Toán cho AI

Lộ trình 35 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

Toán cho AI: lộ trình 35 bài . . . 4
Ký hiệu và kiểm tra nền: học cách đọc ngôn ngữ toán trước khi đọc công thức AI . . . 10
Vector và ma trận như phép biến đổi: mỗi ma trận đưa một điểm đầu vào đến một điểm đầu ra . . . 19
Chuyển vị, trực giao và hạng: đọc hình dạng và số chiều của một phép biến đổi . . . 28
Trị riêng và vector riêng: tìm những hướng đặc biệt của phép biến đổi . . . 38
SVD (Singular Value Decomposition): nhìn mọi ma trận như xoay, co giãn, xoay . . . 48
Đạo hàm, gradient, Jacobian, Hessian: đo sự thay đổi từ một con số đến nhiều biến . . . 59
Chain rule và backprop: nhân độ nhạy trên đường, cộng đóng góp ở chỗ phân nhánh. . . . 69
Xác suất cơ bản: kỳ vọng và phương sai tóm tắt điều gì có thể xảy ra và mức độ dao động . . . 80
Gaussian một chiều và nhiều chiều: một đường cong chuông được điều khiển bởi tâm và độ phân tán. . . 90
Bayes, MLE và MAP: cập nhật điều chưa biết từ dữ liệu . . . 99
Entropy, cross-entropy và KL: đo bất định và cái giá của dự đoán sai . . . 108
Số học máy tính: tính gần đúng có kiểm soát để kết quả không tràn hay mất nghĩa . . . 118
Chuẩn và điều kiện số: đo độ lớn và độ nhạy của phép tính . . . 128
Hàm lồi và tập lồi: nối hai điểm để hiểu hình học của tối ưu . . . 138
00 Preface: Toán nền tảng và hướng dẫn đọc về Toán cho AI . . . 151
01 The Matrix Calculus You Need For Deep Learning (2018) . . . 172
02 Automatic Differentiation in Machine Learning: a Survey (2015) . . . 179
03 A Tutorial on Principal Component Analysis (2014) . . . 186
04 Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions (2009). . . . 195
05 A Tutorial on Spectral Clustering (2007) . . . 204
06 New Insights and Perspectives on the Natural Gradient Method (2014) . . . 213
07 An Elementary Introduction to Information Geometry (2018). . . . 222
08 Variational Inference: A Review for Statisticians (2016) . . . 231
09 An Introduction to Variational Autoencoders (2019) . . . 239
10 A Conceptual Introduction to Hamiltonian Monte Carlo (2017) . . . 247
11 Stochastic Gradient Descent as Approximate Bayesian Inference (2017) . . . 255
12 Computational Optimal Transport (2018) . . . 263
13 Wasserstein GAN (2017) . . . 274
14 Deep Variational Information Bottleneck (2016) . . . 283
15 Online normalizer calculation for softmax (2018) . . . 293
16 FP8 Formats for Deep Learning (2022) . . . 300
17 Spectral Normalization for Generative Adversarial Networks (2018) . . . 309
18 The Shattered Gradients Problem: If resnets are the answer, then what is the question? (2017) . . . 318
19 Measuring the Intrinsic Dimension of Objective Landscapes (2018). . . . 328
20 Benefits of Depth in Neural Networks (2016) . . . 335
21 The Expressive Power of Neural Networks: A View from the Width (2017) . . . 342
22 Deep Neural Networks as Gaussian Processes (2017) . . . 349
23 Neural Tangent Kernel: Convergence and Generalization in Neural Networks (2018) . . . 356
24 On Lazy Training in Differentiable Programming (2018) . . . 363
25 The Principles of Deep Learning Theory (2021) . . . 370
26 Exponential Expressivity in Deep Neural Networks through Transient Chaos (2016) . . . 379

27 Deep Information Propagation (2016) . . . 386
28 Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021) . . . 392
29 Understanding Diffusion Models: A Unified Perspective (2022) . . . 403
30 Opening the Black Box of Deep Neural Networks via Information (2017) . . . 410
31 Deep Learning: A Statistical Viewpoint (2021) . . . 417
32 Benign Overfitting in Linear Regression (2019) . . . 425
33 Gradient Descent Finds Global Minima of Deep Neural Networks (2018) . . . 431
34 A Mean Field View of the Landscape of Two-Layer Neural Networks (2018) . . . 438
35 A Tutorial on Thompson Sampling (2017) . . . 445

Lộ trình

Toán cho AI: lộ trình 35 bài

Thư mục này chứa ghi chú học tập bằng tiếng Việt về **toán nền tảng cho AI**: đại số tuyến tính và matrix calculus, autodiff, suy luận biến phân và MCMC, optimal transport, lý thuyết thông tin, số học dấu phẩy động, rồi lý thuyết học sâu (độ sâu/độ rộng, kernel, NTK, lan truyền tín hiệu, overparameterization). Thứ tự được sắp cho **người chưa biết gì về domain**; nên đọc trước `ml basics content/` và `optimization content/` .

Mọi paper đều được đọc từ **full text trên arXiv** (PDF, tải ngày 09/11/2026). ID của từng bài đã được đối chiếu với tiêu đề trên trang arXiv. Một số paper là sách hoặc khảo sát rất dài: phạm vi đã đọc được ghi rõ trong header và mục 6 của từng bài.

> Đây là ghi chú diễn giải, không phải bản dịch. Mục ”Ứng dụng cho dự án” trong mỗi file là suy luận của chúng ta (cho dự án LightOnOCR-3-0.8B đã nén), **không phải** kết luận của paper. Các phép tính đánh dấu ”tự tính” là của người ghi chú.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	Toán và kiến thức nền không có trong paper, bảng thuật ngữ, cách đọc phản biện, lịch đọc, liên hệ dự án, 5 câu tự kiểm tra. **Đọc trước tiên.**

Giai đoạn 1: Công cụ tính toán và đại số

#	File	Bài	Ý chính
01	`01_2018_matrix_calculus.md`	The Matrix Calculus You Need For Deep Learning (2018)	dựng lại từ đầu phần vi tích phân ma trận (đạo hàm theo vector, Jacobian, 3 dạng quy tắc chuỗi) cần để hiểu backpropagation, rồi áp dụng trực tiếp để tính gradient của…
02	`02_2015_automatic_differentiation_survey.md`	Automatic Differentiation in Machine Learning: a Survey (2015)	khảo sát automatic differentiation (AD) từ góc nhìn machine learning: phân biệt AD với đạo hàm tay/số/ký hiệu, hai chế độ forward và reverse (reverse mode chính là tổng…
03	`03_2014_pca_tutorial.md`	A Tutorial on Principal Component Analysis (2014)	Đại số tuyến tính ứng dụng, giảm chiều dữ liệu (PCA qua eigen-decomposition của ma trận hiệp phương sai và qua SVD)

#	File	Bài	Ý chính
04	`04_2009_randomized_matrix_decomposition.md`	Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions (2009)	Đại số tuyến tính số ngẫu nhiên hoá: tính xấp xỉ ma trận hạng thấp (SVD, phân tích giá trị riêng, QR, interpolative decomposition) bằng lấy mẫu/chiếu ngẫu nhiên
05	`05_2007_spectral_clustering_tutorial.md`	A Tutorial on Spectral Clustering (2007)	Đại số tuyến tính ứng dụng, phân cụm phổ (spectral clustering) dựa trên eigen-decomposition của ma trận Laplacian của đồ thị tương tự (similarity graph)

Giai đoạn 2: Hình học thông tin và suy luận biến phân

#	File	Bài	Ý chính
06	`06_2014_natural_gradient_insights.md`	New Insights and Perspectives on the Natural Gradient Method (2014)	Tối ưu hóa bậc hai, natural gradient, Fisher information matrix, Generalized Gauss-Newton matrix
07	`07_2018_information_geometry_intro.md`	An Elementary Introduction to Information Geometry (2018)	Hình học thông tin (information geometry), ma trận Fisher information như metric tensor, kết nối đối ngẫu (dual connection), đa tạp dually flat, liên hệ với natural…
08	`08_2016_variational_inference_review.md`	Variational Inference: A Review for Statisticians (2016)	Variational inference (suy luận biến phân/suy luận xấp xỉ qua tối ưu hóa), ELBO, mean-field, CAVI, suy luận biến phân ngẫu nhiên (stochastic variational inference, SVI)…
09	`09_2019_vae_introduction.md`	An Introduction to Variational Autoencoders (2019)	Variational autoencoder (VAE), ELBO, reparameterization trick, amortized inference, normalizing flow (inverse autoregressive flow), mô hình sinh biến ẩn sâu (deep latent…

Giai đoạn 3: MCMC và SGD như suy luận Bayes

#	File	Bài	Ý chính
10	`10_2017_hamiltonian_monte_carlo.md`	A Conceptual Introduction to Hamiltonian Monte Carlo (2017)	giải thích trực giác hình học (tránh dùng vi phân hình học hình thức) cho việc tại sao Hamiltonian Monte Carlo (HMC) khám phá hiệu quả các phân phối xác suất nhiều…

#	File	Bài	Ý chính
11	`11_2017_sgd_approximate_bayesian_inference.md`	Stochastic Gradient Descent as Approximate Bayesian Inference (2017)	xem SGD với learning rate hằng số (constant SGD) như một quá trình ngẫu nhiên liên tục có phân phối dừng Gaussian, từ đó chỉnh learning rate/minibatch/preconditioner để…

Giai đoạn 4: Optimal transport và thông tin

#	File	Bài	Ý chính
12	`12_2018_computational_optimal_transport.md`	Computational Optimal Transport (2018)	Một tài liệu chuyên khảo (monograph) dài khoảng 250 trang tổng hợp lý thuyết và thuật toán của Optimal Transport (OT, vận tải tối ưu), từ bài toán Monge/Kantorovich cổ…
13	`13_2017_wasserstein_gan.md`	Wasserstein GAN (2017)	Đề xuất thay khoảng cách Jensen-Shannon (JS, dùng trong GAN gốc) bằng khoảng cách Earth-Mover/Wasserstein-1 để đo sự khác biệt giữa phân phối mô hình (generator) và phân…
14	`14_2016_deep_variational_information_bottleneck.md`	Deep Variational Information Bottleneck (2016)	xây dựng một cận dưới biến phân (variational lower bound) khả vi cho mục tiêu Information Bottleneck (IB) của Tishby et al

Giai đoạn 5: Số học và ổn định số, điều kiện hoá

#	File	Bài	Ý chính
15	`15_2018_online_softmax.md`	Online normalizer calculation for softmax (2018)	một thuật toán tính hàm softmax ”an toàn” (không tràn số) trong một lượt quét duy nhất qua vector đầu vào, bằng cách tính đồng thời giá trị lớn nhất và tổng mũ chuẩn…
16	`16_2022_fp8_formats.md`	FP8 Formats for Deep Learning (2022)	Đề xuất một chuẩn định dạng số thực dấu phẩy động 8-bit (FP8) gồm hai biến thể E4M3 và E5M2 để huấn luyện và suy luận mạng neural, kèm thực nghiệm trên nhiều kiến trúc…
17	`17_2018_spectral_normalization_gan.md`	Spectral Normalization for Generative Adversarial Networks (2018)	Đề xuất ”spectral normalization”, một kỹ thuật chuẩn hoá trọng số (weight normalization) ép ràng buộc Lipschitz lên discriminator của GAN bằng cách chia mỗi ma trận…

#	File	Bài	Ý chính
18	`18_2017_shattered_gradients.md`	The Shattered Gradients Problem: If resnets are the answer, then what is the question? (2017)	Phân tích toán học vì sao gradient của mạng feedforward dùng rectifier (ReLU) càng sâu càng giống nhiễu trắng (white noise), làm huấn luyện khó khăn;

Giai đoạn 6: Biểu diễn, độ sâu, độ rộng, kernel và lan truyền tín hiệu

#	File	Bài	Ý chính
19	`19_2018_intrinsic_dimension.md`	Measuring the Intrinsic Dimension of Objective Landscapes (2018)	đo ”intrinsic dimension” (số chiều nội tại) của objective landscape bằng phương pháp random subspace training;
20	`20_2016_benefits_of_depth.md`	Benefits of Depth in Neural Networks (2016)	chứng minh tồn tại các hàm tính được bởi mạng ReLU rất sâu (Θ(k^3) lớp, Θ(1) node mỗi lớp) mà không mạng nào nông hơn (O(k) lớp) có thể xấp xỉ tốt, trừ khi mạng nông đó…
21	`21_2017_expressive_power_width.md`	The Expressive Power of Neural Networks: A View from the Width (2017)	câu hỏi đối ngẫu của ”benefits of depth”: độ rộng (width) của mạng ReLU ảnh hưởng thế nào tới khả năng biểu diễn hàm;
22	`22_2017_deep_nn_gaussian_processes.md`	Deep Neural Networks as Gaussian Processes (2017)	Lý thuyết mạng nơ-ron vô hạn rộng (infinite-width), tương ứng với Gaussian Process, suy luận Bayes cho deep learning
23	`23_2018_ntk.md`	Neural Tangent Kernel: Convergence and Generalization in Neural Networks (2018)	Neural Tangent Kernel (NTK), động lực huấn luyện mạng nơ-ron vô hạn rộng bằng gradient descent, kernel gradient descent
24	`24_2018_lazy_training.md`	On Lazy Training in Differentiable Programming (2018)	định nghĩa và phân tích hiện tượng ”lazy training” (mô hình phi tuyến hành xử gần như phiên bản tuyến tính hoá quanh điểm khởi tạo khi được scale lên), chỉ ra đây là hệ…
25	`25_2021_principles_dl_theory.md`	The Principles of Deep Learning Theory (2021)	Đây là một cuốn sách giáo trình dài khoảng 470 trang, xây dựng một ”lý thuyết hiệu dụng” (effective theory, mượn từ vật lý) cho mạng nơ-ron sâu tại thời điểm khởi tạo và…
26	`26_2016_exponential_expressivity_chaos.md`	Exponential Expressivity in Deep Neural Networks through Transient Chaos (2016)	lý thuyết mean-field cho sự lan truyền tín hiệu qua mạng nơ-ron ngẫu nhiên (chưa huấn luyện) nhiều lớp;
27	`27_2016_deep_information_propagation.md`	Deep Information Propagation (2016)	hai depth-scale (ξ_q, ξ_c) điều khiển lan truyền tín hiệu trong mạng ngẫu nhiên chưa huấn luyện;

Giai đoạn 7: Hình học, diffusion và thông tin trong mạng

#	File	Bài	Ý chính
28	`28_2021_geometric_deep_learning.md`	Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021)	một khung lý thuyết (gọi là ”Geometric Deep Learning”, GDL) thống nhất phần lớn kiến trúc deep learning phổ biến (CNN, GNN, Deep Sets, Transformer, Mesh CNN, Spherical…
29	`29_2022_understanding_diffusion_models.md`	Understanding Diffusion Models: A Unified Perspective (2022)	suy ra lại toàn bộ họ mô hình diffusion (Variational Diffusion Model) từ một góc nhìn thống nhất: Markovian Hierarchical VAE, 3 cách tham số hoá mục tiêu tương đương (dự…
30	`30_2017_opening_black_box_information.md`	Opening the Black Box of Deep Neural Networks via Information (2017)	quan sát thực nghiệm quá trình huấn luyện Deep Neural Network (DNN) qua lăng kính lý thuyết thông tin (Information Bottleneck, IB): phát hiện 2 pha của SGD (drift và…

Giai đoạn 8: Tổng quát hoá và overparameterization

#	File	Bài	Ý chính
31	`31_2021_deep_learning_statistical_viewpoint.md`	Deep Learning: A Statistical Viewpoint (2021)	Bài khảo sát (survey) tổng hợp các kết quả gần đây của lý thuyết học thống kê nhằm lý giải hai hiện tượng bí ẩn của deep learning: (a) gradient descent giải được bài…
32	`32_2019_benign_overfitting_linear_regression.md`	Benign Overfitting in Linear Regression (2019)	đặc trưng hoá chính xác (điều kiện cần và đủ, qua 2 khái niệm ”effective rank” của ma trận hiệp phương sai) khi nào bộ ước lượng minimum-norm interpolator trong hồi quy…
33	`33_2018_gradient_descent_global_minima_dnn.md`	Gradient Descent Finds Global Minima of Deep Neural Networks (2018)	Chứng minh lý thuyết rằng gradient descent khởi tạo ngẫu nhiên đạt loss huấn luyện bằng 0 với tốc độ tuyến tính trên mạng nơ-ron sâu quá tham số hoá…
34	`34_2018_mean_field_two_layer_networks.md`	A Mean Field View of the Landscape of Two-Layer Neural Networks (2018)	Chứng minh rằng, trong giới hạn tỷ lệ (scaling limit) phù hợp, động lực SGD khi huấn luyện mạng nơ-ron 2 lớp được mô tả xấp xỉ bởi một phương trình đạo hàm riêng phi…

Giai đoạn 9: Bandit và quyết định tuần tự

#	File	Bài	Ý chính
35	`35_2017_tutorial_thompson_sampling.md`	A Tutorial on Thompson Sampling (2017)	Tutorial tổng hợp về thuật toán Thompson Sampling (TS) cho bài toán ra quyết định tuần tự dưới thông tin không đầy đủ (multi-armed bandit và mở rộng), từ công thức cơ…

Cách dùng

Đọc preface (00) trước. Các giai đoạn 1 và 5 dùng ngay trong thực hành; giai đoạn 6 và 8 là lý thuyết nặng, lần đầu có thể chỉ đọc mục 1, 2, 11 của từng bài.

Mỗi bài có 11 mục cố định: bối cảnh, ý tưởng chính, phương pháp chi tiết, thí nghiệm, phân tích, giới hạn, liên hệ, ứng dụng cho dự án (suy luận), thí nghiệm gợi ý, tóm tắt công thức, kết luận. Mục 6 (giới hạn) là nơi ghi các mâu thuẫn nội tại phát hiện trong paper.

Liên quan

- `math content/` (35 bài): toán nền; `ml basics content/` (40 bài): ML cơ bản; `optimization content/` (51 bài): tối ưu, optimizer, hyperparameter.
- `llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`: các lộ trình chuyên đề xây trên ba thư mục nền này.

Nền tảng 01

Ký hiệu và kiểm tra nền: học cách đọc ngôn ngữ toán trước khi đọc công thức AI

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: không cần kiến thức trước; có thể đọc như bản đồ vào cuốn toán

Dùng ở: gần như mọi bài; đặc biệt math 01 (matrix calculus), math 03 (PCA), math 04 (randomized SVD), math 15 (online softmax), math 17 (spectral normalization)

> Chương này viết để người mới đọc được ký hiệu, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần một bản đồ ký hiệu

Một công thức toán thường chứa hai thứ: ý tưởng và cách ghi ý tưởng. Người đã quen ký hiệu nhìn thấy $x \in \mathbb{R}^d$ là hiểu "một danh sách d số thực"; người mới có thể mắc ngay ở ký hiệu $\in$, chữ $\mathbb{R}$, hay số mũ d. Khi đó công thức trông khó hơn khái niệm bên trong.

Chương này dạy cách đọc những ký hiệu lặp đi lặp lại trong cả cuốn. Mục tiêu không phải thuộc lòng một bảng chữ cái, mà là biết hỏi ba câu: đối tượng này thuộc loại nào, nó có kích thước bao nhiêu, và công thức đang làm phép gì với nó. Trong AI, ba câu hỏi đó giúp đọc phương trình của dữ liệu, tham số, hàm mất mát, đạo hàm và xác suất mà không nhầm một số đơn với một mảng số.

Ý tưởng gói gọn: **hãy xem ký hiệu như nhãn loại và kích thước của một vật toán học.** Chữ thường thường đặt tên cho số hoặc vector, chữ hoa thường đặt tên cho ma trận, còn chỉ số cho biết đang nói đến phần tử nào. Đây là quy ước hữu ích, không phải luật tự nhiên; tác giả paper có thể chọn cách khác.

2. Nhắc nền: số, tập hợp và dấu bằng

2.1 Số thực và số nguyên

Tập số thực được ghi $\mathbb{R}$. Số thực gồm các số có thể nằm trên trục số, chẳng hạn -2, 0, 3.5 và $\sqrt{2}$. Tập số nguyên được ghi $\mathbb{Z}$, ví dụ -2, 0, 7. Trong học máy, tham số và dữ liệu thường được mô hình hoá bằng số thực, dù máy tính lưu chúng với độ chính xác hữu hạn.

Khi viết $x \in \mathbb{R}$, ta nói x là một số thực. Ký hiệu $\in$ đọc là "thuộc". Còn $x \notin \mathbb{R}$ nghĩa là x không thuộc tập số thực. Đừng nhầm dấu thuộc với phép chia hoặc dấu bằng.

2.2 Tập hợp, số chiều và kích thước

Ký hiệu $\mathbb{R}^d$ là tập mọi vector có d thành phần thực. Chữ d là số chiều, không phải số mũ theo nghĩa nhân lặp. Chẳng hạn $x \in \mathbb{R}^3$ nghĩa là x có ba toạ độ thực. Ví dụ (tự tính): một vector ba chiều có thể là $x = (2, -1, 4)^\top$.

Ký hiệu $\mathbb{R}^{m \times n}$ là tập các ma trận có m hàng và n cột. Với $A \in \mathbb{R}^{2 \times 3}$, ta biết trước hình dạng của A: hai hàng, ba cột. Chỉ số đầu tiên là hàng, chỉ số thứ hai là cột.

Kích thước và giá trị là hai thông tin khác nhau. Hai ma trận đều có thể thuộc $\mathbb{R}^{2 \times 3}$ nhưng chứa các số khác nhau. Khi một phép nhân không hợp lệ, thường có thể phát hiện bằng kích thước trước khi tính bất kỳ phần tử nào.

3. Bảng ký hiệu dùng xuyên cuốn

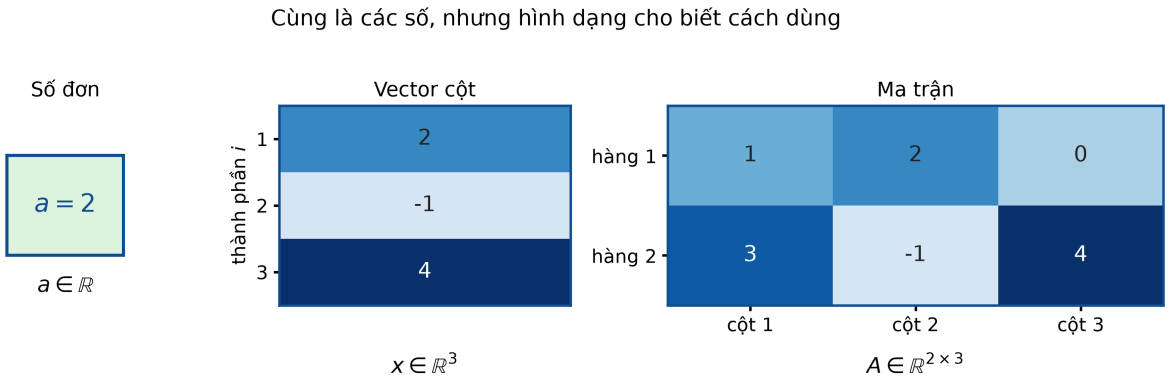
Bảng dưới đây là quy ước đọc nhanh cho chương nền tảng. Một paper có thể dùng chữ khác, vì vậy luôn xem phần ký hiệu riêng của paper nếu có.

Ký hiệu	Đọc là	Ý nghĩa thường gặp	Cách kiểm nhanh
a, b, c	chữ thường	số thực hoặc vô hướng	Có đúng một giá trị không?
x, y, v	chữ thường đậm hoặc thường	vector, thường là cột	Có bao nhiêu thành phần?
A, B, W, X	chữ hoa	ma trận	Có bao nhiêu hàng và cột?
x_i	x chỉ số i	thành phần thứ i của vector	i thường chọn một toạ độ
A_{ij}	A hàng i cột j	một ô của ma trận	Chỉ số đầu là hàng
$x^{(k)}$	x chỉ số trên ngoặc	mẫu thứ k hoặc phiên bản thứ k	Không phải luỹ thừa nếu có quy ước mẫu
$x^\top$	x chuyển vị	đổi vector cột thành hàng	Đọc kích thước sau khi đổi
$\mathbf{1}$	vector một	vector có các phần tử bằng một	Độ dài do ngữ cảnh quyết định
I	ma trận đơn vị	ma trận vuông, đường chéo chính bằng một và các ô ngoài đường chéo bằng không	Kích thước cần khớp
$\mathbf{0}$ hoặc 0	không	số không, vector không hoặc ma trận không	Xem kích thước ngữ cảnh
$\sum_i$	tổng theo i	cộng qua các giá trị chỉ số	Kiểm miền của i
$\prod_i$	tích theo i	nhân qua các giá trị chỉ số	Kiểm miền của i
$\in$	thuộc	là một phần tử của tập	$x \in \mathbb{R}^d$ là vector d chiều
$\forall$	với mọi	phát biểu đúng cho tất cả phần tử	Đọc miền sau dấu này
$\exists$	tồn tại	có ít nhất một phần tử thoả điều kiện	Không nhất thiết duy nhất
$\approx$	xấp xỉ	hai số gần nhau theo độ chính xác nêu rõ	Không phải bằng tuyệt đối
$\propto$	tỉ lệ với	bằng nhau đến một hằng số nhân	Hằng số có thể phụ thuộc ngữ cảnh
$\mathbb{E}[X]$	kỳ vọng của X	trung bình theo một phân phối	X là biến ngẫu nhiên
$\mathrm{Var}(X)$	phương sai	độ phân tán quanh kỳ vọng	Không âm
$\mathcal{L}$	L viết hoa kiểu thư pháp	hàm mất mát hoặc một tập	Đọc định nghĩa tại lần xuất hiện đầu
∇f	nabla f	gradient của hàm vô hướng	Có một đạo hàm riêng cho mỗi đầu vào
$\partial f / \partial x_i$	đạo hàm riêng	độ đối của f theo thành phần x_i	Giữ các biến còn lại cố định
θ	theta	tham số của mô hình	Có thể là số, vector hoặc bó tham số

Chữ đậm như **x** thường nhấn rằng một vật là vector, còn chữ thường nghiêng x có thể dùng theo quy ước toán cổ điển. Trong các bài của cuốn này, vector có lúc được ghi bằng chữ thường không đậm cho dễ đọc. Hãy dựa vào khai báo kích thước và công thức, đừng chỉ dựa vào kiểu chữ.

4. Nhìn hình dạng trước khi đọc phép tính

Trước công thức, nhìn ba lớp đối tượng như ba loại khay: một số đơn, một cột số, và một bảng số. Hình dưới dùng màu khác nhau để tách từng loại.



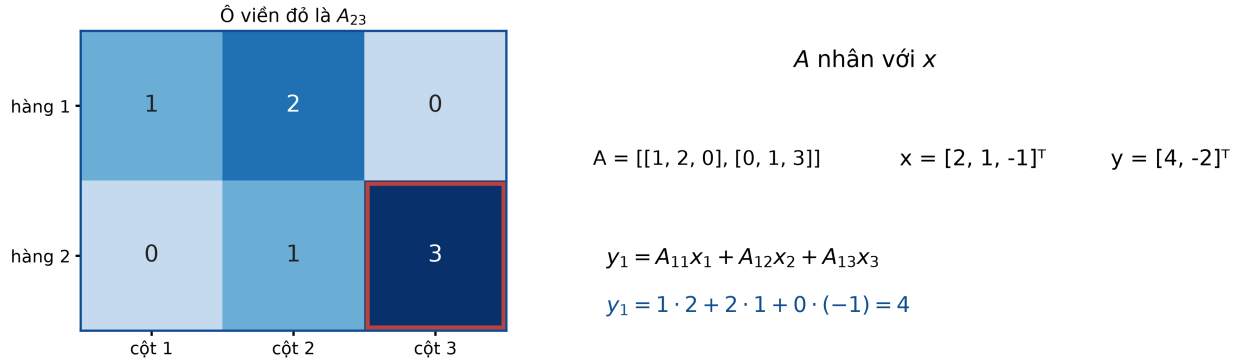
Hình 1: Một số đơn, vector ba thành phần và ma trận hai hàng ba cột, mỗi đối tượng có nhãn kích thước (minh hoạ, tự tính)

Cách đọc hình 1: ô bên trái chỉ chứa một số a. Ở giữa là vector x có ba hàng đứng dọc, nên ta đang xem nó như một cột thuộc $\mathbb{R}^3$. Bên phải, A có hai hàng và ba cột, nên thuộc $\mathbb{R}^{2\times3}$. Màu và giá trị trong hình chỉ để phân biệt vị trí, không biểu diễn độ lớn thực nghiệm nào.

Để ghi kích thước của vector cột, ta có thể viết $x \in \mathbb{R}^{3\times1}$ hoặc chỉ viết $x \in \mathbb{R}^3$. Cách rút gọn thứ hai phổ biến vì vector thường được hiểu là cột. Nếu một tài liệu xem vector là hàng, tài liệu đó cần nói rõ quy ước.

5. Chỉ số: cách mở một mảng ra thành số

Với vector $x = (x_1, x_2, \ldots, x_d)^\top$, ký hiệu x_i lấy thành phần thứ i. Với ma trận A, ký hiệu A_{ij} lấy phần tử tại hàng i, cột j. Thứ tự này thống nhất: chỉ số hàng trước, chỉ số cột sau.



Hình 2: Ma trận A có hàng và cột được tô, cho thấy Aij là phần tử ở hàng i cột j và một hàng của tích Ax là tổng có trọng số (minh hoạ, tự tính)

Cách đọc hình 2: trong bảng bên trái, ô được viền đậm nằm tại hàng $i = 2$, cột $j = 3$, nên nó được ký hiệu là A_{23}. Phần bên phải dùng cùng ma trận A và vector x; hàng đầu của đầu ra bằng $A_{11}x_1 + A_{12}x_2 + A_{13}x_3$. Mỗi đầu ra là tổng của một hàng với vector đầu vào, từng cặp phần tử được nhân rồi cộng.

Nếu viết $A_{i,:}$, dấu hai chấm thường có nghĩa "lấy tất cả cột ở hàng i". Tương tự, $A_{:,j}$ là cột thứ j. Cú pháp này quen thuộc trong lập trình mảng, nhưng trong toán viết tay một tác giả có thể dùng cách khác.

6. Tổng, trung bình và dấu ngoặc

Ký hiệu tổng $\sum_{i=1}^{n} x_i$ yêu cầu cộng các phần tử từ chỉ số 1 đến n. Chỉ số nằm dưới dấu tổng là biến chạy; hai con số dưới và trên chỉ phạm vi. Viết tường minh: $\sum_{i=1}^{3} x_i = x_1 + x_2 + x_3$.

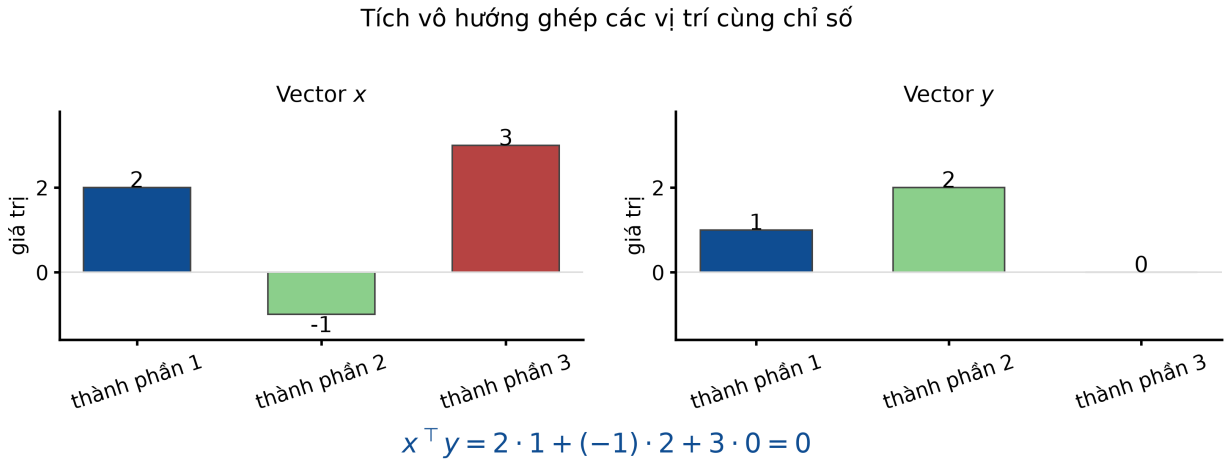
Ví dụ (tự tính): với $x = (2, 1, 3)^\top$, ta có $\sum_{i=1}^{3} x_i = 2 + 1 + 3 = 6$, còn trung bình là $\bar{x} = \frac{1}{3}\sum_{i=1}^{3} x_i = 2$. Ký hiệu gạch trên $\bar{x}$ thường biểu thị trung bình mẫu, nhưng phải xem định nghĩa tại chỗ vì gạch trên còn có nghĩa khác ở một số lĩnh vực.

Các dấu ngoặc giúp nhìn thứ tự phép toán. Trong $\sum_i (x_i - y_i)^2$, trước hết lấy hiệu từng cặp, bình phương từng hiệu, rồi cộng. Nếu bỏ ngoặc thành $\sum_i x_i - y_i^2$, biểu thức đã đổi nghĩa. Phép tổng có thể đặt ngoài một phép toán theo từng phần tử, nhưng không thể tuỳ tiện chuyển qua phép nhân giữa hai tổng.

Một cách đọc an toàn là chỉ tay theo thứ tự: biến chạy là gì, biến chạy từ đâu đến đâu, phần nào được tính bên trong mỗi lượt, cuối cùng các lượt được cộng hay nhân với nhau. Thói quen này hữu ích khi khai triển loss trên một tập ví dụ.

7. Tích vô hướng: ghép hai danh sách thành một số

Trước công thức, hãy hình dung hai danh sách số có cùng số ô. Ta nhân từng cặp ô tương ứng rồi cộng các tích. Hình cho thấy một trường hợp nhỏ, đồng thời tô riêng từng cặp được ghép.



Hình 3: Tích vô hướng của hai vector ghép các thành phần cùng vị trí rồi cộng kết quả (minh hoạ, tự tính)

Cách đọc hình 3: vector x và y đều có ba thành phần, nên ta ghép thành phần thứ nhất với thứ nhất, thứ hai với thứ hai, và thứ ba với thứ ba. Tích vô hướng là một số, không phải vector: $x^\top y = \sum_i x_i y_i$. Trong ví dụ số trên hình, kết quả bằng $2 \cdot 1 + (-1) \cdot 2 + 3 \cdot 0 = 0$ (tự tính).

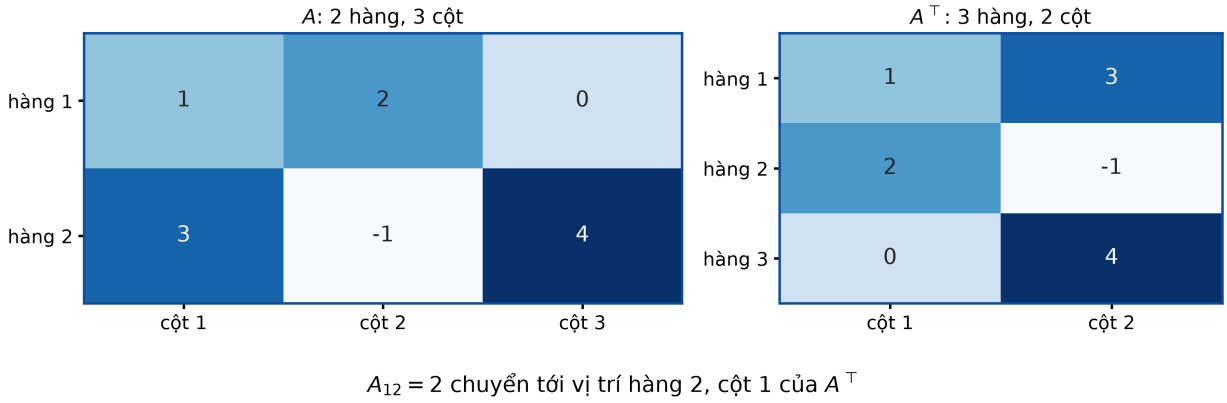
Tích vô hướng chỉ định nghĩa theo cách này khi hai vector có cùng số thành phần. Nếu kích thước khác nhau, không thể ghép trọn vẹn từng vị trí. Về sau, tích vô hướng giúp đo góc và độ giống hướng; hiện giờ chỉ cần nhớ đầu ra là số.

8. Chuyển vị và kích thước của phép nhân

Chuyển vị đổi hàng thành cột và đổi cột thành hàng. Nếu $A \in \mathbb{R}^{m \times n}$ thì $A^\top \in \mathbb{R}^{n \times m}$. Với vector cột $x \in \mathbb{R}^{d \times 1}$, chuyển vị $x^\top$ là vector hàng cỡ $1 \times d$.

Cách đọc hình 4: ma trận bên trái có hai hàng, ba cột. Bên phải có ba hàng, hai cột, và phần tử ở hàng i, cột j của ma trận đầu trở thành phần tử ở hàng j, cột i của ma trận sau. Chẳng hạn A_{12} chuyển sang vị trí hàng 2, cột 1 của $A^\top$. Vì vậy chuyển vị không đổi các giá trị, chỉ sắp xếp lại chỗ.

Quy tắc kích thước của tích ma trận rất gọn: số cột của ma trận bên trái phải bằng số hàng của ma trận bên phải. Nếu $A \in \mathbb{R}^{m \times n}$ và $B \in \mathbb{R}^{n \times p}$ thì $AB \in \mathbb{R}^{m \times p}$. Hai con số ở giữa phải khớp; hai con số ở ngoài là kích thước kết quả.



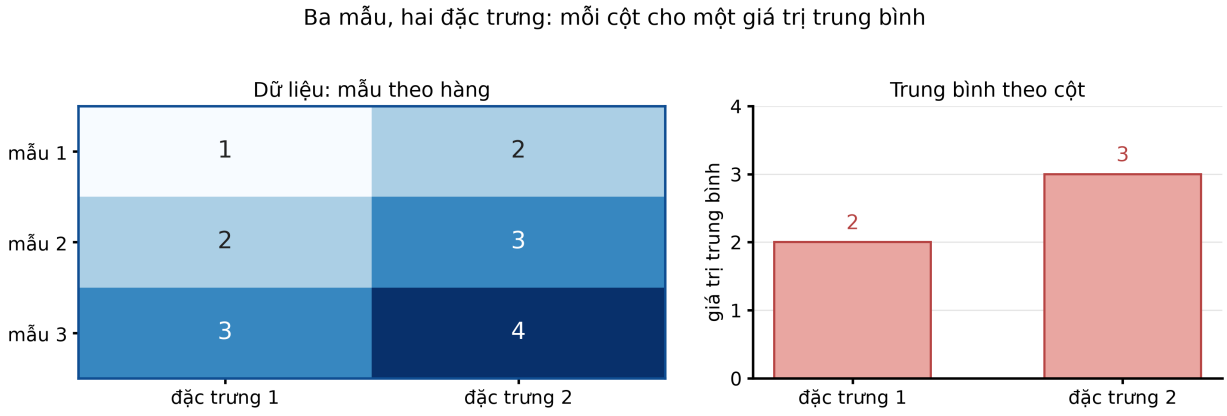
Hình 4: Chuyển vị đổi hàng thành cột và ma trận m nhân n thành ma trận n nhân m (minh hoạ, tự tính)

Ví dụ (tự tính): ma trận A cỡ 2×3 nhân vector x cỡ 3×1 cho vector $y = Ax$ cỡ 2×1. Nếu nhầm rằng A là 3×2, lỗi lộ ra ngay ở phép nhân với vector ba thành phần.

9. Dữ liệu: ma trận có thể xếp mẫu theo hàng hoặc cột

Trong AI, một ma trận dữ liệu thường chứa nhiều mẫu. Một quy ước phổ biến xếp mỗi mẫu thành một hàng: $X \in \mathbb{R}^{n \times d}$, với n mẫu và d đặc trưng. Quy ước khác xếp mẫu theo cột: $X \in \mathbb{R}^{d \times n}$. Cả hai đều đúng nếu người viết nhất quán.

Hình sau lấy ba mẫu, mỗi mẫu có hai đặc trưng, rồi chỉ rõ phép lấy trung bình theo từng đặc trưng khi mẫu nằm theo hàng.



Hình 5: Ba mẫu nằm theo hàng và hai đặc trưng nằm theo cột, trung bình theo mẫu tạo một số cho mỗi đặc trưng (minh hoạ, tự tính)

Cách đọc hình 5: mỗi hàng là một mẫu và mỗi cột là một đặc trưng. Cộng các giá trị trong một cột rồi chia cho số hàng cho ta trung bình của đặc trưng đó; với bảng nhỏ, các trung bình là 2 và 3 (tự tính). Nếu mẫu nằm theo cột thì mỗi hàng là một đặc trưng, nên ta cộng qua các cột để lấy trung bình cho từng hàng. Trước khi dùng hàm thư viện, hãy tìm xem trục nào biểu diễn mẫu.

Trong ký hiệu toán, $x^{(k)}$ thường dùng để chỉ mẫu thứ k, để tránh nhầm với x^k là luỹ thừa. Ví dụ $x^{(3)}$ có thể là vector đặc trưng của mẫu thứ ba. Một số paper lại dùng x_i cho mẫu thứ i. Cách phân biệt an toàn là nhìn số chỉ số và đọc câu khai báo.

10. Các dấu ngoặc và ký hiệu đặc biệt thường gặp

10.1 Ngoặc tròn, vuông, nhọn

Ngoặc tròn $f(x)$ thường gom đối số của hàm. Ngoặc vuông $[x_1, x_2]$ có thể chỉ một khoảng, một vector hàng hoặc một danh sách, tuỳ ngữ cảnh. Ngoặc nhọn $\{x : x > 0\}$ thường mô tả tập hợp các giá trị x thoả điều kiện phía sau dấu hai chấm.

Hai dấu trị tuyệt đối $|x|$ cho độ lớn của một số. Với vector, ký hiệu chuẩn thường viết $\|x\|_2$, là độ dài Euclid. Không nên dùng một cặp dấu gạch đứng đơn trong công thức ma trận rồi đoán ý nghĩa; tác giả có thể muốn nói định thức, chuẩn hay trị tuyệt đối.

10.2 Chữ Hy Lạp và ký hiệu gạch

Các chữ Hy Lạp thường đặt tên cho đại lượng: α có thể là hệ số, λ là trị riêng hoặc trọng số phạt, σ là độ lệch chuẩn hoặc giá trị kỳ dị, θ là tham số. Không có một ý nghĩa cố định xuyên mọi bài.

Chữ hoa viết kiểu thư pháp như $\mathcal{L}$ thường dùng cho hàm mất mát, tập hợp hoặc một đối tượng đặc biệt. Ký hiệu θ có thể chỉ vector tham số nếu tác giả khai báo như vậy, dù nó không in đậm. Dấu mũ trên $\hat{x}$ thường là ước lượng, dự đoán hoặc đại lượng đã chuẩn hoá; dấu ngã $\tilde{x}$ thường là phiên bản biến đổi. Luôn kiểm định nghĩa tại nơi ký hiệu xuất hiện đầu tiên.

11. Khi cùng một ký hiệu có nhiều nghĩa

Đại số toán học có nhiều quy ước sống song song. x có thể là vector, biến vô hướng hoặc đầu vào của hàm. A có thể là ma trận, một sự kiện trong xác suất, hoặc một bộ dữ liệu. σ có thể chỉ độ lệch chuẩn hay giá trị kỳ dị. Cách viết không sai nếu tác giả định nghĩa đủ rõ, nhưng người đọc không nên đoán nghĩa từ một chữ đơn lẻ.

Hãy đọc quanh ký hiệu theo ba bước. Bước một, tìm câu “cho ...” hoặc “đặt ...” gần nơi nó xuất hiện đầu tiên. Bước hai, ghi loại và kích thước cạnh ký hiệu, chẳng hạn $W \in \mathbb{R}^{m \times n}$. Bước ba, kiểm mỗi phép toán sau đó có hợp kích thước không. Nếu một ma trận cỡ $m \times n$ được cộng với vector cỡ p, cần có quy tắc mở rộng hoặc phép cộng đó không hợp lệ.

Ký hiệu có dấu gạch dưới cũng cần đọc cẩn thận. x_i chọn một thành phần. $x_{i:j}$ trong mã thường cắt một đoạn. $x_{i,t}$ có thể chọn mẫu i ở thời điểm t. Chỉ số không tự nói lên vai trò; phần mô tả dữ liệu mới nói điều đó.

12. Nối sang AI: công thức mất mát là một phép đếm có cấu trúc

Một mô hình thường nhận đầu vào x, có tham số θ và tạo dự đoán $f_\theta(x)$. Mỗi mẫu có thể tạo một mức lỗi $\ell(f_\theta(x), y)$. Trên n mẫu, một loss trung bình thường được viết

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x^{(i)}), y^{(i)}\big).$$

Đọc công thức này từ ngoài vào: $\mathcal{L}$ là một số phụ thuộc tham số θ; mỗi vòng i lấy một mẫu và tính một lỗi; tổng cộng các lỗi; chia cho n để có trung bình. Nếu $x^{(i)}$ là vector, $f_\theta(x^{(i)})$ có thể là vector xác suất hoặc một số dự đoán. Chương này chưa cần biết đạo hàm để hiểu cấu trúc phép tính.

Ký hiệu $\nabla_\theta \mathcal{L}$ sẽ xuất hiện khi ta hỏi mỗi thành phần của tham số nên đổi theo hướng nào để giảm loss. Đó là chủ đề của math nen06. Biến ngẫu nhiên và kỳ vọng $\mathbb{E}$ sẽ xuất hiện khi dữ liệu được mô hình hoá bằng một phân phối, được dựng từ math nen08. Vì vậy, đọc được ký hiệu trước giúp người học tập trung vào ý tưởng mới trong các chương đó.

Các bài PCA và randomized SVD dùng ma trận để chứa dữ liệu hoặc phép biến đổi; math 15 dùng tổng theo phần tử để tránh tràn số khi tính softmax; math 17 dùng chuẩn ma trận để kiểm soát độ lớn phép biến đổi. Bảng ký hiệu ở đầu chương là nơi quay lại khi quên dấu hoặc loại đối tượng.

13. Sai lầm hay gặp

- **Coi $\mathbb{R}^d$ là một vector riêng lẻ.** Đây là không gian chứa mọi vector có d thành phần; $x \in \mathbb{R}^d$ nói rằng x là một phần tử cụ thể trong không gian đó.
- **Đảo hàng và cột trong A_{ij}.** Chỉ số thứ nhất là hàng, thứ hai là cột. Khi chưa chắc, hãy viết ra một ma trận nhỏ và chỉ vị trí.
- **Coi số mũ trong $x^{(i)}$ luôn là phép luỹ thừa.** Ngoặc thường được dùng cho chỉ số mẫu; paper phải khai báo, còn người đọc cần tra.
- **Quên số chiều khi nhân.** Kiểm hai kích thước ở giữa trước khi thực hiện phép nhân.
- **Mặc định dữ liệu luôn xếp mẫu theo hàng.** Cả quy ước hàng lẫn cột đều tồn tại; tìm mô tả của X trước khi cộng hoặc lấy trung bình.
- **Cho rằng ký hiệu Hy Lạp có một nghĩa cố định.** σ chẳng hạn có thể chỉ hai đại lượng khác nhau trong hai chương. Dựa vào câu định nghĩa và đơn vị.

14. Tóm tắt và bài tự kiểm

14.1 Tóm tắt ký hiệu

Cách viết	Đọc nhanh	Kiểm tra
$x \in \mathbb{R}^d$	vector d thành phần thực	đếm thành phần
$A \in \mathbb{R}^{m \times n}$	ma trận m hàng, n cột	hàng trước, cột sau
A_{ij}	phần tử hàng i cột j	chỉ số đầu là hàng
$\sum_{i=1}^{n} x_i$	cộng từ thành phần 1 đến n	xác định miền của i
$x^\top y$	tích vô hướng	hai vector cùng chiều, kết quả số
$A^\top$	chuyển vị	đổi kích thước m nhân n thành n nhân m
AB	tích ma trận	cột A phải khớp hàng B
$x^{(i)}$	thường là mẫu i	kiểm định nghĩa trong bài
$\mathbb{E}[X]$	kỳ vọng	X phải là biến ngẫu nhiên
∇f	gradient	một đạo hàm riêng cho mỗi đầu vào

14.2 Tự kiểm tra 10 câu: nên bắt đầu từ đâu?

Hãy thử trả lời trước khi xem đáp án. Mục đích không phải đạt điểm tuyệt đối, mà là chọn chương nền phù hợp. Mỗi câu kiểm một kỹ năng nhỏ, không kiểm mức độ thông minh hay năng lực toán nói chung.

Câu 1. $x \in \mathbb{R}^4$ thường có nghĩa nào? A. Ma trận bốn hàng bốn cột. B. Vector có bốn thành phần thực. C. Một số thực được nhân bốn lần.

Câu 2. Nếu $A \in \mathbb{R}^{2\times 3}$ và $x \in \mathbb{R}^3$ là vector cột, thì Ax có kích thước nào? A. 2×1. B. 3×1. C. Phép nhân không hợp lệ.

Câu 3. Trong A_{23}, chỉ số nào là hàng? A. 2. B. 3. C. Cả hai cùng là hàng.

Câu 4. $x^\top y$ với hai vector ba thành phần thường trả về gì? A. Một số. B. Vector ba thành phần. C. Ma trận ba hàng ba cột.

Câu 5. Nếu A có hai hàng, ba cột thì $A^\top$ có dạng nào? A. Hai hàng, ba cột. B. Ba hàng, hai cột. C. Ba hàng, ba cột.

Câu 6. Điều kiện $Q^\top Q = I$ gắn với khái niệm nào? A. Ma trận trực giao. B. Ma trận xác suất. C. Ma trận đường chéo bất kỳ.

Câu 7. Công thức $Av = \lambda v$ thường nói về gì? A. Tích vô hướng. B. Vector riêng và trị riêng. C. Trung bình mẫu.

Câu 8. Một hàm nhận hai biến và trả một số có gradient gồm bao nhiêu đạo hàm riêng? A. Một. B. Hai. C. Bốn.

Câu 9. $\mathbb{E}[X]$ là ký hiệu nào? A. Kỳ vọng của biến ngẫu nhiên. B. Ma trận đơn vị. C. Định thức của X.

Câu 10. Khi trong bài có σ, cách an toàn nhất để biết nghĩa là gì? A. Mặc định là độ lệch chuẩn. B. Mặc định là giá trị kỳ dị. C. Tìm định nghĩa tại lần xuất hiện đầu.

Đáp án và đường đi. Câu 1: B, câu 2: A, câu 3: A, câu 4: A, câu 5: B, câu 6: A, câu 7: B, câu 8: B, câu 9: A, câu 10: C. Nếu sai câu 1, 3 hoặc 10, đọc lại math nen01 và thử lập một bảng ký hiệu của riêng mình. Nếu sai câu 2, đọc math nen02 về kích thước và phép nhân ma trận. Nếu sai câu 4, xem lại math nen01 mục 7 về tích vô hướng; math nen02 mục 5 sẽ nối nó với phép nhân ma trận. Nếu sai câu 5, xem math nen01 mục 8 về chuyển vị, rồi đọc math nen03 để tìm hiểu các tính chất liên quan sâu hơn. Nếu vướng câu 6 hoặc 7, lần lượt đọc math nen03 về trực giao, hạng và không gian cột, rồi math nen04 về trị riêng. Câu 8 dẫn tới math nen06 về đạo hàm và gradient. Câu 9 dẫn tới math nen08 về xác suất, kỳ vọng và phương sai. Khi các khái niệm đó ổn, quay lại bài paper phù hợp; không cần đọc cả cuốn một lần.

14.3 Bài tập tự tính

Năm bài đi từ đọc ký hiệu đến một phép tính nhỏ. Hãy che đáp án, giải trước, rồi đối chiếu. Các số trong lời giải đã được kiểm bằng numpy.

Bài 1 (dễ). Viết $x = (4, -2, 1)^\top$ thành danh sách thành phần, nêu kích thước của x và $x^\top$.

Đáp án. Các thành phần lần lượt là $x_1 = 4$, $x_2 = -2$, $x_3 = 1$ (tự tính). Vector cột x có kích thước 3×1; vector hàng $x^\top$ có kích thước 1×3.

Bài 2 (dễ vừa). Với $A = \begin{pmatrix} 1 & 2 & 0 \\ 3 & -1 & 4 \end{pmatrix}$, hãy nêu kích thước của A, tính A_{21} và viết $A^\top$.

Đáp án. A thuộc $\mathbb{R}^{2\times 3}$, $A_{21} = 3$. Chuyển vị là $A^\top = \begin{pmatrix} 1 & 3 \\ 2 & -1 \\ 0 & 4 \end{pmatrix}$ (tự tính). Nó có ba hàng và hai cột.

Bài 3 (vừa). Tính $\sum_{i=1}^{3} x_i$ và $x^\top y$ khi $x = (1, 2, -1)^\top$ và $y = (3, 0, 2)^\top$.

Đáp án. Tổng thành phần của x là $1 + 2 + (-1) = 2$. Tích vô hướng là $1 \cdot 3 + 2 \cdot 0 + (-1) \cdot 2 = 1$ (tự tính). Kết quả thứ nhất là tổng các thành phần của một vector; kết quả thứ hai là một số ghép cặp hai vector.

Bài 4 (vừa khó). Cho $A = \begin{pmatrix} 2 & 1 & 0 \\ 0 & -1 & 3 \end{pmatrix}$ và $x = (1, 2, -1)^\top$. Tính Ax bằng cách tính từng hàng, rồi kiểm kích thước kết quả.

Đáp án. Hàng đầu cho $2 \cdot 1 + 1 \cdot 2 + 0 \cdot (-1) = 4$. Hàng sau cho $0 \cdot 1 + (-1) \cdot 2 + 3 \cdot (-1) = -5$. Vậy $Ax = (4, -5)^\top$ có kích thước 2×1 (tự tính). Kết quả có hai thành phần vì ma trận có hai hàng.

Bài 5 (khó). Dữ liệu gồm bốn mẫu, mỗi mẫu có hai đặc trưng. Nếu ma trận được xếp theo quy ước mẫu nằm theo hàng, hãy nêu kích thước của X và của vector trung bình theo đặc trưng $\bar{x}$.

Đáp án. Có bốn hàng mẫu và hai cột đặc trưng, vì vậy $X \in \mathbb{R}^{4\times 2}$. Lấy trung bình mỗi cột cho một giá trị trên mỗi đặc trưng, nên $\bar{x} \in \mathbb{R}^2$ (tự tính). Khi đưa vào công thức cụ thể, kiểm xem vector trung bình được

viết hàng hay cột.

Tài liệu tham khảo nền: Strang, Introduction to Linear Algebra; Deisenroth, Faisal và Ong, Mathematics for Machine Learning; Goodfellow, Bengio và Courville, Deep Learning; các bài math 01, math 03, math 04, math 15 và math 17 trong repo.

Nền tảng 02

Vector và ma trận như phép biến đổi: mỗi ma trận đưa một điểm đầu vào đến một điểm đầu ra

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: math nen01 về ký hiệu, kích thước và phép cộng theo chỉ số

Dùng ở: math 03 (PCA), math 04 (randomized SVD), math 05 (spectral clustering), math 17 (spectral normalization), math 19 (intrinsic dimension), math 20 (độ sâu mạng)

> Chương này xây trực giác hình học cho phép nhân ma trận, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao xem ma trận như một phép biến đổi

Ma trận không chỉ là một bảng số. Nó là một quy tắc nhận một vector đầu vào và trả về một vector đầu ra. Quy tắc đó có thể xoay, kéo dài, nghiêng, phản xạ hoặc làm mất một số chiều. Khi ta đọc ma trận theo cách này, phép nhân ma trận bớt giống một công thức cần học thuộc và trở thành một câu hỏi trực quan: mỗi hướng cơ bản được đưa đi đâu?

Trong AI, một lớp tuyến tính lấy biểu diễn hiện tại của mẫu và đổi nó thành biểu diễn tiếp theo. Ma trận trọng số ghi cách pha trộn các thành phần đầu vào để tạo từng thành phần đầu ra. Ghép nhiều lớp tuyến tính nghĩa là ghép nhiều phép biến đổi. Chương này dựng ý đó từ hình vuông đơn vị, vector cơ sở và phép tính tay cỡ nhỏ.

Định nghĩa giản dị: **một phép biến đổi tuyến tính là một cách đổi vector sao cho gốc vẫn ở gốc và phép cộng, nhân với số được bảo toàn.** Ma trận là cách ghi phép biến đổi ấy trong một hệ toạ độ đã chọn.

2. Nhắc nền: điểm, vector và hướng

Một vector hai chiều có thể được xem như một mũi tên từ gốc toạ độ đến một điểm. Ví dụ $x = (2, 1)^\top$ đi hai đơn vị sang phải và một đơn vị lên trên. Thành phần thứ nhất cho độ dịch ngang, thành phần thứ hai cho độ dịch dọc. Cùng vector đó có thể đặt ở nhiều vị trí khác nhau như mũi tên, nhưng phép biến đổi tuyến tính thường được hình dung qua vector đặt gốc ở $(0, 0)$.

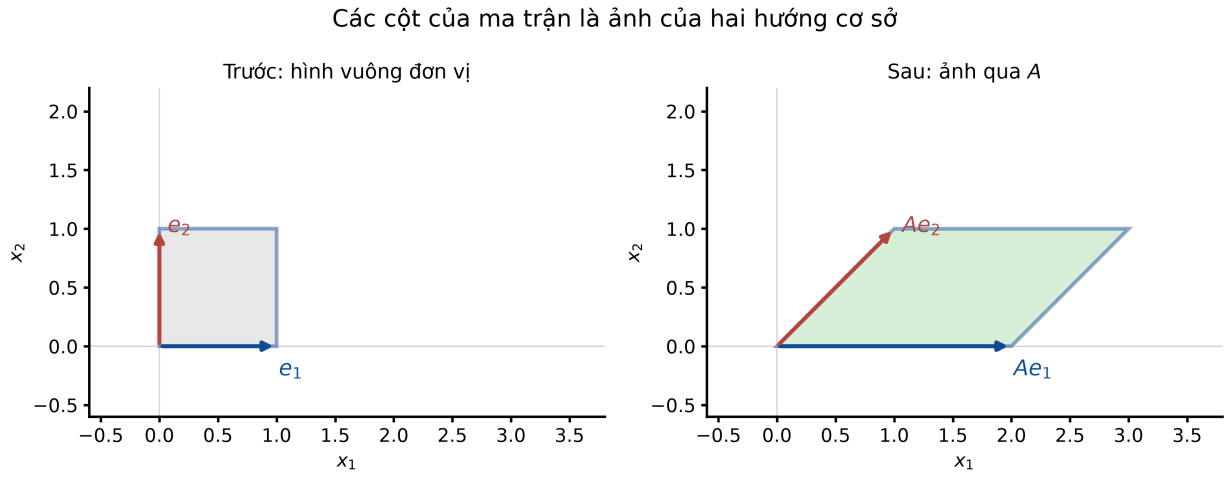
Vector cơ sở chuẩn của mặt phẳng là $e_1 = (1, 0)^\top$ và $e_2 = (0, 1)^\top$. Chúng là các mũi tên đơn vị theo trục ngang và trục dọc. Bất kỳ vector $x = (x_1, x_2)^\top$ nào cũng được ghép từ chúng: $x = x_1 e_1 + x_2 e_2$. Ví dụ (tự tính): $(3, -2)^\top = 3e_1 - 2e_2$.

Điều này quan trọng vì phép biến đổi tuyến tính chỉ cần cho ta biết hai mũi tên cơ sở đi đâu. Tính chất tuyến tính sẽ suy ra ảnh của mọi vector khác bằng cách cộng và co giãn hai ảnh đó.

3. Trực giác đầu tiên: hình vuông thành hình bình hành

Xem ma trận như một phép biến đổi nghĩa là theo dõi nơi mỗi điểm đi đến. Hình vuông đơn vị có bốn đỉnh, trong đó hai cạnh xuất phát từ gốc chính là e_1 và e_2. Khi áp ma trận A lên mọi điểm, hai cạnh đó thành hai cột của A. Hình minh hoạ dùng

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}.$$



Hình 6: Ma trận A đưa hình vuông đơn vị thành hình bình hành, hai cạnh mới là ảnh của hai vector cơ sở (minh hoạ, tự tính)

Cách đọc hình 1: hình vuông xám ở bên trái có hai cạnh e_1 và e_2. Sau biến đổi, cạnh ngang thành cột thứ nhất $(2, 0)^\top$, còn cạnh dọc thành cột thứ hai $(1, 1)^\top$. Hai ảnh này tạo hai cạnh của hình bình hành xanh. Đỉnh đối diện gốc là tổng hai cột $(3, 1)^\top$ (tự tính), vì đỉnh ban đầu là $e_1 + e_2$.

Hình đã gợi ra một quy tắc: **cột thứ j của ma trận là nơi vector cơ sở thứ j được đưa tới.** Phần sau chứng minh quy tắc đó từ tính tuyến tính.

4. Định nghĩa chính thức: bảo toàn cộng và co giãn

Một hàm T từ $\mathbb{R}^n$ sang $\mathbb{R}^m$ là tuyến tính nếu với mọi vector x, y và số a:

$$T(x + y) = T(x) + T(y), \qquad T(ax) = aT(x).$$

Hai quy tắc gộp lại thành $T(ax + by) = aT(x) + bT(y)$. Nói bằng lời, ảnh của một tổng bằng tổng các ảnh; nhân đầu vào với số a thì ảnh cũng được nhân với số a. Riêng khi đặt $a = 0$, ta có $T(0) = 0$. Mọi phép tuyến tính phải giữ nguyên gốc toạ độ.

Với vector cột, một ma trận $A \in \mathbb{R}^{m \times n}$ định nghĩa phép biến đổi $T(x) = Ax$. Đầu vào có n thành phần, đầu ra có m thành phần. Viết theo từng hàng:

$$y_i = \sum_{j=1}^{n} A_{ij} x_j, \qquad i = 1, \ldots, m.$$

Mỗi thành phần đầu ra là tổng có trọng số của các thành phần đầu vào. Hàng i của A là bộ trọng số để tính y_i.

Viết theo cột lại cho một góc nhìn khác:

$$Ax = x_1 A_{:,1} + x_2 A_{:,2} + \cdots + x_n A_{:,n}.$$

Ở đây $A_{:,j}$ là cột thứ j. Đầu ra là tổng các cột, trong đó mỗi cột được nhân với độ lớn tương ứng x_j. Hai cách nhìn, theo hàng và theo cột, là cùng một phép nhân.

5. Ví dụ tay: nhân một vector theo từng hàng và từng cột

Ta dùng lại ma trận của hình:

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}, \qquad x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

5.1 Cách tính theo hàng

Hàng đầu của A cho thành phần đầu ra thứ nhất:

$$y_1 = 2 \cdot 1 + 1 \cdot 2 = 4.$$

Hàng thứ hai cho thành phần đầu ra thứ hai:

$$y_2 = 0 \cdot 1 + 1 \cdot 2 = 2.$$

Vậy $Ax = (4, 2)^\top$ (tự tính). Ma trận có hai hàng nên đầu ra có hai thành phần. Đây là một cách kiểm tra nhanh kết quả.

5.2 Cách tính theo cột

Hai cột của A là $A_{:,1} = (2, 0)^\top$ và $A_{:,2} = (1, 1)^\top$. Vì $x = (1, 2)^\top$, ta lấy cột đầu một lần rồi cột hai hai lần:

$$Ax = 1 \begin{pmatrix} 2 \\ 0 \end{pmatrix} + 2 \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$$

(tự tính). Kết quả khớp với cách tính theo hàng. Cách nhìn theo cột thường dễ hình dung hình học; cách nhìn theo hàng thuận tiện khi tính số.

5.3 Kiểm tra tính tuyến tính bằng ví dụ

Ta có $A(e_1 + 2e_2) = Ae_1 + 2Ae_2$. Vế trái là $A(1, 2)^\top = (4, 2)^\top$, còn vế phải là $(2, 0)^\top + 2(1, 1)^\top = (4, 2)^\top$ (tự tính). Phép biến đổi không cần một quy tắc mới cho vector $(1, 2)$; ảnh được tạo bằng cách ghép ảnh các hướng cơ sở.

6. Những phép biến đổi cơ bản

6.1 Tỉ lệ và co giãn

Ma trận đường chéo $D = \mathrm{diag}(s_x, s_y)$ nhân trục ngang với s_x, trục dọc với s_y. Độ lớn $|s_x|$ và $|s_y|$ cho biết mỗi trục được co hay giãn bao nhiêu; dấu âm còn lật hướng trên trục tương ứng. Nếu hai hệ số cùng dấu và có cùng độ lớn, hình được co hoặc giãn đều. Nếu độ lớn của chúng khác nhau, hai trục bị co giãn không đều. Chẳng hạn $\mathrm{diag}(1, -1)$ giữ độ dài nhưng phản xạ qua trục ngang.

Ví dụ (tự tính): $D = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$ đưa $(1, 2)^\top$ thành $(2, 6)^\top$. Trục ngang được kéo dài gấp hai, trục dọc gấp ba. Gốc vẫn đứng yên.

6.2 Xoay và phản xạ

Ma trận xoay mặt phẳng một góc ϕ là

$$R_\phi = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}.$$

Xoay giữ độ dài và góc giữa các vector. Với góc 90 độ, các số lượng giác trở thành 0 và 1, nên $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. Khi nhân với $(2,1)^\top$, kết quả là $(-1,2)^\top$ (tự tính). Đây là một phép quay quanh gốc.

Phản xạ qua trục ngang có ma trận $F = \mathrm{diag}(1,-1)$. Nó giữ toạ độ ngang và đổi dấu toạ độ dọc. Ví dụ (tự tính): $F(2,1)^\top = (2,-1)^\top$. Xoay và phản xạ đều là phép tuyến tính.

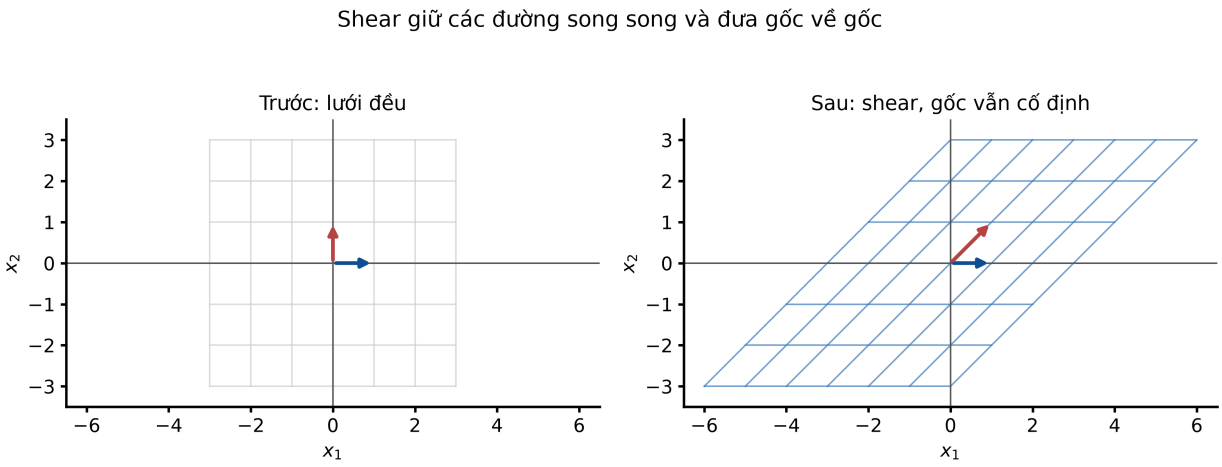
6.3 Nghiêng và cắt xiên

Phép shear (nghiêng) có thể được ghi bằng $S = \begin{pmatrix} 1 & s \\ 0 & 1 \end{pmatrix}$. Nó gửi $(x,y)^\top$ thành $(x+sy, y)^\top$: hàng ngang dịch tuỳ theo độ cao, còn độ cao được giữ nguyên. Với $s = 1$, hình vuông thành hình bình hành nhưng diện tích vẫn bằng nhau. Ví dụ (tự tính): $(1,2)^\top$ đi tới $(3,2)^\top$.

Phép nghiêng hữu ích để thấy ma trận không chỉ xoay các trục. Trong mạng tuyến tính, mỗi đầu ra có thể trộn nhiều đầu vào, giống như một hướng mới được dựng từ tổng có trọng số của các trục cũ.

7. Trực giác qua lưới toạ độ

Khi phép shear không suy biến tác động lên một lưới, các đường thẳng song song vẫn song song và các điểm trên cùng đường biến đổi nhất quán. Lưới giúp thấy biến dạng trên cả mặt phẳng, thay vì chỉ theo dõi một vector.



Hình 7: Lưới toạ độ dưới phép shear vẫn giữ gốc và các đường thẳng song song nhưng bị nghiêng (minh hoạ, tự tính)

Cách đọc hình 2: đường ngang và dọc xám là lưới ban đầu. Sau shear, một hướng cơ sở vẫn nằm ngang, còn hướng kia nghiêng vì ảnh của e_2 là $(1,1)^\top$; công thức $S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ xác nhận phép biến đổi cụ thể này bảo toàn cộng và co giãn. Chỉ thấy gốc đứng yên thì chưa đủ để kết luận một phép biến đổi bất kỳ là tuyến tính. Nếu ta dịch cả lưới sang bên, gốc không còn được giữ, khi đó phép biến đổi là affine chứ không còn tuyến tính.

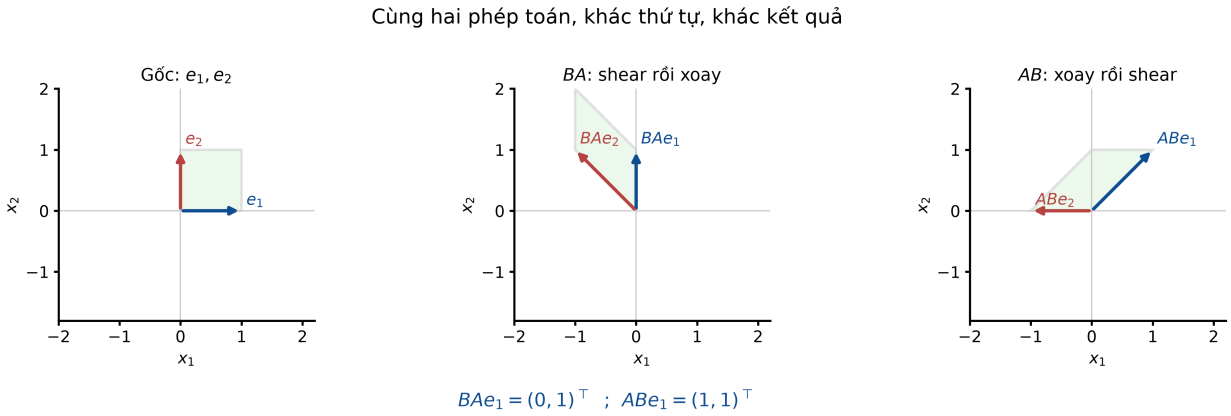
Với phép shear không suy biến trong hình, các đường song song vẫn song song. Một phép tuyến tính tổng

quát có thể suy biến: ảnh của một đường thẳng là đường thẳng hoặc một điểm, còn hai đường song song có thể thành hai đường song song, trùng nhau hoặc hai điểm. Không phải điểm nào cũng bị đưa đi một khoảng bằng nhau: điều được giữ là cấu trúc tuyến tính, không phải độ dài, góc hoặc diện tích nói chung.

8. Ghép phép biến đổi: vì sao thứ tự phép nhân quan trọng

Nếu A biến x thành Ax, rồi B biến kết quả thành $B(Ax)$, ma trận của phép ghép là BA. Ta viết từ trái sang phải theo thứ tự phép tính đại số, nhưng thực hiện từ phải sang trái: ma trận gần vector nhất chạy trước.

Hình so sánh hai thứ tự khác nhau: shear rồi xoay, và xoay rồi shear. Cùng hai thao tác nhưng nói chung cho kết quả khác.



Hình 8: Shear rồi xoay cho ảnh khác với xoay rồi shear, thể hiện thứ tự nhân ma trận (minh hoạ, tự tính)

Cách đọc hình 3: cột giữa minh hoạ BAx, tức shear A trước rồi xoay B; cột phải minh hoạ ABx, tức xoay trước rồi shear. Chọn $x = e_1$, hai đầu ra lần lượt là $(0,1)^\top$ và $(1,1)^\top$ (tự tính). Hai mũi tên cuối khác nhau, vì phép nhân ma trận thường không giao hoán: $BA \neq AB$.

Cụ thể, đặt $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ và $B = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. Ta có $BA = \begin{pmatrix} 0 & -1 \\ 1 & 1 \end{pmatrix}$ còn $AB = \begin{pmatrix} 1 & -1 \\ 1 & 0 \end{pmatrix}$ (tự tính). Hai kết quả đều là ma trận 2×2, nhưng các cột và hình dạng mà chúng tạo ra khác nhau.

Trong mạng nhiều lớp tuyến tính, phép biến đổi của lớp đầu được áp trước, rồi lớp tiếp theo. Nếu không có hàm kích hoạt phi tuyến ở giữa, cả chuỗi có thể gộp thành một ma trận tích. Điều này dẫn tới một câu hỏi quan trọng: vì sao mạng sâu hơn lại hữu ích? Câu trả lời liên quan đến phi tuyến và cách tham số hoá; độ sâu tuyến tính đơn thuần không làm lớp hàm biểu diễn phong phú hơn.

9. Ma trận vuông, ma trận chữ nhật và nghịch đảo

Ma trận vuông có cùng số hàng và cột, nên có thể biến vector trong cùng một không gian thành vector trong không gian đó. Ma trận chữ nhật có thể đổi số chiều, chẳng hạn $A \in \mathbb{R}^{3\times 2}$ nhận vector hai thành phần và trả vector ba thành phần. Ma trận này không thể có phép nghịch đảo hai phía theo nghĩa thông thường, vì đầu vào và đầu ra có kích thước khác nhau.

Với ma trận vuông, nếu tồn tại một phép biến đổi ngược đưa mọi kết quả trở về đúng đầu vào, ma trận khả nghịch. Khi A^{-1} tồn tại, $A^{-1}Ax = x$. Một điều kiện hình học tương đương là các hướng độc lập không bị ép chồng lên nhau và không chiều nào bị xoá. Chương nen03 sẽ định nghĩa hạng và không gian cột kỹ hơn.

Định thức của ma trận vuông hai chiều cung cấp một kiểm tra gọn: với $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, $\det(A) = ad - bc$. Trong mặt phẳng, diện tích được nhân với $|\det(A)|$; dấu của định thức báo định hướng có bị lật hay không.

Nếu định thức khác không, ma trận khả nghịch; nếu bằng không, diện tích bị co về một đường hoặc một điểm. Ta chỉ dùng đây như trực giác, chưa cần thuật toán tính định thức kích thước lớn.

9.1 Một ma trận có thể biến đổi nhiều vector cùng lúc

Gom nhiều vector đầu vào thành các cột của một ma trận X. Khi nhân AX, mỗi cột của X được biến đổi độc lập bởi cùng ma trận A. Nếu A cỡ $m \times n$ và X cỡ $n \times k$, thì AX cỡ $m \times k$: đầu ra có k cột, mỗi cột là một vector mới.

Ví dụ (tự tính): dùng ma trận A ở mục 5 và gom hai đầu vào thành $X = \begin{pmatrix} 1 & 1 \\ 2 & 0 \end{pmatrix}$. Khi đó cột đầu cho $(4, 2)^\top$, cột thứ hai cho $(2, 0)^\top$, vì vậy $AX = \begin{pmatrix} 4 & 2 \\ 2 & 0 \end{pmatrix}$. Đây chính là cách tính theo từng vector viết gọn thành một phép nhân. Khi dữ liệu được xếp theo hàng, ta thường nhân phía phải với ma trận trọng số đã chuyển vị; quy ước phải khớp với cách xếp mẫu.

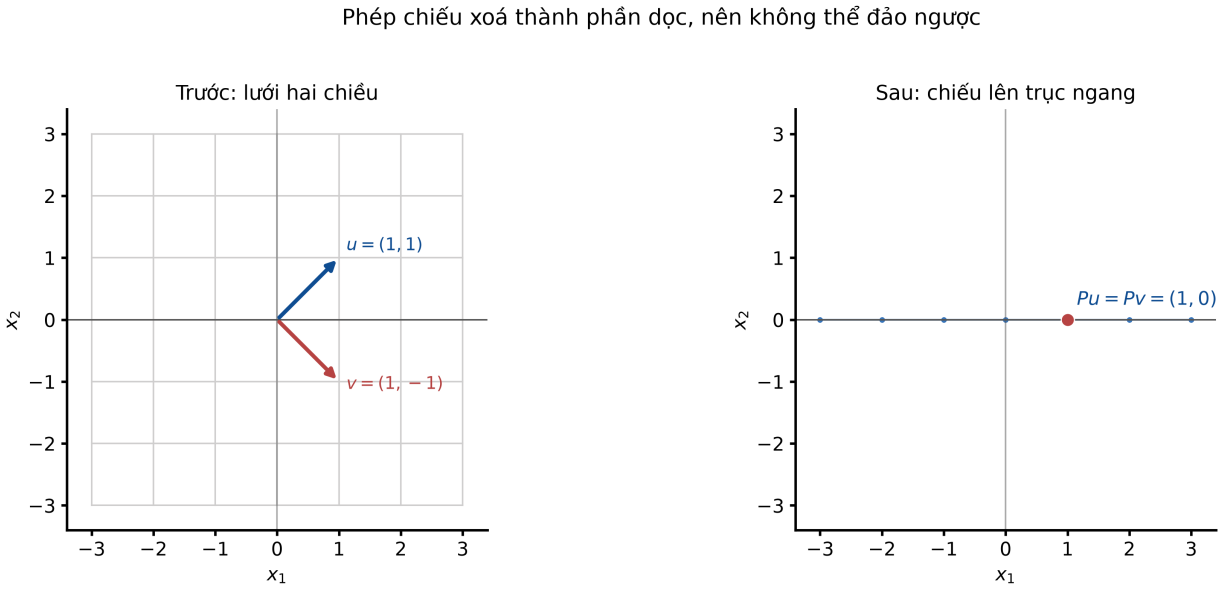
9.2 Vector hình học và danh sách toạ độ

Một vector hình học là đối tượng độc lập với cách ta ghi nó. Danh sách $(1, 2)^\top$ chỉ là toạ độ của vector đó trong một cơ sở đã chọn, chẳng hạn hai trục chuẩn e_1, e_2. Nếu đổi cơ sở, cùng điểm hình học sẽ có một danh sách toạ độ khác. Ma trận biểu diễn một phép biến đổi cũng phụ thuộc cơ sở đầu vào và cơ sở đầu ra.

Ở chương này, mọi hình vẽ dùng cơ sở chuẩn, nên các cột của ma trận đúng là ảnh của e_1, e_2. Khi học PCA, SVD hay trị riêng, ta sẽ đổi sang những hướng khác để mô tả cấu trúc dữ liệu thuận tiện hơn. Đừng lẫn việc đổi toạ độ với việc di chuyển bản thân đối tượng: điểm hình học có thể giữ nguyên trong khi các con số dùng để ghi nó thay đổi.

10. Khi phép biến đổi làm mất thông tin

Xét ma trận chiếu $P = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$. Nó giữ thành phần ngang và xoá thành phần dọc: $P(x, y)^\top = (x, 0)^\top$. Mọi điểm có cùng toạ độ ngang đều bị đưa đến cùng một điểm trên trục ngang. Vì thế không thể phục hồi duy nhất đầu vào từ đầu ra.



Hình 9: Ma trận chiếu diag một không làm phẳng lưới và xoá khác biệt theo phương dọc (minh hoạ, tự tính)

Cách đọc hình 4: lưới ban đầu ở bên trái có cả hai hướng. Sau chiếu, các đường ngang nằm trên trục hoành và các đường dọc sụp xuống thành một điểm trên trục đó. Hai vector $(1, 1)^\top$ và $(1, -1)^\top$ cùng thành $(1, 0)^\top$ (tự tính). Thành phần dọc bị mất, nên không có phép nghịch đảo khôi phục nó.

Các vector bị đưa về vector không tạo thành không gian-kernel, hay không gian-không. Với ma trận chiếu này, mọi vector dọc dạng $(0, t)^\top$ đều bị xoá. Hạng, kernel và chiều ảnh là các khái niệm liên quan trực tiếp; chúng được học hệ thống ở math nen03.

Trong dữ liệu, phép chiếu có thể là lựa chọn có chủ ý: giảm chiều, nén hoặc giữ các đặc trưng quan trọng. Nhưng nếu phép biến đổi xoá thông tin cần cho nhiệm vụ, mô hình sau đó không thể lấy lại thông tin chỉ từ đầu ra đã bị chiếu.

11. Cần phân biệt tuyến tính với affine

Phép biến đổi affine có dạng $T(x) = Ax + b$, trong đó b là vector dịch. Phần Ax đổi hướng và tỷ lệ, phần b tịnh tiến mọi điểm bằng cùng một lượng. Nếu $b \neq 0$, gốc bị đưa tới b, nên phép biến đổi không còn tuyến tính theo định nghĩa ở mục 4.

Ví dụ (tự tính): với $A = I$ và $b = (2, 0)^\top$, điểm $(1, 1)^\top$ được đưa thành $(3, 1)^\top$, còn gốc thành $(2, 0)^\top$. Phép này bảo toàn khoảng cách giữa các cặp điểm vì chỉ tịnh tiến, nhưng không giữ gốc.

Một lớp fully connected thường có trọng số W và bias b, nên ánh xạ của lớp trước hàm kích hoạt là affine: $z = Wx + b$. Khi $b = 0$, nó trở thành tuyến tính. Sự phân biệt hữu ích khi đọc mô hình: "ma trận trọng số" mô tả phần tuyến tính, bias mô tả dịch, còn hàm kích hoạt thường tạo phần phi tuyến.

12. Nối sang AI

(a) Một lớp tuyến tính trộn các đặc trưng. Nếu vector đầu vào chứa các đặc trưng của một mẫu, ma trận trọng số tính từng đầu ra bằng tổng có trọng số. Mỗi hàng của ma trận trả về một đầu ra; mỗi cột cho biết một đặc trưng đầu vào đóng góp thế nào vào toàn bộ đầu ra. Bias thêm một dịch chuyển chung.

(b) PCA đổi hệ trục để biểu diễn dữ liệu. Dữ liệu có thể được chiếu lên một số hướng mới để giữ biến thiên quan trọng. Trực giác phép biến đổi trong chương này là nền để hiểu vì sao PCA có thể quay dữ liệu, rồi bỏ các hướng ít quan trọng. Chi tiết nằm ở math 03; các kích thước và cách ghép các mẫu có thể khác giữa hai quy ước hàng và cột.

(c) Phân cụm phổ dùng ma trận để đổi biểu diễn đồ thị. Một ma trận liên kết hoặc Laplacian tác động lên vector đặc trưng của nút. Vector riêng của nó tạo các hướng hữu ích để biểu diễn cấu trúc đồ thị. Math 05 dùng ý niệm ma trận như toán tử, còn math nen04 sẽ giải thích trị riêng.

(d) Chuẩn phổ đo mức kéo giãn mạnh nhất. Một lớp tuyến tính có thể khuếch đại một hướng đầu vào mạnh hơn các hướng khác. Giá trị kỳ dị lớn nhất đo độ kéo giãn lớn nhất đó; math nen05 và math 17 dùng cách nhìn này để phân tích độ nhạy và spectral normalization.

(e) Ma trận chữ nhật xuất hiện tự nhiên khi số chiều đổi. Lớp có n đặc trưng đầu vào và m đặc trưng đầu ra dùng ma trận trọng số kích thước $m \times n$. Nhiều tham số mô hình thực sự là các ma trận chữ nhật; đừng ép chúng thành ma trận vuông trong suy nghĩ chỉ vì ma trận thường được vẽ vuông.

13. Sai lầm hay gặp

- **Đọc cột thành hàng.** Với quy ước vector cột, cột thứ j của A là Ae_j. Mỗi hàng lại dùng để tính một đầu ra; hai câu này cùng đúng.
- **Nhân theo thứ tự sai.** BAx nghĩa là A chạy trước rồi B. Đọc từ vector ra ngoài để biết thứ tự.
- **Giả sử phép biến đổi tuyến tính giữ độ dài và góc.** Chỉ một số ma trận đặc biệt như xoay mới có tính chất đó; ma trận bất kỳ có thể kéo giãn và nghiêng.

- **Cho rằng ma trận chữ nhật luôn đảo được.** Để có nghịch đảo thông thường, đầu vào và đầu ra phải cùng số chiều và phép biến đổi không được làm mất hướng.
- **Nhầm phép chiếu với phép quay.** Chiếu thường xoá một số thành phần, khiến nhiều đầu vào có cùng đầu ra; quay giữ đủ độ dài và thông tin.
- **Gọi mọi ánh xạ có bias là tuyến tính.** $Ax + b$ là affine; khi b khác vector không, gốc không còn được giữ.

14. Tóm tắt công thức

Khái niệm	Công thức	Cách đọc
Phép biến đổi ma trận	$y = Ax$	A nhận x và trả y
Kích thước	$A \in \mathbb{R}^{m \times n}$	n chiều vào, m chiều ra
Theo hàng	$y_i = \sum_j A_{ij} x_j$	hàng i tính đầu ra thứ i
Theo cột	$Ax = \sum_j x_j A_{:,j}$	ghép các ảnh của hướng cơ sở
Tính tuyến tính	$T(ax + by) = aT(x) + bT(y)$	bảo toàn cộng và co giãn
Cột ma trận	$Ae_j = A_{:,j}$	ảnh của hướng cơ sở thứ j
Ghép phép biến đổi	$B(Ax) = (BA)x$	phép gần vector nhất chạy trước
Phép affine	$T(x) = Ax + b$	tuyến tính cộng thêm dịch
Nghịch đảo	$A^{-1}Ax = x$	phép ngược khôi phục đầu vào
Định thức 2 chiều	$ad - bc$	trị tuyệt đối là hệ số diện tích; dấu cho biết định hướng

15. Bài tập tự tính

Năm bài tăng dần từ cột của ma trận tới ghép phép biến đổi. Hãy tự giải trước khi đọc đáp án. Các phép tính trong đáp án được kiểm bằng numpy.

Bài 1 (dễ). Với $A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$, tìm Ae_1 và Ae_2. Hãy mô tả mỗi kết quả bằng cột tương ứng của A.

Đáp án. $Ae_1 = (2, 0)^\top$ là cột thứ nhất, còn $Ae_2 = (1, 1)^\top$ là cột thứ hai (tự tính). Cột của ma trận ghi ảnh của các hướng cơ sở chuẩn.

Bài 2 (dễ vừa). Tính Ax với cùng A và $x = (1, 2)^\top$ theo cách tính từng hàng.

Đáp án. Thành phần đầu là $2 \cdot 1 + 1 \cdot 2 = 4$, thành phần sau là $0 \cdot 1 + 1 \cdot 2 = 2$. Vậy $Ax = (4, 2)^\top$ (tự tính). Kết quả có hai thành phần vì A có hai hàng.

Bài 3 (vừa). Cho phép shear $S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$. Tính ảnh của $(2, 3)^\top$ và giải thích ảnh của e_2.

Đáp án. $S(2, 3)^\top = (2 + 3, 3)^\top = (5, 3)^\top$ (tự tính). Ảnh của e_2 là $(1, 1)^\top$, nên hướng dọc nghiêng sang phải một đơn vị trong khi giữ thành phần dọc.

Bài 4 (vừa khó). Cho $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ và $B = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. Tính BAe_1 và ABe_1 để thấy thứ tự khác nhau.

Đáp án. $Ae_1 = e_1$, rồi $BAe_1 = Be_1 = (0, 1)^\top$. Mặt khác $Be_1 = (0, 1)^\top$, rồi $A(0, 1)^\top = (1, 1)^\top$, nên $ABe_1 = (1, 1)^\top$ (tự tính). Hai kết quả khác nhau, nên thứ tự phép biến đổi có ý nghĩa.

Bài 5 (khó). Xét $P = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$. Tìm ảnh của $u = (3, 2)^\top$ và $v = (3, -5)^\top$. Có thể dùng P để suy ra vector ban đầu duy nhất từ kết quả không?

Đáp án. $Pu = (3, 0)^\top$ và $Pv = (3, 0)^\top$ (tự tính). Vì hai vector đầu vào khác nhau mà ảnh giống nhau, không thể khôi phục đầu vào duy nhất. Thành phần dọc đã bị xoá.

Tài liệu tham khảo nền: Strang, Introduction to Linear Algebra; Deisenroth, Faisal và Ong, Mathematics for Machine Learning; Goodfellow, Bengio và Courville, Deep Learning; các bài math 03, math 04, math 05, math 17, math 19 và math 20 trong repo.

Nền tảng 03

Chuyển vị, trực giao và hạng: đọc hình dạng và số chiều của một phép biến đổi

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: tích vô hướng ở mục 7 của chương nền 01; vector, phép nhân ma trận với vector và tích ma trận ở chương nền 02. Độ dài vector được nhắc lại ở mục 2 của chương này.

Dùng ở: math 03 (PCA), math 04 (xấp xỉ hạng thấp), math 17 (chuẩn hoá phổ)

> Chương này xây dựng kiến thức từ đầu, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi ”tự tính” đã được kiểm bằng numpy.

1. Vì sao cần các khái niệm này

Một ma trận có thể được xem là một chiếc máy nhận vector và trả về vector. **Chuyển vị** đổi vai trò hàng và cột; **trực giao** mô tả các hướng vuông góc hoặc phép biến đổi giữ góc và độ dài; còn **hạng** đếm số hướng đầu ra độc lập mà máy có thể tạo ra. **Không gian cột** là tập mọi đầu ra có thể của máy. Những khái niệm này trả lời các câu hỏi khác nhau nhưng bổ sung cho nhau.

Nếu một lớp tuyến tính nhận nhiều đặc trưng rồi đưa ra ít hướng độc lập, một phần thông tin đầu vào có thể đã bị gộp. Nếu ta muốn xấp xỉ dữ liệu bằng vài hướng, ta cần biết chúng sinh ra không gian nào và có trùng lặp hay không. Khi lan truyền gradient, phép chuyển vị đưa thông tin từ phía đầu ra về phía đầu vào. Trong PCA, những hướng trực giao giúp ta mô tả dữ liệu bằng các trục dễ phân biệt. Chương này làm rõ từng câu, rồi chương tiếp theo dùng chúng để giải thích trị riêng.

Hãy giữ một phân biệt từ đầu: một cột là danh sách số, còn không gian cột là **cả tập** gồm mọi tổ hợp tuyến tính của các cột. Cũng vậy, hạng không đếm số ô khác 0 mà đếm số hướng độc lập. Hai cột đều khác 0 vẫn có thể cùng một hướng.

2. Nhắc nền: vector, tích vô hướng và tổ hợp tuyến tính

Vector trong mặt phẳng có thể biểu diễn bằng mũi tên xuất phát từ gốc. Với $u = (u_1, u_2)^\top$ và $v = (v_1, v_2)^\top$, tích vô hướng và độ dài được định nghĩa bởi

$$u^\top v = u_1 v_1 + u_2 v_2, \qquad \|u\|_2 = \sqrt{u^\top u}.$$

Chẳng hạn $u = (2, 1)^\top$ và $v = (-1, 2)^\top$ có tích vô hướng $2(-1) + 1(2) = 0$; mỗi vector dài $\sqrt{5}$ (tự tính). Cả hai đều khác 0 và vuông góc nhau. Nói hai vector **trực giao** nghĩa là tích vô hướng bằng 0; nói chúng **trực chuẩn** còn yêu cầu từng vector có độ dài 1. Vector 0 có tích vô hướng bằng 0 với mọi vector, nhưng không thể chuẩn hoá thành vector đơn vị.

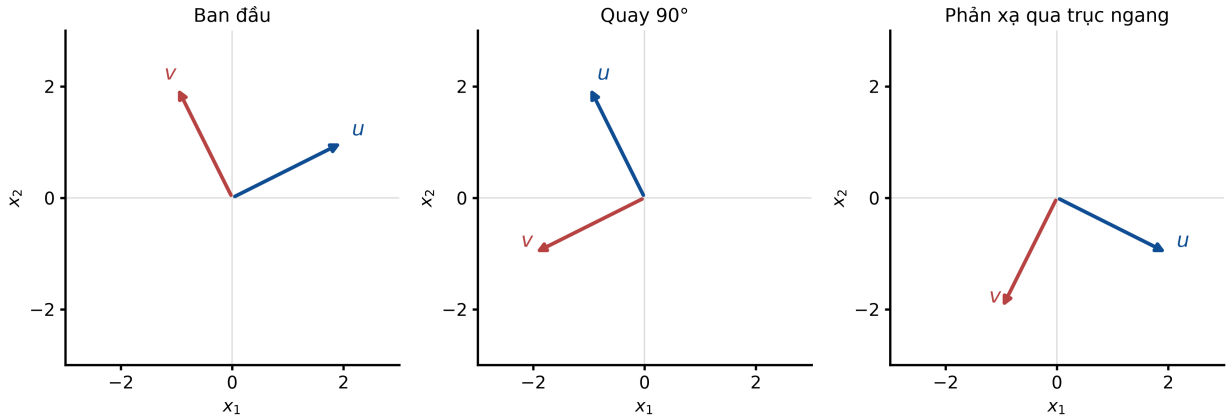
Một tổ hợp tuyến tính là phép cộng sau khi nhân các vector với các hệ số. Ví dụ, $(3, 2)^\top = 2(1, 0)^\top + 1(1, 2)^\top$ (tự tính). Với một ma trận A có các cột $a_1, \dots, a_n$ và vector x có các tọa độ $x_1, \dots, x_n$, phép nhân có thể đọc theo cột:

$$Ax = x_1 a_1 + \cdots + x_n a_n.$$

Đầu vào x ghi các hệ số; đầu ra là tổ hợp tương ứng của các cột. Chẳng hạn $A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$ và $x = (3, -1)^\top$ cho $Ax = 3(1,0)^\top - (2,1)^\top = (1,-1)^\top$ (tự tính). Cách đọc này sẽ giúp ta nhìn ra không gian cột mà không cần tính mọi đầu ra.

3. Nhìn hình trước: giữ hình dạng hay làm mất một chiều

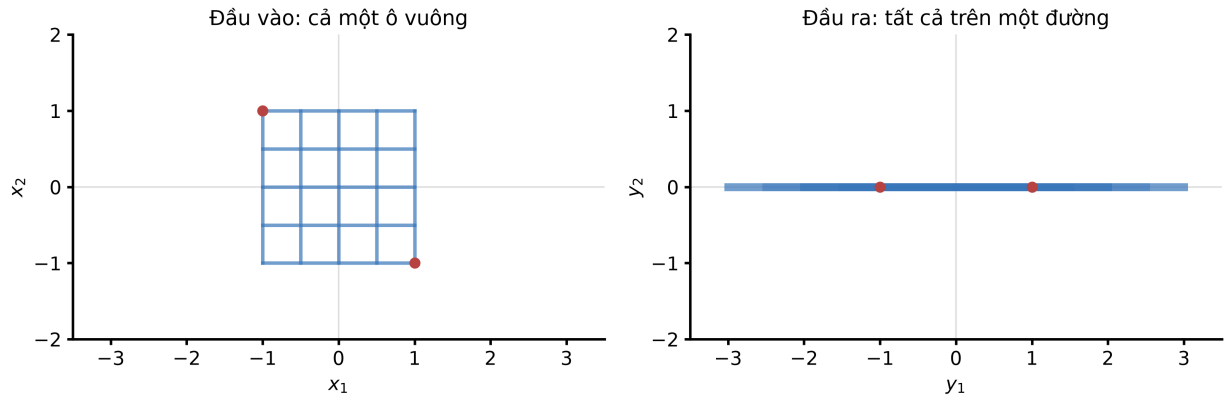
Hình đầu tiên đặt cùng hai mũi tên vuông góc vào ba khung. Phép quay và phép lật thay đổi vị trí của chúng, nhưng không bóp méo độ dài hoặc góc. Các tọa độ trên hai trục dùng cùng tỉ lệ để góc vuông không bị đánh lừa bởi cách vẽ.



Hình 10: Hai vector vuông góc trước và sau phép quay hoặc phản xạ (minh hoạ, tự tính)

Cách đọc hình 1: khung trái là hai vector ban đầu, khung giữa quay cả hai 90 độ, và khung phải phản xạ chúng qua trục ngang. Màu giữ vai trò của từng vector, nên ta thấy cả độ dài lẫn góc vuông đều được bảo toàn. Phản xạ làm đổi hướng của hệ tọa độ nhưng vẫn là một phép trực giao, không chỉ phép quay mới có tính chất đó.

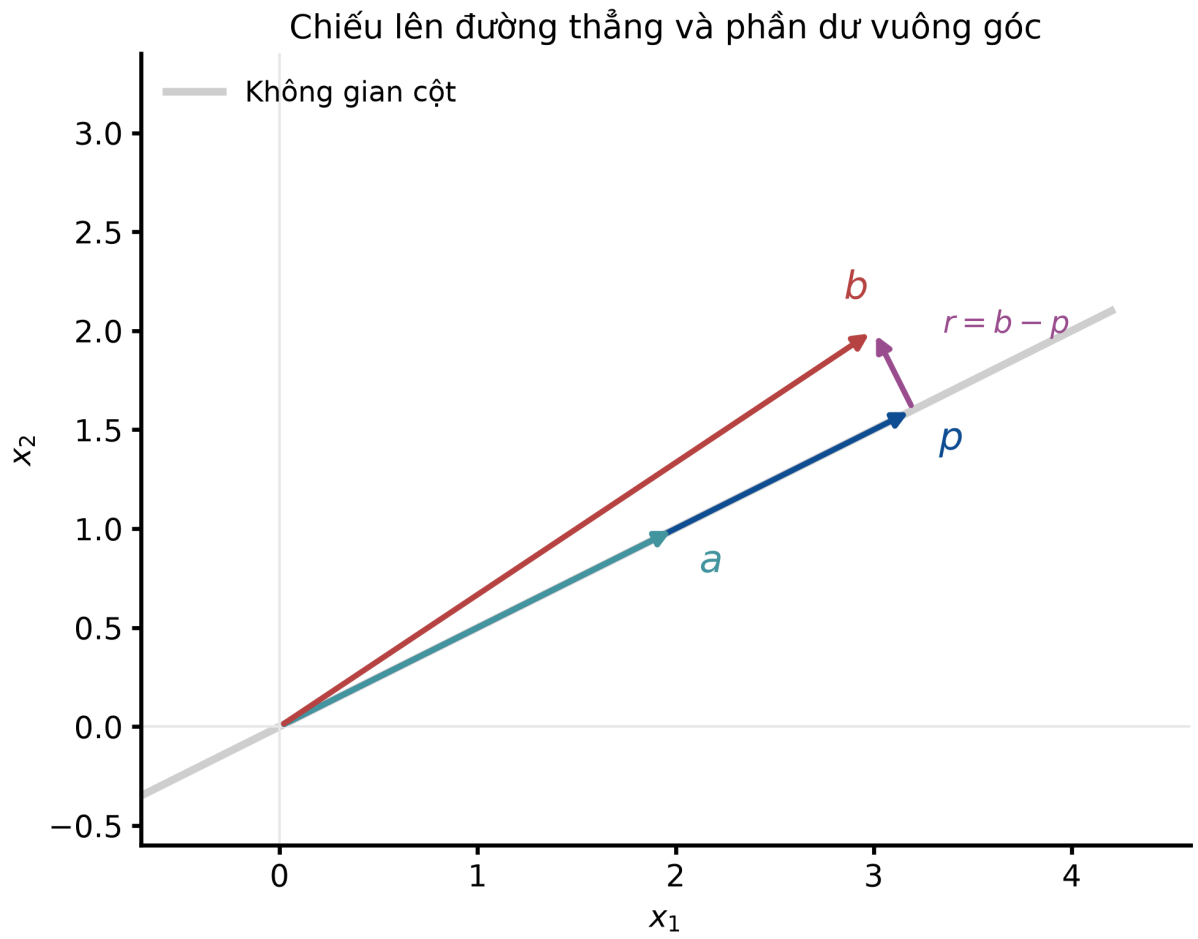
Không phải ma trận nào cũng giữ nguyên số hướng. Ở hình sau, một ô vuông đầy đủ được đưa thành một đoạn nằm ngang. Nhiều điểm đầu vào khác nhau có thể đổ vào cùng một điểm đầu ra: từ một chiều, ta không thể khôi phục duy nhất tất cả thông tin của hai chiều ban đầu.



Hình 11: Một phép biến đổi hạng một đưa lưới hai chiều thành một đoạn thẳng (minh hoạ, tự tính)

Cách đọc hình 2: bên trái là lưới của ô vuông đầu vào; bên phải là ảnh của các đường lưới qua cùng một ma trận. Tất cả ảnh nằm trên trục ngang, nên không gian đầu ra thực sự sử dụng chỉ có một chiều, dù mỗi đầu ra vẫn được ghi bằng hai tọa độ. Hai chấm đỏ là ảnh của hai điểm cụ thể, giúp thấy lưới không chỉ được vẽ lại mà đã bị biến đổi.

Còn trong bài toán xấp xỉ, có thể b không nằm trên đường mà ma trận sinh ra. Khi ấy ta tìm điểm gần nhất p trên đường. Đoạn từ p tới b vuông góc với đường: nếu còn thành phần dọc đường thì ta vẫn di chuyển được để gần b hơn.



Hình 12: Chiếu vector b lên một đường thẳng và phần dư vuông góc (minh hoạ, tự tính)

Cách đọc hình 3: đường xám là mọi bội của a; mũi tên xanh p là điểm trên đường gần đầu mũi tên đỏ b nhất. Mũi tên tím là phần dư $r = b - p$, vuông góc với đường, còn a chỉ một hướng sinh ra đường đó. Ở ví dụ trong hình, $p = (3{,}2,\, 1{,}6)^\top$ và $r = (-0{,}2,\, 0{,}4)^\top$ (tự tính); ta sẽ tính lại từng bước ở mục 7.

4. Chuyển vị: đổi hàng thành cột

Nếu A có m hàng và n cột thì **chuyển vị** $A^\top$ có n hàng và m cột, với $(A^\top)_{ij} = A_{ji}$. Ta lấy hàng thứ nhất của A làm cột thứ nhất của $A^\top$, và tiếp tục như vậy. Với

$$B = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \end{pmatrix}, \qquad B^\top = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 3 & 2 \end{pmatrix},$$

B cỡ 2×3, còn $B^\top$ cỡ 3×2 (tự tính). Chuyển vị tồn tại cho mọi ma trận, kể cả chữ nhật hoặc suy biến; nó không đòi hỏi định thức khác 0. Ngược lại, ma trận nghịch đảo thông thường chỉ tồn tại với ma trận vuông khả nghịch. Hai phép này không giống nhau nói chung.

Các quy tắc thường dùng, với kích thước các tích phù hợp, là

$$(A^\top)^\top = A, \qquad (A+C)^\top = A^\top + C^\top, \qquad (AB)^\top = B^\top A^\top.$$

Thứ tự bị đảo ở quy tắc cuối. Ta có thể kiểm từng ô hoặc kiểm kích thước: nếu A cỡ $m \times n$ và B cỡ $n \times p$, thì $(AB)^\top$ cỡ $p \times m$, đúng cỡ với $B^\top A^\top$. Viết $A^\top B^\top$ không chỉ sai thứ tự, mà trong nhiều trường hợp còn không nhân được.

4.1 Chuyển vị chuyển câu hỏi từ đầu ra về đầu vào

Với A cỡ $m \times n$, $x \in \mathbb{R}^n$ và $y \in \mathbb{R}^m$, ta có

$$(Ax)^\top y = x^\top A^\top y.$$

Vế trái hỏi đầu ra Ax khớp với y thế nào. Vế phải cho cùng một con số bằng cách đưa y qua $A^\top$ trước rồi so với x. Các kích thước cho thấy $A^\top y$ thuộc không gian đầu vào, không phải đầu ra. Ví dụ với B ở trên, $x = (1, 0, -1)^\top$, $y = (2, 1)^\top$, ta được $Bx = (-2, -2)^\top$, $(Bx)^\top y = -6$, $B^\top y = (2, 5, 8)^\top$ và $x^\top B^\top y = -6$ (tự tính).

Khi $A^\top = A$, ma trận được gọi là **đối xứng**. Nó phải vuông; ví dụ $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ là đối xứng. Đối xứng không có nghĩa là trực giao: bình phương ma trận ví dụ không phải ma trận đơn vị (tự tính). Ma trận đối xứng sẽ xuất hiện nhiều khi học trị riêng và Hessian.

4.2 Ma trận Gram

Nếu các cột của A là $a_1, \dots, a_n$ thì ô hàng i, cột j của $A^\top A$ chính là $a_i^\top a_j$. Vì thế $A^\top A$ tập hợp độ dài bình phương và tích vô hướng của các cột; nó được gọi là ma trận Gram. Dùng B ở đầu mục này,

$$B^\top B = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 8 \\ 3 & 8 & 13 \end{pmatrix}.$$

Ví dụ ô giữa đường chéo bằng $2^2 + 1^2 = 5$, còn ô hàng 2 cột 3 bằng $2 \cdot 3 + 1 \cdot 2 = 8$ (tự tính). Ma trận Gram luôn đối xứng và $z^\top A^\top A z = \|Az\|_2^2 \geq 0$. Ta sẽ dùng nhận xét này khi giải thích bình phương tối thiểu và trị riêng của ma trận đối xứng.

5. Trực giao và trực chuẩn

Một tập vector $q_1, \dots, q_k$ là **trực chuẩn** nếu mỗi vector dài 1 và hai vector khác nhau bất kỳ có tích vô hướng 0. Nếu đặt chúng thành các cột của Q, định nghĩa gọn là

$$Q^\top Q = I_k.$$

Các ô ngoài đường chéo của $Q^\top Q$ kiểm tra vuông góc, còn các ô đường chéo kiểm tra độ dài bằng 1. Chẳng hạn $q_1 = (1, 1)^\top / \sqrt{2}$ và $q_2 = (1, -1)^\top / \sqrt{2}$ tạo thành một cặp trực chuẩn (tự tính). Nếu chỉ vuông góc mà độ dài chưa bằng 1, ta có thể chuẩn hoá từng vector khác 0 bằng cách chia cho độ dài của nó.

5.1 Ma trận trực giao vuông

Ma trận vuông thực Q được gọi là **ma trận trực giao** khi $Q^\top Q = I$. Khi đó $Q^{-1} = Q^\top$ và cũng có $QQ^\top = I$. Đẳng thức nghịch đảo là một tính chất đặc biệt của lớp ma trận này, không phải định nghĩa của chuyển vị nói chung.

Ma trận quay ở hình 1 là $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. Tính trực tiếp $R^\top R = I$, $R(2,1)^\top = (-1,2)^\top$ (tự tính). Ma trận $F = \mathrm{diag}(1,-1)$ lật dấu tọa độ thứ hai và cũng thoả $F^\top F = I$ (tự tính). Phép quay có định thức 1; phép phản xạ ví dụ có định thức âm 1. Từ $\det(Q^\top Q) = \det(Q)^2 = 1$, mọi ma trận trực giao vuông có định thức bằng 1 hoặc âm 1.

Chúng giữ tích vô hướng vì

$$(Qu)^\top (Qv) = u^\top Q^\top Q v = u^\top v.$$

Cho $u = v$ ta được $\|Qu\|_2 = \|u\|_2$; vì độ dài và tích vô hướng giữ nguyên, góc giữa hai vector khác 0 cũng giữ nguyên. Hãy liên hệ phép tính này với hình 1: công thức bảo đảm điều mà ta vừa nhìn thấy cho **mọi** cặp vector, không chỉ cho hai mũi tên đã chọn.

5.2 Cột trực chuẩn nhưng ma trận chữ nhật

Nếu Q có m hàng, k cột trực chuẩn với $k < m$, ta vẫn có $Q^\top Q = I_k$ và $\|Qz\|_2 = \|z\|_2$ với $z \in \mathbb{R}^k$. Nhưng $QQ^\top$ cỡ $m \times m$ và **không** bằng I_m: nó chiếu lên không gian do các cột của Q sinh ra. Không gọi $Q^\top$ là nghịch đảo hai phía của ma trận chữ nhật.

Ví dụ $q = (1,0)^\top$ được xem là ma trận 2×1: $q^\top q = (1)$, nhưng $qq^\top = \mathrm{diag}(1,0)$ (tự tính). Vector $(3,4)^\top$ qua $qq^\top$ thành $(3,0)^\top$, mất thành phần dọc trục đứng. Một công thức quen khi các cột trực chuẩn là $p = QQ^\top b$, tức lấy tọa độ $Q^\top b$ rồi tạo lại vector bằng Q.

6. Không gian cột, hạng và không gian nghiệm bằng 0

Giả sử A cỡ $m \times n$ có các cột a_j. **Không gian cột** của A là

$$\mathrm{Col}(A) = \{Ax : x \in \mathbb{R}^n\} = \{x_1 a_1 + \cdots + x_n a_n : x_j \in \mathbb{R}\}.$$

Nó nằm trong $\mathbb{R}^m$, vì mỗi đầu ra có m tọa độ. Tập này luôn chứa vector 0 và đóng dưới cộng và nhân với số thực, nên là một không gian con. Với hình 2, $A = \begin{pmatrix} 1 & 2 \\ 0 & 0 \end{pmatrix}$ có hai cột $(1,0)^\top$ và $(2,0)^\top$. Chúng cùng phương; không gian cột là toàn bộ trục ngang, còn ảnh của riêng ô vuông đầu vào chỉ là **một đoạn** của trục ấy (tự tính).

Hạng $\mathrm{rank}(A)$ là số chiều của không gian cột, hay số cột độc lập tuyến tính lớn nhất. Nó cũng bằng số hàng độc lập lớn nhất, và luôn không vượt quá $\min(m,n)$. Ma trận ở hình 2 có hạng 1 dù có hai cột khác 0. Một ma trận toàn 0 có hạng 0; ma trận vuông $n \times n$ hạng n có nghịch đảo.

6.1 Một ví dụ ba cột, từng bước

Xét ma trận và phương trình

$$C = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \qquad Cx = b.$$

Gọi ba cột của C lần lượt là c_1, c_2, c_3. Ta có $c_3 = c_1 + c_2$, nên cột thứ ba không bổ sung hướng mới (tự tính). Còn c_1 và c_2 độc lập: nếu $\alpha c_1 + \beta c_2 = 0$, xem tọa độ thứ nhất cho $\alpha = 0$, tọa độ thứ hai cho $\beta = 0$. Vì thế $\text{rank}(C) = 2$; không gian cột là mặt phẳng do c_1, c_2 sinh ra trong $\mathbb{R}^3$.

Một vector $b = (b_1, b_2, b_3)^\top$ thuộc mặt phẳng này khi và chỉ khi $b_3 = b_1 + b_2$. Thật vậy, mọi tổ hợp $sc_1 + tc_2 = (s, t, s + t)^\top$ đều có tính chất ấy; chiều ngược lại, chọn $s = b_1$, $t = b_2$. Do đó $b = (2, 1, 3)^\top$ có nghiệm, còn $(2, 1, 4)^\top$ không phải đầu ra chính xác của C (tự tính).

6.2 Không gian null: những thay đổi không xuất hiện ở đầu ra

Không gian null hay nhân của A là

$$\text{Null}(A) = \{x \in \mathbb{R}^n : Ax = 0\}.$$

Nó thuộc không gian **đầu vào**, trái với không gian cột thuộc không gian **đầu ra**. Với C ở trên, $C(-1, -1, 1)^\top = 0$ (tự tính). Mọi nghiệm của $Cx = 0$ đều có dạng $t(-1, -1, 1)^\top$: hai hàng đầu cho $x_1 = -x_3$ và $x_2 = -x_3$, còn hàng cuối không thêm điều kiện. Nhân vì thế có một chiều.

Định lý hạng và số chiều null cho A cỡ $m \times n$:

$$\text{rank}(A) + \dim \text{Null}(A) = n.$$

Với C, ta kiểm được $2 + 1 = 3$ (tự tính). Nếu x_0 là một nghiệm của $Ax = b$, thì mọi $x_0 + z$ với $z \in \text{Null}(A)$ cũng là nghiệm. Ngược lại, hiệu của hai nghiệm bất kỳ luôn nằm trong null. Với $b = (2, 1, 3)^\top$, lấy $x_0 = (2, 1, 0)^\top$, ta được cả họ nghiệm $(2 - t, 1 - t, t)^\top$, trong đó t là số thực tùy ý (tự tính). Ma trận có thể có nghiệm cho một b mà vẫn không có nghiệm duy nhất.

6.3 Độc lập, phụ thuộc và điều kiện đầy hạng

Một tập cột độc lập nếu chỉ có cách đặt mọi hệ số bằng 0 mới cho tổng bằng vector 0. Nếu có một tổ hợp khác 0 cho tổng 0 thì các cột phụ thuộc, tức ít nhất một cột viết được từ các cột còn lại. Hạng cột đầy đủ nghĩa là hạng bằng số cột n, điều này chỉ có thể khi $n \le m$. Hạng hàng đầy đủ nghĩa là hạng bằng số hàng m, chỉ có thể khi $m \le n$. Hai điều này trùng nhau với ma trận vuông khả nghịch, nhưng khác nhau với ma trận chữ nhật.

Với A đầy hạng cột, $Ax = 0$ chỉ có nghiệm 0 nên nếu $Ax = b$ có nghiệm thì nghiệm là duy nhất. Với A đầy hạng hàng, mọi $b \in \mathbb{R}^m$ đều có ít nhất một nghiệm. Một ma trận rộng đầy hạng hàng vẫn có thể có vô số nghiệm vì còn hướng trong null. Các kết luận này được suy ra từ hạng và số chiều, không từ số lượng ô khác 0.

7. Chiếu trực giao và bình phương tối thiểu

Xét lại $a = (2, 1)^\top$ và $b = (3, 2)^\top$ ở hình 3. Ta muốn chọn số t sao cho ta gần b nhất. Viết $p = ta$ và phần dư $r = b - ta$. Ở nghiệm tốt nhất, phần dư vuông góc với hướng có thể di chuyển là a, nên $a^\top(b - ta) = 0$. Giải ra

$$t = \frac{a^\top b}{a^\top a} = \frac{8}{5}, \qquad p = \frac{8}{5}\begin{pmatrix}2\\1\end{pmatrix} = \begin{pmatrix}16/5\\8/5\end{pmatrix}, \qquad r = \begin{pmatrix}-1/5\\2/5\end{pmatrix}.$$

Kiểm từng bước: $a^\top b = 2\cdot 3 + 1\cdot 2 = 8$, $a^\top a = 5$, và $a^\top r = -2/5 + 2/5 = 0$ (tự tính). Vì p và r vuông góc, định lý Pythagore cho $\|b\|_2^2 = \|p\|_2^2 + \|r\|_2^2 = 64/5 + 1/5 = 13$ (tự tính). Điều này cũng xác nhận phân rã ở hình.

Tại sao vuông góc bảo đảm gần nhất? Với mọi điểm khác $p + sa$ trên đường, $b - (p + sa) = r - sa$. Do $r^\top a = 0$,

$$\|r - sa\|_2^2 = \|r\|_2^2 + s^2\|a\|_2^2 \geq \|r\|_2^2.$$

Vì vậy không điểm nào trên đường cho sai số nhỏ hơn p. Cùng lập luận áp dụng cho một không gian con nhiều chiều: phần dư phải vuông góc với mọi cột của A. Nghiệm bình phương tối thiểu $\hat{x}$ thỏa **phương trình chuẩn**

$$A^\top(A\hat{x} - b) = 0, \qquad A^\top A\hat{x} = A^\top b.$$

Nếu A đầy hạng cột thì $A^\top A$ khả nghịch và $\hat{x} = (A^\top A)^{-1}A^\top b$ là công thức đại số. Nếu các cột phụ thuộc, công thức dùng nghịch đảo này không hợp lệ: điểm chiếu $A\hat{x}$ vẫn duy nhất nhưng các hệ số $\hat{x}$ có thể không duy nhất. Trong tính toán số, thường giải bằng QR hoặc SVD thay vì lập nghịch đảo một cách tường minh; nếu lập $A^\top A$, điều kiện số có thể xấu đi.

8. Cơ sở trực chuẩn và QR

Một **cơ sở** của không gian con là một tập vector vừa sinh ra toàn bộ không gian đó vừa độc lập tuyến tính. Có nhiều cơ sở cho cùng một không gian. Cơ sở trực chuẩn đặc biệt tiện vì nếu b nằm trong không gian sinh bởi các cột của Q, tọa độ của b chính là $Q^\top b$. Không cần giải một hệ phương trình tổng quát để lấy tọa độ: tích vô hướng với từng cột đã cho từng hệ số.

Quá trình Gram-Schmidt biến một dãy vector độc lập thành các vector trực chuẩn sinh cùng không gian. Ví dụ $a_1 = (1,1)^\top$, $a_2 = (1,0)^\top$ (tự tính):

1. Chuẩn hoá a_1: $q_1 = (1,1)^\top/\sqrt{2}$.
2. Trừ khỏi a_2 phần dọc q_1: $w = a_2 - (q_1^\top a_2)q_1 = (1/2, -1/2)^\top$.
3. Chuẩn hoá w: $q_2 = (1,-1)^\top/\sqrt{2}$.

Khi đó $q_1^\top q_2 = 0$ và hai độ dài đều bằng 1 (tự tính). Các hệ số có thể được gom vào một ma trận tam giác trên:

$$A = \begin{pmatrix}1 & 1\\1 & 0\end{pmatrix} = QR, \quad Q = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\1 & -1\end{pmatrix}, \quad R = \begin{pmatrix}\sqrt{2} & 1/\sqrt{2}\\0 & 1/\sqrt{2}\end{pmatrix}.$$

Nhân lại QR cho đúng A (tự tính). Đây là **phân tích QR**. Với ma trận cao đầy hạng cột, dạng thu gọn có Q gồm các cột trực chuẩn và R vuông tam giác trên. Thư viện số thường dùng các biến thể ổn định hơn Gram-Schmidt cơ bản; ý tưởng hình học vẫn là tìm một cơ sở trực chuẩn cho không gian cột.

Nếu trong khi trực chuẩn hóa, phần còn lại bằng 0 thì vector đó đã thuộc không gian sinh bởi các vector trước và không tạo thêm hướng. Khi tính bằng số dấu phẩy động, phần còn lại rất nhỏ cần được xét cùng ngưỡng và độ lớn dữ liệu, không nên so sánh máy móc với 0 tuyệt đối.

9. Hạng đúng trên giấy và hạng trong tính toán

Trên giấy, chỉ cần khác 0 là một số khác 0, nên $D = \text{diag}(1, 0{,}001)$ có hạng 2, còn $E = \text{diag}(1, 0)$ có hạng 1 (tự tính). Tuy nhiên D đưa một hướng về độ dài rất nhỏ so với hướng còn lại. Với dữ liệu nhiễu hoặc tính toán hữu hạn chữ số, có thể khó phân biệt hướng nhỏ đó với nhiễu. Ta thường xem **hạng số học** là số giá trị kỳ dị vượt một ngưỡng, sẽ học kỹ ở chương SVD.

Ngưỡng phụ thuộc thang đo và mục đích: cùng một ma trận có thể được coi là gần hạng 1 trong một bài toán xấp xỉ, nhưng vẫn là hạng 2 về mặt toán học. Đừng nói "đã mất hạng" chỉ từ việc một giá trị nhỏ, và cũng đừng kết luận một hướng nhỏ là vô ích cho dự đoán. Nó có thể quan trọng với tác vụ dù đóng góp ít vào độ lớn tổng.

Hạng có các giới hạn hữu ích: $\text{rank}(AB) \le \min(\text{rank}(A), \text{rank}(B))$ khi tích tồn tại, vì ảnh của tích nằm trong ảnh của A và phụ thuộc vào ảnh của B. Một lớp tuyến tính đưa dữ liệu qua nút thắt hạng thấp không thể phục hồi thêm hướng chỉ bằng một lớp tuyến tính khác sau đó. Phát biểu này nói về tích tuyến tính; mạng có phi tuyến hoặc nhánh tắt cần được phân tích với cấu trúc thực của nó.

10. Nối sang AI

(a) PCA và cơ sở trực chuẩn. PCA tìm các hướng trực chuẩn để biểu diễn biến thiên của dữ liệu đã trừ trung bình. Lấy tích vô hướng với mỗi hướng cho tọa độ mới; giữ một số hướng tương ứng với chiếu lên không gian con do chúng sinh ra. Việc hướng trực giao giúp tránh hai trục mô tả cùng một chiều theo nghĩa hình học. Chi tiết các quy ước hàng, cột xem math 03.

(b) Nén qua hạng thấp. Nếu $W = UV$ với U cỡ $m \times r$ và V cỡ $r \times n$, thì $\text{rank}(W) \le r$. Lớp đầu đưa đầu vào về r tọa độ trung gian, lớp sau tạo đầu ra từ r hướng. Điều này giải thích tiềm năng giảm số tham số khi r đủ nhỏ, nhưng thay một ma trận bằng xấp xỉ hạng thấp luôn cần đo sai số và hiệu quả trên tác vụ. Các phương pháp tìm xấp xỉ cho ma trận lớn được bàn trong math 04.

(c) Chuyển vị trong lan truyền ngược. Với lớp $y = Ax$, nếu một hàm mất mát vô hướng có gradient theo đầu ra g_y, gradient theo đầu vào là $g_x = A^\top g_y$. Đây là hệ quả của đồng nhất thức ở mục 4.1 và chain rule, không phải phép đảo đầu ra để tìm lại đầu vào. Nếu A cỡ $m \times n$, g_y dài m và g_x dài n, nên kiểm kích thước giúp phát hiện lỗi sớm.

(d) Trực giao và bảo toàn độ lớn. Ma trận trực giao vuông giữ độ dài vector qua lớp tuyến tính, vì vậy là một trường hợp dễ phân tích khi nghiên cứu dòng tín hiệu. Ràng buộc các cột trực chuẩn là một lựa chọn khác với chỉ giới hạn độ giãn lớn nhất. Ghi chú math 17 so sánh các cách điều chuẩn như vậy; không nên suy ra một điều kiện hình học riêng lẻ tự bảo đảm toàn bộ quá trình huấn luyện tốt.

(e) Dữ liệu phụ thuộc tuyến tính. Nếu hai đặc trưng là bội chính xác của nhau, ma trận thiết kế có cột phụ thuộc. Hồi quy bình phương tối thiểu vẫn có thể dự đoán bằng điểm chiếu, nhưng các hệ số có thể không duy nhất. Nếu chúng chỉ gần phụ thuộc, nghiệm có thể nhạy với nhiễu; cần kiểm thang đo, điều kiện số và ý nghĩa của từng đặc trưng thay vì chỉ đếm số cột.

11. Sai lầm hay gặp

- **Coi chuyển vị là nghịch đảo.** $A^\top$ luôn định nghĩa; A^{-1} cần ma trận vuông khả nghịch. Chúng chỉ bằng nhau trong những trường hợp đặc biệt như ma trận trực giao vuông.
- **Đảo thứ tự sai.** $(AB)^\top = B^\top A^\top$, không phải $A^\top B^\top$. Kiểm kích thước trước khi nhân giúp phát hiện nhiều lỗi.
- **Nhầm vuông góc với trực chuẩn.** Trực chuẩn yêu cầu thêm mỗi vector có độ dài 1. Hai cột vuông góc nhưng chưa chuẩn hoá không thỏa $Q^\top Q = I$.
- **Suy $QQ^\top = I$ từ cột trực chuẩn cho mọi hình dạng.** Với Q cao, $QQ^\top$ là phép chiếu, không phải ma trận đơn vị trên toàn bộ không gian đầu ra.

- **Đếm cột khác 0 để tìm hạng.** Các cột có thể khác 0 nhưng phụ thuộc; phải đếm số hướng độc lập. Không gian cột nằm ở đầu ra, còn không gian null nằm ở đầu vào.
- **Đảo $A^\top A$ khi các cột phụ thuộc hoặc gần phụ thuộc.** Khi phụ thuộc chính xác, nghịch đảo không tồn tại; khi gần phụ thuộc, kết quả có thể rất nhạy. Dùng phương pháp số phù hợp và giải thích ngưỡng hạng.

12. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Chuyển vị	$(A^\top)_{ij} = A_{ji}$	đổi hàng và cột
Tích chuyển vị	$(AB)^\top = B^\top A^\top$	đổi thứ tự khi chuyển vị tích
Cột trực chuẩn	$Q^\top Q = I$	cột dài 1 và vuông góc từng đôi
Trực giao vuông	$Q^{-1} = Q^\top$	phép đảo bằng chuyển vị
Bảo toàn tích	$(Qu)^\top (Qv) = u^\top v$	giữ góc và độ dài
Không gian cột	$\mathrm{Col}(A) = \{Ax : x \in \mathbb{R}^n\}$	mọi đầu ra có thể
Hạng và null	$\mathrm{rank}(A) + \dim \mathrm{Null}(A) = n$	chia số chiều đầu vào
Chiếu trực chuẩn	$p = QQ^\top b$	điểm gần nhất trong không gian cột của Q
Bình phương tối thiểu	$A^\top (A\hat{x} - b) = 0$	phần dư vuông góc với mọi cột

13. Bài tập tự tính

Năm bài từ dễ đến khó. Hãy tự làm trước rồi xem lời giải ngay dưới mỗi bài; các kết quả đã được kiểm bằng numpy.

Bài 1 (dễ). Cho $A = \begin{pmatrix} 1 & 2 & 0 \\ -1 & 3 & 4 \end{pmatrix}$. Viết $A^\top$ và cho biết kích thước của nó.

Đáp án. Đổi hàng thành cột được $A^\top = \begin{pmatrix} 1 & -1 \\ 2 & 3 \\ 0 & 4 \end{pmatrix}$, cỡ 3×2 (tự tính). Ma trận ban đầu cỡ 2×3; không cần xét định thức hoặc tính nghịch đảo.

Bài 2 (dễ vừa). Các vector $u = (3, 4)^\top$ và $v = (-4, 3)^\top$ có trực giao không? Hãy tạo một cặp trực chuẩn cùng hướng với chúng.

Đáp án. $u^\top v = 3(-4) + 4(3) = 0$ và mỗi vector có độ dài 5 (tự tính). Chia từng vector cho 5, ta được $q_1 = (3, 4)^\top / 5$ và $q_2 = (-4, 3)^\top / 5$. Hai vector mới có độ dài 1 và tích vô hướng 0, nên tạo thành cặp trực chuẩn.

Bài 3 (vừa). Với $A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$, tìm hạng, không gian cột và một cơ sở của null.

Đáp án. Cột hai gấp đôi cột một khác 0, nên hạng bằng 1 và $\mathrm{Col}(A) = \{t(1, 2)^\top : t \in \mathbb{R}\}$. Giải $x_1 + 2x_2 = 0$ cho $x = s(-2, 1)^\top$; hàng thứ hai là bội của hàng đầu. Vậy một cơ sở của null là $\{(-2, 1)^\top\}$, và hạng cộng số chiều null bằng 2 (tự tính).

Bài 4 (vừa khó). Chiếu $b = (2, 1)^\top$ lên đường sinh bởi $a = (1, 1)^\top$. Tìm điểm chiếu và kiểm phần dư vuông góc.

Đáp án. $a^\top b = 3$ và $a^\top a = 2$, nên $p = (3/2)a = (3/2, 3/2)^\top$. Phần dư $r = b - p = (1/2, -1/2)^\top$ có $a^\top r = 0$ (tự tính). Như vậy p là điểm gần b nhất trên đường; không phải b vì hai tọa độ của b khác nhau.

Bài 5 (khó). Cho $A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$ và $b = (2, 3)^{\top}$. Tìm tất cả nghiệm $Ax = b$, sau đó xác định nghiệm có chuẩn nhỏ nhất.

Đáp án. Hai hàng cho $x_1 + x_3 = 2$, $x_2 + x_3 = 3$. Đặt $x_3 = t$ được $x = (2 - t, 3 - t, t)^{\top}$. Bình phương chuẩn bằng $(2-t)^2 + (3-t)^2 + t^2 = 3t^2 - 10t + 13 = 3(t - 5/3)^2 + 14/3$, nhỏ nhất khi $t = 5/3$. Vậy nghiệm chuẩn nhỏ nhất là $(1/3, 4/3, 5/3)^{\top}$, có bình phương chuẩn $14/3$ (tự tính). Tập nghiệm có một chiều null sinh bởi $(-1, -1, 1)^{\top}$, và nghiệm chuẩn nhỏ nhất vuông góc với vector đó.

Tài liệu tham khảo nền: Strang, Introduction to Linear Algebra; Trefethen và Bau, Numerical Linear Algebra; Shlens 2014 (math 03); Halko và cộng sự 2009 (math 04).

Nền tảng 04

Trị riêng và vector riêng: tìm những hướng đặc biệt của phép biến đổi

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector, nhân ma trận, tích vô hướng, trực giao, hạng và không gian cột ở chương nền 02 và 03

Dùng ở: math 03 (PCA), math 04 (phân tích ma trận ngẫu nhiên), math 05 (phân cụm phổ), math 17 (lặp lũy thừa)

> Chương này xây dựng khái niệm từ đầu, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần trị riêng và vector riêng

Một ma trận vuông thường vừa đổi độ dài vừa đổi hướng vector. **Vector riêng** là một vector khác 0 mà sau phép biến đổi vẫn nằm trên cùng đường thẳng với nó; **trị riêng** là hệ số cho biết vector đó được nhân lên, thu nhỏ, lật hướng hay đưa về 0. Các hướng này giúp ta đọc một phép biến đổi phức tạp bằng những hành vi đơn giản hơn.

Trong AI, ma trận hiệp phương sai có các hướng riêng cho biết dữ liệu trải rộng theo hướng nào, tạo cơ sở cho PCA. Trị riêng của Hessian mô tả độ cong theo các hướng quanh một điểm, giúp giải thích thung lũng hẹp hoặc điểm yên trong tối ưu. Trị riêng của ma trận Laplacian đồ thị liên quan đến cấu trúc kết nối trong phân cụm phổ. Cùng một công cụ xuất hiện nhiều nơi, nhưng ý nghĩa thực tế của trị riêng phụ thuộc vào **ma trận ta đang phân tích**.

Đây không phải công cụ áp cho mọi ma trận theo cùng một cách. Trị riêng thông thường được định nghĩa cho ma trận vuông vì đầu vào và đầu ra cần cùng không gian để so hướng. Ma trận thực vẫn có thể có trị riêng phức hoặc không có đủ vector riêng độc lập. Với ma trận bất kỳ, kể cả chữ nhật, chương tiếp theo giới thiệu SVD như một công cụ khác.

2. Nhắc nền: nhân ma trận và hướng

Với A cỡ 2×2, phép nhân Ax đưa một vector x trong mặt phẳng tới vector mới. Ví dụ

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad A \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

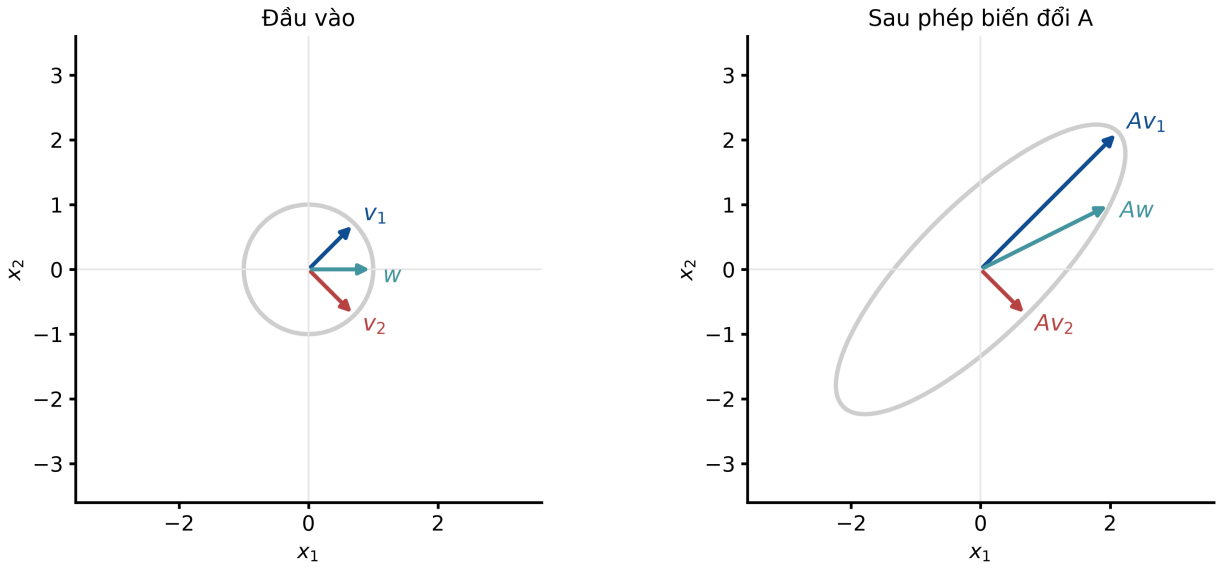
Vector $(1, 0)^\top$ hướng ngang, còn $(2, 1)^\top$ nghiêng lên, nên phép biến đổi đã đổi hướng vector này (tự tính). Nhưng $A(1, 1)^\top = (3, 3)^\top = 3(1, 1)^\top$: vector dọc đường chéo đi qua gốc vẫn nằm trên đường đó, chỉ dài gấp ba (tự tính). Câu hỏi của chương là làm sao tìm tất cả các hướng có tính chất như vậy.

Khi nói "cùng hướng riêng", ta quan tâm đường thẳng qua gốc. Nếu vector ảnh quay ngược chiều dọc đường thẳng, nó vẫn có thể là bội của vector gốc, chỉ là hệ số nhân âm. Phép nhân với 0 đưa vector về gốc, một trường hợp biên vẫn được cho phép trong định nghĩa đại số. Điều bị loại là chọn chính vector đầu vào bằng 0, vì vector 0 luôn thoả phương trình với bất kỳ hệ số nào và không xác định được hướng.

Một nhắc lại khác: nếu các cột của ma trận vuông Q trực chuẩn thì $Q^\top Q = I$ và $Q^{-1} = Q^\top$. Tính chất này giúp viết phép đổi sang hệ trục trực giao và đổi ngược lại. Ta sẽ chỉ dùng cách chéo hóa bằng Q trực giao khi đã có điều kiện thích hợp, đặc biệt là khi ma trận thực đối xứng.

3. Nhìn hình trước: hướng giữ đường thẳng

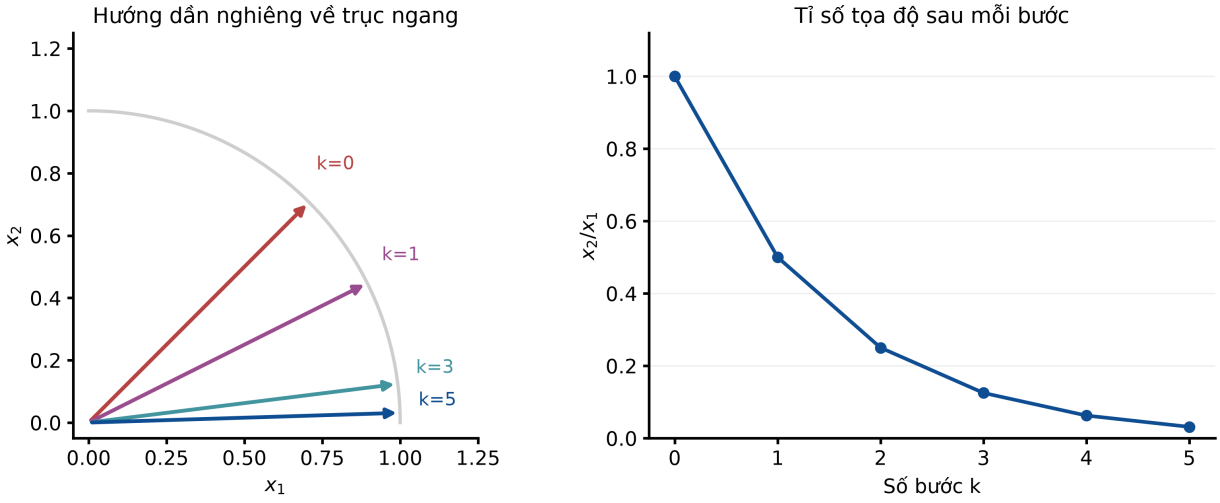
Hình dưới lấy các vector độ dài 1 trên vòng tròn và đưa chúng qua ma trận A của mục 2. Hai mũi tên xanh, đỏ được chọn theo hai đường chéo; mũi tên màu xanh ngọc hướng ngang là một hướng thông thường.



Hình 13: Vector riêng giữ đường thẳng khi vòng tròn bị biến thành elip (minh hoạ, tự tính)

Cách đọc hình 1: bên trái là vòng tròn ban đầu với hai vector riêng v_1, v_2 và một vector w khác. Bên phải, Av_1 vẫn trên đường của v_1 và dài gấp ba, còn Av_2 bằng chính v_2 (tự tính). Ngược lại, Aw không còn nằm ngang, nên w không phải vector riêng. Hình này dùng cùng tỉ lệ trên hai trục và cùng giới hạn ở hai khung, vì vậy có thể so độ dài một cách trực tiếp.

Điều gì xảy ra nếu liên tục áp một phép biến đổi? Với một ví dụ có một hướng được khuếch đại mạnh hơn, thành phần dọc hướng đó ngày càng chiếm ưu thế sau khi ta chuẩn hoá lại ở mỗi bước. Đây là trực giác của lặp lũy thừa, không phải quy luật cho mọi ma trận hoặc mọi điểm bắt đầu.



Hình 14: Lặp lũy thừa cho ma trận chéo với một trị riêng trội (minh hoạ, tự tính)

Cách đọc hình 2: bên trái, các mũi tên đã chuẩn hoá dần tiến về trục ngang khi số bước tăng. Bên phải, tỉ số giữa tọa độ đứng và ngang giảm theo dãy $1, 1/2, 1/4, \ldots$ (tự tính). Trục ngang trong đồ thị phải là số bước,

không phải một tọa độ không gian; ta sẽ giải thích vì sao mỗi bước giảm tỉ số một nửa ở mục 8.

4. Định nghĩa chính thức

Cho ma trận vuông $A \in \mathbb{R}^{n \times n}$. Nếu có vector $v \neq 0$ và một số λ sao cho

$$Av = \lambda v,$$

thì v được gọi là **vector riêng** của A ứng với **trị riêng** λ. Khi xét trên số thực, cả v lẫn λ đều thực; để bao gồm trường hợp phức, ta cho phép chúng thuộc không gian phức tương ứng. Tập mọi trị riêng của một ma trận còn gọi là **phổ trị riêng** của nó.

Khi $\lambda > 1$, vector được kéo dài; khi $0 < \lambda < 1$, vector ngắn đi mà không đổi chiều. Khi $\lambda < 0$, vector ảnh ngược chiều và có độ dài nhân với $|\lambda|$; khi $\lambda = 0$, ảnh là vector 0. Các mô tả độ dài này áp dụng cho vector riêng thực và trị riêng thực. Nếu v là vector riêng và $c \neq 0$, thì cv cũng là vector riêng cùng trị riêng, vì $A(cv) = cAv = \lambda(cv)$. Có thể chọn độ dài 1 để tiện so sánh, nhưng bản thân định nghĩa không bắt buộc chuẩn hoá.

Mọi vector riêng cùng trị riêng λ, cùng với vector 0, tạo thành **không gian riêng**

$$E_\lambda = \text{Null}(A - \lambda I) = \{v : (A - \lambda I)v = 0\}.$$

Không gian riêng có thể nhiều hơn một chiều. Chính vector 0 nằm trong không gian con này nhưng **không** được gọi là vector riêng. Một trị riêng bằng 0 tồn tại khi và chỉ khi null của A có vector khác 0, hay tương đương A không khả nghịch.

5. Tính tay từng bước với ma trận đối xứng

Ta dùng lại $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Đầu vào của phép tìm trị riêng là A, còn đầu ra cần tìm là các số λ và các vector v tương ứng. Mọi phép tính số của ví dụ này do ta tự làm và đã kiểm bằng numpy.

5.1 Bước 1: lập phương trình đặc trưng

Từ $Av = \lambda v$ ta chuyển về $(A - \lambda I)v = 0$. Ta cần nghiệm v khác 0, tức ma trận vuông $A - \lambda I$ không khả nghịch. Điều này tương đương định thức bằng 0:

$$\det(A - \lambda I) = \det \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix} = (2 - \lambda)^2 - 1 = 0.$$

Quy tắc định thức của ma trận $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ là $ad - bc$. Phương trình đặc trưng là một công cụ để tìm trị riêng; không có nghĩa là mọi ma trận chữ nhật đều có đa thức đặc trưng theo cách này.

5.2 Bước 2: giải hai trị riêng

Từ $(2 - \lambda)^2 = 1$ suy ra $2 - \lambda = 1$ hoặc $2 - \lambda = -1$. Do đó $\lambda_1 = 3$ và $\lambda_2 = 1$ (tự tính). Ta chọn đánh số giảm dần ở ví dụ này; việc đánh số không thay đổi bản chất của các trị riêng.

Kiểm nhanh bằng tổng và tích: $3 + 1 = 4 = \text{tr}(A)$ là tổng đường chéo, còn $3 \cdot 1 = 3 = \det(A) = 4 - 1$ (tự tính). Với ma trận vuông, nếu đếm trị riêng theo bội đại số trong trường phức, tổng bằng vết và tích bằng

định thức. Hai phép kiểm này hữu ích, nhưng tự chúng không chứng minh rằng vector riêng ta tìm tiếp theo đã đúng.

5.3 Bước 3: giải vector riêng ứng với từng trị

Với $\lambda_1 = 3$,

$$(A - 3I)v = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = 0.$$

Hàng đầu cho $-v_1 + v_2 = 0$, tức $v_2 = v_1$. Không gian riêng là mọi bội của $(1, 1)^\top$; chọn vector đơn vị $q_1 = (1, 1)^\top/\sqrt{2}$. Với $\lambda_2 = 1$,

$$(A - I)v = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = 0,$$

nên $v_2 = -v_1$ và ta chọn $q_2 = (1, -1)^\top/\sqrt{2}$. Ta kiểm trực tiếp $Aq_1 = 3q_1$, $Aq_2 = q_2$, $q_1^\top q_2 = 0$, và cả hai có độ dài 1 (tự tính). Hai dấu của một vector có thể cùng đổi; chẳng hạn $-q_1$ vẫn là vector riêng hợp lệ.

5.4 Bước 4: ghép lại thành phép đổi hệ trục

Đặt Q gồm hai vector riêng đơn vị làm cột, Λ chứa trị riêng tương ứng theo đúng thứ tự:

$$Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad \Lambda = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}.$$

Hai đẳng thức riêng $Aq_1 = 3q_1$ và $Aq_2 = q_2$ ghép thành $AQ = Q\Lambda$. Vì Q trực giao, nhân bên phải với $Q^\top$ cho

$$A = Q\Lambda Q^\top = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Tích cuối khôi phục đúng ma trận ban đầu (tự tính). Đọc từ phải sang trái: $Q^\top$ lấy tọa độ trong cơ sở riêng, Λ nhân từng tọa độ với trị riêng tương ứng, và Q đưa tọa độ về hệ ban đầu. Một phép biến đổi trông như vừa xoay vừa giãn trong hệ tọa độ cũ trở thành phép nhân theo từng trục trong hệ tọa độ riêng.

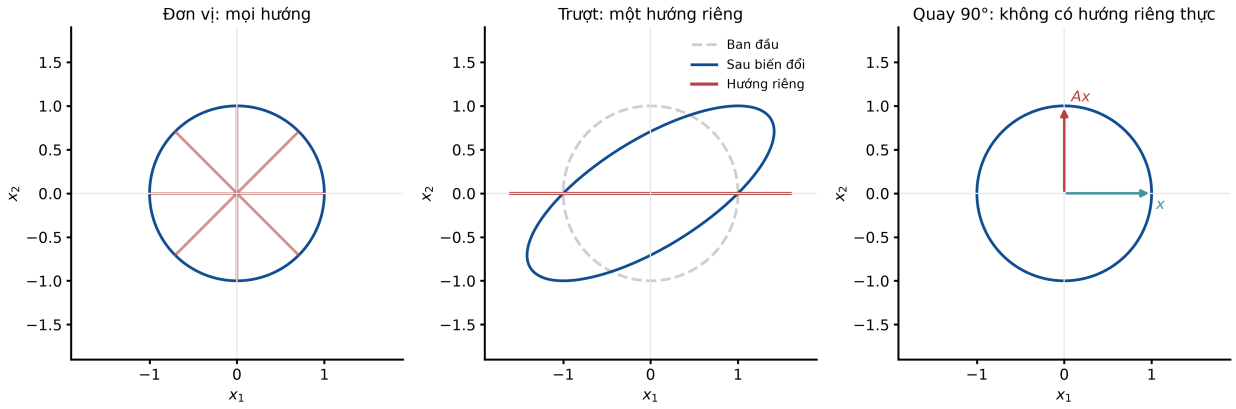
5.5 Bước 5: giải thích một vector không phải vector riêng

Với $w = (1, 0)^\top$, các tọa độ trong cơ sở riêng là $Q^\top w = (1/\sqrt{2}, 1/\sqrt{2})^\top$. Nhân hai tọa độ đó lần lượt với 3 và 1 rồi đổi về, ta thu được $Aw = (2, 1)^\top$ (tự tính). Hai thành phần bị giãn không bằng nhau nên tổng của chúng đổi hướng. Vector w không phải vector riêng, nhưng vẫn biểu diễn được bằng hai vector riêng vì chúng tạo một cơ sở của cả mặt phẳng.

Biểu diễn cũng có thể viết $A = 3q_1q_1^\top + q_2q_2^\top$. Mỗi $q_iq_i^\top$ chiếu lên một đường riêng, rồi hệ số λ_i co giãn phần chiếu tương ứng. Công thức này sẽ không giữ nguyên dưới dạng trực giao như vậy với ma trận tổng quát không đối xứng.

6. Điều gì có thể xảy ra ngoài ví dụ đẹp

Ba khung sau cho thấy vì sao chỉ giải phương trình đặc trưng rồi tuyên bố ”có đủ hướng để chéo hóa” là chưa đủ. Chúng cùng là ma trận thực cỡ 2×2, nhưng số hướng riêng thực khác nhau.



Hình 15: Ba trường hợp trị riêng: mọi hướng, thiếu hướng và không có hướng riêng thực (minh hoạ, tự tính)

Cách đọc hình 3: khung trái là ma trận đơn vị, ở đó bất kỳ đường qua gốc đều là hướng riêng; vài đường đỏ chỉ là ví dụ. Khung giữa là phép trượt với duy nhất một đường riêng màu đỏ, dù trị riêng của nó lặp lại. Khung phải là phép quay 90 độ: mũi tên xanh ngọc đổi thành mũi tên đỏ vuông góc, và mọi vector thực khác 0 đều đổi hướng như vậy (tự tính).

6.1 Trị riêng lặp có thể vẫn có đủ vector riêng

Với I_2, phương trình $I_2 v = 1v$ đúng với mọi $v \neq 0$. Trị riêng 1 xuất hiện hai lần trong đa thức đặc trưng, và không gian riêng là cả $\mathbb{R}^2$ (tự tính). Ta có thể chọn vô số cơ sở trực chuẩn khác nhau; không có một hướng riêng duy nhất để buộc các phần mềm trả về cùng kết quả. Khi trị riêng lặp, so không gian con thường có ý nghĩa hơn so từng vector riêng.

Bội đại số là số lần λ xuất hiện như nghiệm của đa thức đặc trưng. **Bội hình học** là số chiều của không gian riêng $\text{Null}(A - \lambda I)$. Bội hình học luôn ít nhất 1 và không lớn hơn bội đại số. Để ma trận $n \times n$ chéo hóa được trên trường đang xét, tổng số vector riêng độc lập phải bằng n, tương đương tổng các bội hình học bằng n.

6.2 Trị riêng lặp cũng có thể thiếu vector riêng

Xét ma trận trượt $J = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$. Phương trình đặc trưng là $(1-\lambda)^2 = 0$, nên trị riêng duy nhất là 1 với bội đại số 2. Nhưng

$$J - I = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad (J - I)v = 0 \iff v_2 = 0.$$

Không gian riêng chỉ gồm các bội của $(1, 0)^\top$, có chiều 1 (tự tính). Không có hai vector riêng độc lập, nên không tồn tại cơ sở gồm vector riêng và J không chéo hóa được. Ma trận này minh họa rằng trị riêng lặp không tự động gây lỗi, nhưng cũng không tự động bảo đảm đủ vector riêng.

6.3 Có thể không có trị riêng thực

Với $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, phương trình đặc trưng là $\lambda^2 + 1 = 0$. Nó không có nghiệm thực, nhưng có hai nghiệm phức i và $-i$ với $i^2 = -1$ (tự tính). Hình học trong mặt phẳng thực giải thích điều này: quay 90 độ đưa mỗi vector thực khác 0 sang một đường vuông góc, không thể là bội thực của vector gốc.

Không có trị riêng thực không đồng nghĩa với ma trận bằng 0 hoặc không khả nghịch: $\det R = 1$ (tự tính).

Khi mở rộng sang số phức, khái niệm trực giao cần thay chuyển vị bằng chuyển vị liên hợp; phần còn lại của chương tập trung vào các ma trận thực để các hình và tích vô hướng có ý nghĩa quen thuộc.

7. Chéo hóa và định lý phổ

Nếu một ma trận vuông A có n vector riêng độc lập $v_1, \dots, v_n$, đặt V gồm các vector đó làm cột và $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. Khi ấy $AV = V\Lambda$, V khả nghịch, và

$$A = V\Lambda V^{-1}.$$

Phân tích này cho phép tính $A^k = V\Lambda^k V^{-1}$ với số nguyên không âm k: chỉ cần nâng từng trị riêng trên đường chéo lên lũy thừa k. Lưu ý V^{-1} nói chung **không** bằng $V^\top$, bởi các vector riêng của ma trận không đối xứng có thể không trực giao. Dù chéo hóa tồn tại, một cơ sở riêng gần phụ thuộc cũng có thể làm việc tính toán nhạy với sai số số học.

7.1 Định lý phổ cho ma trận thực đối xứng

Nếu $A = A^\top$ và A thực, định lý phổ bảo đảm: mọi trị riêng thực và tồn tại một cơ sở trực chuẩn gồm các vector riêng. Vì vậy có thể viết

$$A = Q\Lambda Q^\top, \qquad Q^\top Q = QQ^\top = I.$$

Điều này vẫn đúng khi có trị riêng lặp; ta chọn một cơ sở trực chuẩn bên trong từng không gian riêng. Lý do hai vector riêng ứng với hai trị riêng khác nhau phải vuông góc có thể thấy trực tiếp: từ $Au = \lambda u$, $Av = \mu v$ và $A = A^\top$, ta có

$$\lambda u^\top v = (Au)^\top v = u^\top A v = \mu u^\top v.$$

Nếu $\lambda \neq \mu$ thì $(\lambda - \mu)u^\top v = 0$, nên $u^\top v = 0$. Chứng minh này không nói rằng hai vector riêng ứng với **cùng** trị riêng tự động vuông góc; trong không gian riêng đó ta cần chọn hoặc trực chuẩn hóa cơ sở.

7.2 Dấu trị riêng và dạng toàn phương

Với ma trận đối xứng thực, viết $x = Qz$, ta được

$$x^\top A x = z^\top \Lambda z = \sum_i \lambda_i z_i^2.$$

Nếu mọi $\lambda_i > 0$, biểu thức dương với mọi $x \neq 0$ và A **xác định dương**. Nếu mọi $\lambda_i \geq 0$, biểu thức không âm và A **nửa xác định dương**; trị riêng 0 được phép. Nếu có cả trị riêng dương lẫn âm thì dạng toàn phương nhận cả giá trị dương lẫn âm, gọi là không xác định.

Ví dụ (tự tính): $\operatorname{diag}(3, 1)$ xác định dương; $\operatorname{diag}(3, 0)$ chỉ nửa xác định dương; $\operatorname{diag}(3, -1)$ không xác định vì cho giá trị dương trên e_1 và âm trên e_2. Ở đây tính đối xứng là một phần quan trọng của cách phân loại. Với Hessian đối xứng tại điểm dừng, trị riêng dương hoặc âm cho thấy độ cong theo từng hướng; khi có trị riêng 0, riêng phép kiểm bậc hai có thể chưa đủ để kết luận cực trị.

7.3 Tỉ số Rayleigh: độ lớn theo một hướng

Với ma trận đối xứng A và $x \neq 0$, định nghĩa tỉ số Rayleigh

$$R_A(x) = \frac{x^\top A x}{x^\top x}.$$

Viết $x = \sum_i c_i q_i$ trong cơ sở riêng trực chuẩn, ta được $R_A(x) = \frac{\sum_i \lambda_i c_i^2}{\sum_i c_i^2}$. Đây là trung bình có trọng số không âm của các trị riêng, nên nằm giữa trị riêng nhỏ nhất và lớn nhất. Giá trị lớn nhất đạt tại một vector riêng ứng với trị riêng lớn nhất, giá trị nhỏ nhất đạt tại một vector riêng ứng với trị riêng nhỏ nhất.

Với A ở mục 5 và $w = (1, 0)^\top$, ta có $R_A(w) = 2$, nằm giữa 1 và 3; với q_1, giá trị là 3 (tự tính). Giá trị lớn nhất ở đây là lớn nhất **theo thứ tự đại số**, không nhất thiết lớn nhất theo trị tuyệt đối. Phân biệt này quan trọng trong PCA và trong lặp lũy thừa.

8. Lũy thừa và lặp lũy thừa

Hình 2 dùng $D = \mathrm{diag}(2, 1)$ và $x_0 = (1, 1)^\top / \sqrt{2}$. Sau k lần nhân, trước chuẩn hoá,

$$D^k x_0 = \frac{1}{\sqrt{2}} \begin{pmatrix} 2^k \\ 1 \end{pmatrix}.$$

Chuẩn hoá không thay tỉ số hai tọa độ, nên tọa độ đứng chia tọa độ ngang bằng 2^{-k}. Với $k = 0, 1, 2, 3$, tỉ số lần lượt là $1, 1/2, 1/4, 1/8$ (tự tính). Nó tiến về 0, nghĩa là hướng vector tiến về hướng của e_1, vector riêng ứng với trị riêng 2.

Thực hiện chuẩn hoá sau mỗi lần nhân để hạn chế số quá lớn hoặc quá nhỏ:

$$y_{k+1} = A x_k, \qquad x_{k+1} = \frac{y_{k+1}}{\|y_{k+1}\|_2}.$$

Bước này chỉ định nghĩa nếu $y_{k+1} \neq 0$. Trong trường hợp ma trận đối xứng có một trị riêng trội duy nhất về trị tuyệt đối và vector bắt đầu có thành phần khác 0 theo hướng riêng tương ứng, hướng của dãy tiến về hướng đó. Tốc độ thường do tỉ số giữa trị tuyệt đối lớn thứ hai và lớn nhất chi phối. Nếu trị riêng trội âm, dấu của vector có thể lật qua từng bước, dù đường thẳng chứa nó hội tụ.

Điều kiện khởi tạo là có thật: với chính D ở trên, bắt đầu bằng e_2 thì mọi bước vẫn là e_2, không bao giờ phát hiện hướng e_1 (tự tính). Nếu hai trị riêng có cùng độ lớn trội, hướng cũng không nhất thiết hội tụ về một vector riêng duy nhất. Trong thực tế người ta theo dõi sai số $\|Ax - \lambda x\|_2$, không chỉ nhìn số vòng lặp. Tỉ số Rayleigh có thể dùng làm ước lượng trị riêng khi đã có hướng gần đúng.

9. Trị riêng, giá trị kỳ dị và ma trận Gram

Trị riêng giải câu hỏi $Av = \lambda v$, nên cần ma trận vuông và so cùng không gian đầu vào, đầu ra. Giá trị kỳ dị được định nghĩa qua SVD và tồn tại cho mọi ma trận thực A cỡ $m \times n$. Cụ thể, các bình phương giá trị kỳ dị là các trị riêng không âm của $A^\top A$; nếu v_i là hướng kỳ dị phải và $\sigma_i > 0$, thì $A v_i = \sigma_i u_i$. Hướng đầu ra u_i không nhất thiết cùng đường với v_i.

Vì $x^\top A^\top A x = \|Ax\|_2^2 \geq 0$, ma trận $A^\top A$ đối xứng nửa xác định dương và mọi trị riêng của nó không âm. Nếu A đối xứng thực, giá trị kỳ dị bằng trị tuyệt đối của các trị riêng, sau khi sắp thứ tự phù hợp. Đặc biệt, nếu A còn nửa xác định dương thì trị riêng chính là giá trị kỳ dị.

Ví dụ $H = \mathrm{diag}(2, -1)$ có trị riêng 2 và âm 1; $H^\top H = \mathrm{diag}(4, 1)$, nên giá trị kỳ dị là 2 và 1 (tự tính). Ví dụ khác, $N = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ có hai trị riêng bằng 0, nhưng $N^\top N = \mathrm{diag}(0, 1)$ và các giá trị kỳ dị là 1 và 0 (tự tính).

Như vậy ma trận có mọi trị riêng bằng 0 vẫn có thể biến đổi một số vector khác 0; không suy chuẩn phổ bằng 0 từ phổ trị riêng bằng 0 của ma trận không đối xứng.

Lập $A^\top A$ là cách giải thích quan hệ trên giấy. Khi tính SVD số, cách này có thể làm bình phương điều kiện số và mất độ chính xác ở hướng nhỏ; nên dùng phương pháp thư viện phù hợp. Chương nền 05 sẽ đi tiếp từ các hướng này tới SVD, nén và xấp xỉ hạng thấp.

10. Ví dụ dữ liệu: hướng phương sai lớn nhất

Giả sử bốn mẫu hai chiều đã được xếp theo hàng của X, và mỗi cột có trung bình 0:

$$X = \begin{pmatrix} 2 & 2 \\ -2 & -2 \\ 1 & -1 \\ -1 & 1 \end{pmatrix}, \qquad C = \frac{1}{4}X^\top X = \begin{pmatrix} 2{,}5 & 1{,}5 \\ 1{,}5 & 2{,}5 \end{pmatrix}.$$

Đây là ma trận hiệp phương sai dùng mẫu số $n = 4$ cho ví dụ này. Một số tài liệu dùng $n - 1$; cần nói rõ quy ước vì nó làm thay đổi trị riêng nhưng không thay hướng riêng. Tính từng ô: tổng bình phương của mỗi cột là 10, tổng tích hai cột là 6, và chia 4 cho các giá trị trong C (tự tính).

Vì cấu trúc giống mục 5, các hướng riêng đơn vị là $q_1 = (1, 1)^\top/\sqrt{2}$ và $q_2 = (1, -1)^\top/\sqrt{2}$, với trị riêng tương ứng 4 và 1 (tự tính). Theo tỉ số Rayleigh, phương sai theo một hướng đơn vị q là $q^\top Cq$ và lớn nhất bằng 4 tại q_1. PCA một chiều chọn đường của q_1, rồi chiếu dữ liệu lên đó.

Tổng phương sai là $\mathrm{tr}(C) = 5$, nên hướng đầu giữ $4/5 = 80\%$ tổng phương sai trong ví dụ này (tự tính). Con số chỉ áp dụng cho bốn điểm đã chọn, không phải đặc tính chung của PCA. Hướng phương sai lớn nhất không tự động là hướng hữu ích nhất cho nhãn hay cho mục tiêu dự đoán; PCA không dùng nhãn khi chọn hướng.

Quy ước chiều ma trận ảnh hưởng công thức. Ở đây hàng là mẫu và cột là đặc trưng, nên ma trận hiệp phương sai đặc trưng là $X^\top X/4$. Nếu dùng mỗi cột là một mẫu như trong một phần của ghi chú math 03, công thức đổi thứ tự. Luôn ghi rõ cách xếp dữ liệu trước khi dùng công thức.

11. Nối sang AI

(a) PCA và giảm chiều. Trị riêng của hiệp phương sai đo phương sai theo các hướng riêng tương ứng; sắp giảm dần rồi giữ một số hướng tạo biểu diễn số chiều thấp. Nếu nhiều trị riêng bằng hoặc gần bằng nhau, từng hướng có thể thay đổi mạnh giữa các mẫu dữ liệu, dù không gian con chung ổn định hơn. Xem math 03 cho cách tiếp cận qua hiệp phương sai và SVD.

(b) Phân cụm phổ. Thay vì lập hiệp phương sai, ta tạo ma trận mô tả kết nối giữa các điểm rồi phân tích ma trận Laplacian thích hợp. Một số vector riêng được dùng làm biểu diễn mới để nhận diện cấu trúc cụm. Trị riêng cần chọn và cách chuẩn hoá phụ thuộc định nghĩa Laplacian và thuật toán cụ thể, vì thế không thể áp quy tắc ”luôn lấy lớn nhất” của ví dụ PCA cho mọi bài toán. Xem math 05.

(c) Hessian và độ cong khi tối ưu. Tại một điểm, các trị riêng của Hessian đối xứng cho biết độ cong bậc hai theo những hướng riêng. Trị riêng dương lớn biểu thị độ cong lớn theo hướng đó, còn sự xuất hiện đồng thời của trị dương và âm tại điểm dừng cho thấy dạng yên ngựa. Độ cong thay đổi theo vị trí và mô hình phi tuyến, nên phân tích tại một điểm không mô tả toàn bộ địa hình.

(d) Lặp lũy thừa để ước lượng chuẩn phổ. Chuẩn phổ của một ma trận trọng số là giá trị kỳ dị lớn nhất, không phải nói chung là trị riêng lớn nhất của chính ma trận trọng số. Một cách liên quan tới mục 8 là lặp trên $W^\top W$ hoặc luân phiên nhân với W và $W^\top$ để ước lượng các hướng kỳ dị. Ghi chú math 17 giải thích ứng dụng cụ thể, còn điều kiện hội tụ vẫn cần được xét.

(e) Tính gần đúng khi dữ liệu lớn. Phân tích đầy đủ có thể quá tốn tài nguyên khi ma trận lớn. Một số phương pháp tìm gần đúng một không gian con quan trọng trước rồi xử lý bài toán nhỏ hơn; các bước chiếu và trực chuẩn ở chương 03 đóng vai trò ở đây. Ghi chú math 04 trình bày phân tích ngẫu nhiên và điều kiện đánh đổi độ chính xác, chi phí.

12. Sai lầm hay gặp

- **Cho vector 0 làm vector riêng.** Nó thỏa phương trình với mọi λ nhưng không mang hướng; vector riêng phải khác 0, dù vector 0 thuộc mọi không gian riêng.
- **Cho rằng mọi ma trận thực có đủ vector riêng thực.** Phép quay 90 độ không có trị riêng thực; ma trận trượt có trị riêng thực nhưng thiếu vector riêng độc lập.
- **Đồng nhất trị riêng lặp với thiếu chéo hóa.** Ma trận đơn vị có trị riêng lặp và vẫn có đủ cơ sở riêng; phải kiểm số chiều từng không gian riêng.
- **Thay V^{-1} bằng $V^\top$ mà không kiểm.** Phân tích $A = V\Lambda V^{-1}$ chỉ trở thành $Q\Lambda Q^\top$ khi chọn được cơ sở riêng trực chuẩn, điều được bảo đảm cho ma trận đối xứng thực.
- **Nhầm trị riêng với giá trị kỳ dị.** Giá trị kỳ dị không âm và áp dụng cho cả ma trận chữ nhật; ngay cả với ma trận vuông, hai dãy có thể khác nhau rõ rệt.
- **Nghĩ lặp lũy thừa luôn tìm được trị riêng dương lớn nhất.** Phương pháp nhắm hướng trội về trị tuyệt đối dưới điều kiện phù hợp, cần thành phần khởi đầu khác 0 và có thể lật dấu nếu trị trội âm.

13. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Cặp riêng	$Av = \lambda v,\ v \neq 0$	ảnh là bội của vector
Phương trình đặc trưng	$\det(A - \lambda I) = 0$	tìm trị riêng của ma trận vuông
Không gian riêng	$E_\lambda = \mathrm{Null}(A - \lambda I)$	mọi vector thỏa phương trình, kể cả 0
Chéo hóa	$A = V\Lambda V^{-1}$	cần đủ vector riêng độc lập
Định lý phổ	$A = Q\Lambda Q^\top$	với A thực đối xứng
Dạng toàn phương	$x^\top Ax = \sum_i \lambda_i z_i^2$	$z = Q^\top x$, dấu trị riêng quyết định dấu
Rayleigh	$R_A(x) = x^\top Ax / (x^\top x)$	nằm giữa hai trị riêng biên khi A đối xứng
Lặp lũy thừa	$x_{k+1} = Ax_k / \lVert Ax_k \rVert_2$	ước lượng hướng trội khi đủ điều kiện
SVD và Gram	$\sigma_i^2 = \lambda_i(A^\top A)$	giá trị kỳ dị từ trị riêng không âm

14. Bài tập tự tính

Năm bài sau đi từ nhận diện một cặp riêng đến chéo hóa, kiểm điều kiện và PCA. Hãy thử làm trước khi xem lời giải; các phép tính số đã được kiểm bằng numpy.

Bài 1 (dễ). Với $D = \mathrm{diag}(3, -2)$, tìm trị riêng và một vector riêng ứng với mỗi trị.

Đáp án. $De_1 = 3e_1$ và $De_2 = -2e_2$, nên hai trị riêng là 3 và âm 2, với vector riêng tương ứng $e_1 = (1, 0)^\top$, $e_2 = (0, 1)^\top$ (tự tính). Vector ở hướng e_2 bị lật chiều và dài gấp hai; mọi bội khác 0 của mỗi vector cũng là nghiệm.

Bài 2 (dễ vừa). Tìm các trị riêng và vector riêng đơn vị của $A = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$, rồi kiểm tổng và tích trị riêng.

Đáp án. $\det(A - \lambda I) = (4 - \lambda)^2 - 4 = 0$, suy ra $\lambda_1 = 6$, $\lambda_2 = 2$. Với trị 6, phương trình cho hai tọa độ bằng nhau nên chọn $q_1 = (1, 1)^\top / \sqrt{2}$. Với trị 2, hai tọa độ đối nhau nên chọn $q_2 = (1, -1)^\top / \sqrt{2}$. Ta có $6 + 2 = 8 = \mathrm{tr}(A)$ và $6 \cdot 2 = 12 = \det(A)$; trực tiếp $Aq_1 = 6q_1$, $Aq_2 = 2q_2$ (tự tính).

Bài 3 (vừa). Xét $J = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$. Nó có chéo hóa được bằng cơ sở vector riêng không?

Đáp án. Đa thức đặc trưng là $(2-\lambda)^2$, nên 2 là trị riêng duy nhất, bội đại số 2. Nhưng $(J-2I)v = 0$ cho $v_2 = 0$, vì vậy không gian riêng chỉ là các bội của $(1,0)^\top$ và có chiều 1 (tự tính). Ta thiếu một vector riêng độc lập, nên không thể chéo hóa J bằng cơ sở riêng dù trị riêng là thực.

Bài 4 (vừa khó). Cho $A = \begin{pmatrix} 2 & 2 \\ 1 & 1 \end{pmatrix}$. Tìm các trị riêng, một vector riêng mỗi trị và nhận xét về tính trực giao.

Đáp án. $\det(A-\lambda I) = (2-\lambda)(1-\lambda) - 2 = \lambda(\lambda-3)$, nên trị riêng là 0 và 3. Với 0, điều kiện là $v_1 + v_2 = 0$, chọn $v_0 = (1,-1)^\top$. Với 3, điều kiện là $v_1 = 2v_2$, chọn $v_3 = (2,1)^\top$. Kiểm $Av_0 = 0$, $Av_3 = 3v_3$, nhưng $v_0^\top v_3 = 1 \neq 0$ (tự tính). Ma trận này không đối xứng, nên không có lý do buộc hai hướng riêng khác trị phải vuông góc.

Bài 5 (khó). Bốn điểm đã trừ trung bình có ma trận hàng là $X = \begin{pmatrix} 2 & 0 \\ -2 & 0 \\ 0 & 1 \\ 0 & -1 \end{pmatrix}$. Dùng $C = X^\top X / 4$, tìm hướng PCA đầu và tỉ lệ tổng phương sai nó giữ lại.

Đáp án. $X^\top X = \operatorname{diag}(8, 2)$, do đó $C = \operatorname{diag}(2, 1/2)$. Hướng đầu là e_1 vì trị riêng 2 lớn hơn $1/2$; tổng phương sai bằng vết $2 + 1/2 = 5/2$. Tỉ lệ giữ lại là $2/(5/2) = 4/5 = 80\%$ (tự tính). Dấu âm của e_1 cũng biểu diễn cùng đường PCA; dùng mẫu số khác có thể đổi các trị riêng nhưng không đổi tỉ lệ trong ví dụ này.

Tài liệu tham khảo nền: Strang, Introduction to Linear Algebra; Trefethen và Bau, Numerical Linear Algebra; Shlens 2014 (math 03); von Luxburg 2007 (math 05).

Nền tảng 05

SVD (Singular Value Decomposition): nhìn mọi ma trận như xoay, co giãn, xoay

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector, nhân ma trận với vector, ma trận chuyển vị, trị riêng ở mức nhớ lại (có nhắc nhanh trong chương)

Dùng ở: bài math 03 (PCA), math 04 (randomized decomposition), math 17 (spectral normalization), LoRA và pruning ở các cuốn khác

> Chương này viết theo cách học từ đầu, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi 'tự tính' đã được kiểm bằng numpy.

1. Vì sao cần SVD

Trước hết, một câu để bạn có hình dung: **mọi ma trận đều là xoay rồi co giãn rồi xoay.** Đó là ý tưởng của SVD (Singular Value Decomposition, phân tích giá trị kỳ dị). Phần còn lại của chương giải thích câu này nghĩa là gì, vì sao nó đúng cả với ma trận chữ nhật, và vì sao ta cần nó.

Ma trận xuất hiện khắp nơi trong AI: trọng số một lớp mạng, bảng dữ liệu, ma trận hiệp phương sai. Chúng thường lớn và rối. Câu hỏi thực dụng là trong đống đó, đâu là phần quan trọng nhất. SVD sắp xếp mọi thứ thành một dãy "lớp" có thứ tự, lớp mạnh nhất đứng đầu. Nhờ vậy ta giữ được vài lớp đầu để nén, đếm được ma trận thực sự dùng bao nhiêu chiều, và biết chỗ nào trong phép đảo ma trận sẽ nguy hiểm (mục 8). SVD cũng là nền cho PCA (math 03), cho xấp xỉ hạng thấp (mục 9 và 11), và cho các thuật toán SVD ngẫu nhiên khi ma trận quá lớn (math 04).

2. Nhắc nhanh nền tảng

2.1 Ma trận là phép biến đổi

Ma trận M cỡ $m \times n$ nhận vector x có n thành phần và trả ra $y = Mx$ có m thành phần. Về hình học, M đưa điểm x đến điểm y. Công thức là $y_i = \sum_{j=1}^{n} M_{ij}\, x_j$.

Ví dụ (tự tính): với $M = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ và $x = (3, 4)^\top$, ta có $Mx = (2 \cdot 3 + 0 \cdot 4,\ 0 \cdot 3 + 1 \cdot 4)^\top = (6, 4)^\top$.

Cách đọc hữu ích hơn là đọc **theo cột**. Vì $x = \sum_j x_j e_j$ và phép biến đổi là tuyến tính, nên $Mx = \sum_j x_j M e_j$: cột thứ j của M là ảnh của e_j. Với ma trận chính của chương, $A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}$, A đưa e_1 đến $(3, 4)^\top$ và e_2 đến $(0, 5)^\top$ (tự tính).

Nhân hai ma trận nghĩa là ghép hai phép biến đổi: $M_2 M_1$ là áp M_1 trước rồi áp M_2. Đó là lý do ma trận hợp với mạng nơ-ron tuyến tính, khi lớp sau nhận đầu ra của lớp trước.

2.2 Chuyển vị, trực giao, đường chéo

Chuyển vị $M^\top$ đổi hàng thành cột. Ví dụ $A^\top = \begin{pmatrix} 3 & 4 \\ 0 & 5 \end{pmatrix}$. Một tính chất cần nhớ: $(Mx)^\top y = x^\top (M^\top y)$.

Ma trận trực giao (orthogonal) Q thoả $Q^\top Q = I$: các cột vuông góc từng đôi và có độ dài 1. Nó giữ độ dài, vì $\|Qx\|_2^2 = x^\top Q^\top Q x = x^\top x$. Về hình học, Q là một **xoay** (quay quanh gốc) hoặc một **phản xạ** (lật qua một đường thẳng). Ví dụ (tự tính): $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ là xoay 90 độ, đưa $(3,4)^\top$ thành $(-4,3)^\top$, vẫn có độ dài 5. Còn $F = \mathrm{diag}(1,-1)$ là lật qua trục hoành.

Trong chương này, ”trực giao” nghĩa là ”xoay hoặc phản xạ, giữ độ dài”. SVD sẽ cho ra những ma trận trực giao có định thức -1, tức phản xạ, và điều đó hợp lệ (mục 4).

Ma trận đường chéo $D = \mathrm{diag}(d_1, \ldots, d_n)$ co giãn từng trục tọa độ: $Dx = (d_1 x_1, \ldots, d_n x_n)^\top$. Ví dụ (tự tính): $\mathrm{diag}(2,1)$ đưa $(1,1)$ thành $(2,1)$, tức trục ngang kéo dài gấp đôi, trục dọc giữ nguyên.

Ba loại này (xoay/phản xạ, co giãn theo trục, và ghép chúng) là toàn bộ nguyên liệu của SVD.

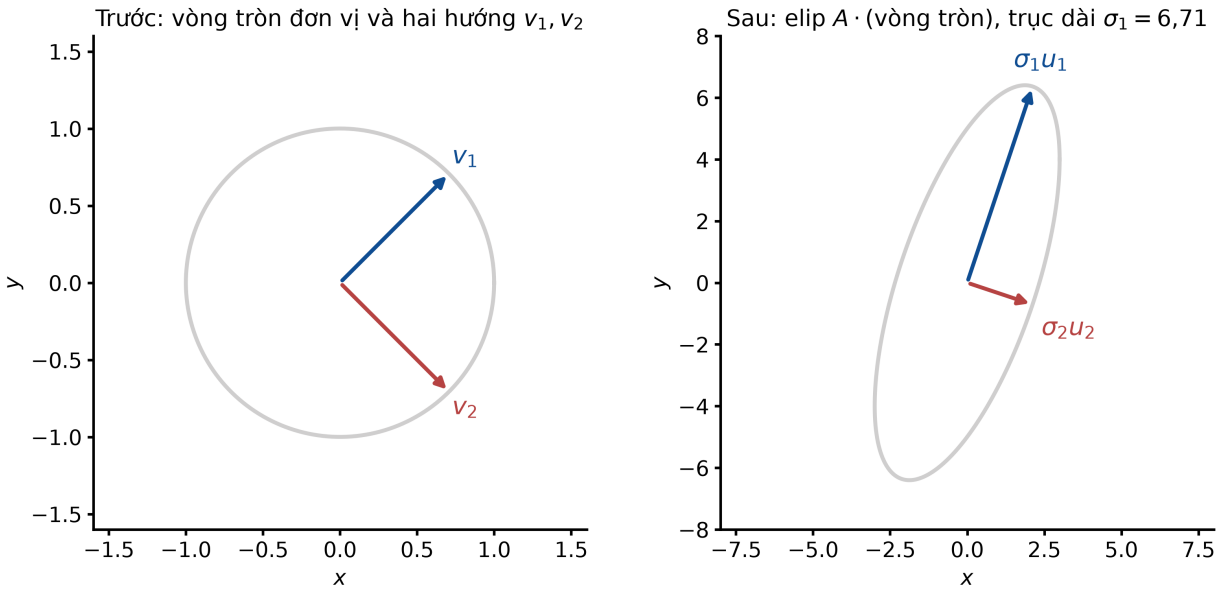
3. Trực giác hình học: vòng tròn thành elip

Lấy mọi vector có độ dài 1 trong mặt phẳng, ta được vòng tròn đơn vị. Áp A lên từng điểm, ta được một elip. Đây là hình ảnh trung tâm của chương.

Hai hướng vuông góc ở đầu vào không luôn biến thành hai hướng vuông góc ở đầu ra. Với A: $e_1 \mapsto (3,4)^\top$, $e_2 \mapsto (0,5)^\top$, và $(3,4)\cdot(0,5) = 20 \neq 0$ (tự tính). SVD cho biết luôn tồn tại một cặp hướng đầu vào vuông góc v_1, v_2 mà ảnh Av_1, Av_2 cũng vuông góc. Các ảnh đó chính là trục của elip:

- v_i là hướng đầu vào (một điểm trên vòng tròn ban đầu);
- $\sigma_i u_i$ là trục thứ i của elip, với u_i là hướng đầu ra và σ_i là độ dài trục.

Với A của chương, $v_1 = \frac{1}{\sqrt{2}}(1,1)^\top$ và $v_2 = \frac{1}{\sqrt{2}}(1,-1)^\top$ được đưa thành $\sigma_1 u_1 \approx (2{,}12;\ 6{,}36)^\top$ và $\sigma_2 u_2 \approx (2{,}12;\ -0{,}71)^\top$ (tự tính). Các số này được tính đầy đủ ở mục 5.



Hình 16: Vòng tròn đơn vị bị ma trận A biến thành elip: v1, v2 thành σ1u1, σ2u2 (minh hoạ, tự tính)

Cách đọc hình 1: bên trái là vòng tròn bán kính 1 với hai mũi tên v_1 (xanh) và v_2 (đỏ), vuông góc nhau. Bên phải là ảnh của cả vòng tròn qua A. Mũi tên xanh $\sigma_1 u_1$ dài hơn hẳn, khớp với $\sigma_1 \approx 6{,}71$ trên tiêu đề. Quan trọng hơn, mũi tên đỏ vuông góc với mũi tên xanh: hai trục elip vuông góc nhau, và đó là điều SVD bảo đảm. Hai hình dùng hai hệ tọa độ khác nhau (đầu vào và đầu ra), nên đừng so trực tiếp góc giữa v_1 và u_1.

4. Phát biểu SVD: ba bước đọc từ phải sang trái

Mọi ma trận thực A cỡ $m \times n$ đều phân tích được thành

$$A = U\Sigma V^\top,$$

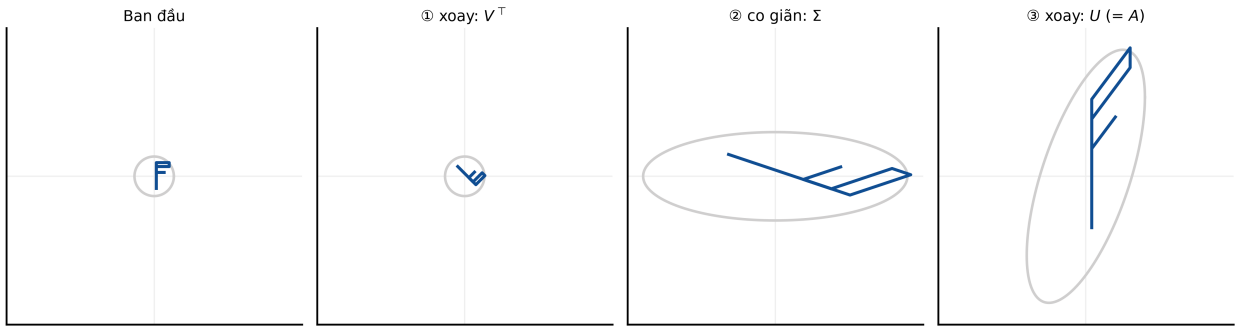
trong đó:

- U cỡ $m \times m$, trực giao; các cột u_i là hướng ở **đầu ra**, sống trong $\mathbb{R}^m$.
- V cỡ $n \times n$, trực giao; các cột v_i là hướng ở **đầu vào**, sống trong $\mathbb{R}^n$.
- Σ cỡ $m \times n$, chỉ có số trên đường chéo, và các số đó là $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$, gọi là **giá trị kỳ dị** (singular value).

Đọc từ phải sang trái, $Ax = U\Sigma V^\top x$ là **ba bước**:

1. $V^\top$ xoay (hoặc phản xạ) đầu vào, đưa mỗi v_i về trục e_i, vì $V^\top v_i = e_i$.
2. Σ co giãn: trục thứ i được nhân với σ_i.
3. U xoay (hoặc phản xạ) kết quả, đưa mỗi e_i đến u_i.

Hình 2 chạy đúng ba bước này trên một chữ F.



Hình 17: Bốn bước của SVD trên hình chữ F: gốc, xoay $V^\top$, co giãn Σ, xoay U (minh hoạ, tự tính)

Cách đọc hình 2: bốn panel từ trái sang phải là chữ F gốc, sau $V^\top$, sau Σ, và sau U. Ở panel ②, vòng tròn đã thành elip nằm ngang vì $\sigma_1 > \sigma_2$. Ở panel ③, kết quả trùng với A áp lên chữ F gốc. Để ý panel ①: chữ F bị lật. Đó là vì $V^\top$ ở đây là một **phản xạ** (det $V = -1$, tự tính), và điều đó hợp lệ. Nếu muốn cả U và V đều là xoay thuần, ta đổi dấu đồng thời v_2 và u_2. Ma trận A không đổi, vì $\sigma_2 u_2 v_2^\top$ không đổi khi đổi dấu cả hai (mục 6.3). Vậy câu ”xoay, co giãn, xoay” nên hiểu là ”xoay hoặc phản xạ”.

Cách đọc theo cột, quan trọng cho phần nén ở mục 9:

$$A = \sum_{i=1}^{r} \sigma_i \, u_i v_i^\top,$$

với r là số giá trị kỳ dị khác 0. Mỗi $u_i v_i^\top$ là một ma trận **hạng 1** (tích ngoài, outer product), nên A là tổng có trọng số của các lớp hạng 1, và lớp thứ i có trọng số σ_i. Lớp mạnh nhất đứng đầu.

5. Tính tay từng bước với $A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}$

Ta làm sáu bước, mỗi bước một phép tính, không nhảy cóc. Định thức $\det A = 15 \neq 0$, nên A khả nghịch và có hạng 2. Mọi phép tính trong mục này do ta tự làm và đã kiểm bằng numpy.

5.1 Bước 1: tính $A^\top A$

$$A^\top A = \begin{pmatrix} 3 & 4 \\ 0 & 5 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}.$$

Từng ô: ô $(1,1)$ là $3 \cdot 3 + 4 \cdot 4 = 25$; ô $(1,2)$ là $3 \cdot 0 + 4 \cdot 5 = 20$; ô $(2,1)$ là $0 \cdot 3 + 5 \cdot 4 = 20$; ô $(2,2)$ là $0 \cdot 0 + 5 \cdot 5 = 25$. Vậy (tự tính)

$$A^\top A = \begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}.$$

Ô $(1,1)$ là bình phương độ dài cột 1 của A, còn ô $(1,2)$ là tích vô hướng giữa cột 1 và cột 2. Nói gọn, $A^\top A$ chứa mọi tích vô hướng giữa các cột của A.

5.2 Bước 2: tìm trị riêng (eigenvalue)

$$\det(A^\top A - \lambda I) = (25 - \lambda)^2 - 20^2 = 0.$$

Suy ra $25 - \lambda = \pm 20$, tức $\lambda = 5$ hoặc $\lambda = 45$ (tự tính). Kiểm: tổng hai trị riêng bằng vết (trace) của ma trận, $45 + 5 = 50 = 25 + 25$.

5.3 Bước 3: tìm v_1 và v_2

Với $\lambda = 45$: $A^\top A - 45I = \begin{pmatrix} -20 & 20 \\ 20 & -20 \end{pmatrix}$. Hàng 1 cho $-20x + 20y = 0$, tức $x = y$. Chuẩn hoá về độ dài 1 ta được

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Với $\lambda = 5$: $A^\top A - 5I = \begin{pmatrix} 20 & 20 \\ 20 & 20 \end{pmatrix}$. Hàng 1 cho $20x + 20y = 0$, tức $y = -x$, nên

$$v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Hai vector này vuông góc và có độ dài 1 (tự tính). Quy ước: v_1 ứng với trị riêng lớn nhất.

5.4 Bước 4: giá trị kỳ dị

Giá trị kỳ dị là căn bậc hai của trị riêng:

$$\sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6{,}708, \qquad \sigma_2 = \sqrt{5} \approx 2{,}236$$

(tự tính). Kiểm: $\sigma_1 \sigma_2 = 3\sqrt{5} \cdot \sqrt{5} = 15 = \det A$. Với ma trận vuông, tích các giá trị kỳ dị luôn bằng $|\det A|$, vì ma trận trực giao có định thức ± 1.

5.5 Bước 5: tìm u_1 và u_2

Từ $A = U\Sigma V^\top$, nhân bên phải với v_i: $Av_i = U\Sigma e_i = \sigma_i u_i$. Vậy $u_i = Av_i / \sigma_i$.

Ta có $A(1,1)^\top = (3,9)^\top$, nên $Av_1 = \frac{1}{\sqrt{2}}(3,9)^\top$ và

$$u_1 = \frac{1}{\sqrt{2}}(3,9)^\top \cdot \frac{1}{3\sqrt{5}} = \frac{1}{\sqrt{10}}\begin{pmatrix}1\\3\end{pmatrix}.$$

Kiểm: $(1^2+3^2)/10 = 1$.

Ta có $A(1,-1)^\top = (3,-1)^\top$, nên $Av_2 = \frac{1}{\sqrt{2}}(3,-1)^\top$ và

$$u_2 = \frac{1}{\sqrt{2}}(3,-1)^\top \cdot \frac{1}{\sqrt{5}} = \frac{1}{\sqrt{10}}\begin{pmatrix}3\\-1\end{pmatrix}.$$

Kiểm: $(9+1)/10 = 1$, và $u_1 \cdot u_2 = (3-3)/10 = 0$ (tự tính).

5.6 Bước 6: ghép lại và kiểm

$$U = \frac{1}{\sqrt{10}}\begin{pmatrix}1 & 3\\3 & -1\end{pmatrix}, \qquad \Sigma = \begin{pmatrix}3\sqrt{5} & 0\\0 & \sqrt{5}\end{pmatrix}, \qquad V = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\1 & -1\end{pmatrix}.$$

Cột j của $U\Sigma$ là $\sigma_j u_j$. Ta có $\sigma_1 u_1 = \frac{3}{\sqrt{2}}(1,3)^\top$, vì $\sqrt{5}/\sqrt{10} = 1/\sqrt{2}$, và $\sigma_2 u_2 = \frac{1}{\sqrt{2}}(3,-1)^\top$. Vậy

$$U\Sigma = \frac{1}{\sqrt{2}}\begin{pmatrix}3 & 3\\9 & -1\end{pmatrix}.$$

Vì V đối xứng nên $V^\top = V$, và

$$U\Sigma V^\top = \frac{1}{2}\begin{pmatrix}3 & 3\\9 & -1\end{pmatrix}\begin{pmatrix}1 & 1\\1 & -1\end{pmatrix}.$$

Từng ô của tích: ô $(1,1)$ là $3\cdot 1 + 3\cdot 1 = 6$; ô $(1,2)$ là $3\cdot 1 + 3\cdot(-1) = 0$; ô $(2,1)$ là $9\cdot 1 + (-1)\cdot 1 = 8$; ô $(2,2)$ là $9\cdot 1 + (-1)\cdot(-1) = 10$. Vậy $U\Sigma V^\top = \frac{1}{2}\begin{pmatrix}6 & 0\\8 & 10\end{pmatrix} = \begin{pmatrix}3 & 0\\4 & 5\end{pmatrix} = A$ (tự tính).

Đọc theo lớp cũng cho đúng kết quả đó: $\sigma_1 u_1 v_1^\top = \begin{pmatrix}1{,}5 & 1{,}5\\4{,}5 & 4{,}5\end{pmatrix}$ và $\sigma_2 u_2 v_2^\top = \begin{pmatrix}1{,}5 & -1{,}5\\-0{,}5 & 0{,}5\end{pmatrix}$, cộng lại bằng A (tự tính). Lớp 1 mang phần lớn độ lớn của A, và mục 9 sẽ đo điều này.

Lưu ý: numpy có thể trả về các cặp (u_i, v_i) với dấu ngược lại, xem mục 6.3.

5.7 Vì sao σ^2 là trị riêng của $A^\top A$

Thay $A = U\Sigma V^\top$ vào $A^\top A$ và dùng $U^\top U = I$:

$$A^\top A = (U\Sigma V^\top)^\top U\Sigma V^\top = V\Sigma^\top U^\top U\Sigma V^\top = V(\Sigma^\top\Sigma)V^\top.$$

Ma trận $\Sigma^\top\Sigma$ là đường chéo với các số σ_i^2. Nhân bên phải với V, rồi dùng $V^\top V = I$, ta được $A^\top A\, v_i = \sigma_i^2 v_i$ với mọi i. Vậy σ_i^2 là trị riêng của $A^\top A$, ứng với vector riêng v_i. Đây cũng là cách tìm SVD trên giấy: phân tích trị riêng của $A^\top A$, lấy căn, rồi tính $u_i = Av_i/\sigma_i$.

6. Các biến thể và những chi tiết hay bị bỏ qua

6.1 Full, thin, compact

- **Full SVD**: U cỡ $m \times m$, Σ cỡ $m \times n$, V cỡ $n \times n$. Đây là dạng numpy trả về mặc định.
- **Thin SVD** (economy): với $k = \min(m, n)$, U cỡ $m \times k$, Σ cỡ $k \times k$, V cỡ $n \times k$. Phần bị bỏ chỉ nhân với các hàng 0 của Σ nên không đổi tích.
- **Compact SVD**: chỉ giữ r cột ứng với $\sigma_i > 0$. Cũng đúng hoàn toàn.

Ví dụ (tự tính) với ma trận 5×3: full cho U cỡ 5×5, Σ cỡ 5×3, $V^\top$ cỡ 3×3; thin cho U cỡ 5×3, Σ cỡ 3×3. Khi viết code, hãy kiểm $U\Sigma V^\top$ có đúng cỡ $m \times n$ trước khi tin kết quả.

6.2 Số giá trị kỳ dị và hạng

Số giá trị kỳ dị bằng $\min(m, n)$: ma trận cao có n giá trị, ma trận rộng có m giá trị. Hạng r bằng số $\sigma_i > 0$. Ví dụ (tự tính): $P = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$ cỡ 2×3 chỉ có hai giá trị kỳ dị, là 2 và 1. Ma trận A của chương có hạng 2, còn ma trận hạng 1 có đúng một giá trị kỳ dị khác 0 (bài 4, mục 14).

6.3 Dấu mơ hồ

Đổi dấu đồng thời u_i và v_i không đổi $\sigma_i u_i v_i^\top$, nên A không đổi. Đổi dấu chỉ một trong hai thì A sai. Trong lần chạy numpy của chương, cả hai cặp đều được trả về ngược dấu so với mục 5 (tự tính). Đó không phải lỗi. Khi so với sách hay code khác, đừng so từng dấu; hãy kiểm $U\Sigma V^\top = A$.

6.4 Giá trị kỳ dị trùng nhau

Nếu $\sigma_i = \sigma_j$, các u_i, v_i không duy nhất: ta được quay tự do trong không gian con tương ứng. Ví dụ: I_2 có $\sigma_1 = \sigma_2 = 1$, và với mọi ma trận trực giao Q, $U = V = Q$ đều là SVD hợp lệ, vì $Q\,I\,Q^\top = I$ (tự tính). Khi các σ trùng nhau, đừng so từng cột u_i giữa hai lần tính. Hãy so không gian con, hoặc so chính A.

7. Liên hệ với trị riêng

Từ mục 5.7 ta có $\sigma_i = \sqrt{\lambda_i(A^\top A)}$. Vì $A^\top A$ đối xứng và không âm, căn bậc hai luôn có nghĩa.

Nếu A đối xứng nửa xác định dương, phân tích trị riêng $A = Q\Lambda Q^\top$ cho ta $A^\top A = Q\Lambda^2 Q^\top$, nên $\sigma_i = \lambda_i$ và có thể chọn $U = V = Q$. Ví dụ $S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ (ma trận ở mục 2.4 của lời nói đầu) có trị riêng và giá trị kỳ dị đều là 6 và 2 (tự tính). Ta đặt tên S để khỏi lẫn với Σ. Nếu A đối xứng nhưng có trị riêng âm thì $\sigma = |\lambda|$: ví dụ $B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ có trị riêng 1 và -1, nhưng hai giá trị kỳ dị đều là 1 (tự tính).

SVD khác trị riêng ở chỗ: nó tồn tại cho mọi ma trận chữ nhật, và U, V luôn trực giao với các σ_i thực, không âm. Trị riêng chỉ có cho ma trận vuông, có thể là số phức (ma trận xoay R ở mục 2.2 không có trị riêng thực), và không phải lúc nào cũng đủ vector riêng.

8. Các đại lượng suy ra từ giá trị kỳ dị

8.1 Chuẩn Frobenius

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\sum_i \sigma_i^2}.$$

Với A của chương: $\|A\|_F^2 = 9 + 0 + 16 + 25 = 50 = 45 + 5 = \sigma_1^2 + \sigma_2^2$, nên $\|A\|_F = \sqrt{50} = 5\sqrt{2} \approx 7{,}071$ (tự tính).

8.2 Chuẩn phổ (spectral norm)

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sigma_1.$$

Lý do: với $x = \sum_i c_i v_i$, ta có $\|x\|_2^2 = \sum_i c_i^2$ và $\|Ax\|_2^2 = \sum_i \sigma_i^2 c_i^2 \leq \sigma_1^2 \sum_i c_i^2$, dấu bằng khi $x = v_1$. Với A của chương, $\|A\|_2 = 3\sqrt{5} \approx 6{,}708$ (tự tính).

8.3 Điều kiện số (condition number)

Với ma trận vuông khả nghịch, $\kappa(A) = \sigma_1 / \sigma_{\min}$. Với A của chương, $\kappa(A) = \frac{3\sqrt{5}}{\sqrt{5}} = 3$ (tự tính). Khi giải $Ax = b$, sai số tương đối của x có thể bị khuếch đại tối đa cỡ κ lần so với sai số tương đối của b. κ gần 1 là tốt, còn κ rất lớn là dấu hiệu nguy hiểm, nhất là khi đảo ma trận. Mục 12 cho thấy vì sao lập $A^\top A$ làm con số này xấu đi.

8.4 Hạng số học

Trên giấy, hạng là số $\sigma_i > 0$. Trên máy, một giá trị kỳ dị lẽ ra bằng 0 có thể ra một số rất nhỏ khác 0, nên ta dùng hạng số học: số σ_i vượt một ngưỡng τ do người làm chọn. Ví dụ: ma trận $C = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 1 \end{pmatrix}$ ở mục 2.3 của lời nói đầu có hạng toán học 2, vì hàng 2 bằng 2 lần hàng 1. Với $\sigma_{\max} \approx 8{,}435$, numpy đếm hạng bằng ngưỡng $\sigma_{\max} \cdot \max(m, n) \cdot \varepsilon \approx 5{,}6 \times 10^{-15}$ (tự tính, ε là epsilon máy). Giá trị kỳ dị thứ ba của C nhỏ hơn ngưỡng này, nên hạng được đếm là 2 (tự tính).

9. Xấp xỉ hạng thấp và định lý Eckart-Young

9.1 Ý tưởng

Giữ k lớp mạnh nhất:

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top.$$

Hạng của A_k là k khi $\sigma_k > 0$. Định lý Eckart-Young (cùng với Mirsky) phát biểu: trong mọi ma trận B có hạng không quá k,

$$\|A - A_k\|_F \leq \|A - B\|_F, \qquad \|A - A_k\|_F = \sqrt{\sum_{i>k} \sigma_i^2}.$$

Nói bằng lời: cắt SVD cho sai số nhỏ nhất theo chuẩn Frobenius. Tương tự, $\|A - A_k\|_2 = \sigma_{k+1}$ là nhỏ nhất theo chuẩn phổ. Trực giác: bình phương sai số bằng tổng bình phương các σ bị bỏ, nên ta bỏ phần nhỏ nhất. Nếu $\sigma_k > \sigma_{k+1}$ thì A_k là duy nhất.

9.2 Ví dụ tay: A_1 của chương (tự tính)

Giữ lớp 1 của mục 5: $A_1 = \begin{pmatrix} 1{,}5 & 1{,}5 \\ 4{,}5 & 4{,}5 \end{pmatrix}$, và $A - A_1 = \begin{pmatrix} 1{,}5 & -1{,}5 \\ -0{,}5 & 0{,}5 \end{pmatrix}$. Sai số Frobenius là $\sqrt{2{,}25 + 2{,}25 + 0{,}25 + 0{,}25} = \sqrt{5} = \sigma_2$, khớp định lý. Sai số tương đối là $\frac{\sqrt{5}}{\sqrt{50}} = \frac{1}{\sqrt{10}} \approx 0{,}316$, tức khoảng 31,6%. Năng lượng giữ lại là $\frac{45}{50} = 90\%$.

9.3 Sai số tương đối và năng lượng

Với ma trận bất kỳ, đặt

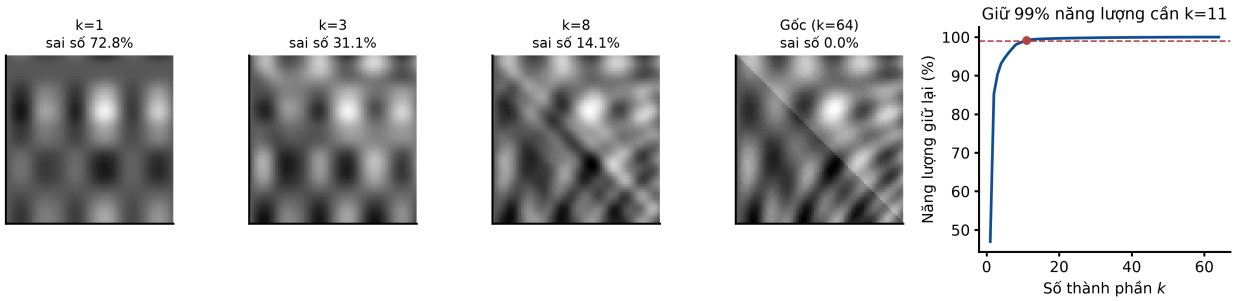
$$e_k = \frac{\|A - A_k\|_F}{\|A\|_F} = \sqrt{\frac{\sum_{i>k} \sigma_i^2}{\sum_i \sigma_i^2}}, \qquad E_k = \frac{\sum_{i \le k} \sigma_i^2}{\sum_i \sigma_i^2}.$$

e_k là sai số tương đối, đo theo độ lớn. E_k là năng lượng giữ lại, đo theo bình phương. Chúng bù nhau: $e_k^2 + E_k = 1$. Với ví dụ trên, $e_1^2 = 5/50 = 0{,}1$ và $E_1 = 0{,}9$ (tự tính). Đây là chỗ hay nhầm: ”giữ 90% năng lượng” không có nghĩa là ”sai số 10%”. Sai số tương đối ở đây là 31,6%, vì $\sqrt{0{,}1} \approx 0{,}316$.

9.4 Hình 3: nén một ảnh

Ảnh thử là một ảnh tổng hợp 64×64 có cấu trúc, sinh bằng hạt giống cố định để kết quả lặp lại được. Lưu A_k dưới dạng U_k, Σ_k, V_k cần $k(64 + 64 + 1) = 129k$ số (tự tính). Vậy $k = 1$ cần 129 số, khoảng 3,1% so với 4096 điểm ảnh gốc; $k = 8$ cần 1032 số, khoảng 25%; và $k = 11$ cần 1419 số, khoảng 35% (tự tính).

Sai số tương đối đo trên ảnh là 72,8% với $k = 1$, 31,1% với $k = 3$, và 14,1% với $k = 8$. Theo $e_k^2 + E_k = 1$, năng lượng giữ lại tương ứng là khoảng 47%, 90% và 98% (tự tính). Cần $k = 11$ để giữ 99% tổng năng lượng $\sum_i \sigma_i^2$.



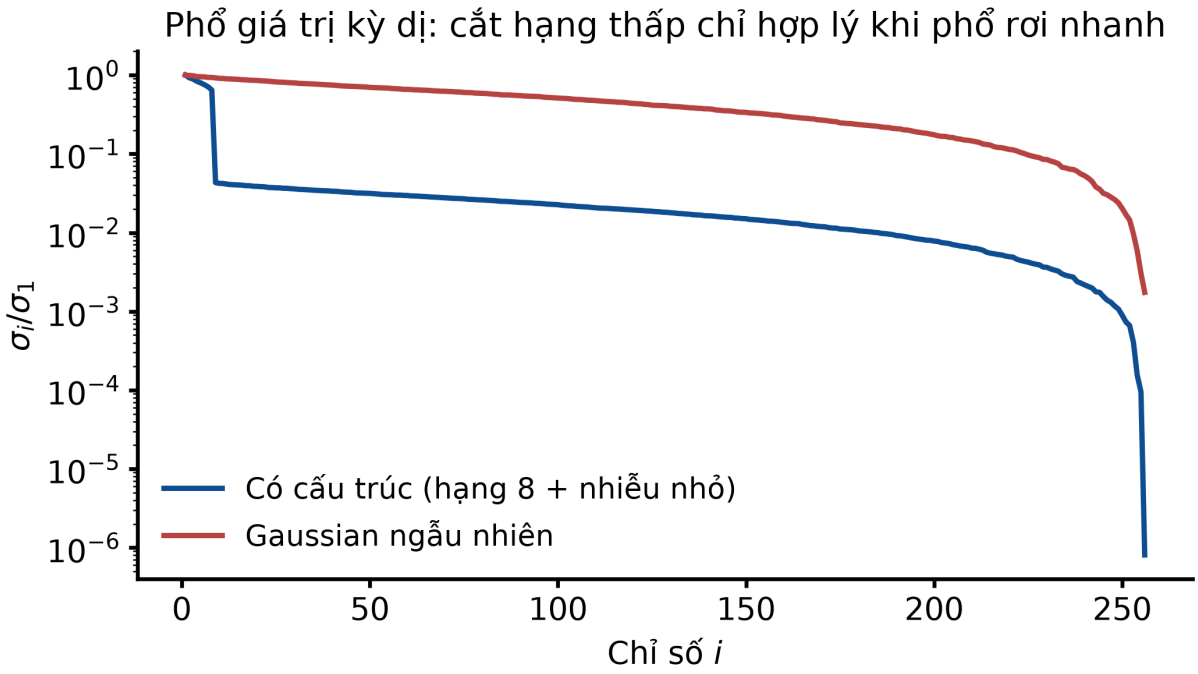
Hình 18: Xấp xỉ hạng thấp của ảnh 64x64 với k=1, 3, 8 và ảnh gốc, cùng đường năng lượng tích luỹ (minh hoạ, tự tính)

Cách đọc hình 3: bốn ảnh nhỏ từ trái sang phải là $k = 1$, $k = 3$, $k = 8$ và ảnh gốc. Với $k = 1$, ảnh chỉ còn vài khối sáng tối lớn, tức ”bộ khung” thô nhất. Với $k = 8$, ảnh gần như nhận ra được so với gốc. Đừng đọc 72,8% như ”mất 72,8% độ đẹp”: đó là độ lớn theo chuẩn Frobenius, không phải cảm nhận thị giác. Panel bên phải là đường năng lượng: nó vọt lên nhanh rồi gần như phẳng, và chấm đỏ đánh dấu $k = 11$ tại mốc 99%. Đây là cách trả lời câu ”nên giữ bao nhiêu lớp”: đặt một ngưỡng năng lượng, rồi đọc k trên trục ngang.

10. Phổ giá trị kỳ dị: khi nào cắt hạng thấp hợp lý

Dãy $\sigma_1 \ge \sigma_2 \ge \dots$ gọi là **phổ** (spectrum) của ma trận. Trước khi nén bất kỳ ma trận trọng số nào, hãy hỏi: phổ có **vực** không? Vực là một bước nhảy lớn sau một chỉ số k. Nếu có vực, ma trận gần hạng thấp và cắt ở vực là hợp lý. Nếu phổ rơi đều và chậm, không có điểm cắt tự nhiên, và cắt sẽ bỏ đi thông tin thật.

Cách đọc hình 4: trục ngang là chỉ số i, trục dọc là σ_i/σ_1 theo thang log (mỗi vạch là một bậc 10). Đường xanh là ma trận 256×256 gồm một phần hạng 8 cộng nhiễu nhỏ. Đường này vẫn ở mức cao đến chỉ số 8, rồi rơi thẳng đứng xuống khoảng 0,04 ở chỉ số 9, tức $\sigma_9/\sigma_1 \approx 0{,}04$ (tự tính từ dữ liệu đã kiểm). Đó là một vực, nên ta cắt ở $k = 8$. Đường đỏ là ma trận Gaussian ngẫu nhiên cùng cỡ: không có bước nhảy nào, σ giảm đều suốt hình, vì ma trận ngẫu nhiên không có cấu trúc hạng thấp. Đuôi đường xanh rơi tới cỡ 10^{-6}. Đó là nhiễu nhỏ, không phải số 0 tuyệt đối.



Hình 19: Phổ giá trị kỳ dị: ma trận có cấu trúc rơi vực sau hạng 8, ma trận Gaussian rơi chậm (minh hoạ, tự tính)

Khi gặp một ma trận mới, hãy hỏi theo thứ tự: phổ có vực rõ không, vực ở chỉ số nào, và nếu cắt ở đó thì sai số e_k bằng bao nhiêu (mục 9.3)?

11. Nối sang AI

(a) PCA là SVD của dữ liệu đã trừ trung bình. Với ma trận dữ liệu X (mỗi hàng một mẫu, đã trừ trung bình theo cột), ta có $X = U\Sigma V^\top$. Các cột của V là hướng chính, và phương sai theo hướng thứ i tỉ lệ với σ_i^2. Chi tiết về hằng số chuẩn hoá nằm trong math 03. PCA giữ k hướng mạnh nhất chính là xấp xỉ hạng thấp ở mục 9, áp lên dữ liệu.

(b) Nén và cắt tỉa trọng số bằng hạng thấp. Thay W cỡ $m \times n$ bằng tích PQ, với P cỡ $m \times k$ và Q cỡ $k \times n$. Số tham số giảm từ mn xuống $k(m+n)$. Cắt SVD cho P và Q tối ưu theo chuẩn Frobenius, nhưng sai số nhỏ theo chuẩn không tự động nghĩa là mô hình vẫn đúng. Hãy đo lại trên tác vụ thật, và xem phổ (mục 10) trước khi chọn k.

(c) LoRA nhìn từ SVD. Cập nhật có dạng $W' = W + BA$, với $B \in \mathbb{R}^{m\times r}$, $A \in \mathbb{R}^{r\times n}$ và r nhỏ. Trong LoRA, A và B là hai ma trận nhân tử của bộ điều hợp (adapter), **không phải** ma trận ví dụ của chương này. Vì $\text{rank}(BA) \le r$, SVD của BA có tối đa r giá trị kỳ dị khác 0. Từ đó ta có thể hỏi: cập nhật tập trung ở vài hướng hay trải đều? Đây là câu hỏi phân tích, không phải kết luận rằng LoRA luôn hoạt động như vậy.

(d) Điều kiện số và chuẩn phổ trong ổn định huấn luyện. $\|W\|_2 = \sigma_1$ cho biết lớp tuyến tính phóng đại độ dài mạnh nhất là bao nhiêu. Nhiều lớp có σ_1 lớn xếp chồng nhau có thể làm tín hiệu và gradient bùng nổ. Spectral normalization chia W cho σ_1 để chuẩn phổ bằng 1 (chi tiết trong math 17). Điều kiện số κ (mục 8.3) cảnh báo độ nhạy khi giải và đảo, nhưng không tự động làm huấn luyện hỏng.

(e) Randomized SVD khi ma trận quá lớn. SVD đầy đủ tốn cỡ $O(mn\min(m,n))$ phép tính (mục 12), không khả thi khi m và n lên hàng trăm nghìn. Khi chỉ cần vài lớp đầu, SVD ngẫu nhiên chiếu A lên một không gian con nhỏ được chọn ngẫu nhiên, rồi làm SVD trên ma trận nhỏ đã chiếu. Chi tiết và các đảm bảo lý thuyết xem math 04.

12. Sai lầm hay gặp

- **Nhầm giá trị kỳ dị với trị riêng của A.** σ_i là căn bậc hai trị riêng của $A^\top A$, không phải trị riêng của A. Ví dụ: $N = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ có hai trị riêng đều bằng 0, nhưng $N^\top N = \mathrm{diag}(0, 1)$, nên giá trị kỳ dị là 1 và 0 (tự tính).
- **Quên v_i là hướng đầu vào, u_i là hướng đầu ra.** v_i sống trong $\mathbb{R}^n$, còn u_i sống trong $\mathbb{R}^m$. Khi $m \neq n$, so góc giữa chúng là vô nghĩa. Đúng là $Av_i = \sigma_i u_i$ và $A^\top u_i = \sigma_i v_i$, đừng đảo hai công thức này.
- **Đổi dấu không đồng bộ.** Đổi dấu cả cặp (u_i, v_i) thì A không đổi. Đổi dấu chỉ một vector thì A sai. Lỗi này hay gặp khi ghép kết quả từ hai nguồn.
- **Tin vào "hạng toán học" thay vì "hạng số học".** Trên máy, hạng là số σ vượt một ngưỡng có lý do rõ ràng (mục 8.4).
- **Dùng sai kích thước thin SVD.** Lấy U thin nhân với Σ đầy đủ, hoặc quên rằng numpy trả về full SVD mặc định. Hãy kiểm trước: U cỡ $m \times k$, Σ cỡ $k \times k$, $V^\top$ cỡ $k \times n$.
- **Đánh giá thấp chi phí, và lập $A^\top A$ rồi tìm trị riêng.** SVD đầy đủ tốn cỡ $O(mn \min(m, n))$ phép tính, nên đừng đoán chi phí bằng trực giác. Còn lập $A^\top A$ bình phương điều kiện số: $\kappa(A^\top A) = \kappa(A)^2$. Với A của chương, $\kappa(A) = 3$ nhưng $\kappa(A^\top A) = 9$ (tự tính). Đó là lý do thư viện số không làm theo cách tính tay ở mục 5.

13. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
SVD	$A = U\Sigma V^\top$	xoay, co giãn, xoay
Dạng lớp hạng 1	$A = \sum_i \sigma_i u_i v_i^\top$	tổng các lớp, lớp mạnh đứng đầu
Cặp hướng	$Av_i = \sigma_i u_i$, $A^\top u_i = \sigma_i v_i$	đầu vào và đầu ra khớp nhau
Giá trị kỳ dị	$\sigma_i = \sqrt{\lambda_i(A^\top A)}$	căn trị riêng của $A^\top A$
Hạng	$r = \#\{i : \sigma_i > 0\}$	số lớp thật sự có mặt
Chuẩn Frobenius	$\|A\|_F = \sqrt{\sum_i \sigma_i^2}$	độ lớn tổng
Chuẩn phổ	$\|A\|_2 = \sigma_1$	phóng đại tối đa
Điều kiện số	$\kappa(A) = \sigma_1/\sigma_{\min}$	độ nhạy khi giải và đảo
Eckart-Young	$\|A - A_k\|_F = \sqrt{\sum_{i>k} \sigma_i^2}$	cắt k lớp đầu là tối ưu
Sai số và năng lượng	$e_k^2 + E_k = 1$	hai cách nhìn cùng một việc

14. Bài tập tự tính

Năm bài từ dễ đến khó. Hãy tự làm trước khi xem đáp án ngay bên dưới. Đáp án đã được kiểm lại bằng numpy, nhưng tính tay mới là phần giúp bạn nhớ lâu.

Bài 1 (dễ). Tìm SVD của $\begin{pmatrix} 2 & 0 \\ 0 & -1 \end{pmatrix}$.

Gợi ý: đây là ma trận chéo. Giá trị kỳ dị là gì, và dấu âm đi vào đâu?

Đáp án. $\sigma = 2$ và 1. Một SVD là $U = I$, $\Sigma = \mathrm{diag}(2, 1)$, $V = \mathrm{diag}(1, -1)$. Kiểm: $I\,\mathrm{diag}(2, 1)\,\mathrm{diag}(1, -1) = \mathrm{diag}(2, -1)$, khớp với ma trận đề bài (tự tính). Dấu âm được đưa vào V. Đưa dấu vào U cũng đúng.

Bài 2 (dễ vừa). Tìm giá trị kỳ dị của $\begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}$.

Gợi ý: tính $A^\top A$ trước.

Đáp án. $A^\top A = \text{diag}(1, 4)$ (tự tính), nên $\sigma = 2$ và 1. Để kiểm thêm: với $v_1 = e_2$, ta có $u_1 = Av_1/2 = (1, 0)^\top$; với $v_2 = e_1$, ta có $u_2 = Av_2/1 = (0, 1)^\top$. Do đó $U = I$, $\Sigma = \text{diag}(2, 1)$, $V = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, và $U\Sigma V^\top = A$ (tự tính).

Bài 3 (vừa). Tìm xấp xỉ hạng 1 tốt nhất của $\begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$ theo chuẩn Frobenius, và tính sai số.

Gợi ý: với ma trận chéo dương, giữ lớp 1 nghĩa là giữ số nào? Phần nào bị bỏ đi?

Đáp án. $\sigma = 3$ và 1. Xấp xỉ tốt nhất là $A_1 = \text{diag}(3, 0) = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix}$. Sai số $A - A_1 = \text{diag}(0, 1)$ có chuẩn Frobenius bằng $1 = \sigma_2$ (tự tính), đúng định lý Eckart-Young.

Bài 4 (vừa khó). Ma trận $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ có hạng 1. Vì sao nó chỉ có một giá trị kỳ dị khác 0, và giá trị đó bằng bao nhiêu?

Đáp án. Hàng 2 bằng 2 lần hàng 1, nên hạng là 1. Vì hạng bằng số giá trị kỳ dị khác 0 (mục 6.2), chỉ có một giá trị kỳ dị khác 0. Cách 1: $\|A\|_F^2 = 1 + 4 + 4 + 16 = 25$ (tự tính). Vì chỉ có một σ khác 0, $\sigma_1^2 = 25$, nên $\sigma_1 = 5$ và $\sigma_2 = 0$. Cách 2: $A^\top A = \begin{pmatrix} 5 & 10 \\ 10 & 20 \end{pmatrix}$ (tự tính) có vết 25 và định thức $100 - 100 = 0$, nên trị riêng là 25 và 0, tức $\sigma_1 = 5$. Kiểm: với $u_1 = v_1 = \frac{1}{\sqrt{5}}(1, 2)^\top$, ta có $5\, u_1 v_1^\top = 5 \cdot \frac{1}{5} \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} = A$.

Bài 5 (khó, lý thuyết). Chứng minh $\|A\|_F^2 = \sum_i \sigma_i^2$ bằng tính chất vết (trace) và tính trực giao của U, V.

Gợi ý: viết $\|A\|_F^2$ thành vết của $A^\top A$, rồi thay $A = U\Sigma V^\top$.

Đáp án.

Bước 1: $\|A\|_F^2 = \sum_{i,j} A_{ij}^2 = \text{tr}(A^\top A)$, vì ô đường chéo $(A^\top A)_{jj}$ là tổng bình phương của cột j, và vết là tổng các ô đường chéo.

Bước 2: thay $A = U\Sigma V^\top$ và dùng $U^\top U = I$ (mục 5.7): $A^\top A = V\Sigma^\top \Sigma V^\top$.

Bước 3: đặt $Z = \Sigma^\top \Sigma$, một ma trận đường chéo. Dùng tính chất xoay vòng của vết, $\text{tr}(XY) = \text{tr}(YX)$, với $X = VZ$ và $Y = V^\top$:

$$\text{tr}(VZV^\top) = \text{tr}(V^\top VZ) = \text{tr}(Z),$$

vì $V^\top V = I$.

Bước 4: $\text{tr}(Z)$ là tổng các ô đường chéo của $\Sigma^\top \Sigma$, mà chúng là $\sigma_1^2, \ldots, \sigma_{\min(m,n)}^2$ và các số 0 còn lại. Vậy $\|A\|_F^2 = \sum_i \sigma_i^2$.

Cùng cơ chế này cho thấy nhân với ma trận trực giao không đổi chuẩn Frobenius: $\|QA\|_F^2 = \text{tr}(A^\top Q^\top QA) = \text{tr}(A^\top A)$.

Tài liệu tham khảo nền: Strang, Introduction to Linear Algebra; Trefethen & Bau, Numerical Linear Algebra; Halko et al. 2009 (math 04); Shlens 2014 (math 03).

Nền tảng 06

Đạo hàm, gradient, Jacobian, Hessian: đo sự thay đổi từ một con số đến nhiều biến

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector, tích vô hướng, ma trận, nhân ma trận với vector và trị riêng cơ bản

Dùng ở: math 01 (vi tích phân ma trận), math 02 (tự động tính đạo hàm), math 06 (gradient tự nhiên), math 18 (gradient trong mạng sâu)

> Chương này là tài liệu học từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần đo sự thay đổi?

Đạo hàm cho biết đầu ra thay đổi nhanh thế nào khi đầu vào thay đổi một lượng rất nhỏ. Nếu đầu vào gồm nhiều số, ta cần ghi riêng ảnh hưởng của từng số. Gradient gom các ảnh hưởng ấy khi đầu ra là một số; Jacobian làm cùng việc khi đầu ra là cả một vector; Hessian ghi sự thay đổi của gradient. Bốn tên gọi thực chất nối tiếp một câu hỏi: nếu ta nhích đầu vào, kết quả sẽ thay đổi ra sao?

Trong AI, tham số của mô hình là các đầu vào của một hàm mất mát. Gradient cho biết nhích từng tham số làm mất mát tăng hay giảm tại trạng thái hiện tại. Người huấn luyện thường đi theo hướng ngược gradient để tìm tham số tốt hơn. Jacobian mô tả độ nhạy của đầu ra theo đầu vào, còn Hessian giúp xét độ cong của mất mát và mức độ khó của bài toán tối ưu. Biết một gradient không có nghĩa ta đã tìm được nghiệm tốt nhất; nó chỉ là thông tin tại một điểm.

Chương này bắt đầu bằng một đường cong, sau đó mở rộng dần sang mặt phẳng và ma trận. Chương tiếp theo giải thích cách kết hợp các phép đo cục bộ để tính gradient của cả mạng.

2. Nhắc nhanh nền tảng

2.1 Hàm số và mức thay đổi

Một hàm f nhận đầu vào rồi trả về đầu ra. Với $f(x) = x^2$, đầu vào $x = 2$ cho $f(2) = 4$, còn đầu vào $x = 3$ cho $f(3) = 9$ (tự tính). Ta đã tăng đầu vào một đơn vị, đầu ra tăng năm đơn vị. Tuy nhiên, tốc độ thay đổi trung bình trên đoạn này không nhất thiết bằng tốc độ ngay tại đầu đoạn: đường cong có thể dốc hơn ở nơi khác.

Ký hiệu Δx đọc là "độ thay đổi của x"; nó có thể dương hoặc âm. Tương tự, $\Delta f = f(x + \Delta x) - f(x)$. Tỉ số $\Delta f / \Delta x$ đo mức tăng trung bình trên đoạn, với điều kiện $\Delta x \neq 0$. Đạo hàm được xây bằng cách nhìn điều gì xảy ra khi đoạn này thu nhỏ dần về một điểm.

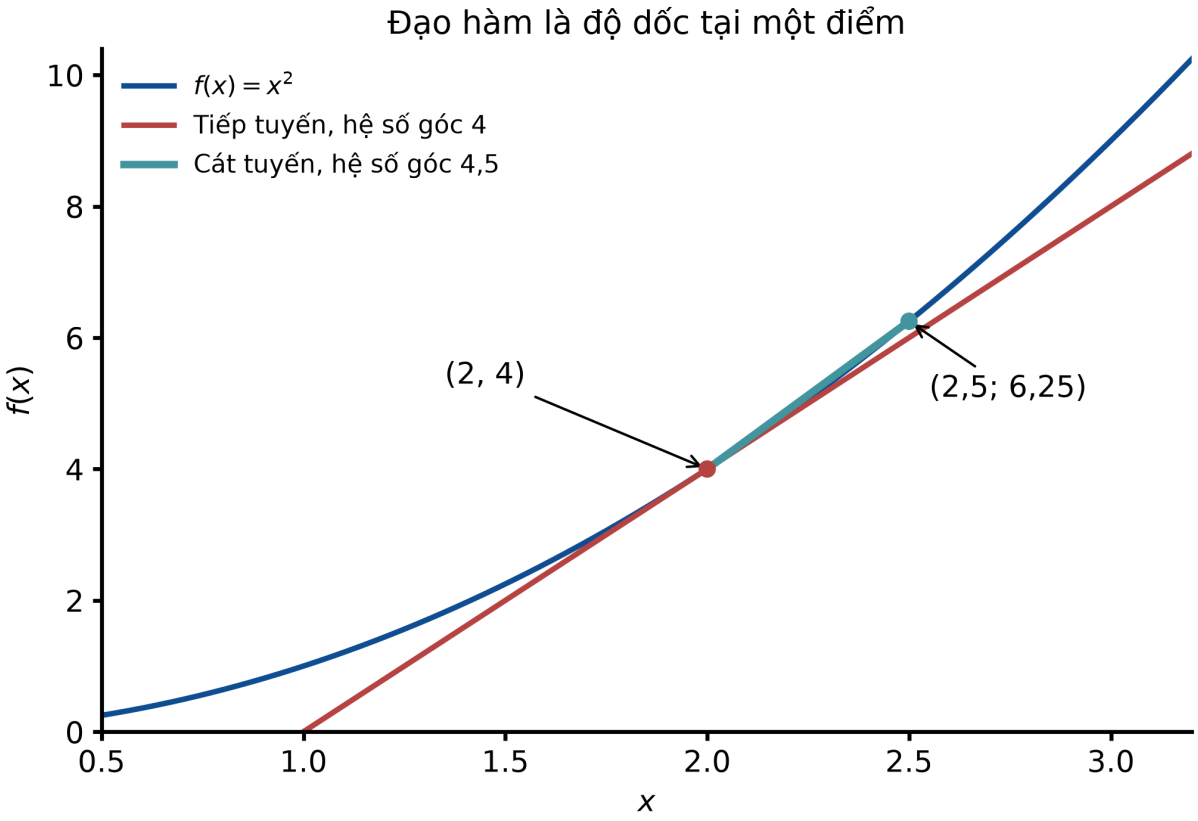
2.2 Vector và ma trận

Ta dùng vector cột, chẳng hạn $p = (1, 2)^\top$. Tích vô hướng của hai vector cùng cỡ là tổng các tích thành phần. Ví dụ $(1, 2)^\top \cdot (3, 4)^\top = 1 \cdot 3 + 2 \cdot 4 = 11$ (tự tính). Ký hiệu $v^\top w$ cũng biểu thị tích này; kết quả là một số.

Ma trận A cỡ $m \times n$ nhân vector cột v cỡ n cho vector cỡ m. Ví dụ $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ và $v = (1, 2)^\top$ cho $Av = (4, 7)^\top$ (tự tính). Kích thước là hàng rào kiểm lỗi: không thể nhân một ma trận với vector có số thành phần không khớp số cột.

3. Nhìn hình trước: cát tuyến tiến tới tiếp tuyến

Đường cong màu xanh trong hình là hàm bình phương. Lấy hai điểm trên đường cong, nối chúng bằng một đoạn thẳng: đó là cát tuyến. Khi điểm thứ hai tiến sát điểm đầu, nếu độ dốc của cát tuyến tiến tới một giá trị xác định thì đường thẳng giới hạn là tiếp tuyến, và độ dốc ấy là đạo hàm. Ta dùng điểm thứ hai cách điểm đầu một khoảng còn nhìn thấy được, nên hai độ dốc trong hình không trùng nhau.



Hình 20: Đường cong bình phương với tiếp tuyến và cát tuyến tại x bằng 2 (minh hoạ, tự tính)

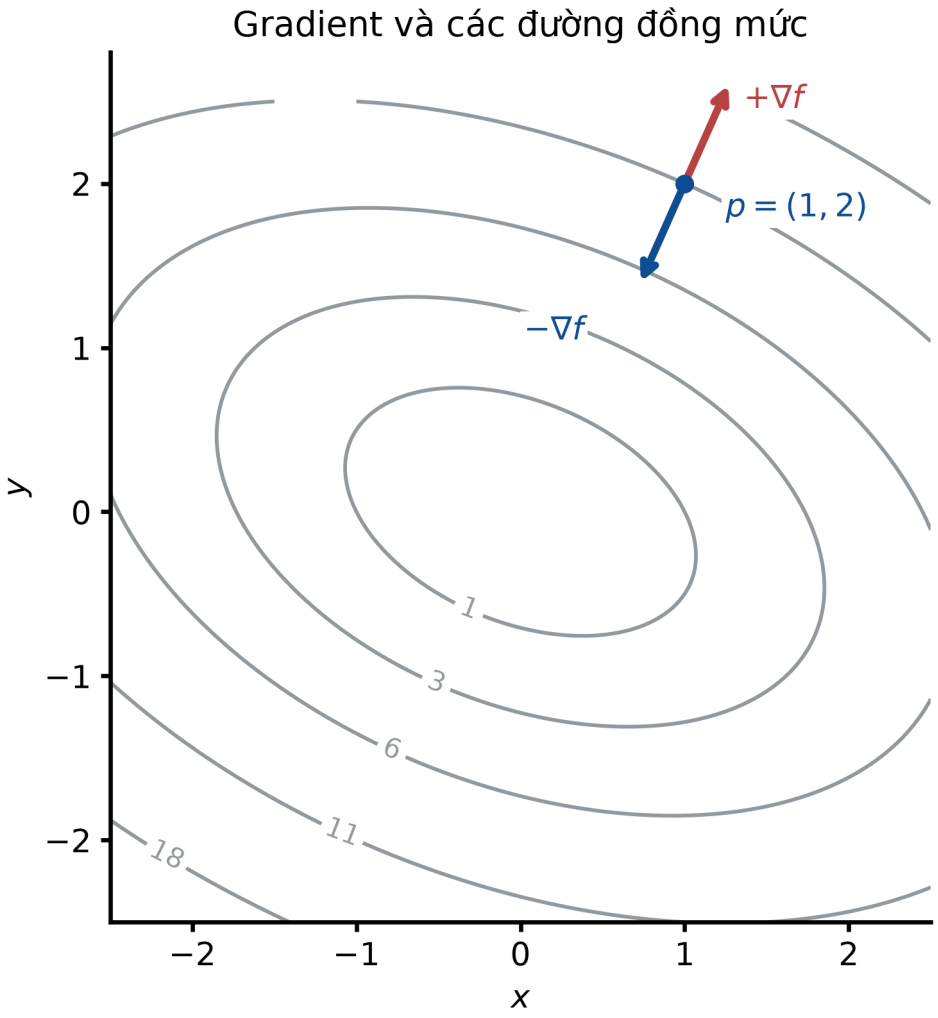
Cách đọc hình: đường xanh là giá trị của hàm, đường đỏ chạm đường cong tại điểm $(2, 4)$ và có độ dốc 4. Đoạn xanh ngọc nối đến điểm $(2{,}5; 6{,}25)$, có độ dốc 4,5 vì đầu ra tăng 2,25 khi đầu vào tăng 0,5 (tự tính). Độ dốc trung bình 4,5 không phải đạo hàm tại điểm đầu; khoảng cách giữa hai con số phản ánh việc đường cong còn uốn trên đoạn đang xét.

Trong nhiều chiều, ta không chỉ chọn điểm mà còn phải chọn hướng đi. Hình dưới cho thấy một mặt cao độ nhìn từ trên xuống: mỗi đường nối các điểm có cùng giá trị. Một mũi tên chỉ hướng tăng cục bộ mạnh nhất, mũi tên ngược lại chỉ hướng giảm cục bộ mạnh nhất khi đo các hướng bằng cùng độ dài.

Cách đọc hình: các đường xám ghi giá trị của hàm; điểm xanh là vị trí đang xét. Mũi tên đỏ chỉ theo gradient, mũi tên xanh chỉ ngược lại; cả hai đều bắt đầu tại cùng điểm. Tại điểm này mũi tên đỏ vuông góc với đường đồng mức đi qua điểm: đi dọc đường đồng mức không làm đổi giá trị ở bậc đầu, còn đi theo mũi tên làm giá trị tăng ở bậc đầu. Hình minh hoạ một vị trí cụ thể, không phải lời hứa rằng mọi bước dài theo mũi tên xanh đều làm hàm giảm.

4. Đạo hàm một biến: định nghĩa bằng giới hạn

Đạo hàm của f tại x là giới hạn của độ dốc cát tuyến khi độ thay đổi đầu vào tiến về 0, nếu giới hạn đó tồn tại và hữu hạn:



Hình 21: Gradient vuông góc đường đồng mức và hướng ngược gradient (minh hoạ, tự tính)

$$f'(x) = \frac{df}{dx}(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

Ký hiệu h chỉ một số khác 0 trong tỉ số, rồi tiến tới 0; ta không trực tiếp chia cho 0. Đạo hàm có đơn vị "đơn vị đầu ra trên một đơn vị đầu vào". Nếu đầu vào là giây và đầu ra là mét, đạo hàm là mét trên giây.

Với $f(x) = x^2$, ta khai triển $f(x+h) - f(x) = 2xh + h^2$; chia cho h được $2x + h$. Khi h tiến về 0, còn lại $f'(x) = 2x$. Ở $x = 2$, đạo hàm là 4. Đúng như hình, với $h = 0{,}5$, độ dốc cát tuyến là $4 + 0{,}5 = 4{,}5$ (tự tính).

Cách dùng thực tế của đạo hàm là xấp xỉ gần điểm đang đứng:

$$f(x+h) \approx f(x) + f'(x)h.$$

Với hàm bình phương tại $x = 2$ và $h = 0{,}1$, vế phải cho $4 + 4 \cdot 0{,}1 = 4{,}4$, còn giá trị chính xác là $2{,}1^2 = 4{,}41$ (tự tính). Sai khác $0{,}01$ là phần bình phương của bước. Một xấp xỉ cục bộ không có nghĩa hai vế bằng nhau với bước bất kỳ.

Một số quy tắc quen dùng là đạo hàm của tổng bằng tổng các đạo hàm, đạo hàm của x^n là nx^{n-1} với số mũ nguyên dương, và đạo hàm của tích uv là $u'v + uv'$. Chương 07 đi sâu vào đạo hàm của hàm hợp. Các quy tắc đều cần điều kiện hàm liên quan có đạo hàm tại điểm đang xét.

5. Đạo hàm riêng và gradient

Cho hàm $f(x_1, \ldots, x_n)$ trả về một số. **Đạo hàm riêng** theo x_i là đạo hàm khi chỉ thay đổi x_i và giữ các tọa độ còn lại cố định. Với vector cột, gradient cũng được viết thành vector cột:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}.$$

Từng thành phần cho biết phản ứng cục bộ theo một trục tọa độ. Một số tài liệu, trong đó có ghi chú math 01, viết gradient thành hàng. Hai quy ước diễn tả cùng các đạo hàm; ở đây ta chọn vector cột để $\nabla f^\top \Delta x$ là một số và ma trận Jacobian có hàng tương ứng đầu ra, cột tương ứng đầu vào. Khi đọc tài liệu khác, hãy kiểm quy ước chuyển vị trước khi so sánh công thức.

Ví dụ với $g(x, y) = x^2 + 3xy$, khi đạo hàm theo x, ta coi y là hằng số, nên $\partial g/\partial x = 2x + 3y$. Khi đạo hàm theo y, ta coi x là hằng số, nên $\partial g/\partial y = 3x$. Tại $(1, 2)$, gradient là $(8, 3)^\top$ (tự tính). Gradient là một vector gắn với điểm: đổi điểm thì các thành phần thường đổi.

Sự tồn tại của tất cả đạo hàm riêng tại một điểm chưa đủ bảo đảm một xấp xỉ tuyến tính tốt theo mọi hướng. Khi nói tới công thức xấp xỉ ở các mục dưới, ta giả định hàm khả vi tại điểm đó; các đa thức trong ví dụ đều thỏa. Sự phân biệt này quan trọng gần những chỗ gãy hoặc hàm có hành vi bất thường.

6. Ví dụ tay: từ đạo hàm riêng đến một bước nhỏ

Xét hàm dùng trong hình gradient và đường đồng mức:

$$f(x, y) = x^2 + xy + 2y^2, \qquad p = (1, 2)^\top.$$

Ta tính từng bước, với các số trong mục này đều là tự tính và đã kiểm bằng numpy.

Bước 1: giá trị tại điểm. Thay $x = 1$, $y = 2$ cho $f(p) = 1 + 2 + 8 = 11$ (tự tính). Đường đồng mức qua điểm vì thế có nhãn 11.

Bước 2: đạo hàm riêng theo từng trục. Giữ y cố định, đạo hàm của x^2 là $2x$, của xy là y, còn $2y^2$ không đổi. Vậy $\partial f/\partial x = 2x + y$. Tương tự, giữ x cố định cho $\partial f/\partial y = x + 4y$.

Bước 3: thế tọa độ vào gradient.

$$\nabla f(x, y) = \begin{pmatrix} 2x + y \\ x + 4y \end{pmatrix}, \qquad \nabla f(p) = \begin{pmatrix} 4 \\ 9 \end{pmatrix}.$$

Nếu tăng riêng x một lượng rất nhỏ, giá trị hàm tăng xấp xỉ 4 lần lượng đó. Nếu tăng riêng y cùng lượng, giá trị tăng xấp xỉ 9 lần. Đây là tốc độ cục bộ theo đơn vị tọa độ, không phải thay đổi chính xác khi tăng hẳn một đơn vị.

Bước 4: kiểm một dịch chuyển. Với $\Delta p = (0{,}1, -0{,}1)^\top$, tích vô hướng là $\nabla f(p)^\top \Delta p = 4{\cdot}0{,}1 + 9{\cdot}(-0{,}1) = -0{,}5$. Dự đoán tuyến tính là $11 - 0{,}5 = 10{,}5$. Giá trị thật tại $(1{,}1; 1{,}9)$ là $1{,}21 + 2{,}09 + 7{,}22 = 10{,}52$; sai khác là $0{,}02$ (tự tính). Sai khác không phủ nhận gradient: nó là phần độ cong bị bỏ qua trong xấp xỉ bậc một.

Bước 5: đi ngược gradient. Chọn hệ số bước $0{,}1$, ta được $p_{\text{mới}} = p - 0{,}1\nabla f(p) = (0{,}6; 1{,}1)^\top$. Giá trị mới $f(p_{\text{mới}}) = 0{,}36 + 0{,}66 + 2{,}42 = 3{,}44$, thấp hơn 11 (tự tính). Nhưng nếu chọn bước quá lớn, dự đoán cục bộ có thể không còn hữu ích. Chẳng hạn hệ số 1 đưa ta tới $(-3, -7)$ và giá trị hàm là 128, lớn hơn 11 (tự tính). Hướng đúng và độ dài bước thích hợp là hai quyết định khác nhau.

7. Đạo hàm theo hướng và ý nghĩa hình học

Chọn một hướng đơn vị u với $\|u\|_2 = 1$. Đạo hàm theo hướng đo tốc độ thay đổi khi đi từ p theo hướng đó:

$$D_u f(p) = \lim_{t \to 0} \frac{f(p + tu) - f(p)}{t} = \nabla f(p)^\top u,$$

khi f khả vi tại p. Yêu cầu vector đơn vị giúp so sánh các hướng với cùng một đơn vị quãng đường; nếu nhân đôi một vector không chuẩn hóa, đạo hàm theo tham số tương ứng cũng bị nhân đôi.

Với ví dụ trên, lấy $u = (3/5, 4/5)^\top$, ta có $\|u\|_2 = 1$ và $D_u f(p) = 4 \cdot 3/5 + 9 \cdot 4/5 = 48/5 = 9{,}6$ (tự tính). Hướng ngược lại cho tốc độ $-9{,}6$. Giá trị dương nghĩa là nhích một chút theo hướng đó làm hàm tăng; giá trị âm nghĩa là giảm.

Bất đẳng thức Cauchy và Schwarz cho $\nabla f(p)^\top u \le \|\nabla f(p)\|_2$. Nếu gradient khác vector 0, dấu bằng đạt tại $u = \nabla f(p)/\|\nabla f(p)\|_2$. Do đó gradient chỉ hướng tăng nhanh nhất trong các vector đơn vị theo chuẩn Euclid, và âm gradient chỉ hướng giảm nhanh nhất theo cùng chuẩn. Ở ví dụ, $\|\nabla f(p)\|_2 = \sqrt{97} \approx 9{,}849$, lớn hơn 9,6 (tự tính).

Nếu $\nabla f(p) = 0$, mọi đạo hàm theo hướng đều bằng 0, nhưng chưa thể kết luận đó là cực tiểu. Ngoài ra, ý nghĩa ”nhanh nhất” phụ thuộc thước đo độ dài và cách ta chuẩn hóa các biến: đổi đơn vị hoặc đổi hình học của không gian có thể làm hướng được chọn thay đổi. Gradient tự nhiên là một cách thay đổi hình học đang dùng, xem math 06.

Một đường đồng mức trơn qua p gồm những điểm cùng giá trị hàm. Nếu t là vector tiếp tuyến của đường đó, đi theo t không đổi giá trị ở bậc đầu, nên $\nabla f(p)^\top t = 0$: gradient vuông góc tiếp tuyến. Phát biểu về pháp tuyến chỉ có hướng rõ ràng khi gradient khác 0 và đường đồng mức đủ trơn.

8. Jacobian: nhiều đầu ra cùng lúc

Một hàm vector $F : \mathbb{R}^n \to \mathbb{R}^m$ có các thành phần $F_1, \dots, F_m$. **Jacobian** tại x là ma trận $m \times n$:

$$J_F(x) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial x_1} & \cdots & \frac{\partial F_m}{\partial x_n} \end{pmatrix}.$$

Hàng thứ i mô tả một đầu ra, cột thứ j mô tả một đầu vào. Với hàm khả vi và dịch chuyển nhỏ, $F(x+\Delta x) \approx F(x) + J_F(x)\Delta x$. Đây là bản nhiều đầu ra của công thức tiếp tuyến: Jacobian là phép biến đổi tuyến tính gần đúng cho các thay đổi nhỏ quanh điểm đang xét, không phải bản thân hàm trên toàn miền.

8.1 Ví dụ tay với hai đầu ra

Xét $F(x, y) = (x^2 + y, xy)^\top$. Các đạo hàm theo x và y của thành phần đầu là $2x$ và 1; của thành phần sau là y và x. Do đó

$$J_F(x, y) = \begin{pmatrix} 2x & 1 \\ y & x \end{pmatrix}, \qquad J_F(1, 2) = \begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix}.$$

Tại $(1, 2)$, đầu ra là $(3, 2)^\top$. Với cùng $\Delta p = (0{,}1, -0{,}1)^\top$ ở mục 6, Jacobian dự đoán thay đổi $(0{,}1; 0{,}1)^\top$. Tính trực tiếp cho $F(1{,}1; 1{,}9) = (3{,}11; 2{,}09)^\top$, tức thay đổi thật $(0{,}11; 0{,}09)^\top$ (tự tính). Sai lệch của dự đoán là $(0{,}01; -0{,}01)^\top$.

Chú ý hai hàng của Jacobian tại điểm này tình cờ giống nhau, dù hai hàm thành phần khác nhau. Chúng có cùng phản ứng bậc một ngay tại điểm đó; điều này không làm hai đầu ra bằng nhau hoặc giống nhau khi đi xa. Một Jacobian tính tại một điểm là thông tin cục bộ.

8.2 Trường hợp đặc biệt và kích thước

Nếu $F(x) = Ax + b$, với A cỡ $m \times n$, thì $J_F(x) = A$ ở mọi điểm. Đây là một phép tuyến tính cộng độ lệch, và xấp xỉ bằng Jacobian trở thành đẳng thức chính xác. Nếu hàm chỉ có một đầu ra, $J_f = \nabla f^\top$ là hàng cỡ $1 \times n$. Nếu chỉ có một đầu vào, Jacobian là một cột cho tốc độ thay đổi của từng đầu ra.

Đối với mạng lớn, ma trận Jacobian đầy đủ có thể rất lớn. Ta thường chỉ cần $J_F v$, tức phản ứng theo một hướng v, hoặc $J_F^\top w$, tức kéo một vector trọng số w từ đầu ra về đầu vào. Chương 07 sẽ nối hai tích này với tự động tính đạo hàm; không cần lập ma trận đầy đủ để tính được một trong hai tích.

9. Hessian: gradient đổi theo vị trí như thế nào

Với hàm scalar khả vi hai lần, **Hessian** là Jacobian của gradient, một ma trận $n \times n$:

$$H_f(x) = \nabla^2 f(x) = \left[\frac{\partial}{\partial x_j} \left(\frac{\partial f}{\partial x_i} \right) \right]_{i,j=1}^{n}.$$

Ô đường chéo cho độ cong theo từng trục; ô ngoài đường chéo cho sự tương tác giữa hai tọa độ trong thay đổi của gradient. Nếu các đạo hàm riêng cấp hai liên tục quanh điểm đang xét, thứ tự lấy đạo hàm hỗn hợp có thể hoán đổi và Hessian đối xứng. Không nên mặc định đối xứng khi thiếu giả thiết này.

Với $f(x, y) = x^2 + xy + 2y^2$ ở mục 6, ta đạo hàm tiếp $(2x + y,\ x + 4y)^\top$ và được

$$H_f = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}.$$

Hessian không phụ thuộc điểm vì hàm là bậc hai. Hai trị riêng là $3 - \sqrt{2}$ và $3 + \sqrt{2}$, đều dương (tự tính). Mọi hướng khác 0 vì thế có độ cong dương, và hàm có dạng một lòng chảo nghiêng. Điều này mạnh hơn việc nhìn riêng hai ô đường chéo: ma trận có đường chéo dương vẫn có thể có trị riêng âm nếu tương tác ngoài đường chéo đủ lớn.

9.1 Khai triển bậc hai

Với hàm đủ trơn và bước Δx nhỏ,

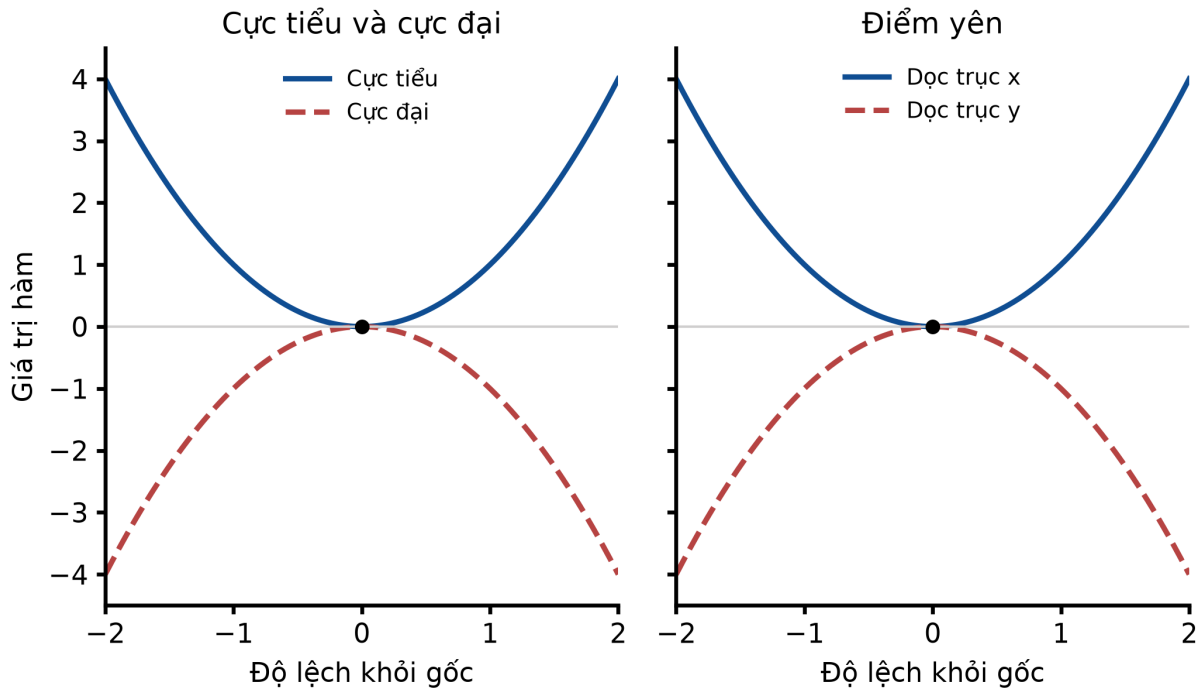
$$f(x + \Delta x) \approx f(x) + \nabla f(x)^\top \Delta x + \frac{1}{2} \Delta x^\top H_f(x) \Delta x.$$

Số hạng đầu là giá trị hiện tại, số hạng tuyến tính là dự đoán bằng gradient, còn số hạng bậc hai điều chỉnh theo độ cong. Với hàm bậc hai ở mục 6, công thức này đúng chính xác cho mọi bước. Thay $\Delta p = (0{,}1, -0{,}1)^\top$ cho phần điều chỉnh $\frac{1}{2} \Delta p^\top H_f \Delta p = 0{,}02$. Do đó $11 - 0{,}5 + 0{,}02 = 10{,}52$, khớp giá trị đã tính trực tiếp (tự tính).

Với hàm không phải bậc hai, còn phần dư và dấu xấp xỉ không thể đổi thành dấu bằng một cách tùy tiện. Hessian được tính tại điểm cũ, nên nó cũng là mô tả cục bộ.

9.2 Cực tiểu, cực đại và điểm yên

Một **điểm dừng** là điểm có gradient bằng 0. Nếu Hessian tại điểm dừng xác định dương, tức mọi trị riêng đều dương, điểm đó là cực tiểu địa phương nghiêm ngặt. Nếu mọi trị riêng âm, đó là cực đại địa phương nghiêm ngặt. Nếu có cả trị riêng dương lẫn âm, đó là điểm yên: có hướng đi làm giá trị tăng và hướng khác làm giá trị giảm trong lân cận đủ nhỏ.



Hình 22: Ba dạng độ cong qua cực tiểu, điểm yên và cực đại theo hai trục (minh hoạ, tự tính)

Cách đọc hình: panel trái vẽ lát cắt của $x^2 + y^2$ và $-x^2 - y^2$, lần lượt là cực tiểu và cực đại tại gốc. Hai lát cắt theo trục x và y của mỗi hàm này trùng nhau. Panel phải vẽ $x^2 - y^2$, theo trục x đường cong mở lên, theo trục y đường cong mở xuống, nên gốc là điểm yên (tự tính). Hình chỉ vẽ hai lát cắt, còn kết luận tổng quát cần kiểm mọi hướng, có thể thực hiện qua trị riêng của Hessian đối xứng.

Khi Hessian có trị riêng bằng 0 và các trị riêng còn lại không đủ cho một trong ba trường hợp trên, phép thử này không kết luận được. Ví dụ x^4 và $-x^4$ đều có đạo hàm bậc một và bậc hai bằng 0 tại gốc, nhưng gốc lần lượt là cực tiểu và cực đại (tự tính). Trị riêng bằng 0 không đồng nghĩa điểm yên.

10. Độ chính xác số và kiểm bằng sai phân

Ta có thể kiểm một thành phần gradient bằng sai phân hữu hạn. Với vector đơn vị tọa độ e_i, công thức trung tâm là

$$\frac{f(x + he_i) - f(x - he_i)}{2h} \approx \frac{\partial f}{\partial x_i}(x).$$

Đây là xấp xỉ, không phải định nghĩa thay thế của gradient khi dùng một giá trị h hữu hạn. Với hàm đủ trơn, sai số cắt cụt của sai phân trung tâm thường giảm theo bậc h^2 khi h nhỏ. Nhưng trên máy, lấy hiệu hai số gần nhau cũng làm sai số làm tròn trở nên đáng kể, nên chọn h nhỏ vô hạn không phải chiến lược tốt. Chương 07 minh họa sự cân bằng này bằng hình và dùng nó để kiểm backprop.

Không nên kiểm ở chỗ hàm không khả vi và đòi sai phân khớp một đạo hàm không tồn tại. Ví dụ hàm $\max(0, x)$ có nhánh trái bằng 0 và nhánh phải có độ dốc 1; tại gốc hai độ dốc một phía khác nhau. Một thư viện có thể chọn một giá trị quy ước để tiếp tục lan truyền, nhưng đó không làm đạo hàm cổ điển tồn tại ở gốc.

11. Nối sang AI

(a) Tối ưu mất mát. Gradient của mất mát theo trọng số chỉ phản ứng của mất mát khi từng trọng số thay đổi. Bộ tối ưu dùng thông tin đó cùng một quy tắc cập nhật; một gradient âm theo một trọng số không có nghĩa trọng số đó luôn phải tăng trong mọi bước tương lai. Xem math 01 để nối các đạo hàm riêng với phép tính trên ma trận.

(b) Độ nhạy của mô hình. Jacobian theo đầu vào mô tả thay đổi nhỏ của vector đầu ra do nhiễu nhỏ của đầu vào. Nếu một hướng bị phóng đại mạnh, đầu ra có thể nhạy theo hướng đó. Đây là kết luận cục bộ tại điểm đang xét, không tự nó chứng minh mô hình sẽ sai trên mọi nhiễu lớn.

(c) Hướng phẳng và hướng dốc. Trị riêng Hessian của mất mát bậc hai quanh một điểm cho độ cong theo các hướng riêng. Khi độ cong khác nhau nhiều, cùng một bước có thể quá ngắn theo hướng phẳng và quá dài theo hướng dốc. Hessian của mạng lớn có thể rất đắt để lập đầy đủ, nên người ta quan tâm đến các xấp xỉ hay tích Hessian với vector.

(d) Gradient tự nhiên. Hướng giảm mạnh nhất phụ thuộc cách đo độ dài trong không gian tham số. Gradient tự nhiên dùng thông tin hình học phân phối để chọn hướng cập nhật, thay vì chỉ dùng hình học Euclid thông thường. Ghi chú math 06 giải thích thêm về lựa chọn này và những điều kiện áp dụng.

(e) Lan truyền gradient trong mạng sâu. Gradient ở lớp đầu phụ thuộc các phép biến đổi tại nhiều lớp phía sau. Độ lớn và cấu trúc của những phép biến đổi này ảnh hưởng tín hiệu ngược; xem math 18 để tìm hiểu một vấn đề cụ thể của mạng sâu. Không cần suy ra mọi mạng sâu đều gặp cùng mức khó khăn.

12. Sai lầm hay gặp

- **Xem cát tuyến là đạo hàm chính xác.** Với bước hữu hạn, tỉ số sai phân đo thay đổi trung bình; đạo hàm là giới hạn khi bước tiến về 0.
- **Quên giữ biến còn lại cố định khi lấy đạo hàm riêng.** Khi tính theo x, biểu thức theo y vẫn có thể là một hằng số khác 0, không phải tự biến mất trong mọi tích.
- **Quên điểm và chiều của gradient.** $\nabla f(p)$ thay đổi theo p và theo quy ước ở đây là vector cột. So với tài liệu viết hàng, hãy chuyển vị trước khi nhân.
- **Nói gradient là hướng tăng nhanh nhất mà quên chuẩn.** Kết luận này so sánh các hướng có cùng độ dài Euclid; đổi thước đo độ dài có thể đổi hướng tối ưu.
- **Đảo hàng và cột Jacobian.** Ở đây hàng là đầu ra, cột là đầu vào. Nhân J_F với một thay đổi đầu vào để kiểm cỡ kết quả.
- **Thấy gradient bằng 0 rồi kết luận cực tiểu, hoặc thấy Hessian có trị riêng 0 rồi kết luận điểm yên.** Cần kiểm độ cong và nhớ rằng phép thử bậc hai có trường hợp không kết luận được.

13. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Đạo hàm	$f'(x) = \lim_{h\to 0}[f(x+h)-f(x)]/h$	độ dốc tức thời nếu giới hạn tồn tại
Gradient cột	$\nabla f = (\partial f/\partial x_i)_i$	gom đạo hàm riêng của một đầu ra
Xấp xỉ bậc một	$f(x+\Delta x) \approx f(x) + \nabla f(x)^\top \Delta x$	dự đoán thay đổi nhỏ
Theo hướng đơn vị	$D_u f = \nabla f^\top u$	tốc độ dọc hướng u
Jacobian	$(J_F)_{ij} = \partial F_i/\partial x_j$	ánh xạ thay đổi đầu vào tới đầu ra
Hessian	$(H_f)_{ij} = \partial(\partial f/\partial x_i)/\partial x_j$	độ cong và tương tác tọa độ
Bậc hai	$f(x+\Delta x) \approx f(x) + \nabla f^\top \Delta x + \frac{1}{2}\Delta x^\top H_f \Delta x$	hiệu chỉnh tuyến tính bằng độ cong
Điểm dừng	$\nabla f = 0$	ứng viên cần kiểm thêm

14. Bài tập tự tính

Năm bài từ dễ đến khó. Hãy tự tính rồi xem ngay lời giải; mọi kết quả số bên dưới đều là tự tính, đã kiểm bằng numpy.

Bài 1 (dễ). Với $f(x) = 3x^2 - 2x + 1$, tính $f(2)$, $f'(2)$ và độ dốc cát tuyến từ $x = 2$ đến $x = 2{,}1$.

Đáp án. $f(2) = 12 - 4 + 1 = 9$ và $f'(x) = 6x - 2$, nên $f'(2) = 10$. Với $x = 2{,}1$, giá trị bằng $10{,}03$, do đó độ dốc cát tuyến là $(10{,}03 - 9)/0{,}1 = 10{,}3$ (tự tính). Nó gần nhưng không bằng độ dốc tức thời.

Bài 2 (vừa). Với $f(x, y) = x^2 y + y^2$, tính gradient tại $(2, 1)$ và đạo hàm theo hướng đơn vị $(3/5, 4/5)^\top$.

Đáp án. Hai đạo hàm riêng là $2xy$ và $x^2 + 2y$. Thay điểm cho $\nabla f(2, 1) = (4, 6)^\top$. Tích vô hướng với hướng đã cho là $4 \cdot 3/5 + 6 \cdot 4/5 = 36/5 = 7{,}2$ (tự tính). Hướng đã có chuẩn bằng 1.

Bài 3 (vừa). Cho $G(x, y) = (x + y, xy, x^2)^\top$. Tính Jacobian và dự đoán thay đổi đầu ra tại $(1, 2)$ khi $\Delta p = (0{,}1, -0{,}2)^\top$. So với thay đổi thật.

Đáp án. Jacobian là $J_G(x, y) = \begin{pmatrix} 1 & 1 \\ y & x \\ 2x & 0 \end{pmatrix}$, cỡ 3×2. Tại điểm đề bài, nhân với Δp cho $(-0{,}1; 0; 0{,}2)^\top$. Đầu ra ban đầu là $(3, 2, 1)^\top$, đầu ra mới là $(2{,}9; 1{,}98; 1{,}21)^\top$, nên thay đổi thật là $(-0{,}1; -0{,}02; 0{,}21)^\top$ (tự tính). Sai khác là phần bậc hai.

Bài 4 (khó hơn). Xét $q(x, y) = x^2 + 4xy + y^2$. Phân loại điểm gốc bằng Hessian và kiểm theo hai hướng.

Đáp án. Gradient là $(2x + 4y, 4x + 2y)^\top$, nên gốc là điểm dừng. Hessian $\begin{pmatrix} 2 & 4 \\ 4 & 2 \end{pmatrix}$ có hai trị riêng 6 và -2, nên gốc là điểm yên. Quả thật $q(t, t) = 6t^2$ và $q(t, -t) = -2t^2$ (tự tính): một hướng tăng, một hướng giảm. Chỉ nhìn hai phần tử đường chéo dương sẽ bỏ sót tương tác $4xy$.

Bài 5 (tổng hợp). Với $f(x, y) = x^2 + y^2$ tại $p = (1, 2)^\top$, lấy một bước gradient với hệ số $0{,}1$. Tính giá trị mới trực tiếp và bằng khai triển bậc hai.

Đáp án. Giá trị ban đầu là 5, gradient là $(2, 4)^\top$, Hessian là $2I$. Bước $\Delta p = -0{,}1(2, 4)^\top = (-0{,}2, -0{,}4)^\top$ đưa điểm tới $(0{,}8; 1{,}6)^\top$, có giá trị $0{,}64 + 2{,}56 = 3{,}2$. Phần tuyến tính là $(2, 4) \cdot (-0{,}2, -0{,}4) = -2$; phần bậc hai là $\frac{1}{2}\Delta p^\top (2I)\Delta p = 0{,}2$, nên $5 - 2 + 0{,}2 = 3{,}2$ (tự tính). Vì đây là hàm bậc hai, khai triển không có phần dư.

Tài liệu tham khảo nền: Parr và Howard, The Matrix Calculus You Need For Deep Learning (math 01); Baydin et al., Automatic Differentiation in Machine Learning (math 02); Goodfellow, Bengio và Courville, Deep Learning.

Nền tảng 07

Chain rule và backprop: nhân độ nhạy trên đường, cộng đóng góp ở chỗ phân nhánh

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: đạo hàm, gradient cột, Jacobian và nhân ma trận với vector ở nền tảng 06

Dùng ở: math 01 (quy tắc chuỗi trên ma trận), math 02 (tự động tính đạo hàm), math 18 (gradient mạng sâu), math 23 (đạo hàm đầu ra theo tham số)

> Chương này là tài liệu học từ đầu, không phải bản dịch paper. Các phép tính ghi ”tự tính” đã được kiểm bằng numpy.

1. Vì sao cần quy tắc chuỗi?

Quy tắc chuỗi cho biết tác động của một thay đổi được truyền qua các phép tính liên tiếp như thế nào. Một mạng không nhận trọng số rồi tạo mất mát bằng một phép tính duy nhất: nó nhân, cộng, áp hàm kích hoạt và tính sai số. Backpropagation, hay lan truyền ngược, áp quy tắc chuỗi qua các phép đó theo chiều từ mất mát về các tham số.

Ta có thể viết toàn bộ mạng thành một công thức khổng lồ rồi tự lấy đạo hàm, nhưng cách đó khó theo dõi và lặp đi lặp lại các phép tính. Thay vào đó, lưu giá trị trung gian khi tính xuôi và tái dùng chúng khi tính ngược. Mỗi phép chỉ cần biết đạo hàm cục bộ của chính nó. Khi một đại lượng được dùng ở nhiều nơi, ảnh hưởng của tất cả những đường đó phải được cộng lại.

Chương này đi từ một hàm hợp một biến đến mạng hai đầu vào, hai neuron ẩn và một đầu ra. Ta tính toàn bộ gradient bằng tay, kiểm bằng sai phân, và chỉ sau đó nhìn sang tự động tính đạo hàm. Một lượt backprop cung cấp gradient; việc chọn cách cập nhật là việc của bộ tối ưu.

2. Nhắc nhanh nền cần thiết

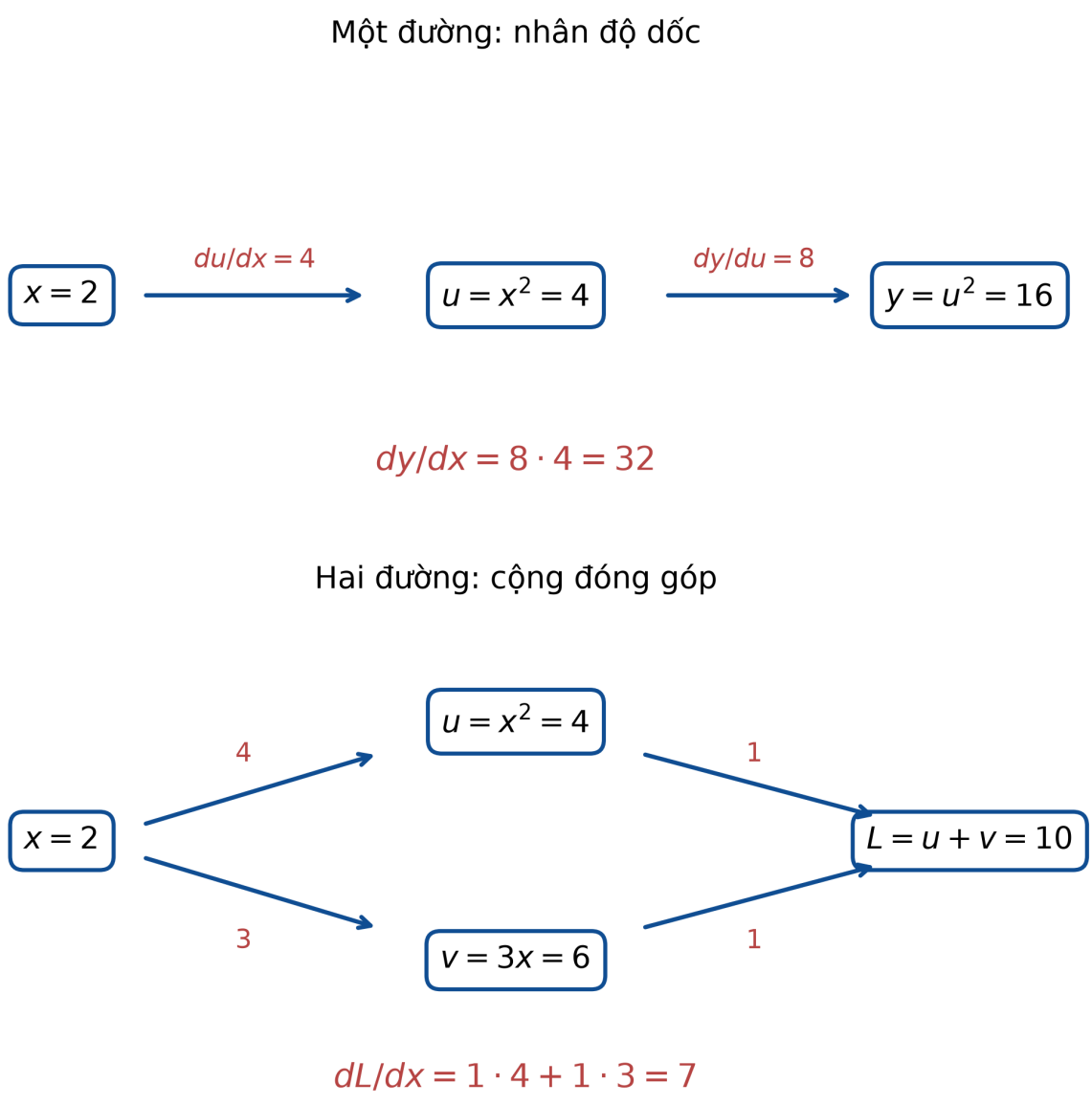
Đạo hàm $f'(x)$ mô tả mức thay đổi cục bộ của đầu ra khi x thay đổi. Nếu $f(x) = x^2$, thì $f'(x) = 2x$, do đó tại $x = 2$ đạo hàm là 4 (tự tính). Với hàm scalar nhận nhiều biến, gradient là vector cột chứa các đạo hàm riêng; Jacobian của một hàm vector có hàng là đầu ra, cột là đầu vào. Ký hiệu và quy ước ở đây giống nền tảng 06.

Ta sẽ dùng các đạo hàm cục bộ đơn giản: tổng có đạo hàm theo mỗi đối số bằng 1; tích ab có đạo hàm theo a bằng b khi giữ b cố định; bình phương u^2 có đạo hàm $2u$. Ví dụ nếu $a = 2$ và $b = 3$, thay đổi nhỏ của a làm tích thay đổi xấp xỉ 3 lần lượng thay đổi đó, còn thay đổi nhỏ của b có hệ số 2 (tự tính). Từ đây, phép ghép quyết định phải nhân hay cộng những hệ số cục bộ ấy.

Một vector cột x cỡ n khi nhân với ma trận W cỡ $m \times n$ cho vector Wx cỡ m. Tích ngoài $uv^\top$ của u cỡ m và v cỡ n là ma trận $m \times n$, với phần tử hàng i, cột j bằng $u_i v_j$. Ta sẽ cần nó để viết gradient của ma trận trọng số.

3. Nhìn hình trước: một đường và nhiều đường

Panel trên có một chuỗi thao tác: đầu vào thành một giá trị trung gian, rồi thành đầu ra. Khi đầu vào nhích, tác động lên đầu ra bằng tích của hai độ nhạy cục bộ. Panel dưới dùng cùng một đầu vào ở hai nhánh; mỗi nhánh đóng góp riêng vào đầu ra, nên phải cộng hai đóng góp.



Hình 23: Quy tắc chuỗi trên một đường và phép cộng ảnh hưởng của hai nhánh (minh hoạ, tự tính)

Cách đọc hình: ở panel trên, tại đầu vào 2, nút giữa bằng 4 và nút cuối bằng 16; các độ dốc cục bộ lần lượt là 4 và 8. Kết quả ở dưới là tích $8 \cdot 4 = 32$ (tự tính). Ở panel dưới, mỗi số màu đỏ cạnh mũi tên là một đạo hàm cục bộ; có hai đường từ đầu vào tới mất mát, cho tổng $1 \cdot 4 + 1 \cdot 3 = 7$ (tự tính). Không được chỉ giữ một đường rồi bỏ đường còn lại.

Có thể hình dung một thay đổi rất nhỏ như tin nhắn đi qua mạng lưới ống: trên một đường, mỗi đoạn nhân mức khuếch đại; tại chỗ nhiều đường cùng ảnh hưởng đến kết quả, các đóng góp được cộng. Đây là trực giác bậc một, không phải quy tắc cho các thay đổi lớn tùy ý.

4. Quy tắc chuỗi một biến

Cho $u = g(x)$ và $y = f(u)$, với g khả vi tại x và f khả vi tại $g(x)$. Khi đó

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx} = f'(g(x))g'(x).$$

Phải tính đạo hàm của hàm ngoài **tại giá trị trung gian** $g(x)$, rồi nhân với đạo hàm của hàm trong tại x. Ký hiệu phân số giúp nhớ thứ tự, nhưng chứng minh dựa trên xấp xỉ tuyến tính, không phải quy tắc khử các ký hiệu một cách máy móc.

Với panel trên của hình quy tắc chuỗi, $u = x^2$ và $y = u^2$. Tại $x = 2$, ta có $u = 4$, $du/dx = 2x = 4$ và $dy/du = 2u = 8$. Vì vậy $dy/dx = 8 \cdot 4 = 32$ (tự tính). Có thể kiểm bằng cách thế trực tiếp $y = x^4$, rồi đạo hàm được $4x^3 = 32$ tại cùng điểm.

Tại sao lại nhân? Khi x đổi một lượng nhỏ Δx, ta có $\Delta u \approx g'(x)\Delta x$, và tiếp đó $\Delta y \approx f'(u)\Delta u$. Thay biểu thức đầu vào biểu thức sau, $\Delta y \approx f'(u)g'(x)\Delta x$. Cách giải thích bằng giới hạn vẫn áp dụng cả khi một độ dốc bằng 0; không cần giả sử được phép chia cho độ thay đổi trung gian khác 0.

Ví dụ nữa, $y = (3x + 1)^2$ có $u = 3x + 1$. Tại $x = 2$, $u = 7$, nên đạo hàm là $2u \cdot 3 = 42$ (tự tính). Chỉ lấy $2u$ sẽ bỏ quên tác động của phép biến đổi bên trong.

5. Nhiều đường ảnh hưởng: cộng tại chỗ dùng chung

Giả sử $u = x^2$, $v = 3x$ và $L = u + v$. Ở $x = 2$, các giá trị xuôi là $u = 4$, $v = 6$, $L = 10$ (tự tính). Với thay đổi nhỏ của x, nhánh u đóng góp xấp xỉ $4\Delta x$, nhánh v đóng góp $3\Delta x$. Cả hai đi vào phép cộng, nên tổng thay đổi xấp xỉ $7\Delta x$.

Viết theo đạo hàm:

$$\frac{dL}{dx} = \frac{\partial L}{\partial u}\frac{du}{dx} + \frac{\partial L}{\partial v}\frac{dv}{dx} = 1 \cdot 4 + 1 \cdot 3 = 7.$$

Kiểm bằng cách rút gọn $L = x^2 + 3x$: đạo hàm $2x + 3$ bằng 7 tại $x = 2$ (tự tính). Nếu L còn nhận trực tiếp x như một đối số riêng, ta phải cộng cả đóng góp trực tiếp đó. Công thức tổng quát là cộng đóng góp qua từng đường độc lập về mặt tính toán, không chỉ đường nhìn thấy đầu tiên.

5.1 Một nút được dùng hai lần trong phép nhân

Xét $L = x \cdot x$ tại $x = 3$. Nếu xem hai vị trí của phép nhân là hai đầu vào riêng tạm thời, đạo hàm cục bộ theo mỗi vị trí bằng giá trị của vị trí kia, đều là 3. Vì cùng một x xuất hiện ở cả hai vị trí, hai đóng góp cộng lại thành $3 + 3 = 6$ (tự tính). Đó là lý do đạo hàm x^2 là $2x$, và cũng là một ví dụ về tích lũy gradient khi tái sử dụng một nút.

Nhầm lẫn phổ biến là nghĩ phải cộng các giá trị xuôi của hai đường thay vì cộng **đạo hàm** mà chúng gửi về. Trong ví dụ $L = u + v$, giá trị xuôi của hai nhánh là 4 và 6; đóng góp ngược tại x lại là 4 và 3. Hai phép cộng diễn ra ở hai bước và mang ý nghĩa khác nhau.

6. Quy tắc chuỗi cho vector và ma trận

Cho $u = g(x)$ với $x \in \mathbb{R}^n$, $u \in \mathbb{R}^k$, rồi $y = f(u)$ với $y \in \mathbb{R}^m$. Theo quy ước hàng là đầu ra, cột là đầu vào,

$$J_y(x) = J_f(u) J_g(x), \qquad (m \times n) = (m \times k)(k \times n).$$

Phần tử hàng i, cột j là tổng $\sum_{r=1}^{k} (\partial y_i / \partial u_r)(\partial u_r / \partial x_j)$. Tổng theo r chính là cộng ảnh hưởng của mọi tọa độ trung gian, còn mỗi tích là chuỗi độ nhạy trên một đường. Nhân ma trận thực hiện cả phép nhân lẫn phép cộng một cách gọn gàng.

6.1 Ví dụ tay: hai đầu vào, hai đầu ra

Xét $u_1 = x_1 + x_2$, $u_2 = x_1 x_2$, sau đó $y_1 = u_1^2 + u_2$, $y_2 = u_1 - u_2$. Tại $x = (1, 2)^\top$, ta có $u = (3, 2)^\top$ và $y = (11, 1)^\top$ (tự tính).

Bắt đầu từ hàm trong, đạo hàm từng thành phần cho

$$J_g(x) = \begin{pmatrix} 1 & 1 \\ x_2 & x_1 \end{pmatrix}, \qquad J_g(1, 2) = \begin{pmatrix} 1 & 1 \\ 2 & 1 \end{pmatrix}.$$

Tiếp theo, coi u_1, u_2 là đầu vào của hàm ngoài. Ta có

$$J_f(u) = \begin{pmatrix} 2u_1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad J_f(3, 2) = \begin{pmatrix} 6 & 1 \\ 1 & -1 \end{pmatrix}.$$

Nhân hai ma trận theo đúng thứ tự:

$$J_y(1, 2) = \begin{pmatrix} 6 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 8 & 7 \\ -1 & 0 \end{pmatrix}.$$

Chẳng hạn ô trên cùng bên trái bằng $6 \cdot 1 + 1 \cdot 2 = 8$: một đóng góp qua u_1, một đóng góp qua u_2 (tự tính). Kiểm độc lập bằng công thức $y_1 = (x_1 + x_2)^2 + x_1 x_2$ cho đạo hàm theo x_1 là $2(x_1 + x_2) + x_2 = 8$ tại điểm đã chọn. Không được đảo thứ tự nhân chỉ vì cả hai ma trận ở ví dụ tình cờ đều là 2×2.

6.2 Khi đầu ra cuối là một số

Với $L = f(u)$ scalar và $u = g(x)$, gradient được viết thành cột. Từ quy tắc Jacobian hàng, chuyển vị hai vế cho

$$\nabla_x L = J_g(x)^\top \nabla_u L.$$

Vector bên phải $\nabla_u L$ có k thành phần, $J_g^\top$ có cỡ $n \times k$, nên đầu ra là vector n thành phần, cùng kích thước x. Đây là một **tích Jacobian chuyển vị với vector**. Nếu chỉ cần gradient của một mất mát scalar, ta có thể truyền vector ngược qua từng phép mà không tạo toàn bộ Jacobian của cả mạng.

Ví dụ nếu giữ y_1 của mục 6.1 làm mất mát, ta có $\nabla_u L = (6, 1)^\top$. Nhân $J_g(1, 2)^\top (6, 1)^\top$ được $(8, 7)^\top$, trùng hàng đầu của Jacobian tổng sau khi chuyển thành cột (tự tính).

7. Đồ thị tính toán và lan truyền ngược

Một **đồ thị tính toán** là cách tách chương trình thành các nút giá trị và các phép tạo ra chúng. Lượt xuôi đi theo mũi tên để tính giá trị từng nút. Lượt ngược bắt đầu ở mất mát, đi theo thứ tự ngược, và cộng mọi ảnh hưởng mà một nút nhận được từ các nút sử dụng nó.

Gọi $\bar{v}$ là đạo hàm của mất mát cuối theo giá trị trung gian v. Để hiểu phép lấy đạo hàm này, hãy thay nút v bằng một đầu vào độc lập, cho nó thay đổi rồi tính lại toàn bộ các phép phía sau, trong khi giữ các nguồn độc lập khác cố định. Tín hiệu tại v cộng ảnh hưởng qua mọi đường từ nút đó tới mất mát. Với phép $z = ab$, khi đã có $\bar{z}$, ta gửi $\bar{z}b$ về a và $\bar{z}a$ về b. Với phép $z = a + b$, ta gửi $\bar{z}$ về mỗi đầu vào. Nếu một đầu vào được dùng nhiều nơi, mỗi lần gửi là một đóng góp và phải cộng vào cùng biến tích lũy.

Mất mát đầu ra nhận hạt giống $\bar{L} = 1$, vì đạo hàm của L theo chính nó bằng 1. Điều đó không có nghĩa mọi gradient đều bằng 1; các đạo hàm cục bộ lần lượt thay đổi tín hiệu trên đường về. Việc lưu một số giá trị xuôi là cần thiết: đạo hàm của phép nhân theo a cần biết giá trị b đã dùng tại lượt xuôi.

8. Ví dụ chính: một mạng nhỏ tính xuôi

Ta xét một mẫu dữ liệu và một mạng có hai đầu vào, hai neuron ẩn ReLU, một đầu ra scalar. Đặt

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad v = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad c = 0, \quad y = 1.$$

Quy ước hàng của W là neuron ẩn, cột là đầu vào. y là nhãn cố định, không phải đầu ra mô hình. Công thức mạng và mất mát cho một mẫu là

$$z = Wx + b, \qquad a = \mathrm{ReLU}(z), \qquad s = v^{\top} a + c, \qquad L = \frac{1}{2}(s - y)^2.$$

ReLU được áp theo từng phần tử: $\mathrm{ReLU}(t) = \max(0, t)$. Hệ số $\frac{1}{2}$ trong mất mát được chọn để đạo hàm theo s gọn là $s - y$. Định nghĩa này áp cho một mẫu, không mặc nhiên là trung bình của cả batch. Các con số của ví dụ dưới đây là tự tính và đã kiểm bằng numpy.

8.1 Bước 1: lớp tuyến tính

$$z = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

Neuron thứ nhất cộng hai đầu vào, nên $z_1 = 1 + 2 = 3$. Neuron thứ hai lấy $1 - 2 = -1$. Dấu của mỗi thành phần quyết định ReLU sẽ truyền hay chặn giá trị đó.

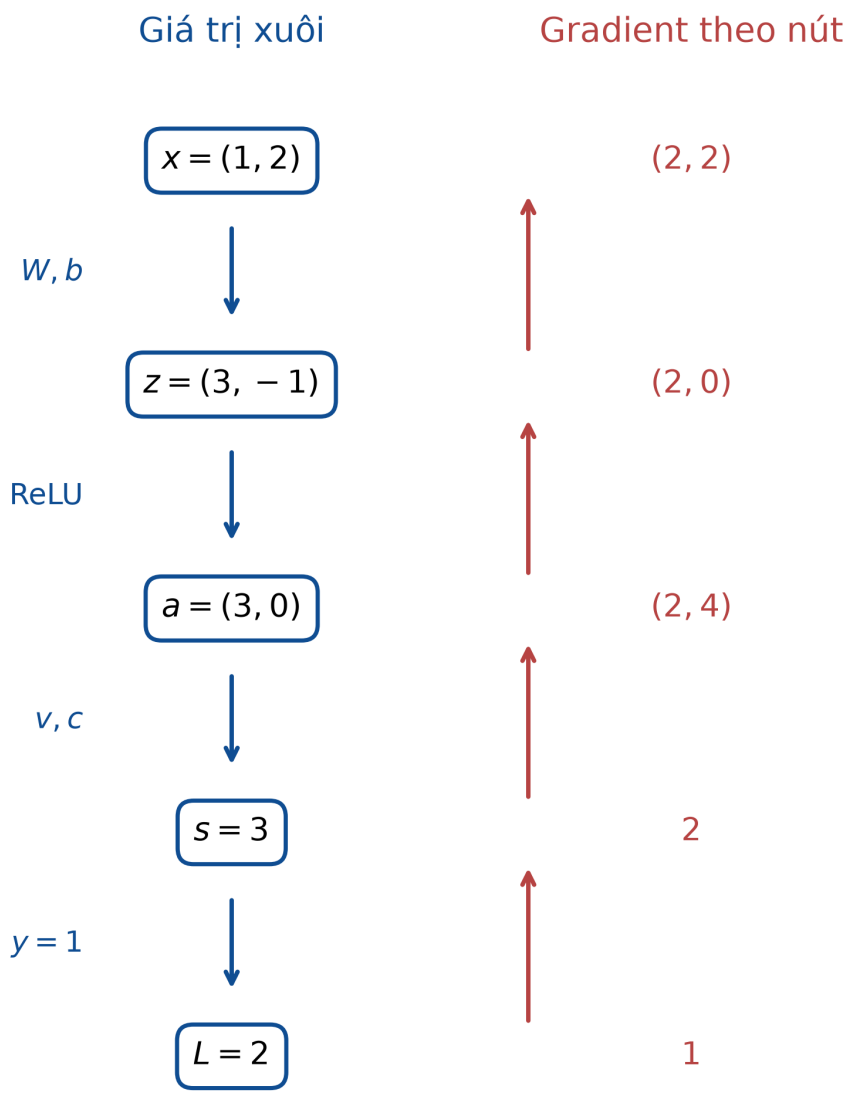
8.2 Bước 2: kích hoạt

ReLU giữ số dương và thay số âm bằng 0, nên $a = (3, 0)^{\top}$ (tự tính). Ta cố ý chọn cả hai giá trị đầu vào ReLU khác 0, do đó đạo hàm cổ điển ở hai điểm này đều xác định. Thành phần đầu đang hoạt động, thành phần sau không hoạt động cho mẫu đang xét.

8.3 Bước 3: đầu ra và mất mát

Nhân với v và cộng c cho $s = 1 \cdot 3 + 2 \cdot 0 + 0 = 3$. So với nhãn 1, sai lệch là $s - y = 2$ và mất mát $L = \frac{1}{2} \cdot 2^2 = 2$ (tự tính). Ta lưu x, z, a, s, y để tính ngược.

Cách đọc hình: cột xanh từ trên xuống dưới ghi các giá trị tại từng nút của ví dụ; tên bên trái mũi tên chỉ phép dùng tham số hoặc hàm kích hoạt. Cột đỏ đi từ dưới lên là đạo hàm của mất mát theo giá trị của nút tương



Hình 24: Lượt xuôi và tín hiệu lan truyền ngược qua mạng hai neuron ẩn (minh hoạ, tự tính)

ứng, không phải giá trị của chính nút đó. Tại đầu vào ReLU, tín hiệu ngược là $(2, 0)^\top$: thành phần thứ hai bị chặn vì đầu vào của nó âm (tự tính). Những con số này sẽ được suy ra từng bước dưới đây.

9. Ví dụ chính: backprop từng tham số

Bắt đầu ở L, dùng các giá trị đã lưu tại mục 8. Ta ký hiệu $\bar{s} = \partial L/\partial s$, $\bar{a} = \nabla_a L$ và tương tự. Mỗi vector gradient là cột; gradient theo ma trận có đúng cùng cỡ với ma trận tham số.

9.1 Từ mất mát về đầu ra

Vì $L = \frac{1}{2}(s - y)^2$, đạo hàm theo s bằng $s - y$. Do đó

$$\bar{s} = s - y = 3 - 1 = 2.$$

Nếu s nhích lên một lượng rất nhỏ, mất mát hiện tăng xấp xỉ hai lần lượng đó. Ta giữ nhãn y cố định trong phép tính gradient theo tham số.

9.2 Từ đầu ra về v, c và a

Khai triển $s = v_1 a_1 + v_2 a_2 + c$. Đạo hàm theo v_i là a_i, còn theo a_i là v_i. Nhân các đạo hàm cục bộ với $\bar{s} = 2$:

$$\nabla_v L = \bar{s} a = \begin{pmatrix} 6 \\ 0 \end{pmatrix}, \qquad \frac{\partial L}{\partial c} = \bar{s} = 2, \qquad \bar{a} = \bar{s} v = \begin{pmatrix} 2 \\ 4 \end{pmatrix}.$$

Tất cả đều tự tính. Đạo hàm theo v_2 bằng 0 vì $a_2 = 0$, dù v_2 bằng 2. Tuy vậy, đạo hàm theo giá trị kích hoạt a_2 là 4: nếu có thể thay đổi trực tiếp a_2 khỏi giá trị 0, đầu ra sẽ phản ứng. Bước ReLU tiếp theo mới quyết định có truyền tín hiệu ấy về z_2 được hay không.

9.3 Qua ReLU

Với đầu vào dương, đạo hàm ReLU bằng 1; với đầu vào âm, bằng 0. Tại $z = (3, -1)^\top$, nhân từng thành phần của $\bar{a}$ với các đạo hàm cục bộ tương ứng cho

$$\bar{z} = \begin{pmatrix} 1 \cdot 2 \\ 0 \cdot 4 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}.$$

Tín hiệu của neuron thứ hai bị chặn cho mẫu này, nên hàng tương ứng của gradient theo W sẽ bằng 0. Điều đó không có nghĩa neuron mãi mãi không ảnh hưởng: một mẫu khác hoặc một bộ tham số khác có thể đưa đầu vào neuron sang phía dương.

9.4 Qua $z = Wx + b$

Mỗi thành phần là $z_i = \sum_j W_{ij} x_j + b_i$. Giữ các thành phần khác cố định, đạo hàm $\partial z_i / \partial W_{ij} = x_j$. Nhân với $\bar{z}_i$ cho $\partial L/\partial W_{ij} = \bar{z}_i x_j$. Gộp lại thành tích ngoài:

$$\nabla_W L = \bar{z} x^\top = \begin{pmatrix} 2 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 4 \\ 0 & 0 \end{pmatrix}, \qquad \nabla_b L = \bar{z} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}.$$

Kiểm một ô: W_{12} tác động tới z_1 qua $x_2 = 2$, còn $\partial L/\partial z_1 = 2$, nên $\partial L/\partial W_{12} = 2 \cdot 2 = 4$ (tự tính). Kích thước cũng khớp: tích vector 2×1 với hàng 1×2 cho ma trận 2×2, cùng kích thước W.

Ta còn có thể hỏi gradient theo đầu vào. Một đầu vào ảnh hưởng tới mọi neuron ẩn, do đó phải cộng ảnh hưởng qua các hàng:

$$\nabla_x L = W^\top \bar{z} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

Mỗi thành phần 2 là tổng đóng góp của hai neuron, trong đó neuron sau góp 0. $W^\top$ xuất hiện vì đang kéo độ nhạy từ hai đầu ra của lớp về hai đầu vào. Dù W của ví dụ tình cờ đối xứng, trong mạng nói chung không được bỏ dấu chuyển vị.

9.5 Một cập nhật và kiểm lại mất mát

Giả sử dùng gradient descent đồng thời cho tất cả tham số với hệ số bước $0{,}1$. Áp công thức tham số mới bằng tham số cũ trừ hệ số bước nhân gradient của nó, ta được

$$W_{\text{mới}} = \begin{pmatrix} 0{,}8 & 0{,}6 \\ 1 & -1 \end{pmatrix}, \quad b_{\text{mới}} = \begin{pmatrix} -0{,}2 \\ 0 \end{pmatrix}, \quad v_{\text{mới}} = \begin{pmatrix} 0{,}4 \\ 2 \end{pmatrix}, \quad c_{\text{mới}} = -0{,}2.$$

Chạy lại cùng mẫu với các tham số mới: $z_{\text{mới}} = (1{,}8, -1)^\top$, $a_{\text{mới}} = (1{,}8, 0)^\top$, $s_{\text{mới}} = 0{,}4 \cdot 1{,}8 - 0{,}2 = 0{,}52$, và mất mát mới bằng $\frac{1}{2}(0{,}52 - 1)^2 = 0{,}1152$ (tự tính). Mất mát đã giảm ở bước cụ thể này. Không được suy ra mọi hệ số bước đều giảm mất mát, hoặc rằng cùng một cập nhật sẽ giảm mất mát trên mọi mẫu khác.

Cập nhật là một thao tác sau khi đã tính đủ gradient ở tham số cũ. Nếu thay W ngay giữa lượt ngược rồi dùng W mới để tính $\nabla_x L$, ta không còn đang tính gradient của cùng lượt xuôi ban đầu.

10. Kiểm gradient bằng sai phân trung tâm

Để kiểm gradient của một tham số scalar θ tại giá trị hiện tại, tính

$$g_{\text{số}} = \frac{L(\theta + h) - L(\theta - h)}{2h}$$

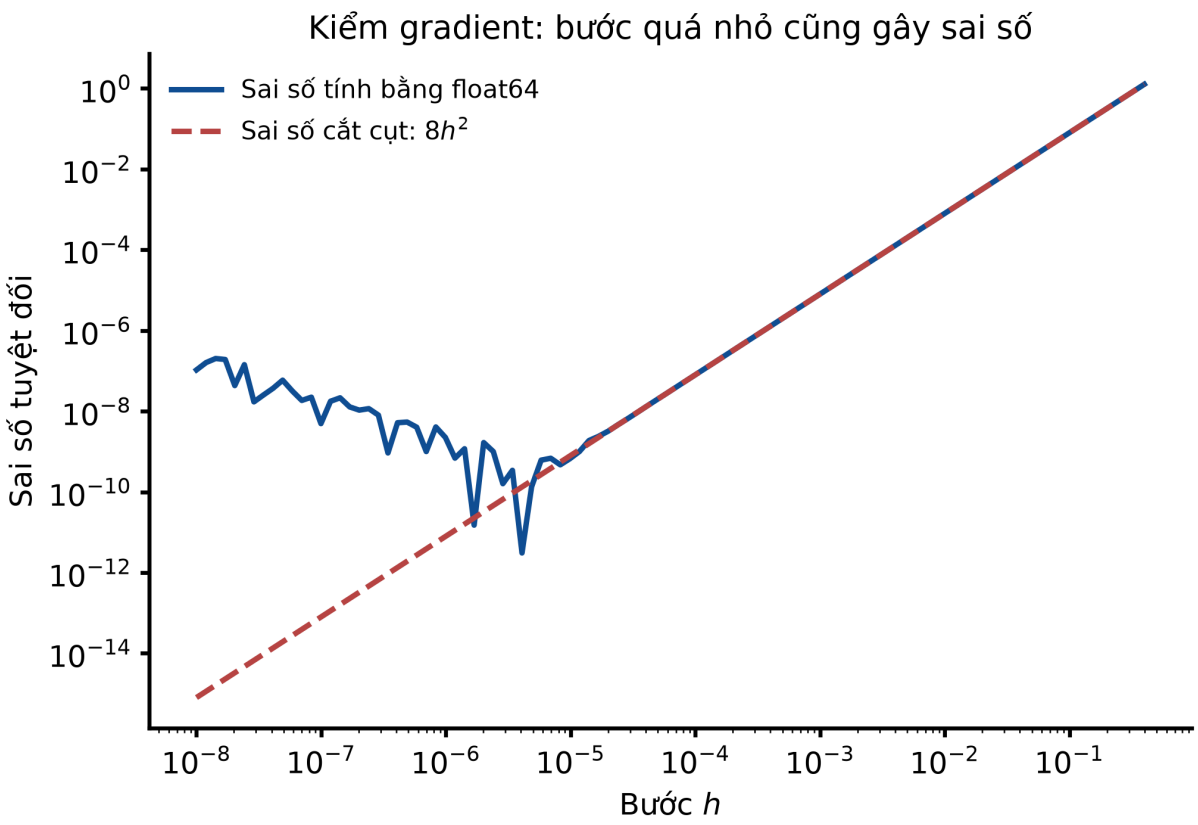
với mọi tham số khác, dữ liệu, nhãn và cách tính mất mát được giữ cố định. Sau đó so $g_{\text{số}}$ với đạo hàm giải tích hoặc backprop. Phải chạy lại lượt xuôi ở cả hai phía; chỉ thay một giá trị đã lưu sau lượt xuôi sẽ không kiểm đúng chương trình.

Trong ví dụ trên, xét riêng W_{11} tại giá trị ban đầu 1. Với $h = 0{,}01$, hai mất mát là $L(1{,}01) = 2{,}02005$ và $L(0{,}99) = 1{,}98005$, nên sai phân trung tâm bằng $(2{,}02005 - 1{,}98005)/0{,}02 = 2$, khớp $\partial L / \partial W_{11} = 2$ (tự tính). Hai phép thử đều giữ neuron đầu ở phía dương và neuron thứ hai ở phía âm. Đây là kiểm trên một tọa độ, chưa thay thế việc kiểm các tham số còn lại.

10.1 Tại sao không chọn bước cực nhỏ?

Hình sau dùng hàm đơn giản hơn $f(x) = x^4$ tại $x = 2$, có đạo hàm thật bằng 32. Trong số học chính xác, sai phân trung tâm bằng $32 + 8h^2$, nên sai số cắt cụt đúng bằng $8h^2$. Trên máy, việc trừ hai giá trị gần nhau gây sai số làm tròn và khuếch đại khi chia cho $2h$.

Cách đọc hình: cả hai trục đều theo thang log, trục ngang là bước h và trục dọc là sai số tuyệt đối. Đường đỏ nét đứt là sai số cắt cụt $8h^2$ trong số học chính xác; đường xanh là sai số thực đo bằng float64 (tự tính). Về phía phải, hai đường gần trùng vì sai số cắt cụt chi phối; về phía trái, làm tròn khiến đường xanh không tiếp tục bám đường đỏ. Hình minh họa xu hướng cho một hàm và kiểu số cụ thể, không ấn định một bước tốt nhất cho mọi bài toán.



Hình 25: Sai số kiểm gradient của hàm x mũ bốn khi thay đổi bước sai phân (minh hoạ, tự tính)

Chọn bước đủ nhỏ để xấp xỉ cục bộ có ích nhưng không quá nhỏ so với độ chính xác máy. So sánh cả sai số tuyệt đối và tương đối, nhất là khi gradient gần 0. Với ReLU, tránh chọn điểm hay bước khiến hai phép thử đi qua điểm gãy: sai phân khi đó có thể phản ánh cả hai nhánh, còn đạo hàm tại điểm gãy có thể không tồn tại.

11. Các biến thể và chi tiết dễ bỏ qua

11.1 Forward mode và reverse mode

Forward mode bắt đầu với một thay đổi đầu vào v, truyền nó cùng chiều tính xuôi và thu được Jv. Reverse mode bắt đầu với một trọng số đầu ra w, truyền nó ngược và thu được $J^\top w$. Hai cách áp cùng quy tắc chuỗi nhưng chọn thứ tự tích khác nhau.

Khi đầu ra là một mất mát scalar và có nhiều tham số, reverse mode đặc biệt hữu ích: bắt đầu bằng hạt giống 1 tại mất mát, một lượt ngược thu gradient cho tất cả tham số ảnh hưởng tới nó. Khi có ít đầu vào và nhiều đầu ra, forward mode có thể thuận lợi hơn cho một số yêu cầu. Chi phí và bộ nhớ thực tế còn phụ thuộc đồ thị, cài đặt, và các giá trị cần lưu; xem math 02 để đọc kỹ hơn.

11.2 Batch, tổng và trung bình

Nếu batch có B mẫu và mất mát được định nghĩa là trung bình $L_{\text{batch}} = \frac{1}{B}\sum_{i=1}^{B} L_i$, gradient cũng là trung bình của các gradient mẫu. Ví dụ hai gradient scalar lần lượt là 2 và 4, thì gradient của trung bình là 3, còn gradient của tổng là 6 (tự tính). Một hệ số $1/B$ thiếu hoặc thừa làm thay đổi độ lớn cập nhật, dù hướng có thể giữ nguyên.

Tính trung bình mất mát cho batch không có nghĩa luôn có thể tách mọi mô hình thành các lượt độc lập: một

số phép dùng thống kê của cả batch hoặc liên kết các mẫu. Khi đó phải vi phân đúng chương trình thực tế, kể cả các liên kết giữa mẫu.

11.3 Chia sẻ tham số

Một trọng số có thể được dùng ở nhiều vị trí hoặc nhiều bước thời gian. Với tham số chung, toàn bộ đường qua các lần dùng đều góp vào **cùng** gradient; không được ghi đè kết quả trước bằng kết quả sau. Nguyên tắc cộng này chính là panel dưới trong hình quy tắc chuỗi, nhưng giờ nút dùng chung là một tham số thay vì đầu vào.

11.4 Chỗ không trơn và gradient có giá trị rất nhỏ

ReLU không có đạo hàm cổ điển tại đầu vào 0, vì độ dốc một phía bên trái là 0 và bên phải là 1. Một cài đặt có thể quy ước một giá trị tại đó cho lượt ngược; cần biết quy ước khi so sánh kết quả. Ví dụ chính tránh điểm này, nên các đạo hàm đều là đạo hàm thông thường.

Khi ghép nhiều lớp, các phép nhân độ nhạy có thể làm tín hiệu rất nhỏ hoặc rất lớn. Đây không phải kết luận chỉ từ số lớp: dấu, hướng, ma trận và dữ liệu đều quan trọng. Ngoài độ lớn, cấu trúc của gradient cũng có thể thay đổi; math 18 nghiên cứu một hiện tượng cụ thể trong mạng sâu.

12. Nối sang AI

(a) Huấn luyện mạng. Backprop tính gradient của mất mát theo trọng số lớp ẩn lẫn lớp đầu ra. Optimizer nhận gradient để tạo cập nhật; backprop không bắt buộc dùng gradient descent thông thường và cũng không tự chọn learning rate.

(b) Các hàm mất mát khác. Khi thay mất mát bình phương bằng một hàm khác, ta thay đạo hàm cục bộ tại đầu ra và giữ cùng nguyên tắc nhân dọc đường, cộng tại nhánh. Phải kiểm quy ước lấy tổng hay trung bình và các phép biến đổi được gộp trong cài đặt.

(c) Độ nhạy đầu vào. Cùng một lượt ngược có thể trả gradient theo đầu vào nếu coi đầu vào là biến quan tâm. Nó mô tả thay đổi cục bộ của mất mát theo đầu vào, không tự động giải thích quan hệ nhân quả trong dữ liệu.

(d) Tự động tính đạo hàm. Thư viện theo dõi các phép tính đã thực thi rồi áp các quy tắc đạo hàm cục bộ. Đây là cách tính đạo hàm của chương trình với sai số số học máy, khác với lấy sai phân để xấp xỉ đạo hàm; ghi chú math 02 giải thích các chế độ và đánh đổi.

(e) Ma trận Jacobian theo tham số. Nếu cần độ nhạy của vector dự đoán theo tham số, ta xét Jacobian tương ứng chứ không chỉ gradient một mất mát scalar. Nó là một đối tượng liên quan khi tìm hiểu neural tangent kernel trong math 23; không cần lập toàn bộ ma trận này trong mọi bước huấn luyện thông thường.

13. Sai lầm hay gặp

- **Chỉ lấy đạo hàm hàm ngoài.** Với hàm hợp, phải tính đạo hàm hàm ngoài tại giá trị trung gian rồi nhân đạo hàm hàm trong.
- **Quên cộng các đường cùng ảnh hưởng đến một nút.** Một trọng số dùng hai lần nhận tổng hai đóng góp, không phải đóng góp cuối cùng ghi đè đóng góp trước.
- **Nhầm giá trị xuôi và tín hiệu ngược.** a là đầu ra ReLU, còn $\bar{a}$ là đạo hàm của mất mát theo a; chúng có thể rất khác nhau.
- **Đảo thứ tự Jacobian hoặc quên chuyển vị khi dùng gradient cột.** Viết kích thước từng ma trận trước khi nhân giúp phát hiện lỗi.
- **Đòi đạo hàm cổ điển của ReLU tại 0 hoặc kiểm sai phân xuyên qua điểm gãy.** Nêu rõ quy ước cài đặt và kiểm ở điểm khả vi.
- **Cập nhật tham số giữa lượt ngược hoặc quên chuẩn hóa batch.** Hãy tính gradient theo cùng trạng thái tham số của lượt xuôi, rồi áp đúng định nghĩa tổng hay trung bình của mất mát.

14. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Chuỗi scalar	$dy/dx = (dy/du)(du/dx)$	nhân độ nhạy trên một đường
Nhiều đường	$dL/dx = \sum_i (\partial L/\partial u_i)(du_i/dx)$	cộng mọi đóng góp qua trung gian
Jacobian hàm hợp	$J_{f\circ g} = J_f J_g$	hàng đầu ra, cột đầu vào
Lan truyền ngược	$\nabla_x L = J_g^\top \nabla_u L$	kéo gradient cột về đầu vào
Lớp tuyến tính	$z = Wx + b$	định nghĩa lớp trước kích hoạt
Gradient lớp	$\nabla_W L = \bar{z} x^\top$, $\nabla_b L = \bar{z}$	tích ngoài theo một mẫu
Gradient đầu vào	$\nabla_x L = W^\top \bar{z}$	cộng ảnh hưởng qua các đầu ra lớp
Kiểm bằng số	$[L(\theta + h) - L(\theta - h)]/(2h)$	xấp xỉ đạo hàm để đối chiếu

15. Bài tập tự tính

Năm bài từ dễ đến khó. Hãy thử viết các giá trị xuôi trước, sau đó lan truyền ngược; các kết quả số dưới đây là tự tính và đã kiểm bằng numpy.

Bài 1 (dễ). Cho $u = 3x + 1$, $y = u^2$. Tính y và dy/dx tại $x = 2$ bằng quy tắc chuỗi.

Đáp án. Giá trị trung gian $u = 7$, nên $y = 49$. Đạo hàm cục bộ $dy/du = 2u = 14$ và $du/dx = 3$, nên $dy/dx = 14 \cdot 3 = 42$ (tự tính). Thế trước rồi đạo hàm $(3x + 1)^2$ cũng cho cùng kết quả.

Bài 2 (vừa). Cho $u = x^2$, $v = x^3$, $L = u + v$. Tính giá trị và gradient tại $x = 2$; chỉ rõ mỗi đường đóng góp bao nhiêu.

Đáp án. Lượt xuôi cho $u = 4$, $v = 8$, $L = 12$. Hai đạo hàm cục bộ theo x lần lượt là $2x = 4$ và $3x^2 = 12$. Cả hai đi vào phép cộng với hệ số 1, nên $dL/dx = 4 + 12 = 16$ (tự tính).

Bài 3 (vừa). Cho $u = (2x_1, x_1 + x_2)^\top$ và $L = u_1 + 3u_2$. Tại $x = (1, 2)^\top$, tính giá trị và gradient theo x bằng tích Jacobian chuyển vị với vector.

Đáp án. $u = (2, 3)^\top$ và $L = 11$. Jacobian của u theo x là $J = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}$, còn $\nabla_u L = (1, 3)^\top$. Nhân $J^\top (1, 3)^\top$ cho $\nabla_x L = (5, 3)^\top$ (tự tính). Kiểm trực tiếp: $L = 5x_1 + 3x_2$.

Bài 4 (khó hơn). Với $z = wx + b$, $a = \mathrm{ReLU}(z)$, $L = \frac{1}{2}(a - 1)^2$, cho $x = 2$, $w = 3$, $b = -1$. Tính L và gradient theo w, b, x.

Đáp án. $z = 3 \cdot 2 - 1 = 5 > 0$ và $a = 5$, nên $L = 8$. Lan truyền ngược: $\bar{a} = a - 1 = 4$, đạo hàm ReLU bằng 1, nên $\bar{z} = 4$. Từ $z = wx + b$, ta có $\partial L/\partial w = 4 \cdot 2 = 8$, $\partial L/\partial b = 4$, $\partial L/\partial x = 4 \cdot 3 = 12$ (tự tính).

Bài 5 (tổng hợp). Một trọng số w được dùng hai lần: $a = wx_1$, $b = wx_2$, $s = a + b$, $L = \frac{1}{2}(s - y)^2$. Cho $w = 2$, $(x_1, x_2) = (1, 2)$ và $y = 3$. Tính L, gradient theo w và gradient theo từng đầu vào.

Đáp án. Lượt xuôi cho $a = 2$, $b = 4$, $s = 6$, nên $L = \frac{1}{2}(6 - 3)^2 = 4{,}5$. Tín hiệu tại s là $s - y = 3$, được gửi nguyên vẹn qua phép cộng tới cả hai nhánh. Nhánh đầu góp $3x_1 = 3$ vào gradient của w, nhánh sau góp $3x_2 = 6$, nên $\partial L/\partial w = 3 + 6 = 9$. Cả hai đầu vào đều có đạo hàm $3w = 6$ (tự tính). Viết thẳng $s = w(x_1 + x_2)$ cho cùng kết quả.

Tài liệu tham khảo nền: Parr và Howard, The Matrix Calculus You Need For Deep Learning (math 01); Baydin et al., Automatic Differentiation in Machine Learning (math 02); Goodfellow, Bengio và Courville, Deep Learning.

Nền tảng 08

Xác suất cơ bản: kỳ vọng và phương sai tóm tắt điều gì có thể xảy ra và mức độ dao động

Loại: chương nền tảng, không phải ghi chú paper

Đọc trước: phân số, tổng hữu hạn, cách đọc bảng; không cần học trước Bayes hay giải tích

Dùng ở: math 06, math 07, math 08, math 09, math 10, math 11, math 22

> Chương này dạy xác suất từ đầu, không phải bản dịch paper nào. Những con số ghi "tự tính" và các đáp án bài tập đã được kiểm bằng numpy.

1. Vì sao cần xác suất

Xác suất là cách gán mức độ có thể xảy ra cho những kết quả chưa biết. Trong AI, nó giúp mô tả dữ liệu nhiễu, dự đoán chưa chắc chắn, biến ẩn, và mục tiêu huấn luyện được tính bằng trung bình trên các kết quả có thể.

Một bộ phân loại có thể cho biết "mèo" với xác suất 0,8 thay vì chỉ trả về nhãn cứng. Một mô hình sinh có thể lấy một biến ngẫu nhiên rồi biến nó thành ảnh. Một thuật toán tối ưu ngẫu nhiên có thể lấy một minibatch khác nhau ở mỗi bước. Những việc này trông khác nhau, nhưng đều cần trả lời các câu hỏi cơ bản: kết quả nào có thể xảy ra, chúng có xác suất bao nhiêu, trung bình sẽ ra sao, và kết quả thường dao động rộng đến mức nào?

Xác suất không hứa rằng một kết quả riêng lẻ sẽ gần trung bình. Nó mô tả một quy luật cho cả các lần lặp. Nếu một sự kiện có xác suất nhỏ, nó vẫn có thể xảy ra ngay lần tới; ý nghĩa của con số chỉ lộ rõ khi lặp thí nghiệm nhiều lần.

2. Nhắc nền cần thiết

2.1 Kết quả, không gian mẫu và biến cố

Một **thí nghiệm ngẫu nhiên** là phép thử mà ta chưa biết trước kết quả, dù điều kiện thử đã được xác định. **Không gian mẫu** Ω là tập mọi kết quả có thể. **Biến cố** là một tập con của Ω, chẳng hạn biến cố "ra mặt ngửa".

Với một lần tung đồng xu, $\Omega = \{N, S\}$, trong đó N là ngửa và S là sấp. Nếu đồng xu công bằng, hai kết quả có cùng khả năng. Khi đó xác suất ra ngửa là một nửa. Với hai lần tung độc lập một đồng xu công bằng, các kết quả có thứ tự là NN, NS, SN, SS và đồng khả năng. Biến cố "có đúng một mặt ngửa" gồm NS và SN, nên xác suất là $2/4 = 1/2$ (tự tính). Thứ tự có ý nghĩa vì lần thứ nhất và lần thứ hai là hai vị trí khác nhau.

Nếu mọi kết quả trong một không gian hữu hạn là như nhau, xác suất biến cố A là số kết quả trong A chia cho tổng số kết quả. Cách đếm này không dùng được nếu các kết quả không đồng đều, hoặc nếu không gian có vô hạn phần tử. Khi ấy ta cần mô tả phân phối xác suất trực tiếp.

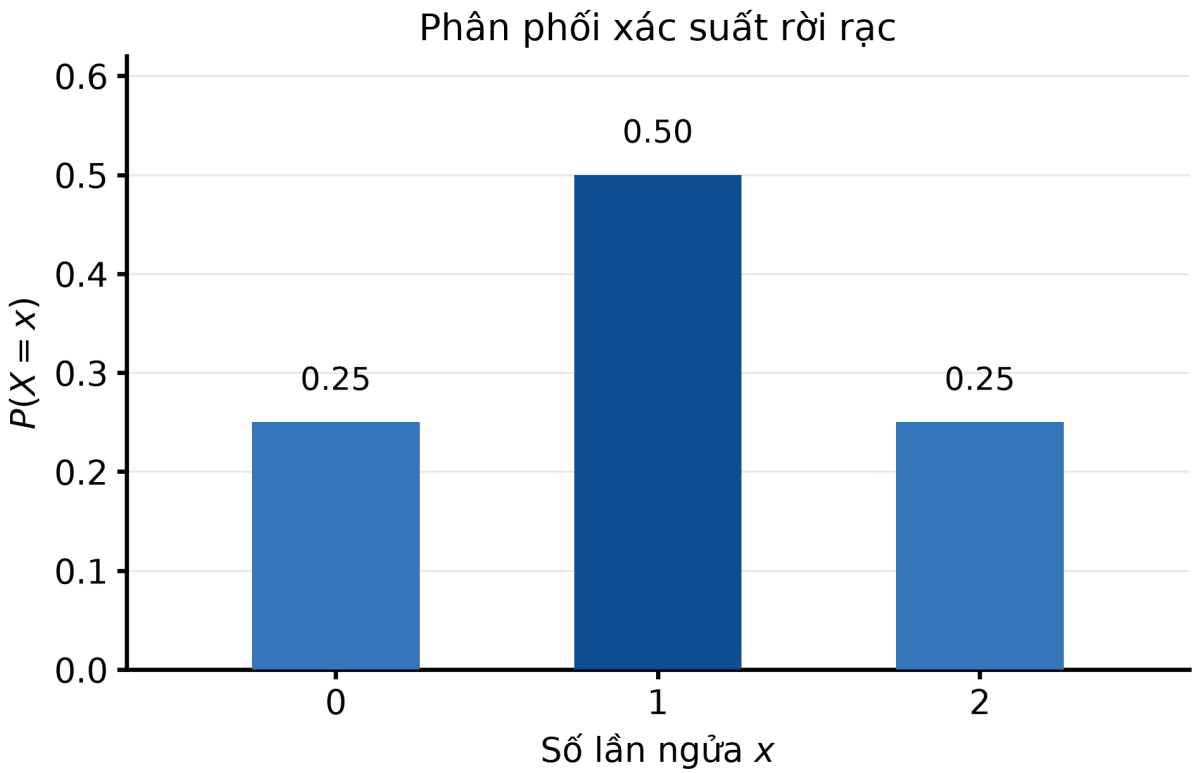
2.2 Biến ngẫu nhiên đổi kết quả thành con số

Biến ngẫu nhiên X là một quy tắc gán một con số cho mỗi kết quả của thí nghiệm. Với hai lần tung đồng xu, có thể đặt $X =$ số lần ra ngửa. Khi kết quả là NS, giá trị X bằng một. Viết X là biến ngẫu nhiên, còn $X = 1$ là biến cố mà biến đó nhận giá trị một.

Sự phân biệt này hữu ích khi đọc công thức. $P(X = 1)$ là xác suất của biến cố "X bằng một", còn X tự nó chưa phải là một số cụ thể trước khi thí nghiệm diễn ra. Sau khi quan sát, ta nhận được một giá trị như $x = 1$.

3. Trực giác bằng hình: một phân phối là bản đồ khả năng

Với hai lần tung đồng xu công bằng và độc lập, biến X là số lần ngửa chỉ nhận các giá trị 0, 1, 2. Hai cách đạt một lần ngửa là NS và SN, nên cột ở giữa cao hơn hai cột ngoài. Các cột không nói kết quả nào “đã xảy ra”; chúng mô tả điều gì dễ gặp trước khi tung.



Hình 26: Phân phối xác suất của số lần ngửa trong hai lần tung đồng xu công bằng (minh hoạ, tự tính)

Cách đọc hình 1: trục ngang liệt kê những số mặt ngửa có thể có, trục dọc là xác suất của từng giá trị. Ba cột có tổng chiều cao bằng một, vì một trong ba trường hợp chắc chắn xảy ra. Cột ở giữa cao gấp đôi mỗi cột ngoài vì có hai chuỗi kết quả tạo ra một mặt ngửa, trong khi mỗi đầu chỉ có một chuỗi (tự tính).

Biểu đồ cột phù hợp với biến rời rạc, tức biến chỉ nhận các giá trị tách biệt như số lần thành công. Với biến liên tục như chiều cao, thời gian hoặc một tọa độ trong không gian, ta thường vẽ một đường mật độ. Khi đó xác suất nằm ở **diện tích dưới đường** trên một khoảng, chứ không phải chiều cao tại một điểm. Mục 4 sẽ viết hai cách mô tả này bằng công thức.

4. Phân phối và các quy tắc xác suất

4.1 Xác suất rời rạc

Nếu X nhận các giá trị $x_1, x_2, \ldots$, ta viết

$$p_X(x_i) = P(X = x_i), \qquad p_X(x_i) \geq 0, \qquad \sum_i p_X(x_i) = 1.$$

Hàm p_X được gọi là **hàm khối xác suất** (probability mass function, PMF). Xác suất một biến cố gồm nhiều giá trị là tổng khối xác suất của các giá trị ấy. Với biểu đồ hình 1, $P(X \leq 1) = p_X(0) + p_X(1) = 3/4$ (tự tính).

Tổng bằng một là điều kiện kiểm tra nhanh. Nếu các xác suất là $0{,}2$, $0{,}3$ và $0{,}4$, chúng chưa thể là một PMF hoàn chỉnh vì tổng chỉ bằng $0{,}9$ (tự tính); có thể còn thiếu một kết quả hoặc có lỗi chuẩn hoá.

4.2 Xác suất liên tục

Với biến liên tục, ta mô tả bằng **mật độ xác suất** $f_X(x)$. Mật độ không âm và tổng diện tích dưới đường bằng một:

$$f_X(x) \geq 0, \qquad \int_{-\infty}^{\infty} f_X(x)\,dx = 1.$$

Xác suất một khoảng được tính bằng tích phân:

$$P(a \leq X \leq b) = \int_a^b f_X(x)\,dx.$$

Với phân phối liên tục, xác suất đúng một điểm bằng không, dù mật độ tại điểm đó có thể dương. Điều này không có nghĩa là điểm ấy không thể được quan sát. Xác suất của một khoảng được tính bằng diện tích dưới mật độ; một khoảng cụ thể có thể có xác suất bằng không nếu mật độ không tạo diện tích trong khoảng đó. Đừng diễn giải $f_X(x)$ như xác suất của đúng giá trị x.

4.3 Hàm phân phối tích luỹ

Một cách mô tả dùng được cho cả biến rời rạc và liên tục là **hàm phân phối tích luỹ**:

$$F_X(x) = P(X \leq x).$$

Khi x tăng, $F_X(x)$ không giảm, bắt đầu từ gần không và tiến tới một. Với biến rời rạc, nó nhảy lên tại các giá trị mà X có thể nhận. Với biến liên tục đủ trơn, độ dốc của nó là mật độ. CDF trả lời trực tiếp câu hỏi “giá trị không vượt quá ngưỡng này có xác suất bao nhiêu?”, hữu ích khi cần tính phân vị hoặc đặt ngưỡng quyết định.

4.4 Ba tiên đề cốt lõi

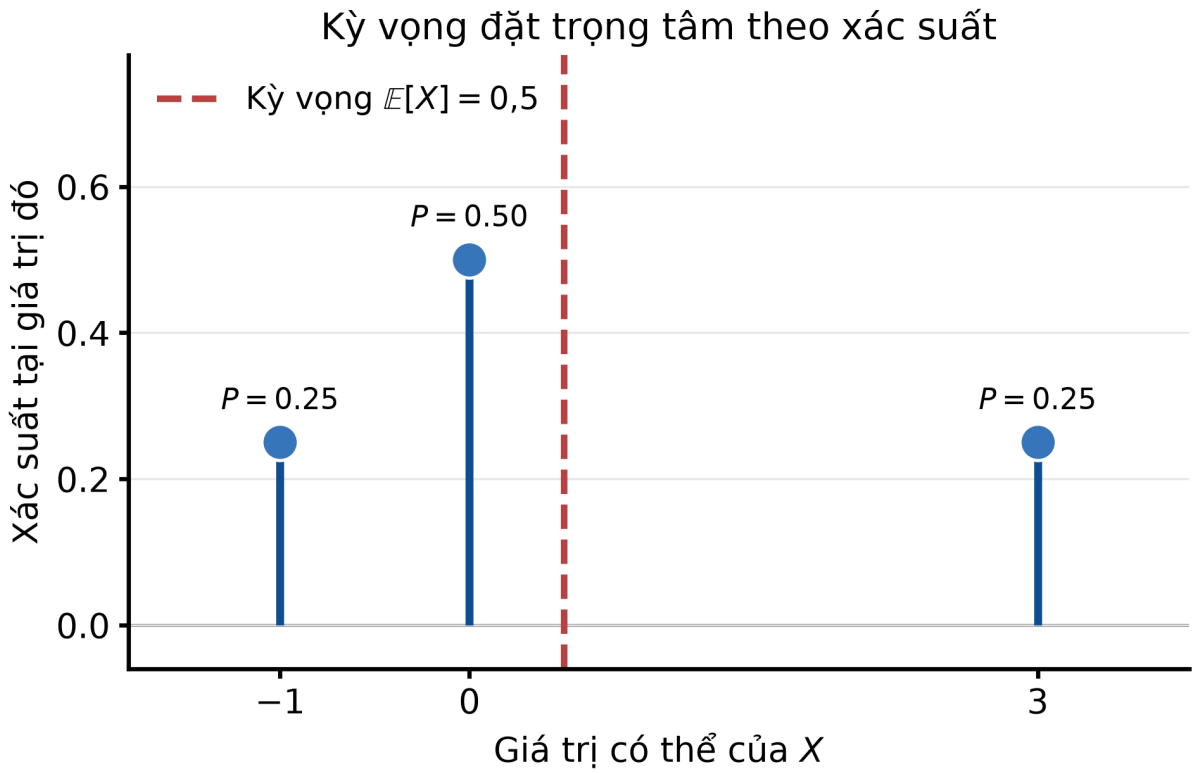
Ba tiên đề nói rằng xác suất không âm, xác suất của toàn không gian mẫu bằng một, và xác suất có tính cộng đếm được trên một dãy biến cố đôi một không giao nhau. Với dãy $A_1, A_2, \ldots$ đôi một không giao nhau, tiên đề cộng là

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

Quy tắc cộng cho hai biến cố không thể cùng xảy ra là trường hợp riêng của tiên đề này, khi dãy chỉ có hai biến cố: $P(A \cup B) = P(A) + P(B)$. Từ đây suy ra xác suất biến cố đối là $P(A^c) = 1 - P(A)$. Ví dụ nếu xác suất một linh kiện lỗi là $0{,}02$, xác suất nó không lỗi là $0{,}98$ (tự tính). Phần bù thường giúp tính nhanh: xác suất có ít nhất một lần xảy ra bằng một trừ xác suất không lần nào xảy ra.

5. Trực giác về kỳ vọng: điểm cân bằng có trọng số

Kỳ vọng là trung bình theo phân phối. Nếu lặp các phép thử độc lập cùng phân phối với kỳ vọng hữu hạn, trung bình mẫu sẽ tiến gần kỳ vọng này khi số lần lặp tăng. Hãy hình dung mỗi giá trị nằm tại một vị trí trên trục số và xác suất của nó là trọng lượng. Kỳ vọng là vị trí trung bình có tính đến các trọng lượng đó.



Hình 27: Kỳ vọng là trung bình có trọng số của các kết quả và xác suất tương ứng (minh hoạ, tự tính)

Cách đọc hình 2: mỗi cột mảnh đặt tại một giá trị có thể, còn độ cao thể hiện xác suất của giá trị ấy. Đường đứt nét đỏ ở $0{,}5$ là trung bình có trọng số, không nhất thiết trùng một giá trị mà biến có thể nhận. Giá trị 3 nằm xa hơn nhưng chỉ xuất hiện với xác suất một phần tư, vì vậy nó kéo tâm sang phải vừa đủ để cân với khối xác suất ở các vị trí còn lại (tự tính).

Kỳ vọng không phải lời tiên đoán cho một lần thử. Nếu biến chỉ nhận 0 hoặc 2 với xác suất bằng nhau, kỳ vọng bằng 1 dù kết quả 1 không bao giờ xảy ra (tự tính). Nó cũng không nhất thiết bằng trung vị hay giá trị phổ biến nhất. Cả ba đại lượng trả lời câu hỏi khác nhau.

6. Định nghĩa kỳ vọng và tính tay từng bước

6.1 Kỳ vọng của biến rời rạc

Nếu X nhận giá trị x_i với xác suất p_i, kỳ vọng là

$$\mathbb{E}[X] = \sum_{i} x_i p_i, \qquad \text{khi } \sum_{i} |x_i| p_i < \infty.$$

Ta nhân từng giá trị với trọng lượng xác suất tương ứng rồi cộng lại. Đây là trung bình có trọng số. Điều kiện hội tụ tuyệt đối bảo đảm kỳ vọng là một số hữu hạn, không phụ thuộc cách sắp thứ tự các số hạng. Với biến liên tục, tổng được thay bằng tích phân: $\mathbb{E}[X] = \int x f_X(x)\,dx$, nếu $\int |x| f_X(x)\,dx < \infty$.

6.2 Ví dụ chậm: tính kỳ vọng của số mặt ngửa

Tiếp tục thí nghiệm tung hai đồng xu. Biến X là số mặt ngửa, với bảng phân phối ở mục 3. Tính từng số hạng:

$$\mathbb{E}[X] = 0 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} = 1.$$

Vậy số lần ngửa trung bình qua rất nhiều cặp tung là một (tự tính). Một cặp cụ thể có thể có 0, 1 hoặc 2 lần ngửa; “trung bình bằng một” không có nghĩa mọi cặp đều có đúng một lần ngửa.

Kỳ vọng của một hàm cũng được tính bằng cách áp hàm lên từng giá trị trước khi lấy trung bình. Chẳng hạn

$$\mathbb{E}[X^2] = 0^2 \cdot \frac{1}{4} + 1^2 \cdot \frac{1}{2} + 2^2 \cdot \frac{1}{4} = \frac{3}{2}$$

(tự tính). Nói chung, $\mathbb{E}[X^2]$ không bằng $\mathbb{E}[X]^2$. Ở đây, hai vế lần lượt là $3/2$ và 1 (tự tính). Sai khác đó chính là một phần để đo mức độ dao động.

6.3 Kỳ vọng có thể nằm ngoài các giá trị thường gặp

Cho Y bằng 0 với xác suất $0{,}99$ và bằng 100 với xác suất $0{,}01$. Khi đó $\mathbb{E}[Y] = 0 \cdot 0{,}99 + 100 \cdot 0{,}01 = 1$ (tự tính), nhưng giá trị phổ biến nhất là 0 và biến không bao giờ nhận đúng 1. Ví dụ này giúp đọc đúng một kỳ vọng trong bài toán mất mát hoặc phần thưởng: nó là mức trung bình theo phân phối, không phải mô tả đầy đủ của một lần chạy.

7. Phương sai: độ lớn của dao động quanh kỳ vọng

7.1 Định nghĩa và ý nghĩa

Phương sai lấy độ lệch khỏi kỳ vọng, bình phương nó để độ lệch âm và dương không triệt tiêu nhau, rồi lấy trung bình:

$$\operatorname{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big].$$

Độ lệch chuẩn $\sigma_X = \sqrt{\operatorname{Var}(X)}$ có cùng đơn vị với X, nên thường dễ diễn giải hơn phương sai. Nếu X đo bằng giây, phương sai đo bằng giây bình phương, còn độ lệch chuẩn lại đo bằng giây.

7.2 Ví dụ chậm: phương sai của hai lần tung

Với X ở mục 6, kỳ vọng bằng một. Ta tính độ lệch bình phương ở từng kết quả: tại 0 là $(0-1)^2 = 1$, tại 1 là 0, tại 2 là 1. Gắn xác suất vào từng số hạng:

$$\operatorname{Var}(X) = 1 \cdot \frac{1}{4} + 0 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} = \frac{1}{2}.$$

Phương sai bằng một nửa (tự tính), độ lệch chuẩn bằng $1/\sqrt{2} \approx 0{,}707$ mặt ngửa (tự tính). Độ lệch chuẩn không có nghĩa “mọi lần cách kỳ vọng đúng 0,707”; nó là một thước đo độ rộng tổng hợp.

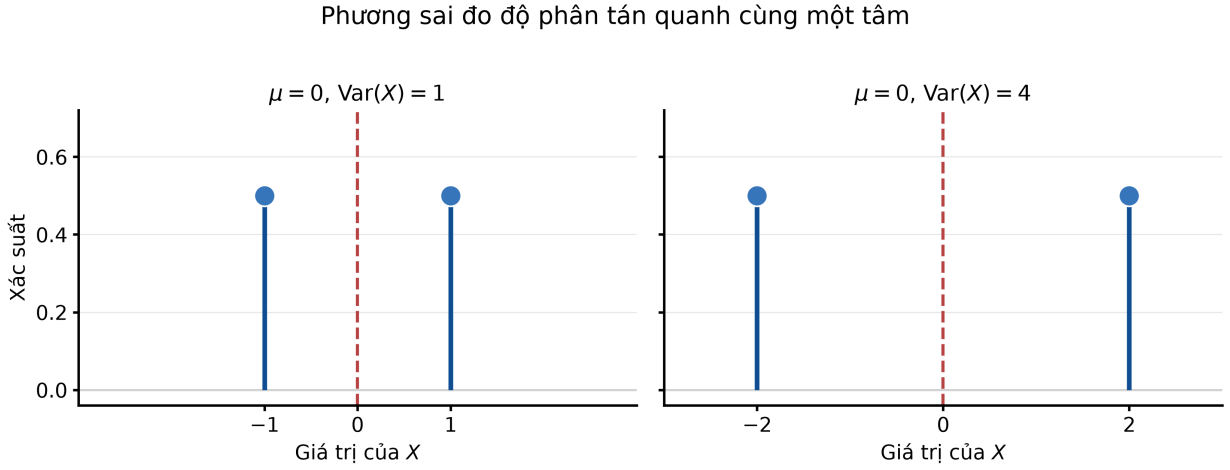
Ta có thể kiểm kết quả bằng công thức tương đương:

$$\operatorname{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

Thay hai số đã tính ở mục 6: $3/2 - 1^2 = 1/2$ (tự tính). Cách này thường ngắn hơn, nhưng cách lấy trung bình bình phương độ lệch cho trực giác rõ hơn và có thể ổn định số tốt hơn trong một số phép tính.

7.3 Cùng tâm, độ phân tán khác nhau

Hai biến có thể cùng kỳ vọng nhưng khác phương sai. Chẳng hạn, phân phối đều trên hai giá trị -1 và 1 có kỳ vọng 0 và phương sai 1 (tự tính). Phân phối đều trên -2 và 2 cũng có kỳ vọng 0, nhưng phương sai 4 (tự tính). Tâm không thay đổi, nhưng các khả năng của phân phối thứ hai xa tâm hơn.



Hình 28: Hai phân phối có cùng kỳ vọng nhưng khác phương sai (minh hoạ, tự tính)

Cách đọc hình 3: đường đỏ đánh dấu cùng tâm 0 ở cả hai panel. Các điểm xanh ở panel phải xa tâm hơn điểm xanh bên trái, nên độ lệch bình phương trung bình lớn hơn. Độ cao của hai điểm bằng nhau vì hai kết quả trong mỗi phân phối có xác suất bằng nhau; diện tích giữa các panel không phải xác suất chung của cả hai phân phối.

8. Quy tắc tính nhanh và liên hệ giữa biến

8.1 Kỳ vọng tuyến tính

Với các hằng số a, b, ta có

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b.$$

Khi các kỳ vọng đều hữu hạn, $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ ngay cả khi X và Y phụ thuộc nhau. Tính tuyến tính của kỳ vọng là công cụ mạnh khi tính số lượt thành công: viết số lượt thành tổng các biến chỉ báo, rồi cộng xác suất từng chỉ báo. Không cần giả sử các lượt độc lập để tìm kỳ vọng tổng.

8.2 Phương sai sau khi dịch hoặc đổi thang

Phép cộng một hằng số chỉ dịch chuyển tâm, không đổi độ rộng. Nhân với a làm mọi độ lệch dài gấp $|a|$, vì vậy phương sai thay đổi theo hệ số bình phương a^2:

$$\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X).$$

Nếu X tính bằng mét và đổi sang centimet, phương sai tăng theo bình phương hệ số đổi đơn vị, nhưng độ lệch chuẩn chỉ tăng theo đúng hệ số đó. Kiểm tra đơn vị giúp phát hiện việc lẫn phương sai với độ lệch chuẩn.

8.3 Hai biến, hiệp phương sai và độc lập

Độ dao động của tổng còn phụ thuộc hai biến cùng tăng giảm ra sao. **Hiệp phương sai** đo xu hướng đó:

$$\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] .$$

Khi hiệp phương sai dương, các giá trị lớn của một biến có xu hướng đi cùng giá trị lớn của biến kia. Khi âm, chúng có xu hướng đi ngược. Với hai biến có phương sai hữu hạn,

$$\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y).$$

Nếu X và Y độc lập thì hiệp phương sai bằng 0, nên phương sai của tổng là tổng phương sai. Chiều ngược lại không luôn đúng: không tương quan không đồng nghĩa độc lập trong mọi phân phối. Với Gaussian nhiều chiều, điều kiện này mạnh hơn; xem chương 09.

8.4 Có điều kiện và độc lập

Xác suất có điều kiện $P(A \mid B)$ hỏi xác suất của A khi biết B đã xảy ra. Nếu $P(B) > 0$, định nghĩa là

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Hai biến cố độc lập khi biết biến cố này không làm thay đổi xác suất của biến cố kia: $P(A \mid B) = P(A)$. Một kiểm tra tương đương là $P(A \cap B) = P(A)P(B)$.

Ví dụ: một túi có hai thẻ đỏ và hai thẻ xanh. Rút một thẻ rồi không bỏ lại. Xác suất lần hai đỏ trước khi biết lần đầu là $1/2$, nhưng nếu lần đầu đã đỏ thì chỉ còn một thẻ đỏ trong ba thẻ, nên xác suất có điều kiện là $1/3$ (tự tính). Hai lần rút không độc lập. Nếu bỏ thẻ về túi trước lần rút sau, xác suất lần hai vẫn là $1/2$ dù lần đầu ra gì; khi đó hai lần rút độc lập theo mô hình này (tự tính).

Bayes đổi hướng điều kiện, từ $P(B \mid A)$ sang $P(A \mid B)$. Đây là chủ đề của chương nền 10 về Bayes, MLE và MAP; ở đây chỉ cần nhớ rằng xác suất có điều kiện luôn phụ thuộc vào điều kiện đang được cho.

9. Lấy mẫu, trung bình mẫu và giới hạn

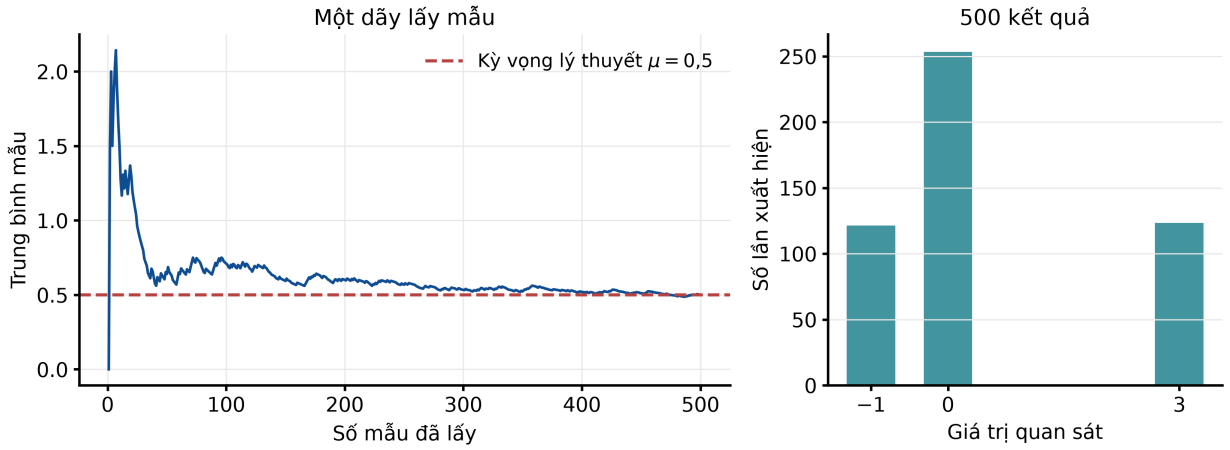
Khi không tính được kỳ vọng bằng công thức, ta có thể lấy mẫu độc lập $X_1, \dots, X_n$ và xấp xỉ bằng trung bình:

$$\widehat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

Nếu các mẫu độc lập cùng phân phối và có kỳ vọng hữu hạn, định luật số lớn nói rằng trung bình mẫu sẽ tiến gần kỳ vọng thật khi số mẫu tăng. Đây là xu hướng dài hạn, không phải lời bảo đảm rằng mỗi lần tăng thêm một mẫu thì đường trung bình phải đi gần hơn.

Cách đọc hình 4: đường xanh là trung bình tích luỹ của một dãy lấy mẫu giả lập, còn đường đỏ là kỳ vọng đúng của phân phối dùng để lấy mẫu. Đầu dãy dao động rõ vì một vài kết quả có ảnh hưởng lớn; về sau từng kết quả mới chiếm phần nhỏ hơn trong trung bình. Panel phải cho thấy tần số, tức số lần mỗi kết quả xuất hiện trong dãy 500 mẫu; đây không phải tần suất đã chia cho tổng số mẫu hay một lần tung riêng lẻ.

Nếu phương sai hữu hạn, phương sai của trung bình mẫu bằng $\operatorname{Var}(X)/n$ cho các mẫu độc lập cùng phân phối. Vì thế độ lệch chuẩn của trung bình mẫu giảm theo $1/\sqrt{n}$. Để giảm độ lệch chuẩn một nửa, cần tăng cỡ mẫu lên gấp bốn (tự tính). Quy luật này giải thích vì sao ước lượng Monte Carlo có thể cải thiện chậm khi số mẫu tăng.



Hình 29: Một trung bình mẫu dao động quanh kỳ vọng lý thuyết và ổn định dần khi lấy thêm mẫu (minh hoạ, tự tính)

Định lý giới hạn trung tâm cho biết trong nhiều điều kiện thông dụng, tổng hoặc trung bình của nhiều biến độc lập có phân phối gần chuẩn sau khi chuẩn hoá, ngay cả khi từng biến ban đầu không chuẩn. Đây là một định lý tiệm cận, không nói mọi tổng hữu hạn đều chính xác Gaussian. Chương 09 dùng phân phối chuẩn làm việc này cụ thể hơn.

10. Một vài khái niệm cần phân biệt

Xác suất lý thuyết đến từ mô hình ta giả định. **Tần suất thực nghiệm** là tỷ lệ quan sát được trong một mẫu hữu hạn. Tần suất có thể lệch khỏi xác suất lý thuyết, nhất là khi số lần thử ít; độ lệch không tự nó chứng minh mô hình sai.

Kỳ vọng là trung bình có trọng số của toàn phân phối. **Trung bình mẫu** là con số tính từ các quan sát đã có. Một trung bình mẫu có thể lệch kỳ vọng vì lấy mẫu ngẫu nhiên. **Phương sai của phân phối** mô tả độ rộng thực của các giá trị; **phương sai của ước lượng** mô tả độ dao động của một đại lượng được tính từ mẫu, như trung bình mẫu. Hai thứ này liên quan nhưng không đồng nhất.

Đối với dữ liệu rời rạc, có thể dùng biến chỉ báo I_A, bằng một nếu biến cố A xảy ra và bằng không nếu không. Khi ấy $\mathbb{E}[I_A] = P(A)$ (tự tính từ hai trường hợp). Tổng các chỉ báo đếm số lần biến cố xảy ra. Ý tưởng này xuất hiện trong phân tích số lượt, xác suất lỗi và một số hàm mất mát phân loại.

11. Nối sang AI

Hàm mất mát kỳ vọng. Một mô hình thường được đánh giá bằng kỳ vọng của mất mát trên phân phối dữ liệu hoặc trên phân phối dự đoán. Tập dữ liệu hữu hạn thay kỳ vọng đó bằng trung bình mẫu. Hai đại lượng gần nhau khi mẫu đại diện đủ tốt, nhưng tập huấn luyện không bảo đảm bao phủ mọi trường hợp có thể.

Lấy mẫu minibatch. Mỗi minibatch cung cấp một ước lượng ngẫu nhiên cho gradient hoặc hàm mất mát toàn tập. Kỳ vọng và phương sai giúp tách hai câu hỏi: ước lượng có đúng trung bình không, và nó nhiễu đến mức nào. Nhiễu không tự động có lợi; tác động phụ thuộc cách lấy mẫu, cỡ lô và lịch học.

Mô hình xác suất và khả năng xảy ra. Thay vì chỉ dự đoán một điểm, mô hình có thể gán xác suất cho nhiều kết quả. Với biến rời rạc, tối đa hoá likelihood tăng xác suất mô hình gán cho kết quả đã quan sát; với biến liên tục, likelihood tại một quan sát được tính từ mật độ ở điểm đó, còn xác suất của một vùng được tính trên một khoảng. Chương nền 10 về Bayes sẽ phân tích kỹ hơn likelihood, prior và posterior.

Biến ẩn và suy luận biến phân. VAE dùng kỳ vọng theo phân phối biến ẩn trong mục tiêu học. Khi không

thể tính tích phân chính xác, ta lấy mẫu để ước lượng. Nền tảng này dẫn trực tiếp đến ghi chú math 08 và math 09.

Độ không chắc chắn trong dự đoán. Một xác suất dự đoán hữu ích cần được hiểu như mức độ tin cậy của mô hình, nhưng con số đó phụ thuộc giả định và dữ liệu huấn luyện. Kỳ vọng và phương sai tóm tắt một số khía cạnh của phân phối dự đoán; chúng không thay thế kiểm tra hiệu chuẩn hay đánh giá trên dữ liệu phù hợp.

Hình học và lấy mẫu. Hiệp phương sai mô tả cách các tọa độ biến động cùng nhau. Nó xuất hiện trong Gaussian, ma trận Fisher, thuật toán HMC và mô hình quá trình Gaussian. Các ví dụ trong math 06, math 07, math 10 và math 22 dùng nhiều cấp độ khác nhau của cùng bộ công cụ.

12. Sai lầm hay gặp

- **Đọc xác suất như lời hứa cho lần kế tiếp.** Xác suất 0,1 không loại trừ việc sự kiện xảy ra ngay lần đầu; nó mô tả tần suất dài hạn nếu mô hình đúng và phép thử lặp lại.
- **Lẫn biến ngẫu nhiên với giá trị đã quan sát.** X là quy tắc hoặc đại lượng trước khi biết kết quả; x là một giá trị cụ thể sau khi quan sát.
- **Coi mật độ tại một điểm là xác suất điểm đó.** Với biến liên tục, xác suất nằm trên khoảng được tính bằng diện tích dưới mật độ.
- **Cho rằng kỳ vọng là kết quả phổ biến nhất.** Kỳ vọng có thể không phải giá trị nào biến nhận được, như ví dụ ở mục 6.3.
- **Nhầm phương sai với độ lệch chuẩn.** Phương sai bình phương độ lệch, còn độ lệch chuẩn lấy căn và trở lại đơn vị ban đầu.
- **Cho rằng không tương quan đồng nghĩa độc lập.** Độc lập kéo theo hiệp phương sai bằng không khi các moment tồn tại, nhưng chiều ngược lại không đúng trong mọi phân phối.

13. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Tổng xác suất rời rạc	$\sum_i p_X(x_i) = 1$	Tất cả khả năng phủ hết kết quả
Xác suất liên tục	$P(a \le X \le b) = \int_a^b f_X(x)\,dx$	Diện tích dưới mật độ
CDF	$F_X(x) = P(X \le x)$	Xác suất tích luỹ tới ngưỡng
Kỳ vọng rời rạc	$\mathbb{E}[X] = \sum_i x_i p_X(x_i)$	Trung bình có trọng số
Phương sai	$\operatorname{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$	Dao động quanh kỳ vọng
Công thức phương sai	$\operatorname{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$	Cách tính tương đương
Đổi thang	$\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X)$	Dịch không đổi độ rộng
Hiệp phương sai	$\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$	Cách hai biến cùng dao động
Xác suất có điều kiện	$P(A \mid B) = P(A \cap B) / P(B)$	Xác suất khi đã biết B
Trung bình mẫu	$\widehat{\mu}_n = \frac{1}{n}\sum_i X_i$	Ước lượng kỳ vọng từ mẫu

14. Bài tập tự tính

Hãy tự giải trước rồi mới đọc lời giải ngay dưới mỗi bài. Các kết quả số trong đáp án đã được kiểm lại bằng numpy.

Bài 1 (dễ). Tung độc lập một đồng xu công bằng hai lần. Gọi X là số lần ngửa. Hãy liệt kê giá trị có thể của X và xác suất từng giá trị.

Đáp án. Bốn chuỗi có thứ tự là NN, NS, SN, SS, cùng xác suất. Không lần nào ngửa chỉ xuất hiện ở SS, một lần ngửa ở NS hoặc SN, hai lần ngửa ở NN. Do đó $P(X = 0) = 1/4$, $P(X = 1) = 1/2$,

$P(X = 2) = 1/4$ (tự tính). Tổng bằng một.

Bài 2 (dễ vừa). Với phân phối của bài 1, tính $\mathbb{E}[X]$ và $\mathrm{Var}(X)$.

Đáp án. $\mathbb{E}[X] = 0 \cdot 1/4 + 1 \cdot 1/2 + 2 \cdot 1/4 = 1$. Ta có $\mathbb{E}[X^2] = 0 + 1/2 + 4 \cdot 1/4 = 3/2$, nên $\mathrm{Var}(X) = 3/2 - 1^2 = 1/2$ (tự tính). Đây khớp phép tính độ lệch bình phương ở mục 7.2.

Bài 3 (vừa). Tung độc lập một đồng xu công bằng 10 lần và gọi S là số lần ngửa. Không cần liệt kê mọi chuỗi, hãy tìm kỳ vọng và phương sai của S.

Đáp án. Viết $S = I_1 + \cdots + I_{10}$, trong đó I_j là chỉ báo lần thứ j ra ngửa. Mỗi chỉ báo có kỳ vọng $1/2$ và phương sai $(1/2)(1 - 1/2) = 1/4$. Các lần tung độc lập, vì vậy kỳ vọng tổng bằng $10 \cdot 1/2 = 5$, phương sai tổng bằng $10 \cdot 1/4 = 5/2$ (tự tính). Độ lệch chuẩn là $\sqrt{5/2} \approx 1{,}581$ (tự tính).

Bài 4 (vừa khó). Một túi có ba thẻ đỏ và một thẻ xanh. Rút hai thẻ liên tiếp không hoàn lại. Tính xác suất thẻ thứ hai đỏ nếu thẻ đầu tiên đỏ, rồi so với xác suất thẻ thứ hai đỏ trước khi biết kết quả đầu.

Đáp án. Sau khi thẻ đầu đỏ, còn hai đỏ trong ba thẻ nên $P(R_2 \mid R_1) = 2/3$. Xác suất không điều kiện của thẻ thứ hai đỏ là $3/4 \cdot 2/3 + 1/4 \cdot 1 = 3/4$ (tự tính), hoặc do mỗi vị trí trong thứ tự rút đối xứng nên tỷ lệ đỏ ở vị trí hai bằng tỷ lệ đỏ ban đầu. Hai con số khác nhau, vì vậy kết quả hai lần rút phụ thuộc nhau.

Bài 5 (khó). Một biến X nhận -1, 0 và 3 với xác suất lần lượt là $1/4$, $1/2$ và $1/4$. Tính kỳ vọng, phương sai và độ lệch chuẩn.

Đáp án. Kỳ vọng là $(-1)\cdot 1/4 + 0 \cdot 1/2 + 3 \cdot 1/4 = 1/2$. Bình phương biến có kỳ vọng $1 \cdot 1/4 + 0 + 9 \cdot 1/4 = 5/2$. Phương sai là $5/2 - (1/2)^2 = 9/4$, nên độ lệch chuẩn là $3/2$ (tự tính). Kỳ vọng $1/2$ không phải giá trị có thể nhận của biến này.

Tài liệu tham khảo nền: Blitzstein & Hwang, Introduction to Probability; Ross, A First Course in Probability; Murphy, Probabilistic Machine Learning: An Introduction; ghi chú math 06, 07, 08, 09, 10, 11 và 22 trong repo.

Nền tảng 09

Gaussian một chiều và nhiều chiều: một đường cong chuông được điều khiển bởi tâm và độ phân tán

Loại: chương nền tảng, không phải ghi chú paper

Đọc trước: chương nền xác suất về kỳ vọng, phương sai và hiệp phương sai; vector, ma trận và phép nhân ma trận với vector ở mức cơ bản

Dùng ở: math 06, math 07, math 08, math 09, math 10, math 11, math 22, math 23

> Chương này giới thiệu phân phối Gaussian từ đầu, không phải bản dịch paper nào. Những con số ghi "tự tính" và các đáp án bài tập đã được kiểm bằng numpy.

1. Vì sao cần Gaussian

Phân phối Gaussian mô tả mật độ lớn nhất ở tâm và giảm dần khi độ lệch tăng; trong nhiều chiều, độ lệch được đo theo từng hướng bằng khoảng cách Mahalanobis. Trong AI, nó là mô hình thuận tiện cho nhiễu, sai số đo, trọng số hoặc biến ẩn chưa biết, và đầu ra bất định của mô hình.

Gaussian còn có tên **phân phối chuẩn** (normal distribution). Trong một chiều, hình dạng của nó là đường cong chuông đối xứng. Trong nhiều chiều, hình dạng thường giống một đám mây hình elip hoặc ellipsoid. Một tâm cho biết vị trí trung bình; độ rộng cho biết mức dao động; ma trận hiệp phương sai cho biết các hướng cùng biến động.

Gaussian quan trọng vì nhiều phép toán với nó có dạng gọn. Biến đổi tuyến tính của Gaussian vẫn là Gaussian, tổng của các Gaussian độc lập vẫn là Gaussian, và khi một tổng gồm nhiều thành phần nhỏ được chuẩn hoá, định lý giới hạn trung tâm thường dẫn tới một xấp xỉ Gaussian. Đây là các lý do thực dụng, không phải bằng chứng rằng mọi dữ liệu thật đều có hình chuông.

2. Nhắc nền cần thiết

Chương trước định nghĩa kỳ vọng là trung bình có trọng số và phương sai là độ dao động quanh kỳ vọng. Với biến X, ta ký hiệu kỳ vọng bằng $\mu = \mathbb{E}[X]$ và độ lệch chuẩn bằng $\sigma = \sqrt{\mathrm{Var}(X)}$.

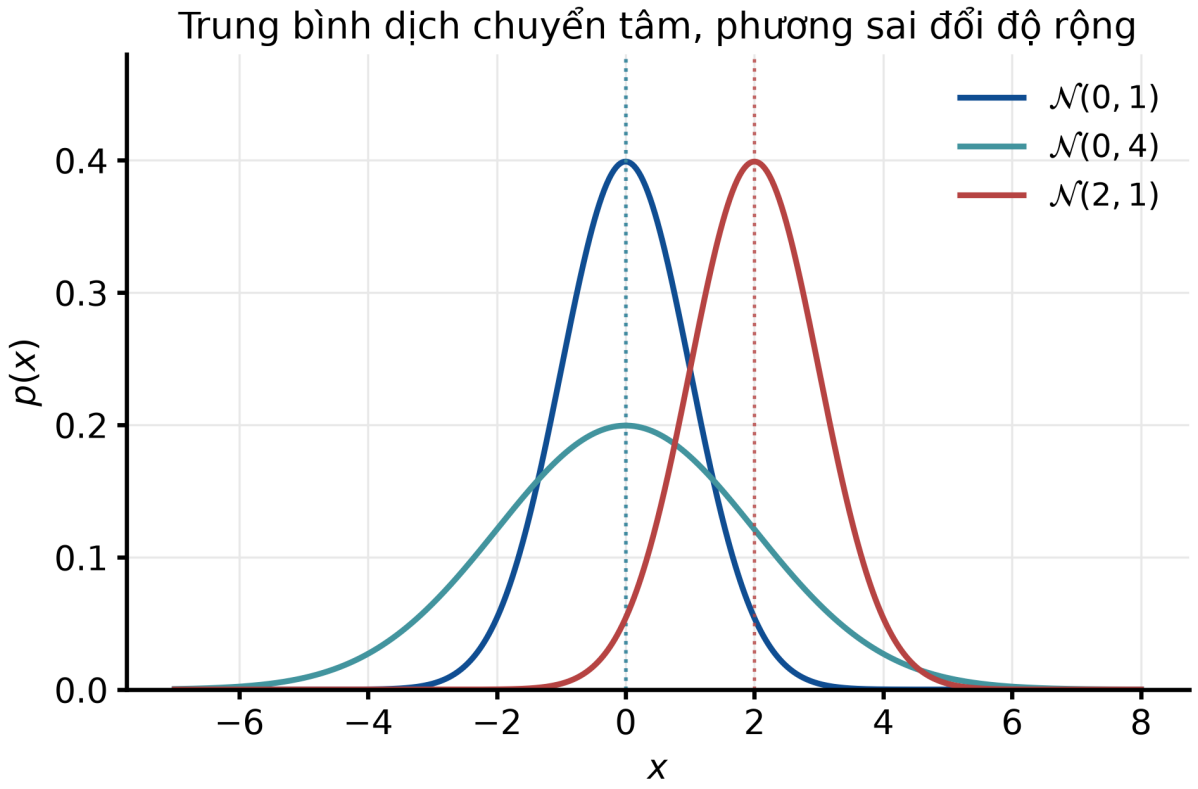
Ví dụ, nếu một phép đo có tâm 10 và độ lệch chuẩn 2, thì một giá trị quan sát là bao nhiêu đơn vị trên hay dưới 10 sẽ dễ đọc hơn số đo ban đầu. Lấy giá trị rồi trừ tâm, sau đó chia cho độ lệch chuẩn. Kết quả đếm khoảng cách theo số độ lệch chuẩn. Công thức chính thức và vai trò của phép chuẩn hoá sẽ xuất hiện sau hình trực giác.

Với nhiều biến, ta gom tâm thành vector μ và độ dao động cùng nhau vào ma trận hiệp phương sai Σ. Phần tử đường chéo là phương sai từng tọa độ; phần tử ngoài đường chéo là hiệp phương sai giữa hai tọa độ. Vector và ma trận giúp viết gọn, nhưng ý nghĩa vẫn là ta đã gặp ở chương xác suất.

3. Trực giác một chiều: tâm dịch chuyển, độ rộng thay đổi

Ba đường trong hình đều có dạng chuông. Đường màu xanh và xanh lục có cùng tâm, nhưng đường xanh lục trải rộng hơn. Đường đỏ có độ rộng tương tự đường xanh nhưng tâm đã dịch sang phải.

Cách đọc hình 1: đỉnh của mỗi đường nằm tại trung bình của phân phối đó. Hai đường có tâm 0 cùng đạt đỉnh tại 0, nhưng đường có phương sai lớn hơn thấp và rộng hơn vì cùng khối xác suất trải trên vùng lớn hơn. Đường đỏ dịch sang phải mà giữ nguyên độ rộng, nên trung bình đổi nhưng độ lệch chuẩn không đổi.



Hình 30: Ba Gaussian một chiều với tâm hoặc độ rộng khác nhau (minh hoạ, tự tính)

Chiều cao của đường không phải xác suất tại một điểm. Xác suất của một khoảng là diện tích dưới đường trong khoảng đó. Khi đường thấp hơn và rộng hơn, tổng diện tích vẫn bằng một, không phải phân phối có "ít xác suất hơn".

4. Định nghĩa Gaussian một chiều

Biến liên tục X có phân phối Gaussian với trung bình μ và phương sai σ^2 khi mật độ của nó là

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \qquad \sigma > 0.$$

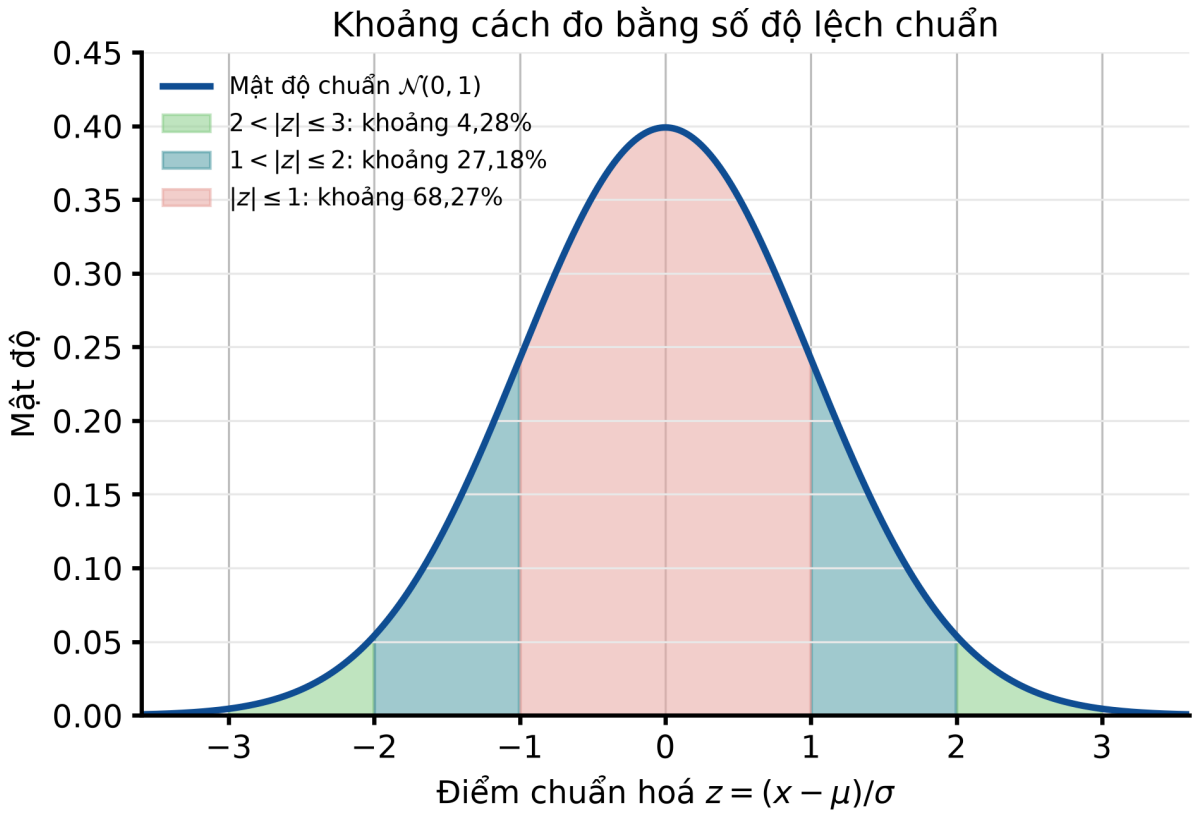
Ta viết ngắn gọn $X \sim \mathcal{N}(\mu, \sigma^2)$. Tham số thứ nhất là tâm; tham số thứ hai là **phương sai**, không phải độ lệch chuẩn. Độ lệch chuẩn là σ.

Có hai thành phần trong công thức. Số mũ phạt các giá trị cách μ xa nhiều lần σ; hệ số phía trước điều chỉnh chiều cao để diện tích dưới đường bằng một. Nếu tăng σ, đường thấp hơn và rộng hơn. Nếu đổi μ, toàn bộ đường tịnh tiến mà hình dạng không đổi.

Vì hàm mũ chỉ phụ thuộc $(x-\mu)^2$, hai phía cách tâm cùng một khoảng có cùng mật độ. Trung bình, trung vị và đỉnh đều nằm ở μ do tính đối xứng. Phương sai đúng bằng σ^2 theo cách tham số hoá này.

5. Chuẩn hoá và điểm z

Để so các giá trị trong những thang đo khác nhau, ta chuyển mỗi giá trị về số độ lệch chuẩn tính từ tâm. Hình dưới cho thấy các khoảng quanh tâm trong phân phối chuẩn chuẩn hoá.



Hình 31: Các khoảng một, hai và ba độ lệch chuẩn quanh trung bình của Gaussian chuẩn (minh hoạ, tự tính)

Cách đọc hình 2: trục ngang đã được chuẩn hoá nên tâm nằm ở 0 và một đơn vị là một độ lệch chuẩn. Vùng đỏ là phần trong khoảng từ -1 đến 1; dải xanh ngọc là phần từ một đến hai độ lệch chuẩn ở mỗi phía; dải xanh lá là phần từ hai đến ba. Cộng dồn từ tâm ra ngoài, các khoảng từ -1 đến 1, từ -2 đến 2 và từ -3 đến 3 chứa xấp xỉ 68,27%, 95,45% và 99,73% xác suất (tự tính bằng tích phân Gaussian chuẩn).

Nếu $X \sim \mathcal{N}(\mu, \sigma^2)$, ta đặt

$$Z = \frac{X - \mu}{\sigma}.$$

Khi đó $Z \sim \mathcal{N}(0, 1)$, gọi là Gaussian chuẩn tắc. Một giá trị $z = 2$ nghĩa là quan sát cao hơn trung bình hai độ lệch chuẩn. Điểm z không phải xác suất; ta cần dùng phân phối chuẩn chuẩn tắc hoặc bảng CDF để đổi điểm z thành xác suất của một khoảng.

Phép đổi này giữ nguyên thứ tự: giá trị lớn hơn μ cho điểm z dương, nhỏ hơn μ cho điểm z âm. Vì thế ta có thể chuyển một xác suất của Gaussian bất kỳ sang xác suất tương ứng dưới đường chuẩn tắc, vốn chỉ có một dạng duy nhất.

6. Ví dụ tay: chuẩn hoá trước khi tìm xác suất

Giả sử thời gian chờ X được mô hình hoá bằng $\mathcal{N}(10, 4)$, tức trung bình 10 phút và phương sai 4 phút bình phương. Độ lệch chuẩn là $\sigma = \sqrt{4} = 2$ phút (tự tính).

Ta muốn xác suất thời gian nằm từ 8 đến 12 phút. Chuẩn hoá hai đầu mút:

$$z_{\text{dưới}} = \frac{8-10}{2} = -1, \qquad z_{\text{trên}} = \frac{12-10}{2} = 1.$$

Do đó xác suất cần tìm chính là phần diện tích từ -1 đến 1 dưới Gaussian chuẩn tắc. Theo CDF chuẩn, giá trị này xấp xỉ $0{,}6827$, tức khoảng 68,27% (tự tính). Đây là phép tích phân của đường cong, không phải lấy chiều cao tại hai đầu mút.

Có thể kiểm tra một khoảng khác mà không cần đổi công thức. Khoảng từ 6 đến 14 phút cách tâm hai độ lệch chuẩn ở mỗi phía, nên xác suất vào khoảng 95,45% (tự tính). Nếu X có cùng trung bình nhưng độ lệch chuẩn lớn hơn, hai ngưỡng 8 và 12 sẽ gần tâm hơn tính theo đơn vị độ lệch chuẩn, nên xác suất trong khoảng đó giảm.

Các con số 68,27% và 95,45% là xấp xỉ cho diện tích của Gaussian chuẩn, không phải quy tắc chắc chắn cho một tập dữ liệu hữu hạn. Nếu phân phối thật không Gaussian, các khoảng một và hai độ lệch chuẩn không nhất thiết chứa những tỷ lệ ấy.

7. Trực giác nhiều chiều: hiệp phương sai định hướng đám mây

Với hai đại lượng, đám mây điểm có thể tròn hoặc kéo dài theo một đường chéo. Tương quan dương làm hai tọa độ thường tăng cùng nhau; tương quan âm làm một tọa độ tăng khi tọa độ kia giảm. Trường hợp không tương quan tạo các đường mức song song với trục tọa độ trong ví dụ đối xứng dưới đây.



Hình 32: Đường mức Gaussian hai chiều với tương quan âm, bằng không và dương (minh hoạ, tự tính)

Cách đọc hình 3: mỗi đường là tập điểm có cùng mật độ, không phải một quỹ đạo mà dữ liệu bắt buộc phải đi theo. Khi tương quan dương, elip nghiêng theo hướng hai tọa độ cùng tăng; khi tương quan âm, nó nghiêng theo hướng một tọa độ tăng còn tọa độ kia giảm. Cả ba hình có cùng phương sai biên bằng một, nhưng ma trận hiệp phương sai khác nhau (tự tính).

Trong nhiều chiều, “độ xa tâm” không còn chỉ là khoảng cách Euclid. Đi xa theo hướng biến động tự nhiên của dữ liệu ít bất thường hơn đi cùng khoảng cách theo hướng rất hẹp. Ma trận hiệp phương sai định nghĩa cách đo có hướng này.

8. Gaussian nhiều chiều: trung bình và ma trận hiệp phương sai

8.1 Ma trận hiệp phương sai

Với vector ngẫu nhiên $\mathbf{X} = (X_1, \ldots, X_d)^\top$, vector trung bình là

$$\mu = \mathbb{E}[\mathbf{X}].$$

Ma trận hiệp phương sai có phần tử ở hàng i, cột j là

$$\Sigma_{ij} = \operatorname{Cov}(X_i, X_j) = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)].$$

Do tính đối xứng của tích, Σ đối xứng. Các phần tử đường chéo Σ_{ii} chính là phương sai của tọa độ thứ i. Ma trận hiệp phương sai luôn nửa xác định dương: với mọi vector $\mathbf{a}$, giá trị $\mathbf{a}^\top \Sigma \mathbf{a}$ là phương sai của tổ hợp tuyến tính $\mathbf{a}^\top \mathbf{X}$, nên không âm.

Nếu mọi tọa độ có phương sai 1, ta vẫn chưa biết chúng có cùng biến động hay không. Phần tử ngoài đường chéo phân biệt các trường hợp đó. Tương quan chuẩn hoá hiệp phương sai để có thang đo từ -1 đến 1, giúp so độ liên hệ giữa các tọa độ có đơn vị khác nhau.

8.2 Định nghĩa Gaussian nhiều chiều

Vector $\mathbf{X}$ có phân phối Gaussian d chiều với trung bình μ và ma trận hiệp phương sai xác định dương Σ nếu mật độ là

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}\sqrt{\det \Sigma}} \exp\left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right).$$

Ta viết $\mathbf{X} \sim \mathcal{N}(\mu, \Sigma)$. Số hạng bậc hai trong hàm mũ là khoảng cách Mahalanobis bình phương: nó tính độ xa tâm nhưng chia theo hướng biến động. Mặt có cùng khoảng cách Mahalanobis tạo ra ellipsoid.

Điều kiện xác định dương bảo đảm mật độ thông thường tồn tại trên toàn không gian và ma trận có nghịch đảo. Nếu Σ chỉ nửa xác định dương, phân phối có thể nằm trên một mặt phẳng hoặc không gian con, khi ấy không có mật độ thông thường theo thể tích của toàn không gian. Đây là trường hợp suy biến, không thể xử lý bằng cách lấy nghịch đảo ma trận như thường.

8.3 Gaussian đẳng hướng, đường chéo và đầy đủ

Nếu $\Sigma = \sigma^2 I$, phân phối **đẳng hướng** có cùng độ rộng theo mọi hướng. Các đường mức là hình tròn trong hai chiều và mặt cầu trong nhiều chiều.

Nếu Σ là đường chéo, mỗi tọa độ có độ rộng riêng nhưng không có hiệp phương sai giữa các tọa độ. Với phân phối Gaussian, các tọa độ không tương quan khi ấy cũng độc lập. Với những phân phối không Gaussian, ma trận đường chéo chỉ nói rằng chúng không tương quan tuyến tính, chưa đủ kết luận độc lập.

Nếu Σ có phần tử ngoài đường chéo khác không, ta có Gaussian **hiệp phương sai đầy đủ**. Dạng này linh hoạt hơn, nhưng cần lưu nhiều tham số hơn và phải bảo đảm ma trận vẫn xác định dương. Nhiều mô hình dùng Gaussian đường chéo vì đơn giản và rẻ hơn, dù giả định độc lập giữa các chiều có thể quá mạnh.

9. Ví dụ tay: tạo Gaussian hai chiều từ nhiễu chuẩn

Ta muốn tạo một Gaussian hai chiều có tâm $\mu = (1, -1)^\top$ và ma trận hiệp phương sai

$$\Sigma = \begin{pmatrix} 1 & 0{,}6 \\ 0{,}6 & 1 \end{pmatrix}.$$

Một cách dựng là lấy hai nhiễu độc lập $\varepsilon_1, \varepsilon_2 \sim \mathcal{N}(0, 1)$ rồi biến đổi affine. Ma trận tam giác

$$L = \begin{pmatrix} 1 & 0 \\ 0{,}6 & 0{,}8 \end{pmatrix}$$

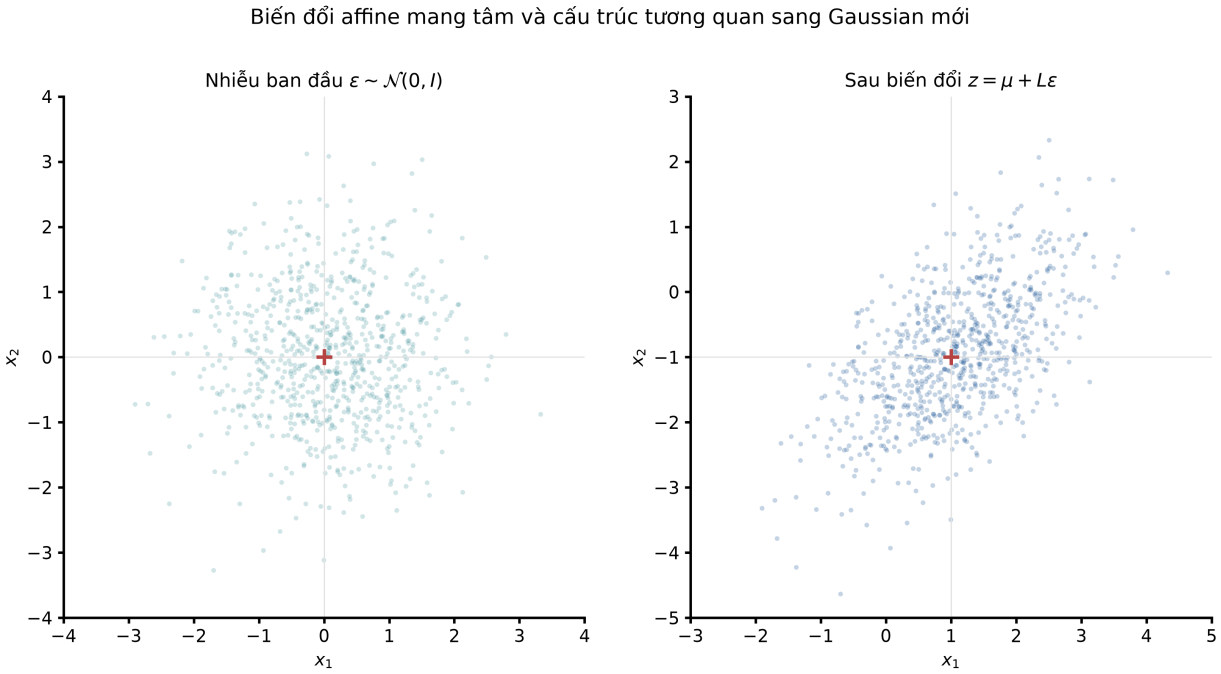
thoả $LL^\top = \Sigma$, vì phần tử dưới phải là $0{,}6^2 + 0{,}8^2 = 1$ và hai phần tử ngoài đường chéo đều là $0{,}6$ (tự tính). Do đó ta đặt

$$\mathbf{Z} = \mu + L\varepsilon, \qquad \varepsilon = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \end{pmatrix}.$$

Từng phép toán có ý nghĩa trực tiếp. Nhân với L tạo các độ rộng và tương quan mong muốn; cộng μ dịch tâm. Vì $\mathbb{E}[\varepsilon] = \mathbf{0}$, trung bình của $\mathbf{Z}$ là μ. Vì hiệp phương sai của nhiễu chuẩn là ma trận đơn vị,

$$\mathrm{Cov}(\mathbf{Z}) = LIL^\top = LL^\top = \Sigma.$$

Với nhiễu cụ thể $\varepsilon = (1, 0)^\top$, ta nhận $\mathbf{Z} = (2, -0{,}4)^\top$ (tự tính). Với nhiễu $(0, 1)^\top$, ta nhận $\mathbf{Z} = (1, -0{,}2)^\top$ (tự tính). Đây chỉ là hai ví dụ đầu vào để thấy phép biến đổi, không phải hai điểm đại diện cho toàn phân phối.



Hình 33: Nhiễu chuẩn độc lập trước và sau phép biến đổi tạo Gaussian có tương quan (minh hoạ, tự tính)

Cách đọc hình 4: panel trái là các mẫu nhiễu chuẩn tập trung quanh gốc, gần tròn; dấu cộng đỏ là tâm. Panel phải là các mẫu sau khi nhân ma trận L và dịch tâm, chúng tạo thành đám mây nghiêng quanh $(1, -1)$. Ma trận biến đổi được chọn để phương sai từng chiều bằng một và tương quan bằng 0,6 (tự tính).

Cách viết $\mathbf{Z} = \mu + L\varepsilon$ còn được gọi là **tái tham số hoá**. Nhiễu ngẫu nhiên nằm trong ε với phân phối cố định; tâm và ma trận L là các tham số điều khiển một phép biến đổi xác định. Điều này hữu ích khi cần tính gradient qua một biến được lấy mẫu, như trong VAE.

10. Các tính chất và chi tiết hay bị bỏ qua

10.1 Biến đổi affine

Nếu $\mathbf{X} \sim \mathcal{N}(\mu, \Sigma)$, thì với ma trận A và vector $\mathbf{b}$,

$$\mathbf{Y} = A\mathbf{X} + \mathbf{b}$$

cũng là Gaussian, với

$$\mathbb{E}[\mathbf{Y}] = A\mu + \mathbf{b}, \qquad \mathrm{Cov}(\mathbf{Y}) = A\Sigma A^{\top}.$$

Điều này gồm trường hợp vô hướng $aX + b$, trong đó trung bình đổi thành $a\mu + b$ và phương sai thành $a^2\sigma^2$. Một phép chiếu tuyến tính của Gaussian nhiều chiều vẫn Gaussian; hiệp phương sai biến đổi cùng phép chiếu.

10.2 Hiệp phương sai, tương quan và góc của ellipsoid

Ma trận hiệp phương sai xác định kích thước và hướng các trục của ellipsoid mật độ. Các vector riêng cho hướng chính, còn trị riêng cho độ rộng bình phương theo hướng đó. Nếu mọi trị riêng bằng nhau, không có hướng nào đặc biệt; nếu một trị riêng lớn hơn, đám mây trải rộng hơn dọc theo vector riêng tương ứng.

Hệ số tương quan giữa hai tọa độ bằng hiệp phương sai chia cho tích độ lệch chuẩn của chúng. Nó không có đơn vị, trong khi hiệp phương sai phụ thuộc đơn vị đo. Khi một tọa độ đổi từ mét sang centimet, trị số hiệp phương sai đổi theo, nhưng hệ số tương quan không đổi.

10.3 Gaussian có nhiều dạng xấp xỉ

Một Gaussian đầy đủ dùng ma trận covariance chung. Gaussian đường chéo chỉ học một phương sai cho mỗi chiều. Gaussian đẳng hướng dùng cùng một phương sai cho mọi chiều. Một số mô hình dùng ma trận hạng thấp cộng phần đường chéo để cân bằng biểu diễn tương quan với chi phí tính toán. Đây là các cấu trúc tham số khác nhau, không phải các định nghĩa xác suất khác nhau.

Một **hỗn hợp Gaussian** chọn một trong vài Gaussian thành phần rồi lấy mẫu từ thành phần đó. Hỗn hợp có thể mô tả nhiều đỉnh, trong khi một Gaussian đơn thường chỉ có một đỉnh. Nếu dữ liệu có nhiều cụm tách biệt, tăng phương sai của một Gaussian duy nhất không nhất thiết mô tả được cấu trúc đúng.

10.4 Mật độ và xác suất thể tích trong nhiều chiều

Mật độ ở tâm không phải xác suất điểm tâm. Xác suất một vùng là tích phân mật độ trên vùng đó. Trong nhiều chiều, thể tích của các vùng thay đổi nhanh theo kích thước, nên trực giác từ đường cong một chiều dễ gây nhầm. Đặc biệt, điểm có mật độ lớn nhất không đồng nghĩa một điểm đơn lẻ chứa nhiều xác suất nhất; mọi điểm riêng lẻ vẫn có xác suất bằng không với phân phối liên tục.

10.5 Ma trận hiệp phương sai số học

Khi ước lượng covariance từ mẫu, sai số lấy mẫu có thể làm ma trận gần suy biến hoặc không xác định dương nếu có quá ít mẫu so với số chiều. Chỉ thêm nghịch đảo trực tiếp không giải quyết được vấn đề. Có thể cần làm trơn, giảm chiều hoặc áp cấu trúc covariance phù hợp. Chương về số học máy tính sẽ bàn thêm về điều kiện số và phép tính ổn định.

11. Nối sang AI

Nhiễu đo và đầu ra liên tục. Nếu mô hình hoá sai số quanh dự đoán bằng Gaussian độc lập, trung bình bằng không và phương sai cố định, tối thiểu hoá tổng bình phương sai số tương đương với tối đa hoá likelihood Gaussian. Nếu sai số chỉ gần Gaussian, đây là một cách mô hình hoá xấp xỉ. Khi phương sai được học, mô

hình có thể dự đoán độ bất định thay đổi theo đầu vào. Giả định này cần được kiểm tra với phần dư, không nên xem là đúng sẵn.

Biến ẩn trong VAE. Encoder thường trả về trung bình và phương sai của Gaussian điều kiện theo dữ liệu. Tái tham số hoá viết mẫu ẩn thành $\mathbf{z} = \mu + L\varepsilon$ để tách nhiễu khỏi tham số học được. Ghi chú math 09 dùng trực tiếp cơ chế này; math 08 đặt Gaussian trong bối cảnh suy luận biến phân.

Lấy mẫu trong HMC và suy luận Bayes. Gaussian đa biến cung cấp phân phối tham chiếu và hình học cục bộ cho nhiều thuật toán lấy mẫu. Ma trận covariance ảnh hưởng hướng đi và thang đo trong không gian tham số. Chi tiết thuật toán nằm trong math 10, nên ở đây chỉ cần nắm cấu trúc xác suất.

Natural gradient và Fisher. Các phân phối xác suất có thể được so sánh thông qua thay đổi của chúng, thay vì chỉ nhìn khoảng cách Euclid giữa tham số. Với Gaussian, trung bình và covariance là những tọa độ tự nhiên để viết likelihood, Fisher information và các phép xấp xỉ địa phương. Các ghi chú math 06 và math 07 phát triển góc nhìn đó.

Quá trình Gaussian và các mô hình gần Gaussian. Trong Gaussian process, một tập hữu hạn giá trị hàm được mô tả bằng Gaussian nhiều chiều, và covariance nói các điểm đầu vào liên hệ với nhau thế nào. Các mô hình mạng sâu cũng có những giới hạn hoặc xấp xỉ Gaussian trong một số điều kiện; xem math 22 và math 23.

Định lý giới hạn trung tâm và nhiễu. Tổng nhiều thành phần độc lập thường gần chuẩn sau chuẩn hoá trong điều kiện thích hợp. Điều này giải thích vì sao Gaussian hay được dùng như mô hình nhiễu hoặc xấp xỉ, nhưng không bảo đảm mọi activation, gradient hoặc dữ liệu đầu ra đều Gaussian.

12. Sai lầm hay gặp

- **Nhầm tham số thứ hai của $\mathcal{N}(\mu, \sigma^2)$.** Đó là phương sai. Muốn lấy độ lệch chuẩn phải căn bậc hai.
- **Đọc chiều cao mật độ như xác suất một điểm.** Chỉ tích phân trên một vùng mới cho xác suất của vùng ấy.
- **Chuẩn hoá bằng phương sai thay vì độ lệch chuẩn.** Điểm z là $(x - \mu)/\sigma$, vì mẫu số cần cùng đơn vị với tử số.
- **Nghĩ mọi dữ liệu đều Gaussian vì định lý giới hạn trung tâm.** Định lý có điều kiện và thường là xấp xỉ cho tổng đã chuẩn hoá; nó không nói mỗi biến riêng lẻ có dạng chuẩn.
- **Cho rằng covariance bằng không luôn có nghĩa độc lập.** Điều này đúng với các tọa độ của Gaussian chung, nhưng không đúng cho mọi phân phối.
- **Đảo ma trận covariance suy biến như thể nó luôn khả nghịch.** Cần kiểm tra tính xác định dương và hiểu phân phối có thể nằm trên một không gian con.

13. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Gaussian một chiều	$X \sim \mathcal{N}(\mu, \sigma^2)$	Tâm μ, phương sai σ^2
Mật độ một chiều	$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp(-\frac{(x-\mu)^2}{2\sigma^2})$	Xác suất khoảng là diện tích
Chuẩn hoá	$Z = (X - \mu)/\sigma$	Đưa về $\mathcal{N}(0, 1)$
Gaussian nhiều chiều	$\mathbf{X} \sim \mathcal{N}(\mu, \Sigma)$	Tâm vector, covariance ma trận
Phần tử covariance	$\Sigma_{ij} = \mathrm{Cov}(X_i, X_j)$	Dao động chung giữa hai tọa độ
Mahalanobis bình phương	$(\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu)$	Khoảng cách có điều chỉnh theo hướng
Biến đổi affine	$\mathbf{Y} = A\mathbf{X} + \mathbf{b}$	Gaussian vẫn là Gaussian
Trung bình sau biến đổi	$\mathbb{E}[\mathbf{Y}] = A\mu + \mathbf{b}$	Tâm được biến đổi tuyến tính

Khái niệm	Công thức	Ý nghĩa
Covariance sau biến đổi	$\mathrm{Cov}(\mathbf{Y}) = A\Sigma A^{\top}$	Độ rộng và hướng được biến đổi
Tái tham số hoá	$\mathbf{Z} = \mu + L\varepsilon$, $LL^{\top} = \Sigma$	Tạo Gaussian từ nhiễu chuẩn

14. Bài tập tự tính

Hãy tự giải trước rồi mới xem đáp án ngay dưới mỗi bài. Các kết quả số trong đáp án đã được kiểm lại bằng numpy.

Bài 1 (dễ). Biến $X \sim \mathcal{N}(5, 9)$. Hãy nêu trung bình, phương sai và độ lệch chuẩn.

Đáp án. Trung bình là 5, phương sai là 9, độ lệch chuẩn là 3 (tự tính), vì độ lệch chuẩn bằng căn phương sai. Cần chú ý tham số thứ hai trong ký hiệu là phương sai.

Bài 2 (dễ vừa). Với $X \sim \mathcal{N}(10, 4)$, đổi khoảng $8 \le X \le 12$ sang khoảng trên trục chuẩn hoá Z.

Đáp án. Độ lệch chuẩn bằng 2. Hai đầu mút cho $z = (8-10)/2 = -1$ và $z = (12-10)/2 = 1$ (tự tính), nên khoảng tương đương là $-1 \le Z \le 1$. Xác suất xấp xỉ 68,27% theo Gaussian chuẩn (tự tính bằng CDF).

Bài 3 (vừa). Nếu $X \sim \mathcal{N}(0, 1)$ và $Y = 3X + 2$, hãy tìm phân phối của Y.

Đáp án. Trung bình của Y là $3 \cdot 0 + 2 = 2$. Phương sai là $3^2 \cdot 1 = 9$, nên $Y \sim \mathcal{N}(2, 9)$ và độ lệch chuẩn bằng 3 (tự tính). Nhân với 3 đổi thang, cộng 2 dịch tâm.

Bài 4 (vừa khó). Ma trận hiệp phương sai là $\begin{pmatrix} 4 & 3 \\ 3 & 9 \end{pmatrix}$. Tìm độ lệch chuẩn của từng tọa độ và hệ số tương quan.

Đáp án. Phương sai biên là 4 và 9, nên độ lệch chuẩn lần lượt là 2 và 3. Hệ số tương quan là $3/(2 \cdot 3) = 1/2$ (tự tính). Hiệp phương sai dương nên hai tọa độ có xu hướng tăng cùng nhau. Ma trận xác định dương vì các phần tử đường chéo dương và định thức $4 \cdot 9 - 3 \cdot 3 = 27 > 0$ (tự tính).

Bài 5 (khó). Với $\Sigma = \begin{pmatrix} 1 & 0{,}6 \\ 0{,}6 & 1 \end{pmatrix}$, tính khoảng cách Mahalanobis bình phương từ tâm đến điểm lệch $(1, 1)^{\top}$ và đến điểm lệch $(1, -1)^{\top}$.

Đáp án. Định thức của Σ bằng $1 - 0{,}6^2 = 0{,}64$, nên ma trận nghịch đảo là $\frac{1}{0{,}64}\begin{pmatrix} 1 & -0{,}6 \\ -0{,}6 & 1 \end{pmatrix}$ (tự tính). Thay vector $(1, 1)^{\top}$ vào dạng bậc hai được $1{,}25 = 5/4$; thay $(1, -1)^{\top}$ được 5 (tự tính). Đi xa theo hướng tương quan dương tốn ít khoảng cách Mahalanobis hơn, vì đám mây phân phối trải dài hơn theo hướng đó.

Tài liệu tham khảo nền: DeGroot & Schervish, Probability and Statistics; Murphy, Probabilistic Machine Learning: An Introduction; Bishop, Pattern Recognition and Machine Learning; ghi chú math 06, 07, 08, 09, 10, 11, 22 và 23 trong repo.

Nền tảng 10

Bayes, MLE và MAP: cập nhật điều chưa biết từ dữ liệu

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: xác suất có điều kiện, kỳ vọng và phương sai ở chương nền 08; Gaussian ở chương nền 09; đạo hàm cơ bản ở chương nền 06

Dùng ở: math 08 (suy luận biến phân), math 09 (VAE), math 10 (MCMC), math 11 và math 35 (Thompson sampling)

> Đây là chương học từ đầu, không phải bản dịch paper. Các phép tính ghi ”tự tính” đã được kiểm bằng numpy.

1. Vì sao cần Bayes, MLE và MAP

Suy luận thống kê là dùng dữ liệu quan sát để nói điều gì đó về thứ ta chưa biết. Một đồng xu có thể lệch, một bộ phân loại có tham số chưa học, hay một xét nghiệm có thể dương tính nhầm. Ta không nhìn trực tiếp được mọi sự thật; ta xây mô hình mô tả dữ liệu có thể xuất hiện thế nào rồi cập nhật điều mình biết.

Ba cái tên trong chương trả lời những câu hơi khác nhau. **Bayes** cho phân phối hậu nghiệm: sau khi thấy dữ liệu, những khả năng nào còn hợp lý và mỗi khả năng được cho trọng lượng bao nhiêu? **MLE** chọn tham số làm dữ liệu đã thấy có likelihood lớn nhất. **MAP** chọn tham số tại đỉnh của phân phối hậu nghiệm, vì thế có tính đến cả dữ liệu và giả định ban đầu. Cả ba đều phụ thuộc vào mô hình, không có chiếc máy nào tự biến dữ liệu thành sự thật chắc chắn.

Trong AI, ta dùng MLE để huấn luyện mô hình dự đoán qua log-likelihood, dùng prior để đưa giả định vào bài toán ít dữ liệu, và dùng posterior để biểu diễn sự không chắc chắn. Biết phân biệt ba đối tượng giúp ta đọc đúng VAE, suy luận biến phân, MCMC và Thompson sampling ở các bài sau.

2. Nhắc nhanh xác suất cần dùng

2.1 Có điều kiện và phân hoạch

Gọi A là sự kiện có bệnh và B là kết quả xét nghiệm dương tính. Xác suất có điều kiện $P(B \mid A)$ trả lời câu: **trong nhóm đã có bệnh, bao nhiêu phần có kết quả dương tính?** Nó không cùng câu hỏi với $P(A \mid B)$, tức trong nhóm dương tính có bao nhiêu phần thật sự có bệnh. Ký hiệu bên phải dấu gạch đứng là điều đang được cho biết.

Nếu $P(A) > 0$, định nghĩa là

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)}, \qquad P(A \cap B) = P(B \mid A)P(A).$$

Ví dụ (tự tính): $P(A) = 0{,}1$ và $P(B \mid A) = 0{,}8$ cho $P(A \cap B) = 0{,}08$. Đó là phần vừa có bệnh vừa dương tính trong toàn bộ nhóm, chưa phải phần có bệnh trong nhóm dương tính.

Nếu mọi người thuộc đúng một trong hai nhóm A hoặc A^c, xác suất toàn phần của dương tính là

$$P(B) = P(B \mid A)P(A) + P(B \mid A^c)P(A^c).$$

Ta cộng hai đường không chồng nhau: dương tính ở nhóm có bệnh và dương tính ở nhóm không bệnh. Số sau dấu bằng là xác suất của **cùng một** sự kiện B, tính trên toàn quần thể.

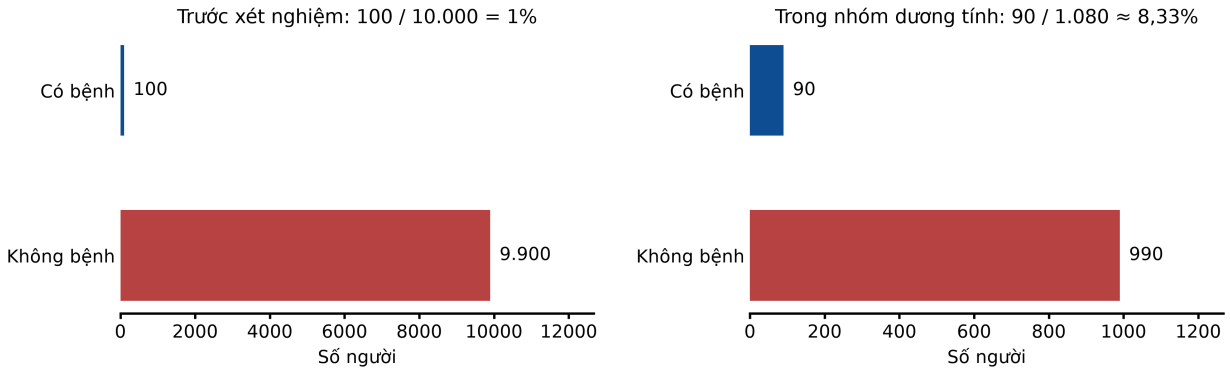
2.2 Xác suất, mật độ và ký hiệu tham số

Một xác suất rời rạc là trọng lượng của từng kết quả và tổng trọng lượng bằng 1. Với biến liên tục, **mật độ** là chiều cao của đường cong; xác suất nằm trong một khoảng bằng diện tích dưới đường cong trên khoảng đó. Mật độ có thể lớn hơn 1, miễn tổng diện tích bằng 1; xác suất tại một điểm chính xác bằng 0 đối với một phân phối liên tục thông thường.

Ta dùng θ cho tham số chưa biết, chẳng hạn xác suất mặt ngửa của đồng xu, và D cho dữ liệu đã quan sát. Ký hiệu $p(D \mid \theta)$ mô tả cách mô hình cho dữ liệu khi chọn một giá trị của θ. Khi D cố định và ta thay đổi θ, cùng biểu thức ấy được nhìn như **likelihood** của tham số. Không được tự động đọc likelihood như xác suất của θ.

3. Trực giác trước công thức: dương tính không đồng nghĩa chắc chắn có bệnh

Xét một ví dụ hoàn toàn giả định, không phải số liệu y khoa. Trong nhóm 10.000 người, 1% có bệnh; xét nghiệm dương tính ở 90% người có bệnh và cũng dương tính nhầm ở 10% người không bệnh. Những số lượng ở đây là số kỳ vọng theo mô hình, hoặc đúng với một bảng giả định có đúng các tỉ lệ đó.



Hình 34: Từ tỉ lệ bệnh ban đầu đến thành phần của nhóm xét nghiệm dương tính (minh hoạ, tự tính)

Cách đọc hình 1: bên trái, trong 10.000 người chỉ có 100 người thuộc nhóm có bệnh. Bên phải chỉ giữ những người có kết quả dương tính: 90 người có bệnh và 990 người không bệnh, vì nhóm không bệnh ban đầu lớn hơn rất nhiều. Trục ngang của hai panel có thang khác nhau, nên không so trực tiếp độ dài thanh giữa hai panel. Vậy trong 1.080 kết quả dương tính, chỉ 90 là người có bệnh, khoảng 8,33% (tự tính).

Điều bất ngờ không nằm ở một phép nhân khó mà ở **mẫu số**. Nếu chỉ nhìn độ nhạy 90%, ta đang chọn mẫu số là những người có bệnh. Câu hỏi sau khi có kết quả dương tính lại yêu cầu mẫu số là tất cả người dương tính, bao gồm cả dương tính nhầm. Phải tính cả hai nhóm trước khi chia.

4. Công thức Bayes và tính tay từng bước

4.1 Quy tắc tổng quát

Từ hai cách viết cùng một xác suất giao, $P(A \cap B) = P(B \mid A)P(A) = P(A \mid B)P(B)$, ta có công thức Bayes, khi $P(B) > 0$:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

Đọc bằng lời: **hậu nghiệm bằng likelihood nhân tiên nghiệm rồi chia cho xác suất của dữ liệu.** Ở ví dụ này, tiên nghiệm là xác suất có bệnh trước khi biết kết quả; hậu nghiệm là xác suất có bệnh sau khi biết kết quả dương tính. Từ ”trước” và ”sau” nói về thông tin đã điều kiện hoá, không đòi hỏi bệnh xuất hiện sau xét nghiệm.

4.2 Thay từng số vào ví dụ

Bước 1, trong 10.000 người có $10.000 \cdot 0{,}01 = 100$ người có bệnh và 9.900 người không bệnh (tự tính).

Bước 2, dương tính thật có $100 \cdot 0{,}9 = 90$ người. Dương tính nhầm có $9.900 \cdot 0{,}1 = 990$ người. Cộng lại được $90 + 990 = 1.080$ kết quả dương tính (tự tính).

Bước 3, lấy phần có bệnh trong nhóm dương tính:

$$P(A \mid B) = \frac{90}{1080} = \frac{1}{12} \approx 0{,}0833.$$

Bằng công thức xác suất thay vì đếm người, ta được cùng kết quả (tự tính):

$$\frac{0{,}9 \cdot 0{,}01}{0{,}9 \cdot 0{,}01 + 0{,}1 \cdot 0{,}99} = \frac{0{,}009}{0{,}108} = \frac{1}{12}.$$

Lưu ý hai con số có vai trò khác nhau: 90% là $P(B \mid A)$, còn khoảng 8,33% là $P(A \mid B)$. Cả hai đều đúng trong ví dụ. Tỉ lệ nền, độ nhạy và tỉ lệ dương tính nhầm đều phải phù hợp với quần thể thực tế để kết luận có ý nghĩa.

4.3 Nhiều giả thuyết và bằng chứng

Với các giả thuyết loại trừ nhau và bao hết khả năng $H_1, \ldots, H_m$, quy tắc giống hệt:

$$P(H_i \mid D) = \frac{P(D \mid H_i)P(H_i)}{\sum_j P(D \mid H_j)P(H_j)}.$$

Tử số là trọng lượng chưa chuẩn hoá của giả thuyết H_i; mẫu số cộng trọng lượng của **mọi** giả thuyết để tổng hậu nghiệm bằng 1. Đừng bỏ giả thuyết khác khỏi mẫu số chỉ vì ta đang quan tâm một giả thuyết. Nếu bằng chứng có xác suất bằng 0 trong mô hình, phép chia này không xác định: phải xem lại mô hình hoặc thông tin điều kiện hoá.

Dạng tỉ số cũng hữu ích: tỉ số hậu nghiệm giữa hai giả thuyết bằng tỉ số tiên nghiệm nhân **hệ số Bayes** $P(D \mid H_1)/P(D \mid H_2)$, miễn các mẫu số liên quan khác 0. Hệ số này hỏi dữ liệu hợp với giả thuyết thứ nhất gấp mấy lần giả thuyết thứ hai, chứ không một mình cho xác suất hậu nghiệm.

5. Từ dữ liệu đến likelihood và MLE

5.1 Đồng xu là một mô hình

Cho một đồng xu có xác suất ngửa $\theta \in [0, 1]$. Giả sử các lần tung **độc lập có điều kiện theo cùng** θ. Ta thấy chính xác dãy $D = (1, 1, 1, 0)$, với 1 là ngửa và 0 là sấp. Khi đã chọn một giá trị của θ, xác suất đúng dãy có thứ tự này là

$$p(D \mid \theta) = \theta^3(1 - \theta).$$

Nếu dữ liệu chỉ ghi ”3 ngửa trong 4 lần” mà không ghi thứ tự, còn phải nhân số cách sắp xếp là 4. Hệ số 4 không phụ thuộc vào θ, nên không làm đổi MLE hay posterior của θ nếu dùng đúng cùng loại dữ liệu; nó có làm đổi xác suất tuyệt đối của sự kiện được hỏi.

Giữ dãy dữ liệu cố định và thử hai giá trị (tự tính): với $\theta = \frac{1}{2}$, likelihood là $\frac{1}{16} = 0{,}0625$; với $\theta = \frac{3}{4}$, likelihood là $\frac{27}{256} \approx 0{,}1055$. Dữ liệu này hợp với giá trị sau hơn theo mô hình, nhưng phép so sánh đó chưa cho xác suất rằng một giá trị tham số là đúng.

5.2 MLE và log-likelihood

Maximum likelihood estimate, viết tắt MLE, là một giá trị làm likelihood lớn nhất trong không gian tham số:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} p(D \mid \theta).$$

Ta thường lấy log tự nhiên vì nó biến tích thành tổng, và log là hàm tăng nên không đổi vị trí cực đại ở nơi likelihood dương. Đây là thuận tiện tính toán, không phải một giả định mới. Với k lần ngửa trong n lần độc lập có điều kiện:

$$\ell(\theta) = k \log \theta + (n-k) \log(1-\theta), \qquad 0 < \theta < 1.$$

Đạo hàm $\ell'(\theta) = \frac{k}{\theta} - \frac{n-k}{1-\theta}$. Cho bằng 0, ta có $k(1-\theta) = (n-k)\theta$, nên $\hat{\theta}_{\text{MLE}} = k/n$. Đạo hàm bậc hai âm ở bên trong khi có cả hai kết quả, do đó điểm dừng là cực đại. Với dãy của ta, $k = 3$, $n = 4$, vậy $\hat{\theta}_{\text{MLE}} = 3/4$ (tự tính).

Nếu mọi lần đều ngửa hoặc mọi lần đều sấp, nghiệm ở biên lần lượt là 1 hoặc 0; không được thế máy móc vào đạo hàm chứa log tại biên. Đây là lời nhắc thực tế: chỉ một quan sát ngửa đã làm MLE bằng 1, nhưng điều đó không chứng minh sấp là bất khả thi.

5.3 Likelihood không phải phân phối trên tham số

Với θ cố định, $p(D \mid \theta)$ là xác suất hoặc mật độ theo dữ liệu. Với D cố định, biểu thức này là một hàm theo θ; nó không bắt buộc có tổng hay tích phân theo θ bằng 1. Ta có thể nhân nó với một hằng số dương để vẽ cho dễ so sánh mà MLE không đổi, nhưng diện tích chuẩn hoá ấy **chưa làm nó thành posterior** nếu chưa chọn prior.

Cách đọc hình 2: trục ngang là các giá trị có thể của θ, trục dọc là chiều cao sau chuẩn hoá theo θ. Đường đỏ nét đứt là likelihood được chia cho diện tích của chính nó, có đỉnh ở $3/4$; đó là MLE. Đường xanh đậm là posterior sau khi nhân likelihood với prior Beta(2, 2), có đỉnh ở $2/3$; đó là MAP (tự tính). Việc đỉnh xanh ở bên trái đỉnh đỏ thể hiện tác động của prior trong ví dụ này, không phải quy luật rằng MAP luôn nhỏ hơn MLE.

6. Bayes cho tham số liên tục và MAP

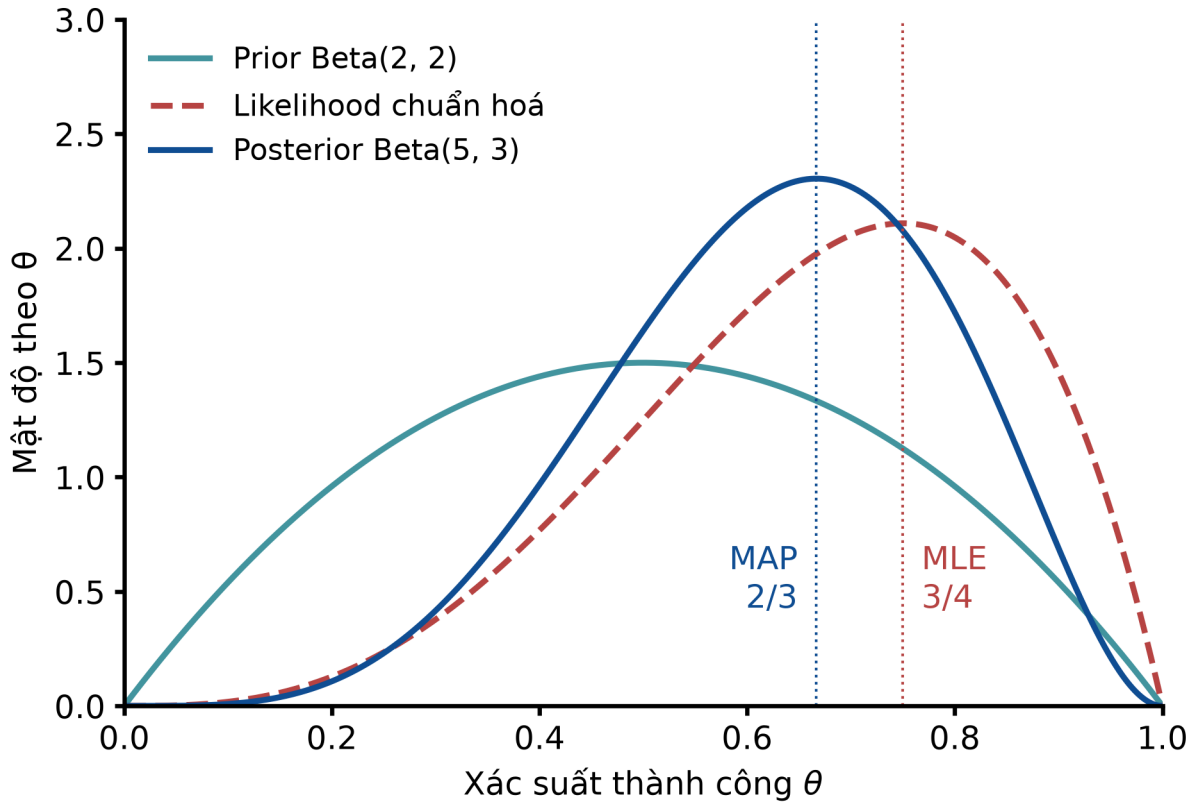
6.1 Prior, posterior và mẫu số

Với tham số liên tục, Bayes dùng mật độ:

$$p(\theta \mid D) = \frac{p(D \mid \theta) p(\theta)}{p(D)}, \qquad p(D) = \int p(D \mid t) p(t)\, dt.$$

$p(\theta)$ là **prior**, mô tả độ hợp lý của các giá trị tham số trước khi đưa D vào cập nhật. $p(D \mid \theta)$ là likelihood; $p(\theta \mid D)$ là **posterior**, mật độ sau cập nhật. $p(D)$ là likelihood biên hay evidence, trung bình likelihood theo prior, và làm diện tích posterior bằng 1. Khi tìm đỉnh theo θ với cùng dữ liệu và prior, $p(D)$ không đổi nên có thể bỏ khỏi phép tối ưu.

Điều quan trọng: prior không nhất thiết là ”ý kiến tuỳ tiện”. Nó là một phần của mô hình, có thể dựa trên kiến thức có trước hoặc được chọn để regularize. Tuy vậy, prior có thể sai hoặc ảnh hưởng mạnh khi dữ liệu ít. Cần nói rõ prior và kiểm tra mức nhạy của kết luận khi thay nó.



Hình 35: Tiên nghiệm, likelihood chuẩn hoá và hậu nghiệm cho ba ngửa một sấp (minh hoạ, tự tính)

6.2 Tính posterior Beta cho ví dụ đồng xu

Một họ prior thông dụng cho xác suất θ là Beta. Với $a, b > 0$, mật độ có dạng

$$p(\theta) = \frac{\theta^{a-1}(1-\theta)^{b-1}}{B(a,b)}, \qquad 0 < \theta < 1.$$

$B(a,b)$ là hằng số làm diện tích bằng 1; nó không phụ thuộc θ. Chọn prior Beta(2, 2), ta có $p(\theta) = 6\theta(1-\theta)$ (tự tính). Đường này cân quanh $1/2$, ưu tiên giá trị trung tâm hơn hai đầu, nhưng vẫn cho nhiều giá trị khác trọng lượng dương.

Bây giờ cập nhật bằng dãy ba ngửa một sấp, từng bước (tự tính):

1. Nhân likelihood với prior: $\theta^3(1-\theta) \cdot 6\theta(1-\theta) = 6\theta^4(1-\theta)^2$.
2. Bỏ hệ số 6 để nhận ra hình dạng: $\theta^4(1-\theta)^2$ là nhân của mật độ Beta(5, 3), vì số mũ lần lượt là $5-1$ và $3-1$.
3. Tích phân nhân đó trên $[0,1]$ bằng $1/105$, nên mật độ hậu nghiệm là $105\theta^4(1-\theta)^2$. Evidence của **dãy có thứ tự cụ thể** là $6/105 = 2/35$.
4. Trung bình posterior là $5/(5+3) = 5/8 = 0{,}625$. Đây cũng là xác suất dự đoán mặt ngửa ở lần kế tiếp sau khi lấy trung bình qua mọi giá trị θ.

Họ Beta được gọi là **liên hợp** với Bernoulli vì sau cập nhật posterior vẫn thuộc họ Beta. Nói chung, a, b chuyển thành $a + k, b + n - k$ khi có k thành công trong n lần độc lập có điều kiện. Không cần một họ liên hợp mới áp dụng được Bayes, chỉ là không phải lúc nào tích phân cũng có công thức gọn.

6.3 MAP là đỉnh hậu nghiệm

Maximum a posteriori estimate, viết tắt MAP, chọn một điểm làm mật độ posterior lớn nhất:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} p(\theta \mid D) = \arg\max_{\theta} \big[\log p(D \mid \theta) + \log p(\theta)\big].$$

Đẳng thức sau dùng khi log xác định và ta tối ưu trên miền phù hợp. Nó cho thấy MAP cân bằng độ hợp dữ liệu với log-prior. Với Beta(5, 3), log posterior bỏ hằng số là $4\log\theta + 2\log(1-\theta)$. Đạo hàm bằng 0 cho $4/\theta = 2/(1-\theta)$, nên $\hat{\theta}_{\text{MAP}} = 2/3$ (tự tính). Đạo hàm bậc hai âm ở miền trong, nên đây là cực đại duy nhất.

Phân biệt ba kết quả của cùng ví dụ: MLE là $3/4$, MAP là $2/3$, trung bình posterior là $5/8$ (tự tính). MLE nhìn đỉnh likelihood; MAP nhìn đỉnh posterior; trung bình posterior tính toàn bộ khối xác suất. Điểm nào hữu ích phụ thuộc mục tiêu, và không điểm nào thay thế được toàn bộ phân phối hậu nghiệm.

7. Dự đoán sau cập nhật và dữ liệu tăng lên

7.1 Lấy trung bình thay vì chỉ chọn một tham số

Phân phối dự đoán hậu nghiệm cho kết quả mới $x_{\text{mới}}$ lấy trung bình dự đoán theo posterior. Nói tổng quát, dự đoán theo từng giá trị θ vẫn có thể phụ thuộc dữ liệu đã thấy:

$$p(x_{\text{mới}} \mid D) = \int p(x_{\text{mới}} \mid \theta, D) p(\theta \mid D)\, d\theta.$$

Nếu $x_{\text{mới}}$ độc lập với D khi đã biết θ, như các lần tung đồng xu trong mô hình của mục 5, thì $p(x_{\text{mới}} \mid \theta, D) = p(x_{\text{mới}} \mid \theta)$. Khi đó công thức rút gọn thành

$$p(x_{\text{mới}} \mid D) = \int p(x_{\text{mới}} \mid \theta) p(\theta \mid D)\, d\theta.$$

Trong ví dụ đồng xu, $p(x_{\text{mới}} = 1 \mid D) = E[\theta \mid D] = 5/8$ (tự tính). Ta cũng có thể **cắm điểm** MLE hoặc MAP vào mô hình và nhận các dự đoán khác: lần lượt là $3/4$ và $2/3$. Ba cách trả lời khác nhau vì ba phương pháp giữ lại lượng thông tin khác nhau, không phải vì một phép cộng bị sai.

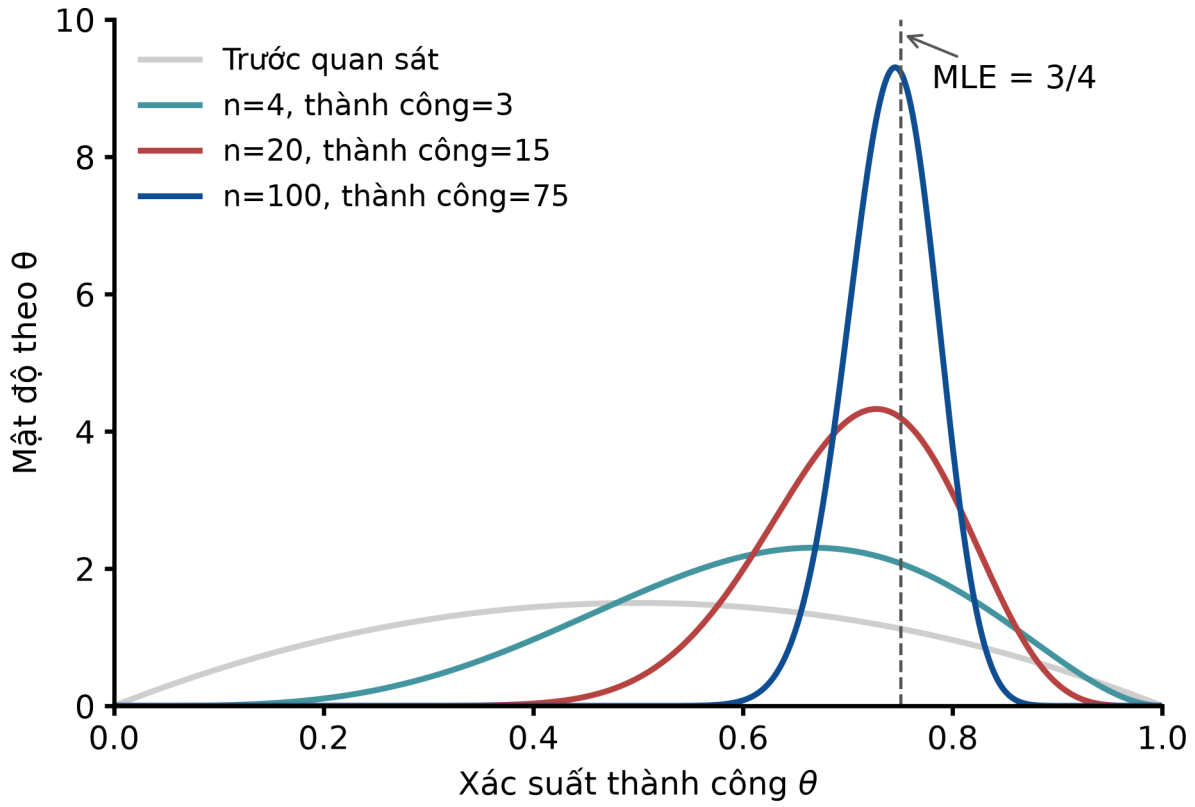
Với hai lần ngửa mới liên tiếp, tích phân còn phản ánh sự không chắc chắn chung về cùng một θ. Với posterior Beta(5, 3), xác suất là $(5/8)(6/9) = 5/12 \approx 0{,}4167$. Nếu cắm trung bình $5/8$ rồi coi hai lần độc lập với tham số cố định đó, kết quả là $(5/8)^2 = 25/64 \approx 0{,}3906$ (tự tính). Hai phép tính khác nhau: hai lần độc lập **khi biết** θ, nhưng nhìn chung không độc lập sau khi đã lấy trung bình qua θ.

7.2 Thêm dữ liệu thay đổi hình dạng hậu nghiệm

Tiếp tục với prior Beta(2, 2), xét ba tập dữ liệu có cùng tỉ lệ thành công $3/4$: 3 trên 4, 15 trên 20 và 75 trên 100. Posterior lần lượt là Beta(5, 3), Beta(17, 7) và Beta(77, 27). Các trung bình tương ứng là $5/8 = 0{,}625$, $17/24 \approx 0{,}7083$ và $77/104 \approx 0{,}7404$ (tự tính).

Cách đọc hình 3: đường xám là prior, các đường còn lại là posterior sau số quan sát ghi trong chú thích. Khi số quan sát tăng trong ví dụ này, đường hẹp và cao hơn gần $3/4$, nghĩa là khối xác suất tập trung hơn. Diện tích dưới mỗi đường đều bằng 1, nên chiều cao lớn không phải xác suất của một giá trị chính xác. Hình chỉ minh hoạ trường hợp mô hình phù hợp và cùng tỉ lệ mẫu; nó không bảo đảm mọi posterior ngoài thực tế đều tập trung quanh chân lý.

Prior không nhất thiết ”biến mất” trong mọi bài toán lớn. Nếu mô hình sai, tham số không được dữ liệu nhận diện, hoặc prior loại bỏ vùng chứa giá trị đúng, thêm dữ liệu không sửa được mọi vấn đề. Ngoài ra, xác suất hậu nghiệm vẫn là kết luận có điều kiện theo mô hình và prior đã chọn.



Hình 36: Hậu nghiệm hẹp dần khi số lần quan sát tăng cùng tỉ lệ thành công (minh hoạ, tự tính)

8. Những chi tiết thường bị bỏ qua

8.1 MLE của trung bình Gaussian

Giả sử $x_1, \dots, x_n$ độc lập có cùng phân phối Gaussian với trung bình μ chưa biết và phương sai σ^2 cố định, dương. Sau khi bỏ các hằng số không phụ thuộc μ, log-likelihood tỉ lệ với $-\sum_i (x_i - \mu)^2$. Cực đại likelihood vì thế tương đương cực tiểu tổng bình phương sai số, cho

$$\hat{\mu}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

Ví dụ dữ liệu 2, 4, 6 cho $\hat{\mu} = 4$ và tổng bình phương sai số là $(-2)^2 + 0^2 + 2^2 = 8$ (tự tính). Nếu phương sai cũng chưa biết và dùng cùng tập dữ liệu, MLE của phương sai là $8/3$, trong khi ước lượng phương sai mẫu không chệch dùng mẫu số $n - 1$ cho kết quả 4 (tự tính). Hai mục tiêu khác nhau, không có mâu thuẫn. Nếu tất cả quan sát bằng nhau, likelihood Gaussian với phương sai tự do có thể không đạt cực đại hữu hạn khi phương sai tiến tới 0.

8.2 MAP và regularization

Ví dụ hồi quy Gaussian với nhiễu có phương sai cố định và prior Gaussian tâm 0 trên vector trọng số w. Bỏ các hằng số không phụ thuộc w, âm log posterior có dạng

$$\frac{1}{2\sigma^2} \sum_i (y_i - \hat{y}_i)^2 + \frac{1}{2\tau^2} \|w\|_2^2.$$

Số hạng thứ nhất đo sai số dữ liệu; số hạng thứ hai đến từ prior $w \sim \mathcal{N}(0, \tau^2 I)$ và phạt trọng số lớn. Vì thế cực tiểu mất mát bình phương cộng phạt L2 tương ứng với một MAP dưới những giả định và hệ số cụ thể này. Khi mã dùng trung bình thay vì tổng loss, hệ số phạt phải đổi theo cùng quy ước. Không thể nói mọi cách weight decay trong mọi optimizer đều là phép MAP này.

8.3 Phụ thuộc tham số hoá và cách biểu diễn

MLE có tính tương thích với phép đổi tham số một-một: đổi tham số sau khi chọn MLE cho nghiệm MLE tương ứng, nếu xử lý miền và các nghiệm không duy nhất đúng cách. **MAP mật độ nói chung không có tính chất đó.** Đổi biến liên tục làm mật độ nhân với hệ số Jacobian; đỉnh mật độ có thể dịch chuyển. Điều này không làm posterior thay đổi về xác suất sự kiện, chỉ nhắc rằng ”đỉnh” phụ thuộc hệ tọa độ và thước đo.

Ngoài ra, giả sử độc lập có điều kiện là một phần quan trọng của các tích ở trên. Nếu các quan sát lặp tương quan mạnh mà ta coi là độc lập, likelihood có thể quá tự tin và posterior quá hẹp. Chỉ ghi thêm nhiều dòng dữ liệu không tự tạo ra nhiều thông tin độc lập.

9. Nối sang AI

(a) Phân loại và dự đoán token. Một mạng xuất xác suất có điều kiện cho nhãn hoặc token tiếp theo. Cực đại tổng log-likelihood của dữ liệu huấn luyện tương đương cực tiểu âm log-likelihood, cơ sở của cross-entropy với nhãn quan sát. Cách nối entropy, cross-entropy và KL được làm kỹ ở chương nền 11. Tối ưu trên tập huấn luyện không tự bảo đảm xác suất hiệu chỉnh tốt trên dữ liệu mới.

(b) Suy luận biến phân. Trong math 08, posterior chính xác thường khó tính. Ta chọn một họ phân phối dễ xử lý, rồi tối ưu một thành viên để xấp xỉ posterior, thay vì chỉ lấy một điểm MAP. ELBO giúp thực hiện việc này mà không cần tính trực tiếp evidence khó.

(c) VAE và biến ẩn. Math 09 dùng prior cho biến ẩn và một mô hình tạo dữ liệu có điều kiện theo biến ẩn. Mạng suy luận cho một posterior xấp xỉ đối với mỗi dữ liệu đầu vào; không nên nhầm posterior này với likelihood của dữ liệu. Chất lượng xấp xỉ còn phụ thuộc họ phân phối và cách tối ưu.

(d) Lấy mẫu posterior. Math 10 thảo luận Hamiltonian Monte Carlo, còn math 11 xét mối liên hệ giữa SGD và suy luận Bayes xấp xỉ. Lấy mẫu tìm cách biểu diễn nhiều khả năng của tham số, trong khi MAP chỉ giữ một điểm. Một tập mẫu có ích chỉ khi phương pháp thực sự khám phá được các vùng posterior liên quan.

(e) Ra quyết định và khám phá. Ở math 35, Thompson sampling lấy mẫu từ posterior rồi chọn hành động theo mẫu ấy. Độ không chắc chắn không chỉ là thanh sai số trang trí: nó ảnh hưởng cách cân bằng khám phá với khai thác. Một prior hay likelihood sai cũng có thể làm chiến lược này tự tin sai.

10. Sai lầm hay gặp

- **Đảo điều kiện.** $P(B \mid A)$ và $P(A \mid B)$ có mẫu số khác nhau. Hãy viết lại bằng lời và vẽ hai nhóm như hình 1 trước khi thay số.
- **Gọi likelihood là posterior.** $p(D \mid \theta)$ là hàm theo tham số khi dữ liệu cố định; posterior còn cần prior và phép chuẩn hoá. Likelihood được chuẩn hoá để vẽ cũng không tự bao gồm prior của bài toán.
- **Quên mẫu số hoặc trường hợp còn lại trong Bayes.** Phải cộng mọi đường có thể sinh ra bằng chứng. Không thể chỉ chia cho likelihood của giả thuyết mình thích.
- **Coi MLE hay MAP là mức chắc chắn.** Một điểm không cho biết posterior rộng hay hẹp; muốn nói về bất định cần giữ phân phối hoặc tóm tắt phù hợp.
- **Dùng công thức đỉnh bên trong ở biên.** Nếu không có cả hai loại kết quả, hoặc tham số Beta không làm đỉnh nằm trong khoảng mở, phải kiểm tra biên và sự tồn tại của cực đại.
- **Quên điều kiện mô hình và prior.** Quan sát phụ thuộc, prior loại trừ vùng đúng, hoặc mô hình sai có thể làm kết luận quá tự tin, dù phép biến đổi đại số hoàn toàn đúng.

11. Tóm tắt công thức

Khái niệm	Công thức	Cách đọc
Bayes cho sự kiện	$P(A \mid B) = P(B \mid A)P(A)/P(B)$	đổi chiều điều kiện nếu $P(B) > 0$
Evidence	$p(D) = \int p(D \mid \theta)p(\theta)\, d\theta$	trung bình likelihood theo prior
Posterior	$p(\theta \mid D) \propto p(D \mid \theta)p(\theta)$	nhân rồi chuẩn hoá theo tham số
MLE	$\hat{\theta}_{\text{MLE}} = \arg\max_\theta p(D \mid \theta)$	chọn đỉnh likelihood
MAP	$\hat{\theta}_{\text{MAP}} = \arg\max_\theta p(\theta \mid D)$	chọn đỉnh mật độ posterior
Beta và Bernoulli	$\text{Beta}(a, b) \to \text{Beta}(a + k, b + n - k)$	thêm thành công và thất bại
Trung bình Beta	$E[\theta] = a/(a + b)$	trung bình theo toàn phân phối
Posterior dự đoán	$p(x \mid D) = \int p(x \mid \theta, D)p(\theta \mid D)\, d\theta$	trung bình dự đoán qua posterior

12. Bài tập tự tính

Hãy thử từng bài trước khi xem đáp án. Các phép tính trong phần này do ta tự làm và đã kiểm bằng numpy.

Bài 1 (dễ). Có hai giả thuyết H_1, H_2 với prior lần lượt là $0{,}4$ và $0{,}6$. Xác suất thấy dữ liệu D theo hai giả thuyết lần lượt là $0{,}5$ và $0{,}2$. Tính $P(H_1 \mid D)$.

Đáp án. Hai trọng lượng chưa chuẩn hoá là $0{,}4 \cdot 0{,}5 = 0{,}2$ và $0{,}6 \cdot 0{,}2 = 0{,}12$. Mẫu số là $0{,}32$, nên hậu nghiệm cần tìm là $0{,}2/0{,}32 = 5/8 = 0{,}625$ (tự tính). Hậu nghiệm còn lại bằng $3/8$, và tổng bằng 1.

Bài 2 (dễ vừa). Tung đồng xu độc lập có điều kiện 5 lần, thấy 4 ngửa và 1 sấp. Tìm MLE của xác suất ngửa và giải thích tại sao không thể nói ngay xác suất tham số đó đúng là 80%.

Đáp án. Likelihood tỉ lệ với $\theta^4(1 - \theta)$; đỉnh ở $\hat{\theta}_{\text{MLE}} = 4/5 = 0{,}8$ (tự tính). Đây là giá trị tham số làm dữ liệu hợp lý nhất trong mô hình, không phải xác suất của chính giả thuyết. Để nói về phân phối của tham số cần thêm prior và tính posterior.

Bài 3 (vừa). Prior của xác suất ngửa là Beta(2, 3). Sau đó thấy 3 ngửa và 1 sấp độc lập có điều kiện. Tìm posterior, trung bình posterior và MAP.

Đáp án. Cộng số ngửa vào tham số thứ nhất, số sấp vào thứ hai, được Beta(5, 4). Trung bình bằng $5/9 \approx 0{,}5556$. Hai tham số đều lớn hơn 1 nên đỉnh nằm bên trong: MAP bằng $(5-1)/(5+4-2) = 4/7 \approx 0{,}5714$ (tự tính). MLE của dữ liệu riêng là $3/4$, cho thấy ba tóm tắt trả lời ba câu hỏi khác nhau.

Bài 4 (vừa khó). Các số 1, 2, 6 là ba quan sát độc lập Gaussian có cùng trung bình và phương sai chưa biết. Tìm MLE của trung bình và phương sai.

Đáp án. Trung bình mẫu là $(1 + 2 + 6)/3 = 3$. Bình phương sai lệch là 4, 1, 9, tổng bằng 14. Do đó MLE phương sai là $14/3 \approx 4{,}6667$ (tự tính). Mẫu số là 3 vì bài hỏi MLE, không phải ước lượng phương sai không chệch.

Bài 5 (khó). Prior của H_1 bằng $1/4$, của H_2 bằng $3/4$. Bằng chứng D có likelihood lần lượt $4/5$ và $1/5$. Tính hậu nghiệm của H_1 bằng tỉ số, rồi kiểm bằng công thức Bayes.

Đáp án. Tỉ số prior H_1 so với H_2 là $(1/4)/(3/4) = 1/3$. Hệ số Bayes là $(4/5)/(1/5) = 4$, nên tỉ số posterior bằng $4/3$. Hai xác suất cộng bằng 1, vì thế $P(H_1 \mid D) = 4/(4 + 3) = 4/7$ và $P(H_2 \mid D) = 3/7$ (tự tính). Kiểm trực tiếp: trọng lượng chưa chuẩn hoá là $1/5$ và $3/20$; chia $1/5$ cho tổng $7/20$ cũng cho $4/7$.

Tài liệu tham khảo nền: Bishop, Pattern Recognition and Machine Learning; Murphy, Probabilistic Machine Learning; ghi chú math 08, math 09 và math 35.

Nền tảng 11

Entropy, cross-entropy và KL: đo bất định và cái giá của dự đoán sai

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: xác suất rời rạc, kỳ vọng và log ở chương nền 08; likelihood và Bayes ở chương nền 10

Dùng ở: math 07 (hình học thông tin), math 08 (suy luận biến phân), math 09 (VAE), math 14 (information bottleneck), math 15 (softmax) và math 29 (diffusion)

Đây là chương học từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần ba khái niệm này

Entropy đo lượng bất định trung bình của một phân phối; cross-entropy đo chi phí trung bình khi dự đoán dữ liệu bằng một phân phối; KL đo phần chênh lệch giữa hai đại lượng đó. Ba câu này cần được hiểu với cùng một không gian kết quả, cùng cơ số log và cùng phân phối dùng để lấy trung bình.

Giả sử mô hình dự đoán nhãn cho ảnh. Chỉ hỏi "có chọn đúng nhãn nhiều nhất không" bỏ qua độ tự tin: hai mô hình có thể đều chọn sai nhưng một mô hình đã gán xác suất gần 0 cho nhãn đúng. Cross-entropy phạt mạnh trường hợp tự tin sai. KL dùng để so sánh phân phối xấp xỉ với phân phối đích trong nhiều mô hình; entropy cho ta một mốc đo bất định vốn có của nguồn.

Đây không phải ba thước đo thay thế mọi tiêu chí. Một loss thấp trên dữ liệu huấn luyện không tự bảo đảm dự đoán tốt ở dữ liệu mới, và entropy cao không cho biết nguyên nhân của sự không chắc chắn. Ta bắt đầu với một đồng xu rồi tính tay một ví dụ ba kết quả, sau đó nối đến phân loại, VAE và information bottleneck.

2. Nhắc nhanh xác suất, kỳ vọng và log

Một phân phối rời rạc $P = (p_1, \dots, p_m)$ cho mỗi kết quả một xác suất không âm, với $\sum_i p_i = 1$. Nếu một đại lượng nhận giá trị f_i khi kết quả là i, trung bình theo P là

$$E_P[f(X)] = \sum_i p_i f_i.$$

Ví dụ (tự tính): nếu một trò chơi trả 2 điểm với xác suất $3/4$ và 0 điểm với xác suất $1/4$, điểm trung bình là $(3/4)\cdot 2 + (1/4)\cdot 0 = 1{,}5$. Không cần mọi lần chơi đều trả đúng 1,5 điểm; đây là trung bình có trọng số theo phân phối.

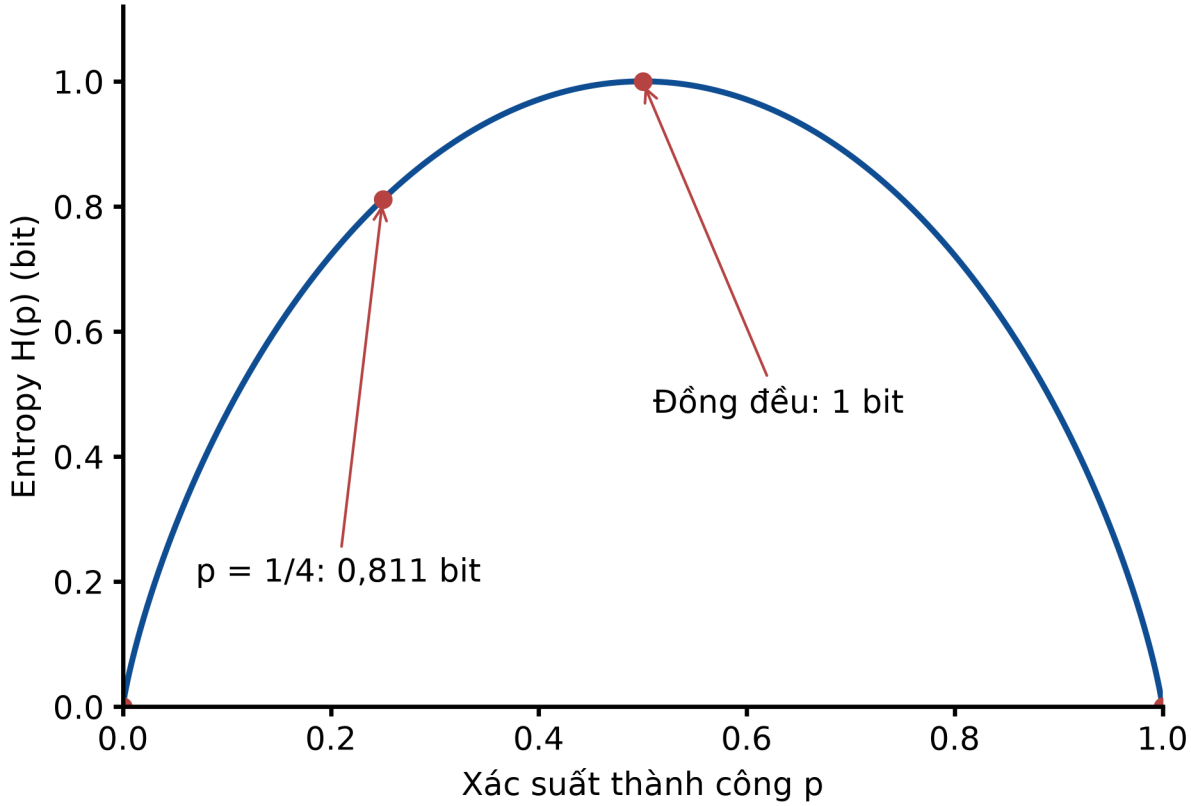
Log biến tích thành tổng: $\log(ab) = \log a + \log b$ khi $a, b > 0$. Đặc biệt, $\log_2(1/2) = -1$ và $\log_2(1/4) = -2$ (tự tính). Vì log của xác suất nhỏ hơn hoặc bằng 1 là không dương, dấu trừ trước log làm mức bất ngờ không âm. Trong chương, công thức thông tin dùng $\log_2$ và đơn vị **bit**, trừ khi nêu rõ log tự nhiên ln và đơn vị **nat**.

Phân biệt hai việc: lấy log của xác suất của **một kết quả**, và lấy trung bình của đại lượng đó qua **mọi kết quả**. Entropy thuộc loại thứ hai. Khi một kết quả có xác suất 0, quy ước $0 \log 0 = 0$ là viết gọn của giới hạn $\lim_{u \to 0^+} u \log u = 0$, không phải coi $\log 0$ là số hữu hạn.

3. Trực giác hình: đồng xu nào khó đoán hơn?

Nếu một đồng xu luôn ngửa, ta biết kết quả trước khi tung; không có bất định về kết quả của phép tung ấy. Nếu ngửa và sấp có cùng xác suất, ta không có lựa chọn nào có lợi thế theo mô hình. Giữa hai trường hợp là

những đồng xu lệch ít hoặc nhiều.



Hình 37: Entropy của đồng xu thay đổi theo xác suất mặt ngửa (minh hoạ, tự tính)

Cách đọc hình 1: trục ngang là xác suất ngửa, trục dọc là entropy tính bằng bit. Ở hai đầu, kết quả là chắc chắn nên entropy bằng 0; chính giữa, hai kết quả đồng khả năng và entropy đạt 1 bit. Tại xác suất ngửa $1/4$, entropy khoảng $0{,}811$ bit (tự tính), nhỏ hơn 1 vì mặt còn lại dễ xuất hiện hơn. Hình đối xứng qua $1/2$ vì đổi tên ngửa với sấp không đổi lượng bất định.

Entropy nói về phân phối trước khi quan sát kết quả, không phải cảm giác bất ngờ của riêng kết quả đã xảy ra. Đồng xu lệch vẫn có thể cho ra một kết quả hiếm và bất ngờ, dù entropy trung bình của nó thấp.

4. Lượng tin và entropy chính thức

4.1 Bất ngờ của một kết quả

Nếu một kết quả có xác suất $p > 0$, ta gán cho nó **lượng tin riêng** (self-information)

$$I(x) = -\log_2 p(x).$$

Một kết quả chắc chắn có lượng tin 0 bit. Kết quả xác suất $1/2$ có lượng tin 1 bit; kết quả xác suất $1/4$ có lượng tin 2 bit (tự tính). Kết quả càng hiếm thì lượng tin khi nó xảy ra càng lớn. Nếu hai sự kiện độc lập cùng xảy ra, xác suất của chúng nhân nhau nên lượng tin cộng lại.

Đây là một thước đo theo mô hình xác suất, không đo mức quan trọng, độ hữu ích hay giá trị con người gán cho thông điệp. Một chuỗi ký tự ngẫu nhiên có thể khó dự đoán mà chẳng mang ý nghĩa thực dụng. Lượng tin của kết quả có xác suất đúng bằng 0 được xem là vô hạn, nhưng sự kiện ấy không xuất hiện theo chính mô hình đã đặt.

4.2 Trung bình theo phân phối

Entropy Shannon của phân phối rời rạc P là kỳ vọng lượng tin riêng:

$$H(P) = E_P[-\log_2 p(X)] = -\sum_i p_i \log_2 p_i.$$

Chú ý trọng số p_i đứng ở ngoài log. Một kết quả rất hiếm có lượng tin lớn, nhưng chỉ đóng góp theo xác suất rất nhỏ của nó. Với đồng xu có xác suất ngửa p, công thức trở thành

$$H(p) = -p\log_2 p - (1-p)\log_2(1-p).$$

Tại $p = 1/2$, mỗi số hạng đóng góp $1/2$ bit, tổng là 1 bit. Tại $p = 1/4$, $H = -(1/4)(-2) - (3/4)\log_2(3/4) \approx 0{,}8113$ bit (tự tính). Hai đầu được hiểu theo quy ước giới hạn ở mục 2.

Với m kết quả và phân phối đều, $H(P) = \log_2 m$. Đó là entropy lớn nhất trên không gian có đúng m kết quả khả dĩ; mức 0 đạt được khi một kết quả có xác suất 1. Điều này không có nghĩa rằng entropy của mọi phân phối trên các không gian có kích thước khác nhau đều nằm dưới 1 bit.

4.3 Bit, nat và cách đọc theo mã hoá

Nếu dùng log tự nhiên, $H_{\text{nat}} = H_{\text{bit}} \ln 2$; ngược lại chia cho $\ln 2$ để đổi nat thành bit. Chẳng hạn một đồng xu cân bằng có entropy 1 bit bằng $\ln 2 \approx 0{,}6931$ nat (tự tính). Cơ số thay độ lớn số đo, không thay thứ tự các mô hình khi cơ số được giữ nhất quán.

Một trực giác mã hoá: để mô tả sự kiện hiếm, ta dành một đoạn mã dài hơn; sự kiện phổ biến được mô tả ngắn hơn. Độ dài lý tưởng $-\log_2 p_i$ có trung bình là entropy. Một mã nhị phân tiền tố cho từng ký hiệu thường cần độ dài nguyên, nên không phải lúc nào entropy cũng chính là chiều dài của một từ mã thật; mã hoá các khối dài cho phép tiến gần giới hạn trung bình trong điều kiện thích hợp. Ở đây ”bit trung bình” là đơn vị đo, không có nghĩa mỗi quan sát riêng luôn được biểu diễn bằng số bit phân số.

5. Ví dụ tay: ba kết quả A, B, C

Giả sử dữ liệu có phân phối $P = (1/2, 1/4, 1/4)$ trên ba kết quả A, B, C. Một mô hình dự đoán $Q = (1/4, 1/2, 1/4)$ theo đúng thứ tự ấy. Mô hình đã đổi trọng lượng của A và B. Ta sẽ tính entropy của dữ liệu, cross-entropy của dự đoán, rồi KL; tất cả các phép tính dưới đây (tự tính) đã được kiểm bằng numpy.

5.1 Entropy của P

Bước 1, tính lượng tin khi biết phân phối thật: A có $-\log_2(1/2) = 1$ bit; B và C mỗi kết quả có $-\log_2(1/4) = 2$ bit.

Bước 2, lấy trung bình **theo P**, vì đó là cách dữ liệu xuất hiện:

$$H(P) = \frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + \frac{1}{4}\cdot 2 = 1{,}5 \text{ bit}.$$

Tuy B và C bất ngờ hơn từng lần xuất hiện, A chiếm một nửa số lần trong mô hình. Entropy là trung bình của tất cả các trường hợp, không phải giá trị lớn nhất của chúng.

5.2 Cross-entropy khi dùng Q

Bây giờ dùng xác suất dự đoán của Q để chấm mỗi kết quả, nhưng vẫn lấy trung bình theo P vì dữ liệu đến từ P. A bị gán xác suất $1/4$, nên chi phí là 2 bit; B được gán $1/2$, nên chi phí là 1 bit; C có chi phí 2 bit như trước. Vậy

$$H(P,Q) = -\sum_i p_i \log_2 q_i = \frac{1}{2}\cdot 2 + \frac{1}{4}\cdot 1 + \frac{1}{4}\cdot 2 = 1{,}75 \text{ bit}.$$

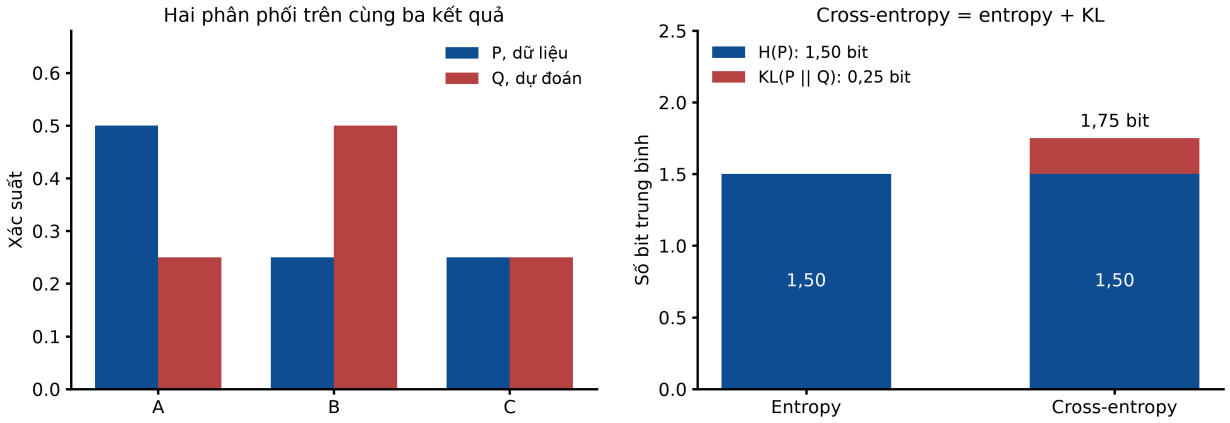
Dự đoán Q cho A quá ít xác suất, mà A xuất hiện tương đối thường. Tổng chi phí vì vậy lớn hơn entropy của P là $0{,}25$ bit. Đây là kết quả cho các phân phối giả định ở ví dụ, không phải một con số đo trên paper hay dữ liệu thật.

5.3 KL: phân tích phần chênh lệch

Lấy chênh lệch hai công thức:

$$H(P,Q) - H(P) = \sum_i p_i \log_2 \frac{p_i}{q_i} = D_{\mathrm{KL}}(P\|Q).$$

Cộng từng hạng: A cho $(1/2)\log_2 2 = 1/2$ bit; B cho $(1/4)\log_2(1/2) = -1/4$ bit; C cho 0. Tổng KL là $1/2 - 1/4 = 1/4$ bit. Một số hạng riêng lẻ có thể âm, vì Q gán xác suất cho B lớn hơn P; điều không âm là **tổng** khi tính đúng mọi kết quả.



Hình 38: Hai phân phối ba kết quả và phân tích cross-entropy thành entropy cộng KL (minh hoạ, tự tính)

Cách đọc hình 2: panel trái so xác suất P màu xanh và Q màu đỏ cho cùng từng kết quả A, B, C. Ở panel phải, cột entropy cao 1,50 bit và cột cross-entropy cao 1,75 bit; phần đỏ thêm vào biểu diễn KL bằng 0,25 bit. Độ cao được lấy trung bình theo P, không phải theo Q. Q bằng P thì phần chênh sẽ bằng 0, nhưng entropy của P vẫn có thể lớn hơn 0.

6. Cross-entropy: phân phối dữ liệu và phân phối dự đoán

6.1 Định nghĩa và ý nghĩa

Với P và Q trên cùng một tập kết quả hữu hạn, cross-entropy của P đối với Q là

$$H(P,Q) = E_P[-\log_2 q(X)] = -\sum_i p_i \log_2 q_i.$$

Chữ ”cross” gợi hai phân phối tham gia: P quyết định trọng số trong kỳ vọng, Q cho điểm bằng âm log xác suất dự đoán. Nếu $q_i = 0$ ở nơi $p_i > 0$, cross-entropy bằng vô hạn vì mô hình tuyên bố một kết quả có thể xảy ra là bất khả thi. Nếu $p_i = 0$, đóng góp tương ứng được quy ước bằng 0, kể cả khi $q_i = 0$.

Nếu P cố định và Q có thể là mọi phân phối trên cùng không gian, $H(P, Q)$ nhỏ nhất khi $Q = P$. Khi chỉ được chọn Q trong một họ mô hình hạn chế, Q tốt nhất trong họ có thể vẫn khác P. Khi P cũng thay đổi, chẳng hạn so các tập dữ liệu có độ khó khác nhau, giá trị cross-entropy thô thay đổi cả do entropy nền; không thể quy toàn bộ chênh lệch cho sai lệch mô hình.

6.2 Một nhãn quan sát là phân phối one-hot

Trong phân loại có m lớp, nhãn đúng y có thể viết thành vector t với $t_y = 1$, các thành phần còn lại bằng 0. Nếu mô hình dự đoán xác suất $q_1, \ldots, q_m$, loss của một mẫu là

$$L = -\sum_{i=1}^{m} t_i \ln q_i = -\ln q_y.$$

Ở đây dùng log tự nhiên nên đơn vị là nat. Ví dụ (tự tính): Q dự đoán $(0{,}7, 0{,}2, 0{,}1)$ và nhãn đúng là lớp thứ hai; loss bằng $-\ln 0{,}2 \approx 1{,}6094$ nat, hay $-\log_2 0{,}2 \approx 2{,}3219$ bit. Nếu nhãn đúng là lớp thứ nhất với cùng Q, loss là $-\ln 0{,}7 \approx 0{,}3567$ nat. Mô hình được chấm theo xác suất nó gán cho nhãn thực, không chỉ theo chỉ số có xác suất lớn nhất.

Trung bình loss trên tập gồm n mẫu là $-(1/n)\sum_j \ln q(y_j \mid x_j)$. Với cùng dữ liệu và mô hình xác suất có điều kiện, cực tiểu loss trung bình này tương đương cực đại log-likelihood có điều kiện ở chương nền 10. Ký hiệu one-hot ở đây là nhãn của **một quan sát**, còn P trong công thức tổng quát có thể là phân phối thật hay phân phối thực nghiệm trên nhiều quan sát.

6.3 Softmax và gradient

Nếu mạng đưa ra các điểm số thực z_i, softmax biến chúng thành xác suất:

$$q_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Với nhãn one-hot và log tự nhiên, đạo hàm loss theo logit z_i là $q_i - t_i$. Ví dụ $Q = (0{,}7, 0{,}2, 0{,}1)$ và nhãn đúng là lớp thứ hai cho gradient theo logits bằng $(0{,}7, -0{,}8, 0{,}1)$ (tự tính). Một bước nhỏ ngược gradient trực tiếp trên logits sẽ tăng logit lớp đúng và giảm các logit còn lại. Với tham số của mạng, gradient còn phải đi qua chain rule; không thể kết luận từng trọng số riêng luôn thay đổi theo chiều cố định ấy.

Để tránh tràn số, thư viện thường tính log-softmax bằng log-sum-exp ổn định thay vì tính e^{z_i} trực tiếp rồi lấy log. Trừ cùng hằng số khỏi mọi logit không đổi softmax, và thường chọn logit lớn nhất làm hằng số. Xem math 15 cho bối cảnh tính softmax hiệu quả và chương nền 12 cho số học máy tính.

7. KL divergence: vì sao có hướng

7.1 Định nghĩa và điều kiện có nghĩa

KL divergence từ P đến Q, đọc theo đúng thứ tự ký hiệu, là

$$D_{\mathrm{KL}}(P\|Q) = \sum_i p_i \log_2 \frac{p_i}{q_i} = E_P\left[\log_2 \frac{p(X)}{q(X)}\right].$$

Tổng lấy theo P, không lấy theo Q. Nếu P cho một kết quả trọng lượng dương còn Q cho nó 0, KL bằng vô hạn. Nếu $p_i = 0$, số hạng ấy bằng 0 theo quy ước giới hạn. Với phân phối hữu hạn và điều kiện hỗ trợ tương thích, KL không âm và bằng 0 khi hai phân phối bằng nhau.

Ý nghĩa mã hoá lý tưởng: entropy là chi phí trung bình tối thiểu khi biết P; cross-entropy là chi phí lý tưởng khi gán độ dài theo Q nhưng nguồn thật là P; KL là số bit phụ trung bình. Đây là chi phí **có hướng** vì vai trò của nguồn và bộ dự đoán là khác nhau.

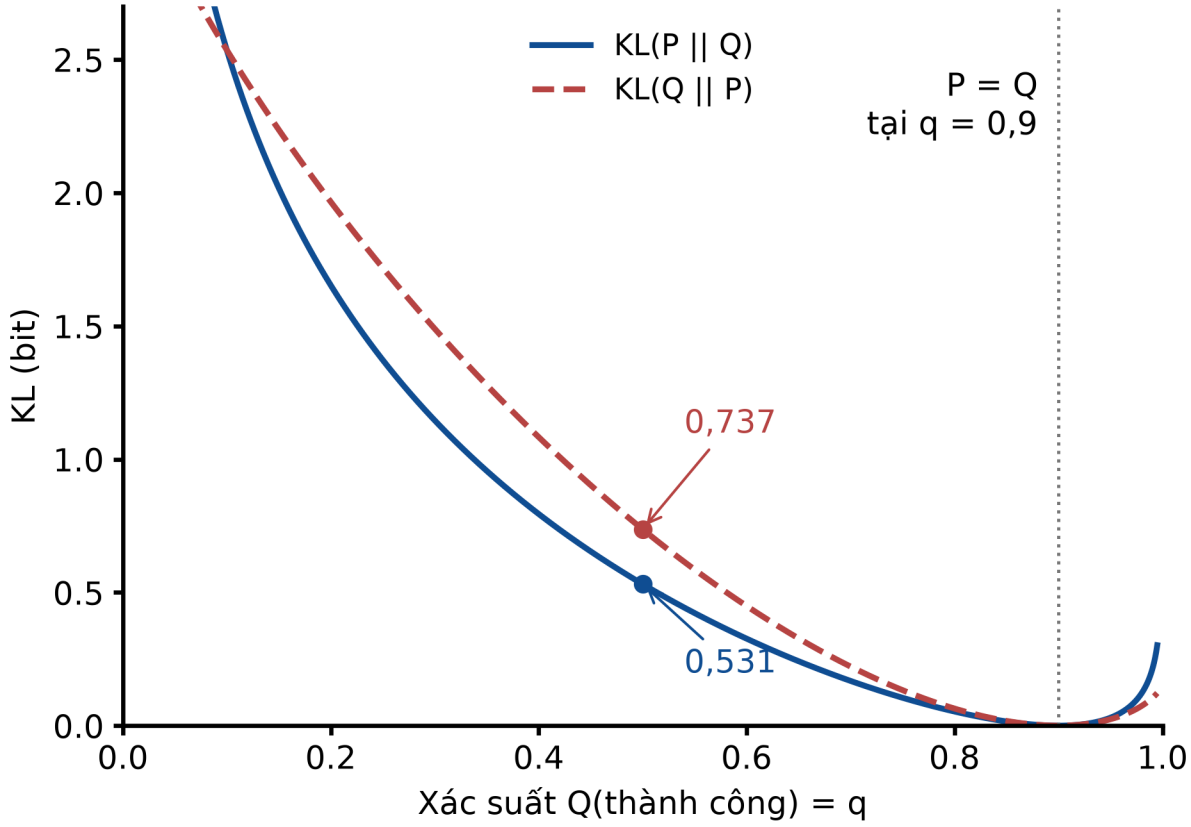
7.2 Không đối xứng, không phải khoảng cách thông thường

Đổi chỗ P và Q thì cả tỉ số bên trong log lẫn phân phối dùng để lấy trung bình đều đổi. Xét P có xác suất thành công $0{,}9$, Q có xác suất thành công $0{,}5$; xác suất thất bại lần lượt là $0{,}1$ và $0{,}5$. Tính từng chiều (tự tính):

$$D_{\mathrm{KL}}(P\|Q) = 0{,}9\log_2\frac{0{,}9}{0{,}5} + 0{,}1\log_2\frac{0{,}1}{0{,}5} \approx 0{,}5310 \text{ bit},$$

$$D_{\mathrm{KL}}(Q\|P) = 0{,}5\log_2\frac{0{,}5}{0{,}9} + 0{,}5\log_2\frac{0{,}5}{0{,}1} \approx 0{,}7370 \text{ bit}.$$

Hai số không bằng nhau, dù cùng so sánh một cặp phân phối. KL không phải một metric vì không đối xứng và nói chung không thoả bất đẳng thức tam giác. Có những cách xây đại lượng đối xứng từ KL, chẳng hạn Jensen-Shannon, nhưng đó là khái niệm khác.



Hình 39: Hai chiều KL giữa một phân phối Bernoulli cố định và một phân phối thay đổi (minh hoạ, tự tính)

Cách đọc hình 3: P cố định với xác suất thành công $0{,}9$; trục ngang thay đổi xác suất thành công q của Q. Đường xanh là KL từ P đến Q, đường đỏ nét đứt là chiều ngược lại; tại $q = 0{,}5$, hai chấm cho khoảng 0,531 và 0,737 bit (tự tính). Cả hai bằng 0 khi Q trùng P tại $q = 0{,}9$. Trục dọc được giới hạn để nhìn rõ vùng trung tâm, vì KL từ P đến Q có thể tăng không giới hạn khi Q gán xác suất rất gần 0 cho một kết quả có xác suất dương theo P.

7.3 Các chiều KL phạt kiểu sai khác nhau

Trong $D_{\mathrm{KL}}(P\|Q)$, kỳ vọng theo P khiến Q bị phạt rất nặng nếu bỏ gần hết xác suất ở một vùng P thật sự có khối lượng. Trong $D_{\mathrm{KL}}(Q\|P)$, một vùng được Q gán nhiều khối lượng nhưng P gần như không có cũng có thể bị phạt nặng. Đó là trực giác về sự khác biệt, không phải một định lý rằng một chiều luôn bao phủ đủ mọi vùng hay chiều kia luôn chỉ chọn đúng một mode.

Kết quả tối ưu còn phụ thuộc họ Q, hình dạng P, ràng buộc hỗ trợ, và khả năng tối ưu thực tế. Khi đọc một công thức biến phân, hãy xác định rõ cái nào đứng trước dấu $\|$, cái nào đứng sau, và kỳ vọng đang lấy theo phân phối nào trước khi mô tả hành vi.

8. Vì sao KL không âm và cross-entropy có cận dưới

Với $u > 0$, bất đẳng thức $-\ln u \geq 1 - u$ bằng nhau đúng khi $u = 1$. Nó đi từ tính lồi của $-\ln u$: đồ thị nằm trên tiếp tuyến tại $u = 1$. Ta dùng log tự nhiên cho chứng minh, rồi đổi sang bit bằng cách chia cho $\ln 2 > 0$.

Giả sử Q dương ở mọi nơi P dương. Trên tập $S = \{i : p_i > 0\}$, thay $u = q_i/p_i$, nhân với p_i rồi cộng:

$$D_{\mathrm{KL}}^{(\mathrm{nat})}(P\|Q) \geq \sum_{i\in S}(p_i - q_i) = 1 - \sum_{i\in S} q_i \geq 0.$$

Nếu Q bằng 0 ở nơi P dương thì KL là vô hạn và kết luận không âm vẫn đúng. Với các phân phối hữu hạn, dấu bằng xảy ra khi $p_i = q_i$ trên hỗ trợ của P; vì tổng xác suất của P đã bằng 1, Q khi đó cũng không còn khối lượng ngoài hỗ trợ ấy, tức P bằng Q.

Áp dụng đẳng thức ở mục 5.3:

$$H(P, Q) = H(P) + D_{\mathrm{KL}}(P\|Q) \geq H(P).$$

Vì $H(P)$ không đổi khi ta chỉ thay Q, tối thiểu hoá cross-entropy tương đương tối thiểu hoá KL từ P đến Q. Điều kiện **P cố định** quan trọng: nếu cả nguồn dữ liệu lẫn mô hình thay đổi, phần entropy nền cũng có thể thay đổi.

9. Thêm vài biến thể cần biết

9.1 Entropy có điều kiện và thông tin tương hỗ

Với hai biến rời rạc X và Y, $H(X \mid Y)$ là entropy của X sau khi biết Y, được lấy trung bình trên các giá trị Y. Nếu X và Y là hai bản sao của cùng một đồng xu cân bằng, biết Y cho biết X hoàn toàn: $H(X) = 1$ bit và $H(X \mid Y) = 0$ bit (tự tính). Nếu chúng độc lập, biết Y không đổi phân phối của X nên $H(X \mid Y) = H(X)$.

Thông tin tương hỗ (mutual information) là lượng giảm bất định trung bình:

$$I(X; Y) = H(X) - H(X \mid Y) = D_{\mathrm{KL}}(P_{X,Y}\|P_X P_Y).$$

Dạng KL so phân phối chung với tích của hai phân phối biên, nên $I(X; Y) \geq 0$ và bằng 0 khi X, Y độc lập, trong trường hợp rời rạc hữu hạn. Với hai bản sao đồng xu trên, thông tin tương hỗ bằng 1 bit (tự tính). Thông tin tương hỗ đo phụ thuộc thống kê nói chung, nhưng tự nó không cho biết chiều nhân quả.

9.2 Phân phối liên tục: entropy vi phân

Với mật độ $p(x)$, người ta định nghĩa entropy vi phân $h(X) = -\int p(x) \ln p(x)\, dx$ khi tích phân có nghĩa. Nó không hoàn toàn giống entropy rời rạc: mật độ không phải xác suất điểm, và entropy vi phân có thể âm.

Ví dụ phân phối đều trên khoảng từ 0 đến $1/2$ có mật độ bằng 2 trên khoảng đó, nên $h = -\ln 2 \approx -0{,}6931$ nat (tự tính). Giá trị âm không có nghĩa xác suất âm hay "âm bất định" theo nghĩa entropy rời rạc.

Đổi đơn vị đo hoặc đổi tọa độ có thể thay đổi entropy vi phân do mật độ đổi theo Jacobian. KL giữa hai phân phối liên tục được biến đổi cùng bằng một phép đổi biến khả nghịch phù hợp thì không đổi; nhưng vẫn phải xét hỗ trợ và sự tồn tại của tích phân. Đừng áp dụng trực tiếp cận $H \geq 0$ của trường hợp rời rạc cho entropy vi phân.

9.3 Nhãn mềm và label smoothing

Nhãn không nhất thiết là one-hot. Nếu một bài toán cho phân phối mục tiêu $t = (t_1, \ldots, t_m)$, cross-entropy là $-\sum_i t_i \ln q_i$, chấm cả các xác suất trong nhãn mềm. Label smoothing chủ động phân bổ một phần trọng lượng mục tiêu cho lớp khác theo một quy tắc đã chọn. Cách đó thay đổi mục tiêu tối ưu; không thể giả định nó luôn cải thiện mọi tác vụ hoặc thay thế kiểm tra hiệu chỉnh xác suất.

Ví dụ với $t = (0{,}9, 0{,}05, 0{,}05)$ và $q = (0{,}7, 0{,}2, 0{,}1)$, loss bằng $-0{,}9 \ln 0{,}7 - 0{,}05 \ln 0{,}2 - 0{,}05 \ln 0{,}1 \approx 0{,}5166$ nat (tự tính). Đây không phải loss one-hot của lớp đầu, vì hai lớp còn lại cũng có trọng số dương.

9.4 Zero xác suất và số học hữu hạn

Các công thức giới hạn giải thích vì sao $0 \log 0$ đóng góp 0, nhưng máy tính gọi log trực tiếp tại 0 sẽ cho vô cực hoặc lỗi số. Một mô hình softmax lý tưởng với logits hữu hạn cho xác suất dương, nhưng phép tính số hữu hạn vẫn có thể underflow. Dùng log-softmax ổn định là lựa chọn thường gặp; cắt xác suất về một ngưỡng dương có thể hữu ích cho tính toán nhưng làm thay đổi giá trị loss và gradient ở vùng bị cắt. Cần biết chương trình đang tính đại lượng nào, thay vì coi mọi thủ thuật số là cùng một công thức toán.

10. Nối sang AI

(a) Huấn luyện phân loại. Cross-entropy với nhãn quan sát được dùng rộng rãi vì phạt xác suất thấp cho nhãn đúng và là âm log-likelihood có điều kiện. Accuracy chỉ nhìn nhãn có điểm lớn nhất; cross-entropy dùng cả độ tự tin. Cả hai có ích nhưng đo các khía cạnh khác nhau.

(b) Dự đoán token và perplexity. Với loss trung bình L theo nat cho mỗi token, perplexity tương ứng là $\exp(L)$; nếu loss dùng bit, perplexity là 2^L. So sánh chỉ có ý nghĩa khi cách tách token và tập đánh giá tương thích. Con số thấp hơn mô tả xác suất mô hình gán cho chuỗi đánh giá, không tự chứng minh mọi câu sinh ra đều đúng.

(c) Suy luận biến phân và VAE. Math 08 và math 09 dùng KL để đo sai khác của phân phối suy luận và một phân phối đích. Trong đồng nhất thức ELBO, khoảng cách giữa log evidence và ELBO là KL từ posterior xấp xỉ đến posterior thật. Một số công thức VAE khác cũng chứa KL từ posterior xấp xỉ đến prior; hai KL có đối số và vai trò khác nhau.

(d) Information bottleneck. Math 14 dùng thông tin tương hỗ để bàn về biểu diễn giữ thông tin liên quan đến đầu ra trong khi hạn chế thông tin về đầu vào. Trong mô hình cụ thể, các chặn hoặc xấp xỉ khả tính có thể thay thế thông tin tương hỗ khó tính trực tiếp. Không nên đồng nhất một đại lượng xấp xỉ với giá trị chính xác nếu chưa xét giả định.

(e) Hình học thông tin và mô hình sinh. Math 07 nghiên cứu cấu trúc của các phân phối; KL có vai trò liên hệ với độ cong cục bộ nhưng bản thân nó không đối xứng. Trong math 29, các mục tiêu biến phân có các số hạng liên quan tới phân phối và KL. Đọc đúng chiều và điều kiện của từng số hạng là bước cần thiết trước khi diễn giải một loss mô hình sinh.

11. Sai lầm hay gặp

- **Nhầm entropy với bất ngờ của một mẫu.** $-\log p(x)$ là lượng tin của kết quả x; entropy là trung bình của đại lượng ấy qua phân phối.
- **Lấy trung bình theo Q trong $H(P,Q)$.** P là phân phối của dữ liệu trong định nghĩa đang dùng, còn Q cho điểm dự đoán. Đổi vai trò là một đại lượng khác.
- **Cho rằng cross-entropy bằng 0 khi dự đoán khớp.** Khi $Q = P$, KL bằng 0 nhưng cross-entropy bằng $H(P)$, có thể dương nếu dữ liệu còn bất định.
- **Gọi KL là khoảng cách đối xứng.** Hai chiều có thể khác nhau hoặc một chiều vô hạn, trong khi chiều còn lại hữu hạn. Luôn viết đủ thứ tự đối số.
- **Trộn bit với nat hoặc bỏ qua xác suất 0.** Dùng một cơ số log cho cùng phép so sánh; kiểm tra hỗ trợ trước khi thay số vào log hay phép chia.
- **Áp kết luận rời rạc cho mật độ liên tục.** Entropy vi phân có thể âm và phụ thuộc đơn vị đo; mật độ lớn hơn 1 không làm xác suất vượt 1.

12. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Lượng tin	$I(x) = -\log_2 p(x)$	bất ngờ của một kết quả
Entropy	$H(P) = -\sum_i p_i \log_2 p_i$	bất ngờ trung bình theo P
Cross-entropy	$H(P,Q) = -\sum_i p_i \log_2 q_i$	chi phí trung bình theo P khi dùng Q
KL	$D_{\mathrm{KL}}(P\|Q) = \sum_i p_i \log_2(p_i/q_i)$	chi phí phụ có hướng
Phân tích	$H(P,Q) = H(P) + D_{\mathrm{KL}}(P\|Q)$	dự đoán khớp khi phần KL bằng 0
One-hot loss	$L = -\ln q_y$	âm log xác suất của nhãn đúng
Softmax gradient	$\partial L/\partial z_i = q_i - t_i$	với nhãn one-hot và log tự nhiên
Thông tin tương hỗ	$I(X;Y) = H(X) - H(X \mid Y)$	giảm bất định trung bình khi biết Y

13. Bài tập tự tính

Hãy tự làm trước, rồi so với lời giải. Các phép tính trong phần này do ta tự làm và đã kiểm bằng numpy.

Bài 1 (dễ). Một đồng xu cân bằng có xác suất ngửa và sấp đều bằng $1/2$. Tính lượng tin của mặt ngửa và entropy của đồng xu theo bit.

Đáp án. Mặt ngửa có lượng tin $-\log_2(1/2) = 1$ bit. Mặt sấp cũng 1 bit, nên entropy là $(1/2)\cdot 1 + (1/2)\cdot 1 = 1$ bit (tự tính). Một giá trị là cho kết quả riêng, giá trị còn lại là trung bình.

Bài 2 (dễ vừa). Với $P = (1/2, 1/2)$ và $Q = (1/4, 3/4)$, tính cross-entropy và KL theo bit.

Đáp án. $H(P) = 1$ bit. Cross-entropy là $-(1/2)\log_2(1/4) - (1/2)\log_2(3/4) \approx 1{,}2075$ bit, nên $D_{\mathrm{KL}}(P\|Q) \approx 0{,}2075$ bit (tự tính). Có thể kiểm bằng tổng hai hạng KL: $(1/2)\log_2 2 + (1/2)\log_2(2/3)$.

Bài 3 (vừa). Với $P = (1, 0)$ và $Q = (3/4, 1/4)$, tính hai chiều KL. Vì sao kết quả khác nhau?

Đáp án. Theo P, chỉ kết quả thứ nhất đóng góp, nên $D_{\mathrm{KL}}(P\|Q) = \log_2(4/3) \approx 0{,}4150$ bit (tự tính). Ngược lại, $D_{\mathrm{KL}}(Q\|P) = +\infty$, vì Q cho kết quả thứ hai xác suất dương còn P cho xác suất 0. Đổi chiều làm đổi cả trọng số lẫn mẫu số của tỉ số xác suất.

Bài 4 (vừa khó). Mô hình cho Q bằng $(0{,}7, 0{,}2, 0{,}1)$, nhãn đúng là lớp thứ hai. Tính cross-entropy một mẫu bằng nat và gradient theo ba logits softmax.

Đáp án. Vector nhãn là $(0, 1, 0)$, loss bằng $-\ln 0{,}2 \approx 1{,}6094$ nat. Gradient theo logits là $Q - t = (0{,}7, -0{,}8, 0{,}1)$ (tự tính). Không được thay $0{,}2$ bằng xác suất lớn nhất $0{,}7$, vì lớp có xác suất lớn nhất không

phải nhãn đúng.

Bài 5 (khó). X là đồng xu cân bằng, Y luôn bằng X. Viết phân phối chung và tính $I(X;Y)$ theo hai cách.

Đáp án. Phân phối chung có $P(0,0) = P(1,1) = 1/2$, hai cặp còn lại có xác suất 0. Vì biết Y thì biết X, $H(X) = 1$ và $H(X \mid Y) = 0$, nên $I(X;Y) = 1$ bit. Kiểm qua KL: mỗi biên đều cân bằng, nên tích các biên cho mỗi cặp xác suất $1/4$; trên hai cặp có xác suất dương, tỉ số là $(1/2)/(1/4) = 2$. Vậy KL bằng $2 \cdot (1/2) \log_2 2 = 1$ bit (tự tính).

Tài liệu tham khảo nền: Cover và Thomas, Elements of Information Theory; Bishop, Pattern Recognition and Machine Learning; ghi chú math 07, math 08, math 09 và math 14.

Nền tảng 12

Số học máy tính: tính gần đúng có kiểm soát để kết quả không tràn hay mất nghĩa

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: số thực, lũy thừa, hàm mũ và logarit ở mức phổ thông; vector và ma trận có thể xem lại ở math nen02 và nen03

Dùng ở: bài math 15 (online softmax), math 16 (định dạng FP8)

> Chương này giải thích khái niệm từ đầu, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần số học máy tính

Máy tính không lưu được mọi số thực chính xác; nó lưu một tập hữu hạn các số gần chúng nhất. **Số học máy tính** nghiên cứu chuyện gì xảy ra khi ta làm phép cộng, nhân, lấy hàm mũ hay chia trên những số hữu hạn đó. Hiểu được điều này giúp ta viết phép tính sao cho sai số làm tròn nhỏ, tránh tràn số và giữ được ý nghĩa của kết quả.

Trong AI, một phép tính sai nhỏ có thể lặp đi lặp lại qua hàng triệu phép nhân. Một logit lớn có thể làm hàm mũ tràn thành vô cực; một xác suất rất nhỏ có thể bị làm tròn thành không; phép trừ hai số gần nhau có thể xóa mất những chữ số có ích. Mô hình vẫn có thể chạy hết mà loss hoặc gradient đã thành inf, nan, hay một số sai đáng kể. Học số học hữu hạn giúp ta nhận ra những lỗi đó và chọn công thức ổn định hơn.

2. Nhắc lại số thực, sai số và độ chính xác

2.1 Số thực và số được lưu

Trên giấy, giữa hai số thực luôn có vô hạn số khác. Trong một kiểu số máy tính, mỗi số chỉ có hữu hạn bit, vì vậy chỉ một số hữu hạn giá trị được biểu diễn chính xác. Khi muốn lưu một số không nằm trong tập ấy, máy làm tròn nó về một giá trị có thể lưu được.

Ví dụ, phân số $1/2$ có biểu diễn nhị phân kết thúc là 0.1_2. Phân số $1/10$ không có biểu diễn nhị phân hữu hạn, cũng như $1/3$ không có biểu diễn thập phân hữu hạn. Máy phải chọn số nhị phân gần nhất. Vì vậy, trong nhiều kiểu số, biểu thức lưu trữ 0.1 không bằng chính xác $1/10$ (tự tính, kiểm tra bằng numpy bằng cách so sánh giá trị máy lưu với số thập phân có độ chính xác cao).

Một điều quan trọng: làm tròn không có nghĩa là chọn tùy tiện. Phần cứng và thư viện thường dùng một quy tắc được chuẩn hoá, thường là chọn số biểu diễn gần nhất; nếu đúng giữa hai lựa chọn thì chọn số có bit cuối chẵn. Tên tiếng Anh thường gặp là round to nearest, ties to even. Quy tắc cụ thể còn tuỳ phép toán và môi trường, nên khi cần kết quả bit-for-bit phải kiểm tra đúng phần cứng, kiểu số và thư viện.

2.2 Ba loại sai số

Ta tách ba việc dễ bị gom làm một:

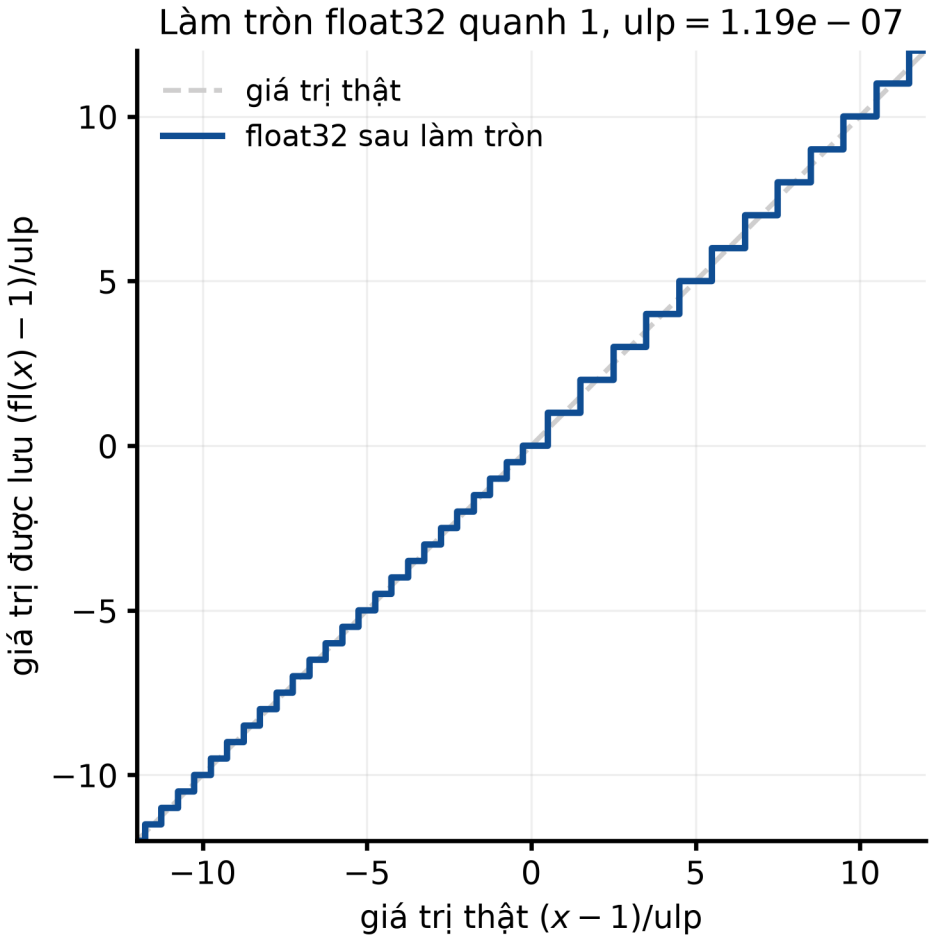
- **Sai số biểu diễn:** số đầu vào được thay bằng số máy gần nó nhất. Ví dụ thập phân 0.1 không có biểu diễn nhị phân hữu hạn.
- **Sai số làm tròn:** kết quả chính xác của một phép toán được thay bằng số máy gần nhất. Nhiều phép làm tròn liên tiếp có thể làm sai khác tích luỹ.

- **Sai số do xấp xỉ thuật toán:** ta cố ý thay bài toán bằng phép tính rẻ hơn, chẳng hạn dừng một phép lặp trước khi hội tụ. Đây là lựa chọn thuật toán, khác với giới hạn bit của số máy.

Một thuật toán có thể cho sai số lớn vì dữ liệu đầu vào nhạy cảm, dù từng phép toán được làm tròn rất chính xác. Ngược lại, một bài toán nhạy vẫn có thể được giải bằng thuật toán ổn định nếu dữ liệu được đo đủ tốt và ta hiểu giới hạn của kết quả. Chương này tập trung vào biểu diễn số và cách sắp xếp phép tính.

3. Trực giác bằng hình: số máy tạo thành một lưới

Hãy phóng to đoạn số thực quanh 1. Các số dấu phẩy động không nằm dày đặc như một đường liền; chúng tạo thành các bậc. Khi số thật nằm giữa hai bậc, phép làm tròn đưa nó về một trong hai bậc gần nhất. Khoảng cách giữa các bậc phụ thuộc độ lớn của số: quanh 1 khoảng cách nhỏ hơn nhiều so với quanh một số rất lớn.

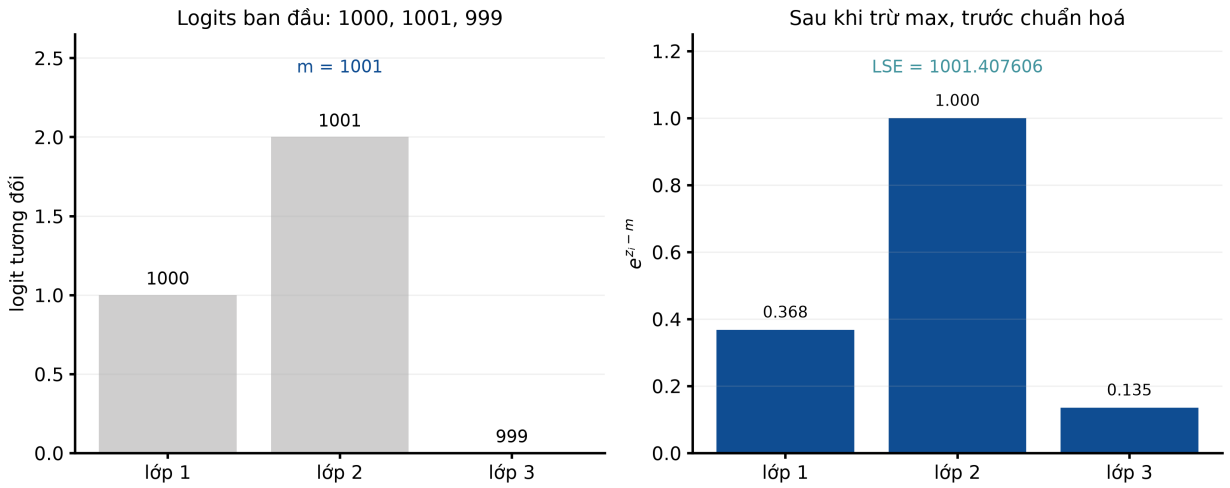


Hình 40: Các số float32 quanh 1 tạo thành những bậc rời rạc, giá trị đầu vào được làm tròn về bậc gần nhất (minh hoạ, tự tính)

Cách đọc hình 1: đường chéo xám là giá trị thật nếu không bị làm tròn; bậc xanh là giá trị float32 được lưu. Trục ngang và trục dọc đo độ lệch khỏi 1 theo $\text{ulp}(1) = 2^{-23}$, khoảng cách từ 1 tới float32 kế tiếp lớn hơn 1. Ngay dưới 1, khoảng cách số máy chỉ bằng 2^{-24}, vì vậy bậc xanh dày hơn ở nửa trái.

Một công thức AI khác cần nhìn bằng trực giác trước là log-sum-exp. Khi các logits đều lớn, lấy hàm mũ trực tiếp có thể tạo số quá lớn. Nhưng nếu trừ cùng một hằng số trước khi lấy hàm mũ, các tỉ lệ giữa những số mũ không đổi; nhờ đó ta có thể chuyển các giá trị về miền dễ tính.

Cách đọc hình 2: panel trái ghi logits ban đầu trên từng cột; chiều cao cột biểu diễn độ lệch so với logit thấp



Hình 41: Trừ logit lớn nhất đưa các số mũ về miền nhỏ hơn mà không đổi tỉ lệ softmax (minh hoạ, tự tính)

nhất 999. Panel phải là các số mũ sau khi trừ logit lớn nhất. Giá trị lớn nhất sau dịch bằng 1, các giá trị còn lại nằm trong khoảng từ 0 đến 1; tỉ lệ softmax vẫn giữ nguyên.

4. Định nghĩa dấu phẩy động

4.1 Dấu, phần định trị và số mũ

Một số dấu phẩy động nhị phân thường được hình dung qua ba phần: bit dấu, phần định trị (significand) và số mũ (exponent). Với số chuẩn hoá, dạng khái quát là

$$x = (-1)^s (1.f)_2 2^e,$$

trong đó s chọn dấu, f chứa các bit phần lẻ, và e điều chỉnh thang độ lớn. Dạng này cho phép dùng cùng số bit để biểu diễn số rất nhỏ lẫn số rất lớn. Đổi lại, số biểu diễn được thưa dần khi độ lớn tăng.

Trong IEEE 754, các kiểu nhị phân phổ biến khác nhau ở số bit dành cho phần định trị và số mũ. Nhiều bit định trị hơn thường giúp phân biệt các số gần nhau tốt hơn; nhiều bit số mũ hơn thường mở rộng miền giá trị. Không có kiểu nào đồng thời cho độ chính xác và miền giá trị vô hạn.

4.2 Khoảng cách giữa số máy và epsilon

Gọi $\mathrm{fl}(x)$ là số máy thu được khi làm tròn x. **Epsilon máy** thường được hiểu là khoảng cách giữa 1 và số máy kế tiếp lớn hơn 1. Với IEEE binary32, khoảng cách này là 2^{-23}; với binary64 là 2^{-52} (tự tính, lấy từ numpy.finfo). Ở các lũy thừa của hai, khoảng cách ngay phía dưới có thể khác khoảng cách phía trên, nên hãy coi đây là quy ước tham chiếu quanh 1.

Với làm tròn về số gần nhất, sai số tương đối của một số chuẩn hoá thường bị chặn bởi một nửa epsilon, miễn là kết quả không tràn, không rơi vào vùng dưới chuẩn và phép toán tuân quy tắc làm tròn đã giả định. Nửa epsilon này thường được gọi là unit roundoff, ký hiệu u. Vì thế, với một phép toán cơ bản có kết quả chuẩn hoá, người ta hay viết

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \le u,$$

trong đó $\circ$ là phép toán, còn δ là sai số tương đối của bước đó. Đây là mô hình phân tích hữu ích, không phải lời hứa cho mọi tình huống: số gần 0, tràn, NaN, phép toán đặc biệt và kiểu phần cứng có thể cần mô hình

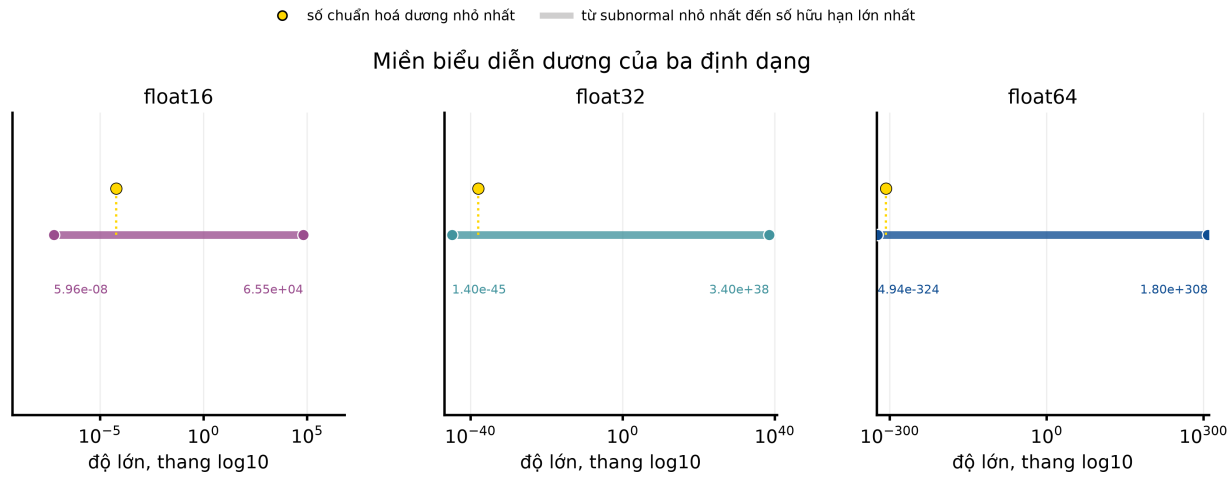
khác.

4.3 Miền giá trị: số chuẩn hoá, dưới chuẩn, tràn và hụt

Mỗi kiểu số chỉ biểu diễn được một miền hữu hạn. Nếu kết quả dương vượt giới hạn trên, phép toán có thể tạo $+\infty$; nếu kết quả quá nhỏ, nó có thể được biểu diễn bằng số dưới chuẩn với ít bit chính xác hơn, rồi cuối cùng làm tròn về 0. Ta gọi trường hợp đầu là **tràn số** (overflow), trường hợp sau là **hụt số** (underflow).

Số dưới chuẩn lấp một phần khoảng trống giữa số chuẩn hoá nhỏ nhất và 0, giúp giá trị tiến dần về không thay vì nhảy ngay. Độ chính xác tương đối giảm khi xuống vùng này. Một số bộ xử lý hoặc chế độ tăng tốc có thể chủ động đưa số dưới chuẩn về không (flush to zero); vì vậy, độ chính xác thực tế của phép tính còn phụ thuộc đường chạy.

Các hằng số miền của float16, float32 và float64 có thể lấy trực tiếp từ numpy.finfo, thay vì ghi nhớ. Hình tiếp theo so sánh khoảng số dương nhỏ nhất có thể biểu diễn với số lớn nhất của từng kiểu (tự tính từ NumPy).



Hình 42: Miền dương của float16, float32 và float64, mỗi panel có thang log riêng (minh hoạ, tự tính)

Cách đọc hình 3: mỗi panel dùng một thang ngang logarit riêng, nên hãy đọc nhãn lũy thừa trên trục của panel thay vì so chiều dài các đoạn giữa các panel. Hai đầu đoạn ghi số dưới chuẩn dương nhỏ nhất và số hữu hạn lớn nhất; chấm vàng chỉ số chuẩn hoá dương nhỏ nhất. Riêng độ chính xác quanh một số cụ thể còn phụ thuộc số bit định trị, không chỉ miền giá trị.

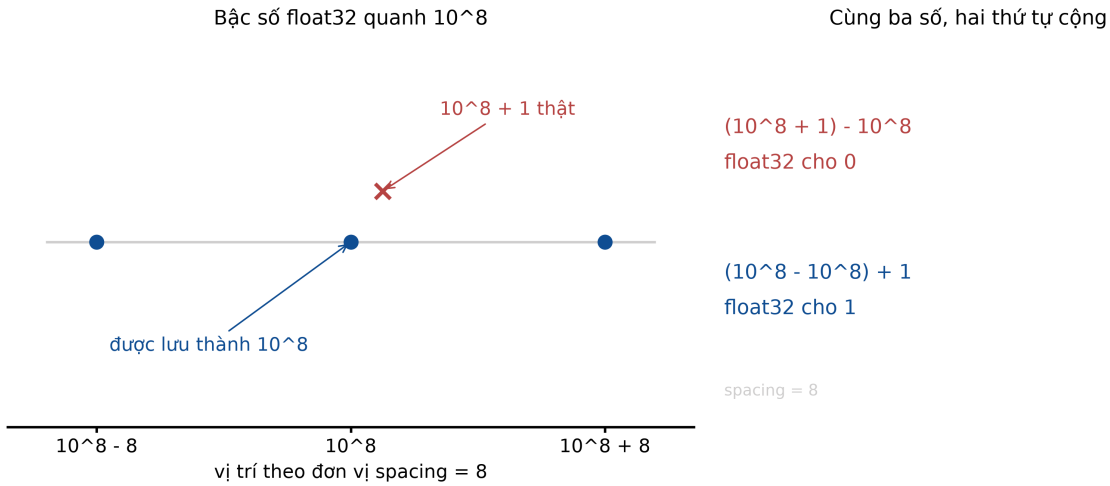
5. Một phép tính từng bước: vì sao thứ tự cộng quan trọng

Ta xét $10^8 + 1 - 10^8$ trong float32. Số 10^8 đủ lớn để khoảng cách giữa các số float32 lân cận quanh nó là 8 (tự tính bằng numpy.spacing). Vì vậy, cộng 1 vào 10^8 nhỏ hơn nửa bước lưới. Phép cộng đầu tiên làm tròn kết quả trở lại 10^8.

Viết rõ từng bước (tự tính):

1. $a = \text{float32}(10^8)$.
2. $b = \text{float32}(a + 1) = 10^8$, vì 1 không đủ để đưa kết quả sang float32 kế tiếp.
3. $c = \text{float32}(b - a) = 0$.

Nếu đổi ngoặc, ta tính $(10^8 - 10^8) + 1 = 1$ (tự tính). Vậy cùng ba giá trị toán học cho hai kết quả máy khác nhau, do phép cộng dấu phẩy động không kết hợp chính xác như phép cộng số thực. Đó là lý do phép tổng song song, đổi thứ tự reduction hay thay kích cỡ batch đôi khi cho kết quả khác ở những chữ số cuối.



Hình 43: Cộng 1 vào 10 mũ 8 trong float32 làm tròn về cùng một bậc, nên hai thứ tự cộng cho kết quả khác (minh hoạ, tự tính)

Cách đọc hình 4: các vạch xanh là số float32 quanh 10^8; khoảng cách giữa hai vạch kế tiếp là 8. Điểm đỏ là tổng thật $10^8 + 1$, nhưng nằm gần vạch 10^8 hơn vạch kế tiếp, nên được làm tròn về vạch đó. Panel bên phải so sánh hai thứ tự cộng: ở thứ tự thứ nhất, số 1 biến mất trước khi phép trừ xảy ra.

5.1 Cộng nhiều số và giảm sai số

Khi cộng nhiều số, các số nhỏ cộng vào một tổng lớn có thể bị mất. Một mẹo là cộng các số theo thứ tự tăng dần độ lớn để hạn chế việc số nhỏ bị nuốt sớm. Một cách khác là **cộng bù** (compensated summation), chẳng hạn Kahan summation: ta lưu riêng phần sai số bị bỏ qua ở mỗi phép cộng rồi đưa nó trở lại ở bước sau. Cách này không làm số máy chính xác vô hạn, nhưng thường cải thiện tổng khi có nhiều số khác độ lớn.

Với tích vô hướng hay tổng gradient, kết quả còn tuỳ thứ tự reduction song song. Hai GPU, hai số luồng hoặc hai thư viện có thể gom các hạng tử theo cây cộng khác nhau. Điều đó không tự động là lỗi; câu hỏi là độ chênh có phù hợp với độ chính xác và mục tiêu ứng dụng không.

6. Mất chữ số khi trừ hai số gần nhau

Triệt tiêu số học (cancellation) xảy ra khi trừ hai số gần nhau: các chữ số đầu giống nhau bị xoá, còn lại kết quả nhỏ chịu ảnh hưởng tương đối lớn từ sai số trước đó. Ví dụ kinh điển là $\sqrt{1+x} - 1$ khi x rất nhỏ. Nếu phép căn được làm tròn gần 1, phép trừ tiếp theo có thể làm mất hầu hết chữ số có ích.

Ta biến đổi đại số bằng cách nhân cả tử và mẫu với biểu thức liên hợp (tự tính):

$$\sqrt{1+x} - 1 = \frac{x}{\sqrt{1+x} + 1}.$$

Công thức bên phải tránh trừ hai số gần bằng nhau. Với $x = 10^{-16}$ trong float64, biểu thức trực tiếp có thể làm tròn thành 0, trong khi dạng hữu tỉ hoá cho giá trị gần 5×10^{-17} (tự tính, kiểm bằng NumPy). Đây là cùng một hàm toán học nhưng hai cách đánh giá có độ ổn định khác nhau.

Triệt tiêu không phải lúc nào cũng có thể chữa bằng một biến đổi đại số đơn giản. Trước hết cần hỏi bài toán có nhạy hay không, rồi chọn cách tính giảm tổn thất. Trong bài toán phương sai, chẳng hạn, công thức $\mathbb{E}[X^2] - \mathbb{E}[X]^2$ có thể trừ hai số lớn gần bằng nhau; thuật toán Welford cập nhật trung bình và phương sai theo từng phần để hạn chế vấn đề này.

7. Log-sum-exp và softmax ổn định

7.1 Hàm mũ làm tràn nhanh

Với vector logits $z = (z_1, \ldots, z_n)$, softmax được định nghĩa bởi

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Hàm mũ tăng rất nhanh. Nếu một logit lớn, tính riêng e^{z_i} có thể tràn dù xác suất sau chia hoàn toàn bình thường. Nếu logit rất âm, e^{z_i} có thể hụt về 0. Công thức toán học đúng chưa chắc là trình tự phép toán máy an toàn.

7.2 Dịch các logits bằng số lớn nhất

Đặt $m = \max_i z_i$. Ta trừ m khỏi mọi logit:

$$\text{softmax}(z)_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}.$$

Ta được phép làm vậy vì tử và mẫu đều được nhân với cùng hệ số e^{-m}. Mọi số mũ mới không lớn hơn 1, và ít nhất một số bằng 1. Vì vậy mẫu số không thể tràn chỉ do một logit lớn. Chênh lệch logits quá lớn vẫn có thể làm một số hạng nhỏ hụt về 0, nhưng xác suất đó khi ấy cũng cực nhỏ ở độ chính xác đang dùng.

Phép biến đổi này giả sử có ít nhất một logit hữu hạn và không có NaN hay $+\infty$. Nếu mọi lớp đều bị mask bằng $-\infty$, thì $m = -\infty$ và phép trừ tạo $-\infty - (-\infty)$, tức NaN; trường hợp đó phải được xử lý riêng theo quy ước của bài toán. Max-trick cũng không bảo đảm tổng mẫu số không tràn trong accumulator hẹp: nếu cộng 65536 số hạng đều bằng 1 trong float16, tổng thật vượt giá trị hữu hạn lớn nhất 65504 của kiểu này (tự tính, kiểm bằng NumPy). Với vector lớn, cần tích luỹ ở kiểu đủ rộng hoặc dùng cách chuẩn hoá trực tuyến phù hợp.

Max-trick ngăn tràn do số mũ lớn, nhưng không giữ mọi chữ số của tổng nhỏ sau khi thêm vào số hạng lớn nhất. Chẳng hạn, trong float32 với hai logit đã dịch $(0, -20)$, phép tính $\log(1 + e^{-20})$ có thể làm tròn thành 0 nếu phép cộng trong dấu log làm mất e^{-20}. Với dạng hai số hạng, $\log 1p(e^{-20})$ giữ được giá trị gần 2.06×10^{-9} (tự tính bằng NumPy); với nhiều số hạng, dùng accumulator đủ rộng hoặc phép cộng log ổn định khi cần giữ loss rất nhỏ.

Để giữ các hiệu chỉnh nhỏ khi logits có nền rất lớn, ta lưu riêng phần dịch: $d_i = z_i - m$ và $s = \log \sum_j e^{d_j}$. Softmax dùng e^{d_i}/e^{s}; log-softmax dùng $d_i - s$; cross-entropy của lớp đúng t dùng $s - d_t$. Đừng tính $m + s$ rồi trừ lại z_t nếu cần loss hay log-probability trong kiểu chính xác hữu hạn.

7.3 Log-sum-exp

Log-sum-exp là

$$\text{LSE}(z) = \log\left(\sum_i e^{z_i}\right).$$

Phiên bản ổn định được viết

$$\text{LSE}(z) = m + \log\left(\sum_i e^{z_i - m}\right), \qquad m = \max_i z_i.$$

Đây là công thức đã thấy trong hình 2. Trên số thực, log-softmax bằng $z_i - \mathrm{LSE}(z)$; nhưng trong tính toán máy hãy giữ $d_i = z_i - m$ và $s = \log \sum_j e^{d_j}$ rồi lấy $d_i - s$. Nếu cộng $m + s$ trước, số hạng nhỏ s có thể bị làm tròn mất khi m rất lớn; phép trừ sau đó không thể khôi phục nó. Cũng không nên tính softmax rồi mới lấy log, vì xác suất dưới chuẩn có thể làm tròn về đúng 0 và log thành $-\infty$.

7.4 Ví dụ tay với logits lớn

Lấy $z = (1000, 1001, 999)$. Tính trực tiếp sẽ cần e^{1000}, vượt miền số hữu hạn của float64. Dùng max-trick, ta có $m = 1001$ và

$$\mathrm{LSE}(z) = 1001 + \log(e^{-1} + 1 + e^{-2}) \approx 1001.407606.$$

Đặt $d = z - m = (-1, 0, -2)$ và $s = \log(e^{-1} + 1 + e^{-2}) \approx 0.407606$. Các xác suất tương ứng xấp xỉ $(0.244728, 0.665241, 0.090031)$; tổng bằng 1 tới sai số làm tròn (tự tính bằng NumPy). Các giá trị này giống hệt khi dùng logits $(0, 1, -1)$ vì đã trừ cùng một số 1000 khỏi từng phần tử (tự tính). Nếu cần log-probability hoặc cross-entropy, dùng $d_i - s$ hay $s - d_t$ trực tiếp để không làm mất phần hiệu chỉnh nhỏ s.

Với lớp đúng có chỉ số t, tính cross-entropy ổn định bằng phần dịch:

$$\mathcal{L}(z, t) = s - d_t, \qquad d_i = z_i - m, \qquad s = \log \sum_j e^{d_j}.$$

Trên số thực, biểu thức này bằng $\mathrm{LSE}(z) - z_t$. Cách tính $s - d_t$ tránh cả việc lưu xác suất nhỏ rồi mới lấy log lẫn việc cộng $m + s$ rồi trừ logit lớn z_t trong kiểu số hẹp. Vì thế, số học ổn định là một phần của cách cài loss, không chỉ là chi tiết phần cứng.

8. Các chi tiết hay bị bỏ qua

8.1 Sai số có thể tích luỹ

Nếu mỗi phép toán mắc sai số tương đối nhỏ, nhiều phép toán nối tiếp vẫn có thể tạo sai số đáng kể. Một phép nhân ma trận có rất nhiều phép nhân-cộng; sai số phụ thuộc số hạng, thứ tự cộng, mức triệt tiêu và điều kiện của bài toán. Không nên suy ra rằng “mỗi phép toán chỉ sai một chút” đồng nghĩa “kết quả cuối luôn đúng”.

8.2 Độ chính xác và miền giá trị là hai việc khác nhau

Hai kiểu số có thể cùng độ rộng bit nhưng phân bổ bit khác nhau. Kiểu ưu tiên nhiều bit số mũ chứa được số lớn hơn, còn kiểu có nhiều bit định trị phân biệt được các giá trị gần nhau tốt hơn. FP16 và bfloat16 thường được chọn vì những đánh đổi khác nhau; FP8 trong bài math 16 có nhiều biến thể với cùng mục tiêu giảm dung lượng và tăng thông lượng. Tên kiểu không nói hết khả năng của đường tính: accumulator có thể dùng độ chính xác cao hơn kiểu lưu đầu vào.

8.3 Số đặc biệt

IEEE 754 định nghĩa các giá trị đặc biệt như $+\infty$, $-\infty$ và NaN (not a number). Chia số khác không cho một số hữu hạn có thể tạo vô cực; phép toán không xác định như $0/0$ thường tạo NaN. NaN có tính chất khó chịu là so sánh bằng với chính nó cũng sai. Khi loss bỗng thành NaN, hãy lần ngược phép toán đầu tiên tạo NaN hoặc vô cực, thay vì chỉ nhìn giá trị cuối.

8.4 Chọn độ chính xác cho từng phần

Huấn luyện hỗn hợp độ chính xác thường lưu activation hoặc trọng số bằng kiểu hẹp để giảm bộ nhớ, còn một số phép cộng tích luỹ dùng kiểu rộng hơn. Một số loss được tính lại ở kiểu rộng, gradient có thể được

scale trước khi lưu để tránh hụt số. Những lựa chọn này là chính sách triển khai, không thay đổi định nghĩa mô hình; nhưng chúng có thể thay đổi sai số và khả năng tái lập.

9. Nối sang AI

(a) Softmax an toàn trong phân loại và mô hình ngôn ngữ. Hàm softmax biến logits thành phân phối xác suất. Max-trick và log-sum-exp giúp tránh tràn số; khi cần log-probability hoặc loss, giữ riêng $d_i = z_i - m$ và $s = \log \sum_j e^{d_j}$ để không mất hiệu chỉnh nhỏ sau khi cộng với nền logit lớn. Bài math 15 đi thêm một bước, tính max và tổng mẫu số trực tuyến để giảm số lần đọc bộ nhớ. Công thức trong chương này là nền để hiểu ý tưởng an toàn số học của bài đó, không phải các kết quả benchmark của paper.

(b) Loss phân loại từ logits. Trên số thực, cross-entropy bằng log-sum-exp trừ logit của lớp đích; khi tính máy, hãy giữ $d_i = z_i - m$ và $s =$
log
$sum_j e^{d_j}$ rồi tính $s - d_t$. Nhiều thư viện gộp softmax với cross-entropy để tránh tạo một xác suất rất nhỏ rồi mới lấy log. Nếu tách hai bước, cần kiểm tra lớp xác suất bị làm tròn thành không.

(c) Mixed precision và FP8. Mỗi phép nhân tensor có thể lưu bằng kiểu hẹp nhưng cộng dồn bằng kiểu rộng hơn. Bài math 16 phân tích định dạng FP8 và những đánh đổi về miền giá trị, độ chính xác, scale và cách dùng theo từng loại tensor. Không thể kết luận một kiểu số "tốt nhất" nếu chưa xét phân bố giá trị và phép toán tiếp theo.

(d) Số học xác suất. Xác suất nhiều biến cố độc lập thường là tích của nhiều số nhỏ. Lấy log đổi tích thành tổng, nhờ vậy tránh hụt số và biến bài toán cực đại hoá likelihood thành phép cộng log-likelihood. Khi chuẩn hoá log-probability, log-sum-exp là phép toán tự nhiên.

(e) Gradient và phép cộng song song. Gradient là tổng đóng góp từ nhiều mẫu hoặc nhiều thiết bị. Đổi thứ tự reduction có thể đổi vài bit cuối, nhất là khi các hạng tử có độ lớn rất khác nhau. Sai số số học thường không giải thích toàn bộ hiện tượng gradient biến mất hay bùng nổ, nhưng nó có thể làm tình trạng đó nặng hơn hoặc làm khó xác định phép tính đầu tiên bị lỗi.

10. Sai lầm hay gặp

- **Nghĩ số được in ra là số được lưu.** Màn hình có thể hiển thị 0.1, trong khi giá trị nhị phân bên trong chỉ gần bằng nó.
- **Cho rằng dấu phẩy động có tính kết hợp như số thực.** Ví dụ tổng float32 ở mục 5 cho thấy đổi ngoặc có thể đổi kết quả.
- **Tính softmax bằng hàm mũ trực tiếp.** Một logit lớn có thể làm tràn trước khi phép chia tạo xác suất.
- **Lấy log sau khi xác suất đã hụt về không.** Nếu cần log-probability, hãy tính $d_i - s$ với $d_i = z_i - m$ và $s =$
  log
  $sum_j e^{d_j}$, không tạo xác suất nhỏ trước rồi lấy log.
- **Nhầm epsilon với độ chính xác tuyệt đối ở mọi độ lớn.** Khoảng cách số máy phụ thuộc độ lớn của số, và epsilon chỉ là tham chiếu quanh 1.
- **Cho rằng tăng số bit luôn sửa được bài toán nhạy.** Độ chính xác cao giảm sai số tính toán, nhưng không loại bỏ độ nhạy vốn có của đầu vào hoặc mô hình.

11. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Số dấu phẩy động	$x = (-1)^s (1.f)_2 2^e$	dấu, định trị, số mũ

Khái niệm	Công thức	Ý nghĩa
Làm tròn	$\mathrm{fl}(x)$	số máy gần x nhất theo quy tắc đã chọn
Sai số tương đối một bước	$\mathrm{fl}(y) = y(1+\delta)$	mô hình phân tích, thường $\lvert\delta\rvert \le u$ với kết quả chuẩn hoá
Epsilon máy	$\mathrm{eps} = \mathrm{nextafter}(1, +\infty) - 1$	khoảng cách kế tiếp quanh 1
Softmax ổn định	$e^{z_i - m} / \sum_j e^{z_j - m}$	trừ $m = \max_j z_j$ trước khi lấy mũ
Log-sum-exp	$m + \log \sum_i e^{z_i - m}$	logarit của tổng mũ, dạng ổn định
Cross-entropy từ logits	$s - d_t$, với $d_i = z_i - m$, $s = \log \sum_j e^{d_j}$	không làm mất hiệu chỉnh nhỏ khi logits có nền lớn
Hụt số	kết quả nhỏ hơn miền biểu diễn hữu hiệu	mất độ chính xác, có thể thành 0
Tràn số	kết quả lớn hơn giới hạn hữu hạn	có thể thành $+\infty$ hoặc $-\infty$

12. Bài tập tự tính

Hãy tự làm trước khi xem đáp án. Các phép tính trong phần đáp án đã được kiểm lại bằng NumPy.

Bài 1 (dễ). Viết 13.25 dưới dạng nhị phân hữu hạn và giải thích vì sao có thể lưu chính xác trong số dấu phẩy động nhị phân.

Đáp án. $13 = 1101_2$ và $0.25 = 0.01_2$, nên $13.25 = 1101.01_2$ (tự tính). Phần lẻ kết thúc sau hai bit nhị phân, vì vậy biểu diễn chính xác nếu kiểu số có đủ bit định trị và số mũ.

Bài 2 (dễ vừa). Trong float32, so sánh $(10^8 + 1) - 10^8$ với $(10^8 - 10^8) + 1$. Vì sao kết quả khác nhau?

Đáp án. Phép thứ nhất cho 0, phép thứ hai cho 1 (tự tính bằng NumPy). Khoảng cách float32 quanh 10^8 là 8, nên phép cộng đầu tiên làm tròn $10^8 + 1$ về 10^8; số 1 bị mất trước phép trừ.

Bài 3 (vừa). Tính ổn định $\mathrm{LSE}(100, 101)$ và xác suất softmax tương ứng.

Đáp án. Trừ max 101, ta được $101 + \log(1 + e^{-1}) \approx 101.313262$ (tự tính). Hai xác suất lần lượt là $1/(1+e)$ và $e/(1+e)$, xấp xỉ 0.268941 và 0.731059 (tự tính). Tổng của chúng bằng 1 tới sai số làm tròn.

Bài 4 (vừa khó). Với $x = 10^{-16}$ trong float64, vì sao $\sqrt{1+x} - 1$ có thể cho 0, còn $x/(\sqrt{1+x}+1)$ vẫn cho giá trị khác 0?

Đáp án. Khi tính trong float64, $1 + x$ có thể làm tròn thành đúng 1; khi đó căn bậc hai cũng là 1 và phép trừ trả về 0. Dạng hữu tỉ hoá có mẫu gần 2, nên kết quả gần 5×10^{-17} (tự tính bằng NumPy). Hai biểu thức bằng nhau trên số thực, nhưng cách đánh giá thứ hai tránh triệt tiêu.

Bài 5 (khó). Chứng minh phép trừ cùng một hằng số khỏi mọi logit không đổi softmax và suy ra công thức log-sum-exp ổn định.

Đáp án. Với mọi hằng số c,

$$\frac{e^{z_i - c}}{\sum_j e^{z_j - c}} = \frac{e^{-c} e^{z_i}}{e^{-c} \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Do đó có thể chọn $c = m = \max_j z_j$, khiến mọi số mũ không vượt quá 1. Với log-sum-exp, lấy log của tổng sau khi tách thừa số e^m:

$$\log \sum_i e^{z_i} = \log \left(e^m \sum_i e^{z_i - m} \right) = m + \log \sum_i e^{z_i - m}.$$

Đây là công thức ổn định, vì các số mũ bên trong tổng có miền giá trị dễ xử lý hơn. Phép biến đổi đại số là chính xác; phép tính hữu hạn vẫn có sai số làm tròn.

Tài liệu tham khảo nền: Goldberg, What Every Computer Scientist Should Know About Floating-Point Arithmetic; Higham, Accuracy and Stability of Numerical Algorithms; IEEE Standard for Floating-Point Arithmetic; math 15 (Online normalizer calculation for softmax); math 16 (FP8 Formats for Deep Learning).

Nền tảng 13

Chuẩn và điều kiện số: đo độ lớn và độ nhạy của phép tính

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector, ma trận và phép nhân ma trận với vector; SVD ở mức trực giác trong math nen05; số học máy tính ở math nen12

Dùng ở: bài math 06 (nghịch đảo Fisher và damping), math 17 (chuẩn phổ), math 18 (độ lớn gradient), math 19 (conditioning)

> Chương này dạy chuẩn và điều kiện số từ đầu, không phải bản dịch hay tóm tắt paper nào. Các phép tính ghi ”tự tính” đã được kiểm bằng numpy.

1. Vì sao cần chuẩn và điều kiện số

Chuẩn là một cách đo độ lớn của vector hay ma trận. **Điều kiện số** mô tả đầu ra có thể đổi mạnh đến đâu khi đầu vào đổi một lượng nhỏ. Hai ý này giúp ta trả lời những câu hỏi thực tế: một ma trận có khuếch đại tín hiệu không, hai nghiệm gần nhau hay không, residual nhỏ có đủ để tin nghiệm không, hoặc phép nghịch đảo có nhạy với nhiễu không.

Trong AI, chuẩn xuất hiện khi nói về độ lớn của trọng số, gradient, sai số dự đoán và độ nhạy của một lớp. Điều kiện số xuất hiện khi giải hệ tuyến tính, tính least squares, dùng ma trận Fisher hay Hessian, và khi phân tích vì sao sai số đầu vào nhỏ có thể thành sai số đầu ra lớn. Chúng là công cụ đo, không tự mình bảo đảm rằng mô hình tốt hay thuật toán ổn định.

2. Nhắc nhanh nền tảng: vector, độ dài, phép biến đổi

Một vector $x = (x_1, \dots, x_d)$ có nhiều cách đo độ lớn. Trong mặt phẳng, khoảng cách quen thuộc từ gốc đến $(3, 4)$ là 5, vì $3^2 + 4^2 = 25$ (tự tính). Nhưng nếu ta cộng độ lớn từng tọa độ, con số là 7; nếu chỉ quan tâm tọa độ lớn nhất, con số là 4 (tự tính). Các con số khác nhau vì chúng trả lời những câu hỏi hơi khác nhau.

Ma trận A biến một vector x thành vector mới Ax. Ma trận có thể kéo dài một hướng, co một hướng khác, hoặc xoay chúng. Ví dụ trực quan, ma trận đường chéo

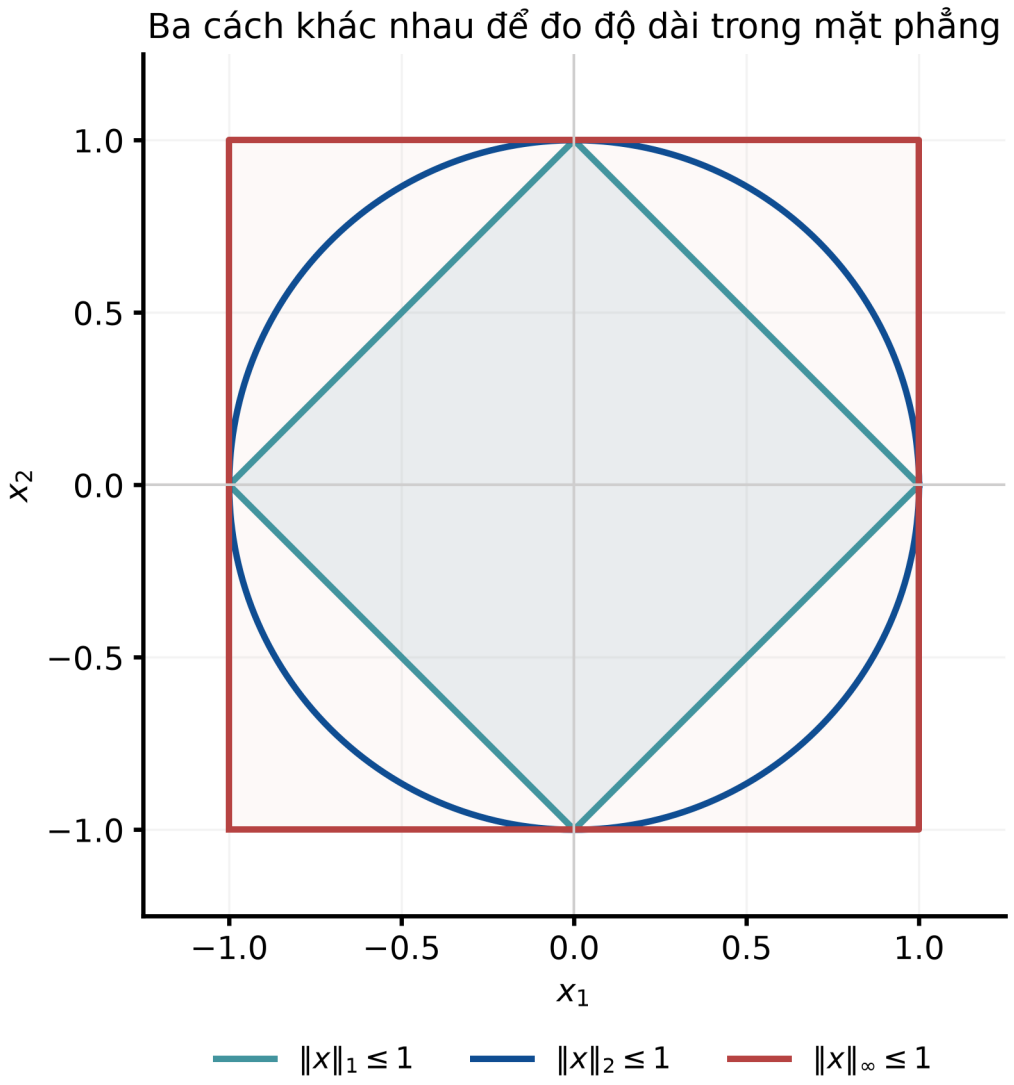
$$A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$$

kéo dài hướng ngang gấp ba và giữ nguyên hướng dọc (tự tính). Nếu đầu vào là $e_1 = (1, 0)^\top$, đầu ra có độ dài 3; nếu là $e_2 = (0, 1)^\top$, đầu ra có độ dài 1 (tự tính). Câu hỏi ”ma trận này lớn đến đâu” còn phụ thuộc ta xét hướng nào và cách đo độ dài ra sao.

3. Trực giác bằng hình: quả cầu đơn vị biến dạng

Một chuẩn xác định tập các vector có độ dài không quá 1, gọi là quả cầu đơn vị. Với độ dài Euclid, tập đó là hình tròn; với tổng trị tuyệt đối, đó là hình thoi; với độ lớn tọa độ lớn nhất, đó là hình vuông. Hình dạng khác nhau làm lộ ra điều mỗi phép đo ưu tiên.

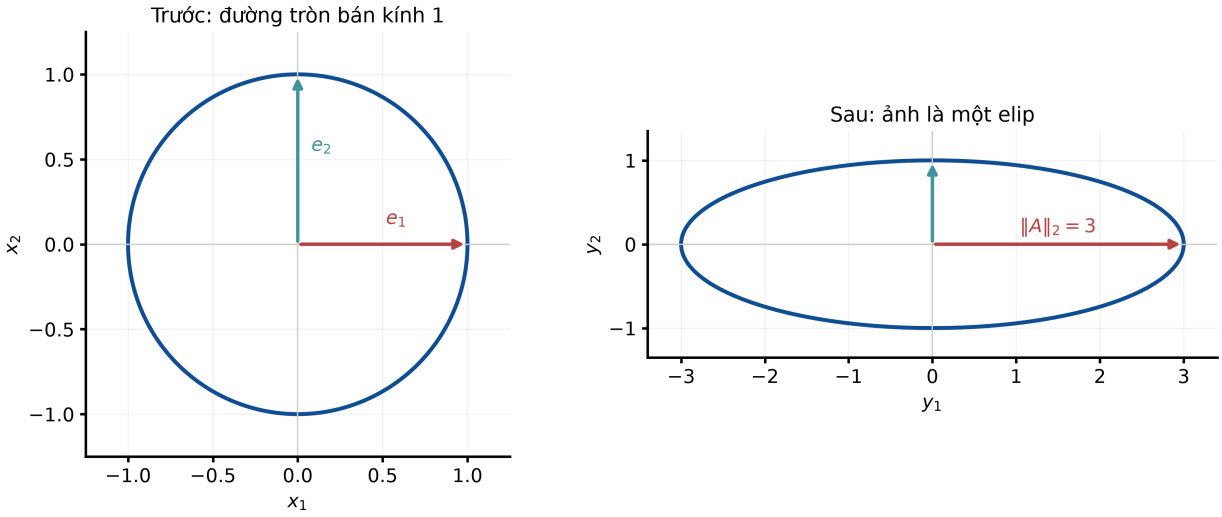
Cách đọc hình 1: ba miền đều là các vector có độ dài không quá 1, nhưng độ dài được đo theo ba quy tắc khác nhau. Hình thoi giới hạn tổng độ lớn hai tọa độ; hình tròn giới hạn khoảng cách Euclid; hình vuông giới



Hình 44: Quả cầu đơn vị của chuẩn 1, chuẩn 2 và chuẩn vô cực trong mặt phẳng (minh hoạ, tự tính)

hạn riêng từng tọa độ. Vì vậy, một vector có thể nằm trong quả cầu của chuẩn này nhưng ở ngoài quả cầu của chuẩn khác.

Giờ áp ma trận A lên mọi điểm trên hình tròn đơn vị. Nếu ảnh kéo dài xa nhất một hệ số 3, ta gọi 3 là mức khuếch đại tối đa theo độ dài Euclid. Hướng đạt được hệ số ấy là hướng quan trọng khi đo chuẩn phổ.



Hình 45: Ảnh của hình tròn đơn vị qua ma trận đường chéo diag(3,1), hướng ngang được kéo dài tối đa (minh hoạ, tự tính)

Cách đọc hình 2: panel trái là vector đầu vào có độ dài 1; panel phải là ảnh của cả đường tròn qua A. Elip vươn xa nhất theo trục ngang, từ đó thấy phép biến đổi có thể khuếch đại độ dài tối đa gấp 3 lần. Trục dọc chỉ dài 1, nên không phải mọi hướng đều bị khuếch đại như nhau.

4. Định nghĩa các chuẩn thường dùng

4.1 Chuẩn vector

Một chuẩn $\|x\|$ là một phép đo độ lớn thoả ba điều kiện: không âm và chỉ bằng 0 với vector không; nhân vector với số c làm độ lớn nhân với $|c|$; chuẩn của tổng không lớn hơn tổng các chuẩn. Điều kiện thứ ba là bất đẳng thức tam giác.

Họ chuẩn p được viết

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{1/p}, \qquad p \geq 1.$$

Ba trường hợp phổ biến là

$$\|x\|_1 = \sum_i |x_i|, \qquad \|x\|_2 = \sqrt{\sum_i x_i^2}, \qquad \|x\|_\infty = \max_i |x_i|.$$

Chuẩn 1 cộng độ lớn từng thành phần, chuẩn 2 đo độ dài Euclid, chuẩn vô cực nhìn thành phần lớn nhất. Trong bài toán thưa, chuẩn 1 thường được dùng làm đại lượng phạt; trong hình học và sai số, chuẩn 2 thường là lựa chọn mặc định; chuẩn vô cực hữu ích khi muốn giới hạn sai lệch lớn nhất ở một tọa độ.

4.2 Chuẩn ma trận cảm sinh

Một ma trận là phép biến đổi tuyến tính, nên ta đo nó bằng mức lớn nhất mà nó có thể kéo dài một vector đơn vị:

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}.$$

Đây là **chuẩn ma trận cảm sinh** từ chuẩn vector p. Theo định nghĩa, với mọi vector x,

$$\|Ax\|_p \leq \|A\|_p \, \|x\|_p.$$

Chuẩn cảm sinh còn thoả $\|AB\|_p \leq \|A\|_p \|B\|_p$. Đọc theo phép biến đổi, ma trận A khuếch đại một vector nhiều nhất $\|A\|_p$ lần, còn hai phép biến đổi nối tiếp không thể khuếch đại nhiều hơn tích các mức riêng của chúng.

Với chuẩn 1 và chuẩn vô cực, có thể tính trực tiếp:

$$\|A\|_1 = \max_j \sum_i |A_{ij}|, \qquad \|A\|_\infty = \max_i \sum_j |A_{ij}|.$$

Chuẩn 1 lấy tổng trị tuyệt đối lớn nhất trong các cột; chuẩn vô cực lấy tổng lớn nhất trong các hàng.

4.3 Chuẩn phổ và chuẩn Frobenius

Chuẩn cảm sinh từ chuẩn Euclid được gọi là **chuẩn phổ**:

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sigma_{\max}(A),$$

trong đó $\sigma_{\max}(A)$ là giá trị kỳ dị lớn nhất của A. Hình 2 minh hoạ định nghĩa này: bán kính lớn nhất của ảnh quả cầu Euclid chính là chuẩn phổ.

Một cách đo khác là **chuẩn Frobenius**, giống căn tổng bình phương mọi ô của ma trận:

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}.$$

Chuẩn phổ và Frobenius đều là chuẩn ma trận, nhưng trả lời câu hỏi khác nhau. Chuẩn phổ đo khuếch đại tối đa theo một hướng; Frobenius cộng năng lượng trên mọi hướng và mọi ô. Chúng liên hệ với nhau qua các giá trị kỳ dị:

$$\|A\|_2 = \sigma_1, \qquad \|A\|_F = \sqrt{\sum_i \sigma_i^2}.$$

Nếu chỉ cần biết một lớp có thể phóng đại tín hiệu mạnh đến đâu, dùng chuẩn phổ. Nếu cần đo tổng bình phương trọng số hoặc sai số ma trận, chuẩn Frobenius thường tiện hơn.

5. Tính tay các chuẩn của một ma trận

Xét lại

$$A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}.$$

Với chuẩn 1, tổng trị tuyệt đối hai cột lần lượt là 3 và 1, nên $\|A\|_1 = 3$ (tự tính). Với chuẩn vô cực, tổng trị tuyệt đối hai hàng lần lượt là 3 và 1, nên $\|A\|_\infty = 3$ (tự tính).

Để tính chuẩn phổ, lấy một vector đơn vị $x = (x_1, x_2)$, thoả $x_1^2 + x_2^2 = 1$. Khi đó

$$\|Ax\|_2^2 = 9x_1^2 + x_2^2 \le 9(x_1^2 + x_2^2) = 9.$$

Vậy $\|Ax\|_2 \le 3$. Dấu bằng đạt được khi $x = (1, 0)$, nên $\|A\|_2 = 3$ (tự tính). Đây cũng là giá trị kỳ dị lớn nhất của ma trận đường chéo này.

Cuối cùng, tổng bình phương bốn ô là $3^2 + 0^2 + 0^2 + 1^2 = 10$, do đó $\|A\|_F = \sqrt{10} \approx 3.1623$ (tự tính). Kết quả không mâu thuẫn với chuẩn phổ bằng 3: Frobenius đo tổng đóng góp theo nhiều hướng, còn chuẩn phổ chỉ đo hướng bị kéo dài nhiều nhất.

Một cặp bất đẳng thức hữu ích cho ma trận $m \times n$ là

$$\|A\|_2 \le \|A\|_F \le \sqrt{\min(m, n)}\, \|A\|_2.$$

Nó cho ta kiểm tra nhanh xem hai phép đo có cùng bậc độ lớn không. Nếu ma trận có một hướng chi phối mạnh, chuẩn phổ và Frobenius có thể gần nhau; nếu nhiều giá trị kỳ dị cùng lớn, Frobenius có thể lớn hơn đáng kể.

6. Điều kiện số: bài toán nhạy đến đâu

6.1 Điều kiện số của một ma trận

Với ma trận vuông khả nghịch A, điều kiện số theo chuẩn p được định nghĩa

$$\kappa_p(A) = \|A\|_p \, \|A^{-1}\|_p.$$

Vì $1 = \|I\|_p = \|AA^{-1}\|_p \le \|A\|_p \|A^{-1}\|_p$, ta luôn có $\kappa_p(A) \ge 1$. Giá trị gần 1 cho biết phép biến đổi không quá méo theo chuẩn đó; giá trị lớn báo rằng có hướng bị co rất mạnh so với hướng khác và nghịch đảo sẽ khuếch đại sai số theo hướng bị co.

Theo chuẩn phổ, SVD cho công thức dễ đọc:

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

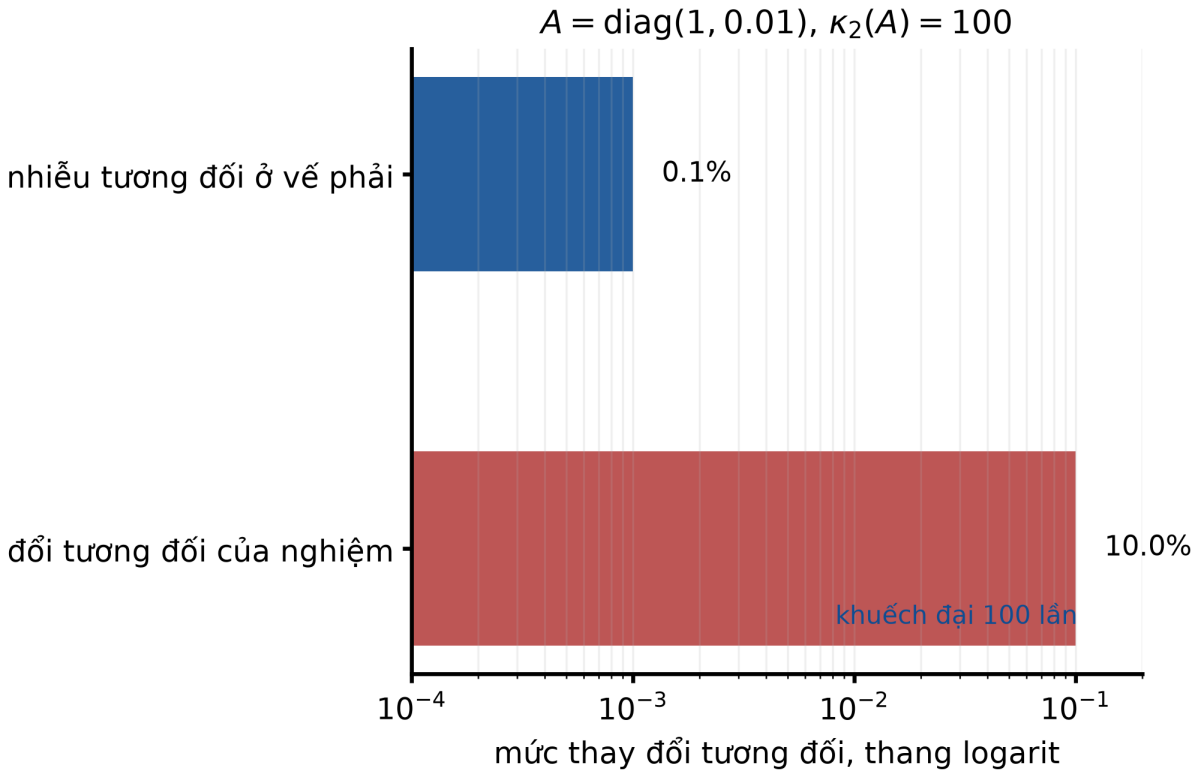
Ma trận suy biến có giá trị kỳ dị nhỏ nhất bằng 0, không có nghịch đảo và có điều kiện số vô hạn. Với ma trận chữ nhật hạng đầy đủ, ta dùng tỉ số giữa giá trị kỳ dị lớn nhất và nhỏ nhất khác 0 để mô tả độ nhạy trên không gian mà phép biến đổi còn phân biệt được. Phần SVD ở math nen05 giải thích các giá trị kỳ dị này theo hình học.

6.2 Nhiễu đầu vào được phóng đại

Xét hệ $Ax = b$ với $A = \mathrm{diag}(1, 0.01)$, $b = (1, 0)$, nên nghiệm là $x = (1, 0)$ (tự tính). Ta làm nhiễu vế phải một lượng $\delta b = (0, 0.001)$. Khi giải lại, nghiệm thành $\hat{x} = (1, 0.1)$ (tự tính). Nhiễu tương đối ở b là 0.001;

sai khác tương đối của nghiệm là 0.1 (tự tính). Tỉ số phóng đại là 100, đúng bằng $\kappa_2(A) = 1/0.01 = 100$ (tự tính).

Đây là tình huống gần mức xấu nhất: nghiệm ban đầu nằm theo hướng được khuếch đại tốt, còn nhiễu đi theo hướng mà A co lại nhiều. Với một vector khác, độ khuếch đại có thể nhỏ hơn. Điều kiện số là giới hạn xấu nhất theo hướng, không phải dự đoán chính xác sai số cho mọi đầu vào.



Hình 46: Độ nhiễu tương đối ở vế phải và độ đổi tương đối của nghiệm, chênh nhau 100 lần (minh hoạ, tự tính)

Cách đọc hình 3: thanh xanh biểu diễn nhiễu tương đối ở vế phải, thanh đỏ biểu diễn độ đổi tương đối của nghiệm. Ma trận co thành phần thứ hai xuống hệ số 0.01, nên nhiễu đó được khuếch đại thành thay đổi nghiệm lớn gấp 100 lần theo tỉ lệ. Đây là trường hợp đạt chặn điều kiện số $\kappa_2(A) = 100$.

Với A vuông khả nghịch, $b \neq 0$ và nghiệm $x \neq 0$, nếu chỉ nhiễu b và A cố định, ta có chặn. Dùng cùng một chuẩn vector cho $x, b, \delta x, \delta b$ và chuẩn ma trận cảm sinh tương ứng cho A:

$$\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|}.$$

Nếu cả A và b cùng bị nhiễu, đặt $\hat{A} = A + \delta A$ và $\hat{b} = b + \delta b$. Dùng cùng giả thiết khả nghịch và $b \neq 0$ như trên, cùng các chuẩn tương thích; khi $\kappa(A)\|\delta A\|/\|A\| < 1$, một chặn thường dùng là

$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \frac{\kappa(A)}{1 - \kappa(A)\|\delta A\|/\|A\|} \left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right).$$

Các chuẩn ở vế trái và phải phải cùng loại. Khi ma trận bị nhiễu đủ lớn so với $1/\kappa(A)$, mẫu số của chặn tiến gần 0: ta không còn bảo đảm nghiệm nhiễu gần nghiệm ban đầu.

6.3 Điều kiện số của một hàm

Ý tưởng điều kiện số không chỉ dành cho ma trận. Với một hàm vô hướng $y = f(x)$, điều kiện tuyệt đối cục bộ đo gần đúng mức đổi của đầu ra trên một mức đổi đầu vào nhỏ, thường liên quan đến $|f'(x)|$. Nếu quan tâm sai số tương đối, điều kiện tương đối cục bộ là

$$\kappa_f(x) = \left| \frac{x f'(x)}{f(x)} \right|,$$

khi các đại lượng xác định và khác 0. Ví dụ $f(x) = x^2$ có $\kappa_f(x) = 2$ với $x \neq 0$ (tự tính), nên sai số tương đối nhỏ ở x tạo sai số tương đối xấp xỉ gấp đôi ở đầu ra. Điều kiện luôn phụ thuộc cách đo sai số và vùng đầu vào đang xét.

7. Điều kiện số không phải độ ổn định của thuật toán

Điều kiện số thuộc về **bài toán**: nó hỏi nghiệm thay đổi bao nhiêu nếu dữ liệu thay đổi. Độ ổn định số thuộc về **thuật toán**: nó hỏi cách tính cụ thể có thêm bao nhiêu sai số so với sai số vốn đã có trong bài toán.

Một bài toán có điều kiện số xấu không thể được biến thành bài toán nhạy thấp chỉ bằng cách viết lại mã. Tuy vậy, thuật toán ổn định có thể tìm nghiệm tốt nhất mà độ chính xác đầu vào cho phép. Ngược lại, với bài toán điều kiện số tốt, một thuật toán bất ổn vẫn có thể khuếch đại sai số tính toán không cần thiết.

7.1 Residual và sai số nghiệm

Ta có nghiệm đúng x của $Ax = b$, nhưng trên máy tìm được $\hat{x}$. **Residual** là

$$r = b - A\hat{x}.$$

Residual đo mức vi phạm phương trình. **Sai số tiến** đo khoảng cách giữa nghiệm tính được và nghiệm đúng, chẳng hạn $\|\hat{x} - x\|/\|x\|$. Hai đại lượng này không giống nhau. Từ $A(\hat{x} - x) = -r$, suy ra

$$\|\hat{x} - x\| \leq \|A^{-1}\| \, \|r\|.$$

Nếu A^{-1} có chuẩn lớn, một residual nhỏ vẫn có thể tương ứng với sai số nghiệm đáng kể.

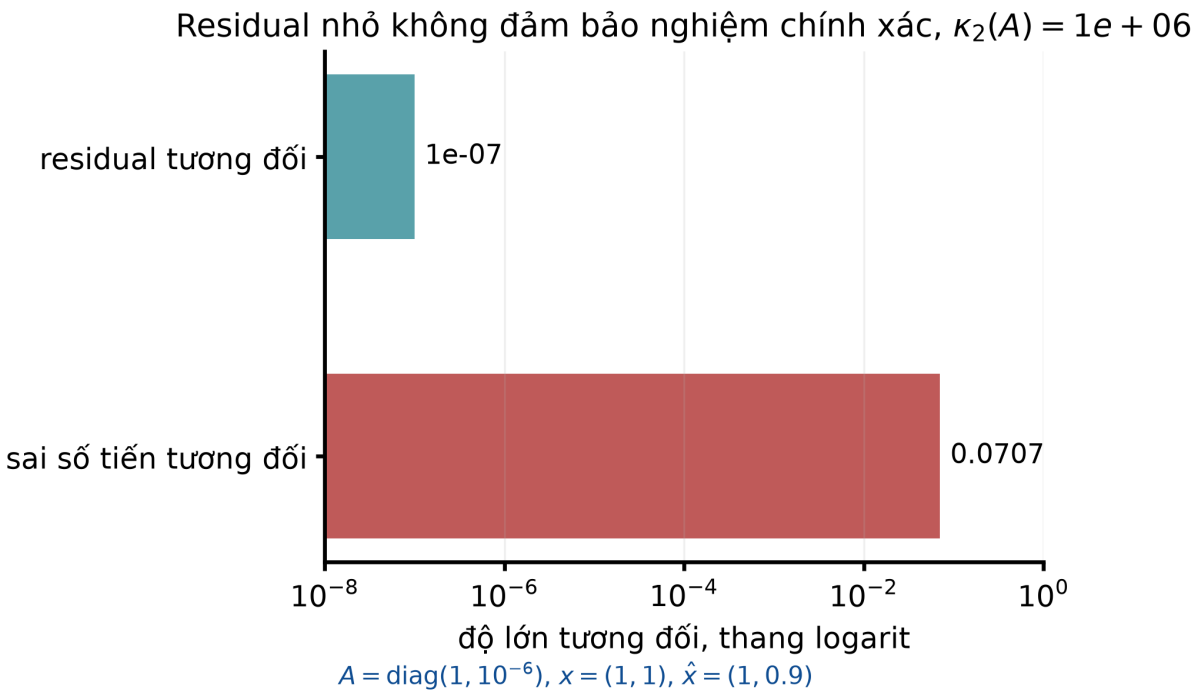
Ví dụ, lấy $A = \mathrm{diag}(1, 10^{-6})$, nghiệm đúng $x = (1, 1)$ và $b = (1, 10^{-6})$. Nếu ta có $\hat{x} = (1, 0.9)$, residual bằng $(0, 10^{-7})$ (tự tính). Chuẩn residual tương đối theo b xấp xỉ 10^{-7}, nhưng sai số tương đối của nghiệm là khoảng 0.0707, tức 7.07% (tự tính). Nguyên nhân là $\kappa_2(A) = 10^6$ (tự tính).

Cách đọc hình 4: thanh trên là residual tương đối cỡ 10^{-7}, nhìn rất nhỏ; thanh dưới là sai số tiến của nghiệm, khoảng 7.07%. Cùng ví dụ có điều kiện số 10^6, nên một sai lệch nhỏ trong phương trình chưa đủ để kết luận từng thành phần nghiệm chính xác. Hãy xem cả residual, điều kiện số và độ chính xác đầu vào.

7.2 Sai số lùi

Một cách đánh giá thuật toán khác là hỏi: nghiệm tính được có phải nghiệm chính xác của một bài toán hơi khác không? Với hệ tuyến tính, sai số lùi liên hệ với residual và kích cỡ ma trận, nghiệm, vế phải. Nếu sai số lùi nhỏ, thuật toán đã giải rất chính xác một hệ gần hệ ban đầu. Sai số tiến vẫn có thể lớn khi bài toán điều kiện số xấu.

Đây là lý do một solver tốt thường được gọi là backward stable: kết quả của nó có thể xem như nghiệm chính xác của dữ liệu hơi nhiễu. Để kết luận sai số tiến nhỏ, ta còn phải biết điều kiện số của bài toán đủ tốt. Cặp khái niệm này phân biệt lỗi của cách tính với độ nhạy vốn có của mô hình toán.



Hình 47: Một residual nhỏ có thể cùng tồn tại với sai số nghiệm đáng kể khi ma trận có điều kiện số lớn (minh hoạ, tự tính)

8. Liên hệ giữa chuẩn, điều kiện số và SVD

Với $A = U\Sigma V^\top$, các ma trận trực giao U, V chỉ đổi hướng; ma trận đường chéo Σ co giãn từng hướng riêng. Giá trị kỳ dị lớn nhất cho mức khuếch đại tối đa, còn giá trị nhỏ nhất là hệ số giãn nhỏ nhất. Nếu nó nhỏ hơn 1, hướng tương ứng bị co nhiều nhất; nếu lớn hơn hoặc bằng 1, hướng đó vẫn bị giãn hoặc giữ nguyên. Tỉ số của chúng chính là $\kappa_2(A)$ nếu A vuông và khả nghịch.

Ví dụ $A = \mathrm{diag}(1, 0.01)$ có các giá trị kỳ dị 1 và 0.01, nên điều kiện số theo chuẩn 2 bằng 100 (tự tính). Nếu đổi thành $\mathrm{diag}(1, 10^{-6})$, hướng thứ hai bị co mạnh hơn và điều kiện số thành 10^6 (tự tính). Chỉ cần một hướng gần như bị ép về 0 là phép nghịch đảo phải khuếch đại theo hướng đó rất mạnh.

8.1 Bình phương điều kiện khi lập phương trình chuẩn

Trong least squares, một cách quen thuộc là lập phương trình chuẩn $A^\top Ax = A^\top b$. Các giá trị kỳ dị khác 0 của $A^\top A$ là bình phương các giá trị kỳ dị khác 0 của A; nếu A thiếu hạng cột, $A^\top A$ có thêm giá trị kỳ dị bằng 0. Vì thế, chỉ khi A có hạng cột đầy đủ,

$$\kappa_2(A^\top A) = \kappa_2(A)^2, \qquad A \text{ có hạng cột đầy đủ.}$$

Nếu A có hạng cột đầy đủ và $\kappa_2(A) = 100$, ma trận bình phương có điều kiện số 10000 (tự tính). Nếu A thiếu hạng cột, $A^\top A$ có giá trị kỳ dị bằng 0 và điều kiện số vô hạn. Chẳng hạn $A = (1 \;\; 0)$ có giá trị kỳ dị khác 0 duy nhất bằng 1, nhưng $A^\top A = \mathrm{diag}(1, 0)$ suy biến (tự tính). Vì vậy, giải least squares bằng QR hoặc SVD thường ổn định hơn khi ma trận kém điều kiện, dù phương trình chuẩn có thể dễ viết. Các ví dụ về SVD và hạng ở math nen05 giúp nhìn thấy vì sao giá trị kỳ dị nhỏ gây rắc rối khi đảo.

9. Nối sang AI

(a) Chuẩn phổ và giới hạn Lipschitz. Chuẩn phổ của ma trận trọng số là mức khuếch đại tối đa của lớp tuyến tính theo chuẩn Euclid. Với mạng chuỗi, nếu các hàm kích hoạt là 1-Lipschitz, tích chuẩn phổ của các ma trận cho một chặn Lipschitz của mạng; nếu activation có hằng số Lipschitz khác 1, cần nhân thêm các hằng số đó. Với nhánh dư, phải cộng các chặn Lipschitz của những nhánh được cộng. Bài math 17 dùng giá trị kỳ dị lớn nhất để chuẩn hoá trọng số của discriminator. Chuẩn nhỏ hơn không tự động bảo đảm GAN ổn định, nhưng cung cấp một đại lượng rõ ràng để kiểm soát mức khuếch đại của từng lớp tuyến tính.

(b) Gradient, Jacobian và chuỗi lớp. Jacobian biến một nhiễu nhỏ ở đầu vào thành nhiễu nhỏ ở đầu ra. Chuẩn phổ của Jacobian chặn mức khuếch đại cục bộ; qua nhiều lớp, các mức khuếch đại có thể nhân lên. Điều này giúp diễn giải vì sao gradient có thể co hoặc phình khi lan truyền qua mạng. Bài math 18 nghiên cứu một hiện tượng khác liên quan đến cấu trúc tương quan của gradient; không nên đồng nhất mọi vấn đề gradient với điều kiện số của một ma trận duy nhất.

(c) Fisher, Hessian và damping. Natural gradient dùng nghịch đảo Fisher để đổi hướng cập nhật. Nếu Fisher có trị riêng rất nhỏ, nghịch đảo khuếch đại mạnh theo hướng tương ứng. Damping cộng một lượng lên đường chéo để tránh chia cho giá trị quá nhỏ. Bài math 06 phân tích vai trò của damping và ma trận độ cong; điều kiện số ở đây giúp hiểu vì sao phép nghịch đảo nhạy.

(d) Không gian tham số hiệu dụng. Một bài toán có thể khó giải trực tiếp trong không gian tọa độ ban đầu nhưng dễ hơn trong không gian con hoặc sau tái tham số hoá. Bài math 19 thảo luận conditioning như một cách lý giải khả dĩ cho sự khác nhau giữa tối ưu trực tiếp và tối ưu trong không gian con. Điều kiện số là một dấu hiệu chẩn đoán, không phải bằng chứng duy nhất về nguyên nhân hội tụ.

(e) Regularization và least squares. Khi ma trận gần suy biến, thêm một hạng tử như λI có thể làm bài toán nghịch đảo bớt nhạy. Cái giá phải trả là nghiệm bị thay đổi theo mức regularization. Cần cân bằng sai số dữ liệu, độ nhạy và sai lệch do regularization, thay vì chỉ cố làm condition number nhỏ nhất.

10. Sai lầm hay gặp

- **Dùng từ "chuẩn" mà không nói rõ loại nào.** Chuẩn 1, chuẩn 2, chuẩn vô cực và Frobenius không đo cùng một thứ.
- **Nghĩ residual nhỏ nghĩa là nghiệm chính xác.** Nếu ma trận có điều kiện số lớn, sai số nghiệm vẫn có thể lớn.
- **Nhầm điều kiện số với độ ổn định thuật toán.** Bài toán có thể nhạy nhưng thuật toán ổn định, hoặc bài toán tốt nhưng thuật toán bất ổn.
- **Nghịch đảo ma trận chỉ vì công thức có A^{-1}.** Trong tính toán, nên dùng solver phù hợp; lập nghịch đảo tường minh thường thêm sai số và tốn công.
- **Bình phương ma trận điều kiện mà không để ý.** Lập $A^\top A$ trong phương trình chuẩn làm bình phương điều kiện số theo chuẩn 2.
- **Cho rằng chuẩn phổ nhỏ bảo đảm mọi hành vi mạng tốt.** Nó chặn mức khuếch đại tuyến tính theo một thước đo, không đo độ đúng của dự đoán hay toàn bộ động lực huấn luyện.

11. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Chuẩn vector p	$\|x\|_p = (\sum_i \|x_i\|^p)^{1/p}$	đo độ lớn theo cách chọn p
Chuẩn cảm sinh	$\|A\|_p = \max_{x\neq 0} \|Ax\|_p / \|x\|_p$	mức khuếch đại tối đa
Chuẩn phổ	$\|A\|_2 = \sigma_{\max}(A)$	khuếch đại Euclid lớn nhất
Frobenius	$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}$	tổng độ lớn bình phương của các ô

Khái niệm	Công thức	Ý nghĩa
Điều kiện số	$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p$	độ nhạy của phép nghịch đảo
Điều kiện theo SVD	$\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$	tỉ số co giãn lớn nhất và nhỏ nhất
Nhiễu vế phải	$\|\delta x\|/\|x\| \le \kappa(A)\|\delta b\|/\|b\|$	chặn khuếch đại sai số tương đối
Residual	$r = b - A\hat{x}$	mức vi phạm hệ phương trình
Bình phương điều kiện	$\kappa_2(A^\top A) = \kappa_2(A)^2$, nếu A hạng cột đầy đủ	hệ quả của bình phương giá trị kỳ dị

12. Bài tập tự tính

Hãy thử làm trước khi đọc đáp án. Các phép tính trong đáp án đã được kiểm lại bằng NumPy.

Bài 1 (dễ). Với $x = (3, -4)$, tính chuẩn 1, chuẩn 2 và chuẩn vô cực.

Đáp án. $\|x\|_1 = |3| + |-4| = 7$, $\|x\|_2 = \sqrt{3^2 + (-4)^2} = 5$, và $\|x\|_\infty = 4$ (tự tính).

Bài 2 (dễ vừa). Với $A = \begin{pmatrix} 1 & -2 \\ 3 & 0 \end{pmatrix}$, tính chuẩn ma trận 1 và chuẩn ma trận vô cực.

Đáp án. Tổng trị tuyệt đối theo cột là 4 và 2, nên $\|A\|_1 = 4$. Tổng theo hàng là 3 và 3, nên $\|A\|_\infty = 3$ (tự tính).

Bài 3 (vừa). Tính chuẩn phổ, chuẩn Frobenius và điều kiện số theo chuẩn 2 của $A = \mathrm{diag}(4, 0.5)$.

Đáp án. Giá trị kỳ dị là 4 và 0.5. Do đó $\|A\|_2 = 4$, $\|A\|_F = \sqrt{16 + 0.25} = \sqrt{65}/2 \approx 4.0311$, và $\kappa_2(A) = 4/0.5 = 8$ (tự tính).

Bài 4 (vừa khó). Với $A = \mathrm{diag}(1, 0.01)$, $b = (1, 0)$, và $\delta b = (0, 0.001)$, tính nghiệm trước và sau nhiễu. Mức nhiễu tương đối ở b được khuếch đại bao nhiêu lần trong nghiệm?

Đáp án. Nghiệm ban đầu là $x = (1, 0)$; sau nhiễu, $\hat{x} = A^{-1}(b + \delta b) = (1, 0.1)$. Độ nhiễu tương đối của b là 0.001, sai khác tương đối của nghiệm là 0.1, nên hệ số phóng đại bằng 100, đúng với $\kappa_2(A) = 100$ (tự tính).

Bài 5 (khó). Cho $A = \mathrm{diag}(1, 10^{-6})$, nghiệm thật $x = (1, 1)$ và $b = Ax$. Nếu nghiệm tính được là $\hat{x} = (1, 0.9)$, hãy tính residual tương đối theo b, sai số tiến tương đối và điều kiện số.

Đáp án. $b = (1, 10^{-6})$ và $r = b - A\hat{x} = (0, 10^{-7})$, nên residual tương đối là khoảng 10^{-7}. Sai số nghiệm có độ dài 0.1, còn $\|x\|_2 = \sqrt{2}$, nên sai số tiến tương đối là $0.1/\sqrt{2} \approx 0.07071$, tức khoảng 7.07%. Điều kiện số là $1/10^{-6} = 10^6$ (tự tính). Ví dụ này cho thấy residual nhỏ không tự nó đảm bảo sai số nghiệm nhỏ.

Tài liệu tham khảo nền: Trefethen & Bau, Numerical Linear Algebra; Golub & Van Loan, Matrix Computations; Higham, Accuracy and Stability of Numerical Algorithms; math 06 (Natural Gradient), math 17 (Spectral Normalization), math 18 (The Shattered Gradients Problem), math 19 (Intrinsic Dimension).

Nền tảng 14

Hàm lồi và tập lồi: nối hai điểm để hiểu hình học của tối ưu

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector và tích vô hướng, đạo hàm, gradient và Hessian, kỳ vọng rời rạc (đều được nhắc nhanh dưới đây)

Dùng ở: math 07 (hàm sinh lồi và Bregman divergence), math 08 và math 09 (suy luận biến phân), math 24 (giả thiết lồi mạnh)

> Chương này được viết để học từ đầu, không phải bản dịch hay tóm tắt một paper. Các phép tính được ghi "tự tính" đã kiểm bằng numpy; hình minh hoạ cũng dùng các giá trị tự tính.

1. Vì sao cần tập lồi và hàm lồi

Một **tập lồi** có tính chất: nối hai điểm bất kỳ trong tập bằng đoạn thẳng thì cả đoạn vẫn ở trong tập. Một **hàm lồi** có đồ thị nằm không cao hơn dây cung nối hai điểm bất kỳ trên đồ thị. Cả hai đều bắt đầu bằng cùng một thao tác: chọn hai điểm rồi xét mọi điểm ở giữa.

Trong học máy, ta hay chọn tham số để hàm mất mát nhỏ nhất. Nếu miền được phép là tập lồi và hàm mất mát là hàm lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục. Điều này giúp ta chứng minh một điểm tìm được thật sự tốt nhất trong miền, thay vì chỉ tốt hơn các điểm ở gần. Hồi quy tuyến tính và logistic với đặc trưng cố định là những ví dụ quan trọng; chúng cũng cho ta bài toán nhỏ để hiểu các phương pháp tối ưu.

Lồi không có nghĩa là thuật toán bất kỳ lập tức tìm ra nghiệm, và cũng không có nghĩa là mô hình dự đoán tốt trên dữ liệu mới. Mạng nơ-ron sâu thường không lồi theo toàn bộ tham số; nhiều kết quả của lý thuyết lồi vẫn hữu ích cho một khối con, một bài toán thay thế, hoặc để phân biệt điều được bảo đảm với điều chỉ quan sát được trong thí nghiệm.

2. Nhắc nhanh: đoạn thẳng, độ dốc và trung bình

Tổ hợp có trọng số. Cho hai vector x, y và một số t từ 0 đến 1. Điểm

$$z = (1 - t)x + ty$$

nằm trên đoạn nối x với y. Khi $t = 0$ ta ở x; khi $t = 1$ ta ở y; khi t tăng, ta đi từ x về phía y. Hai trọng số không âm và cộng lại bằng 1, vì thế z là một **tổ hợp lồi**. Ví dụ (tự tính), với $x = (-0{,}8, 0)$, $y = (0{,}8, 0)$ và $t = 0{,}25$ thì $z = (-0{,}4, 0)$.

Gradient và tích vô hướng. Với hàm khả vi f của vector x, gradient $\nabla f(x)$ chứa các đạo hàm riêng. Với bước nhỏ d, thay đổi hàm gần bằng $\nabla f(x)^\top d$. Tích này dương khi hướng d có thành phần cùng hướng gradient, âm khi nó có thành phần ngược hướng. Chương này dùng nó để đọc tiếp tuyến và điều kiện tối ưu trên một miền có biên.

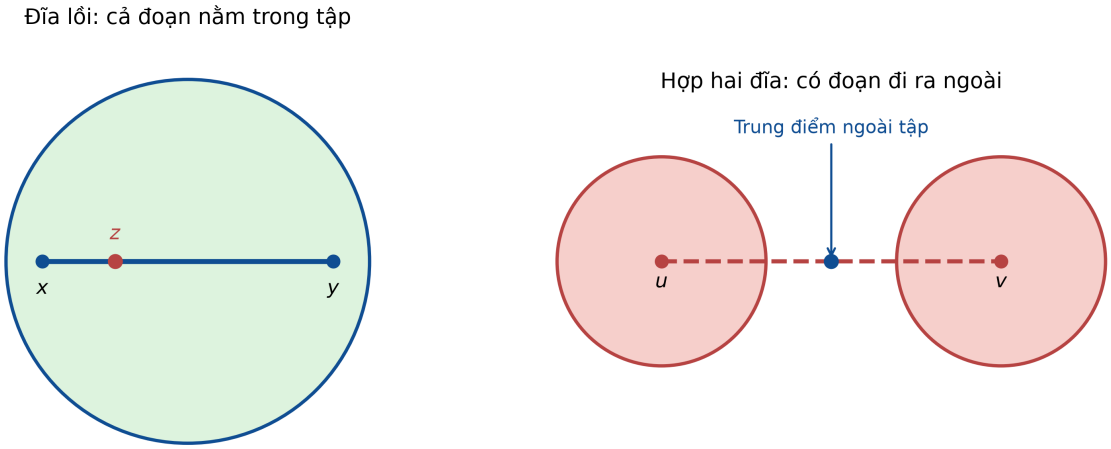
Hessian. Ma trận Hessian $\nabla^2 f(x)$ chứa các đạo hàm riêng cấp hai. Với một hướng v, số $v^\top \nabla^2 f(x) v$ cho biết độ cong cấp hai theo hướng đó. Nếu nó không âm với mọi v, Hessian được gọi là **nửa xác định dương**. Một hướng có độ cong bằng 0 là có thể, nên tên này không yêu cầu mọi trị riêng đều dương.

Trung bình có trọng số. Nếu các xác suất p_i không âm và có tổng bằng 1, kỳ vọng của một biến nhận các giá trị x_i là $\mathbb{E}[X] = \sum_i p_i x_i$. Đó cũng là một tổ hợp lồi. Bất đẳng thức Jensen ở mục 12 nối thao tác lấy trung bình này với tính lồi.

3. Nhìn tập lồi trước khi viết định nghĩa

Hãy tưởng tượng một tấm bìa hình đĩa, tính cả đường viền. Chọn hai điểm bất kỳ trên tấm bìa, kéo căng một sợi chỉ giữa chúng. Sợi chỉ luôn nằm trên tấm bìa. Tập lồi có thể có góc, cạnh thẳng hoặc nằm trong không gian nhiều chiều, không nhất thiết phải tròn hay có diện tích.

Bây giờ đặt hai tấm bìa hình đĩa rời nhau. Hai tâm đều thuộc hợp hai đĩa, nhưng trung điểm ở khe trống không thuộc hợp đó. Chỉ cần tìm được **một cặp** như vậy là đủ bác bỏ tính lồi.



Hình 48: Một đĩa lồi và hợp hai đĩa không lồi (minh hoạ, tự tính)

Cách đọc hình 1: bên trái, đoạn xanh nối x và y nằm hoàn toàn trong đĩa, còn điểm đỏ z là một điểm ở giữa. Bên phải, hai điểm đỏ là tâm của hai đĩa, đường đứt nối chúng đi qua phần trắng. Chấm xanh là trung điểm nằm ngoài hợp hai đĩa: cặp điểm này là phản ví dụ chứng minh hợp không lồi.

Tập hợp là đối tượng chứa các điểm đầu vào. Nó chưa nói hàm mất mát cao hay thấp tại những điểm đó. Hình 1 kiểm tra **miền**, còn dây cung ở mục 5 kiểm tra **giá trị của hàm** trên miền.

4. Định nghĩa và ví dụ của tập lồi

Một tập $C \subseteq \mathbb{R}^n$ gọi là **lồi** nếu với mọi $x, y \in C$ và mọi $t \in [0, 1]$,

$$(1 - t)x + ty \in C.$$

Cụm ”với mọi” xuất hiện hai lần: mọi cặp điểm và mọi vị trí trên đoạn. Để chứng minh một tập lồi, ta lấy cặp và trọng số tổng quát rồi chỉ ra điểm pha trộn còn thoả mọi điều kiện của tập. Để chứng minh không lồi, một cặp cụ thể có một điểm giữa đoạn nằm ngoài tập là đủ.

4.1 Nửa không gian và tam giác

Với vector $a \neq 0$ và số b cố định, tập $H = \{x : a^\top x \leq b\}$ là một **nửa không gian**. Thật vậy, nếu $a^\top x \leq b$ và $a^\top y \leq b$, thì với mọi $t \in [0, 1]$,

$$a^\top\big((1 - t)x + ty\big) = (1 - t)a^\top x + ta^\top y \leq (1 - t)b + tb = b.$$

Tập nghiệm của một phương trình affine $a^\top x = b$ cũng lồi: phép pha trộn vẫn cho đúng b. Giao của bất kỳ họ tập lồi nào cũng lồi, vì đoạn giữa hai điểm thuộc giao nằm trong từng tập. Trái lại, hợp của hai tập lồi có thể không lồi, như hình 1.

Một miền hay dùng trong chương là tam giác, tính cả biên:

$$C = \{(x, y) : x \geq 0,\ y \geq 0,\ x + y \leq 2\}.$$

Mỗi điều kiện xác định một nửa không gian, nên giao của chúng lồi. Ba đỉnh của tam giác là $(0, 0)$, $(2, 0)$ và $(0, 2)$ (tự tính). Bỏ hoặc giữ đường biên không tự động quyết định tính lồi: một đĩa mở và một đĩa đóng đều lồi. Tuy nhiên, việc có biên ảnh hưởng tới **sự tồn tại** của nghiệm tối ưu, xem mục 9.4.

4.2 Tổ hợp lồi của nhiều điểm

Nếu $x_1, \dots, x_k \in C$, $\alpha_i \geq 0$ và $\sum_i \alpha_i = 1$, thì tính lồi kéo theo $\sum_i \alpha_i x_i \in C$. Ta có thể hiểu điều này bằng cách pha lần lượt hai điểm, rồi tiếp tục pha kết quả với điểm kế tiếp.

Bao lồi của một tập điểm là tập gồm mọi tổ hợp lồi hữu hạn của các điểm đó. Với ba đỉnh không thẳng hàng, bao lồi là tam giác đầy đủ tính cả cạnh và phần trong. Bao lồi là tập lồi nhỏ nhất chứa các điểm đã cho. Đây là một khái niệm hình học, không phụ thuộc hàm mất mát nào.

5. Nhìn hàm lồi: đồ thị ở dưới dây cung

Bây giờ lấy một hàm số một biến. Chọn hai vị trí trên trục ngang, tìm hai điểm trên đồ thị, rồi nối chúng bằng một đoạn thẳng gọi là **dây cung**. Với hàm lồi, giữa hai vị trí đó, đồ thị không vượt lên trên dây cung. Còn đường tiếp tuyến của hàm lồi khả vi nằm không cao hơn đồ thị trên toàn miền lồi.

Cách đọc hình 2: đường xanh đậm là $f(t) = t^2$, dây cung đỏ nối hai điểm của đồ thị tại 0 và 2, còn đường xanh đứt là tiếp tuyến tại 1. Tại vị trí 1,5, hình vuông đỏ ở mức 3 cao hơn chấm xanh ở mức 2,25; tam giác trên tiếp tuyến ở mức 2 thấp hơn chấm xanh. Hai so sánh này dùng cùng một hoành độ nhưng nói về hai đường khác nhau, nên hãy theo màu và ký hiệu.

Định nghĩa chính thức: cho miền lồi C và hàm $f : C \to \mathbb{R}$. Hàm f **lồi trên** C nếu với mọi $x, y \in C$ và mọi $t \in [0, 1]$,

$$f\big((1 - t)x + ty\big) \leq (1 - t)f(x) + tf(y).$$

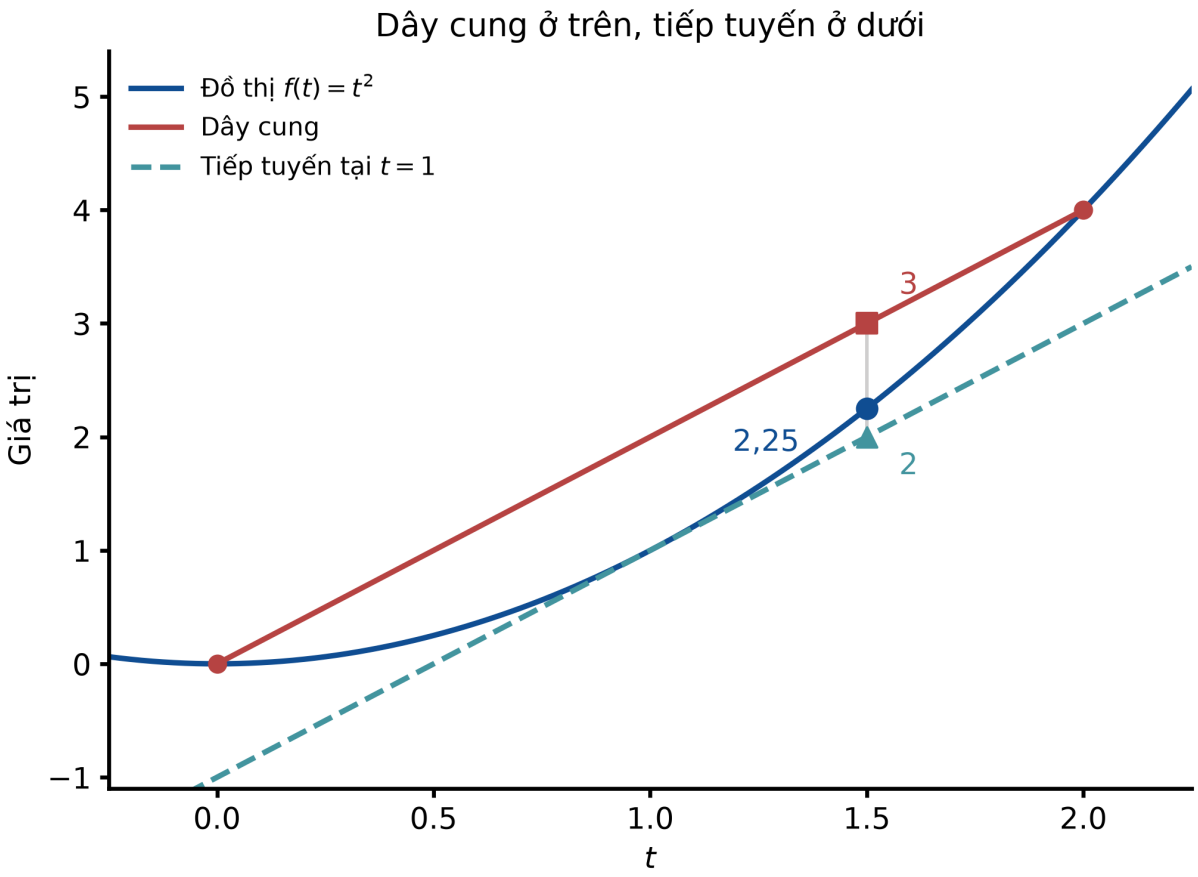
Vế trái là giá trị hàm tại điểm ở giữa. Vế phải là giá trị của dây cung, tức trung bình có cùng trọng số của hai giá trị ở đầu. Dấu bằng được phép xảy ra; do đó hàm hằng và hàm affine đều lồi. **Hàm lõm** được định nghĩa bằng bất đẳng thức ngược lại, hay tương đương $-f$ là hàm lồi. Định nghĩa nhiều chiều hoàn toàn giống vậy: chỉ cần nối hai vector đầu vào, không cần vẽ toàn bộ đồ thị.

Tính lồi phụ thuộc miền đang xét. Chẳng hạn $f(t) = t^2$ lồi trên toàn bộ đường thẳng, nên cũng lồi trên mọi khoảng con. Một biểu thức có thể lồi trên một khoảng nhưng không lồi trên khoảng lớn hơn; luôn nêu miền trước khi kết luận.

6. Ví dụ tay: kiểm dây cung và tiếp tuyến

Xét $f(t) = t^2$ với hai đầu mút $a = 0$, $b = 2$, và trọng số $\theta = \frac{3}{4}$ đặt lên b. Các phép tính trong mục này (tự tính).

Bước 1: điểm đầu vào ở giữa. Tính cùng trọng số cho hai đầu mút:



Hình 49: Dây cung và tiếp tuyến của hàm bình phương (minh hoạ, tự tính)

$$z = \left(1 - \frac{3}{4}\right) \cdot 0 + \frac{3}{4} \cdot 2 = \frac{3}{2} = 1{,}5.$$

Bước 2: tính giá trị của hàm tại điểm đó. Ta được $f(z) = (1{,}5)^2 = 2{,}25$.

Bước 3: tính giá trị dây cung. Hai giá trị đầu mút là $f(0) = 0$ và $f(2) = 4$. Pha chúng bằng đúng các trọng số ở bước 1:

$$\frac{1}{4} f(0) + \frac{3}{4} f(2) = \frac{1}{4} \cdot 0 + \frac{3}{4} \cdot 4 = 3.$$

Kết quả là $2{,}25 \le 3$; khoảng cách theo phương thẳng đứng là $0{,}75$. Đây là một lần kiểm đúng bất đẳng thức, chưa đủ chứng minh tính lồi cho mọi cặp. Nhưng có thể kiểm trực tiếp tổng quát: với mọi $a, b \in \mathbb{R}$ và $0 \le \theta \le 1$,

$$(1-\theta)a^2 + \theta b^2 - \big((1-\theta)a + \theta b\big)^2 = \theta(1-\theta)(a-b)^2 \ge 0.$$

Vì chênh lệch luôn không âm, t^2 thật sự lồi trên toàn đường thẳng. Chênh lệch dương khi hai đầu mút khác nhau và trọng số nằm hẳn giữa 0 và 1; ta sẽ gọi tính chất mạnh hơn này là lồi chặt.

Bước 4: so với tiếp tuyến. Tại $t = 1$, $f(1) = 1$ và $f'(1) = 2$. Công thức đường tiếp tuyến là $\ell(t) = 1 + 2(t-1) = 2t - 1$. Tại $t = 1{,}5$, đường này cho $\ell(1{,}5) = 2$, nhỏ hơn $f(1{,}5) = 2{,}25$ đúng $0{,}25$. Khác với dây cung, tiếp tuyến được dựng từ giá trị và độ dốc tại **một** điểm.

7. Gradient, tiếp tuyến và điểm tối ưu

Với f khả vi trên một tập lồi, điều kiện bậc nhất của tính lồi là

$$f(y) \ge f(x) + \nabla f(x)^{\top}(y - x) \qquad \text{với mọi } x, y \in C.$$

Vế phải là xấp xỉ tuyến tính tại x, tức đường tiếp tuyến trong một chiều hoặc mặt phẳng tiếp xúc trong nhiều chiều. Điều kiện nói rằng xấp xỉ đó là một **cận dưới toàn cục** trên miền, không chỉ là xấp xỉ gần x. Trong trường hợp miền có biên, hiểu gradient là gradient của hàm khả vi trên một lân cận của miền. Đối với hàm khả vi và miền lồi phù hợp, bất đẳng thức này tương đương với định nghĩa bằng dây cung.

Nếu tối thiểu hoá không ràng buộc trên toàn $\mathbb{R}^n$ và tìm được $x^\star$ có $\nabla f(x^\star) = 0$, thay $x = x^\star$ vào cận dưới cho mọi y:

$$f(y) \ge f(x^\star) + 0 = f(x^\star).$$

Vậy $x^\star$ là nghiệm toàn cục. Chiều ngược lại cũng đúng với một nghiệm nằm ở miền mở và hàm khả vi: gradient tại cực tiểu phải bằng 0. Nhưng điều kiện gradient bằng 0 một mình không đủ với hàm không lồi: điểm yên ở hình 3 là phản ví dụ.

Nếu có ràng buộc $x \in C$, nghiệm có thể nằm trên biên và gradient không bằng 0. Với f lồi khả vi và C lồi, $x^\star \in C$ là nghiệm toàn cục khi và chỉ khi

$$\nabla f(x^\star)^{\top}(y - x^\star) \ge 0 \qquad \text{với mọi } y \in C.$$

Điều này nói rằng từ nghiệm, mọi hướng đi tới một điểm khả thi đều có đạo hàm theo hướng không âm. Hướng ngược gradient có thể làm giảm hàm nhưng lại đi ra ngoài miền. Mục 10 sẽ tính từng bước một ví dụ mà gradient ở nghiệm khác 0.

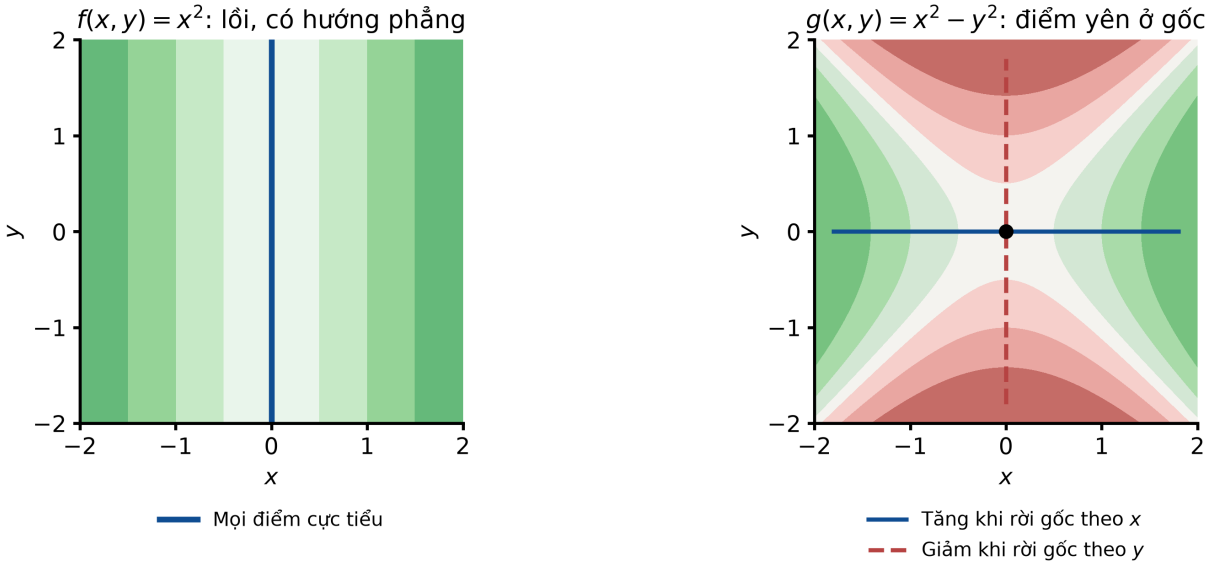
Vì sao cực tiểu địa phương cũng là toàn cục? Giả sử x là cực tiểu địa phương nhưng có $y \in C$ tốt hơn, tức $f(y) < f(x)$. Tính lồi cho thấy tại điểm $(1-t)x+ty$ với $t > 0$ bất kỳ, giá trị không vượt $(1-t)f(x)+tf(y)$, vốn nhỏ hơn $f(x)$. Lấy t rất nhỏ, điểm này vừa ở gần x vừa khả thi, mâu thuẫn với cực tiểu địa phương. Lập luận cần cả hàm lồi lẫn miền lồi.

8. Hessian: nhìn độ cong theo mọi hướng

Nếu f có đạo hàm cấp hai liên tục trên một miền mở lồi, thì f lồi khi và chỉ khi Hessian của nó nửa xác định dương tại mọi điểm trong miền:

$$v^\top \nabla^2 f(x) v \geq 0 \quad \text{với mọi } x \text{ trong miền và mọi } v.$$

Trong một chiều, điều này trở thành $f''(t) \geq 0$ trên cả khoảng mở. Trong nhiều chiều, nhìn riêng các phần tử đường chéo là chưa đủ, vì các thành phần ngoài đường chéo cũng ảnh hưởng độ cong khi nhiều tọa độ cùng thay đổi. Với Hessian thực đối xứng, điều kiện tương đương là mọi trị riêng đều không âm.



Hình 50: Hàm lồi có hướng phẳng và hàm có điểm yên (minh hoạ, tự tính)

Cách đọc hình 3: mỗi panel nhìn từ trên xuống, hai trục là các tọa độ đầu vào. Bên trái, màu không thay đổi khi đi dọc đường xanh $x = 0$: mọi điểm trên đường đó đều có cùng giá trị nhỏ nhất. Bên phải, đi ngang từ gốc làm g tăng còn đi dọc làm g giảm; gốc là điểm yên, không phải cực tiểu.

Ta tính hai Hessian trên hình (tự tính):

$$f(x,y) = x^2, \qquad \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \qquad g(x,y) = x^2 - y^2, \qquad \nabla^2 g = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}.$$

Hessian của f có trị riêng 2 và 0, cả hai không âm, nên f lồi dù phẳng theo y. Hàm có vô số nghiệm $(0, y)$. Hessian của g có trị riêng 2 và -2: một hướng cong lên, một hướng cong xuống, nên g không lồi. Cụ thể, $g(1,0) = 1$ nhưng $g(0,1) = -1$ trong khi $g(0,0) = 0$.

Kiểm Hessian ở một điểm chỉ cho thông tin tại điểm đó. Để kết luận tính lồi trên cả miền bằng tiêu chuẩn này, phải chứng minh điều kiện ở **mọi** điểm. Ngược lại, một điểm bên trong miền tại đó Hessian có hướng cong âm đã đủ bác bỏ tính lồi của hàm khả vi hai lần.

9. Lồi, lồi chặt, lồi mạnh và sự tồn tại nghiệm

9.1 Lồi chặt

Hàm **lồi chặt** nếu bất đẳng thức dây cung trở thành dấu nhỏ hơn nghiêm ngặt khi $x \neq y$ và $0 < t < 1$. Đồ thị khi đó nằm hẳn dưới dây cung tại mọi điểm bên trong đoạn. Ví dụ t^2 lồi chặt trên $\mathbb{R}$ theo phép khai triển ở mục 6; $f(x, y) = x^2$ trên $\mathbb{R}^2$ không lồi chặt vì nó hằng trên mọi đoạn thẳng đứng.

Nếu một hàm lồi chặt có nghiệm cực tiểu trên miền lồi thì nghiệm đó **duy nhất**. Thật vậy, giả sử có hai nghiệm khác nhau cùng đạt giá trị nhỏ nhất. Điểm giữa chúng khả thi và có giá trị thấp hơn cả hai theo tính lồi chặt, trái với giả thiết. Cụm ”nếu có nghiệm” rất quan trọng: tính duy nhất không tự tạo ra nghiệm.

9.2 Lồi mạnh

Với $\mu > 0$, hàm f được gọi là **lồi mạnh với hệ số** μ trên miền lồi nếu

$$f\big((1-t)x + ty\big) \leq (1-t)f(x) + tf(y) - \frac{\mu}{2}t(1-t)\|x - y\|_2^2.$$

Số hạng trừ yêu cầu một khoảng cách dương dưới dây cung khi hai điểm khác nhau và trọng số nằm bên trong đoạn. Vì vậy lồi mạnh kéo theo lồi chặt, rồi lồi chặt kéo theo lồi. Nếu hàm khả vi hai lần trên miền mở lồi, điều kiện tương đương là $\nabla^2 f(x) \succeq \mu I$ tại mọi điểm, nghĩa là $v^\top \nabla^2 f(x) v \geq \mu \|v\|_2^2$ cho mọi v.

Điều ngược lại không đúng (tự tính): hàm t^4 lồi chặt trên $\mathbb{R}$ nhưng không lồi mạnh trên toàn đường thẳng, vì đạo hàm cấp hai $12t^2$ bằng 0 tại gốc. Hằng số μ còn phụ thuộc cách chuẩn hoá biểu thức và miền xét, nên phải đọc định nghĩa của từng bài.

9.3 Lồi mạnh và độ trơn là hai điều khác nhau

Lồi mạnh đặt **cận dưới** cho độ cong, còn độ trơn thường đặt **cận trên** cho tốc độ thay đổi của gradient. Nếu hàm khả vi hai lần và $\mu I \preceq \nabla^2 f(x) \preceq L I$ trên miền đang xét, trị riêng Hessian nằm giữa μ và L. Với $\mu > 0$, tỉ số L/μ mô tả chênh lệch giữa các hướng cong mạnh và cong yếu trong mô hình bậc hai; tỉ số lớn thường làm gradient descent nhạy với chọn bước. Cần đọc đủ giả thiết của từng định lý trước khi suy ra tốc độ hội tụ.

9.4 Có một giá trị thấp nhất hay chỉ tiến gần?

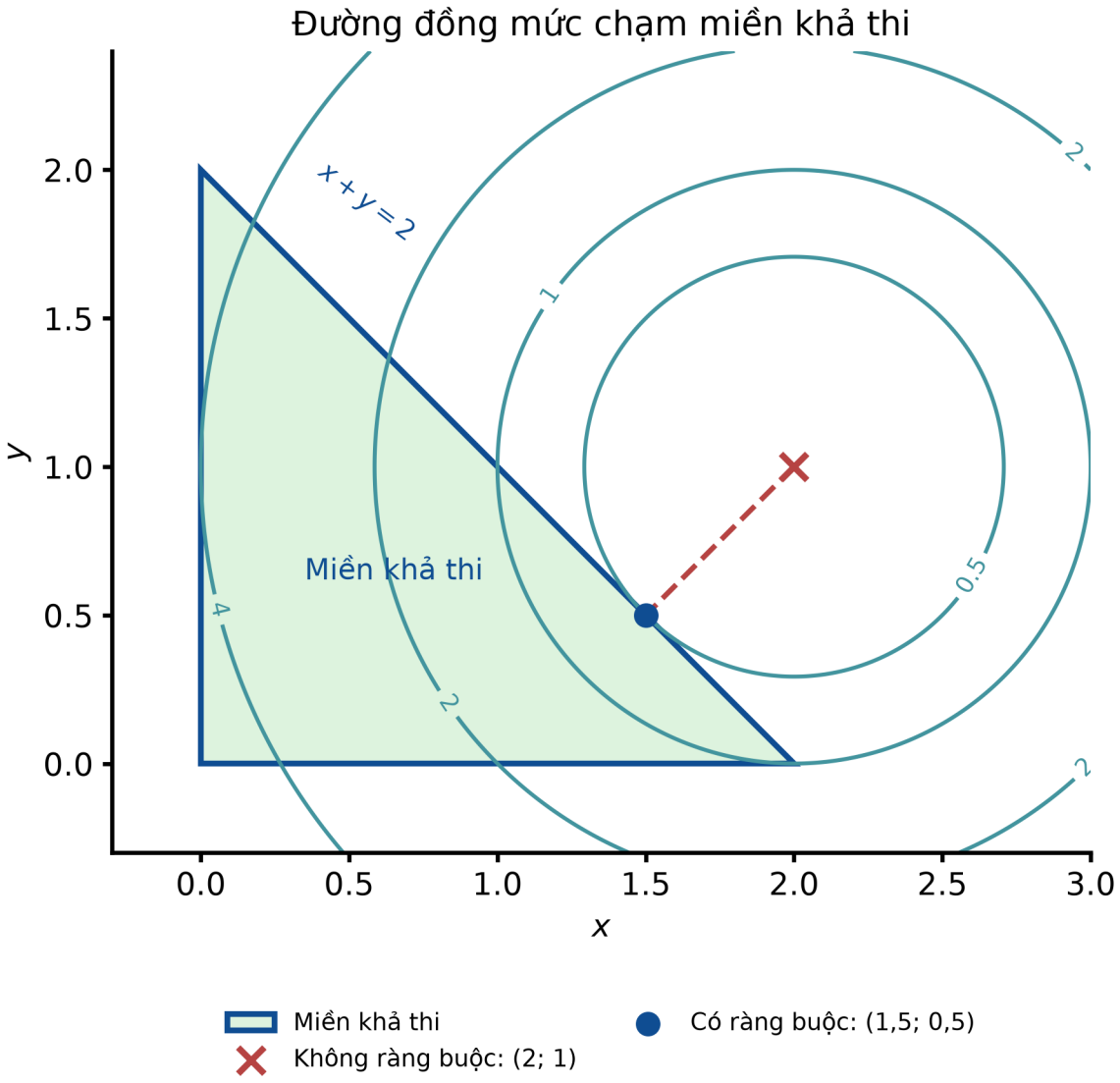
Tập nghiệm có thể rỗng dù bài toán lồi. Ví dụ (tự tính), $f(t) = t$ trên miền mở $(0, 1)$ lồi, và giá trị hàm tiến tới 0 khi t tiến tới 0 từ bên phải. Nhưng 0 không thuộc miền nên không có điểm nào đạt giá trị đó. Số 0 là **cận dưới đúng** (infimum), không phải giá trị cực tiểu đạt được. Nếu hàm liên tục và miền khác rỗng, đóng, bị chặn trong không gian hữu hạn chiều, một nghiệm cực tiểu tồn tại; điều này là một điều kiện đủ hữu ích, không phải điều kiện cần duy nhất.

10. Ví dụ tay: tối ưu trên tam giác

Ta muốn chọn một điểm khả thi gần điểm $(2, 1)$ nhất theo khoảng cách Euclid bình phương:

$$\min_{(x,y)\in C} f(x, y) = (x-2)^2 + (y-1)^2, \qquad C = \{x \geq 0,\ y \geq 0,\ x + y \leq 2\}.$$

Hãy đọc hình trước, sau đó mới tính. Các phép tính và các giá trị trong mục này (tự tính).



Hình 51: Đường đồng mức của khoảng cách bình phương chạm tam giác khả thi (minh hoạ, tự tính)

Cách đọc hình 4: miền xanh nhạt là tam giác khả thi, dấu nhân đỏ là tâm các đường đồng mức và là nghiệm không ràng buộc. Khi thu nhỏ các đường đồng mức quanh dấu nhân, đường mức nhỏ nhất còn chạm miền là đường ghi 0,5, tại chấm xanh. Đường đứt nối hai nghiệm vuông góc cạnh xiên; các đường mức được ghi bằng **giá trị bình phương khoảng cách**, không phải bán kính.

10.1 Bước 1: kiểm bài toán lồi

Miền C lồi như đã chứng minh ở mục 4. Đạo hàm và Hessian của mục tiêu là

$$\nabla f(x,y) = \begin{pmatrix} 2(x-2) \\ 2(y-1) \end{pmatrix}, \qquad \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = 2I.$$

Với mọi vector khác 0, $v^\top (2I) v = 2\|v\|_2^2 > 0$. Mục tiêu lồi mạnh với $\mu = 2$, do đó nếu có nghiệm thì nghiệm duy nhất. Miền là tam giác đóng, bị chặn và khác rỗng; hàm liên tục, nên nghiệm tồn tại.

10.2 Bước 2: nhìn nghiệm tự do và tìm một ứng viên trên biên

Không xét ràng buộc, hai bình phương đồng thời bằng 0 tại $(2,1)$, cho giá trị 0. Nhưng $2+1=3>2$, nên điểm đó nằm ngoài miền. Theo hình, ta thử cạnh xiên $x+y=2$. Đặt $y = 2-x$, với $0 \le x \le 2$, rồi thay vào mục tiêu:

$$\begin{aligned} f(x, 2-x) &= (x-2)^2 + (1-x)^2 \\ &= 2x^2 - 6x + 5. \end{aligned}$$

Đạo hàm theo x là $4x-6$. Cho đạo hàm bằng 0 được $x = \frac{3}{2}$, suy ra $y = \frac{1}{2}$. Điểm này thoả cả ba ràng buộc và trên cạnh đang xét. Giá trị tại đó là

$$f\left(\frac{3}{2}, \frac{1}{2}\right) = \left(-\frac{1}{2}\right)^2 + \left(-\frac{1}{2}\right)^2 = \frac{1}{2}.$$

Giải trên cạnh chỉ mới tìm được ứng viên tốt nhất của cạnh đó. Ta chưa được mặc nhiên bỏ qua phần trong và các cạnh khác. Bước tiếp theo chứng minh ứng viên là tốt nhất trên **toàn miền**.

10.3 Bước 3: chứng nhận toàn cục

Gọi $z^\star = (\frac{3}{2}, \frac{1}{2})$. Gradient tại đó là $\nabla f(z^\star) = (-1,-1)^\top$, khác vector 0. Với điểm khả thi bất kỳ $w = (u,v)$, ta có $u+v \le 2$, nên

$$\nabla f(z^\star)^\top (w - z^\star) = -(u - \tfrac{3}{2}) - (v - \tfrac{1}{2}) = 2 - u - v \ge 0.$$

Đó chính là điều kiện tối ưu ở mục 7, vậy $z^\star$ là nghiệm toàn cục. Tính lồi mạnh bảo đảm không có nghiệm thứ hai. Ví dụ kiểm riêng điểm $(1, \frac{1}{2})$: tích vô hướng trên bằng $\frac{1}{2} > 0$; đây chỉ là một phép kiểm minh hoạ, còn chứng minh dựa trên w khả thi bất kỳ.

Có thể đọc hình theo phép chiếu: $z^\star$ là điểm gần $(2,1)$ nhất trong C. Đoạn đứt từ $z^\star$ tới tâm nằm theo hướng $(1,1)$, vuông góc cạnh $x+y=2$. Không phải phép chiếu nào cũng chạm cạnh xiên hay nằm ngoài miền: điều đó tùy điểm và tập đang xét.

10.4 Mở rộng: điều kiện KKT

Điều kiện Karush-Kuhn-Tucker (KKT) cho bất đẳng thức $h_i(z) \le 0$ gồm: điểm khả thi, nhân tử $\lambda_i \ge 0$, gradient Lagrange theo z bằng 0 và $\lambda_i h_i(z) = 0$ (bù trừ).

Với ràng buộc cạnh xiên $h(x, y) = x + y - 2 \le 0$, hàm Lagrange dùng nhân tử $\lambda \ge 0$ là

$$\mathcal{L}(x, y, \lambda) = f(x, y) + \lambda(x + y - 2).$$

Tại nghiệm, cạnh xiên đang hoạt động vì $h(z^\star) = 0$. Chọn $\lambda = 1$ thì $\nabla f(z^\star) + \lambda(1, 1)^\top = (0, 0)^\top$. Hai ràng buộc $x \ge 0$, $y \ge 0$ không chạm biên tại nghiệm nên chọn nhân tử tương ứng bằng 0. Các điều kiện khả thi, nhân tử không âm, dừng và bù trừ đều được thoả.

Trong bài toán lồi khả vi, KKT là chứng nhận đủ cho tối ưu toàn cục. Chiều ngược cần điều kiện bổ sung: ở đây Slater được thoả vì $(\frac{1}{2}, \frac{1}{2})$ thoả nghiêm mọi bất đẳng thức (tự tính), nên nghiệm có nhân tử KKT.

11. Những phép toán thường giữ tính lồi

Để phân tích một hàm dài, ta có thể ghép những mảnh đã biết. Các quy tắc dưới đây luôn đi kèm miền được xác định và là miền lồi thích hợp.

- **Tổng không âm:** nếu mỗi f_i lồi và $\alpha_i \ge 0$, thì $\sum_i \alpha_i f_i$ lồi. Trọng số âm có thể phá tính lồi. Trung bình của các loss lồi theo cùng tham số vì vậy cũng lồi.
- **Đổi đầu vào affine:** nếu f lồi, thì $g(x) = f(Ax + b)$ lồi trên tập các x mà $Ax + b$ thuộc miền của f. Lý do là phép affine giữ các tổ hợp lồi. Phép đổi biến phi tuyến không tự động giữ tính chất này.
- **Lấy giá trị lớn nhất:** nếu các f_i lồi, thì $g(x) = \max_i f_i(x)$ lồi trên miền lồi chung. Hàm này có thể không khả vi tại nơi nhiều nhánh bằng nhau, nhưng định nghĩa dây cung vẫn dùng được.
- **Tập dưới mức:** với f lồi trên C, tập $\{x \in C : f(x) \le c\}$ lồi. Với hai điểm trong tập, bất đẳng thức dây cung cho giá trị tại mọi điểm giữa chúng không vượt c. Chiều đảo không đúng nói chung: mọi tập dưới mức lồi mới cho tính **tựa lồi**, yếu hơn lồi.

Các hàm chuẩn như $\|x\|_1$ và $\|x\|_2$ lồi nhờ bất đẳng thức tam giác và tính thuần nhất không âm. Trị tuyệt đối là trường hợp một chiều, $|t| = \max(t, -t)$: nó lồi nhưng không có đạo hàm thông thường ở 0. Nếu thiếu đạo hàm, vẫn có thể dùng định nghĩa dây cung và khái niệm **dưới gradient**; không nên áp máy móc công thức Hessian.

Một vector s là dưới gradient của hàm lồi f tại x nếu $f(y) \ge f(x) + s^\top(y - x)$ với mọi y trong miền. Với $f(t) = |t|$ tại 0, mọi $s \in [-1, 1]$ đều hợp lệ (tự tính). Khi f khả vi, gradient là dưới gradient tương ứng. Đây là lý do phương pháp dưới gradient có thể xử lý một số loss lồi không trơn.

12. Jensen: lấy trung bình trước hay sau hàm?

Với hàm lồi f, các điểm x_i trong miền và trọng số $p_i \ge 0$ có tổng bằng 1, bất đẳng thức Jensen hữu hạn nói rằng

$$f\left(\sum_i p_i x_i\right) \le \sum_i p_i f(x_i).$$

Bên trái: trung bình đầu vào trước, rồi áp hàm. Bên phải: áp hàm cho từng đầu vào, rồi lấy trung bình. Trường hợp hai điểm chính là định nghĩa dây cung; trường hợp nhiều điểm có thể chứng minh bằng cách ghép dần từng điểm. Đối với biến ngẫu nhiên tổng quát, viết $f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$ khi các kỳ vọng liên quan tồn tại và phép áp dụng có nghĩa.

Ví dụ (tự tính), cho X nhận 0 với xác suất $\frac{1}{4}$ và nhận 2 với xác suất $\frac{3}{4}$, chọn $f(t) = t^2$. Khi đó

$$\mathbb{E}[X] = \frac{3}{2}, \qquad f(\mathbb{E}[X]) = \frac{9}{4} = 2{,}25,$$

trong khi

$$\mathbb{E}[f(X)] = \frac{1}{4} \cdot 0^2 + \frac{3}{4} \cdot 2^2 = 3.$$

Vậy $2{,}25 \leq 3$, đúng như hình 2 và mục 6. Chênh lệch là $\frac{3}{4}$, cũng chính là phương sai của biến X trong ví dụ này. Sự trùng nhau với phương sai là đặc điểm của hàm bình phương, không phải công thức chênh lệch chung cho mọi hàm lồi.

Với hàm **lõm** như log trên $(0, \infty)$, dấu đảo chiều:

$$\mathbb{E}[\log X] \leq \log \mathbb{E}[X],$$

với $X > 0$ gần như chắc chắn và các kỳ vọng hữu hạn thích hợp. Đây là hướng thường gặp khi xây dựng một cận dưới cho log của một kỳ vọng trong suy luận biến phân. Trước khi dùng, hãy kiểm hàm nào lồi hay lõm, xác suất nào đang lấy trung bình và các biểu thức có xác định không. Một bất đẳng thức chỉ cho hướng của cận; nó không tự nói khoảng cách giữa hai vế nhỏ.

13. Nối sang học máy

Hồi quy tuyến tính. Với dữ liệu cố định X và y, loss bình phương $\|Xw - y\|_2^2$ là hàm lồi theo w: đó là bình phương chuẩn sau một phép affine. Hessian bằng $2X^\top X$, luôn nửa xác định dương vì $v^\top X^\top X v = \|Xv\|_2^2 \geq 0$. Nếu các cột của X độc lập tuyến tính thì Hessian xác định dương và loss lồi mạnh theo w; nếu không, có thể có nhiều nghiệm. Kết luận này nói về hàm theo tham số khi dữ liệu cố định.

Hồi quy logistic. Với đặc trưng cố định và nhãn cố định, negative log-likelihood logistic là tổng các loss lồi theo trọng số tuyến tính. Thêm một hình phạt $\lambda\|w\|_2^2$ với $\lambda > 0$ cho mọi tọa độ đang xét làm tăng độ cong theo các tọa độ ấy. Nếu một hệ số chặn không bị phạt, không thể chỉ dựa vào hình phạt trên w để khẳng định lồi mạnh theo toàn bộ tham số gồm cả hệ số chặn.

Tối ưu mạng nơ-ron. Một loss lồi theo đầu ra mô hình không tự kéo theo loss lồi theo toàn bộ trọng số của mạng phi tuyến. Việc ghép với mạng là một phép phụ thuộc phi tuyến theo tham số, không thuộc quy tắc affine ở mục 11. Vì thế nhận xét về cực tiểu toàn cục của bài lồi không thể tự động chuyển sang mạng sâu. Bài math 24 dùng thêm giả thiết cụ thể về hàm loss và mô hình khi nghiên cứu lazy training.

Hình học thông tin. Trong math 07, hàm sinh lồi khả vi tạo Bregman divergence bằng chênh lệch giữa giá trị hàm và mặt phẳng tiếp xúc. Bất đẳng thức tiếp tuyến ở mục 7 cho thấy đại lượng này không âm. Tính lồi chặt giúp phân biệt hai điểm khác nhau, nhưng Bregman divergence nói chung không đối xứng nên không nên tự gọi nó là khoảng cách metric.

Suy luận biến phân. Bài math 08 thảo luận tối ưu ELBO và giới hạn cực tiểu hay cực đại địa phương; bài math 09 trình bày một cách suy ra ELBO riêng. Jensen giúp hiểu vì sao những biểu thức log và kỳ vọng tạo ra một cận, nhưng không bảo đảm mục tiêu ELBO lồi hay lõm theo mọi tham số của mô hình. Cần tách hướng của bất đẳng thức khỏi tính chất tối ưu của bài toán cụ thể.

Miền xác suất. Một vector xác suất có các phần tử không âm và tổng bằng 1. Miền này lồi: pha hai phân phối với trọng số không âm có tổng 1 vẫn là phân phối. Điều đó không tự bảo đảm rằng một hàm bất kỳ trên các phân phối là lồi; cần kiểm chính hàm và cả cách tham số hoá.

14. Những sai lầm thường gặp

- **Chỉ nhìn một đoạn rồi kết luận hàm lồi.** Một đoạn thỏa bất đẳng thức chỉ là ví dụ. Cần chứng minh cho mọi đoạn, hoặc dùng một tiêu chuẩn tương đương có đủ giả thiết.

- **Nhầm tập lồi với hàm lồi.** Tập nói về đoạn giữa các đầu vào khả thi; hàm nói thêm về giá trị ở các điểm đó. Cần cả hai khi dùng kết luận về cực tiểu địa phương.
- **Đảo chiều dây cung và tiếp tuyến.** Với hàm lồi, dây cung ở trên đồ thị giữa hai đầu mút, còn tiếp tuyến của hàm khả vi ở dưới đồ thị trên miền.
- **Đòi Hessian phải dương xác định.** Nửa xác định dương đã đủ cho lồi khi tiêu chuẩn Hessian áp dụng. Trị riêng 0 có thể tạo một hướng phẳng; không được kết luận nghiệm duy nhất chỉ từ tính lồi.
- **Cho rằng gradient tại mọi nghiệm có ràng buộc đều bằng 0.** Tại biên, gradient có thể khác 0. Phải kiểm các hướng khả thi, hoặc dùng điều kiện KKT với đúng các ràng buộc.
- **Coi lồi chặt là bảo đảm nghiệm tồn tại, thuật toán nhanh và dự đoán tốt.** Lồi chặt chỉ bảo đảm nhiều nhất một nghiệm cực tiểu trên miền lồi. Sự tồn tại, tốc độ và chất lượng tổng quát hoá cần các giả thiết riêng.

15. Bảng công thức cần nhớ

Khái niệm	Điều kiện hoặc công thức	Cách đọc
Tập lồi C	$(1-t)x+ty \in C$ với $x,y \in C$, $t \in [0,1]$	Cả đoạn nối ở trong miền
Hàm lồi f	$f((1-t)x+ty) \le (1-t)f(x)+tf(y)$	Đồ thị không cao hơn dây cung
Bậc nhất	$f(y) \ge f(x)+\nabla f(x)^\top (y-x)$	Tiếp tuyến là cận dưới
Bậc hai	$\nabla^2 f(x) \succeq 0$ trên miền mở lồi	Mọi hướng có độ cong không âm
Tối ưu có ràng buộc	$\nabla f(x^\star)^\top (y-x^\star) \ge 0$ với mọi $y \in C$	Không có hướng khả thi giảm bậc nhất
Jensen	$f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$	Trung bình trước hàm không lớn hơn trung bình sau hàm
Lồi mạnh	$\nabla^2 f(x) \succeq \mu I$, $\mu > 0$, khi khả vi hai lần trên miền mở lồi	Độ cong có cận dưới dương

Các điều kiện gradient trong bảng dùng cho hàm lồi khả vi; với miền có biên, gradient được hiểu như ở mục 7. Các công thức kỳ vọng đòi hỏi các đại lượng liên quan xác định. Với hàm không khả vi, trở về định nghĩa dây cung hoặc dùng dưới gradient.

16. Bài tập tự tính, có lời giải

Hãy làm trước khi xem đáp án. Mọi phép tính số ở phần lời giải (tự tính) đã được kiểm bằng numpy.

Bài 1. Một điểm trên đoạn

Cho $x=(0,0)$, $y=(2,4)$, $t=\frac{1}{4}$. Tính $z=(1-t)x+ty$. Nếu C là tập lồi chứa x, y, ta kết luận gì về z?

Đáp án. Ta có $z=\frac{3}{4}(0,0)+\frac{1}{4}(2,4)=(\frac{1}{2},1)$. Vì trọng số không âm và cộng bằng 1, z thuộc đoạn nối x, y nên $z \in C$. Không cần biết hình dạng chính xác của C.

Bài 2. Kiểm một dây cung

Cho $f(t)=t^2$, hai đầu mút 0 và 2. Tính giá trị hàm tại trung điểm và trung bình hai giá trị đầu mút. Từ một phép kiểm này, đã chứng minh được hàm lồi chưa?

Đáp án. Trung điểm là 1, nên $f(1)=1$. Hai giá trị đầu mút là $f(0)=0$, $f(2)=4$; trung bình là 2. Ta có $1 \le 2$, phù hợp định nghĩa. Một cặp điểm chưa đủ chứng minh cho mọi cặp; phép khai triển tổng quát tại mục 6 mới làm được điều đó.

Bài 3. Xét Hessian một hàm hai biến

Cho $f(x,y) = x^2 + xy + y^2$. Tìm Hessian, trị riêng của Hessian và kết luận tính lồi. Hàm có nghiệm cực tiểu duy nhất ở đâu?

Đáp án. Gradient là $(2x + y,\ x + 2y)^\top$ và Hessian là

$$H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Ta có $\det(H - \lambda I) = (2-\lambda)^2 - 1$, cho hai trị riêng 1 và 3. Chúng đều dương, nên $H \succeq I$ và hàm lồi mạnh trên $\mathbb{R}^2$. Gradient bằng 0 tại $(0,0)$, do đó đó là nghiệm toàn cục duy nhất, với giá trị 0. Có thể kiểm hướng $(1,-1)$: $v^\top H v = 2 > 0$.

Bài 4. Jensen với ba giá trị

Biến ngẫu nhiên X nhận các giá trị 0, 2, 4 với xác suất lần lượt là $\frac{1}{4}$, $\frac{1}{2}$, $\frac{1}{4}$. Tính $\mathbb{E}[X]$, $f(\mathbb{E}[X])$ và $\mathbb{E}[f(X)]$ với $f(t) = t^2$.

Đáp án. Ta được $\mathbb{E}[X] = 0\cdot\frac{1}{4} + 2\cdot\frac{1}{2} + 4\cdot\frac{1}{4} = 2$, nên $f(\mathbb{E}[X]) = 4$. Mặt khác, $\mathbb{E}[f(X)] = 0^2\cdot\frac{1}{4} + 2^2\cdot\frac{1}{2} + 4^2\cdot\frac{1}{4} = 6$. Jensen cho $4 \le 6$, và chênh lệch 2 là phương sai trong bài này.

Bài 5. Nghiệm nằm ở đỉnh tam giác

Dùng miền $C = \{(x,y) : x \ge 0,\ y \ge 0,\ x + y \le 2\}$. Tìm điểm trong C gần $(3,1)$ nhất theo khoảng cách bình phương. Chứng minh nghiệm bằng điều kiện bậc nhất, không chỉ đoán từ hình.

Đáp án. Thử $z^\star = (2,0)$, là một đỉnh khả thi. Hàm $f(x,y) = (x-3)^2 + (y-1)^2$ có gradient $\nabla f(z^\star) = (-2,-2)^\top$ và Hessian $2I$, nên lồi mạnh. Với mọi $w = (u,v) \in C$,

$$\nabla f(z^\star)^\top (w - z^\star) = -2(u-2) - 2v = 2(2 - u - v) \ge 0.$$

Điều kiện bậc nhất chứng nhận nghiệm toàn cục; lồi mạnh bảo đảm nghiệm duy nhất. Giá trị nhỏ nhất là $f(2,0) = (-1)^2 + (-1)^2 = 2$. Ở đây hai ràng buộc $x + y \le 2$ và $y \ge 0$ đều chạm biên; một ràng buộc chạm biên vẫn có thể có nhân tử KKT bằng 0.

Tài liệu tham khảo nền: Boyd và Vandenberghe, Convex Optimization; Deisenroth, Faisal và Ong, Mathematics for Machine Learning; các ghi chú math 07, math 08, math 09 và math 24 trong cuốn này.

Bài 00

Preface: Toán nền tảng và hướng dẫn đọc về Toán cho AI

Tác giả: ghi chú tổng hợp từ kiến thức nền của người biên soạn (không phải ghi chú một paper cụ thể) **Venue / arXiv:** không áp dụng **Nguồn:** các bài 01-35 trong thư mục này; phần kiến thức nền chung (định nghĩa, định lý kinh điển trong đại số tuyến tính, giải tích, xác suất, lý thuyết thông tin, số học dấu phẩy động) được trình bày lại ở đây, không trích từ một paper cụ thể **Chủ đề:** toán nền cho AI từ con số 0: đại số tuyến tính (vector, ma trận, hạng, trị riêng, SVD, chuẩn, điều kiện số), giải tích nhiều biến (gradient, Jacobian, Hessian, chain rule, autodiff, backprop tay), xác suất (Gaussian đa biến, Bayes, MLE/MAP, bất đẳng thức tập trung, hình học chiều cao), lý thuyết thông tin (entropy, cross-entropy, KL, MI), số học máy tính (dấu phẩy động, log-sum-exp, ổn định số), và lý thuyết học sâu ở mức khái niệm (NTK, overparameterization) đủ để đọc các bài 19-34 của thư mục **Vị trí trong lộ trình:** Bài số 00, đọc trước tất cả. Thứ tự đề xuất của cả dự án: `math content/` (thư mục này) rồi `ml basics content/` rồi `optimization content/`, sau đó mới tới các thư mục chuyên đề (`llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`).

> Đây là phần kiến thức nền chung do chúng ta tự soạn lại, không phải ghi chú diễn giải của một paper cụ thể và không phải bản dịch của bất kỳ nguồn nào. Khi cần một cuốn sách giáo khoa để tham khảo thêm, chúng tôi gọi tên: Boyd và Vandenberghe, ”Convex Optimization”; Deisenroth, Faisal và Ong, ”Mathematics for Machine Learning”; Goodfellow, Bengio và Courville, ”Deep Learning”; Murphy, ”Probabilistic Machine Learning”; Strang, ”Introduction to Linear Algebra”. Đây là các nguồn tham khảo được công nhận rộng rãi, nhưng chúng tôi không trích số trang hay số chương cụ thể vì không đối chiếu trực tiếp với một ấn bản nào. Mọi ví dụ số trong bài này đánh dấu ”tự tính” đã được kiểm bằng Python (numpy) trước khi ghi vào đây, có thể chạy lại để kiểm tra. Khi một bài trong `math content/` nói khác với phần tổng quan ở đây (ví dụ cách paper định nghĩa một ký hiệu), hãy tin bài đó, vì nó được đọc trực tiếp từ full text, hơn là tin phần nền tảng chung này.

1. Vì sao AI cần toán và bản đồ phân loại

1.1 Một câu tóm tắt

Một mạng neural là một hàm số f_θ có rất nhiều tham số θ (ma trận trọng số). Huấn luyện là tìm θ làm nhỏ một hàm mất mát $\mathcal{L}(\theta)$ trên dữ liệu. Toàn bộ 35 bài trong thư mục này chỉ xoay quanh bốn câu hỏi:

1. Dữ liệu và tham số được **biểu diễn** bằng gì, và biến đổi tuyến tính giữa chúng có cấu trúc gì? (đại số tuyến tính, mục 2)
2. Làm sao **tính được hướng giảm** của $\mathcal{L}(\theta)$ một cách tự động và đúng, kể cả khi θ có hàng tỷ chiều? (giải tích nhiều biến và autodiff, mục 3)
3. Dữ liệu và nhiễu được mô tả bằng luật nào, và ta tin vào một ước lượng đến mức nào? (xác suất, mục 4)
4. ”Khoảng cách” giữa hai phân phối (dự đoán và thực tế) được đo bằng gì, và tại sao loss của language model lại là một con số như vậy? (lý thuyết thông tin, mục 5)

Hai câu hỏi phụ nhưng rất thực dụng: mọi phép tính trên chạy bằng số có độ chính xác hữu hạn trên GPU, vậy khi nào nó tràn số hoặc mất chính xác (số học máy tính, mục 6); và vì sao một mạng quá nhiều tham số so với dữ liệu vẫn học được mà không chỉ ”học vẹt” (lý thuyết học sâu, mục 7).

1.2 Bản đồ: nhánh toán nào dùng ở đâu

Nhánh toán	Công cụ chính	Dùng để làm gì trong AI	Bài trong thư mục này
Đại số tuyến tính	vector, ma trận, hạng, trị riêng, SVD	biểu diễn dữ liệu/tham số; nén, giảm chiều, phân tích trọng số	01, 03, 04, 05, 17
Hình học thông tin	ma trận Fisher, natural gradient	tối ưu hoá ”đúng hình học” của không gian phân phối	06, 07
Giải tích nhiều biến	gradient, Jacobian, Hessian, autodiff	tính gradient để chạy gradient descent	01, 02
Suy luận biến phân và MCMC	ELBO, HMC, SGD như lấy mẫu Bayes	huấn luyện mô hình sinh, định lượng độ bất định	08, 09, 10, 11
Vận tải tối ưu	khoảng cách Wasserstein, Sinkhorn	đo khác biệt giữa hai phân phối khi KL không dùng được	12, 13
Lý thuyết thông tin	entropy, KL, information bottleneck	định nghĩa loss, nén biểu diễn, phân tích huấn luyện	14, 30
Số học máy tính	dấu phẩy động, log-sum-exp	giữ phép tính ổn định, chọn định dạng số khi huấn luyện	15, 16
Chuẩn hoá và Lipschitz	chuẩn ma trận, trị kỳ dị lớn nhất	ổn định huấn luyện GAN, kiểm soát độ nhạy	17, 18
Lý thuyết học sâu	NTK, mean-field, overparameterization	giải thích vì sao mạng rất lớn vẫn tối ưu và tổng quát hoá được	19-27, 31-34
Hình học hình học sâu, mô hình sinh, quyết định tuần tự	đối xứng/đa tạp, diffusion, bandit	các nhánh phụ, chi tiết ở mục 11	28, 09/13/29, 35

Thư mục `ml basics content/` và `optimization content/` dùng lại gần như toàn bộ các dòng trên, nhưng áp vào tình huống cụ thể hơn (một optimizer, một kiến trúc). Các thư mục chuyên đề (`llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`) dùng toán ở đây làm ”ngôn ngữ chung”: ví dụ KL divergence (mục 5) xuất hiện lại ở phạt KL trong RLHF, SVD và chuẩn Frobenius (mục 2) xuất hiện lại khi phân tích trọng số bị cắt tỉa.

1.3 Vì sao thứ tự math rồi mới ml basics rồi optimization

Thư mục này dạy **công cụ** (vector làm gì, gradient là gì, entropy đo gì) mà không gắn với một kiến trúc cụ thể. `ml basics content/` dùng công cụ đó để giải thích **hiện tượng** (double descent, generalization, adversarial example). `optimization content/` dùng công cụ đó để giải thích **thuật toán cập nhật tham số** (Adam, learning rate schedule). Đọc đúng thứ tự tránh việc phải dừng giữa một bài optimizer để tự hỏi ”Hessian là gì”.

---

2. Đại số tuyến tính

Đây là nhánh dùng nhiều nhất trong thư mục (01, 03, 04, 05, 17 trực tiếp; gần như mọi bài khác dùng ký hiệu ma trận). Mục này dựng lại từ số 0, có một **ví dụ xuyên suốt** để bạn thấy các khái niệm liên hệ với nhau.

2.1 Vector, chuẩn, tích vô hướng

Một vector $x \in \mathbb{R}^d$ là một cột d số. **Tích vô hướng** (dot product) $x^\top y = \sum_i x_i y_i$ đo ”độ giống hướng”. **Chuẩn Euclid** (độ dài) là $\|x\|_2 = \sqrt{x^\top x}$.

Ví dụ (tự tính): $x = (3, 4)$, $\|x\|_2 = \sqrt{9 + 16} = \sqrt{25} = 5$.

Góc giữa hai vector: $\cos\theta = \dfrac{x^\top y}{\|x\|_2\|y\|_2}$. Khi $\cos\theta = 0$, hai vector **vuông góc** (orthogonal), tức "không mang thông tin chung" theo nghĩa tuyến tính.

2.2 Ma trận là một phép biến đổi tuyến tính

$W \in \mathbb{R}^{m\times n}$ biến một vector n chiều thành một vector m chiều: $y = Wx$. Nhân hai ma trận W_2W_1 nghĩa là **ghép hai phép biến đổi**: áp W_1 trước, W_2 sau. Đây chính xác là cách một mạng nhiều lớp tuyến tính hoạt động (bỏ qua phi tuyến tạm thời): lớp 2 áp lên đầu ra của lớp 1.

2.3 Hạng (rank) và không gian con

Hạng của W là số chiều thật sự của ảnh (image) của phép biến đổi, tức số cột (hoặc hàng) độc lập tuyến tính. W cỡ $m \times n$ có hạng tối đa $\min(m, n)$.

Ví dụ rank-deficient (tự tính). Xét

$$C = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 1 \end{pmatrix}.$$

Hàng 2 bằng 2 lần hàng 1, nên hai hàng này phụ thuộc tuyến tính. Tính bằng `numpy.linalg.matrix_rank`: hạng của C là 2 (không phải 3). SVD của C cho ba giá trị kỳ dị khoảng 8.435, 0.918, và 5.3×10^{-16} (tự tính, giá trị thứ ba chỉ là nhiễu số do dấu phẩy động, về lý thuyết đúng là 0). Đây là cách thực dụng để phát hiện hạng thấp trên máy: đếm số giá trị kỳ dị "đáng kể" thay vì kỳ vọng đúng số 0 tuyệt đối.

2.4 Trị riêng, vector riêng, và định lý phổ (spectral theorem)

Với ma trận vuông A: $Av = \lambda v$ nghĩa là A chỉ **co giãn** v theo hệ số λ, không đổi hướng. λ là trị riêng (eigenvalue), v là vector riêng (eigenvector).

Ví dụ xuyên suốt (tự tính). Đặt

$$\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}.$$

Σ đối xứng ($\Sigma = \Sigma^\top$), nên theo định lý phổ, nó có đủ $d = 2$ vector riêng **trực chuẩn** và trị riêng **thực**. Tính bằng `numpy.linalg.eigh`: trị riêng là 2 và 6; vector riêng tương ứng là $v_1 = (1, -1)/\sqrt{2}$ (ứng với $\lambda = 2$) và $v_2 = (1, 1)/\sqrt{2}$ (ứng với $\lambda = 6$). Kiểm tra tay: $\Sigma v_2 = \Sigma\,(1,1)^\top/\sqrt{2} = (6,6)^\top/\sqrt{2} = 6\,v_2$. Đúng.

Ý nghĩa hình học: Σ kéo giãn không gian theo hướng v_2 gấp 6 lần, theo hướng v_1 gấp 2 lần. Nếu Σ là ma trận hiệp phương sai (sẽ dùng lại ở mục 4), đây chính là "trục chính" của một đám mây điểm elip.

Lưu ý: ma trận **không đối xứng** nói chung có thể có trị riêng phức (ví dụ ma trận xoay 90 độ không có trục nào giữ nguyên hướng trong số thực). Phần lớn ma trận gặp trong AI (hiệp phương sai, Hessian, Fisher information, Gram matrix $X^\top X$) đối xứng hoặc đối xứng hoá được, nên trị riêng thực là trường hợp phổ biến nhất ta cần.

2.5 Phân tích giá trị kỳ dị (SVD)

Mọi ma trận $A \in \mathbb{R}^{m\times n}$ (vuông hay không, đối xứng hay không) đều phân tích được:

$$\boxed{A = U\Sigma_s V^\top}$$

với $U \in \mathbb{R}^{m\times m}$, $V \in \mathbb{R}^{n\times n}$ trực giao ($U^\top U = I$, $V^\top V = I$), và Σ_s là ma trận đường chéo không âm, các phần tử $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ gọi là **giá trị kỳ dị** (singular value). Đọc theo chiều V, co giãn theo Σ_s, ghi theo chiều U.

Liên hệ eigen và SVD: nếu A đối xứng và bán xác định dương (positive semi-definite, mọi trị riêng ≥ 0) như Σ ở mục 2.4, SVD và phân tích trị riêng **trùng nhau**: $U = V$ = các vector riêng, giá trị kỳ dị = trị riêng. Kiểm tra (tự tính) trên Σ ở trên: `numpy.linalg.svd` cho giá trị kỳ dị $[6, 2]$, đúng bằng trị riêng (sắp xếp giảm dần) ở mục 2.4.

Ví dụ không đối xứng (tự tính). Với

$$A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix},$$

SVD cho giá trị kỳ dị khoảng 6.708 và 2.236 (không phải số nguyên đẹp như trị riêng của ma trận đối xứng, đây là điều bình thường). Hạng của A là 2 (cả hai giá trị kỳ dị đều khác 0, khớp với $\det A = 15 \neq 0$).

2.6 Chuẩn Frobenius, điều kiện số (condition number), xấp xỉ hạng thấp

Chuẩn Frobenius: $\|A\|_F = \sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\sum_k \sigma_k^2}$ (tổng bình phương mọi phần tử, bằng tổng bình phương giá trị kỳ dị).

Điều kiện số: $\kappa(A) = \sigma_{\max}/\sigma_{\min}$. κ lớn nghĩa là A gần suy biến: nghịch đảo A^{-1} khuếch đại nhiễu rất mạnh theo hướng $\sigma_{\min}$.

Ví dụ (tự tính): $\kappa(\Sigma) = 6/2 = 3$ (ổn). Ngược lại, với

$$B = \begin{pmatrix} 1 & 0.9999 \\ 0.9999 & 1 \end{pmatrix},$$

trị riêng là khoảng 1.9999 và 0.0001, nên $\kappa(B) \approx 20000$ (tự tính): hai hàng gần như trùng nhau, nên nghịch đảo B^{-1} khuếch đại nhiễu lên khoảng 20000 lần. Đây chính là lý do thuật toán natural gradient (bài 06) hay least-squares với λ ridge (gặp lại ở mục 2.8) cần một hệ số chính quy hoá: cộng thêm λI để tránh nghịch đảo một ma trận gần suy biến.

Xấp xỉ hạng thấp tốt nhất (định lý Eckart-Young): với mọi $k < \text{rank}(A)$, xấp xỉ hạng k gần A nhất theo chuẩn Frobenius là giữ lại k giá trị kỳ dị lớn nhất, bỏ phần còn lại:

$$\boxed{A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top, \qquad \|A - A_k\|_F = \sqrt{\sum_{i>k} \sigma_i^2}}$$

Ví dụ (tự tính) trên Σ: xấp xỉ hạng 1 tốt nhất là $\Sigma_1 = 6\, v_2 v_2^\top = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$, sai số Frobenius $\|\Sigma - \Sigma_1\|_F = \sigma_2 = 2$ (khớp với tính trực tiếp bằng numpy). Đây chính là nguyên lý của nén mô hình bằng phân tích hạng thấp (PCA ở bài 03, randomized SVD ở bài 04): giữ vài hướng có "năng lượng" (giá trị kỳ dị) lớn nhất, bỏ phần nhỏ.

2.7 Tích Kronecker

Tích Kronecker $A \otimes B$ ghép mỗi phần tử A_{ij} thành một khối $A_{ij}B$. Với A, B cỡ 2×2, kết quả cỡ 4×4.

Ví dụ (tự tính):

$$P = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad Q = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P \otimes Q = \begin{pmatrix} 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \\ 0 & 3 & 0 & 4 \\ 3 & 0 & 4 & 0 \end{pmatrix}.$$

Tích Kronecker không xuất hiện trực tiếp trong 35 bài của thư mục này, nhưng xuất hiện trong `optimization content/` (Shampoo, bài opt 14; KFAC) để xây preconditioner xấp xỉ mà không cần lưu toàn bộ ma trận Hessian: nếu Hessian xấp xỉ bằng $A \otimes B$, nghịch đảo nó chỉ cần nghịch đảo riêng A và B (hai ma trận nhỏ) thay vì một ma trận khổng lồ, vì $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$.

2.8 Bình phương tối thiểu và giả nghịch đảo

Bài toán: tìm W sao cho $WX \approx Y$ (ví dụ X là activation của một mạng, Y là mục tiêu cần khớp). Nghiệm bình phương tối thiểu có chính quy hoá ridge:

$$\boxed{W^\star = \arg\min_W \|WX - Y\|_F^2 + \lambda \|W\|_F^2 = YX^\top (XX^\top + \lambda I)^{-1}}$$

λI giữ vai trò giống mục 2.6: tránh nghịch đảo $XX^\top$ khi nó gần suy biến (ví dụ số mẫu ít hơn số chiều). Công thức này xuất hiện lại nguyên vẹn ở `interp content/00` (mục 2.8, phép tính D_{lin} cho dự án pruning): tìm ma trận tuyến tính tốt nhất ”vá” lại activation của model con cho khớp với model gốc.

---

3. Giải tích nhiều biến

3.1 Gradient, Jacobian, Hessian

Cho hàm scalar (một số ra) $f : \mathbb{R}^d \to \mathbb{R}$. **Gradient** là vector đạo hàm riêng:

$$\nabla f(x) = \Big(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_d} \Big)^\top.$$

∇f chỉ **hướng tăng nhanh nhất** của f; đi ngược hướng đó là gradient descent.

Cho hàm vector (nhiều số ra) $g : \mathbb{R}^n \to \mathbb{R}^m$, **Jacobian** $J \in \mathbb{R}^{m \times n}$ có $J_{ij} = \partial g_i / \partial x_j$: hàng i là gradient của đầu ra thứ i.

Hessian của f là ma trận đạo hàm bậc hai, $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. Khi f khả vi liên tục hai lần, H **đối xứng** (đạo hàm chéo không phụ thuộc thứ tự lấy đạo hàm, định lý Schwarz).

Ví dụ (tự tính). $f(x, y) = x^2 y + \sin(y)$ tại điểm $(x, y) = (1, 0)$.

$$\nabla f = \big(2xy,\ x^2 + \cos y\big), \qquad \nabla f(1, 0) = (0,\ 2).$$

$$H = \begin{pmatrix} 2y & 2x \\ 2x & -\sin y \end{pmatrix}, \qquad H(1, 0) = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}.$$

Kiểm bằng sai phân hữu hạn (finite difference, bước $h = 10^{-4}$ đến 10^{-6}): gradient khớp $(0, 2.0)$, Hessian khớp $\begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$ (sai số dưới 10^{-3}, đúng như lý thuyết sai phân hữu hạn dự đoán).

Trị riêng của $H(1, 0)$ là $+2$ và -2 (tự tính, `numpy.linalg.eigvalsh`): một dương một âm, nghĩa là $(1, 0)$ là một **điểm yên ngựa** (saddle point): f tăng theo một hướng, giảm theo hướng khác. Phân biệt cực tiểu, cực đại, điểm yên bằng dấu trị riêng Hessian là công cụ trung tâm khi phân tích loss landscape (liên hệ `optimization content/` bài 26, 27).

3.2 Quy tắc chuỗi (chain rule): dạng scalar, dạng vector, dạng ma trận

Dạng scalar quen thuộc: $\dfrac{d}{dx} h(g(x)) = h'(g(x))\, g'(x)$.

Dạng vector (tổng quát hoá, đây là công cụ cốt lõi của bài 01): nếu $z = g(x)$ ($x \in \mathbb{R}^n$, $z \in \mathbb{R}^m$) và $y = h(z)$ ($y \in \mathbb{R}$), thì

$$\boxed{\nabla_x y = J_g^\top \, \nabla_z y}$$

với $J_g \in \mathbb{R}^{m \times n}$ là Jacobian của g. Đây chính là công thức mà backpropagation lặp lại ở mỗi lớp.

Ví dụ (tự tính). $g(x, y) = (x + y,\ xy)$, $h(u, v) = u^2 + v$, tại $(x, y) = (2, 3)$.

$$J_g = \begin{pmatrix} 1 & 1 \\ y & x \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 3 & 2 \end{pmatrix}, \qquad \nabla h = (2u,\ 1) = (2(x + y),\ 1) = (10,\ 1).$$

$$\nabla_{(x,y)}(h \circ g) = J_g^\top \nabla h = \begin{pmatrix} 1 & 3 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 10 \\ 1 \end{pmatrix} = \begin{pmatrix} 13 \\ 12 \end{pmatrix}.$$

Kiểm bằng sai phân hữu hạn trực tiếp trên hàm hợp $f(x, y) = (x + y)^2 + xy$: ra đúng $(13, 12)$ (tự tính).

Dạng ma trận (dùng trong mọi lớp linear của mạng neural). Với $L = \|Wx - y\|_2^2$:

$$\boxed{\frac{\partial L}{\partial W} = 2(Wx - y)\, x^\top}$$

Ví dụ (tự tính): $W = \begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix}$, $x = (1, 2)$, $y = (2, 1)$. $Wx = (5, 1)$, $Wx - y = (3, 0)$, $L = 9$.

$$\frac{\partial L}{\partial W} = 2(3, 0)\,(1, 2) = \begin{pmatrix} 6 & 12 \\ 0 & 0 \end{pmatrix}.$$

Sai phân hữu hạn trên từng phần tử của W cho đúng cùng ma trận này (tự tính, khớp đến 10^{-6}).

3.3 Automatic differentiation: forward mode và reverse mode

Autodiff (bài 02) tính đạo hàm **chính xác** (không phải xấp xỉ như sai phân hữu hạn) bằng cách phân rã hàm thành các phép toán cơ bản và áp chain rule lặp lại qua một ”đồ thị tính toán” (computational graph).

Ví dụ. $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ tại $(x_1, x_2) = (2, 3)$. Đồ thị tính toán: $v_1 = x_1 x_2$, $v_2 = \sin(x_1)$, $f = v_1 + v_2$.

Forward mode: chọn một hướng "seed" cho đầu vào, lan truyền đạo hàm **cùng chiều** với giá trị, từ input ra output. Seed $(\dot{x}_1, \dot{x}_2) = (1, 0)$ (muốn $\partial f/\partial x_1$): $\dot{v}_1 = x_2\dot{x}_1 + x_1\dot{x}_2 = 3$, $\dot{v}_2 = \cos(x_1)\dot{x}_1 = \cos(2) \approx -0.416$, $\dot{f} = \dot{v}_1 + \dot{v}_2 \approx 2.584$. Seed $(0, 1)$ (muốn $\partial f/\partial x_2$): $\dot{v}_1 = x_1 = 2$, $\dot{v}_2 = 0$, $\dot{f} = 2$. **Cần một lượt chạy cho mỗi input.**

Reverse mode (chính là backpropagation): lan truyền đạo hàm **ngược chiều**, từ output về input, dùng "adjoint" $\bar{v} = \partial f/\partial v$. $\bar{f} = 1$ (gốc), $\bar{v}_1 = \bar{v}_2 = 1$ (vì $f = v_1 + v_2$), rồi $\bar{x}_1 = \bar{v}_1\, x_2 + \bar{v}_2 \cos(x_1) \approx 3 + (-0.416) = 2.584$, $\bar{x}_2 = \bar{v}_1\, x_1 = 2$. **Một lượt chạy duy nhất cho mọi input**, miễn là chỉ có một (hoặc ít) output.

Cả hai cho cùng kết quả $\partial f/\partial x_1 \approx 2.584$, $\partial f/\partial x_2 = 2$ (tự tính, khớp sai phân hữu hạn).

Vì sao mạng neural luôn dùng reverse mode: loss là **một số scalar**, còn số tham số có thể lên đến hàng tỷ. Forward mode cần một lượt chạy riêng cho **mỗi tham số** (không khả thi). Reverse mode cần **một** lượt backward duy nhất để lấy gradient theo **toàn bộ** tham số cùng lúc. Forward mode chỉ có lợi khi số input ít hơn hẳn số output (hiếm gặp trong deep learning, nhưng gặp trong một số phép tính Hessian-vector product).

3.4 Ví dụ tay: backpropagation qua mạng 2 lớp

Mạng: input $x \in \mathbb{R}^2$, lớp ẩn $z_1 = W_1 x + b_1 \in \mathbb{R}^2$, ReLU $a_1 = \max(0, z_1)$, lớp ra $\hat{y} = W_2 a_1 + b_2 \in \mathbb{R}$, loss $L = \frac{1}{2}(\hat{y} - t)^2$.

Số liệu (tự tính, đã kiểm bằng sai phân hữu hạn cho mọi tham số):

$$x = (1, 2), \quad W_1 = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix}, \quad b_1 = (0.05, -0.1), \quad W_2 = (0.4, -0.6), \quad b_2 = 0.2, \quad t = 1.$$

Forward:

$$z_1 = W_1 x + b_1 = (0.5 \cdot 1 - 0.3 \cdot 2 + 0.05,\ 0.2 \cdot 1 + 0.8 \cdot 2 - 0.1) = (-0.05,\ 1.7).$$

$$a_1 = \max(0, z_1) = (0,\ 1.7) \qquad \text{(thành phần thứ nhất của } z_1 \text{ bị ReLU chặn vì âm).}$$

$$\hat{y} = W_2 a_1 + b_2 = 0.4 \cdot 0 + (-0.6) \cdot 1.7 + 0.2 = -0.82.$$

$$L = \tfrac{1}{2}(-0.82 - 1)^2 = \tfrac{1}{2}(1.82)^2 = 1.6562.$$

Backward (reverse-mode, từng bước):

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - t = -1.82.$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}}\, a_1^\top = -1.82\,(0,\ 1.7) = (0,\ -3.094), \qquad \frac{\partial L}{\partial b_2} = -1.82.$$

$$\frac{\partial L}{\partial a_1} = \frac{\partial L}{\partial \hat{y}}\, W_2 = -1.82\,(0.4, -0.6) = (-0.728,\ 1.092).$$

Qua ReLU: đạo hàm là 1 nơi $z_1 > 0$, 0 nơi $z_1 < 0$ (mask $= (0, 1)$ cho ví dụ này):

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \odot \mathbb{1}[z_1 > 0] = (-0.728 \cdot 0,\ 1.092 \cdot 1) = (0,\ 1.092).$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} x^\top = \begin{pmatrix} 0 \\ 1.092 \end{pmatrix} (1,\ 2) = \begin{pmatrix} 0 & 0 \\ 1.092 & 2.184 \end{pmatrix}, \qquad \frac{\partial L}{\partial b_1} = (0,\ 1.092).$$

Mọi con số trên khớp chính xác với đạo hàm tính bằng sai phân hữu hạn trên từng phần tử của W_1, b_1, W_2, b_2 (tự tính, sai số dưới 10^{-6}).

Điểm đáng chú ý: ReLU chết (dead ReLU). Đơn vị ẩn thứ nhất có thành phần tương ứng của z_1 bằng $-0.05 < 0$, nên thành phần tương ứng của a_1 bằng 0, và toàn bộ dòng đầu của $\partial L/\partial W_1$ cùng thành phần đầu của $\partial L/\partial b_1$ đều **bằng 0**. Nếu đơn vị này giữ nguyên bị chặn qua nhiều bước cập nhật, dòng đầu của W_1 và thành phần đầu của b_1 sẽ không đổi, đơn vị đó không còn học được gì: đây chính là hiện tượng "ReLU chết" sẽ nhắc lại ở mục 8. Hiện tượng này không phải lỗi tính toán, mà là hệ quả trực tiếp của đạo hàm ReLU bằng 0 phía âm.

---

4. Xác suất

4.1 Biến ngẫu nhiên, kỳ vọng, phương sai

Một biến ngẫu nhiên X có phân phối xác suất mô tả khả năng nó nhận từng giá trị. **Kỳ vọng** $\mathbb{E}[X]$ là trung bình có trọng số xác suất; **phương sai** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ đo độ phân tán. Với hai biến, **hiệp phương sai** $\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$ đo mức hai biến thay đổi cùng nhau; ma trận hiệp phương sai của một vector ngẫu nhiên gom tất cả các hiệp phương sai cặp đôi lại, và luôn **đối xứng, bán xác định dương**, nên dùng được đúng bộ công cụ ở mục 2.4-2.6.

4.2 Gaussian đa biến

Vector ngẫu nhiên $X \in \mathbb{R}^d$ theo phân phối Gaussian đa biến $\mathcal{N}(\mu, \Sigma)$ có mật độ

$$\boxed{p(x) = \frac{1}{(2\pi)^{d/2}\,|\Sigma|^{1/2}} \exp\Big(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big)}$$

Đại lượng $(x-\mu)^\top \Sigma^{-1}(x-\mu)$ gọi là **khoảng cách Mahalanobis bình phương**: đo x cách μ bao nhiêu "độ lệch chuẩn", có tính đến hướng và độ co giãn của Σ (khác chuẩn Euclid thường, vốn coi mọi hướng như nhau).

Ví dụ (tự tính), dùng lại $\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ **ở mục 2.4,** $\mu = (0, 0)$**, điểm** $x = (1, 1)$**.**

$$\Sigma^{-1} = \frac{1}{12}\begin{pmatrix} 4 & -2 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} 1/3 & -1/6 \\ -1/6 & 1/3 \end{pmatrix}.$$

$$\text{Mahalanobis}^2 = x^\top \Sigma^{-1} x = \tfrac{1}{3}, \qquad \text{Mahalanobis} = \sqrt{1/3} \approx 0.577.$$

$$|\Sigma| = 16 - 4 = 12, \qquad p(1,1) = \frac{1}{2\pi\sqrt{12}} \exp(-\tfrac{1}{6}) \approx 0.0389.$$

Vì $x = (1,1)$ nằm dọc hướng $v_2 = (1,1)/\sqrt{2}$ (trục có trị riêng **lớn** là 6, phương sai lớn), khoảng cách Mahalanobis của nó (0.577) nhỏ hơn khoảng cách Euclid ($\|x\|_2 = \sqrt{2} \approx 1.414$): theo hướng "dễ xảy ra" của Σ, cùng một khoảng cách Euclid trở nên "gần" hơn về xác suất.

4.3 Định lý Bayes

$$\boxed{P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}}$$

Ví dụ kinh điển (tự tính). Bệnh hiếm, tỉ lệ mắc trong dân số $P(\text{bệnh}) = 1\%$. Test có độ nhạy (sensitivity) 99% ($P(\text{dương} \mid \text{bệnh}) = 0.99$) và tỉ lệ dương giả (false positive rate) 5% ($P(\text{dương} \mid \text{không bệnh}) = 0.05$). Hỏi: dương tính thì xác suất mắc bệnh thật là bao nhiêu?

$$P(\text{dương}) = 0.01 \cdot 0.99 + 0.99 \cdot 0.05 = 0.0099 + 0.0495 = 0.0594.$$

$$P(\text{bệnh} \mid \text{dương}) = \frac{0.0099}{0.0594} \approx 0.1667 \ (16.7\%).$$

Dù test có độ nhạy 99% (nghe rất chắc chắn), xác suất mắc bệnh thật khi dương tính chỉ khoảng 1/6, vì bệnh hiếm (prior nhỏ) và tỉ lệ dương giả không phải 0. Đây là ví dụ kinh điển về "base-rate fallacy": đánh giá một phân loại (reward model, bộ kiểm tra tự động) chỉ bằng độ nhạy, bỏ qua tỉ lệ cơ sở, dễ hiểu sai mức tin cậy thực tế của nó.

4.4 MLE và MAP

Maximum Likelihood Estimation (MLE): chọn tham số θ làm cực đại hoá xác suất quan sát được dữ liệu, $\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_i p(x_i \mid \theta)$.

Ví dụ (tự tính). Tung một đồng xu lệch 10 lần, được 7 lần mặt "đúng" (head). Với $n = 10$, số lần đúng $k = 7$, log-likelihood là $\ell(p) = k \log p + (n-k)\log(1-p)$; đạo hàm bằng 0 cho $\hat{p}_{\text{MLE}} = k/n = 7/10 = 0.7$ (nghiệm quen thuộc: tần suất quan sát).

Maximum A Posteriori (MAP): thêm một **prior** $p(\theta)$ (niềm tin trước khi thấy dữ liệu), cực đại hoá hậu nghiệm $p(\theta \mid x) \propto p(x \mid \theta)p(\theta)$.

Ví dụ (tự tính). Prior $\text{Beta}(2,2)$ (niềm tin yếu rằng p gần 0.5) cho cùng dữ liệu 7/10. Hậu nghiệm của mô hình Bernoulli-Beta là $\text{Beta}(2+7,\ 2+3) = \text{Beta}(9,5)$. Mode (giá trị MAP) của $\text{Beta}(\alpha,\beta)$ là $(\alpha-1)/(\alpha+\beta-2)$:

$$\hat{p}_{\text{MAP}} = \frac{9-1}{9+5-2} = \frac{8}{12} \approx 0.667.$$

So với MLE 0.7, MAP bị "kéo" nhẹ về phía prior 0.5. Khi dữ liệu càng nhiều, ảnh hưởng của prior càng nhỏ và MAP tiến về MLE; đây là nguyên lý chung của suy luận biến phân và hậu nghiệm Bayes (bài 08, 09, 10).

4.5 Bất đẳng thức tập trung (concentration inequality)

Các bất đẳng thức này trả lời: trung bình của n mẫu lệch khỏi giá trị kỳ vọng thật bao xa, với xác suất bao nhiêu? **Bất đẳng thức Hoeffding** cho biến bị chặn trong $[0,1]$:

$$\boxed{P\big(|\bar{X}_n - \mathbb{E}X| > \epsilon\big) \le 2\exp(-2n\epsilon^2)}$$

Ví dụ (tự tính). $\epsilon = 0.05$, $n = 100$: cận $= 2\exp(-2 \cdot 100 \cdot 0.0025) = 2\exp(-0.5) \approx 1.213$. Cận này **lớn hơn 1**, tức vô nghĩa (một xác suất không thể vượt 1): với $n = 100$ và sai số 0.05, Hoeffding không cho thông

tin gì, cần n lớn hơn. Với $n = 1000$, cùng $\epsilon = 0.05$: cận $= 2\exp(-5) \approx 0.0135$, có ý nghĩa (xác suất lệch quá 5% dưới khoảng 1.35%). Với $\epsilon = 0.1$, $n = 1000$: cận $\approx 4.12 \times 10^{-9}$, rất chặt.

Bài học thực dụng: cận lý thuyết **chặt hay lỏng phụ thuộc mạnh vào n và ϵ**; luôn tính thử bằng số trước khi trích dẫn một bound như một bảo đảm thực tế (liên hệ mục 10, cách đọc paper lý thuyết phản biện).

4.6 Hình học không gian nhiều chiều

Trực giác hình học 2-3 chiều (hình cầu, góc vuông) **đánh lừa** khi d lớn (hàng trăm đến hàng nghìn chiều, đúng kích thước của một residual stream trong transformer).

Nồng độ của chuẩn (tự tính, mô phỏng). Với $z \sim \mathcal{N}(0, I_d)$, $d = 1000$: lấy 20000 mẫu, $\|z\|_2/\sqrt{d}$ có trung bình ≈ 0.9995 và độ lệch chuẩn chỉ ≈ 0.0224. Hầu hết khối lượng xác suất của một Gaussian chuẩn chiều cao tập trung rất gần **mặt cầu** bán kính $\sqrt{d}$, không gần tâm (khác trực giác 2 chiều, nơi điểm "ngẫu nhiên" có thể ở bất kỳ đâu trong hình tròn).

Gần vuông góc thì có rất nhiều (tự tính, mô phỏng). Lấy một vector đơn vị cố định $u \in \mathbb{R}^{1000}$ và 5000 vector đơn vị ngẫu nhiên độc lập: cos góc với u có trung bình ≈ -0.0001 (coi như 0) và độ lệch chuẩn ≈ 0.0318, rất gần $1/\sqrt{d} = 1/\sqrt{1000} \approx 0.0316$ theo lý thuyết. Hai vector ngẫu nhiên trong không gian nhiều chiều **gần như luôn vuông góc**. Hệ quả (đã khai triển kỹ ở `interp content/00` mục 3.1): một không gian d chiều có thể chứa **nhiều hơn** d hướng "gần như độc lập" nếu chấp nhận một chút nhiễu khi chúng không hoàn toàn vuông góc nhau, đây là nền tảng hình học của superposition trong interpretability.

---

5. Lý thuyết thông tin

5.1 Entropy

Entropy $H(p) = -\sum_i p_i \log p_i$ đo độ "bất định" của một phân phối (đơn vị bit nếu dùng $\log_2$, nat nếu dùng ln). Entropy cực đại khi phân phối đều, bằng 0 khi phân phối tập trung hoàn toàn vào một giá trị.

Ví dụ (tự tính): đồng xu lệch $p = (0.3, 0.7)$: $H(p) = -0.3\log_2 0.3 - 0.7\log_2 0.7 \approx 0.881$ bit (so với đồng xu công bằng $p = (0.5, 0.5)$ có $H = 1$ bit, lớn nhất có thể cho 2 giá trị).

5.2 Cross-entropy và KL divergence

$$H(p, q) = -\sum_i p_i \log q_i, \qquad \boxed{\mathrm{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i} = H(p, q) - H(p) \ \geq 0}$$

$\mathrm{KL}(p\|q) = 0$ khi và chỉ khi $p = q$. KL **không đối xứng**: $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$ nói chung, nên không phải một "khoảng cách" thật theo nghĩa toán học (không thoả bất đẳng thức tam giác); đây là lý do bài 12, 13 (optimal transport, Wasserstein GAN) đề xuất một thước đo khác khi KL không phù hợp.

Ví dụ (tự tính): $p = (0.7, 0.2, 0.1)$, $q = (0.5, 0.3, 0.2)$:

$$H(p) \approx 1.157 \text{ bit}, \quad H(p, q) \approx 1.280 \text{ bit}, \quad \mathrm{KL}(p\|q) \approx 0.123 \text{ bit}.$$

5.3 Vì sao loss của language model là cross-entropy (và bằng NLL)

Khi huấn luyện, nhãn thật ở mỗi vị trí là **một token cụ thể**, tức phân phối p suy biến thành một "one-hot": $p_{\text{đúng}} = 1$, còn lại 0. Khi đó $H(p) = 0$ (không còn bất định gì để đo, nhãn đã biết chắc), và

$$H(p, q) = -\log q_{\text{đúng}} = \mathrm{KL}(p\|q).$$

Đây đúng là **negative log-likelihood (NLL)** của token đúng theo phân phối dự đoán q của model. Vậy ”tối thiểu cross-entropy” và ”tối thiểu KL tới phân phối thật” và ”tối đa log-likelihood” là **cùng một bài toán** khi nhãn là one-hot. Ví dụ (tự tính): $q_{\text{đúng}} = 0.5$: NLL $= -\ln(0.5) \approx 0.693$ nat $= -\log_2(0.5) = 1$ bit. **Perplexity** (dùng để báo cáo chất lượng LM) chỉ là $\exp(\text{NLL trung bình theo nat})$, tức ”số lựa chọn hiệu dụng trung bình mà model còn phân vân”.

5.4 Mutual information (MI)

$$I(X;Y) = \mathrm{KL}\big(p(x,y) \,\|\, p(x)p(y)\big) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

đo mức X và Y phụ thuộc nhau; $I = 0$ khi và chỉ khi độc lập.

Ví dụ (tự tính): bảng đồng thời $P(X,Y) = \begin{pmatrix} 0.4 & 0.1 \\ 0.1 & 0.4 \end{pmatrix}$ (cả hai biên $P(X) = P(Y) = (0.5, 0.5)$). $I(X;Y) \approx 0.278$ bit. Có thể kiểm chéo bằng công thức cho biến nhị phân đối xứng: $I = 1 - H_2(0.2)$ với H_2 là entropy nhị phân của tỉ lệ ”lệch” $0.1/0.5 = 0.2$: $H_2(0.2) = -0.2\log_2 0.2 - 0.8\log_2 0.8 \approx 0.722$ bit, $1 - 0.722 = 0.278$ bit, khớp. Information bottleneck (bài 14, 30) dùng chính I này để đo một biểu diễn ẩn Z giữ được bao nhiêu thông tin về nhãn Y và bỏ đi bao nhiêu thông tin dư thừa từ input X.

---

6. Số học máy tính

6.1 Số dấu phẩy động: 3 phần, và đánh đổi exponent/mantissa

Một số dấu phẩy động chuẩn hoá được viết $(-1)^S \times 1.M \times 2^{E-\text{bias}}$: 1 bit dấu S, phần mũ E (quyết định **thang đo**, dynamic range), phần định trị M (quyết định **độ chính xác** trong một thang đo). Nhiều bit E hơn: biểu diễn được số rất lớn và rất nhỏ, nhưng mỗi ”nấc” giữa hai số liền kề thưa hơn. Nhiều bit M hơn: chính xác hơn trong cùng một thang đo.

Định dạng	Dấu	Exponent	Mantissa	Max normal	Dùng khi nào
fp32	1	8	23	$\approx 3.4 \times 10^{38}$	mặc định, tích lũy số (optimizer state)
fp16 (IEEE half)	1	5	10	65504	suy luận/huấn luyện tiết kiệm bộ nhớ, dễ tràn số
bf16	1	8	7	$\approx 3.39 \times 10^{38}$	cùng dynamic range với fp32, ít chính xác hơn, khó tràn số
fp8 E4M3	1	4	3	448	trọng số, activation (bài 16)
fp8 E5M2	1	5	2	57344	gradient, cần dynamic range rộng (bài 16)

Hai dòng cuối lấy đúng định nghĩa và số liệu từ bài 16 trong thư mục này (paper định nghĩa chính xác E4M3: 1 dấu + 4 exponent + 3 mantissa, bias 7, max normal 448; E5M2: 1 dấu + 5 exponent + 2 mantissa, bias 15, max normal 57344). Khuyến nghị của paper 16: dùng E4M3 cho trọng số/activation (cần chính xác hơn, phân phối hẹp quanh 0), E5M2 cho gradient (cần dynamic range rộng hơn, nhiều giá trị rất nhỏ lẫn vài giá trị lớn).

6.2 Khoảng cách giữa các số biểu diễn được (ví dụ bf16, tự tính)

Với mantissa 7 bit (bf16), khoảng cách giữa hai số liền kề trong cùng một ”octave” $[2^k, 2^{k+1})$ là 2^{k-7}: khoảng cách **nhân đôi** mỗi khi giá trị vượt một lũy thừa của 2. Mô phỏng làm tròn kiểu bf16 (tự tính, cắt mantissa về 7 bit, làm tròn đến số chẵn gần nhất):

Giá trị gốc	Làm tròn về bf16	Octave
255	255 (đúng)	$[128, 256)$, bước 1
256	256 (đúng)	$[256, 512)$, bước 2
257	256	$[256, 512)$, bước 2
258	258 (đúng)	$[256, 512)$, bước 2
1000	1000 (đúng)	$[512, 1024)$, bước 4
1001	1000	$[512, 1024)$, bước 4

Quan sát: 257 không biểu diễn chính xác được trong bf16 (làm tròn về 256), vì bước nhảy ở octave $[256, 512)$ đã là 2; nhưng 255 (octave thấp hơn, bước nhảy 1) thì vẫn chính xác. Bài học: **sai số làm tròn của bf16/fp16/fp8 tăng theo độ lớn giá trị tuyệt đối**, không phải hằng số, đây là lý do cần "scaling factor" trước khi ép tensor về một định dạng hẹp (bài 16, mục 3.3 của chính bài đó).

6.3 log-sum-exp: vì sao softmax "ngây thơ" tràn số

Softmax chuẩn: $p_i = e^{z_i} / \sum_j e^{z_j}$. Khi logit z lớn (ví dụ sau một vài lớp không chuẩn hoá tốt), e^{z_i} có thể tràn số.

Ví dụ (tự tính). Logits $z = (1000, 1001, 1002)$ tính trực tiếp bằng `numpy` ở độ chính xác float64: `exp(z)` cho ra `inf` ở cả ba phần tử (RuntimeWarning: overflow encountered in exp), tổng là `inf`, và `inf/inf = nan`. Softmax "ngây thơ" cho ra `[nan, nan, nan]`, hoàn toàn vô dụng.

Thủ thuật log-sum-exp: trừ giá trị lớn nhất trước khi lấy exp, vì softmax không đổi khi cộng một hằng số vào mọi logit:

$$\boxed{p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \qquad m = \max_j z_j, \qquad \log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}}$$

Với cùng $z = (1000, 1001, 1002)$, $m = 1002$: logit đã trừ là $(-2, -1, 0)$, hoàn toàn an toàn, softmax $\approx (0.0900,\ 0.2447,\ 0.6652)$ (tự tính, khớp ba chữ số thập phân với kỳ vọng lý thuyết $e^{-2} : e^{-1} : 1$ chuẩn hoá). $\log \sum_j e^{z_j} \approx 1002.408$ (tự tính): một số hợp lý, không tràn.

Đây chính là nội dung của bài 15 (online softmax): tính m và tổng theo kiểu "chạy" (online), cập nhật dần khi quét qua vector, để không phải lưu toàn bộ vector logit hai lần; đây cũng là kỹ thuật lõi bên trong FlashAttention (`llm content/44, 45`) khi tính attention theo khối (tile) mà không vật chất hoá toàn bộ ma trận attention.

6.4 Catastrophic cancellation: công thức "đúng về đại số" nhưng sai về số

Công thức $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ đúng về đại số, nhưng khi $\mathbb{E}[X^2]$ và $(\mathbb{E}X)^2$ đều lớn và gần nhau, phép trừ hai số lớn gần bằng nhau ở độ chính xác hữu hạn gây **mất hết các chữ số có nghĩa**.

Ví dụ (tự tính). 10^5 mẫu Gaussian với trung bình 10^4, phương sai thật 1. Tính ở độ chính xác float64 làm chuẩn: phương sai ≈ 0.993. Ép dữ liệu và phép tính về float32 rồi dùng công thức một lượt $\mathbb{E}[X^2] - (\mathbb{E}X)^2$: trong lần chạy cụ thể của chúng tôi, kết quả ra đúng 0, sai khác 100% so với giá trị thật (kết quả cụ thể có thể lệch chút theo nền tảng/BLAS, nhưng hiện tượng mất chính xác gần như hoàn toàn là ổn định). Đổi sang công thức hai lượt (trừ trung bình trước, rồi bình phương, rồi lấy trung bình): Var ≈ 0.993, sai số tương đối chỉ khoảng 6×10^{-7} (tự tính), đúng bằng độ chính xác float32 kỳ vọng.

Bài học: khi viết code tính phương sai, chuẩn hoá (normalization layer), hay bất kỳ công thức "trừ hai số lớn gần nhau" nào, nên dùng công thức hai lượt (hoặc thuật toán Welford), đặc biệt khi đang chạy ở fp16/bf16.

7. Lý thuyết học sâu ở mức khái niệm

Mục này **không** chứng minh lại các định lý của bài 19-27, 31-34 (các bài đó tự làm việc đó, với giả thiết chặt chẽ hơn nhiều những gì nêu ở đây). Mục tiêu chỉ là đủ trực giác để đọc các bài đó **không bị chặn ngay ở trang đầu**.

7.1 Overparameterization: vì sao ”quá nhiều tham số” không tất yếu là xấu

Trực giác thống kê cổ điển (bias-variance trade-off, `ml basics content/` bài 02): mô hình nhiều tham số hơn dữ liệu sẽ ”khớp vẹt” (overfit), tổng quát hoá kém. Thực nghiệm hiện đại cho thấy mạng neural rất lớn, tham số nhiều hơn hẳn số điểm dữ liệu, **vẫn** tổng quát hoá tốt (`ml basics content/` bài 03, double descent). Bài 31 (khảo sát) tổng hợp hai mảng lý thuyết giải thích hiện tượng này: (a) overparameterization giúp **bài toán tối ưu dễ hơn**, gradient descent tới nghiệm khớp hoàn toàn dữ liệu dù landscape không lồi (bài 33 chứng minh cho vài kiến trúc cụ thể); (b) cách gradient descent **chọn nghiệm** trong số rất nhiều nghiệm khớp dữ liệu (implicit regularization) giúp tổng quát hoá tốt dù khớp hoàn hảo dữ liệu nhiễu (”benign overfitting”, bài 32 đặc trưng hoá chính xác điều kiện này cho hồi quy tuyến tính).

7.2 Neural Tangent Kernel (NTK): tuyến tính hoá quanh điểm khởi tạo

Ý tưởng (bài 23, 24): khi mạng **rất rộng**, huấn luyện bằng gradient descent gần giống huấn luyện phiên bản **tuyến tính hoá** của nó quanh tham số khởi tạo θ_0:

$$f(x;\theta) \approx f(x;\theta_0) + \nabla_\theta f(x;\theta_0)^\top (\theta - \theta_0).$$

Đây là một khai triển Taylor bậc 1 (liên hệ mục 3.1: đúng là tuyến tính hoá bằng gradient, giống ∇f ở đầu bài). Gradient descent trên vế phải tương đương **kernel gradient descent** với kernel

$$\boxed{\Theta(x,x') = \nabla_\theta f(x;\theta_0)^\top \nabla_\theta f(x';\theta_0)}$$

Bài 23 chứng minh: khi độ rộng tiến ra vô hạn, Θ gần như **không đổi** suốt quá trình huấn luyện (gọi là NTK), và động lực huấn luyện trở thành một bài toán **hồi quy kernel** tuyến tính, giải được bằng công cụ giải tích cổ điển, giải thích được tốc độ hội tụ.

Ví dụ tối giản (tự tính, minh hoạ, không phải NTK đầy đủ của một mạng thật). Model tuyến tính 1 tham số $f(x;w) = wx$: $\partial f/\partial w = x$, nên kernel là $\Theta(x,x') = x\,x'$. Với $x \in \{1,2,3\}$:

$$\Theta = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$$

(tự tính, mỗi phần tử $\Theta_{ij} = x_i x_j$). Đây là ma trận hạng 1 (liên hệ mục 2.3): với model càng đơn giản, kernel càng thoái hoá; với mạng sâu thật, Θ có một công thức đóng dạng đệ quy qua các lớp (bài 23 suy ra cho ReLU) phức tạp hơn nhiều, không trình bày lại ở đây.

7.3 Lazy training và feature learning: hai chế độ khác nhau

Bài 24 chỉ ra: ”huấn luyện giống NTK” (gọi là **lazy training**, tham số hầu như không di chuyển xa khỏi θ_0, chỉ output thay đổi) là hệ quả của cách **scale** hàm số theo kích thước mô hình, không chỉ của việc có nhiều tham số. Ở chế độ ngược lại, **feature learning**, biểu diễn ẩn của mạng thực sự thay đổi trong lúc huấn luyện (học đặc trưng mới, không chỉ nội suy trong không gian đặc trưng cố định tại khởi tạo). Bài 24 cũng thực nghiệm cho thấy các CNN hiệu năng cao **không** nằm trong chế độ lazy. Bài học thực dụng: NTK là một công

cụ phân tích rất đẹp về mặt toán nhưng mô tả đúng nhất một chế độ (lazy) mà các mô hình tốt nhất trong thực tế **không** hoạt động ở đó, nên cần đọc có phê phán (xem lại mục 10).

7.4 Lan truyền tín hiệu qua mạng sâu chưa huấn luyện (mean-field)

Câu hỏi của bài 26, 27: một mạng rất sâu, **ngẫu nhiên, chưa huấn luyện**, còn truyền được tín hiệu không? Bằng lý thuyết mean-field: với một số phân phối trọng số/bias ban đầu, tồn tại một **ranh giới order-to-chaos**; gần ranh giới (”critical”), tín hiệu và gradient truyền ổn định qua nhiều lớp; xa ranh giới, tín hiệu co về hằng số (order) hoặc khuếch đại hỗn loạn (chaos), cả hai triệt tiêu thông tin phân biệt giữa các input khi lớp sâu dần. Đây là nền tảng cho các công thức khởi tạo trọng số có kiểm soát phương sai (`optimization content/` bài 37, Kaiming init) và một phần động lực của skip-connection (bài 18: resnet làm chậm hẳn quá trình ”nhiễu hoá” gradient theo độ sâu).

7.5 Information bottleneck nhìn từ huấn luyện thật

Bài 30 quan sát thực nghiệm (không phải lý thuyết hình thức như 26/27): trong lúc huấn luyện một DNN, thông tin tương hỗ (mục 5.4) giữa biểu diễn ẩn và input/nhãn trải qua hai pha, drift (khớp dữ liệu nhanh) rồi diffusion (nén biểu diễn, bỏ thông tin dư thừa không liên quan nhãn). Bài 14 xây một cận dưới biến phân khả vi cho đúng mục tiêu information bottleneck này để huấn luyện trực tiếp bằng gradient descent (không chỉ quan sát), kết nối lại với suy luận biến phân (mục 4.4, bài 08, 09).

7.6 Khi nào đủ để đọc bài 19-34, khi nào cần đọc kỹ hơn

Nếu mục tiêu là **hiểu trực giác** (tại sao mô hình lớn vẫn học được, tại sao init quan trọng, NTK nghĩa là gì): mục 7.1-7.5 ở đây là đủ để đọc phần dẫn nhập và kết luận của các bài 19-34. Nếu mục tiêu là **dùng lại chứng minh hoặc công thức chính xác** (ví dụ suy ra công thức NTK cho một kiến trúc cụ thể, hay điều kiện chính xác của benign overfitting): phải đọc trực tiếp bài tương ứng, phần dẫn nhập ở đây không đủ.

---

8. Cạm bẫy thực hành

8.1 Quy ước hàng/cột và layout của gradient

Có hai quy ước phổ biến cho gradient của hàm scalar theo vector: ”denominator layout” (∇f là vector cột, cùng chiều x, dùng trong phần lớn tài liệu machine learning) và ”numerator layout” (gradient là vector hàng, phổ biến hơn trong một số sách giải tích). Bài 01 trong thư mục này chọn rõ một quy ước và nói thẳng lý do; khi so một công thức giữa hai paper khác nhau mà thấy ”bị chuyển vị” (transpose) so với trí nhớ của bạn, nhiều khả năng chỉ là khác quy ước layout, không phải một bên sai.

8.2 ReLU chết (dead ReLU) và gradient biến mất

Mục 3.4 đã cho một ví dụ cụ thể: khi $z < 0$ ở một đơn vị ReLU, gradient qua đơn vị đó bằng 0 tuyệt đối, không phải ”rất nhỏ”. Nếu toàn bộ batch khiến một neuron luôn nhận $z < 0$, neuron đó không học được nữa cho tới khi (nếu có) một thay đổi ở lớp trước đẩy z dương trở lại. Khắc phục thường gặp: learning rate nhỏ hơn, khởi tạo cẩn thận (mục 7.4), hoặc đổi activation (Leaky ReLU, GELU) để giữ một gradient nhỏ khác 0 ở phía âm.

8.3 Nhầm công thức mẫu (sample) và tổng thể (population)

Phương sai mẫu ”không chệch” chia cho $n - 1$ (hiệu chỉnh Bessel), phương sai MLE chia cho n. Hai công thức cho hai số **khác nhau** (chênh nhau hệ số $n/(n-1)$, đáng kể khi n nhỏ). Nhiều thư viện (ví dụ `numpy.std` mặc định chia n, `pandas.std` mặc định chia $n-1$) chọn mặc định khác nhau: luôn kiểm tra tài liệu hàm trước khi so sánh số giữa hai công cụ.

8.4 Hướng bất đẳng thức Jensen

Với f **lồi** (convex): $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$. Với f **lõm** (concave, ví dụ log): $\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$. Dấu bất đẳng thức bị đảo khi nhầm lồi với lõm là lỗi rất thường gặp khi suy ra ELBO (evidence lower bound, bài 08, 09): ELBO là một **cận dưới** của log-likelihood chính vì log lõm.

8.5 Tràn số và mất số khi không dùng log-sum-exp

Lặp lại từ mục 6.3: bất kỳ chỗ nào có `softmax`, `logsumexp`, hay tính log-likelihood của một tích nhiều xác suất nhỏ (ví dụ log-likelihood của cả một câu dài), luôn làm việc trong không gian **log** và dùng log-sum-exp, không nhân trực tiếp nhiều số nhỏ hơn 1 rồi mới lấy log.

8.6 Ma trận gần suy biến làm nổ số (ill-conditioning)

Mục 2.6 cho ví dụ cụ thể: $\kappa \approx 20000$ khi hai hàng gần như trùng nhau. Bất kỳ phép nghịch đảo ma trận trong code (natural gradient, Gauss-Newton, least-squares không ridge) nên luôn cộng thêm một λI nhỏ, hoặc dùng giả nghịch đảo (pseudo-inverse) có cắt ngưỡng giá trị kỳ dị nhỏ, trước khi tin vào kết quả.

8.7 Lưu ý khi viết công thức markdown/pandoc cho các bài trong thư mục

Khi build các bài (01-35) hoặc file này thành PDF qua pandoc: (a) luôn đóng đủ cặp dấu mở và đóng công thức, một dấu lẻ làm pandoc đọc sai toàn bộ phần còn lại của file; (b) không để một chữ số đứng dính ngay sau phần đóng công thức mà không có khoảng trắng ở giữa, ví dụ viết "k = 5" (có khoảng trắng trước số 5) thay vì để số 5 dính liền ngay sau phần đóng công thức của k; (c) trong bảng markdown, không dùng dấu gạch đứng trần `|` bên trong công thức (nó bị hiểu là dấu phân cột), dùng `\mid` cho điều kiện xác suất, `\vert` hoặc `\Vert` cho giá trị tuyệt đối/chuẩn; (d) tránh viết địa chỉ dạng `tên@miền` ngoài khối code, một số bộ render Markdown tự biến nó thành liên kết; (e) không dùng `\par` trong tiêu đề.

8.8 So sánh không cùng điều kiện

Khi đọc kết quả thực nghiệm (`ml basics content/`, `optimization content/`), luôn hỏi: cùng số bước huấn luyện hay cùng số giờ GPU? Cùng kích thước batch? Cùng được tune hyperparameter công bằng không? Rất nhiều "phương pháp mới thắng baseline" chỉ vì baseline chưa được tune đủ kỹ (nguyên tắc này lặp lại ở hầu hết thư mục chuyên đề, ví dụ `rl content/00` mục 7).

---

9. Bảng thuật ngữ Anh-Việt

English	Tiếng Việt / ghi chú
Vector, matrix, tensor	vector, ma trận, tensor (mảng nhiều chiều)
Rank	hạng (số chiều độc lập tuyến tính)
Eigenvalue, eigenvector	trị riêng, vector riêng
Singular value, SVD	giá trị kỳ dị, phân tích giá trị kỳ dị
Condition number	điều kiện số (độ "khó nghịch đảo")
Low-rank approximation	xấp xỉ hạng thấp
Frobenius norm	chuẩn Frobenius
Orthogonal, orthonormal	trực giao (vuông góc), trực chuẩn (vuông góc và chuẩn 1)
Positive (semi-)definite	(bán) xác định dương
Gradient	gradient (vector đạo hàm riêng)
Jacobian	ma trận Jacobian (đạo hàm của hàm vector)
Hessian	ma trận Hessian (đạo hàm bậc hai)
Chain rule	quy tắc chuỗi

English	Tiếng Việt / ghi chú
Automatic differentiation (AD)	vi phân tự động (autodiff)
Forward mode / reverse mode	chế độ thuận / chế độ ngược
Backpropagation	lan truyền ngược
Saddle point	điểm yên ngựa
Random variable	biến ngẫu nhiên
Expectation, variance, covariance	kỳ vọng, phương sai, hiệp phương sai
Multivariate Gaussian	Gaussian đa biến (phân phối chuẩn nhiều chiều)
Mahalanobis distance	khoảng cách Mahalanobis
Bayes' theorem	định lý Bayes
Prior, posterior, likelihood	niềm tin trước (tiên nghiệm), hậu nghiệm, hàm hợp lý
Maximum likelihood estimation (MLE)	ước lượng hợp lý cực đại
Maximum a posteriori (MAP)	ước lượng hậu nghiệm cực đại
Estimator, bias, consistency	bộ ước lượng, độ chệch, tính nhất quán
Concentration inequality	bất đẳng thức tập trung
Entropy	entropy (độ bất định)
Cross-entropy	entropy chéo
Kullback-Leibler (KL) divergence	độ lệch KL
Mutual information (MI)	thông tin tương hỗ
Information bottleneck	nút cổ chai thông tin
Negative log-likelihood (NLL)	log-hợp lý âm
Perplexity	độ rối (của language model)
Floating point	số dấu phẩy động
Mantissa, exponent	định trị, phần mũ
Overflow, underflow	tràn số (lên), tràn số (xuống 0)
Numerical stability	ổn định số
log-sum-exp	log-tổng-exp (thủ thuật tính softmax an toàn)
Catastrophic cancellation	mất số do trừ hai số gần nhau
Overparameterization	quá tham số hoá
Neural Tangent Kernel (NTK)	nhân tiếp tuyến thần kinh
Lazy training / feature learning	huấn luyện "lười" (gần tuyến tính hoá) / học đặc trưng
Mean-field theory	lý thuyết mean-field (trường trung bình)
Benign overfitting	khớp quá độ lành tính
Implicit regularization	chính quy hoá ngầm
Kronecker product	tích Kronecker
Natural gradient	gradient tự nhiên
Fisher information matrix	ma trận thông tin Fisher
Evidence lower bound (ELBO)	cận dưới chứng cớ
Optimal transport, Wasserstein distance	vận tải tối ưu, khoảng cách Wasserstein

---

10. Cách đọc paper toán, lý thuyết một cách phản biện

1. **Phân biệt ba loại câu:** điều paper *chứng minh* (có giả thiết rõ, có bước suy luận), điều paper *quan sát thực nghiệm* (đúng trên setup cụ thể, chưa chắc tổng quát), và điều *ta suy ra thêm*. Ghi chú trong thư mục này luôn tách rõ bằng "(tự tính)" và các cụm "paper nói" / "suy luận của người ghi chú".

2. **Đọc giả thiết trước khi đọc kết luận.** Một định lý ”mạng rất rộng thì NTK không đổi” chỉ đúng dưới giả thiết độ rộng tiến ra vô hạn và cách khởi tạo/scale cụ thể (mục 7.2-7.3); áp trực tiếp cho một mạng hữu hạn, scale khác, là ngoài phạm vi định lý.
3. **Luôn thử tính bằng số trước khi tin một bound.** Mục 4.5 cho thấy cùng một bất đẳng thức đúng về đại số có thể vô nghĩa (> 1) hoặc rất chặt, tuỳ n, ϵ. Cắm số cụ thể vào trước khi dùng một bound để ”chứng minh” điều gì trong thực hành.
4. **Phân biệt kết quả tồn tại (existence) và kết quả xây dựng được (constructive).** ”Tồn tại một hàm mà mạng sâu tính được nhưng mạng nông không tính được” (bài 20) không có nghĩa là mọi hàm thực tế đều như vậy.
5. **Chú ý thang đo của thực nghiệm minh hoạ.** Nhiều kết quả lý thuyết học sâu (mục 7) chứng minh trên mạng tuyến tính, mạng 2 lớp, hoặc giới hạn vô hạn chiều rộng; khoảng cách giữa đó và một transformer vài tỷ tham số có thể rất lớn, paper thường tự nêu rõ giới hạn này ở phần ”limitations” hoặc ”discussion”.
6. **Khi một paper là sách/khảo sát rất dài** (bài 12, 25, 28, 31), kiểm phạm vi đã đọc được ghi ở đầu bài ghi chú tương ứng (thường trong trường ”Vị trí trong lộ trình” hoặc mục 1) trước khi coi đó là tóm tắt toàn bộ cuốn sách.
7. **Theo dõi ký hiệu riêng của từng paper.** d có thể là số chiều ở một bài, là ”depth” (độ sâu) ở bài khác; σ có thể là độ lệch chuẩn hoặc một hàm activation. Không mang ký hiệu từ bài này sang bài khác mà không kiểm lại định nghĩa.

---

11. Lịch đọc gợi ý

Thư mục đã có `README.md` chia 35 bài thành 10 giai đoạn (giai đoạn 0 là preface này); lịch dưới đây dùng lại đúng nhóm đó, kèm ghi chú bài nào nặng và có thể lùi lại, kiểm bằng `ls` tên file thật.

Giai đoạn 1 (bài 01-05): công cụ tính toán và đại số. `01_2018_matrix_calculus.md`, `02_2015_automatic_differentiation_survey.md`, `03_2014_pca_tutorial.md`, `04_2009_randomized_matrix_decomposition.md`, `05_2007_spectral_clustering_tutorial.md`. Lý do nhóm: đây là công cụ tính toán dùng ngay trong code, không cần nền lý thuyết nào trước. Đọc đầy đủ, không bài nào nên bỏ qua.

Giai đoạn 2 (bài 06-09): hình học thông tin và suy luận biến phân. `06_2014_natural_gradient_insights.md`, `07_2018_information_geometry_intro.md`, `08_2016_variational_inference_review.md`, `09_2019_vae_introduction.md`. Bài 07 (hình học thông tin, đa tạp dually flat) là bài **nặng** nhất nhóm này về mức độ hình thức; nếu chỉ cần hiểu natural gradient để dùng, đọc kỹ bài 06 và lướt bài 07.

Giai đoạn 3 (bài 10-11): MCMC và SGD như suy luận Bayes. `10_2017_hamiltonian_monte_carlo.md`, `11_2017_sgd_approximate_bayesian_inference.md`. Có thể lùi lại nếu không làm việc trực tiếp với lấy mẫu Bayes hay uncertainty quantification.

Giai đoạn 4 (bài 12-14): vận tải tối ưu và thông tin. `12_2018_computational_optimal_transport.md` (monograph khoảng 250 trang, **nặng nhất của cả thư mục** về độ dài, lần đầu chỉ nên đọc phần Monge/Kantorovich và Sinkhorn), `13_2017_wasserstein_gan.md`, `14_2016_deep_variational_information_bottleneck.md`.

Giai đoạn 5 (bài 15-18): số học và ổn định số. `15_2018_online_softmax.md`, `16_2022_fp8_formats.md`, `17_2018_spectral_normalization_gan.md`, `18_2017_shattered_gradients.md`. Thực dụng, dùng ngay khi viết code; không bài nào nên bỏ qua.

Giai đoạn 6 (bài 19-27): biểu diễn, độ sâu, độ rộng, kernel, lan truyền tín hiệu. `19_2018_intrinsic_dimension.md`, `20_2016_benefits_of_depth.md`, `21_2017_expressive_power_width.md`, `22_2017_deep_nn_gaussian_processes.md`, `23_2018_ntk.md`, `24_2018_lazy_training.md`, `25_2021_`

`principles_dl_theory.md` (sách ~470 trang, chỉ đọc phần được chọn, xem ghi chú đầu bài), `26_2016_exponential_expressivity_chaos.md`, `27_2016_deep_information_propagation.md`. README đã ghi rõ: đây là **lý thuyết nặng nhất** của thư mục cùng với giai đoạn 8; lần đầu đọc, chỉ cần mục 1, 2, 11 (bối cảnh, ý tưởng chính, kết luận) của từng bài, dựa vào mục 7 ở trên để nắm trực giác, rồi quay lại đọc kỹ sau khi đã quen. Nếu phải chọn ưu tiên trong nhóm này: 23 (NTK) và 24 (lazy training) hữu ích nhất cho việc đọc các bài LLM theory hiện đại; 20, 21, 26, 27 có thể lùi lại nếu không làm việc trực tiếp với lý thuyết biểu diễn.

Giai đoạn 7 (bài 28-30): hình học, diffusion, thông tin trong mạng. `28_2021_geometric_deep_learning.md` (dài, chỉ đọc phạm vi đã chọn ở đầu bài), `29_2022_understanding_diffusion_models.md`, `30_2017_opening_black_box_information.md`. Bài 28 hữu ích nếu làm việc với kiến trúc có cấu trúc đối xứng (graph, mesh); bài 29 cần nếu đụng tới mô hình diffusion.

Giai đoạn 8 (bài 31-34): tổng quát hoá và overparameterization. `31_2021_deep_learning_statistical_viewpoint.md` (khảo sát, đọc phần tổng quan trước), `32_2019_benign_overfitting_linear_regression.md`, `33_2018_gradient_descent_global_minima_dnn.md`, `34_2018_mean_field_two_layer_networks.md`. Cùng mức nặng như giai đoạn 6 (nhiều chứng minh hình thức); đọc bài 31 trước để có bản đồ tổng quan, 3 bài còn lại là ví dụ cụ thể hoá, có thể chỉ đọc kết luận lần đầu.

Giai đoạn 9 (bài 35): bandit và quyết định tuần tự. `35_2017_tutorial_thompson_sampling.md`. Độc lập với các giai đoạn trước, có thể đọc bất cứ lúc nào, hữu ích trước khi đọc `rl content/` (bandit là RL đơn giản nhất).

Đường tắt cho người chỉ cần đọc để hiểu các thư mục chuyên đề khác (không cần lý thuyết sâu): preface này (mục 1-6, 9) → 01, 02 (matrix calculus, autodiff) → 03 (PCA) → 15, 16 (ổn định số, fp8) → đọc mục 7 ở trên thay cho 19-27, 31-34 → quay lại đọc chi tiết bài cụ thể chỉ khi một bài ở thư mục chuyên đề trích dẫn trực tiếp.

---

12. Liên hệ với dự án LightOnOCR-3-0.8B và các thư mục khác

Bối cảnh dự án (suy luận, không phải nội dung một paper nào): dự án đang nén và hậu huấn luyện LightOnOCR-3-0.8B, một VLM đọc OCR với decoder lai GatedDeltaNet cộng full attention, khoảng 3.600 token mỗi trang (khoảng 1.500 token văn bản cộng 1,6-2,1 nghìn token hình ảnh), chạy trên GPU tối đa 8 GB, batch 1. Các hướng pruning, distillation, RL, interpretability đã có thư mục riêng; mục này chỉ nối những công cụ toán ở trên với các hướng đó, không lặp lại nội dung của chúng.

Với đại số tuyến tính (mục 2): mọi kỹ thuật cắt tỉa có cấu trúc (structural pruning) đều là một bài toán xấp xỉ hạng thấp hoặc chọn không gian con trần trụi: cắt một lớp (`pruning content/` bài 31 `31_2024_shortgpt.md`, 33, 34, 35) hay cắt theo chiều ẩn (bài 29 `29_2024_slicegpt.md`) đều có thể đọc như "tìm một phép chiếu/một tập cột để bỏ mà sai số Frobenius nhỏ nhất" (đúng tinh thần Eckart-Young ở mục 2.6). Công thức least-squares với ridge ở mục 2.8 là chính xác phép tính D_{lin} đã dùng ở `interp content/00` mục 2.8 để vá lại activation sau khi cắt một layer.

Với số học máy tính (mục 6): GPU ≤ 8 GB bắt buộc cân nhắc fp16/bf16/fp8 khi huấn luyện và lượng tử hoá khi suy luận. Bài 16 (fp8 formats) trong thư mục này đã phân tích trực tiếp việc này; log-sum-exp (mục 6.3) là đúng cơ chế bên trong các kernel attention tiết kiệm bộ nhớ (`llm content/` bài 44, 45 FlashAttention) cần cho chuỗi dài khoảng 3.600 token/trang ở batch 1.

Với lý thuyết học sâu (mục 7): sau khi cắt từ 0.8B xuống một kích thước nhỏ hơn, mô hình con có thể rời khỏi vùng "rất overparameterized" so với dữ liệu phục hồi (KD). Trực giác NTK/lazy training (mục 7.2-7.3) gợi ý một câu hỏi suy luận: liệu giai đoạn phục hồi bằng KD có hoạt động gần chế độ "lazy" (chủ yếu nội suy, ít học đặc trưng mới) hay "feature learning" thật; nếu gần lazy, kỳ vọng phục hồi nhanh nhưng khó học hành

vi mới ngoài những gì teacher đã cho thấy ở layer tương ứng, một câu hỏi có thể kiểm bằng cách đo biểu diễn ẩn trước/sau KD (liên hệ CKA, `interp content/00` mục 7) thay đổi bao nhiêu.

Với thư mục `distillation content/`: chưng cất tri thức (bài 02 Hinton) vốn dùng trực tiếp cross-entropy và KL (mục 5) giữa phân phối softmax của teacher (có temperature) và student làm loss; mọi lựa chọn công thức KD (forward KL, reverse KL, các biến thể ở bài 31 `31_2024_rethinking_kl_llm_kd.md`) là lựa chọn **hướng** của cùng một công cụ ở mục 5.2, và tính không đối xứng của KL (mục 5.2) chính là lý do các lựa chọn này cho hành vi khác nhau (forward KL có xu hướng "cover" mọi mode của teacher, reverse KL có xu hướng tập trung vào mode chính, suy luận chung từ tính chất không đối xứng, không phải số đo trực tiếp từ bài này).

Với thư mục `pruning content/`: chẩn đoán độ nhạy của trọng số trước khi cắt (bài 01 Optimal Brain Damage, bài 07 Taylor importance, bài 22 Optimal Brain Compression, bài 23 SparseGPT, bài 24 Wanda, bài 25 ZipLM) đều quy về xấp xỉ Taylor bậc 2 bằng Hessian hoặc một dạng least-squares theo cột (đúng mục 2.8, 3.1): càng hiểu rõ vì sao cần λI khi nghịch đảo một ma trận gần suy biến (mục 2.6), càng dễ đọc vì sao các paper này phải thêm "damping" khi tính Hessian xấp xỉ trên một tập calibration nhỏ.

Với thư mục `rl content/`: nếu có một giai đoạn RLVR nhỏ sau KD (như `rl content/00` mục 9 đã suy luận), phạt KL về checkpoint sau KD dùng đúng công thức mục 5.2-5.3 ở đây; ước lượng KL trên mẫu (ba công thức k_1, k_2, k_3 nêu ở `rl content/00` mục 4.2) là một ứng dụng trực tiếp của cùng định nghĩa KL.

Với thư mục `ml basics content/` và `optimization content/`: cả hai dùng lại gần như toàn bộ mục 2, 3, 6 ở đây (ma trận, gradient, số học) nhưng áp vào kiến trúc/optimizer cụ thể; nên đọc hai thư mục đó tiếp theo thư mục này trước khi quay lại các thư mục chuyên đề, đúng thứ tự đã nêu ở mục 1.3.

Tổng kết (suy luận): trong số 35 bài, nhóm hữu ích trực tiếp nhất cho công việc nén/hậu huấn luyện LightOnOCR là 01, 02 (công cụ tính gradient), 03, 04 (hạng thấp, PCA), 15, 16 (ổn định số khi huấn luyện ở GPU nhỏ), và 23, 24 (NTK/lazy training, để suy luận về hành vi phục hồi sau cắt tỉa). Nhóm ít liên quan trực tiếp hơn (nhưng vẫn hữu ích nếu mở rộng dự án sang mô hình sinh hay quyết định tuần tự): 09, 12, 13, 29 (biến phân, vận tải tối ưu, GAN, diffusion, vì dự án hiện tại là OCR chứ không phải mô hình sinh ảnh) và 35 (Thompson sampling, chỉ liên quan nếu làm bandit để chọn hyperparameter RLVR).

---

13. Năm câu tự kiểm tra

Câu 1. Ma trận $\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ có trị riêng 2 và 6. Viết xấp xỉ hạng 1 tốt nhất của Σ theo chuẩn Frobenius, và tính sai số xấp xỉ.

Đáp án.

Theo định lý Eckart-Young (mục 2.6), xấp xỉ hạng 1 tốt nhất giữ lại giá trị kỳ dị (ở đây bằng trị riêng, vì Σ đối xứng bán xác định dương) lớn nhất, tức $\lambda = 6$ với vector riêng $v_2 = (1,1)/\sqrt{2}$: $\Sigma_1 = 6\, v_2 v_2^\top = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$.

Sai số Frobenius bằng giá trị kỳ dị bị bỏ: $\|\Sigma - \Sigma_1\|_F = 2$ (tự tính, khớp numpy).

Câu 2. Vì sao reverse-mode autodiff (backpropagation) hiệu quả hơn forward-mode cho việc huấn luyện mạng neural, trong khi cả hai đều tính đạo hàm chính xác?

Đáp án.

Loss của mạng neural là **một số scalar** (ít output), nhưng số tham số (input của phép tính gradient) có thể lên tới hàng tỷ. Forward-mode cần một lượt chạy riêng cho **mỗi** input (ở đây là mỗi tham số) để lấy đạo hàm theo input đó; với hàng tỷ tham số, điều này không khả thi. Reverse-mode cần đúng **một** lượt chạy xuôi và một

lượt chạy ngược để lấy gradient theo **toàn bộ** tham số cùng lúc (mục 3.3), vì nó lan truyền đạo hàm từ phía có ít output (1) về phía có nhiều input.

Câu 3. Một reward model hay bộ phân loại có "độ nhạy" (sensitivity) 99% khi phát hiện một hành vi hiếm (tỉ lệ cơ sở 1%), và tỉ lệ dương giả 5%. Nếu nó báo "phát hiện", xác suất hành vi đó thật sự xảy ra là bao nhiêu? Bài học rút ra cho việc đánh giá một bộ kiểm tra tự động là gì?

Đáp án.

Theo Bayes (mục 4.3), $P(\text{thật} \mid \text{báo phát hiện}) = \dfrac{0.01 \times 0.99}{0.01 \times 0.99 + 0.99 \times 0.05} = \dfrac{0.0099}{0.0594} \approx 0.167$ (khoảng 16,7%). Dù độ nhạy rất cao, xác suất đúng khi có báo động chỉ khoảng 1/6, vì hành vi hiếm (tỉ lệ cơ sở nhỏ) và vẫn có dương giả. Bài học: không thể chỉ nhìn độ nhạy của một bộ kiểm tra để tin vào kết quả nó báo, phải tính cả tỉ lệ cơ sở thật của thứ đang tìm.

Câu 4. Trong ví dụ backprop 2 lớp ở mục 3.4, vì sao $\partial L/\partial W_1$ có dòng đầu bằng 0 hoàn toàn? Nếu tình trạng này giữ nguyên qua nhiều bước cập nhật, hệ quả là gì?

Đáp án.

Vì đơn vị ẩn thứ nhất có thành phần tương ứng của z_1 bằng $-0.05 < 0$, nên thành phần tương ứng của $a_1 = \text{ReLU}(z_1)$ bằng 0 và đạo hàm của ReLU tại điểm âm bằng 0 tuyệt đối; gradient không thể "chảy qua" một cổng có đạo hàm 0, nên mọi gradient theo dòng tương ứng của W_1 và thành phần tương ứng của b_1 đều bị nhân với 0. Nếu tình trạng đó giữ nguyên qua nhiều bước (ReLU chết, mục 8.2), đơn vị đó đóng băng vĩnh viễn: không còn đóng góp gì vào việc học, tương đương lãng phí một phần công suất của mạng.

Câu 5. Vì sao cross-entropy loss khi huấn luyện language model lại đúng bằng negative log-likelihood (NLL) của token đúng, và vì sao hai thứ tưởng như khác nhau (một là "khoảng cách giữa phân phối", một là "log-hợp lý") lại là cùng một số?

Đáp án.

Nhãn thật ở một vị trí là một token cụ thể, nên phân phối "thật" p là one-hot: $p_{\text{đúng}} = 1$, còn lại 0, và $H(p) = 0$ (mục 5.1, không còn bất định vì đã biết chắc). Khi đó cross-entropy rút gọn: $H(p, q) = -\sum_i p_i \log q_i = -\log q_{\text{đúng}}$ (chỉ còn một số hạng khác 0), và vì $H(p) = 0$ nên $\text{KL}(p\|q) = H(p, q) - H(p) = H(p, q)$ cũng bằng đúng số đó. Mà $-\log q_{\text{đúng}}$ chính là định nghĩa của NLL. Ba cách nhìn (tối thiểu cross-entropy, tối thiểu KL tới nhãn thật, tối đa log-likelihood) trùng nhau chỉ vì nhãn là one-hot; với nhãn "mềm" (soft label, như trong distillation, mục 12) $H(p) \neq 0$ và ba cách nhìn **không còn trùng nhau tuyệt đối**, dù vẫn liên hệ qua đúng công thức $H(p, q) = \text{KL}(p\|q) + H(p)$.

---

14. Bảng tra công thức

Khái niệm	Công thức	Mục
Chuẩn Euclid	$\|x\|_2 = \sqrt{x^\top x}$	2.1
Trị riêng	$Av = \lambda v$	2.4
SVD	$A = U\Sigma_s V^\top$	2.5
Điều kiện số	$\kappa(A) = \sigma_{\max}/\sigma_{\min}$	2.6
Xấp xỉ hạng thấp (Eckart-Young)	$A_k = \sum_{i \le k} \sigma_i u_i v_i^\top$	2.6
Least squares có ridge	$W^\star = YX^\top(XX^\top + \lambda I)^{-1}$	2.8
Gradient, Jacobian, Hessian	∇f, $J_{ij} = \partial g_i/\partial x_j$, $H_{ij} = \partial^2 f/\partial x_i \partial x_j$	3.1
Chain rule (vector)	$\nabla_x y = J_g^\top \nabla_z y$	3.2
$\partial/\partial W$ của $\|Wx - y\|^2$	$2(Wx - y)x^\top$	3.2

Khái niệm	Công thức	Mục
Gaussian đa biến	$p(x) = \frac{1}{(2\pi)^{d/2}\lvert\Sigma\rvert^{1/2}} \exp(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu))$	4.2
Định lý Bayes	$P(A \mid B) = P(B \mid A)P(A)/P(B)$	4.3
MAP của Beta-Bernoulli	$\hat{p} = (\alpha - 1)/(\alpha + \beta - 2)$	4.4
Hoeffding	$P(\lvert\bar{X}_n - \mathbb{E}X\rvert > \epsilon) \le 2\exp(-2n\epsilon^2)$	4.5
Entropy	$H(p) = -\sum_i p_i \log p_i$	5.1
Cross-entropy, KL	$H(p,q) = -\sum p_i \log q_i$, $\mathrm{KL}(p\Vert q) = H(p,q) - H(p)$	5.2
NLL = cross-entropy khi p one-hot	$-\log q_{\text{đúng}}$	5.3
Mutual information	$I(X;Y) = \mathrm{KL}(p(x,y)\Vert p(x)p(y))$	5.4
log-sum-exp ổn định	$\log\sum_j e^{z_j} = m + \log\sum_j e^{z_j - m}$, $m = \max_j z_j$	6.3
NTK	$\Theta(x,x') = \nabla_\theta f(x;\theta_0)^\top \nabla_\theta f(x';\theta_0)$	7.2

Tài liệu này không có đường link gốc vì không ghi chú một paper cụ thể. Phần công cụ nên đọc kèm: Boyd và Vandenberghe, "Convex Optimization"; Deisenroth, Faisal và Ong, "Mathematics for Machine Learning"; Goodfellow, Bengio và Courville, "Deep Learning"; Murphy, "Probabilistic Machine Learning"; Strang, "Introduction to Linear Algebra". Bước tiếp theo: mở `01_2018_matrix_calculus.md`*.*

Bài 01

The Matrix Calculus You Need For Deep Learning (2018)

Tác giả: Terence Parr, Jeremy Howard (University of San Francisco, chương trình MS in Data Science; đồng thời liên quan tới fast.ai).
Venue / arXiv: arXiv:1802.01528 (v3, 02/07/2018); paper không nêu venue hội nghị/tạp chí (đây là tài liệu tutorial tự xuất bản, có bản HTML song song tạo từ cùng nguồn markup; trang đầu còn ghi tay ngày "July 3, 2018", lệch 1 ngày so với stamp arXiv v3 thật là 02/07/2018, chi tiết nhỏ không ảnh hưởng nội dung).
Nguồn: https://arxiv.org/abs/1802.01528
Chủ đề: dựng lại từ đầu phần vi tích phân ma trận (đạo hàm theo vector, Jacobian, 3 dạng quy tắc chuỗi) cần để hiểu backpropagation, rồi áp dụng trực tiếp để tính gradient của một neuron ReLU và của hàm loss MSE.
Vị trí trong lộ trình: đây là bài ĐẦU TIÊN trong batch math 01-09 (không có bài trước trong batch này). Nó dựng nền (đạo hàm vector, Jacobian, chain rule) mà bài tiếp theo, math 02 (automatic differentiation survey), dùng trực tiếp để định nghĩa chính xác forward-mode/reverse-mode AD. Đọc trước math 02.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Muốn huấn luyện một mạng neural, ta cần cực tiểu hoá một hàm loss bằng gradient descent, và điều đó đòi hỏi tính đạo hàm của loss theo mọi tham số (trọng số w, bias b). Các thư viện như PyTorch tính việc này tự động (automatic differentiation, xem bài sau math 02), nên về mặt thực hành người dùng không bắt buộc phải tự tay làm toán. Nhưng để đọc hiểu paper về deep learning, hoặc để debug một phép tính gradient tự viết, cần nắm được **matrix calculus**: phần giao giữa đại số tuyến tính (vector, matrix) và giải tích nhiều biến (đạo hàm riêng).

Vấn đề paper nêu ra: tài liệu online về matrix calculus thường là danh sách công thức khô khan, ít giải thích, hướng tới người đọc đã quen ký hiệu toán chặt. Mục tiêu của bài là **dựng lại từ đầu** một số quy tắc đạo hàm ma trận quan trọng nhất cho deep learning, chỉ cần kiến thức calculus 1 (đạo hàm hàm một biến) làm nền.

Các khái niệm cần nắm trước khi đọc (paper giả định người đọc đã quen mạng neural ở mức thực hành):

- **Đạo hàm (derivative)** $\frac{d}{dx}f(x)$: tốc độ thay đổi của f theo x.
- **Đạo hàm riêng (partial derivative)** $\frac{\partial}{\partial x}f(x,y)$: đạo hàm theo x, coi các biến khác là hằng số.
- **Gradient** $\nabla f(x,y) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$: vector (hàng, theo quy ước paper) gom tất cả đạo hàm riêng của MỘT hàm trả về scalar.
- **Jacobian**: khi có NHIỀU hàm trả về scalar (gộp thành vector hàm $\mathbf{y} = \mathbf{f}(\mathbf{x})$), Jacobian là ma trận gồm các gradient xếp thành hàng.

Bài toán cụ thể paper giải quyết tới cuối: tính $\frac{\partial C}{\partial w}$ và $\frac{\partial C}{\partial b}$ cho một neuron đơn `activation(x) = max(0, w · x + b)` (fully-connected + ReLU) và loss MSE, đây chính là bước cần cho một lần cập nhật gradient descent.

2. Ý tưởng chính

Paper không trình bày matrix calculus như một danh sách quy tắc cần nhớ, mà **dựng lại từng bước** bằng cách luôn quy một biểu thức vector/matrix phức tạp về các phương trình scalar tương đương, áp dụng quy tắc đạo hàm scalar quen thuộc, rồi "gom" lại thành dạng vector/matrix. Qua quá trình đó, tác giả định danh rõ **ba loại "chain rule"** hay bị lẫn với nhau trong các tài liệu khác:

1. **Single-variable chain rule**: $\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$, chỉ đúng khi có DUY NHẤT một đường truyền dữ liệu từ x tới y.
2. **Single-variable total-derivative chain rule**: khi một biến trung gian phụ thuộc x qua nhiều đường, phải CỘNG đóng góp của tất cả các đường: $\frac{\partial f(x,u_1,\ldots,u_n)}{\partial x} = \frac{\partial f}{\partial x} + \sum_i \frac{\partial f}{\partial u_i}\frac{\partial u_i}{\partial x}$.
3. **Vector chain rule**: $\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$ (tích hai Jacobian), và đây là dạng **tổng quát nhất**: hai dạng trên chỉ là trường hợp đặc biệt của nó. Thông điệp cốt lõi của paper: "chỉ cần nhớ vector chain rule, còn lại tự suy ra".

Áp dụng toàn bộ máy này cho một neuron `max(0, w·x + b)` và loss MSE cho ra công thức gradient cuối cùng (mục 3.6), đúng là công thức mọi người quen dùng khi lập trình gradient descent tay cho hồi quy tuyến tính/logistic.

3. Phương pháp chi tiết

3.1 Ôn lại quy tắc đạo hàm scalar (Mục 2)

Bảng các quy tắc cơ bản (Power rule $\frac{d}{dx}x^n = nx^{n-1}$, Sum/Difference rule, Product rule $\frac{d}{dx}fg = f\frac{dg}{dx} + \frac{df}{dx}g$, Chain rule $\frac{d}{dx}f(g(x)) = \frac{df(u)}{du}\frac{du}{dx}$ với $u = g(x)$). Paper nhấn mạnh nên coi $\frac{d}{dx}$ là một **toán tử** (map một hàm số thành hàm đạo hàm của nó), có tính phân phối và cho phép đưa hằng số ra ngoài, giúp đơn giản hoá biểu thức phức tạp bằng cách chia nhỏ.

3.2 Đạo hàm riêng và gradient (Mục 3)

Với $f(x, y)$, mỗi biến có một đạo hàm riêng ($\frac{\partial}{\partial x}$ coi y là hằng số, và ngược lại). Gom các đạo hàm riêng của MỘT hàm số scalar lại thành vector hàng gọi là **gradient**: $\nabla f(x, y) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$. Ví dụ paper: $f(x, y) = 3x^2y \Rightarrow \nabla f = [6xy, 3x^2]$.

3.3 Jacobian (Mục 4, 4.1)

Khi có nhiều hàm scalar cùng nhận các tham số đó, xếp các gradient thành HÀNG của một ma trận, ta được **Jacobian**. Với $\mathbf{y} = \mathbf{f}(\mathbf{x})$, $\mathbf{f}$ gồm m hàm $f_1, \ldots, f_m$, $\mathbf{x}$ có n phần tử, Jacobian là ma trận $m \times n$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \vdots \\ \nabla f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

Paper dùng **numerator layout** (hàng = hàm, cột = biến); nhiều tài liệu khác dùng denominator layout, chỉ là Jacobian transpose. Ví dụ minh hoạ paper đưa: Jacobian của hàm đồng nhất $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ là ma trận đơn vị I (chứng minh bằng cách tính từng phần tử: $\frac{\partial}{\partial x_j} x_i = 0$ nếu $j \neq i$, $= 1$ nếu $j = i$).

3.4 Phép toán element-wise và "diagonal condition" (Mục 4.2, 4.3, 8.2, 8.3)

Với phép toán element-wise $\mathbf{y} = \mathbf{f}(\mathbf{w}) \odot \mathbf{g}(\mathbf{x})$ (ví dụ $\mathbf{w} + \mathbf{x}$, $\mathbf{w} - \mathbf{x}$, tích Hadamard $\mathbf{w} \otimes \mathbf{x}$, chia element-wise), nếu f_i chỉ phụ thuộc w_i và g_i chỉ phụ thuộc x_i ("element-wise diagonal condition"), Jacobian suy biến thành **ma trận đường chéo**:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \text{diag}\left(\ldots, \frac{\partial}{\partial w_i}(f_i(w_i)g_i(x_i)), \ldots\right)$$

Bảng kết quả cụ thể (paper, Mục 8.2): $\frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{w}} = I$; $\frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{w}} = I$, $\frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{x}} = -I$; $\frac{\partial(\mathbf{w}\otimes\mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\mathbf{x})$, $\frac{\partial(\mathbf{w}\otimes\mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\mathbf{w})$. Mở rộng tự nhiên là **scalar expansion** ($\mathbf{y} = \mathbf{x} + z$ hay $\mathbf{y} = \mathbf{x}z$ với z scalar): $\frac{\partial(\mathbf{x}+z)}{\partial \mathbf{x}} = I$, $\frac{\partial(\mathbf{x}+z)}{\partial z} = \mathbf{1}$ (vector cột toàn 1); $\frac{\partial(\mathbf{x}z)}{\partial \mathbf{x}} = Iz$, $\frac{\partial(\mathbf{x}z)}{\partial z} = \mathbf{x}$.

3.5 Vector sum reduction và đạo hàm dot product (Mục 4.4, 8.4)

Với $y = \text{sum}(\mathbf{f}(\mathbf{x})) = \sum_i f_i(\mathbf{x})$, gradient là $\nabla y = \left[\sum_i \frac{\partial f_i}{\partial x_1}, \ldots, \sum_i \frac{\partial f_i}{\partial x_n}\right]$. Trường hợp riêng quan trọng nhất cho deep learning: **đạo hàm của dot product** $y = \mathbf{w} \cdot \mathbf{x} = \text{sum}(\mathbf{w} \otimes \mathbf{x})$. Đặt biến trung gian $\mathbf{u} = \mathbf{w} \otimes \mathbf{x}$, dùng vector chain rule (xem 3.6):

$$\frac{\partial \mathbf{u}}{\partial \mathbf{w}} = \text{diag}(\mathbf{x}), \qquad \frac{\partial y}{\partial \mathbf{u}} = \mathbf{1}^T \qquad \Rightarrow \qquad \frac{\partial y}{\partial \mathbf{w}} = \mathbf{1}^T \,\text{diag}(\mathbf{x}) = \mathbf{x}^T, \quad \text{tương tự } \frac{\partial y}{\partial \mathbf{x}} = \mathbf{w}^T$$

Paper kiểm lại bằng cách tính trực tiếp từng phần tử scalar ($\frac{\partial}{\partial w_j} \sum_i w_i x_i = x_j$) và khớp với kết quả trên.

3.6 Ba loại chain rule (Mục 4.5)

(a) Single-variable chain rule. Với $y = f(g(x))$, đặt $u = g(x)$: $\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$. Chỉ áp dụng khi x ảnh hưởng tới y qua **đúng một** đường dữ liệu (data-flow path). Ví dụ paper: $y = \sin(x^2)$, đặt $u = x^2$: $\frac{dy}{dx} = \cos(u) \cdot 2x = 2x\cos(x^2)$.

(b) Single-variable total-derivative chain rule. Khi biểu thức có biến trung gian phụ thuộc x qua NHIỀU đường (ví dụ $y = x + x^2$, viết lại $u_1 = x^2$, $u_2(x, u_1) = x + u_1$), áp dụng trực tiếp single-variable chain rule sẽ SAI (paper minh hoạ: nếu "mù quáng" lấy $\frac{du_2}{du_1}\frac{du_1}{dx} = 1 \times 2x = 2x$ thì thiếu hẳn đường trực tiếp $x \to y$, kết quả đúng phải là $1 + 2x$). Công thức tổng quát (total derivative, "luật tổng các đạo hàm"):

$$\frac{\partial f(x, u_1, \ldots, u_n)}{\partial x} = \frac{\partial f}{\partial x} + \sum_{i=1}^{n} \frac{\partial f}{\partial u_i}\frac{\partial u_i}{\partial x}$$

Paper kiểm bằng số: với $x = 1$ (nên $y = 1 + 1^2 = 2$), tăng $\Delta x = 1$ ($\hat{y} = 2 + 2^2 = 6$), $\Delta y = 6 - 2 = 4$ khớp với công thức đúng $1 + 2x = 1 + 2 = 3$... paper thực ra dùng ví dụ số để chỉ ra đạo hàm **đúng** phải lớn hơn giá trị sai $2x = 2$ (ở $x = 1$), còn $\Delta y/\Delta x = 4$ là xấp xỉ sai phân hữu hạn (không bằng đúng đạo hàm giải tích $1 + 2x = 3$ tại điểm này, chỉ minh hoạ hướng đúng/sai, paper không khẳng định hai số này phải bằng nhau).

(c) Vector chain rule. Tổng quát hoá (b) sang vector: với $\mathbf{y} = \mathbf{f}(\mathbf{g}(\mathbf{x}))$,

$$\boxed{\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}}$$

là **tích hai Jacobian** (Jacobian $m \times k$ nhân Jacobian $k \times n$ ra Jacobian $m \times n$, với $m = |\mathbf{f}|, k = |\mathbf{g}|, n = |\mathbf{x}|$; thứ tự nhân quan trọng vì nhân ma trận không giao hoán). Paper chứng minh bằng ví dụ 2 hàm $f_1 = \ln(g_1)$, $f_2 = \sin(g_2)$ với $g_1 = x^2, g_2 = 3x$: tính tay từng phần tử theo quy tắc (b) rồi gom lại khớp đúng với công thức tích Jacobian. Khi các hàm thoả "diagonal condition" (mỗi f_i chỉ phụ thuộc g_i, mỗi g_i chỉ phụ thuộc x_i), vector chain rule suy biến thành diag $\left(\frac{\partial f_i}{\partial g_i}\frac{\partial g_i}{\partial x_i}\right)$, tức chain rule scalar áp từng phần tử.

Ba dạng chain rule này không phải ba công cụ tách biệt: (a) và (b) chỉ là trường hợp đặc biệt của (c) khi các đại lượng là scalar.

Ví dụ số "tự tính" minh hoạ (a): tại $x = 1$, công thức $\frac{dy}{dx} = 2x\cos(x^2)$ cho $\frac{dy}{dx}|_{x=1} = 2 \times 1 \times \cos(1) \approx 2 \times 0{,}5403 = 1{,}0806$ (tự tính; paper chỉ đưa công thức ký hiệu, không tính giá trị số cụ thể này).

3.7 Gradient của một neuron ReLU (Mục 5)

Với $z(\mathbf{w}, b, \mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ và activation$(\mathbf{x}) = \max(0, z)$:

$$\frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}^T, \qquad \frac{\partial z}{\partial b} = 1$$

Đạo hàm của $\max(0, z)$ là hàm từng khúc: $\frac{\partial}{\partial z}\max(0,z) = 0$ nếu $z \le 0$, $= 1$ nếu $z > 0$. Dùng vector chain rule với biến trung gian z:

$$\frac{\partial \,\text{activation}}{\partial \mathbf{w}} = \begin{cases} \mathbf{0}^T & z \le 0 \\ \mathbf{x}^T & z > 0 \end{cases}, \qquad \frac{\partial \,\text{activation}}{\partial b} = \begin{cases} 0 & z \le 0 \\ 1 & z > 0 \end{cases}$$

3.8 Gradient của loss MSE theo $\mathbf{w}$ và b (Mục 6)

Với N mẫu, $C(\mathbf{w}, b, X, \mathbf{y}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \max(0, \mathbf{w}\cdot\mathbf{x}_i + b))^2$. Đặt $u =$ activation, $v = y - u$, $C = \frac{1}{N}\sum v^2$, áp dụng chain rule liên tiếp ($\frac{\partial C}{\partial v} = \frac{2}{N}\sum v$, rồi $\frac{\partial v}{\partial u} = -1$, rồi $\frac{\partial u}{\partial \mathbf{w}}$ như ở 3.7), đặt $e_i = \mathbf{w}\cdot\mathbf{x}_i + b - y_i$ (lỗi, dấu ngược với $v_i = y_i - u_i$):

$$\boxed{\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N}\sum_{i=1}^{N} e_i\,\mathbf{x}_i^T \;\;(\text{khi } z_i > 0), \qquad \frac{\partial C}{\partial b} = \frac{2}{N}\sum_{i=1}^{N} e_i \;\;(\text{khi } z_i > 0)}$$

(cả hai bằng $\mathbf{0}$/0 khi $z_i \le 0$ vì ReLU chặn gradient). Gradient là **trung bình có trọng số theo lỗi**: mẫu có lỗi lớn kéo gradient mạnh hơn theo hướng $\mathbf{x}_i$. Cập nhật gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\frac{\partial C}{\partial \mathbf{w}}$, $b_{t+1} = b_t - \eta\frac{\partial C}{\partial b}$.

Ví dụ số "tự tính" minh hoạ công thức trên (paper không có ví dụ số kiểu này, đây là của người ghi chú để kiểm tra công thức): cho $N = 1$, $\mathbf{w} = [1, 2]$, $\mathbf{x} = [3, 4]$, $b = -5$, mục tiêu $y = 4$. Tính $z = \mathbf{w}\cdot\mathbf{x} + b = 1\times 3 + 2\times 4 - 5 = 3 + 8 - 5 = 6 > 0$, nên activation $= 6$ và $e = z - y = 6 - 4 = 2$. Áp công thức: $\frac{\partial C}{\partial \mathbf{w}} = 2 \times e \times \mathbf{x}^T = 2 \times 2 \times [3, 4] = [12, 16]$ (tự tính) và $\frac{\partial C}{\partial b} = 2 \times e = 4$ (tự tính). Với $\eta = 0{,}01$: $\mathbf{w}_{t+1} = [1, 2] - 0{,}01 \times [12, 16] = [0{,}88,\ 1{,}84]$, $b_{t+1} = -5 - 0{,}01 \times 4 = -5{,}04$ (tự tính), đúng hướng giảm z (và do đó giảm e^2) vì $e > 0$.

3.9 Kết hợp $\mathbf{w}$ và b

Paper gợi ý thủ thuật thực hành: ghép $\hat{\mathbf{w}} = [\mathbf{w}^T, b]^T$ và thêm 1 vào cuối $\mathbf{x}$: $\hat{\mathbf{x}} = [\mathbf{x}^T, 1]^T$, khi đó $\mathbf{w}\cdot\mathbf{x} + b = \hat{\mathbf{w}}\cdot\hat{\mathbf{x}}$, chỉ cần tính MỘT gradient thay vì hai (đây chính là "augmented weight vector", một mẹo thường thấy khi lập trình hồi quy tuyến tính).

4. Thí nghiệm và kết quả chính

Đây là paper tutorial về toán, **không có thí nghiệm/benchmark thực nghiệm** theo nghĩa huấn luyện mô hình hay so sánh số liệu trên dữ liệu thật. "Kết quả chính" của bài là các công thức/định danh được dựng lại từ đầu và kiểm chứng chéo (bằng cách tính cả theo đường vector và theo đường quy về scalar, rồi so khớp):

Kết quả	Công thức
Jacobian hàm đồng nhất	$\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = I$
Jacobian phép element-wise (diagonal condition)	$\text{diag}(\dots)$
Đạo hàm dot product	$\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{w}} = \mathbf{x}^T$, $\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{x}} = \mathbf{w}^T$
Vector chain rule	$\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$
Gradient neuron ReLU + MSE	$\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N}\sum e_i\mathbf{x}_i^T$

Mỗi công thức đều được tác giả **kiểm hai lần**: một lần dựng bằng máy vector/matrix, một lần quy hoàn toàn về các phương trình scalar rồi lấy đạo hàm từng phần tử, và hai kết quả khớp nhau (ví dụ rõ nhất ở Mục 4.5.3 cho vector chain rule, và ở Mục 5 cho đạo hàm dot product).

5. Phân tích / ablation

- **Numerator layout vs denominator layout:** paper chọn numerator layout (hàng Jacobian = hàm, cột = biến); nhiều tài liệu/bài báo khác dùng denominator layout (transpose của numerator). Đây không phải một ”lựa chọn trung lập”; người đọc papers khác cần luôn kiểm layout trước khi so khớp công thức, nếu không shape sẽ không khớp.
- **Điều kiện áp dụng từng loại chain rule:** paper nhấn mạnh sai lầm phổ biến là áp dụng single-variable chain rule (a) khi thực ra cần total-derivative chain rule (b) vì có nhiều đường dữ liệu (ví dụ $y = x + x^2$). Việc luôn luôn dùng vector chain rule (c) tránh được lỗi này vì nó tự động ”cộng gộp” mọi đường qua phép nhân Jacobian.
- **Diagonal condition là chìa khoá tối ưu hoá tính toán:** phần lớn các phép toán trong neuron (element-wise +, -, $\otimes$) có Jacobian là ma trận đường chéo thưa, không cần lưu toàn bộ ma trận $n \times n$ dày; đây chính là lý do các framework autodiff không bao giờ vật lý hoá Jacobian đầy đủ mà làm việc với vector-Jacobian product (liên hệ bài sau, math 02).
- **Trường hợp ReLU bị ”tắt” ($z \leq 0$):** gradient của cả activation và loss triệt tiêu hoàn toàn. Đây chính là nguồn gốc hiện tượng ”dying ReLU” quen biết trong thực hành (không được paper này đặt tên, nhưng là hệ quả toán học trực tiếp của công thức đạo hàm từng khúc ở Mục 5).

6. Giới hạn

Do tác giả nêu:

- Phạm vi chỉ là đạo hàm theo **vector**, không phải theo **ma trận** đầy đủ (ví dụ đạo hàm của loss theo toàn bộ ma trận trọng số một layer, không phải theo một vector **w** của một neuron); tác giả nói rõ ở Mục 7 (Summary) đây là ”bước tiếp theo” để người đọc tự học thêm, không nằm trong bài.
- Automatic differentiation (forward/backward mode cho cả một đồ thị tính toán lớn) được coi là ”beyond the scope of this article, but we’re setting the stage for a future article” (Mục 4.5.1): tức paper chỉ dừng ở quy tắc đạo hàm toán học tay, không trình bày thuật toán AD thực thụ (đó là nội dung bài 02).
- Ví dụ xuyên suốt chỉ là MỘT neuron fully-connected + ReLU + MSE, chưa mở rộng cho nhiều layer, hàm activation khác, hay loss khác (ví dụ cross-entropy/softmax).

Người ghi chú nhận xét:

- Lối trình bày ”quy về scalar rồi suy ra công thức vector” rất dễ hiểu cho người mới nhưng **không phải chứng minh toán học chặt** theo chuẩn mực của một bài báo lý thuyết; mục tiêu là sư phạm (pedagogical), không phải rigor.
- Vì chỉ xử lý một neuron/một layer, công thức Mục 6 không trực tiếp cho thấy vì sao backpropagation qua nhiều layer lại hiệu quả hơn forward-mode AD (đó là lý do thực sự reverse-mode được ưa dùng trong deep learning, chỉ được nêu sơ lược ở cuối Mục 4.5.1 và sẽ được bài 02 làm rõ).
- Có một điểm không nhất quán nhỏ về ngày tháng đã nêu ở header (trang đầu ”July 3, 2018” so với stamp arXiv ”02/07/2018”), không ảnh hưởng nội dung toán.

7. Liên hệ với các bài khác

- **math 02** (Automatic Differentiation in Machine Learning: a Survey): dùng trực tiếp vector chain rule/Jacobian của bài này để định nghĩa chính xác forward-mode (tương ứng $\frac{du}{dx}\frac{dy}{du}$, nhân ”từ trái”, theo hướng $x \to y$) và reverse-mode (tương ứng $\frac{dy}{du}\frac{du}{dx}$, nhân ”từ phải”, theo hướng $y \to x$), paper 01 đã gợi mở chính hai hướng nhân này ở cuối Mục 4.5.1 nhưng không đi sâu.
- **math 23 (Neural Tangent Kernel), math 24 (Lazy Training):** cả hai phân tích mạng neural qua tuyến tính hoá bậc nhất (Jacobian của hàm mạng theo tham số tại điểm khởi tạo), chính là Jacobian $\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ tổng quát hoá từ công thức neuron đơn ở bài này.

- **pruning content/07_2019_importance_estimation_taylor.md:** chấm "độ quan trọng" của một tham số/kênh khi cắt tỉa bằng khai triển Taylor bậc 1 (dùng gradient) hoặc bậc 2 (dùng tích gradient, xấp xỉ Hessian) của loss; đây chính là ứng dụng trực tiếp của máy đạo hàm dựng ở bài này cho một mục đích khác (ước lượng thay đổi loss khi xoá một tham số) thay vì cập nhật gradient descent.
- **interp content/02_2021_mathematical_framework.md** (A Mathematical Framework for Transformer Circuits): phân tích attention bằng cách phân rã các phép nhân ma trận tuyến tính thành tích các Jacobian/ma trận con, cùng tinh thần "theo dõi shape và phép nhân ma trận cẩn thận" mà bài 01 rèn luyện.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 text + ~1.6-2.1K token visual, chạy trên GPU $\leq$ 8GB, batch 1). Bài này là toán nền, không phải kỹ thuật compress cụ thể, nhưng vẫn có vài điểm hữu ích trực tiếp (suy luận):

1. **Đọc/debug code backward tự viết.** Nếu cần viết custom autograd Function (ví dụ cho một biến thể attention sparsity hay một lớp lượng tử hoá tuỳ biến trong pipeline distill GatedDeltaNet), việc nắm chắc vector chain rule và quy tắc "diagonal condition" (3.4) giúp suy ra đúng shape và công thức backward thủ công, tránh lỗi về layout (numerator/denominator) hay quên cộng đóng góp qua nhiều đường (total-derivative chain rule).
2. **Hiểu vì sao reverse-mode (backprop) hợp lý cho mạng có nhiều tham số, ít output** (một gợi ý ở cuối Mục 4.5.1 của bài, sẽ được math 02 làm rõ định lượng): mô hình OCR có hàng trăm triệu tham số nhưng loss là một con số duy nhất mỗi batch, đúng khớp điều kiện mà bài này chỉ ra là lợi thế của reverse-mode.
3. **Phân tích "điểm chết" (dying ReLU-like) khi cắt tỉa:** công thức 3.7-3.8 cho thấy gradient triệt tiêu hoàn toàn khi kích hoạt bị "tắt" ($z \leq 0$ cho ReLU); khi đánh giá độ nhạy của một kênh/head trước khi cắt, cần nhớ rằng gradient bằng 0 không có nghĩa kênh đó vô dụng, mà có thể do activation đang nằm trong vùng bão hoà của hàm phi tuyến tại batch hiện tại.

Phần không liên quan trực tiếp: công thức cụ thể cho neuron ReLU/MSE không áp dụng thẳng cho attention hay GatedDeltaNet (các kiến trúc này cần Jacobian của các phép toán phức tạp hơn nhiều: softmax, phép đệ quy tuyến tính...), nhưng đây đúng là lý do các framework dùng autodiff (math 02) thay vì suy ra công thức tay cho từng kiến trúc mới.

9. Thí nghiệm gợi ý

1. **Tự viết backward cho một neuron ReLU + MSE (như Mục 5-6 của paper) bằng NumPy, so với `torch.autograd`:** tạo $\mathbf{w}, b, \mathbf{x}, y$ ngẫu nhiên nhỏ, tính `activation` và `loss` tay, tính $\frac{\partial C}{\partial \mathbf{w}}, \frac{\partial C}{\partial b}$ theo công thức 3.8, so với gradient PyTorch tính qua `loss.backward()`. Kỳ vọng: sai số tuyệt đối ở mức 10^{-6} hoặc nhỏ hơn (float32); nếu lệch nhiều, kiểm tra lại dấu hoặc điều kiện ReLU ($z \leq 0$).
2. **Kiểm tra numerator vs denominator layout bằng code:** tính Jacobian của một hàm vector nhỏ $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^2$ bằng `torch.autograd.functional.jacobian`, so sánh shape trả về với quy ước numerator layout của bài; diễn giải tại sao cần transpose nếu dùng quy ước khác.
3. **Minh hoạ "diagonal condition":** lấy một phép toán element-wise (ví dụ tanh áp lên vector) qua `torch.autograd.functional.jacobian`, in toàn bộ ma trận Jacobian $n \times n$ và kiểm tra trực quan nó đúng là ma trận đường chéo (các phần tử ngoài đường chéo bằng 0 trong giới hạn sai số số học).
4. **So sánh chi phí reverse-mode vs forward-mode tay:** với một hàm tổng hợp nhiều lớp phi tuyến ($f_4(f_3(f_2(f_1(x))))$ như ví dụ Mục 4.5.1 của paper), đo thời gian tính gradient bằng `torch.autograd.functional.jvp` (forward-mode, kiểu hình minh hoạ "forward differentiation" của bài) so với `vjp`/`backward` (reverse-mode) khi tăng số chiều input, giữ output là scalar; kỳ vọng reverse-mode

nhanh hơn rõ rệt khi input lớn, khớp với trực giác bài 01 đã gợi mở.

5. **Khảo sát vùng "chết" của ReLU trên mô hình nhỏ thật:** lấy 1 layer Linear+ReLU nhỏ của một mô hình toy, chạy forward trên một batch dữ liệu, đo tỉ lệ neuron có $z \leq 0$ (gradient = 0 theo công thức 3.7) và quan sát tỉ lệ này thay đổi thế nào khi khởi tạo trọng số khác đi.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Gradient	$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right]$
Jacobian (numerator layout)	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$, hàng $i = \nabla f_i(\mathbf{x})$
Element-wise diagonal condition	$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \text{diag}\left(\frac{\partial}{\partial w_i}(f_i(w_i) \odot g_i(x_i))\right)$
Đạo hàm dot product	$\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{w}} = \mathbf{x}^T$, $\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{x}} = \mathbf{w}^T$
Single-variable chain rule	$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$ (1 đường duy nhất)
Total-derivative chain rule	$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial x} + \sum_i \frac{\partial f}{\partial u_i}\frac{\partial u_i}{\partial x}$
Vector chain rule (tổng quát nhất)	$\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$
Gradient neuron ReLU	$\frac{\partial \text{ activation}}{\partial \mathbf{w}} = \mathbf{x}^T$ nếu $z > 0$, $\mathbf{0}^T$ nếu $z \leq 0$
Gradient loss MSE (ReLU)	$\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N}\sum_i e_i \mathbf{x}_i^T$, $e_i = \mathbf{w}\cdot\mathbf{x}_i + b - y_i$
Cập nhật gradient descent	$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\frac{\partial C}{\partial \mathbf{w}}$

11. Kết luận ngắn

Bài này dựng lại từ đầu, bằng cách quy mọi biểu thức vector/matrix về các phương trình scalar rồi gom lại, các công cụ đạo hàm cần cho deep learning: gradient, Jacobian (numerator layout), điều kiện để Jacobian suy biến thành ma trận đường chéo, và ba dạng chain rule mà tất cả đều chỉ là trường hợp đặc biệt của **vector chain rule** $\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$. Áp dụng toàn bộ máy đó cho một neuron ReLU và loss MSE cho ra đúng công thức gradient descent quen thuộc. Đây là nền toán thuần tuý (không thí nghiệm, không benchmark), chuẩn bị trực tiếp cho bài sau (math 02) trình bày automatic differentiation như một thuật toán tổng quát hoá chính quy tắc chuỗi này cho một đồ thị tính toán lớn bất kỳ, thay vì tính tay cho từng kiến trúc.

Nguồn gốc: https://arxiv.org/abs/1802.01528

Bài 02

Automatic Differentiation in Machine Learning: a Survey (2015)

Tác giả: Atılım Güneş Baydin (University of Oxford), Barak A. Pearlmutter (National University of Ireland Maynooth), Alexey Andreyevich Radul (Massachusetts Institute of Technology), Jeffrey Mark Siskind (Purdue University).
Venue / arXiv: arXiv:1502.05767 (v4, 05/02/2018); paper không nêu venue hội nghị/tạp chí trong văn bản (bản arXiv này không có dòng tạp chí; đã kiểm kỹ toàn văn, không có self-citation venue).
Nguồn: https://arxiv.org/abs/1502.05767
Chủ đề: khảo sát automatic differentiation (AD) từ góc nhìn machine learning: phân biệt AD với đạo hàm tay/số/ký hiệu, hai chế độ forward và reverse (reverse mode chính là tổng quát hoá của backpropagation), ứng dụng trong optimization/neural network/computer vision/NLP/suy luận xác suất, và các cách cài đặt (elemental library, source transformation, operator overloading).
Vị trí trong lộ trình: đọc sau math 01 (Matrix Calculus), dùng trực tiếp vector chain rule/Jacobian của bài đó để định nghĩa chính xác forward-mode và reverse-mode AD; đọc trước math 03.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Trước AD, có 3 cách tính đạo hàm trong code: (1) **tay** (derive công thức rồi code cứng), tốn thời gian và dễ sai; (2) **numerical differentiation** (sai phân hữu hạn), đơn giản nhưng kém chính xác và tốn $O(n)$ lần gọi hàm cho gradient n chiều; (3) **symbolic differentiation** (biến đổi biểu thức bằng hệ thống như Mathematica/Maple), cho kết quả đúng về mặt ký hiệu nhưng dễ bị ”expression swell” (biểu thức đạo hàm phình to theo cấp số nhân qua mỗi lần áp quy tắc tích). AD là lựa chọn thứ (4), bài này khảo sát AD từ góc nhìn machine learning.

Lịch sử ngắn (Mục 3.3): ý tưởng AD có từ thập niên 1950; forward mode được Wengert (1964) phát hiện như một phương pháp tổng quát; reverse mode có nguồn gốc từ Pontryagin maximum principle (lý thuyết điều khiển, cuối 1950), được Linnainmaa (1970, 1976) công bố sớm, Werbos (1974) hình thức hoá, và Speelpenning (1980) đưa ra cài đặt **tự động hoàn toàn** đầu tiên. Trong machine learning, backpropagation (trường hợp đặc biệt của reverse-mode AD) bị ”phát hiện lại” nhiều lần độc lập (Parker 1985, Werbos, nhóm Parallel Distributed Processing) trước khi Rumelhart và đồng nghiệp (1986) làm nó nổi tiếng. Paper nhấn mạnh: hiểu toán học sau backprop là đủ nền để hiểu AD nói chung, vì backprop **chỉ là một trường hợp** của reverse-mode AD áp cho hàm mạng neural ghép với loss.

Một nguồn nhầm lẫn paper muốn giải quyết: nhiều người gọi bất cứ thứ gì không phải đạo hàm tay là ”automatic differentiation”, trong khi AD là một **kỹ thuật cụ thể**: tính giá trị số của đạo hàm (không phải biểu thức đạo hàm) bằng cách ”tích luỹ” (accumulate) giá trị trong lúc chạy code, khác hẳn cả numerical diff (xấp xỉ) và symbolic diff (biến đổi biểu thức).

2. Ý tưởng chính

AD coi mọi phép tính số là tổ hợp của một **tập hữu hạn các phép toán sơ cấp** (cộng, nhân, hàm lượng giác, log, exp...) có đạo hàm đã biết. AD thực hiện một **”non-standard interpretation”** của chương trình: chạy chương trình như bình thường nhưng tại mỗi phép toán sơ cấp, đồng thời áp dụng chain rule để lan truyền thêm một giá trị đạo hàm, dựa trên **evaluation trace** (còn gọi là Wengert list) ghi lại chính xác những phép toán nào đã thực thi với giá trị cụ thể nào.

Vì làm việc trên **evaluation trace cụ thể** (không phải biểu thức closed-form trừu tượng), AD có 2 lợi thế lớn: (a) **chính xác tới độ chính xác máy** (không phải xấp xỉ như numerical diff) chỉ với **chi phí hằng số** so với hàm gốc (không bị expression swell như symbolic diff); (b) **hoạt động với code có control flow** (if, loop, recursion) vì bất kể đường thực thi nào được chọn, kết quả cuối vẫn chỉ là một trace số cụ thể để chain rule áp lên, điều symbolic differentiation không làm được tốt (phải biết trước biểu thức đóng).

Có **hai chế độ** AD, khác nhau về **hướng** áp chain rule: **forward mode** (lan truyền đạo hàm theo cùng hướng dữ liệu, từ input ra output) và **reverse mode** (lan truyền ngược, từ output về input); reverse mode chính là backpropagation tổng quát hoá cho MỌI chương trình số, không chỉ cho mạng neural.

3. Phương pháp chi tiết

3.1 AD không phải numerical hay symbolic differentiation (Mục 2)

Numerical differentiation dùng xấp xỉ giới hạn:

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h} \tag{1}$$

với $\mathbf{e}_i$ là vector đơn vị thứ i, $h > 0$ nhỏ. Có hai loại lỗi đối nghịch nhau khi chọn h: **truncation error** (lỗi do h không thực sự là 0, giảm khi $h \to 0$) và **round-off error** (lỗi do giới hạn độ chính xác dấu phẩy động, tăng khi h quá nhỏ); Hình 3 của paper minh họa hai đường lỗi này cắt nhau. Dạng center difference giảm bớt lỗi bậc nhất:

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h} + O(h^2) \tag{2}$$

nhưng vẫn cần $O(n)$ lần gọi hàm cho gradient n chiều (Jacobian $m \times n$ cần $2mn$ lần gọi), không khả thi khi n lên tới hàng triệu/tỷ tham số như deep learning hiện đại.

Symbolic differentiation áp quy tắc biến đổi biểu thức (ví dụ quy tắc tích, công thức 3) một cách máy móc. Vấn đề: nếu $h(x) = f(x)g(x)$ có phần chung giữa h và h' (là f, g), việc thế trực tiếp đạo hàm con vào công thức sẽ **nhân đôi lặp lại** các biểu thức con, dẫn tới "expression swell". Paper dùng ví dụ logistic map $l_{n+1} = 4l_n(1 - l_n)$ (Bảng 1): đạo hàm của l_4 theo x nếu không rút gọn có độ dài tăng vượt xa cả hàm gốc qua mỗi bước lặp.

3.2 Evaluation trace / Wengert list (Mục 3)

AD biểu diễn một hàm $f : \mathbb{R}^n \to \mathbb{R}^m$ bằng ba nhóm biến trung gian theo ký hiệu Griewank & Walther: $v_{i-n} = x_i$ ($i = 1, \ldots, n$, biến input), v_i ($i = 1, \ldots, l$, biến trung gian), $y_{m-i} = v_{l-i}$ (biến output). Ví dụ xuyên suốt bài (Hình 4, Bảng 2-3): $y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$.

3.3 Forward mode (Mục 3.1)

Gắn mỗi biến trung gian v_i một "tangent" $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$ (đạo hàm theo MỘT input cố định), rồi chạy chain rule trên từng phép toán sơ cấp để tạo "tangent trace" đi **song song, cùng hướng** với trace gốc (primal trace).

Ví dụ paper (Bảng 2), tại $(x_1, x_2) = (2, 5)$, seed $\dot{x}_1 = 1, \dot{x}_2 = 0$ (tính $\partial y/\partial x_1$):

Primal	Giá trị	Tangent	Giá trị
$v_{-1} = x_1$	2	$\dot{v}_{-1} = \dot{x}_1$	1
$v_0 = x_2$	5	$\dot{v}_0 = \dot{x}_2$	0
$v_1 = \ln v_{-1}$	$\ln 2$	$\dot{v}_1 = \dot{v}_{-1}/v_{-1}$	$1/2$

Primal	Giá trị	Tangent	Giá trị
$v_2 = v_{-1} \times v_0$	10	$\dot{v}_2 = \dot{v}_{-1}v_0 + \dot{v}_0 v_{-1}$	5
$v_3 = \sin v_0$	$\sin 5 \approx -0{,}959$	$\dot{v}_3 = \dot{v}_0 \cos v_0$	0
$v_4 = v_1 + v_2$	$0{,}693 + 10$	$\dot{v}_4 = \dot{v}_1 + \dot{v}_2$	$5{,}5$
$v_5 = v_4 - v_3$	$10{,}693 + 0{,}959 = 11{,}652$	$\dot{v}_5 = \dot{v}_4 - \dot{v}_3$	$5{,}5$
$y = v_5$	$11{,}652$	$\dot{y}$	$5{,}5$

Một lần "forward pass" chỉ cho **một cột** của Jacobian (đạo hàm theo MỘT input, với MỌI output). Forward mode thuận lợi khi $f : \mathbb{R} \to \mathbb{R}^m$ (một pass lấy hết); khi $f : \mathbb{R}^n \to \mathbb{R}$ (một output, nhiều input như loss trong ML) cần n **lần pass** để có đủ gradient $\nabla f = [\partial y/\partial x_1, \ldots, \partial y/\partial x_n]$, đây là điểm bất lợi chính của forward mode cho ML. Forward mode cũng cho phép tính **Jacobian-vector product** $J_f \mathbf{r}$ (công thức 4) chỉ bằng MỘT pass, bằng cách seed $\dot{\mathbf{x}} = \mathbf{r}$; trường hợp đặc biệt $f : \mathbb{R}^n \to \mathbb{R}$ cho đạo hàm theo hướng $\nabla f \cdot \mathbf{r}$.

Dual numbers (Mục 3.1.1): về mặt toán, forward mode tương đương tính hàm trên **số đôi** (dual number) $v + \dot{v}\epsilon$ với $\epsilon^2 = 0, \epsilon \neq 0$ (chuỗi Taylor cắt bậc 1). Quy tắc cộng/nhân trên số đôi tự động khớp quy tắc đạo hàm: $(v + \dot{v}\epsilon)(u + \dot{u}\epsilon) = (vu) + (v\dot{u} + \dot{v}u)\epsilon$ (đúng là quy tắc tích). Hệ số của ϵ sau khi áp hàm f trên $v + 1\epsilon$ chính là $f'(v)$ (công thức 5); chain rule hoạt động tự nhiên qua 2 lần áp công thức này.

3.4 Reverse mode (Mục 3.2)

Gắn mỗi biến trung gian một "adjoint" $\bar{v}_i = \frac{\partial y_j}{\partial v_i}$ (độ nhạy của MỘT output y_j theo mọi biến trung gian). Quy trình 2 giai đoạn: (1) chạy forward, lưu lại trace và giá trị các v_i (bookkeeping); (2) chạy backward từ output ($\bar{v}_{\text{output}} = \bar{y} = \partial y/\partial y = 1$), lan truyền ngược $\bar{v}_i$ theo total-derivative chain rule (xem math 01, mục 3.6b): mỗi biến nhận đóng góp từ TẤT CẢ biến "con" nó ảnh hưởng tới.

Cùng ví dụ, cùng điểm $(2, 5)$ **(Bảng 3), MỘT lần reverse pass cho CẢ HAI đạo hàm cùng lúc:**

$$\bar{x}_1 = \frac{\partial y}{\partial x_1} = 5{,}5, \qquad \bar{x}_2 = \frac{\partial y}{\partial x_2} = 1{,}716$$

Kiểm bằng tay (tự tính, đối chiếu công thức giải tích trực tiếp của $y = \ln(x_1) + x_1 x_2 - \sin(x_2)$**):** $\frac{\partial y}{\partial x_1} = \frac{1}{x_1} + x_2 = \frac{1}{2} + 5 = 5{,}5$ (khớp $\bar{x}_1$); $\frac{\partial y}{\partial x_2} = x_1 - \cos(x_2) = 2 - \cos(5) \approx 2 - 0{,}2837 = 1{,}7163$ (khớp $\bar{x}_2 = 1{,}716$, sai số làm tròn). Hai cách tính (đạo hàm giải tích trực tiếp và reverse-mode AD theo Bảng 3 của paper) cho cùng kết quả, đúng như lý thuyết chain rule đảm bảo.

Chi phí (Mục 3.2): với $f : \mathbb{R}^n \to \mathbb{R}^m$, gọi $\text{ops}(f)$ là số phép toán để tính f một lần. Forward mode tính Jacobian $m \times n$ trong thời gian $n\, c\, \text{ops}(f)$; reverse mode trong thời gian $m\, c\, \text{ops}(f)$, với hằng số $c < 6$, thường $c \in [2, 3]$ (dẫn theo Griewank & Walther 2008, không phải tự chứng minh trong bài này). Khi $m \ll n$ (loss scalar, triệu tham số, đúng tình huống ML), **reverse mode thắng rõ**: chỉ MỘT pass cho cả gradient n chiều, đây chính là lý do backpropagation (= reverse-mode AD cho mạng neural ghép loss) là kỹ thuật chủ đạo trong deep learning, không phải forward mode.

Đánh đổi: reverse mode cần lưu lại toàn bộ trace trung gian (tape) trong bộ nhớ, tăng theo số phép toán của hàm, đây là bất lợi chính so với forward mode (gần như không cần bộ nhớ ngoài). Paper dẫn ví dụ cụ thể từ nghiên cứu khác (Gruslys và đồng nghiệp 2016, không phải thí nghiệm của chính tác giả bài này): kỹ thuật checkpointing cho backpropagation-through-time (BPTT) trên mạng hồi quy, tiết kiệm **tới 95% bộ nhớ, đổi lại tăng 33% thời gian tính toán** trong một trường hợp cụ thể họ báo cáo.

3.5 Hessian-vector product không cần lưu toàn bộ Hessian (Mục 4.1)

Kỹ thuật "reverse-on-forward" (Pearlmutter 1994, được paper dẫn lại): với $f : \mathbb{R}^n \to \mathbb{R}$, điểm $\mathbf{x}$ và vector $\mathbf{v}$, trước tiên dùng **forward mode** để tính đạo hàm theo hướng $\nabla f \cdot \mathbf{v}$ (seed $\dot{\mathbf{x}} = \mathbf{v}$), sau đó áp **reverse mode**

lên kết quả đó để được $\nabla^2 f \cdot \mathbf{v} = H_f \mathbf{v}$. Toàn bộ chỉ tốn $O(n)$ (không phải $O(n^2)$ như vật lý hoá Hessian đầy đủ rồi nhân vector), dù H_f là ma trận $n \times n$. Đây là cơ sở cho các phương pháp tối ưu bậc hai xấp xỉ (quasi-Newton, stochastic Newton) hoạt động được ở quy mô lớn.

3.6 Ứng dụng trong machine learning (Mục 4)

Paper liệt kê 5 nhóm ứng dụng, mỗi nhóm kèm trích dẫn công trình cụ thể (không phải thí nghiệm riêng của bài): (i) **gradient-based optimization** nói chung (SGD, Newton/quasi-Newton dùng Hessian-vector product ở 3.5); (ii) **neural network/deep learning/differentiable programming**, nhấn mạnh backprop chỉ là ”trường hợp đặc biệt của AD” và framework hiện đại (autograd, Chainer, PyTorch) cung cấp AD tổng quát ”define-by-run” (dynamic computational graph) thay cho kiểu ”define-and-run” (static graph, như Theano/TensorFlow thời đó); (iii) **computer vision** (inverse graphics/render vi phân, bundle adjustment, đăng ký ảnh y tế: dẫn một kết quả từ nghiên cứu khác báo cáo **tăng tốc 6 lần** so với numerical diff center-difference, không phải số của chính bài này); (iv) **NLP** (parsing, sentence tracking); (v) **suy luận xác suất** (HMC/NUTS trong Stan, variational inference, reparameterization trick cho biến liên tục, REINFORCE/REBAR/RELAX cho biến rời rạc).

3.7 Các cách cài đặt (Mục 5)

Ba nhóm chính: **(a) Elemental libraries** (người code tự gọi hàm thư viện AD cho từng phép sơ cấp, đơn giản nhất nhưng thủ công nhất); **(b) Source code transformation/compiler** (tiền xử lý code nguồn để sinh code đạo hàm, ví dụ ADIFOR, Tapenade, Tangent cho Python/Numpy); **(c) Operator overloading** (định nghĩa lại các phép toán cơ bản trên kiểu dữ liệu AD-aware, cách phổ biến nhất trong ML hiện đại: ADOL-C dùng ”tape” ghi lại phép toán để ”chạy lại ngược”, PyTorch/Chainer/autograd theo hướng này). Paper cảnh báo các chi phí/rủi ro thực hành: cấp phát bộ nhớ cho số đôi tốn kém nếu làm ẩu, method dispatch của operator overloading có thể chậm cả bậc độ lớn so với tính toán số thuần, và lớp lỗi **”perturbation confusion”**: khi hai lượt AD lồng nhau tác động cùng đoạn code, hai ϵ hình thức phải được giữ phân biệt, dễ lẫn nếu cài không cẩn thận.

Lưu ý về xấp xỉ số (Mục 5): AD lấy đúng đạo hàm của **chương trình đã viết**, không phải đạo hàm ”lý tưởng” của hàm toán mà chương trình xấp xỉ. Ví dụ paper nêu: nếu e^x được tính bằng một công thức xấp xỉ từng khúc (piecewise-rational), AD trên công thức đó cho đạo hàm của XẤP XỈ đó, có thể kém hơn chính xấp xỉ gốc của $\frac{de^x}{dx} = e^x$ (paper ghi chú trong thực hành hiện đại điều này không còn là vấn đề vì e^x thường là hàm nguyên thuỷ cứng trong phần cứng).

4. Thí nghiệm và kết quả chính

Đây là paper **survey**, nhưng khác math 01/03, bài này CÓ MỘT thí nghiệm định lượng thật của chính tác giả (Mục 4.1, Bảng 4 và Hình 5): đo thời gian tính hàm năng lượng tự do Helmholtz (dựa trên phương trình trạng thái Peng-Robinson, có n biến độc lập) và gradient của nó bằng numerical diff (center difference), forward AD, và reverse AD, với n từ 1 đến 50, trên máy Intel Core i7-4785T 2.20GHz, 16GB RAM, dùng công cụ DiffSharp 0.5.7, lấy trung bình 1000 lần chạy. Thời gian hàm gốc tại $n = 1$ là $0{,}0023$ ms (mốc chuẩn hoá).

n	f gốc (so n=1)	∇f numerical (so n=1)	∇f forward AD (so n=1)	∇f reverse AD (so n=1)
1	1	1,08	1,34	1,52
8	5,12	35,55	13,69	11,12
15	14,51	176,79	51,54	31,37
22	29,11	499,43	132,33	67,27
29	52,58	1045,29	251,32	113,99
36	84,00	1986,70	469,84	174,62
43	127,33	3269,36	815,55	254,15

n	f gốc (so n=1)	∇f numerical (so n=1)	∇f forward AD (so n=1)	∇f reverse AD (so n=1)
50	174,44	4995,96	1342,07	342,33

Paper còn cho cột "so với f cùng n" (chi phí tương đối của gradient so với hàm gốc CÙNG kích thước): tại $n = 50$, numerical diff tốn $28{,}63\times$ chi phí của $f(n = 50)$, forward AD tốn $7{,}69\times$, còn **reverse AD chỉ tốn** $1{,}96\times$ và tỉ số này gần như không đổi khi n tăng (từ $1{,}52\times$ ở $n = 1$ xuống dao động quanh $2\times$), đúng khớp lý thuyết $m\,c\,\text{ops}(f)$ với $m = 1$ (gradient scalar) không phụ thuộc n. Đây là bằng chứng số cụ thể cho tuyên bố "reverse mode AD gần như không đắt hơn hàm gốc là bao, bất kể số chiều input", trong khi numerical diff và cả forward mode đều tăng chi phí tuyến tính theo n.

Các số liệu khác trong Mục 4 (tăng tốc 6 lần của Grabner và đồng nghiệp, tiết kiệm 95% bộ nhớ của Gruslys và đồng nghiệp) là **trích dẫn từ công trình khác**, không phải thí nghiệm của chính 4 tác giả bài survey này; paper chỉ tổng hợp lại làm minh chứng cho luận điểm.

5. Phân tích / ablation

- **Forward vs reverse, phụ thuộc hình dạng n so với m:** bảng ở Mục 4 cho thấy rõ cả numerical, forward, và reverse đều tăng theo n về giá trị tuyệt đối (vì hàm Helmholtz có $m = n$ do gradient có n thành phần), nhưng tốc độ tăng khác hẳn: numerical tăng gần $O(n^2)$ hiệu dụng khi tính đủ gradient (vì mỗi thành phần cần 1-2 lần gọi f có chi phí tăng theo n), forward AD tăng gần tuyến tính theo n do cần n pass, còn reverse AD gần hằng số tương đối so với f cùng kích thước.
- **Đánh đổi bộ nhớ-thời gian của reverse mode:** là trục đánh đổi DUY NHẤT mà reverse mode phải trả so với forward mode (vốn gần như không cần nhớ thêm); checkpointing là kỹ thuật chính để kiểm soát đánh đổi này, đặc biệt quan trọng trên GPU bộ nhớ hạn chế.
- **AD không "sửa" vấn đề số học:** paper chỉ rõ tổng nhiều số lớn gần triệt tiêu nhau (ví dụ $\nabla_w E = \sum_i \nabla_w E_i \to 0$ khi gần cực tiểu) vẫn gặp đúng rủi ro cộng số dấu phẩy động dù tính bằng AD; AD chỉ đảm bảo đúng đạo hàm của code đã viết, không đảm bảo code đó ổn định số học.
- **Khác biệt workload ML và workload AD truyền thống:** paper nhận xét ML hiện đại compute-bound (dồn vào kernel ma trận tối ưu cao), phù hợp operator-overloading trên phép toán mức cao; mô phỏng khoa học truyền thống (nguồn gốc lịch sử của AD) thường bandwidth-bound, nên hợp với source-code transformation/compiler hơn. Đây là lý do operator overloading (PyTorch-style) thắng thế trong ML dù cộng đồng AD truyền thống từng ưu tiên source transformation.

6. Giới hạn

Do tác giả nêu:

- Các vấn đề số học sâu (subgradient cho hàm không khả vi, xấp xỉ gần 0 cho $|\cdot|, \|\cdot\|_2, \sqrt{\cdot}$) được nói rõ là "ngoài phạm vi hiện tại" (Mục 5), chỉ trỏ tới tài liệu khác.
- Hằng số chi phí $c < 6$, $c \in [2, 3]$ của reverse/forward mode là dẫn lại từ Griewank & Walther (2008), không phải kết quả tự chứng minh của 4 tác giả.
- Nested/higher-order AD (cần cho hypergradient, tối ưu siêu tham số, HMC Riemannian) được xếp vào "hướng nghiên cứu tương lai" (Mục 6 Kết luận), chưa trình bày chi tiết kỹ thuật trong bài.
- Bài khảo sát các framework tại thời điểm viết (Theano, TensorFlow tĩnh, autograd, Chainer, PyTorch bản đầu); không thể phản ánh các phát triển AD sau 2018 (ví dụ JAX, chế độ compile-and-trace hiện đại).

Người ghi chú nhận xét:

- Thí nghiệm định lượng DUY NHẤT của bài (Bảng 4/Hình 5) dùng một hàm năng lượng tự do kiểu khoa học tính toán (scientific computing), KHÔNG phải một mạng neural/deep learning workload thật;

với một paper ”khảo sát AD cho machine learning”, việc thiếu một benchmark trực tiếp trên kiến trúc neural (ví dụ MLP/CNN nhỏ) để đo chi phí AD trong bối cảnh ML cụ thể là một khoảng trống đáng chú ý, dù không ảnh hưởng tới tính đúng đắn lý thuyết được trình bày.
- Các số liệu tăng tốc/giảm bộ nhớ dẫn từ công trình khác (6 lần, 95%) chỉ là MỘT trường hợp cụ thể được báo cáo trong bài gốc đó, không phải mức cải thiện trung bình hay đảm bảo chung; cần cẩn trọng khi tổng quát hoá.

7. Liên hệ với các bài khác

- **math 01** (The Matrix Calculus You Need For Deep Learning): cung cấp nền chain rule/Jacobian mà bài này dùng trực tiếp; forward mode AD tương ứng cách nhân Jacobian ”từ trong ra ngoài” theo hướng dữ liệu, reverse mode tương ứng cách nhân ”từ ngoài vào trong” mà math 01 đã gợi mở ở cuối Mục 4.5.1 của bài đó nhưng không khai triển thành thuật toán tổng quát.
- **math 03** (A Tutorial on Principal Component Analysis): bài tiếp theo trong batch, không phụ thuộc trực tiếp vào AD nhưng cùng mạch ”công cụ toán nền cho ML” của coordinator.
- **math 23 (Neural Tangent Kernel), math 24 (Lazy Training):** cả hai phân tích mạng neural qua Jacobian của hàm mạng theo tham số; trong thực hành, Jacobian đó luôn được TÍNH bằng chính kỹ thuật AD (forward-mode hoặc Jacobian-vector product) mô tả ở Mục 3.3-3.5 của bài này, dù hai bài đó tập trung vào hệ quả lý thuyết hơn là cách tính.
- **math 25** (The Principles of Deep Learning Theory): các phân tích lý thuyết sâu về mạng neural ở quy mô lớn đều mặc định có sẵn gradient/Hessian chính xác tới máy nhờ AD, đây là tiền đề thực hành mà bài 02 giải thích vì sao khả thi ở quy mô triệu/tỷ tham số.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 text + ~1.6-2.1K token visual, GPU $\leq$ 8GB, batch 1). Bài này giải thích CHÍNH CÔNG NGHỆ đang chạy ngầm mỗi lần gọi `loss.backward()`, nên có vài hệ quả thực hành trực tiếp (suy luận):

1. **Vì sao reverse-mode (= backprop) là lựa chọn duy nhất hợp lý ở đây.** Mô hình OCR có hàng trăm triệu tham số (n rất lớn) nhưng loss mỗi batch là một số scalar ($m = 1$); theo đúng phân tích chi phí $m\,c\,\mathrm{ops}(f)$ ở Mục 3.4, đây chính xác là trường hợp reverse mode áp đảo forward mode, không có lý do gì để cân nhắc forward-mode AD cho việc huấn luyện/distill mô hình này.
2. **Gradient checkpointing khi GPU chỉ 8GB.** Đánh đổi bộ nhớ-thời gian của reverse mode (3.4) chính là cơ sở lý thuyết của kỹ thuật `torch.utils.checkpoint` hay dùng trong thực hành: không lưu toàn bộ activation trung gian (đặc biệt tốn với 3.600 token/trang và nhiều layer attention/GatedDeltaNet), chấp nhận tính lại một phần forward khi backward để đổi lấy bộ nhớ, ví dụ cụ thể paper dẫn (95% bộ nhớ, +33% compute cho BPTT) cho thấy đánh đổi này có thể rất có lợi khi bộ nhớ là nút thắt (đúng tình huống GPU $\leq$ 8GB).
3. **Hessian-vector product (3.5) cho ước lượng độ nhạy bậc hai khi cắt tỉa**, nếu muốn chấm ”độ quan trọng” một tham số/kênh bằng xấp xỉ bậc hai của thay đổi loss (liên hệ pruning content/07, dùng Taylor bậc 2) mà không cần vật lý hoá Hessian đầy đủ (không khả thi ở quy mô mô hình này): kỹ thuật reverse-on-forward chỉ tốn $O(n)$, khả thi trên GPU nhỏ cho một vài hướng $\mathbf{v}$ được chọn.
4. **Operator overloading (PyTorch) là lựa chọn mặc định, không cần cân nhắc thêm**, vì toàn bộ pipeline train/distill hiện tại đều dựa trên nó; hiểu rủi ro ”perturbation confusion” (3.7) chỉ thực sự liên quan nếu cần lồng nhiều lượt đạo hàm (ví dụ meta-learning hyperparameter qua gradient của gradient), hiện chưa phải nhu cầu của dự án.

9. Thí nghiệm gợi ý

1. **Tái lập mini-bản của Bảng 4/Hình 5:** viết một hàm toy nhiều biến (ví dụ tổng vài hạng log/mũ của n biến, n từ 1 đến 50), đo thời gian numerical diff (center difference, NumPy) so với `torch.autograd.backward` (reverse) và `torch.func.jvp` (forward) trên CPU/GPU nhỏ; kỳ vọng tái hiện đúng xu hướng: reverse AD gần hằng số tương đối so với f, numerical/forward tăng theo n.
2. **Kiểm lại ví dụ số của paper bằng code:** cài $y = \ln(x_1) + x_1 x_2 - \sin(x_2)$ trong PyTorch với $x_1 = 2, x_2 = 5$ (`requires_grad=True`), gọi `.backward()`, so `x1.grad, x2.grad` với $\bar{x}_1 = 5{,}5, \bar{x}_2 \approx 1{,}716$ của Bảng 3; đo sai số tuyệt đối, kỳ vọng dưới 10^{-4}.
3. **Đo đánh đổi của gradient checkpointing** trên một MLP/Transformer-block nhỏ 15-20 lớp: so sánh peak memory (`torch.cuda.max_memory_allocated`) và thời gian một bước train khi bật/tắt `torch.utils.checkpoint`, đối chiếu tỉ lệ tiết kiệm bộ nhớ/tăng compute với con số paper dẫn (95%/33%) để xem mức đánh đổi có cùng cỡ không trên kiến trúc nhỏ của mình.
4. **Hessian-vector product toy:** trên một mạng nhỏ (vài nghìn tham số), tính $H\mathbf{v}$ bằng kỹ thuật reverse-on-forward (`torch.autograd.functional.vhp` hoặc double-backward tay) và so thời gian/bộ nhớ với vật lý hoá toàn bộ Hessian rồi nhân vector trực tiếp; kỳ vọng chênh lệch rõ khi số tham số tăng.
5. **Minh hoạ "perturbation confusion" có kiểm soát:** viết một đoạn code AD lồng 2 cấp (ví dụ differentiate một hàm mà bên trong nó lại gọi `.backward()` của một bài toán tối ưu con) theo đúng kiểu cảnh báo ở Mục 5, quan sát khi nào gradient tính sai nếu không `detach()`/`create_graph` đúng cách trong PyTorch, rút ra checklist tránh lỗi tương tự khi code của dự án có vòng tối ưu lồng nhau (ví dụ learned learning-rate).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Numerical diff	$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x}+h\mathbf{e}_i)-f(\mathbf{x})}{h}$, $O(n)$ lần gọi f
Center difference	$\frac{f(\mathbf{x}+h\mathbf{e}_i)-f(\mathbf{x}-h\mathbf{e}_i)}{2h} + O(h^2)$
Dual number	$v + \dot{v}\epsilon$, $\epsilon^2 = 0$; $f(v + \dot{v}\epsilon) = f(v) + f'(v)\dot{v}\epsilon$
Forward mode	tangent $\dot{v}_i = \partial v_i / \partial x_1$; 1 pass = 1 cột Jacobian; tốt khi $f : \mathbb{R} \to \mathbb{R}^m$
Reverse mode	adjoint $\bar{v}_i = \partial y_j / \partial v_i$; 1 pass = 1 hàng Jacobian (cả gradient nếu $m = 1$); tốt khi $f : \mathbb{R}^n \to \mathbb{R}$, n lớn
Chi phí	forward: $n\, c\, \text{ops}(f)$; reverse: $m\, c\, \text{ops}(f)$, $c < 6$, $c \in [2, 3]$
Jacobian-vector product (forward)	$J_f\mathbf{r}$, seed $\dot{\mathbf{x}} = \mathbf{r}$, 1 pass
Hessian-vector product	reverse-on-forward: $H_f\mathbf{v}$ trong $O(n)$, không cần vật lý hoá H_f
Backpropagation	trường hợp đặc biệt của reverse-mode AD, với $y =$ loss scalar

11. Kết luận ngắn

Bài khảo sát định nghĩa chính xác AD như một kỹ thuật khác hẳn đạo hàm tay/số/ký hiệu: tính đúng tới độ chính xác máy với chi phí chỉ tăng hằng số so với hàm gốc, nhờ lan truyền chain rule trên evaluation trace cụ thể của chương trình. Hai chế độ forward (tangent, theo hướng dữ liệu) và reverse (adjoint, ngược hướng dữ liệu, tổng quát hoá backpropagation) có chi phí đối nghịch theo tỉ lệ input/output, và thí nghiệm định lượng của chính bài (hàm Helmholtz, Bảng 4/Hình 5) xác nhận bằng số rằng reverse mode gần như không đắt hơn hàm gốc bất kể số chiều, đúng lý do nó là nền tảng của mọi pipeline huấn luyện deep learning hiện đại. Bài này chuyển hoá các quy tắc toán tay ở math 01 thành một thuật toán tổng quát cho bất kỳ chương trình số nào, đặt nền cho mọi framework autodiff (PyTorch và tương tự) mà các bài sau trong lộ trình sử dụng mà không cần suy lại từ đầu.

Nguồn gốc: https://arxiv.org/abs/1502.05767

Bài 03

A Tutorial on Principal Component Analysis (2014)

Tác giả: Jonathon Shlens (Google Research, Mountain View, CA)
Venue / arXiv: arXiv:1404.1100 (v1, 03/04/2014); paper không nêu venue hội nghị/tạp chí, đây là một bản tutorial tự xuất bản (trang đầu ghi ”Dated: April 7, 2014; Version 3.02”)
Nguồn: https://arxiv.org/abs/1404.1100
Chủ đề: Đại số tuyến tính ứng dụng, giảm chiều dữ liệu (PCA qua eigen-decomposition của ma trận hiệp phương sai và qua SVD)
Vị trí trong lộ trình: Bài số 3 trong nhóm 9 bài toán nền (math 01-09) chuẩn bị cho dự án nén/hậu huấn luyện mô hình. Đọc sau math 01 (matrix calculus) và math 02 (automatic differentiation), vì các khái niệm eigenvector, eigenvalue, đạo hàm ma trận ở hai bài đó sẽ được dùng lại. Đây là bài mở đầu nhóm con ”phương pháp eigen/SVD”, đứng trước math 04 (randomized SVD) và math 05 (spectral clustering).

> Ghi chú này được viết lại bằng tiếng Việt dựa trên toàn văn bài báo gốc (đọc từ bản PDF chuyển sang text, bao gồm cả hai phụ lục chứng minh và mã Matlab), không phải bản dịch từng câu. Mục tiêu là giải thích lại ý tưởng cho người mới bắt đầu, có bổ sung một ví dụ tính tay (đánh dấu ”tự tính”) để kiểm chứng công thức. Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Principal Component Analysis (PCA) là một trong những công cụ phổ biến nhất trong phân tích dữ liệu hiện đại, được dùng trong rất nhiều lĩnh vực: thần kinh học, đồ họa máy tính, nén dữ liệu, học máy. Paper mở đầu bằng một nhận xét khá thẳng: PCA là ”a mainstay of modern data analysis, a black box that is widely used but (sometimes) poorly understood”. Mục tiêu của tác giả Jonathon Shlens không phải đưa ra kết quả nghiên cứu mới, mà là giải thích lại cho người đọc, từ trực giác đơn giản đến công thức đại số tuyến tính đầy đủ, vì sao và như thế nào PCA hoạt động.

Đây là một bản tutorial tự xuất bản trên arXiv, không công bố ở hội nghị hay tạp chí nào. Một điểm khác thường so với các bài báo khoa học điển hình: bài không có danh sách tài liệu tham khảo (reference list) nào ở cuối. Toàn bộ nội dung được tác giả tự suy diễn từ các định lý đại số tuyến tính cơ bản (được chứng minh lại trong Phụ lục A), kèm hai đoạn mã Matlab minh họa ở Phụ lục B. Giả định duy nhất mà tác giả đặt ra là người đọc đã có kiến thức đại số tuyến tính cơ bản (vector, ma trận, tích vô hướng, giá trị riêng).

Vì đây là bài ngắn (khoảng 12 trang kể cả phụ lục, 791 dòng text sau khi chuyển từ PDF), ghi chú này đọc và dùng toàn bộ nội dung: phần thân bài (giới thiệu, ví dụ lò xo, đổi cơ sở, phương sai/hiệp phương sai, hai cách giải PCA, thảo luận) và cả hai phụ lục (Phụ lục A: chứng minh các định lý đại số tuyến tính; Phụ lục B: mã Matlab).

Trong lộ trình học của dự án, bài này đóng vai trò nền tảng cho nhóm các phương pháp dựa trên eigen-decomposition/SVD: những kỹ thuật như randomized SVD (math 04) hay spectral clustering (math 05) đều xoay quanh việc tìm một tập vector riêng (eigenvector) ”tốt” của một ma trận nào đó, và PCA là ví dụ đơn giản nhất, dễ hình dung nhất của mô thức này khi ma trận đó là ma trận hiệp phương sai của dữ liệu.

2. Ý tưởng chính

Ý tưởng trung tâm của PCA, theo cách trình bày của paper, có thể tóm gọn như sau: cho một tập dữ liệu được ghi lại trong một cơ sở ”ngây thơ” (naive basis, ví dụ hệ tọa độ do cách đặt cảm biến/camera quyết định, không nhất thiết phản ánh đúng cấu trúc vật lý thật), PCA đi tìm một cơ sở trực chuẩn (orthonormal basis) mới, là một phép quay của cơ sở cũ, sao cho khi biểu diễn lại dữ liệu trong cơ sở mới này:

- Các chiều (biến) mới không còn tương quan với nhau (hiệp phương sai chéo bằng 0), nghĩa là loại bỏ được sự dư thừa (redundancy) giữa các biến đo.
- Các chiều được xếp hạng theo phương sai giảm dần, chiều có phương sai lớn nhất được coi là mang nhiều ”cấu trúc tín hiệu” nhất (theo giả định signal-to-noise ratio, SNR).

Để đi đến công thức cụ thể, paper đặt ra ba giả định tường minh (paper tự liệt kê ở mục ”Summary of Assumptions”):

1. **Tuyến tính (Linearity):** cơ sở mới chỉ được phép là một tổ hợp tuyến tính của cơ sở cũ, tức bài toán được quy về bài toán đổi cơ sở $Y = PX$ với P là ma trận biến đổi.
2. **Phương sai lớn = cấu trúc quan trọng:** các hướng có phương sai lớn nhất được giả định chứa thông tin tín hiệu quan trọng nhất, còn phương sai nhỏ được coi là nhiễu.
3. **Các thành phần chính trực giao với nhau (orthogonal):** đây là một giả định kỹ thuật giúp bài toán có nghiệm đại số tuyến tính rõ ràng (không phải điều kiện bắt buộc về mặt vật lý).

Với ba giả định này, PCA trở thành bài toán: tìm ma trận trực chuẩn P sao cho ma trận hiệp phương sai của dữ liệu đã biến đổi, $C_Y = \frac{1}{n}YY^T$, là một ma trận đường chéo (diagonal), và các giá trị trên đường chéo (chính là các phương sai) được xếp theo thứ tự giảm dần. Paper chứng minh nghiệm của bài toán này chính là: các hàng của P là các eigenvector của ma trận hiệp phương sai $C_X = \frac{1}{n}XX^T$ của dữ liệu gốc, xếp theo eigenvalue giảm dần. Đây gọi là các ”principal components”.

Một cách diễn đạt tương đương (và được paper trình bày là cách tổng quát hơn) là qua phân tích suy biến giá trị kỳ dị (Singular Value Decomposition, SVD): các principal component chính là các vector riêng bên phải (right singular vectors) của một ma trận liên quan trực tiếp đến X.

3. Phương pháp chi tiết

3.1. Ví dụ dẫn nhập: lò xo và ba camera

Paper mở đầu bằng một ví dụ vật lý: một quả bóng gắn vào lò xo lý tưởng dao động dọc theo một trục duy nhất (gọi là trục x). Vì người thực nghiệm không biết trước đâu là trục đúng, họ đặt ba camera A, B, C ở các góc tùy ý quanh hệ thống, mỗi camera ghi lại một hình chiếu 2D của vị trí quả bóng với tần số 120 Hz. Mỗi mẫu dữ liệu tại một thời điểm là một vector 6 chiều $\vec{X} = (x_A, y_A, x_B, y_B, x_C, y_C)^T$. Nếu ghi 10 phút, ta có $10 \times 60 \times 120 = 72000$ vector như vậy.

Dù bản chất vật lý chỉ có 1 chiều chuyển động thật (x), do cách đo đạc (ba camera ở các góc bất kỳ), dữ liệu thô ”trông” như có 6 chiều độc lập. Câu hỏi của PCA là: từ 6 chiều đo được (nhiễu, dư thừa, không phải cơ sở ”đúng”), làm sao suy ra được hướng chuyển động thật?

Paper gọi cơ sở đo đạc ban đầu (ví dụ (x_A, y_A) của một camera) là ”naive basis” (cơ sở ngây thơ), vì nó chỉ phản ánh cách đặt thiết bị đo, không phản ánh cấu trúc vật lý thật của hệ thống.

3.2. Đổi cơ sở (change of basis)

Gọi X là ma trận dữ liệu gốc kích thước $m \times n$ (m là số loại đo đạc, mỗi cột là một mẫu/thời điểm; trong ví dụ lò xo thì $m = 6$, $n = 72000$). PCA tìm một ma trận P (kích thước $m \times m$, trực chuẩn) để biến đổi:

$$PX = Y$$

Trong đó các hàng p_i của P là tập cơ sở mới, các cột x_i của X là các mẫu dữ liệu gốc, và các cột y_i của Y là dữ liệu đã biểu diễn lại. Paper chỉ ra: mỗi phần tử của y_i là tích vô hướng của x_i với một hàng p_j của P, tức y_i chính là hình chiếu của x_i lên các vector $p_1, \ldots, p_m$. Nói cách khác, các hàng của P đóng vai trò một cơ sở trực chuẩn mới để biểu diễn lại các cột của X.

Hai câu hỏi còn lại mà paper đặt ra: ”cách biểu diễn lại X tốt nhất là gì?” và ”chọn P như thế nào?”. Để trả lời, cần thêm tiêu chí: thế nào là ”biểu diễn tốt”.

3.3. Phương sai, nhiễu và dư thừa

Paper định nghĩa tỉ số tín hiệu trên nhiễu (SNR) là SNR $= \sigma^2_{\text{signal}}/\sigma^2_{\text{noise}}$: SNR cao nghĩa là đo đạc chính xác, SNR thấp nghĩa là dữ liệu nhiễu nhiều. Minh họa bằng Hình 2 của paper (dữ liệu mô phỏng của một camera): hướng có phương sai lớn nhất của đám mây điểm không trùng với trục đo đạc ban đầu (x_A, y_A), mà là một hướng xiên (đường fit tốt nhất qua đám mây). Giả định cốt lõi: hướng có phương sai lớn nhất chứa động lực học (dynamics) quan trọng nhất của hệ thống.

Bên cạnh nhiễu, còn có vấn đề dư thừa (redundancy): nhiều cảm biến có thể đo cùng một thông tin (ví dụ hai camera quá gần nhau, hoặc cùng một đại lượng đo bằng hai đơn vị khác nhau). Paper minh họa bằng Hình 3: nếu hai biến đo r_1, r_2 không tương quan (plot bên trái), không có gì để loại bỏ; nếu chúng tương quan cao (plot bên phải), chỉ cần giữ lại một biến là đủ, đây chính là ý tưởng cốt lõi của giảm chiều dữ liệu.

Để định lượng dư thừa, paper định nghĩa hiệp phương sai (covariance) giữa hai tập đo $A = \{a_1, \ldots, a_n\}$ và $B = \{b_1, \ldots, b_n\}$ (giả định trung bình bằng 0):

$$\sigma^2_A = \frac{1}{n}\sum_i a_i^2, \quad \sigma^2_B = \frac{1}{n}\sum_i b_i^2, \quad \sigma^2_{AB} = \frac{1}{n}\sum_i a_i b_i = \frac{1}{n} a b^T$$

với a, b là các vector hàng. Paper nêu rõ hai tính chất: $\sigma_{AB} = 0$ khi và chỉ khi A, B không tương quan; $\sigma^2_{AB} = \sigma^2_A$ khi $A = B$. Paper cũng có một chú thích (footnote 2) quan trọng: trong công thức chính của bài, hiệp phương sai dùng hệ số $1/n$ để đơn giản hóa việc suy diễn, nhưng trong thực hành (và lý thuyết ước lượng thống kê), người ta thường dùng $1/(n-1)$. Đây là điểm cần lưu ý: **công thức suy diễn chính trong mục III-V của paper dùng** $1/n$**, nhưng hai đoạn mã Matlab ở Phụ lục B lại dùng** $1/(N-1)$ (`covariance = 1/(N-1) * data * data';` và `Y = data' / sqrt(N-1);`). Đây không phải một lỗi hay mâu thuẫn bị bỏ sót; paper đã tự nêu rõ sự khác biệt này ở footnote 2, nhưng người đọc cần nhận ra chủ động rằng công thức lý thuyết (dùng n) và mã thực hành (dùng $n-1$) không giống nhau tuyệt đối, chỉ xấp xỉ nhau khi n lớn.

Tổng quát hóa cho m loại đo đạc, định nghĩa ma trận hiệp phương sai:

$$C_X \equiv \frac{1}{n} X X^T$$

Paper liệt kê các tính chất của C_X: là ma trận vuông đối xứng $m \times m$ (chứng minh ở Phụ lục A, Định lý 2); các phần tử trên đường chéo là phương sai của từng loại đo; các phần tử ngoài đường chéo là hiệp phương sai giữa các cặp đo đạc. Mục tiêu: tìm cách biến đổi để có một ma trận hiệp phương sai mới C_Y sao cho (1) mọi phần tử ngoài đường chéo bằng 0 (dữ liệu mới không còn tương quan), và (2) các phần tử trên đường chéo xếp theo thứ tự giảm dần (các chiều xếp hạng theo ”độ quan trọng”).

3.4. Lời giải 1: eigen-decomposition của ma trận hiệp phương sai

Paper chọn cách đơn giản nhất để chéo hóa C_Y: giả định P là ma trận trực chuẩn (orthonormal). Khi đó:

$$C_Y = \frac{1}{n} Y Y^T = \frac{1}{n}(PX)(PX)^T = P\left(\frac{1}{n} X X^T\right) P^T = P C_X P^T$$

Theo định lý đại số tuyến tính (Phụ lục A, Định lý 4): mọi ma trận đối xứng A có thể chéo hóa bằng ma trận các eigenvector của nó, $A = EDE^T$, với D là ma trận đường chéo chứa các eigenvalue, E là ma trận các eigenvector (đã chuẩn hóa) xếp theo cột. Chọn $P \equiv E^T$ (mỗi hàng của P là một eigenvector của C_X), và dùng tính chất ma trận trực chuẩn có nghịch đảo bằng chuyển vị ($P^{-1} = P^T$, Định lý 1), ta có:

$$C_Y = PC_XP^T = P(E^TDE)P^T = (PP^T)D(PP^T) = D$$

Vậy: **các thành phần chính (principal components) của X chính là các eigenvector của $C_X = \frac{1}{n}XX^T$**, và phần tử thứ i trên đường chéo của C_Y chính là phương sai của dữ liệu đã chiếu lên eigenvector thứ i. Thuật toán thực hành (paper nêu): (1) trừ đi giá trị trung bình của mỗi loại đo, (2) tính các eigenvector của C_X, xếp theo eigenvalue giảm dần.

3.5. Lời giải 2: qua SVD

Paper trình bày một cách tổng quát hơn qua phân tích SVD, và thừa nhận đây là phần "mathematically involved" nhất, có thể bỏ qua mà không mất tính liên tục của bài. Cho một ma trận X bất kỳ kích thước $n \times m$ (chú ý: paper đối chiếu quy ước $m \times n$ thành $n \times m$ chỉ trong mục này), X^TX là ma trận đối xứng $m \times m$ hạng r, có các eigenvector trực chuẩn $\hat{v}_1, \ldots, \hat{v}_r$ với eigenvalue $\lambda_1, \ldots, \lambda_r$. Định nghĩa singular value $\sigma_i \equiv \sqrt{\lambda_i}$ và vector $\hat{u}_i \equiv \frac{1}{\sigma_i}X\hat{v}_i$. Dạng vô hướng của SVD:

$$X\hat{v}_i = \sigma_i\hat{u}_i$$

Ghép tất cả thành dạng ma trận (xếp các $\hat{v}_i$ thành cột của V, các $\hat{u}_i$ thành cột của U, các σ_i lên đường chéo của Σ):

$$X = U\Sigma V^T$$

Đây chính là định nghĩa SVD: mọi ma trận X đều có thể phân tích thành (trực giao) $\times$ (đường chéo) $\times$ (trực giao), tương ứng với phép quay, phép co giãn, rồi một phép quay khác.

Lưu ý về ký hiệu: ở đoạn này paper định nghĩa một ma trận mới cũng được gọi là Y, nhưng với vai trò hoàn toàn khác so với $Y = PX$ ở mục III-IV. Cụ thể, để kết nối SVD với PCA, paper đặt $Y \equiv \frac{1}{\sqrt{n}}X^T$ (kích thước $n \times m$, mỗi cột có trung bình 0), rồi chỉ ra $Y^TY = \frac{1}{n}XX^T = C_X$ (chính là ma trận hiệp phương sai). Nếu lấy SVD của ma trận Y này, các cột của V (trong $Y = U\Sigma V^T$) chính là các eigenvector của C_X, tức chính là các principal component. Đây không phải một mâu thuẫn toán học, nhưng là một điểm **dễ gây nhầm lẫn khi đọc**: chữ "Y" được paper dùng lại cho hai đối tượng khác nhau ở hai mục khác nhau (nhận xét của người ghi chú, không phải do paper nói sai).

Thuật toán thực hành thứ hai (SVD): (1) trừ trung bình mỗi loại đo, (2) tính SVD của ma trận $\frac{1}{\sqrt{n}}X^T$, lấy V làm các thành phần chính. Phụ lục B cho hai đoạn mã Matlab tương ứng hai thuật toán này (`pca1` dùng eigen-decomposition của hiệp phương sai, `pca2` dùng SVD).

3.6. Ví dụ số tự tính: PCA tay cho 4 mẫu, 2 chiều

Để kiểm chứng toàn bộ công thức ở trên bằng số cụ thể (tự tính), xét một bộ dữ liệu giả định đã có trung bình 0, 2 loại đo A, B (tức $m = 2$), 4 mẫu ($n = 4$):

mẫu	A	B
1	2	2
2	-2	-2
3	1	-1
4	-1	1

Dễ thấy mẫu 1, 2 nằm đúng trên đường $B = A$, còn mẫu 3, 4 nằm đúng trên đường $B = -A$, tức đây là một

phiên bản số của ”dữ liệu tương quan cao” ở Hình 3 của paper, nhưng với hai cụm tương quan ngược dấu nhau.

Bước 1, tính hiệp phương sai (dùng công thức $1/n$ như mục III-V của paper):

$$\sigma_A^2 = \frac{1}{4}(4+4+1+1) = 2.5, \quad \sigma_B^2 = \frac{1}{4}(4+4+1+1) = 2.5$$

$$\sigma_{AB}^2 = \frac{1}{4}\big(2\cdot 2 + (-2)\cdot(-2) + 1\cdot(-1) + (-1)\cdot 1\big) = \frac{1}{4}(4+4-1-1) = 1.5$$

Vậy $C_X = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{pmatrix}$.

Bước 2, tìm eigenvalue: phương trình đặc trưng $(2.5-\lambda)^2 - 1.5^2 = 0 \Rightarrow \lambda = 2.5 \pm 1.5$, cho $\lambda_1 = 4$, $\lambda_2 = 1$. Kiểm tra: $\lambda_1 + \lambda_2 = 5 = \sigma_A^2 + \sigma_B^2$ (tổng phương sai được bảo toàn qua phép quay, đúng như tính chất trace của ma trận đối xứng khi chéo hóa bằng ma trận trực chuẩn).

Bước 3, tìm eigenvector: với $\lambda_1 = 4$: $(2.5-4)x + 1.5y = 0 \Rightarrow -x + y = 0 \Rightarrow x = y$, chuẩn hóa được $p_1 = (1/\sqrt{2}, 1/\sqrt{2})$. Với $\lambda_2 = 1$: $(2.5-1)x + 1.5y = 0 \Rightarrow x + y = 0 \Rightarrow x = -y$, chuẩn hóa $p_2 = (1/\sqrt{2}, -1/\sqrt{2})$.

Bước 4, chiếu dữ liệu lên các thành phần chính và kiểm tra lại phương sai: chiếu mẫu 1, $(2,2)$, lên p_1: $(2+2)/\sqrt{2} = 2\sqrt{2} \approx 2.828$; lên p_2: $(2-2)/\sqrt{2} = 0$. Chiếu mẫu 3, $(1,-1)$, lên p_1: $(1-1)/\sqrt{2} = 0$; lên p_2: $(1+1)/\sqrt{2} = \sqrt{2} \approx 1.414$. Làm tương tự cho mẫu 2, 4, ta được tọa độ chiếu lên p_1 là $\{2\sqrt{2}, -2\sqrt{2}, 0, 0\}$ và lên p_2 là $\{0, 0, \sqrt{2}, -\sqrt{2}\}$.

Phương sai của tọa độ chiếu lên p_1: $\frac{1}{4}\big[(2\sqrt{2})^2 + (-2\sqrt{2})^2 + 0 + 0\big] = \frac{1}{4}(8+8) = 4$, đúng bằng λ_1. Phương sai của tọa độ chiếu lên p_2: $\frac{1}{4}\big[0 + 0 + (\sqrt{2})^2 + (-\sqrt{2})^2\big] = \frac{1}{4}(2+2) = 1$, đúng bằng λ_2. Vậy phép tính khớp hoàn toàn với công thức $C_Y = D$ ở mục 3.4: sau khi đổi sang cơ sở $P = \{p_1, p_2\}$, hiệp phương sai mới là $\begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, đường chéo, đúng như mục tiêu của PCA. Về mặt hình học, p_1 chính là hướng mà mẫu 1, 2 nằm trên đó (đường $B = A$), còn p_2 là hướng mà mẫu 3, 4 nằm trên đó (đường $B = -A$): PCA đã ”tìm lại” chính xác hai trục ẩn mà dữ liệu được tạo ra từ đó, không cần biết trước quy luật sinh dữ liệu (toàn bộ ví dụ này là tự tính, không lấy từ paper).

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial lý thuyết, **không có benchmark định lượng** theo nghĩa so sánh với baseline trên bộ dữ liệu thật. Paper không chạy PCA trên một dataset thực tế nào và không báo cáo số liệu (thời gian chạy, độ chính xác, v.v.). ”Kết quả chính” của paper là các kết quả suy diễn đại số tuyến tính và các ví dụ minh họa mang tính khái niệm:

- **Kết quả toán học trung tâm:** PCA tương đương với việc tìm eigenvector của ma trận hiệp phương sai $C_X = \frac{1}{n}XX^T$ (mục 3.4), hoặc tương đương tìm right singular vector của ma trận $\frac{1}{\sqrt{n}}X^T$ qua SVD (mục 3.5). Hai cách này cho cùng một tập thành phần chính.
- **Ví dụ minh họa lò xo:** paper khẳng định (không kèm số liệu, chỉ là lập luận định tính) rằng áp dụng PCA lên 6 chiều dữ liệu camera sẽ ”khôi phục” được một chiều chính mang hầu hết phương sai, tương ứng với hướng chuyển động $\hat{x}$ thật của lò xo, mặc dù dữ liệu thô có 6 chiều.
- **”Quick Summary of PCA” (Hình 5 của paper):** paper tổng kết quy trình thực hành thành 3 bước: (1) tổ chức dữ liệu thành ma trận $m \times n$ (m loại đo, n mẫu), (2) trừ trung bình của mỗi loại đo, (3) tính SVD hoặc eigenvector của ma trận hiệp phương sai.

- **Hai ví dụ phản biện (Hình 6):** (a) theo dõi một người trên vòng đu quay (ferris wheel): động lực học thực chất chỉ phụ thuộc một biến góc quay θ (phi tuyến với tọa độ Descartes), PCA không khôi phục được biến này; (b) một bộ dữ liệu có phân phối không-Gauss và các trục không trực giao, khiến hướng phương sai lớn nhất không phải là câu trả lời ”đúng” mà ta mong đợi. Cả hai ví dụ này cũng chỉ minh họa bằng hình vẽ, không có số liệu định lượng kèm theo.
- **Phụ lục B (mã nguồn):** hai hàm Matlab `pca1` (dùng eigen-decomposition của hiệp phương sai) và `pca2` (dùng SVD) được cung cấp để người đọc tự chạy, nhưng paper không so sánh tốc độ hay độ chính xác giữa hai cách cài đặt này trên dữ liệu cụ thể nào.

Nói cách khác, đóng góp chính của paper nằm ở phần suy diễn và trực giác, không phải ở phần thực nghiệm. Đây là điểm khác biệt rõ với math 02 (bài trước trong batch), nơi vẫn có một bảng thực nghiệm định lượng thật (benchmark thời gian chạy của forward/reverse-mode AD).

5. Phân tích / ablation

Vì không có thí nghiệm định lượng, phần ”ablation” ở đây được hiểu theo nghĩa phân tích các giả định và điều kiện áp dụng mà paper tự đặt ra:

So sánh hai lời giải (eigen-decomposition vs SVD): về mặt đại số, hai cách hoàn toàn tương đương: eigenvector của C_X cũng chính là các cột V trong SVD của $\frac{1}{\sqrt{n}}X^T$. Paper trình bày cách 1 (mục V) là ”straightforward” và cách 2 (mục VI) là tổng quát hơn nhưng phức tạp hơn về đại số. Paper không bàn đến sự khác biệt về độ ổn định số học (numerical stability) giữa việc tính trực tiếp eigenvector của XX^T so với tính SVD của X trực tiếp; đây là một điểm thường được nhắc tới trong thực hành tính toán số (tính XX^T rồi lấy eigen có thể khuếch đại lỗi số học do bình phương điều kiện số của X) nhưng **không được paper này đề cập**, nên nếu muốn dùng lập luận đó phải ghi là ”(ngoài paper, kiến thức chung)”.

Ba giả định của PCA và khi nào chúng có vấn đề (paper tự thảo luận ở mục VII):

Giả định	Nội dung	Khi nào có vấn đề (theo paper)
I. Tuyến tính	Cơ sở mới là tổ hợp tuyến tính của cơ sở cũ	Khi động lực học thật là phi tuyến (ví dụ Hình 6a, vòng đu quay với biến góc θ)
II. Phương sai lớn = cấu trúc quan trọng	Thành phần phương sai cao mang thông tin, phương sai thấp là nhiễu	Giả định này ”có thể sai” khi SNR thấp hoặc khi cấu trúc quan trọng lại nằm ở phương sai nhỏ; paper nêu đây là giả định ”mạnh và đôi khi không đúng”
III. Các thành phần chính trực giao	P là ma trận trực chuẩn	Khi dữ liệu thật phân bố theo các trục không trực giao (Hình 6b); đây là giả định kỹ thuật để bài toán có nghiệm đại số gọn, không phải yêu cầu vật lý

Về tính tối ưu của PCA: paper khẳng định ”it can be proved that under a common loss function, mean squared error (i.e. L2 norm), PCA provides the optimal reduced representation of the data” nhưng **không đưa ra chứng minh hay trích dẫn nguồn** cho khẳng định này trong văn bản được đọc. Đây là một khẳng định được nêu nhưng không được chứng minh trong bài, cần ghi nhận rõ là ”paper nói” chứ ghi chú này không tự kiểm chứng được tính đúng đắn của phát biểu đó.

Về ”phi tham số” (non-parametric): paper tự nêu đây vừa là ưu điểm (không cần chọn tham số, kết quả duy nhất và không phụ thuộc người dùng) vừa là nhược điểm (PCA ”không quan tâm” đến cách dữ liệu được tạo ra, nên không thể tận dụng kiến thức tiên nghiệm, ví dụ không nhận ra cấu trúc tuần hoàn của ví dụ vòng đu quay).

6. Giới hạn

Các giới hạn mà **paper tự nêu** (mục VII ”Discussion”):

- PCA chỉ loại bỏ được các phụ thuộc bậc hai (second-order dependencies, tức tương quan tuyến tính đo bằng hiệp phương sai). Nếu cấu trúc quan trọng của dữ liệu nằm ở các phụ thuộc bậc cao hơn (higher-order dependencies), PCA sẽ không phát hiện ra. Paper có chú thích (footnote 7) nêu: phụ thuộc bậc hai là đủ để mô tả toàn bộ cấu trúc khi dữ liệu phân phối Gauss (trong trường hợp đó bậc một và bậc hai là thống kê đủ, sufficient statistics).
- Ví dụ thất bại cụ thể: vòng đu quay (Hình 6a, động lực học phi tuyến theo góc θ) và dữ liệu không-Gauss/trục không trực giao (Hình 6b).
- Paper gợi ý hai hướng khắc phục nhưng không đi sâu: kernel PCA (áp dụng một phi tuyến/kernel lên dữ liệu trước khi làm PCA, ví dụ đổi sang tọa độ cực cho ví dụ vòng đu quay) và Independent Component Analysis, ICA (yêu cầu các chiều giảm xuống độc lập thống kê, không chỉ không tương quan). Về ICA, paper nhận xét thẳng: ”suffers from the fact that solutions are (sometimes) difficult to compute”, nhưng không trình bày thuật toán ICA cụ thể nào.

Một số quan sát **của người ghi chú** (không phải do paper tự phê bình, nhưng là đặc điểm khách quan của văn bản):

- Paper hoàn toàn không có danh mục tài liệu tham khảo; mọi khẳng định lịch sử hoặc liên hệ với các phương pháp khác (ICA, kernel PCA) đều không có trích dẫn kèm theo, nên không thể lần ngược lại nguồn gốc các khẳng định đó từ chính văn bản này.
- Như đã nêu ở mục 3.5, paper dùng lại ký hiệu Y cho hai đối tượng khác nhau ở hai mục khác nhau (dữ liệu đã đổi cơ sở ở mục III-IV, và ma trận chuẩn hóa $\frac{1}{\sqrt{n}}X^T$ dùng để tính SVD ở mục VI.C). Đây là một điểm dễ gây nhầm lẫn khi đọc nhanh.
- Công thức hiệp phương sai trong thân bài dùng hệ số $1/n$, còn mã Matlab ở Phụ lục B dùng $1/(N-1)$; paper có nêu khác biệt này ở một footnote ngắn nhưng không nhấn mạnh, người đọc dễ bỏ sót nếu chỉ đọc phần công thức chính.
- Toàn bộ các ví dụ số của paper (ví dụ lò xo, Hình 2, 3, 6) chỉ mang tính minh họa định tính qua hình vẽ, không có dữ liệu số cụ thể nào được in kèm trong văn bản để người đọc tự kiểm tra lại bằng tay; đây là lý do ghi chú này phải tự xây dựng một ví dụ số riêng ở mục 3.6 để có một ví dụ ”tự tính” kiểm chứng công thức.

7. Liên hệ với các bài khác

- **math 04 (Finding Structure with Randomness, Halko-Martinsson-Tropp):** bài này mở rộng trực tiếp lời giải SVD của PCA (mục 3.5 ở trên) sang quy mô lớn, dùng random projection để tính một SVD xấp xỉ nhanh hơn nhiều so với tính SVD đầy đủ khi ma trận dữ liệu rất lớn. Nói cách khác, math 04 trả lời câu hỏi ”làm sao tính PCA/SVD hiệu quả khi m, n rất lớn”, trong khi bài hiện tại chỉ trình bày lời giải ”đúng nhưng tốn kém” (eigen-decomposition hoặc SVD đầy đủ).
- **math 05 (A Tutorial on Spectral Clustering, von Luxburg):** cũng dựa trên eigen-decomposition, nhưng của một ma trận khác, ma trận Laplacian của đồ thị tương tự (similarity graph) giữa các điểm dữ liệu, thay vì ma trận hiệp phương sai. Sự tương phản thú vị: PCA tìm hướng phương sai lớn nhất (dữ liệu ”trải rộng” nhất) để giữ thông tin, còn spectral clustering tìm các eigenvector gắn với eigenvalue *nhỏ nhất* của Laplacian để tìm cấu trúc cụm (cluster). Cả hai đều minh họa mô thức chung ”phân tích dữ liệu bằng eigenvector của một ma trận mô tả cấu trúc dữ liệu”, nhưng chọn ma trận và tiêu chí khác nhau.
- **math 19 (Measuring the Intrinsic Dimension of Objective Landscapes, Li và cộng sự 2018):** đặt câu hỏi cùng họ với PCA, ”dữ liệu/bài toán thực chất cần bao nhiêu chiều để mô tả đầy đủ”, nhưng áp dụng cho không gian tham số của mạng neural (landscape mục tiêu huấn luyện) thay vì không gian đặc trưng của dữ liệu, và dùng phương pháp random subspace training thay vì eigen-decomposition của

hiệp phương sai.

- **ml basics 40 (UMAP, McInnes và cộng sự):** một phương pháp giảm chiều phi tuyến hiện đại, trực tiếp giải quyết đúng loại thất bại mà paper này tự nêu ở Hình 6 (dữ liệu có cấu trúc phi tuyến hoặc phân phối không phù hợp với giả định tuyến tính/Gauss của PCA).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B, một VLM OCR dùng kiến trúc lai GatedDeltaNet kết hợp full attention, xử lý khoảng 3600 token/trang (trong đó khoảng 1500 token văn bản và 1600-2100 token hình ảnh), chạy trên GPU không quá 8GB, batch size 1. Dưới đây là các liên hệ **suy luận** (không phải nội dung paper, do người ghi chú tự đề xuất dựa trên hiểu biết về PCA):

- **Chẩn đoán chiều hiệu dụng của activation trước khi cắt tỉa (pruning) theo width:** có thể lấy activation (hidden state) của một tầng cụ thể trên một batch dữ liệu đại diện, chạy PCA, xem bao nhiêu principal component giữ được, ví dụ, 90-95% phương sai. Nếu số lượng này nhỏ hơn nhiều so với số chiều hidden thực tế của tầng, đó là tín hiệu (không phải bằng chứng chắc chắn) cho thấy tầng đó có thể bị ”over-provisioned” về chiều, và có thể là ứng viên tốt để giảm width hoặc áp dụng nén low-rank.
- **Phân tích độ dư thừa của visual token:** với khoảng 1600-2100 token hình ảnh mỗi trang, có thể dùng PCA để kiểm tra xem embedding của các token này có tập trung phương sai vào một số ít thành phần không. Nếu có, đây là gợi ý ủng hộ các kỹ thuật gộp/giảm token hình ảnh (token merging/pruning) để giảm số token thực tế cần xử lý, giúp tiết kiệm bộ nhớ GPU quan trọng khi chạy batch size 1 trên GPU 8GB.
- **Giới hạn quan trọng cần nhớ khi áp dụng suy luận trên:** như paper tự nêu ở mục VII (Hình 6), PCA chỉ bắt được cấu trúc tuyến tính bậc hai. Biểu diễn nội tại của một mô hình attention/GatedDeltaNet nhiều khả năng mang cấu trúc phi tuyến phức tạp hơn nhiều so với giả định của PCA, vì vậy kết quả PCA trên activation chỉ nên dùng như một **chỉ báo thăm dò ban đầu** (quick diagnostic), không nên coi là bằng chứng đủ để quyết định cắt tỉa trực tiếp. Có thể cần kết hợp với các phương pháp phi tuyến hơn (kernel PCA, autoencoder, hoặc UMAP như ở math 40) để có bức tranh đầy đủ hơn.

Tổng thể, bài này **liên quan gián tiếp nhưng hữu ích** cho dự án: nó không đưa ra một kỹ thuật nén cụ thể nào, nhưng cung cấp công cụ chẩn đoán đơn giản, rẻ về tính toán (so với các phương pháp phức tạp hơn), dễ chạy thử trên GPU nhỏ trước khi đầu tư vào các chiến lược nén phức tạp hơn.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây đều có thể chạy trên một GPU 8GB, dùng PyTorch (hàm `torch.pca_lowrank` hoặc tự viết qua `torch.linalg.eigh`/`torch.linalg.svd`):

1. **Variance-explained curve theo tầng:** lấy hidden state (ví dụ đầu ra sau mỗi block attention/GatedDeltaNet) của một mô hình nhỏ cho một batch vài chục mẫu, chạy PCA, vẽ đường cong phương sai giải thích lũy tích theo số thành phần. Đo: số PC cần để đạt 90%, 95% phương sai. Kỳ vọng: số PC này nhỏ hơn số chiều hidden thực tế, thể hiện một mức độ dư thừa biểu diễn; nếu đường cong gần như tuyến tính (không bão hòa sớm) thì ngược lại, biểu diễn gần như ”full-rank”, ít dư thừa.
2. **So sánh tầng đầu và tầng cuối:** lặp lại thí nghiệm 1 cho tầng gần đầu vào và tầng gần đầu ra, so sánh hai đường cong. Giả thuyết cần kiểm tra: các tầng sâu hơn có xu hướng ”nén” biểu diễn vào không gian chiều thấp hơn (đường cong dốc nhanh hơn) so với tầng đầu.
3. **PCA trên visual token embedding của các trang OCR khác nhau:** chọn vài trang tài liệu khác nhau về mật độ chữ/hình, lấy embedding token hình ảnh trước khi vào decoder, chạy PCA riêng cho mỗi trang. Đo số PC cần giữ 90% phương sai cho mỗi loại trang. Kỳ vọng: trang chữ dày đặc cần nhiều PC hơn (thông tin ”đặc” hơn) so với trang có nhiều khoảng trắng hoặc hình đơn giản.

4. **Nén một lớp Linear bằng chiếu PCA của input activation:** với một lớp Linear cụ thể trong mô hình, tính PCA của activation đầu vào trên một tập dữ liệu mẫu, chiếu input qua k thành phần chính đầu tiên trước khi đưa vào lớp (tương đương một dạng nén low-rank theo activation), đo độ chính xác OCR (hoặc perplexity) khi giảm k dần. Kỳ vọng: có một ngưỡng k mà dưới đó độ chính xác giảm mạnh, đây là thông tin hữu ích cho việc chọn rank khi nén.
5. **Tái tạo lại hai thuật toán `pca1`/`pca2` của Phụ lục B bằng PyTorch/NumPy trên dữ liệu toy 2D/3D có nhiễu Gauss** (mô phỏng lại ví dụ lò xo bằng code, ví dụ sinh dữ liệu dọc theo một trục cộng nhiễu Gauss nhỏ ở các trục khác), xác nhận hai cách (eigen-decomposition của hiệp phương sai và SVD) cho đúng cùng kết quả (các eigenvalue và eigenvector phải khớp, trong giới hạn sai số số học), đồng thời đo thời gian chạy của hai cách khi tăng dần n (số mẫu) để cảm nhận sự khác biệt thực hành giữa hai cách cài đặt.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa
Đổi cơ sở	$Y = PX$, P trực chuẩn ($P^{-1} = P^T$)
Ma trận hiệp phương sai	$C_X = \frac{1}{n} XX^T$ (đường chéo: phương sai; ngoài đường chéo: hiệp phương sai)
Mục tiêu PCA	Tìm P sao cho $C_Y = PC_XP^T = D$ (ma trận đường chéo, các phần tử giảm dần)
Lời giải 1	Các hàng của P là eigenvector của C_X, xếp theo eigenvalue λ_i giảm dần; phương sai theo hướng p_i bằng λ_i
SVD tổng quát	$X = U\Sigma V^T$; với $X\hat{v}_i = \sigma_i \hat{u}_i$, $\sigma_i = \sqrt{\lambda_i}$, λ_i là eigenvalue của X^TX
Lời giải 2 (qua SVD)	Đặt $Z \equiv \frac{1}{\sqrt{n}} X^T$; các cột V trong SVD của Z là các principal component của X
Quy trình thực hành (3 bước)	(1) Tổ chức dữ liệu thành ma trận $m \times n$; (2) trừ trung bình mỗi loại đo; (3) tính SVD hoặc eigenvector của hiệp phương sai
Ví dụ tự tính (mục 3.6)	$C_X = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{pmatrix}$, $\lambda_1 = 4$ ứng với $p_1 = (1/\sqrt{2}, 1/\sqrt{2})$, $\lambda_2 = 1$ ứng với $p_2 = (1/\sqrt{2}, -1/\sqrt{2})$, kiểm tra $\lambda_1 + \lambda_2 = 5 = \sigma_A^2 + \sigma_B^2$

11. Kết luận ngắn

Paper trình bày lại PCA từ gốc, cho thấy toàn bộ kỹ thuật thực chất chỉ là: chọn một phép quay trực chuẩn của hệ trục tọa độ sao cho dữ liệu, biểu diễn trong hệ trục mới, trở nên ”decorrelated” (hiệp phương sai chéo) và được xếp hạng theo phương sai, và lời giải của bài toán đó chính là eigenvector của ma trận hiệp phương sai, hoặc tương đương, các right singular vector trong SVD của dữ liệu đã chuẩn hóa. Đây là một bài tutorial thuần lý thuyết, không có benchmark thực nghiệm, nhưng là nền tảng quan trọng cho nhóm các phương pháp dựa trên eigen-decomposition sẽ gặp lại ở các bài tiếp theo của lộ trình (randomized SVD ở math 04, spectral clustering ở math 05). Giới hạn cốt lõi mà paper tự thừa nhận: PCA chỉ xử lý được cấu trúc tuyến tính, phụ thuộc bậc hai; khi dữ liệu có cấu trúc phi tuyến hoặc phân phối không trực giao/không-Gauss, PCA có thể cho kết quả không phản ánh đúng cấu trúc thật của hệ thống.

Nguồn gốc: https://arxiv.org/abs/1404.1100

Bài 04

Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions (2009)

Tác giả: N. Halko (University of Colorado at Boulder), P. G. Martinsson (University of Colorado at Boulder), J. A. Tropp (California Institute of Technology)
Venue / arXiv: arXiv:0909.4061 (v2, 14/12/2010); paper không nêu venue hội nghị/tạp chí trong phần văn bản chính (chỉ có các trích dẫn tới tạp chí SIAM khác trong danh mục tham khảo, không phải venue của chính bài này)
Nguồn: https://arxiv.org/abs/0909.4061
Chủ đề: Đại số tuyến tính số ngẫu nhiên hoá: tính xấp xỉ ma trận hạng thấp (SVD, phân tích giá trị riêng, QR, interpolative decomposition) bằng lấy mẫu/chiếu ngẫu nhiên
Vị trí trong lộ trình: Bài số 4 trong nhóm 9 bài toán nền (math 01-09). Đọc sau math 03 (PCA/SVD cổ điển, FILE ĐÃ TỒN TẠI), vì bài này chính là cách tính nhanh, ngẫu nhiên hoá cho đúng bài toán SVD/eigen-decomposition đã trình bày ở math 03 khi ma trận quá lớn để tính SVD đầy đủ. Đứng trước math 05 (spectral clustering), nơi cũng cần tính eigenvector của ma trận lớn (Laplacian) một cách hiệu quả.

> Ghi chú này được viết lại bằng tiếng Việt dựa trên toàn văn bài báo gốc (chuyển PDF sang text, 4271 dòng, khoảng 74 trang), không phải bản dịch từng câu. Đây là bài DÀI nhất trong nhóm 9 bài, nên ghi chú tập trung vào phần cốt lõi: toàn bộ Part I (Giới thiệu, mục 1-3) và Part II (Thuật toán, mục 4-7, bao gồm cả mục 7 ”Numerical examples” với các thí nghiệm số thật), còn Part III (Lý thuyết, mục 8-11) chỉ lấy các định lý chính (phát biểu kết quả, không chép lại chi tiết từng bước chứng minh kỹ thuật dài của phần lý thuyết độ tập trung đo). Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Paper mở đầu bằng một trích dẫn của G. W. Stewart: cách tiếp cận ”decompositional” (phân tích ma trận thành các nhân tử như QR, SVD, eigen-decomposition) là một trong 10 thuật toán có ảnh hưởng lớn nhất tới khoa học và kỹ thuật thế kỷ 20, vì nó xây dựng ”nền tính toán” chung cho nhiều bài toán khác nhau, không chỉ giải riêng từng bài toán cụ thể. Tuy nhiên, các thuật toán cổ điển để tính các phân tích này (QR có pivot, SVD, eigen-decomposition) đang gặp khó khi áp dụng cho dữ liệu hiện đại, vì bốn lý do mà paper tự liệt kê:

1. Ma trận trong các ứng dụng hiện đại (data mining) có kích thước ”khủng”.
2. Dữ liệu thường bị thiếu hoặc không chính xác, nên việc tính một phân tích chính xác tuyệt đối tới sai số máy là lãng phí tính toán.
3. Chi phí truyền dữ liệu (data transfer) ngày càng quan trọng hơn số phép toán (flop); các kỹ thuật cần ít lượt đọc dữ liệu (pass) hơn có thể nhanh hơn trong thực tế dù số flop tương đương hoặc nhiều hơn.
4. Kiến trúc phần cứng (ví dụ GPU) đòi hỏi thuật toán số phải thích nghi.

Mục tiêu của paper là chứng minh rằng các thuật toán ngẫu nhiên (randomized algorithms) giải quyết tốt các vấn đề này: nhanh hơn, ổn định hơn (robust hơn) so với thuật toán tất định cổ điển trong nhiều trường hợp, và cho phép đánh đổi độ chính xác với tốc độ. Paper nhấn mạnh ngay từ đầu (Remark 1.1): khác với phương pháp Monte Carlo cổ điển (độ chính xác thấp, không chắc chắn), các thuật toán được trình bày ở đây ”tương đối không nhạy với chất lượng của nguồn ngẫu nhiên” và cho kết quả chính xác cao, với xác suất thất bại có thể làm nhỏ tuỳ ý (ví dụ dưới 10^{-15}) mà chi phí tính toán tăng thêm không đáng kể.

Về cấu trúc: paper có 3 phần chính theo cách paper tự chia (”Outline of paper”, mục 1.7): Part I là Giới thiệu (mục 1-3, gồm tổng quan, lịch sử/công trình liên quan, và kiến thức nền đại số tuyến tính); Part II là Thuật

toán (mục 4-7, trình bày chi tiết các thuật toán cho Stage A và Stage B, kèm mục 7 là các thí nghiệm số thật); Part III là Lý thuyết (mục 8-11, phân tích sai số chặt chẽ bằng cả đại số tuyến tính tất định và xác suất). Ba tác giả thuộc University of Colorado at Boulder (Halko, Martinsson) và Caltech (Tropp), được tài trợ bởi NSF và ONR.

Các từ khóa paper tự liệt kê trong abstract cho thấy phạm vi rộng: dimension reduction, eigenvalue decomposition, interpolative decomposition, Johnson-Lindenstrauss lemma, matrix approximation, parallel algorithm, pass-efficient algorithm, principal component analysis, randomized algorithm, random matrix, rank-revealing QR factorization, singular value decomposition, streaming algorithm. Paper tự mô tả mình là một bài khảo sát (survey) kết hợp với mở rộng các kết quả có sẵn trong văn liệu ("This work surveys and extends recent research").

2. Ý tưởng chính

Ý tưởng trung tâm là chia bài toán tính một phân tích hạng thấp của ma trận A (kích thước $m \times n$) thành hai giai đoạn, gọi là Stage A và Stage B:

- **Stage A:** tính một ma trận Q có các cột trực chuẩn (orthonormal), sao cho $A \approx QQ^*A$. Nói cách khác, cột của Q tạo thành một cơ sở xấp xỉ cho "range" (không gian cột) của A, và ta muốn số cột của Q càng ít càng tốt mà vẫn giữ độ chính xác.
- **Stage B:** dùng Q đã có để tính một phân tích chuẩn (SVD, QR, eigen-decomposition) của A bằng các phương pháp tất định cổ điển, áp dụng lên ma trận đã "thu nhỏ" qua Q.

Điểm mới cốt lõi của paper nằm ở Stage A: thay vì dùng các thuật toán tất định (ví dụ rank-revealing QR hay Krylov subspace), paper dùng lấy mẫu ngẫu nhiên (random sampling) để tìm cơ sở này một cách rất hiệu quả. Trực giác (mục 1.3.2 của paper): nếu A có hạng chính xác k, và ta lấy k vector ngẫu nhiên $\omega^{(i)}$ rồi tính $y^{(i)} = A\omega^{(i)}$, thì vì tính ngẫu nhiên, tập $\{\omega^{(i)}\}$ "nhìn chung" (generic) độc lập tuyến tính và không rơi vào không gian null của A, nên tập $\{y^{(i)}\}$ cũng độc lập tuyến tính và trải khắp toàn bộ range của A. Chỉ cần trực chuẩn hóa các $y^{(i)}$ này là có được cơ sở cho range của A.

Khi $A = B + E$ với B là phần hạng-k "tín hiệu" cần giữ và E là nhiễu nhỏ, lấy đúng k mẫu có thể không đủ (vì E làm lệch hướng các vector mẫu ra khỏi range của B). Giải pháp: lấy thêm một số mẫu dư ra, gọi là tham số lấy dư (oversampling) p, tức lấy $k + p$ mẫu thay vì k. Paper chỉ ra xác suất thất bại giảm siêu mũ (superexponentially) theo p, nên trong thực hành chỉ cần $p = 5$ hoặc $p = 10$ đã cho kết quả rất tốt.

Về hiệu quả tính toán, paper tóm tắt ba kịch bản (mục 1.4), trong đó bài toán mô hình là tìm k thành phần SVD chủ đạo của ma trận $m \times n$:

Kịch bản	Thuật toán cổ điển	Thuật toán ngẫu nhiên
Ma trận dày, vừa bộ nhớ	$O(mnk)$ flop (rank-revealing QR)	$O(mn \log(k))$ flop (dùng ma trận ngẫu nhiên có cấu trúc)
Ma trận thưa/có cấu trúc (tích ma trận-vector nhanh)	Krylov subspace (Lanczos/Arnoldi), chi phí $\sim k\,T_{mult} + (m+n)k^2$	Cùng bậc chi phí $O(k\,T_{mult} + (m+n)k^2)$ nhưng ổn định hơn, dễ song song hóa
Ma trận quá lớn, lưu ngoài bộ nhớ nhanh (out-of-core/streaming)	cần $O(k)$ lượt đọc dữ liệu	chỉ cần một số hằng lượt đọc, có thể chỉ 1 lượt

Đây là ý tưởng mang tính "hạ tầng tính toán": không thay đổi đích đến (vẫn là SVD/QR/eigen-decomposition xấp xỉ), mà thay đổi cách tìm subspace quan trọng ở Stage A để tận dụng kiến trúc máy tính hiện đại (nhân ma trận song song, ít lượt đọc dữ liệu).

3. Phương pháp chi tiết

3.1. Hai bài toán hình thức hóa

Paper hình thức hóa hai biến thể của bài toán xấp xỉ hạng thấp (mục 1.3.1):

- **Fixed-precision problem:** cho ma trận A và sai số cho phép ε, tìm Q với số cột $k = k(\varepsilon)$ ít nhất sao cho $\|A - QQ^*A\| \le \varepsilon$ (chuẩn toán tử ℓ_2). Theo lý thuyết SVD cổ điển, nghiệm tối ưu của bài toán này chính là σ_{j+1} (giá trị kỳ dị thứ $j+1$) khi Q chứa j vector kỳ dị trái chủ đạo.
- **Fixed-rank problem:** cho trước hạng đích k và tham số lấy dư p, tìm Q với $k+p$ cột trực chuẩn sao cho $\|A - QQ^*A\| \approx \min_{\text{rank}(X)\le k} \|A - X\|$.

Hai bài toán này có thể quy đổi qua lại: thuật toán cho fixed-rank có thể chỉnh để giải fixed-precision bằng cách tăng dần số mẫu và theo dõi sai số còn lại (mục 4.3-4.4).

3.2. Proto-Algorithm và Randomized Range Finder (Algorithm 4.1)

Phiên bản đơn giản nhất của thuật toán Stage A (paper gọi là "Proto-Algorithm", sau đó hình thức hóa thành Algorithm 4.1 "Randomized Range Finder"):

1. Vẽ một ma trận kiểm thử (test matrix) ngẫu nhiên Gauss Ω kích thước $n \times \ell$ (với $\ell = k + p$).
2. Tính $Y = A\Omega$ (kích thước $m \times \ell$).
3. Trực chuẩn hóa các cột của Y (ví dụ qua phân tích QR, $Y = QR$) để được Q.

Số flop cần cho thuật toán này: $T_{basic} \sim \ell n T_{rand} + \ell\, T_{mult} + \ell^2 m$, trong đó T_{rand} là chi phí sinh một số ngẫu nhiên Gauss, T_{mult} là chi phí nhân A với một vector. Paper nhận xét thực nghiệm: hiệu năng của thuật toán phụ thuộc rất ít vào chất lượng bộ sinh số ngẫu nhiên; khâu trực chuẩn hóa (bước 3) nên dùng Gram-Schmidt có trực chuẩn hóa kép (double orthogonalization) hoặc phép phản xạ Householder/phép quay Givens vì cột của Y thường gần phụ thuộc tuyến tính (ill-conditioned).

3.3. Ước lượng sai số và thuật toán thích nghi (Algorithm 4.2)

Để giải bài toán fixed-precision (khi hạng số k chưa biết trước), paper phát triển một ước lượng sai số xác suất, rẻ về tính toán, dựa trên Lemma 4.1: với B là ma trận thực $m \times n$, r là số nguyên dương, $\alpha > 1$, và $\{\omega^{(i)}\}_{i=1}^{r}$ là các vector Gauss chuẩn độc lập, thì

$$\|B\| \le \alpha \sqrt{\frac{2}{\pi}} \max_{i=1,\dots,r} \|B\omega^{(i)}\|$$

trừ với xác suất α^{-r}. Áp dụng với $B = (I - QQ^*)A$ và $\alpha = 10$, ta có công cụ kiểm tra xem Q hiện tại đã đủ tốt chưa mà chỉ cần vài tích ma trận-vector, không cần tính lại sai số đầy đủ. Dựa trên đó, Algorithm 4.2 ("Adaptive Randomized Range Finder") xây dựng Q tăng dần theo từng cột, dừng khi r vector mẫu liên tiếp có chuẩn đủ nhỏ, đạt sai số mong muốn với xác suất ít nhất $1 - \min\{m,n\} \cdot 10^{-r}$.

3.4. Lặp luỹ thừa (power iteration) khi phổ giá trị kỳ dị suy giảm chậm

Nhược điểm của Algorithm 4.1: nếu các giá trị kỳ dị của A suy giảm chậm (phổ "phẳng"), các vector kỳ dị tương ứng giá trị nhỏ vẫn "chen vào" mẫu ngẫu nhiên và làm giảm chất lượng cơ sở Q. Giải pháp (mục 4.5, Algorithm 4.3 "Randomized Power Iteration"): áp dụng thuật toán cho ma trận $B = (AA^*)^q A$ với q là một số nguyên nhỏ (thường $q = 1$ hoặc 2), thay vì áp dụng trực tiếp cho A. Ma trận B có cùng vector kỳ dị với A nhưng giá trị kỳ dị suy giảm nhanh hơn rất nhiều: $\sigma_j(B) = \sigma_j(A)^{2q+1}$. Heuristic của paper: nếu thuật toán cơ bản cho sai số trong khoảng C lần tối ưu, thì phiên bản lũy thừa cho sai số trong khoảng $C^{1/(2q+1)}$ lần tối ưu, tức khoảng cách tới tối ưu giảm theo cấp số mũ khi q tăng.

Paper cảnh báo (Remark 4.3): khi thực hiện trong số học dấu phẩy động, việc nhân lặp A, A^* nhiều lần sẽ làm mất hết thông tin về các mode kỳ dị nhỏ do lỗi làm tròn (nếu độ chính xác máy là μ, thông tin của các

giá trị kỳ dị nhỏ hơn $\mu^{1/(2q+1)}\|A\|$ sẽ bị "xóa"). Khắc phục bằng cách trực chuẩn hóa lại sau mỗi lần nhân A hoặc A^*, cho ra Algorithm 4.4 ("Randomized Subspace Iteration"), được paper khuyến nghị dùng trong thực hành vì chi phí tương đương Algorithm 4.3 nhưng chính xác hơn nhiều trong số học thực tế.

3.5. Ma trận ngẫu nhiên có cấu trúc (SRFT) để tăng tốc

Nút nghẽn tính toán của Algorithm 4.1 là tích $A\Omega$. Nếu Ω là ma trận Gauss chuẩn, chi phí này là $O(mn\ell)$, không hơn gì rank-revealing QR cổ điển. Paper đề xuất dùng ma trận ngẫu nhiên có cấu trúc, ví dụ Subsampled Random Fourier Transform (SRFT):

$$\Omega = \sqrt{\frac{n}{\ell}}\, DFR$$

với D là ma trận đường chéo ngẫu nhiên (các phần tử trên vòng tròn đơn vị phức), F là biến đổi Fourier rời rạc unitary $n \times n$, R là ma trận chọn ngẫu nhiên ℓ trong n cột của ma trận đơn vị. Vì có thể tính $A\Omega$ qua FFT lấy mẫu con (subsampled FFT), chi phí giảm xuống $O(mn\log(\ell))$. Đây là Algorithm 4.5 ("Fast Randomized Range Finder").

3.6. Stage B: từ cơ sở Q tới phân tích chuẩn

Khi đã có Q với $\|A - QQ^*A\| \le \varepsilon$, paper trình bày nhiều cách để lấy ra SVD/eigen-decomposition cụ thể:

- **Direct SVD (Algorithm 5.1):** tính $B = Q^*A$ (ma trận nhỏ $\ell \times n$), lấy SVD của B là $B = \tilde{U}\Sigma V^*$, rồi $U = Q\tilde{U}$. Sai số không suy giảm so với sai số của Q: $\|A - U\Sigma V^*\| \le \varepsilon$.
- **Row extraction (Algorithm 5.2):** dùng interpolative decomposition của Q để trích ra k hàng của chính A mà không cần tính tích ma trận-ma trận nào, nhanh hơn nhưng kém chính xác hơn Algorithm 5.1.
- **Biến thể cho ma trận Hermitian/positive semidefinite (Algorithm 5.3-5.5, trong đó có phương pháp Nyström)** và **thuật toán một lượt đọc (single-pass, Algorithm 5.6)** cho trường hợp không thể quay lại đọc A lần hai (dữ liệu stream).

Tổng hợp Stage A (với power iteration) + Stage B (Direct SVD) cho ra "Prototype for Randomized SVD" mà paper trình bày ngay từ phần giới thiệu:

1. Sinh ma trận kiểm thử Gauss Ω kích thước $n \times 2k$.
2. $Y = (AA^*)^q A\Omega$ (nhân xen kẽ A và A^*).
3. Trực chuẩn hóa Y để được Q.
4. $B = Q^*A$.
5. SVD của B: $B = \tilde{U}\Sigma V^*$.
6. $U = Q\tilde{U}$.

Chi phí: $T_{randSVD} = (2q+2)k\,T_{mult} + O(k^2(m+n))$.

3.7. Ví dụ tự tính: minh hoạ cơ chế "lấy mẫu ngẫu nhiên tìm range" trên ma trận hạng 1

Để minh họa cụ thể trực giác ở mục 1.3.2 của paper (nếu A có hạng chính xác k, lấy k mẫu đủ để trải khắp range của A), xét ma trận hạng 1 đơn giản, tự chọn (không lấy từ paper), kích thước 3×2:

$$A = \begin{pmatrix} 1 & 1 \\ 2 & 2 \\ 2 & 2 \end{pmatrix}$$

Đây chính là $A = xy^T$ với $x = (1,2,2)^T$, $y = (1,1)^T$, nên $\text{rank}(A) = 1$ và $\text{range}(A) = \text{span}\{x\}$. Để tính tay dễ hiểu, thay cho vector Gauss thật, dùng hai vector kiểm thử cụ thể $\omega^{(1)} = (1,0)^T$ và $\omega^{(2)} = (0,1)^T$ (đóng vai trò minh họa cơ chế, không phải mẫu Gauss thật theo đúng Algorithm 4.1):

$$y^{(1)} = A\omega^{(1)} = (1,2,2)^T = x, \qquad y^{(2)} = A\omega^{(2)} = (1,2,2)^T = x$$

Cả hai mẫu đều cho đúng x (vì A hạng 1, mọi cột của A đều là một hằng số nhân với x). Trực chuẩn hóa $y^{(1)}$: $q = x/\|x\| = (1,2,2)^T/3 = (1/3,\ 2/3,\ 2/3)^T$. Đây chính xác là một cơ sở trực chuẩn cho range của A, đạt được chỉ với một mẫu (vì hạng đúng là 1, không cần lấy dư $p > 0$ với ví dụ "sạch", không nhiễu này). Điều này khớp chính xác với phát biểu của paper ở mục 1.3.2: "to produce an orthonormal basis for the range of A, we just need to orthonormalize the sample vectors" khi A có hạng chính xác k (tự tính, minh họa cho trường hợp lý tưởng không nhiễu; paper tự nói rõ trường hợp có nhiễu $A = B + E$ cần lấy dư thêm p mẫu, không được minh họa số cụ thể ở đây vì cần ma trận phức tạp hơn).

4. Thí nghiệm và kết quả chính

4.1. Kết quả lý thuyết chính

Paper phát biểu ngay từ phần giới thiệu hai định lý trung tâm (các bản đơn giản hóa của Theorem 10.6 và Corollary 10.10 ở Part III):

Theorem 1.1 (sai số trung bình của Proto-Algorithm với ma trận Gauss): với A là ma trận thực $m \times n$, hạng đích $k \geq 2$, tham số lấy dư $p \geq 2$, $k + p \leq \min\{m, n\}$, thực hiện proto-algorithm với ma trận kiểm thử Gauss chuẩn để được Q ($m \times (k+p)$ cột trực chuẩn), thì

$$\mathbb{E}\|A - QQ^*A\| \leq \left(1 + \frac{4\sqrt{k+p}}{p-1}\sqrt{\min\{m,n\}}\right)\sigma_{k+1}$$

trong đó σ_{k+1} là giá trị kỳ dị thứ $k+1$ của A (sai số nhỏ nhất có thể đạt được bằng bất kỳ cơ sở nào). Với $p = 5$, với xác suất ít nhất $1 - 6p^{-p}$, sai số thực tế nằm trong khoảng $\left(1 + 11\sqrt{k+p}\sqrt{\min\{m,n\}}\right)\sigma_{k+1}$, cho thấy chỉ cần lấy dư $p = 5$ là đủ dùng được trong thực hành.

Theorem 1.2 (Randomized SVD với power iteration): với ma trận A, số mũ q, hạng đích k ($2 \leq k \leq 0.5\min\{m,n\}$), chạy thuật toán Randomized SVD (phần 3.6 ở trên) để được phân tích hạng $2k$, $U\Sigma V^*$, thì

$$\mathbb{E}\|A - U\Sigma V^*\| \leq \left[1 + 4\sqrt{\frac{2\min\{m,n\}}{k-1}}\right]^{1/(2q+1)}\sigma_{k+1}$$

Đây là kết quả mà paper tự nhận "is new" (chưa có trong các công trình trước): hệ số nhân trong ngoặc bị đẩy về 1 theo cấp số mũ khi q tăng.

Phần lý thuyết chi tiết (Part III, mục 9) cung cấp một định lý tất định tổng quát hơn (Theorem 9.1), chỉ phụ thuộc vào tương tác giữa ma trận kiểm thử Ω và các vector kỳ dị phải của A:

$$\|(I - P_Y)A\|^2 \leq \|\Sigma_2\|^2 + \|\Sigma_2\Omega_2\Omega_1^\dagger\|^2$$

(với Σ_1, Σ_2 là khối giá trị kỳ dị lớn/nhỏ, Ω_1, Ω_2 là Ω biểu diễn trong hệ trục kỳ dị phải). Theorem 9.2 chỉ ra power scheme cải thiện sai số theo lũy thừa $1/(2q+1)$; Theorem 9.3 chỉ ra bước cắt (truncation) về đúng hạng k chỉ làm tăng sai số thêm tối đa σ_{k+1}.

4.2. Thí nghiệm số thật (mục 7 của paper, cuối Part II)

Khác với math 01/03 (lý thuyết thuần, không benchmark), paper này có mục 7 "Numerical examples" với **nhiều thí nghiệm số thật**, chạy trên các ma trận cụ thể:

- **Toán tử tích phân Laplace (mục 7.1):** ma trận 200×200 rời rạc hóa một toán tử lớp đơn (single-layer potential) cho phương trình Laplace, phổ giá trị kỳ dị suy giảm nhanh. Chạy Algorithm 4.2 trong Matlab v6.5. Hình 7.2 cho thấy sai số thực tế e_ℓ rất gần sai số tối thiểu lý thuyết $\sigma_{\ell+1}$, và ước lượng sai số rẻ f_ℓ luôn là chặn trên của sai số thực, bảo thủ (pessimistic) khoảng 10 lần. Lặp lại 2000 lần (Hình 7.3) xác nhận hành vi này ổn định qua các lần thử.
- **Ma trận Laplace rời rạc trên lưới điện trở (mục 7.1 tiếp):** ma trận 1596×532 định nghĩa ngầm (chỉ truy cập qua tích ma trận-vector, mỗi lần cần giải một hệ thưa $\sim 140{,}000 \times 140{,}000$), cho kết quả định tính tương tự (Hình 7.4).
- **Ma trận Laplacian đồ thị từ ảnh (mục 7.2):** từ ảnh grayscale 95×95 pixel, xây ma trận trọng số 9025×9025 (dùng kernel Gauss trên các patch 5×5), rồi Laplacian chuẩn hóa. Dùng power scheme Algorithm 4.3 với $q = 0, 1, 2, 3$: Hình 7.5 cho thấy $q = 0$ (không lặp lũy thừa) cho kết quả xấp xỉ eigenvalue rất kém, nhưng $q = 1$ đã cải thiện rõ rệt, các eigenvalue lớn đã khá chính xác.
- **Eigenfaces (mục 7.3):** ma trận $98{,}304 \times 7254$ từ 7254 ảnh khuôn mặt (cơ sở dữ liệu FERET), cần 5.4 GB nếu lưu dạng số thực kép, không vừa bộ nhớ nhanh. Dùng power scheme pass-efficient, ước lượng sai số qua kỹ thuật Remark 4.1 (vì không thể tính sai số thật ở quy mô này). Hình 7.6: $q = 0$ cho kết quả rất kém, cải thiện nhanh khi q tăng, hầu như không cải thiện thêm sau 3-5 lượt đọc dữ liệu.
- **So sánh tốc độ với ma trận ngẫu nhiên có cấu trúc (mục 7.4, Bảng 7.1):** so sánh 4 phương pháp (`direct`: rank-revealing QR cổ điển, `gauss`: Algorithm 4.1 với Gauss, `srft`: Algorithm 4.1 với SRFT, `svd`: SVD đầy đủ LAPACK `dgesdd`) trên ma trận Gauss ngẫu nhiên $n \times n$, đo thời gian thật (giây) trên một máy "office desktop" Pentium IV 3.2 GHz, 2 GB RAM. Một vài số liệu trích từ Bảng 7.1 (tự chọn trích, số liệu thật của paper): với $n = 4096$, $\ell = 320$: `direct` $= 5.00 \times 10^1$ giây, `gauss` $= 2.41 \times 10^1$ giây, `srft` $= 1.21 \times 10^1$ giây; SVD đầy đủ cho $n = 4096$ tốn 6.90×10^2 giây. Hình 7.7 (acceleration factor) cho thấy SRFT thường tăng tốc mạnh so với `direct`, Gauss tăng tốc vừa phải.

Paper tự nhấn mạnh: các ví dụ ở mục 7 chỉ ở quy mô vừa phải; "Computational examples illustrating truly large data matrices have been reported elsewhere" (dẫn một công trình khác [112]), tức benchmark ở quy mô siêu lớn không nằm trong chính bài báo này.

5. Phân tích / ablation

Ảnh hưởng của số bước lặp lũy thừa q (ablation có sẵn trong paper): cả hai thí nghiệm ở mục 7.2 (Laplacian đồ thị) và 7.3 (Eigenfaces) đều là một dạng ablation theo $q = 0, 1, 2, 3$. Kết luận nhất quán: $q = 0$ (không lặp lũy thừa) luôn cho kết quả kém khi phổ giá trị kỳ dị suy giảm chậm; chỉ cần $q = 1$ hoặc $q = 2$ đã cải thiện mạnh; tăng q quá mức không mang lại lợi ích đáng kể thêm (Eigenfaces: "hầu như không cải thiện thêm sau 3-5 lượt đọc"). Đây là bằng chứng thực nghiệm trực tiếp cho Theorem 9.2 (sai số giảm theo lũy thừa $1/(2q+1)$ khi q tăng).

Phân tích sâu hơn Bảng 7.1: SRFT không phải luôn thắng. Khi đọc kỹ toàn bộ Bảng 7.1, có một quan sát quan trọng mà phần văn bản chính của paper không nhấn mạnh rõ: ở $n = 2048$, $\ell = 1280$ (tức ℓ gần bằng n, khoảng 62%), `srft` mất 3.13×10^2 giây, CHẬM HƠN cả `direct` (7.97×10^1 giây)! Tương tự ở $n = 4096$, $\ell = 1280$: `srft` $= 4.03 \times 10^2$ giây, cũng chậm hơn `direct` (2.40×10^2 giây). Đây là **quan sát của người ghi chú dựa trên số liệu thật trong Bảng 7.1 của paper**, không phải khẳng định mà paper tự nêu rõ bằng lời: khi tỉ lệ ℓ/n đủ lớn (ở đây ℓ chiếm hơn một nửa n), phương pháp SRFT (được quảng cáo là nhanh hơn nhờ FFT lấy mẫu con) lại trở nên chậm hơn phương pháp tất định cổ điển. Điều này cho thấy tuyên bố tổng quát trong abstract ("randomized algorithms... beat its classical competitors in terms of accuracy, speed, and robustness") cần hiểu có điều kiện: lợi thế tốc độ của SRFT chủ yếu xảy ra khi $\ell \ll n$ (hạng đích nhỏ hơn nhiều so với kích thước ma trận), đúng với giả định "numerical rank thấp" mà toàn bộ phương pháp dựa vào, nhưng không phải lúc nào cũng đúng.

Ước lượng sai số luôn bảo thủ, không chỉ đúng trung bình: Hình 7.3 (2000 lần thử) cho thấy ước lượng f_ℓ luôn là chặn trên của sai số thật e_ℓ, lệch khoảng hệ số 10, đúng như thiết kế lý thuyết (hệ số an toàn $\alpha = 10$

trong Lemma 4.1). Đây là một đánh đổi thiết kế có chủ ý: người dùng ”trả giá” bằng việc có thể lấy nhiều mẫu hơn mức tối thiểu cần thiết, để đổi lại một cam kết xác suất chắc chắn hơn.

So sánh phân phối ma trận kiểm thử (mục 7.4, Hình 7.8): trên cùng ma trận Laplace 200×200, so 4 phân phối cho Ω (Gauss, Ortho tức ma trận trực chuẩn ngẫu nhiên đều, SRFT, GSRFT) qua 100.000 lần thử. Kết quả: SRFT thực ra hơi tốt hơn Gauss trong ví dụ cụ thể này; SRFT và GSRFT cho sai số gần như giống nhau; Ortho chỉ tốt hơn rõ rệt khi ma trận kiểm thử gần vuông (không tốt hơn đáng kể trong 3/4 trường hợp còn lại). Đây là bằng chứng thực nghiệm cho thấy lựa chọn phân phối ngẫu nhiên cụ thể ít quan trọng hơn so với việc có lấy mẫu ngẫu nhiên hay không.

6. Giới hạn

Các giới hạn **paper tự nêu rõ**:

- Thuật toán cơ bản (Algorithm 4.1, không lặp lũy thừa) hoạt động kém khi phổ giá trị kỳ dị của A suy giảm chậm; bắt buộc phải dùng power iteration (Algorithm 4.3/4.4), làm tăng số lượt đọc dữ liệu lên $2q + 1$ lần.
- Power iteration thực hiện trong số học dấu phẩy động dễ mất thông tin về các mode kỳ dị nhỏ do lỗi làm tròn (Remark 4.3); phải trực chuẩn hóa lại giữa các bước (Algorithm 4.4) để khắc phục, với chi phí gần như không đổi nhưng cần thêm bước triển khai.
- Ma trận ngẫu nhiên có cấu trúc (SRFT) không thích nghi tốt với bài toán fixed-precision, vì mẫu thường phải tính hàng loạt (Remark 4.4); và paper tự nói rõ ”it is not currently known whether such an iterative scheme can be accelerated to $O(mn \log(k))$ complexity using ’fast’ random transforms such as the SRFT” (Remark 4.7, câu hỏi mở tại thời điểm viết bài).
- Krylov subspace method (đối thủ cổ điển cho ma trận thưa/có cấu trúc) ”đôi khi” (sometimes) vẫn cho kết quả chính xác hơn phương pháp ngẫu nhiên với cùng ngân sách flop, mặc dù phương pháp ngẫu nhiên ổn định hơn và dễ song song hóa hơn (mục 1.4.2, paper tự nêu đây không phải phương pháp ngẫu nhiên luôn thắng tuyệt đối).
- Row extraction (Algorithm 5.2) nhanh hơn Direct SVD (Algorithm 5.1) nhưng kém chính xác hơn; paper khuyến nghị dùng Algorithm 5.1 trừ khi tốc độ quan trọng hơn (mục 5.2).
- Các thí nghiệm số trong chính bài báo chỉ ở quy mô vừa (lớn nhất là ma trận $98{,}304 \times 7254$, 5.4 GB); các khẳng định về ”truly massive data sets” dựa một phần vào trích dẫn các công trình khác, không phải thí nghiệm trực tiếp trong bài này.

Quan sát **của người ghi chú** (không phải do paper tự phê bình bằng lời, nhưng là đặc điểm khách quan rút ra từ chính dữ liệu của paper, như đã nêu ở mục 5): trong Bảng 7.1, khi ℓ/n lớn (ví dụ $\ell = 1280$, $n = 2048$ hoặc 4096), phương pháp `srft` chậm hơn cả phương pháp tất định `direct`, một trường hợp mà tuyên bố tổng quát ”randomized beats classical” của paper không áp dụng.

7. Liên hệ với các bài khác

- **math 03 (A Tutorial on Principal Component Analysis, Shlens, FILE ĐÃ TỒN TẠI):** bài này mở rộng trực tiếp lời giải SVD của PCA trình bày ở math 03. Khi ma trận dữ liệu X quá lớn để tính SVD đầy đủ, các thuật toán ở đây (Randomized Range Finder, Randomized SVD) cho một cách tính PCA xấp xỉ nhanh hơn nhiều, với sai số kiểm soát được theo Theorem 1.1/1.2.
- **math 05 (A Tutorial on Spectral Clustering, von Luxburg):** cần tính các eigenvector của ma trận Laplacian của đồ thị similarity. Thí nghiệm ở chính mục 7.2 của paper này (ma trận Laplacian đồ thị 9025×9025 từ ảnh) là một ví dụ trực tiếp của đúng loại bài toán mà math 05 cần giải, chỉ khác là ở đây Laplacian xây từ patch ảnh cho khử nhiễu, còn math 05 xây Laplacian từ similarity graph cho phân cụm.
- **opt 14 (Shampoo: Preconditioned Stochastic Tensor Optimization):** liên hệ gián tiếp, suy luận của người ghi chú (không phải nội dung của paper này hay của Shampoo nêu rõ): Shampoo cần tính nghịch

đảo căn bậc hai của các ma trận preconditioner lớn cho mỗi lớp; về nguyên tắc, các kỹ thuật xấp xỉ hạng thấp ngẫu nhiên ở đây có thể dùng để giảm chi phí tính toán này, nhưng đây là một liên hệ khái niệm, không phải điều hai bài báo trực tiếp thảo luận.

- **math 19 (Measuring the Intrinsic Dimension of Objective Landscapes):** cùng mạch "tìm hạng/chiều hiệu dụng thấp của một đối tượng lớn" như ở math 03, nhưng math 19 áp dụng cho không gian tham số mạng neural bằng một phương pháp hoàn toàn khác (random subspace training), không dùng SVD ngẫu nhiên.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet kết hợp full attention, khoảng 3600 token/trang, GPU không quá 8GB, batch size 1). Các liên hệ dưới đây là **suy luận của người ghi chú**, không phải nội dung của paper:

- **Chẩn đoán hạng hiệu dụng của weight matrix nhanh hơn SVD đầy đủ:** với GPU 8GB, tính SVD đầy đủ của một ma trận trọng số lớn (ví dụ ma trận chiếu trong attention hay FFN) có thể tốn thời gian/bộ nhớ đáng kể. Dùng Randomized SVD (Algorithm trong mục 3.6, chỉ cần vài tích ma trận-vector/ma trận-ma trận với Ω nhỏ) có thể ước lượng nhanh phổ giá trị kỳ dị và hạng hiệu dụng của từng ma trận trọng số, hỗ trợ quyết định nên cắt tỉa/nén low-rank lớp nào trước, đúng tinh thần độ phức tạp $O(mn \log k)$ so với $O(mnk)$ mà paper nêu.
- **Chỉ cần tích ma trận-vector, phù hợp khi muốn tránh load toàn bộ ma trận lớn vào bộ nhớ GPU hạn chế:** vì Algorithm 4.1/4.3 chỉ cần $A\Omega$ (và $A^*\Omega'$ cho power iteration), có thể áp dụng cho các phép biến đổi tuyến tính lớn mà không cần vật chất hóa toàn bộ ma trận, một lợi thế đáng chú ý khi làm việc trong giới hạn 8GB và batch size 1.
- **Giới hạn cần lưu ý khi áp dụng suy luận trên:** các ma trận trọng số của một mô hình 0.8B tham số thường không quá lớn theo chuẩn "ma trận khủng" mà paper hướng tới (paper thử nghiệm tới quy mô $98{,}304 \times 7254$ hay lớn hơn); ở quy mô nhỏ hơn, chi phí cố định của việc sinh ma trận ngẫu nhiên và thực hiện power iteration (cần nhiều lượt nhân ma trận) có thể không mang lại lợi ích tốc độ rõ rệt so với dùng trực tiếp `torch.linalg.svd` hoặc `svd_lowrank` có sẵn, đặc biệt nếu không cần lặp đi lặp lại việc này nhiều lần. Đây là nhận định cần đo thực nghiệm cụ thể (mục 9 dưới đây), không nên giả định tốc độ luôn thắng chỉ vì lý thuyết độ phức tạp tốt hơn, đúng như chính quan sát ở mục 5 (Bảng 7.1) cho thấy lợi thế SRFT không phải luôn đúng.

9. Thí nghiệm gợi ý

1. **So sánh Randomized SVD với SVD đầy đủ trên một weight matrix thật:** lấy một ma trận trọng số (ví dụ lớp Linear của một mô hình nhỏ, hoặc chính một ma trận trong LightOnOCR-3 nếu khả thi) trên GPU 8GB, cài Algorithm trong mục 3.6 bằng PyTorch (hoặc dùng `torch.svd_lowrank` sẵn có) với các $q = 0, 1, 2$, so với `torch.linalg.svd` đầy đủ. Đo thời gian chạy và sai số Frobenius $\|A - U\Sigma V^*\|_F$ theo hạng k. Kỳ vọng: thời gian randomized tăng chậm hơn nhiều theo kích thước ma trận; sai số giảm về gần σ_{k+1} khi q tăng, khớp mẫu hình của Theorem 9.2.
2. **Ablation theo số bước lặp lũy thừa q:** lặp lại thí nghiệm kiểu Hình 7.5/7.6 của paper nhưng trên một weight matrix thật của mô hình (thay cho ảnh/Eigenfaces), đo sai số xấp xỉ giá trị kỳ dị lớn nhất với $q = 0, 1, 2, 3$. Kỳ vọng: $q = 0$ cho kết quả rõ ràng kém hơn, cải thiện mạnh ngay ở $q = 1$, giống mẫu hình quan sát được ở hai thí nghiệm thật của paper.
3. **Ablation theo tham số lấy dư p:** cố định hạng đích k, thay đổi $p = 0, 5, 10, 20$, đo sai số xấp xỉ. Kỳ vọng: sai số giảm nhanh khi tăng p từ 0 lên 5-10, sau đó lợi ích giảm dần (diminishing returns), khớp khẳng định "rarely any advantage to select $p > k$" của paper.
4. **Thử nghiệm ma trận ngẫu nhiên có cấu trúc ở quy mô mô hình nhỏ:** so sánh thời gian chạy giữa Gauss và một biến thể cấu trúc đơn giản (ví dụ random sign matrix kết hợp FFT/Hadamard gần giống SRFT) trên một ma trận kích thước thực tế của mô hình (ví dụ 2048×2048 hoặc hidden size tương tự),

với ℓ (hạng đích + lấy dư) ở các tỉ lệ khác nhau so với n (ví dụ $\ell/n = 0.05, 0.3, 0.6$). Kỳ vọng kiểm chứng quan sát ở mục 5: SRFT nhanh khi ℓ/n nhỏ nhưng có thể chậm hơn phương pháp tất định khi ℓ/n lớn.

5. **Kiểm tra độ tin cậy của ước lượng sai số rẻ (Lemma 4.1 kiểu):** với một ma trận đủ nhỏ để tính được sai số thật $\|A - QQ^*A\|$ chính xác, so sánh với ước lượng xác suất rẻ (dùng vài vector dò ngẫu nhiên). Đo tỉ lệ giữa ước lượng và sai số thật qua nhiều lần thử. Kỳ vọng: ước lượng luôn là chặn trên, bảo thủ khoảng một hệ số không đổi, giống quan sát ở Hình 7.3 của paper (hệ số khoảng 10).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / mô tả
Khung 2 giai đoạn	Stage A: tìm Q trực chuẩn với $A \approx QQ^*A$. Stage B: tính phân tích chuẩn từ Q
Proto-Algorithm / Algorithm 4.1	Ω ngẫu nhiên $n \times \ell$; $Y = A\Omega$; $Q =$ cơ sở trực chuẩn của Y
Oversampling	$\ell = k + p$; $p = 5$ hoặc 10 thường đủ dùng
Power iteration	$B = (AA^*)^q A$; $\sigma_j(B) = \sigma_j(A)^{2q+1}$; cần trực chuẩn hóa giữa các bước (Algorithm 4.4) để tránh mất thông tin do làm tròn
SRFT	$\Omega = \sqrt{n/\ell}\, DFR$; cho phép tính $A\Omega$ trong $O(mn \log \ell)$ qua FFT lấy mẫu con
Theorem 1.1	$\mathbb{E}\|A - QQ^*A\| \le \left(1 + \frac{4\sqrt{k+p}}{p-1}\sqrt{\min\{m,n\}}\right)\sigma_{k+1}$
Theorem 1.2 (Randomized SVD)	$\mathbb{E}\|A - U\Sigma V^*\| \le \left[1 + 4\sqrt{2\min\{m,n\}/(k-1)}\right]^{1/(2q+1)} \sigma_{k+1}$
Chi phí dày đặc	Cổ điển $O(mnk)$ so với ngẫu nhiên có cấu trúc $O(mn \log k)$
Ví dụ tự tính (mục 3.7)	A hạng 1 (3×2); mẫu $A\omega$ cho đúng hướng $x = (1,2,2)^T$; trực chuẩn hóa còn $q = (1/3, 2/3, 2/3)^T$, khớp range(A) chỉ với 1 mẫu
Quan sát từ Bảng 7.1 (người ghi chú)	SRFT nhanh khi $\ell \ll n$ nhưng có thể chậm hơn `direct` khi ℓ/n lớn (ví dụ $\ell = 1280$, $n = 2048$ hoặc 4096)

11. Kết luận ngắn

Paper trình bày một khung hai giai đoạn để tính xấp xỉ hạng thấp của ma trận lớn: dùng lấy mẫu ngẫu nhiên (random projection) để tìm nhanh một subspace nắm bắt phần lớn ”hành động” của ma trận (Stage A), rồi dùng các phương pháp tất định cổ điển trên ma trận đã thu nhỏ để lấy ra SVD/QR/eigen-decomposition cụ thể (Stage B). Đóng góp chính không chỉ là ý tưởng (đã có từ trước trong một số công trình riêng lẻ) mà là tổng hợp, mở rộng, và đặc biệt là phân tích sai số chặt chẽ (cả tất định và xác suất) cùng một lượng thí nghiệm số thực tế đáng kể trên nhiều loại ma trận khác nhau (toán tử tích phân, Laplacian đồ thị, cơ sở dữ liệu khuôn mặt). Bài học thực hành quan trọng nhất: khi phổ giá trị kỳ dị suy giảm chậm, phải kết hợp power iteration; khi hạng đích k nhỏ hơn nhiều kích thước ma trận, ma trận ngẫu nhiên có cấu trúc (SRFT) mang lại tốc độ vượt trội, nhưng lợi thế này không phải tuyệt đối, như chính số liệu Bảng 7.1 của paper cho thấy ở các tỉ lệ ℓ/n lớn. Đây là bài quan trọng để hiểu cách các ý tưởng ở math 03 (PCA/SVD) có thể mở rộng sang quy mô lớn, và là bước đệm kỹ thuật cho các phương pháp eigen-based khác trong lộ trình (math 05).

Nguồn gốc: https://arxiv.org/abs/0909.4061

Bài 05

A Tutorial on Spectral Clustering (2007)

Tác giả: Ulrike von Luxburg (Max Planck Institute for Biological Cybernetics, Tübingen, Đức)
Venue / arXiv: arXiv:0711.0189 (v1, 01/11/2007); paper tự nêu rõ venue ngay trang đầu: ”This article appears in Statistics and Computing, 17(4), 2007”
Nguồn: https://arxiv.org/abs/0711.0189
Chủ đề: Đại số tuyến tính ứng dụng, phân cụm phổ (spectral clustering) dựa trên eigen-decomposition của ma trận Laplacian của đồ thị tương tự (similarity graph)
Vị trí trong lộ trình: Bài số 5 trong nhóm 9 bài toán nền (math 01-09). Đọc sau math 03 (PCA) và math 04 (randomized SVD), cùng thuộc nhóm con ”phương pháp dựa trên eigenvector/SVD” của batch, nhưng áp dụng eigen-decomposition cho một ma trận khác (Laplacian của đồ thị) thay vì ma trận hiệp phương sai dữ liệu. Đây là bài cuối của nhóm con này, mở đường sang nhóm ”hình học thông tin” ở math 06-07.

> Ghi chú này được viết lại bằng tiếng Việt dựa trên toàn văn bài báo gốc (đọc từ bản PDF chuyển sang text, bao gồm cả phần tài liệu tham khảo để xác nhận các khẳng định trích dẫn), không phải bản dịch từng câu. Mục tiêu là giải thích lại ý tưởng cho người mới bắt đầu, có bổ sung các ví dụ tính tay (đánh dấu ”tự tính”) để kiểm chứng công thức. Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Clustering (phân cụm) là một trong những kỹ thuật phân tích dữ liệu khám phá (exploratory data analysis) được dùng rộng rãi nhất, từ thống kê, khoa học máy tính, sinh học đến khoa học xã hội. Bài toán cơ bản: cho một tập điểm dữ liệu, tìm cách chia chúng thành các nhóm sao cho các điểm trong cùng nhóm ”giống nhau”, còn các điểm ở nhóm khác ”khác nhau”. Paper mở đầu bằng nhận xét: so với các thuật toán truyền thống như k-means hay single linkage, spectral clustering (phân cụm phổ) thường cho kết quả tốt hơn, dễ cài đặt, và có thể giải hiệu quả bằng các công cụ đại số tuyến tính tiêu chuẩn, nhưng ”on the first glance spectral clustering appears slightly mysterious, and it is not obvious to see why it works at all and what it really does”. Mục tiêu của tutorial là giải thích trực giác đằng sau phương pháp này.

Khác với math 03 (PCA tutorial, không có danh mục tài liệu tham khảo), bài này có danh mục tham khảo đầy đủ với hơn 60 trích dẫn, kể cả phần ”Outlook and further reading” (mục 9) tổng kết lịch sử: spectral clustering bắt nguồn từ Donath và Hoffman (1973, dùng eigenvector của ma trận kề để phân hoạch đồ thị) và Fiedler (1973, phát hiện mối liên hệ giữa phân hoạch đồ thị và eigenvector thứ hai của Laplacian), rồi được phổ biến trong cộng đồng machine learning qua các công trình của Shi và Malik (2000), Ng, Jordan và Weiss (2002), Meila và Shi (2001).

Paper tự nhận đây là ”a self-contained introduction”, không yêu cầu kiến thức toán nào ngoài đại số tuyến tính cơ bản, nhưng cũng tự nhận không cố gắng tổng kết toàn bộ văn liệu (literature) về spectral clustering vì khối lượng quá lớn. Cấu trúc: mục 2-3 giới thiệu các đối tượng toán học (similarity graph, graph Laplacian); mục 4 trình bày các thuật toán; mục 5-7 giải thích tại sao thuật toán hoạt động theo ba góc nhìn khác nhau (graph cut, random walk, perturbation theory); mục 8 bàn các vấn đề thực hành; mục 9 tổng kết lịch sử và mở rộng.

Ghi chú này đọc toàn bộ văn bản (1866 dòng sau khi chuyển PDF, khoảng 32 trang kể cả tài liệu tham khảo), bao gồm cả các mệnh đề/chứng minh ngắn trong mục 3, 5, 7.

2. Ý tưởng chính

Ý tưởng trung tâm: biểu diễn dữ liệu dưới dạng một đồ thị tương tự (similarity graph) $G = (V, E)$, trong đó mỗi điểm dữ liệu là một đỉnh, và cạnh giữa hai đỉnh được gán trọng số bằng độ tương tự giữa hai điểm đó. Bài

toán phân cụm được phát biểu lại thành bài toán phân hoạch đồ thị: tìm một phân hoạch sao cho tổng trọng số các cạnh *giữa* các nhóm càng nhỏ càng tốt (điểm khác nhóm thì khác nhau), còn tổng trọng số cạnh *trong* mỗi nhóm càng lớn càng tốt (điểm cùng nhóm thì giống nhau).

Phát biểu bằng công thức, phân hoạch đồ thị tối ưu theo nghĩa ”mincut” (cắt nhỏ nhất) là bài toán dễ giải, nhưng trong thực hành nó thường cho nghiệm tầm thường (cắt tách một đỉnh lẻ ra khỏi phần còn lại). Để tránh điều này, người ta thêm điều kiện ”cân bằng” (các nhóm phải đủ lớn), dẫn đến hai tiêu chí phổ biến: RatioCut (đo kích thước nhóm bằng số đỉnh) và Ncut, normalized cut (đo kích thước nhóm bằng tổng trọng số cạnh gắn với nhóm, gọi là vol(A)). Cả hai bài toán tối ưu có điều kiện cân bằng này đều là NP-hard.

Spectral clustering chính là một phép **thư giãn liên tục (continuous relaxation)** của hai bài toán NP-hard đó: thay vì tìm một vector chỉ báo rời rạc (chỉ nhận 2 giá trị, biểu thị ”thuộc nhóm nào”), ta cho phép vector này nhận giá trị thực bất kỳ. Nghiệm của bài toán thư giãn chính là các eigenvector của một ma trận Laplacian của đồ thị (ứng với các eigenvalue nhỏ nhất). Quy trình thực hành tổng quát: (1) xây dựng đồ thị tương tự, (2) tính ma trận Laplacian, (3) lấy k eigenvector đầu tiên (ứng với k eigenvalue nhỏ nhất), (4) dùng các eigenvector này để ”nhúng” mỗi điểm dữ liệu vào một không gian $\mathbb{R}^k$ mới (mỗi điểm trở thành một hàng của ma trận chứa các eigenvector làm cột), (5) chạy k-means trên không gian nhúng đó để lấy cụm cuối cùng.

3. Phương pháp chi tiết

3.1. Đồ thị tương tự (similarity graph)

Cho n điểm dữ liệu $x_1, \ldots, x_n$ và độ tương tự $s_{ij} \geq 0$ giữa mỗi cặp. Ký hiệu: $W = (w_{ij})$ là ma trận kề có trọng số (weighted adjacency matrix), $d_i = \sum_j w_{ij}$ là bậc (degree) của đỉnh i, D là ma trận đường chéo chứa các d_i. Với một tập con đỉnh $A \subset V$, paper định nghĩa hai cách đo ”kích thước”: $|A|$ (số đỉnh) và $\text{vol}(A) = \sum_{i \in A} d_i$ (tổng bậc). $\text{cut}(A, \bar{A}) := \sum_{i \in A, j \in \bar{A}} w_{ij}$ là tổng trọng số các cạnh nối A với phần bù $\bar{A}$.

Paper nêu ba cách xây dựng đồ thị tương tự phổ biến:

- **ε-neighborhood graph:** nối hai điểm nếu khoảng cách giữa chúng nhỏ hơn ε; thường coi là đồ thị không trọng số vì mọi cạnh đã có khoảng cách tương đương.
- **k-nearest neighbor graph:** nối v_i với v_j nếu v_j thuộc k láng giềng gần nhất của v_i (hoặc ngược lại). Vì quan hệ ”láng giềng gần nhất” không đối xứng, có hai cách làm đồ thị vô hướng: bỏ qua chiều (kNN graph thông thường) hoặc chỉ giữ cạnh khi cả hai chiều đều thỏa (mutual kNN graph).
- **Fully connected graph:** nối mọi cặp điểm có độ tương tự dương, trọng số bằng chính s_{ij}; thường dùng với hàm Gauss $s(x_i, x_j) = \exp(-\|x_i - x_j\|^2/(2\sigma^2))$, với σ đóng vai trò tương tự ε.

3.2. Ma trận Laplacian của đồ thị

Laplacian không chuẩn hóa (unnormalized):

$$L = D - W$$

Paper chứng minh (Proposition 1) các tính chất quan trọng:

1. Với mọi $f \in \mathbb{R}^n$: $f^\top L f = \frac{1}{2} \sum_{i,j=1}^n w_{ij}(f_i - f_j)^2$ (dạng toàn phương).
2. L đối xứng và bán xác định dương (positive semi-definite).
3. Eigenvalue nhỏ nhất của L là 0, với eigenvector tương ứng là vector hằng $\mathbf{1}$.
4. L có n eigenvalue thực không âm $0 = \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_n$.

Một tính chất quan trọng khác (Proposition 2): **số thành phần liên thông của đồ thị bằng đúng số bội (multiplicity) của eigenvalue** 0, và không gian riêng (eigenspace) ứng với eigenvalue 0 được trải (span) bởi các vector chỉ báo (indicator vector) $\mathbf{1}_{A_1}, \ldots, \mathbf{1}_{A_k}$ của từng thành phần liên thông $A_1, \ldots, A_k$.

Hai Laplacian chuẩn hóa (normalized):

$$L_{sym} := D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}, \qquad L_{rw} := D^{-1} L = I - D^{-1} W$$

Paper nêu L_{rw} liên hệ chặt với một random walk trên đồ thị (ma trận chuyển $P = D^{-1}W$, $L_{rw} = I - P$). Các tính chất quan trọng (Proposition 3, 4): (a) λ là eigenvalue của L_{rw} với eigenvector u khi và chỉ khi λ là eigenvalue của L_{sym} với eigenvector $w = D^{1/2}u$; (b) 0 là eigenvalue của L_{rw} với eigenvector $\mathbf{1}$ (vector hằng), còn 0 là eigenvalue của L_{sym} với eigenvector $D^{1/2}\mathbf{1}$ (không phải vector hằng); (c) số thành phần liên thông cũng bằng bội của eigenvalue 0 cho cả hai Laplacian chuẩn hóa, tương tự Proposition 2.

3.3. Ba thuật toán spectral clustering

Paper trình bày ba thuật toán, khác nhau ở loại Laplacian dùng:

Thuật toán	Laplacian dùng	Bước đặc biệt
Unnormalized spectral clustering	L	không có
Normalized theo Shi và Malik (2000)	giải bài toán eigenvalue tổng quát $Lu = \lambda Du$ (tương đương eigenvector của L_{rw})	không có
Normalized theo Ng, Jordan và Weiss (2002)	L_{sym}	chuẩn hóa mỗi hàng của ma trận eigenvector về norm 1 trước khi chạy k-means

Cả ba đều theo khung chung: (1) xây đồ thị tương tự từ ma trận độ tương tự S, lấy W; (2) tính Laplacian tương ứng; (3) lấy k eigenvector đầu $u_1, \dots, u_k$, xếp thành các cột của ma trận $U \in \mathbb{R}^{n \times k}$ (riêng thuật toán Ng-Jordan-Weiss còn chuẩn hóa hàng của U thành ma trận T); (4) coi mỗi hàng của U (hoặc T) là một điểm $y_i \in \mathbb{R}^k$; (5) chạy k-means trên $\{y_i\}$ để được cụm cuối cùng.

3.4. Ví dụ số tự tính: Laplacian của đồ thị nhỏ

Ví dụ 1 (kiểm chứng Proposition 1 và 2, đồ thị 4 đỉnh, 2 thành phần liên thông): xét đồ thị gồm 2 cạnh tách biệt, $w_{12} = 1$ (nối đỉnh 1, 2) và $w_{34} = 1$ (nối đỉnh 3, 4), không có cạnh nào khác. Degree: $d_1 = d_2 = d_3 = d_4 = 1$, nên $D = I$. Ma trận Laplacian:

$$L = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

Đây là ma trận khối chéo gồm hai khối $\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ giống nhau. Mỗi khối có eigenvalue 0 (eigenvector $(1,1)$) và eigenvalue 2 (eigenvector $(1,-1)$). Vậy L có 4 eigenvalue: $0, 0, 2, 2$, tức **eigenvalue 0 có bội 2, đúng bằng số thành phần liên thông (2)**, khớp với Proposition 2 (tự tính). Kiểm tra eigenvector của eigenvalue 0: với $f = (1,1,0,0)^\top$ (vector chỉ báo của thành phần $\{1,2\}$), $Lf = (1\cdot1 - 1\cdot1,\ -1\cdot1 + 1\cdot1,\ 0,\ 0)^\top = (0,0,0,0)^\top$, đúng là eigenvector của eigenvalue 0. Kiểm tra bằng công thức dạng toàn phương của Proposition 1: $f^\top L f = \frac{1}{2}\big[w_{12}(f_1 - f_2)^2 + w_{21}(f_2 - f_1)^2 + w_{34}(f_3 - f_4)^2 + w_{43}(f_4 - f_3)^2\big] = \frac{1}{2}[0 + 0 + 0 + 0] = 0$, khớp với $\lambda \|f\|^2 = 0 \cdot 2 = 0$.

Với $f = (1,-1,0,0)^\top$ (hướng thứ hai trong khối đầu): $Lf = (1 \cdot 1 - (-1)\cdot(-1)$ sai, tính lại: Lf hàng 1 $= 1\cdot1 + (-1)\cdot(-1) = 1 + 1 = 2$, hàng 2 $= -1\cdot1 + 1\cdot(-1) = -1 - 1 = -2$, hàng 3, 4 bằng 0), vậy

$Lf = (2, -2, 0, 0)^\top = 2f$, đúng eigenvalue 2. Kiểm tra dạng toàn phương: $f^\top Lf = \frac{1}{2}[1 \cdot (1-(-1))^2 + 1 \cdot ((-1)-1)^2] = \frac{1}{2}[4+4] = 4$, khớp $\lambda\|f\|^2 = 2 \cdot (1+1) = 4$ (tự tính).

Ví dụ 2 (kiểm chứng sự khác biệt giữa L, L_{sym}, L_{rw} trên đồ thị bậc không đều): xét đồ thị hình sao 3 đỉnh: đỉnh 1 nối với đỉnh 2 ($w_{12} = 1$) và đỉnh 1 nối với đỉnh 3 ($w_{13} = 1$), không có cạnh 2-3. Degree: $d_1 = 2, d_2 = 1, d_3 = 1$, $D = \text{diag}(2, 1, 1)$.

$$L = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}$$

Kiểm tra eigenvector 0 của L là vector hằng $(1,1,1)^\top$: $L(1,1,1)^\top = (2-1-1, -1+1, -1+1)^\top = (0,0,0)^\top$, đúng (tự tính).

Tính $L_{sym} = D^{-1/2} L D^{-1/2}$ với $D^{-1/2} = \text{diag}(1/\sqrt{2}, 1, 1)$: phần tử $(1,1)$ là $2 \cdot \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = 1$; phần tử $(1,2) = (1,3)$ là $-1 \cdot \frac{1}{\sqrt{2}} \cdot 1 = -1/\sqrt{2}$; phần tử $(2,2) = (3,3) = 1$; phần tử $(2,3) = 0$. Vậy:

$$L_{sym} = \begin{pmatrix} 1 & -1/\sqrt{2} & -1/\sqrt{2} \\ -1/\sqrt{2} & 1 & 0 \\ -1/\sqrt{2} & 0 & 1 \end{pmatrix}$$

Theo Proposition 3/4, eigenvector của eigenvalue 0 phải là $D^{1/2}\mathbf{1} = (\sqrt{2}, 1, 1)^\top$ (không còn là vector hằng). Kiểm tra: hàng 1 của $L_{sym}(\sqrt{2},1,1)^\top = 1 \cdot \sqrt{2} + (-1/\sqrt{2}) \cdot 1 + (-1/\sqrt{2}) \cdot 1 = \sqrt{2} - 2/\sqrt{2} = \sqrt{2} - \sqrt{2} = 0$; hàng 2: $-1/\sqrt{2} \cdot \sqrt{2} + 1 \cdot 1 + 0 = -1 + 1 = 0$; hàng 3 tương tự 0. Vậy $(\sqrt{2}, 1, 1)^\top$ đúng là eigenvector của eigenvalue 0 của L_{sym}, khớp chính xác với phát biểu của paper (tự tính).

Tính $L_{rw} = D^{-1}L = \text{diag}(1/2, 1, 1) \cdot L = \begin{pmatrix} 1 & -0.5 & -0.5 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}$. Kiểm tra eigenvector 0 là vector hằng $(1,1,1)^\top$ (theo Proposition 3/4, phần đầu): hàng 1: $1 - 0.5 - 0.5 = 0$; hàng 2: $-1 + 1 + 0 = 0$; hàng 3: $-1 + 0 + 1 = 0$. Đúng. Vậy trong ví dụ bậc không đều này, L_{rw} vẫn giữ eigenvector hằng cho eigenvalue 0 giống L, còn L_{sym} thì không, đúng như sự phân biệt mà paper nêu giữa hai loại Laplacian chuẩn hóa (tự tính, toàn bộ ví dụ 2 là phép tính độc lập, không chép từ paper, chỉ dùng công thức paper cung cấp).

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial, không có benchmark định lượng so với các thuật toán khác trên dữ liệu thật theo nghĩa bảng số liệu độ chính xác. "Kết quả chính" của paper là các minh họa trên **dữ liệu mô phỏng nhỏ (toy data)**, dùng để làm rõ trực giác, không phải so sánh hiệu năng:

- **Hình 1 (mục 4):** dữ liệu 1 chiều gồm 200 điểm lấy mẫu từ hỗn hợp 4 phân phối Gauss (mixture of four Gaussians). Paper tính eigenvalue/eigenvector của L và L_{rw} cho cả đồ thị 10-nearest-neighbor và đồ thị fully-connected. Kết quả minh họa: với đồ thị kNN (các cụm tách thành các thành phần liên thông riêng), 4 eigenvalue đầu đúng bằng 0 và các eigenvector tương ứng là vector chỉ báo cụm; với đồ thị fully-connected (chỉ 1 thành phần liên thông vì hàm Gauss luôn dương), eigenvalue 0 có bội 1 (eigenvector hằng), còn 3 eigenvector tiếp theo mang thông tin phân biệt 4 cụm qua dấu của các phần tử (threshold ở 0).
- **Hình 3 (mục 8.1):** so sánh ε-neighborhood graph, kNN graph và mutual kNN graph trên một bộ dữ liệu 2D có 3 cụm (hai "vành trăng" mật độ khác nhau và một cụm Gauss). Minh họa định tính: ε-graph

khó chọn tham số khi dữ liệu có mật độ khác nhau theo vùng; kNN graph nối được các vùng mật độ khác nhau nhưng dễ tách rời thành nhiều thành phần; mutual kNN graph nằm ”ở giữa”, không trộn các mức mật độ khác nhau.

- **Hình 4 (mục 8.3):** minh họa ”eigengap heuristic” (chọn số cụm k dựa vào khoảng cách lớn giữa eigenvalue thứ k và $k+1$) trên 3 bộ dữ liệu có phương sai cụm tăng dần (cụm càng ”nhòe” thì khoảng cách eigenvalue càng khó phát hiện).
- **Hình 5 (mục 8.5):** minh họa hiện tượng unnormalized spectral clustering (dùng L) có thể hội tụ về eigenvector xấp xỉ hàm Dirac (gần như chỉ khác 0 tại một đỉnh duy nhất) khi eigenvalue tiến gần $\min_j d_j$ (bậc nhỏ nhất trong đồ thị), minh họa bằng cách tăng σ của kernel Gauss ($\sigma = 2$ và $\sigma = 5$) trên cùng bộ dữ liệu của Hình 1.

Ngoài các minh họa toy data này, paper còn trình bày một **kết quả lý thuyết định lượng thật** (không phải thí nghiệm số của chính paper, mà là kết quả được paper diễn giải lại từ Guattery và Miller, 1998): trên một lớp đồ thị gọi là ”cockroach graph” (dạng cái thang với vài thanh ngang bị bỏ), nhát cắt RatioCut tối ưu cho $k = 2$ có giá trị $\text{RatioCut}(A, \bar{A}) = 2/k$ (cắt dọc thang), nhưng unnormalized spectral clustering luôn tìm ra nhát cắt ngang với giá trị $\text{RatioCut}(B, \bar{B}) = 1$, tức **tệ hơn nghiệm tối ưu một hệ số** $k/2$ (paper nói rõ đây là ”a factor in the order of n”). Đây là một bằng chứng định lượng cụ thể (dù không phải paper tự chạy thí nghiệm, mà dẫn lại kết quả đã công bố) cho thấy phép thư giãn phổ (spectral relaxation) không có đảm bảo xấp xỉ (approximation guarantee) nào so với bài toán tổ hợp gốc.

5. Phân tích / ablation

Paper giải thích vì sao spectral clustering hoạt động qua **ba góc nhìn độc lập**, và phần này tổng hợp lại như một dạng ”ablation” về việc thuật toán nào phù hợp khi nào:

Góc nhìn 1, graph cut (mục 5): minh họa cho $k = 2$: đặt $f_i = \sqrt{|\bar{A}|/|A|}$ nếu $v_i \in A$ và $-\sqrt{|A|/|\bar{A}|}$ nếu $v_i \in \bar{A}$, paper chứng minh $f^\top L f = |V| \cdot \text{RatioCut}(A, \bar{A})$ và $f \perp \mathbf{1}$, $\|f\| = \sqrt{n}$. Thư giãn điều kiện rời rạc của f (cho f nhận giá trị thực tùy ý) biến bài toán tối thiểu RatioCut thành bài toán Rayleigh-Ritz chuẩn, nghiệm là eigenvector thứ hai của L. Tương tự, thư giãn Ncut (dùng $g = D^{1/2} f$) cho nghiệm là eigenvector thứ hai của L_{sym} (tương đương L_{rw}). Vậy: **thư giãn RatioCut dẫn tới unnormalized spectral clustering, thư giãn Ncut dẫn tới normalized spectral clustering**.

Góc nhìn 2, random walk (mục 6): $L_{rw} = I - P$ với $P = D^{-1}W$ là ma trận chuyển (transition matrix) của một random walk trên đồ thị; eigenvalue λ của L_{rw} tương ứng eigenvalue $1 - \lambda$ của P. Paper chứng minh (Proposition 5, dẫn từ Meila và Shi 2001) $\text{Ncut}(A, \bar{A}) = P(\bar{A} \mid A) + P(A \mid \bar{A})$ khi random walk khởi động từ phân phối dừng: tối thiểu Ncut tương đương tìm một nhát cắt sao cho random walk hiếm khi ”nhảy” qua lại giữa hai nhóm. Paper cũng giới thiệu khoảng cách commute (commute distance, hay resistance distance) qua ma trận nghịch đảo tổng quát (pseudo-inverse) $L^\dagger$ của L, và tự cảnh báo rằng mối liên hệ giữa phép nhúng của spectral clustering và commute-time embedding chỉ là **”a rather loose relation”** (không hoàn toàn trùng khớp, hai phép nhúng chỉ gần giống nhau trong một số trường hợp đặc biệt), paper nói rõ ”we have not yet seen a precise mathematical statement about this”.

Góc nhìn 3, perturbation theory (mục 7): dùng định lý Davis-Kahan để chặn khoảng cách giữa không gian riêng của Laplacian ”lý tưởng” (các cụm hoàn toàn tách biệt) và Laplacian ”bị nhiễu” (các cụm có một ít kết nối chéo) theo chuẩn của phần nhiễu $\|H\|$ chia cho khoảng cách phổ (eigengap) δ. Kết luận: **eigengap lớn thì nhiễu nhỏ không làm lệch nhiều eigenvector**, tức thuật toán ổn định hơn. Mục này cũng nêu hai điều kiện kỹ thuật cần có để lập luận trên hợp lý: (a) thứ tự eigenvalue phải có ý nghĩa (đúng với các Laplacian, nhưng không đúng nếu dùng trực tiếp eigenvector của ma trận tương tự S hoặc W, nên paper khuyến cáo không dùng S/W trực tiếp để phân cụm); (b) các phần tử eigenvector trên mỗi thành phần phải ”đủ xa 0”, nếu không (như trường hợp L_{sym} khi đồ thị có đỉnh bậc rất thấp) bước chuẩn hóa hàng có thể khuếch đại nhiễu và gây phân loại sai cho các đỉnh bậc thấp (outlier).

So sánh tổng hợp ba thuật toán (mục 8.5): paper đưa ra khuyến nghị rõ ràng, dựa trên ba lập luận:

Lập luận	Kết luận của paper
Mục tiêu phân cụm (tối thiểu cắt giữa nhóm và tối đa tương tự trong nhóm)	Normalized (dùng Ncut) đạt cả 2 mục tiêu; Unnormalized (dùng RatioCut) chỉ đạt mục tiêu tối thiểu cắt, không trực tiếp tối đa tương tự nội nhóm
Tính nhất quán (consistency) khi $n \to \infty$	Normalized (cả L_{sym} và L_{rw}) được chứng minh hội tụ về một phân hoạch có ý nghĩa; Unnormalized có thể không hội tụ, hoặc hội tụ về nghiệm suy biến (cụm chỉ gồm 1 điểm), liên quan hiện tượng eigenvector xấp xỉ Dirac khi eigenvalue vượt quá $\min_j d_j$
L_{sym} so với L_{rw}	Cả hai đều có cơ sở lý thuyết tốt, nhưng eigenvector của L_{rw} là vector chỉ báo cụm "sạch" (không bị nhân thêm $D^{1/2}$ như L_{sym}); L_{sym} không có ưu thế tính toán nào hơn, nên paper khuyến nghị dùng L_{rw}

Kết luận cuối của paper ở mục này: nên dùng **normalized spectral clustering với** L_{rw} làm lựa chọn mặc định.

6. Giới hạn

Các giới hạn mà **paper tự nêu rõ**, khá nhiều và khá thẳng thắn:

- **Không có cơ sở lý thuyết vững cho việc chọn đồ thị tương tự:** paper viết thẳng "to our knowledge there has been no systematic study which investigates the effects of the similarity graph and its parameters... None of the recommendations above is based on a firm theoretic ground". Với mutual kNN graph, paper thậm chí thừa nhận "we have to admit that we are a bit lost for rules of thumb" khi chọn tham số k.
- **Eigengap heuristic chỉ hiệu quả khi cụm tách biệt rõ:** khi các cụm "nhòe" hoặc chồng lấp, khoảng cách eigenvalue không còn rõ ràng, heuristic trở nên mơ hồ, như minh họa ở Hình 4.
- **Không có đảm bảo xấp xỉ:** phép thư giãn phổ không đảm bảo nghiệm gần với nghiệm tối ưu của bài toán tổ hợp gốc (RatioCut/Ncut); ví dụ cockroach graph cho thấy sai khác có thể lớn tới hệ số bậc n (mục 4, dẫn từ Guattery và Miller 1998). Paper cũng nhắc tới kết quả tổng quát hơn: xấp xỉ balanced graph cut trong hệ số hằng là NP-hard (Bui và Jones, 1992).
- **Mối quan hệ giữa chọn k (số cụm) và chọn tham số đồ thị (ví dụ k của kNN graph) không được hiểu rõ:** paper nói "to our knowledge nothing non-trivial is known on their interactions".
- **Unnormalized spectral clustering có thể hội tụ về nghiệm suy biến** (cụm 1 điểm) khi $n \to \infty$, trừ khi các eigenvalue dùng để phân cụm nhỏ hơn hẳn $\min_j d_j$; đây là lý do chính paper khuyến cáo tránh dùng L trực tiếp.
- L_{sym} **có thể phân loại sai các đỉnh bậc thấp (outlier):** vì eigenvector lý tưởng của L_{sym} là $D^{1/2}\mathbf{1}_{A_i}$, các đỉnh có bậc rất thấp sẽ có giá trị eigenvector rất nhỏ; bước chuẩn hóa hàng trong thuật toán Ng-Jordan-Weiss có thể khuếch đại nhiễu ở các giá trị nhỏ này và gây phân loại nhầm.
- **Bước k-means cuối cùng không có gì "đặc biệt bắt buộc"**: paper tự nói việc dùng k-means ở bước cuối là lựa chọn tùy ý (arbitrary), chỉ là lựa chọn phổ biến nhất; có thể thay bằng siêu phẳng (Lang, 2006) hoặc hậu xử lý khác (Bach và Jordan, 2004).

Quan sát **của người ghi chú**: paper này khác biệt rõ so với math 01-03 trong batch (những bài không có hoặc có rất ít chỗ tự phê bình định lượng), vì tác giả liên tục tự nêu các giới hạn thực hành kèm trích dẫn cụ thể, khiến phần "giới hạn" ở đây có nhiều nội dung xác thực từ chính văn bản hơn so với các bài trước.

7. Liên hệ với các bài khác

- **math 03 (A Tutorial on Principal Component Analysis, Shlens):** cả hai phương pháp đều dựa trên eigen-decomposition, nhưng của hai ma trận khác nhau và theo hai mục tiêu khác nhau. PCA tìm eigenvector ứng với eigenvalue *lớn nhất* của ma trận hiệp phương sai (hướng phương sai cao nhất để giữ thông tin), còn spectral clustering tìm eigenvector ứng với eigenvalue *nhỏ nhất* của Laplacian (hướng mà hàm f biến thiên ít giữa các điểm tương tự, để tách cụm). Đáng chú ý, paper hiện tại tự trích dẫn trực tiếp mối liên hệ này ở mục 9: Bengio và cộng sự (2004) diễn giải ma trận $D^{-1/2}WD^{-1/2}$ như một kernel matrix (liên hệ kernel PCA), còn Saerens, Fouss, Yen và Dupont (2004) có bài báo tên "The principal components analysis of a graph, and its relationships to spectral clustering" nêu đúng mối liên hệ giữa hai bài toán.
- **math 04 (Finding Structure with Randomness, Halko-Martinsson-Tropp):** paper hiện tại (mục 8.2) nói rằng để tính k eigenvector đầu của một Laplacian lớn nhưng thưa (sparse), người ta dùng power method hoặc phương pháp Krylov subspace như Lanczos; các kỹ thuật ngẫu nhiên hóa của math 04 (randomized range finder, randomized SVD) là một hướng hiện đại hơn để giải cùng bài toán con này (tìm vài eigenvector/singular vector đầu một cách hiệu quả ở quy mô lớn), dù bản thân paper 2007 này không đề cập tới randomized linear algebra (hợp lý vì ra đời 2 năm trước math 04).
- **math 06 (New Insights and Perspectives on the Natural Gradient Method, Martens):** bài tiếp theo trong batch, chuyển hẳn sang nhóm chủ đề "hình học thông tin / Fisher information", không chia sẻ công cụ toán trực tiếp với spectral clustering, nhưng cùng thuộc lộ trình "các phương pháp dùng eigen-decomposition của một ma trận mô tả cấu trúc bài toán" nếu nhìn rộng (Fisher information matrix cũng cần eigen-decomposition/ma trận đối xứng dương, dù không phải Laplacian đồ thị).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B, một VLM OCR dùng kiến trúc lai GatedDeltaNet kết hợp full attention, xử lý khoảng 3600 token/trang (khoảng 1500 token văn bản, 1600-2100 token hình ảnh), chạy trên GPU không quá 8GB, batch size 1. Các liên hệ dưới đây là **suy luận của người ghi chú**, không phải nội dung của paper:

- **Dùng attention map như một similarity graph để nhóm token/head:** ma trận attention trung bình (hoặc attention weight giữa các token trong một layer) có thể coi là một ma trận tương tự W (sau khi làm đối xứng, ví dụ $W = (A + A^\top)/2$ với A là attention weight). Từ đó tính Laplacian L_{rw} và chạy spectral clustering để xem token nào "thường chú ý vào nhau" có xu hướng tạo thành cụm tự nhiên, phục vụ việc gộp token (token merging) hoặc phân nhóm head attention tương tự nhau để tỉa (prune) những head dư thừa trong cùng một nhóm.
- **Phân tích cấu trúc head attention để tỉa theo nhóm:** nếu coi mỗi attention head là một "đỉnh" và độ tương tự giữa hai head được đo bằng độ tương quan giữa pattern attention của chúng trên một batch dữ liệu, spectral clustering có thể gợi ý các nhóm head có hành vi gần giống nhau, từ đó chỉ giữ lại một đại diện cho mỗi nhóm khi cắt tỉa (một dạng structured pruning dựa trên cụm thay vì dựa trên ngưỡng độ lớn trọng số đơn lẻ).
- **Giới hạn cần lưu ý khi áp dụng suy luận trên:** như paper tự nêu ở mục 6, 8.1, việc xây similarity graph (chọn kNN hay fully-connected, chọn tham số σ hay k) rất nhạy và không có hướng dẫn lý thuyết chắc chắn; áp dụng cho attention map của một mô hình cụ thể sẽ cần thử nghiệm tham số cẩn thận, và kết quả cụm có thể không ổn định giữa các batch dữ liệu khác nhau. Ngoài ra toàn bộ cách tiếp cận này chỉ nên coi là một công cụ **chẩn đoán/khám phá cấu trúc**, không phải một thuật toán nén đã được paper này kiểm chứng cho bối cảnh attention của transformer.

Tổng thể, bài này **liên quan gián tiếp**: công cụ toán (Laplacian, eigen-decomposition của ma trận mô tả quan hệ giữa các thực thể) có thể mượn để phân tích cấu trúc nội tại của mô hình (token, head), nhưng đích thực của bài là phân cụm dữ liệu, không phải nén mô hình, nên việc áp dụng cho dự án đòi hỏi một bước chuyển

đổi khá xa so với nội dung gốc.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây chạy được trên 1 GPU 8GB (dùng PyTorch/NumPy/scikit-learn cho bước k-means và eigen-decomposition cuối):

1. **Tái tạo ví dụ toy của Hình 1:** sinh 200 điểm 1D từ hỗn hợp 4 phân phối Gauss, xây đồ thị 10-NN và đồ thị fully-connected (kernel Gauss $\sigma = 1$), tính eigenvalue/eigenvector của L và L_{rw}, kiểm tra xem 4 eigenvalue đầu có gần 0 không (đồ thị kNN) và xem 3 eigenvector tiếp theo của đồ thị fully-connected có phân biệt đúng 4 cụm qua dấu không, so với kỳ vọng mô tả ở mục 4.
2. **Spectral clustering trên attention map của một layer nhỏ:** lấy attention weight trung bình (qua vài batch) của một layer attention trong một mô hình nhỏ, đối xứng hóa thành W, tính L_{rw}, lấy k eigenvector đầu, chạy k-means để nhóm các token/vị trí. Đo: các nhóm tìm được có tương ứng với cấu trúc ngữ nghĩa hợp lý (ví dụ token trong cùng một từ, hoặc token hình ảnh cùng một vùng ảnh) không.
3. **So sánh eigengap heuristic trên dữ liệu tổng hợp với mức nhiễu khác nhau:** lặp lại thí nghiệm giống Hình 4 của paper (3 bộ Gauss mixture với phương sai tăng dần), vẽ phổ eigenvalue của L_{rw}, quan sát khoảng cách eigenvalue thứ k và $k+1$ thu hẹp dần khi cụm chồng lấp nhiều hơn, kiểm tra trực tiếp tuyên bố của paper rằng heuristic "kém hiệu quả khi cụm nhòe".
4. **Kiểm tra hiện tượng Dirac-eigenvector của unnormalized Laplacian:** trên cùng bộ dữ liệu Gauss mixture, tăng dần σ của kernel Gauss khi xây đồ thị fully-connected, tính eigenvector của L (không chuẩn hóa), đo độ "tập trung" của mỗi eigenvector (ví dụ bằng entropy hoặc tỉ lệ phần tử lớn nhất trên tổng), kiểm tra xem khi eigenvalue tiến gần $\min_j d_j$ thì eigenvector có thực sự trở nên gần giống hàm Dirac (chỉ lớn ở một tọa độ) như paper mô tả ở Hình 5 không.
5. **So sánh ba thuật toán (unnormalized, Shi-Malik, Ng-Jordan-Weiss) trên một đồ thị có vài đỉnh bậc rất thấp (outlier):** tạo một đồ thị tổng hợp có 2 cụm dày đặc và thêm vài điểm "lạc" (bậc rất thấp), chạy cả ba thuật toán, so sánh tỉ lệ các điểm lạc bị phân loại sai giữa Ng-Jordan-Weiss (dùng L_{sym}, dự kiến nhạy với outlier do bước chuẩn hóa hàng) và Shi-Malik/unnormalized, kiểm tra trực tiếp cảnh báo ở mục 7.2 của paper.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa
Ma trận kề, bậc	$W = (w_{ij})$, $d_i = \sum_j w_{ij}$, $D = \text{diag}(d_1, \dots, d_n)$
Laplacian không chuẩn hóa	$L = D - W$; $f^\top L f = \frac{1}{2}\sum_{ij} w_{ij}(f_i - f_j)^2$; eigenvalue nhỏ nhất 0, eigenvector $\mathbf{1}$
Số thành phần liên thông	bằng bội của eigenvalue 0 của L (và của L_{sym}, L_{rw}); eigenspace trải bởi các vector chỉ báo thành phần
Laplacian chuẩn hóa	$L_{sym} = D^{-1/2} L D^{-1/2}$ (eigenvector 0 là $D^{1/2}\mathbf{1}$); $L_{rw} = D^{-1}L = I - P$ với $P = D^{-1}W$ (eigenvector 0 là $\mathbf{1}$)
RatioCut, Ncut	RatioCut $= \sum_i \text{cut}(A_i, \bar{A}_i)/\lvert A_i\rvert$; Ncut $= \sum_i \text{cut}(A_i, \bar{A}_i)/\text{vol}(A_i)$
Thư giãn phổ	thư giãn RatioCut $\to$ eigenvector của L (unnormalized); thư giãn Ncut $\to$ eigenvector của L_{sym}/L_{rw} (normalized)
Thuật toán chung	xây đồ thị, tính Laplacian, lấy k eigenvector đầu làm cột của U, mỗi hàng của U là điểm nhúng $y_i \in \mathbb{R}^k$, chạy k-means trên $\{y_i\}$
Eigengap heuristic	chọn k sao cho $\lambda_1, \dots, \lambda_k$ rất nhỏ nhưng λ_{k+1} lớn hẳn
Ví dụ tự tính 1	đồ thị 2 cạnh tách biệt: L có eigenvalue $\{0, 0, 2, 2\}$, bội của 0 bằng 2 = số thành phần liên thông
Ví dụ tự tính 2	đồ thị sao 3 đỉnh, $d = (2, 1, 1)$: eigenvector 0 của L là $(1, 1, 1)$, của L_{sym} là $(\sqrt{2}, 1, 1)$, của L_{rw} là $(1, 1, 1)$

11. Kết luận ngắn

Paper giải thích spectral clustering từ ba góc nhìn độc lập (cắt đồ thị, random walk, lý thuyết nhiễu loạn), cho thấy bản chất thuật toán là một phép thư giãn liên tục của các bài toán phân hoạch đồ thị NP-hard (RatioCut, Ncut), với nghiệm là các eigenvector ứng với eigenvalue nhỏ nhất của ma trận Laplacian (không chuẩn hóa hoặc chuẩn hóa). Khuyến nghị thực hành của paper: dùng đồ thị k-nearest-neighbor, Laplacian chuẩn hóa L_{rw}, vì có cơ sở lý thuyết tốt hơn (nhất quán khi $n \to \infty$, không có hiện tượng eigenvector suy biến như Laplacian không chuẩn hóa). Giới hạn cốt lõi mà paper tự nhận: việc chọn đồ thị tương tự và các tham số liên quan thiếu cơ sở lý thuyết chắc chắn, và phép thư giãn phổ không có đảm bảo xấp xỉ nào so với bài toán tổ hợp gốc. Trong lộ trình học, bài này minh họa một mô thức dùng lại nhiều lần: phân tích cấu trúc dữ liệu/bài toán qua eigen-decomposition của một ma trận đối xứng mô tả quan hệ giữa các thực thể, cùng họ với PCA (math 03) nhưng áp dụng cho ma trận và mục tiêu hoàn toàn khác.

Nguồn gốc: https://arxiv.org/abs/0711.0189

Bài 06

New Insights and Perspectives on the Natural Gradient Method (2014)

Tác giả: James Martens (DeepMind, London)
Venue / arXiv: arXiv:1412.1193 (v11, 19/09/2020); công bố chính thức tại Journal of Machine Learning Research 21 (2020), trang 1-76 (Submitted 11/17, Revised 6/20, Published 8/20, Editor: Léon Bottou)
Nguồn: https://arxiv.org/abs/1412.1193
Chủ đề: Tối ưu hóa bậc hai, natural gradient, Fisher information matrix, Generalized Gauss-Newton matrix
Vị trí trong lộ trình: Bài số 6 trong nhóm 9 bài toán nền (math 01-09). Mở đầu nhóm con ”hình học thông tin / Fisher information” (06-07), đọc sau math 05 (spectral clustering, vẫn thuộc nhóm eigen-decomposition cổ điển), dùng lại khái niệm Hessian và gradient từ math 01 nhưng áp dụng cho một ma trận ”độ cong” khác (Fisher). Đứng trước math 07 (tổng quát hóa thành khung hình học thông tin đầy đủ cho các họ phân phối).

> Ghi chú này được viết lại bằng tiếng Việt dựa trên toàn văn bài báo gốc (đọc hết phần thân bài, mục 1-15, qua bản PDF chuyển sang text), không phải bản dịch từng câu. Đây là một bài rất dài (76 trang) và kỹ thuật cao (nhiều định lý, chứng minh chi tiết); ghi chú đọc đầy đủ các mục 1-15 (định nghĩa, lý luận hình học, GGN, Fisher, empirical Fisher, tính bất biến tham số hóa, tốc độ hội tụ tiệm cận) nhưng CHỈ LƯỢC QUA phần Phụ lục A-E (khoảng 1900 dòng, chứng minh chi tiết các định lý 3, 5, 6 và Corollary 2), vì đây là các chứng minh kỹ thuật thuần đại số, không cần chép lại để hiểu ý tưởng chính. Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Natural gradient descent được Amari đề xuất từ cuối những năm 1990 (paper trích ”Amari, 1998”), là một phương pháp tối ưu thay thế cho gradient descent tiêu chuẩn, đã được áp dụng cho nhiều lĩnh vực: blind source separation, reinforcement learning, huấn luyện mạng neural. Paper của James Martens (lúc viết các phiên bản đầu công tác tại DeepMind, London) không đề xuất một thuật toán mới, mà là một bài phân tích phê bình (critical analysis) lại toàn bộ phương pháp natural gradient: nó thực sự là gì, vì sao nó hoạt động tốt, và những cách hiểu truyền thống về nó (dựa trên information geometry) có những hạn chế gì.

Đây là một trong những bài có lịch sử công bố khác thường nhất trong batch: bản arXiv đầu tiên (v1) được đăng tháng 12/2014 (mã arXiv 1412.1193), nhưng bài chỉ được nộp chính thức cho JMLR vào tháng 11/2017, qua chỉnh sửa (revised) tháng 6/2020, và xuất bản tháng 8/2020, với phiên bản arXiv cuối cùng là v11 (tổng cộng 11 lần sửa trong 6 năm). Venue cuối cùng là Journal of Machine Learning Research, tập 21 (2020), trang 1-76, do Léon Bottou làm editor. Ghi chú này dùng năm 2014 cho tên file theo quy ước lấy năm của mã arXiv gốc, nhưng cần hiểu rằng nội dung đã được tinh chỉnh qua nhiều năm trước khi thành bài JMLR hoàn chỉnh.

Tóm tắt (abstract) của paper nêu rõ ba đóng góp chính: (1) chỉ ra natural gradient descent có thể xem như một phương pháp bậc hai (2nd-order method), với Fisher information matrix đóng vai trò thay thế cho Hessian; (2) trong nhiều trường hợp quan trọng, Fisher tương đương với Generalized Gauss-Newton matrix (GGN); (3) phân tích tốc độ hội tụ tiệm cận của natural gradient descent ngẫu nhiên (stochastic) trên hàm mục tiêu lồi bậc hai (quadratic), phê bình ”empirical Fisher approximation” hay bị dùng sai trong thực hành, và phân tích tính bất biến (gần đúng) theo tham số hóa.

2. Ý tưởng chính

Natural gradient được định nghĩa là gradient nhân với nghịch đảo của Fisher information matrix F của mô hình:

$$\tilde{\nabla} h = F^{-1} \nabla h$$

trong đó h là hàm mục tiêu (thường là trung bình loss trên tập huấn luyện). Paper trình bày hai cách hiểu bổ sung cho nhau về natural gradient, đây chính là ”insight” trung tâm của bài:

Cách hiểu truyền thống (information geometry): gradient tiêu chuẩn $-\nabla h$ là hướng giảm dốc nhất (steepest descent) khi khoảng cách trong không gian tham số được đo bằng chuẩn Euclid. Natural gradient là hướng giảm dốc nhất khi khoảng cách được đo bằng một ”metric” khác, nội tại với không gian các phân phối xác suất mà mô hình biểu diễn (không gắn với cách tham số hóa cụ thể), cụ thể là xấp xỉ của KL divergence $KL(P_{x,y}(\theta + d) \| P_{x,y}(\theta))$ trong một lân cận nhỏ. Vì khoảng cách KL không phụ thuộc cách tham số hóa, hướng natural gradient (trong không gian phân phối) bất biến với mọi cách đổi tham số trơn, khả nghịch, trong khi gradient thường thì không.

Cách hiểu mới mà paper đề xuất là trọng tâm (bậc hai / 2nd-order): paper lập luận cách hiểu truyền thống trên có một lỗ hổng thực tế: nó chỉ đúng khi bước cập nhật vô cùng nhỏ, còn trong thực hành ta luôn cập nhật một bước rời rạc hữu hạn trong không gian tham số θ, lúc đó lý giải hình học bị phá vỡ (đi thẳng trong không gian θ không tương ứng với đi thẳng trong không gian phân phối). Paper đề xuất cách hiểu thứ hai: natural gradient descent chính là một dạng phương pháp bậc hai (2nd-order method), trong đó Fisher F đóng vai trò ma trận độ cong (curvature matrix) thay cho Hessian H thật. Và quan trọng hơn: trong nhiều trường hợp phổ biến, F lại chính bằng Generalized Gauss-Newton matrix (GGN), một ma trận xấp xỉ Hessian đã được biết đến từ trước trong tối ưu hóa cổ điển, có tính chất luôn nửa xác định dương (PSD). Cách hiểu này giải thích được vì sao các kỹ thuật ”damping” (giảm xóc, ví dụ Tikhonov regularization, trust region) từ tối ưu hóa bậc hai cổ điển lại cần thiết để natural gradient hoạt động tốt và ổn định trong thực hành với bước cập nhật lớn.

3. Phương pháp chi tiết

3.1. Khung bài toán và KL divergence objective

Paper xét bài toán học có giám sát (supervised learning) tổng quát: mạng neural với tham số $\theta \in \mathbb{R}^n$ tính hàm $f(x, \theta)$, mục tiêu tối thiểu $h(\theta) = \frac{1}{|S|} \sum_{(x,y) \in S} L(y, f(x, \theta))$ với L là hàm loss. Paper nhấn mạnh natural gradient có lý thuyết chặt nhất khi $L(y, z) = -\log r(y|z)$ cho một phân phối dự đoán $R_{y|z}$ nào đó (ví dụ: squared error tương ứng phân phối Gauss, cross-entropy tương ứng phân phối multinomial). Khi đó bài toán học tương đương tối thiểu $KL(Q_{x,y} \| P_{x,y}(\theta))$, với $Q_{x,y}$ là phân phối dữ liệu thật, $P_{x,y}(\theta)$ là phân phối mà mô hình biểu diễn. Khi thay Q_x bằng phân phối huấn luyện thực nghiệm $\hat{Q}_x$ và chỉ có một mẫu y cho mỗi x, mục tiêu KL quy về chính hàm negative log-likelihood tiêu chuẩn.

3.2. Fisher information matrix và natural gradient

Fisher của $P_{x,y}(\theta)$ theo θ được định nghĩa:

$$F = \mathbb{E}_{P_{x,y}} \left[\nabla \log p(x, y|\theta) \, \nabla \log p(x, y|\theta)^\top \right] = -\mathbb{E}_{P_{x,y}} \left[H_{\log p(x,y|\theta)} \right]$$

Dạng thứ nhất (tích ngoài của gradient log-density với chính nó, lấy kỳ vọng) cho thấy ngay F luôn PSD (vì là kỳ vọng của một outer-product, luôn PSD). Dạng thứ hai (âm của kỳ vọng Hessian của log-density) cho thấy F có thể hiểu là ”Hessian kỳ vọng dưới phân phối của chính mô hình” (expected Hessian under the model’s own predictive distribution), khác với Hessian thật H của h, vốn là kỳ vọng Hessian dưới phân phối thực nghiệm $\hat{Q}_{x,y}$. Natural gradient định nghĩa là $\tilde{\nabla} h = F^{-1} \nabla h$.

3.3. Lý giải hình học (mục 6 của paper)

Paper dùng khai triển Taylor bậc hai của KL divergence:

$$KL(P_{x,y}(\theta+d)\|P_{x,y}(\theta)) = \frac{1}{2}d^\top F d + O(d^3)$$

Dựa trên một kết quả của Arnold và cộng sự (2011) về steepest descent với ma trận PSD A tổng quát, paper cho thấy khi lấy $A = \frac{1}{2}F$, hướng $-F^{-1}\nabla h$ (tỉ lệ bởi hằng số) chính là hướng giảm dốc nhất trong không gian phân phối, đo bằng khoảng cách KL cục bộ. Vì KL chỉ phụ thuộc phân phối dự đoán $P_{y|x}$, không phụ thuộc cách tham số hóa θ, hướng này (trong không gian phân phối) bất biến với đổi tham số hóa, trong khi hướng gradient Euclid thường thì phụ thuộc tham số hóa.

3.4. Phương pháp bậc hai (2nd-order) tổng quát và Generalized Gauss-Newton matrix

Paper định nghĩa khung chung cho các phương pháp bậc hai: cập nhật $\delta_k^* = \arg\min_\delta M_k(\delta)$ với mô hình bậc hai cục bộ $M_k(\delta) = \frac{1}{2}\delta^\top B_k\delta + \nabla h(\theta_k)^\top\delta + h(\theta_k)$, B_k là "ma trận độ cong". Gradient descent ứng với $B_k = \beta I$ (ước lượng độ cong rất thô, một hằng số theo mọi hướng). Newton's method ứng với $B_k = H$ (Hessian thật), chính xác nhất về mặt cục bộ nhưng có vấn đề: H của mạng neural có thể không xác định (indefinite, có hướng cong âm), khiến $M_k(\delta)$ không chặn dưới theo các hướng đó, gây ra bước cập nhật quá lớn, không đáng tin cậy ("model trust" issue).

Generalized Gauss-Newton matrix (GGN), ký hiệu G, giải quyết vấn đề này: với loss $L(y,z)$ lồi theo z,

$$G = \frac{1}{|S|}\sum_{(x,y)\in S} J_f^\top H_L J_f$$

với J_f là Jacobian của $f(x,\theta)$ theo θ, H_L là Hessian của $L(y,z)$ theo z tại $z = f(x,\theta)$. Vì L lồi theo z nên H_L luôn PSD, do đó G luôn PSD, không bao giờ có hướng cong âm. G có thể suy ra bằng cách tuyến tính hóa f (lấy xấp xỉ affine của f quanh θ_k) trước khi lấy Hessian của h theo cách đó; nói cách khác G chính là phần của Hessian thật H còn lại sau khi "bỏ đi" phần đóng góp từ độ cong (phi tuyến) của riêng hàm f (mạng neural), chỉ giữ lại độ cong đến từ hàm loss L. Paper lý luận (mục 8.1, mang tính "suy đoán" theo chính lời paper tự nhận): độ cong đến từ L "đáng tin cậy" hơn độ cong đến từ f (vì H_L luôn PSD, ổn định, không đổi dấu bất thường như độ cong của các hàm kích hoạt phi tuyến bên trong mạng), đây là một lời giải thích khả dĩ cho việc GGN thường hoạt động tốt hơn Hessian thật trong huấn luyện mạng neural.

3.5. Mối liên hệ Fisher và GGN (mục 9, kết quả trung tâm của paper)

Paper chỉ ra Fisher cũng có thể viết dưới dạng rất giống GGN:

$$F = \mathbb{E}_{Q_x}\left[J_f^\top F_R J_f\right]$$

với F_R là Fisher của phân phối dự đoán $R_{y|z}$ tại $z = f(x,\theta)$. So với công thức GGN ở trên, hai biểu thức giống nhau hoàn toàn, chỉ khác "ma trận trong" là F_R (Fisher của R) thay cho H_L (Hessian của L). Paper chứng minh (và đây là kết quả định lý trung tâm của bài): $F_R = H_L$ xảy ra khi $R_{y|z}$ thuộc họ phân phối mũ (exponential family) với z là "tham số tự nhiên" (natural parameter), tức $\log r(y|z) = z^\top T(y) - \log Z(z)$. Các ví dụ cụ thể mà paper nêu: phân phối Gauss đa biến với z tham số hóa trực tiếp trung bình μ (tương ứng loss squared error), và phân phối multinomial với z là input vào softmax (tương ứng loss cross-entropy). Đây chính là lý do, trong phần lớn các bài toán phân loại/hồi quy tiêu chuẩn của deep learning, Fisher và GGN trùng nhau, nên hai cách lý giải ở mục 2 (hình học thông tin và bậc hai) đều áp dụng được đồng thời.

3.6. Xây dựng phương pháp thực hành và vai trò của damping (mục 10)

Vì bước cập nhật thực tế luôn là một đoạn hữu hạn trong không gian θ (không phải một đường đi mượt vô cùng nhỏ trong không gian phân phối), lý giải hình học ở mục 3.3 không còn chính xác với bước lớn; paper minh họa bằng Hình 1 của paper (đường đi trong không gian tham số do bước natural gradient lớn tạo ra có

thể lệch xa so với hướng ban đầu). Thay vào đó, paper đề xuất dựa trên lý giải bậc hai: vì $F \approx G$ trong nhiều trường hợp, cập nhật $-F^{-1}\nabla h$ với hệ số $\alpha = 1$ chính là nghiệm tối ưu của mô hình bậc hai cục bộ dùng G. Khi mô hình bậc hai không còn chính xác ở khoảng cách lớn (và đặc biệt khi F có một số eigenvalue rất nhỏ khiến natural gradient rất lớn theo hướng đó), các kỹ thuật "update damping" quen thuộc trong tối ưu hóa bậc hai cổ điển (Tikhonov regularization, trust region, kiểu Levenberg-Marquardt) nên được dùng, thay vì chỉ đơn giản giảm hệ số bước α một cách đồng nhất cho mọi hướng (vì cách này ảnh hưởng đều lên mọi eigen-direction của F, kể cả những hướng mà natural gradient vốn đã hợp lý).

3.7. Phê bình "empirical Fisher" và ví dụ số minh họa (mục 11, kèm tự tính)

Paper định nghĩa "empirical Fisher" $\bar{F}$ là một xấp xỉ hay bị dùng (và hay bị gọi nhầm là Fisher thật): lấy kỳ vọng tích ngoài gradient log-density dưới phân phối thực nghiệm $\hat{Q}_{x,y}$ (dữ liệu thật) thay vì dưới phân phối dự đoán của chính mô hình $P_{x,y}$:

$$\bar{F} = \frac{1}{|S|} \sum_{(x,y)\in S} \nabla \log p(y|x,\theta)\, \nabla \log p(y|x,\theta)^\top$$

$\bar{F}$ dễ tính hơn F (chỉ cần gradient, không cần lấy thêm kỳ vọng theo phân phối dự đoán của mô hình), nhưng paper lập luận nó khác F theo cách "tinh vi nhưng quan trọng": $\bar{F}$ không có tính chất xấp xỉ Hessian hay tương đương GGN như F có. Paper đưa ra một ví dụ cụ thể để chứng minh điều này (paper chỉ cho công thức tổng quát, không có số liệu cụ thể; phần số liệu dưới đây là tính toán minh họa của người ghi chú, tự tính):

Xét $n = 1$, $f(x,\theta) = \theta$, $R_{y|z} = \mathcal{N}(z, 1)$, $S = \{(0,0)\}$ (một mẫu, input và target đều bằng 0). Khi đó $h(\theta) = \frac{1}{2}\theta^2$, $\nabla h = \theta$. Paper tính được: $\bar{F} = \theta^2$ (phụ thuộc θ hiện tại), còn $F = 1$ (hằng số, không đổi theo θ).

Tính tay (tự tính) hai dãy cập nhật với bước cố định $\alpha = 0.5$, bắt đầu từ $\theta_0 = 2$:

Cập nhật dùng Fisher thật: $\theta_{k+1} = \theta_k - \alpha F^{-1}\theta_k = (1-\alpha)\theta_k = 0.5\,\theta_k$. Dãy: $\theta_0 = 2$, $\theta_1 = 1$, $\theta_2 = 0.5$, $\theta_3 = 0.25$, $\theta_4 = 0.125$, giảm đều đặn theo cấp số nhân với tỉ lệ 0.5 mỗi bước, hội tụ nhanh về 0 (đúng như paper dự đoán: với $0 < \alpha < 2$, cập nhật dùng F cho hội tụ tuyến tính ổn định).

Cập nhật dùng empirical Fisher (với $\xi = 1$ như trong công thức tổng quát của paper): $\theta_{k+1} = \theta_k - \alpha(\theta_k^2)^{-1}\theta_k = \theta_k - \dfrac{\alpha}{\theta_k} = \theta_k - \dfrac{0.5}{\theta_k}$. Dãy tính tay: $\theta_0 = 2 \to \theta_1 = 2 - 0.25 = 1.75 \to \theta_2 = 1.75 - 0.5/1.75 \approx 1.464 \to \theta_3 \approx 1.464 - 0.342 \approx 1.123 \to \theta_4 \approx 1.123 - 0.445 \approx 0.677 \to \theta_5 \approx 0.677 - 0.738 \approx -0.061$. Tại bước 5, giá trị vượt qua 0 và trở nên rất nhỏ (gần 0), khiến bước tiếp theo $\theta_6 \approx -0.061 - 0.5/(-0.061) \approx -0.061 + 8.2 \approx 8.14$ nhảy rất xa khỏi nghiệm. Dãy số không hội tụ êm mà dao động với biên độ tăng vọt, đúng như paper cảnh báo: với bước cố định, cập nhật dựa trên empirical Fisher "sẽ không hội tụ về nghiệm tối ưu trừ khi hệ số mũ $\xi < 1$ và bước α_k giảm về 0 đủ nhanh". Đây là minh chứng số cụ thể (tự tính, dựng từ đúng ví dụ và công thức mà paper đưa ra) cho thấy vì sao dùng empirical Fisher làm ma trận độ cong có thể gây mất ổn định mà Fisher thật không gặp phải.

4. Thí nghiệm và kết quả chính

Đây là một bài **lý thuyết hoàn toàn**: toàn bộ 15 mục chính không chứa bất kỳ thí nghiệm huấn luyện mạng neural thực tế nào (không có bảng số liệu, không có hình vẽ loss curve, không có so sánh trên dataset chuẩn như MNIST/CIFAR). "Kết quả chính" của paper là các định lý toán học và các lập luận phân tích:

- **Kết quả 1 (mục 9):** điều kiện chính xác để Fisher F tương đương Generalized Gauss-Newton matrix G: khi $R_{y|z}$ là họ phân phối mũ với z là tham số tự nhiên (điều kiện này đúng cho squared-error loss với Gauss, và cross-entropy loss với multinomial/softmax).
- **Kết quả 2 (mục 11, ví dụ $n = 1$):** một ví dụ phân tích (không phải thí nghiệm số lớn, nhưng là một case study cụ thể, định lượng) cho thấy cập nhật dựa trên empirical Fisher với bước cố định có thể phân

kỳ, trong khi cập nhật dựa trên Fisher thật hội tụ tuyến tính ổn định. Mục 3.7 trên đã mở rộng case study này thành một dãy số cụ thể (tự tính) để minh họa trực quan.

- **Kết quả 3 (mục 12, Theorem 1 và Corollary 2):** điều kiện đủ $J_\zeta^\top B_\theta J_\zeta = B_\gamma$ để một ma trận độ cong B cho tính bất biến (gần đúng) theo đổi tham số hóa ζ. Paper chứng minh điều kiện này tự động thỏa mãn cho mọi ma trận độ cong có dạng $\mathbb{E}[J_f^\top A J_f]$ (bao gồm GGN, Fisher, cả empirical Fisher), nhưng **không** thỏa mãn cho Hessian thật (trừ các trường hợp đặc biệt: ζ là hàm affine, hoặc $\nabla h = 0$ tại điểm đang xét).
- **Kết quả 4 (mục 14, Theorem 3, 5, 6, phần trung tâm về lý thuyết hội tụ):** với hàm mục tiêu lồi bậc hai $h(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H^*(\theta - \theta^*)$ và cập nhật ngẫu nhiên (stochastic) dạng $\theta_{k+1} = \theta_k - \alpha_k B^{-1} g_k(\theta_k)$ với B là ma trận độ cong cố định bất kỳ, paper đưa ra biểu thức tiệm cận chính xác cho $\mathbb{E}[h(\theta_k)] - h(\theta^*)$, tách thành hai phần: một phần phụ thuộc nhiễu gradient (noise-dependent, chiếm ưu thế khi $k \to \infty$) và một phần phụ thuộc điểm khởi tạo θ_0 (starting-point-dependent, giảm nhanh hơn theo k). Kết luận gây bất ngờ mà paper tự nêu: **phần phụ thuộc nhiễu không được cải thiện bởi việc dùng ma trận độ cong bậc hai** (natural gradient hay Newton) nếu kết hợp với kỹ thuật Polyak averaging, SGD thường + averaging đạt cùng tốc độ tiệm cận (”Fisher efficient”) mà không cần bất kỳ ma trận độ cong nào; lợi ích thực sự của phương pháp bậc hai nằm ở phần phụ thuộc điểm khởi tạo, quan trọng khi số vòng lặp còn hạn chế (chưa đạt chế độ tiệm cận).

Paper không chạy bất kỳ experiment deep learning thực nghiệm nào để kiểm chứng trực tiếp các định lý 3, 5, 6 trên mạng neural thật; mọi kết quả đều là suy diễn giải tích thuần túy cho trường hợp lý tưởng hóa (hàm mục tiêu lồi bậc hai).

5. Phân tích / ablation

So sánh các ma trận độ cong (curvature matrix) theo các tiêu chí mà paper tự đặt ra:

Ma trận	PSD?	Bất biến tham số hóa?	Tương đương Hessian khi nào
Hessian H	Không (có thể âm)	Không, trừ trường hợp đặc biệt	Luôn (chính là H)
GGN G	Luôn PSD	Có (dạng $\mathbb{E}[J_f^\top A J_f]$)	Khi dự đoán tối ưu ($\nabla_z L \approx 0$) hoặc f tuyến tính (mạng tuyến tính)
Fisher F	Luôn PSD	Có	Khi $Q_{x,y} = P_{x,y}(\theta)$ (điều kiện ”realizability”)
Empirical Fisher $\bar{F}$	Luôn PSD	Có (paper chứng minh riêng ở mục 12)	Chỉ khi ”realizability” đúng, nói chung không xấp xỉ tốt H

Phân tích tốc độ hội tụ tiệm cận (mục 14), điểm mấu chốt của ablation lý thuyết: paper chỉ ra rằng kết quả ”Fisher efficiency” nổi tiếng của Amari (1998), hàm ý natural gradient descent đạt phương sai nhỏ nhất có thể theo chặn Cramér-Rao, đi kèm bốn điều kiện ngặt mà paper tự liệt kê: (1) phải giả định hội tụ về optimum toàn cục (chỉ hợp lý khi h lồi, không đúng cho mạng neural thường không lồi); (2) Fisher F phải tính trên toàn bộ tập huấn luyện (full-batch), không cho phép mini-batch; (3) kết quả đo theo sai số tham số $\theta_k - \theta^*$, không trực tiếp theo giá trị hàm mục tiêu; (4) cần điều kiện ”realizability” (mô hình phải khớp hoàn toàn phân phối huấn luyện tại θ^*), một điều kiện mà paper gọi là ”tinh vi” (subtle) và thường không đúng trong thực hành.

Về vai trò thực sự của phương pháp bậc hai: kết quả 4 ở mục 4 (Theorem 5, 6) cho thấy lợi ích của natural gradient/GGN không nằm ở tốc độ hội tụ tiệm cận dài hạn (nơi SGD với Polyak averaging cũng đạt cùng tốc độ, theo đúng chặn Cramér-Rao, mà không cần ma trận độ cong nào), mà nằm ở việc cải thiện mạnh thành phần phụ thuộc điểm khởi tạo khi Hessian thật H^* bị ”ill-conditioned” (tỉ số điều kiện $\kappa(H^*) = \lambda_1(H^*)/\lambda_n(H^*)$ lớn) và số vòng lặp còn hạn chế, chưa đạt chế độ tiệm cận. Đây là một phân tích khá tinh tế: nó giải thích **tại sao** natural gradient có lợi trong thực hành (giai đoạn đầu huấn luyện, hữu hạn bước) nhưng cũng chỉ ra rõ **giới hạn** của lợi ích đó (không giúp gì ở chế độ tiệm cận dài hạn nếu đã dùng averaging).

Về tính bất biến tham số hóa (mục 12): đây là một phân tích ”phê bình” đúng nghĩa của paper, không chỉ khẳng định tính bất biến mà còn chỉ rõ nó chỉ là xấp xỉ (approximate), với sai số tỉ lệ $O(\|d_\gamma\|^2)$ phụ thuộc độ cong của hàm đổi tham số hóa ζ và độ lớn bước đi; tính bất biến chính xác chỉ giữ được trong giới hạn bước đi tiến tới 0 (đường đi mượt lý tưởng), không giữ được chính xác cho bước hữu hạn trong thực hành.

6. Giới hạn

Các giới hạn mà **paper tự nêu rõ**, đặc biệt ở mục 15 ”Conclusions and Open Questions” (một phần thể hiện paper này khá trung thực/tự phê bình so với nhiều bài khác trong batch):

- **Lý thuyết hội tụ của chính paper không giải thích được hiện tượng thực nghiệm mà nó muốn lý giải.** Paper viết thẳng: các chặn hội tụ ở mục 14 (dựa trên giả định hàm mục tiêu là bậc hai với độ cong không đổi) ”luôn được cải thiện khi dùng Hessian thay vì GGN” theo lý thuyết đó, do vậy chính các chặn này **không thể giải thích** được việc GGN quan sát thực nghiệm lại tốt hơn Hessian khi huấn luyện mạng neural, và cũng không giải thích được vì sao kỹ thuật damping lại quan trọng đến vậy trong thực hành. Đây là một giới hạn cốt lõi, paper tự nhận và để lại như câu hỏi mở.
- Phần lý giải vì sao GGN có thể tốt hơn Hessian (mục 8.1) được paper tự gọi là ”speculation” (suy đoán), không phải kết quả đã chứng minh chặt.
- Kết quả ”Fisher efficiency” cổ điển của Amari cần bốn điều kiện ngặt (mục 5 trên đã liệt kê: hội tụ toàn cục, Fisher tính full-batch, đo theo tham số không theo hàm mục tiêu, và điều kiện ”realizability”), hầu hết không đúng với huấn luyện mạng neural thực tế.
- Phân tích tốc độ hội tụ (Theorem 5, 6) chỉ áp dụng cho hàm mục tiêu lồi bậc hai với nhiễu gradient có hiệp phương sai không đổi theo θ, một giả định paper tự nhận là ”somewhat unrealistic” (không thực tế lắm), chỉ xem là gần đúng khi θ đã gần điểm tối ưu.
- Tính bất biến tham số hóa chỉ là xấp xỉ cho bước hữu hạn, không chính xác tuyệt đối (mục 12, đã nêu ở mục 5).
- Paper để lại ba câu hỏi mở tường minh ở cuối bài: có thể chứng minh chặt lợi thế của GGN so với Hessian cho mạng neural không; có tình huống nào Fisher và GGN khác biệt rõ ràng và một trong hai chắc chắn tốt hơn; khi nào lợi thế ”trước tiệm cận” (pre-asymptotic) của phương pháp bậc hai so với SGD+averaging thực sự quan trọng trong thực hành.

Quan sát của người ghi chú (không phải paper tự phê bình): toàn bộ bài không có một thí nghiệm huấn luyện mạng neural cụ thể nào để kiểm chứng trực tiếp các định lý; mọi minh họa định lượng (như ví dụ $n = 1$ ở mục 11.1) đều là các case study toán học đơn giản, không phải benchmark thực nghiệm. Đây khác với kỳ vọng thông thường về một bài JMLR dài 76 trang, phần lớn độ dài nằm ở chứng minh toán học (phụ lục A-E, khoảng 1900 dòng) chứ không phải ở phần thực nghiệm.

7. Liên hệ với các bài khác

- **rl content/04_2015_trpo.md (đã xác nhận tồn tại):** TRPO (Trust Region Policy Optimization) là ví dụ ứng dụng trực tiếp và nổi tiếng nhất của natural gradient trong RL: TRPO ước lượng bước cập nhật policy bằng cách giải gần đúng $F^{-1}\nabla h$ (natural gradient) thông qua Fisher-vector product và conjugate gradient, chính là kỹ thuật tính matrix-vector product với F mà paper này mô tả ở mục 9.1 (nhân bằng J_f, rồi bằng F_R, rồi bằng $J_f^\top$, giống hệt cách Schraudolph tính matrix-vector product với GGN). Khái niệm ”trust region” trong TRPO cũng chính là kỹ thuật damping mà paper này đề xuất ở mục 10 để làm natural gradient ổn định với bước lớn.
- **math 07 (An Elementary Introduction to Information Geometry, bài sau trong batch):** tổng quát hóa chính cách hiểu hình học ở mục 3.3 của bài này (Fisher như một metric tensor Riemannian trên không gian phân phối) thành một khung đầy đủ của information geometry (affine connections, dually flat manifolds, họ phân phối mũ).
- **math 05 (A Tutorial on Spectral Clustering, bài trước trong batch):** cả hai bài đều xoay quanh một

ma trận ”đặc trưng” của mô hình/dữ liệu (Laplacian của đồ thị ở math 05, Fisher information ở bài này) mà ta cần hiểu rõ cấu trúc riêng (eigenvalue/eigenvector) của nó để thiết kế thuật toán hiệu quả; tuy nhiên hai bài dùng ma trận này theo cách khác nhau hẳn (phân cụm so với tối ưu hóa).

- **math 01 (The Matrix Calculus You Need For Deep Learning, bài đầu batch):** Fisher information matrix và GGN trong bài này đều được xây dựng trực tiếp từ Jacobian J_f của mạng neural theo tham số, đúng loại đối tượng (Jacobian, Hessian của hàm hợp) mà math 01 dạy cách tính bằng chain rule.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B, VLM OCR kiến trúc lai GatedDeltaNet kết hợp full attention, khoảng 3600 token/trang, chạy trên GPU không quá 8GB, batch size 1. Các liên hệ dưới đây đều là **suy luận** của người ghi chú:

- **Hạn chế bộ nhớ là rào cản chính để dùng natural gradient/K-FAC trực tiếp:** Fisher information matrix đầy đủ có kích thước $n \times n$ với n là số tham số, không thể lưu trữ hay nghịch đảo với mạng cỡ 0.8B tham số trên GPU 8GB. Các xấp xỉ thực hành (K-FAC dùng cấu trúc Kronecker theo từng lớp, hoặc xấp xỉ block-diagonal) vẫn tốn thêm đáng kể bộ nhớ và tính toán so với optimizer bậc một, nên **khó áp dụng trực tiếp** trong điều kiện ràng buộc GPU nhỏ, batch size 1 của dự án. Đây là lý do để nói ”ít liên quan trực tiếp” cho việc huấn luyện/fine-tune full model.
- **Liên hệ gián tiếp qua Adam/AdaGrad:** paper (mục 11.2) chỉ rõ Adam, RMSProp, AdaGrad đều dùng xấp xỉ diagonal của **empirical Fisher** (không phải Fisher thật) làm preconditioner. Nếu dự án đang dùng Adam để fine-tune hoặc distill mô hình, đây chính là một dạng ”natural-gradient-rất-thô” đang được áp dụng ngầm; hiểu rõ điểm yếu của empirical Fisher (mục 3.7, ví dụ tự tính) giúp giải thích tại sao Adam có thể kém ổn định trên một số lịch học rate lớn, và gợi ý rằng việc tinh chỉnh ϵ/weight decay trong Adam thực chất là đang ”vá” đúng vấn đề về sai lệch quy mô (scale) mà paper mô tả cho empirical Fisher.
- **K-FAC/natural gradient có thể hữu ích hơn ở quy mô nhỏ hơn**, ví dụ khi chỉ fine-tune một adapter nhỏ (LoRA) hoặc một đầu phân loại nhỏ gắn thêm vào mô hình OCR, nơi số tham số cần tối ưu nhỏ hơn nhiều, khiến việc tính/lưu một xấp xỉ Fisher trở nên khả thi trên GPU 8GB. Đây là một gợi ý suy luận, paper không đề cập cụ thể tới LoRA hay fine-tuning adapter.

Tổng thể, bài này **ít liên quan trực tiếp** đến việc huấn luyện/nén mô hình full-scale của dự án do ràng buộc bộ nhớ nghiêm trọng của natural gradient ở quy mô lớn, nhưng cung cấp nền tảng lý thuyết hữu ích để hiểu cơ chế của các optimizer bậc một (Adam) đang dùng trong thực hành, và có thể liên quan trực tiếp hơn nếu dự án áp dụng kỹ thuật fine-tuning quy mô nhỏ (adapter/LoRA).

9. Thí nghiệm gợi ý

1. **Tái tạo ví dụ $n = 1$ của paper (mục 11.1) bằng code:** cài đặt chính xác ví dụ $f(x, \theta) = \theta$, $R_{y|z} = \mathcal{N}(z, 1)$, $h(\theta) = \frac{1}{2}\theta^2$ bằng PyTorch, chạy cả hai dãy cập nhật (dùng $F = 1$ và dùng $\bar{F} = \theta^2$) với vài giá trị α và θ_0 khác nhau, vẽ đồ thị θ_k theo k. Đo: giá trị α tối đa còn giữ được sự hội tụ cho mỗi loại cập nhật. Kỳ vọng: xác nhận lại kết quả tự tính ở mục 3.7 (Fisher ổn định, empirical Fisher dễ phân kỳ với bước cố định).
2. **So sánh SGD, Adam, và một xấp xỉ diagonal Fisher thật (không phải empirical) trên một mạng nhỏ** (ví dụ MLP 2-3 lớp trên MNIST hoặc một bài toán hồi quy toy), ước lượng diagonal Fisher thật bằng cách sample y từ phân phối dự đoán của mô hình rồi lấy gradient theo mẫu đó (đúng kỹ thuật mà paper gợi ý ở mục 11.1, trích dẫn Chapelle & Erhan 2011). Đo tốc độ giảm loss theo số iteration. Kỳ vọng: diagonal Fisher thật hội tụ ổn định hơn diagonal empirical Fisher thô ở learning rate cao.
3. **Đo điều kiện (condition number) của Hessian/GGN cho một lớp Linear nhỏ** trong một mô hình OCR toy (hoặc một mạng con nhỏ của LightOnOCR nếu khả thi trích xuất), ước lượng Hessian-vector product và GGN-vector product bằng kỹ thuật forward-over-reverse (liên hệ math 02), so sánh phổ

eigenvalue (qua power iteration) của hai ma trận này trên vài batch nhỏ. Đo: tỉ số điều kiện κ, tần suất xuất hiện eigenvalue âm của Hessian thật. Kỳ vọng: xác nhận GGN luôn PSD trong khi Hessian có thể có vài eigenvalue âm nhỏ.

4. **Kiểm tra tính bất biến tham số hóa gần đúng:** lấy một mạng nhỏ, huấn luyện 2 phiên bản tương đương về mặt hàm số nhưng tham số hóa khác nhau (ví dụ: w trực tiếp so với $w = \exp(u)$ cho một trọng số dương), so sánh đường đi loss khi dùng SGD thường (kỳ vọng khác nhau rõ rệt giữa 2 tham số hóa) so với khi dùng một bước xấp xỉ natural-gradient/diagonal-Fisher (kỳ vọng gần giống nhau hơn, theo đúng Theorem 1/Corollary 2 của paper).
5. **Thử K-FAC đơn giản hóa (per-layer, không Kronecker đầy đủ) trên 1-2 lớp Linear nhỏ nhất của một mô hình nhỏ**, đo thời gian và bộ nhớ cần thêm so với Adam trên cùng GPU, để tự kiểm chứng trực tiếp ”rào cản bộ nhớ” đã nêu ở mục 8, từ đó định lượng cụ thể xem natural-gradient-xấp-xỉ có thực sự khả thi ở quy mô lớn hơn một chút (ví dụ vài triệu tham số) trên GPU 8GB hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Natural gradient	$\tilde{\nabla} h = F^{-1} \nabla h$
Fisher information matrix	$F = \mathbb{E}_{P_{x,y}}[\nabla \log p(x,y\|\theta) \nabla \log p(x,y\|\theta)^\top] = -\mathbb{E}_{P_{x,y}}[H_{\log p}]$, luôn PSD
Xấp xỉ KL bậc hai	$KL(P(\theta+d)\|P(\theta)) \approx \frac{1}{2} d^\top F d$, cho lý giải hình học của natural gradient
Mô hình bậc hai cục bộ	$M_k(\delta) = \frac{1}{2}\delta^\top B_k \delta + \nabla h(\theta_k)^\top \delta + h(\theta_k)$; $B_k = \beta I$ là gradient descent, $B_k = H$ là Newton
Generalized Gauss-Newton (GGN)	$G = \frac{1}{\|S\|} \sum J_f^\top H_L J_f$, luôn PSD vì L lồi theo z
Điều kiện $F = G$	Đúng khi $R_{y\|z}$ thuộc họ phân phối mũ với z là tham số tự nhiên (ví dụ Gauss với $z = \mu$, softmax/multinomial)
Empirical Fisher	$\bar{F} = \frac{1}{\|S\|} \sum \nabla \log p(y\|x,\theta) \nabla \log p(y\|x,\theta)^\top$, dễ tính nhưng không xấp xỉ tốt H nói chung
Ví dụ tự tính (mục 3.7)	$\theta_0 = 2$, $\alpha = 0.5$: cập nhật bằng $F = 1$ cho dãy $2, 1, 0.5, 0.25, \ldots$ (hội tụ đều); cập nhật bằng $\bar{F} = \theta^2$ cho dãy $2, 1.75, 1.464, 1.123, 0.677, -0.061, 8.14, \ldots$ (phân kỳ)
Điều kiện bất biến tham số hóa	$J_\zeta^\top B_\theta J_\zeta = B_\gamma$; đúng cho $B = \mathbb{E}[J_f^\top A J_f]$ (GGN, Fisher, empirical Fisher), không đúng cho Hessian (trừ ζ affine)
Tốc độ hội tụ tiệm cận (quadratic, SGD bậc hai tổng quát)	$\mathbb{E}[h(\theta_k)] - h(\theta^*)$ tách thành phần phụ thuộc nhiễu (không cải thiện bởi 2nd-order khi có Polyak averaging) và phần phụ thuộc θ_0 (cải thiện mạnh bởi 2nd-order khi H^* ill-conditioned)

11. Kết luận ngắn

Paper đóng góp một cách nhìn mới, mang tính thống nhất cho natural gradient: thay vì chỉ là một phép ”chuyển không gian” theo information geometry, natural gradient nên được hiểu là một phương pháp tối ưu bậc hai dùng Fisher information matrix làm ma trận độ cong, và trong rất nhiều trường hợp quan trọng (loss cross-entropy, squared error với các phân phối mũ tương ứng), Fisher trùng với Generalized Gauss-Newton matrix, một ma trận luôn PSD. Cách nhìn này lý giải tại sao các kỹ thuật damping/trust-region của tối ưu hóa bậc hai cổ điển lại cần thiết trong thực hành, và giúp phê bình chính xác điểm yếu của ”empirical Fisher” hay bị dùng nhầm (minh họa bằng ví dụ tính tay ở mục 3.7). Tuy nhiên, điểm đáng chú ý nhất về mặt khoa học là sự trung thực của tác giả ở phần kết luận: chính lý thuyết hội tụ mà paper xây dựng công nhận không giải thích được đầy đủ vì sao GGN/Fisher lại vượt trội so với Hessian trong thực hành huấn luyện mạng neural, để lại đây như câu hỏi mở cho nghiên cứu sau này.

Nguồn gốc: https://arxiv.org/abs/1412.1193

Bài 07

An Elementary Introduction to Information Geometry (2018)

Tác giả: Frank Nielsen (Sony Computer Science Laboratories Inc, Tokyo, Japan)
Venue / arXiv: arXiv:1808.08271 (v2, 06/09/2020); paper không nêu venue hội nghị/tạp chí, phần lời cảm ơn cho biết đây là bài viết dựa trên một bài nói chuyện (invited talk) tại workshop ”Geometry in Machine Learning” (GiMLi) năm 2018
Nguồn: https://arxiv.org/abs/1808.08271
Chủ đề: Hình học thông tin (information geometry), ma trận Fisher information như metric tensor, kết nối đối ngẫu (dual connection), đa tạp dually flat, liên hệ với natural gradient
Vị trí trong lộ trình: Bài số 7 trong 9 bài toán nền (math 01-09). Đọc sau math 06 (natural gradient method), vì bài này tổng quát hóa khung hình học đứng sau khái niệm Fisher information matrix đã gặp ở math 06. Đứng trước math 08 (variational inference), vốn cũng dùng KL divergence nhưng cho mục đích suy luận Bayes xấp xỉ, khác với mục đích xây cấu trúc hình học ở đây.

> Ghi chú này dựa trên toàn văn bài báo (đọc từ PDF chuyển sang text, khoảng 64 trang, 3132 dòng). Bài có một danh mục tài liệu tham khảo rất dài ở cuối (khoảng 5-6 trang, gần 150 mục); phần này chỉ được đọc lướt để xác nhận đó thực sự là danh mục trích dẫn thuần túy, không được tóm tắt từng mục. Toàn bộ phần nội dung chính (Mục 1-5, Phụ lục A, Phụ lục B, trang tổng hợp ký hiệu) đã được đọc đầy đủ. Đây là bài có mật độ công thức rất cao và nhiều khái niệm hình học vi phân trừu tượng (manifold, connection, curvature); ghi chú này cố gắng diễn giải lại các khái niệm cốt lõi cho người chưa học hình học vi phân (không phải bản dịch), có thể phải bỏ qua một số chi tiết kỹ thuật rất sâu (ví dụ công thức tổng quát của Bregman ball, phần α-embedding chi tiết) để giữ trọng tâm. Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Bài này là một survey của Frank Nielsen, trình bày lại các cấu trúc hình học vi phân nền tảng của ”information geometry” (hình học thông tin, viết tắt IG), nêu ”định lý nền tảng của hình học thông tin” (fundamental theorem of information geometry), và minh họa một vài ứng dụng trong khoa học thông tin (kiểm định giả thuyết Bayes, phân cụm hỗn hợp thống kê). Paper tự nêu rõ ngay phần tóm tắt: ”proofs are omitted for brevity” (các chứng minh bị bỏ qua để bài ngắn gọn), nghĩa là đây là một bài tổng hợp kiến thức có tính giáo dục, không phải một công bố kết quả nghiên cứu mới, và phần lớn các định lý được trích dẫn lại từ các giáo trình/bài báo khác (đặc biệt là hai cuốn sách chuyên khảo của Amari, ký hiệu [8] và [25] trong paper) chứ không được chứng minh tại chỗ.

Theo lời cảm ơn ở cuối bài, đây là bài viết dựa trên một bài nói chuyện mời (invited talk) tại workshop ”Geometry in Machine Learning” (GiMLi) năm 2018, không công bố tại một hội nghị hay tạp chí cụ thể nào trong chính văn bản arXiv này.

Paper có một phần lịch sử khá chi tiết (mục 5.2) đáng chú ý: hình học thông tin bắt nguồn từ việc Harold Hotelling (cuối những năm 1920, ghi chú tay năm 1929) và độc lập C. R. Rao (1945, trong bài báo nổi tiếng nêu ra Cramér-Rao bound) coi ma trận Fisher information là một metric tensor Riemannian trên không gian tham số của một họ phân phối xác suất. Nikolai Chentsov (thập niên 1960-1970, chuyên khảo tiếng Nga) chứng minh Fisher information metric là metric bất biến duy nhất (up to hệ số tỉ lệ) dưới các phép biến đổi Markov. Shun-ichi Amari (từ 1980) phát triển cấu trúc đối ngẫu (dualistic) đầy đủ và đặt tên lĩnh vực là ”information geometry”. Paper trích dẫn trực tiếp định nghĩa của Amari: ”Information geometry is a method of exploring the world of information by means of modern geometry.”

Ghi chú này đọc toàn bộ nội dung chính của bài (các mục 1 đến 5, Phụ lục A về ước lượng Monte Carlo cho

f-divergence, Phụ lục B về phân rã canonical của họ Gaussian đa biến, và trang tổng hợp ký hiệu ở cuối), chỉ đọc lướt phần danh mục tham khảo (reference list) vì đó thuần túy là các mục trích dẫn, không chứa nội dung kỹ thuật cần tóm tắt.

2. Ý tưởng chính

Ý tưởng cốt lõi: một họ tham số các phân phối xác suất $\mathcal{P} = \{p_\theta(x)\}_{\theta\in\Theta}$ có thể được nhìn như một "đa tạp" (manifold) M, trong đó mỗi điểm trên đa tạp là một phân phối xác suất cụ thể, và tọa độ địa phương của điểm đó chính là vector tham số θ. Trên một đa tạp trơn như vậy, paper phân biệt rõ hai cấu trúc độc lập có thể đặt lên:

1. **Metric tensor g:** cho phép đo "độ dài" và góc giữa các vector tiếp xúc (tangent vector) tại mỗi điểm, tức đo khoảng cách vô cùng nhỏ giữa hai phân phối rất gần nhau.
2. **Affine connection ∇:** một toán tử vi phân cho phép định nghĩa đạo hàm hiệp biến (covariant derivative) của một trường vector theo một trường vector khác, từ đó định nghĩa được khái niệm "đường thẳng" tổng quát trên đa tạp (gọi là geodesic, đường tự song song/autoparallel) và cách "vận chuyển song song" (parallel transport) một vector giữa các mặt phẳng tiếp xúc khác nhau.

Điểm khác biệt căn bản giữa hình học thông tin và hình học Riemannian cổ điển: trong hình học Riemannian, với một metric g cho trước, có **duy nhất một** connection thỏa hai điều kiện (không xoắn, tương thích với metric), gọi là connection Levi-Civita ${}^{LC}\nabla$ (Định lý nền tảng của hình học Riemannian, Theorem 1 trong paper). Hình học thông tin đi xa hơn: nó xét **một cặp connection đối ngẫu (conjugate)** (∇, ∇^*) cùng khớp với một metric g (cấu trúc (M, g, ∇, ∇^*)), và connection Levi-Civita chỉ là trường hợp đặc biệt tự đối ngẫu $\nabla = \nabla^* = {}^{LC}\nabla$.

Với một họ tham số thống kê, lựa chọn chính tắc cho metric g là **ma trận Fisher information (FIM)**: paper chứng minh (trích dẫn Chentsov/Amari-Nagaoka) đây là metric **duy nhất** (sai khác một hệ số tỉ lệ) bất biến dưới các phép biến đổi gọi là "Markov embedding" (Theorem 8). Cặp connection đối ngẫu chính tắc tương ứng là connection mũ (exponential connection ∇^e) và connection hỗn hợp (mixture connection ∇^m).

"Định lý nền tảng của hình học thông tin" (Theorem 3): nếu một connection không xoắn ∇ có độ cong hằng κ, thì connection đối ngẫu ∇^* của nó cũng có độ cong hằng κ. Trường hợp đặc biệt và quan trọng nhất trong thực hành là $\kappa = 0$: khi cả ∇ và ∇^* đều "flat" (phẳng, không có độ cong), ta có một **đa tạp dually flat** (Corollary 2). Loại đa tạp này xuất hiện tự nhiên với họ hàm mũ (exponential family, ví dụ Gaussian) và họ hỗn hợp (mixture family), và có cấu trúc đại số rất đẹp: hai hệ tọa độ toàn cục θ và η liên hệ qua biến đổi Legendre-Fenchel, cùng một định lý Pythagore tổng quát (bất đối xứng).

3. Phương pháp chi tiết

3.1. Manifold, metric tensor và affine connection

Một đa tạp trơn D-chiều M là một không gian tôpô mà cục bộ (ở lân cận mỗi điểm) "giống" không gian Euclid $\mathbb{R}^D$. Tại mỗi điểm $p \in M$, có một mặt phẳng tiếp xúc (tangent plane) T_p, là một không gian vector chứa các "hướng đi" khả dĩ tại p. Paper định nghĩa hai cấu trúc trên M:

- **Metric tensor g:** với hai vector tiếp xúc $u, v \in T_p$, $g_p(u, v) = \langle u, v\rangle_p$ là một dạng song tuyến tính đối xứng, xác định dương, cho phép tính độ dài $\|v\| = \sqrt{g(v, v)}$ và tính trực giao ($u \perp v$ khi $g(u, v) = 0$).
- **Affine connection ∇:** cho phép định nghĩa đạo hàm hiệp biến $\nabla_X Y$ của trường vector Y theo trường vector X. Trong hệ tọa độ địa phương, connection được xác định bởi các hệ số Christoffel Γ^k_{ij}, với $\nabla_{\partial_i}\partial_j = \Gamma^k_{ij}\partial_k$.

Từ connection, ta có được: (1) phép vận chuyển song song của vector giữa các mặt tiếp xúc dọc theo một đường cong trơn, (2) các ∇-geodesic (đường tự song song, nghiệm của phương trình vi phân bậc hai $\ddot\gamma + \Gamma^k_{ij}\dot\gamma\dot\gamma = 0$), (3) độ cong và độ xoắn (torsion) nội tại của đa tạp.

Connection gọi là tương thích với metric (metric-compatible) nếu $X\langle Y,Z\rangle = \langle \nabla_X Y, Z\rangle + \langle Y, \nabla_X Z\rangle$ với mọi trường vector X, Y, Z. Định lý nền tảng của hình học Riemannian (Theorem 1 của paper): tồn tại duy nhất một connection không xoắn tương thích với metric, gọi là connection Levi-Civita ${}^{LC}\nabla$, và hình học Riemannian cổ điển (M, g) chính là trường hợp đặc biệt $(M, g, {}^{LC}\nabla)$.

3.2. Cấu trúc đối ngẫu: connection liên hợp (conjugate connection)

Đây là bước mở rộng cốt lõi của hình học thông tin. Paper định nghĩa (Definition 1): connection ∇^* được gọi là **liên hợp (conjugate)** với ∇ đối với metric g nếu với mọi trường vector X, Y, Z:

$$X\langle Y,Z\rangle = \langle \nabla_X Y, Z\rangle + \langle Y, \nabla^*_X Z\rangle$$

So với điều kiện tương thích metric thông thường (mục 3.1), ở đây đạo hàm của Y dùng connection ∇, còn đạo hàm của Z dùng connection khác ∇^*. Tính chất liên hợp là một phép đối hợp: $(\nabla^*)^* = \nabla$. Cấu trúc (M, g, ∇, ∇^*) gọi là **Conjugate Connection Manifold (CCM)**.

Một hệ quả quan trọng (Property 1, công thức 38-39 của paper): phép vận chuyển song song đối ngẫu bảo toàn metric, tức nếu vận chuyển u theo ∇ và v theo ∇^* dọc cùng một đường cong, tích vô hướng $\langle u, v\rangle$ không đổi theo thời gian. Paper cũng chỉ ra (Property 3): connection trung bình $\bar{\nabla} = (\nabla + \nabla^*)/2$ luôn bằng chính connection Levi-Civita, tức ${}^{LC}\nabla$ nằm "ở giữa" cặp connection đối ngẫu.

Từ cặp (∇, ∇^*), paper định nghĩa tensor khối (cubic tensor) Amari-Chentsov, hoàn toàn đối xứng:

$$C_{ijk} := \Gamma^k_{ij} - \Gamma^{*k}_{ij}$$

Cấu trúc (M, g, C) gọi là **statistical manifold** theo định nghĩa của Lauritzen (1987); paper lưu ý tên gọi này có thể gây nhầm, vì đây thuần túy là một cấu trúc hình học, không bắt buộc liên quan đến thống kê (dù luôn có thể tìm một mô hình thống kê tương ứng).

3.3. Họ connection α và định lý nền tảng

Từ một cặp conjugate (∇, ∇^*), paper xây dựng một họ liên tục theo tham số $\alpha \in \mathbb{R}$, gọi là α-connection:

$$\Gamma^{\alpha}_{ij,k} = \Gamma^0_{ij,k} - \frac{\alpha}{2} C_{ij,k}$$

với Γ^0 là hệ số Christoffel của connection Levi-Civita. Khi $\alpha = 0$ ta có lại Levi-Civita; khi $\alpha = 1$ ta có lại ∇; khi $\alpha = -1$ ta có lại ∇^*.

Định lý nền tảng của hình học thông tin (Theorem 3 của paper, trích dẫn chứng minh từ [25]): nếu một connection không xoắn ∇ có độ cong hằng κ, thì connection liên hợp ∇^* của nó cũng có độ cong hằng κ (y hệt). Hệ quả quan trọng nhất (Corollary 2): đa tạp (M, g, ∇, ∇^*) là ∇-flat (độ cong $\kappa = 0$) khi và chỉ khi nó cũng ∇^*-flat. Trường hợp $\kappa = 0$ này được gọi là **dually flat manifold**, và đây là trường hợp quan trọng nhất trong các ứng dụng thực hành vì có cấu trúc đại số đơn giản (xem mục 3.4).

3.4. Đa tạp dually flat và hình học Bregman

Paper xây dựng đa tạp dually flat từ một hàm sinh (generator) $F(\theta)$ lồi chặt, trơn C^3, gọi là potential function. Từ F, định nghĩa **Bregman divergence**:

$$B_F(\theta : \theta') := F(\theta) - F(\theta') - (\theta - \theta')^\top \nabla F(\theta')$$

Cấu trúc hình học sinh ra từ B_F có các hệ số Christoffel $\Gamma^F_{ij,k}(\theta) = 0$ với mọi θ (Eq. 59 của paper), tức connection này luôn phẳng (flat) theo xây dựng, với **bất kỳ** hàm lồi F nào, không riêng gì các họ phân phối xác suất. Metric tensor tương ứng chính là Hessian của F: $g^F = \nabla^2 F(\theta)$.

Dùng biến đổi Legendre-Fenchel, định nghĩa hàm đối ngẫu $F^*(\eta) := \sup_\theta\{\theta^\top \eta - F(\theta)\}$, và hệ tọa độ đối ngẫu $\eta = \nabla F(\theta)$, $\theta = \nabla F^*(\eta)$. Paper nêu identity của Crouzeix: $\nabla^2 F(\theta)\,\nabla^2 F^*(\eta) = I$. Trên đa tạp dually flat, mọi điểm đều có hai hệ tọa độ toàn cục θ và η, và một cặp điểm P, Q có thể nối bằng geodesic "thẳng theo θ" (∇-geodesic) hoặc "thẳng theo η" (∇^*-geodesic).

Kết quả đẹp nhất của cấu trúc này là **định lý Pythagore đối ngẫu** (Theorem 6, minh họa ở Hình 6 của paper): nếu geodesic đối ngẫu $\gamma^*(P, Q)$ vuông góc với geodesic $\gamma(Q, R)$ (theo metric g) thì:

$$D(P : R) = D(P : Q) + D(Q : R)$$

đây là một dạng tổng quát hóa định lý Pythagore của Euclid cho không gian "cong" nhưng dually flat, với D là divergence (không đối xứng) thay cho khoảng cách Euclid thường.

3.5. Ma trận Fisher information và tính bất biến duy nhất

Cho một họ tham số $\mathcal{P} = \{p_\theta(x)\}_\theta$, với log-likelihood $l(\theta; x) := \log p_\theta(x)$ và score vector $s_\theta = \nabla_\theta l$. Ma trận Fisher information (FIM) được định nghĩa:

$$I(\theta) := \mathbb{E}_\theta[\partial_i l\, \partial_j l]_{ij} \succeq 0$$

Paper chỉ ra FIM là bất biến dưới đổi biến không gian mẫu và biến đổi hiệp biến (covariant) dưới đổi tham số: nếu $\bar{p}(x; \eta) = p(\theta(\eta); x)$, thì $\bar{I}(\eta) = J^\top I(\theta(\eta)) J$ với $J = [\partial\theta_i/\partial\eta_j]$ là ma trận Jacobian (Eq. 81).

Theorem 8 (tính duy nhất của Fisher information metric): Fisher information metric là metric tensor bất biến duy nhất (sai khác một hệ số tỉ lệ) dưới các phép nhúng Markov (Markov embedding). Nói cách khác, trong số vô số metric tensor có thể đặt lên không gian tham số của một mô hình thống kê, chỉ có Fisher metric (và các phiên bản co giãn của nó) thỏa mãn một yêu cầu bất biến tự nhiên trong thống kê. Đây là lý do toán học vì sao Fisher information "phải" được dùng làm metric chuẩn, không phải một lựa chọn tùy ý trong số nhiều lựa chọn tương đương.

Paper cho công thức FIM cho hai loại họ tham số quan trọng:

- **Họ hàm mũ (exponential family)** $p_\theta(x) = \exp(\sum_i t_i(x)\theta_i - F(\theta) + k(x))$ với F là hàm cumulant (log-normalizer) lồi chặt: $I(\theta) = \mathrm{Cov}_{p_\theta}[t(x)] = \nabla^2 F(\theta)$ (Eq. 90). Tức với họ hàm mũ, FIM chính là Hessian của hàm cumulant, và connection sinh ra tự động là dually flat.
- **Họ hỗn hợp (mixture family)** $p_\theta(x) = \sum_i \theta_i F_i(x) + C(x)$: $I(\theta) = \mathbb{E}\left[\dfrac{F_i(x)F_j(x)}{p_\theta(x)^2}\right]$ (Eq. 94).

Ví dụ cụ thể paper cho với Gaussian đơn biến $\mathcal{N}(\mu, \sigma)$: theo tham số hóa $\lambda = (\mu, \sigma)$, $I_\lambda(\lambda) = \mathrm{diag}(1/\sigma^2,\ 2/\sigma^2)$; theo tham số hóa $\lambda' = (\mu, \sigma^2)$, $I_{\lambda'}(\lambda') = \mathrm{diag}(1/\sigma^2,\ 1/(2\sigma^4))$ (Eq. 83-84). Hai ma trận này khác nhau nhưng liên hệ đúng theo công thức biến đổi hiệp biến ở trên, không mâu thuẫn.

3.6. Fisher-Rao distance và hai connection chính tắc cho thống kê

Khi coi FIM là metric Riemannian $g = I(\theta)$ trên $\mathcal{P}$, khoảng cách geodesic Riemannian tương ứng gọi là **Fisher-Rao distance**:

$$D_\rho(p_{\theta_1}, p_{\theta_2}) = \inf_\gamma \int_0^1 \sqrt{\dot\gamma(t)^\top g_{\gamma(t)} \dot\gamma(t)}\, dt$$

Paper cho ba ví dụ cụ thể về hình học sinh ra: họ phân phối rời rạc hữu hạn (categorical) cho hình học cầu (spherical); họ location-scale hai biến cho hình học hyperbolic; họ chỉ-dịch-chuyển (location family, scale cố định) cho hình học Euclid thường.

Hai connection chính tắc gắn với Fisher metric cho một họ tham số thống kê là connection mũ ∇^e và connection hỗn hợp ∇^m (Eq. 96-97 của paper). Paper chỉ ra (Eq. 104): với chính họ hàm mũ E và họ hỗn hợp M, các hệ số Christoffel của cả ∇^e và ∇^m đều triệt tiêu, nghĩa là cả hai loại họ phân phối này tự động là dually flat theo đúng cặp connection chính tắc của chúng, không cần xây dựng gượng ép.

3.7. Ứng dụng natural gradient (liên hệ trực tiếp math 06)

Paper trình bày lại natural gradient trong khung hình học này (mục 4.1). Gradient thường (ordinary gradient descent) không bất biến khi đổi tham số hóa: nếu $\eta = \eta(\theta)$ là một đổi biến trơn, hai dãy gradient descent trên θ và trên η nói chung hội tụ về các điểm khác nhau, vì gradient thường không tính đến hình học (độ cong, metric) của không gian tham số. Paper minh họa bằng ví dụ rất trực quan: tối thiểu hóa một hàm nhiệt độ theo độ C và theo độ F (cùng một đại lượng vật lý, hai cách đo) có thể cho hai đường đi tối ưu khác nhau nếu dùng gradient thường.

Natural gradient khắc phục vấn đề này bằng cách định nghĩa:

$$^{NG}\nabla L_\theta(\theta) := g_\theta^{-1}(\theta)\,\nabla_\theta L_\theta(\theta)$$

đây xấp xỉ hướng dốc nhất nội tại (intrinsic Riemannian steepest descent) qua một phép xấp xỉ bậc một của ánh xạ mũ Riemannian (Riemannian exponential map), vốn thường không tính được trong thực hành (Property 5 của paper). Paper chứng minh một kết quả đẹp (**Property 7**, suy ra trực tiếp từ chain rule, Eq. 183-186): trên một đa tạp dually flat sinh từ hàm F, với $g_\theta(\theta) = \nabla^2 F(\theta) = \nabla_\theta \eta$, ta có:

$$\nabla_\theta L_\theta(\theta) = (\nabla_\theta \eta)(\nabla_\eta L_\eta(\eta)) \quad \Rightarrow \quad {}^{NG}\nabla L_\theta(\theta) = g_\theta^{-1}(\theta)\nabla_\theta L_\theta(\theta) = (\nabla_\theta \eta)^{-1}(\nabla_\theta \eta)\nabla_\eta L_\eta(\eta) = \nabla_\eta L_\eta(\eta)$$

Nói cách khác: **natural gradient descent trên tham số hóa θ chính là gradient descent thường trên tham số hóa đối ngẫu $\eta = \nabla F(\theta)$**. Đây là một cách diễn đạt rất gọn và sâu sắc cho chính hiện tượng "bất biến theo tham số hóa" mà bài math 06 (Martens) thảo luận bằng ngôn ngữ đại số tuyến tính (Fisher = Generalized Gauss-Newton).

3.8. Ví dụ số tự tính: Fisher information của phân phối Bernoulli

Paper chỉ cho công thức FIM minh họa cụ thể bằng số cho họ Gaussian (mục 3.5 trên), không tính cho phân phối Bernoulli. Để tự kiểm chứng công thức tổng quát $I(\theta) = \mathbb{E}_\theta[(\partial_\theta l)^2]$ (Eq. 80, trường hợp 1 chiều) bằng một ví dụ khác (tự tính), xét $X \sim \text{Bernoulli}(\theta)$, $p_\theta(x) = \theta^x(1-\theta)^{1-x}$ với $x \in \{0, 1\}$.

Log-likelihood: $l(\theta; x) = x \log\theta + (1-x)\log(1-\theta)$. Đạo hàm theo θ:

$$\partial_\theta l = \frac{x}{\theta} - \frac{1-x}{1-\theta}$$

Tại $x = 1$: $\partial_\theta l = 1/\theta$. Tại $x = 0$: $\partial_\theta l = -1/(1-\theta)$. Áp dụng định nghĩa FIM (lấy kỳ vọng theo $x \sim \text{Bernoulli}(\theta)$, $P(x=1) = \theta$, $P(x=0) = 1-\theta$):

$$I(\theta) = \theta \cdot \left(\frac{1}{\theta}\right)^2 + (1-\theta)\cdot\left(\frac{1}{1-\theta}\right)^2 = \frac{1}{\theta} + \frac{1}{1-\theta} = \frac{1}{\theta(1-\theta)}$$

Thay số cụ thể (tự tính): tại $\theta = 0.5$: $I(0.5) = 1/(0.5 \times 0.5) = 4$. Tại $\theta = 0.2$: $I(0.2) = 1/(0.2 \times 0.8) = 1/0.16 = 6.25$. Tại $\theta = 0.05$: $I(0.05) = 1/(0.05 \times 0.95) \approx 21.05$.

Quan sát (của người ghi chú, suy ra từ phép tính trên, không phải trích từ paper): $I(\theta)$ nhỏ nhất tại $\theta = 0.5$ và tăng vọt khi θ tiến về 0 hoặc 1. Điều này có nghĩa là, theo đúng tinh thần "Fisher information đo lượng thông tin về θ mà một quan sát mang lại", khi θ gần biên (0 hoặc 1, kết quả gần như chắc chắn), một mẫu quan sát duy nhất lại mang nhiều thông tin hơn về giá trị chính xác của θ so với khi $\theta = 0.5$ (kết quả ngẫu nhiên nhất). Theo Cramér-Rao bound mà paper nhắc tới (Eq. 88), phương sai của một ước lượng không chệch của θ bị chặn dưới bởi $\frac{1}{n} I(\theta)^{-1} = \frac{\theta(1-\theta)}{n}$, đây đúng là công thức phương sai chuẩn của ước lượng tỉ lệ mẫu (sample proportion) quen thuộc trong thống kê cơ bản, cho thấy công thức FIM tổng quát của paper khớp với một kết quả đã biết.

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial/survey lý thuyết, tự nêu rõ bỏ qua các chứng minh chi tiết, và **không có benchmark thực nghiệm học máy nào** theo nghĩa huấn luyện mô hình, so sánh với baseline, hay đo trên một bộ dữ liệu chuẩn. "Kết quả chính" của bài là các định lý và cấu trúc hình học:

- **Theorem 3 (định lý nền tảng):** độ cong hằng của ∇ và connection liên hợp ∇^* của nó luôn bằng nhau; hệ quả quan trọng nhất là tính dually flat (Corollary 2).
- **Theorem 8:** Fisher information metric là metric bất biến duy nhất dưới Markov embedding.
- **Theorem 6 (Pythagore đối ngẫu)** trên đa tạp dually flat.
- **Property 7:** natural gradient trên θ bằng gradient thường trên tọa độ đối ngẫu η.

Về phần minh họa bằng số, paper có một vài hình vẽ mang tính mô phỏng nhỏ, gần nhất với "thực nghiệm" trong bài, cần phân biệt rõ đây không phải benchmark có baseline:

- **Hình 8 (minh họa Cramér-Rao bound):** paper lấy các vị trí lưới (μ, σ) của tham số Gaussian, với mỗi vị trí lặp lại 200 lần việc ước lượng MLE của (μ, σ) từ 100 mẫu iid $\sim \mathcal{N}(\mu, \sigma)$, rồi vẽ elip biểu diễn ma trận hiệp phương sai mẫu của các ước lượng MLE (elip đen) so với elip biểu diễn chính ma trận Fisher information tại vị trí lưới đó (elip đỏ). Paper quan sát: elip Fisher (đỏ) luôn có trục song song với hệ tọa độ (vì FIM của Gaussian là ma trận đường chéo theo (μ, σ)), còn elip hiệp phương sai mẫu (đen) của MLE thì không, và tâm của elip đen có thể lệch khỏi đúng vị trí lưới.
- **Hình 14-15 (minh họa phân cụm hỗn hợp thống kê):** paper dựng một họ hỗn hợp (mixture family) bậc $D = 2$ từ 3 phân phối thành phần cho trước (Laplace, Gaussian, Cauchy), minh họa 2 hỗn hợp cụ thể (Hình 14), sau đó áp dụng Bregman k-means trên một đa tạp dually flat xấp xỉ bằng Monte Carlo (vì entropy của hỗn hợp không có dạng đóng) để phân một tập các w-GMM (Gaussian Mixture Model với trọng số khác nhau nhưng cùng tham số thành phần) thành $k = 2$ cụm (Hình 15).

Cả hai minh họa này đều mang tính trực quan hóa khái niệm, không có so sánh định lượng với phương pháp khác hay số liệu bảng biểu kèm theo trong văn bản được đọc.

5. Phân tích / ablation

Vì không có thí nghiệm định lượng, phần này tập trung phân tích các lựa chọn cấu trúc mà paper trình bày và sự khác biệt giữa chúng:

Fisher-Rao distance (metric thật) vs Bregman/KL divergence (không đối xứng): paper tự tổng kết sự khác biệt này trong Hình 10 (được mô tả lại bằng lời ở đây): Fisher-Rao distance là một khoảng cách Riemannian thật (đối xứng, thỏa bất đẳng thức tam giác) nhưng **thường không có dạng đóng** và phải tính xấp xỉ bằng số; còn Bregman divergence (sinh từ đa tạp dually flat) không đối xứng nói chung, nhưng geodesic trên đa tạp dually flat là đường thẳng trong hệ tọa độ phù hợp, nên tính toán rất hiệu quả. Đây là một trade-off cốt lõi của toàn bộ khung lý thuyết: đổi tính đối xứng/đúng nghĩa "khoảng cách" để lấy khả năng tính toán khép kín.

Vai trò của tham số α: họ α-connection cho một cách nội suy liên tục giữa connection Levi-Civita tự đối ngẫu ($\alpha = 0$) và cặp connection "cực trị" (∇, ∇^*) ($\alpha = \pm 1$). Với họ hàm mũ và họ hỗn hợp, $\alpha = \pm 1$ (connection mũ/hỗn hợp) cho dually flat; các giá trị α khác nói chung không cho connection phẳng.

Fisher information thực nghiệm có thể không đường chéo dù Fisher information lý thuyết là đường chéo: đây là một quan sát rất cụ thể mà paper tự nêu ở mục 4.1.4 (ứng dụng Natural Evolution Strategies): với Gaussian (μ, σ^2), FIM lý thuyết (kỳ vọng, "expected FIM") là ma trận đường chéo $\text{diag}(1/\sigma^2, 1/(2\sigma^4))$, nhưng FIM **thực nghiệm** (trung bình mẫu của tích ngoài score vector, Eq. 189 của paper) "các phần tử ngoài đường chéo gần 0 nhưng thường không chính xác bằng 0" (nguyên văn gần đúng ý paper). Paper gợi ý một cách thực hành: chỉ ước lượng FIM cho tới khi các phần tử ngoài đường chéo nhỏ hơn một ngưỡng ϵ cho trước. Đây là một điểm quan trọng, liên hệ trực tiếp tới phê bình "empirical Fisher approximation" ở bài math 06.

So sánh hình học Fisher-Rao và hình học optimal transport (nêu ở phần kết luận, mục 5.1 của paper): paper đưa ra một quan sát thú vị (trích dẫn công trình khác, không tự chứng minh trong bài): với cùng một họ location-scale, hình học Fisher-Rao cho độ cong **âm** (hyperbolic), còn hình học Wasserstein/optimal-transport cho độ cong **dương**. Đây là một minh chứng rõ ràng rằng "chọn cách đo khoảng cách nào giữa các phân phối" (Fisher-Rao hay Wasserstein) dẫn tới các cấu trúc hình học rất khác nhau, dù cùng áp dụng cho một họ phân phối.

6. Giới hạn

Các giới hạn **paper tự nêu hoặc có thể nhận thấy rõ từ cách trình bày**:

- Paper tự tuyên bố ngay từ tóm tắt: "proofs are omitted for brevity". Hầu hết các định lý quan trọng (Theorem 3, Theorem 8, các công thức Pythagore) được trích dẫn từ các chuyên khảo khác của Amari và các đồng nghiệp, không được chứng minh lại trong chính bài. Người đọc muốn kiểm chứng chặt chẽ phải tra cứu tài liệu gốc.
- Fisher-Rao distance (khoảng cách Riemannian thật) "thường không có dạng đóng" (paper nêu rõ, mục 2.5) và cần phương pháp số như geodesic shooting để tính; đây là một hạn chế thực hành đáng kể nếu muốn dùng chính Fisher-Rao distance (thay vì Bregman/KL divergence) trong ứng dụng.
- Entropy của một hỗn hợp thống kê (ví dụ Gaussian mixture) nói chung **không có dạng đóng** (paper nêu rõ ở mục 4.4, do có tổng log bên trong tích phân), nên việc dựng đa tạp dually flat cho họ hỗn hợp trong thực hành phải dựa vào xấp xỉ Monte Carlo, không phải công thức chính xác.
- Không phải mọi α-geometry đều tự nhiên là dually flat: paper nêu ví dụ họ phân phối Cauchy là một trường hợp α-geometry không dually flat theo cấu trúc tự nhiên, cần một hàm sinh đặc biệt ("Bregman-Tsallis generator") để ép về dạng dually flat.
- Bài không cung cấp mã nguồn, không có thí nghiệm có thể tái lập theo nghĩa thông thường của học máy (không huấn luyện mô hình, không báo cáo độ chính xác/tốc độ trên dữ liệu thật).
- Mật độ ký hiệu và khái niệm hình học vi phân (connection, curvature, tensor, biconjugation Legendre-Fenchel) rất cao đối với người mới; paper giả định người đọc đã quen với đại số tuyến tính và có thể theo dõi các định nghĩa hình thức, dù tự gọi là "elementary introduction".

7. Liên hệ với các bài khác

- **math 06 (New Insights and Perspectives on the Natural Gradient Method, Martens):** liên hệ trực tiếp và sâu nhất. Mục 4.1 của bài này trình bày lại chính natural gradient bằng ngôn ngữ hình học vi phân, và Property 7 (mục 3.7 ở trên) chứng minh một cách rất gọn tính bất biến theo tham số hóa mà math 06 lập luận qua đẳng thức Fisher = Generalized Gauss-Newton. Ngoài ra, cả hai bài đều nêu đúng cùng một vấn đề thực hành: ma trận Fisher information **thực nghiệm/xấp xỉ** (empirical Fisher) có thể không giữ đúng cấu trúc (ví dụ tính đường chéo) của Fisher information lý thuyết, dù dùng hai cách

diễn đạt khác nhau (math 07 minh họa cụ thể bằng ví dụ Gaussian ở §4.1.4; math 06 phê bình tổng quát hơn).

- **math 08 (Variational Inference: A Review for Statisticians, bài sau):** cùng dùng công cụ KL divergence nhưng cho mục đích hoàn toàn khác, suy luận Bayes xấp xỉ bằng tối ưu hóa (tối đa ELBO), không phải xây cấu trúc hình học của không gian tham số. Công thức KL giữa hai Gaussian ở Phụ lục B của bài này (Eq. 235) là chính công thức sẽ cần dùng lại khi tính ELBO với họ biến phân Gaussian ở math 08/09.
- **math 12 (Computational Optimal Transport):** paper tự nêu ở phần kết luận (mục 5.1) một sự đối lập cụ thể giữa hình học Fisher-Rao và hình học Wasserstein/optimal-transport trên cùng một họ location-scale (độ cong âm so với dương). Math 12 sẽ đi sâu vào chính loại hình học "khoảng cách giữa phân phối" thứ hai này.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet kết hợp full attention, khoảng 3600 token/trang, GPU không quá 8GB, batch size 1). Nhận xét thẳng: bài này **ít liên quan trực tiếp** đến các quyết định kỹ thuật cụ thể của dự án (nó không đưa ra một kỹ thuật nén, một kiến trúc, hay một thuật toán huấn luyện cụ thể nào), nhưng có hai liên hệ gián tiếp đáng ghi nhận (đều là suy luận của người ghi chú):

- **KL divergence trong distillation/RLHF là một trường hợp của "divergence" trong khung hình học thông tin:** các hàm loss phổ biến khi distill mô hình nhỏ từ mô hình lớn (ví dụ match phân phối output của student với teacher qua forward-KL hoặc reverse-KL) chính xác là các divergence được paper phân tích ở mục 3.4 (và công thức cụ thể ở mục 3.13/4.3 của paper, không trình bày chi tiết ở đây). Hiểu rằng forward-KL và reverse-KL là hai divergence đối ngẫu (reference-dual) của nhau trong khung này có thể giúp giải thích tại sao chọn forward-KL hay reverse-KL cho loss distillation lại cho hành vi khác nhau (ví dụ reverse-KL có xu hướng "mode-seeking", forward-KL có xu hướng "mode-covering"), dù paper này không trực tiếp thảo luận distillation.
- **Natural gradient = gradient thường trên tọa độ đối ngẫu (Property 7) là một ý tưởng đẹp nhưng khó áp dụng trực tiếp ở quy mô dự án:** để tận dụng được kết quả này, cần biết hàm sinh lồi F cho tham số hóa hiện tại của mô hình, điều này không thực tế với một mạng neural lớn như LightOnOCR-3-0.8B (không gian tham số của nó không có cấu trúc dually flat tường minh). Vì vậy ý tưởng này **khó áp dụng trực tiếp**, nhất quán với giới hạn bộ nhớ đã nêu ở math 06 (K-FAC/natural gradient đầy đủ quá tốn kém cho GPU 8GB, batch 1).

Tổng thể, nên coi bài này là nền tảng lý thuyết giúp hiểu *vì sao* các lựa chọn divergence (KL, reverse-KL) trong các bài thực hành khác lại có ý nghĩa hình học riêng, chứ không phải nguồn cung cấp kỹ thuật áp dụng trực tiếp cho dự án.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây chạy được trên 1 GPU 8GB (PyTorch/NumPy):

1. **Tái hiện ý tưởng Hình 8 của paper cho Bernoulli:** với θ cố định (ví dụ 0.3), lấy mẫu n quan sát Bernoulli, tính MLE $\hat{\theta}$ (tỉ lệ mẫu), lặp lại 200 lần, đo phương sai thực nghiệm của $\hat{\theta}$ qua 200 lần, so sánh với cận Cramér-Rao $\theta(1-\theta)/n$ tính từ công thức $I(\theta) = 1/(\theta(1-\theta))$ ở mục 3.8 (tự tính). Kỳ vọng: phương sai thực nghiệm xấp xỉ cận lý thuyết khi n đủ lớn.
2. **Fisher information thực nghiệm vs lý thuyết cho một lớp softmax nhỏ:** lấy một mô hình phân loại nhỏ (vài lớp), tính FIM thực nghiệm (trung bình ngoài tích của gradient log-likelihood trên một batch) theo đúng công thức Eq. 189 của paper, so sánh với cấu trúc đường chéo kỳ vọng (nếu mô hình tương đương một họ hàm mũ đơn giản như softmax). Đo mức độ lệch khỏi đường chéo khi tăng kích thước batch dùng để ước lượng.

3. **Kiểm tra tính bất biến của natural gradient (Property 5/7):** cài một bài toán tối ưu nhỏ (ví dụ logistic regression) với 2 cách tham số hóa khác nhau cho một tham số dương (ví dụ θ và $\log\theta$), chạy gradient descent thường trên cả hai và quan sát đường đi khác nhau; chạy natural gradient descent (dùng Fisher xấp xỉ diagonal) trên cả hai tham số hóa và kiểm tra đường đi có "giống nhau hơn" (ít nhất về quỹ đạo của phân phối dự đoán) hay không.
4. **Tính Bregman/KL divergence giữa hai Gaussian đa biến** dùng công thức đóng ở Phụ lục B của paper (Eq. 235), áp dụng cho embedding/activation của hai checkpoint mô hình nhỏ khác nhau (giả định gần Gaussian), xem khoảng cách này biến đổi ra sao khi nén mô hình nhiều hay ít.
5. **Mô phỏng Chernoff information đơn giản (mục 4.3 của paper):** với hai Gaussian 1 chiều cụ thể, tính Bhattacharyya distance B_α theo công thức paper cho một lưới giá trị $\alpha \in (0,1)$, tìm α^* tối thiểu hóa bằng grid search hoặc tối ưu số, minh họa khái niệm "best error exponent" của bài toán kiểm định giả thuyết.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Metric tensor g	$g_p(u,v) = \langle u, v\rangle_p$, đo độ dài/góc vector tiếp xúc
Affine connection ∇	Xác định đạo hàm hiệp biến $\nabla_X Y$, geodesic, parallel transport
Connection liên hợp ∇^*	$X\langle Y,Z\rangle = \langle \nabla_X Y, Z\rangle + \langle Y, \nabla^*_X Z\rangle$
Tensor Amari-Chentsov C	$C_{ijk} = \Gamma^k_{ij} - \Gamma^{*k}_{ij}$, hoàn toàn đối xứng
α-connection	$\Gamma^\alpha = \Gamma^0 - \frac{\alpha}{2}C$; $\alpha = 0$: Levi-Civita; $\alpha = \pm 1$: cặp (∇, ∇^*)
Định lý nền tảng IG	∇ độ cong hằng $\kappa \Leftrightarrow \nabla^*$ độ cong hằng κ; $\kappa = 0$: dually flat
Bregman divergence	$B_F(\theta:\theta') = F(\theta) - F(\theta') - (\theta-\theta')^\top \nabla F(\theta')$, F lồi chặt
Tọa độ đối ngẫu	$\eta = \nabla F(\theta)$, $\theta = \nabla F^*(\eta)$ (Legendre-Fenchel)
Pythagore đối ngẫu	$\gamma^*(P,Q) \perp \gamma(Q,R) \Rightarrow D(P:R) = D(P:Q) + D(Q:R)$
Fisher information matrix	$I(\theta) = \mathbb{E}_\theta[\partial_i l\, \partial_j l]$; duy nhất bất biến (Theorem 8)
Natural gradient	${}^{NG}\nabla L_\theta(\theta) = g_\theta^{-1}(\theta)\nabla_\theta L_\theta(\theta)$; bằng $\nabla_\eta L_\eta(\eta)$ trên đa tạp dually flat
FIM Gaussian 1D, tham số (μ,σ)	$\mathrm{diag}(1/\sigma^2,\ 2/\sigma^2)$
Ví dụ tự tính: FIM Bernoulli	$I(\theta) = 1/(\theta(1-\theta))$; $I(0.5) = 4$, $I(0.2) = 6.25$, $I(0.05) \approx 21.05$

11. Kết luận ngắn

Bài khảo sát lại toàn bộ khung lý thuyết của information geometry: coi một họ phân phối xác suất như một đa tạp, đặt lên nó một metric (chính tắc là Fisher information, metric bất biến duy nhất theo Theorem 8) và một cặp connection đối ngẫu (chính tắc là connection mũ/hỗn hợp). Kết quả trung tâm là định lý nền tảng (độ cong của connection và đối ngẫu của nó luôn bằng nhau) và trường hợp đặc biệt dually flat, nơi tồn tại hai hệ tọa độ đối ngẫu liên hệ qua Legendre-Fenchel và một định lý Pythagore tổng quát. Ứng dụng quan trọng nhất đối với lộ trình học của dự án là mục 4.1: natural gradient trên θ chính là gradient thường trên tọa độ đối ngẫu η của một đa tạp dually flat, một cách diễn đạt hình học rất gọn cho cùng hiện tượng bất biến tham số hóa đã gặp ở math 06. Giới hạn lớn nhất của bài, như paper tự nhận, là tính chất "introduction" của nó: hầu hết chứng minh bị bỏ qua, và nhiều kết quả đẹp (dually flat, Pythagore đối ngẫu) chỉ áp dụng trơn tru cho các họ phân phối có cấu trúc đặc biệt (hàm mũ, hỗn hợp), không phải cho không gian tham số tổng quát của một mạng neural lớn.

Nguồn gốc: https://arxiv.org/abs/1808.08271

Bài 08

Variational Inference: A Review for Statisticians (2016)

Tác giả: David M. Blei (Đại học Columbia, Khoa Khoa học Máy tính và Khoa Thống kê), Alp Kucukelbir (Đại học Columbia, Khoa Khoa học Máy tính), Jon D. McAuliffe (Đại học California, Berkeley, Khoa Thống kê)
Venue / arXiv: arXiv:1601.00670 (bản v9, ngày 11/05/2018); văn bản arXiv không tự nêu tên hội nghị hay tạp chí công bố nào; theo kiến thức chung (ngoài paper, không xác nhận được trong chính văn bản arXiv) bài này được xuất bản trên Journal of the American Statistical Association (JASA) năm 2017
Nguồn: https://arxiv.org/abs/1601.00670
Chủ đề: Variational inference (suy luận biến phân/suy luận xấp xỉ qua tối ưu hóa), ELBO, mean-field, CAVI, suy luận biến phân ngẫu nhiên (stochastic variational inference, SVI) dùng natural gradient
Vị trí trong lộ trình: Bài số 8 trong 9 bài toán nền (math 01-09). Đọc sau math 06 (natural gradient method) và math 07 (information geometry) vì mục 4.3 của bài này (SVI) dùng trực tiếp khái niệm natural gradient đã học ở hai bài đó. Đứng trước math 09 (variational autoencoder), nơi khung ELBO/variational family được áp dụng cho mô hình sinh sâu (deep generative model) với kỹ thuật reparameterization mà chính bài này cũng nhắc tới ở mục 5.3 như một hướng mở rộng ngoài phạm vi chính của bài.

> Ghi chú này dựa trên toàn văn bài báo (đọc từ PDF chuyển sang text, khoảng 41 trang, 2334 dòng), gồm Mục 1-5, Phụ lục A (hồi quy tuyến tính Bayes với ARD), Phụ lục B (mô hình GMM ảnh và mã Stan), Phụ lục C (LDA), đọc đầy đủ; phần danh mục tài liệu tham khảo ở cuối (khoảng 5 trang, hơn 150 mục) chỉ đọc lướt để xác nhận đó là danh mục trích dẫn thuần túy. Đây không phải bản dịch mà là ghi chú diễn giải lại cho người mới học suy luận Bayes xấp xỉ. Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Bài này là một bài tổng quan (review) viết cho đối tượng là các nhà thống kê đã quen với suy luận Bayes bằng MCMC (Markov chain Monte Carlo) nhưng chưa quen với một họ phương pháp khác: variational inference (VI, suy luận biến phân). Mục tiêu paper tự nêu ở phần tóm tắt là trình bày lại VI một cách có hệ thống, kèm ví dụ cụ thể (mô hình hỗn hợp Gaussian, Gaussian mixture model viết tắt GMM), để người đọc có thể áp dụng ngay cho bài toán của mình.

Bối cảnh lịch sử theo paper (mục 1): ý tưởng ”mean field” (trường trung bình) có gốc từ vật lý thống kê, được đưa vào học máy/thống kê qua Peterson và Anderson (1987, ”A mean field theory learning algorithm for neural networks”) và Hinton và Van Camp (1993, ”Keeping the neural networks simple by minimizing the description length of the weights”). VI sau đó được hệ thống hóa cho mô hình đồ thị (graphical model) bởi Jordan, Ghahramani, Jaakkola, Saul (1999, ”Introduction to variational methods for graphical models”). Đối trọng truyền thống của VI là các phương pháp MCMC (Metropolis và cộng sự 1953; Hastings 1970; Gibbs sampling của Geman và Geman 1984): MCMC xây một xích Markov có phân phối dừng chính là posterior cần tính, còn VI biến bài toán suy luận thành một bài toán tối ưu hóa.

Paper không tự nêu tên hội nghị/tạp chí công bố trong chính văn bản arXiv (chỉ có từ khóa ”Algorithms; Statistical Computing; Computationally Intensive Methods” ở đầu bài). Việc bài này sau đó xuất bản ở JASA 2017 là thông tin ngoài paper, nêu ở đây chỉ để định vị, không dùng làm cơ sở cho nội dung kỹ thuật.

Ghi chú này đọc toàn bộ nội dung chính (mục 1 đến 5) và cả ba phụ lục (A, B, C), vì ba phụ lục này chứa chi tiết suy luận toán học cho hai thí nghiệm thực sự quan trọng của bài (so sánh với Stan ở Phụ lục B, mô hình LDA ở Phụ lục C).

2. Ý tưởng chính

Bài toán Bayes tổng quát: có dữ liệu quan sát $x = x_{1:n}$ và biến ẩn/tham số $z = z_{1:m}$, mô hình xác định mật độ chung $p(z, x)$. Ta muốn tính posterior

$$p(z \mid x) = \frac{p(z, x)}{p(x)}, \qquad p(x) = \int p(z, x)\, dz.$$

Vấn đề: với nhiều mô hình, $p(x)$ (evidence, "bằng chứng") không tính được ở dạng đóng. Ví dụ paper dùng xuyên suốt bài là mô hình hỗn hợp Gaussian Bayes (Bayesian GMM, mục 2.1): với K cụm,

$$\mu_k \sim \mathcal{N}(0, \sigma^2), \quad k = 1, \ldots, K; \qquad c_i \sim \text{Categorical}\left(\tfrac{1}{K}, \ldots, \tfrac{1}{K}\right); \qquad x_i \mid c_i, \mu \sim \mathcal{N}(c_i^\top \mu, 1).$$

Ở đây c_i là vector chỉ số (indicator) cho biết điểm i thuộc cụm nào. Để tính $p(x)$ phải lấy tổng trên mọi cách gán cụm c có thể, tức K^n số hạng (với n điểm dữ liệu): không thể tính trực tiếp khi n lớn.

Ý tưởng của VI: thay vì tính posterior đúng, đặt ra một họ phân phối xấp xỉ "dễ" $\mathcal{Q} = \{q(z; \nu)\}$ (gọi là variational family), rồi tìm thành viên q^* trong họ đó gần posterior thật nhất, theo nghĩa cực tiểu hóa Kullback-Leibler divergence

$$q^* = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(z) \,\|\, p(z \mid x)\big).$$

Vấn đề con: biểu thức $\mathrm{KL}(q \| p(z \mid x))$ tự nó chứa $\log p(x)$ (xem mục 3), nên không tính trực tiếp được. Paper giải quyết bằng cách tối ưu một đại lượng khác, tương đương về mặt tối ưu nhưng tính được: ELBO (evidence lower bound). Đây chính là chuyển bài toán suy luận thành bài toán tối ưu hóa, điểm khác biệt căn bản so với MCMC (một họ thuật toán lấy mẫu).

Để ELBO tính được, paper giả định họ biến phân có dạng mean-field: z chia thành m nhóm/khối $z_1, \ldots, z_m$, và

$$q(z) = \prod_{j=1}^{m} q_j(z_j),$$

tức giả định các nhóm biến ẩn độc lập với nhau dưới q (nhưng không giả định dạng hàm cụ thể nào cho mỗi q_j; dạng của q_j sẽ "lộ" ra từ chính bài toán tối ưu).

3. Phương pháp chi tiết

3.1 ELBO và liên hệ với KL

Paper định nghĩa (mục 2.2):

$$\mathrm{ELBO}(q) = \mathbb{E}_q[\log p(z, x)] - \mathbb{E}_q[\log q(z)].$$

Khai triển $\mathrm{KL}(q(z) \| p(z \mid x))$ và dùng $p(z \mid x) = p(z, x)/p(x)$:

$$\mathrm{KL}(q(z) \| p(z \mid x)) = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(z, x)] + \log p(x) = -\mathrm{ELBO}(q) + \log p(x).$$

Sắp xếp lại:

$$\log p(x) = \text{ELBO}(q) + \text{KL}\big(q(z) \,\|\, p(z \mid x)\big).$$

Vì $\text{KL} \geq 0$ luôn luôn, nên $\text{ELBO}(q) \leq \log p(x)$: ELBO là chặn dưới của log-evidence (đúng như tên gọi). Vì $\log p(x)$ không phụ thuộc q, cực đại hóa $\text{ELBO}(q)$ theo q đúng bằng cực tiểu hóa $\text{KL}(q \| p(z \mid x))$ theo q, mà không cần biết $\log p(x)$. Paper cũng chỉ ra liên hệ với thuật toán EM (Expectation-Maximization): ELBO chính là đại lượng được cực đại hóa ở bước E của EM; VI tổng quát hóa ý tưởng này khi chính posterior có điều kiện đầy đủ (complete conditional) cũng không tính được dạng đóng.

3.2 Mean-field và minh họa Hình 1 của paper

Mục 2.3 của paper minh họa (Hình 1, chỉ là hình vẽ định tính, không có số liệu) rằng khi posterior thật là một Gaussian hai chiều có tương quan mạnh, xấp xỉ mean-field (tích hai Gaussian một chiều độc lập) khớp đúng vị trí trung tâm (mean) nhưng cho ra một "khối" xấp xỉ nhỏ hơn nhiều so với hình elip thật, tức đánh giá thấp phương sai biên (marginal variance).

Ví dụ (tự tính): paper không cho con số cụ thể nào ở Hình 1, nên ở đây tính một ví dụ số cụ thể để định lượng hiện tượng đó. Đây là tính toán độc lập của người ghi chú dựa trên một kết quả toán học chuẩn về xấp xỉ mean-field Gaussian theo KL ngược (không phải điều paper tự chứng minh, nhưng nhất quán với Hình 1 và với kết quả lý thuyết ở mục 5.2 dưới đây).

Giả sử posterior thật là $p(z_1, z_2) = \mathcal{N}(0, \Sigma)$ với

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}, \quad \rho = 0.8.$$

Xấp xỉ mean-field $q(z_1, z_2) = q_1(z_1) q_2(z_2)$, tối thiểu $\text{KL}(q \| p)$ (đúng chiều KL mà VI luôn dùng). Với p là Gaussian có ma trận hiệp phương sai nghịch đảo (precision) $\Lambda = \Sigma^{-1}$, kết quả chuẩn là: q_j^* có trung bình đúng bằng trung bình thật, còn phương sai bằng $1/\Lambda_{jj}$ (nghịch đảo của phần tử chéo của ma trận precision), không phải Σ_{jj} (phần tử chéo của hiệp phương sai thật).

Tính Λ:

$$\Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix} = \frac{1}{1-0.64} \begin{pmatrix} 1 & -0.8 \\ -0.8 & 1 \end{pmatrix} = \frac{1}{0.36} \begin{pmatrix} 1 & -0.8 \\ -0.8 & 1 \end{pmatrix}.$$

Vậy $\Lambda_{11} = \Lambda_{22} = 1/0.36 \approx 2.778$. Phương sai của q_j^* là $1/\Lambda_{jj} = 0.36$, trong khi phương sai biên thật $\Sigma_{jj} = 1$. Tỉ lệ $0.36/1 = 0.36$: xấp xỉ mean-field chỉ giữ lại 36% phương sai thật, đánh giá thấp phương sai tới 64% trong ví dụ này. Khi $\rho \to 1$ (tương quan gần tuyệt đối), $1 - \rho^2 \to 0$ nên phương sai mean-field $\to 0$: VI mean-field trở nên "tự tin" quá mức một cách không giới hạn khi độ tương quan tăng. Đây là minh họa số cụ thể cho câu paper nói ở Hình 1 và cho kết quả lý thuyết ở mục 5.2.

3.3 CAVI (Coordinate Ascent Variational Inference)

Với giả định mean-field, paper dẫn ra (mục 2.4, công thức tổng quát) quy tắc cập nhật tối ưu cho từng nhân tử, giữ các nhân tử khác cố định:

$$q_j^*(z_j) \;\propto\; \exp\Big\{\mathbb{E}_{-j}\big[\log p(z_j \mid z_{-j}, x)\big]\Big\} \;\propto\; \exp\Big\{\mathbb{E}_{-j}\big[\log p(z, x)\big]\Big\},$$

trong đó $\mathbb{E}_{-j}$ là kỳ vọng lấy theo tích các nhân tử q_i với $i \neq j$ (giữ cố định ở vòng cập nhật hiện tại). Thuật toán CAVI (Algorithm 1 của paper): khởi tạo các q_j, sau đó lặp qua $j = 1, \ldots, m$, mỗi lần cập nhật q_j theo

công thức trên rồi tính lại ELBO, lặp tới khi ELBO tăng rất ít (hội tụ). Vì mỗi bước chỉ tăng hoặc giữ nguyên ELBO (không giảm), CAVI là một phương pháp coordinate ascent đúng nghĩa, bảo đảm hội tụ tới một điểm tối ưu cục bộ (không bảo đảm toàn cục, vì ELBO như hàm của toàn bộ $(q_1, \ldots, q_m)$ nói chung không lồi).

Paper nêu rõ hai vấn đề thực hành (mục 2.5): (1) nhạy với điểm khởi tạo và tối ưu cục bộ (minh họa thực nghiệm ở mục 4 dưới); (2) cần kỹ thuật log-sum-exp để tránh tràn số (underflow) khi chuẩn hóa các nhân tử dạng categorical (ví dụ xác suất gán cụm trong GMM).

3.4 Ví dụ đầy đủ: GMM

Mục 3 của paper suy ra chi tiết CAVI cho mô hình GMM ở mục 2.1. Nhân tử biến phân cho biến gán cụm c_i là categorical với "trách nhiệm" (responsibility) φ_{ik} tính theo softmax của $\mathbb{E}[\mu_k]\, x_i - \mathbb{E}[\mu_k^2]/2$; nhân tử cho mỗi μ_k là Gaussian với trung bình $m_k = \dfrac{\sum_i \varphi_{ik} x_i}{1/\sigma^2 + \sum_i \varphi_{ik}}$ và phương sai $s_k^2 = \dfrac{1}{1/\sigma^2 + \sum_i \varphi_{ik}}$. Công thức ELBO đầy đủ cho mô hình này là Eq. (21) của paper; Algorithm 2 là CAVI cụ thể hóa cho GMM (lặp cập nhật φ rồi (m, s^2)); Eq. (35) cho mật độ dự đoán của một điểm dữ liệu mới bằng cách trộn K thành phần đã khớp.

3.5 Mở rộng: họ hàm mũ và SVI

Mục 4.1: nếu mỗi complete conditional $p(z_j \mid z_{-j}, x)$ thuộc một họ hàm mũ (exponential family) với tham số tự nhiên $\eta_j(z_{-j}, x)$, thì nhân tử tối ưu q_j^* cũng thuộc đúng họ hàm mũ đó, với tham số tự nhiên bằng kỳ vọng của $\eta_j(z_{-j}, x)$ dưới các q_i khác. Mục 4.2 hình thức hóa lớp mô hình "conditionally conjugate" phù hợp cho dữ liệu lớn: có biến toàn cục β (chung cho toàn bộ dữ liệu, ví dụ tham số mô hình) và biến địa phương z_i (riêng từng điểm dữ liệu, ví dụ biến gán cụm), mỗi z_i chỉ phụ thuộc β và (x_i, z_i).

Mục 4.3 trình bày Stochastic Variational Inference (SVI), đóng góp phương pháp chính của bài để xử lý dữ liệu quy mô lớn. Với cấu trúc conditionally conjugate trên, natural gradient của ELBO theo tham số biến phân toàn cục λ (tham số của $q(\beta)$) có dạng rất gọn (Eq. 51-52 của paper):

$$\nabla_\lambda^{\text{nat}} \text{ELBO} = F(\lambda)^{-1} \nabla_\lambda \text{ELBO},$$

và với $q(\beta)$ thuộc họ hàm mũ, công thức này rút gọn thành một biểu thức dễ tính (không cần nghịch đảo ma trận Fisher một cách tường minh), vì Fisher information của họ hàm mũ theo tham số tự nhiên chính là Hessian của hàm log-partition, triệt tiêu gọn với các số hạng chain-rule trong gradient thường. SVI tận dụng điều này: ước lượng natural gradient bằng cách lấy ngẫu nhiên một (hoặc một minibatch) điểm dữ liệu, nhân với hệ số tỉ lệ $N/|\text{minibatch}|$ để không chệch (unbiased), rồi cập nhật

$$\lambda_{t+1} = \lambda_t + \rho_t \, \widehat{\nabla_\lambda^{\text{nat}} \text{ELBO}}_t,$$

với dãy bước học ρ_t thỏa điều kiện Robbins-Monro (Robbins và Monro, 1951): $\sum_t \rho_t = \infty$ và $\sum_t \rho_t^2 < \infty$ (ví dụ $\rho_t = (t + t_0)^{-\kappa}$ với $\kappa \in (0.5, 1]$), đảm bảo hội tụ hầu chắc chắn tới một điểm tối ưu cục bộ. Đây là Algorithm 3 của paper, và chính là áp dụng cụ thể của natural gradient (math 06) kết hợp với tối ưu ngẫu nhiên (stochastic optimization).

4. Thí nghiệm và kết quả chính

Paper có ba khảo sát thực nghiệm thật sự (không chỉ là ví dụ minh họa trên giấy), ở mục 3.4 và 4.3:

(a) Mô phỏng (mục 3.4): dữ liệu tổng hợp từ hỗn hợp 5 Gaussian hai chiều ($K = 5$). Hình 3 của paper cho thấy đường cong ELBO theo số vòng lặp CAVI có hình dạng "khuỷu tay" (elbow): có những đoạn gần như đi ngang (ELBO tăng rất chậm) rồi đột ngột nhảy lên, tương ứng với lúc thuật toán "thoát" khỏi một cấu hình gán cụm chưa tốt sang cấu hình tốt hơn về chất.

(b) So sánh với Stan/HMC trên dữ liệu ảnh thật (mục 3.4, Phụ lục B): dùng tập dữ liệu imageCLEF, mỗi ảnh được biểu diễn bằng histogram màu RGB 576 chiều, với 10.000 ảnh huấn luyện và 10.000 ảnh kiểm tra. Khớp một GMM đường chéo (diagonal covariance) với $K = 30$ cụm bằng CAVI, so với lấy mẫu bằng Stan (dùng Hamiltonian Monte Carlo/NUTS). Siêu tham số cụ thể ghi ở Phụ lục B: $a_0 = 1/K$, $m_0 = 0$, $b_0 = 1$, $\alpha_0 = 1$, $\beta_0 = 1$; Phụ lục B cũng cho toàn bộ mã Stan dùng trong so sánh (Hình 8 của paper). Kết luận của paper: CAVI nhanh hơn Gibbs sampling và HMC/NUTS "nhiều cấp độ lớn" (orders of magnitude), khớp được mô hình cho 10.000 ảnh trong chưa đầy một phút. Paper tự có một chú thích (footnote) nhấn mạnh đây chỉ là một nghiên cứu trường hợp cụ thể, không phải một khẳng định tổng quát rằng VI luôn nhanh hơn mọi biến thể MCMC.

(c) LDA trên 1,8 triệu bài báo New York Times (mục 4.3, Phụ lục C): áp dụng SVI cho mô hình Latent Dirichlet Allocation (LDA, Blei và cộng sự 2003), dựa trên suy luận CAVI cụ thể cho LDA ở Phụ lục C (cập nhật Eq. 59-61, Algorithm 4, theo Hoffman và cộng sự 2013): nhân tử $q(\beta_k; \lambda_k)$ cho mỗi chủ đề (Dirichlet V chiều), $q(\theta_d; \gamma_d)$ cho tỉ lệ chủ đề mỗi văn bản (Dirichlet K chiều), $q(z_{dn}; \phi_{dn})$ cho gán chủ đề mỗi từ (multinomial). Hình 7 của paper hiển thị 15 chủ đề trích ra được (dưới dạng các từ có xác suất cao trong mỗi chủ đề); vì đây là một hình ảnh minh họa trong bản PDF chứ không phải bảng số liệu dạng văn bản, ghi chú này không trích lại danh sách từ cụ thể của 15 chủ đề (không có trong văn bản text trích xuất được một cách đáng tin cậy). Paper nhấn mạnh (diễn giải lại, không trích nguyên văn chính xác từng chữ do giới hạn khi chuyển PDF sang text): nếu không có SVI, một phân tích ở quy mô 1,8 triệu văn bản như vậy sẽ không thể thực hiện được bằng CAVI theo lô (batch) thông thường.

Ba thí nghiệm này là bằng chứng thực nghiệm thật, không phải chỉ lý thuyết, cho hai tuyên bố chính của paper: CAVI nhanh hơn MCMC trên bài toán cỡ vừa (thí nghiệm b), và SVI mở ra khả năng xử lý dữ liệu cỡ rất lớn mà CAVI theo lô không làm được (thí nghiệm c).

5. Phân tích / ablation

Paper không chạy một bảng ablation có kiểm soát theo nghĩa thường gặp trong các bài học máy hiện đại (không có bảng so sánh hệ thống nhiều lựa chọn siêu tham số với số liệu đầy đủ). Thay vào đó, phần "phân tích" gần nhất gồm:

- **Độ nhạy với điểm khởi tạo (mục 2.5, Hình 2):** chạy CAVI cho GMM với 10 điểm khởi tạo ngẫu nhiên khác nhau trên cùng dữ liệu, cho ra các kết quả hội tụ khác nhau (tối ưu cục bộ khác nhau). Đây là minh họa thực nghiệm thật, không phải chỉ cảnh báo lý thuyết.
- **Khảo sát lý thuyết về sai số của VI (mục 5.2):** paper tổng hợp (không tự chứng minh lại) một kết quả quan trọng của Wang và Titterington (2005): với một lớp mô hình nhất định, ma trận hiệp phương sai posterior theo mean-field VI khác với hiệp phương sai "đúng" (theo ước lượng hợp lý cực đại/nghịch đảo Fisher information) bởi một ma trận nửa xác định dương bị trừ đi, nghĩa là VI **luôn đánh giá thấp** phương sai posterior theo hướng đó. Đây là cơ sở lý thuyết (không phải thực nghiệm của chính paper này) cho hiện tượng đã minh họa định lượng ở mục 3.2 của ghi chú này (ví dụ tự tính $\rho = 0.8$).
- Paper cũng tổng hợp (không tự chứng minh) các kết quả về tính nhất quán (consistency) của ước lượng VI khi $n \to \infty$ từ nhiều tác giả khác, nhưng tự nhận lý thuyết về độ chính xác thống kê đầy đủ của VI (ví dụ phân phối tiệm cận, khoảng tin cậy) còn là vấn đề mở (xem mục 6).

6. Giới hạn

Giới hạn paper tự nêu rõ:

- **Đánh giá thấp phương sai một cách có hệ thống:** do giả định mean-field (độc lập giữa các nhóm biến ẩn) và do chọn chiều KL $\mathrm{KL}(q\|p)$ (không phải $\mathrm{KL}(p\|q)$), xấp xỉ mean-field về cấu trúc không thể tái tạo tương quan giữa các nhóm biến khác nhau, và có xu hướng cho posterior "tự tin quá mức" (xem mục 3.2 và 5.2).

- **Chỉ tìm tối ưu cục bộ:** CAVI/SVI đều là các thuật toán tối ưu cục bộ trên một hàm mục tiêu (ELBO) không lồi nói chung; nhạy với điểm khởi tạo (Hình 2).
- **Yêu cầu cấu trúc conditional conjugacy để có công thức đóng:** phần suy luận chi tiết (CAVI, SVI với công thức tường minh) của paper chỉ áp dụng trực tiếp khi mỗi complete conditional thuộc họ hàm mũ nhận biết được. Với mô hình không liên hợp (non-conjugate), ví dụ hồi quy logistic Bayes mà paper nêu ở mục 5.3, cần kỹ thuật khác (Monte Carlo gradient, ”black box” VI của Ranganath, Gerrish, Blei 2014) mà paper chỉ khảo sát sơ lược, không suy diễn chi tiết như với CAVI/SVI.
- **Mục 5.4 (Open problems) paper tự liệt kê là vấn đề mở, không phải đã giải quyết:** lựa chọn phân kỳ (KL hay một họ phân kỳ khác, ví dụ α-divergence) ảnh hưởng tới hành vi của nghiệm (mode-seeking so với mode-covering) nhưng chưa có hướng dẫn tổng quát; expectation propagation được paper mô tả như ”dùng KL divergence theo chiều ngược lại”; giao diện giữa VI và MCMC (kết hợp hai họ phương pháp) còn là hướng nghiên cứu đang mở; lý thuyết thống kê đầy đủ cho ước lượng VI (tính nhất quán, phân phối tiệm cận, định lượng độ không chắc chắn khi suy luận sai) còn chưa hoàn chỉnh.

Quan sát riêng của người ghi chú (không phải khẳng định của paper): bài không đưa ra một bảng số liệu thời gian chạy (giây/phút) cụ thể, có thể tái lập từng con số, cho so sánh imageCLEF ở mục 4(b) trên đây; các câu như ”nhanh hơn nhiều cấp độ lớn” hay ”dưới một phút” là phát biểu định tính trong văn bản chính, không phải một bảng benchmark với số liệu từng cấu hình. Người muốn tái lập chính xác cần tự chạy lại dựa trên mã Stan ở Phụ lục B.

7. Liên hệ với các bài khác

- **math 06 (New Insights and Perspectives on the Natural Gradient Method):** SVI ở mục 4.3 của bài này là một ứng dụng trực tiếp của natural gradient đã học ở math 06; công thức $\nabla_\lambda^{\text{nat}}\text{ELBO} = F(\lambda)^{-1}\nabla_\lambda\text{ELBO}$ chính là đối tượng trung tâm của math 06, áp dụng cho một bài toán cụ thể (tối ưu ELBO).
- **math 07 (An Elementary Introduction to Information Geometry):** lý do natural gradient của SVI rút gọn gọn gàng khi $q(\beta)$ thuộc họ hàm mũ chính là cấu trúc tọa độ đối ngẫu (tham số tự nhiên và tham số kỳ vọng) đã trình bày ở math 07 (Property 7 ở đó: natural gradient theo tọa độ tự nhiên bằng gradient thường theo tọa độ đối ngẫu). Công thức Eq. 51 của bài này là một trường hợp cụ thể của sự kiện tổng quát đó.
- **math 09 (An Introduction to Variational Autoencoders, bài tiếp theo):** chính paper này, ở mục 5.3 (”Beyond conditional conjugacy”), trích dẫn trực tiếp Kingma và Welling (2014, ”Auto-encoding variational Bayes”) và Rezende, Mohamed, Wierstra (2014, ”Stochastic backpropagation and approximate inference in deep generative models”) như ví dụ về suy luận biến phân cho mô hình không liên hợp, dùng gradient kiểu Monte Carlo (tiền đề của reparameterization trick ở math 09). Đây là liên hệ được chính văn bản paper xác nhận, không phải suy diễn của người ghi chú.
- **math 10 (A Conceptual Introduction to Hamiltonian Monte Carlo):** mục 1 và mục 3.4(b) của bài này so sánh trực tiếp VI/CAVI với HMC (được triển khai qua Stan) như đối trọng chính cần cân nhắc khi chọn phương pháp suy luận Bayes xấp xỉ; math 10 trình bày chi tiết phía HMC của sự so sánh đó.

8. Ứng dụng cho dự án (suy luận)

Dự án tham chiếu: LightOnOCR-3-0.8B, một mô hình VLM OCR (vision-language model cho nhận dạng chữ), kiến trúc hybrid GatedDeltaNet kết hợp full attention, khoảng 3.600 token/trang (gồm khoảng 1.500 token văn bản và 1.600-2.100 token hình ảnh), chạy trên GPU tối đa 8GB, batch size 1. Phần sau đây là **suy luận** của người ghi chú, không phải nội dung trong paper.

Đánh giá tổng quát: **ít liên quan trực tiếp**. LightOnOCR được huấn luyện bằng maximum likelihood (cross-entropy) theo kiểu sequence-to-sequence chuẩn, không phải bằng suy luận posterior Bayes qua CAVI/SVI; mô hình cũng không có cấu trúc conditionally-conjugate (mạng attention sâu không thuộc họ hàm mũ theo

nghĩa bài này dùng), nên không thể áp trực tiếp công thức CAVI hay ”mẹo” natural gradient rẻ của SVI (mục 3.5 trên đây) cho việc huấn luyện chính mô hình, cùng kết luận với ghi chú math 06 về việc natural gradient tổng quát khó khả thi cho mạng lớn.

Tuy vậy có hai điểm liên quan gián tiếp đáng ghi:

1. Nếu cần xây một lớp xử lý hậu kỳ nhỏ (ví dụ một mô hình hỗn hợp Bayes nhỏ để gộp/làm sạch các giá trị trường dữ liệu trích xuất được từ OCR, hoặc mô hình hóa độ không chắc chắn của việc phân loại loại trang/layout), thì CAVI/GMM Bayes trình bày ở mục 3-4 của bài này hoàn toàn có thể dùng được, vì các mô hình phụ trợ này rất nhỏ, chạy trên CPU trong vài milli-giây, không vướng ràng buộc 8GB GPU của mô hình chính.
2. Cảnh báo ở mục 5.2/6 (mean-field VI đánh giá thấp phương sai) là bài học thực tế đáng nhớ nếu có ai định dùng VI để lượng hóa độ không chắc chắn cho một tham số phụ trợ nào đó của hệ thống (ví dụ ngưỡng tin cậy/temperature calibration cho đầu ra OCR): con số ”phương sai posterior” thu được từ mean-field VI, theo chính kết quả paper trích dẫn, có thể nhỏ hơn thực tế một cách hệ thống, nên không nên diễn giải nó như một khoảng tin cậy đáng tin cậy nếu không kiểm tra thêm.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây chạy được trên một GPU 8GB (một số thực ra không cần GPU, vì các mô hình liên quan rất nhỏ, nhưng đặt trong khuôn khổ phần cứng của dự án để nhất quán):

1. **CAVI so với EM trên GMM tổng hợp nhỏ.** Sinh dữ liệu tổng hợp 2 chiều từ 3-5 Gaussian (vài nghìn điểm), cài CAVI theo Algorithm 2 của paper (numpy/PyTorch, CPU hoặc GPU đều được) và so với `sklearn.mixture.GaussianMixture` (EM). Đo: log-likelihood cuối cùng, thời gian chạy, số vòng lặp tới hội tụ. Kỳ vọng: chất lượng khớp tương đương, cả hai nhanh hơn hẳn một phương pháp MCMC đầy đủ (ví dụ chạy thêm Gibbs sampler thủ công để so sánh thời gian). Diễn giải: xác nhận trực tiếp hiện tượng paper mô tả ở mục 4(b).
2. **Định lượng đánh giá thấp phương sai của mean-field trên một posterior Gaussian tương quan.** Lặp lại ví dụ tự tính ở mục 3.2 (toy $\rho = 0.8$) bằng mô phỏng số: chạy coordinate-ascent thật (không dùng công thức đóng) cho q_1, q_2 Gaussian xấp xỉ $p = \mathcal{N}(0, \Sigma)$, kiểm tra phương sai hội tụ về $1/\Lambda_{jj}$ như tính tay. Thử nhiều giá trị $\rho \in \{0, 0.3, 0.6, 0.8, 0.95\}$ và vẽ đồ thị tỉ lệ phương sai mean-field/phương sai thật theo ρ. Kỳ vọng: tỉ lệ giảm về 0 khi $\rho \to 1$. Diễn giải: minh họa trực quan, có số liệu, cho giới hạn ở mục 6.
3. **SVI so với CAVI theo lô trên LDA cỡ nhỏ.** Dùng một corpus nhỏ (vài nghìn văn bản, ví dụ một phần của 20 Newsgroups), cài CAVI theo lô cho LDA (Phụ lục C, Algorithm 4) và SVI (Algorithm 3) với vài kích thước minibatch và vài giá trị $\kappa \in \{0.5, 0.7, 1.0\}$ cho bước học $\rho_t = (t + t_0)^{-\kappa}$. Đo: perplexity trên tập giữ lại (held-out) theo thời gian chạy thực (wall-clock), không theo số vòng lặp. Kỳ vọng: SVI đạt perplexity tương đương CAVI nhanh hơn khi corpus đủ lớn để amortize chi phí overhead, nhưng có thể chậm hơn CAVI trên corpus rất nhỏ (do nhiễu từ việc lấy mẫu minibatch). Diễn giải: tái hiện ở quy mô nhỏ bài học từ thí nghiệm LDA 1,8 triệu bài báo của paper.
4. **GMM Bayes trên embedding token hình ảnh của chính LightOnOCR.** Trích embedding của các visual token từ bộ mã hóa hình ảnh (vision encoder) của LightOnOCR trên một vài trang tài liệu scan, khớp một GMM Bayes nhỏ (K khoảng 5-10) bằng CAVI lên các embedding đó, so với baseline k-means trên cùng embedding. Đo: liệu các cụm CAVI có tách biệt theo vùng layout hợp lý (vùng chữ, vùng nền/margin, vùng bảng) khi quan sát bằng mắt (overlay nhãn cụm lên ảnh gốc) hay không; so sánh ”trách nhiệm” (responsibility) mềm của CAVI tại các patch biên giới chữ/nền với quyết định cứng của k-means. Kỳ vọng: CAVI cho thấy các patch biên giới có trách nhiệm chia đều giữa hai cụm (phản ánh đúng sự mơ hồ), còn k-means buộc phải chọn một nhãn cứng. Diễn giải: đây là thí nghiệm mang tính suy luận/khám phá (không có trong paper), kiểm tra xem mean-field VI có mang lại tín hiệu hữu ích nào ngoài một baseline phân cụm đơn giản cho dữ liệu của chính dự án hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Posterior mục tiêu	$p(z \mid x) = p(z,x)/p(x)$, với $p(x) = \int p(z,x)\,dz$ thường không tính được
ELBO	$\text{ELBO}(q) = \mathbb{E}_q[\log p(z,x)] - \mathbb{E}_q[\log q(z)]$
Liên hệ ELBO và KL	$\log p(x) = \text{ELBO}(q) + \text{KL}(q(z) \Vert p(z \mid x))$, nên cực đại ELBO $\equiv$ cực tiểu KL
Họ mean-field	$q(z) = \prod_{j=1}^{m} q_j(z_j)$ (các nhóm biến ẩn độc lập dưới q)
Cập nhật CAVI	$q_j^*(z_j) \propto \exp\{\mathbb{E}_{-j}[\log p(z,x)]\}$, lặp qua j tới hội tụ
Nhân tử họ hàm mũ	Nếu complete conditional thuộc họ hàm mũ, q_j^* cũng thuộc họ đó, tham số = kỳ vọng tham số tự nhiên của conditional
Natural gradient của SVI	$\nabla_\lambda^{\text{nat}}\text{ELBO} = F(\lambda)^{-1}\nabla_\lambda \text{ELBO}$, rút gọn khi $q(\beta)$ thuộc họ hàm mũ
Cập nhật SVI	$\lambda_{t+1} = \lambda_t + \rho_t \widehat{\nabla_\lambda^{\text{nat}}\text{ELBO}}_t$, ước lượng không chệch từ một (mini)batch dữ liệu
Điều kiện Robbins-Monro	$\sum_t \rho_t = \infty$, $\sum_t \rho_t^2 < \infty$ (ví dụ $\rho_t = (t+t_0)^{-\kappa}$, $\kappa \in (0.5, 1]$)
Hạn chế đã biết	Mean-field đánh giá thấp phương sai posterior (Wang và Titterington, 2005); chỉ tìm tối ưu cục bộ

11. Kết luận ngắn

Đây là một bài tổng quan có chất lượng sư phạm cao: trình bày lại ELBO, CAVI, và SVI một cách mạch lạc, có ví dụ GMM suy luận đầy đủ từng bước, và quan trọng là có hai thực nghiệm thật của chính nhóm tác giả (so sánh CAVI với Stan/HMC trên dữ liệu ảnh imageCLEF, và SVI trên 1,8 triệu bài báo New York Times bằng LDA) để minh chứng cho lợi ích tốc độ của VI, không chỉ dừng ở lý thuyết. Vì đây là bài review, phần lớn ”kết quả chính” là trình bày lại/hệ thống hóa các kết quả đã biết (CAVI, SVI của Hoffman và cộng sự 2013) hơn là đề xuất phương pháp hoàn toàn mới, ngoại trừ chính hai thí nghiệm thực nghiệm so sánh nêu trên do nhóm tác giả tự thực hiện. Paper cũng rất thẳng thắn về giới hạn của chính họ phương pháp (mean-field đánh giá thấp phương sai, chỉ tìm tối ưu cục bộ, cần cấu trúc conjugate để có công thức đóng), và liệt kê rõ các vấn đề mở (mục 5.4) thay vì tuyên bố VI đã giải quyết xong mọi bài toán suy luận Bayes. Ghi chú này đã đọc toàn bộ mục 1-5 và cả ba phụ lục A, B, C; phần danh mục tham khảo chỉ đọc lướt để xác nhận đó là trích dẫn thuần túy.

Nguồn gốc: https://arxiv.org/abs/1601.00670

Bài 09

An Introduction to Variational Autoencoders (2019)

Tác giả: Diederik P. Kingma (Google), Max Welling (Universiteit van Amsterdam; Qualcomm)
Venue / arXiv: arXiv:1906.02691 (bản v3, ngày 11/12/2019); paper tự nêu venue ngay trên trang đầu: ”Foundations and Trends® in Machine Learning”, dù số volume/issue ghi là ”xx” (placeholder, có thể do đây là phiên bản tiền in trước khi có số trang tạp chí chính thức)
Nguồn: https://arxiv.org/abs/1906.02691
Chủ đề: Variational autoencoder (VAE), ELBO, reparameterization trick, amortized inference, normalizing flow (inverse autoregressive flow), mô hình sinh biến ẩn sâu (deep latent variable model)
Vị trí trong lộ trình: Bài số 9/9 trong nhóm bài toán nền (math 01-09), bài cuối cùng của nhóm này. Đọc sau math 08 (Variational Inference: A Review for Statisticians) vì VAE dùng đúng khung ELBO/KL của math 08 nhưng thay biến phân cục bộ theo từng điểm dữ liệu bằng một mạng nơ-ron suy luận dùng chung (amortized inference). Cũng dùng lại chain rule/automatic differentiation của math 01-02 (để lan truyền gradient qua phép lấy mẫu) và có liên hệ trực tiếp với PCA ở math 03 (giới hạn tuyến tính không nhiễu của VAE chính là PCA).

> Ghi chú này dựa trên toàn văn bài báo (đọc từ PDF chuyển sang text, khoảng 89 trang, 3225 dòng), gồm Chương 1-5 và Phụ lục A, đọc đầy đủ; phần danh mục tài liệu tham khảo ở cuối (khoảng 10 trang, hơn 150 mục) chỉ đọc lướt để xác nhận đó là danh mục trích dẫn thuần túy, không chứa nội dung kỹ thuật cần tóm tắt. Đây không phải bản dịch mà là ghi chú diễn giải lại cho người mới học mô hình sinh biến ẩn sâu. Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Đây là bản mở rộng chính thức (paper tự nói ở mục 1.2: ”an expanded version of our earlier work”) của bài báo gốc nổi tiếng Kingma và Welling (2014), ”Auto-Encoding Variational Bayes” (viết tắt AEVB), người đưa ra khái niệm variational autoencoder (VAE, bộ tự mã hóa biến phân). Bài 2019 này viết dưới dạng một chuyên khảo (monograph) cho ”Foundations and Trends in Machine Learning”, trình bày lại nền tảng VAE kỹ hơn và bổ sung các mở rộng quan trọng sau năm 2014 (inverse autoregressive flow, mô hình biến ẩn nhiều tầng, v.v.).

Động lực theo paper (mục 1.1): phân biệt mô hình sinh (generative, học phân phối chung trên mọi biến) với mô hình phân biệt (discriminative, chỉ học ánh xạ dự đoán). VAE là một khung giúp học đồng thời một mô hình sinh biến ẩn sâu (deep latent-variable model, DLVM) và một mô hình suy luận tương ứng, bằng stochastic gradient descent (SGD). Paper mô tả VAE gồm hai mô hình ghép cặp nhưng tham số hóa độc lập: encoder (mô hình nhận dạng/suy luận) và decoder (mô hình sinh). Theo paper, đóng góp lớn nhất của khung VAE có thể là nhận ra rằng có thể giảm nhiễu trong gradient bằng ”reparameterization trick” (mẹo đổi biến). VAE được paper mô tả là lấy cảm hứng từ Helmholtz Machine (Dayan và cộng sự, 1995) nhưng khác ở điểm quan trọng: Helmholtz Machine dùng thuật toán wake-sleep, tối ưu hai hàm mục tiêu riêng biệt không tương ứng với một bound chung nào của log-likelihood, còn VAE tối ưu một mục tiêu duy nhất.

Paper giả định người đọc có kiến thức cơ bản về đại số, giải tích và lý thuyết xác suất, và nói rõ đây không nhằm là một review toàn diện mọi công trình liên quan, mà là mở rộng có chọn lọc của bài 2014.

2. Ý tưởng chính

Mô hình biến ẩn sâu (DLVM): Có dữ liệu quan sát x và biến ẩn z (không quan sát được), mô hình sinh $p_\theta(x, z) = p_\theta(z)\, p_\theta(x \mid z)$, trong đó $p_\theta(z)$ là prior và $p_\theta(x \mid z)$ (decoder) được tham số hóa bằng mạng nơ-ron. Likelihood biên (marginal likelihood/evidence)

$$p_\theta(x) = \int p_\theta(x, z)\, dz$$

không tính được dạng đóng khi decoder là một mạng nơ-ron phi tuyến (mục 1.8 của paper). Vì $p_\theta(z \mid x) = p_\theta(x, z)/p_\theta(x)$, posterior cũng không tính được khi $p_\theta(x)$ không tính được, và ngược lại.

Ý tưởng VAE: đưa vào một mô hình suy luận (encoder) $q_\phi(z \mid x)$ xấp xỉ posterior thật, $q_\phi(z \mid x) \approx p_\theta(z \mid x)$, cũng tham số hóa bằng mạng nơ-ron, ví dụ

$$(\mu, \log \sigma) = \text{EncoderNeuralNet}_\phi(x), \qquad q_\phi(z \mid x) = \mathcal{N}(z; \mu, \text{diag}(\sigma^2)).$$

Khác biệt quan trọng với variational inference ”cổ điển” (chính là math 08): ở VI cổ điển, mỗi điểm dữ liệu có một bộ tham số biến phân riêng, tối ưu lặp đi lặp lại cho từng điểm (như CAVI ở math 08). VAE dùng MỘT mạng encoder chung cho toàn bộ tập dữ liệu: tham số ϕ (trọng số mạng) được chia sẻ giữa tất cả điểm dữ liệu, mỗi lần suy luận chỉ cần một lượt truyền xuôi (feedforward). Paper gọi đây là ”amortized inference” (suy luận khấu hao/chia sẻ chi phí): tránh phải tối ưu lặp cho từng điểm riêng lẻ, đổi lại phải chịu nhiễu từ việc lấy mẫu z trong quá trình huấn luyện.

Mục tiêu tối ưu: vẫn là ELBO như ở math 08, nhưng bây giờ tối ưu đồng thời cả θ (decoder) và ϕ (encoder) bằng SGD, nhờ một kỹ thuật cho phép lan truyền gradient qua bước lấy mẫu ngẫu nhiên $z \sim q_\phi(z \mid x)$: đó là reparameterization trick, viết $z = g(\varepsilon, \phi, x)$ với ε là nhiễu độc lập với ϕ, x, để ”ngoại hóa” phần ngẫu nhiên và lan truyền gradient qua phần còn lại (hàm g) bằng backpropagation thông thường.

3. Phương pháp chi tiết

3.1 ELBO (dẫn theo cách của paper, không qua bất đẳng thức Jensen)

Paper chọn một cách dẫn ELBO khác với cách dùng bất đẳng thức Jensen thường thấy, để ”cho thấy rõ độ chặt (tightness) của bound” (mục 2.2). Với $q_\phi(z \mid x)$ bất kỳ:

$$\log p_\theta(x) = \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\Big] + \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\Big] = \mathcal{L}_{\theta,\phi}(x) + D_{KL}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)),$$

trong đó

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x, z) - \log q_\phi(z \mid x)]$$

là ELBO. Vì $D_{KL} \geq 0$, $\mathcal{L}_{\theta,\phi}(x) \leq \log p_\theta(x)$. Paper nhấn mạnh điểm ”two for one” (mục 2.2.1): cực đại ELBO theo cả θ và ϕ cùng lúc vừa gần đúng cực đại likelihood biên $p_\theta(x)$ (mô hình sinh tốt hơn), vừa cực tiểu $D_{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$ (mô hình suy luận tốt hơn): một hàm mục tiêu, hai mục tiêu cùng đạt được.

3.2 Gradient theo θ dễ, theo ϕ khó

Gradient theo θ đơn giản vì có thể đổi chỗ kỳ vọng và gradient (kỳ vọng không phụ thuộc θ):

$$\nabla_\theta \mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z|x)}[\nabla_\theta \log p_\theta(x, z)] \approx \nabla_\theta \log p_\theta(x, z)$$

(ước lượng Monte Carlo một mẫu). Nhưng gradient theo ϕ khó vì kỳ vọng được lấy theo chính $q_\phi(z \mid x)$, một phân phối phụ thuộc ϕ: không thể đổi chỗ ∇_ϕ và $\mathbb{E}_{q_\phi(z|x)}[\cdot]$ một cách trực tiếp.

3.3 Reparameterization trick

Với biến ẩn liên tục và encoder/decoder khả vi, paper viết lại $z \sim q_\phi(z \mid x)$ thành một phép biến đổi khả vi, khả nghịch của một biến nhiễu độc lập:

$$z = g(\varepsilon, \phi, x), \qquad \varepsilon \sim p(\varepsilon) \text{ (không phụ thuộc } \phi, x).$$

Khi đó $\mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{p(\varepsilon)}[f(z)]$, và kỳ vọng/gradient theo ϕ giao hoán được:

$$\nabla_\phi \mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{p(\varepsilon)}[\nabla_\phi f(z)] \approx \nabla_\phi f(z).$$

ELBO viết lại: $\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{p(\varepsilon)}[\log p_\theta(x, z) - \log q_\phi(z \mid x)]$ với $z = g(\varepsilon, \phi, x)$, và ước lượng một mẫu $\tilde{\mathcal{L}}_{\theta,\phi}(x) = \log p_\theta(x, z) - \log q_\phi(z \mid x)$. Thuật toán tổng quát (Algorithm 1 của paper) gọi là Auto-Encoding Variational Bayes (AEVB); vì nhiễu đến từ cả việc lấy minibatch và việc lấy mẫu ε, đây là một thủ tục "doubly stochastic" (ngẫu nhiên hai lớp). Paper chứng minh tính không chệch (unbiasedness) của gradient này bằng đại số trực tiếp (Eq. 2.30-2.32 của paper).

Để tính $\log q_\phi(z \mid x)$, paper dùng công thức đổi biến với định thức Jacobian:

$$\log q_\phi(z \mid x) = \log p(\varepsilon) - \log d_\phi(x, \varepsilon), \qquad \log d_\phi(x, \varepsilon) = \log \left| \det \frac{\partial z}{\partial \varepsilon} \right|.$$

3.4 Posterior Gaussian factorized và full-covariance

Với encoder Gaussian factorized $q_\phi(z \mid x) = \mathcal{N}(z; \mu, \text{diag}(\sigma^2))$, paper dùng $\varepsilon \sim \mathcal{N}(0, I)$, $z = \mu + \sigma \odot \varepsilon$ ($\odot$ là tích từng phần tử); Jacobian là $\text{diag}(\sigma)$, log-định thức $= \sum_i \log \sigma_i$; mật độ posterior

$$\log q_\phi(z \mid x) = \sum_i [\log \mathcal{N}(\varepsilon_i; 0, 1) - \log \sigma_i].$$

Mở rộng sang full-covariance Gaussian (mục 2.5.1): $z = \mu + L\varepsilon$ với L là ma trận tam giác (phân rã Cholesky $\Sigma = LL^\top$), xây dựng qua $L \leftarrow L_{\text{mask}} \odot L_0 + \text{diag}(\sigma)$ (L_{mask} là ma trận mặt nạ 0 trên/trên đường chéo, 1 dưới đường chéo); định thức Jacobian vẫn đơn giản vì L tam giác: $\log |\det L| = \sum_i \log |L_{ii}|$. Paper cho thuật toán cụ thể (Algorithm 2) tính ước lượng ELBO một mẫu cho một VAE ví dụ: encoder Gaussian full-covariance, decoder Bernoulli factorized (cho dữ liệu nhị phân).

Ví dụ (tự tính): paper cho công thức tổng quát ở Algorithm 2 nhưng không có con số cụ thể nào được tính sẵn trong văn bản chính. Dưới đây tính tay một trường hợp đơn giản (rút gọn về encoder factorized Gaussian một chiều thay vì full-covariance, để dễ tính tay) theo đúng công thức của Algorithm 2, với dữ liệu Bernoulli 2 chiều.

Giả sử với một điểm dữ liệu $x = (1, 0)$, encoder cho ra $\mu = 0.5$, $\log \sigma = -0.5$ (vậy $\sigma = e^{-0.5} \approx 0.6065$), và lấy mẫu $\varepsilon = 1.0$:

$$z = \mu + \sigma\varepsilon = 0.5 + 0.6065 \times 1.0 = 1.1065.$$

Tính $\tilde{\mathcal{L}}_{\log q(z)} = -(\frac{1}{2}\varepsilon^2 + \frac{1}{2}\log(2\pi) + \log\sigma) = -(0.5 + 0.91894 - 0.5) = -0.91894$.

Tính $\tilde{\mathcal{L}}_{\log p(z)} = -(\frac{1}{2}z^2 + \frac{1}{2}\log(2\pi)) = -(0.5 \times 1.1065^2 + 0.91894) = -(0.61215 + 0.91894) = -1.53109$.

Giả sử decoder cho $p = (0.7, 0.3)$ (xác suất Bernoulli cho hai chiều của x). Với $x = (1, 0)$:

$\tilde{\mathcal{L}}_{\log p(x|z)} = [1\cdot\log 0.7 + 0\cdot\log 0.3] + [0\cdot\log 0.3 + 1\cdot\log(1-0.3)] = \log 0.7 + \log 0.7 = 2\times(-0.356675) = -0.71335.$

ELBO ước lượng một mẫu: $\tilde{\mathcal{L}} = \tilde{\mathcal{L}}_{\log p(x|z)} + \tilde{\mathcal{L}}_{\log p(z)} - \tilde{\mathcal{L}}_{\log q(z)} = -0.71335 - 1.53109 - (-0.91894) = -1.3255$ nat.

So sánh với công thức đóng (ngoài phạm vi các công thức viết rõ trong các chương đã đọc của paper, nhưng là kết quả chuẩn, thường thấy ở phụ lục của bài Kingma và Welling 2014 gốc): với $q = \mathcal{N}(\mu, \sigma^2)$ và prior $\mathcal{N}(0, 1)$, có công thức đóng

$$D_{KL}(q\|p) = \tfrac{1}{2}(\sigma^2 + \mu^2 - 1 - \log\sigma^2).$$

Với $\mu = 0.5$, $\sigma^2 = e^{-1} \approx 0.3679$: $D_{KL} = 0.5 \times (0.3679 + 0.25 - 1 - (-1)) = 0.5 \times 0.6179 = 0.30895$, tức $\mathbb{E}[\log p(z) - \log q(z)] = -0.30895$ (giá trị kỳ vọng đúng). Số tính bằng một mẫu ở trên cho $-\tilde{\mathcal{L}}_{\log q(z)} + \tilde{\mathcal{L}}_{\log p(z)} = -0.61215$, khác với -0.30895. Đây không phải sai số tính toán, mà minh họa đúng bản chất "ước lượng Monte Carlo một mẫu không chệch trong kỳ vọng, nhưng có nhiễu với từng mẫu cụ thể" mà paper nhấn mạnh ở mục 2.4.3 (tính không chệch chỉ đúng khi lấy trung bình trên nhiều mẫu ε, không đúng cho một mẫu đơn lẻ).

3.5 Vượt ra ngoài Gaussian: normalizing flow và IAF

Chương 3 của paper thảo luận hai hướng làm posterior linh hoạt hơn: biến ẩn phụ trợ (auxiliary latent variables) và normalizing flow. Normalizing flow (Rezende và Mohamed, 2015) xây một chuỗi biến đổi khả nghịch $z_t = f_t(z_{t-1}, x)$ bắt đầu từ $\varepsilon_0 \sim p(\varepsilon)$ đơn giản; định thức Jacobian của cả chuỗi là tích các định thức từng bước, nên log-định thức là tổng. Paper đề xuất (dựa trên Kingma và cộng sự, 2016) Inverse Autoregressive Flow (IAF): mỗi bước là một biến đổi tự hồi quy nghịch (inverse autoregressive), có hai tính chất quan trọng: (1) có thể tính song song trên các chiều (khác với việc lấy mẫu từ một mô hình tự hồi quy thường, vốn phải tính tuần tự); (2) Jacobian tam giác với định thức đơn giản là tích các σ_i. IAF ghép T bước như vậy (Algorithm 3), cho phép posterior linh hoạt hơn nhiều so với Gaussian factorized, đặc biệt quan trọng khi không gian biến ẩn có chiều cao. Hình 3.2 của paper minh họa trực quan: trên một tập đồ chơi 4 điểm dữ liệu, posterior factorized Gaussian cho ra các "đốm" tròn đều, còn posterior IAF cho ra hình dạng lệch, phi-Gaussian, khớp tốt hơn với posterior thật.

Chương 4 bàn về mô hình sinh sâu hơn: nhiều tầng biến ẩn $z = \{z_1, ..., z_K\}$ (lấy mẫu tuần tự theo thứ tự topo), và lựa chọn thứ tự topo giữa inference model và generative model (cùng chiều "top-down" hay ngược chiều "bottom-up"); paper trích nhiều công trình (Salimans 2016; Sønderby và cộng sự 2016a; Kingma và cộng sự 2016) cho thấy dùng cùng thứ tự top-down cho cả hai giúp chia sẻ tham số, học nhanh hơn.

4. Thí nghiệm và kết quả chính

Đây là một bài mang tính chuyên khảo/sư phạm hơn là một bài công bố thực nghiệm mới; phần lớn các "kết quả" được trích dẫn lại từ các bài khác mà paper tổng hợp, cần phân biệt rõ với phần thực nghiệm (nếu có) của chính paper này:

- **Hình 3.2 (thực nghiệm minh họa của chính paper hoặc kế thừa trực tiếp từ Kingma và cộng sự 2016, cùng nhóm tác giả):** trên một tập dữ liệu đồ chơi 4 điểm, so sánh posterior factorized Gaussian với posterior IAF; kết quả hình ảnh cho thấy IAF linh hoạt hơn rõ rệt.
- Mục 3.2.1 có câu "In our experiment with CIFAR-10, we make use of multiple layers of stochastic variables" (trong thí nghiệm của chúng tôi với CIFAR-10, chúng tôi dùng nhiều tầng biến ngẫu nhiên) nhưng không kèm bảng số liệu định lượng cụ thể nào trong các chương chính đã đọc; đây là một câu nhắc tới thực nghiệm mà không trình bày chi tiết kết quả trong văn bản.

- **Mục 2.9.1 (kết quả định lượng trích dẫn từ một bài khác):** paper trích Kucukelbir và cộng sự (2017) cho biết ước lượng gradient kiểu score-function (REINFORCE) cần ”hai cấp độ lớn” (two orders of magnitude) nhiều mẫu hơn ước lượng theo reparameterization để đạt phương sai tương đương. Đây là một con số định lượng thật, nhưng là kết quả của một bài khác được paper này trích dẫn lại, không phải thực nghiệm tự paper này chạy.
- **Mục 4.5.1 (các kết quả trích dẫn từ nhiều bài follow-up khác):** ví dụ Maaløe và cộng sự (2016) đạt dưới 1% lỗi phân loại trên MNIST khi chỉ dùng 10 ảnh có nhãn mỗi lớp (hơn 99.8% nhãn bị bỏ) bằng kỹ thuật bán giám sát dựa trên VAE; Pu và cộng sự (2016) so sánh GoogLeNet huấn luyện với 1% ảnh ImageNet có nhãn (khoảng 5% độ chính xác top-1) với một cách tiếp cận bán giám sát dùng VAE (khoảng 45% độ chính xác) trên cùng thiết lập. Cả hai là kết quả thật, có số liệu cụ thể, nhưng đều là kết quả của các bài khác, được chương ”Follow-Up Work” của paper này tổng hợp lại, không phải thực nghiệm gốc của chính Kingma và Welling trong bài 2019 này.
- Các ứng dụng khác được liệt kê (thiết kế hóa học của Gómez-Bombarelli và cộng sự 2018, sinh câu văn của Bowman và cộng sự 2015, mô phỏng ảnh thiên văn của Ravanbakhsh và cộng sự 2017, ”smile vector” của White 2016) đều là ví dụ minh họa từ các bài khác, trình bày lại dưới dạng hình ảnh trong paper, không phải số liệu định lượng của chính paper 2019 này.

Nhận xét của người ghi chú: khác với math 08 (có ít nhất hai thực nghiệm thật tự tác giả chạy: so sánh CAVI với Stan, và LDA trên 1,8 triệu bài báo), bài VAE 2019 này gần như không có một bảng số liệu thực nghiệm mới nào do chính Kingma và Welling tự chạy và báo cáo đầy đủ trong văn bản chính; vai trò chính của bài là trình bày lại khung lý thuyết (ELBO, reparameterization, IAF) và tổng hợp có hệ thống các kết quả thực nghiệm từ rất nhiều bài khác.

5. Phân tích / ablation

Không có bảng ablation có kiểm soát do chính paper này tự thực hiện. Các phần gần nhất với ”phân tích” là:

- Mục 2.9.1: so sánh định tính và trích một con số định lượng (hai cấp độ lớn số mẫu) giữa hai loại ước lượng gradient (reparameterization và score function), giải thích bằng trực giác ”information bottleneck”: ước lượng score function chỉ dùng giá trị vô hướng $\log p_\theta(x, z)$, còn ước lượng reparameterization dùng được cả vector gradient $\nabla_z \log p_\theta(x, z)$ (rộng hơn, nhiều thông tin hơn), nên phương sai thấp hơn.
- Mục 2.8.1: khoảng giá trị free-bits $\lambda \in \{0.125, 0.25, 0.5, 1, 2\}$ được báo cáo là ”hoạt động tốt cho một khoảng khá rộng giá trị”, cải thiện log-likelihood trên một bộ benchmark, nhưng đây là kết quả trích từ Kingma và cộng sự (2016), không phải khảo sát mới của bài 2019 này.
- Hình 3.2 là phép so sánh định tính (trực quan, không có bảng số liệu) giữa hai lựa chọn kiến trúc posterior (factorized Gaussian và IAF).

6. Giới hạn

Giới hạn/nhược điểm paper tự nêu rõ:

- **Vấn đề tối ưu/”posterior collapse” (mục 2.8.1):** paper báo cáo (nhất quán với Bowman và cộng sự 2015 và Sønderby và cộng sự 2016a) rằng tối ưu ngẫu nhiên với ELBO nguyên bản có thể bị kẹt ở một điểm cân bằng ổn định không mong muốn, nơi $q(z \mid x) \approx p(z)$ (biến ẩn gần như bị ”bỏ qua”) ngay từ đầu huấn luyện, vì khi đó số hạng likelihood $\log p(x \mid z)$ còn yếu. Giải pháp (KL annealing, hoặc ”free bits” của Kingma và cộng sự 2016) chỉ là các biện pháp khắc phục thực nghiệm (heuristic), không phải một sửa đổi có cơ sở lý thuyết đầy đủ.
- **”Blurriness” (mục 2.7-2.8.2):** vì ELBO tương đương cực tiểu $D_{KL}(q_{D,\phi}(x, z) \| p_\theta(x, z))$ theo một chiều cụ thể của KL (zero-avoiding, không phải zero-forcing), nếu không khớp hoàn hảo được, mô hình sinh p_θ có xu hướng trải rộng xác suất hơn phân phối dữ liệu thật (vì bị phạt rất nặng nếu bỏ sót

vùng dữ liệu thật có xác suất, nhưng bị phạt nhẹ nếu ”lan” xác suất ra vùng không có dữ liệu thật). Đây là lời giải thích của chính paper cho hiện tượng ảnh sinh ra từ VAE thường bị mờ/nhòe so với ảnh thật.

- **Posterior factorized/Gaussian đơn giản hạn chế độ linh hoạt:** động lực cho cả chương 3 (normalizing flow, IAF); ngay cả flow tổng quát cũng bị giới hạn bởi yêu cầu khả nghịch và Jacobian dễ tính (planar/radial flow của Rezende và Mohamed 2015 được paper tự nhận xét là ”không rõ cách mở rộng lên không gian biến ẩn có chiều rất cao”).
- **IWAE (mục 2.6):** paper tự nhận xét ước lượng importance-weighted có ”scaling rất tệ” (notoriously bad scaling properties) khi không gian biến ẩn có chiều cao.
- **Reparameterization trick chỉ áp dụng cho biến ẩn liên tục:** biến ẩn rời rạc cần ước lượng kiểu score function/REINFORCE (mục 2.9.1), có phương sai cao hơn hẳn theo chính kết quả paper trích dẫn.
- **Tuyên bố mạnh ở phần kết luận (mục 5):** paper tự nhận ”VAE vẫn là khung duy nhất trong tài liệu cho phép cả biến quan sát rời rạc và liên tục, cho phép suy luận biến ẩn khấu hao hiệu quả và tổng hợp (synthesis) nhanh, và có thể đạt hiệu năng gần trạng thái tiên tiến nhất về log-likelihood dữ liệu”. **Quan sát riêng của người ghi chú:** đây là đánh giá của chính các tác giả tại thời điểm viết (bản v3, cuối 2019); đây là một tuyên bố mạnh, mang tính chủ quan của tác giả về vị trí của phương pháp so với các họ mô hình khác, không phải một kết quả đo lường được, nên cần đọc như một nhận định có bối cảnh thời điểm, không phải một sự thật toán học.

7. Liên hệ với các bài khác

- **math 08 (Variational Inference: A Review for Statisticians):** paper này dùng đúng khung ELBO/KL của math 08 nhưng khác biệt cốt lõi được chính văn bản paper nêu rõ (mục 2.1): VAE dùng ”amortized inference” (một mạng suy luận dùng chung cho mọi điểm dữ liệu), trong khi ”các phương pháp variational inference truyền thống” (chính là CAVI ở math 08) dùng tham số biến phân riêng cho từng điểm dữ liệu, kém hiệu quả với tập dữ liệu lớn. Phụ lục A.2.2 của paper này còn viết lại rõ công thức variational EM với tham số cục bộ $\phi^{(i)}$ (đúng là công thức CAVI của math 08) để làm nền so sánh.
- **math 01/02 (Matrix Calculus / Automatic Differentiation):** reparameterization trick (mục 2.4) hoạt động được chính là nhờ lan truyền gradient qua phép biến đổi $z = g(\varepsilon, \phi, x)$ bằng backpropagation/automatic differentiation; Phụ lục A.3 của paper này gọi thẳng đây là ”reverse-mode automatic differentiation (còn gọi là thuật toán backpropagation)”, đúng chủ đề math 02; chain rule dùng trong suy luận gradient chính là chủ đề math 01.
- **math 03 (PCA Tutorial):** mục 2.9 của paper trích dẫn Roweis (1998) chỉ ra rằng ở giới hạn nhiễu decoder tiến về 0 của một VAE linear-Gaussian ($p(z) = \mathcal{N}(0, I)$, $p(x \mid z) = \mathcal{N}(x; Wz, \epsilon I)$), nghiệm maximum-likelihood của W chính là PCA, và posterior trở thành một phân phối Dirac delta (không còn độ không chắc chắn). Đây là liên hệ trực tiếp, được chính văn bản xác nhận, không phải suy diễn của người ghi chú.
- **math 10 (A Conceptual Introduction to Hamiltonian Monte Carlo):** Phụ lục A.2.3 (”MCMC-EM”) của paper này thảo luận một cách khác để tối ưu likelihood bằng EM kết hợp MCMC, trích dẫn Neal (2011) ”MCMC Using Hamiltonian Dynamics”, đúng chủ đề cốt lõi của math 10, và paper tự nhận xét nhược điểm của hướng này là cần chạy một xích MCMC độc lập cho mỗi điểm dữ liệu (khác hẳn với việc dùng một mạng encoder chung của VAE).

8. Ứng dụng cho dự án (suy luận)

Dự án tham chiếu: LightOnOCR-3-0.8B, một mô hình VLM OCR, kiến trúc hybrid GatedDeltaNet kết hợp full attention, khoảng 3.600 token/trang, GPU tối đa 8GB, batch size 1. Toàn bộ phần này là **suy luận** của người ghi chú, không phải nội dung trong paper.

Đánh giá tổng quát: **ít liên quan trực tiếp**. LightOnOCR là một mô hình sinh văn bản có điều kiện (conditional sequence model) huấn luyện bằng maximum likelihood trực tiếp trên token đầu ra, không có biến ẩn z với

prior/posterior rõ ràng, không tối ưu ELBO, nên không phải là một VAE và không dùng reparameterization trick trong kiến trúc công bố.

Hai điểm liên quan gián tiếp, mang tính suy luận:

1. Nếu dự án từng cân nhắc thêm một ”nút cổ chai” (bottleneck) ngẫu nhiên để nén số token hình ảnh (hiện khoảng 1.600-2.100 token thị giác mỗi trang) xuống ít hơn nhằm giảm chi phí tính toán trên GPU 8GB, thì reparameterization trick là công cụ chuẩn để làm bước nén đó có thể huấn luyện được bằng gradient (thay vì phải dùng ước lượng gradient kiểu score function, có phương sai cao hơn theo đúng kết quả mục 2.9.1 của paper). Đây là một hướng kỹ thuật khả thi về mặt lý thuyết, không phải điều paper hay kiến trúc LightOnOCR hiện có đề cập.
2. Cảnh báo ”blurriness” (mục 2.8.2, mục 6 trên đây) là bài học đáng cân nhắc nếu có ý tưởng dùng kiến trúc kiểu VAE cho việc nén/tái tạo token thị giác: xu hướng làm mờ/trung bình hóa đặc trưng của VAE có thể đi ngược lại yêu cầu giữ độ chính xác chi tiết (ví dụ nét chữ nhỏ) cần cho OCR; nguy cơ ”posterior collapse” (mục 2.8.1) cũng là rủi ro thực tế cần theo dõi nếu áp dụng bất kỳ bottleneck biến ẩn ngẫu nhiên nào vào pipeline.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ sau chạy được trên một GPU 8GB:

1. **VAE nhỏ trên dữ liệu ảnh đơn giản.** Cài một VAE nhỏ (encoder/decoder vài tầng fully-connected hoặc conv nhỏ, latent dim 2-20) trên MNIST hoặc một tập ký tự/glyph nhỏ, dùng đúng Algorithm 1 (reparameterization trick). Đo: đường cong ELBO theo epoch, chất lượng ảnh tái tạo (reconstruction), và chất lượng ảnh sinh từ mẫu prior $z \sim \mathcal{N}(0, I)$ thuần túy (không qua encoder). Kỳ vọng: ảnh tái tạo rõ hơn ảnh sinh từ prior thuần túy (nhất quán với mục 2.8.2 về blurriness khi sinh không điều kiện).
2. **So sánh posterior factorized Gaussian và IAF trên dữ liệu đồ chơi.** Lặp lại tinh thần Hình 3.2: một tập vài điểm dữ liệu 2D tổng hợp, huấn luyện một encoder Gaussian factorized và một encoder có thêm 1-2 bước IAF, trực quan hóa hình dạng posterior của từng điểm. Đo: ELBO cuối cùng của hai cấu hình. Kỳ vọng: IAF cho ELBO cao hơn (bound chặt hơn) nhờ posterior linh hoạt hơn.
3. **So sánh phương sai gradient: reparameterization và score function.** Trên một mô hình nhỏ cố định (ví dụ 1 tầng ẩn), tính gradient ELBO theo ϕ bằng cả hai ước lượng (reparameterization và REINFORCE/score function, mục 2.9.1 và 2.72-2.73 của paper) trên nhiều lần lấy mẫu lại, đo phương sai thực nghiệm của mỗi ước lượng. Kỳ vọng: ước lượng score function có phương sai lớn hơn đáng kể, kiểm chứng lại con số ”hai cấp độ lớn” mà paper trích dẫn từ Kucukelbir và cộng sự (2017), ở quy mô nhỏ hơn.
4. **Tái hiện posterior collapse và cách khắc phục.** Huấn luyện một VAE với decoder đủ mạnh (ví dụ decoder có nhiều tham số hơn hẳn encoder) trên dữ liệu tuần tự tổng hợp đơn giản, không dùng KL annealing; đo KL trung bình mỗi chiều latent theo epoch để kiểm tra có bị ”collapse” về 0 không (dấu hiệu $q(z \mid x) \approx p(z)$). Lặp lại với KL annealing hoặc free-bits (λ trong khoảng paper gợi ý, ví dụ 0.5) và so sánh KL trung bình, chất lượng tái tạo. Kỳ vọng: không annealing dễ bị collapse (KL gần 0), có annealing/free-bits giữ được KL dương ổn định hơn.
5. **Kiểm chứng liên hệ VAE tuyến tính và PCA (liên hệ trực tiếp với math 03).** Huấn luyện một VAE ”tuyến tính” (encoder và decoder đều là hàm tuyến tính, không có lớp phi tuyến) trên một tập dữ liệu nhỏ, với nhiễu decoder rất nhỏ; so sánh không gian con học được bởi ma trận W của decoder với các thành phần chính (principal components) tính trực tiếp bằng eigen-decomposition của ma trận hiệp phương sai (như ở math 03). Kỳ vọng: khi nhiễu decoder càng nhỏ, không gian con của W càng gần không gian các thành phần chính đầu, xác nhận số liệu cho liên hệ Roweis (1998) mà paper trích ở mục 2.9.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Mô hình sinh biến ẩn sâu (DLVM)	$p_\theta(x,z) = p_\theta(z)\,p_\theta(x \mid z)$, likelihood biên $p_\theta(x) = \int p_\theta(x,z)\,dz$ thường không tính được
Encoder / mô hình suy luận	$q_\phi(z \mid x) \approx p_\theta(z \mid x)$, tham số ϕ dùng chung cho mọi điểm dữ liệu (amortized inference)
ELBO	$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z\|x)}[\log p_\theta(x,z) - \log q_\phi(z \mid x)]$
Liên hệ ELBO - log likelihood	$\log p_\theta(x) = \mathcal{L}_{\theta,\phi}(x) + D_{KL}(q_\phi(z \mid x)\|p_\theta(z \mid x)) \geq \mathcal{L}_{\theta,\phi}(x)$
Reparameterization trick	$z = g(\varepsilon,\phi,x)$, $\varepsilon \sim p(\varepsilon)$ độc lập ϕ, x; cho phép $\nabla_\phi \mathbb{E}_{q_\phi}[f(z)] \approx \nabla_\phi f(z)$
Log-mật độ qua đổi biến	$\log q_\phi(z \mid x) = \log p(\varepsilon) - \log\|\det(\partial z/\partial\varepsilon)\|$
Gaussian factorized	$z = \mu + \sigma \odot \varepsilon$, log-định thức Jacobian $= \sum_i \log\sigma_i$
Gaussian full-covariance	$z = \mu + L\varepsilon$, L tam giác (Cholesky của $\Sigma = LL^\top$), log-định thức $= \sum_i \log\|L_{ii}\|$
Normalizing flow / IAF	Chuỗi biến đổi khả nghịch $z_t = f_t(z_{t-1}, x)$, log-định thức Jacobian toàn chuỗi là tổng log-định thức từng bước
KL Gaussian-Gaussian (công thức đóng, không nằm trong các chương chính đã đọc)	$D_{KL}(\mathcal{N}(\mu,\sigma^2)\|\mathcal{N}(0,1)) = \frac{1}{2}(\sigma^2 + \mu^2 - 1 - \log\sigma^2)$
Hạn chế đã biết	Posterior collapse (mục 2.8.1); ”blurriness” do chiều KL (mục 2.8.2); reparameterization chỉ cho biến liên tục

11. Kết luận ngắn

Bài này là bản mở rộng có tính sư phạm của bài AEVB gốc (Kingma và Welling, 2014), trình bày lại ELBO, reparameterization trick, và các mở rộng (normalizing flow/IAF, mô hình nhiều tầng biến ẩn) một cách mạch lạc, có chứng minh từng bước (ví dụ tính không chệch của gradient reparameterization). Khác với math 08, bài này gần như không có thực nghiệm số liệu mới do chính tác giả chạy và báo cáo chi tiết trong văn bản chính; phần lớn ”kết quả” ở chương 4 là tổng hợp lại các con số thật nhưng lấy từ rất nhiều bài follow-up khác (bán giám sát trên MNIST/ImageNet, thiết kế hóa học, sinh câu văn, v.v.), cần đọc với sự phân biệt rõ nguồn gốc. Về mặt lý thuyết, đóng góp cốt lõi (reparameterization trick, amortized inference) được trình bày chặt chẽ, và paper cũng thẳng thắn nêu các hạn chế đã biết của VAE (posterior collapse, blurriness, giới hạn của reparameterization với biến rời rạc). Ghi chú này đã đọc toàn bộ 5 chương và Phụ lục A; phần danh mục tham khảo chỉ đọc lướt để xác nhận đó là trích dẫn thuần túy.

Nguồn gốc: https://arxiv.org/abs/1906.02691

Bài 10

A Conceptual Introduction to Hamiltonian Monte Carlo (2017)

Tác giả: Michael Betancourt (Applied Statistics Center, Columbia University; phần lớn bài viết hoàn thành khi là Research Fellow tại Centre for Research in Statistical Methodology, Đại học Warwick).
Venue / arXiv: arXiv:1701.02434 (v2, 16/07/2018); paper không nêu venue hội nghị hay tạp chí cụ thể trên bản arXiv.
Nguồn: https://arxiv.org/abs/1701.02434
Chủ đề: giải thích trực giác hình học (tránh dùng vi phân hình học hình thức) cho việc tại sao Hamiltonian Monte Carlo (HMC) khám phá hiệu quả các phân phối xác suất nhiều chiều, cách chỉnh các bậc tự do của HMC (kinetic energy, thời gian tích phân, symplectic integrator) để triển khai tốt, và cách chẩn đoán khi phương pháp thất bại.
Vị trí trong lộ trình: đọc sau math 08 (Variational Inference: A Review for Statisticians), vì cả hai là họ phương pháp suy luận xấp xỉ hậu nghiệm Bayes nhưng theo hướng ngược nhau (lấy mẫu MCMC so với tối ưu biến phân); đọc trước math 11 (Stochastic Gradient Descent as Approximate Bayesian Inference), vì bài 11 dùng chính ngôn ngữ ”động lực học” tương tự (Langevin dynamics, nhiệt độ, nhiễu) để biến SGD thành một dạng lấy mẫu Bayes gần đúng và có nhắc tới HMC như một họ phương pháp liên quan. Cũng liên quan tới math 06 (natural gradient) qua khái niệm Riemannian metric.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Bài toán trung tâm của thống kê tính toán là ước lượng kỳ vọng $\mathbb{E}_\pi[f]$ của một hàm f theo một phân phối mục tiêu π trên không gian tham số D chiều Q (ví dụ hậu nghiệm Bayes, hoặc phân phối likelihood trong thống kê tần suất). Khi π không có dạng giải tích đơn giản, ta phải dùng phương pháp số, và với bài toán nhiều chiều phương pháp phổ biến nhất là **Markov chain Monte Carlo (MCMC)**: xây một xích Markov có phân phối dừng (stationary distribution) chính là π, rồi lấy trung bình hàm f trên các mẫu của xích.

Khó khăn nằm ở **hình học của không gian nhiều chiều**. Paper chỉ ra (Hình 1, Hình 2 trong bài) rằng khi chia không gian D chiều thành các hộp quanh mode, số hộp lân cận mode là $3^D - 1$: với $D = 1$ có 2 hộp lân cận, $D = 3$ đã có 26. Thể tích bên ngoài áp đảo thể tích quanh mode theo cấp lũy thừa khi D tăng, trong khi mật độ xác suất $\pi(q)$ lại lớn nhất tại mode. Kỳ vọng được tính bằng tích phân $\int \pi(q) f(q)\, dq$, mà tích phân này là **tích của mật độ và thể tích**; vùng mật độ lớn thì thể tích nhỏ, vùng thể tích lớn thì mật độ nhỏ, nên vùng đóng góp chính cho kỳ vọng là một ”vành” hẹp giữa hai cực gọi là **typical set**. Khi D tăng, typical set càng mỏng và càng xa mode (hiện tượng ”concentration of measure”). Hệ quả: lưới đều (quadrature) cần N^D điểm để phủ không gian nhưng hầu như không điểm nào rơi vào typical set, nên quadrature thất bại theo cấp lũy thừa với D. Mọi thuật toán lấy mẫu hiệu quả phải tập trung tính toán vào typical set.

Khái niệm cần nắm trước khi đọc (paper giả định người đọc chỉ cần xác suất và giải tích cơ bản, không cần hình học vi phân):

- **Markov transition** $T(q' \mid q)$: xác suất chuyển từ điểm q sang q'. Transition ”bảo toàn” π nếu $\pi(q) = \int dq'\, \pi(q') T(q \mid q')$.
- **Hiệu quả (effective sample size, ESS):** $\text{ESS} = N / (1 + 2 \sum_{l=1}^{\infty} \rho_l)$ với ρ_l là tự tương quan trễ l của f theo lịch sử xích; ESS đo số mẫu độc lập ”tương đương” chứa trong xích.

- **Metropolis-Hastings (MH):** cơ chế đề xuất $Q(q' \mid q)$ rồi chấp nhận với xác suất $a(q' \mid q) = \min\left(1, \frac{Q(q|q')\pi(q')}{Q(q'|q)\pi(q)}\right)$.

2. Ý tưởng chính

Random Walk Metropolis (RWM, đề xuất Gaussian $Q(q' \mid q) = \mathcal{N}(q' \mid q, \Sigma)$) là thuật toán MCMC cổ điển nhất, nhưng nó chỉ "đoán và kiểm tra" ngẫu nhiên, không dùng thông tin cục bộ của π. Trong không gian nhiều chiều, hầu hết các đề xuất rơi ra ngoài typical set (bị density thấp từ chối) hoặc phải co nhỏ bước đi để được chấp nhận (nhưng khi đó xích di chuyển quá chậm). Paper lập luận: để mở rộng MCMC lên các bài toán nhiều chiều thực tế, ta cần **tận dụng thông tin hình học của typical set**, cụ thể là gradient $\nabla \log \pi(q)$.

Ý tưởng chính (Mục 3.1): một vector field thẳng theo gradient sẽ kéo ta về **mode** (nơi gradient bằng 0), không phải về typical set (vì gradient phụ thuộc cách tham số hoá, còn typical set thì không). Để "bẻ" gradient cho thẳng hàng với typical set, paper dùng một phép tương tự vật lý: nghĩ mode như một hành tinh, gradient như lực hấp dẫn, và typical set như quỹ đạo quay quanh hành tinh đó. Một vật ở trạng thái nghỉ sẽ rơi vào hành tinh (giống như một bước gradient thuần sẽ rơi vào mode); nhưng nếu trang bị đúng **động lượng (momentum)** vuông góc với lực hấp dẫn, vật sẽ duy trì quỹ đạo ổn định quanh hành tinh. Cách duy nhất để thêm động lượng có cấu trúc xác suất đúng (bảo toàn thể tích pha, volume-preserving) chính là **Hamiltonian Monte Carlo**: mở rộng không gian tham số q thành không gian pha (q, p) với p là biến động lượng phụ trợ, định nghĩa một hàm Hamiltonian $H(q, p) = -\log \pi(q, p)$, và dùng phương trình Hamilton để sinh ra các quỹ đạo bám theo typical set của phân phối mở rộng $\pi(q, p)$, sau đó chiếu ngược lại không gian q để được mẫu từ $\pi(q)$.

3. Phương pháp chi tiết

3.1 Không gian pha và phương trình Hamilton (Mục 3.2-3.3)

Mở rộng mỗi chiều q_n thành cặp (q_n, p_n), được không gian pha $2D$ chiều. Phân phối hậu nghiệm được "nâng" lên phân phối chính tắc (canonical distribution) trên không gian pha bằng cách chọn một phân phối có điều kiện $\pi(p \mid q)$ cho động lượng:

$$\pi(q, p) = \pi(p \mid q)\, \pi(q).$$

Vì $\pi(q, p)$ không phụ thuộc cách tham số hoá, ta viết nó qua hàm **Hamiltonian** (bất biến):

$$\pi(q, p) = e^{-H(q,p)}, \qquad H(q, p) = -\log \pi(q, p) = \underbrace{-\log \pi(p \mid q)}_{K(p,q)\ \text{(kinetic energy)}} + \underbrace{-\log \pi(q)}_{V(q)\ \text{(potential energy)}}.$$

$V(q)$ hoàn toàn do phân phối mục tiêu quyết định; $K(p, q)$ là bậc tự do do người triển khai chọn. Vector field sinh ra quỹ đạo khám phá là **phương trình Hamilton**:

$$\frac{dq}{dt} = \frac{\partial H}{\partial p} = \frac{\partial K}{\partial p}, \qquad \frac{dp}{dt} = -\frac{\partial H}{\partial q} = -\frac{\partial K}{\partial q} - \frac{\partial V}{\partial q}.$$

Vì $\partial V/\partial q = -\nabla \log \pi(q)$, phương trình Hamilton chính là cách "truyền" gradient qua động lượng để tạo chuyển động thẳng hàng với typical set, đúng như ý tưởng ở mục 2. Một Markov transition Hamiltonian lý tưởng gồm 3 bước: (1) lấy mẫu động lượng $p \sim \pi(p \mid q)$ (nâng điểm hiện tại lên không gian pha), (2) tích phân phương trình Hamilton trong thời gian t: $(q, p) \to \phi_t(q, p)$, (3) chiếu $(q, p) \to q$ (bỏ động lượng). Vì phương trình Hamilton bảo toàn H (do đó bảo toàn thể tích pha, shear transformation), quỹ đạo luôn ở trên một **mức năng lượng** (energy level set) $H^{-1}(E) = \{(q, p) : H(q, p) = E\}$.

3.2 Phân rã vi chính tắc (microcanonical decomposition, Mục 4.1)

Không gian pha phân tầng (foliate) thành các mức năng lượng lồng nhau. Mỗi điểm được xác định bởi năng lượng E và vị trí θ_E trên mức đó. Phân phối chính tắc phân rã thành:

$$\pi(q, p) = \pi(\theta_E \mid E)\, \pi(E),$$

gọi $\pi(\theta_E \mid E)$ là **phân phối vi chính tắc (microcanonical)** và $\pi(E)$ là **phân phối năng lượng biên (marginal energy distribution)**. Một xích Hamiltonian Markov chia thành hai pha xen kẽ: **tất định** (quỹ đạo Hamiltonian khám phá một mức năng lượng) và **ngẫu nhiên** (lấy lại mẫu động lượng tạo một bước random-walk giữa các mức năng lượng). Hiệu năng pha tất định phụ thuộc thời gian tích phân và độ đều của mức năng lượng; hiệu năng pha ngẫu nhiên phụ thuộc việc phân phối chuyển tiếp năng lượng $\pi(E \mid q)$ (sinh ra khi lấy lại mẫu p tại một điểm q) có "khớp" với $\pi(E)$ hay không. Nếu $\pi(E \mid q)$ hẹp hơn nhiều so với $\pi(E)$, random walk giữa các mức năng lượng rất chậm.

3.3 Chọn kinetic energy (Mục 4.2)

Euclidean-Gaussian: nếu không gian tham số có một metric Euclid g (ma trận $D \times D$), ta định nghĩa $M = R\, S\, g\, S^{\mathsf{T}} R^{\mathsf{T}}$ ("mass matrix"), và chọn $\pi(p \mid q) = \mathcal{N}(p \mid 0, M)$, cho kinetic energy:

$$K(q, p) = \tfrac{1}{2}\, p^{\mathsf{T}} M^{-1} p + \log |M| + \text{const.}$$

Lựa chọn tối ưu (Mục 4.2.1) là đặt nghịch đảo metric bằng hiệp phương sai mục tiêu, $M^{-1} = \mathbb{E}_\pi[(q-\mu)(q-\mu)^{\mathsf{T}}]$, ước lượng thực nghiệm trong giai đoạn warm-up (chạy thử, cập nhật M, chạy lại). Việc này tương đương xoay/co giãn không gian tham số để "khử tương quan" phân phối mục tiêu, làm các mức năng lượng đều hơn.

Riemannian-Gaussian (Mục 4.2.2, Girolami và Calderhead 2011): nếu phân phối mục tiêu có độ cong thay đổi theo vị trí (không Gaussian toàn cục), dùng metric biến thiên $\Sigma(q)$: $\pi(p \mid q) = \mathcal{N}(p \mid 0, \Sigma(q))$, $K(q, p) = \frac{1}{2} p^{\mathsf{T}} \Sigma^{-1}(q) p + \frac{1}{2} \log |\Sigma(q)|$. Nếu $\Sigma(q)$ gần giống Hessian của $-\log \pi(q)$ (ví dụ SoftAbs metric, Betancourt 2013a), các mức năng lượng được "sửa cục bộ" nên đều hơn ở mọi vùng. Paper nhận xét non-Gaussian kinetic energy thường kém hơn thực nghiệm, vì khi D lớn, $\pi(E)$ có xu hướng tiến về Gaussian theo định lý giới hạn trung tâm (CLT trên tổng nhiều biến), nên chỉ kinetic energy Gaussian mới khớp tốt.

3.4 Thời gian tích phân tối ưu và NUTS (Mục 4.3)

Với thời gian tích phân T cố định, kỳ vọng dọc theo quỹ đạo E_T hội tụ về kỳ vọng không gian E_ϕ trên "quỹ đạo kín" (orbit) khi $T \to \infty$ (gọi là "dynamic ergodicity"). Giai đoạn đầu hội tụ siêu tuyến tính (mỗi bước mới đều khám phá vùng chưa thăm), sau đó chuyển sang giai đoạn tiệm cận chậm (quỹ đạo quay lại vùng đã thăm). Với họ mục tiêu 1 chiều $\pi_\beta(q) \propto e^{-|q|^\beta}$ và kinetic energy chuẩn $\mathcal{N}(0, 1)$, thời gian tích phân tối ưu tỉ lệ:

$$T_{\text{optimal}}(q, p) \propto H(q, p)^{\frac{2-\beta}{2\beta}}.$$

Với $\beta < 2$ (đuôi nặng hơn Gaussian), T_{optimal} tăng khi năng lượng (tức đi sâu vào đuôi) tăng, nên một T tĩnh không bao giờ tối ưu cho mọi vùng; chỉ $\beta = 2$ (đúng Gaussian) mới có T tối ưu hằng số. Đây là động lực cho tiêu chí dừng động (dynamic): **No-U-Turn Sampler (NUTS)** (Hoffman và Gelman, 2014; Betancourt 2013b) dừng quỹ đạo khi nó bắt đầu "quay đầu" lại điểm xuất phát, chi tiết thuật toán nằm ở Phụ lục A.4-A.5 của paper.

3.5 Symplectic integrator và sửa lỗi MH (Mục 5)

Không thể giải chính xác phương trình Hamilton (trừ vài trường hợp đặc biệt), nên phải tích phân số. Bộ giải số thông thường bị ”trôi” (drift) tích luỹ lỗi theo thời gian; **symplectic integrator** (ví dụ **leapfrog**) thì bảo toàn thể tích pha đúng như hệ thật, nên quỹ đạo số chỉ dao động quanh mức năng lượng đúng, không trôi xa. Với kinetic energy không phụ thuộc vị trí, leapfrog với bước ϵ:

$$p_{n+\frac{1}{2}} \leftarrow p_n - \tfrac{\epsilon}{2}\frac{\partial V}{\partial q}(q_n), \qquad q_{n+1} \leftarrow q_n + \epsilon\, p_{n+\frac{1}{2}}, \qquad p_{n+1} \leftarrow p_{n+\frac{1}{2}} - \tfrac{\epsilon}{2}\frac{\partial V}{\partial q}(q_{n+1}).$$

Vì leapfrog có sai số nhỏ nhưng khác 0, cần một bước sửa Metropolis-Hastings để có mẫu đúng từ phân phối chính tắc. Vì quỹ đạo số là tất định và không ”nghịch” (không thể đề xuất lùi), đề xuất trần (chỉ lấy điểm cuối q_L, p_L) luôn có xác suất chấp nhận MH bằng 0 (Hình 30 của paper). Cách sửa: thêm bước lật dấu động lượng $(q,p) \to (q,-p)$ để đề xuất trở thành nghịch được, cho xác suất chấp nhận:

$$a((q_L,-p_L) \mid (q_0,p_0)) = \min\Big(1,\ \exp\big(-H(q_L,-p_L) + H(q_0,p_0)\big)\Big).$$

Vì leapfrog gần bảo toàn H, xác suất chấp nhận này suy giảm rất ít khi D tăng, khác hẳn RWM. Paper cũng trình bày cách tổng quát hoá: đề xuất đa điểm trên cả quỹ đạo (không chỉ điểm cuối) bằng lấy mẫu đa thức (multinomial) có trọng số $\propto \pi(z)$ trên quỹ đạo, xây dựng tiến dần (progressive sampling) bằng cách gộp hai nửa quỹ đạo cũ/mới theo trọng số $w_{\text{old}}/(w_{\text{old}} + w_{\text{new}})$ (Mục A.2-A.3, công thức (1)-(2) trong paper). Khi độ dài quỹ đạo là lũy thừa của 2, việc mở rộng tương đương xây cây nhị phân cân đối theo đệ quy (Hình 37-38), đây là cấu trúc được Stan dùng cho NUTS hiện đại (Phụ lục A.5).

Cuối cùng, cấu hình symplectic integrator gồm **bước** ϵ và **bậc** K (số lần tính gradient mỗi bước). Bước nhỏ hơn chính xác hơn nhưng tốn nhiều bước hơn ($L = T/\epsilon$); bậc cao chính xác hơn nhưng tốn hơn mỗi bước. Paper dùng hình học của ”Hamiltonian bị điều chỉnh” (modified/shadow Hamiltonian, mà symplectic integrator giải đúng) để suy ra cận trên/dưới giữa chi phí và xác suất chấp nhận MH trung bình, từ đó đề xuất **điều chỉnh ϵ tự động trong warm-up để đạt xác suất chấp nhận trung bình trong khoảng 0.6 đến 0.8** (Mục 5.3).

4. Thí nghiệm và kết quả chính

Đây là bài **tổng quan khái niệm (conceptual review)**, không có thí nghiệm thực nghiệm kiểu deep learning (không có dataset, không có bảng so sánh accuracy trên nhiều mô hình). ”Kết quả chính” của paper là các kết luận giải tích/định lý/ví dụ minh hoạ, cụ thể:

Kết quả	Nội dung	Vị trí trong paper
Thể tích vùng lân cận mode	$3^D - 1$ hộp lân cận trong D chiều; tỉ trọng thể tích hộp trung tâm là $1/3, 1/9, 1/27$ với $D = 1, 2, 3$	Hình 1, Mục 1.3
Thời gian tích phân tối ưu (giải tích, 1D)	$T_{\text{optimal}} \propto H^{(2-\beta)/(2\beta)}$ cho họ $\pi_\beta(q) \propto e^{-\lvert q\rvert^\beta}$	Mục 4.3
Cận chi phí / xác suất chấp nhận	Cận trên và dưới giữa chi phí tính toán và xác suất chấp nhận MH trung bình, khuyến nghị huấn luyện ϵ để đạt chấp nhận 0.6-0.8	Mục 5.3, Hình 33
E-BFMI (Bayesian fraction of missing information cho năng lượng)	Chỉ số chẩn đoán kinetic energy kém; thực nghiệm cho thấy giá trị dưới 0.3 là dấu hiệu có vấn đề	Mục 6.1

Kết quả	Nội dung	Vị trí trong paper
Phân kỳ (divergences) của symplectic integrator	Xảy ra khi quỹ đạo số bay ra vô cùng ở vùng độ cong cao; dùng như chẩn đoán trực tiếp pathology hình học	Mục 6.2, Hình 29, 35
Phạm vi geometric ergodicity	HMC đơn giản đã geometric ergodic trên lớp phân phối lớn hơn RWM (Livingstone et al. 2016)	Mục 6

Paper không chạy benchmark thực nghiệm trên dữ liệu thật trong bản review này; các khẳng định thực nghiệm (ví dụ ngưỡng E-BFMI 0.3, khoảng chấp nhận 0.6-0.8) được paper trích dẫn là quan sát thực nghiệm đã biết trong các công trình trước (Betancourt và đồng nghiệp), không phải thí nghiệm mới chạy trong bài này.

Ví dụ tính tay (tự tính): xét mục tiêu Gaussian chuẩn 1 chiều $\pi(q) = \mathcal{N}(0,1)$, $V(q) = q^2/2$, $\partial V/\partial q = q$. Lấy $p_0 \sim \mathcal{N}(0,1)$, giả sử $q_0 = 1$, $p_0 = 0$, kinetic energy Euclidean-Gaussian $K = p^2/2$. Chạy leapfrog với $\epsilon = 0.5$ một bước:

$$p_{0.5} = p_0 - \tfrac{0.5}{2} \cdot q_0 = 0 - 0.25 \cdot 1 = -0.25,$$
$$q_1 = q_0 + 0.5 \cdot p_{0.5} = 1 + 0.5 \cdot (-0.25) = 0.875,$$
$$p_1 = p_{0.5} - \tfrac{0.5}{2} \cdot q_1 = -0.25 - 0.25 \cdot 0.875 = -0.46875.$$

Năng lượng ban đầu $H_0 = V(q_0) + K(p_0) = 0.5 + 0 = 0.5$. Năng lượng sau 1 bước $H_1 = V(q_1) + K(p_1) = 0.875^2/2 + (-0.46875)^2/2 \approx 0.3828 + 0.1099 = 0.4927$ (tự tính). Sai số $|H_1 - H_0| \approx 0.0073$, rất nhỏ so với $\epsilon = 0.5$, minh hoạ tính chất "dao động quanh mức năng lượng đúng" của symplectic integrator (không trôi xa như integrator thường) đúng như Hình 27-28 mô tả định tính.

5. Phân tích / ablation

Vì đây là bài lý thuyết, "ablation" ở đây là các so sánh định tính paper tự trình bày giữa các lựa chọn thiết kế:

- **RWM so với HMC (Mục 2.4 so với Mục 3):** RWM dùng đề xuất Gaussian không thông tin, xác suất chấp nhận suy biến theo cấp lũy thừa với D vì gần như mọi đề xuất rơi ra ngoài typical set hoặc phải co cực nhỏ. HMC dùng gradient qua động lượng, xác suất chấp nhận chỉ suy giảm nhẹ với D vì bản chất bảo toàn thể tích pha của symplectic integrator.
- **Euclidean so với Riemannian kinetic energy (Mục 4.2):** Euclidean chỉ sửa được tương quan toàn cục (một ma trận hiệp phương sai cố định); Riemannian sửa cục bộ theo vị trí, cần thiết cho các mô hình phân cấp (hierarchical model) nơi độ cong biến đổi mạnh theo vùng (ví dụ "funnel" của Neal), nhưng tốn chi phí tính Hessian/metric mỗi bước.
- **Thời gian tích phân tĩnh so với động (NUTS) (Mục 4.3):** tĩnh chỉ tối ưu cho một mức năng lượng; đuôi nặng ($\beta < 2$) cần T lớn hơn ở vùng đuôi nên một T cố định sẽ làm chậm toàn chuỗi. NUTS thích ứng độ dài quỹ đạo theo từng điểm khởi đầu.
- **Lấy mẫu đều (uniform) so với lệch (biased) progressive sampling (Phụ lục A.3.2):** biến thể lệch ưu tiên lấy mẫu từ nửa quỹ đạo mới, tạo "phản tương quan" (anti-correlation) giữa các mẫu liên tiếp của xích, có thể cải thiện hiệu năng nhưng paper không đưa ra số liệu định lượng cụ thể trong phần chính (chỉ chứng minh tính đúng đắn, phân phối dừng được bảo toàn).
- **Chẩn đoán kép (E-BFMI và divergences):** cả hai là điều kiện *cần* nhưng không *đủ* cho geometric ergodicity (Mục 6.3); paper nói rõ không có đảm bảo hình thức loại trừ hết các pathology tinh vi hơn.

6. Giới hạn

Do tác giả nêu:

- Không có lý thuyết hình thức toàn diện cho geometric ergodicity của HMC; các kết quả (Livingstone et al. 2016) chỉ là ”preliminary results” trên một lớp phân phối, chưa phải tổng quát.
- E-BFMI và divergence chỉ là điều kiện cần, không đủ (Mục 6.3): một phân phối có thể có pathology không bị hai chẩn đoán này phát hiện.
- SoftAbs metric cho Riemannian-HMC là ”heuristic regularization” (Mục 4.2.2), chưa có lý thuyết hình thức hoá đầy đủ.
- Higher-order symplectic integrator có thêm bậc tự do có thể làm hỏng hiệu năng nếu chọn sai, và việc tinh chỉnh các bậc tự do này vẫn là hướng nghiên cứu mở (Mục 5.3).
- Non-Gaussian kinetic energy ”không có nhiều lý thuyết hỗ trợ” và thực nghiệm cho thấy chúng thường kém hơn, nhưng paper không đưa số liệu cụ thể trong bài (chỉ dẫn trực giác qua CLT).

Do chúng ta nhận thấy:

- Bài không có bảng số liệu định lượng nào từ thực nghiệm chạy trong bài; mọi ”kết quả thực nghiệm” (ngưỡng 0.3, khoảng 0.6-0.8) là trích dẫn công trình khác, không tự kiểm chứng lại trong văn bản này.
- Ví dụ giải tích 1D (họ π_β) minh họa trực giác tốt nhưng không tổng quát hoá trực tiếp sang không gian nhiều chiều với tương tác phức tạp giữa các toạ độ.
- Phần kỹ thuật NUTS đầy đủ (bao gồm cả xử lý khi quỹ đạo phân kỳ giữa cây nhị phân) nằm ở Phụ lục A.4-A.5 với độ phức tạp cao; bài báo tự nhận ”a more thorough discussion of the technical details is available in Appendix A” cho thấy phần thân bài chỉ là giản lược khái niệm, Stan thực tế dùng phiên bản phức tạp hơn (xử lý đa chiều khác biệt, điều kiện dừng theo nhiều hướng).

7. Liên hệ với các bài khác

- **math 06** (New insights and perspectives on the natural gradient method, 1412.1193): cùng dùng khái niệm Riemannian metric để sửa hình học không gian tham số; natural gradient dùng Fisher information metric cho bước tối ưu hoá, còn Riemannian-HMC (Mục 4.2.2 của bài này) dùng metric tương tự (có thể là Fisher metric) cho kinetic energy khi lấy mẫu.
- **math 11** (Stochastic Gradient Descent as Approximate Bayesian Inference, 1704.04289): bài tiếp theo trong lộ trình, dùng SGD như một xấp xỉ liên tục (continuous-time, Langevin-type dynamics tương tự Hamiltonian dynamics nhưng có nhiễu và ma sát) để suy luận Bayes gần đúng mà không cần tích phân Hamilton chính xác như HMC.
- **math 35** (A Tutorial on Thompson Sampling, 1707.02038) đã kiểm tra tồn tại qua ls (file sẽ nằm cùng thư mục math, số theo titles.tsv): cùng nằm trong nhóm phương pháp Bayes thực hành (ở đây là bandit), tuy ứng dụng khác HMC nhưng cùng triết lý ”suy luận hậu nghiệm làm nền cho quyết định”, đáng đọc nếu quan tâm thêm về suy luận Bayes ứng dụng.
- `ls "/home/duy/Downloads/duy_dev/research_/rl content/"` xác nhận tồn tại **rl 12** (`12_2018_sac.md`, Soft Actor-Critic): SAC dùng entropy như một dạng ”nhiệt độ” tương tự cách HMC/Bayes xử lý độ bất định, dù mục tiêu khác (RL so với suy luận hậu nghiệm).
- `ls "/home/duy/Downloads/duy_dev/research_/interp content/"` xác nhận tồn tại **interp 01** (`01_2022_mech_interp_essay.md`): không liên hệ kỹ thuật trực tiếp, nhưng cả hai đều nhấn mạnh việc ”tin tưởng có điều kiện” vào một công cụ dựa trên chẩn đoán thất bại rõ ràng (ở HMC là divergences/E-BFMI, ở interpretability là circuit analysis), một điểm chung về triết lý khoa học.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM đọc OCR, kiến trúc hybrid Gated-DeltaNet + full attention, batch size 1, GPU $\leq$ 8 GB). HMC bản thân là công cụ suy luận Bayes đầy đủ (lấy

mẫu hậu nghiệm tham số), **rất ít được dùng trực tiếp để huấn luyện mạng nơ-ron cỡ lớn** vì chi phí một bước HMC (nhiều lần tính gradient cho một quỹ đạo) quá cao so với một bước SGD/Adam. Vì vậy phần lớn kỹ thuật chi tiết của bài này (tuning kinetic energy, NUTS, symplectic integrator order) **ít liên quan trực tiếp** đến việc train/nén model OCR của dự án.

Tuy nhiên có 2 ý tưởng khái niệm có thể dùng gián tiếp (suy luận):

1. **Chẩn đoán bằng "năng lượng"/độ phân kỳ số:** ý tưởng dùng một đại lượng vô hướng dễ tính (năng lượng Hamiltonian, hoặc ở đây có thể là loss/logit-norm) để phát hiện vùng "bệnh lý" của không gian tham số (ở HMC là divergent transitions) có thể truyền cảm hứng cho một chẩn đoán tương tự khi fine-tune hoặc cắt tỉa model: theo dõi độ lớn gradient hoặc norm activation bất thường tăng vọt ở một layer cụ thể có thể là dấu hiệu layer đó "nhạy" (tương tự vùng độ cong cao trong HMC) và dễ gãy khi lượng tử hoá hoặc cắt bớt.
2. **Ước lượng độ bất định (uncertainty) của model nhỏ sau khi cắt/distill:** nếu muốn ước lượng độ tin cậy của model student (ví dụ cho các trang OCR khó đọc), về lý thuyết có thể dùng HMC/các biến thể xấp xỉ của nó để lấy mẫu hậu nghiệm trên một tập con tham số nhỏ (ví dụ chỉ các lớp LoRA hoặc đầu ra cuối) rồi đo phương sai dự đoán, nhưng với model 0.8B tham số đầy đủ, chi phí HMC thực tế là không khả thi trên GPU 8 GB; cách thực dụng hơn (không thuộc phạm vi bài này) là Dropout as Bayesian Approximation hoặc Deep Ensembles.

Nói thẳng: **phần lõi kỹ thuật của bài này (tuning HMC) ít liên quan trực tiếp** tới huấn luyện/nén LightOnOCR; giá trị chính là nền tảng khái niệm (typical set, phase space, symplectic integrator) giúp hiểu các bài liên quan khác (ví dụ math 11 về SGD-as-Bayes, có liên quan gần hơn tới optimizer thực dụng).

9. Thí nghiệm gợi ý

1. **So sánh RWM và HMC trên Gaussian 2 chiều tương quan cao:** tạo $\pi(q) = \mathcal{N}(0, \Sigma)$ với Σ có tương quan 0.95 giữa 2 chiều; chạy RWM (đề xuất Gaussian đẳng hướng) và HMC leapfrog thủ công (vài chục bước, $L = 10$-20, ϵ nhỏ) bằng NumPy/PyTorch trên CPU. Đo effective sample size (ESS) trên cùng số lần tính hàm mục tiêu. Kỳ vọng: HMC có ESS/evaluation cao hơn RWM rõ rệt khi tương quan lớn, minh họa trực tiếp Mục 2.4 và 3.
2. **Đo xác suất chấp nhận MH theo step size ϵ:** với cùng target Gaussian chuẩn, chạy leapfrog $L = 20$ bước với nhiều $\epsilon \in \{0.01, 0.05, 0.1, 0.3, 0.5, 1.0\}$, vẽ xác suất chấp nhận trung bình theo ϵ. Kỳ vọng: xác suất chấp nhận giảm đơn điệu khi ϵ tăng; xác định ϵ cho chấp nhận quanh 0.65 để kiểm chứng khuyến nghị 0.6-0.8 ở Mục 5.3.
3. **Tạo "Neal's funnel" (phân phối phân cấp điển hình) và quan sát divergence:** $v \sim \mathcal{N}(0, 3^2)$, $q \mid v \sim \mathcal{N}(0, e^v)$ (tham số hoá "centered"); chạy HMC/leapfrog đơn giản, log số bước phân kỳ (năng lượng bùng nổ) khi v nhỏ (vùng cổ hẹp của funnel). Kỳ vọng: số divergence tăng mạnh ở vùng cổ, minh hoạ trực tiếp Mục 6.2 và gợi ý kỹ thuật "non-centered parameterization" paper nhắc tới.
4. **Tính E-BFMI tự chế trên một xích HMC toy:** log năng lượng $E_n = H(q_n, p_n)$ qua các bước, tính $\widehat{\text{E-BFMI}} = \sum(E_n - E_{n-1})^2 / \sum(E_n - \bar{E})^2$ theo công thức Mục 6.1; thử với kinetic energy "đúng" (mass matrix = hiệp phương sai target) và "sai" (mass matrix = identity khi target có tương quan mạnh). Kỳ vọng: cấu hình sai cho E-BFMI thấp hơn (gần hoặc dưới 0.3), khớp ngưỡng paper nêu.
5. **Minh hoạ ba giai đoạn của xích Markov (Hình 7):** chạy một xích RWM từ điểm khởi tạo xa mode, vẽ $|\hat{f}_N - \mathbb{E}_\pi[f]|$ theo số bước; quan sát pha hội tụ ban đầu (bias lớn), pha "sojourn" đầu tiên (bias giảm nhanh), và pha tinh chỉnh chậm. Kỳ vọng: đường cong có 3 đoạn dốc khác nhau rõ rệt, minh họa trực tiếp Mục 2.2.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Kỳ vọng mục tiêu	$\mathbb{E}_\pi[f] = \int_Q dq\, \pi(q) f(q)$
Hiệu quả xích Markov	$\text{ESS} = N/\left(1 + 2\sum_{l\geq 1} \rho_l\right)$
Metropolis-Hastings	$a(q' \mid q) = \min\left(1, \frac{Q(q\mid q')\pi(q')}{Q(q'\mid q)\pi(q)}\right)$
Phân phối chính tắc	$\pi(q,p) = e^{-H(q,p)}$, $H = K(p,q) + V(q)$
Phương trình Hamilton	$\dot{q} = \partial K/\partial p$, $\dot{p} = -\partial K/\partial q - \partial V/\partial q$
Kinetic energy Euclidean-Gaussian	$K = \frac{1}{2} p^\intercal M^{-1} p + \log\lvert M\rvert$, $M^{-1} \approx \text{Cov}_\pi[q]$
Kinetic energy Riemannian-Gaussian	$K = \frac{1}{2} p^\intercal \Sigma^{-1}(q) p + \frac{1}{2}\log\lvert\Sigma(q)\rvert$
Thời gian tích phân tối ưu (1D, $\pi_\beta \propto e^{-\lvert q\rvert^\beta}$)	$T_{\text{optimal}} \propto H^{(2-\beta)/(2\beta)}$
Leapfrog	$p_{+\frac{\epsilon}{2}} = p - \frac{\epsilon}{2}\nabla V(q)$; $q' = q + \epsilon p_{+\frac{\epsilon}{2}}$; $p' = p_{+\frac{\epsilon}{2}} - \frac{\epsilon}{2}\nabla V(q')$
MH sửa lỗi integrator (momentum flip)	$a = \min\left(1, \exp(-H(q_L, -p_L) + H(q_0, p_0))\right)$
Xác suất chấp nhận tối ưu khuyến nghị	khoảng 0.6 đến 0.8
E-BFMI	$\widehat{\text{E-BFMI}} = \sum_n (E_n - E_{n-1})^2 / \sum_n (E_n - \bar{E})^2$; ngưỡng cảnh báo thực nghiệm < 0.3

11. Kết luận ngắn

Bài này xây dựng trực giác hình học cho Hamiltonian Monte Carlo mà không cần vi phân hình học chính thức: không gian nhiều chiều khiến khối lượng xác suất tập trung vào một ”typical set” mỏng và xa mode; các phương pháp ”đoán-kiểm tra” như Random Walk Metropolis thất bại vì không khai thác được hình học này, còn HMC tận dụng gradient qua một hệ động lực học Hamiltonian (động lượng phụ trợ, bảo toàn thể tích pha) để tạo quỹ đạo bám theo typical set. Phần còn lại của bài là cách chỉnh các bậc tự do thực tế (kinetic energy, thời gian tích phân/NUTS, symplectic integrator) và cách chẩn đoán khi các giả định hình học bị vi phạm (E-BFMI, divergent transitions). Giá trị chính cho người đọc không chuyên Bayes là bộ khung khái niệm (typical set, phase space, năng lượng) hữu ích khi đọc các bài liên quan về suy luận Bayes gần đúng (math 11) và các phương pháp tối ưu dùng metric hình học (math 06).

Nguồn gốc: https://arxiv.org/abs/1701.02434

Bài 11

Stochastic Gradient Descent as Approximate Bayesian Inference (2017)

Tác giả: Stephan Mandt (Data Science Institute, Columbia University), Matthew D. Hoffman (Adobe Research), David M. Blei (Columbia University).
Venue / arXiv: arXiv:1704.04289 (v2, 19/01/2018); paper ghi rõ venue là Journal of Machine Learning Research (JMLR) 18 (2017) 1-35, nộp 4/2017, sửa 10/2017, xuất bản 12/2017.
Nguồn: https://arxiv.org/abs/1704.04289
Chủ đề: xem SGD với learning rate hằng số (constant SGD) như một quá trình ngẫu nhiên liên tục có phân phối dừng Gaussian, từ đó chỉnh learning rate/minibatch/preconditioner để phân phối dừng này xấp xỉ hậu nghiệm Bayes; phân tích thêm momentum, SGLD, SGFS, và Polyak iterate averaging như một bộ lấy mẫu MCMC.
Vị trí trong lộ trình: đọc sau math 10 (A Conceptual Introduction to Hamiltonian Monte Carlo), vì cả hai cùng dùng ngôn ngữ ”động lực học ngẫu nhiên” (Hamiltonian/Langevin) để suy luận Bayes gần đúng, nhưng bài 10 tích phân chính xác một hệ Hamiltonian còn bài này xấp xỉ SGD rời rạc bằng một phương trình vi phân ngẫu nhiên (SDE) liên tục; đọc trước math 12 (Computational Optimal Transport) vì sau khi nắm các công cụ suy luận xấp xỉ (sampling, biến phân), phần tiếp theo của lộ trình chuyển sang đo khoảng cách giữa phân phối (OT, Wasserstein). Cũng liên quan math 08 (Variational Inference review) vì bài này tự nhận là ”một dạng variational inference” theo nghĩa rộng.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

SGD với learning rate giảm dần (theo Robbins-Monro) hội tụ tới một điểm tối ưu (cục bộ) của hàm mất mát. Nhưng nếu giữ learning rate **hằng số** (”constant SGD”), thuật toán tiến tới gần optimum rồi **”nảy quanh”** vùng lân cận mãi mãi, không bao giờ dừng hẳn. Paper quan sát: hành vi ”nảy quanh” này chính là một **quá trình Markov có phân phối dừng (stationary distribution)**, và ý tưởng trung tâm của bài là: nếu chỉnh đúng các siêu tham số của constant SGD (learning rate ϵ, kích thước minibatch S, ma trận tiền điều kiện/preconditioner H), phân phối dừng đó có thể được làm cho **gần với hậu nghiệm Bayes** $p(\theta \mid x) = \exp\{\log p(\theta, x) - \log p(x)\}$ (công thức (1) trong paper).

Đây khác với các phương pháp **stochastic gradient MCMC (SG-MCMC)** đã có trước đó như SGLD (Stochastic Gradient Langevin Dynamics, Welling và Teh 2011), SG-HMC (Chen et al. 2014), SG thermostats (Ding et al. 2014), SGFS (Stochastic Gradient Fisher Scoring, Ahn et al. 2012): các phương pháp này cố tình thêm nhiễu để **tiệm cận chính xác** hậu nghiệm (nhưng mixing chậm khi learning rate phải giảm về 0). Bài báo đi theo hướng ngược lại: chấp nhận một xấp xỉ **không chính xác tuyệt đối** nhưng **mixing nhanh hơn**, bằng cách chạy SGD thông thường (không cần thêm nhiễu đặc biệt) với learning rate hằng số đã được tinh chỉnh.

Công cụ toán học chính: xấp xỉ quá trình rời rạc của SGD bằng một **phương trình vi phân ngẫu nhiên (SDE)** liên tục thời gian, cụ thể là **quá trình Ornstein-Uhlenbeck (OU)** đa biến, một quá trình ngẫu nhiên có nghiệm dừng Gaussian giải tích được. Công cụ thứ hai: **variational inference** theo nghĩa tối thiểu hoá Kullback-Leibler (KL) divergence giữa phân phối dừng (vai trò q) và hậu nghiệm (vai trò f hoặc p); khác biệt so với VI thông thường là ở đây ta không tối ưu tham số của q trực tiếp, mà tối ưu **siêu tham số của thuật toán tối ưu hoá** (learning rate, preconditioner) để định hình q.

2. Ý tưởng chính

Với hàm mất mát $L(\theta) = \frac{1}{N}\sum_{n=1}^{N} \ell_n(\theta)$ (ví dụ $\ell_n(\theta) = -\log p(x_n \mid \theta) - \frac{1}{N}\log p(\theta)$, công thức (3), chính là MAP loss), SGD cập nhật $\theta(t+1) = \theta(t) - \epsilon\, \hat{g}_S(\theta(t))$ với gradient ước lượng trên minibatch S mẫu. Bốn giả định (Assumption 1-4, Mục 3.2) biến quá trình rời rạc này thành một SDE:

1. **A1:** theo định lý giới hạn trung tâm, nhiễu gradient minibatch là Gaussian: $\hat{g}_S(\theta) \approx g(\theta) + \frac{1}{\sqrt{S}}\Delta g(\theta)$, $\Delta g(\theta) \sim \mathcal{N}(0, C(\theta))$.
2. **A2:** $C(\theta) \approx C = BB^{\mathsf{T}}$ (hằng số, không đổi theo θ trong vùng quanh optimum).
3. **A3:** xấp xỉ sai phân $\Delta\theta(t) = \theta(t+1) - \theta(t)$ bằng SDE liên tục $d\theta = -g(\theta)dt + \frac{\epsilon}{\sqrt{S}} B\, dW(t)$.
4. **A4:** gần optimum, loss là hàm bậc hai $L(\theta) = \frac{1}{2}\theta^{\mathsf{T}} A\theta$ (A là Hessian tại optimum, giả sử optimum ở $\theta = 0$).

Bốn giả định này biến SGD thành một **quá trình Ornstein-Uhlenbeck đa biến**: $d\theta(t) = -A\theta(t)\,dt + \frac{1}{\sqrt{S}} B\, dW(t)$ (công thức (11)), có phân phối dừng Gaussian $q(\theta) \propto \exp(-\frac{1}{2}\theta^{\mathsf{T}}\Sigma^{-1}\theta)$ với hiệp phương sai Σ nghiệm của phương trình Lyapunov $\Sigma A + A\Sigma = \frac{\epsilon}{S} BB^{\mathsf{T}}$ (công thức (13)). Ý tưởng chính: Σ **tỉ lệ thuận với** ϵ **và tỉ lệ nghịch với** A**,** S, nên ta có thể "vặn" ϵ, S, hoặc thêm ma trận tiền điều kiện H để Σ khớp với hiệp phương sai hậu nghiệm thật $N^{-1}A^{-1}$ (xấp xỉ Laplace/Bernstein-von Mises của hậu nghiệm, công thức (14)). Việc khớp này được thực hiện bằng cách **tối thiểu hoá KL**$(q\|f)$ giải tích (vì cả hai đều Gaussian).

3. Phương pháp chi tiết

3.1 Learning rate và preconditioner tối ưu (Mục 4.1)

Định lý 1 (constant SGD, công thức (15)): learning rate hằng số tối thiểu hoá $\mathrm{KL}(q\|f)$ là

$$\epsilon^* = \frac{2\,S\,D}{N\,\mathrm{Tr}(BB^{\mathsf{T}})},$$

với D là số chiều của θ. Chứng minh (paper, Mục 4.1): $\mathrm{KL}(q\|f) = \frac{c\,\epsilon N}{2S}\mathrm{Tr}(BB^{\mathsf{T}}) - D\log(\epsilon/S)$ (công thức (16)); lấy đạo hàm theo ϵ/S và giải cho 0 ra công thức (15). Diễn giải: learning rate nên **tỉ lệ nghịch với độ lớn trung bình nhiễu gradient** ($\mathrm{Tr}(BB^{\mathsf{T}})$) và **tỉ lệ thuận với** S/N.

Định lý 2 (preconditioned constant SGD, công thức (17)): nếu dùng cập nhật $\theta_{t+1} = \theta_t - H\hat{g}_S(\theta_t)$ với ma trận H xác định dương thay cho ϵ vô hướng, preconditioner tối ưu là

$$H^* = \frac{2S}{N}(BB^{\mathsf{T}})^{-1}.$$

Hệ quả 3: nếu hạn chế H chéo (diagonal, cần thiết khi D lớn vì H đầy đủ tốn $O(D^2)$), preconditioner chéo tối ưu là $H^*_{kk} = \dfrac{2S}{N\,(BB^{\mathsf{T}})_{kk}}$.

Hệ quả 4: nếu dùng đúng H^* đầy đủ (công thức (17)), phân phối dừng trở thành **chính xác** hậu nghiệm ($\Sigma = A^{-1}/N$), không chỉ là xấp xỉ KL-tối ưu.

Để áp dụng thực tế cần ước lượng $C = BB^{\mathsf{T}}$ trực tuyến (online). Paper dùng ước lượng dựa trên hiệu giữa gradient của 1 mẫu và gradient minibatch đầy đủ, cập nhật đệ quy (công thức (21)):

$$C_t = (1-\kappa_t)\,C_{t-1} + \kappa_t\,(\hat{g}_{1,t} - \hat{g}_{S,t})(\hat{g}_{1,t} - \hat{g}_{S,t})^{\mathsf{T}},$$

với κ_t giảm dần ($\kappa_t \sim 1/t$). Chi phí: $O(D)$ cho xấp xỉ chéo (tương đương AdaGrad/RMSProp), $O(D^2)$ cho preconditioner đầy đủ (paper nhận xét "impractical in many real-word setups" cho D lớn).

3.2 Variational EM cho siêu tham số (Mục 4.2)

Với mô hình có siêu tham số λ (ví dụ hệ số regularization), paper đề xuất chạy **đồng thời hai learning rate**: $\theta_{t+1} = \theta_t - \epsilon^* \nabla_\theta L(\theta_t, \lambda_t)$ (hằng số, để θ lấy mẫu từ hậu nghiệm xấp xỉ) và $\lambda_{t+1} = \lambda_t - \rho_t \nabla_\lambda L(\theta_t, \lambda_t)$ (giảm dần, để λ hội tụ về một điểm tối ưu), công thức (22). Đây là một dạng **variational EM (VEM)**: bước E (lấy mẫu θ từ phân phối dừng của constant SGD, đóng vai trò $q(\theta)$) và bước M (tối ưu λ theo gradient của log-joint kỳ vọng) diễn ra đan xen trong cùng một vòng lặp SGD kép, không cần cross-validation.

3.3 SGD với Momentum (Mục 4.3)

Thêm biến động lượng v: $v(t+1) = (1-\mu)v(t) - \epsilon\, \hat{g}_S(\theta(t))$, $\theta(t+1) = \theta(t) + v(t+1)$, với hệ số hãm (damping) $\mu \in [0, 1]$ ($\mu = 1$ là SGD thường, không có "quán tính"). Dùng cùng 4 giả định ở trên cho hệ 2 biến (θ, v), giải hệ phương trình mô-men bậc 2 (chi tiết Phụ lục D), paper thu được quan hệ năng lượng kiểu định lý "fluctuation-dissipation" (công thức (24)):

$$\mathbb{E}[vv^\mathsf{T}] = \tfrac{\epsilon}{2}\mathbb{E}[\theta\theta^\mathsf{T}]A + \tfrac{\epsilon}{2}A\,\mathbb{E}[\theta\theta^\mathsf{T}], \qquad \mu\,\mathbb{E}[vv^\mathsf{T}] = \tfrac{\epsilon^2}{2S}BB^\mathsf{T}.$$

Kết hợp lại cho (công thức (25)):

$$\Sigma A + A\Sigma = \frac{\epsilon}{\mu S} BB^\mathsf{T},$$

giống hệt công thức (13) của SGD không momentum, chỉ thay ϵ/S bằng $\epsilon/(\mu S)$. Hệ quả quan trọng: **momentum không đổi hình dạng của phân phối dừng, chỉ đổi tỉ lệ (scale)** theo hệ số $1/\mu$; chỉ tổ hợp $\epsilon/(\mu S)$ mới ảnh hưởng đến KL divergence, nên "không có một learning rate tối ưu duy nhất" khi có momentum, mà có cả một họ (ϵ, μ, S) cho cùng kết quả xấp xỉ (nhưng tốc độ hội tụ động lực học thì khác nhau).

3.4 SGLD và SGFS với learning rate không tiệm cận 0 (Mục 5)

SGLD (Welling-Teh 2011) thêm nhiễu Wiener $V(t)$ vào update: $\theta_{t+1} = \theta_t - \frac{\epsilon}{2}N\hat{\nabla}L(\theta_t) + \sqrt{\epsilon}\,V(t)$. Phân tích OU cho phương trình dừng: $\frac{1}{2}\epsilon N(A\Sigma + \Sigma A) = I + \frac{\epsilon N}{2S}BB^\mathsf{T}$. Khi $\epsilon \to 0$: $\Sigma^{-1} = NA$ đúng hậu nghiệm. Nhưng với ϵ hữu hạn (như dùng trong thực tế), các hệ số hiệu chỉnh dương khiến **SGLD hệ thống hoá đánh giá quá cao (overestimate) hiệp phương sai hậu nghiệm**, hệ quả của learning rate không tiệm cận 0.

SGFS (Ahn et al. 2012) dùng update $\theta_{t+1} = \theta_t - H\hat{g}(\theta_t) + \sqrt{H}EW(t)$ với H là preconditioner và EE^T là hiệp phương sai nhiễu nhân tạo. **Định lý 5 (công thức (27)):** $H^* = \frac{2}{N}(BB^\mathsf{T} + EE^\mathsf{T})^{-1}$ tối thiểu hoá KL **và đưa KL về đúng 0** (nghĩa là, nếu biết chính xác BB^T, SGFS với H^* tạo mẫu đúng từ hậu nghiệm, không chỉ là xấp xỉ). Paper cũng đưa ra bản chéo/vô hướng (Hệ quả 6) và một thủ thuật ổn định số: chọn nhiễu nhân tạo E sao cho $H_{kk} \equiv h_{\max}$ cho mọi k (tiêm nhiễu Gaussian vào các chiều có gradient-noise quá nhỏ để tránh learning rate hiệu dụng quá lớn ở chiều đó); paper nhấn mạnh thủ thuật này **ngược lại** khuyến nghị gốc của Ahn et al. (2012) là chọn B tỉ lệ với E.

3.5 Polyak iterate averaging như một bộ lấy mẫu (Mục 6)

Tối ưu hoá: trung bình các iterate $\hat{\mu}_{t+1} = \frac{t}{t+1}\hat{\mu}_t + \frac{1}{t+1}\theta_{t+1}$ (công thức (28)) đạt tốc độ hội tụ **tốt nhất có thể trong số các thuật toán chỉ dùng stochastic-gradient oracle** (kết quả gốc của Polyak và Juditsky 1992); paper chứng minh lại ngắn hơn bằng công cụ OU, cho hiệp phương sai của ước lượng $D \approx \frac{1}{TS}A^{-1}BB^\mathsf{T}(A^{-1})^\mathsf{T}$ (công thức được paper rút gọn), **không phụ thuộc learning rate hay preconditioner** (vì preconditioning thay đổi cả phân phối dừng và ma trận tự tương quan theo cách triệt tiêu lẫn nhau).

Lấy mẫu: với cửa sổ trung bình $T = N/S$ (một epoch), dưới giả định bổ sung **A5** (định lý Bernstein-von Mises áp dụng với N lớn, và dữ liệu sinh đúng từ model với $\theta^* = 0$, nên $A = BB^\mathsf{T}$ chính là ma trận thông tin Fisher), paper suy ra thuật toán **Iterate Averaging Stochastic Gradient sampler (IASG, Algorithm 1**

trong paper): chạy SGD learning rate hằng, cứ mỗi $T = N/S$ bước thì tính trung bình T iterate gần nhất thành một mẫu hậu nghiệm. Kết quả lý thuyết nổi bật: **IASG cần đúng khoảng N lượt gọi gradient (một lượt qua hết dữ liệu) để sinh một mẫu hậu nghiệm xấp xỉ "hiệu dụng độc lập"**, và vì iterate averaging đạt chặn Cramér-Rao tối ưu, **không thuật toán SG-MCMC "hộp đen" nào có thể sinh mẫu hậu nghiệm nhanh hơn $O(ND)$ (thời gian tuyến tính theo kích thước dữ liệu nhân số chiều)**, trừ khi khai thác thêm thông tin về nguồn nhiễu (như SVRG, SAGA, được paper dẫn ra ở chú thích).

4. Thí nghiệm và kết quả chính

Đây là bài có **thí nghiệm thực nghiệm thật** (không phải thuần lý thuyết), chia 3 phần (Mục 7):

7.1 Kiểm tra hiệp phương sai dừng trên dữ liệu thật. Ba tập dữ liệu UCI: Wine Quality ($N = 4\,898$, 11 đặc trưng, hồi quy tuyến tính), Protein Tertiary Structure ($N = 45\,730$, 8 đặc trưng, hồi quy tuyến tính), Skin Segmentation ($N = 245\,057$, 3 đặc trưng, hồi quy logistic). Minibatch $S = 100, 100, 10\,000$ theo thứ tự; regularizer bậc 2 bằng 1; learning rate theo công thức (15). Paper báo cáo hình ảnh (Hình 1-2) cho thấy hiệp phương sai thực nghiệm khớp tốt với dự đoán từ OU; sai khác đáng kể nhất ở Wine (do tập nhỏ hơn nên hậu nghiệm rộng hơn, cần learning rate lớn hơn, hiệu ứng rời rạc hoá lớn hơn).

Bảng 1 (KL divergence giữa hậu nghiệm xấp xỉ bằng Laplace và phân phối dừng của từng thuật toán, cột là 3 tập dữ liệu Wine/Skin/Protein):

Phương pháp	Wine	Skin	Protein
constant SGD	18.7	0.471	1000.9
constant SGD-d (chéo)	14.0	0.921	678.4
constant SGD-f (đầy đủ)	**0.7**	**0.005**	**1.8**
SGLD (Welling-Teh)	2.9	0.905	4.5
SGFS-d	12.8	0.864	597.4
SGFS-f	0.8	**0.005**	1.3
BBVI	44.7	5.74	478.1

Nhận xét của paper: preconditioner **đầy đủ** (SGD-f, SGFS-f) áp sát hậu nghiệm tốt nhất (KL nhỏ nhất); phiên bản chéo (hay dùng trong thực hành vì rẻ) kém hơn hẳn; **SGD hằng số chỉnh đúng có thể cạnh tranh với các thuật toán MCMC chuyên dụng** (SGLD, SGFS) và tốt hơn BBVI (black-box VI dùng reparameterization gradient) trên cả 3 tập; paper kết luận yếu tố quan trọng nhất không phải nhiễu nhân tạo thêm vào mà là **chất lượng xấp xỉ ma trận tiền điều kiện**.

7.2 Iterate averaging như MCMC. Dữ liệu tổng hợp đúng theo model: $N = 10\,000$ covariates $D = 10$ chiều từ Gaussian, prior precision $\lambda = 1$. Thí nghiệm xác nhận (Hình 3): chạy constant SGD $\epsilon = 0.005$, $10 \cdot D \cdot N = 10^6$ bước, $S = 1$, cửa sổ trung bình $N/S = 10^4$; hiệp phương sai của iterate đã trung bình hoá **giống hệt** hiệp phương sai hậu nghiệm thật. So sánh tốc độ hội tụ (Hình 4-5) giữa IASG, SGLD, và NUTS (trên cùng bài toán hồi quy tuyến tính): $S = 10$, cửa sổ $N/S = 1000$, $\epsilon_{\text{IASG}} = 0.003$; SGLD dùng lịch Robbins-Monro $\epsilon_t = \epsilon_0/\sqrt{1000 + t}$ với $\epsilon_0 = 10^{-3}$; NUTS tự động chỉnh bước và dùng gradient đầy đủ (không stochastic). Kết quả: khi khởi tạo ngẫu nhiên (không phải tại MAP), **IASG hội tụ nhanh hơn và ổn định hơn SGLD và NUTS** trong việc ước lượng phương sai biên lớn nhất/nhỏ nhất của hậu nghiệm; SGLD có độ lệch chuẩn ước lượng lớn hơn hẳn do learning rate giảm dần làm chậm cân bằng ở các bước sau; khi khởi tạo tại MAP (loại bỏ pha hội tụ ban đầu) cả 3 thuật toán đều hoạt động hợp lý, nhưng IASG vẫn ổn định nhanh nhất, tuy có chút lệch (biased) ở phương sai nhỏ nhất.

7.3 Tối ưu siêu tham số. Mô hình hồi quy logistic đa lớp (softmax) với prior Gaussian, hệ số regularization λ (công thức (35)). Dữ liệu thật: **MNIST** (60.000 huấn luyện, 10.000 kiểm tra, 784 đặc trưng) và **Covertype**

(500.000 huấn luyện, 81.012 kiểm tra, 54 đặc trưng). Hình 6: vẽ validation loss theo λ (đường cong liên tục), đánh dấu điểm λ mà VEM dựa trên constant-SGD (Mục 4.2) chọn, so với điểm λ mà BBVI chọn. Kết quả: hai phương pháp chọn λ **gần giống nhau** và đều gần đáy đường cong validation loss; paper kết luận constant SGD-VEM là **giải pháp rẻ hơn cross-validation** để chọn siêu tham số trong mô hình xác suất phức tạp.

5. Phân tích / ablation

- **Preconditioner đầy đủ so với chéo (Bảng 1):** chéo luôn kém hơn đầy đủ một khoảng đáng kể (ví dụ Protein: 678.4 so với 1.8 cho SGD), xác nhận trực giác lý thuyết ở Hệ quả 3/4: chỉ preconditioner đầy đủ $\propto (BB^{\mathsf{T}})^{-1}$ mới cho mẫu đúng hậu nghiệm; bản chéo là một xấp xỉ có hệ thống (biased).
- **SGD hằng số so với SGLD/SGFS/BBVI:** với preconditioning đúng, constant SGD **không hề kém hơn** các thuật toán MCMC chuyên dụng, dù không chủ đích ”được thiết kế để là một sampler”. Điểm nhấn của paper: sự khác biệt chính giữa các phương pháp không phải là ”có thêm nhiễu Bayes hay không” mà là **chất lượng xấp xỉ ma trận hiệp phương sai nhiễu gradient**.
- **Momentum:** không đổi hình dạng phân phối dừng, chỉ đổi tỉ lệ theo $1/\mu$; về mặt lấy mẫu Bayes, dùng momentum tương đương dùng một constant SGD ”hiệu dụng” với learning rate ϵ/μ, nhưng paper trích Sutskever et al. (2013) rằng μ nhỏ hơn có thể giúp **ổn định số học** của động lực học rời rạc hơn với cùng ϵ/μ.
- **Khởi tạo ngẫu nhiên so với tại MAP (Hình 4-5):** IASG robust hơn SGLD/NUTS trong trường hợp khởi tạo ngẫu nhiên (gần với thực tế hơn), cho thấy ưu điểm thực dụng của một sampler dùng learning rate hằng không cần ”ủ nhiệt” (annealing) phức tạp.
- **Biên hiệu quả lý thuyết:** kết quả ”không SG-MCMC nào nhanh hơn $O(ND)$” (Mục 6.2 cuối) là một dạng ”ablation lý thuyết” ngược: paper tự chỉ ra giới hạn trên của chính lớp phương pháp mình đang phát triển, dựa trên optimality đã biết của Polyak averaging.

6. Giới hạn

Do tác giả nêu:

- Giả định A1-A4 chỉ hợp lý trong pha ”đã hội tụ gần optimum, nhiễu nhỏ so với gradient trung bình” (Mục 3.2, ”Discussion of Assumptions”); trong pha tìm kiếm ban đầu (search phase) các giả định bị vi phạm và paper ”không nói được gì chung” về hành vi thuật toán ở pha đó.
- Nếu hiệp phương sai nhiễu gradient C không có hạng đầy đủ (full rank), Σ cũng không đầy hạng, làm toàn bộ phân tích Mục 4 phức tạp hơn; paper đề xuất thêm nhiễu Gaussian đẳng hướng nhỏ làm biện pháp khắc phục nhưng không phân tích sâu tác động của biện pháp này.
- KL ngược $KL(f\|q)$ (Phụ lục C) chỉ giải được dạng đóng khi A và Σ giao hoán, điều **hiếm khi đúng trong thực tế**; do đó paper chỉ dùng $KL(q\|f)$ trong toàn bộ phần chính.
- Kết quả ”chặn dưới $O(ND)$” dựa trên Giả định A5 (Bernstein-von Mises áp dụng, N lớn, và mô hình đúng/well-specified: dữ liệu sinh thật từ model với $\theta^* = 0$ nên $A = BB^{\mathsf{T}}$); đây là ”các giả định lạc quan” (optimistic assumptions), và paper tự nhận ”có thể dễ dàng xây dựng tình huống mà các giả định 1-5 bị vi phạm”, chỉ lập luận (không chứng minh) rằng các tình huống vi phạm không dễ hơn.
- Yêu cầu dataset N đủ lớn so với số điều kiện (condition number) $\lambda_{\text{max}}/\lambda_{\text{min}}$ của Hessian A để IASG hoạt động tốt (Mục 6.2, ”Approximation error”); nếu không, cần preconditioner $H \approx A^{-1}$ thay learning rate vô hướng.

Do chúng ta nhận thấy:

- Thí nghiệm 7.1/7.3 đều dùng mô hình tuyến tính/log-linear (hồi quy tuyến tính, logistic, softmax); paper **không thử nghiệm trên mạng nơ-ron sâu phi tuyến** nào trong văn bản chính, nên tính tổng quát của phương pháp cho các loss không lồi/không xấp xỉ bậc hai tốt (như transformer) chỉ là suy luận từ lý thuyết, chưa được paper kiểm chứng thực nghiệm.

- Việc xem A (Hessian) là hằng số quanh một điểm tối ưu (Assumption 4) rất mạnh cho mạng sâu hiện đại, nơi loss landscape có nhiều điểm yên ngựa/minimum phẳng và Hessian biến đổi mạnh theo vị trí (liên hệ math 18, Shattered Gradients, và math 34).
- Bảng 1 không có cột ghi thời gian chạy thực tế (wall-clock) cho mỗi phương pháp, nên khó so sánh trực tiếp "chi phí tính toán thực" giữa SGD-f (preconditioner $O(D^2)$) và SGD-d (chéo, $O(D)$) ngoài con số KL.

7. Liên hệ với các bài khác

- **math 10** (A Conceptual Introduction to Hamiltonian Monte Carlo, đã viết, `10_2017_hamiltonian_monte_carlo.md`): cùng chủ đề "dùng động lực học liên tục để lấy mẫu Bayes"; HMC tích phân Hamilton chính xác (symplectic integrator), bài này xấp xỉ SGD bằng SDE Ornstein-Uhlenbeck; phần SGD-with-momentum (Mục 3.3 ở trên) có cấu trúc năng lượng động/tĩnh rất giống Mục 3.2 của bài 10 (động năng $\mathbb{E}[vv^\mathsf{T}]$, "nhiệt độ" BB^T).
- **math 08** (Variational Inference: A Review for Statisticians, 1601.00670): bài này tự gọi phương pháp của mình là một dạng VI (tối thiểu hoá $\mathrm{KL}(q\|f)$), nhưng khác VI cổ điển ở chỗ không tối ưu trực tiếp tham số của q mà tối ưu siêu tham số thuật toán; nên đọc math 08 trước để hiểu rõ khung KL-divergence chuẩn.
- **math 06** (New insights and perspectives on the natural gradient method, 1412.1193): cả hai bài đều liên quan đến vai trò ma trận Fisher information/preconditioner; $H^* = (2S/N)(BB^\mathsf{T})^{-1}$ ở Định lý 2 có cùng vai trò với Fisher information trong natural gradient (khi $A = BB^\mathsf{T}$, tức Giả định A5).
- `ls "/home/duy/Downloads/duy_dev/research_/llm content/"` xác nhận tồn tại **llm 01** (`01_2017_attention_is_all_you_need.md`): không liên hệ kỹ thuật trực tiếp, nhưng cùng năm 2017, và các tối ưu hoá Adam/preconditioner trong attention đều có thể soi qua lăng kính "noise covariance" của bài này.
- `ls "/home/duy/Downloads/duy_dev/research_/distillation content/"` xác nhận tồn tại **distillation 02** (`02_2015_distilling_knowledge_hinton.md`): phần Mục 8 dưới đây liên hệ ý tưởng "nhiệt độ/learning rate" của bài này tới quá trình distill.
- Trong thư mục mới cùng mã 'math': **opt 06** (Adam, 1412.6980) và **opt 08** (On the Convergence of Adam and Beyond, 1904.09237) nên đọc cùng, vì công thức $\epsilon^* \propto 1/\mathrm{Tr}(BB^\mathsf{T})$ ở Định lý 1 chính là cơ sở lý thuyết (từ góc nhìn Bayes) cho trực giác "learning rate nên nghịch với độ lớn nhiễu gradient" mà Adam/RMSProp dùng theo hướng thực dụng (empirical).

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, GPU $\leq$ 8 GB, batch size 1 khi suy luận). Bài này **liên quan khá trực tiếp** tới phần huấn luyện/fine-tune/distill, qua các ý tưởng sau (đều là suy luận, paper không thử nghiệm trên model như vậy):

1. **Batch size và learning rate như "nhiệt độ" ngầm.** Công thức $\Sigma \propto \epsilon/S$ (công thức (13)) cho thấy noise scale của SGD (và do đó độ "khám phá" không gian tham số, liên quan tới cực tiểu phẳng/sharp) phụ thuộc vào **tỉ lệ** ϵ/S, không phải riêng ϵ. Khi fine-tune/distill LightOnOCR trên GPU nhỏ, batch size thường bị ép nhỏ (do giới hạn 8 GB); theo logic bài này, để giữ "nhiệt độ" lấy mẫu tương đương một cấu hình batch lớn hơn, cần **giảm learning rate tỉ lệ với batch size** (hoặc dùng gradient accumulation để tăng S hiệu dụng mà không tăng VRAM đỉnh) để tránh nhiễu quá lớn làm quá trình huấn luyện "không bao giờ ổn định" quanh một cực tiểu tốt.
2. **Variational EM cho siêu tham số nén.** Ý tưởng Mục 4.2 (chạy đồng thời learning rate hằng cho tham số chính và learning rate giảm dần cho siêu tham số) có thể suy luận áp dụng để **tự động chỉnh hệ số regularization/weight decay hoặc trọng số loss distillation** trong lúc fine-tune model đã cắt tỉa, thay vì quét lưới (grid search) tốn nhiều lần chạy trên GPU hạn chế.

3. **Giới hạn tốc độ lấy mẫu $O(ND)$ như một lời nhắc.** Nếu dự án có ý định dùng SG-MCMC (ví dụ để ước lượng độ bất định của model student) thì kết quả lý thuyết ở Mục 6.2 cảnh báo: không thể sinh mẫu hậu nghiệm "nhanh hơn một epoch dữ liệu", nên với tập OCR lớn, chi phí ước lượng uncertainty kiểu Bayes đầy đủ theo hướng này không rẻ; các phương pháp thực dụng hơn (Dropout Bayesian Approximation, Deep Ensembles, nằm ngoài phạm vi bài này) có thể khả thi hơn trên GPU 8 GB.

Nói rõ: phần preconditioning đầy đủ ($O(D^2)$, Định lý 2) **ít khả thi trực tiếp** cho model 0.8B tham số (ma trận $D \times D$ với D hàng trăm triệu là không thể lưu); chỉ bản **chéo** (Hệ quả 3, tương đương AdaGrad/RMSProp) là thực dụng, và đây chính là lý do Adam/RMSProp phổ biến hơn SGFS đầy đủ trong deep learning hiện đại.

9. Thí nghiệm gợi ý

1. **Kiểm chứng Định lý 1 trên hồi quy logistic nhỏ:** tạo dữ liệu tổng hợp 2 lớp, $D = 5$-10 chiều, chạy constant SGD với ϵ tính theo công thức (15) (ước lượng BB^T bằng công thức (21)); so hiệp phương sai thực nghiệm của các iterate với hiệp phương sai Laplace của hậu nghiệm thật (tính bằng Hessian nghịch đảo). Kỳ vọng: hai hiệp phương sai gần nhau, xác nhận Hình 1-2 của paper trên quy mô nhỏ hơn.
2. **So sánh preconditioner chéo so với đầy đủ (tái tạo thu nhỏ Bảng 1):** cùng bài toán trên, so KL divergence giữa hậu nghiệm và phân phối dừng của (a) constant SGD không preconditioner, (b) preconditioner chéo (Hệ quả 3), (c) preconditioner đầy đủ (Định lý 2). Kỳ vọng: (c) < (b) < (a) về KL, khớp xu hướng Bảng 1.
3. **Đo ảnh hưởng của batch size tới "nhiệt độ" lấy mẫu:** cố định ϵ, chạy constant SGD với $S \in \{1, 10, 100\}$ trên cùng mô hình tuyến tính nhỏ; đo phương sai thực nghiệm của iterate ở trạng thái dừng. Kỳ vọng: phương sai tỉ lệ nghịch với S (từ $\Sigma \propto \epsilon/S$), minh hoạ trực tiếp cho gợi ý Mục 8.1 về batch size nhỏ trên GPU hạn chế.
4. **Tái tạo thu nhỏ IASG so với SGLD (Hình 4-5):** trên hồi quy tuyến tính $D = 5$, dữ liệu sinh đúng từ model (Assumption 5), so tốc độ hội tụ của phương sai biên ước lượng bởi IASG (cửa sổ N/S) so với SGLD (learning rate giảm theo Robbins-Monro) khi khởi tạo ngẫu nhiên. Kỳ vọng: IASG ổn định nhanh hơn SGLD, giống xu hướng paper báo cáo.
5. **Mô phỏng ảnh hưởng của momentum lên "nhiệt độ hiệu dụng":** với cùng mô hình, chạy SGD-momentum với (ϵ, μ) khác nhau nhưng giữ ϵ/μ không đổi; đo hiệp phương sai dừng. Kỳ vọng: hiệp phương sai gần như không đổi giữa các cấu hình (theo công thức (25)), xác nhận "momentum chỉ đổi tỉ lệ, không đổi hình dạng".

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
SDE xấp xỉ SGD (OU process)	$d\theta = -A\theta\, dt + \frac{1}{\sqrt{S}} B\, dW$, (11)
Phân phối dừng	$q(\theta) \propto \exp(-\frac{1}{2}\theta^\mathsf{T}\Sigma^{-1}\theta)$, $\Sigma A + A\Sigma = \frac{\epsilon}{S}BB^\mathsf{T}$
Learning rate tối ưu (Định lý 1)	$\epsilon^* = \dfrac{2SD}{N\,\mathrm{Tr}(BB^\mathsf{T})}$
Preconditioner đầy đủ tối ưu (Định lý 2)	$H^* = \dfrac{2S}{N}(BB^\mathsf{T})^{-1}$
Preconditioner chéo tối ưu (Hệ quả 3)	$H^*_{kk} = \dfrac{2S}{N\,(BB^\mathsf{T})_{kk}}$
Ước lượng online noise covariance	$C_t = (1-\kappa_t)C_{t-1} + \kappa_t(\hat{g}_{1,t} - \hat{g}_{S,t})(\hat{g}_{1,t} - \hat{g}_{S,t})^\mathsf{T}$, (21)
Momentum, hiệp phương sai dừng	$\Sigma A + A\Sigma = \frac{\epsilon}{\mu S}BB^\mathsf{T}$, (25)
SGFS preconditioner tối ưu (Định lý 5)	$H^* = \frac{2}{N}(BB^\mathsf{T} + EE^\mathsf{T})^{-1}$
Hiệp phương sai iterate averaging	$D \approx \frac{1}{TS}A^{-1}BB^\mathsf{T}(A^{-1})^\mathsf{T}$, không phụ thuộc preconditioner
Cửa sổ IASG	$T = N/S$ (một epoch cho 1 mẫu hậu nghiệm hiệu dụng)

Khái niệm	Công thức / ý nghĩa
Chặn dưới SG-MCMC	không thuật toán nào nhanh hơn $O(ND)$ cho 1 mẫu hiệu dụng độc lập (dưới Giả định A1-A5)

Ví dụ tính tay (tự tính): với $D = 2$, $N = 1000$, $S = 10$, $\text{Tr}(BB^\mathsf{T}) = 4$, công thức (15) cho $\epsilon^* = \frac{2 \cdot 10 \cdot 2}{1000 \cdot 4} = \frac{40}{4000} = 0.01$. Nếu tăng batch lên $S = 100$ (giữ $\text{Tr}(BB^\mathsf{T})$ không đổi, một xấp xỉ vì nhiễu thực ra giảm theo S nhưng ở đây $\text{Tr}(BB^\mathsf{T})$ định nghĩa là hiệp phương sai của **một mẫu đơn**, không đổi theo S), ϵ^* tăng 10 lần lên 0.1, khớp trực giác ”batch lớn thì chịu được learning rate lớn hơn” thường thấy trong thực hành huấn luyện mạng sâu (tự tính, minh hoạ công thức, không phải số trong paper).

11. Kết luận ngắn

Bài báo chỉ ra rằng SGD với learning rate hằng số, khi xấp xỉ bằng một quá trình Ornstein-Uhlenbeck liên tục, có phân phối dừng Gaussian mà ta có thể chỉnh (qua learning rate, minibatch, preconditioner) để tối thiểu hoá KL divergence tới hậu nghiệm Bayes, biến SGD thuần thành một bộ suy luận Bayes gần đúng rẻ tiền. Phân tích mở rộng sang momentum (chỉ đổi tỉ lệ, không đổi hình dạng phân phối), SGLD/SGFS (định lượng được độ chệch khi learning rate không tiệm cận 0, và SGFS với preconditioner đúng cho mẫu chính xác), và Polyak iterate averaging (tối ưu cho cả tối ưu hoá và lấy mẫu, nhưng kéo theo chặn dưới $O(ND)$ cho tốc độ sinh mẫu của mọi SG-MCMC). Thực nghiệm trên hồi quy tuyến tính/logistic xác nhận các dự đoán lý thuyết và cho thấy constant SGD với preconditioner tốt cạnh tranh được với MCMC chuyên dụng. Với dự án nén/distill LightOnOCR, giá trị chính là trực giác về batch size/learning rate như ”nhiệt độ” lấy mẫu và ý tưởng variational EM cho tự động chỉnh siêu tham số, còn phần preconditioner đầy đủ không khả thi ở quy mô hàng trăm triệu tham số.

Nguồn gốc: https://arxiv.org/abs/1704.04289

Bài 12

Computational Optimal Transport (2018)

Tác giả: Gabriel Peyré, Marco Cuturi
Venue / arXiv: arXiv:1803.00567 (v4, 18/03/2020); bản tạp chí ghi trong bibtex ở đầu tài liệu là *Foundations and Trends in Machine Learning*, tập 11, số 5-6, trang 355-607, năm 2019; tuy nhiên dòng footer ở đầu tài liệu lại ghi ”now Publishers Inc. Computational Optimal Transport. Foundations and Trends in Computer Graphics and Vision, vol. XX, no. XX, pp. 1-205, 2020” - **đây là một điểm paper tự mâu thuẫn giữa hai dòng metadata** (tên tạp chí và số trang khác nhau ở hai chỗ trong cùng tài liệu), ghi chú này không tự chọn một trong hai, chỉ nêu rõ cả hai như paper viết.
Nguồn: https://arxiv.org/abs/1803.00567
Chủ đề: Một tài liệu chuyên khảo (monograph) dài khoảng 250 trang tổng hợp lý thuyết và thuật toán của Optimal Transport (OT, vận tải tối ưu), từ bài toán Monge/Kantorovich cổ điển tới chính quy hoá entropy (entropic regularization) và thuật toán Sinkhorn, cùng các ứng dụng trong khoa học dữ liệu.
Vị trí trong lộ trình: nên đọc trước math 13 (Wasserstein GAN) vì bài đó dùng trực tiếp đối ngẫu Kantorovich-Rubinstein mà không tự chứng minh; có thể đọc sau math 10/11 (Hamiltonian Monte Carlo, SGD Bayesian) vì không có liên hệ kỹ thuật trực tiếp nhưng cả ba đều thuộc nhóm ”nền tảng toán học định lượng” của lộ trình; nên đọc trước hoặc cùng lúc với math 17 (Spectral Normalization for GANs) vì cả hai cùng xoay quanh ràng buộc Lipschitz.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

> **LƯU Ý QUAN TRỌNG VỀ PHẠM VI ĐỌC**: đây là một tài liệu chuyên khảo khoảng 250 trang (10 chương, mục lục đầy đủ: 1. Introduction, 2. Theoretical Foundations, 3. Algorithmic Foundations, 4. Entropic Regularization of Optimal Transport, 5. Semidiscrete Optimal Transport, 6. W_1 Optimal Transport, 7. Dynamic Formulations, 8. Statistical Divergences, 9. Variational Wasserstein Problems, 10. Extensions of Optimal Transport), không thể đọc và tóm tắt toàn bộ trong một ghi chú có độ dài hợp lý. Theo hướng dẫn của brief cho các bài khảo sát/sách dài, ghi chú này **đọc đầy đủ và kỹ**: Chương 1 (Introduction), Chương 2 (Theoretical Foundations, toàn bộ §2.1-2.6), và Chương 4 (Entropic Regularization, trọng tâm §4.1, §4.2, một phần §4.5-4.6, vì đây là thuật toán thực hành quan trọng nhất cho machine learning). Ghi chú **đọc một phần** Chương 8 (§8.2, phần Integral Probability Metrics, vì liên hệ trực tiếp tới đối ngẫu Kantorovich-Rubinstein dùng trong WGAN). Ghi chú **KHÔNG đọc chi tiết** Chương 3 (Algorithmic Foundations: network simplex, thuật toán auction, các giải thuật linear-programming cổ điển cho OT rời rạc), Chương 5 (Semidiscrete Optimal Transport), Chương 6 (W_1 Optimal Transport trên không gian metric/đồ thị), Chương 7 (Dynamic Formulations, công thức Benamou-Brenier), phần còn lại của Chương 8, Chương 9 (Variational Wasserstein Problems: Wasserstein barycenter, gradient flow, minimum Kantorovich estimator, chỉ được nhắc tên không đọc chi tiết công thức), và Chương 10 (Extensions: multimarginal, unbalanced OT, sliced Wasserstein, Gromov-Wasserstein) - các chương này chỉ được nêu tên và chủ đề chính trong mục 6 (Giới hạn) để người đọc biết chúng tồn tại, không được tóm tắt nội dung kỹ thuật vì chưa đọc kỹ.

1. Bối cảnh

Tài liệu mở đầu bằng một ẩn dụ của Gaspard Monge (1746-1818): một người công nhân có xẻng trong tay phải di chuyển một đống cát lớn nằm ở công trường để dựng lên một đống cát khác có hình dạng cho trước (ví dụ một ”lâu đài cát”). Người công nhân muốn tối thiểu hoá tổng công sức, đo bằng tổng khoảng cách hoặc thời gian mang từng xẻng cát. Các nhà toán học quan tâm Optimal Transport (OT, vận tải tối ưu) coi bài toán này như việc **so sánh hai phân phối xác suất** (hai đống cát có cùng thể tích): xét mọi cách có thể để biến đổi, vận chuyển hoặc tái định hình đống cát thứ nhất thành đống thứ hai, gán cho mỗi cách vận chuyển một ”chi phí toàn cục”, dựa trên xem xét ”cục bộ” là chi phí di chuyển một hạt cát từ nơi này sang nơi khác. OT quan tâm tới tính chất của phép vận chuyển ít tốn kém nhất, cũng như cách tính nó hiệu quả. Chi phí nhỏ nhất đó

không chỉ định nghĩa một khoảng cách giữa các phân phối, mà còn mang lại một **cấu trúc hình học phong phú** trên không gian các phân phối xác suất.

OT đã được ”khám phá lại” (re-discovered) nhiều lần dưới nhiều hình thức khác nhau: công trình khởi đầu của Monge xuất phát từ một bài toán kỹ thuật; Tolstoi (những năm 1920) và Hitchcock, Kantorovich, Koopmans (những năm 1940) thiết lập ý nghĩa của nó với logistics và kinh tế học; Dantzig giải nó bằng số vào năm 1949 trong khung quy hoạch tuyến tính (linear programming); OT được các nhà phân tích toán học xem lại vào những năm 1990, đặc biệt là Brenier; và gần đây gắn liền với tầm nhìn máy tính dưới tên ”earth mover’s distance”. Những năm gần đây chứng kiến một cuộc ”cách mạng” khác nhờ sự xuất hiện của các bộ giải xấp xỉ (approximate solvers) có thể mở rộng (scale) tới các bài toán chiều lớn, khiến OT ngày càng được dùng trong xử lý ảnh, đồ hoạ, và machine learning (hồi quy, phân loại, mô hình sinh).

Tài liệu tự định vị mục tiêu: ”đây là một tổng quan về OT nghiêng về các phương pháp số, bao quát các tính chất lý thuyết của OT có thể hướng dẫn thiết kế thuật toán mới, tập trung vào làn sóng thuật toán hiệu quả gần đây giúp OT có ý nghĩa trong khoa học dữ liệu” (dòng 132-135, diễn giải). Tài liệu cũng tự nói rõ về cấu trúc 3 tầng độ khó: phần chính của mỗi chương (Chương 2, 3, 4, 9, 10) chỉ xét hình học trên không gian các vector xác suất/histogram rời rạc (đủ để hiện thực thuật toán); các khung xám nhạt (light gray box) trình bày công thức cho **đo rời rạc** (discrete measures, có vị trí hỗ trợ trong không gian metric liên tục); các khung xám đậm (dark gray box) trình bày cho **đo tổng quát** (arbitrary measures, có thể có mật độ). Chương 5-8 nhắm tới độc giả thiên về toán học hơn (dòng 212-213).

2. Ý tưởng chính

Ý tưởng cốt lõi của toàn bộ OT: thay vì so sánh hai phân phối bằng các phân kỳ cổ điển (như Kullback-Leibler, chỉ so sánh **mật độ điểm-theo-điểm**), OT so sánh bằng cách tìm **kế hoạch vận chuyển rẻ nhất** biến một phân phối thành phân phối khác, dựa trên một ”chi phí nền” (ground cost) $c(x, y)$ đo khoảng cách/chi phí di chuyển khối lượng từ điểm x sang điểm y. Vì OT ”nâng” (lift) một khoảng cách nền giữa các điểm lên thành một khoảng cách giữa các phân phối trên các điểm đó, nó giải quyết được một hạn chế căn bản của các phân kỳ cổ điển: khi hai phân phối có **support rời nhau** hoặc không chồng lấp (ví dụ hai phân phối rời rạc đặt tại các điểm khác nhau), KL divergence hoặc total variation có thể không định nghĩa được hoặc cho giá trị không phản ánh ”khoảng cách hình học” thực sự giữa chúng, còn khoảng cách OT (Wasserstein) vẫn phản ánh đúng mức độ gần/xa về vị trí.

Ý tưởng thứ hai: bài toán gốc của Monge (tìm một **ánh xạ** xác định/deterministic biến mỗi điểm nguồn thành một điểm đích duy nhất) rất khó giải vì không lồi (nonconvex) và có thể không có nghiệm. Kantorovich đưa ra ý tưởng **nới lỏng (relaxation)** mang tính đột phá: thay ánh xạ xác định bằng một **coupling** (phân phối đồng thời π hay ma trận ghép đôi P) cho phép ”chia khối lượng” (mass splitting) từ một điểm nguồn tới nhiều điểm đích. Bài toán Kantorovich là một **quy hoạch tuyến tính (linear program)** lồi, dễ phân tích và giải hơn hẳn bài toán Monge gốc.

Ý tưởng thứ ba, mang tính thực hành quan trọng nhất của toàn chương trình nghiên cứu được tài liệu này tổng hợp: giải trực tiếp quy hoạch tuyến tính Kantorovich có độ phức tạp cao (với n điểm, cần xử lý ma trận $n \times n$ qua các thuật toán linear-programming cổ điển). Thêm một **số hạng chính quy hoá entropy (entropic regularization)** vào bài toán biến nó thành một bài toán **lồi chặt (strictly convex)**, có nghiệm duy nhất dạng đóng theo hai vector tỉ lệ (scaling vector), giải được bằng một thuật toán lặp rất đơn giản và song song hoá tốt trên GPU: **thuật toán Sinkhorn**. Đây là ý tưởng đã ”làm sống lại” OT trong machine learning hiện đại.

3. Phương pháp chi tiết

3.1 Histogram, đo rời rạc, và bài toán Monge (§2.1-2.2)

Một **histogram** (hay vector xác suất) a là một phần tử của simplex xác suất $\Sigma_n = \{a \in \mathbb{R}_+^n : \sum_i a_i = 1\}$. Một **đo rời rạc** (discrete measure) với trọng số a và vị trí $x_1, \dots, x_n \in \mathcal{X}$ được viết $\alpha = \sum_{i=1}^n a_i \delta_{x_i}$ (công

thức 2.1), với δ_x là khối Dirac tại x (một đơn vị khối lượng tập trung vô hạn tại vị trí x). Tài liệu cũng định nghĩa **đo tổng quát (general measure)** $\alpha \in \mathcal{M}(\mathcal{X})$ không nhất thiết rời rạc, truy cập được bằng cách "lấy tích phân" (test) với hàm liên tục $f \in C(\mathcal{X})$.

Bài toán gán tối ưu (optimal assignment problem): cho ma trận chi phí $(C_{i,j})$, với $n = m$, tìm phép hoán vị (permutation) σ tối thiểu $\frac{1}{n}\sum_i C_{i,\sigma(i)}$ (công thức 2.2). Tài liệu nhấn mạnh độ khó tính toán trực tiếp: tập hoán vị có kích thước $n!$, "có hơn 10^{100} phần tử khi n chỉ là 70" (dòng 432-433, trích Dantzig 1983).

Bài toán Monge (giữa hai đo rời rạc $\alpha = \sum a_i \delta_{x_i}$, $\beta = \sum b_j \delta_{y_j}$): tìm ánh xạ T sao cho $T_\# \alpha = \beta$ (push-forward, nghĩa là với mọi j: $b_j = \sum_{i:T(x_i)=y_j} a_i$, công thức 2.4), tối thiểu hoá $\sum_i c(x_i, T(x_i))$ (công thức 2.5). Tài liệu chỉ rõ hạn chế: khi $n \neq m$ "các ánh xạ Monge có thể không tồn tại giữa một đo rời rạc và đo khác, điều này luôn xảy ra khi đo đích có nhiều điểm hơn đo nguồn" (dòng 498-502). Bài toán Monge nói chung **không lồi** (nonconvex) vì tập khả thi của ánh xạ không có cấu trúc lồi, khiến việc giải trực tiếp khó khăn.

3.2 Nới lỏng Kantorovich (§2.3)

Kantorovich (1942) nới lỏng bản chất "xác định" của vận chuyển: thay vì khối lượng tại x_i chỉ được gán cho một điểm đích duy nhất, khối lượng tại x_i có thể được "phân tán" (dispatched) tới nhiều vị trí. Thay cho phép hoán vị σ hoặc ánh xạ T, dùng một **ma trận coupling** $P \in \mathbb{R}_+^{n \times m}$, với $P_{i,j}$ là lượng khối lượng chảy từ bin i sang bin j. Tập coupling khả thi:

$$U(a, b) := \{P \in \mathbb{R}_+^{n \times m} : P\mathbf{1}_m = a \text{ và } P^\top \mathbf{1}_n = b\} \tag{2.10}$$

Tài liệu chỉ ra $U(a, b)$ là một **đa diện lồi bị chặn (bounded convex polytope)**, và khác với Monge (bất đối xứng), công thức Kantorovich **luôn đối xứng**: $P \in U(a, b) \Leftrightarrow P^\top \in U(b, a)$. Bài toán Kantorovich:

$$L_C(a, b) := \min_{P \in U(a,b)} \langle C, P \rangle = \sum_{i,j} C_{i,j} P_{i,j} \tag{2.11}$$

một **quy hoạch tuyến tính**. Tài liệu minh hoạ bằng ví dụ "nhà kho và nhà máy" (Remark 2.10): một người điều hành n nhà kho, m nhà máy, cần vận chuyển nguyên liệu với chi phí đơn vị $C_{i,j}$, giải (2.11) để tìm kế hoạch vận chuyển rẻ nhất.

Khi $n = m$ và $a = b = \mathbf{1}_n/n$ (phân phối đều), **Proposition 2.1 (Kantorovich cho matching)** chứng minh nghiệm tối ưu của (2.11) có thể đạt được tại một ma trận hoán vị (dùng định lý Birkhoff: tập điểm cực trị của $U(\mathbf{1}_n/n, \mathbf{1}_n/n)$ chính là tập ma trận hoán vị; và một định lý cơ bản của linear programming nói cực tiểu một hàm tuyến tính trên một đa diện đạt tại điểm cực trị). Nghĩa là: nới lỏng Kantorovich là **"tight"** (chặt, không mất tính tối ưu) trên bài toán gán, dù không gian tìm kiếm lớn hơn hẳn (coupling thay vì hoán vị).

3.3 Tính chất khoảng cách / khoảng cách Wasserstein (§2.4)

Proposition 2.2: nếu $n = m$ và $C = D^p$ với D là một khoảng cách trên $\{1, \dots, n\}$ (đối xứng, $D_{i,j} = 0 \Leftrightarrow i = j$, thoả bất đẳng thức tam giác), thì:

$$W_p(a, b) := L_{D^p}(a, b)^{1/p} \tag{2.17}$$

định nghĩa **khoảng cách p-Wasserstein** trên Σ_n: đối xứng, dương, $W_p(a, b) = 0 \Leftrightarrow a = b$, và thoả bất đẳng thức tam giác. Chứng minh bất đẳng thức tam giác dùng kỹ thuật "gluing lemma" (bổ đề ghép nối): nếu P là coupling tối ưu giữa a, b và Q là coupling tối ưu giữa b, c, xây dựng $S = P\,\mathrm{diag}(1/\tilde{b})\,Q$ (với $\tilde{b}_j = b_j$ nếu $b_j > 0$, $= 1$ nếu không) là một coupling khả thi giữa a, c (không nhất thiết tối ưu), từ đó dùng bất đẳng thức

Minkowski suy ra $W_p(a,c) \le W_p(a,b) + W_p(b,c)$. Đây là một chứng minh **cấu trúc**, không trừu tượng, phù hợp cho đo rời rạc.

Điểm đáng chú ý nhất về tính chất "khoảng cách yếu" (weak distance) của Wasserstein (Remark 2.18): với mọi $p > 0$, $W^p(\delta_x, \delta_y) = d(x,y)$ (vì $U(\delta_x, \delta_y) = \{\delta_{(x,y)}\}$ chỉ có một coupling khả thi duy nhất), nên $W^p(\delta_x, \delta_y) \to 0$ khi $x \to y$. Đây chính là tính chất mà "các khoảng cách cổ điển không có": chuẩn L^2 chỉ áp dụng được cho đo liên tục có mật độ, trong khi Wasserstein **luôn định nghĩa được** và phản ánh đúng khoảng cách hình học, ngay cả giữa hai khối Dirac. Tính chất này tương đương với **hội tụ yếu (weak convergence)**: $\alpha_k \rightharpoonup \alpha \Leftrightarrow W_p(\alpha_k, \alpha) \to 0$ (Villani 2009, Theorem 6.8).

3.4 Bài toán đối ngẫu (§2.5)

Proposition 2.4: bài toán Kantorovich (2.11) có đối ngẫu:

$$L_C(a,b) = \max_{(f,g) \in R(C)} \langle f, a \rangle + \langle g, b \rangle \tag{2.20}$$

với $R(C) = \{(f,g) \in \mathbb{R}^n \times \mathbb{R}^m : \forall (i,j), f_i + g_j \le C_{i,j}\}$ (công thức 2.21). (f,g) gọi là "thế vị Kantorovich" (Kantorovich potentials).

Tài liệu dùng ẩn dụ "thuê ngoài logistics" (Remark 2.21) để giải thích trực giác: một nhà vận chuyển (vendor) đề nghị tính giá "thu gom" f_i tại mỗi nhà kho i và giá "giao hàng" g_j tại mỗi nhà máy j, độc lập với tuyến đường cụ thể; tổng giá là $\langle f, a \rangle + \langle g, b \rangle$. Để người điều hành chấp nhận đề nghị, cần $f_i + g_j \le C_{i,j}$ với mọi cặp (i,j) (nếu không, tự vận chuyển rẻ hơn). Nhà vận chuyển muốn tối đa hoá giá nhận được dưới ràng buộc này - chính là bài toán đối ngẫu (2.20). Tài liệu chứng minh (bằng Lagrangian duality, trao đổi min-max) rằng giá tối ưu của nhà vận chuyển **bằng chính xác** chi phí vận chuyển tối ưu $L_C(a,b)$ - đây là **strong duality** của quy hoạch tuyến tính.

Định lý Brenier (Theorem 2.1), một kết quả cổ điển quan trọng được tài liệu trình bày với sơ đồ chứng minh: trong trường hợp $\mathcal{X} = \mathcal{Y} = \mathbb{R}^d$, $c(x,y) = \|x-y\|^2$, nếu ít nhất một trong hai đo (gọi là α) có mật độ so với đo Lebesgue, thì coupling tối ưu π của bài toán Kantorovich là **duy nhất** và được hỗ trợ trên đồ thị của một "ánh xạ Monge" $T : \mathbb{R}^d \to \mathbb{R}^d$. Hơn nữa, T được xác định duy nhất là **gradient của một hàm lồi** φ: $T(x) = \nabla \varphi(x)$. Ý nghĩa: trong trường hợp này, **bài toán Kantorovich và bài toán Monge là tương đương** (nới lỏng là chặt), đây là phiên bản liên tục của Proposition 2.1 (trường hợp rời rạc, đều). Tài liệu cũng liên hệ φ với phương trình Monge-Ampère $\det(\partial^2 \varphi(x)) \rho_\beta(\nabla \varphi(x)) = \rho_\alpha(x)$ (công thức 2.32, Remark 2.25), một PDE phi tuyến khó giải số trực tiếp (toán tử Monge-Ampère được tài liệu gọi là "Laplacian suy biến phi tuyến").

3.5 Trường hợp đặc biệt: 1 chiều (§2.6)

Khi $\mathcal{X} = \mathbb{R}$ (một chiều), OT có **công thức đóng đơn giản**: nếu $\alpha = \frac{1}{n}\sum \delta_{x_i}$, $\beta = \frac{1}{n}\sum \delta_{y_j}$ với $x_1 \le \cdots \le x_n$ và $y_1 \le \cdots \le y_n$ đã **sắp thứ tự**, thì:

$$W_p(\alpha, \beta)^p = \frac{1}{n} \sum_{i=1}^{n} |x_i - y_i|^p \tag{2.33}$$

nghĩa là chỉ cần **sắp xếp (sort)** hai tập điểm rồi ghép theo thứ tự, không cần giải quy hoạch tuyến tính đầy đủ. Tài liệu minh hoạ ứng dụng thực tế: **cân bằng histogram ảnh (histogram equalization)**, dùng OT 1D để tìm phép tương ứng giữa mức xám của hai ảnh (Remark 2.29), và công cụ hàm phân phối tích luỹ nghịch (quantile function, công thức 2.34-2.35) để tổng quát hoá cho đo liên tục bất kỳ trên $\mathbb{R}$.

3.6 Chính quy hoá Entropy (§4.1)

Entropy rời rạc của một coupling: $H(P) := -\sum_{i,j} P_{i,j}(\log(P_{i,j}) - 1)$ (công thức 4.1), là hàm **1-lõm mạnh (1-strongly concave)**. Ý tưởng chính quy hoá entropy:

$$L_C^\varepsilon(a,b) := \min_{P \in U(a,b)} \langle P, C\rangle - \varepsilon H(P) \tag{4.2}$$

Vì mục tiêu là hàm ε**-lồi mạnh (ε-strongly convex)**, bài toán (4.2) có nghiệm **duy nhất**. Tài liệu lưu ý nguồn gốc lịch sử: ý tưởng này "có thể truy về các mô hình trong lý thuyết giao thông [Wilson, 1969]: các mẫu giao thông thực tế không khớp với dự đoán của bài toán vận tải tối ưu, vì giao thông thực tế phân tán (diffuse) hơn, trong khi OT có xu hướng dựa vào một vài tuyến đường do tính thưa (sparsity) của coupling tối ưu" (dòng 2694-2698, diễn giải).

Proposition 4.1 (hội tụ theo ε): nghiệm duy nhất P_ε của (4.2) thoả:

- $\varepsilon \to 0$: $P_\varepsilon \to \arg\min\{-H(P) : P \in U(a,b), \langle P, C\rangle = L_C(a,b)\}$, nghĩa là hội tụ về nghiệm **có entropy lớn nhất** trong tập nghiệm tối ưu của bài toán gốc không chính quy hoá, và $L_C^\varepsilon(a,b) \to L_C(a,b)$.
- $\varepsilon \to \infty$: $P_\varepsilon \to a \otimes b = ab^\top$ (tích tensor, coupling của hai biến **độc lập**).

Diễn giải trực giác (tự tổng hợp từ tài liệu): ε nhỏ cho nghiệm gần giống bài toán OT gốc (thưa, "sắc nét"); ε lớn cho nghiệm giống hệt trường hợp hai biến độc lập hoàn toàn (không còn thông tin vận chuyển, chỉ còn tích hai marginal). Giữa hai cực đó, nghiệm **trở nên ít thưa hơn** (less sparse) khi ε tăng, điều này vừa giúp tăng tốc thuật toán (mục 3.7) vừa cải thiện tốc độ hội tụ thống kê (Chương 8, không đọc chi tiết trong ghi chú này).

Một cách biểu diễn khác của (4.2): định nghĩa phân kỳ KL giữa hai ma trận $KL(P|K) := \sum_{i,j} P_{i,j}\log(P_{i,j}/K_{i,j}) - P_{i,j} + K_{i,j}$ (công thức 4.6), và kernel Gibbs $K_{i,j} := e^{-C_{i,j}/\varepsilon}$, thì nghiệm P_ε chính là **phép chiếu KL** của K lên $U(a,b)$: $P_\varepsilon = \text{Proj}_{U(a,b)}^{KL}(K) = \arg\min_{P \in U(a,b)} KL(P|K)$ (công thức 4.7). Bài toán này, với đo tổng quát, được gọi là "bài toán Schrödinger tĩnh" (static Schrödinger problem), có nguồn gốc từ vật lý thống kê (Schrödinger 1931).

3.7 Thuật toán Sinkhorn (§4.2)

Proposition 4.3: nghiệm của (4.2) có dạng $P_{i,j} = u_i K_{i,j} v_j$ cho hai vector tỉ lệ (scaling) $(u,v) \in \mathbb{R}_+^n \times \mathbb{R}_+^m$ (công thức 4.12). Chứng minh: đưa vào 2 biến đối ngẫu f, g cho ràng buộc marginal, điều kiện bậc nhất (first-order condition) cho $\frac{\partial}{\partial P_{i,j}}\left[\langle P, C\rangle - \varepsilon H(P) - \langle f, P\mathbf{1}_m - a\rangle - \langle g, P^\top \mathbf{1}_n - b\rangle\right] = 0$ cho ra $C_{i,j} + \varepsilon\log(P_{i,j}) - f_i - g_j = 0$, tức $P_{i,j} = e^{f_i/\varepsilon} e^{-C_{i,j}/\varepsilon} e^{g_j/\varepsilon}$, đặt $u_i = e^{f_i/\varepsilon}$, $v_j = e^{g_j/\varepsilon}$ cho dạng trên.

Viết dưới dạng ma trận: $P = \text{diag}(u) K \text{diag}(v)$. Ràng buộc marginal $P\mathbf{1}_m = a$, $P^\top \mathbf{1}_n = b$ trở thành hệ phương trình phi tuyến (bài toán "matrix scaling" trong giải tích số):

$$u \odot (Kv) = a \quad \text{và} \quad v \odot (K^\top u) = b \tag{4.14}$$

($\odot$ là phép nhân theo từng phần tử). **Thuật toán Sinkhorn** giải hệ này bằng lặp xen kẽ:

$$u^{(\ell+1)} := \frac{a}{Kv^{(\ell)}}, \qquad v^{(\ell+1)} := \frac{b}{K^\top u^{(\ell+1)}} \tag{4.15}$$

khởi tạo $v^{(0)} = \mathbf{1}_m$ (phép chia giữa hai vector hiểu theo từng phần tử). Tài liệu nhấn mạnh: dù (u,v) không duy nhất (chỉ xác định tới một hằng số nhân: nếu (u,v) thoả (4.13) thì $(\lambda u, v/\lambda)$ cũng thoả với mọi $\lambda > 0$), **coupling diag**$(u)K$**diag**(v) **thu được là duy nhất** và các lần lặp đều hội tụ về nó.

Remark 4.5 (lịch sử): các lần lặp (4.15) ”xuất hiện đầu tiên trong [Yule 1912, Kruithof 1937], sau đó được biết là thủ tục iterative proportional fitting (IPFP) và phương pháp RAS. Chứng minh hội tụ được gán cho Sinkhorn [1964], do đó có tên thuật toán này... quan tâm trở lại trong khoa học dữ liệu sau [Cuturi 2013], người chỉ ra thuật toán Sinkhorn cung cấp một xấp xỉ hiệu quả và có thể mở rộng của OT, nhờ song song hoá dễ dàng (đặc biệt trên GPU), và đại lượng chính quy hoá này cũng định nghĩa, khác với công thức quy hoạch tuyến tính, một hàm mất mát khả vi” (dòng 2994-3011, diễn giải).

Remark 4.6 (độ phức tạp tổng thể): Altschuler et al. [2017] chứng minh với $\varepsilon = \frac{4\log(n)}{\tau}$, cần $O(\|C\|_\infty^3 \log(n)\tau^{-3})$ lần lặp Sinkhorn (kèm một bước làm tròn/rounding để đưa về coupling khả thi $\hat{P} \in U(a,b)$) để đảm bảo $\langle \hat{P}, C\rangle \leq L_C(a,b) + \tau$. Điều này kéo theo Sinkhorn tính được nghiệm τ**-xấp xỉ** của OT không chính quy hoá trong $O(n^2 \log(n)\tau^{-3})$ phép toán.

Remark 4.7 (ổn định số học): tài liệu cảnh báo ”trong thực hành, hội tụ Sinkhorn thường không bị chậm đi vì một lý do đơn giản hơn: thuật toán Sinkhorn thường **không kết thúc được** ngay khi một số phần tử của kernel K trở nên quá nhỏ để lưu dưới dạng số dương trong bộ nhớ, và trở thành 0 thay vào đó”, dẫn tới phép chia cho 0. Giải pháp: tính toán trên các số nhân tử u, v trong **log-domain** (Remark 4.23, không đọc chi tiết trong ghi chú này).

3.8 Ví dụ số nhỏ tính tay: một bước Sinkhorn (tự tính)

Để minh hoạ cơ chế thuật toán Sinkhorn, xét bài toán rất nhỏ: $n = m = 2$, $a = (0.5, 0.5)$, $b = (0.5, 0.5)$ (hai histogram đều trên 2 bin), ma trận chi phí $C = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (chi phí 0 nếu cùng bin, 1 nếu khác bin), và $\varepsilon = 1$.

Kernel Gibbs: $K_{i,j} = e^{-C_{i,j}/\varepsilon}$, nên $K = \begin{pmatrix} e^0 & e^{-1} \\ e^{-1} & e^0 \end{pmatrix} = \begin{pmatrix} 1 & 0.3679 \\ 0.3679 & 1 \end{pmatrix}$ (tự tính: $e^{-1} \approx 0.3679$).

Khởi tạo $v^{(0)} = (1, 1)$. Bước 1: $u^{(1)} = a/(Kv^{(0)})$. Tính $Kv^{(0)} = \begin{pmatrix} 1 \times 1 + 0.3679 \times 1 \\ 0.3679 \times 1 + 1 \times 1 \end{pmatrix} = \begin{pmatrix} 1.3679 \\ 1.3679 \end{pmatrix}$ (tự tính). Vậy $u^{(1)} = (0.5/1.3679,\ 0.5/1.3679) = (0.3655,\ 0.3655)$ (tự tính, $0.5/1.3679 \approx 0.3655$).

Bước 2: $v^{(1)} = b/(K^\top u^{(1)})$. Vì K đối xứng, $K^\top = K$. $Ku^{(1)} = \begin{pmatrix} 1 \times 0.3655 + 0.3679 \times 0.3655 \\ 0.3679 \times 0.3655 + 1 \times 0.3655 \end{pmatrix} = \begin{pmatrix} 0.3655 \times 1.3679 \\ 0.3655 \times 1.3679 \end{pmatrix} = \begin{pmatrix} 0.5000 \\ 0.5000 \end{pmatrix}$ (tự tính: $0.3655 \times 1.3679 \approx 0.5000$). Vậy $v^{(1)} = (0.5/0.5,\ 0.5/0.5) = (1, 1)$ (tự tính).

Vì $v^{(1)} = v^{(0)} = (1, 1)$, thuật toán đã **hội tụ ngay sau 1 vòng lặp** đối với bài toán đối xứng đặc biệt này (do tính đối xứng của $a = b$ và C). Coupling tối ưu: $P_{i,j} = u_i K_{i,j} v_j$, với $u = (0.3655, 0.3655)$, $v = (1, 1)$:

$$P = \begin{pmatrix} 0.3655 \times 1 \times 1 & 0.3655 \times 0.3679 \times 1 \\ 0.3655 \times 0.3679 \times 1 & 0.3655 \times 1 \times 1 \end{pmatrix} = \begin{pmatrix} 0.3655 & 0.1345 \\ 0.1345 & 0.3655 \end{pmatrix}$$

(tự tính: $0.3655 \times 0.3679 \approx 0.1345$). Kiểm tra marginal: tổng hàng 1 = $0.3655 + 0.1345 = 0.5000$, khớp $a_1 = 0.5$ (tự tính), xác nhận $P \in U(a, b)$. Nhận xét tự rút ra: nghiệm P có khối lượng tập trung nhiều hơn trên đường chéo ($0.3655 > 0.1345$), phản ánh đúng trực giác ”chi phí $C_{1,1} = C_{2,2} = 0$ rẻ hơn $C_{1,2} = C_{2,1} = 1$”, nhưng **không hoàn toàn tập trung** trên đường chéo (khác với nghiệm OT không chính quy hoá $\varepsilon = 0$, lẽ ra sẽ cho $P = \text{diag}(0.5, 0.5)$ hoàn toàn trên đường chéo vì đây chính là trường hợp $a = b$ nên coupling tối ưu không chính quy hoá là ma trận đơn vị nhân 0.5), đúng với lý thuyết Proposition 4.1 rằng chính quy hoá entropy ”làm mờ” (blur) nghiệm ra khỏi các đỉnh thưa của đa diện.

3.9 Hàm mất mát khả vi và Sinkhorn divergence (§4.5)

Vì $L_C^\varepsilon(a,b)$ là hàm **lồi chung (jointly convex)** theo cả a, b khi $\varepsilon \geq 0$, và khi $\varepsilon > 0$ gradient của nó bằng chính các thế vị đối ngẫu tối ưu: $\nabla L_C^\varepsilon(a,b) = (f^\star, g^\star)$ (Proposition 4.6, chọn $f^\star, g^\star$ sao cho tổng toạ độ bằng 0). Điều này rất quan trọng cho machine learning: L_C^ε **có thể dùng trực tiếp làm hàm mất mát khả vi** trong các bài toán biến phân (Chương 9, không đọc chi tiết).

Tài liệu định nghĩa **Sinkhorn divergence** (nguyên và đối ngẫu, Definition 4.1): $P_C^\varepsilon(a,b) := \langle C, P^\star \rangle$ và $D_C^\varepsilon(a,b) := \langle f^\star, a \rangle + \langle g^\star, b \rangle$, thoả quan hệ $D_C^\varepsilon(a,b) \leq L_C^\varepsilon(a,b) \leq P_C^\varepsilon(a,b)$ và $P_C^\varepsilon(a,b) - D_C^\varepsilon(a,b) = \varepsilon(H(P^\star) + 1)$ (Proposition 4.7, công thức 4.47). Tài liệu tự lưu ý một điểm thực tế quan trọng: các chặn này "chỉ đúng khi Sinkhorn đã hội tụ hoàn toàn và do đó không bao giờ thực sự hữu dụng trong thực hành" (dòng 3879-3881, diễn giải), vì trong thực hành số vòng lặp luôn bị giới hạn (ngưỡng dừng sớm). Khi dừng sau L vòng lặp cố định, $D_C^{(L)}(a,b)$ (dùng $f^{(L)}, g^{(L)}$ chưa hội tụ) vẫn cho một **chặn dưới hợp lệ** của L_C^ε (Proposition 4.8), nhưng tài liệu cũng cảnh báo (Remark 4.27) rằng "khác với L_C^ε, Sinkhorn divergence hữu hạn $D_C^{(L)}(a,b)$ nói chung **không phải hàm lồi** theo tham số của nó (có thể kiểm tra dễ dàng bằng số)" - một điểm hạn chế lý thuyết quan trọng khi dùng Sinkhorn divergence làm loss trong tối ưu không lồi.

3.10 Liên hệ Integral Probability Metric và Kantorovich-Rubinstein (§8.2, đọc một phần)

Tài liệu định nghĩa khung tổng quát **dual norm**/Integral Probability Metric (IPM): cho tập lồi đối xứng B các hàm đo được, $\|\alpha\|_B := \max_f \{\int f \, d\alpha : f \in B\}$ (công thức 8.10). **Example 8.7** xác nhận trực tiếp: khoảng cách W^1 (Wasserstein-1, định nghĩa ở Chương 6 mà ghi chú này không đọc chi tiết) chính là một trường hợp đặc biệt của dual norm này với $B = \{f : \text{Lip}(f) \leq 1\}$ (tập hàm 1-Lipschitz) - đây chính là công thức đối ngẫu Kantorovich-Rubinstein mà math 13 (Wasserstein GAN) dùng trực tiếp làm nền tảng. Tài liệu còn định nghĩa **flat norm** (còn gọi là chuẩn "Kantorovich-Rubinstein" theo Hanin 1992, dùng $B = \{f : \|\nabla f\|_\infty \leq 1, \|f\|_\infty \leq 1\}$, công thức 8.11) và **Maximum Mean Discrepancy (MMD)** như một trường hợp IPM khác (dùng B là hình cầu đơn vị của một không gian Hilbert tái tạo nhân, RKHS) - những biến thể này không được đọc chi tiết trong ghi chú này.

4. Thí nghiệm và kết quả chính

Vì đây là một tài liệu khảo sát/chuyên khảo lý thuyết-thuật toán, không phải một bài báo thực nghiệm theo nghĩa "so sánh hệ thống mới với baseline trên benchmark cố định". Phần "thực nghiệm" chủ yếu là các hình minh hoạ số học để làm rõ hành vi của các khái niệm lý thuyết, trong các chương mà ghi chú này đã đọc kỹ:

- Figure 2.8 (đã đọc): minh hoạ coupling tối ưu $P^\star$ và thế vị đối ngẫu $f^\star, g^\star$ cho một bài toán OT cụ thể giữa hai đo rời rạc 2D với chi phí Euclid bình phương, trực quan hoá "giá tương đối" tại mỗi điểm.
- Figure 2.9 (đã đọc): minh hoạ coupling tối ưu 1D (công thức 2.33) dưới dạng mũi tên nối các điểm đã sắp xếp.
- Figure 4.1-4.3 (đã đọc): minh hoạ ảnh hưởng của ε lên nghiệm chính quy hoá entropy trên simplex Σ_3 (tam giác 2D) và giữa hai đo liên tục/rời rạc 1D-2D, xác nhận trực quan Proposition 4.1 (nghiệm dịch từ góc đa diện về "trung tâm entropy" khi ε tăng).
- Figure 4.5 (đã đọc): minh hoạ sự tiến triển của coupling Sinkhorn qua các vòng lặp ℓ, và tốc độ hội tụ (đo bằng log của vi phạm ràng buộc marginal) với các giá trị $\varepsilon = 10, 0.1, 10^{-3}$ - xác nhận trực quan rằng ε nhỏ hơn làm Sinkhorn hội tụ chậm hơn.

Vì ghi chú này không đọc chi tiết các chương thực nghiệm ứng dụng cụ thể hơn (ví dụ các ví dụ ảnh màu/texture trong Chương 2 Remark 2.12, hay các ứng dụng Wasserstein barycenter trong Chương 9), không có bảng số liệu định lượng kiểu "accuracy/FID" nào để báo cáo ở mục này - bản chất tài liệu này là lý thuyết và thuật toán, không phải một paper thực nghiệm đơn lẻ.

5. Phân tích / ablation

Tương tự mục 4, không có ”ablation” theo nghĩa thực nghiệm máy học thông thường. Thay vào đó, phần có giá trị phân tích gần nhất với ”ablation” là các **so sánh chế độ cực hạn của tham số** ε (Proposition 4.1, đã trình bày ở mục 3.6): $\varepsilon \to 0$ (gần với OT gốc, nghiệm thưa/sắc nét, entropy lớn nhất trong tập nghiệm tối ưu) và $\varepsilon \to \infty$ (nghiệm hội tụ về tích hai marginal độc lập, hoàn toàn ”mờ”). Đây là một phân tích lý thuyết có vai trò tương đương một ”ablation tham số” giúp người đọc hiểu rõ đánh đổi (trade-off) giữa độ chính xác xấp xỉ OT gốc và tốc độ/ổn định tính toán (Remark 4.6-4.7: ε nhỏ hơn cho xấp xỉ tốt hơn nhưng Sinkhorn hội tụ chậm hơn và dễ gặp vấn đề số học).

Một phân tích thứ hai có giá trị: so sánh chặn trên/dưới của Sinkhorn divergence ($D_C^\varepsilon \le L_C^\varepsilon \le P_C^\varepsilon$, với khoảng cách giữa chúng chính xác bằng $\varepsilon(H(P^\star)+1)$) cho thấy rõ **chi phí chính xác của việc chính quy hoá**: sai số giữa bài toán chính quy hoá và OT gốc có thể định lượng trực tiếp qua ε và entropy của nghiệm.

6. Giới hạn

Các giới hạn **paper tự nêu** trong các phần ghi chú này đã đọc:

1. Nghịch lý thực tế của Sinkhorn: lý thuyết nói tốc độ hội tụ chậm đi khi $\varepsilon \to 0$, nhưng tài liệu tự nói ”trong thực hành, sự chậm đi này hiếm khi được quan sát vì một lý do đơn giản hơn: thuật toán sẽ **không hội tụ được** (not terminate) khi phần tử kernel K quá nhỏ để lưu dưới dạng số dương, dẫn tới chia cho 0” (Remark 4.7) - một hạn chế số học thực tế nghiêm trọng hơn cả vấn đề tốc độ hội tụ lý thuyết.
2. Chặn lý thuyết của Sinkhorn divergence ($D_C^\varepsilon \le L_C^\varepsilon \le P_C^\varepsilon$) ”chỉ đúng khi đã hội tụ hoàn toàn, và do đó không bao giờ thực sự hữu dụng trong thực hành” (dòng 3879-3881) - paper tự thừa nhận một kết quả lý thuyết đẹp nhưng **không áp dụng được trực tiếp** khi dừng sớm (early stopping), cần công thức thay thế (Proposition 4.8) chỉ cho chặn một chiều.
3. Sinkhorn divergence hữu hạn bước $D_C^{(L)}$ **không lồi** nói chung theo tham số (Remark 4.27) - khác với L_C^ε lý thuyết (lồi chung theo a, b), khiến việc tối ưu các mô hình dùng Sinkhorn divergence làm loss (ví dụ khi L nhỏ để tiết kiệm tính toán) mất đi đảm bảo lồi.
4. Bài toán Monge gốc có thể **không có nghiệm** khi $n \neq m$ và trọng số không tương thích (Remark 2.4 cuối) - nới lỏng Kantorovich luôn có nghiệm (vì tập khả thi compact và không rỗng), nhưng điều này đồng nghĩa Kantorovich và Monge **không luôn tương đương**; định lý Brenier chỉ đảm bảo tương đương trong trường hợp đặc biệt (một đo có mật độ, chi phí Euclid bình phương).
5. Về mặt phạm vi đọc của ghi chú này (giới hạn do người viết chọn, không phải do paper tự nêu): các chương không đọc chi tiết chứa những nội dung quan trọng khác của lý thuyết OT hiện đại mà ghi chú này **không thể đánh giá/tóm tắt chính xác**, cụ thể: Chương 3 (các thuật toán linear-programming cổ điển cho OT rời rạc: network simplex, auction algorithm - các thuật toán chính xác, không xấp xỉ, có độ phức tạp đa thức nhưng chậm hơn Sinkhorn trong thực hành theo hiểu biết chung của lĩnh vực, ”ngoài paper, kiến thức chung”); Chương 5 (Semidiscrete OT, khi một đo rời rạc một đo liên tục); Chương 6 (W^1 trên không gian metric/đồ thị, có công thức đóng nhanh hơn Sinkhorn trong một số trường hợp); Chương 7 (công thức động Benamou-Brenier, biến OT thành bài toán PDE phụ thuộc thời gian); Chương 9 (Wasserstein barycenter, gradient flow trong không gian Wasserstein, minimum Kantorovich estimator - các ứng dụng biến phân quan trọng cho machine learning mà ghi chú này chỉ nêu tên); Chương 10 (OT đa biên/multimarginal, OT không cân bằng/unbalanced khi hai đo không cùng tổng khối lượng, sliced Wasserstein distance, khoảng cách Gromov-Wasserstein cho so sánh cấu trúc giữa hai không gian metric khác nhau).

7. Liên hệ với các bài khác

- **math 13 (Wasserstein GAN, 1701.07875)**: đã có file tại ”math content/13_2017_wasserstein_gan.md” (xác nhận bằng ls). WGAN dùng trực tiếp đối ngẫu Kantorovich-Rubinstein (công thức 2.20 của tài liệu này, hoặc tương đương IPM với B = hàm 1-Lipschitz, Example 8.7 mục 3.10) như một ”hộp đen”

lý thuyết không tự chứng minh; tài liệu hiện tại là nơi chứng minh/trình bày đầy đủ hơn kết quả đối ngẫu đó (Proposition 2.4, 2.5).

- **math 17 (Spectral Normalization for GANs, 1802.05957)**: đã có file tại "math content/17_2018_spectral_normalization_gan.md" (xác nhận bằng ls). Cả hai bài (math 13, math 17) đều cần ép ràng buộc Lipschitz lên một hàm thực hiện bởi mạng neural; tài liệu hiện tại cung cấp nền tảng lý thuyết **tại sao** ràng buộc 1-Lipschitz quan trọng (chính là định nghĩa của W^1 qua IPM).
- **math 15 (Online normalizer calculation for softmax, 1805.02867)**: đã có file tại "math content/15_2018_online_softmax.md" (xác nhận bằng ls). Không có liên hệ kỹ thuật trực tiếp, nhưng cả hai bài cùng thuộc nhóm "thuật toán số học/tối ưu nền tảng cho machine learning hiện đại" trong lộ trình math.
- **ml basics content/14_2014_gan.md, ml basics content/15_2013_vae_aevb.md** (đã xác nhận tồn tại bằng ls): GAN gốc dùng Jensen-Shannon divergence; VAE dùng KL divergence trong ELBO. Cả hai đều là các "phân kỳ cổ điển" mà OT/Wasserstein (tài liệu này) được đề xuất như một thay thế có tính chất hình học tốt hơn khi hai phân phối không chồng lấp.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ nội dung mục này là suy luận. Tài liệu này là một chuyên khảo lý thuyết/thuật toán tổng quát về OT, không nói về LightOnOCR hay về distillation/pruning của mô hình ngôn ngữ-hình ảnh.

Dự án người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3600 token/trang, GPU $\leq$ 8GB, batch 1 khi suy luận; các hướng pruning/distillation/RL/interpretability đã có thư mục riêng, không lặp lại chi tiết kỹ thuật của các hướng đó). Nội dung OT/Sinkhorn **không phải** một kỹ thuật nén trực tiếp, nhưng có hai liên hệ ý tưởng hợp lý (suy luận, mức độ chắc chắn khác nhau):

1. **Hàm mất mát khả vi cho distillation (liên hệ trung bình, qua math 13)**: như đã nói ở ghi chú math 13 mục 8, khi hai phân phối đầu ra (của teacher và student đã nén) ít chồng lấp (ví dụ ở giai đoạn đầu fine-tune sau khi cắt/lượng tử hoá mạnh), KL divergence tiêu chuẩn có thể kém ổn định hơn một khoảng cách kiểu Wasserstein. Tài liệu hiện tại cho thấy cụ thể **cách tính** một xấp xỉ khả vi của Wasserstein distance bằng Sinkhorn (mục 3.9, L_C^ε lồi và có gradient dạng đóng theo thế vị đối ngẫu) - đây là công cụ cụ thể (không chỉ ý tưởng) có thể thử nghiệm làm hàm mất mát thay thế hoặc bổ sung cho KL trong distillation của LightOnOCR, nếu gặp vấn đề ổn định huấn luyện ở giai đoạn đầu.

2. **Chi phí tính toán Sinkhorn trên GPU 8GB (ít liên quan trực tiếp, cần cân nhắc thực tế)**: thuật toán Sinkhorn (công thức 4.15) chỉ cần các phép nhân ma trận-vector, song song hoá tốt trên GPU. Tuy nhiên, nếu áp dụng Sinkhorn divergence giữa phân phối đầu ra của teacher/student trên **từng cặp mẫu trong một batch** (ví dụ so sánh phân phối token-level), kích thước ma trận kernel K sẽ là $n \times m$ với n, m là kích thước từ vựng (vocab size) hoặc số lượng visual token (1600-2100 theo đặc tả LightOnOCR) - với vocab size điển hình hàng chục nghìn, ma trận kernel đầy đủ ($K \in \mathbb{R}^{n \times m}$ với n cỡ 30.000-128.000) có thể **vượt quá bộ nhớ của GPU 8GB** nếu tính trực tiếp không có cấu trúc đặc biệt (ví dụ tận dụng chi phí nền có cấu trúc riêng). Đây là một hạn chế thực tế quan trọng cần kiểm tra trước khi áp dụng, không được tài liệu này đề cập (vì tài liệu không nói về mô hình ngôn ngữ cụ thể).

3. **Không liên quan trực tiếp tới pruning/lượng tử hoá cấu trúc mô hình**: các kỹ thuật cốt lõi của tài liệu (Kantorovich relaxation, Sinkhorn, Wasserstein barycenter) không có liên hệ trực tiếp tới việc cắt lớp/đầu attention hoặc lượng tử hoá trọng số của LightOnOCR; các hướng đó đã có thư mục riêng (pruning content/, không lặp lại ở đây).

Kết luận mục này (suy luận): tài liệu này **ít liên quan trực tiếp** tới việc nén LightOnOCR như một kỹ thuật nén cụ thể, nhưng cung cấp **công cụ toán học cụ thể** (Sinkhorn, Wasserstein distance khả vi) có thể thử nghiệm như một hàm mất mát thay thế trong distillation, với lưu ý quan trọng về chi phí bộ nhớ khi áp dụng trên không gian có số chiều lớn (vocab size/token count) trên phần cứng hạn chế.

9. Thí nghiệm gợi ý

1. **Tái tạo ví dụ Sinkhorn tay (mục 3.8) bằng code và mở rộng lên kích thước lớn hơn**: viết hàm Python/PyTorch hiện thực chính xác công thức (4.15) cho một bài toán $n = m = 50$ (thay vì 2), với a, b ngẫu nhiên trên simplex và ma trận chi phí C là khoảng cách Euclid bình phương giữa các điểm ngẫu nhiên trong $\mathbb{R}^2$, chạy trên GPU 8GB hoặc CPU (bài toán rất nhẹ). Đo số vòng lặp cần để vi phạm marginal ($\|P\mathbf{1}_m - a\|_1$) nhỏ hơn một ngưỡng, với vài giá trị ε khác nhau (10, 1, 0.1, 0.01). Kết quả kỳ vọng: số vòng lặp cần tăng mạnh khi ε giảm, có thể gặp lỗi số học (NaN/0) ở ε rất nhỏ nếu không dùng log-domain. Diễn giải: xác nhận trực tiếp Remark 4.6-4.7 của tài liệu (đánh đổi giữa độ chính xác xấp xỉ và tốc độ/ổn định hội tụ) trên dữ liệu tự tạo.

2. **So sánh Sinkhorn divergence vs KL divergence khi hai phân phối ít chồng lấp**: tạo hai phân phối rời rạc nhỏ (ví dụ 10 bin) có support hoàn toàn hoặc gần như rời nhau (ví dụ a đặt khối lượng ở bin 1-3, b ở bin 8-10), tính cả Sinkhorn divergence (công thức ở mục 3.9, dùng chi phí $|i - j|$ giữa chỉ số bin) và KL divergence tiêu chuẩn, thay đổi dần để hai phân phối "xích lại gần nhau" (bin chồng lấp tăng dần). Kết quả kỳ vọng: KL có thể nhảy giá trị đột ngột hoặc không định nghĩa khi support rời hoàn toàn (giống Example 1 của WGAN, math 13), còn Sinkhorn divergence thay đổi mượt theo khoảng cách bin. Diễn giải: minh hoạ trực tiếp động lực toán học cho mục 8 (lý do Wasserstein/Sinkhorn có thể ổn định hơn KL khi phân phối ít chồng lấp).

3. **Đo chi phí bộ nhớ của kernel Gibbs theo kích thước vocab**: tính kích thước bộ nhớ (bytes) cần để lưu ma trận $K \in \mathbb{R}^{n \times m}$ ở độ chính xác FP32 với n, m lần lượt bằng vocab size điển hình của một tokenizer nhỏ (ví dụ 32.000) và so với giới hạn 8GB. Kết quả kỳ vọng (tính tay/code đơn giản): $32000 \times 32000 \times 4$ bytes (tự tính: $32000^2 = 1.024 \times 10^9$, nhân 4 bytes $= 4.096 \times 10^9$ bytes ≈ 4.1 GB theo quy ước thập phân, tương đương ≈ 3.81 GiB theo quy ước nhị phân 1GiB $= 2^{30}$ bytes - cùng một số lượng byte, chỉ khác đơn vị quy đổi) - đã chiếm gần hết 8GB chỉ cho MỘT ma trận kernel của MỘT cặp mẫu, cho thấy ngay vấn đề mở rộng (scalability) nếu áp dụng trực tiếp trên vocab đầy đủ. Diễn giải: định lượng cụ thể giới hạn thực tế đã nêu ở mục 8 ý 2, giúp quyết định có cần giảm chiều (ví dụ chỉ áp dụng Sinkhorn trên top-k logit, hoặc trên embedding chiều thấp) trước khi thử nghiệm trên LightOnOCR.

4. **Kiểm chứng định lý Brenier bằng số trên ví dụ 2D nhỏ**: với hai phân phối Gaussian 2D (một có mật độ, theo đúng giả định định lý Brenier), tính OT bằng Sinkhorn với ε rất nhỏ, trích xuất coupling, và kiểm tra bằng số liệu xem ánh xạ "trung bình có điều kiện" $\hat{T}(x) = \mathbb{E}[y|x]$ suy ra từ coupling có gần với một ánh xạ affine (gradient của hàm lồi bậc 2, vì với hai Gaussian ánh xạ Brenier có dạng tuyến tính đã biết trong kiến thức chung) hay không. Kết quả kỳ vọng: với ε đủ nhỏ, ánh xạ suy ra gần với nghiệm affine lý thuyết (ngoài paper, kiến thức chung: với 2 Gaussian, ánh xạ Monge-Brenier tối ưu là một phép biến đổi tuyến tính cụ thể theo ma trận hiệp phương sai). Diễn giải: kiểm chứng thực nghiệm trực tiếp định lý 2.1 (Brenier) trên một trường hợp đơn giản có thể kiểm tra độc lập.

5. **Sinkhorn trên embedding của token OCR (thăm dò, liên hệ xa tới LightOnOCR)**: nếu có quyền truy cập embedding của một vài token văn bản/token hình ảnh từ một mô hình nhỏ tương tự, thử tính khoảng cách Sinkhorn giữa phân phối embedding của hai batch ảnh OCR khác nhau (ví dụ ảnh "rõ" vs ảnh "mờ/nén mạnh"), xem khoảng cách này có tương quan với chất lượng OCR (ví dụ character error rate) hay không. Kết quả kỳ vọng: không có kỳ vọng cụ thể từ tài liệu (đây là thí nghiệm khám phá hoàn toàn mới, không được tài liệu hay bất kỳ paper nào trong dự án đề xuất trực tiếp). Diễn giải: thí nghiệm này mang tính dò đường (exploratory) để kiểm tra liệu khoảng cách OT có thể dùng như một chỉ số chẩn đoán chất lượng representation khi nén mô hình OCR, với kết quả âm tính cũng là thông tin hữu ích.

10. Tóm tắt công thức / khái niệm

- Khoảng cách p-Wasserstein (rời rạc): $W_p(a,b) = L_{D^p}(a,b)^{1/p}$ với $L_C(a,b) = \min_{P\in U(a,b)}\langle C,P\rangle$ (công thức 2.11, 2.17).
- Tập coupling: $U(a,b) = \{P \geq 0 : P\mathbf{1}_m = a, P^\top \mathbf{1}_n = b\}$ (công thức 2.10).
- Đối ngẫu Kantorovich: $L_C(a,b) = \max_{(f,g):f_i+g_j\leq C_{i,j}} \langle f,a\rangle + \langle g,b\rangle$ (công thức 2.20).
- Định lý Brenier: với $c = \|x-y\|^2$, một đo có mật độ, coupling tối ưu hỗ trợ trên đồ thị $T = \nabla\varphi$, φ lồi.
- Công thức đóng 1D: $W_p(\alpha,\beta)^p = \frac{1}{n}\sum_i |x_i - y_i|^p$ với các điểm đã sắp thứ tự (công thức 2.33).
- Entropy: $H(P) = -\sum_{i,j} P_{i,j}(\log P_{i,j} - 1)$ (công thức 4.1).
- OT chính quy hoá entropy: $L_C^\varepsilon(a,b) = \min_{P\in U(a,b)}\langle P,C\rangle - \varepsilon H(P)$ (công thức 4.2); $\varepsilon \to 0$ cho nghiệm entropy lớn nhất của OT gốc, $\varepsilon\to\infty$ cho $P \to ab^\top$.
- Nghiệm Sinkhorn: $P_{i,j} = u_i K_{i,j} v_j$, $K_{i,j} = e^{-C_{i,j}/\varepsilon}$ (công thức 4.12).
- Lặp Sinkhorn: $u^{(\ell+1)} = a/(Kv^{(\ell)})$, $v^{(\ell+1)} = b/(K^\top u^{(\ell+1)})$ (công thức 4.15).
- Độ phức tạp Sinkhorn (τ-xấp xỉ): $O(n^2\log(n)\tau^{-3})$ phép toán (Altschuler et al. 2017).
- Gradient của OT chính quy hoá: $\nabla L_C^\varepsilon(a,b) = (f^\star, g^\star)$ (Proposition 4.6).
- IPM tổng quát: $\|\alpha\|_B = \max_{f\in B}\int f\,d\alpha$ (công thức 8.10); W^1 là trường hợp B=hàm 1-Lipschitz (Example 8.7), chính là Kantorovich-Rubinstein duality dùng trong WGAN.
- Ví dụ tự tính (mục 3.8): bài toán 2×2 đối xứng, $\varepsilon = 1$, hội tụ sau 1 vòng lặp, $P = \begin{pmatrix} 0.3655 & 0.1345 \\ 0.1345 & 0.3655 \end{pmatrix}$, kiểm tra marginal khớp $a = b = (0.5, 0.5)$ (tự tính).

11. Kết luận ngắn

Tài liệu này tổng hợp nền tảng lý thuyết (Monge, nới lỏng Kantorovich, tính chất khoảng cách Wasserstein, đối ngẫu, định lý Brenier) và thuật toán thực hành quan trọng nhất (chính quy hoá entropy và thuật toán Sinkhorn) của Optimal Transport, với trọng tâm hướng tới ứng dụng khoa học dữ liệu/machine learning. Ý tưởng cốt lõi: OT cho một cách "nâng" khoảng cách nền giữa các điểm thành khoảng cách giữa các phân phối, khắc phục hạn chế của các phân kỳ cổ điển khi hai phân phối không chồng lấp; chính quy hoá entropy biến bài toán quy hoạch tuyến tính khó thành một bài toán lồi chặt có nghiệm duy nhất, giải được bằng thuật toán Sinkhorn đơn giản, song song hoá tốt, và **khả vi** - tính chất khả vi này là cầu nối trực tiếp tới các ứng dụng machine learning hiện đại như WGAN (dùng đối ngẫu Kantorovich-Rubinstein). Ghi chú này đọc đầy đủ Chương 1, 2, phần lớn Chương 4, và một phần Chương 8 của một tài liệu dài khoảng 250 trang; các chương về thuật toán linear-programming cổ điển (Chương 3), semidiscrete OT (Chương 5), W^1 trên không gian metric/đồ thị (Chương 6), công thức động Benamou-Brenier (Chương 7), các ứng dụng biến phân (Chương 9), và các mở rộng (multimarginal, unbalanced OT, sliced Wasserstein, Gromov-Wasserstein - Chương 10) không được đọc chi tiết và chỉ được nêu tên.

Nguồn gốc: https://arxiv.org/abs/1803.00567

Bài 13

Wasserstein GAN (2017)

Tác giả: Martin Arjovsky, Soumith Chintala, Léon Bottou
Venue / arXiv: arXiv:1701.07875 (v3, 06/12/2017); paper không nêu venue hội nghị/journal cụ thể trên trang đầu (chỉ ghi tên tác giả và affiliation: Courant Institute of Mathematical Sciences, Facebook AI Research); đây là một trong các paper nền tảng được biết rộng rãi trong cộng đồng, công bố qua arXiv.
Nguồn: https://arxiv.org/abs/1701.07875
Chủ đề: Đề xuất thay khoảng cách Jensen-Shannon (JS, dùng trong GAN gốc) bằng khoảng cách Earth-Mover/Wasserstein-1 để đo sự khác biệt giữa phân phối mô hình (generator) và phân phối dữ liệu thật, giải quyết vấn đề gradient biến mất và mode collapse trong huấn luyện GAN, thông qua một thuật toán thực hành gọi là WGAN dùng Kantorovich-Rubinstein duality và weight clipping.
Vị trí trong lộ trình: nên đọc sau ml basics 14 (Generative Adversarial Networks, Goodfellow et al. 2014) để hiểu GAN gốc trước; nên đọc cùng lúc hoặc trước math 12 (Computational Optimal Transport) vì bài này dùng trực tiếp lý thuyết Kantorovich-Rubinstein duality (một kết quả cốt lõi của optimal transport) mà không tự chứng minh chi tiết; nên đọc trước math 17 (Spectral Normalization for GANs) vì cả hai cùng giải quyết vấn đề ”ép ràng buộc Lipschitz cho discriminator/critic” nhưng math 17 là một cải tiến kỹ thuật thay cho weight clipping của bài này.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper mở đầu bằng câu hỏi nền tảng của học không giám sát (unsupervised learning): ”học một phân phối xác suất nghĩa là gì?” (dòng 16-17). Cách trả lời cổ điển là học một **mật độ xác suất (density)** bằng cách định nghĩa một họ tham số các mật độ $(P_\theta)_{\theta \in \mathbb{R}^d}$ và tìm θ cực đại hoá likelihood trên dữ liệu: $\max_{\theta \in \mathbb{R}^d} \frac{1}{m} \sum_{i=1}^{m} \log P_\theta(x^{(i)})$. Nếu phân phối dữ liệu thật P_r có mật độ, việc này tương đương (khi $m \to \infty$) với tối thiểu hoá **phân kỳ Kullback-Leibler (KL)** $KL(P_r \| P_\theta)$.

Vấn đề: cách này cần **mật độ mô hình P_θ tồn tại**. Điều này không đúng trong tình huống phổ biến khi phân phối được hỗ trợ (supported) trên một **manifold chiều thấp** (low dimensional manifold) nhúng trong không gian chiều cao (ví dụ không gian ảnh). Khi đó, ”khó có khả năng manifold của mô hình và support của phân phối thật có giao nhau đáng kể” (dòng 33-34), khiến khoảng cách KL ”không được định nghĩa (hoặc đơn giản là vô hạn)” (dòng 35). Cách khắc phục thông thường là thêm nhiễu (noise) vào phân phối mô hình, nhưng paper chỉ ra chi phí thực tế của việc này: trích dẫn một paper khác cho thấy ”độ lệch chuẩn tối ưu của nhiễu thêm vào mô hình khi tối đa hoá likelihood là khoảng 0.1 cho mỗi pixel của ảnh sinh ra, khi pixel đã được chuẩn hoá về $[0, 1]$” (dòng 42-44) - ”đây là một lượng nhiễu rất lớn, đến mức khi các paper báo cáo mẫu sinh ra từ mô hình, họ không thêm hạng tử nhiễu mà họ dùng để tính likelihood” (dòng 44-46). Paper nhận xét thẳng: ”hạng tử nhiễu thêm vào rõ ràng không đúng cho bài toán, nhưng cần thiết để phương pháp maximum likelihood hoạt động” (dòng 46-48).

Cách thay thế: thay vì ước lượng mật độ P_r (có thể không tồn tại), định nghĩa một biến ngẫu nhiên Z với phân phối cố định $p(z)$, cho qua một hàm tham số $g_\theta : \mathcal{Z} \to \mathcal{X}$ (thường là một mạng neural) để **sinh trực tiếp mẫu** theo một phân phối P_θ nào đó. VAE (Variational Auto-Encoders) và GAN (Generative Adversarial Networks) là hai ví dụ nổi tiếng của cách tiếp cận này. Vì VAE tập trung vào likelihood xấp xỉ, nó vẫn chia sẻ hạn chế cần thêm nhiễu; GAN cho phép linh hoạt hơn trong việc định nghĩa hàm mục tiêu (bao gồm Jensen-Shannon và các f-divergence khác), nhưng ”huấn luyện GAN được biết rộng rãi là tinh vi và bất ổn, với lý do được điều tra về mặt lý thuyết” trong một paper trước đó của cùng tác giả thứ nhất (dòng 70-71, trích [1] Arjovsky

& Bottou 2017).

Trọng tâm của paper: các cách khác nhau để đo ”độ gần” giữa phân phối mô hình và phân phối thật, tức là các cách định nghĩa khoảng cách hoặc phân kỳ $\rho(P_\theta, P_r)$. Sự khác biệt cơ bản nhất giữa các khoảng cách này là **ảnh hưởng tới sự hội tụ của dãy phân phối xác suất**: một dãy (P_t) hội tụ nếu và chỉ nếu tồn tại P_∞ sao cho $\rho(P_t, P_\infty) \to 0$, và điều này phụ thuộc vào cách định nghĩa ρ. Paper nói đại khái (informally): ”một khoảng cách ρ tạo ra một tô-pô (topology) yếu hơn khi nó làm cho một dãy phân phối dễ hội tụ hơn” (dòng 78-79). Mục tiêu then chốt: cần phép ánh xạ $\theta \mapsto P_\theta$ **liên tục** để hàm mất mát $\theta \mapsto \rho(P_\theta, P_r)$ liên tục (có thể dùng gradient descent hiệu quả); khoảng cách yếu hơn giúp ánh xạ này dễ liên tục hơn.

2. Ý tưởng chính

Ý tưởng cốt lõi: thay khoảng cách Jensen-Shannon (JS, dùng trong GAN gốc) bằng **khoảng cách Earth-Mover (EM)**, còn gọi là **khoảng cách Wasserstein-1**, để đo sự khác biệt giữa phân phối mô hình P_θ và phân phối thật P_r. Lý do: Wasserstein là một khoảng cách **yếu hơn hẳn** JS (paper chứng minh bằng Theorem 2, xem mục 3), nghĩa là nhiều dãy phân phối hội tụ dưới Wasserstein nhưng không hội tụ dưới JS/KL/TV. Cụ thể, với hai phân phối có **support rời nhau** (disjoint support, tình huống rất phổ biến khi dữ liệu nằm trên manifold chiều thấp), Wasserstein vẫn cho một khoảng cách **liên tục và có gradient khả dụng theo tham số**, còn JS/KL/TV cho giá trị hằng số (không đổi, gradient = 0) hoặc vô hạn.

Vì tính toán trực tiếp định nghĩa gốc của khoảng cách EM (một bài toán tối ưu vận tải, infimum trên mọi joint distribution) là **cực kỳ khó** (intractable), paper dùng **đối ngẫu Kantorovich-Rubinstein (Kantorovich-Rubinstein duality)**, một định lý từ lý thuyết optimal transport, để biến đổi bài toán infimum khó thành bài toán **supremum trên các hàm 1-Lipschitz**, dễ xấp xỉ bằng một mạng neural (gọi là **critic**, không còn gọi là ”discriminator” vì nó không còn là bộ phân loại nhị phân, mà xuất ra một giá trị thực vô hướng).

Ý tưởng thứ hai, mang tính thực hành: để mạng neural xấp xỉ hàm 1-Lipschitz, paper dùng một kỹ thuật đơn giản là **weight clipping** (cắt trọng số về một khoảng cố định $[-c, c]$ sau mỗi lượt cập nhật gradient). Paper tự thừa nhận đây là ”một cách rõ ràng là tệ (clearly terrible way)” (dòng 316) để ép ràng buộc Lipschitz, nhưng chọn vì đơn giản và ”đã cho hiệu năng tốt” (dòng 322), đồng thời khuyến khích nghiên cứu tiếp để cải thiện (điều mà math 17, Spectral Normalization, thực hiện sau đó một năm).

3. Phương pháp chi tiết

3.1 Các khoảng cách/phân kỳ cơ bản (§2)

Paper định nghĩa chính thức (với $\mathcal{X}$ là tập metric compact, ví dụ không gian ảnh $[0,1]^d$, Σ là tập các Borel subset của $\mathcal{X}$, $\text{Prob}(\mathcal{X})$ là không gian các độ đo xác suất trên $\mathcal{X}$):

- **Total Variation (TV)**: $\delta(P_r, P_g) = \sup_{A \in \Sigma} |P_r(A) - P_g(A)|$.
- **Kullback-Leibler (KL)**: $KL(P_r \| P_g) = \int \log\left(\frac{P_r(x)}{P_g(x)}\right) P_r(x)\, d\mu(x)$, giả định cả hai có mật độ theo cùng một độ đo μ. Paper nhấn mạnh: ”KL nổi tiếng là bất đối xứng và có thể vô hạn khi có điểm $P_g(x) = 0$ mà $P_r(x) > 0$” (dòng 140-141).
- **Jensen-Shannon (JS)**: $JS(P_r, P_g) = KL(P_r \| P_m) + KL(P_g \| P_m)$, với $P_m = (P_r + P_g)/2$. ”Phân kỳ này đối xứng và luôn được định nghĩa vì có thể chọn $\mu = P_m$” (dòng 155-156).
- **Earth-Mover (EM) / Wasserstein-1**:

$$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|] \tag{1}$$

  với $\Pi(P_r, P_g)$ là tập mọi phân phối đồng thời (joint distribution) $\gamma(x, y)$ có hai biên (marginal) tương ứng là P_r và P_g. Giải thích trực giác (paper tự nói, dòng 164-167): ”$\gamma(x, y)$ cho biết bao nhiêu ’khối lượng’ (mass) cần được vận chuyển từ x sang y để biến đổi P_r thành P_g. Khoảng cách EM là ’chi phí’ của kế hoạch vận tải tối ưu”.

3.2 Ví dụ minh hoạ (Example 1, "Learning parallel lines")

Paper đưa ra một ví dụ cụ thể để minh hoạ sự khác biệt giữa các khoảng cách. Cho $Z \sim U[0,1]$ (phân phối đều trên đoạn đơn vị). P_0 là phân phối của $(0, Z) \in \mathbb{R}^2$ (một đường thẳng dọc đi qua gốc toạ độ). Đặt $g_\theta(z) = (\theta, z)$ với θ là một tham số thực đơn (đường thẳng dọc dịch sang phải/trái theo θ). Paper tính trực tiếp (dòng 176-188):

- $W(P_0, P_\theta) = |\theta|$.
- $JS(P_0, P_\theta) = \log 2$ nếu $\theta \neq 0$, $= 0$ nếu $\theta = 0$.
- $KL(P_\theta \| P_0) = KL(P_0 \| P_\theta) = +\infty$ nếu $\theta \neq 0$, $= 0$ nếu $\theta = 0$.
- $\delta(P_0, P_\theta) = 1$ nếu $\theta \neq 0$, $= 0$ nếu $\theta = 0$.

Khi $\theta_t \to 0$, dãy (P_{θ_t}) **hội tụ dưới khoảng cách EM** (vì $|\theta_t| \to 0$), nhưng **không hội tụ chút nào** dưới JS, KL, KL ngược, hoặc TV (vì các giá trị này nhảy đột ngột từ một hằng số dương về 0 chỉ tại $\theta = 0$, không có gì "liên tục" khi θ_t tiến dần về 0). Paper tự kiểm chứng lại bằng tay (tự tính, kiểm tra tính hợp lý): với $\theta = 0.1$ (rất gần 0 nhưng khác 0), $W = 0.1$ (nhỏ, phản ánh đúng trực giác "hai đường thẳng gần nhau"), nhưng $JS = \log 2 \approx 0.693$ (giống hệt trường hợp $\theta = 1$ rất xa, không phân biệt được mức độ gần) - đây chính là minh hoạ bằng số cụ thể cho việc JS "bão hoà" (saturate) ngay khi hai phân phối có support rời nhau, bất kể chúng gần nhau bao nhiêu về khoảng cách Euclid.

Đây là một ví dụ có **support rời nhau** (hai đường thẳng khác θ không giao nhau trừ khi $\theta = 0$); paper nói thêm "kết luận tương tự vẫn đúng khi support có giao nhau nhưng giao đó nằm trong một tập có độ đo 0" (dòng 196-198), tức là tình huống tổng quát hơn (hai manifold chiều thấp giao nhau ở vị trí tổng quát/general position) vẫn gặp cùng vấn đề.

3.3 Theorem 1 và Theorem 2 (tính liên tục và quan hệ thứ bậc giữa các khoảng cách)

Theorem 1: cho P_r cố định, $g_\theta(z)$ là một hàm (thường là mạng neural). Thì:

1. Nếu g liên tục theo θ, thì $W(P_r, P_\theta)$ cũng liên tục.
2. Nếu g liên tục Lipschitz cục bộ (locally Lipschitz) và thoả một giả định chính quy (Assumption 1, không đi sâu chi tiết kỹ thuật trong ghi chú này), thì $W(P_r, P_\theta)$ liên tục ở mọi nơi, và khả vi hầu khắp nơi (differentiable almost everywhere).
3. Hai phát biểu (1), (2) **sai** đối với $JS(P_r, P_\theta)$ và mọi KL.

Corollary 1 (hệ quả quan trọng cho thực hành): nếu g_θ là bất kỳ mạng neural feedforward (định nghĩa: "hợp các phép biến đổi affine và phi tuyến điểm-theo-điểm mượt và Lipschitz như sigmoid, tanh, elu, softplus", chú thích 4, dòng 229-232; paper ghi chú thêm "phát biểu cũng đúng với rectifier nonlinearity nhưng chứng minh kỹ thuật hơn nên bỏ qua"), và $p(z)$ là một prior sao cho $\mathbb{E}_{z \sim p(z)}[\|z\|] < \infty$ (ví dụ Gaussian, uniform), thì $W(P_r, P_\theta)$ liên tục ở mọi nơi và khả vi hầu khắp nơi. Đây chính là cơ sở lý thuyết cho việc "an toàn" khi dùng gradient descent trên Wasserstein distance với một mạng neural bất kỳ làm generator.

Theorem 2 mô tả **thứ bậc độ mạnh (relative strength)** của các tô-pô sinh ra bởi các khoảng cách này, "KL mạnh nhất, tiếp theo là JS và TV, và EM yếu nhất" (dòng 242-243):

1. $\delta(P_n, P) \to 0 \Leftrightarrow JS(P_n, P) \to 0$ (TV và JS tương đương).
2. $W(P_n, P) \to 0 \Leftrightarrow P_n \xrightarrow{D} P$ (hội tụ Wasserstein tương đương hội tụ theo phân phối, convergence in distribution).
3. $KL(P_n \| P) \to 0$ hoặc $KL(P \| P_n) \to 0 \Rightarrow$ phát biểu (1).
4. Phát biểu (1) $\Rightarrow$ phát biểu (2).

Kết luận rút ra (paper tự nói, dòng 260-262): "điều này nhấn mạnh rằng KL, JS, TV không phải là hàm mất mát hợp lý khi học các phân phối được hỗ trợ trên manifold chiều thấp. Nhưng khoảng cách EM hợp lý trong tình huống đó".

3.4 Đối ngẫu Kantorovich-Rubinstein và thuật toán WGAN (§3)

Vì infimum trong định nghĩa (1) ”cực kỳ khó tính” (highly intractable, dòng 269), paper dùng **đối ngẫu Kantorovich-Rubinstein** (trích dẫn [22], không chứng minh trong bài này, dẫn người đọc tới Appendix A để hiểu nguồn gốc trực giác):

$$W(P_r, P_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_\theta}[f(x)] \tag{2}$$

trong đó supremum lấy trên mọi hàm **1-Lipschitz** $f : \mathcal{X} \to \mathbb{R}$. Nếu thay $\|f\|_L \leq 1$ bằng $\|f\|_L \leq K$ (K-Lipschitz), ta được $K \cdot W(P_r, P_g)$ (chỉ khác một hằng số nhân). Nếu có một họ hàm tham số $\{f_w\}_{w \in \mathcal{W}}$ đều là K-Lipschitz với K nào đó, có thể giải bài toán:

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{z \sim p(z)}[f_w(g_\theta(z))] \tag{3}$$

Nếu supremum trong (2) đạt được tại một $w \in \mathcal{W}$ nào đó (một giả định mạnh, ”tương tự giả định khi chứng minh tính nhất quán (consistency) của một ước lượng”, dòng 286-287), quá trình này cho một tính toán của $W(P_r, P_\theta)$ (sai một hằng số nhân).

Theorem 3: cho P_r bất kỳ, P_θ là phân phối của $g_\theta(Z)$ thoả Assumption 1. Thì tồn tại nghiệm f cho bài toán cực đại $\max_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_\theta}[f(x)]$, và:

$$\nabla_\theta W(P_r, P_\theta) = -\mathbb{E}_{z \sim p(z)}[\nabla_\theta f(g_\theta(z))]$$

khi cả hai vế được định nghĩa tốt. Đây là cơ sở lý thuyết cho việc **lan truyền ngược trực tiếp** qua công thức đối ngẫu để huấn luyện generator.

Thuật toán WGAN cụ thể (Algorithm 1) - để f_w nằm trong không gian compact $\mathcal{W}$ (điều kiện cần cho Lipschitz với hằng số K phụ thuộc $\mathcal{W}$, không phụ thuộc trọng số cụ thể), paper dùng cách đơn giản nhất: **cắt (clamp) trọng số về một hộp cố định** $\mathcal{W} = [-c, c]^l$ sau mỗi lượt cập nhật gradient. Mã giả (paraphrase từ Algorithm 1, dòng 329-351), với giá trị mặc định dùng cho mọi thí nghiệm trong bài: learning rate $\alpha = 0.00005$, clipping parameter $c = 0.01$, batch size $m = 64$, $n_{critic} = 5$ (số lượt cập nhật critic cho mỗi lượt cập nhật generator):

Trong khi θ chưa hội tụ:

- Với $t = 0, \ldots, n_{critic}$: lấy mẫu batch dữ liệu thật $\{x^{(i)}\}$ và batch prior $\{z^{(i)}\}$; tính $g_w \leftarrow \nabla_w \left[\frac{1}{m}\sum f_w(x^{(i)}) - \frac{1}{m}\sum f_w(g_\theta(z^{(i)}))\right]$; cập nhật $w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$; cắt $w \leftarrow \text{clip}(w, -c, c)$.
- Lấy mẫu batch prior mới; tính $g_\theta \leftarrow -\nabla_\theta \frac{1}{m}\sum f_w(g_\theta(z^{(i)}))$; cập nhật $\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$.

Lưu ý quan trọng: critic được cập nhật dùng **gradient tăng** (ascent, dấu $+$), còn generator dùng **gradient giảm** trên âm của hàm mục tiêu (dấu $-\nabla_\theta$) - khác biệt dấu này phản ánh đúng cấu trúc $\max_w \min_\theta$ (hoặc tương đương) của bài toán gốc.

Paper **tự phê bình thẳng** kỹ thuật weight clipping ngay sau khi trình bày (dòng 316-325): ”Weight clipping rõ ràng là một cách tệ để ép ràng buộc Lipschitz. Nếu tham số clipping lớn, có thể mất nhiều thời gian để bất kỳ trọng số nào đạt tới giới hạn của nó, khiến việc huấn luyện critic tới tối ưu khó hơn. Nếu clipping nhỏ, điều này có thể dễ dẫn tới gradient biến mất khi số lớp lớn, hoặc khi không dùng batch normalization (như trong RNN)”. Paper cũng nói đã thử ”các biến thể đơn giản (như chiếu trọng số lên một mặt cầu) với ít khác biệt, và giữ nguyên weight clipping vì sự đơn giản và hiệu năng đã tốt” (dòng 321-322), đồng thời ”để lại chủ đề ép ràng buộc Lipschitz trong bối cảnh mạng neural cho nghiên cứu sau này, và tích cực khuyến khích các

nhà nghiên cứu quan tâm cải thiện phương pháp này" (dòng 323-325) - đây chính là lời "mời gọi" trực tiếp mà các paper sau (như Spectral Normalization, math 17) đáp lại.

3.5 Lý do huấn luyện critic tới tối ưu lại có ích (§3, cuối)

Vì khoảng cách EM "liên tục và khả vi hầu khắp nơi" (continuous and differentiable a.e.), paper lập luận "ta có thể (và nên) huấn luyện critic tới tối ưu. Lý giải đơn giản: càng huấn luyện critic nhiều, gradient của Wasserstein mà ta nhận được càng đáng tin cậy" (dòng 355-358). Ngược lại, với JS, "khi discriminator tốt hơn, gradient trở nên đáng tin cậy hơn nhưng gradient thật lại bằng 0 vì JS bị bão hoà cục bộ (locally saturated), dẫn tới vanishing gradient" (dòng 359-361). Figure 2 của paper minh hoạ: khi học phân biệt hai phân phối Gaussian, "discriminator học rất nhanh cách phân biệt fake và real, và như dự đoán không cung cấp thông tin gradient đáng tin cậy. Critic, tuy nhiên, không thể bão hoà, và hội tụ về một hàm tuyến tính cho gradient rất sạch ở mọi nơi" (dòng 363-366). Paper giải thích lý do kỹ thuật: "việc ràng buộc trọng số giới hạn mức tăng trưởng tối đa của hàm là tuyến tính ở các phần khác nhau của không gian, buộc critic tối ưu phải có hành vi này" (dòng 366-368).

Một lợi ích thứ hai, "có thể quan trọng hơn" (dòng 369): việc huấn luyện critic tới tối ưu làm **không thể xảy ra mode collapse** khi làm vậy, vì "mode collapse xuất phát từ việc generator tối ưu cho một discriminator cố định là một tổng các hàm delta tại các điểm discriminator gán giá trị cao nhất" (dòng 370-373, trích dẫn hiện tượng đã quan sát trong [4] Goodfellow 2014 gốc và nhấn mạnh lại trong [11]).

4. Thí nghiệm và kết quả chính

Thực nghiệm sinh ảnh trên **LSUN-Bedrooms** (bộ sưu tập ảnh phòng ngủ), so sánh với baseline **DCGAN** (GAN tích chập huấn luyện theo thủ tục GAN chuẩn dùng mẹo $-\log D$). Mẫu sinh ra là ảnh 3 kênh màu 64x64 pixel. Dùng đúng hyperparameter mặc định ở Algorithm 1 cho mọi thí nghiệm.

Tuyên bố hai lợi ích chính (§4): (a) một thước đo loss có ý nghĩa, tương quan với sự hội tụ của generator và chất lượng mẫu; (b) cải thiện độ ổn định của quá trình tối ưu.

4.2 Thước đo loss có ý nghĩa (Figure 3, 4): vì WGAN huấn luyện critic f tương đối tốt trước mỗi lượt cập nhật generator, hàm loss tại điểm đó là một ước lượng của khoảng cách EM (sai một hằng số nhân phụ thuộc cách ràng buộc Lipschitz của f). Paper thử 3 kiến trúc: DCGAN tích chập, generator là MLP 4 lớp ẩn 512 unit mỗi lớp (critic DCGAN), và cả generator+critic đều là MLP (learning rate cao để huấn luyện thất bại có chủ đích). Kết quả (Figure 3): "các biểu đồ cho thấy rõ các đường cong này tương quan tốt với chất lượng trực quan của mẫu sinh ra" (dòng 428-430). Paper tự nhận đây là phát hiện đáng chú ý: "theo hiểu biết của chúng tôi, đây là lần đầu tiên trong văn liệu GAN một thuộc tính như vậy được chỉ ra, nơi loss của GAN thể hiện tính chất hội tụ" (dòng 431-433). Ngược lại, Figure 4 cho JS estimate của GAN chuẩn: "cả hai đường cong đều có lỗi tăng dần. Mẫu tốt hơn cho DCGAN nhưng ước lượng JS tăng hoặc giữ không đổi, cho thấy không có tương quan đáng kể giữa chất lượng mẫu và loss" (dòng 439-441); "ước lượng JS thường vẫn gần $\log 2 \approx 0.69$, giá trị cao nhất mà khoảng cách JS đạt được" (dòng 469-471, con số $\log 2 \approx 0.693$ khớp chính xác với giá trị tính ở Example 1 mục 3.2, tự kiểm chứng: $\ln 2 \approx 0.6931$).

Paper tự cảnh báo giới hạn của tuyên bố này ngay sau (dòng 448-452): "chúng tôi không khẳng định đây là một phương pháp mới để đánh giá định lượng mô hình sinh. Hằng số tỷ lệ phụ thuộc kiến trúc của critic khiến khó so sánh các mô hình có critic khác nhau. Hơn nữa, trong thực tế việc critic không có capacity vô hạn khiến khó biết chính xác ước lượng của chúng ta gần khoảng cách EM thật đến mức nào".

Một kết quả tiêu cực paper tự báo cáo (dòng 479-487): "huấn luyện WGAN trở nên bất ổn đôi lúc khi dùng một optimizer dựa trên momentum như Adam (với $\beta_1 > 0$) trên critic, hoặc khi dùng learning rate cao. Vì loss cho critic không ổn định theo thời gian (nonstationary), các phương pháp dựa trên momentum có vẻ hoạt động kém hơn". Paper xác định nguyên nhân khả dĩ bằng quan sát thực nghiệm cụ thể: "khi loss nổ lên và mẫu xấu đi, cosine giữa bước Adam và gradient thường trở nên âm. Nơi duy nhất cosine này âm là trong các

tình huống bất ổn này" - đây là một chẩn đoán thực nghiệm cụ thể, dẫn tới quyết định "chuyển sang RMSProp, được biết hoạt động tốt ngay cả trên các bài toán rất nonstationary" (dòng 485-487).

4.3 Cải thiện độ ổn định (Figure 5, 6, 7): thử 3 kiến trúc generator: (1) DCGAN tích chập chuẩn, (2) DCGAN tích chập **không** batch normalization và số filter không đổi qua các lớp, (3) MLP ReLU 4 lớp 512 unit. Hai kiến trúc cuối "được biết hoạt động rất kém với GAN" (dòng 499-500). Kết quả: với kiến trúc (2), "GAN chuẩn thất bại trong việc học trong khi WGAN vẫn có thể tạo mẫu" (dòng 522-523, chú thích Figure 6). Với kiến trúc (3) MLP, "phương pháp WGAN vẫn có thể tạo mẫu, chất lượng thấp hơn DCGAN, và cao hơn MLP của GAN chuẩn. Lưu ý mức độ mode collapse đáng kể trong GAN MLP" (dòng 531-533, chú thích Figure 7). Kết luận tổng quát mà paper nhấn mạnh bằng chữ đậm trong văn bản gốc: "**Trong không thí nghiệm nào chúng tôi thấy bằng chứng của mode collapse cho thuật toán WGAN**" (dòng 505-506).

5. Phân tích / ablation

Paper không có một bảng "ablation" theo nghĩa thống kê định lượng (so sánh số liệu qua nhiều cấu hình trong một bảng), nhưng có hai dạng phân tích tương đương:

1. **So sánh kiến trúc generator (độ bền kiến trúc)**: Figure 5, 6, 7 chính là một ablation theo kiến trúc - cố định critic/discriminator (DCGAN) và thay đổi generator qua 3 mức độ "khó" tăng dần (DCGAN chuẩn → DCGAN không BN → MLP thuần). Kết quả cho thấy WGAN **bền (robust)** hơn hẳn GAN chuẩn khi kiến trúc generator trở nên "khó" hơn, trong khi GAN chuẩn thất bại hoàn toàn ở kiến trúc (2) và mode-collapse nặng ở kiến trúc (3).

2. **So sánh optimizer (momentum vs RMSProp)**: phát hiện ở dòng 479-487 (đã nêu ở mục 4) là một dạng ablation về lựa chọn optimizer, cho thấy tính nonstationary của bài toán WGAN nhạy với momentum, dẫn tới khuyến nghị thực hành cụ thể (dùng RMSProp, tránh Adam với $\beta_1 > 0$ hoặc learning rate cao).

Phần §5 "Related Work" cũng đóng vai trò như một phân tích lý thuyết đối chiếu (không phải ablation thực nghiệm): paper định nghĩa khung tổng quát **Integral Probability Metric (IPM)**: $d_{\mathcal{F}}(P_r, P_\theta) = \sup_{f \in \mathcal{F}} \mathbb{E}_{P_r}[f] - \mathbb{E}_{P_\theta}[f]$ (công thức 4), và chỉ ra các lựa chọn khác nhau của $\mathcal{F}$ cho các khoảng cách rất khác nhau: $\mathcal{F}$ = hàm 1-Lipschitz → Wasserstein (qua Kantorovich-Rubinstein); $\mathcal{F}$ = hàm đo được bị chặn trong $[-1, 1]$ → **total variation** δ (paper nhấn mạnh: "điều này cho thấy đi từ 1-Lipschitz sang 1-Bounded thay đổi mạnh tô-pô của không gian", dòng 560-562); Energy-based GAN (EBGAN) được paper chứng minh (Appendix D, không đọc chi tiết trong ghi chú này) tương đương với cách tiếp cận sinh dựa trên total variation, nên "sẽ gặp cùng vấn đề của GAN cổ điển về việc không thể huấn luyện discriminator tới tối ưu" (dòng 572-575); Maximum Mean Discrepancy (MMD) là trường hợp $\mathcal{F} = \{f \in \mathcal{H} : \|f\|_\infty \leq 1\}$ cho một không gian Hilbert tái tạo nhân (RKHS) - paper phân tích chi tiết chi phí tính toán MMD tăng theo bậc hai với số mẫu, và tính toán cụ thể "cho ảnh chuẩn 64x64, cần minibatch kích thước ít nhất 4096... chi phí mỗi vòng lặp là 4096^2, hơn 5 bậc độ lớn so với một vòng lặp GAN dùng batch size chuẩn 64" (dòng 620-625, tự kiểm chứng: $4096^2 = 16.777.216$, so với $64^2 = 4096$ của MMD ở batch 64 chuẩn, tỷ lệ $4096/1 \approx 4096$ lần, xấp xỉ $\log_{10}(4096) \approx 3.6$ bậc độ lớn chỉ riêng từ kích thước batch, cộng thêm hệ số khác theo paper nói "hơn 5 bậc"; tự tính).

6. Giới hạn

Các giới hạn **paper tự nêu rõ**, không phải suy diễn của người ghi chú:

1. Weight clipping "rõ ràng là một cách tệ" để ép ràng buộc Lipschitz (dòng 316): clipping parameter lớn → chậm đạt giới hạn, khó huấn luyện critic tới tối ưu; clipping nhỏ → dễ vanishing gradient khi mạng sâu hoặc không dùng batch normalization (ví dụ RNN).
2. Paper tự nhận: "chúng tôi không khẳng định đây là một phương pháp mới để đánh giá định lượng mô hình sinh" (dòng 448-449) - loss WGAN chỉ tương quan tốt với chất lượng mẫu, không phải một chỉ

số tuyệt đối có thể so sánh giữa các mô hình có kiến trúc critic khác nhau, vì hằng số tỷ lệ (scaling constant) phụ thuộc kiến trúc.

3. "Trong thực tế việc critic không có capacity vô hạn khiến khó biết chính xác ước lượng của chúng ta gần khoảng cách EM thật đến mức nào" (dòng 451-452) - đây là một giới hạn nhận thức luận quan trọng: WGAN chỉ tính được một **xấp xỉ** của Wasserstein distance thật, không phải giá trị chính xác.
4. Huấn luyện WGAN có thể bất ổn khi dùng optimizer dựa trên momentum (Adam với $\beta_1 > 0$) hoặc learning rate cao trên critic (đã nêu ở mục 4, "kết quả tiêu cực" paper tự báo cáo).
5. Giả định "supremum trong (2) đạt được tại một $w \in \mathcal{W}$ nào đó" để công thức (3) tính đúng $W(P_r, P_\theta)$ (sai hằng số nhân) được paper tự gọi là "một giả định khá mạnh, tương tự giả định khi chứng minh tính nhất quán của một ước lượng" (dòng 286-287) - không được chứng minh luôn đúng trong thực hành với mạng neural cụ thể.
6. Về mặt lý thuyết, Corollary 1 chỉ chứng minh cho activation "mượt và Lipschitz" (sigmoid, tanh, elu, softplus); với rectifier (ReLU), paper nói "phát biểu cũng đúng nhưng chứng minh kỹ thuật hơn (dù rất tương tự) nên bỏ qua" (chú thích 4, dòng 232-233) - tức chứng minh đầy đủ cho ReLU **không có trong bài báo này**, chỉ được khẳng định bằng lời.
7. Paper tự để lại "chủ đề ép ràng buộc Lipschitz trong bối cảnh mạng neural cho nghiên cứu sau này" (dòng 323-324) - weight clipping được paper trình bày như một giải pháp tạm, không phải giải pháp cuối cùng.

7. Liên hệ với các bài khác

- **ml basics content/14_2014_gan.md** (đã xác nhận tồn tại bằng ls): GAN gốc (Goodfellow et al. 2014) là nền tảng trực tiếp mà bài này cải tiến; GAN gốc dùng JS divergence (qua công thức cross-entropy chuẩn), bài này thay bằng Wasserstein distance để giải quyết vấn đề gradient biến mất khi discriminator "quá tốt".
- **math 12 (Computational Optimal Transport)**: đang được viết song song theo số quy ước trong titles.tsv (chưa có file khi viết ghi chú này). Bài hiện tại dùng trực tiếp đối ngẫu Kantorovich-Rubinstein (công thức 2) như một "hộp đen" lý thuyết (không tự chứng minh, dẫn người đọc tới Appendix A và trích dẫn [22]); math 12 là nơi lý thuyết optimal transport tổng quát (bao gồm chính định lý Kantorovich-Rubinstein duality) được trình bày đầy đủ hơn.
- **math 17 (Spectral Normalization for GANs, 1802.05957)**: đã có file tại "math content/17_2018_spectral_normalization_gan.md" (xác nhận bằng ls). Math 17 chính là lời đáp trực tiếp cho lời "kêu gọi nghiên cứu tiếp" của bài hiện tại (dòng 323-325): thay weight clipping (gây rank-deficiency theo phân tích của math 17 §3) bằng chuẩn hoá giá trị kỳ dị lớn nhất, đạt ràng buộc Lipschitz hiệu quả hơn.
- **distillation content/02_2015_distilling_knowledge_hinton.md** (đã xác nhận tồn tại bằng ls): không trực tiếp cùng chủ đề, nhưng liên hệ ở mục 8 (dùng ý tưởng Wasserstein/Earth-Mover distance thay KL khi đo khoảng cách giữa phân phối đầu ra của teacher/student).
- **ml basics content/15_2013_vae_aevb.md** (đã xác nhận tồn tại bằng ls): VAE là mô hình sinh thay thế dùng likelihood xấp xỉ (ELBO), được paper này nhắc tới ở phần Bối cảnh (dòng 64-66) như một ví dụ khác của cách tiếp cận "sinh mẫu qua hàm g_θ", chia sẻ hạn chế cần thêm nhiễu mà GAN/WGAN không gặp phải.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ nội dung mục này là suy luận, paper không nói về LightOnOCR hay về distillation/nén mô hình ngôn ngữ-hình ảnh.

Dự án người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, GPU $\leq$ 8GB, batch 1 khi suy luận). Mô hình này **không phải GAN**: không có generator/discriminator

đối nghịch, không có bài toán min-max. Cần nêu rõ: **phần lớn nội dung thực nghiệm của paper này (sinh ảnh LSUN-Bedrooms bằng GAN) ít liên quan trực tiếp** tới công việc nén LightOnOCR.

Tuy nhiên có một liên hệ ý tưởng hợp lý (suy luận, không phải kỹ thuật paper tự đề xuất cho bối cảnh này): trong distillation (nén qua ”dạy” từ teacher sang student, chủ đề đã có thư mục riêng, không lặp lại chi tiết kỹ thuật ở đây), cách phổ biến để đo ”khoảng cách” giữa phân phối đầu ra (logits/xác suất token) của teacher và student là **KL divergence** (ví dụ trong soft-label distillation kiểu Hinton). Bài học cốt lõi của WGAN (mục 3.2, 3.3) là: khi hai phân phối có **ít chồng lấp (overlap)** - tình huống rất có thể xảy ra ở **đầu quá trình huấn luyện** khi student còn khởi tạo xa với teacher, hoặc khi nén mạnh (aggressive compression) khiến phân phối đầu ra student lệch nhiều so với teacher - KL divergence có thể cho giá trị rất lớn hoặc không ổn định, trong khi một khoảng cách kiểu Wasserstein/Earth-Mover vẫn cho một giá trị ”mượt” và gradient hữu ích, vì nó đo *khoảng cách hình học* giữa hai phân phối (dùng khoảng cách giữa các giá trị token/embedding) thay vì chỉ đo *tỷ lệ mật độ*.

Áp dụng cụ thể (suy luận, mức độ chắc chắn trung bình): nếu trong quá trình nén LightOnOCR gặp hiện tượng huấn luyện bất ổn ở giai đoạn đầu (ví dụ do student mới cắt/lượng tử hoá có đầu ra ban đầu quá khác teacher), có thể thử một hàm mất mát distillation dựa trên khoảng cách kiểu Wasserstein-1 giữa phân phối xác suất đầu ra (ví dụ qua Sinkhorn/entropic OT, chủ đề của math 12) thay cho hoặc kết hợp với KL divergence thông thường, như một biện pháp ổn định huấn luyện ở giai đoạn đầu. Đây **không phải** kỹ thuật được paper hiện tại hay bất kỳ paper distillation nào trong dự án trực tiếp đề xuất; đây là một suy luận loại suy (analogy) từ bài học ”Wasserstein mượt hơn KL khi support rời nhau” sang bối cảnh distillation, cần kiểm chứng thực nghiệm riêng trước khi áp dụng.

Kết luận mục này (suy luận): bài này **ít liên quan trực tiếp** tới việc nén/hậu huấn luyện LightOnOCR vì bối cảnh (huấn luyện đối nghịch GAN sinh ảnh) khác hẳn (nén một VLM OCR không đối nghịch); giá trị chính là cung cấp **khung lý thuyết về việc lựa chọn khoảng cách phân phối phù hợp** (đặc biệt khi hai phân phối có thể không chồng lấp nhiều), có thể gợi ý một hướng thử nghiệm cho hàm mất mát distillation, nhưng không nên áp dụng trực tiếp thuật toán WGAN (weight clipping, critic đối nghịch) vào pipeline nén.

9. Thí nghiệm gợi ý

1. **Tái tạo Example 1 (hai đường thẳng song song) bằng tay và bằng code**: viết hàm Python tính cả 4 khoảng cách (EM/Wasserstein, JS, KL, TV) cho hai phân phối P_0 và P_θ như định nghĩa ở mục 3.2, với θ chạy từ 0.001 đến 1, vẽ biểu đồ 4 đường. Kết quả kỳ vọng: đường Wasserstein là một đường thẳng $|\theta|$ liên tục qua gốc; các đường JS/KL/TV có bước nhảy gián đoạn ngay tại $\theta = 0^+$. Diễn giải: minh hoạ trực quan, tự làm, cho đúng bằng chứng cốt lõi của paper (Theorem 1-2) mà không cần GPU, chạy trên CPU trong vài giây.

2. **Toy GAN 1D/2D: JS GAN vs WGAN trên hai phân phối không chồng lấp**: tạo dữ liệu thật là hỗn hợp 2 Gaussian hẹp cách xa nhau trong không gian 2D, generator là MLP nhỏ, huấn luyện cả hai phiên bản (GAN chuẩn dùng JS, và WGAN dùng critic + weight clipping) trên GPU 8GB hoặc CPU (bài toán rất nhỏ). Theo dõi gradient của generator qua các bước đầu huấn luyện. Kết quả kỳ vọng: GAN chuẩn có gradient rất nhỏ/không ổn định ở giai đoạn đầu (khi hai phân phối rời xa), WGAN cho gradient ổn định hơn và giảm đều. Diễn giải: tái hiện trực tiếp phát hiện cốt lõi của Figure 2 trong paper (discriminator bão hoà vs critic cho gradient sạch) ở quy mô tự kiểm soát hoàn toàn.

3. **Kiểm tra tương quan loss-chất lượng mẫu (tái hiện Figure 3-4 ở quy mô nhỏ)**: huấn luyện WGAN và DCGAN chuẩn trên MNIST (nhỏ, nhanh trên GPU 8GB), theo dõi đồng thời: (a) giá trị loss/ước lượng khoảng cách của mỗi thuật toán qua các epoch, (b) một chỉ số chất lượng mẫu đơn giản (ví dụ accuracy của một bộ phân loại MNIST có sẵn khi phân loại ảnh sinh ra, hoặc đơn giản là xem mẫu bằng mắt theo epoch). Kết quả kỳ vọng: với WGAN, đường loss giảm dần tương quan với chất lượng mẫu tăng dần; với GAN chuẩn, loss discriminator có thể không tương quan rõ (như Figure 4 của paper).

Diễn giải: xác nhận hoặc phản biện tuyên bố ”thước đo loss có ý nghĩa” của paper trên một bài toán nhỏ hơn hẳn (MNIST thay LSUN-Bedrooms).

4. **Ảnh hưởng của clipping constant** c: huấn luyện WGAN trên một bài toán toy (ví dụ MNIST hoặc dữ liệu 2D tổng hợp) với vài giá trị c khác nhau (ví dụ 0.001, 0.01, 0.1, 1.0), đo tốc độ hội tụ và độ ổn định huấn luyện (có ”nổ” loss hay không). Kết quả kỳ vọng: c quá nhỏ làm chậm học (giống cảnh báo ”vanishing gradient” của paper ở dòng 319-320); c quá lớn làm critic mất nhiều lượt cập nhật để ”chạm” giới hạn (giống cảnh báo dòng 317-318). Diễn giải: xác nhận trực tiếp bằng thực nghiệm chính lời tự phê bình của paper về weight clipping, và cho thấy tại sao cần một kỹ thuật tốt hơn (dẫn tới động lực học math 17).

5. **So sánh Adam vs RMSProp cho critic WGAN**: lặp lại thí nghiệm nhỏ ở mục 2/3, huấn luyện critic bằng Adam (với $\beta_1 = 0.9$, giống mặc định phổ biến) và bằng RMSProp (không momentum), theo dõi độ ổn định (loss có ”nổ” đột ngột hay không), và nếu có điều kiện, tính cosine similarity giữa bước cập nhật Adam và gradient thô như paper đã làm (dòng 483-485) để tìm mối liên hệ giữa cosine âm và thời điểm bất ổn. Kết quả kỳ vọng: cấu hình Adam với $\beta_1 > 0$ dễ bất ổn hơn RMSProp trên bài toán WGAN, nhất là ở learning rate cao. Diễn giải: tái hiện trực tiếp ”kết quả tiêu cực” mà paper tự báo cáo (dòng 479-487), giúp hiểu rõ bản chất ”nonstationary” của bài toán tối ưu WGAN.

10. Tóm tắt công thức / khái niệm

- Khoảng cách Earth-Mover/Wasserstein-1: $W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|]$ (công thức 1).
- Đối ngẫu Kantorovich-Rubinstein: $W(P_r, P_\theta) = \sup_{\|f\|_L \le 1} \mathbb{E}_{P_r}[f(x)] - \mathbb{E}_{P_\theta}[f(x)]$ (công thức 2); với K-Lipschitz, kết quả nhân thêm K.
- Mục tiêu thực hành WGAN: $\max_{w \in \mathcal{W}} \mathbb{E}_{x\sim P_r}[f_w(x)] - \mathbb{E}_{z\sim p(z)}[f_w(g_\theta(z))]$ (công thức 3).
- Gradient của Wasserstein theo tham số generator (Theorem 3): $\nabla_\theta W(P_r, P_\theta) = -\mathbb{E}_{z\sim p(z)}[\nabla_\theta f(g_\theta(z))]$.
- Weight clipping: $w \leftarrow \text{clip}(w, -c, c)$ sau mỗi lượt cập nhật, giữ $w \in \mathcal{W} = [-c, c]^l$ compact (siêu tham số mặc định $c = 0.01$, $\alpha = 0.00005$, $m = 64$, $n_{critic} = 5$).
- Integral Probability Metric (IPM) tổng quát: $d_{\mathcal{F}}(P_r, P_\theta) = \sup_{f\in\mathcal{F}} \mathbb{E}_{P_r}[f] - \mathbb{E}_{P_\theta}[f]$ (công thức 4); $\mathcal{F}$ = 1-Lipschitz cho Wasserstein, $\mathcal{F}$ = bị chặn $[-1, 1]$ cho total variation.
- Ví dụ tự kiểm chứng (Example 1 của paper, mục 3.2): với $\theta = 0.1$, $W = 0.1$ nhưng $JS = \log 2 \approx 0.6931$ (giống $\theta = 1$), minh hoạ JS ”bão hoà” bất kể mức độ gần của hai phân phối có support rời nhau (tự tính xác nhận giá trị $\ln 2 \approx 0.6931$ khớp số paper nêu ≈ 0.69).
- Thứ bậc độ mạnh tô-pô (Theorem 2): KL (mạnh nhất) $\Rightarrow$ (TV $\Leftrightarrow$ JS) $\Rightarrow$ (Wasserstein $\Leftrightarrow$ hội tụ theo phân phối, yếu nhất).

11. Kết luận ngắn

Paper đề xuất thay khoảng cách Jensen-Shannon trong GAN chuẩn bằng khoảng cách Earth-Mover/Wasserstein-1, chứng minh bằng lý thuyết (Theorem 1-2, Example 1) rằng Wasserstein tạo ra một tô-pô yếu hơn hẳn, giữ được tính liên tục/khả vi của hàm mất mát ngay cả khi hai phân phối có support rời nhau (tình huống phổ biến khi học manifold chiều thấp). Nhờ đối ngẫu Kantorovich-Rubinstein, bài toán tối thiểu hoá Wasserstein khó tính trực tiếp được chuyển thành bài toán tối đa hoá trên hàm 1-Lipschitz, xấp xỉ bằng một mạng ”critic” và ép Lipschitz bằng weight clipping (kỹ thuật mà paper tự nhận là chưa tối ưu). Thực nghiệm trên LSUN-Bedrooms xác nhận hai lợi ích chính: loss có ý nghĩa tương quan với chất lượng mẫu (lần đầu trong văn liệu GAN theo paper tự nhận), và độ ổn định cải thiện rõ rệt, không quan sát thấy mode collapse trong bất kỳ thí nghiệm nào, dù vẫn còn hạn chế về độ ổn định khi dùng optimizer có momentum.

Nguồn gốc: https://arxiv.org/abs/1701.07875

Bài 14

Deep Variational Information Bottleneck (2016)

Tác giả: Alexander A. Alemi, Ian Fischer, Joshua V. Dillon, Kevin Murphy (Google Research).
Venue / arXiv: arXiv:1612.00410 (v7, 23/10/2019); paper tự ghi ”Published as a conference paper at ICLR 2017” ở đầu mỗi trang.
Nguồn: https://arxiv.org/abs/1612.00410
Chủ đề: xây dựng một cận dưới biến phân (variational lower bound) khả vi cho mục tiêu Information Bottleneck (IB) của Tishby et al. (1999), cho phép huấn luyện bằng mạng nơ-ron sâu và reparameterization trick; mục tiêu là học một encoding ngẫu nhiên Z vừa nén thông tin dư thừa từ input X vừa giữ thông tin cần cho nhãn Y.
Vị trí trong lộ trình: đọc sau math 11 (SGD as Approximate Bayesian Inference) và math 10 (Hamiltonian Monte Carlo), vì ba bài cùng nằm trong nhóm ”suy luận xấp xỉ/biến phân cho deep learning”, nhưng bài này áp mục tiêu biến phân vào một hàm mất mát huấn luyện mới (IB) chứ không áp vào việc lấy mẫu hậu nghiệm của tham số; đọc cùng nhóm với ml basics 15 (Auto-Encoding Variational Bayes, VAE) vì VIB dùng lại nguyên khối ”encoder ngẫu nhiên + reparameterization trick + KL divergence tới prior” của VAE nhưng trong bài toán có giám sát (supervised) thay vì sinh dữ liệu (generative); nên đọc cùng `pruning content/` và `interp content/` vì khái niệm ”nén biểu diễn trung gian nhưng giữ thông tin cho nhãn” là một khung lý thuyết hữu ích để nghĩ về việc cắt bớt token/kênh trong VLM.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper nhìn mạng nơ-ron sâu qua lăng kính lý thuyết thông tin (information theoretic view): biểu diễn nội bộ của một lớp trung gian được coi là một **encoding ngẫu nhiên** Z của input X, sinh ra bởi một encoder có tham số $p(z|x;\theta)$. ”Mutual information” (thông tin tương hỗ, viết $I(\cdot,\cdot)$) giữa hai biến ngẫu nhiên đo lượng thông tin mà biết biến này giảm được bao nhiêu độ bất định về biến kia; công thức (1) của paper:

$$I(Z,Y;\theta) = \int dx\, dy\; p(z,y|\theta)\; \log \frac{p(z,y|\theta)}{p(z|\theta)p(y|\theta)}.$$

(Ghi chú: trong tích phân trên z cũng được tích hợp ẩn, công thức paper viết hơi rút gọn; ý nghĩa là $I(Z,Y) = \mathbb{E}_{p(z,y)} \log \frac{p(z,y)}{p(z)p(y)}$, tích phân trên cả y và z.)

Mục tiêu tưởng chừng hợp lý: học encoding Z **tối đa hoá** $I(Z,Y;\theta)$, thông tin tương hỗ giữa encoding và nhãn Y. Nhưng paper chỉ ra một cái bẫy: theo *data processing inequality* (thông tin không thể tăng khi đi qua một hàm xử lý) và tính bất biến của mutual information qua các phép biến đổi khả nghịch (reparameterization), nếu chỉ tối đa hoá $I(Z,Y)$ mà không có ràng buộc gì khác, cách ”tối ưu” nhất đơn giản là lấy $Z = X$ (encoding đồng nhất, identity encoding): giữ nguyên toàn bộ input. Đây không phải một biểu diễn hữu ích (không nén được gì).

Do đó cần thêm một **ràng buộc về độ phức tạp (complexity constraint)** của encoding: giới hạn $I(X,Z) \le I_c$, với I_c gọi là information constraint. Mục tiêu đầy đủ (công thức 2):

$$\max_{\theta} I(Z,Y;\theta) \quad \text{s.t.} \quad I(X,Z;\theta) \le I_c.$$

Dùng Lagrange multiplier β, bài toán ràng buộc trên tương đương tối đa hoá (công thức 3):

$$R_{IB}(\theta) = I(Z, Y; \theta) - \beta I(Z, X; \theta).$$

Đây chính là mục tiêu **Information Bottleneck (IB)** gốc của Tishby et al. (1999): số hạng thứ nhất khuyến khích Z dự đoán tốt Y (expressive), số hạng thứ hai khuyến khích Z "quên" X (compressive); $\beta \geq 0$ kiểm soát độ đánh đổi (tradeoff). Trực giác: Z bị ép trở thành một dạng **minimal sufficient statistic** của X cho việc dự đoán Y (thống kê đủ, tối thiểu).

Lưu ý quy ước dấu (paper, chú thích 3): ở đây β lớn cho ra encoding **nén nhiều hơn** (compressed nhiều); một số tài liệu IB khác viết mục tiêu theo chiều ngược (min $I(Z, X) - \beta I(Z, Y)$), lúc đó β lớn lại tương ứng *ít* nén hơn. Đây là một điểm dễ nhầm giữa các paper IB khác nhau, không phải mâu thuẫn nội tại của bài này.

Vấn đề tính toán: mutual information nói chung khó tính (computationally challenging). Paper chỉ nêu hai trường hợp đã biết giải được trước đây: (a) khi X, Y, Z đều rời rạc (như trong Tishby et al. 1999, dùng để gom cụm từ - cluster discrete data như words), giải bằng thuật toán lặp **Blahut-Arimoto**; (b) khi X, Y, Z cùng Gaussian liên hợp (Chechik et al., 2005). Cả hai đều giới hạn nghiêm trọng lớp mô hình có thể học. Blahut-Arimoto cũng được nêu ở Mục 2 (Related Work) là "infeasible to apply to deep neural networks" vì tính lặp của nó.

Đóng góp của bài: dùng **variational inference** để xây một cận dưới (lower bound) khả tính cho R_{IB} ở công thức (3), rồi dùng **reparameterization trick** (Kingma & Welling, 2014) để lấy gradient Monte Carlo không chệch (unbiased), tối ưu bằng SGD với mạng nơ-ron sâu, xử lý được dữ liệu liên tục chiều cao (ảnh) mà không cần giả định rời rạc hay Gaussian liên hợp.

2. Ý tưởng chính

Ý tưởng cốt lõi gồm ba bước:

1. **Chặn dưới số hạng** $I(Z, Y)$ (số hạng muốn tối đa hoá) bằng cách đưa vào một **decoder biến phân** $q(y|z)$ xấp xỉ phân phối $p(y|z)$ thật (vốn bất khả thi vì phải tích phân qua X). Vì $KL[p(Y|Z), q(Y|Z)] \geq 0$ luôn đúng, ta có $I(Z, Y) \geq \mathbb{E}_{p(x,y,z)}[\log q(y|z)] + H(Y)$ (bỏ qua entropy nhãn $H(Y)$ vì không phụ thuộc tham số tối ưu).
2. **Chặn trên số hạng** $I(Z, X)$ (số hạng muốn tối thiểu hoá) bằng cách đưa vào một **marginal biến phân** $r(z)$ xấp xỉ marginal thật $p(z) = \int dx\, p(z|x)p(x)$ (cũng bất khả thi tính trực tiếp vì phải tích phân qua toàn bộ X). Vì $KL[p(Z), r(Z)] \geq 0$, ta có $I(Z, X) \leq \mathbb{E}_{p(x,z)}[\log \frac{p(z|x)}{r(z)}]$.
3. **Ghép hai chặn lại** thành một cận dưới duy nhất L cho toàn bộ R_{IB} (vì trừ đi một chặn trên thì được một chặn dưới), rồi dùng **reparameterization trick** để lấy mẫu $z = f(x, \epsilon)$ khả vi theo tham số encoder, cho phép backpropagate qua mẫu ngẫu nhiên và tối ưu bằng SGD.

Khác với VAE (ml basics 15), ở đây không có số hạng likelihood cho X (vì bài toán là discriminative, có giám sát, không cần tái tạo lại input), và vai trò của β không cố định bằng 1 mà là một siêu tham số cần quét để kiểm soát độ nén.

3. Phương pháp chi tiết

3.1 Giả định cấu trúc (Mục 3, công thức 5)

Paper giả định phân phối chung factorizes theo **Markov chain** $Y \leftrightarrow X \leftrightarrow Z$:

$$p(X, Y, Z) = p(Z|X, Y)p(Y|X)p(X) = p(Z|X)p(Y|X)p(X),$$

nghĩa là $p(Z|X, Y) = p(Z|X)$: **encoding** Z **không được phép phụ thuộc trực tiếp vào nhãn** Y, chỉ qua X. Ràng buộc này mở đường cho việc học biểu diễn không giám sát (Phụ lục B). Thành phần duy nhất cần

mô hình hoá là encoder $p(Z|X)$; mọi phân phối khác suy ra từ nó và cấu trúc Markov chain.

3.2 Chặn dưới cho $I(Z,Y)$

Viết lại (công thức 6):

$$I(Z,Y) = \int dy\, dz\, p(y,z) \log \frac{p(y,z)}{p(y)p(z)} = \int dy\, dz\, p(y,z) \log \frac{p(y|z)}{p(y)}.$$

$p(y|z)$ được xác định đầy đủ bởi encoder và Markov chain (công thức 7): $p(y|z) = \int dx\, \frac{p(y|x)p(z|x)p(x)}{p(z)}$, nhưng **bất khả thi tính trực tiếp** vì mẫu số $p(z)$ cũng cần tích phân qua X. Đặt $q(y|z)$ (decoder, một mạng nơ-ron riêng) là xấp xỉ biến phân cho $p(y|z)$. Dùng $KL[p(Y|Z), q(Y|Z)] \geq 0$ (bất đẳng thức Gibbs), paper suy ra (công thức 8-11):

$$I(Z,Y) \geq \int dy\, dz\, p(y,z) \log q(y|z) + H(Y).$$

Thay $p(y,z) = \int dx\, p(x)p(y|x)p(z|x)$ (dùng giả định Markov), ta được cận dưới chỉ cần mẫu từ dữ liệu và từ encoder (công thức 12):

$$I(Z,Y) \geq \int dx\, dy\, dz\; p(x)p(y|x)p(z|x)\; \log q(y|z).$$

3.3 Chặn trên cho $I(Z,X)$

Viết lại (công thức 13): $I(Z,X) = \int dz\, dx\, p(x,z) \log p(z|x) - \int dz\, p(z) \log p(z)$. Số hạng thứ hai cần marginal $p(z) = \int dx\, p(z|x)p(x)$, cũng khó tính. Đặt $r(z)$ là xấp xỉ biến phân cho marginal này; dùng $KL[p(Z), r(Z)] \geq 0$ để thay $p(z)$ bằng $r(z)$ ở chiều bất lợi (tạo ra một **chặn trên**, công thức 14):

$$I(Z,X) \leq \int dx\, dz\; p(x)p(z|x) \log \frac{p(z|x)}{r(z)}.$$

3.4 Cận dưới toàn bộ và hàm mất mát thực dụng

Ghép hai chặn (công thức 15):

$$I(Z,Y) - \beta I(Z,X) \geq \int dx\, dy\, dz\; p(x)p(y|x)p(z|x) \log q(y|z) - \beta \int dx\, dz\; p(x)p(z|x) \log \frac{p(z|x)}{r(z)} = L.$$

Thay phân phối dữ liệu thật $p(x,y)$ bằng phân phối thực nghiệm (empirical distribution) trên N mẫu huấn luyện, $p(x,y) = \frac{1}{N} \sum_n \delta_{x_n}(x)\delta_{y_n}(y)$ (công thức 16):

$$L \approx \frac{1}{N} \sum_{n=1}^{N} \int dz \Big[p(z|x_n) \log q(y_n|z) - \beta\, p(z|x_n) \log \frac{p(z|x_n)}{r(z)} \Big].$$

Lưu ý: số hạng thứ hai trong công thức này chính là $\beta\, KL[p(Z|x_n), r(Z)]$ viết dưới dạng tích phân.

Tham số hoá cụ thể: giả sử encoder dạng Gaussian $p(z|x) = \mathcal{N}(z \mid f_e^{\mu}(x), f_e^{\Sigma}(x))$, với f_e là một MLP xuất ra cả mean μ (K chiều) và ma trận hiệp phương sai Σ ($K \times K$). Áp dụng **reparameterization trick** (Kingma & Welling, 2014): viết $p(z|x)\, dz = p(\epsilon)\, d\epsilon$ với $z = f(x, \epsilon)$ là hàm xác định (deterministic) của x và biến ngẫu nhiên Gaussian ϵ. Vì nhiễu ϵ độc lập với tham số mô hình, có thể lấy gradient dễ dàng qua z.

Giả sử $p(z|x)$ và $r(z)$ chọn sao cho KL divergence giữa chúng có dạng giải tích (analytic KL, ví dụ cả hai Gaussian), paper đưa ra hàm mất mát cần **tối thiểu hoá** (công thức 17):

$$J_{IB} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{E}_{\epsilon\sim p(\epsilon)}\big[-\log q(y_n \mid f(x_n,\epsilon))\big] + \beta\, KL\big[p(Z|x_n), r(Z)\big].$$

Số hạng thứ nhất là cross-entropy giữa nhãn thật và dự đoán của decoder qua một mẫu ngẫu nhiên $z = f(x_n, \epsilon)$ (giống loss phân loại chuẩn, nhưng qua một bước lấy mẫu); số hạng thứ hai là KL divergence giữa phân phối encoding và prior $r(z)$, nhân với β: **chính xác cùng cấu trúc với loss VAE** (reconstruction + β·KL), nhưng số hạng "reconstruction" ở đây là phân loại nhãn y thay vì tái tạo x.

Chú thích 4 của paper: nếu encoder/prior không cho KL giải tích, vẫn có thể reparameterize qua một mẫu của chính divergence đó (Kingma & Welling, 2014; Blundell et al., 2015), nên công thức (17) không bắt buộc phải Gaussian.

Ví dụ số nhỏ minh hoạ reparameterization trick (tự tính): giả sử với một ảnh x_n, encoder cho ra $\mu = 2.0$, $\sigma = 0.5$ (trường hợp $K = 1$ chiều để đơn giản). Lấy một mẫu nhiễu chuẩn $\epsilon = 0.3$ (từ $\mathcal{N}(0,1)$), reparameterization cho $z = \mu + \sigma \cdot \epsilon = 2.0 + 0.5 \times 0.3 = 2.15$ (tự tính). Vì z là hàm khả vi của μ, σ (tham số encoder) với ϵ cố định/không phụ thuộc tham số, gradient $\partial z/\partial\mu = 1$ và $\partial z/\partial\sigma = \epsilon = 0.3$ (tự tính) tính được trực tiếp bằng quy tắc chuỗi (chain rule) qua decoder, không cần đạo hàm qua một phép lấy mẫu "không khả vi".

Ví dụ nhỏ minh hoạ mutual information rời rạc (tự tính): xét Z, Y rời rạc nhị phân với phân phối chung $p(z=0,y=0) = 0.4$, $p(z=0,y=1) = 0.1$, $p(z=1,y=0) = 0.1$, $p(z=1,y=1) = 0.4$ (tự tính, ví dụ minh hoạ, không phải số trong paper). Marginal: $p(z=0) = p(z=1) = 0.5$, $p(y=0) = p(y=1) = 0.5$. Mutual information: $I(Z,Y) = \sum_{z,y} p(z,y)\log_2 \frac{p(z,y)}{p(z)p(y)} = 2\times 0.4\log_2\frac{0.4}{0.25} + 2\times 0.1\log_2\frac{0.1}{0.25}$ $\approx 2\times 0.4\times 0.678 + 2\times 0.1\times(-1.322) \approx 0.542 - 0.264 = 0.278$ bit (tự tính). Số này minh hoạ cách tính $I(Z,Y)$ ở công thức (1) khi Z, Y rời rạc hữu hạn, trường hợp mà paper nói đã giải được bằng Blahut-Arimoto nhưng không mở rộng được cho deep network.

4. Thí nghiệm và kết quả chính

4.1 MNIST, bottleneck $K = 256$ (Mục 4.1.1)

Kiến trúc: MLP $784-1024-1024-10$, ReLU, giống kiến trúc trong (Pereyra et al., 2017) để có "headroom" so sánh. Baseline (không regularization) đạt lỗi **1.38%**. Encoder VIB: MLP $784-1024-1024-2K$ ($K = 256$), nửa đầu output là μ, nửa sau là σ (qua softplus). Decoder: hồi quy logistic (softmax) đơn giản từ z ra 10 lớp. $r(z)$ cố định là Gaussian cầu (spherical) K chiều, $\mathcal{N}(0, I)$. Huấn luyện 200 epoch, TensorFlow, Adam, learning rate 10^{-4} (chi tiết đầy đủ ở Phụ lục A).

Bảng 1 (lỗi phân loại trên test set MNIST permutation-invariant, $K = 256$):

Model	error
Baseline	1.38%
Dropout	1.34%
Dropout (Pereyra et al., 2017)	1.40%
Confidence Penalty	1.36%
Confidence Penalty (Pereyra et al., 2017)	1.17%
Label Smoothing	1.40%
Label Smoothing (Pereyra et al., 2017)	1.23%
VIB ($\beta = 10^{-3}$)	**1.13%**

Paper tự nêu rõ: sự khác biệt giữa kết quả Confidence Penalty/Label Smoothing của chính họ và số trong (Pereyra et al., 2017) là **do siêu tham số khác nhau** (không phải lỗi tái tạo). VIB đạt lỗi thấp nhất trong bảng (1.13%), nhưng paper tự nhận "các kết quả này không phải state of the art, mục tiêu chính không phải chứng minh VIB là regularizer tốt nhất", chỉ cho thấy VIB hoạt động như một regularizer hợp lý khi dùng cùng kiến trúc cho mọi so sánh.

Hình 1(a) vẽ lỗi train/test theo β (trung bình 5 lần chạy độc lập, có error bar), cho cả "1 shot eval" (1 mẫu posterior z) và "avg eval" (trung bình $S = 12$ mẫu Monte Carlo, $p(y|x) = \frac{1}{S}\sum_s q(y|z^s)$). Quan sát của paper:

- Lỗi tăng vọt khi β vượt ngưỡng tới hạn $\beta \sim 10^{-2}$, tương ứng với $I(X, Z)$ nhỏ hơn $\log_2(10)$ bit: không đủ thông tin để phân biệt 10 lớp.
- Với β nhỏ, test error cao hơn train error (overfitting), vì network học $\sigma \approx 0$ (gần xác định/deterministic), làm mất hiệu lực regularization.
- Ở vùng β trung gian, Monte Carlo averaging (12 mẫu) giúp cải thiện kết quả; vùng β cho kết quả tốt nhất gần trùng với vùng mà lợi ích của averaging biến mất, gợi ý một cách chọn β chỉ dựa vào thống kê trên training set (paper nói rõ **chưa khảo sát sâu** khả năng này).

Hình 1(c) vẽ "IB curve" ($I(Z, Y)$ so với $I(Z, X)$ khi biến β): giảm β (cho qua nhiều thông tin hơn) tăng $I(Z, Y)$ trên **training set**, nhưng không nhất thiết trên test set.

Hình 1(d) vẽ $I(Z, X)$ (trục log) theo β: vùng β tốt nhất (10^{-3} đến 10^{-2}) tương ứng $I(Z, X)$ chỉ khoảng **10 đến 100 bit**.

4.2 MNIST, bottleneck $K = 2$ (Mục 4.1.2)

Dùng full-covariance Gaussian ($K = 2$, mạng dự đoán Cholesky decomposition của Σ). Hình 1(b) cho thấy classification kém hơn (do bottleneck hẹp hơn nhiều), nhưng xu hướng chung giống trường hợp $K = 256$; gap train-test lớn hơn.

Hình 2 trực quan hoá posterior $p(z|x)$ như ellipse tin cậy 95% cho 1000 ảnh test, màu theo nhãn, nền là entropy của $q(y|z)$. Ba giá trị β minh hoạ với lỗi kèm theo (đọc từ chú thích Hình 2): $\beta = 10^{-3}$: $err_{mc} = 3.18\%$, $err_1 = 3.24\%$; $\beta = 10^{-1}$: $err_{mc} = 3.44\%$, $err_1 = 4.32\%$; $\beta = 10^{0}$: $err_{mc} = 33.82\%$, $err_1 = 62.81\%$ (ở đây err_1 dùng 1 mẫu, err_{mc} dùng 12 mẫu). Khi β tăng, covariance của embedding tăng tương đối, các lớp bắt đầu chồng lấp; qua một ngưỡng tới hạn, encoding "sụp" (collapse) và mất hầu hết thông tin về lớp.

4.3 Robustness với adversarial examples (Mục 4.2)

Paper dùng hai loại tấn công: **FGS (Fast Gradient Sign)** của Goodfellow et al. (2015): một bước theo chiều gradient; và **L2 optimization attack** của Carlini & Wagner (2016): tấn công tối ưu hoá, cho perturbation nhỏ nhất đã công bố tới lúc đó, "capable of fooling the target network 100% of the time". Paper trích lại rằng trên MNIST, FGS gốc đạt khoảng 90% tỉ lệ đánh lừa mạng maxout với $\epsilon = 0.25$ (số do Goodfellow et al. báo cáo, không phải thí nghiệm của bài này).

Thiết lập (Mục 4.2.3): cùng kiến trúc $K = 256$ ở Mục 4.1; so sánh baseline deterministic, dropout, và VIB các β. Với VIB, dùng 12 mẫu posterior khi tính $p(y|x)$ cho cả việc tấn công và việc đánh giá; paper cũng thử chạy ở "mean mode" ($\sigma = 0$) và thấy **không khác biệt đáng kể**, nên mọi kết quả báo cáo dùng stochastic evaluation 12 mẫu.

Kết quả MNIST, Carlini-Wagner L2 (0 → 1), Mục 4.2.4: lấy 10 chữ số "0" đầu tiên trong test set, tấn công buộc phân loại thành "1". Hình 3-4 cho thấy mô hình deterministic bị lừa dễ bằng perturbation nhỏ, còn VIB với β đủ lớn thường làm tấn công **thất bại** (trong số bước lặp cho phép); khi tấn công thành công, perturbation phải lớn hơn. Hình 4: khi β tăng, L_0 (số pixel bị thay đổi) giảm nhưng L_2 và L_∞ tăng: kẻ tấn công bị buộc đổi ít pixel hơn nhưng với biên độ lớn hơn mỗi pixel.

Hình 5 (FGS, 1000 ảnh test đầu tiên, 3 lần lặp mỗi β): tất cả model test đạt trên 98.4% accuracy trên test set sạch (không bị méo bởi khác biệt accuracy nền). Accuracy tương đối của VIB (so baseline) tăng mạnh theo β. Baseline deterministic đạt khoảng 1% accuracy (adversarial) khi $\epsilon = 0.5$ đến khoảng 5% khi $\epsilon = 0.35$. Dropout khá hơn baseline (khoảng 5%-16% cùng dải ϵ) nhưng kém VIB.

Hình 6 (Carlini-Wagner L2, cả targeted và untargeted, trên toàn 1000 ảnh mọi lớp): **deterministic và dropout đều đạt accuracy 0%** trên cả targeted và untargeted attack (khớp với số 0% mà Carlini & Wagner (2016) báo cáo cho một ConvNet huấn luyện trên MNIST). VIB cho accuracy dương đáng kể trên một dải β rộng; ngay cả khi $\beta \to 0$ (thực nghiệm dùng $\beta = 10^{-11}$, "indistinguishable" so với $\beta = 0$), VIB vẫn robust hơn hẳn 0%. Paper giải thích hiện tượng này (dù β rất nhỏ) bằng khung lý thuyết của Fawzi et al. (2016): phân loại **quadratic** có khả năng robust cao hơn phân loại **linear**; Phụ lục C chứng minh gần đúng rằng bộ encoder/decoder Gaussian/softmax của VIB xấp xỉ một hàm quadratic với mọi $\beta < \infty$.

4.4 ImageNet (Mục 4.2.5)

Dùng checkpoint pretrained công khai của **Inception ResNet V2** (Szegedy et al., 2016) trên ImageNet (Deng et al., 2009), đạt **80.4%** classification accuracy trên validation set. Trích đặc trưng (feature) ở tầng penultimate (1536 chiều), whitened, rồi gắn một kiến trúc giống thiết lập MNIST: 2 lớp fully-connected 1024 unit, sau đó lớp encoding ngẫu nhiên Gaussian cầu 1024 chiều, rồi decoder là hồi quy logistic.

Kết quả classification: accuracy tốt nhất đạt được là **80.12%** tại $\beta = 0.01$: "gần như bằng" 80.4% gốc nhưng chỉ cần $I(X, Z) \sim 45$ bit (paper gọi là "a surprisingly small amount of information"). Ngược lại, khi $\beta = 0$ (không nén, trên 10,000 bit theo paper), accuracy chỉ **78.87%**. Baseline deterministic (cùng kiến trúc nhưng không VIB loss, lớp linear 1024 chiều thay encoding ngẫu nhiên) chỉ đạt **78.75%**. Paper tự nhận các con số thấp hơn 80.4% gốc "likely due to suboptimal hyperparameters settings or inadequate training" (không phải do bản chất VIB).

Kết quả robustness (Mục 4.2.5, "Adversarial Robustness"): dùng tấn công Carlini targeted L2. Hình 7-8 và Bảng 2 so sánh 3 kiến trúc: Determ (baseline deterministic trên precomputed feature), IRv2 (Inception ResNet V2 gốc không chỉnh sửa), VIB(0.01) ($\beta = 0.01$). Mục tiêu tấn công là nhãn 222 ("soccer ball"), trên 30/40 ảnh đầu validation set mà VIB phân loại đúng.

Bảng 2 (metric, Determ, IRv2, VIB(0.01)):

Metric	Determ	IRv2	VIB(0.01)
Successful target	1.0	1.0	0.567
L_2	6.45	14.43	43.27
L_∞	0.18	0.44	0.92

"Successful target" là tỉ lệ ảnh bị tấn công thành công ép về đúng nhãn mục tiêu (thấp hơn tốt hơn). Cả Determ và IRv2 đều bị tấn công thành công **100%**; VIB(0.01) chỉ bị thành công **56.7%**. Trong văn bản chính (không phải bảng), paper diễn giải cùng thí nghiệm theo cách khác: "chỉ 17/30 cuộc tấn công thành công" và "VIB model can resist about 43.3% of the attacks". Hai cách nói này **nhất quán với nhau** ($17/30 \approx 0.567$ là tỉ lệ thành công, $1 - 0.567 \approx 0.433$ là tỉ lệ chống được), nhưng phần caption Hình 7 lại ghi "in 13 out of 30 examples the attack fails" ($13/30 \approx 0.433$, khớp phần "resist 43.3%"): tức ba cách diễn đạt (bảng 0.567, văn bản 17/30, caption 13/30 fail) **đều tự nhất quán khi quy đổi qua lại**, không phải mâu thuẫn thật, nhưng người đọc cần chú ý bảng ghi "successful" còn văn bản đôi khi ghi theo "fail"/"resist" nên dễ nhầm dấu khi đọc lướt.

L_2 và L_∞ trung bình của perturbation cần để tấn công thành công đều **lớn hơn nhiều** cho VIB (43.27, 0.92) so với Determ (6.45, 0.18) và IRv2 (14.43, 0.44): Determ và IRv2 chỉ cần perturbation nhỏ không thể nhận

ra bằng mắt, còn với VIB, trong nhiều trường hợp perturbation **có thể nhận thấy được** (paper tự nói "the perturbations for the VIB network can be perceived", Hình 7-8).

5. Phân tích / ablation

- $\beta = 0$ **như giới hạn deterministic:** khi $\beta \to 0$, quá trình tối ưu VIB có xu hướng làm $f_e^{\Sigma}(x) \to 0$, khiến network gần như xác định (deterministic); paper cũng huấn luyện riêng một model deterministic tường minh (bỏ hẳn lớp Gaussian, chỉ dùng $z = f_e^{\mu}(x)$) để so sánh, và công thức (18) (J_{IB_0}) là mục tiêu tương ứng trường hợp $\beta = 0$.
- **Số mẫu Monte Carlo khi đánh giá (1 shot so với 12 mẫu):** ở vùng β "tốt" (cho kết quả gần tối ưu), một mẫu đơn (1 shot) đã gần sát kết quả trung bình 12 mẫu (Hình 2 caption: "For 'good' values of β, a single sample suffices"); lợi ích của việc trung bình nhiều mẫu rõ nhất ở vùng β trung gian, và paper quan sát vùng lợi ích này biến mất gần đúng tại vùng β cho kết quả tốt nhất: một "ablation tự nhiên" gợi ý cách chọn β không cần validation set riêng (nhưng chưa khảo sát kỹ).
- **Dropout so với VIB trên adversarial robustness:** dropout cải thiện robustness so với baseline deterministic (ví dụ 5%-16% so với 1%-5% trên FGS), nhưng vẫn kém rõ VIB, đặc biệt khi β tăng (Hình 4b, 5b). Đây là so sánh trực tiếp giữa hai cơ chế "tiêm nhiễu" khác nhau: dropout tắt ngẫu nhiên neuron, VIB tiêm nhiễu Gaussian có kiểm soát bởi mục tiêu thông tin.
- **"Mean mode" so với stochastic evaluation (Mục 4.2.3):** paper thử tắt nhiễu khi đánh giá ($\sigma \to 0$) và thấy "no noticeable impact", nghĩa là lợi ích robustness của VIB không chỉ đến từ việc *đánh giá* ngẫu nhiên (mà có thể đến từ việc *huấn luyện* ngẫu nhiên định hình decision boundary).
- L_0 **so với** L_2/L_∞ **khi** β **tăng (Hình 4, MNIST):** một hiện tượng tinh tế: β tăng khiến L_0 (số pixel) của perturbation thành công **giảm** nhưng L_2, L_∞ (biên độ) **tăng**: attacker bị ép tập trung vào ít pixel hơn nhưng phải "đục" sâu hơn ở các pixel đó.
- **Robustness khi** $\beta \to 0$ **vẫn cao hơn 0% (Hình 6):** đây là quan sát bất ngờ nhất của bài, được paper giải thích không phải bởi lượng nén (vì β gần 0 gần như không nén) mà bởi **dạng hàm quadratic** của bộ encoder/decoder Gaussian/softmax (Phụ lục C): một cơ chế robustness khác, độc lập một phần với cơ chế "nén thông tin".

6. Giới hạn

Do tác giả nêu:

- Kết quả MNIST (Bảng 1) "không phải state of the art", và bài không chủ đích chứng minh VIB là phương pháp regularization tốt nhất, chỉ dùng cùng kiến trúc để so sánh công bằng các dạng regularization khác nhau; cần "much more experimentation" để khẳng định tổng quát.
- Kết quả ImageNet thấp hơn baseline gốc 80.4% ở mọi β thử nghiệm; paper tự nhận nguyên nhân có thể là "inadequate training time or suboptimal hyperparameters", không loại trừ khả năng VIB có thể làm tốt hơn nữa nếu tối ưu kỹ hơn.
- Phụ lục C chỉ đưa ra một **xấp xỉ** (dùng Taylor series expansion bậc 2 cho hàm log-sum-exp), không phải một chặn chặt (tight bound) hoàn chỉnh; paper ghi rõ "using the TSE is sufficient to sketch the derivation", và có thể làm chặt hơn qua kỹ thuật của Browne & McNicholas (2015) nhưng "merely changes the scaling... the result is still log-quadratic" (không đổi bản chất kết luận).
- Mục "Future Directions" (Mục 5) tự liệt hàng loạt hướng **chưa làm** trong bài này: đặt mục tiêu VIB ở nhiều/mọi lớp của network (bài chỉ dùng một bottleneck); test trên ảnh thật đa dạng hơn (not fully explored); dùng $r(z)$ phong phú hơn thay vì cố định $\mathcal{N}(0, I)$; liên hệ với differential privacy (chỉ trích dẫn, chưa làm thực nghiệm); mở rộng sang open-universe classification; và mở rộng sang sequence prediction/predictive information (chỉ nêu ý tưởng).

Do chúng ta nhận thấy:

- Thí nghiệm ImageNet không huấn luyện toàn bộ mạng từ đầu mà **đóng băng** backbone Inception ResNet

V2 và chỉ gắn/huấn luyện một đầu (head) nhỏ phía trên feature trích sẵn; do đó kết luận về classification/robustness trên ImageNet chỉ áp dụng cho setting ”fine-tune head nhỏ trên feature cố định”, chưa chứng minh được VIB hoạt động tốt khi huấn luyện end-to-end một mạng lớn từ đầu trên dữ liệu ảnh thật.

- Số liệu adversarial robustness ImageNet (Bảng 2, Hình 7-8) chỉ dựa trên một tập rất nhỏ (30 ảnh, một nhãn mục tiêu cố định ”soccer ball”); đây là mẫu nhỏ so với quy mô 1000 lớp của ImageNet, nên độ tin cậy thống kê của tỉ lệ 56.7%/43.3% khá hạn chế (không có error bar hay nhiều lần lặp như thí nghiệm MNIST ở Hình 5-6).
- Phần so sánh với Confidence Penalty/Label Smoothing (Bảng 1) dùng lại kiến trúc của Pereyra et al. (2017) nhưng paper tự nhận siêu tham số khác, khiến phép so sánh ”VIB tốt hơn Confidence Penalty của chính paper này” không hoàn toàn tương đương phép so sánh ”VIB tốt hơn số 1.17% trong paper gốc của Pereyra”: hai mức kết quả Confidence Penalty (1.36% so với 1.17%) chênh khá lớn, cho thấy độ nhạy cao với tuning, một yếu tố cần cẩn trọng khi diễn giải ”VIB thắng” ở Bảng 1.

7. Liên hệ với các bài khác

- **ml basics 15** (Auto-Encoding Variational Bayes, đã tồn tại tại `ml basics content/15_2013_vae_aevb.md`, đã `ls` xác nhận): paper tự nói ở Mục 2 (Related Work) rằng VAE ”là một trường hợp đặc biệt của một phiên bản không giám sát (unsupervised) của VIB, nhưng với β cố định ở 1.0” (Phụ lục B chứng minh chi tiết). VIB dùng lại nguyên cơ chế reparameterization trick + KL-to-prior của VAE, nhưng cho bài toán có giám sát và cho phép β thay đổi tự do. Nên đọc VAE trước để hiểu rõ nguồn gốc của kỹ thuật reparameterization và cấu trúc ”encoder + KL-regularizer” trước khi đọc VIB.
- **math 11** (SGD as Approximate Bayesian Inference, đã viết, `math content/11_2017_sgd_approximate_bayesian_inference.md`): cả hai bài cùng nhóm ”làm cho suy luận xấp xỉ/biến phân chạy được bằng SGD trên mạng sâu”, nhưng math 11 áp KL divergence vào việc chọn **siêu tham số của bộ tối ưu** (learning rate, preconditioner) để mô phỏng một hậu nghiệm Bayes trên **tham số mô hình** θ, còn VIB áp KL divergence trực tiếp vào **hàm mất mát** để kiểm soát biểu diễn trung gian Z; cùng công cụ (biến phân, reparameterization) nhưng áp vào hai đối tượng khác nhau (tham số mô hình so với biểu diễn trung gian).
- **math 10** (Hamiltonian Monte Carlo, đã viết): cùng nhóm ”suy luận xấp xỉ cho deep learning” trong lộ trình, tuy không liên hệ kỹ thuật trực tiếp (HMC không xuất hiện trong bài VIB).
- Trong thư mục mới cùng mã ’math’: paper tự trích dẫn **Higgins et al. (2017), beta-VAE** (ICLR 2017, cùng ý tưởng đổi trọng số KL term trong VAE nhưng từ góc nhìn neuroscience) là công trình ”đã khám phá đúng thiết lập này” với động cơ khác; nếu thư mục `math` có bài về beta-VAE trong tương lai (numbering chưa xác định ở đây), nên đọc cùng nhóm.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, khoảng 3600 token/trang gồm khoảng 1500 token text và 1600-2100 token visual, GPU $\leq$ 8 GB, batch size 1 khi suy luận). VIB có liên hệ khá trực tiếp về **ý tưởng**, dù paper không hề thử nghiệm trên OCR hay token hoá ảnh (toàn bộ phần dưới là suy luận):

1. **Khung ”nén $I(Z,X)$, giữ $I(Z,Y)$” như mô hình lý thuyết cho việc nén visual token.** LightOnOCR xử lý khoảng 1600-2100 token visual mỗi trang; nếu coi X là toàn bộ patch ảnh gốc và Z là tập visual token sau một bước merge/pruning, mục tiêu IB gợi ý: thước đo ”tốt” cho một phương pháp token-merging không nên chỉ là giảm số token (giảm $I(Z,X)$ một cách thô, ví dụ bằng cách cắt ngẫu nhiên), mà nên **đồng thời kiểm soát** $I(Z,Y)$ với Y là nhãn OCR/text đích: tức giữ lại thông tin cần cho việc đọc chữ, bỏ thông tin dư thừa về texture/background pixel. Đây chỉ là một khung tư duy (suy luận), không phải công thức sẵn có để cắm thẳng vào token merging.

2. β **như một "núm xoay" tỉ lệ nén-chất lượng có thể đo bằng bit.** Hình 1(d) và đoạn ImageNet (chỉ cần ~ 45 bit để giữ 80.12% accuracy, so với > 10000 bit ở $\beta = 0$ chỉ đạt 78.87%) là minh chứng mạnh rằng một bottleneck ngẫu nhiên nhỏ (vài chục bit) có thể giữ hầu hết thông tin cần cho dự đoán. Nếu dự án muốn đo "mức độ dư thừa" của biểu diễn visual token hiện tại trước khi pruning, có thể suy luận áp dụng ý tưởng ước lượng $I(Z, X)$ qua $KL[p(Z|x), r(Z)]$ như một **chỉ số chẩn đoán** (diagnostic), không nhất thiết phải huấn luyện lại toàn bộ model với VIB loss.
3. **Stochastic bottleneck như một dạng "soft" của việc cắt token/kênh.** Thay vì cắt cứng (hard pruning, giữ/bỏ token tuyệt đối, đã có nhiều kỹ thuật riêng trong thư mục pruning), VIB gợi ý một cơ chế **mềm**: thêm nhiễu Gaussian có học được vào biểu diễn trung gian (ví dụ activation trước một lớp merge), với độ lớn nhiễu học được qua β. Về mặt suy luận, đây gần với ý tưởng "representation nên robust với perturbation nhỏ" mà các hướng distillation/pruning phục hồi (thư mục riêng, không lặp lại chi tiết kỹ thuật ở đây) cũng quan tâm, nhưng VIB cho một công thức hoá rõ ràng bằng mutual information.
4. **Thí nghiệm adversarial robustness (ImageNet, Mục 4.4) ít liên quan trực tiếp tới OCR.** Các kết quả về Carlini-Wagner L2/FGS trên ảnh tự nhiên nhỏ (MNIST 28x28, ImageNet 299x299 classification) không đo cùng loại rủi ro mà một pipeline OCR quan tâm (lỗi nhận dạng ký tự do noise/scan quality, không phải tấn công đối kháng chủ động); nên xem phần robustness của paper này là **ít liên quan trực tiếp**, chỉ hữu ích như minh chứng gián tiếp rằng bottleneck ngẫu nhiên làm decision boundary "trơn" hơn.

9. Thí nghiệm gợi ý

1. **Tái tạo thu nhỏ Bảng 1 trên MNIST với K nhỏ hơn:** MLP $784 - 256 - 256 - 2K$ với $K = 32$ hoặc 64, so sánh baseline deterministic, VIB với vài giá trị $\beta \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$. Đo: test error, và $I(Z, X)$ ước lượng qua $KL[p(Z|x), r(Z)]$ trung bình trên test set. Kỳ vọng: tái lập hình dạng chữ U của lỗi theo β (Hình 1a) và quan hệ tỉ lệ thuận giữa β nhỏ-$I(Z, X)$ lớn.
2. **Visualize bottleneck 2 chiều (tái tạo thu nhỏ Hình 2):** dùng $K = 2$ như Mục 4.1.2, vẽ ellipse $p(z|x)$ cho 200-500 ảnh test với 3-4 giá trị β. Đo: tỉ lệ các lớp chồng lấp (ví dụ bằng diện tích overlap ước lượng) theo β. Kỳ vọng: overlap tăng đơn điệu theo β, khớp mô tả "classes start to overlap" của paper.
3. **Đo adversarial robustness cơ bản với FGS trên MNIST (quy mô nhỏ):** huấn luyện baseline deterministic và VIB ($\beta = 10^{-2}$ chẳng hạn) trên MNIST đầy đủ, tấn công FGS với $\epsilon \in \{0.1, 0.2, 0.3\}$ trên 200 ảnh test. Đo: accuracy trên ảnh adversarial của hai model. Kỳ vọng: VIB giữ accuracy cao hơn deterministic, khớp xu hướng Hình 5 (không kỳ vọng khớp số chính xác vì quy mô thí nghiệm nhỏ hơn).
4. **Kiểm tra ảnh hưởng của số mẫu Monte Carlo khi đánh giá (1 shot so với trung bình):** với model VIB đã huấn luyện ở thí nghiệm 1, so test error khi dùng 1 mẫu so với trung bình 12 mẫu, ở vài giá trị β khác nhau. Kỳ vọng: khác biệt lớn ở β trung gian, gần như không khác biệt ở vùng β tốt nhất, khớp quan sát Mục 4.1.1 (Hình 1a, caption Hình 2).
5. **Minh hoạ "collapse" của encoding khi β quá lớn:** quét β từ 10^{-4} đến 10 trên cùng thiết lập $K = 2$, đo entropy trung bình của $q(y|z)$ đánh giá tại $z = \mu(x)$ cho các ảnh test. Kỳ vọng: entropy tăng mạnh (dự đoán trở nên bất định) khi β vượt một ngưỡng, minh hoạ hiện tượng "encoding collapses" mô tả ở Mục 4.1.2.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Mục tiêu Information Bottleneck	$R_{IB}(\theta) = I(Z, Y; \theta) - \beta I(Z, X; \theta)$, (3)
Markov chain giả định	$Y \leftrightarrow X \leftrightarrow Z$, $p(Z\|X, Y) = p(Z\|X)$, (5)
Chặn dưới $I(Z, Y)$ (decoder biến phân $q(y\|z)$)	$I(Z, Y) \geq \int p(x)p(y\|x)p(z\|x) \log q(y\|z)$, (12)

Khái niệm	Công thức / ý nghĩa
Chặn trên $I(Z,X)$ (marginal biến phân $r(z)$)	$I(Z,X) \le \int p(x)p(z\|x) \log \frac{p(z\|x)}{r(z)}$, (14)
Cận dưới toàn bộ L	ghép (12) và (14), (15)
Hàm mất mát thực dụng J_{IB}	$\frac{1}{N}\sum_n \mathbb{E}_\epsilon[-\log q(y_n\|f(x_n,\epsilon))] + \beta\, KL[p(Z\|x_n), r(Z)]$, (17)
Reparameterization	$z = f(x,\epsilon)$, ϵ độc lập tham số, (Kingma & Welling 2014)
Quy ước dấu β	β lớn = nén nhiều hơn (ngược với một số paper IB khác), chú thích 3
Giới hạn deterministic $\beta = 0$	$J_{IB_0} = -\frac{1}{N}\sum_n \mathbb{E}_{z\sim\mathcal{N}(f_e^\mu, f_e^\Sigma)}[\log S(y_n\|f_d(z))]$, (18)
Softplus cho σ (MNIST $K = 256$)	$\sigma = \log(1+\exp(x-5.0))$, (19), Phụ lục A
VAE là trường hợp đặc biệt	VAE = VIB không giám sát với $\beta = 1$ cố định, Phụ lục B
Quadratic bound cho encoder/decoder Gaussian/softmax	xấp xỉ bằng Taylor series expansion bậc 2 của log-sum-exp, Phụ lục C

Ví dụ tính tay minh hoạ chặn KL (tự tính): giả sử $p(z\|x) = \mathcal{N}(1, 0.25)$ (tức $\sigma^2 = 0.25$, $\sigma = 0.5$) và $r(z) = \mathcal{N}(0,1)$ (prior cố định, 1 chiều). Công thức KL giữa hai Gaussian 1 chiều: $KL[\mathcal{N}(\mu,\sigma^2)\|\mathcal{N}(0,1)] = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log\sigma^2\right)$. Thay số: $\frac{1}{2}(0.25 + 1 - 1 - \log 0.25) = \frac{1}{2}(0.25 - (-1.386)) = \frac{1}{2}(1.636) \approx 0.818$ nat (tự tính). Nhân với β (ví dụ $\beta = 10^{-2}$) cho đóng góp của riêng mẫu này vào số hạng regularization trong công thức (17): $0.818 \times 0.01 \approx 0.00818$ nat (tự tính): minh hoạ cụ thể cách mỗi mẫu x_n góp một số hạng $\beta \cdot KL[p(Z|x_n), r(Z)]$ nhỏ vào tổng loss khi β nhỏ, khớp trực giác Mục 4.1.1 rằng vùng β tốt chỉ cần vài chục bit (ở đây minh hoạ bằng nat, 1 bit $= \ln 2$ nat ≈ 0.693 nat, nên khoảng 10 bit ≈ 6.93 nat).

11. Kết luận ngắn

Deep VIB xây một cận dưới biến phân khả vi cho mục tiêu Information Bottleneck gốc, bằng cách đưa vào một decoder biến phân $q(y|z)$ (chặn dưới $I(Z,Y)$) và một marginal biến phân $r(z)$ (chặn trên $I(Z,X)$), rồi dùng reparameterization trick để huấn luyện bằng SGD trên mạng sâu. Thực nghiệm trên MNIST (lỗi phân loại 1.13% tại $\beta = 10^{-3}$, tốt hơn các regularizer so sánh trong cùng kiến trúc) và ImageNet (80.12% accuracy chỉ với ~ 45 bit thông tin, so với 78.87% khi không nén) cho thấy VIB vừa cải thiện generalization vừa tăng mạnh robustness với adversarial example (Carlini-Wagner L2, FGS), kể cả khi β rất nhỏ, một phần giải thích được bằng tính chất quadratic gần đúng của bộ encoder/decoder Gaussian/softmax. VIB cũng tổng quát hoá VAE (trường hợp đặc biệt không giám sát với $\beta = 1$) và mở đường áp dụng khung IB cho dữ liệu liên tục chiều cao, điều mà các phương pháp IB rời rạc/Gaussian cổ điển không làm được. Với dự án nén LightOnOCR, giá trị chính là khung tư duy "nén $I(Z,X)$ giữ $I(Z,Y)$" để đánh giá các phương pháp cắt/gộp visual token, còn phần thực nghiệm adversarial trên ảnh tự nhiên nhỏ ít liên quan trực tiếp tới rủi ro thực tế của một pipeline OCR.

Nguồn gốc: https://arxiv.org/abs/1612.00410

Bài 15

Online normalizer calculation for softmax (2018)

Tác giả: Maxim Milakov, Natalia Gimelshein (NVIDIA).
Venue / arXiv: arXiv:1805.02867 (v2, 28/07/2018); paper tự ghi ”Preprint. Work in progress.” ở cuối trang đầu, nên không có venue hội nghị hay tạp chí chính thức.
Nguồn: https://arxiv.org/abs/1805.02867
Chủ đề: một thuật toán tính hàm softmax ”an toàn” (không tràn số) trong một lượt quét duy nhất qua vector đầu vào, bằng cách tính đồng thời giá trị lớn nhất và tổng mũ chuẩn hoá, giảm số lần truy cập bộ nhớ so với cách làm 3-pass tiêu chuẩn, và hợp nhất (fuse) với phép TopK để giảm thêm.
Vị trí trong lộ trình: đọc sau math 14 (Deep Variational Information Bottleneck) theo thứ tự titles.tsv; đọc trước math 16 (FP8 Formats for Deep Learning) vì cả hai cùng chủ đề số học hiệu năng (numerics) cho deep learning trên phần cứng thật. Liên quan rất chặt tới **llm 44/45** (FlashAttention, FlashAttention-2): công thức đệ quy ở bài này là nền tảng toán học trực tiếp cho kỹ thuật ”online softmax theo block” mà FlashAttention dùng để tránh lưu toàn bộ ma trận attention.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Hàm **softmax** biến một vector logit $x \in \mathbb{R}^V$ (V là kích thước từ vựng hoặc số lớp) thành một vector xác suất:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^{V} e^{x_j}}. \qquad (1)$$

Softmax xuất hiện ở lớp cuối của hầu hết mô hình phân loại/ngôn ngữ (sau lớp Projection ”chiếu” biểu diễn ẩn ra không gian từ vựng), và trong hồi quy logistic đa lớp. Paper liệt kê nhiều công trình trước đó cố **tránh phải tính softmax đầy đủ** (Differentiated Softmax, SVD-Softmax, Hierarchical Softmax, Importance Sampling, Noise Contrastive Estimation, Self-Normalized Softmax), nhưng hầu hết vẫn phải chạy hàm softmax gốc trên một vector đầy đủ hoặc rút gọn. Paper này đi theo hướng khác: **không thay đổi thuật toán**, mà làm cho chính phép tính softmax gốc **chạy nhanh hơn trên phần cứng thật**, bằng cách giảm số lần truy cập bộ nhớ (memory access).

Vấn đề kỹ thuật cốt lõi: công thức (1) tính trực tiếp có thể **tràn số (overflow)** nếu x_i lớn (vì e^{x_i} tăng rất nhanh), hoặc **mất độ chính xác (underflow)** nếu x_i rất âm. Giải pháp kinh điển, được mọi framework lớn (TensorFlow v1.7, PyTorch v0.4.0, MXNet v1.1.0, Microsoft CNTK v2.5.1, Chainer v5.0.0a1, theo paper liệt kê) dùng, là trừ đi giá trị lớn nhất trước khi lấy exponent:

$$y_i = \frac{e^{x_i - \max_k x_k}}{\sum_{j=1}^{V} e^{x_j - \max_k x_k}}. \qquad (2)$$

Công thức (2) cần **3 lượt quét** qua vector đầu vào: lượt 1 tìm max, lượt 2 tính tổng mẫu số, lượt 3 tính y_i và ghi ra, tổng cộng **4 lần truy cập bộ nhớ mỗi phần tử** (3 lần đọc + 1 lần ghi). Paper gọi đây là ”Safe Softmax” và muốn cải tiến số lần truy cập này.

2. Ý tưởng chính

Ý tưởng chính: **gộp việc tính max và tính tổng mẫu số vào CÙNG MỘT lượt quét**, bằng một công thức đệ quy ”trực tuyến” (online) lấy cảm hứng từ thuật toán tính phương sai trực tuyến ổn định số của Welford (1962, paper trích dẫn tài liệu tham khảo [18]). Mỗi khi gặp một giá trị max mới lớn hơn, ta ”điều chỉnh lại” (rescale) tổng mẫu số đã tích lũy bằng một hệ số mũ, rồi cộng thêm phần tử mới vào. Nhờ vậy chỉ cần **1 lượt quét để tính xong cả max và tổng mẫu số**, giảm từ 4 xuống **3 lần truy cập bộ nhớ mỗi phần tử** (1 đọc cho lượt tính chuẩn hoá + 1 đọc, 1 ghi cho lượt tính y_i). Paper gọi thuật toán này là **”Online Softmax”**.

Ý tưởng thứ hai, mở rộng từ ý tưởng một: vì công thức đệ quy có thể viết lại thành một **phép toán kết hợp (associative) và giao hoán (commutative)** trên cặp (m, d) (max cục bộ, tổng mẫu số cục bộ), ta có thể **tính song song** (chia vector thành nhiều đoạn, tính từng đoạn độc lập, rồi gộp lại), rất phù hợp với GPU nhiều luồng.

Ý tưởng thứ ba: khi softmax được theo sau bởi **TopK** (ví dụ trong beam search để sinh văn bản tự hồi quy), ta không cần vật chất hoá (materialize) toàn bộ vector xác suất y; chỉ cần giữ K giá trị lớn nhất cùng chỉ số của chúng trong lúc quét, hợp nhất (fuse) luôn vào vòng lặp của Online Softmax, giảm xuống chỉ **1 lần truy cập bộ nhớ mỗi phần tử** cho cả Softmax+TopK.

3. Phương pháp chi tiết

3.1 Thuật toán Online Softmax (Mục 3, Algorithm 3)

Khởi tạo $m_0 = -\infty$, $d_0 = 0$. Với $j = 1, \dots, V$:

$$m_j \leftarrow \max(m_{j-1}, x_j), \qquad d_j \leftarrow d_{j-1} \cdot e^{m_{j-1} - m_j} + e^{x_j - m_j}.$$

Sau lượt quét này, $m_V = \max_k x_k$ và $d_V = \sum_{j=1}^{V} e^{x_j - m_V}$ đúng như công thức (2) cần. Lượt quét thứ hai (bắt buộc phải có, vì y_i cần m_V và d_V cuối cùng, không thể biết trước khi quét hết vector) tính $y_i \leftarrow e^{x_i - m_V} / d_V$.

Định lý 1 (paper, chứng minh bằng quy nạp): các dòng 1-6 của Algorithm 3 tính đúng $m_V = \max_k x_k$ và $d_V = \sum_j e^{x_j - m_V}$. Chứng minh quy nạp: cơ sở $V = 1$ hiển nhiên đúng ($m_1 = x_1$, $d_1 = e^{x_1 - m_1} = 1$); bước quy nạp giả sử đúng tới $S-1$, paper triển khai $d_S = d_{S-1} \cdot e^{m_{S-1} - m_S} + e^{x_S - m_S} = \sum_{j=1}^{S-1} e^{x_j - m_S} + e^{x_S - m_S} = \sum_{j=1}^{S} e^{x_j - m_S}$ (dùng giả thiết quy nạp và tính chất $e^{a-b} \cdot e^{b-c} = e^{a-c}$).

Tính an toàn số học: m_j luôn nằm trong $[\min_k x_k, \max_k x_k]$ nên không thể tràn/underflow; d_j luôn thoả $1 \le d_j \le j$ (vì mỗi số hạng $e^{x_i - m_j} \in (0, 1]$ và có ít nhất một số hạng bằng 1 khi $x_i = m_j$). Với lưu trữ số thực 32-bit, paper tính được d_j không tràn số cho tới 1.7×10^{37} phần tử, một con số ”khá lớn” theo paper, nhưng nếu vector lớn hơn mức này cần chuyển sang số thực 64-bit.

3.2 Tính song song (Mục 3.1, công thức (3)-(4))

Định nghĩa phép toán hai ngôi $\oplus : \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}^2$ trên cặp (max, tổng):

$$\begin{pmatrix} m_i \\ d_i \end{pmatrix} \oplus \begin{pmatrix} m_j \\ d_j \end{pmatrix} = \begin{pmatrix} \max(m_i, m_j) \\ d_i \cdot e^{m_i - \max(m_i, m_j)} + d_j \cdot e^{m_j - \max(m_i, m_j)} \end{pmatrix}. \qquad (4)$$

Áp dụng $\oplus$ liên tiếp từ trái sang phải cho các cặp $\binom{x_1}{1}, \binom{x_2}{1}, \dots, \binom{x_V}{1}$ tương đương chạy thuật toán tuần tự ở trên (công thức (3)). Paper nêu (không chứng minh chi tiết, ”we omit the proofs for brevity”) rằng $\oplus$ **kết hợp được (associative)** và **giao hoán được (commutative)**, nên có thể chia vector thành nhiều đoạn, tính (m, d) cục bộ cho mỗi đoạn độc lập (song song trên nhiều luồng GPU), rồi gộp bằng $\oplus$ theo cây giảm (tree reduction) theo thứ tự tuỳ ý.

3.3 Hợp nhất với TopK (Mục 4, Algorithm 4)

Hàm TopK(y) trả về K chỉ số và K giá trị lớn nhất của y (công thức (5)). Nếu chạy Safe Softmax rồi TopK tách biệt, tổng cộng tốn 5 lần truy cập bộ nhớ mỗi phần tử (4 của Safe Softmax + 1 đọc của TopK); dùng Online Softmax riêng rồi TopK riêng tốn 4 lần. Thuật toán 4 của paper **hợp nhất cả hai**: trong lúc quét một lượt để cập nhật (m_j, d_j), đồng thời giữ một buffer $K+1$ phần tử đã sắp xếp giảm dần (chèn phần tử mới vào đúng vị trí bằng một vòng lặp chèn, insertion sort cục bộ, dòng 10-15 của Algorithm 4), rồi ở cuối chỉ cần chuẩn hoá K giá trị trong buffer bằng m_V, d_V đã biết. Kết quả: **chỉ 1 lần truy cập bộ nhớ mỗi phần tử** cho toàn bộ Softmax+TopK (không cần ghi ra toàn bộ vector y kích thước V, chỉ cần ghi ra K giá trị).

Ví dụ tính tay (tự tính): xét vector $x = [1, 3, 2, 5]$ ($V = 4$). Theo công thức (2), Safe Softmax: $m = 5$; các số hạng $e^{1-5} = e^{-4} \approx 0.0183$, $e^{3-5} = e^{-2} \approx 0.1353$, $e^{2-5} = e^{-3} \approx 0.0498$, $e^{5-5} = 1$; tổng $d = 0.0183 + 0.1353 + 0.0498 + 1 \approx 1.2034$; $y \approx [0.0152,\ 0.1124,\ 0.0414,\ 0.8310]$ (tổng xấp xỉ 1, tự tính).

Chạy Algorithm 3 (Online Softmax) theo từng bước: $m_0 = -\infty, d_0 = 0$.

- $j = 1, x_1 = 1$: $m_1 = \max(-\infty, 1) = 1$; $d_1 = 0 \cdot e^{-\infty} + e^{1-1} = 1$.
- $j = 2, x_2 = 3$: $m_2 = \max(1, 3) = 3$; $d_2 = 1 \cdot e^{1-3} + e^{3-3} = e^{-2} + 1 \approx 0.1353 + 1 = 1.1353$.
- $j = 3, x_3 = 2$: $m_3 = \max(3, 2) = 3$; $d_3 = 1.1353 \cdot e^{3-3} + e^{2-3} = 1.1353 + e^{-1} \approx 1.1353 + 0.3679 = 1.5032$.
- $j = 4, x_4 = 5$: $m_4 = \max(3, 5) = 5$; $d_4 = 1.5032 \cdot e^{3-5} + e^{5-5} = 1.5032 \cdot e^{-2} + 1 \approx 1.5032 \times 0.1353 + 1 \approx 0.2034 + 1 = 1.2034$.

$d_4 \approx 1.2034$ khớp đúng với d tính trực tiếp theo công thức (2) ở trên (tự tính, kiểm chứng Định lý 1 trên ví dụ số cụ thể). Nhận xét: mỗi lần gặp max mới ($j = 2$ và $j = 4$), tổng cũ bị "chiết khấu" lại bằng hệ số $e^{m_{\text{cũ}} - m_{\text{mới}}} < 1$ trước khi cộng số hạng mới, đúng cơ chế rescale của thuật toán.

4. Thí nghiệm và kết quả chính

Benchmark thực hiện trên **GPU NVIDIA Tesla V100 PCIe 16 GB**, CUDA Toolkit 9.1, ECC bật, persistent mode bật, dùng thư viện CUB v1.8.0 cho phép reduction song song; mã nguồn benchmark công khai tại github.com/NVIDIA/online-softmax (theo paper). Tất cả thí nghiệm dùng kiểu số **fp32**.

Benchmark Softmax đơn (Mục 5.1, Hình 1-2): so 3 thuật toán Naive, Safe, Online trên nhiều kích thước vector V từ 10^2 đến 10^5 (hoặc 10^7 cho batch nhỏ), với 2 kích thước batch: 4.000 vector (mô phỏng huấn luyện/suy luận theo lô lớn) và 10 vector (mô phỏng suy luận trực tuyến từng mẫu nhỏ).

- Batch lớn (4.000): cả 3 thuật toán tương đương tới $V = 1000$; sau đó bộ nhớ cache L1/L2 bị "thrashing" (giao động liên tục), hiệu năng bị giới hạn bởi băng thông DRAM; Online và Naive (đều chỉ cần 3 truy cập) vượt Safe (4 truy cập), đạt khoảng **1.3x** tại $V = 4000$, gần đúng tỉ lệ giảm truy cập bộ nhớ lý thuyết $4/3 \approx 1.33$ (tự tính: $4/3 = 1.333$).
- Batch nhỏ (10 vector): GPU không được dùng hết (chỉ 10 threadblock), hiệu năng tuyệt đối thấp hơn hẳn, hiện tượng latency chi phối; Online/Naive vượt Safe khoảng **1.15x** cho $V \geq 1000$ (thấp hơn mức 1.3x của batch lớn, vì không còn bị giới hạn thuần bởi băng thông).

Benchmark Softmax+TopK (Mục 5.2, Hình 3-4), $K = 5$**:** so Safe Softmax+TopK chạy tách biệt, Safe Softmax+TopK hợp nhất (fused), và Online Softmax+TopK hợp nhất.

- Batch lớn (4.000): cải thiện bắt đầu từ **1.5x**, tăng dần theo V, đạt gần **5x** tại $V = 25000$; paper phân tích 5x này = 2.5x từ việc hợp nhất (fusion, giảm từ 5 xuống ~2 lượt truy cập tổng thể) **nhân** 2x từ chính Online Softmax (giảm thêm so với Safe).
- Batch nhỏ (10 vector): cải thiện **1.5x-2.5x**, không đạt 5x vì vùng latency-bound (GPU chưa bão hoà), nhưng vẫn có lợi.

- Với K lớn hơn, cải thiện giảm dần: **3.5x cho** $K = 10$**, 2x cho** $K = 15$**, 1.4x cho** $K = 30$, vì chi phí chèn-sắp-xếp (insertion sort) buffer TopK là $O(K)$ mỗi phần tử, dần chiếm ưu thế so với chi phí Softmax khi K tăng.

Kết quả tổng hợp (Mục 6): Online Softmax giảm truy cập bộ nhớ 1.33 lần (lý thuyết) cho softmax đơn, đo được **1.15x đến 1.3x** cải thiện tốc độ thực tế tuỳ chế độ batch/kích thước vector; khi hợp nhất với TopK, đạt **1.5x đến 5x** cải thiện; và các cải thiện này **độc lập (orthogonal)** với các kỹ thuật tối ưu softmax khác (Hierarchical Softmax, Importance Sampling, SVD-Softmax) theo nhận định của paper, vì Online Softmax chỉ thay đổi cách tính softmax gốc, không đổi thuật toán cấp cao hơn.

Bài báo **không có bảng số liệu dạng "accuracy/benchmark mô hình học máy"** (không so sánh độ chính xác mô hình ngôn ngữ hay bất kỳ task ML cụ thể nào); toàn bộ "kết quả chính" là đo tốc độ/throughput (elements per second) trên một loại GPU cụ thể, đúng tính chất của một báo cáo kỹ thuật hiệu năng (performance engineering report), không phải nghiên cứu mô hình.

5. Phân tích / ablation

- **Naive so với Safe so với Online:** Naive và Online có cùng số lần truy cập bộ nhớ (3), nên tốc độ tương đương trên benchmark; khác biệt chỉ ở **độ an toàn số học** (Naive có thể tràn số, Online thì không) với "chi phí thêm gần như không đáng kể" (paper dùng từ "negligible additional cost of two operations per vector element"). Đây là điểm bán hàng chính: Online Softmax cho "miễn phí" (gần như không mất tốc độ) sự an toàn số học mà trước đây phải đánh đổi bằng cách chạy Safe Softmax chậm hơn.
- **Batch lớn so với batch nhỏ:** sự khác biệt về mức cải thiện (1.3x so với 1.15x; 5x so với 1.5-2.5x) cho thấy lợi ích của việc giảm truy cập bộ nhớ chỉ **phát huy tối đa khi hệ thống bị giới hạn bởi băng thông bộ nhớ** (memory-bandwidth-bound, batch lớn, V lớn); khi hệ thống bị giới hạn bởi độ trễ (latency-bound, batch nhỏ, GPU chưa bão hoà), lợi ích vẫn có nhưng nhỏ hơn.
- **Ảnh hưởng của** K **trong TopK fusion:** cải thiện giảm đơn điệu khi K tăng (5x ở $K = 5$ xuống 1.4x ở $K = 30$), vì chi phí chèn-sắp-xếp tăng tuyến tính theo K mỗi phần tử trong khi lợi ích từ giảm truy cập bộ nhớ là hằng số; đây là một đánh đổi (trade-off) rõ ràng mà paper trình bày định lượng.
- **Fusion so với chỉ đổi thuật toán:** phân tích 5x = 2.5x (fusion) × 2x (online) cho thấy **hai nguồn cải thiện độc lập có thể cộng gộp theo tích**, không chỉ theo tổng, một quan sát hữu ích khi thiết kế kernel GPU hợp nhất nhiều phép toán.

6. Giới hạn

Do tác giả nêu (Mục 7, Discussion):

- Chỉ benchmark trên **một loại GPU** (Tesla V100), không có thí nghiệm trên CPU; paper suy đoán (không kiểm chứng) rằng nếu mã nguồn được vector hoá tốt thì CPU cũng có thể đạt cải thiện tương tự.
- Online Softmax và Softmax+TopK fused "vẫn bị giới hạn bởi băng thông bộ nhớ" (still limited by the memory bandwidth); paper gợi ý hướng cải thiện tiếp theo là hợp nhất với **lớp đứng trước** (ví dụ phép nhân ma trận chiếu) để tránh một lượt đọc/ghi bộ nhớ round-trip nữa, nhưng tự nhận "this change is more challenging" và không triển khai trong bài.
- Lợi ích của TopK fusion giảm mạnh khi K lớn (xuống 1.4x ở $K = 30$ và "degrades further for bigger Ks"), nên kỹ thuật này chủ yếu có giá trị cho K nhỏ (ví dụ beam search với beam width nhỏ).
- Đây là "Preprint. Work in progress.", không qua bình duyệt chính thức (peer review) tại một venue cụ thể.

Do chúng ta nhận thấy:

- Toàn bộ benchmark dùng **fp32**; paper không thử nghiệm với fp16/bf16 (các kiểu số phổ biến hơn trong huấn luyện/suy luận LLM hiện nay), nên các số liệu tốc độ cụ thể (1.3x, 5x) không chắc giữ nguyên trên các kiểu số khác hoặc trên GPU đời mới hơn V100 (bài viết năm 2018).

- Không có thí nghiệm nào mô phỏng workload thực tế của attention trong Transformer (chuỗi dài, nhiều ”hàng” softmax đồng thời như trong self-attention); các con số benchmark chỉ cho một hàm softmax độc lập trên vector/batch vector, chưa tính tới chi phí/đặc thù khi softmax là một bước trong một kernel attention lớn hơn (đây chính là việc FlashAttention giải quyết sau này, nhưng không phải nội dung của bài báo 2018 này).
- Không có phân tích độ phức tạp hình thức (Big-O) cho overhead của thuật toán song song (Mục 3.1) khi số luồng tăng; chỉ nói ”enables parallel evaluation” mà không đo riêng hiệu năng của phiên bản song song so với tuần tự.

7. Liên hệ với các bài khác

- `ls "/home/duy/Downloads/duy_dev/research_/llm content/"` xác nhận tồn tại **llm 01** (`01_2017_attention_is_all_you_need.md`): softmax trong attention (softmax$(QK^{\mathsf{T}}/\sqrt{d})$) là nơi trực tiếp áp dụng kỹ thuật ở bài này; mỗi ”hàng” của ma trận attention là một vector softmax cần tính an toàn và nhanh.
- `ls "/home/duy/Downloads/duy_dev/research_/llm content/"` xác nhận tồn tại **llm 44** (`44_2022_flashattention.md`) và **llm 45** (`45_2023_flashattention2.md`): đây là liên hệ quan trọng nhất của bài này. FlashAttention tổng quát hoá chính xác công thức đệ quy (4) ở Mục 3.2 của bài này (phép $\oplus$ kết hợp được trên cặp max/tổng) sang **nhiều block** của ma trận attention (không chỉ trong một vector mà qua các khối của chiều chuỗi), cho phép tính attention đầy đủ mà không bao giờ phải vật chất hoá ma trận $N \times N$ trong bộ nhớ HBM, chỉ cần $O(N)$ bộ nhớ phụ trợ thay vì $O(N^2)$. Online Softmax (bài này, 2018) là tiền đề toán học trực tiếp, dù FlashAttention (2022) không trích dẫn bài này trong các phiên bản phổ biến (quan hệ về toán học, không phải trích dẫn chính thức, đây là nhận xét của người ghi chú).
- `ls "/home/duy/Downloads/duy_dev/research_/llm content/"` xác nhận tồn tại **llm 46** (`46_2023_pagedattention_vllm.md`): hệ thống serving LLM như vLLM cũng cần tính softmax hiệu quả trên KV cache phân trang; cùng chủ đề hiệu năng suy luận.
- **math 16** (FP8 Formats for Deep Learning, bài tiếp theo cùng thư mục): cùng chủ đề số học hiệu năng (numerics) cho deep learning trên phần cứng thật, nhưng ở tầng khác (định dạng số FP8 cho lưu trữ/tính toán, so với thuật toán tính softmax ổn định ở tầng này).
- **math 18** (The Shattered Gradients Problem): khác chủ đề (ổn định huấn luyện mạng sâu qua gradient, không phải numerics suy luận), nhưng cùng tinh thần ”phân tích chi tiết hành vi số học/thống kê của một phép toán cơ bản trong deep learning”.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid Gated-DeltaNet + full attention, mỗi trang khoảng 3600 token gồm khoảng 1500 token text và 1600-2100 token visual, GPU $\leq$ 8 GB, batch size 1 khi suy luận). Bài này **liên quan trực tiếp**, vì phần **full attention** trong kiến trúc hybrid chắc chắn cần tính softmax trên các hàng của ma trận attention, với độ dài chuỗi ~3600 token (và còn dài hơn nếu ghép nhiều trang hoặc dùng context dài).

1. **Peak VRAM khi suy luận batch 1 trên GPU 8 GB:** nếu implementation attention của model (hoặc thư viện bên dưới, ví dụ PyTorch `scaled_dot_product_attention` hay các kernel FlashAttention) đã dùng kỹ thuật online-softmax theo block (kế thừa trực tiếp ý tưởng bài này), thì bộ nhớ đỉnh cho attention sẽ $O(N)$ thay vì $O(N^2)$ với $N \approx 3600$; nếu không, việc vật chất hoá ma trận attention $3600 \times 3600 \approx 1.3 \times 10^7$ phần tử (khoảng 52 MB ở fp32 cho một head, nhân với số head và số layer, tự tính) có thể là một phần đáng kể của ngân sách 8 GB. Khi tối ưu suy luận trên GPU nhỏ, nên kiểm tra xem backend attention đang dùng có áp dụng online softmax hay không (ví dụ bật `torch.backends.cuda.enable_flash_sdp(True)` hoặc dùng kernel FlashAttention trực tiếp).

2. **Giải mã OCR dùng beam search/TopK:** nếu pipeline giải mã văn bản dùng beam search (TopK trên phân phối từ vựng ở mỗi bước), kỹ thuật hợp nhất Softmax+TopK (Mục 4, Algorithm 4) có thể giảm đáng kể số lần truy cập bộ nhớ ở bước decode, đặc biệt quan trọng khi V (kích thước từ vựng tokenizer) lớn; tuy nhiên lợi ích giảm nhanh khi beam width K lớn (paper cho thấy $K = 30$ chỉ còn 1.4x), nên chỉ đáng đầu tư công sức nếu beam width thực tế nhỏ (ví dụ $K \leq 10$).
3. **Nên tận dụng implementation có sẵn hơn tự viết lại:** kỹ thuật này đã được tích hợp sẵn vào các kernel attention hiện đại (FlashAttention và các thư viện suy luận LLM); với một model 0.8B trên GPU tiêu dùng, việc tự viết lại kernel CUDA tối ưu như paper mô tả **không hiệu quả về công sức** so với việc dùng thư viện sẵn có; giá trị chính của bài này cho dự án là **hiểu nguyên lý** để biết nên bật/chọn đúng backend attention nào khi deploy, không phải để tự triển khai lại.

Phần ít liên quan: các con số benchmark cụ thể (1.3x, 5x) đo trên Tesla V100 (datacenter GPU, 2017) không áp dụng trực tiếp cho GPU tiêu dùng 8 GB hiện đại; cần đo lại trên phần cứng thực tế của dự án nếu muốn số liệu chính xác.

9. Thí nghiệm gợi ý

1. **Kiểm chứng tính đúng đắn của Online Softmax trên CPU/GPU nhỏ:** viết 3 hàm Naive/Safe/Online softmax bằng NumPy hoặc PyTorch trên vector ngẫu nhiên (vài nghìn phần tử, giá trị logit lớn để test overflow), so sánh đầu ra Online với `torch.softmax` (Safe); đo sai số tuyệt đối tối đa. Kỳ vọng: sai số ở mức epsilon máy (khoảng 10^{-7} với fp32), xác nhận Định lý 1; thử Naive với logit lớn (ví dụ nhân 100) để quan sát NaN/Inf xuất hiện, minh hoạ lý do cần phiên bản an toàn.
2. **Mô phỏng online softmax theo block (tiền đề FlashAttention):** chia một vector dài (ví dụ 4096 phần tử, tương đương độ dài chuỗi gần với LightOnOCR) thành các block 128/256/512 phần tử, áp dụng công thức $\oplus$ (Mục 3.2) để gộp dần kết quả qua các block, so với tính Safe Softmax một lượt trên toàn vector. Kỳ vọng: kết quả giống nhau trong sai số số học, bất kể kích thước block, minh hoạ tính ”kết hợp được” của $\oplus$ là nền tảng cho attention theo block.
3. **Đo peak VRAM attention trên GPU thật:** trên GPU 8 GB, dựng một lớp self-attention đơn giản với độ dài chuỗi $N \in \{512, 1024, 2048, 3600\}$, so sánh peak memory khi (a) vật chất hoá toàn bộ ma trận attention $N \times N$ rồi softmax kiểu Safe, so với (b) dùng `torch.nn.functional.scaled_dot_product_attention` (có backend FlashAttention). Kỳ vọng: (a) tăng memory theo N^2, (b) gần như tuyến tính theo N, xác nhận lợi ích thực tế trên chính phần cứng của dự án.
4. **Đo tốc độ fused Softmax+TopK ở mức Python (giới hạn):** implement một vòng lặp Python/PyTorch tính softmax rồi `topk` tách biệt, so với một cách tính ”giả lập hợp nhất” (tính trong cùng một hàm, tránh ghi tensor trung gian ra bộ nhớ nhiều lần qua `torch.no_grad()` và viết thủ công). Đo thời gian với $K \in \{5, 10, 30\}$. Kỳ vọng: cải thiện ở mức Python nhỏ hơn nhiều so với 5x của paper (vì overhead Python/kernel launch chi phối, không phải băng thông bộ nhớ thuần như CUDA kernel chuyên dụng); kết luận: lợi ích thật của kỹ thuật này chỉ hiện rõ ở tầng kernel CUDA viết tay, không phải ở tầng framework cấp cao.
5. **Đo ảnh hưởng của vector size V tới tốc độ softmax trên GPU của dự án:** lặp lại ý tưởng Hình 1 của paper (nhưng trên GPU 8 GB tiêu dùng thay vì V100): đo throughput (phần tử/giây) của `torch.softmax` theo V từ 10^2 đến 10^5, với batch lớn và batch nhỏ. Kỳ vọng: quan sát được ngưỡng chuyển từ compute-bound sang memory-bandwidth-bound tương tự $V \approx 1000$ của paper (giá trị cụ thể có thể khác do cache size khác nhau giữa GPU).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Softmax gốc	$y_i = e^{x_i} / \sum_j e^{x_j}$, (1)

Khái niệm	Công thức / giá trị
Safe softmax	$y_i = e^{x_i - \max_k x_k} / \sum_j e^{x_j - \max_k x_k}$, (2)
Đệ quy Online Softmax	$m_j = \max(m_{j-1}, x_j)$; $d_j = d_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}$
Phép kết hợp song song	$\binom{m_i}{d_i} \oplus \binom{m_j}{d_j} = \binom{\max(m_i, m_j)}{d_i e^{m_i - \max} + d_j e^{m_j - \max}}$, (4)
Số lần truy cập bộ nhớ/phần tử	Naive: 3; Safe: 4; Online: 3; Online+TopK fused: 1
Cận an toàn của d_j	$1 \le d_j \le j$; fp32 an toàn tới 1.7×10^{37} phần tử
Tốc độ Softmax đơn (V100, fp32, batch 4000)	$\approx$ 1.3x so với Safe tại $V = 4000$
Tốc độ Softmax đơn (batch 10)	$\approx$ 1.15x so với Safe tại $V \ge 1000$
Tốc độ Softmax+TopK ($K = 5$, batch 4000)	tới $\approx$ 5x ($= 2.5\times$ fusion $\times\, 2\times$ online) tại $V = 25000$
Suy giảm theo K (Softmax+TopK, batch 4000)	5x ($K = 5$) $\to$ 3.5x ($K = 10$) $\to$ 2x ($K = 15$) $\to$ 1.4x ($K = 30$)

11. Kết luận ngắn

Bài báo kỹ thuật ngắn này giải quyết một vấn đề thực dụng: softmax ”an toàn” tiêu chuẩn cần 3 lượt quét/4 lần truy cập bộ nhớ mỗi phần tử; bằng một công thức đệ quy tính đồng thời max và tổng mẫu số (lấy cảm hứng từ thuật toán phương sai trực tuyến của Welford), Online Softmax giảm xuống 3 lần truy cập mà vẫn giữ an toàn số học, đo được cải thiện tốc độ 1.15x-1.3x trên GPU Tesla V100; khi hợp nhất thêm với TopK (cho beam search), cải thiện lên tới 5x với K nhỏ. Giá trị lâu dài của bài này vượt xa các con số benchmark cụ thể năm 2018: chính công thức đệ quy/phép kết hợp $\oplus$ ở đây là nền tảng toán học cho kỹ thuật ”online softmax theo block” trong FlashAttention, công cụ gần như bắt buộc để chạy attention hiệu quả về bộ nhớ trên các mô hình hiện đại, bao gồm phần full-attention trong kiến trúc hybrid của LightOnOCR.

Nguồn gốc: https://arxiv.org/abs/1805.02867

Bài 16

FP8 Formats for Deep Learning (2022)

Tác giả: Paulius Micikevicius, Dusan Stosic, Patrick Judd, John Kamalu, Stuart Oberman, Mohammad Shoeybi, Michael Siu, Hao Wu (NVIDIA); Neil Burgess, Sangwon Ha, Richard Grisenthwaite (Arm); Naveen Mellempudi, Marius Cornea, Alexander Heinecke, Pradeep Dubey (Intel)
Venue / arXiv: arXiv:2209.05433 (v2, 29/09/2022); paper không nêu venue hội nghị/journal cụ thể (là một technical report/whitepaper liên công ty NVIDIA-Arm-Intel, công bố kèm đề xuất chuẩn)
Nguồn: https://arxiv.org/abs/2209.05433
Chủ đề: Đề xuất một chuẩn định dạng số thực dấu phẩy động 8-bit (FP8) gồm hai biến thể E4M3 và E5M2 để huấn luyện và suy luận mạng neural, kèm thực nghiệm trên nhiều kiến trúc và kích cỡ mô hình (đến 175 tỷ tham số).
Vị trí trong lộ trình: đọc sau math 15 (Online normalizer calculation for softmax, cùng chủ đề ”làm toán số học hiệu quả trên phần cứng”), có thể đọc cùng lúc hoặc trước opt 42 (Mixed Precision Training, FP16) vì FP8 là bước tiếp theo tự nhiên của FP16 mixed-precision; nên đọc trước các bài lượng tử hoá LLM (llm 51 LLM.int8, llm 52 SmoothQuant, llm 53 GPTQ, llm 54 AWQ) để hiểu sự khác biệt giữa lượng tử hoá dấu phẩy động (floating-point, không đều) và lượng tử hoá nguyên (int8, đều).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Để hiểu bài này cần biết số thực dấu phẩy động (floating point) được biểu diễn bằng 3 phần: bit dấu (sign, S), phần mũ (exponent, E, quyết định ”thang đo”/dynamic range), và phần định trị (mantissa hay ”trailing significand”, M, quyết định độ chính xác trong một thang đo). Paper gọi rõ: ”mantissa” ở đây là từ đồng nghĩa với ”trailing significand field” theo chuẩn IEEE 754, tức là **không tính** bit 1 ẩn ở đầu của số chuẩn hoá (dòng 119-121 của paper). Giá trị của một số chuẩn hoá (normal) là:

$$(-1)^S \times 1.M_2 \times 2^{(E_2 - \text{bias})}$$

trong đó M_2 là mantissa đọc theo nhị phân (dạng $1.xxx$), bias là độ lệch mũ để cho phép biểu diễn cả số mũ âm bằng một số nguyên không dấu.

Bối cảnh rộng hơn: các mô hình deep learning (DL) ngày càng lớn (GPT-3, Turing-Megatron, PaLM, OPT được paper trích dẫn ở phần Introduction) cần hàng nghìn processor chạy trong nhiều tuần. Biểu diễn số với độ chính xác thấp hơn (reduced precision) là ”trụ cột” (corner stone, nguyên văn dòng 44 có lỗi đánh máy ”corner stoen”) để tăng tốc huấn luyện/suy luận. Các định dạng phổ biến trước FP8 gồm: FP32 (single precision, IEEE), TF32 (dùng cho Tensor Core NVIDIA), FP16 (IEEE half precision), bfloat16 (BF16, Google). Với suy luận (inference), int8 cố định điểm (fixed-point) là lựa chọn phổ biến, nhưng paper nêu rõ ”trong một số trường hợp int8 gặp khó khăn để duy trì độ chính xác cần cho việc triển khai ứng dụng” (dòng 48-49, trích [1] Facebook inference accelerator deployment). Những hướng cực đoan hơn như mạng 1-bit (binary networks) được paper liệt ra (BinaryConnect, Binarized Neural Networks, XNOR-Net, DoReFa-Net) nhưng ”không thành công trong việc duy trì chất lượng kết quả cần cho nhiều ứng dụng thực tế” (dòng 46-48).

Các công trình tiền nhiệm trực tiếp mà paper tự liệt ra trong Introduction (đoạn 2 của §1): Wang et al. [22] (NeurIPS 2018) dùng exponent 5-bit để huấn luyện CNN trên CIFAR-10/ILSVRC12; Mellempudi et al. [12] (2019) nghiên cứu định dạng exponent 5-bit cho CNN lớn hơn và mạng dịch ngôn ngữ (RNN, Transformer); cả hai đều nghiên cứu weight update 16-bit và stochastic rounding; Sun et al. [20] (NeurIPS 2019, ”HFP8”)

giới thiệu việc dùng **hai** định dạng FP8 (exponent 4-bit và 5-bit) cho huấn luyện, nghiên cứu nhiều CNN, speech, dịch ngôn ngữ, và cũng khảo sát suy luận FP8 của mạng huấn luyện ở độ chính xác cao hơn, đưa ra kỹ thuật ”returning of batch normalization statistics” để cải thiện độ chính xác; Noune et al. [16] (2022) đề xuất biến thể FP8 dành một encoding duy nhất cho giá trị đặc biệt để tăng dynamic range. Bài hiện tại (2209.05433) là nỗ lực hợp nhất của 3 công ty (NVIDIA, Arm, Intel) để đưa ra **một chuẩn interchange format** duy nhất (không phải chỉ một kỹ thuật nghiên cứu), nhằm các nhà sản xuất phần cứng khác nhau có thể tương thích với nhau khi trao đổi tensor ở dạng FP8.

2. Ý tưởng chính

Ý tưởng cốt lõi của paper là: định nghĩa **hai** encoding 8-bit cho số dấu phẩy động, đặt tên theo số bit exponent và mantissa:

- **E4M3**: 1 bit dấu + 4 bit exponent + 3 bit mantissa.
- **E5M2**: 1 bit dấu + 5 bit exponent + 2 bit mantissa.

Lý do cần **hai** định dạng (không chỉ một) là có sự đánh đổi (trade-off) giữa dynamic range (thang đo giá trị biểu diễn được, do số bit exponent quyết định) và độ chính xác (precision, do số bit mantissa quyết định). Tensor trọng số (weight) và activation thường có phân phối giá trị tương đối hẹp quanh 0 nên cần **độ chính xác cao hơn, dynamic range thấp hơn** → dùng E4M3. Tensor gradient trong lan truyền ngược (backward pass) thường có dynamic range rộng hơn rất nhiều (nhiều giá trị rất nhỏ gần 0 và một số giá trị lớn) nên cần **dynamic range cao hơn, có thể chấp nhận độ chính xác thấp hơn** → dùng E5M2. Paper khuyến nghị rõ: ”khuyến nghị sử dụng E4M3 cho tensor trọng số và activation, và E5M2 cho tensor gradient” (dòng 122, ”The recommended use of FP8 encodings is E4M3 for weight and activation tensors, and E5M2 for gradient tensors”), đồng thời nói một số mạng cần cả hai loại cùng lúc.

Ý tưởng thứ hai, mang tính triết lý thiết kế: **giữ tối đa quy ước IEEE 754**, chỉ lệch khỏi chuẩn khi lợi ích cho ứng dụng DL là đáng kể. Cụ thể, E5M2 giữ nguyên hoàn toàn quy ước IEEE (có vô cực, có nhiều mã NaN, có số 0 có dấu), còn E4M3 ”hi sinh” việc biểu diễn số vô cực (infinity) và chỉ giữ **một** mẫu bit cho NaN, để ”lấy lại” không gian mã đó cho các giá trị số bình thường lớn hơn, mở rộng dynamic range. Đây là đánh đổi có chủ đích: trong DL, giá trị tràn số (overflow) thường được xử lý bằng cách bão hoà (saturate) về giá trị lớn nhất biểu diễn được, nên không cần một mã riêng cho ”vô cực” như toán học thuần túy cần.

3. Phương pháp chi tiết

3.1 Bảng mã hoá chi tiết (Table 1 của paper)

Paper dùng ký hiệu S.E.M (ví dụ S.1111.1102) để biểu diễn chuỗi bit: S là bit dấu, nhóm tiếp là exponent (4 hoặc 5 bit), nhóm cuối là mantissa (3 hoặc 2 bit), số có chỉ số dưới 2 là số nhị phân, không có chỉ số là số thập phân. Nguyên văn Table 1 (dòng 131-141):

	E4M3	E5M2
Exponent bias	7	15
Infinities	N/A (không biểu diễn)	$S.11111.00_2$
NaN	$S.1111.111_2$	$S.11111.\{01, 10, 11\}_2$
Zeros	$S.0000.000_2$	$S.00000.00_2$
Max normal	$S.1111.110_2 = 1.75 \times 2^8 = 448$	$S.11110.11_2 = 1.75 \times 2^{15} = 57\,344$
Min normal	$S.0001.000_2 = 2^{-6}$	$S.00001.00_2 = 2^{-14}$
Max subnormal	$S.0000.111_2 = 0.875 \times 2^{-6}$	$S.00000.11_2 = 0.75 \times 2^{-14}$
Min subnormal	$S.0000.001_2 = 2^{-9}$	$S.00000.01_2 = 2^{-16}$

Giải thích các khái niệm trong bảng cho người chưa quen:

- **Exponent bias**: số cộng/trừ để đổi exponent (được lưu dưới dạng số nguyên không âm trong các bit) thành exponent thực (có thể âm). Với E4M3, bias = 7, nên exponent field $E_2 = 1111_2 = 15$ (thập phân) tương ứng exponent thực $15 - 7 = 8$.
- **Số chuẩn hoá (normal)**: có bit ẩn đầu là 1, công thức $(-1)^S \times 1.M \times 2^{E-\text{bias}}$.
- **Số subnormal (hay denormal)**: khi phần exponent toàn 0, bit ẩn đầu trở thành 0 (không phải 1), công thức $(-1)^S \times 0.M \times 2^{1-\text{bias}}$, cho phép biểu diễn các số rất gần 0 (nhỏ hơn min normal) với độ chính xác giảm dần (gradual underflow), đổi lại để tránh "rơi thẳng về 0" đột ngột.
- **Max normal** của E4M3: pattern exponent 1111_2 = 15 nhưng **không** dùng để biểu diễn vô cực (vì E4M3 bỏ hẳn vô cực), nên pattern mantissa 110_2 (không phải 111_2, pattern đó dành cho NaN) với exponent lớn nhất 1110_2... paper viết trực tiếp là S.1111.110_2, nghĩa là exponent field vẫn là 1111 (=15, thực là $15 - 7 = 8$), mantissa 110_2 = 0.75, tổng mantissa đầy đủ $1 + 0.75 = 1.75$, giá trị $= 1.75 \times 2^8 = 1.75 \times 256 = 448$. (tự tính: $1.75 \times 256 = 448$, khớp số paper ghi.)

3.2 Mở rộng dynamic range của E4M3 (§3.1)

Đây là điểm kỹ thuật quan trọng nhất của thiết kế. Với quy ước IEEE 754 chuẩn, pattern exponent toàn 1 (1111_2) được dành riêng cho vô cực (mantissa = 0) và NaN (mantissa $\neq 0$, có nhiều mẫu NaN khác nhau). Nếu E4M3 tuân theo đúng quy ước này, giá trị lớn nhất biểu diễn được chỉ là khi exponent field = 1110_2 (=14, thực $14 - 7 = 7$) với mantissa lớn nhất 111_2=0.875, tức $1.875 \times 2^7 = 240$ (paper nói rõ: "Giá trị lớn nhất biểu diễn được nếu không có sửa đổi này sẽ là 240", dòng 159).

Bằng cách **bỏ vô cực** và **chỉ giữ một mẫu NaN** (S.1111.111_2), E4M3 "lấy lại" 7 mẫu bit mantissa còn lại ở exponent field = 1111_2 (mantissa từ 000 đến 110, trừ 111 dành cho NaN) để biểu diễn 7 giá trị số bình thường mới: 256, 288, 320, 352, 384, 416, 448 (paper liệt kê chính xác các số này ở dòng 158). Việc này "mở rộng dynamic range thêm một luỹ số 2" (extends the dynamic range by one extra power of 2), từ 17 lên 18 binade. Khái niệm **binade**: một "binade" là một dải giá trị giữa hai luỹ thừa liên tiếp của 2 (ví dụ từ 2^k đến 2^{k+1}); "17 binade" nghĩa là khoảng cách giữa giá trị nhỏ nhất và lớn nhất biểu diễn được trải qua 17 lần nhân đôi liên tiếp.

Paper cũng giải thích lý do **không** đi xa hơn (bỏ luôn việc phân biệt +0/-0 và dùng chung một mã NaN duy nhất, có thể biểu diễn thêm số 480): việc này sẽ phá vỡ tính đối xứng dấu dương/âm cố hữu của IEEE 754, làm hỏng các thuộc tính thực thi quan trọng như "so sánh và sắp xếp số dấu phẩy động bằng phép toán số nguyên" (dòng 163-164: "IEEE floating point formats allow comparison and sorting of floating point values using integer operations"). Lợi ích tăng từ 448 lên 480 "không đáng để đánh đổi" quy ước này.

Ngược lại, **E5M2 giữ đầy đủ quy ước IEEE** cho giá trị đặc biệt (có vô cực S.11111.00_2, nhiều mã NaN). Lý do: E5M2 đã có 32 binade (nhờ 5 bit exponent), lợi ích của việc "lấy lại" không gian mã dành cho đặc biệt chỉ thêm được 3 giá trị số nữa (do mantissa chỉ 2 bit) - "một binade bổ sung ít quan trọng hơn hẳn khi E5M2 đã có 32 (so với E4M3 chỉ có 17 nếu không chỉnh sửa)" (dòng 169-172). Ngoài ra việc giữ đúng IEEE giúp chuyển đổi (convert) thẳng giữa E5M2 và FP16 dễ dàng ("straightforward conversion between E5M2 and IEEE FP16 formats", dòng 147-148), vì E5M2 có thể coi là "FP16 với ít bit mantissa hơn" giống như cách bfloat16/TF32 được coi là "FP32 với ít bit hơn".

3.3 Exponent bias cố định, scaling factor xử lý trong software (§3.2)

Cả hai định dạng **giữ bias kiểu IEEE** (E4M3: bias=7, E5M2: bias=15), không dùng "programmable exponent bias" (bias có thể cấu hình được theo tensor). Lý do paper đưa ra: dùng **scaling factor** (hệ số nhân, lưu ở độ chính xác cao hơn, ví dụ FP32) trước khi chuyển một tensor sang FP8 đạt được hiệu quả tương đương với việc thay đổi bias, nhưng linh hoạt hơn, vì scaling factor "có thể nhận bất kỳ giá trị thực nào" còn programmable bias "chỉ tương đương cho phép các luỹ thừa của 2 làm scaling factor" (dòng 183-185). Cơ chế này tương tự hệt với **loss scaling** trong mixed-precision training FP16 (paper trích dẫn trực tiếp Mixed Precision Training, [14] = arXiv 1710.03740, chính là opt 42 trong lộ trình này): gradient được nhân với một hệ số trước khi ép

về FP16/FP8 để tránh bị tràn dưới (underflow) về 0, rồi được chia lại hệ số đó sau khi tính toán ở độ chính xác cao hơn.

Khác biệt quan trọng so với FP16 automatic mixed precision: với FP8, **không nên** dùng chiến lược ”bỏ qua cập nhật trọng số khi tràn số rồi giảm scaling factor” (weight update skipping) như FP16 AMP vẫn làm, vì ”tràn số có khả năng xảy ra nhiều hơn hẳn do dynamic range hẹp hơn, dẫn đến quá nhiều lượt cập nhật bị bỏ qua” (dòng 97-99). Thay vào đó FP8 dùng **bão hoà (saturation)**: giá trị vượt ngưỡng tối đa bị cắt về giá trị lớn nhất biểu diễn được, không bị coi là lỗi cần bỏ qua bước cập nhật.

Paper cũng mô tả quy trình tính toán thực tế: phép toán toán học trên input FP8 (ví dụ matrix-multiplication/dot-product trong một lớp fully-connected hoặc convolution, gọi chung là **GEMM**, viết tắt General Matrix Multiply) tạo ra **output ở độ chính xác cao hơn** (ví dụ FP32), không viết trực tiếp ra FP8; chỉ khi cần lưu vào bộ nhớ (memory) thì mới convert (tuỳ chọn) về FP8. Các phép toán ít nặng về số học hơn (nonlinearity, normalization, cập nhật trọng số bởi optimizer) thường bị giới hạn bởi băng thông bộ nhớ (memory-bandwidth limited) hơn là bởi tốc độ tính toán, nên ”không nhạy với một phép toán số học bổ sung cho mỗi giá trị” (dòng 103-105) khi phải unscale (nhân nghịch đảo scaling factor).

3.4 Ví dụ số tính tay (tự tính)

Để hiểu rõ cơ chế biểu diễn và sai số làm tròn, ta thử biểu diễn số thực $x = 300$ bằng E4M3.

Bước 1: tìm luỹ thừa của 2 gần nhất không vượt quá 300: $2^8 = 256 \le 300 < 512 = 2^9$. Vậy exponent thực $= 8$, và dạng chuẩn hoá $x = 1.m \times 2^8$ với $1.m = 300/256 = 1.171875$.

Bước 2: $m = 0.171875$. Với 3 bit mantissa, các giá trị m biểu diễn được là các số nguyên $k/8$ với $k = 0, \dots, 7$ (vì $2^3 = 8$ mức). $0.171875 \times 8 = 1.375$. Số nguyên gần nhất là $k = 1$, cho $m = 1/8 = 0.125$, hoặc $k = 2$ cho $m = 2/8 = 0.25$.

- Với $k = 1$: giá trị biểu diễn $= 1.125 \times 256 = 288$.
- Với $k = 2$: giá trị biểu diễn $= 1.25 \times 256 = 320$.

300 cách 288 là 12, cách 320 là 20, nên làm tròn đến số gần nhất (round-to-nearest-even) sẽ chọn 288. Sai số tuyệt đối là $|300 - 288| = 12$ (tự tính), sai số tương đối là $12/300 \approx 4\%$ (tự tính). Đây minh hoạ trực tiếp bảng Table 1: 288 chính là một trong 7 giá trị ”lấy lại” được nhờ bỏ vô cực mà paper liệt ở dòng 158 (256, 288, 320, ...), nên ví dụ này nằm đúng trong vùng dynamic range mở rộng của E4M3.

Nếu dùng E5M2 cho cùng số $x = 300$: mantissa chỉ có 2 bit, các mức $m = k/4$, $k = 0, \dots, 3$. $0.171875 \times 4 = 0.6875$, gần nhất là $k = 1$ ($m = 0.25$, giá trị $= 320$) hoặc $k = 0$ ($m = 0$, giá trị $= 256$). 300 cách 256 là 44, cách 320 là 20, chọn 320, sai số tuyệt đối 20, sai số tương đối $\approx 6.7\%$ (tự tính). Kết luận tự rút ra (suy luận của người ghi chú, không phải paper nói trực tiếp bằng con số này): với cùng một giá trị cụ thể, E4M3 (nhiều bit mantissa hơn) cho sai số làm tròn nhỏ hơn E5M2, đúng với nguyên lý thiết kế của paper rằng E4M3 ưu tiên độ chính xác còn E5M2 ưu tiên dynamic range.

4. Thí nghiệm và kết quả chính

Paper nhấn mạnh phương pháp luận thực nghiệm: dùng **FP8 mô phỏng (simulated FP8)**, nghĩa là giá trị tensor được ”cắt” (clip) về đúng các giá trị có thể biểu diễn trong FP8 (bao gồm cả áp dụng scaling factor và bão hoà), nhưng **phép tính số học thực tế vẫn thực hiện ở độ chính xác rộng hơn** (FP16 hoặc bfloat16). Lý do nêu rõ: định dạng interchange là trọng tâm của paper (vì các hãng phần cứng khác nhau có thể chọn cách hiện thực phép toán vector/matrix khác nhau), và việc giả lập số học FP8 thật trên phần cứng chưa hỗ trợ sẽ ”quá chậm cho việc huấn luyện các mô hình lớn” (dòng 192-194).

Table 2 - Image Classification (ILSVRC12, Top-1 Accuracy validation), paper so Baseline (FP16/bfloat16) với FP8, các mô hình: VGG-16 (71.27 vs 71.11), VGG-16 BN (73.95 vs 73.69), Inception v3 (77.23 vs 77.06), DenseNet 121 (75.59 vs 75.33), DenseNet 169 (76.97 vs 76.83), ResNet18 (70.58 vs 70.12), ResNet34 (73.84

vs 73.72), ResNet50 v1.5 (76.71 vs 76.76, FP8 còn **cao hơn** baseline), ResNet101 v1.5 (77.51 vs 77.48), ResNeXt50 (77.68 vs 77.62), Xception (79.46 vs 79.17), MobileNet v2 (71.65 vs 71.04, chênh lệch lớn nhất trong bảng, 0.61 điểm, tự tính: $71.65 - 71.04 = 0.61$), DeiT small (80.08 vs 80.02, mô hình Transformer-based duy nhất trong bảng này). Paper tự nhận xét: ”ngoại trừ MobileNet v2, độ chính xác đạt được bởi huấn luyện FP8 nằm trong biến thiên run-to-run của huấn luyện độ chính xác cao hơn” (dòng 230-232), và nói rõ ”chúng tôi tiếp tục nghiên cứu để khôi phục phần độ chính xác còn thiếu cho MobileNet v2” (dòng 232), tức là **paper tự nhận một hạn chế cụ thể**, không che giấu.

Table 3 - Language Translation (WMT 2016 English→German, BLEU qua sacreBLEU trên newstest2014): GNMT (RNN-based, 24.83 vs 24.65), Transformer Base (26.87 vs 26.83), Transformer Large (28.43 vs 28.35). Tất cả đều ”trong biên độ biến thiên run-to-run”.

Table 4 - NLP Models, Perplexity (thấp hơn tốt hơn): Transformer-XL Base (22.98 vs 22.99), Transformer-XL Large (17.80 vs 17.75, FP8 tốt hơn), GPT 126M (19.14 vs 19.24), GPT 1.3B (10.62 vs 10.66), GPT 5B (8.94 vs 8.98), GPT 22B (7.21 vs 7.24), **GPT 175B (6.65 vs 6.68)**. Paper ghi chú quan trọng: với mô hình 175B, ”perplexity được báo cáo ở 75% quá trình huấn luyện, vì run baseline bfloat16 chưa hoàn thành” (dòng 257-258) - đây là một **giới hạn thực nghiệm được paper tự khai báo rõ ràng**, không phải so sánh đầy đủ 100% huấn luyện. Dù vậy, ”phiên huấn luyện FP8 đã hoàn thành và loss curve của nó phù hợp với một quá trình huấn luyện thành công” (Figure 1 minh hoạ perplexity theo số iteration chuẩn hoá cho nhiều mô hình GPT-3).

Table 5 - Post-training quantization (PTQ) của model huấn luyện FP16, so int8 vs E4M3: BERT Base trên SQuAD v1.1 (F1): 16-bit FP = 88.19, int8 = 76.89 (giảm mạnh, mất $88.19 - 76.89 = 11.3$ điểm, tự tính), E4M3 = 88.09 (chỉ giảm 0.1 điểm, tự tính). BERT Large: 16-bit = 90.87, int8 = 89.65, E4M3 = 90.94 (FP8 **vượt** baseline). GPT3 126M trên wikitext103 (perplexity): 16-bit = 19.01, int8 = 28.37 (tăng mạnh/tệ hơn hẳn), E4M3 = 19.43. GPT3 1.3B: 16-bit = 10.19, int8 = 12.74, E4M3 = 10.29. GPT3 6.7B: 16-bit = 8.51, int8 = 10.29, E4M3 = 8.41 (FP8 vượt baseline). Kết luận chính của bảng này (paper nói trực tiếp, dòng 293-296): ”BERT trên SQuAD cho thấy FP8 PTQ duy trì độ chính xác trong khi int8 PTQ dẫn tới mất độ chính xác đáng kể... GPT trên wikitext103 cho thấy FP8 PTQ tốt hơn hẳn trong việc giữ độ chính xác so với int8”. Paper cũng thử ”cast tensor sang FP8 mà không áp dụng scaling factor” và thấy ”mất độ chính xác đáng kể, tăng perplexity lên 11.0” (dòng 294-295) - đây là minh chứng trực tiếp cho tầm quan trọng của scaling factor đã nói ở mục 3.3.

5. Phân tích / ablation

Mục §4.3 ”Per-tensor scaling factors” là phần phân tích sâu nhất của paper, dùng Figure 2: đo perplexity suy luận trên wikitext103 khi lấy một mô hình GPT3 1.3B đã huấn luyện bằng bfloat16, rồi cast weight/activation sang E4M3 với **các giá trị exponent bias khác nhau**, không dùng calibration, không dùng per-tensor scaling riêng (tức là dùng đúng cơ chế ”bias cố định” thay cho ”scaling factor mềm” để xem khi nào nó đủ dùng).

Kết quả: khi chỉ quantize các tensor input cho phép toán GEMM (bao gồm cả hai phép nhân ma trận theo batch trong attention), có ”một vài lựa chọn exponent bias trong khoảng [7, 10] cho kết quả khớp baseline bfloat16” (dòng 317-319) - tức là tồn tại một vùng bias hoạt động tốt, không chỉ một giá trị duy nhất, cho thấy có biên độ dung sai (margin). Nhưng khi **quantize thêm cả tensor residual connection** (input cho phép toán Add, giúp giảm thêm áp lực lưu trữ/băng thông bộ nhớ), ”không một giá trị exponent bias đơn lẻ nào cho độ chính xác đủ tốt, thậm chí bias = 7 cho perplexity 12.59” (dòng 323-325), so với baseline 10.19, tức **tệ hơn baseline** $12.59 - 10.19 = 2.4$ **điểm perplexity** (tự tính). Đây là một phát hiện quan trọng về giới hạn: residual connection có phân phối giá trị khó khớp với bất kỳ bias cố định chung nào.

Giải pháp mà paper đưa ra ngay sau đó: nếu **calibrate riêng từng tensor** (per-channel scaling cho weight, per-tensor scaling cho activation, giống quy ước lượng tử hoá int8 thường dùng, trích dẫn [23] = Integer quantization for deep learning inference), thì đạt perplexity 10.29 (chỉ GEMM) và 10.44 (GEMM + residual)

(dòng 326-328), rất gần baseline 10.19. Đây chính là bằng chứng thực nghiệm trực tiếp ủng hộ lựa chọn thiết kế ở §3.2: giữ bias cố định kiểu IEEE và đẩy việc scaling sang software linh hoạt, thay vì cố gắng làm ”programmable bias” trong phần cứng.

Về mức độ ”ablation” theo nghĩa cắt bớt thành phần: paper **không** có một bảng ablation kiểu ”bật/tắt từng thành phần của thuật toán” như nhiều paper deep learning khác (vì đây là một đề xuất định dạng số, không phải một pipeline nhiều bước); phân tích gần nhất với ablation là việc so sánh ”chỉ GEMM” vs ”GEMM + residual” vừa nêu, và việc so sánh E4M3 vs int8 trong Table 5 (coi int8 như một ”ablation” của thiết kế dấu phẩy động, thay bằng cố định điểm).

6. Giới hạn

Các giới hạn mà **paper tự nêu** (không phải suy luận của người ghi chú):

1. MobileNet v2 chưa khớp hoàn toàn baseline, còn một phần độ chính xác cần ”khôi phục” (dòng 232), paper nói đang tiếp tục nghiên cứu.
2. Kết quả GPT 175B dựa trên so sánh với baseline bfloat16 **mới hoàn thành 75%** quá trình huấn luyện tại thời điểm viết paper (dòng 257-258), nên con số so sánh ở Table 4 cho mô hình lớn nhất chưa phải so sánh đầy đủ 100% các bước huấn luyện.
3. Residual connection không thể quantize tốt bằng exponent bias cố định chung, cần calibration riêng từng tensor (per-tensor/per-channel scaling factor) (§4.3, đã phân tích ở mục 5).
4. Phạm vi tensor được quantize trong thí nghiệm huấn luyện chỉ là ”activation, weight, và activation gradient tensor là input cho GEMM”; **tensor output để ở độ chính xác cao hơn** vì chúng thường được tiêu thụ bởi phép toán non-GEMM (nonlinearity, normalization) và ”việc chuyển thêm nhiều tensor sang FP8 là đề tài nghiên cứu tương lai” (dòng 206-207) - paper tự nói rõ phạm vi áp dụng FP8 trong bài **chưa phải toàn bộ** pipeline huấn luyện.
5. Thí nghiệm dùng **FP8 mô phỏng**: số học thực tế vẫn chạy ở FP16/bfloat16, chỉ giá trị tensor bị ”cắt” về lưới giá trị FP8; paper không báo cáo tốc độ (throughput/latency) thực đo trên phần cứng FP8 thật, vì bài báo là một **đề xuất định dạng** đi trước phần cứng hỗ trợ đầy đủ, không phải một báo cáo benchmark hiệu năng. Đây là điểm quan trọng người đọc dễ hiểu lầm: paper không chứng minh tốc độ thực tế tăng bao nhiêu lần, chỉ chứng minh **độ chính xác được giữ nguyên** khi ép tensor về lưới giá trị FP8.
6. Rounding mode (round-to-nearest-even, stochastic rounding, v.v.) được paper nói ”là lựa chọn của việc hiện thực, phần mềm và có thể phần cứng, để tối đa linh hoạt” (dòng 112-113) - nghĩa là **không được chuẩn hoá** trong đề xuất này, có thể dẫn tới kết quả khác nhau giữa các hiện thực khác nhau của cùng một chuẩn FP8.
7. Paper không thảo luận chi tiết heuristic chọn scaling factor (”vượt quá phạm vi của bài báo này”, dòng 94-95), chỉ nói ý tưởng chung là chọn sao cho giá trị lớn nhất trong tensor gần với giá trị lớn nhất biểu diễn được của định dạng.

7. Liên hệ với các bài khác

- **math 15 (Online normalizer calculation for softmax, 1805.02867)**: đã viết tại ”math content/15_2018_online_softmax.md” (xác nhận tồn tại bằng ls). Cả hai bài đều thuộc nhóm ”làm toán số học hiệu quả và ổn định trên phần cứng tăng tốc deep learning”: math 15 quan tâm tới **thứ tự tính toán** (giảm số lần truy cập bộ nhớ, giữ ổn định số học khi tính softmax theo từng khối), còn math 16 quan tâm tới **cách biểu diễn số** (bao nhiêu bit, định dạng nào); hai hướng tối ưu này độc lập nhưng bổ sung nhau khi thiết kế kernel GPU hiệu năng cao như FlashAttention.
- **opt 42 (Mixed Precision Training, arXiv 1710.03740)**: chưa tồn tại file khi viết ghi chú này (đang được viết song song theo số quy ước trong titles.tsv), nhưng paper FP8 **trích dẫn trực tiếp** [14] = chính paper này làm nguồn cho kỹ thuật ”loss scaling” mà FP8 kế thừa (dòng 92-93, 404-406 trong reference list), và dòng 68 nói rõ FP8 ”match FP16 or bfloat16 training results” - tức FP8 là bước tiếp theo của

chính kỹ thuật mixed-precision mà opt 42 khởi xướng cho FP16.

- **llm content/51_2022_llm_int8.md, llm content/52_2023_smoothquant.md, llm content/53_2023_gptq.md, llm content/54_2024_awq.md** (đã xác nhận tồn tại bằng ls): đây là các kỹ thuật lượng tử hoá **int8/int4** (cố định điểm) cho LLM đã huấn luyện. So sánh trực tiếp với kết quả Table 5 của bài này: FP8 PTQ giữ độ chính xác tốt hơn int8 PTQ ”ngây thơ” (naive), nhưng các bài LLM.int8/SmoothQuant/GPTQ/AWQ giải quyết đúng vấn đề ”int8 PTQ mất độ chính xác” bằng các kỹ thuật thông minh hơn (ví dụ SmoothQuant di chuyển độ khó lượng tử hoá từ activation sang weight bằng một hệ số scale theo channel, GPTQ dùng xấp xỉ Hessian để tối ưu làm tròn). Nói cách khác: nếu phần cứng không hỗ trợ FP8, các kỹ thuật int8 ”thông minh” này là con đường thay thế để đạt độ chính xác tương đương.
- **llm content/43_2026_lightonocr.md** (đã xác nhận tồn tại bằng ls): đây là model mục tiêu của dự án người dùng (xem mục 8).

8. Ứng dụng cho dự án (suy luận)

Toàn bộ nội dung mục này là suy luận, không phải nội dung paper nói trực tiếp về LightOnOCR.

Dự án người dùng là nén/hậu huấn luyện (post-training) LightOnOCR-3-0.8B, một VLM OCR với kiến trúc hybrid GatedDeltaNet + full attention, xử lý khoảng 3600 token/trang (gồm ~1500 token văn bản + 1600-2100 token hình ảnh), chạy trên GPU $\leq$ 8 GB, batch size 1 khi suy luận.

Liên quan trực tiếp nhất: nếu mục tiêu là **giảm bộ nhớ và tăng tốc suy luận** trên GPU 8GB, FP8 (đặc biệt E4M3 cho weight/activation) là một hướng khả thi về mặt lý thuyết độ chính xác, vì Table 5 của paper cho thấy FP8 PTQ giữ độ chính xác tốt hơn int8 PTQ trên các mô hình ngôn ngữ cỡ tương tự (BERT, GPT 126M-6.7B) - những cỡ mô hình này **gần với** 0.8B tham số của LightOnOCR hơn là GPT-175B, nên kết quả định tính (FP8 giữ độ chính xác tốt) có khả năng chuyển giao hợp lý (suy luận, không phải paper đo trực tiếp trên mô hình 0.8B hay trên dữ liệu OCR/VLM).

Tuy nhiên có một giới hạn phần cứng **quan trọng và thực tế** cần nêu rõ: FP8 muốn có tốc độ tăng tốc thật (không chỉ ”mô phỏng” như paper này làm) cần **Tensor Core hỗ trợ FP8 chuyên dụng trong phần cứng**, chỉ có từ các GPU kiến trúc NVIDIA Hopper (H100) và Ada Lovelace (RTX 4000-series) trở lên (ngoài paper, kiến thức chung). Nếu GPU 8GB của người dùng là đời cũ hơn (ví dụ Turing/Ampere thế hệ thấp như RTX 2060/3050/3060 8GB không có FP8 Tensor Core), thì **suy luận thực tế FP8 trên phần cứng đó không khả thi trực tiếp** - chỉ có thể làm giống cách paper này làm: ”mô phỏng” FP8 bằng cách ép giá trị tensor về lưới rời rạc rồi tính lại ở FP16/FP32, điều này **không tiết kiệm bộ nhớ thật** (vẫn lưu ở FP16/FP32) và **không tăng tốc thật** (vẫn tính ở độ chính xác cao hơn), chỉ có giá trị để **nghiên cứu/mô phỏng sai số lượng tử hoá** trước khi triển khai trên phần cứng thật có hỗ trợ.

Nếu GPU của người dùng không hỗ trợ FP8 phần cứng, bài học về **dynamic range theo số bit exponent** và **đánh đổi exponent/mantissa** vẫn hữu ích khi nghĩ về lượng tử hoá int8/int4 nói chung (liên hệ tới llm 51-54 ở mục 7): ví dụ hiểu rằng gradient/activation có outlier lớn cần dynamic range rộng (ứng với ”E5M2-style” trong tư duy, dù thực hiện bằng int cố định điểm) còn weight ổn định hơn cần độ chính xác cao hơn (ứng với ”E4M3-style”), giúp chọn chiến lược scaling (per-channel, per-tensor) hợp lý hơn khi quantize LightOnOCR sang int8.

Tóm lại (suy luận): FP8 **có liên quan về nguyên lý** (dynamic range vs precision trade-off, per-tensor scaling) nhưng **việc áp dụng trực tiếp phụ thuộc hoàn toàn vào phần cứng GPU cụ thể** của người dùng; nếu không chắc GPU hỗ trợ FP8 Tensor Core, nên ưu tiên các kỹ thuật int8/int4 đã được thiết kế cho phần cứng phổ biến hơn (llm 51-54).

9. Thí nghiệm gợi ý

1. **Mô phỏng lượng tử hoá E4M3/E5M2 bằng tay trên một layer nhỏ**: viết hàm Python/PyTorch nhận một tensor float32, tính scaling factor (max giá trị tuyệt đối chia cho max biểu diễn được của định dạng, 448 cho E4M3 hoặc 57344 cho E5M2), làm tròn mỗi giá trị về lưới rời rạc gần nhất của E4M3 (dùng công thức ở mục 3.4), rồi khử scale lại. Đo sai số tương đối trung bình (MAE/giá trị trung bình tuyệt đối) giữa tensor gốc và tensor ”giả-FP8”. Kết quả kỳ vọng: sai số tương đối trung bình khoảng vài phần trăm (tương tự ví dụ tự tính 4-6.7% ở mục 3.4), và sai số của E5M2 lớn hơn E4M3 với cùng tensor weight thông thường (phân phối hẹp quanh 0). Diễn giải: xác nhận trực tiếp nguyên lý ”E4M3 chính xác hơn, E5M2 dynamic range rộng hơn” trên dữ liệu thật của người dùng, không cần phần cứng FP8.

2. **So sánh outlier gradient vs weight**: lấy một mô hình nhỏ (ví dụ 2-3 lớp linear hoặc một lớp attention của LightOnOCR), chạy vài bước forward/backward trên GPU 8GB, lưu lại histogram giá trị tuyệt đối của weight, activation, và gradient. Đo tỷ lệ giữa giá trị lớn nhất và giá trị nhỏ nhất (không kể 0) của mỗi loại tensor (ước lượng dynamic range cần thiết, tính bằng $\log_2(\max / \min)$). Kết quả kỳ vọng: gradient có dynamic range (số binade cần) lớn hơn weight/activation rõ rệt. Diễn giải: xác nhận hoặc phản biện khuyến nghị ”E4M3 cho weight/activation, E5M2 cho gradient” của paper (dòng 122) trên chính mô hình của dự án, trước khi quyết định chiến lược lượng tử hoá.

3. **Ảnh hưởng của residual connection khi quantize**: lấy đầu ra của một khối residual (ví dụ tổng của attention output và input residual stream trong LightOnOCR) trước khi cộng, mô phỏng ép cả hai nhánh về lưới E4M3 với cùng một exponent bias cố định, so với dùng scaling factor riêng cho mỗi nhánh (per-tensor). Đo sai số tái tạo đầu ra sau Add. Kết quả kỳ vọng: giống phát hiện ở §4.3 của paper, dùng chung một bias cố định cho cả residual branch sẽ tệ hơn hẳn so với scale riêng từng tensor. Diễn giải: nếu kết quả khớp, xác nhận bài học của paper (per-tensor scaling cần thiết cho residual) cũng áp dụng cho kiến trúc hybrid GatedDeltaNet + attention của LightOnOCR, nhắc nhở không nên dùng một scale chung cho toàn mô hình khi nén.

4. **PTQ int8 ”ngây thơ” vs E4M3-mô phỏng trên một bài toán nhỏ**: lấy một mô hình phân loại ảnh nhỏ (ví dụ CNN 3-4 lớp trên MNIST/CIFAR-10, chạy được trên GPU 8GB trong vài phút), huấn luyện ở FP32, rồi (a) lượng tử hoá int8 theo kiểu max-calibration đơn giản, (b) lượng tử hoá ”giả-E4M3” theo công thức mục 3.4, so độ giảm accuracy của (a) và (b). Kết quả kỳ vọng: (b) giảm accuracy ít hơn (a), lặp lại định tính phát hiện ở Table 5 của paper (FP8 PTQ tốt hơn int8 PTQ) ở quy mô nhỏ hơn hẳn. Diễn giải: đây là một kiểm tra ”sanity check” rẻ, không cần phần cứng FP8, để tin tưởng hơn vào kết luận định tính trước khi đầu tư công sức triển khai thật.

5. **Đo ảnh hưởng của không dùng scaling factor**: lặp lại thí nghiệm 1 nhưng cố tình đặt scaling factor = 1 (không chuẩn hoá trước khi ép về FP8), so với có scaling factor đúng. Kết quả kỳ vọng: sai số tăng mạnh (tương tự paper báo cáo perplexity tăng lên 11.0 khi bỏ scaling factor, dòng 294-295), đặc biệt nếu giá trị tensor nhỏ hơn hẳn giá trị biểu diễn tối thiểu hoặc lớn hơn hẳn giá trị biểu diễn tối đa của định dạng. Diễn giải: minh hoạ trực quan tại sao scaling factor là bước bắt buộc, không phải chi tiết tối ưu tuỳ chọn.

10. Tóm tắt công thức / khái niệm

- Giá trị số chuẩn hoá (normal): $(-1)^S \times 1.M_2 \times 2^{(E_2 - \text{bias})}$.
- Giá trị số subnormal: $(-1)^S \times 0.M_2 \times 2^{(1 - \text{bias})}$.
- E4M3: 1 bit dấu, 4 bit exponent (bias = 7), 3 bit mantissa; không có vô cực; 1 mẫu NaN; max normal = 448; min normal = 2^{-6}; max subnormal = 0.875×2^{-6}; min subnormal = 2^{-9}.
- E5M2: 1 bit dấu, 5 bit exponent (bias = 15), 2 bit mantissa; tuân IEEE đầy đủ (có vô cực, nhiều mẫu NaN); max normal = 57 344; min normal = 2^{-14}; max subnormal = 0.75×2^{-14}; min subnormal = 2^{-16}.

- Khuyến nghị sử dụng: E4M3 cho weight/activation (cần độ chính xác), E5M2 cho gradient (cần dynamic range).
- Scaling factor s: trước khi ép tensor x (độ chính xác cao) về FP8, tính $x' = x \times s$ sao cho max $|x'|$ gần giá trị lớn nhất biểu diễn được của định dạng; sau phép toán, khử scale bằng $x'' = y \times s^{-1}$.
- Bão hoà (saturation): giá trị $|x'|$ vượt ngưỡng tối đa bị cắt về giá trị lớn nhất biểu diễn được, thay cho việc bỏ qua cập nhật như FP16 AMP.
- Ví dụ tự tính (mục 3.4): biểu diễn $x = 300$ bằng E4M3 cho kết quả gần nhất 288 (sai số tuyệt đối 12, sai số tương đối $\approx 4\%$); bằng E5M2 cho kết quả gần nhất 320 (sai số tuyệt đối 20, sai số tương đối $\approx 6.7\%$) (tự tính).
- Binade: khoảng giá trị giữa hai luỹ thừa liên tiếp của 2; E4M3 (đã mở rộng) có 18 binade, E5M2 có 32 binade.

11. Kết luận ngắn

Paper đề xuất chuẩn FP8 gồm hai encoding E4M3 (ưu tiên độ chính xác, dùng cho weight/activation) và E5M2 (ưu tiên dynamic range, dùng cho gradient), thiết kế bám sát tối đa quy ước IEEE 754 và chỉ lệch khi lợi ích DL rõ ràng (bỏ vô cực và giảm mã NaN ở E4M3 để mở rộng dynamic range). Thực nghiệm trên nhiều kiến trúc (CNN, RNN, Transformer) và kích cỡ (đến GPT 175B) cho thấy huấn luyện FP8 (mô phỏng) khớp kết quả huấn luyện FP16/bfloat16 mà không cần đổi hyperparameter, và suy luận FP8 PTQ giữ độ chính xác tốt hơn int8 PTQ, đặc biệt khi dùng per-tensor/per-channel scaling factor thay cho exponent bias cố định chung, nhất là với tensor khó như residual connection.

Nguồn gốc: https://arxiv.org/abs/2209.05433

Bài 17

Spectral Normalization for Generative Adversarial Networks (2018)

Tác giả: Takeru Miyato, Toshiki Kataoka, Masanori Koyama, Yuichi Yoshida
Venue / arXiv: arXiv:1802.05957 (v1, 16/02/2018); Published as a conference paper at ICLR 2018 (paper tự ghi rõ ở đầu mỗi trang)
Nguồn: https://arxiv.org/abs/1802.05957
Chủ đề: Đề xuất ”spectral normalization”, một kỹ thuật chuẩn hoá trọng số (weight normalization) ép ràng buộc Lipschitz lên discriminator của GAN bằng cách chia mỗi ma trận trọng số cho giá trị kỳ dị (singular value) lớn nhất của nó, nhằm ổn định huấn luyện GAN.
Vị trí trong lộ trình: nên đọc sau math 13 (Wasserstein GAN, 1701.07875, cùng mã ’math’, số 13) vì cả hai cùng giải quyết bài toán ”ép ràng buộc Lipschitz cho discriminator/critic” nhưng bằng hai cách khác nhau (WGAN dùng weight clipping, bài này dùng chuẩn hoá giá trị kỳ dị lớn nhất); có thể đọc cùng lúc hoặc trước math 12 (Computational Optimal Transport) để hiểu rõ hơn vì sao ràng buộc Lipschitz quan trọng với khoảng cách Wasserstein.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

GAN (Generative Adversarial Network, Goodfellow et al. 2014) là một khung huấn luyện mô hình sinh (generative model) gồm hai mạng: **generator** G sinh ra phân phối mô hình (model distribution) mô phỏng phân phối dữ liệu thật, và **discriminator** D phân biệt mẫu từ phân phối mô hình với mẫu từ phân phối dữ liệu thật. Hai mạng được huấn luyện xen kẽ theo bài toán min-max: $\min_G \max_D V(G, D)$, với mục tiêu gốc (Goodfellow et al. 2014): $V(G, D) = \mathbb{E}_{x\sim q_{data}}[\log D(x)] + \mathbb{E}_{x'\sim p_G}[\log(1 - D(x'))]$.

Vấn đề kéo dài của GAN là **sự bất ổn (instability) của huấn luyện**. Paper nêu rõ nguyên nhân cụ thể (dòng 52-60): trong không gian nhiều chiều, việc ước lượng tỷ số mật độ (density ratio) bởi discriminator thường không chính xác và bất ổn trong quá trình huấn luyện, khiến generator không học được cấu trúc đa phương thức (multimodal) của phân phối mục tiêu. Tệ hơn, khi phân phối mô hình và phân phối dữ liệu thật có **support rời nhau** (disjoint support), tồn tại một discriminator có thể phân biệt hoàn hảo hai phân phối (trích dẫn Arjovsky & Bottou, 2017). Khi discriminator đó xuất hiện, quá trình huấn luyện generator ”hoàn toàn ngừng lại, vì đạo hàm của discriminator đó theo input bằng 0” (dòng 57-59). Đây chính là động lực để **giới hạn tập hợp các discriminator** được phép chọn.

Khái niệm cần giải thích: **hàm Lipschitz liên tục (Lipschitz continuous)**, cụ thể một hàm f là **K-Lipschitz** nếu $\|f(x) - f(x')\|/\|x - x'\| \le K$ với mọi x, x' (dùng chuẩn ℓ_2), tức là đầu ra của hàm không thể thay đổi ”quá nhanh” so với đầu vào. Nhiều công trình trước đó (Uehara et al. 2016; Qi 2017; Gulrajani et al. 2017) đã chỉ ra tầm quan trọng của ràng buộc Lipschitz liên tục để đảm bảo các đại lượng thống kê bị chặn (bounded). Cụ thể, paper dẫn ra discriminator tối ưu của công thức GAN gốc: $D^*_G(x) = \text{sigmoid}(f^*(x))$ với $f^*(x) = \log q_{data}(x) - \log p_G(x)$ (công thức 3), và đạo hàm của nó $\nabla_x f^*(x) = \frac{1}{q_{data}(x)}\nabla_x q_{data}(x) - \frac{1}{p_G(x)}\nabla_x p_G(x)$ (công thức 4) ”có thể không bị chặn hoặc thậm chí không tính được” (dòng 118-119). Đây là lý do cần một điều kiện chính quy (regularity condition) cho đạo hàm của f.

Các công trình thành công trước đó đã kiểm soát hằng số Lipschitz bằng cách thêm hạng tử điều chuẩn (regularization) được định nghĩa trên các mẫu input cụ thể: weight clipping (Arjovsky et al. 2017, chính là WGAN = math 13 trong lộ trình này) và gradient penalty (Gulrajani et al. 2017, WGAN-GP). Paper chỉ ra

hạn chế chung của các phương pháp "dựa trên mẫu input" (input-based regularization): "chúng không thể áp đặt điều chuẩn trên không gian **bên ngoài** support của phân phối generator và dữ liệu mà không cần những biện pháp mang tính heuristic" (dòng 138-140). Đây là động lực để đề xuất **spectral normalization**: một phương pháp điều chuẩn trực tiếp trên **ma trận trọng số**, không phụ thuộc vào mẫu dữ liệu cụ thể.

2. Ý tưởng chính

Ý tưởng cốt lõi: ép **chuẩn kỳ dị (spectral norm)** của mỗi ma trận trọng số W trong discriminator bằng đúng 1, bằng cách chia W cho giá trị kỳ dị lớn nhất (largest singular value) của nó:

$$\bar{W}_{SN}(W) := W/\sigma(W)$$

trong đó $\sigma(W)$ là chuẩn kỳ dị, bằng chính giá trị kỳ dị lớn nhất của W. Lý do việc này đảm bảo tính 1-Lipschitz của toàn bộ discriminator: paper dùng bất đẳng thức cho hợp hàm $\|g_1 \circ g_2\|_{Lip} \leq \|g_1\|_{Lip} \cdot \|g_2\|_{Lip}$, và quan sát rằng với lớp tuyến tính $g(h) = Wh$, chuẩn Lipschitz $\|g\|_{Lip} = \sigma(W)$ (chuẩn kỳ dị chính là hằng số Lipschitz của một phép biến đổi tuyến tính). Nếu hàm kích hoạt (activation) giữa các lớp có $\|a_l\|_{Lip} = 1$ (đúng với ReLU, leaky ReLU), thì toàn mạng f (gồm $L+1$ lớp tuyến tính xen activation) thoả: $\|f\|_{Lip} \leq \prod_{l=1}^{L+1} \sigma(W^l)$ (công thức 7). Nếu mỗi W^l đã được chuẩn hoá để $\sigma(\bar{W}_{SN}(W^l)) = 1$, tích trên $= 1$, nên $\|f\|_{Lip} \leq 1$: toàn bộ discriminator được đảm bảo là 1-Lipschitz, **không cần biết gì về phân phối dữ liệu**, khác hẳn các phương pháp dựa trên mẫu input.

Ý tưởng thứ hai, mang tính thực tiễn: tính đúng giá trị kỳ dị lớn nhất bằng phân rã giá trị kỳ dị (SVD) đầy đủ ở mỗi bước huấn luyện là quá tốn kém tính toán; paper dùng **phương pháp lặp lũy thừa (power iteration method)**, một thuật toán xấp xỉ rẻ, chỉ cần vài phép nhân ma trận-vector mỗi bước, để ước lượng $\sigma(W)$ gần như "miễn phí" so với chi phí forward/backward propagation của toàn mạng.

Ý tưởng thứ ba (phân tích gradient, §2.3): chuẩn hoá phổ không chỉ là một phép biến đổi hình thức mà có một hiệu ứng điều chuẩn tự nhiên (implicit regularization): gradient của hàm mất mát theo W (sau chuẩn hoá) có thêm một hạng tử trừ đi thành phần dọc theo hướng kỳ dị thứ nhất $u_1 v_1^\top$, với hệ số thích nghi λ. Điều này **ngăn không gian cột của W tập trung (concentrate) vào một hướng duy nhất** trong quá trình huấn luyện, khác hẳn một điều chuẩn "tĩnh" như L2 regularization.

3. Phương pháp chi tiết

3.1 Thiết lập discriminator và chuẩn kỳ dị (§2, §2.1)

Paper xét discriminator dạng mạng neural nhiều lớp:

$$f(x, \theta) = W^{L+1} a_L(W^L(a_{L-1}(W^{L-1}(\dots a_1(W^1 x)\dots))))$$

(công thức 1), với $\theta = \{W^1, \dots, W^L, W^{L+1}\}$, $W^l \in \mathbb{R}^{d_l \times d_{l-1}}$, bỏ qua bias term để đơn giản. Đầu ra cuối $D(x, \theta) = A(f(x, \theta))$ với A là hàm kích hoạt tương ứng loại divergence được chọn (ví dụ sigmoid cho GAN gốc).

Định nghĩa chuẩn kỳ dị (spectral norm) của ma trận A:

$$\sigma(A) := \max_{h: h \neq 0} \frac{\|Ah\|_2}{\|h\|_2} = \max_{\|h\|_2 \leq 1} \|Ah\|_2$$

(công thức 6). Khái niệm giải thích cho người mới: $\sigma(A)$ đo "mức giãn (stretch) tối đa" mà phép biến đổi tuyến tính A có thể gây ra lên độ dài của một vector đơn vị bất kỳ; về mặt đại số, $\sigma(A)$ **chính là giá trị kỳ dị**

lớn nhất của A (giá trị kỳ dị, singular value, là căn bậc hai của trị riêng của $A^\top A$). Bằng định nghĩa, chuẩn Lipschitz của một lớp tuyến tính $\|g\|_{Lip} = \sup_h \sigma(\nabla g(h)) = \sigma(W)$.

3.2 Thuật toán lặp lũy thừa (power iteration) để ước lượng $\sigma(W)$ (Appendix A, Algorithm 1)

Paper trình bày thuật toán cụ thể (Algorithm 1 "SGD with spectral normalization"):

- Khởi tạo $\tilde{u}^l \in \mathbb{R}^{d_l}$ cho mỗi lớp $l = 1, \ldots, L$ bằng một vector ngẫu nhiên (lấy mẫu từ phân phối đẳng hướng, isotropic).
- Với mỗi lượt cập nhật và mỗi lớp l:
  1. Áp dụng lặp lũy thừa cho ma trận trọng số **chưa chuẩn hoá** W^l:

$$\tilde{v}_l \leftarrow (W^l)^\top \tilde{u}_l / \|(W^l)^\top \tilde{u}_l\|_2 \quad (20)$$

$$\tilde{u}_l \leftarrow W^l \tilde{v}_l / \|W^l \tilde{v}_l\|_2 \quad (21)$$

  2. Tính $\bar{W}_{SN}$ với chuẩn kỳ dị ước lượng:

$$\bar{W}^l_{SN}(W^l) = W^l / \sigma(W^l), \quad \text{trong đó } \sigma(W^l) = \tilde{u}_l^\top W^l \tilde{v}_l \quad (22)$$

  3. Cập nhật W^l bằng SGD trên mini-batch D_M với learning rate α:

$$W^l \leftarrow W^l - \alpha \nabla_{W^l} \ell(\bar{W}^l_{SN}(W^l), D_M) \quad (23)$$

Điểm mấu chốt về hiệu quả tính toán: paper chỉ chạy **một bước** lặp lũy thừa mỗi lượt cập nhật (không lặp tới khi hội tụ hoàn toàn), và nhận xét "trong thực tế, chúng ta có thể an toàn giả định $\tilde{u}$ sinh từ phân phối đều trên mặt cầu không trực giao với vector kỳ dị thứ nhất, vì điều này chỉ xảy ra với xác suất 0" (chú thích 7, dòng 1105-1107) - vì $\tilde{u}$ được cập nhật liên tục qua các bước huấn luyện (không reset mỗi lần), nó "theo dõi" (track) hướng kỳ dị thứ nhất một cách liên tục, nên chỉ cần 1 bước lặp mỗi lượt vẫn hội tụ tốt theo thời gian huấn luyện. Paper nhấn mạnh "thủ tục này rất rẻ về tính toán, thậm chí so với tính forward/backward propagation của mạng neural" (dòng 1066-1068).

Với trọng số của **lớp convolution** $W \in \mathbb{R}^{d_{out} \times d_{in} \times h \times w}$, paper xử lý bằng cách "coi phép toán như một ma trận 2 chiều kích thước $d_{out} \times (d_{in} hw)$" (dòng 324-325), kèm chú thích rằng vì convolution được thực hiện rời rạc, chuẩn kỳ dị sẽ phụ thuộc vào kích thước stride và padding, nhưng "câu trả lời chỉ khác nhau bởi một hằng số K xác định trước" (dòng 330-331, chú thích 3) - tức đây là một **xấp xỉ có chủ đích**, không phải chuẩn kỳ dị chính xác của phép toán convolution thật.

3.3 Ví dụ số tính tay: lặp lũy thừa trên ma trận 2x2 cụ thể (tự tính)

Để hiểu rõ cơ chế lặp lũy thừa, ta thử trên một ma trận 2×2 cụ thể: $W = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$. Giá trị kỳ dị của ma trận đường chéo này chính là các giá trị tuyệt đối trên đường chéo: $\sigma_1 = 2$, $\sigma_2 = 1$ (đây là trường hợp đặc biệt dễ kiểm chứng trực tiếp vì $W^\top W = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$ đã chéo hoá, trị riêng 4 và 1, căn bậc hai cho $\sigma = 2$ và $\sigma = 1$; tự tính).

Khởi tạo $\tilde{u}_0 = (1, 1)^\top / \sqrt{2}$ (vector đơn vị, không trực giao với vector kỳ dị thứ nhất $(1, 0)^\top$, đúng như giả định của paper).

Bước 1 (công thức 20): $\tilde{v}_1 \leftarrow W^\top \tilde{u}_0 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \begin{pmatrix} 2/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \begin{pmatrix} \sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$ (vì W đối xứng nên $W^\top = W$). Chuẩn hoá: $\|\tilde{v}_1\|_2 = \sqrt{2 + 0.5} = \sqrt{2.5} \approx 1.5811$. Vậy $\tilde{v}_1 \approx (0.8944, 0.3162)$ (tự tính: $\sqrt{2}/1.5811 \approx 0.8944$; $(1/\sqrt{2})/1.5811 \approx 0.3162$).

Bước 2 (công thức 21): $\tilde{u}_1 \leftarrow W\tilde{v}_1 = (2 \times 0.8944,\ 1 \times 0.3162) = (1.7888, 0.3162)$. Chuẩn hoá: $\|\cdot\|_2 = \sqrt{1.7888^2 + 0.3162^2} = \sqrt{3.1998 + 0.1} \approx \sqrt{3.2998} \approx 1.8165$. $\tilde{u}_1 \approx (0.9850, 0.1741)$ (tự tính).

Ước lượng σ sau 1 bước (công thức 22): $\sigma \approx \tilde{u}_1^\top W \tilde{v}_1 = \tilde{u}_1 \cdot (1.7888, 0.3162) = 0.9850 \times 1.7888 + 0.1741 \times 0.3162 \approx 1.7620 + 0.0551 \approx 1.8171$ (tự tính).

Sau chỉ 1 bước lặp, ước lượng $\sigma \approx 1.817$, đã gần với giá trị đúng $\sigma_1 = 2$ (sai số tương đối $\approx (2 - 1.817)/2 = 9.15\%$, tự tính). Nếu lặp thêm một bước nữa (bước 2), $\tilde{u}, \tilde{v}$ sẽ hội tụ nhanh hơn nữa về $(1, 0)^\top$ vì tỷ lệ $\sigma_1/\sigma_2 = 2$ (hội tụ theo tốc độ hình học $(\sigma_2/\sigma_1)^k = (1/2)^k$ sau k bước, kiến thức chung về power iteration, ngoài paper). Minh hoạ này cho thấy trực quan tại sao chỉ cần 1 bước mỗi lượt cập nhật vẫn đủ tốt trong thực hành khi $\tilde{u}$ được giữ liên tục qua nhiều lượt cập nhật (không reset), đúng như lý giải của paper ở chú thích 7.

3.4 Phân tích gradient của trọng số đã chuẩn hoá (§2.3)

Paper tính gradient của $\bar{W}_{SN}(W)$ theo từng phần tử W_{ij}:

$$\frac{\partial \bar{W}_{SN}(W)}{\partial W_{ij}} = \frac{1}{\sigma(W)} \left(E_{ij} - [u_1 v_1^\top]_{ij}\, \bar{W}_{SN} \right) \quad (10)$$

với E_{ij} là ma trận có phần tử (i, j) bằng 1, còn lại bằng 0; u_1, v_1 là vector kỳ dị trái/phải thứ nhất của W. Từ đó, đạo hàm của mục tiêu $V(G, D)$ theo W của discriminator (tính trên mini-batch):

$$\frac{\partial V(G, D)}{\partial W} = \frac{1}{\sigma(W)} \left(\hat{\mathbb{E}}[\delta h^\top] - \lambda u_1 v_1^\top \right) \quad (12)$$

trong đó $\delta := \partial V(G, D)/\partial(\bar{W}_{SN} h)$, $\lambda := \hat{\mathbb{E}}[\delta^\top \bar{W}_{SN} h]$, $\hat{\mathbb{E}}[\cdot]$ là kỳ vọng thực nghiệm trên mini-batch. Paper giải thích ý nghĩa (dòng 232-237): ”hạng tử đầu $\hat{\mathbb{E}}[\delta h^\top]$ giống hệt đạo hàm của trọng số khi không chuẩn hoá. Hạng tử thứ hai có thể coi là một hạng tử điều chuẩn, phạt (penalize) thành phần kỳ dị thứ nhất với hệ số điều chuẩn thích nghi λ. λ dương khi δ và $\bar{W}_{SN} h$ cùng hướng, và điều này ngăn không gian cột của W tập trung vào một hướng cụ thể trong quá trình huấn luyện”. Nói cách khác: spectral normalization **tự động** tạo ra một áp lực chống lại việc ma trận trọng số trở nên ”gần hạng-1” (rank-deficient), khác hẳn với một điều chuẩn độc lập dữ liệu như L2.

3.5 So sánh với các kỹ thuật khác (§3)

Paper so sánh kỹ với 3 kỹ thuật thay thế:

1. **Weight normalization (Salimans & Kingma 2016)**: chuẩn hoá ℓ_2-norm của mỗi vector hàng trong W, tương đương với yêu cầu $\sigma_1(\bar{W}_{WN})^2 + \cdots + \sigma_T(\bar{W}_{WN})^2 = d_o$ (công thức 13, gần giống chuẩn Frobenius). Paper chỉ ra: để tối đa hoá $\|\bar{W}_{WN} h\|_2$ cho một vector đơn vị cố định h, cách tốt nhất là đặt $\sigma_1(\bar{W}_{WN}) = \sqrt{d_o}$ và tất cả giá trị kỳ dị còn lại bằng 0, nghĩa là $\bar{W}_{WN}$ **có hạng 1 (rank one)**. Weight clipping (Arjovsky et al. 2017, WGAN = math 13) cũng ”gặp cùng một cạm bẫy” (dòng 275-276, ”suffers from same pitfall”). Hệ quả: các phương pháp này có **xung đột lợi ích** giữa muốn giữ norm lớn (để discriminator nhạy hơn) và việc giữ norm lớn lại làm giảm hạng ma trận (giảm số đặc trưng hữu dụng), khiến discriminator chỉ phân biệt hai phân phối dựa trên ”một vài đặc trưng chọn lọc” (dòng 260-263).

2. **Orthonormal regularization (Brock et al. 2016)**: thêm hạng tử điều chuẩn $\|W^\top W - I\|_F^2$ (công thức 14) vào mục tiêu đối nghịch. Khác biệt cốt lõi mà paper nhấn mạnh (dòng 289-292): orthonormal regularization ”phá hủy thông tin về phổ (spectrum) bằng cách đặt TẤT CẢ giá trị kỳ dị bằng 1”, còn spectral normalization ”chỉ scale (tỷ lệ) phổ sao cho giá trị lớn nhất bằng 1” - giữ nguyên tỷ lệ tương đối giữa các giá trị kỳ dị, không ép tất cả về 1.

3. **Gradient penalty / WGAN-GP (Gulrajani et al. 2017)**: thêm hạng tử điều chuẩn khuyến khích $\|\nabla_{\hat{x}} f\|_2 = 1$ tại các điểm nội suy $\hat{x} = \epsilon\tilde{x} + (1-\epsilon)x$ giữa mẫu sinh và mẫu thật. Paper chỉ ra điểm yếu: phương pháp này "phụ thuộc nặng vào support của phân phối generator hiện tại" (dòng 298-300), mà support đó thay đổi dần trong quá trình huấn luyện, "có thể làm mất ổn định hiệu ứng của điều chuẩn" (dòng 300-302); paper quan sát thực nghiệm "learning rate cao có thể làm mất ổn định hiệu năng của WGAN-GP" (dòng 302-303). Ngược lại, spectral normalization "điều chuẩn hàm trên không gian toán tử (operator space), hiệu ứng điều chuẩn ổn định hơn với lựa chọn batch" (dòng 303-305), và không dễ mất ổn định với learning rate mạnh. Về chi phí tính toán: "WGAN-GP cần chi phí tính toán nhiều hơn spectral normalization với 1 bước lặp lũy thừa, vì tính $\|\nabla_{\hat{x}} f\|_2$ cần một vòng forward-backward propagation đầy đủ" (dòng 306-309).

4. Thí nghiệm và kết quả chính

Paper thực nghiệm trên CIFAR-10, STL-10, và ILSVRC2012 (ImageNet). Dùng Adam optimizer, thử 6 cấu hình hyperparameter (Table 1): A ($\alpha = 0.0001, \beta_1 = 0.5, \beta_2 = 0.9, n_{dis} = 5$, theo Gulrajani et al. 2017), B ($\alpha = 0.0001, \beta_1 = 0.5, \beta_2 = 0.999, n_{dis} = 1$, theo Warde-Farley & Bengio 2017), C ($\alpha = 0.0002, \beta_1 = 0.5, \beta_2 = 0.999, n_{dis} = 1$, theo Radford et al. 2016), và D, E, F là các cấu hình learning rate "hung hăng" hơn ($\alpha = 0.001$) để kiểm tra độ bền (robustness). n_{dis} là số lượt cập nhật discriminator cho mỗi lượt cập nhật generator.

Kết quả Inception Score và FID (Table 2) trên CIFAR-10 (kiến trúc Standard CNN): Real data = 11.24±.12 (Inception score). Weight clipping = 6.41±.11 (IS), FID 42.6. GAN-GP = 6.93±.08, FID 37.7. WGAN-GP = 6.68±.06, FID 40.2. Batch Norm = 6.27±.10, FID 56.3. Layer Norm = 7.19±.12, FID 33.9. Weight Norm = 6.84±.07, FID 34.7. Orthonormal = 7.40±.12, FID 29.0. **(ours) SN-GANs = 7.42±.08, FID 29.3**. SN-GANs với hinge loss (công thức 17) = 7.58±.12, FID 25.5 (tốt nhất trong nhóm Standard CNN). Trên kiến trúc **ResNet**: Orthonormal với hinge loss = 7.92±.04 (IS), FID 23.8±.58; **SN-GANs với hinge loss = 8.22±.05 (IS), FID 21.7±.21** (tốt nhất toàn bảng cho CIFAR-10 ResNet).

Trên **STL-10** (Standard CNN): Real data = 26.08±.26. Weight clipping = 7.57±.10. WGAN-GP = 8.42±.13. Orthonormal = 8.56±.07. SN-GANs = 8.28±.09 (thấp hơn Orthonormal ở cùng số lượt cập nhật). Nhưng với 2x số lượt cập nhật: Orthonormal (2x) = 8.67±.08, SN-GANs (2x) = 8.69±.09 (SN-GANs vượt nhẹ). Trên kiến trúc ResNet: Orthonormal với hinge loss = 8.72±.06, **SN-GANs với hinge loss = 9.10±.04** (tốt nhất); FID: Orthonormal = 42.4±.99, SN-GANs = 40.1±.50 (tốt nhất, thấp hơn là tốt hơn).

Paper tự nhận một trường hợp SN-GANs **không** vượt trội: "với cùng số lượt huấn luyện, SN-GANs thua Orthonormal regularization trên STL-10" (dòng 490-491, Standard CNN, không phải ResNet) - đây là điểm paper thẳng thắn thừa nhận không phải lúc nào phương pháp đề xuất cũng thắng tuyệt đối.

Độ bền với hyperparameter (Figure 1, 2): "spectral normalization tương đối bền (robust) với learning rate và tham số momentum hung hăng. WGAN-GP thất bại trong việc huấn luyện GAN tốt ở learning rate và momentum cao trên cả CIFAR-10 và STL-10. Orthonormal regularization hoạt động kém ở cấu hình E trên STL-10, nhưng hoạt động tốt hơn một chút so với phương pháp của chúng tôi ở cấu hình tối ưu" (dòng 478-482).

Phân tích giá trị kỳ dị của trọng số đã huấn luyện (Figure 3, §4.1.1): "giá trị kỳ dị của các lớp 1 đến 5 huấn luyện bằng weight clipping và weight normalization tập trung vào một vài thành phần... Ngược lại, giá trị kỳ dị của ma trận trọng số huấn luyện bằng spectral normalization được phân bố rộng hơn hẳn" (dòng 770-774). Paper giải thích hệ quả: "khi mục tiêu là phân biệt một cặp phân phối xác suất trên manifold phi tuyến chiều thấp nhúng trong không gian chiều cao, sự thiếu hạng (rank deficiency) ở các lớp thấp có thể đặc biệt tai hại" (dòng 774-776), vì đầu ra của các lớp thấp "chỉ đi qua một vài phép biến đổi rectified linear, nên có xu hướng nằm trên không gian tuyến tính ở phần lớn vùng" (dòng 776-778).

Chi phí thời gian huấn luyện: "trên CIFAR-10, SN-GANs chậm hơn một chút so với weight normalization

(khoảng 110-120% thời gian tính toán), nhưng nhanh hơn đáng kể so với WGAN-GP" (dòng 784-786), vì WGAN-GP cần tính gradient-của-gradient $\|\nabla_x D\|_2$.

Kết quả trên ImageNet (§4.2): "GAN không điều chuẩn và GAN với layer normalization sụp đổ (collapsed) ngay từ đầu huấn luyện và không tạo được ảnh có ý nghĩa. GAN với orthonormal normalization và spectral normalization có thể tạo ảnh. Inception score của orthonormal normalization chững lại (plateau) quanh vòng lặp thứ 20.000, trong khi SN tiếp tục cải thiện sau đó" (dòng 883-887). Paper tự nhận đây là "nghiên cứu đầu tiên thuộc loại này thành công trong việc tạo ảnh tạm ổn (decent) từ ImageNet với một cặp discriminator-generator duy nhất" (dòng 887-889). Đo mức độ mode-collapse bằng intra MS-SSIM: "SN-GANs (intra MS-SSIM = 0.101) chịu mode-collapse ít hơn AC-GANs (intra MS-SSIM $\sim$ 0.25)" (dòng 890-892).

5. Phân tích / ablation

Mục §4.1.2 là phần ablation/phân tích sâu nhất: so sánh trực tiếp SN-GANs với Orthonormal regularization bằng cách **tăng dần kích thước feature map ở lớp cuối** (lớp conv thứ 7), giữ các thiết lập khác cố định. Kết quả (Figure 4): "hiệu năng của orthonormal regularization giảm dần khi tăng kích thước feature map ở lớp cuối. SN-GANs của chúng tôi, mặt khác, không hề suy giảm với thay đổi kiến trúc này" (dòng 857-860). Paper giải thích lý do: orthonormal regularization "ép discriminator sử dụng TẤT CẢ chiều, bao gồm cả những chiều không cần thiết" (dòng 827-828), vì nó đặt mọi giá trị kỳ dị về 1 (không được phép có giá trị kỳ dị nhỏ/bằng 0 để "bỏ qua" một số chiều), nên khi không gian feature lớn hơn mức cần thiết, việc ép dùng hết mọi chiều trở nên có hại. Spectral normalization không gặp vấn đề này vì "chuẩn kỳ dị độc lập với hạng (rank)" (dòng 279): một ma trận có nhiều giá trị kỳ dị nhỏ/gần 0 vẫn thoả $\sigma(W) = 1$ miễn giá trị **lớn nhất** bằng 1, nên mô hình có "tự do hơn trong việc chọn số thành phần kỳ dị (đặc trưng) cần dùng" (dòng 280-282).

Đây là một ablation có giá trị cao vì nó tách bạch rõ **sự khác biệt bản chất toán học** giữa hai phương pháp tưởng chừng tương tự (cả hai đều liên quan tới giá trị kỳ dị của W): orthonormal ép TẤT CẢ giá trị kỳ dị bằng nhau (và bằng 1), còn spectral normalization chỉ ép giá trị kỳ dị **lớn nhất** bằng 1, cho phép các giá trị kỳ dị khác tự do nhỏ hơn (bao gồm gần 0, tương đương "tắt" một số đặc trưng không cần thiết).

Về mặt ablation hyperparameter (Figure 1-2, Table 1, đã mô tả ở mục 4): paper tự thực hiện một dạng "stress test" có hệ thống qua 6 cấu hình optimizer khác nhau (từ "an toàn" A-C đến "hung hăng" D-F), cho phép so sánh độ bền (không chỉ hiệu năng tốt nhất) của các phương pháp điều chuẩn khác nhau - đây là một thiết kế ablation chặt chẽ hơn so với nhiều paper GAN cùng thời chỉ báo cáo kết quả tốt nhất.

6. Giới hạn

Các giới hạn **paper tự nêu hoặc tự thừa nhận**:

1. "Trong công việc tương lai, chúng tôi muốn tiếp tục điều tra vị trí của phương pháp này so với các phương pháp khác trên cơ sở lý thuyết chặt chẽ hơn" (dòng 905-906, phần Conclusion) - paper tự nhận phân tích lý thuyết hiện tại **chưa đầy đủ**, đặc biệt về việc so sánh định lượng nghiêm ngặt với các phương pháp khác (ví dụ chưa có một chứng minh hội tụ GAN tổng thể khi dùng spectral normalization).
2. Với cấu hình Standard CNN trên STL-10 (cùng số lượt cập nhật), SN-GANs thua Orthonormal regularization (đã nêu ở mục 4); chỉ khi tăng gấp đôi số lượt cập nhật SN-GANs mới vượt nhẹ.
3. Việc tính chuẩn kỳ dị cho lớp convolution là một **xấp xỉ** (coi phép toán như ma trận 2D $d_{out} \times (d_{in}hw)$), "câu trả lời chỉ khác nhau bởi một hằng số K xác định trước" (dòng 330-331) - tức không phải chuẩn kỳ dị thực sự chính xác của phép toán convolution (bản chất convolution là một phép biến đổi tuyến tính có cấu trúc đặc biệt, Toeplitz-like, chuẩn kỳ dị chính xác của nó phức tạp hơn một ma trận dày đặc thông thường).
4. Thuật toán chỉ chạy **1 bước** lặp lũy thừa mỗi lượt cập nhật, nên $\sigma(W)$ ước lượng được chỉ là xấp xỉ (không hội tụ hoàn toàn về giá trị kỳ dị lớn nhất thật, xem ví dụ tự tính ở mục 3.3 với sai số $\sim$ 9% sau

1 bước cho một ma trận cụ thể); paper lý giải điều này ”an toàn” trong thực hành nhờ $\tilde{u}$ được giữ liên tục qua các lượt cập nhật, nhưng đây vẫn là một xấp xỉ, không phải phép tính chính xác tuyệt đối.

5. Kết quả thực nghiệm chỉ trên ảnh (CIFAR-10, STL-10, ImageNet); paper không thử trên các loại dữ liệu khác (văn bản, âm thanh), nên tổng quát hoá sang các loại GAN khác chưa được paper tự kiểm chứng.
6. Paper không khẳng định dứt khoát spectral normalization luôn tốt hơn gradient penalty/WGAN-GP về mặt lý thuyết (chỉ nói ”có thể kết hợp cả hai”, ”can possibly used in combinations”, dòng 904-905), nghĩa là hai phương pháp được paper coi là **bổ sung** nhau hơn là loại trừ lẫn nhau.

7. Liên hệ với các bài khác

- **math 13 (Wasserstein GAN, 1701.07875)**: chưa có file tại thời điểm viết ghi chú này (đang được viết song song theo số quy ước trong titles.tsv). Cả hai bài cùng giải quyết bài toán ”ép ràng buộc Lipschitz cho discriminator/critic của GAN” nhưng bằng hai kỹ thuật khác nhau: WGAN dùng **weight clipping** (cắt trực tiếp giá trị trọng số về một khoảng $[-c, c]$ cố định) - paper hiện tại chỉ ra weight clipping ”gặp cùng cạm bẫy” với weight normalization là làm ma trận trọng số trở thành gần hạng-1 (rank deficient, dòng 275-276); spectral normalization giải quyết vấn đề này bằng cách chỉ ràng buộc giá trị kỳ dị **lớn nhất**, giữ tự do cho các giá trị kỳ dị khác.
- **math 12 (Computational Optimal Transport)**: chưa có file (đang viết song song theo số quy ước). WGAN và lý thuyết Kantorovich-Rubinstein duality (chủ đề của math 12) định nghĩa khoảng cách Wasserstein-1 thông qua supremum trên các hàm 1-Lipschitz; spectral normalization của bài này là một kỹ thuật **thực hành cụ thể** để ép ràng buộc 1-Lipschitz đó lên một mạng neural, nên có thể coi là ”công cụ hiện thực hoá” cho chính ràng buộc lý thuyết mà Kantorovich-Rubinstein duality đòi hỏi.
- **distillation content/02_2015_distilling_knowledge_hinton.md** (đã xác nhận tồn tại bằng ls): không liên quan trực tiếp về nội dung kỹ thuật, nhưng cùng chia sẻ một chủ đề meta: cả hai đều quan tâm tới việc kiểm soát ”độ nhạy” (sensitivity) của một mạng neural theo đầu vào (distillation qua temperature trong softmax, spectral normalization qua ràng buộc Lipschitz theo norm trọng số) để đạt một mục tiêu huấn luyện ổn định hơn.
- **llm content/43_2026_lightonocr.md** (đã xác nhận tồn tại bằng ls): model mục tiêu của dự án người dùng, xem mục 8.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ nội dung mục này là suy luận. LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3600 token/trang, GPU $\leq$ 8GB, batch 1 khi suy luận) **không phải GAN**: dự án không có discriminator, không có generator đối nghịch, không có bài toán min-max. Do đó, cần nêu rõ ngay: **phần lớn nội dung của paper này (huấn luyện ổn định GAN trên ảnh) ít liên quan trực tiếp** tới công việc nén/hậu huấn luyện LightOnOCR.

Tuy vậy có hai liên hệ gián tiếp hợp lý (suy luận, mức độ chắc chắn thấp):

1. **Ràng buộc Lipschitz và ổn định số học khi lượng tử hoá/nén**: ý tưởng cốt lõi của spectral normalization là **kiểm soát giá trị kỳ dị lớn nhất của ma trận trọng số** để tránh một lớp ”khuếch đại” tín hiệu quá mức theo một hướng cụ thể. Khi lượng tử hoá (quantize) trọng số của LightOnOCR về độ chính xác thấp hơn (int8, FP8, xem math 16), **dynamic range của trọng số** (khoảng cách giữa giá trị lớn nhất và nhỏ nhất) ảnh hưởng trực tiếp tới sai số lượng tử hoá: một ma trận có giá trị kỳ dị lớn nhất rất lớn so với phần còn lại (giống hiện tượng ”outlier” quan sát trong llm 51 LLM.int8) có thể gây khó khăn khi chọn scale lượng tử hoá. Về nguyên lý (suy luận), việc theo dõi hoặc kiểm soát chuẩn kỳ dị lớn nhất của các ma trận trọng số trong LightOnOCR (dùng power iteration như thuật toán ở mục 3.2, vốn rất rẻ tính toán) có thể là một **chỉ số chẩn đoán** hữu ích để phát hiện lớp nào ”nhạy” (dễ khuếch đại nhiễu lượng tử hoá) trước khi nén, dù bài này không thiết kế cho mục đích đó.

2. **Residual stream và tính ổn định khi fine-tune/nén**: nếu trong quá trình nén LightOnOCR cần fine-tune lại một phần mạng (ví dụ sau khi cắt bớt một số thành phần, liên hệ tới pruning content đã có thư mục riêng, không lặp lại chi tiết kỹ thuật pruning ở đây), việc một ma trận trọng số có chuẩn kỳ dị lớn nhất quá lớn có thể khiến gradient hoặc activation ”nổ” cục bộ theo một hướng, tương tự vấn đề mà spectral normalization giải quyết cho discriminator GAN. Áp dụng một ràng buộc tương tự (ví dụ theo dõi hoặc giới hạn $\sigma(W)$ của các lớp mới khởi tạo/fine-tune) có thể giúp ổn định quá trình huấn luyện lại, nhưng đây là một **suy luận loại suy (analogy)**, không phải kỹ thuật được paper kiểm chứng cho bối cảnh non-GAN.

Kết luận mục này (suy luận): bài này **ít liên quan trực tiếp** tới việc nén/hậu huấn luyện LightOnOCR vì bối cảnh ứng dụng (ổn định huấn luyện đối nghịch GAN trên ảnh) khác hẳn bối cảnh dự án (nén một mô hình VLM OCR không đối nghịch); giá trị tham khảo chủ yếu nằm ở **khái niệm chuẩn kỳ dị như một công cụ chẩn đoán/kiểm soát độ nhạy của ma trận trọng số**, có thể vận dụng một cách gián tiếp khi phân tích hoặc ổn định hoá các lớp trọng số trong quá trình lượng tử hoá/fine-tune, không nên áp dụng trực tiếp thuật toán Algorithm 1 như một phần của pipeline nén mà không kiểm chứng thực nghiệm riêng.

9. Thí nghiệm gợi ý

1. **Cài thuật toán power iteration và kiểm chứng trên ma trận ngẫu nhiên**: viết hàm Python/PyTorch hiện thực chính xác Algorithm 1 (công thức 20-22) cho một ma trận ngẫu nhiên kích thước vài trăm chiều, chạy GPU 8GB (hoặc CPU, vì bài toán rất nhẹ). So sánh ước lượng $\sigma(W)$ sau 1, 5, 20 bước lặp với giá trị kỳ dị lớn nhất tính bằng `torch.linalg.svd` (chính xác). Kết quả kỳ vọng: sai số giảm theo cấp số nhân theo tỷ lệ $(\sigma_2/\sigma_1)^k$; sau khoảng 10-20 bước, sai số nhỏ hơn 1%. Diễn giải: xác nhận trực tiếp cơ chế hội tụ của power iteration đã minh hoạ bằng tay ở mục 3.3, với một ma trận lớn hơn và nhiều bước hơn.

2. **Toy GAN có/không spectral normalization trên dữ liệu 2D**: huấn luyện một GAN rất nhỏ (generator/discriminator vài lớp fully-connected) để học một phân phối 2D đơn giản (ví dụ hỗn hợp Gaussian nhiều mode), chạy nhanh trên CPU/GPU 8GB. So sánh độ ổn định huấn luyện (ví dụ dao động của loss, có mode collapse hay không) giữa phiên bản có và không có spectral normalization trên discriminator, với learning rate ”hung hăng” cố ý (giống thiết lập D-F của paper). Kết quả kỳ vọng: phiên bản không chuẩn hoá dễ mất ổn định/mode collapse hơn ở learning rate cao, giống quan sát ở Figure 1-2 của paper (dù ở quy mô nhỏ hơn hẳn). Diễn giải: tái hiện trực tiếp phát hiện cốt lõi của paper (SN giúp bền hơn với learning rate cao) ở quy mô kiểm soát được.

3. **Đo phân bố giá trị kỳ dị của một mô hình đã huấn luyện (không phải GAN)**: lấy một mô hình nhỏ đã huấn luyện (ví dụ một mạng phân loại ảnh nhỏ, hoặc nếu có thể truy cập, một vài ma trận trọng số của LightOnOCR), tính giá trị kỳ dị của mỗi ma trận trọng số (dùng SVD đầy đủ vì mô hình nhỏ, không cần power iteration), vẽ phân phối giá trị kỳ dị đã chuẩn hoá theo giá trị lớn nhất (giống Figure 3 của paper). Kết quả kỳ vọng: không có kỳ vọng cụ thể từ paper (paper chỉ đo trên discriminator GAN), nhưng nếu một số lớp có giá trị kỳ dị tập trung mạnh vào vài thành phần (gần hạng thấp), đó là dấu hiệu lớp đó có thể nhạy với lượng tử hoá. Diễn giải: đây là thí nghiệm mang tính thăm dò (exploratory) áp dụng công cụ phân tích của paper (phân tích giá trị kỳ dị) sang bối cảnh khác (mô hình không đối nghịch), hữu ích để hiểu cấu trúc nội tại của mô hình trước khi nén.

4. **So sánh weight clipping vs spectral normalization trên cùng một discriminator nhỏ**: huấn luyện hai phiên bản của cùng một GAN nhỏ (toy, trên MNIST hoặc dữ liệu 2D), một dùng weight clipping (cắt trọng số về $[-0.01, 0.01]$ như WGAN gốc), một dùng spectral normalization, đo và so sánh phân phối giá trị kỳ dị của ma trận trọng số discriminator sau huấn luyện (giống Figure 3). Kết quả kỳ vọng: weight clipping cho giá trị kỳ dị tập trung mạnh vào thành phần đầu (gần hạng-1), spectral normalization cho phân phối rộng hơn. Diễn giải: xác nhận trực tiếp tuyên bố cốt lõi của paper ở §3 (weight clipping

gây rank deficiency, spectral normalization không) bằng thực nghiệm tự làm, không chỉ dựa vào lý thuyết.

5. **Đo chi phí tính toán thêm của spectral normalization**: đo thời gian một bước huấn luyện (forward + backward + update) của một mạng nhỏ-trung bình trên GPU 8GB, có và không có power iteration (1 bước) mỗi lớp. Kết quả kỳ vọng: chi phí thêm rất nhỏ (paper báo cáo 110-120% thời gian so với không chuẩn hoá trên CIFAR-10, dòng 784-786), xác nhận tuyên bố "tính toán nhẹ" (computationally light) của paper. Diễn giải: giúp người dùng tự đánh giá liệu overhead của việc theo dõi chuẩn kỳ dị (nếu dùng như công cụ chẩn đoán cho LightOnOCR, xem mục 8) có đáng chi phí tính toán trên phần cứng GPU 8GB hạn chế hay không.

10. Tóm tắt công thức / khái niệm

- Chuẩn kỳ dị (spectral norm): $\sigma(A) = \max_{h \neq 0} \|Ah\|_2 / \|h\|_2$ = giá trị kỳ dị lớn nhất của A.
- Spectral normalization: $\bar{W}_{SN}(W) = W / \sigma(W)$, đảm bảo $\sigma(\bar{W}_{SN}) = 1$.
- Chặn Lipschitz toàn mạng: $\|f\|_{Lip} \leq \prod_{l=1}^{L+1} \sigma(W^l)$ (công thức 7); sau chuẩn hoá mỗi lớp, $\|f\|_{Lip} \leq 1$.
- Power iteration (Algorithm 1, công thức 20-22): $\tilde{v} \leftarrow W^\top \tilde{u} / \|W^\top \tilde{u}\|_2$; $\tilde{u} \leftarrow W\tilde{v} / \|W\tilde{v}\|_2$; $\sigma(W) \approx \tilde{u}^\top W \tilde{v}$. Chỉ 1 bước mỗi lượt cập nhật, $\tilde{u}$ được giữ liên tục qua các lượt.
- Gradient của trọng số chuẩn hoá (công thức 10, 12): $\frac{\partial V}{\partial W} = \frac{1}{\sigma(W)} \left(\hat{\mathbb{E}}[\delta h^\top] - \lambda u_1 v_1^\top \right)$, với $\lambda = \hat{\mathbb{E}}[\delta^\top \bar{W}_{SN} h]$; hạng tử thứ hai phạt thành phần kỳ dị thứ nhất, chống rank collapse.
- Weight normalization (so sánh, công thức 13): $\sigma_1(\bar{W}_{WN})^2 + \cdots + \sigma_T(\bar{W}_{WN})^2 = d_o$; tối ưu hoá norm dẫn tới ma trận hạng 1.
- Orthonormal regularization (so sánh, công thức 14): hạng tử điều chuẩn $\|W^\top W - I\|_F^2$, ép mọi giá trị kỳ dị về 1 (khác spectral normalization chỉ ép giá trị lớn nhất về 1).
- Ví dụ tự tính (mục 3.3): với $W = \text{diag}(2, 1)$, $\sigma_1 = 2$ đúng; sau 1 bước power iteration từ $\tilde{u}_0 = (1, 1)/\sqrt{2}$, ước lượng $\sigma \approx 1.817$ (sai số $\approx 9.15\%$), minh hoạ tốc độ hội tụ hình học $(\sigma_2/\sigma_1)^k = (1/2)^k$ (tự tính).
- Inception Score (IS): $I(\{x_n\}) = \exp(\mathbb{E}[D_{KL}[p(y|x) \| p(y)]])$, cao hơn tốt hơn; FID (Fréchet Inception Distance): khoảng cách 2nd-order giữa phân phối feature của ảnh thật và ảnh sinh, thấp hơn tốt hơn.

11. Kết luận ngắn

Paper đề xuất spectral normalization: chia mỗi ma trận trọng số của discriminator cho giá trị kỳ dị lớn nhất của nó, đảm bảo toàn mạng discriminator là 1-Lipschitz một cách trực tiếp (không cần mẫu dữ liệu), tính toán rẻ nhờ power iteration một bước mỗi lượt cập nhật. Khác biệt cốt lõi so với weight normalization/weight clipping (gây rank deficiency) và orthonormal regularization (phá huỷ thông tin phổ) là spectral normalization chỉ ràng buộc giá trị kỳ dị lớn nhất, giữ tự do cho các giá trị kỳ dị khác, cho phép discriminator dùng nhiều đặc trưng hơn. Thực nghiệm trên CIFAR-10, STL-10, ImageNet cho thấy SN-GANs đạt Inception Score/FID tốt hơn hoặc tương đương các kỹ thuật ổn định hoá GAN trước đó, bền hơn với learning rate cao, và là nghiên cứu đầu tiên (theo paper tự nhận) tạo được ảnh chất lượng tạm ổn từ ImageNet với một cặp generator-discriminator duy nhất.

Nguồn gốc: https://arxiv.org/abs/1802.05957

Bài 18

The Shattered Gradients Problem: If resnets are the answer, then what is the question? (2017)

Tác giả: David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, Brian McWilliams
Venue / arXiv: arXiv:1702.08591 (v2, 06/06/2018); Proceedings of the 34th International Conference on Machine Learning (ICML), Sydney, Australia, PMLR 70, 2017 (paper tự ghi rõ ở trang đầu)
Nguồn: https://arxiv.org/abs/1702.08591
Chủ đề: Phân tích toán học vì sao gradient của mạng feedforward dùng rectifier (ReLU) càng sâu càng giống nhiễu trắng (white noise), làm huấn luyện khó khăn; chứng minh skip-connection (resnet) làm chậm hẳn quá trình này, và đề xuất một cách khởi tạo khác (”looks linear”) giải quyết vấn đề mà không cần skip-connection.
Vị trí trong lộ trình: nên đọc sau opt 37 (Delving Deep into Rectifiers / He init, 1502.01852) và opt 38 (Batch Normalization, 1502.03167) vì bài này dùng trực tiếp giả định khởi tạo của He et al. và phân tích tương tác với batch normalization; liên hệ với math 20 (Benefits of depth in neural networks) về số lượng vùng tuyến tính (linear region) tăng theo độ sâu, là nguyên nhân gốc của hiện tượng ”vỡ gradient” theo chính paper này.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Một trong những trở ngại lâu đời nhất của deep learning là vấn đề **gradient biến mất/nổ** (vanishing/exploding gradients): khi lan truyền ngược (backpropagation) qua nhiều lớp, gradient có thể co lại về 0 hoặc phình to vô hạn, khiến việc cập nhật trọng số ở các lớp sâu trở nên vô nghĩa. Paper mở đầu bằng nhận xét: vấn đề này ”đã phần lớn được giải quyết” nhờ khởi tạo trọng số cẩn thận (careful initialization, ví dụ Glorot & Bengio 2010, He et al. 2015) kết hợp batch normalization (BN, Ioffe & Szegedy 2015). He initialization đảm bảo phương sai (variance) của activation được giữ nguyên qua các lớp rectifier (ReLU), còn BN đảm bảo lan truyền ngược không bị ảnh hưởng bởi độ lớn (scale) của trọng số.

Nhưng có một nghịch lý thực nghiệm: dù đã dùng khởi tạo tốt và BN, **kiến trúc có skip-connection** (như highway network và resnet) vẫn hoạt động tốt hơn hẳn kiến trúc feedforward chuẩn khi mạng đủ sâu (He et al. 2016a;b). Câu hỏi mà paper đặt ra ngay từ tiêu đề: ”Nếu resnet là câu trả lời, thì câu hỏi là gì?” (”If resnets are the answer, then what is the question?”). Nói cách khác: nếu vanishing/exploding gradient đã được giải quyết bằng khởi tạo + BN, thì **tại sao resnet vẫn cần thiết và vẫn tốt hơn**? Paper xác định một vấn đề mới, trực giao (orthogonal) với vanishing/exploding gradient, gọi là **vấn đề gradient bị vỡ (shattered gradients problem)**.

Khái niệm cần giải thích trước khi đọc tiếp: **rectifier** (ReLU, Rectified Linear Unit) là hàm kích hoạt $\rho(a) = \max(0, a)$, không mượt (not smooth) tại $a = 0$ (có một điểm gãy, đạo hàm không liên tục). Paper phân biệt rõ giữa **đầu ra thực (real-valued output)** của rectifier và **activation nhị phân (binary activation)**: activation = 1 nếu đầu ra dương, = 0 nếu không (dòng 39-40 phần Terminology). Đây là phân biệt quan trọng cho phần phân tích sau: mỗi neuron rectifier, tại một điểm dữ liệu cụ thể, ”đang hoạt động” (active, output > 0, hành xử như tuyến tính) hoặc ”không hoạt động” (inactive, output = 0).

2. Ý tưởng chính

Ý tưởng cốt lõi: **tương quan (correlation) giữa gradient tại các điểm dữ liệu khác nhau** trong mạng feedforward chuẩn **suy giảm theo hàm mũ (exponentially) với độ sâu**, khiến gradient (xét như một hàm theo input, hoặc xét giữa các mini-batch) trông giống **nhiễu trắng (white noise)**: không có cấu trúc không gian,

giá trị tại các điểm gần nhau không còn tương quan với nhau. Ngược lại, trong kiến trúc có skip-connection (resnet, highway network), tương quan này **chỉ suy giảm theo tốc độ dưới tuyến tính (sublinear)**, cụ thể là theo $1/\sqrt{L}$ (L là độ sâu), khiến gradient giữ được nhiều cấu trúc hơn, gần giống **nhiễu nâu (brown noise, tức chuyển động Brown / Brownian motion)** hơn là nhiễu trắng.

Hệ quả thực tiễn: khi gradient giống nhiễu trắng, các thuật toán tối ưu giả định ”gradient tại các điểm gần nhau là giống nhau” (ví dụ phương pháp dùng momentum, phương pháp tăng tốc/accelerated methods) sẽ hoạt động kém, vì ”ảnh hưởng của một neuron lên đầu ra mạng (tăng trọng số làm đầu ra tăng hay giảm) trở nên cực kỳ không ổn định, khiến việc học khó khăn” (dòng 183-186, ”becomes extremely unstable making learning difficult”). Paper cũng nhấn mạnh phân tích này là **tại thời điểm khởi tạo (at initialization)**, chưa xét quá trình huấn luyện: ”shattering nên giảm trong quá trình huấn luyện. Hiểu cách shattering ảnh hưởng huấn luyện là một vấn đề mở quan trọng” (dòng 34-35 phần Abstract).

Ý tưởng thứ hai, mang tính kỹ thuật phân tích: gradient của mạng rectifier có thể biểu diễn như **tổng theo các đường đi (path-sum)** qua mạng, trong đó mỗi ”đường đi” (path) chỉ góp phần nếu TẤT CẢ neuron trên đường đi đó đang ”hoạt động” (active). Khi mạng sâu hơn, số lượng đường đi tăng theo hàm mũ với độ sâu, nhưng tỷ lệ đường đi ”cùng hoạt động” ở hai điểm dữ liệu khác nhau giảm nhanh hơn, dẫn đến tương quan giữa gradient giảm theo hàm mũ.

3. Phương pháp chi tiết

3.1 Mô hình tối giản và quan sát trực quan (§1.1)

Để loại bỏ các yếu tố gây nhiễu (gradient phụ thuộc cả loss và mẫu dữ liệu ngẫu nhiên, và có số chiều rất lớn), paper xây dựng **mô hình tối giản**: mạng $f_W : \mathbb{R} \to \mathbb{R}$ nhận số thực vào số thực (không phải dữ liệu thật), mỗi lớp ẩn có $N = 200$ neuron rectifier. Paper quan sát đạo hàm $\frac{df_W}{dx}(x^{(i)})$ với $x^{(i)} \in [-2, 2]$ trên lưới $M = 256$ ”điểm dữ liệu” (Eq. 1).

Quan sát trực quan (Figure 1 của paper, mô tả bằng lời vì không đọc được hình trực tiếp từ text, nhưng chú thích hình được đọc đầy đủ): mạng feedforward 1 lớp cho gradient có cấu trúc giống **nhiễu nâu (brown noise)** (so sánh trực tiếp với xấp xỉ rời rạc của chuyển động Brown $B^N(t) = \sum_{s=1}^{t} W_s$ với $W_s \sim N(0, 1/N)$, công thức dòng 182-184); mạng feedforward 24 lớp cho gradient giống **nhiễu trắng (white noise, lấy mẫu $W_k \sim N(0, 1)$)**; mạng resnet 50 lớp (dùng batch normalization với $\beta = 0.1$) cho gradient có cấu trúc ***ở giữa*** hai cực đó, ”nhiều cấu trúc hơn hẳn mạng feedforward tương đương” (dòng 242-243).

Paper giải thích cơ sở lý thuyết của sự tương đồng với Brownian motion: ”mục A1 áp dụng định lý Donsker để chứng minh gradient hội tụ về chuyển động Brown khi $N \to \infty$” (dòng 137-138). Về mặt trực giác: nếu tất cả neuron là tuyến tính (linear), gradient sẽ là một đường ngang (constant theo x); nhưng rectifier không mượt nên gradient có các điểm gián đoạn (discontinuous) tại các ngưỡng bias của từng neuron, và việc tích luỹ nhiều điểm gián đoạn ngẫu nhiên (độc lập) qua nhiều lớp tạo ra hành vi giống bước ngẫu nhiên (random walk).

Để đo lường định lượng cấu trúc không gian của gradient, paper dùng **hàm tự tương quan (Autocorrelation Function, ACF)**, so sánh ACF của gradient mạng feedforward/resnet ở các độ sâu khác nhau với ACF của nhiễu trắng và nhiễu nâu (Figure 2). Kết quả định tính: ”ACF của mạng nông giống ACF nhiễu nâu. Khi mạng sâu hơn, ACF nhanh chóng giống ACF nhiễu trắng” (dòng 155-159) cho feedforward; nhưng với resnet, ”mặc dù gradient trở nên kém có cấu trúc hơn dần, chúng không ’trắng hoá’ (whiten) đến mức độ của mạng feedforward chuẩn... vẫn còn tương quan trong resnet 50 lớp trong khi ở mạng feedforward tương đương, gradient không thể phân biệt được với nhiễu trắng” (dòng 244-249). Paper cũng chỉ ra hiệu ứng của β-**rescaling** (Szegedy et al. 2016): ”hiệu ứng ấn tượng của β-rescaling mới được đề xuất: ACF của mạng 50 lớp giống nhiễu nâu” khi dùng β nhỏ (dòng 250-252).

3.2 Định nghĩa toán học chính thức (§3)

Paper định nghĩa (Definition 1): cho mỗi input $x^{(i)} \in D$, $\nabla_i := \sum_{p=1}^{P} \frac{\partial f_p}{\partial n}(x^{(i)})$ là đạo hàm của (tổng các toạ độ đầu ra) f_p theo đầu ra n của một neuron cụ thể, tính tại input $x^{(i)}$. Vì trọng số được lấy mẫu từ phân phối trung bình 0, ∇_i có kỳ vọng 0. Hiệp phương sai (covariance) và tương quan (correlation) giữa gradient tại hai điểm dữ liệu được định nghĩa:

$$C(i,j) = \mathbb{E}[\nabla_i \nabla_j] \quad \text{và} \quad R(i,j) = \frac{\mathbb{E}[\nabla_i \nabla_j]}{\sqrt{\mathbb{E}[\nabla_i^2] \cdot \mathbb{E}[\nabla_j^2]}}$$

(kỳ vọng lấy theo phân phối của trọng số). Lưu ý quan trọng paper tự giải thích (dòng 146-154): theo quy tắc chuỗi (chain rule), cập nhật trọng số $\frac{\partial f_W}{\partial w_{ij}} = \frac{\partial f_W}{\partial n_j} \cdot \frac{\partial n_j}{\partial w_{ij}}$; thành phần $\frac{\partial f_W}{\partial n_j}$ (đạo hàm đầu ra mạng theo đầu ra neuron ở một lớp, tức là chính đại lượng ∇_i mà paper phân tích) là thành phần **phụ thuộc vào cấu trúc mạng (kiến trúc, độ sâu)**, còn các thành phần khác không phụ thuộc. Đây là lý do paper tập trung phân tích ∇_i thay vì toàn bộ gradient trọng số.

Giả định điểm dữ liệu điển hình (Assumption 1, "typical datapoints"): hai điểm dữ liệu $x^{(i)}, x^{(j)}$ được coi là "điển hình" nếu một nửa neuron mỗi lớp đang hoạt động cho mỗi điểm, và một phần tư neuron mỗi lớp cùng hoạt động cho cả hai điểm (tức là xác suất "cùng active" của hai neuron độc lập là $0.5 \times 0.5 = 0.25$, tự tính, khớp với giả định "một phần tư"). Giả định này được paper kiểm chứng thực nghiệm bằng Figure 3: với batch normalization, tỷ lệ "cùng hoạt động" (coactivation) của một cặp input phân biệt là **đúng 1/4** (dòng 324, "neurons are co-active for 1/4 of distinct pairs of inputs, which is what would happen if activations were decided by unbiased coin flips"), còn **không có** batch normalization, tỷ lệ cùng hoạt động "tăng dần theo độ sâu, cho thấy phản hồi neuron ngày càng dư thừa (redundant)" (dòng 326-328) - tức là giả định "điểm dữ liệu điển hình" chỉ đúng tốt khi có BN.

3.3 Định lý 1: mạng feedforward (§3.1)

Giả định trọng số được khởi tạo với phương sai $\sigma^2 = 2/N$ theo He et al. (2015). Với một neuron n cách đầu ra L lớp, **Định lý 1 (hiệp phương sai gradient trong mạng feedforward)**:

a) Phương sai gradient tại $x^{(i)}$: $C^{fnn}(i) = 1$.

b) Hiệp phương sai: $C^{fnn}(i,j) = \frac{1}{2^L}$.

Paper giải thích ý nghĩa: phần (a) "khôi phục lại quan sát của He et al. (2015) rằng đặt $\sigma^2 = 2/N$ giữ nguyên phương sai qua các lớp trong mạng rectifier" (dòng 388-389) - tức đây là một kiểm chứng nhất quán với lý thuyết He init đã biết. Phần (b) là **phát hiện mới** (dòng 389, "Part (b) is new"), giải thích trực tiếp quan sát thực nghiệm ở Figure 2a rằng gradient trong mạng feedforward "trắng hoá" theo độ sâu: "trực giác, gradient trắng hoá vì số đường đi (path) qua mạng tăng theo hàm mũ nhanh hơn hẳn phần đường đi cùng hoạt động (co-active)" (dòng 395-398). Vì $C^{fnn}(i) = 1$ (hằng số, không đổi theo L) nhưng $C^{fnn}(i,j) = 1/2^L \to 0$ theo hàm mũ, **tương quan** $R^{fnn}(i,j) = C^{fnn}(i,j)/C^{fnn}(i) = 1/2^L$ cũng suy giảm theo hàm mũ với L.

3.4 Định lý 2: resnet không có batch normalization (§3.2)

Module residual gốc (He et al. 2016a): $x_l = x_{l-1} + W_l \rho_{BN}(V_l \rho_{BN}(x_{l-1}))$, trong đó $\rho_{BN}(a) = \rho(BN(a))$. Paper phân tích phiên bản đơn giản hoá (bỏ V_l, không làm thay đổi bản chất phân tích):

$$x_l = \alpha \cdot x_{l-1} + \beta \cdot W_l \rho_{BN}(x_{l-1}) \tag{2}$$

với α, β là hệ số tái chuẩn hoá (rescaling factor). **Định lý 2 (hiệp phương sai gradient trong resnet)**: xét resnet **không** dùng batch normalization, $\alpha = \beta = 1$, $\sigma^2 = 2/N$ như trên:

a) Phương sai: $C^{res}(i) = 2^L$.

b) Hiệp phương sai: $C^{res}(i,j) = \left(\frac{3}{2}\right)^L$.

Suy ra tương quan $R^{res}(i,j) = \left(\frac{3}{4}\right)^L$ (tự tính: $(3/2)^L/2^L = (3/4)^L$, khớp với công thức paper ghi trực tiếp ở dòng 439). Paper chỉ rõ **hệ quả tiêu cực**: định lý này ngụ ý resnet không có BN có **hai vấn đề**: (i) phương sai gradient tăng (theo 2^L, tức là *nổ*/explode theo độ sâu), và (ii) tương quan vẫn suy giảm theo hàm mũ (theo $(3/4)^L$, dù chậm hơn $(1/2)^L$ của feedforward vì $3/4 > 1/2$). Cả hai vấn đề này "đều quan sát được thực nghiệm" (dòng 443-444).

3.5 Rescaling và Định lý 3: resnet có batch normalization (§3.3)

Giải pháp cho phương sai nổ: rescale bằng $\alpha = 1/\sqrt{2}$, cho $C^{res}_{\alpha=1/\sqrt{2}}(i) = 1$ (ổn định phương sai) nhưng $R^{res}_{\alpha=1/\sqrt{2}}(i,j) = (3/4)^L$ **vẫn giữ nguyên tốc độ suy giảm theo hàm mũ** của tương quan (dòng 450-459) - tức rescaling α đơn thuần không giải quyết được vấn đề "vỡ gradient", chỉ giải quyết vấn đề phương sai nổ. Trong thực tế, α-rescaling không được dùng; thay vào đó activation được tái chuẩn hoá bằng batch normalization, kết hợp đặt $\beta \in [0.1, 0.3]$ theo Szegedy et al. (2016).

Định lý 3 (hiệp phương sai gradient trong resnet có BN và β-rescaling): với resnet có batch normalization và β-rescaling,

a) Phương sai: $C^{res}_{\beta,BN}(i) = \beta^2(L-1) + 1$.

b) Hiệp phương sai: $C^{res}_{\beta,BN}(i,j) \sim \beta\sqrt{L}$.

c) Tương quan: $R^{res}_{\beta,BN}(i,j) \sim \frac{1}{\beta\sqrt{L}}$ (kèm chú thích [1] trỏ tới mục A3.4 phụ lục để xem tính toán chính xác, paper không chứng minh đầy đủ trong thân bài).

Đây là kết quả **then chốt** của paper: tốc độ suy giảm tương quan đổi từ **hàm mũ** $1/2^L$ (feedforward) sang **dưới tuyến tính** $1/\sqrt{L}$ (resnet có BN). Paper giải thích trực giác: "phương sai đầu ra của các lớp tăng tuyến tính, nên batch normalization tái chuẩn hoá chúng theo các lượng khác nhau. Rescale bằng β thêm một hệ số hằng" (dòng 492-495).

3.6 Ví dụ số minh hoạ từ chính paper, và tính tay kiểm chứng (tự tính)

Paper tự đưa ra một ví dụ số cụ thể (dòng 496-501): "mô hình dự đoán khi dùng batch normalization với $\beta = 0.1$ trên resnet 100 lớp cho tương quan điển hình $R^{res}_{0.1,BN}(i,j) = 0.7$. Đặt $\beta = 1.0$ cho $R^{res}_{1.0,BN}(i,j) = 0.1$. Ngược lại, mạng feedforward 100 lớp có tương quan không thể phân biệt được với 0."

Kiểm tra lại bằng công thức $R \sim 1/(\beta\sqrt{L})$ (tự tính): với $\beta = 0.1$, $L = 100$: $1/(0.1 \times \sqrt{100}) = 1/(0.1 \times 10) = 1/1 = 1.0$. Với $\beta = 1.0$, $L = 100$: $1/(1.0 \times 10) = 0.1$. Nhận xét tự rút ra (suy luận của người ghi chú): công thức "$\sim$" (xấp xỉ tỉ lệ, không phải đẳng thức chính xác, ký hiệu paper dùng ở Theorem 3b/3c) khớp đúng với trường hợp $\beta = 1.0$ (ra đúng 0.1 như paper nói), nhưng với $\beta = 0.1$ công thức xấp xỉ cho ra 1.0 trong khi paper nói con số cụ thể là 0.7; đây không phải mâu thuẫn mà là bản chất của ký hiệu tỷ lệ bậc lớn ($\sim$, tương tự $O(\cdot)$ trong giải tích), không phải công thức chính xác tuyệt đối - công thức chính xác đầy đủ được paper nói "xem mục A3.4" (phụ lục, không đọc chi tiết công thức đóng chính xác trong ghi chú này, chỉ đọc phần kết quả §3 chính). Đây là điểm cần lưu ý khi dùng công thức Theorem 3 cho mục đích tính toán cụ thể: chỉ nên dùng để hiểu xu hướng (order of magnitude), không nên coi là công thức đóng chính xác.

3.7 Highway network (§3.4)

Highway network (Srivastava et al. 2015) có dạng $x_l = (1 - T(x_{l-1})) \cdot x_{l-1} + T(x_{l-1}) \cdot H(x_{l-1})$ với T, H là cổng (gate) và đặc trưng (feature) học được. Paper xét biến thể đơn giản hoá với cổng là **số vô hướng**

(scalar) cố định γ_1, γ_2 thoả $\gamma_1^2 + \gamma_2^2 = 1$: $x_l = \gamma_1 \cdot x_{l-1} + \gamma_2 \cdot W_l \rho(x_{l-1})$.

Corollary 1 (hiệp phương sai gradient trong highway network): a) phương sai $C_\gamma^{HN}(i) = 1$; b) tương quan $R_\gamma^{HN}(i,j) = \left(\gamma_1^2 + \frac{1}{2}\gamma_2^2\right)^L$.

Điểm đặc biệt: nếu chọn $\gamma_1 = \sqrt{1 - 1/L}$ và $\gamma_2 = \sqrt{1/L}$ (tức là các hệ số phụ thuộc vào chính độ sâu L), thì khi $L \to \infty$:

$$\lim_{L\to\infty} R_\gamma^{HN}(i,j) = \frac{1}{\sqrt{e}}$$

(tự kiểm chứng bằng giới hạn toán học: $\left(1 - \frac{1}{L} + \frac{1}{2L}\right)^L = \left(1 - \frac{1}{2L}\right)^L \to e^{-1/2} = 1/\sqrt{e} \approx 0.6065$ khi $L \to \infty$, tự tính, khớp với công thức paper nêu). Đây là kết quả ấn tượng nhất về mặt lý thuyết: tương quan **không suy giảm theo độ sâu** (tiến tới một hằng số dương $1/\sqrt{e} \approx 0.6065$ chứ không về 0), đạt "độ sâu độc lập" (depth-independent correlation structure, đúng như tiêu đề Corollary 1). Đánh đổi (trade-off) mà paper nêu ngay sau: "sự đánh đổi là đóng góp của các lớp trở nên ngày càng tầm thường (gần với identity) khi $L \to \infty$" (dòng 478) - vì $\gamma_2 = \sqrt{1/L} \to 0$, mỗi lớp gần như không biến đổi gì cả khi mạng rất sâu.

4. Thí nghiệm và kết quả chính

4.1 Hiệu quả neuron và batch normalization (§2)

Paper quan sát (Figure 3, 100 mạng fully-connected rectifier với 100 unit ẩn mỗi lớp, trung bình trên 100 lần chạy): tỷ lệ neuron "hoạt động" (activation) trung bình luôn khoảng 1/2 bất kể có/không có BN, nhưng **phân phối trở nên lưỡng cực (bimodal) theo độ sâu** khi không có BN: "ở lớp 50 trong mạng feedforward không BN, neuron hoặc luôn hoạt động hoặc luôn không hoạt động" (dòng 334-337). BN làm "hầu hết neuron hoạt động cho đúng một nửa input" (dòng 339-341). Paper rút ra: "neuron luôn hoạt động thì cũng như tuyến tính; neuron luôn không hoạt động thì cũng như không tồn tại. Do đó batch normalization tăng hiệu quả sử dụng tính phi tuyến rectifier" (dòng 343-346) - đây là một đóng góp phụ của paper, độc lập với phân tích chính về gradient.

Figure 4 minh hoạ bằng raster-plot: activation của mạng feedforward không BN "giống nhiễu tĩnh truyền hình (static television noise), cấu trúc không gian bị phá huỷ hoàn toàn" (dòng 282-283); resnet "thể hiện một sự thoả hiệp: neuron được sử dụng hiệu quả nhưng cấu trúc không gian cũng được bảo toàn phần nào" (dòng 283-285), đo bằng "contiguity histogram" (biểu đồ đếm độ dài các dãy hoạt động liên tục).

4.2 Gradient vỡ trong convnet trên CIFAR-10 (§4)

Paper kiểm tra các kết quả lý thuyết (vốn chứng minh cho mạng fully-connected) có đúng với **convnet trên dữ liệu thật** (CIFAR-10) không. Thiết lập: mạng feedforward và resnet với 2, 4, 10, 24, 50 lớp, dùng biến thể kiến trúc "bottleneck" (He et al. 2016a), chèn một skip-connection cho mỗi hai lớp convolution, cả hai kiến trúc đều dùng batch normalization.

Figure 5a-b: so sánh ma trận hiệp phương sai của gradient ở lớp đầu tiên, minibatch 256 mẫu ngẫu nhiên từ CIFAR-10, độ sâu 2 và 50. Kết quả: "cấu trúc không gian hiện rõ ở cả hai mạng 2 lớp, dù rõ hơn ở resnet. Trong mạng feedforward, cấu trúc nhanh chóng biến mất theo độ sâu. Trong resnet, cấu trúc vẫn hiện rõ ở 50 lớp" (dòng 498-501).

Figure 5c: đo **relative effective rank** (hạng hiệu dụng tương đối), định nghĩa $r(\Delta) = \mathrm{tr}(\Delta^\top \Delta)/\|\Delta\|_2^2$ (đo "chiều nội tại" của ma trận Δ có các cột là gradient theo input, so với một ma trận "trắng" Y cùng kích thước có phần tử Gauss độc lập). "Relative effective rank (trung bình trên 30 minibatch) tăng nhanh hơn hẳn theo độ sâu cho mạng không có skip-connection. Với resnet, tham số β làm chậm tốc độ tăng của effective rank, đúng như dự đoán của Theorem 3" (dòng 567-572).

Figure 5d: norm ℓ_2 trung bình của gradient trên mỗi toạ độ (chuẩn hoá theo độ lệch chuẩn tính trên minibatch). "Đại lượng này giảm nhanh hơn hẳn theo độ sâu cho mạng feedforward. Điều này do hiệu ứng lấy trung bình gradient ngày càng trắng hoá trong mỗi minibatch. Nói cách khác, nhiễu trong minibatch lấn át tín hiệu. Hiện tượng này ít rõ rệt hơn hẳn trong resnet" (dòng 574-582).

4.3 LL-init: khởi tạo "trông tuyến tính" (§5)

Paper đề xuất một giải pháp **thay thế** cho skip-connection: khởi tạo rectifier sao cho nó "trông tuyến tính" (looks linear, LL-init) ngay từ đầu. Dùng CReLU (concatenated ReLU, Shang et al. 2016): $x \mapsto \begin{pmatrix} \rho(x) \\ \rho(-x) \end{pmatrix}$.
Quan sát chính: khởi tạo trọng số với cấu trúc khối đối xứng (mirrored block structure) $W, -W$ cho đầu ra: $W \cdot \rho(x) - W \cdot \rho(-x) = W \cdot x$ (tự kiểm chứng: $\rho(x) - \rho(-x) = \max(0, x) - \max(0, -x) = x$ với mọi x thực, vì nếu $x > 0$ thì $= x - 0 = x$, nếu $x < 0$ thì $= 0 - (-x) = x$, đúng với mọi x; tự tính). Nghĩa là **tại thời điểm khởi tạo**, toàn mạng hành xử đúng như một mạng tuyến tính (không có gradient vỡ, vì mạng tuyến tính không có vấn đề này - xem nhận xét ngay sau Eq. 1), nhưng vẫn giữ được tính phi tuyến *tiềm năng* một khi hai khối trọng số phân kỳ (diverge) trong quá trình huấn luyện.

Thí nghiệm CIFAR-10 (Figure 6, Adam optimizer, auto-scheduler giảm learning rate khi loss training chững lại 5 lần đo liên tiếp trong 10 epoch gần nhất, data augmentation dịch tối đa 4 pixel + lật ngang xác suất 0.5): so sánh 5 kiến trúc ở các độ sâu 6, 14, 30, 54, 102, 198 lớp - "Linear" (baseline tuyến tính thuần), "ReLU" (feedforward chuẩn), "CReLU w/o LL" (CReLU nhưng khởi tạo thường), "Resnet", "CReLU w/ LL" (đề xuất của paper). Kết quả: "mạng ReLU và CReLU hoạt động ngày càng kém theo độ sâu; mạng ReLU hoạt động kém hơn cả baseline tuyến tính (40%) ở độ sâu tối đa 198. Mạng feedforward với LL-init hoạt động **tương đương resnet**, cho thấy gradient vỡ là một phần lớn của vấn đề khi huấn luyện mạng rất sâu" (dòng 618-625). Đây là bằng chứng thực nghiệm mạnh nhất ủng hộ giả thuyết trung tâm của paper: nếu một kiến trúc **không có skip-connection nhưng không bị vỡ gradient** (nhờ LL-init) cũng huấn luyện tốt như resnet, thì "shattered gradients" đúng là nguyên nhân cốt lõi khiến mạng feedforward sâu khó huấn luyện, không phải bản thân việc thiếu skip-connection.

Paper lưu ý rõ mục tiêu thí nghiệm **không phải** đạt state-of-the-art ("The aim was not to match the state-of-the-art, but rather to test the hypothesis", dòng 641-642) - đây là thí nghiệm kiểm định giả thuyết (hypothesis test) có kiểm soát, không phải thí nghiệm benchmark.

5. Phân tích / ablation

Về bản chất, các Định lý 1-3 và Corollary 1 chính là "ablation lý thuyết": mỗi định lý xét một biến thể kiến trúc (feedforward thường, resnet không BN, resnet có BN, highway network) và tách bạch ảnh hưởng của từng thành phần:

- So Định lý 1 (feedforward, $R \sim 1/2^L$) với Định lý 2 (resnet không BN, $R \sim (3/4)^L$): thêm skip-connection (không BN) làm chậm tốc độ suy giảm tương quan (vì $3/4 > 1/2$) nhưng **vẫn là hàm mũ**, và gây ra vấn đề mới là phương sai nổ (2^L).
- So Định lý 2 với §3.3 (α-rescaling, R vẫn $(3/4)^L$): α-rescaling giải quyết phương sai nổ nhưng **không** đổi được bản chất hàm mũ của suy giảm tương quan - cho thấy vấn đề "tương quan suy giảm" và vấn đề "phương sai nổ" là hai hiện tượng tách biệt, cần cơ chế khác nhau để giải quyết.
- So §3.3 với Định lý 3 (BN thay cho α-rescaling): đổi hẳn dạng hàm từ mũ ($(3/4)^L$) sang dưới tuyến tính ($1/\sqrt{L}$) - đây là "thành phần" quan trọng nhất, cho thấy **chính batch normalization** (không phải chỉ bản thân phép cộng residual) là yếu tố làm thay đổi bản chất tốc độ suy giảm.
- Corollary 1 (highway network với γ phụ thuộc L) đẩy xa hơn nữa: tương quan có thể **không suy giảm theo độ sâu**, đổi lại mỗi lớp góp phần ngày càng nhỏ.

Phần thực nghiệm §4.3 (LL-init) là một "ablation" theo nghĩa khác: giữ cố định việc không có skip-connection,

chỉ thay đổi cách khởi tạo, để tách bạch xem ”gradient vỡ” (chứ không phải ”thiếu residual path” per se) có phải nguyên nhân chính khiến mạng sâu khó huấn luyện.

Paper tự ghi nhận một điểm chưa giải thích được (một dạng ”giới hạn của phân tích ablation”): ”các thí nghiệm sơ bộ (không trình bày) cho thấy LL-init hiệu quả hơn trên kiến trúc dựa CReLU so với PReLU. Lý do chưa rõ” (dòng 584-588, ”The reason is unclear”). Đây là một khoảng trống trong hiểu biết mà paper **tự thừa nhận công khai**, không cố gắng giải thích thêm.

6. Giới hạn

Các giới hạn **paper tự nêu rõ**:

1. Phân tích Định lý 1-3 và Corollary 1 chỉ chứng minh cho **mạng fully-connected** với số neuron bằng nhau ở mỗi lớp; ”mở rộng sang convnet cần thêm công việc sổ sách (bookkeeping) đáng kể” (dòng 311-312). Phần thực nghiệm §4 kiểm chứng định tính trên convnet thật (CIFAR-10) nhưng không có định lý toán học tổng quát cho convnet.
2. Giả định ”điểm dữ liệu điển hình” (Assumption 1) ”sẽ không đúng cho mọi cặp điểm dữ liệu” (dòng 336), chỉ được kiểm chứng **trung bình** đúng khi có batch normalization (Figure 3); không có BN, giả định này sai dần theo độ sâu (tỷ lệ cùng hoạt động tăng, không giữ ở 1/4).
3. Toàn bộ phân tích là **tại thời điểm khởi tạo (at initialization)**, chưa phải trong suốt quá trình huấn luyện: ”shattering nên giảm trong quá trình huấn luyện. Hiểu cách shattering ảnh hưởng huấn luyện là một vấn đề mở quan trọng” (dòng 34-35, trích lại từ Abstract) - paper tự nhận đây là câu hỏi chưa trả lời.
4. Công thức Theorem 3 (b, c) chỉ là xấp xỉ tỷ lệ bậc lớn ($\sim$), không phải đẳng thức chính xác (xem mục 3.6: ví dụ số với $\beta = 0.1$ không khớp hoàn toàn khi dùng công thức xấp xỉ, dù paper nói công thức chính xác đầy đủ có trong phụ lục A3.4 mà ghi chú này không đọc chi tiết).
5. Thí nghiệm LL-init (§5.1) dùng kiến trúc ”mỏng” (thin, số filter nhỏ, từ 8 đến 64) cố ý để ”đặc biệt khó cho gradient lan truyền ở độ sâu cao” (dòng 655-656) - đây là một lựa chọn thiết kế thí nghiệm có chủ đích làm nổi bật hiệu ứng, không phải kiến trúc thực tế dùng trong sản xuất; paper tự nói rõ mục tiêu ”không phải đạt state-of-the-art” (dòng 641-642).
6. Lý do LL-init hiệu quả hơn trên CReLU so với PReLU ”chưa rõ” (đã nêu ở mục 5), paper không giải thích được.
7. Paper không đề xuất một giải pháp duy nhất ”tốt nhất”; LL-init được trình bày như một hướng thay thế khả thi (alternate solution), không phải thay thế hoàn toàn skip-connection trong thực hành (paper gợi ý hướng tương lai là ”kết hợp LL-init với skip-connection”, dòng 661-662, chưa thử nghiệm).

7. Liên hệ với các bài khác

- **opt 37 (Delving Deep into Rectifiers / He initialization, 1502.01852)**: chưa có file, nhưng được trích dẫn trực tiếp và dùng làm nền tảng giả định $\sigma^2 = 2/N$ xuyên suốt Định lý 1-3 của bài này (He et al. 2015 trong reference list). Bài hiện tại chỉ ra rằng He init giải quyết đúng vấn đề nó nhắm tới (giữ phương sai activation qua các lớp) nhưng **không** giải quyết vấn đề ”vỡ tương quan gradient” mà paper này mới phát hiện.
- **opt 38 (Batch Normalization, 1502.03167)**: chưa có file, nhưng là đối tượng phân tích trung tâm của §2 và Định lý 3; phát hiện bất ngờ của bài này (”An alternate solution... an unexpected side-effect of batch normalization”, dòng 657-659) là BN không chỉ giảm covariate shift như mục tiêu gốc mà còn **thay đổi hẳn tốc độ suy giảm tương quan gradient** từ hàm mũ sang $1/\sqrt{L}$ khi kết hợp với resnet.
- **math 20 (Benefits of depth in neural networks, 1602.04485)**: đã có file tại ”math content/20_2016_benefits_of_depth.md” (xác nhận bằng ls). Phần Kết luận của bài hiện tại trích trực tiếp Montufar et al. (2014) và Telgarsky (2016): ”số vùng tuyến tính (linear region) có thể tăng theo hàm mũ với độ sâu... một hệ quả ít được nhận ra của sự tăng hàm mũ này là sự gia tăng các điểm

gián đoạn trong gradient của mạng rectifier" (dòng 629-637) - đây là **liên hệ nhân quả trực tiếp** mà paper tự nêu: chính hiện tượng "số vùng tuyến tính tăng theo hàm mũ" (chủ đề của math 20) là nguyên nhân sâu xa của "shattered gradients problem".

- **llm content/01_2017_attention_is_all_you_need.md** (đã xác nhận tồn tại bằng ls): Transformer (và do đó LightOnOCR) dùng residual connection quanh mỗi sub-layer (attention, FFN) theo đúng tinh thần resnet; phân tích của bài này về lý do residual connection giúp gradient "ít vỡ hơn" khi mạng rất sâu cung cấp nền tảng lý thuyết cho lựa chọn kiến trúc đó, dù Attention Is All You Need không tự phân tích theo góc độ tương quan gradient.
- **pruning content/31_2024_shortgpt.md, pruning content/33_2024_shortened_llama.md, pruning content/34_2024_sleb.md** (đã xác nhận tồn tại bằng ls): các bài này về cắt bớt lớp (layer pruning) của LLM đã huấn luyện xong, khác bối cảnh với bài hiện tại (bài hiện tại nói về gradient *lúc khởi tạo*, trước huấn luyện); liên hệ gián tiếp (suy luận): khi cắt một lớp residual giữa mạng đã huấn luyện, phần còn lại của mạng (đặc biệt các lớp sau điểm cắt) có thể gặp một dạng "sốc" tương tự về cấu trúc tương quan tín hiệu lan truyền, dù đây không phải cùng cơ chế gradient-tại-khởi-tạo mà paper này phân tích.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ nội dung mục này là suy luận, paper không nói trực tiếp về LightOnOCR hay về giai đoạn hậu huấn luyện/nén mô hình.

Dự án người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, ~3600 token/trang, GPU $\leq$ 8GB, batch size 1 khi suy luận). Bài này nói về **tại thời điểm khởi tạo** của một mạng rectifier feedforward thuần, trong khi LightOnOCR là mô hình **transformer/SSM hybrid đã huấn luyện xong** (không khởi tạo lại từ đầu trong bối cảnh nén/hậu huấn luyện thông thường), nên phần lớn các định lý định lượng (Định lý 1-3) **không áp dụng trực tiếp** - cần nói rõ điều này để không đánh giá quá cao mức liên quan.

Tuy vậy có hai liên hệ hợp lý (suy luận):

1. **Khi cắt bớt lớp hoặc fine-tune lại một phần mạng (liên quan tới các hướng pruning đã có thư mục riêng, không lặp lại chi tiết kỹ thuật pruning tại đây)**: nếu quá trình nén LightOnOCR bao gồm **khởi tạo lại** một số thành phần mới (ví dụ thêm một lớp adapter, hoặc khởi tạo lại trọng số của một khối bị thay thế sau khi cắt), bài học về "He init không đủ để tránh vỡ gradient, cần residual + rescaling phù hợp" vẫn có giá trị tham khảo: nên giữ các đường residual nguyên vẹn quanh thành phần mới khởi tạo, và cân nhắc dùng một hệ số β nhỏ (tương tự β-rescaling của paper) khi "chèn" thành phần mới vào một residual stream đã huấn luyện, để tránh làm nhiễu tín hiệu đã ổn định của các lớp còn lại.

2. **Kiến trúc hybrid GatedDeltaNet + attention**: GatedDeltaNet (một dạng linear attention/SSM có cổng) không có cấu trúc rectifier-feedforward thuần như mô hình toán học của paper này (không có ReLU phi tuyến xen giữa theo kiểu mạng được phân tích), nên **không thể áp trực tiếp** các công thức $1/2^L$ hay $1/\sqrt{L}$ lên "residual stream" của riêng GatedDeltaNet; tuy nhiên về **nguyên lý chung** ("tương quan tín hiệu/gradient qua nhiều lớp suy giảm nhanh hơn trong kiến trúc không có đường tắt so với có đường tắt"), nguyên lý này có khả năng vẫn đúng về định tính cho residual stream tổng thể của mô hình hybrid (attention/GatedDeltaNet xen lớp FFN có phi tuyến), vì bản thân FFN trong transformer vẫn dùng hàm phi tuyến dạng rectifier-like (GELU/SiLU, gần giống ReLU về mặt "có điểm bẻ gãy mượt"). Đây là suy luận có mức độ chắc chắn thấp, chỉ nên dùng để **cảnh giác** khi thiết kế thử nghiệm nén (ví dụ theo dõi tương quan gradient giữa các mẫu dữ liệu nếu huấn luyện gặp bất ổn khó hiểu), không nên dùng để quyết định kiến trúc cụ thể.

Kết luận mục này (suy luận): bài này **ít liên quan trực tiếp** tới công việc nén/hậu huấn luyện LightOnOCR theo nghĩa áp dụng công thức cụ thể, vì bối cảnh (mạng rectifier feedforward thuần, tại khởi tạo) khác hẳn bối cảnh dự án (transformer/SSM hybrid, đã huấn luyện, đang fine-tune/nén). Giá trị chính của bài là **cung**

cấp trực giác về lý do residual connection quan trọng cho việc lan truyền tín hiệu ổn định qua nhiều lớp, một nguyên lý nền tảng đáng nhớ khi thiết kế bất kỳ thay đổi kiến trúc nào (thêm/bớt lớp, khởi tạo lại thành phần) trong quá trình nén.

9. Thí nghiệm gợi ý

1. **Tái tạo Figure 1-2 ở quy mô nhỏ**: cài mô hình tối giản của paper ($f_W : \mathbb{R} \to \mathbb{R}$, $N = 200$ neuron mỗi lớp ẩn, vài lớp) bằng PyTorch, chạy được trên CPU hoặc GPU 8GB trong vài giây; tính đạo hàm df_W/dx trên lưới 256 điểm trong $[-2, 2]$ bằng autograd, với độ sâu 1, 10, 24 lớp. Đo ma trận hiệp phương sai của vector gradient 256 chiều này, vẽ heatmap. Kết quả kỳ vọng: mạng 1 lớp cho ma trận có cấu trúc dải (band structure, giống brown noise), mạng 24 lớp cho ma trận gần với ma trận đường chéo nhiễu (giống white noise). Diễn giải: xác nhận trực tiếp hiện tượng ”vỡ gradient” trên máy của người dùng, không cần phần cứng mạnh.

2. **Kiểm chứng Định lý 1 bằng Monte Carlo**: với mô hình tối giản trên, lấy L cố định (ví dụ $L = 5, 10, 15$), khởi tạo lại mạng (resample trọng số) nhiều lần (ví dụ 200 lần), mỗi lần tính ∇_i tại hai điểm dữ liệu cố định $x^{(i)}, x^{(j)}$ cách nhau một khoảng cố định, ước lượng hiệp phương sai thực nghiệm $\widehat{C}(i, j) = \frac{1}{200} \sum \nabla_i \nabla_j$. So với dự đoán lý thuyết $1/2^L$. Kết quả kỳ vọng: giá trị ước lượng giảm theo đúng tốc độ hàm mũ 2^{-L} khi vẽ trên thang log. Diễn giải: kiểm chứng định lượng trực tiếp công thức toán học của paper bằng mô phỏng, giúp hiểu rõ bản chất ”suy giảm theo hàm mũ” không chỉ là lời văn mà là một con số đo được.

3. **So sánh feedforward vs resnet toy trên tác vụ nhỏ**: huấn luyện hai mạng nhỏ (feedforward thuần và resnet, ví dụ 20-30 lớp, vài trăm tham số mỗi lớp) trên MNIST (chạy nhanh trên GPU 8GB), đo loss training qua epoch. Đồng thời đo relative effective rank của gradient (theo công thức $r(\Delta) = \text{tr}(\Delta^\top \Delta)/\|\Delta\|_2^2$ ở mục 4.2) tại vài checkpoint trong quá trình huấn luyện (không chỉ lúc khởi tạo). Kết quả kỳ vọng: feedforward hội tụ chậm hơn hoặc kẹt ở loss cao hơn resnet ở độ sâu lớn; effective rank của feedforward giảm (trắng hoá) rõ hơn resnet, đặc biệt ở các epoch đầu. Diễn giải: đây trực tiếp trả lời câu hỏi mở mà paper tự nêu (”shattering thay đổi thế nào trong quá trình huấn luyện”, dòng 34-35) ở quy mô nhỏ do chính người dùng tự làm.

4. **Thử LL-init trên một mạng nhỏ**: cài CReLU với khởi tạo khối đối xứng $W, -W$ theo công thức mục 4.3, huấn luyện một mạng feedforward 30-50 lớp (không skip-connection) trên CIFAR-10 hoặc MNIST, so với mạng ReLU thường cùng độ sâu và một resnet cùng độ sâu. Kết quả kỳ vọng (lặp lại định tính Figure 6 ở quy mô nhỏ hơn): CReLU+LL-init huấn luyện được tốt gần bằng resnet, còn ReLU thường kém hẳn ở độ sâu lớn. Diễn giải: xác nhận hoặc phản biện kết luận cốt lõi ”shattered gradients là nguyên nhân chính, không nhất thiết cần skip-connection nếu giải quyết được gradient vỡ bằng cách khác” trên tác vụ/quy mô mà người dùng tự kiểm soát.

5. **Đo tỷ lệ coactivation trong một lớp Transformer thật (liên hệ gián tiếp tới LightOnOCR)**: lấy một checkpoint nhỏ có kiến trúc giống LightOnOCR (hoặc một mô hình mã nguồn mở nhỏ tương tự nếu LightOnOCR chưa sẵn để thử nghiệm nhanh), đưa một batch input qua, đo tỷ lệ ”đồng kích hoạt” của activation phi tuyến (GELU/SiLU, coi dương là ”active” tương tự ReLU) giữa các cặp token khác nhau ở các lớp khác nhau, theo đúng cách đo ở Figure 3 (dòng 322-328) của paper. Kết quả kỳ vọng: không có kỳ vọng định lượng cụ thể từ paper (paper chỉ đo trên rectifier feedforward thuần, không đo transformer), nhưng nếu tỷ lệ đồng kích hoạt tăng dần theo lớp (giống hiện tượng ”redundant” mà paper mô tả khi không có BN), có thể là dấu hiệu cảnh báo tương tự. Diễn giải: thí nghiệm này mang tính khám phá (exploratory), không kỳ vọng chắc chắn, chỉ nhằm kiểm tra xem trực giác của paper (dù xây dựng cho rectifier feedforward) có ”dư âm” nào quan sát được trên kiến trúc hybrid hiện đại hay không; kết quả âm tính (không thấy hiện tượng) cũng là thông tin hữu ích, cho thấy sự khác biệt kiến trúc (có LayerNorm/RMSNorm, có residual dày đặc hơn mạng thí nghiệm của paper) đã đủ để tránh vấn đề này.

10. Tóm tắt công thức / khái niệm

- Định nghĩa hiệp phương sai/tương quan gradient: $C(i,j) = \mathbb{E}[\nabla_i \nabla_j]$, $R(i,j) = \dfrac{\mathbb{E}[\nabla_i \nabla_j]}{\sqrt{\mathbb{E}[\nabla_i^2]\mathbb{E}[\nabla_j^2]}}$.
- Định lý 1 (feedforward, He init $\sigma^2 = 2/N$): $C^{fnn}(i) = 1$; $C^{fnn}(i,j) = 1/2^L$; suy ra $R^{fnn}(i,j) = 1/2^L$ (suy giảm hàm mũ).
- Định lý 2 (resnet không BN, $\alpha = \beta = 1$): $C^{res}(i) = 2^L$ (phương sai nổ); $C^{res}(i,j) = (3/2)^L$; $R^{res}(i,j) = (3/4)^L$ (tự tính từ tỉ số hai công thức trên, khớp paper).
- Rescale $\alpha = 1/\sqrt{2}$: ổn định phương sai ($C = 1$) nhưng tương quan vẫn $(3/4)^L$ (không giải quyết vỡ gradient).
- Định lý 3 (resnet có BN và β-rescaling): $C^{res}_{\beta,BN}(i) = \beta^2(L-1)+1$; $C^{res}_{\beta,BN}(i,j) \sim \beta\sqrt{L}$; $R^{res}_{\beta,BN}(i,j) \sim 1/(\beta\sqrt{L})$ (suy giảm dưới tuyến tính, chậm hơn hẳn hàm mũ).
- Corollary 1 (highway network, cổng vô hướng γ_1, γ_2, $\gamma_1^2 + \gamma_2^2 = 1$): $C^{HN}_{\gamma}(i) = 1$; $R^{HN}_{\gamma}(i,j) = (\gamma_1^2 + \frac{1}{2}\gamma_2^2)^L$; với $\gamma_1 = \sqrt{1-1/L}$, $\gamma_2 = \sqrt{1/L}$: $\lim_{L\to\infty} R^{HN}_{\gamma}(i,j) = 1/\sqrt{e} \approx 0.6065$ (tự tính bằng giới hạn $(1-\frac{1}{2L})^L \to e^{-1/2}$).
- Ví dụ số tự kiểm tra (mục 3.6): công thức $R \sim 1/(\beta\sqrt{L})$ với $\beta = 1.0, L = 100$ cho 0.1 (khớp đúng số paper nêu); với $\beta = 0.1, L = 100$ cho 1.0 (paper nêu số thực tế là 0.7, chênh lệch do "$\sim$" chỉ là xấp xỉ bậc lớn, không phải đẳng thức chính xác) (tự tính).
- LL-init (CReLU, khởi tạo khối đối xứng $W, -W$): $W\rho(x) - W\rho(-x) = Wx$ với mọi x (tự kiểm chứng đại số), khiến mạng hành xử tuyến tính tại khởi tạo dù dùng rectifier.
- Relative effective rank: $r(\Delta) = \mathrm{tr}(\Delta^\top \Delta)/\|\Delta\|_2^2$, đo "chiều nội tại" của ma trận gradient, dùng để định lượng mức độ "trắng hoá".

11. Kết luận ngắn

Paper xác định "vấn đề gradient vỡ" (shattered gradients problem), trực giao với vanishing/exploding gradient đã biết: tương quan giữa gradient tại các điểm dữ liệu khác nhau suy giảm theo hàm mũ với độ sâu trong mạng feedforward rectifier chuẩn (Định lý 1), khiến gradient giống nhiễu trắng; skip-connection kết hợp batch normalization đổi tốc độ suy giảm này thành dưới tuyến tính $1/\sqrt{L}$ (Định lý 3), giải thích tại sao resnet huấn luyện được ở độ sâu lớn hơn hẳn mạng feedforward dù cả hai đều dùng khởi tạo và BN tốt. Thực nghiệm trên CIFAR-10 (cả fully-connected và convnet) xác nhận các dự đoán lý thuyết, và một khởi tạo thay thế (LL-init, không cần skip-connection) đạt hiệu quả gần bằng resnet, ủng hộ mạnh giả thuyết rằng chính "gradient vỡ", không phải sự thiếu vắng đường tắt per se, là nguyên nhân cốt lõi khiến mạng feedforward rất sâu khó huấn luyện.

Nguồn gốc: https://arxiv.org/abs/1702.08591

Bài 19

Measuring the Intrinsic Dimension of Objective Landscapes (2018)

Tác giả: Chunyuan Li (Duke University), Heerad Farkhoor, Rosanne Liu, Jason Yosinski (Uber AI Labs).
Venue / arXiv: arXiv:1804.08838 (v1, 24/04/2018); venue ghi trong paper: ”Published as a conference paper at ICLR 2018”.
Nguồn: https://arxiv.org/abs/1804.08838
Chủ đề: đo ”intrinsic dimension” (số chiều nội tại) của objective landscape bằng phương pháp random subspace training; dùng nó để so sánh độ khó bài toán giữa các domain (supervised, RL) và để nén mạng.
Vị trí trong lộ trình: đọc sau math 18 (Shattered Gradients, cùng khảo sát ”objective landscape” nhưng theo hướng gradient thay đổi hỗn loạn theo độ sâu) và trước math 22-25 (NNGP, NTK, Principles of Deep Learning Theory, cùng đặt câu hỏi ”cần bao nhiêu chiều/tham số hiệu dụng để mạng hoạt động tốt”), và math 24 (Lazy Training, cũng dùng khái niệm ”số chiều hiệu dụng” của mô hình quanh điểm khởi tạo). Trong thư mục pruning, liên hệ trực tiếp ”pruning 08” (Lottery Ticket Hypothesis) và ”pruning 12” (state of pruning).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Một mạng nơ-ron có D tham số (trọng số) có thể được hình dung như đang đi tìm điểm thấp nhất trên một ”mặt địa hình” (objective landscape, hay loss landscape) trong không gian D chiều: mỗi điểm $\theta \in \mathbb{R}^D$ cho một giá trị loss (độ cao của địa hình). Khi $D = 2$ ta có thể vẽ và nhìn trực quan mặt địa hình này như đồi núi, nhưng khi D lên tới hàng trăm nghìn hay hàng triệu (như các mạng hiện đại), trực giác hình học thấp chiều không còn đáng tin.

Trước bài này, một số công trình đã chỉ ra các tính chất ”lạ” của landscape nhiều chiều: Dauphin et al. (2014) cho thấy các điểm tới hạn gần như luôn là saddle point (yên ngựa) chứ không phải valley (hõm cục bộ) như trực giác 2D hay gợi ý; Goodfellow et al. (2015) cho thấy đường đi thẳng từ điểm khởi tạo tới điểm tối ưu thường giảm loss một cách đơn điệu. Những quan sát này gợi ý rằng landscape nhiều chiều ”đơn giản” hơn ta nghĩ theo một số nghĩa.

Câu hỏi trung tâm của paper: một mạng có D tham số, nhưng thực chất cần bao nhiêu ”độ tự do” (degrees of freedom) để giải được một bài toán cụ thể? Số tham số D là thước đo thô (rough gauge) cho độ khó bài toán, nhưng paper muốn có một thước đo định lượng tốt hơn, áp dụng được cho cả supervised learning và reinforcement learning (RL), để so sánh độ khó giữa các bài toán khác nhau một cách công bằng.

Khái niệm cần nhớ: **random subspace** là một không gian con d chiều ($d < D$) được chọn ngẫu nhiên và cố định (không học) bên trong không gian tham số đầy đủ $\mathbb{R}^D$; **codimension** của một tập con $S \subset \mathbb{R}^D$ là D trừ số chiều của S (số chiều ”vuông góc” với S).

2. Ý tưởng chính

Paper định nghĩa **intrinsic dimension** d_{int} của một ”objective landscape” (gắn với một bài toán + mô hình cụ thể) là **codimension của tập nghiệm** (solution set) bên trong không gian tham số đầy đủ $\mathbb{R}^D$. Nếu gọi s là số chiều của tập nghiệm (bao nhiêu hướng di chuyển mà vẫn giữ nghiệm tốt), thì:

$$D = d_{int} + s \tag{1}$$

Ý tưởng đo d_{int} trong thực tế: thay vì huấn luyện trực tiếp trong $\mathbb{R}^D$ (gọi là **direct method**), ta huấn luyện trong một không gian con ngẫu nhiên d chiều của $\mathbb{R}^D$ (gọi là **subspace method**), tăng dần d, và ghi nhận giá trị d nhỏ nhất mà tại đó mô hình đạt hiệu năng đủ tốt (ngưỡng 90% so với baseline huấn luyện trực tiếp). Giá trị d đó được định nghĩa là d_{int90} (đo được, ước lượng của d_{int} lý thuyết).

Hai hệ quả bất ngờ mà paper nhấn mạnh:

- Nhiều bài toán có d_{int} **thấp hơn nhiều** so với D (ví dụ mạng fully-connected (FC) cho MNIST có $D = 199.210$ nhưng chỉ cần khoảng 750 chiều để đạt 90% hiệu năng), cho thấy landscape ”đơn giản” hơn số tham số gợi ý.
- Khi tăng D (thêm layer rộng hơn, sâu hơn) cho **cùng một bài toán**, d_{int} gần như **không đổi**: paper diễn giải điều này là ”mỗi tham số thêm vào chỉ làm tăng số chiều dư thừa s của tập nghiệm, không làm bài toán trở nên khó hơn hay dễ hơn về bản chất”. Đây là cách nhìn định lượng cho hiện tượng ”big models waste capacity” đã được quan sát từ trước (Dauphin & Bengio, 2013).

Phương pháp đo d_{int} cũng dẫn tới một hệ quả phụ: vì chỉ cần lưu d số thực (cộng 2 random seed) để tái tạo mạng, đây là một cách **nén mạng** đơn giản, và d_{int} cho một **chặn trên (upper bound) của Minimum Description Length (MDL)** của nghiệm.

3. Phương pháp chi tiết

3.1 Huấn luyện trong không gian con ngẫu nhiên

Gọi $\theta^{(D)} \in \mathbb{R}^D$ là vector tham số đầy đủ, $\theta_0^{(D)}$ là điểm khởi tạo ngẫu nhiên (ví dụ He init hay Glorot init), P là ma trận chiếu ngẫu nhiên $D \times d$ (cột xấp xỉ trực chuẩn, chuẩn hoá độ dài 1), và $\theta^{(d)} \in \mathbb{R}^d$ là tham số thực sự được học, khởi tạo bằng 0. Tham số đầy đủ dùng để forward/backward là:

$$\theta^{(D)} = \theta_0^{(D)} + P\,\theta^{(d)} \tag{2}$$

P và $\theta_0^{(D)}$ được sinh ngẫu nhiên và **đóng băng** (không cập nhật); chỉ $\theta^{(d)}$ được huấn luyện bằng gradient descent. Vì $\theta^{(d)}$ khởi tạo bằng 0, mạng bắt đầu đúng tại điểm khởi tạo ”chuẩn” $\theta_0^{(D)}$ (vẫn hưởng lợi từ các kỹ thuật init tốt).

Hai trường hợp biên: nếu $d = D$ và P là ma trận đơn vị, ta có đúng bài toán huấn luyện trực tiếp; nếu $d = D$ và P là ma trận xoay ngẫu nhiên, ta có bài toán xoay (với optimizer ”rotation-invariant” như SGD thì kết quả như cũ, nhưng với Adam/RMSProp thì có thể khác vì các optimizer này giả định các trục toạ độ có ý nghĩa riêng).

Ví dụ toy (chính paper dùng để minh hoạ, không phải tự tính): $D = 1000$, bài toán ép tổng 100 phần tử đầu bằng 1, 100 phần tử tiếp bằng 2, ..., tổng cộng 10 nhóm với 10 ràng buộc tổng. Tập nghiệm là siêu phẳng (hyperplane) 990 chiều (990 hướng di chuyển vẫn giữ cost bằng 0), nên theo công thức (1): $d_{int} = D - s = 1000 - 990 = 10$, khớp với số ràng buộc đặt ra. Khi quét d từ 0 lên, performance đạt 90% và 100% cùng tại $d = 10$.

Tự tính (minh hoạ nhỏ hơn, số khác với paper): xét một bài toán toy $D = 10$ với 2 ràng buộc tuyến tính độc lập (ví dụ 5 phần tử đầu phải tổng 1, 5 phần tử sau phải tổng 2). Theo công thức (1): $s = D - d_{int} = 10 - 2 = 8$ chiều dư thừa trong tập nghiệm, $d_{int} = 2$ (đúng bằng số ràng buộc). Đây chỉ là áp dụng công thức (1) cho một bộ số nhỏ hơn để kiểm tra trực giác, không phải số trong paper.

3.2 Định nghĩa d_{int90} và d_{int100}

Vì loss hiếm khi về đúng 0 với dữ liệu thật, paper dùng ngưỡng hiệu năng tương đối so với baseline (mô hình huấn luyện trực tiếp tốt nhất): d_{int100} là chiều tại đó hiệu năng ”không thể phân biệt thống kê” với baseline; d_{int90} là chiều tại đó hiệu năng đạt **tối thiểu 90%** của baseline. Paper nhận thấy d_{int100} dao động mạnh (có thể

gần D nếu baseline được tune rất kỹ, hoặc giảm mạnh nếu subspace training vô tình có tác dụng regularize), nên chọn d_{int90} làm thước đo chính, vì nó ổn định hơn với nhiễu huấn luyện. Ngưỡng 90% được chọn bằng quan sát thực nghiệm trên nhiều đồ thị performance-vs-d (paper lưu ý: ngưỡng khác đi thì kết luận định tính không đổi).

Thước đo hiệu năng: validation accuracy cho supervised classification; tổng reward đã dịch về không âm cho RL.

3.3 Ba phương pháp chiếu ngẫu nhiên (để mở rộng quy mô)

Khi D lớn (hàng triệu), việc lưu và nhân ma trận P dày đặc $D \times d$ tốn $O(Dd)$ thời gian và không gian, không khả thi. Paper dùng 3 cách:

1. **Dense**: P là ma trận chuẩn Gauss dày đặc, $O(Dd)$ thời gian/không gian.
2. **Sparse** (Li et al., 2006): mỗi phần tử khác 0 với xác suất $1/\sqrt{D}$, cho $O(\sqrt{Dd})$ thời gian/không gian (xấp xỉ, thực nghiệm chỉ nhanh hơn dense khoảng hằng số 10x do giới hạn cài đặt TensorFlow SparseTensor, không đạt đúng hệ số lý thuyết $\sqrt{D}$).
3. **Fastfood transform** (Le et al., 2013): phân tích $M = HG\Pi HB$ (Hadamard, Gauss chéo, hoán vị, dấu ngẫu nhiên), cho $O(D \log d)$ thời gian và $O(D)$ không gian, không cần lưu ma trận P hiển thị. Đây là phương pháp cho phép scale tới ImageNet/SqueezeNet ($D \approx 1,25$ triệu) và Atari Pong ($D \approx 1$ triệu).

Bảng S4 (đo thời gian forward+backward qua M, $d = 1\%$ của D): Dense tại $D = 10^5$ là 0,0169s, tại $D = 10^6$ là 1,0742s (ước lượng, hết nhớ ở $D = 6 \times 10^7$); Sparse tại $D = 10^5$ là 0,0002s, tại $D = 10^6$ là 0,0019s; Fastfood tại $D = 10^5$ là 0,0181s, tại $D = 10^6$ là 0,0195s, và vẫn chạy được ở $D = 6 \times 10^7$ (0,7949s), trong khi Dense/Sparse hết bộ nhớ ở quy mô đó.

4. Thí nghiệm và kết quả chính

Đây là paper thực nghiệm (không phải lý thuyết thuần); mục này tóm tắt các bảng/số liệu chính, lấy trực tiếp từ Bảng 1, Bảng S5 và các hình trong text.

MNIST (FC và LeNet). Mạng FC 784-200-200-10 ($D = 199.210$) đạt $d_{int90} \approx 750$ (chỉ dùng 0,4% số tham số để đạt 90% hiệu năng). LeNet ($D = 44.426$) đạt $d_{int90} = 290$ (hệ số nén khoảng 150 lần). Quét 20 mạng FC với độ sâu $\{1, ..., 5\}$ lớp ẩn và độ rộng $\{50, 100, 200, 400\}$: D thay đổi theo hệ số 24,1 lần nhưng d_{int90} chỉ thay đổi theo hệ số 1,33 lần (dùng baseline toàn cục 100%) hoặc 1,42 lần (dùng baseline riêng từng mô hình) **(Hình 3, Hình S7)**, minh chứng chính cho luận điểm "d_{int} ổn định khi D tăng".

MNIST với dữ liệu xáo trộn. Xáo trộn pixel (shuffled pixels): d_{int90} của FC không đổi (750, vì FC bất biến với hoán vị input), nhưng của LeNet tăng từ 290 lên 1400 (convnet mất lợi thế cấu trúc cục bộ). Xáo trộn label (shuffled labels, ghi nhớ nhãn ngẫu nhiên hoàn toàn trên toàn bộ 50.000 ảnh train, đo bằng training accuracy): $d_{int90} = 190.000$, tương đương 3,8 số thực/nhãn được ghi nhớ. Khi giảm cỡ tập ghi nhớ xuống 50% và 10%, d_{int90} giảm xuống 130.000 và 90.000, tức là **chi phí ghi nhớ mỗi nhãn tăng lên (5,2 và 18,0 số thực/nhãn)** khi tập dữ liệu nhỏ hơn, gợi ý có "chia sẻ hạ tầng" (shared infrastructure) giúp ghi nhớ hiệu quả hơn khi dữ liệu lớn.

CIFAR-10 và ImageNet. Theo Bảng 1 (văn bản chính): FC $D = 656.810 \rightarrow d_{int90} = 9000$; LeNet $D = 62.006 \rightarrow d_{int90} = 2900$; để đạt $> 50\%$ accuracy toàn cục, FC/LeNet/ResNet cần d_{int90} xấp xỉ 9k/2,9k/1k tương ứng, cho thấy **ResNet hiệu quả tham số hơn** FC và LeNet trên CIFAR-10. ImageNet (SqueezeNet, $D = 1.248.424$): do hạn chế thời gian/bộ nhớ, chỉ xác nhận được $d_{int90} > 500.000$ (chưa đo được giá trị chính xác; tại $d = 500k$ accuracy mới đạt 38,95%, dưới ngưỡng 90% của baseline 55,5%).

Reinforcement learning (ES và DQN). Dùng Evolution Strategies (ES): InvertedPendulum $d_{int100} = d_{int90} = 4$ (chỉ 4 tham số đủ giải bài toán!); Humanoid (đi bộ) $d_{int90} = 700$ (tương đương độ khó MNIST-FC,

thấp hơn nhiều so với trực giác ”đi bộ khó hơn phân loại ảnh”); Atari Pong (từ pixel) $d_{int90} = 6000$ (cùng bậc với CIFAR-10). Dùng DQN cho CartPole/Pole/Cart: $d_{int90} = 25/23/7$.

Bảng tổng hợp (Bảng 1, văn bản chính, trích các giá trị chính):

Bài toán	Mạng	D	d_{int90}
MNIST	FC	199.210	750
MNIST	LeNet	44.426	290
MNIST (shuffled pixel)	LeNet	44.426	1.400
MNIST (shuffled label, 100%)	FC	959.610	190.000
CIFAR-10	FC	656.810	9.000
CIFAR-10	LeNet	62.006	2.900
ImageNet	SqueezeNet	1.248.424	> 500.000
Inverted Pendulum	FC	562	4
Humanoid	FC	166.673	700
Atari Pong	ConvNet	1.005.974	6.000

Ablation kiến trúc convnet (Mục S10): so 4 biến thể LeNet (Standard, Untied weights, FC-tied kernel toàn ảnh, FC hoàn toàn) trên MNIST: $d_{int90} = 290, 600, 425, 2000$; trên CIFAR-10: $d_{int90} = 1000, 2750, 2500, 35000$. Kết luận: **cả weight-tying và local receptive field đều quan trọng**, loại bỏ một trong hai làm tăng d_{int90} đáng kể (tăng độ khó bài toán theo thước đo này).

5. Phân tích / ablation

Vai trò regularizer (Mục S8, Hình S15-S17). So sánh subspace training với weight decay (ℓ_2 penalty) và dropout trên FC ($L = 2, W = 200$) ở CIFAR-10: cả hai regularizer truyền thống đều giảm khoảng cách train-test accuracy; subspace training tự nó cũng có tác dụng regularize mạnh (đặc biệt khi d nhỏ). Khi $\ell_2 < 5 \times 10^{-4}$ hoặc dropout $< 0,1$ (tức ít regularize truyền thống), subspace training vượt direct training về test NLL, cho thấy nó có thể là một **regularizer thay thế** hiệu quả trong vùng ít regularize khác.

Ổn định huấn luyện mạng sâu (Mục S5.3, Hình S9). Với FC lên tới 10 lớp, SGD step 0,1, ReLU + He init: **direct training luôn thất bại khi độ sâu** $L > 5$ (nhiều lần thất bại ở $L = 4$, luôn thất bại ở $L > 4$), trong khi **subspace training ổn định ở mọi độ sâu được thử**. Paper giải thích: mỗi cơ sở ngẫu nhiên của P ”nhìn” qua toàn bộ D tham số trực tiếp, nên bài toán tối ưu trong không gian con có thể có điều kiện số (conditioning) tốt hơn hẳn so với không gian gốc (nơi các tham số có scale rất khác nhau theo layer).

Vai trò optimizer (Mục S5.4, Hình S10). So SGD (lr 0,1) và Adam (lr 0,001): Adam cho d_{int90} cao hơn SGD nói chung, vì Adam đạt baseline cao hơn (đặc biệt khi dùng baseline riêng từng mô hình).

Hai cách nhìn khác về d_{int} (Mục S5.1, S5.2, S9): d_{int} đo với baseline toàn cục khác với baseline riêng từng mô hình (số liệu lệch nhẹ: hệ số thay đổi 1,33 so với 1,42); với ImageNet, do hạn chế thời gian (mỗi lần đo tốn 6-7 ngày trên 4 GPU), kết luận chỉ là ”chặn dưới $> 500k$”, không có giá trị hội tụ.

6. Giới hạn

Paper tự nêu một số giới hạn: (a) phương pháp subspace training **không giúp tăng tốc inference** (khác với các phương pháp nén khác có kèm lợi ích tính toán khi dự đoán); (b) d_{int} chỉ là **chặn trên** của MDL thực sự, độ ”chặt” của chặn này chưa rõ, có thể khác nhau giữa các bài toán; (c) kết luận ”ResNet tốt hơn FC cho MNIST” (dựa trên d_{int} thấp hơn) là dễ đoán, nhưng với mô hình lớn/phức tạp/không đồng nhất hơn, kết luận từ d_{int} sẽ không hiển nhiên như vậy; (d) không nên khẳng định mọi FC-net trên MNIST đều có $d_{int} \approx 750$,

chỉ nên hiểu đó là một ”cao nguyên” (plateau) giá trị gần đúng cho một họ kiến trúc/siêu tham số cụ thể đã thử.

Hai điểm paper tự mâu thuẫn giữa văn bản chính và phụ lục (người ghi chú phát hiện khi đọc toàn văn, không phải paper tự nêu):

- Bảng 1 (văn bản chính) ghi CIFAR-10/FC có $D = 656.810$; Bảng S5 (phụ lục) lại ghi CIFAR-10/FC có $D = 1.055.610$ cho cùng cấu hình ”FC” ($d_{int90} = 9000$ ở cả hai bảng). Hai số D này khác nhau đáng kể (chênh khoảng 61%) mà không có giải thích trong text; có thể do khác kiến trúc FC cụ thể (số layer/width) giữa khi lập hai bảng, nhưng paper không nói rõ.
- Mục 3.2 (văn bản chính, Hình 2 caption) ghi MNIST/LeNet có $d_{int90} = 290$; Bảng S5 (phụ lục) lại ghi MNIST/LeNet có $d_{int90} = 275$. Chênh lệch nhỏ (15 đơn vị) nhưng là số khác nhau cho cùng một cấu hình được nêu ở hai nơi.

Hai điểm trên không ảnh hưởng tới kết luận định tính của paper (landscape thường có $d_{int} \ll D$, và d_{int} ổn định khi D tăng), nhưng người đọc kỹ số liệu nên biết có sự không khớp này giữa bảng chính và bảng phụ lục.

7. Liên hệ với các bài khác

- **math 23 (Neural Tangent Kernel)** và **math 22 (Deep NN as Gaussian Processes)**: cùng đặt câu hỏi ”mạng có bao nhiêu độ tự do hiệu dụng”, nhưng theo hướng lý thuyết infinite-width thay vì đo thực nghiệm bằng random subspace.
- **math 24 (On Lazy Training)**: dùng khái niệm gần giống ”số chiều hiệu dụng quanh điểm khởi tạo” (qua rank của $Dh(w_0)$ và độ lớn κ_h) để xác định khi nào mạng rơi vào chế độ tuyến tính hoá; bổ sung góc nhìn ”tĩnh” (landscape) của bài 19 bằng góc nhìn ”động” (quỹ đạo huấn luyện).
- **math 25 (Principles of Deep Learning Theory)**: khái niệm ”principle of sparsity” và tỉ lệ $r = L/n$ trong sách này cũng nói về việc mạng có nhiều tham số ”dư” theo nghĩa thống kê, cùng tinh thần với nhận xét ”extra parameters chỉ tăng s, không tăng d_{int}” của bài 19, dù công cụ toán khác hẳn (effective theory/RG flow so với đo thực nghiệm).
- **pruning 08 (Lottery Ticket Hypothesis, đã kiểm tồn tại bằng `ls "pruning content"`)**: cùng tinh thần ”mạng dùng ít tham số hơn số D gốc vẫn giải được bài toán”, nhưng lottery ticket tìm **sub-network thưa** (một tập con trục toạ độ gốc), còn bài 19 tìm **subspace tuyến tính ngẫu nhiên** (tổ hợp tuyến tính của mọi tham số); đây là hai cách khác nhau để lượng hoá ”độ dư thừa tham số” của cùng một hiện tượng.
- **pruning 12 (state of neural network pruning, đã kiểm tồn tại)**: khảo sát các phương pháp đo độ dư thừa tham số/nén mạng nói chung, là bối cảnh rộng hơn cho kỹ thuật nén-bằng-subspace của bài 19 (paper 19 tự nhận chỉ là một ”cách nén đơn giản”, không cạnh tranh về tỉ lệ nén với các phương pháp group-sparsity như Louizos et al. 2017).

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1.6-2.1K token visual, GPU ≤ 8GB, batch 1).

- **Ý tưởng khả dụng nhất (suy luận):** khái niệm d_{int90} gợi ý một cách định lượng để **chọn rank cho LoRA/adapter** khi fine-tune lại mô hình sau khi đã prune/distill. Nếu đo được rằng với một tác vụ cụ thể (ví dụ fine-tune lại OCR trên một domain văn bản hẹp), hiệu năng đạt 90% baseline chỉ cần subspace vài trăm chiều, đó là gợi ý mạnh để chọn rank LoRA tương ứng (thay vì chọn rank tuỳ ý). Đây là suy luận ngoại suy: paper không thử nghiệm trên VLM/Transformer hiện đại, chỉ trên FC/LeNet/ResNet nhỏ và RL toy task.

- **Hạn chế khi áp dụng trực tiếp:** phương pháp random subspace training trong paper tốn kém hơn direct training (phải backprop qua cả phép chiếu P, dù Fastfood giảm chi phí), và không tăng tốc inference, nên **không phù hợp làm phương pháp nén chính** cho một mô hình 0.8B cần chạy suy luận nhanh trên GPU 8GB; giá trị chủ yếu nằm ở **chẩn đoán** (đo xem một layer/module có đang "dùng dư" tham số hay không) hơn là triển khai thực tế.
- **Liên hệ với kiến trúc hybrid:** nhận xét "cấu trúc cục bộ (local receptive field) và chia sẻ trọng số (weight tying) đều quan trọng, thiếu một trong hai làm tăng d_{int90}" gợi ý rằng với kiến trúc lai Gated-DeltaNet (có cấu trúc gated/recurrent đặc thù) + attention, việc prune "phá vỡ" cấu trúc chia sẻ tham số theo chiều thời gian của GatedDeltaNet có thể làm tăng mạnh độ khó tái huấn luyện, tương tự như untie-weight làm tăng d_{int90} trong thí nghiệm LeNet. Đây hoàn toàn là suy luận tương tự hoá, paper không nói về SSM/gated linear attention.

9. Thí nghiệm gợi ý

1. **Đo d_{int90} cho một MLP nhỏ trên tập con MNIST** (ví dụ 5.000 ảnh, mạng 784-100-100-10) bằng random subspace training với P dense, quét $d \in \{10, 50, 100, ..., 1000\}$ trên 1 GPU 8GB. Kỳ vọng: đường cong performance-vs-d dạng chữ S giống Hình 2 của paper, d_{int90} cỡ vài trăm. Diễn giải: xác nhận lại hiện tượng cơ bản của paper ở quy mô nhỏ, kiểm tra code trước khi áp dụng ý tưởng cho mô hình thật.
2. **So sánh d_{int90} của cùng một MLP ở 3 độ rộng khác nhau** (width 50/100/200, cùng 2 lớp ẩn) trên cùng tập dữ liệu. Đo: D tăng theo width nhưng d_{int90} có đổi nhiều không? Kỳ vọng: giống Hình 3 của paper, d_{int90} gần như không đổi dù D tăng vài lần. Diễn giải: nếu đúng, xác nhận "extra width chỉ tăng độ dư thừa"; nếu sai nhiều, có thể do mạng/dữ liệu toy quá nhỏ để quan sát hiệu ứng.
3. **Đo d_{int90} khi fine-tune một adapter nhỏ (giống LoRA) trên một mô hình ngôn ngữ nhỏ (ví dụ GPT-2 nhỏ) cho một tác vụ phân loại văn bản đơn giản**, dùng chính kỹ thuật random subspace training của paper (chiếu ngẫu nhiên không gian tham số của adapter, không phải toàn mô hình, để tiết kiệm bộ nhớ). Đo: d_{int90} của adapter cho tác vụ đó là bao nhiêu so với rank LoRA thường chọn theo kinh nghiệm (ví dụ rank 8/16/32)? Diễn giải: nếu d_{int90} đo được nhỏ hơn rank thường dùng, gợi ý có thể giảm rank mà không mất hiệu năng; nếu lớn hơn, gợi ý rank hiện tại đang hạn chế hiệu năng.
4. **Thí nghiệm ghi nhớ nhãn ngẫu nhiên (shuffled label) trên tập con nhỏ** (ví dụ 1.000 ảnh MNIST, nhãn xáo trộn hoàn toàn), đo d_{int90} (theo training accuracy, không phải validation vì validation sẽ luôn ở mức ngẫu nhiên). Kỳ vọng: d_{int90} gần với số tham số cần để ghi nhớ hoàn toàn (xấp xỉ vài số thực mỗi nhãn, tăng khi tập nhỏ hơn, giống Bảng S2 của paper). Diễn giải: minh hoạ trực quan sự khác biệt giữa "học" (generalization, d_{int} thấp) và "nhớ" (memorization, d_{int} cao gần D).
5. **So sánh thời gian huấn luyện của 2 phương pháp chiếu (dense vs. sparse ngẫu nhiên, bỏ Fastfood vì phức tạp cài đặt hơn) trên một mạng CNN nhỏ** với D cỡ 10^5-10^6 trên GPU 8GB, đo thời gian mỗi bước forward+backward. Kỳ vọng: sparse nhanh hơn dense ở D lớn, giống xu hướng Bảng S4 của paper (có thể không đạt đúng hệ số lý thuyết $\sqrt{D}$ do giới hạn thư viện, giống paper tự ghi nhận). Diễn giải: hiểu chi phí thực tế của kỹ thuật này trước khi áp dụng cho mô hình lớn hơn.

10. Tóm tắt công thức / khái niệm

- **Intrinsic dimension qua codimension:** $D = d_{int} + s$ (1), với s là số chiều của tập nghiệm.
- **Huấn luyện trong subspace:** $\theta^{(D)} = \theta_0^{(D)} + P\,\theta^{(d)}$ (2), $P \in \mathbb{R}^{D \times d}$ cố định ngẫu nhiên, chỉ $\theta^{(d)}$ được học, khởi tạo $\theta^{(d)} = 0$.
- d_{int90} / d_{int100}: chiều subspace nhỏ nhất mà hiệu năng đạt tương ứng 90%/100% so với baseline trực tiếp (ngưỡng đo trên validation accuracy hoặc reward đã dịch chuẩn).
- **3 phương pháp chiếu và độ phức tạp:** Dense $O(Dd)$; Sparse (mật độ $1/\sqrt{D}$) lý thuyết $O(\sqrt{Dd})$; Fastfood $O(D \log d)$ thời gian, $O(D)$ không gian, dùng phân tích $M = HG\Pi HB$.
- **Hệ số nén (tự tính, theo ví dụ MNIST-FC của paper):** $D/d_{int90} = 199.210/750 \approx 265,6$ lần (paper

nói ”260x”, khớp xấp xỉ với tự tính ở đây, chênh nhỏ do cách quy đổi byte/bit). Kích thước lưu trữ ước tính $d_{int90} \times 4$ byte (float32) $= 750 \times 4 = 3.000$ byte $\approx 2,93$ KB (paper ghi ”3,2kB”; tự tính ra 2,93KB, chênh nhỏ có thể do paper tính thêm vài byte cho 2 random seed hoặc quy đổi KB kiểu khác, không ảnh hưởng tới kết luận định tính về hệ số nén rất lớn).

- **MDL diễn giải:** d_{int} (đo bằng số bậc tự do, không phải bit) là chặn trên xây dựng được (constructive upper bound) cho Minimum Description Length của nghiệm, theo tinh thần Rissanen (1978) và Hinton & Van Camp (1993): với độ chính xác bằng nhau, mô hình MDL ngắn hơn được xem là ”tốt hơn” theo nguyên lý này.

11. Kết luận ngắn

Bài này đưa ra một định nghĩa đo được (random subspace training) cho ”intrinsic dimension” của objective landscape, cho thấy nhiều bài toán deep learning có d_{int} thấp hơn số tham số D rất nhiều, và d_{int} khá ổn định khi mô hình được mở rộng (width/depth) cho cùng một bài toán, tức tham số thêm vào chủ yếu làm tăng độ dư thừa của tập nghiệm chứ không thay đổi độ khó cốt lõi. Phát hiện phụ: d_{int} cho phép so sánh độ khó giữa supervised và RL trên cùng một thước đo, và mở ra một kỹ thuật nén mạng đơn giản (lưu seed + d số thực) dù không có lợi ích tốc độ suy luận. Người ghi chú phát hiện hai điểm số liệu không khớp giữa bảng chính và bảng phụ lục (xem mục 6), nên khi trích dẫn số cụ thể của paper này cần đối chiếu lại nguồn gốc bảng.

Nguồn gốc: https://arxiv.org/abs/1804.08838

Bài 20

Benefits of Depth in Neural Networks (2016)

Tác giả: Matus Telgarsky (University of Michigan, Ann Arbor; email ghi trong paper là @cs.ucsd.edu, giữ nguyên như bản in).
Venue / arXiv: arXiv:1602.04485 (v2, 27/05/2016); venue in trong paper: ”JMLR: Workshop and Conference Proceedings vol 49:1-23, 2016” (đây chính là kỷ yếu COLT 2016, nhưng paper chỉ in dòng JMLR W&CP; tên ”COLT 2016” là ”(ngoài paper, kiến thức chung)”).
Nguồn: https://arxiv.org/abs/1602.04485
Chủ đề: chứng minh tồn tại các hàm tính được bởi mạng ReLU rất sâu (Θ(k^3) lớp, Θ(1) node mỗi lớp) mà không mạng nào nông hơn (O(k) lớp) có thể xấp xỉ tốt, trừ khi mạng nông đó cực lớn (Ω(2^k) node); kết quả tổng quát cho một lớp gate rộng gọi là ”semi-algebraic gate” (ReLU, max, piecewise polynomial, decision tree).
Vị trí trong lộ trình: đọc cùng cặp với math 21 (Expressive Power of Neural Networks: A View from the Width) để so sánh hai câu hỏi đối ngẫu ”độ sâu giúp gì” và ”độ rộng giúp gì”. Nên đọc sau khi đã quen các khái niệm xấp xỉ hàm (không bắt buộc bài trước). Sau cặp 20-21, có thể đọc math 25 (Principles of Deep Learning Theory) vì bài đó định nghĩa tỉ số khía cạnh (aspect ratio) L/n (depth-to-width) làm trục trung tâm, một cách nhìn định lượng hơn cho câu hỏi ”sâu bao nhiêu, rộng bao nhiêu là đủ” mà bài 20/21 đặt ra dưới dạng định lý tồn tại/không tồn tại.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Một mạng neural là một mô hình tính toán thực-trị, biểu diễn bằng một đồ thị có hướng không chu trình: mỗi node nhận các số thực trên cạnh vào, tính một hàm của chúng, rồi truyền kết quả ra cạnh ra. Điểm khác biệt của mạng neural so với các mô hình học máy khác (SVM, boosted decision stump) là nó có **nhiều lớp (layer)**: đầu ra là kết quả của việc **ghép (compose)** nhiều phép toán phi tuyến liên tiếp, trong khi các mô hình kia tương đương mạng với số lớp hằng số.

Định lý xấp xỉ tổng quát cổ điển (universal approximation, Cybenko 1989, Hornik et al. 1989, không trích dẫn chi tiết trong paper này nhưng được nhắc tới gián tiếp qua các công trình liên quan) nói rằng mạng có độ sâu cố định nhỏ (ví dụ 1 lớp ẩn) nhưng **đủ rộng** đã có thể xấp xỉ bất kỳ hàm liên tục trên miền compact. Nhưng độ rộng cần thiết có thể tăng theo cấp số nhân với số chiều đầu vào. Câu hỏi đặt ra: tăng độ sâu có giúp mạng biểu diễn được các hàm ”khó” với kích thước (số node, số tham số) nhỏ hơn hẳn so với mạng nông không?

Công trình gần nhất trước đó, Eldan và Shamir (2015), chỉ ra sự tách biệt giữa mạng 2 lớp và 3 lớp (trường hợp riêng k=2) khi số chiều đầu vào d lớn: có hàm 3 lớp không thể xấp xỉ bởi mạng 2 lớp subexponential-size. Paper này (Telgarsky 2016) tổng quát hoá cho **mọi k nguyên dương** và cho **nhiều loại gate** (không chỉ ReLU chuẩn), đồng thời làm việc với d cố định bất kỳ (bao gồm d=1), một chế độ mà các công trình trước không xử lý triệt để.

Các khái niệm cần nắm trước khi đọc mục 2-3:

- **ReLU**: $\sigma_R(z) := \max\{0, z\}$.
- **Semi-algebraic gate** (t, α, β)-sa: một hàm $f : \mathbb{R}^k \to \mathbb{R}$ được ghép từ t đa thức bậc $\leq \alpha$ (dùng làm ”ngưỡng” chia vùng) và các đa thức bậc $\leq \beta$ trên mỗi vùng. Khung này gộp được ReLU, max/min-gate, piecewise polynomial, và decision tree vào một định nghĩa duy nhất.

- **Khoảng cách L1**: sai lệch điểm $|f(x) - g(x)|$ lấy trung bình (tích phân) trên khối $[0,1]^d$.

2. Ý tưởng chính

Kết quả trung tâm là **Định lý 1.1**: với mọi số nguyên $k \geq 1$ và số chiều $d \geq 1$, tồn tại hàm $f : \mathbb{R}^d \to \mathbb{R}$ tính được bởi một mạng ReLU chuẩn có $2k^3 + 8$ lớp, $3k^3 + 12$ node, và $4 + d$ tham số riêng biệt (một tham số có thể xuất hiện ở nhiều cạnh), sao cho

$$\inf_{g \in \mathcal{C}} \int_{[0,1]^d} |f(x) - g(x)|\, dx \geq \frac{1}{64},$$

trong đó $\mathcal{C}$ là hợp của (a) các hàm tính bởi mạng (t, α, β)-semi-algebraic với $\leq k$ lớp và $\leq 2^k/(t\alpha\beta)$ node, và (b) các hàm là tổ hợp tuyến tính của $\leq t$ decision tree, mỗi cây $\leq 2^{k^3}/t$ node.

Nói cách khác: có một hàm cần $\Theta(k^3)$ lớp nhưng chỉ $\Theta(1)$ node mỗi lớp để tính chính xác, mà bất kỳ mạng nông hơn ($\leq k$ lớp, tức là giảm độ sâu đi hệ số k^2) muốn xấp xỉ nó trong sai số L1 nhỏ hơn 1/64 đều phải có kích thước **lớn theo cấp số nhân** ($\Omega(2^k)$ node, hoặc với decision tree thì $\Omega(2^{k^3})$ tổng số node cây, một phân biệt mạnh hơn).

Ý tưởng chứng minh dựa trên quan sát về **dao động (oscillation)**: ghép (compose) các hàm "sóng tam giác" (triangle wave) qua nhiều lớp làm số lần hàm cắt ngang ngưỡng 1/2 tăng theo **cấp số nhân** với số lớp, còn việc chỉ thêm node trong một lớp (addition, không tăng độ sâu) chỉ làm số lần cắt tăng theo cấp số nhân với độ sâu hiện có (bị chặn bởi một hàm e mũ theo depth, chứ không mũ theo chính depth đó lần nữa) - nói đơn giản: composition "thắng" addition trong việc tạo dao động, giống cách ghép đa thức (degree nhân nhau) tạo nhiều nghiệm hơn cộng đa thức (degree chỉ lấy max).

Paper cũng đưa ra **kết quả đồng hành** (companion, Định lý 1.2 và Lemma 1.3): số lượng hàm "khó" như Định lý 1.1 không nhiều - mạng $\Theta(k^3)$ lớp, dù biểu diễn được hàm dao động mạnh, có **VC dimension** bị chặn khá chặt, nên trên n điểm dữ liệu với nhãn ngẫu nhiên đồng xác suất, mạng đó cũng chỉ fit đúng được khoảng một nửa nhãn (sai số $\geq 1/4$) khi n đủ lớn so với số tham số. Điều này cho thấy "mạng sâu mạnh" không có nghĩa là "mạng sâu fit được mọi nhãn ngẫu nhiên".

3. Phương pháp chi tiết

3.1 Semi-algebraic gate và khung ký hiệu mạng

Định nghĩa 2.1: $f : \mathbb{R}^k \to \mathbb{R}$ là (t, α, β)-sa nếu có t đa thức $(q_i)_{i=1}^t$ bậc $\leq \alpha$ và m bộ ba (U_j, L_j, p_j) với p_j bậc $\leq \beta$, sao cho $f(v) = \sum_j p_j(v) \prod_{i \in L_j} \mathbb{1}[q_i(v) < 0] \prod_{i \in U_j} \mathbb{1}[q_i(v) \geq 0]$. Số hạng m không ảnh hưởng tới các chặn độ phức tạp trong bài, nên không xuất hiện trong tên gọi.

Các trường hợp riêng quan trọng (Lemma 2.3, đã kiểm trong text):

- ReLU chuẩn $v \mapsto \sigma_R(\langle a, v\rangle + b)$ là $(1, 1, 1)$-sa.
- max/min gate trên r đa thức bậc $\leq \alpha$ là $(r(r-1), \alpha, \alpha)$-sa.
- Decision tree k node là $(k, 1, 0)$-sa; boosted decision tree t cây mỗi cây $\leq k$ node là $(tk, 1, 0)$.

Ký hiệu mạng: lớp 0 là input (d node, node i là $x \mapsto x_i$); $\mathcal{N}_d((m_i, t_i, \alpha_i, \beta_i)_{i=1}^l)$ là lớp hàm tính bởi mạng có l lớp (không tính lớp 0), lớp i có $\leq m_i$ node (t_i, α_i, β_i)-sa.

3.2 Đếm dao động (crossing number)

Với hàm 1 biến $f : \mathbb{R} \to \mathbb{R}$, đặt $\tilde{f}(x) := \mathbb{1}[f(x) \geq 1/2]$ và **crossing number** $\mathrm{Cr}(f) := |I_f|$ với I_f là phân hoạch $\mathbb{R}$ thành các đoạn mà $\tilde{f}$ không đổi (ví dụ $\mathrm{Cr}(\sigma_R) = 2$).

Lemma 3.1 (xấp xỉ qua đếm dao động): nếu $\mathrm{Cr}(f) \gg \mathrm{Cr}(g)$ thì

$$\frac{1}{\mathrm{Cr}(f)} \sum_{U \in I_f} \mathbb{1}[\forall x \in U,\ \tilde{f}(x) \neq \tilde{g}(x)] \ \geq\ \frac{1}{2}\Big(1 - \frac{2\,\mathrm{Cr}(g)}{\mathrm{Cr}(f)}\Big).$$

Ý nghĩa: nếu g dao động ít hơn f rất nhiều, thì trên phần lớn các đoạn mà $\tilde{f}$ dao động, $\tilde{g}$ không theo kịp nên khác dấu với $\tilde{f}$, dẫn tới sai số xấp xỉ lớn.

Lemma 3.2 (ít lớp, ít dao động - mấu chốt của "benefits of depth"): với h là phép affine $\mathbb{R} \to \mathbb{R}^d$ và $f \in \mathcal{N}_d((m_i, t_i, \alpha_i, \beta_i)_{i=1}^{l})$, đặt $\alpha = \max_i \alpha_i$, $\beta = \max_i \beta_i$, $t = \max_i t_i$, $m = \sum_i m_i$:

$$\mathrm{Cr}(f \circ h) \leq 2\Big(\frac{2tm\alpha}{l}\Big)^{l} \beta^{l^2}.$$

Đây là **điểm mấu chốt**: số lớp l nằm ở **số mũ**, còn số node m nằm ở **cơ số**. Muốn tăng crossing number theo cấp số nhân mà chỉ tăng m (số node, giữ l cố định) thì cần m tăng theo cấp số nhân; nhưng nếu tăng l (độ sâu) thì crossing number tăng theo cấp số nhân của l với cơ số nhỏ.

Việc chứng minh Lemma 3.2 dựa trên một chuỗi bổ đề về việc ghép hàm semi-algebraic với hàm piecewise-polynomial (Lemma 3.3-3.6): hợp các phân hoạch của k hàm $(t, \cdot)$-poly thành một phân hoạch mịn hơn có $\leq kt$ đoạn (Lemma 3.4), từ đó suy luận quy nạp theo lớp rằng node ở lớp i là $(2^i T_i A_i C_{i-1} M_{i-1}, B_i)$-poly với $T_i, A_i, \ldots$ là tích lũy các $t_j, \alpha_j, \ldots$ của các lớp trước.

3.3 Xây dựng hàm nhiều dao động: sóng tam giác

Hàm $f(z) := \sigma_R(2\sigma_R(z) - 4\sigma_R(z - 1/2)) = \min\{\sigma_R(2z), \sigma_R(2 - 2z)\}$ (Lemma 3.10) là một mạng ReLU **1 lớp, 2 node** ($\in \mathcal{N}_1(2, 1, 1, 1, 1)$), đồ thị là một tam giác từ $(0, 0)$ lên $(1/2, 1)$ xuống $(1, 0)$.

Định nghĩa "triangle" (3.7): f là $(t, [0, 1])$-triangle nếu liên tục, có thể chia $[0, 1]$ thành $2t$ đoạn xen kẽ tăng/giảm, giá trị tại các mốc bằng 0 hoặc 1 xen kẽ.

Lemma 3.8: nếu f là $(s, [0, 1])$-triangle và g là $(t, [0, 1])$-triangle thì $f \circ g$ là $(2st, [0, 1])$-triangle. Ghép (composition) nhân số "đỉnh" lại.

Corollary 3.9: với $f \in \mathcal{N}_1(m, l, t, \alpha, \beta)$ là $(t, [0, 1])$-triangle, thì f^k (ghép k lần) $\in \mathcal{N}_1(m, kl, t, \alpha, \beta)$ là $(2^{k-1}t^k, [0, 1])$-triangle với $\mathrm{Cr}(f^k) = (2t)^k + 1$.

Áp dụng cho f ở trên ($t = 1$): f^k chỉ cần k lớp (vì f gốc 1 lớp) nhưng có $\mathrm{Cr}(f^k) = 2^k + 1$ - **dao động tăng theo cấp số nhân với số lớp**, đúng như trực giác composition-thắng-addition ở mục 2.

3.4 Chứng minh Định lý 1.1 (phác thảo đã kiểm trong text)

Đặt $f_0(z) := h^{k^3+4}(z)$ với h là hàm tam giác ReLU ở trên; theo Corollary 3.9, $\mathrm{Cr}(f_0) = 2^{k^3+4} + 1$. Định nghĩa $f(x) = f_0(x_1)$ (chỉ phụ thuộc toạ độ đầu).

Với mạng nông g (semi-algebraic, $\leq k$ lớp, $m \leq 2^k/(t\alpha\beta)$ node), áp Lemma 3.2 cho $g \circ p_y$ (với $p_y(z) := (z, y)$ chiếu lên 1 chiều):

$$\mathrm{Cr}(g \circ p_y) \leq 2\Big(\frac{2tm\alpha}{k}\Big)^{k^2} \beta^{k^3} \leq 4(tm\alpha\beta)^{k^2} \leq 2^{k^3+2}$$

(nếu g là (t, k)-bdt thay vào đó thì $\mathrm{Cr}(g \circ p_y) \leq 2t\, 2^{k^3}/t \leq 2^{k^3+2}$ tương tự).

Áp Lemma 3.1 với $\mathrm{Cr}(f) = 2^{k^3+4} + 1 \gg \mathrm{Cr}(g) \leq 2^{k^3+2}$, kết hợp với tính "đều" (regularity) của hàm tam giác ReLU để chuyển từ bất đẳng thức trên tập chỉ số sang tích phân L1 thật, paper suy ra sai số $\geq 1/64$ (chi tiết số học nằm ở việc chia hằng số 1/16, 1/32 qua các bước).

Ví dụ tự tính (minh hoạ, $k = 2$, không phải số trong bảng paper vì paper không có bảng số cho Định lý 1.1): kiến trúc mạng sâu có $2k^3 + 8 = 24$ lớp, $3k^3 + 12 = 36$ node, $4 + d$ tham số (lấy $d = 1$: 5 tham số). Hàm f_0 tương ứng với ghép $k^3 + 4 = 12$ lần, crossing number $\mathrm{Cr}(f_0) = 2^{12} + 1 = 4097$ (tự tính). Mạng nông bị hạn chế $\leq k = 2$ lớp, giả sử $t = \alpha = \beta = 1$ (ReLU gate thuần) thì $m \leq 2^2/1 = 4$ node mỗi phía, và theo công thức trên $\mathrm{Cr}(g \circ p_y) \leq 2^{k^3+2} = 2^{10} = 1024$ (tự tính). Tỉ số $\mathrm{Cr}(f)/\mathrm{Cr}(g) \approx 4097/1024 \approx 4$ (tự tính): mạng nông dao động kém hơn mạng sâu khoảng 4 lần ngay ở $k = 2$, và khoảng cách này tăng theo cấp số nhân với k (ở số mũ k^3) - minh hoạ trực quan cho "benefits of depth", dù phép tính bằng tay này chỉ mang tính minh hoạ cơ chế, không phải số liệu paper công bố.

3.5 Hạn chế dao động: VC dimension (Mục 4 của paper)

Lemma 4.2: mạng $\leq p$ tham số, $\leq l$ lớp, $\leq m$ node (t, α, β)-sa có VC $\leq 6p(l+1)\big(\ln(2p(l+1)) + \ln(8emt\alpha) + l\ln\beta\big)$.

Lemma 4.1 (kiểu Gilbert-Varshamov qua bất đẳng thức Hoeffding): với n điểm và nhãn ngẫu nhiên đều, xác suất $\geq 1 - \delta$ là mọi f trong lớp hàm có growth function $\mathrm{Sh}(\mathcal{F}; n)$ đều fit sai $\geq \frac{1}{2}\big(1 - \sqrt{(\ln \mathrm{Sh}(\mathcal{F}; n) + \ln(1/\delta))/(2n)}\big)$ tỉ lệ nhãn.

Kết hợp hai bổ đề này (Định lý 1.2): với $n \geq 8pl^2 \ln(8emt\alpha\beta p(l+1)) + 4\ln(1/\delta)$ điểm, xác suất $\geq 1 - \delta$, mạng $\Theta(k^3)$-lớp ở Định lý 1.1 fit sai $\geq 1/4$ tỉ lệ nhãn ngẫu nhiên trên $\Theta(k^9)$ điểm - dù mạng này biểu diễn được hàm dao động cực mạnh, nó **không** fit được nhãn ngẫu nhiên tốt hơn một mạng nhỏ nhiều.

4. Thí nghiệm và kết quả chính

Đây là paper lý thuyết túy; **không có thí nghiệm thực nghiệm số nào** trong toàn văn (đã đọc hết Mục 1-5 và Phụ lục A, không có hình minh hoạ chạy số, không có bảng số liệu huấn luyện). "Kết quả chính" ở đây là các định lý toán học:

Định lý	Nội dung	Loại chặn
1.1	Hàm $\Theta(k^3)$-lớp, $\Theta(1)$-node/lớp không xấp xỉ được (sai L1 $\geq 1/64$) bởi mạng $\leq k$ lớp trừ khi $\Omega(2^k)$ node (hoặc $\Omega(2^{k^3})$ node cây cho boosted tree)	tồn tại (lower bound cho depth efficiency)
1.2 / Lemma 1.3	VC dimension mạng semi-algebraic $O(pl\log(\cdot))$	chặn trên, hạn chế khả năng fit nhãn ngẫu nhiên
3.12	Tổng quát 1.1 cho đo rời rạc/liên tục khác, hằng số sai số khác (1/32, 1/8...)	mở rộng kỹ thuật
Corollary A.1	Tồn tại đa thức bậc 2^k không xấp xỉ được bởi đa thức bậc $\leq 2^{k-3}$ (sai $\geq 1/32$)	hệ quả phụ, liên hệ lý thuyết đa thức cổ điển (Weierstrass, Kolmogorov, Warren)

Vì không có benchmark thực nghiệm, Mục 4 của ghi chú này chỉ tóm tắt các định lý/chặn như trên theo đúng yêu cầu brief cho paper lý thuyết.

5. Phân tích / ablation

Paper không có ablation theo nghĩa thực nghiệm; dưới đây là phân tích cấu trúc của các chặn, dựa hoàn toàn trên nội dung đã đọc:

- **Khoảng giữa depth và width trong chặn chính**: hàm khó cần $\Theta(k^3)$ lớp để xấp xỉ bởi mạng k lớp cần $\Omega(2^k)$ node - đây là **tách biệt theo hệ số đa thức bậc 3 về độ sâu** đối lấy **tách biệt theo cấp số nhân về kích thước**. Tác giả không claim đây là chặn chặt tối ưu (ví dụ không chứng minh được việc giảm độ sâu đi 1 đơn vị, từ $k+1$ xuống k, đã đòi hỏi cỡ lớn theo cấp số nhân; chặn chỉ áp dụng khi giảm độ sâu đi hệ số k^2, từ $\Theta(k^3)$ xuống $O(k)$).

- **Vai trò của decision tree**: với boosted decision tree, chặn mạnh hơn ($\Omega(2^{k^3})$ tổng số node cây so với $\Omega(2^k)$ cho mạng neural nông) - paper lý giải vì cấu trúc cây "cộng tuyến tính nhiều cây nông" vốn đã yếu hơn khi so composition.
- **Vai trò của chiều dữ liệu** d: Định lý 1.1 đúng với d cố định bất kỳ (bao gồm $d = 1$), khác với Eldan-Shamir (2015) cần d lớn. Đây là điểm paper nhấn mạnh là cải tiến kỹ thuật (Mục 5, "Bibliographic notes").
- **Bổ đề đếm dao động là công cụ 1 chiều**: toàn bộ chứng minh quy về hàm 1 biến (sau khi chiếu qua affine map p_y) rồi lấy trung bình trên các hướng chiếu còn lại $y \in \mathbb{R}^{d-1}$ - kỹ thuật "random projection" mà paper tự nhận là "rất thô sơ" (extremely primitive form) so với kỹ thuật dùng trong depth-hierarchy cho mạch boolean (Håstad 1986; Rossman et al. 2015).
- **Lemma 1.3/4.2 không làm yếu Định lý 1.1**: VC dimension nhỏ của mạng sâu không mâu thuẫn với khả năng biểu diễn hàm dao động mạnh của nó, vì VC dimension đo khả năng fit nhãn tùy ý trên **n điểm cho trước**, còn Định lý 1.1 nói về một hàm **cụ thể** (không phải nhãn ngẫu nhiên) cần nhiều lớp để xấp xỉ.

6. Giới hạn

- Chỉ là **kết quả tồn tại** (existence): paper chứng minh có một hàm cụ thể khó xấp xỉ, không mô tả được "có bao nhiêu" hàm như vậy hay chúng có xuất hiện trong các hàm mục tiêu thực tế (dữ liệu ảnh, văn bản...) hay không. Tác giả tự nêu trong Mục 5: hướng mở là đặc trưng hoá **tất cả** các hàm khó xấp xỉ bởi độ sâu nhỏ, không chỉ một ví dụ.
- Hàm xây dựng (sóng tam giác lặp) có cấu trúc rất đặc biệt (dao động đều, tần số tăng dần theo cấp số nhân) - không có bằng chứng hàm như vậy xuất hiện tự nhiên trong các bài toán học máy thực tế.
- Chặn chỉ so sánh $\Theta(k^3)$ lớp với $O(k)$ lớp (một khoảng cách bậc 3 theo k, không phải $k + 1$ so với k); do đó không trả lời được câu hỏi "giảm đúng 1 lớp có luôn cần tăng kích thước theo cấp số nhân không" - đây là giới hạn mà Eldan-Shamir (2015) với trường hợp $k = 2$ (so 2 và 3 lớp) từng trả lời được cho trường hợp riêng khi d lớn, nhưng paper 20 không mở rộng điều đó cho k tổng quát.
- VC dimension bound (Định lý 1.2/Lemma 4.2) dùng kỹ thuật của Warren (1968) cho đa thức, chỉ là "điều chỉnh nhỏ" (minor departure) theo lời tác giả, không tối ưu hoá riêng cho semi-algebraic gate.
- Không có thí nghiệm số nào để kiểm chứng trực giác (khác với bài 21 có bảng thực nghiệm).

7. Liên hệ với các bài khác

- **math 21** (The Expressive Power of Neural Networks: A View from the Width): câu hỏi đối ngẫu của chính bài này - thay vì hỏi "sâu giúp gì" thì hỏi "rộng giúp gì". Bài 21 trích dẫn trực tiếp bài 20 ("Benefits of depth in neural networks (Telgarsky, 2016)") làm nền depth-efficiency để so sánh với width-efficiency họ chứng minh (đã kiểm: trong references của 1709.02540 có "[14] Matus Telgarsky. Benefits of depth in neural networks. COLT 2016").
- **math 22 / math 23** (Deep Neural Networks as Gaussian Processes; Neural Tangent Kernel): nhìn mạng neural theo hướng hoàn toàn khác - không phải "mạng nào biểu diễn được hàm nào" mà là hành vi thống kê của mạng **rất rộng** (width $\to \infty$) tại khởi tạo và trong huấn luyện. Khái niệm "rộng" trong 2 bài đó là lợi thế (đưa về Gaussian process/kernel), trái với tinh thần bài 20 là "sâu mới là lợi thế biểu diễn" - một tương phản thú vị giữa lý thuyết xấp xỉ (approximation theory, bài 20/21) và lý thuyết huấn luyện (training theory, bài 22/23).
- **math 25** (The Principles of Deep Learning Theory): định nghĩa tỉ số khía cạnh $r = L/n$ (depth-to-width) làm biến điều khiển chính của lý thuyết hiệu dụng (effective theory); khi $r \to 0$ mạng ứng xử như vô hạn rộng (mất khả năng học biểu diễn), khi $r \gg 1$ mạng "hỗn loạn" (chaotic, không tính được). Bài 20 góp một mảnh ghép định tính cho phía "tăng L (độ sâu)" của tỉ số này từ góc độ biểu diễn hàm, chứ không phải từ góc độ huấn luyện như math 25.
- **pruning content/32_2024_unreasonable_ineffectiveness_deeper_layers.md** (đã kiểm tồn tại bằng

`ls`): thực nghiệm cho thấy nhiều lớp sâu trong LLM hiện đại có thể bị xoá mà độ chính xác giảm ít - một quan sát thực nghiệm dường như ”mâu thuẫn” với trực giác lý thuyết của bài 20 (sâu hơn luôn biểu diễn được hàm khó hơn); cách dung hoà hợp lý (suy luận của người ghi chú, không phải trong 2 paper): bài 20 chỉ nói có **tồn tại** hàm cần nhiều lớp, không nói **mọi** lớp của **mọi** mạng đã huấn luyện đều cần thiết cho hàm mục tiêu thực tế mà mạng đó đang biểu diễn.

- **pruning content/38_2023_sheared_llama.md** (đã kiểm tồn tại bằng `ls`): pruning theo chiều sâu (loại bỏ layer) là một trong các chiều cắt được dùng thực tế; bài 20 cho trực giác lý thuyết (yếu) về việc giảm độ sâu có thể ”cắt cầu” một số hàm dao động mạnh mà mô hình đã học, cần đền bù bằng việc tăng mạnh kích thước các lớp còn lại.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU $\leq$ 8 GB, batch 1). Đây là paper lý thuyết xấp xỉ hàm thuần túy trên mạng ReLU feedforward đơn giản, không nói trực tiếp về transformer/attention/SSM, nên liên hệ chỉ mang tính **suy luận định tính**, không phải áp dụng công thức trực tiếp:

- Khi quyết định **cắt theo chiều sâu** (depth pruning, bỏ nguyên layer) so với **cắt theo chiều rộng** (width pruning, giảm d_{model}/số head) cho LightOnOCR, bài 20 gợi ý rằng cắt sâu ”mạnh tay” (bỏ nhiều lớp liên tiếp) có nguy cơ làm mất khả năng biểu diễn các hàm cần nhiều bước ghép phi tuyến (ví dụ các phép biến đổi OCR nhiều giai đoạn: dò layout, nhận ký tự, ghép ngữ cảnh) nhiều hơn là cắt rộng với số lượng tham số tương đương, vì theo trực giác của Lemma 3.2, giảm l (số lớp) làm giảm **số mũ** trong chặn trên khả năng dao động/biểu diễn, còn giảm m (số node) chỉ giảm **cơ số**.
- Điều này gợi ý (suy luận) một **ưu tiên thực hành** khi nén mô hình 0.8B trên GPU nhỏ: nếu phải chọn giữa giảm độ sâu hay giảm độ rộng ở mức độ tương đương về số tham số, nên nghiêng về giảm độ rộng (width) nhiều hơn độ sâu (depth), rồi bù lại bằng fine-tune/distillation nhẹ - khớp với quan sát thực nghiệm ở pruning 32 và pruning 38 rằng xoá layer ”tốn ít” hơn tưởng, nhưng không nên xoá quá nhiều layer liên tiếp ở vùng giữa mạng nếu mô hình cần biểu diễn các phép biến đổi nhiều bước (như luồng OCR).
- Nhìn chung: **ít liên quan trực tiếp về công thức** (không có công thức nào trong bài 20 áp dụng thẳng vào kiến trúc hybrid attention/SSM), nhưng **có giá trị trực giác** cho việc thiết kế chiến lược pruning theo depth vs. width.

9. Thí nghiệm gợi ý

1. **Tái dựng hiệu ứng dao động theo độ sâu (toy, 1 GPU 8GB, thậm chí chạy CPU)**: xây mạng ReLU 1 chiều ghép k lần hàm tam giác $f(z) = \min\{2z, 2 - 2z\}$ (PyTorch, vài dòng), vẽ đồ thị f^k với $k = 1, ..., 6$, đếm số lần cắt ngang 0.5 bằng code, so với công thức $\text{Cr}(f^k) = 2^k + 1$ (kỳ vọng khớp đúng - nếu không khớp, kiểm tra lại điều kiện biên).
2. **Đo sai số xấp xỉ mạng nông vs mạng sâu cùng số tham số**: cố định tổng số tham số (ví dụ 200), chia cho 2 mạng: (a) sâu (nhiều lớp, ít node/lớp) ghép hàm tam giác nhiều lần, (b) nông (ít lớp, nhiều node/lớp), train mạng nông để fit lại hàm f^k của mạng sâu bằng MSE loss, đo sai số L1 trên $[0, 1]$. Kỳ vọng: sai số mạng nông tăng nhanh khi k tăng, minh hoạ Định lý 1.1 trong môi trường có train (không phải existence proof) - một phép kiểm định kinh nghiệm, không phải chứng minh lại định lý.
3. **Depth-pruning một mạng ReLU nhỏ đã huấn luyện fit hàm dao động**: huấn luyện mạng 12 lớp fit $f^6(z)$ (hàm tam giác ghép 6 lần) trên $[0, 1]$, sau đó xoá dần từng layer (giữ nguyên các trọng số còn lại, có thể fine-tune nhẹ), đo lại sai số MSE theo số layer còn lại. So sánh với cắt rộng (giảm số node mỗi lớp xuống một nửa, giữ đủ layer). Kỳ vọng: cắt sâu cho hàm dao động nhiều này sẽ giảm chất lượng nhanh hơn cắt rộng tương đương về số tham số, đúng trực giác Mục 8 (nhưng đây là hàm tổng hợp đặc biệt, không đại diện cho toàn bộ không gian hàm mục tiêu thực tế).

4. **Kiểm chứng VC dimension/khả năng fit nhãn ngẫu nhiên**: lấy mạng ReLU 12 lớp nhỏ (≤ 50 tham số), gán nhãn ngẫu nhiên nhị phân cho 200 điểm ngẫu nhiên trong $[0, 1]$, train tới khi hội tụ (nếu có thể), đo train accuracy. Kỳ vọng theo Định lý 1.2: với n điểm đủ lớn so với tham số, train accuracy khó vượt quá khoảng 75% (sai $\geq 1/4$) dù mạng có thể biểu diễn hàm dao động rất mạnh - minh hoạ rằng biểu diễn được hàm khó không đồng nghĩa fit được nhãn ngẫu nhiên tùy ý.

10. Tóm tắt công thức / khái niệm

Khái niệm/công thức	Ý nghĩa
$\sigma_R(z) = \max\{0, z\}$	ReLU
(t, α, β)-sa	semi-algebraic gate: t đa thức ngưỡng bậc $\leq \alpha$, giá trị là đa thức bậc $\leq \beta$ trên mỗi vùng
$\mathrm{Cr}(f) = \lvert I_f \rvert$	crossing number: số đoạn mà $\tilde{f} = \mathbb{1}[f \geq 1/2]$ không đổi
Lemma 3.1	$\mathrm{Cr}(f) \gg \mathrm{Cr}(g) \Rightarrow$ sai số xấp xỉ $\geq \frac{1}{2}(1 - 2\mathrm{Cr}(g)/\mathrm{Cr}(f))$ (dạng rời rạc)
Lemma 3.2	$\mathrm{Cr}(f \circ h) \leq 2(2tm\alpha/l)^l \beta^{l^2}$: số lớp l ở số mũ, số node m ở cơ số
$(t, [0, 1])$-triangle, Cor. 3.9	ghép k lần hàm tam giác t-đỉnh cho $\mathrm{Cr}(f^k) = (2t)^k + 1$
Định lý 1.1	$\exists f$: $2k^3{+}8$ lớp, $3k^3{+}12$ node, $4{+}d$ tham số, không mạng $\leq k$ lớp và $o(2^k)$ node nào xấp xỉ sai $< 1/64$
Định lý 1.2 / Lemma 1.3	$\mathrm{VC}(\mathcal{N}(G)) \leq 6p(l{+}1)\big(\ln(2p(l{+}1)) + \ln(8emt\alpha) + l \ln \beta\big)$
Corollary A.1	tồn tại đa thức bậc 2^k không xấp xỉ được bởi đa thức bậc $\leq 2^{k-3}$, sai $\geq 1/32$
Ví dụ tự tính ($k = 2$)	$\mathrm{Cr}(f_0) = 2^{12} + 1 = 4097$; mạng nông ≤ 2 lớp: $\mathrm{Cr}(g) \leq 2^{10} = 1024$; tỉ số ≈ 4

11. Kết luận ngắn

Bài báo chứng minh bằng công cụ ”đếm dao động” (crossing number) rằng có những hàm ReLU cần số lớp tăng theo k^3 nhưng số node mỗi lớp chỉ là hằng số, trong khi bất kỳ mạng nông hơn ($\leq k$ lớp, hoặc boosted decision tree tương đương) muốn xấp xỉ gần đúng hàm đó đều cần kích thước tăng theo cấp số nhân với k. Cơ chế cốt lõi: composition (ghép lớp) nhân dao động theo cấp số nhân, còn addition (thêm node) chỉ cộng dao động theo cấp số nhân với **độ sâu sẵn có** - số lớp nằm ở số mũ của chặn trên dao động (Lemma 3.2), đó là lý do toán học cho ”benefits of depth”. Đồng thời, paper cũng chỉ ra (qua VC dimension) rằng khả năng biểu diễn hàm dao động mạnh không đồng nghĩa với khả năng fit nhãn ngẫu nhiên tùy ý. Đây là kết quả tồn tại (existence), không phải đặc trưng hoá đầy đủ, và không có thực nghiệm số - phần thực nghiệm kiểm tra trực giác tương tự (có train) được bài 21 (math 21) thực hiện cho câu hỏi đối ngẫu về độ rộng.

Nguồn gốc: https://arxiv.org/abs/1602.04485

Bài 21

The Expressive Power of Neural Networks: A View from the Width (2017)

Tác giả: Zhou Lu, Hongming Pu, Feicheng Wang, Zhiqiang Hu, Liwei Wang (Peking University: Department of Mathematics; Key Laboratory of Machine Perception, MOE, School of EECS; Center for Data Science, Beijing Institute of Big Data Research).
Venue / arXiv: arXiv:1709.02540 (v3, 01/11/2017); venue in trong paper: ”31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.”
Nguồn: https://arxiv.org/abs/1709.02540
Chủ đề: câu hỏi đối ngẫu của ”benefits of depth”: độ rộng (width) của mạng ReLU ảnh hưởng thế nào tới khả năng biểu diễn hàm; chứng minh định lý xấp xỉ tổng quát cho mạng width-bounded, một ”phase transition” ở width = n (số chiều đầu vào), và một chặn dưới đa thức (không phải mũ) cho ”width efficiency”.
Vị trí trong lộ trình: đọc cùng cặp với math 20 (Benefits of Depth, Telgarsky 2016) mà bài này trích dẫn trực tiếp làm nền so sánh depth-efficiency; đọc trước khi vào các bài infinite-width (math 22 Gaussian Process, math 23 NTK) để thấy rằng ”rộng” ở đây (xấp xỉ hàm, width hữu hạn nhỏ) và ”rộng” ở các bài sau (width vô hạn, hành vi thống kê khi khởi tạo) là hai khái niệm khác hẳn nhau dù dùng chung từ ”width”.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Định lý xấp xỉ tổng quát cổ điển (universal approximation theorem, dẫn trong paper qua Cybenko 1989, Hornik-Stinchcombe-White 1989, Funahashi 1989, Barron 1994) nói rằng mạng **depth-2** (1 lớp ẩn) với hàm kích hoạt phù hợp và **đủ rộng** có thể xấp xỉ bất kỳ hàm liên tục trên miền compact tới sai số tuỳ ý. Nhưng kích thước (width) của mạng depth-2 như vậy có thể tăng theo cấp số nhân với số chiều đầu vào. Vì vậy, từ góc nhìn học, chỉ biết ”có thể xấp xỉ được” (universal approximation) là chưa đủ, cần biết **hiệu quả** (efficiency): kích thước mạng cần bao nhiêu.

Hướng nghiên cứu ”depth efficiency” (độ sâu giúp giảm kích thước) đã có nhiều kết quả: Eldan và Shamir (2015) cho trường hợp riêng 2 vs 3 lớp khi chiều dữ liệu lớn; Cohen, Sharir, Shashua (2016, dùng phân tích tensor) cho mạng tích chập; và **Telgarsky (2016)** (= math 20 trong thư mục này, trích dẫn trực tiếp là tài liệu tham khảo [14]) chứng minh tồn tại mạng $O(k^3)$ lớp không thể xấp xỉ bởi mạng $O(k)$ lớp trừ khi kích thước mũ 2^k.

Paper này đặt câu hỏi **đối ngẫu**: độ **rộng** (width) có vai trò gì cho khả năng biểu diễn? Nếu tráo vai trò độ sâu và độ rộng trong các định lý depth-efficiency mà vẫn đúng, thì width có ”sức mạnh” tương đương depth về mặt biểu diễn. Paper nghiên cứu mạng **fully-connected ReLU**, với **depth** h = số lớp (tính cả output, không tính input) và **width** d_m = số node tối đa trong một lớp; số chiều đầu vào là n.

2. Ý tưởng chính

Ba kết quả chính (đã kiểm từng định lý trong text, Mục 3-4 của paper):

Định lý 1 (Universal Approximation cho mạng width-bounded): với mọi hàm khả tích Lebesgue $f : \mathbb{R}^n \to \mathbb{R}$ và mọi $\varepsilon > 0$, tồn tại mạng ReLU fully-connected **width** $\leq n + 4$ (độ sâu không hạn chế) sao cho $\int_{\mathbb{R}^n} |f(x) - F_{\mathcal{A}}(x)|\, dx < \varepsilon$. Đây là bản đối ngẫu của universal approximation cổ điển: thay vì cố định depth=2 và cho width tùy ý lớn, bài này cố định width nhỏ ($n + 4$, không phụ thuộc ε) và cho depth tùy ý lớn.

Định lý 2 và 3 (phase transition tại width = n): Định lý 2 nói rằng, trừ một tập đo Lebesgue bằng 0, **không** hàm khả tích nào có thể được xấp xỉ bởi mạng width $\leq n$ trên toàn $\mathbb{R}^n$ (sai số chỉ có thể là 0 hoặc $+\infty$, không có "gần đúng"). Định lý 3 làm chặt hơn trên miền bị chặn $[-1,1]^n$: với hàm liên tục không hằng theo bất kỳ hướng nào, mạng width $\leq n-1$ có sai số dưới chặn dương $\varepsilon^* > 0$ cố định (không giảm được dù mạng sâu bao nhiêu). Hai định lý này cho thấy **width = n là ngưỡng chuyển pha**: trên ngưỡng (n+4) xấp xỉ được tùy ý tốt, dưới ngưỡng (n-1 hoặc n) thì không bao giờ xấp xỉ tốt được dù tăng độ sâu vô hạn.

Định lý 4 (width efficiency, chặn dưới đa thức): với số nguyên $k \geq n+4$, tồn tại mạng rộng $F_{\mathcal{A}}$ (width $2k^2$, depth 3) sao cho **không** mạng hẹp $F_{\mathcal{B}}$ nào (width $\leq k^{1.5}$, depth $\leq k+2$, tham số bị chặn trong $[-b,b]$) có thể đạt $\int (F_{\mathcal{A}} - F_{\mathcal{B}})^2\, dx < \epsilon$ cho một $\epsilon > 0$ nào đó. Đây là chặn dưới cho "width efficiency" - nhưng **chỉ là đa thức** ($k^{1.5}$ so với k^2), khác hẳn chặn **mũ** (2^k) của depth efficiency (bài 20). Tác giả kết luận (lời paper, Mục 5): "kết quả cung cấp bằng chứng toàn diện hơn rằng độ sâu có thể hiệu quả hơn độ rộng cho tính biểu diễn của mạng ReLU" - nhưng đây là so sánh một chặn dưới mũ (depth) với một chặn dưới đa thức (width) đã chứng minh được, **không phải** chứng minh width-efficiency chỉ có thể là đa thức (chặn trên mũ cho width chưa ai chứng minh, và chặn trên đa thức cũng chưa chứng minh - đây là open problem tác giả tự nêu).

Paper còn tiến hành **thực nghiệm** (Mục 4.1) để gợi ý rằng chặn dưới đa thức của Định lý 4 có thể gần với chặn trên thật (tức là mạng hẹp hơn một chút so với chặn dưới đã đủ xấp xỉ tốt mạng rộng) - xem Mục 4 của ghi chú này.

3. Phương pháp chi tiết

3.1 Khung ký hiệu

Mạng ReLU fully-connected: depth h (số lớp tính cả output, không tính input), width d_m (số node tối đa 1 lớp), input dimension n. $F_{k,\mathcal{A}}$ là hàm của lớp k, $R_{k,\mathcal{A}}$ là hàm sau ReLU của k lớp đầu, $R_{0,\mathcal{A}}$ là lớp input.

3.2 Chứng minh Định lý 1 (phác thảo đã kiểm trong Phụ lục A.1)

Ý tưởng 3 bước:

1. Với f khả tích Lebesgue, chọn N đủ lớn để phần "đuôi" $|x_i| \geq N$ có tích phân $|f|$ nhỏ (bỏ qua $< \varepsilon/2$); tách $f = f_1 - f_2$ (phần dương/âm trên khối $E = [-N,N]^n$); phủ vùng dưới đồ thị f_1, f_2 bằng hữu hạn khối $(n+1)$-chiều (Lebesgue covering), quy hàm về **tổng có trọng số của các hàm chỉ báo (indicator) trên khối n-chiều**.
2. Xây một **block** (khối) mạng ReLU width $n+4$, depth $4n+1$, gọi là **Single ReLU Unit (SRU)**, có khả năng: (a) "chặt" (chop) từng chiều của một khối n-chiều để tạo ra hàm chỉ báo xấp xỉ trên khối đó, (b) lưu lại (nhớ) tổng các khối đã xử lý trước, (c) cộng dồn kết quả hiện tại vào bộ nhớ. Trong $n+4$ node mỗi lớp: n node "truyền" tọa độ input không đổi suốt mạng, 2 node lưu phần dương/âm tích lũy, 2 node còn lại dùng để "chặt" dần từng chiều ($k = 1,\ldots,n$) bằng hàm bậc thang trơn hóa qua tham số δ nhỏ (xem công thức $R_{1,n+3,A_k}$, $R_{4,n+3,A_k}$ trong Phụ lục A.1 - đây chính là cách chuyển một "lát cắt" hộp n-chiều thành một dãy ReLU 4 lớp lặp n lần).
3. Ghép (concatenate) các block cho từng khối indicator, dùng 2 node nhớ để cộng dồn; chọn δ đủ nhỏ để sai số tổng các bước cộng lại nhỏ hơn ε (các hằng số $1/2, 1/4, 1/8, 3/4$ được paper phân bổ cụ thể qua các bất đẳng thức (7)-(26) trong Phụ lục A.1).

3.3 Chứng minh Định lý 2 và 3 (lập luận hình học, Phụ lục A.2-A.3)

Ý tưởng mấu chốt khi $d_m \leq n$ (ma trận trọng số lớp đầu là ánh xạ $\mathbb{R}^n \to \mathbb{R}^m$ với $m < n$): ánh xạ tuyến tính này **không đơn (not injective)**, nên tồn tại hướng $x_0 \neq 0$ sao cho mọi điểm dịch theo x_0 cho cùng giá trị lớp 1, do đó hàm toàn mạng **hằng số dọc theo tia** x_0 (Lemma trong A.3). Nếu hàm bị chặn tích phân hữu hạn trên toàn $\mathbb{R}^n$ (Định lý 2) mà lại hằng số không đổi trên một tia vô hạn với giá trị khác 0, tích phân trên tia đó đã là $+\infty$ (mâu thuẫn) - nên hàm phải bằng 0 trên tia đó; lập luận quy nạp theo từng lớp (Lemma 1-3 trong A.2, dùng tính Lipschitz của hàm ReLU-mạng và định lý hội tụ bị chặn - Dominated Convergence Theorem)

mở rộng điều này ra toàn miền, suy ra hàm $\equiv 0$ nếu tích phân hữu hạn - tức **không xấp xỉ được** hàm khác 0 có tích phân hữu hạn khác 0, trừ khi hàm đó cũng gần 0 (tập đo 0 các trường hợp biên).

Định lý 3 (miền bị chặn) dùng lập luận tương tự nhưng đơn giản hơn: nếu $d_m \leq n-1$, tồn tại hướng x_0 cố định (không phụ thuộc input cụ thể, chỉ phụ thuộc tham số mạng) mà hàm hằng số dọc x_0; với f liên tục không hằng theo hướng đó trên $[-1,1]^n$, khoảng cách L1 giữa f và bất kỳ hàm hằng-theo-x_0 có chặn dưới dương ε^* xác định bởi tính liên tục đều (uniform continuity) của f trên miền compact.

3.4 Chứng minh Định lý 4 (lập luận đo độ - measure zero, Phụ lục A.4)

Với $n=1$: chọn $2k^4$ điểm rời rạc cụ thể $x^{(1)},\ldots,x^{(2k^4)}$; định nghĩa tập E_0 các vector giá trị "hợp lệ" (Lemma 4) sao cho **mọi** vector trong E_0 đều có thể được một mạng rộng (width $2k^2$, depth 3) biểu diễn đúng tại các điểm đó - tức $E_0 \subset E_w$ (tập giá trị khả đạt của mạng rộng). Mặt khác (Lemma 5), ánh xạ từ không gian tham số của mạng hẹp (width $\leq k^{1.5}$, depth $\leq k+2$, có **số chiều tự do** $= d^2(h-2)+d(h-1)+2d+1 \leq k^4+k^3 < 2k^4$) sang không gian giá trị $\mathbb{R}^{2k^4}$ có **miền ảnh đo-0** theo định lý biến đổi Jacobian (vì chiều miền xác định nhỏ hơn chiều không gian đích $2k^4$). Do E_0 có đo dương, E_0 giao với miền ảnh đó là tập con "bỏ qua được" (negligible) trong E_0 - nghĩa là hầu hết hàm trong lớp mạng rộng **không thể** khớp đúng bởi mạng hẹp tại các điểm đã chọn. Lemma 6 dùng tính liên tục + compact (dãy con hội tụ, định lý hội tụ bị chặn) để chuyển từ "không khớp đúng tại điểm rời rạc" sang "có khoảng cách dương $\epsilon_j > 0$ đều" cho MỖI hình dạng mạng hẹp cụ thể, từ đó lấy $\epsilon = \min_j \epsilon_j$ trên tập hữu hạn các hình dạng khả dĩ để có hằng số chung. Mở rộng từ $n=1$ sang $n>1$: cho mạng rộng chỉ phụ thuộc x_1 (trọng số từ $x_2,\ldots,x_n$ gán 0), lập luận tương tự áp dụng trực tiếp.

4. Thí nghiệm và kết quả chính

Khác với bài 20 (không có thực nghiệm), bài này **có** một thực nghiệm thật (Mục 4.1), nhằm kiểm tra liệu chặn dưới đa thức của Định lý 4 có "gần" chặn trên thật không.

Thiết lập: với $n \in \{1,2\}$, $k \in \{3,4,5\}$: mạng mục tiêu (target network) là mạng ReLU ngẫu nhiên, input dim n, output dim 1, **width** $2k^2$**, depth 3** (trọng số lấy theo chuẩn tắc standard normal, bias theo uniform $[-1,1)$). Mạng xấp xỉ (approximator network) có **width** $3k^{1.5}$**, depth** $k+2$ (kiến trúc theo đúng tỉ lệ nêu trong Định lý 4), huấn luyện bằng mini-batch **AdaDelta, learning rate 1.0**, 100 epoch (với $n=1$) hoặc 200 epoch ($n=2$), trên 20000 điểm input ($n=1$) hoặc 40000 điểm ($n=2$) lấy đều trong $[-1,1)^n$ (một nửa làm train, đo sai số trên phần còn lại theo mô tả trong Mục 4.1 - "half of all the test inputs... constitute the training set"), lặp lại 50 lần lấy mẫu tham số mạng mục tiêu, báo cáo MSE (mean square error) trường hợp xấu nhất (worst) và trung bình (average).

Bảng 1 (nguyên văn số liệu trong paper):

n	k	width mục tiêu	depth mục tiêu	width xấp xỉ	depth xấp xỉ	MSE xấu nhất	MSE trung bình
1	3	18	3	16	5	0.002248	0.000345
1	4	36	3	24	6	0.003263	0.000892
1	5	50	3	34	7	0.005643	0.001296
2	3	18	3	16	5	0.008729	0.001990
2	4	36	3	24	6	0.018852	0.006251
2	5	50	3	34	7	0.030114	0.007984

Kiểm tra khớp công thức (tự tính): width mục tiêu $2k^2$: $k=3 \to 2\cdot 9 = 18$ ✓, $k=4 \to 2\cdot 16 = 32$... khoan, paper ghi 36 cho $k=4$: $2k^2 = 2\cdot 16 = 32 \neq 36$. Kiểm tra lại: có thể $2k^2$ với $k=4$ nên là 32, nhưng bảng paper in "36" (đã đọc trực tiếp từ bảng, dòng "1 4 36 3 24 6..."). Đây là **điểm paper tự mâu thuẫn giữa**

công thức Định lý 4 ($2k^2$) và số liệu bảng (ghi rõ ở Mục 6 "Giới hạn" dưới đây, không tự sửa). Với $k = 3$: $2k^2 = 18$ khớp; $k = 5$: $2k^2 = 50$ khớp; chỉ riêng dòng $k = 4$ cho 36 là lệch so với $2 \cdot 4^2 = 32$ (có thể lỗi đánh máy trong bảng gốc, hoặc công thức áp dụng khác cho $k = 4$ - không đủ thông tin trong text để kết luận chắc).

Kiểm tra width xấp xỉ $3k^{1.5}$ (tự tính): $k = 3$: $3 \times 3^{1.5} = 3 \times 5.196 = 15.59 \approx 16$ (paper ghi 16, khớp làm tròn lên); $k = 4$: $3 \times 4^{1.5} = 3 \times 8 = 24$ (khớp đúng, paper ghi 24); $k = 5$: $3 \times 5^{1.5} = 3 \times 11.18 = 33.54 \approx 34$ (paper ghi 34, khớp làm tròn lên). Depth xấp xỉ $k + 2$: $3 \to 5$, $4 \to 6$, $5 \to 7$ - khớp đúng cả 3.

Kết luận thực nghiệm (lời paper): sai số MSE rất nhỏ (từ 3×10^{-4} đến 3×10^{-2}) trong khi giá trị hàm mục tiêu dao động với biên độ ~ 10 (paper tự ghi), nên sai số tuyệt đối này được xem là nhỏ một cách hợp lý; kết quả "cho thấy một gợi ý rằng độ sâu lớn hơn theo tỉ lệ đa thức có thể đủ để mạng hẹp xấp xỉ mạng rộng" - đây **chỉ là gợi ý thực nghiệm trên vài cấu hình nhỏ ($n \leq 2$, $k \leq 5$)**, không phải chứng minh chặn trên đa thức (vẫn là open problem theo lời paper).

5. Phân tích / ablation

Paper không có ablation nghiêm ngặt (không thử các tỉ lệ width/depth khác với công thức Định lý 4), nhưng có thể phân tích các điểm sau từ chính nội dung:

- **"Khoảng trống" giữa Định lý 1 và Định lý 2/3**: Định lý 1 chứng minh xấp xỉ được với width $n + 4$; Định lý 2/3 chứng minh **không** xấp xỉ được với width $\leq n$ (toàn miền) hoặc $\leq n - 1$ (miền bị chặn). Vùng width $\in \{n, n+1, n+2, n+3\}$ (toàn miền) hoặc $\{n\}$ (miền bị chặn) **chưa được xác định** trong paper - đây không phải lỗi mà là khoảng chưa giải quyết, paper không claim đã có chặn chặt khít (tight).
- **Đối xứng kỹ thuật nhưng không đối xứng kết luận**: cấu trúc chứng minh Định lý 1 (dùng khối SRU) là bản "lật ngược vai trò" khá trực tiếp của universal approximation cổ điển (ở đó depth=2, width lớn; ở đây width=$n + 4$, depth lớn) - nhưng Định lý 4 (width efficiency) chỉ đạt chặn **đa thức**, không đối xứng với chặn **mũ** của depth efficiency (bài 20). Bản thân paper thừa nhận (Mục 4) điều này "không loại trừ khả năng có chặn mũ cho width efficiency" - nghĩa là kết luận "depth hiệu quả hơn width" của paper dựa trên **so sánh chặn dưới đã biết, không phải chặn dưới chặt nhất có thể**.
- **Vai trò thực nghiệm**: vì chặn trên đa thức không chứng minh được bằng lý thuyết, thực nghiệm Mục 4.1 được dùng như "bằng chứng gián tiếp" (indirect evidence) - đây là điểm khác biệt về chuẩn mực chứng minh giữa phần lý thuyết (chặt, định lý) và phần này (gợi ý, không tổng quát hoá được).
- **Lemma 2 (trong A.2) có phát biểu "ngược đời"**: nếu mạng hẹp ($d_m \leq n$) thoả mãn tích phân toàn miền hữu hạn (Assumption 1) thì hàm mạng đó **phải bằng 0 khắp nơi** - tác giả tự ghi chú "kết luận này có thể lạ lúc đầu vì $F_{\mathcal{A}}$ giống hàm tuyến tính" (trích dẫn gần nguyên văn từ text), nhấn mạnh rằng kết luận chỉ đúng dưới giả thiết tích phân hữu hạn - một cách khác để nói "hàm khác 0 có tích phân hữu hạn thì không thể biểu diễn bởi mạng width $\leq n$".

6. Giới hạn

- **Mâu thuẫn số liệu trong Bảng 1** đã nêu ở Mục 4: dòng $k = 4$ ghi width mục tiêu = 36 nhưng công thức Định lý 4 là $2k^2 = 32$; không rõ là lỗi đánh máy của paper hay cách tính khác, paper không giải thích - ghi nhận ở đây, không tự sửa theo brief.
- Định lý 1 dùng khoảng cách **L1** trên hàm **khả tích Lebesgue toàn không gian** $\mathbb{R}^n$, khác định lý xấp xỉ cổ điển dùng **L∞** trên hàm **liên tục** miền **compact** - hai phát biểu không hoàn toàn so sánh ngang hàng được dù cùng gọi "universal approximation" (paper tự nêu rõ sự khác biệt này ở cuối Mục 3).
- Định lý 4 chỉ xây dựng **một họ cụ thể** (random weight target network cấu hình cố định), không phải đặc trưng hoá mọi hàm khó xấp xỉ bởi mạng hẹp.
- Thực nghiệm chỉ chạy $n \in \{1, 2\}$, $k \in \{3, 4, 5\}$ - quy mô rất nhỏ, không có thực nghiệm trên dữ liệu hoặc kiến trúc thực tế (CNN, Transformer).

- Chặn trên đa thức (narrow network chỉ cần tăng depth đa thức để xấp xỉ wide network) và chặn dưới mũ cho width efficiency đều là **open problem** tự paper nêu (Mục 4, cuối): "Currently, neither the exponential lower bound nor the polynomial upper bound seems within the reach."
- Không bàn tới hàm kích hoạt khác ReLU, không bàn tới kiến trúc không fully-connected (CNN, attention).

7. Liên hệ với các bài khác

- **math 20** (Benefits of Depth in Neural Networks, Telgarsky 2016): nền tảng depth-efficiency mà bài này trích dẫn trực tiếp (reference [14]) để đối chiếu; cùng dùng khái niệm "efficiency" (kích thước cần tăng bao nhiêu khi giảm chiều kia) nhưng đo bằng công cụ khác (bài 20: crossing number/oscillation; bài 21: đo độ Lebesgue trên không gian tham số).
- **math 22 / math 23** (Deep Neural Networks as Gaussian Processes; Neural Tangent Kernel): khái niệm "width" ở hai bài này là **width** $\to \infty$ theo nghĩa thống kê tại khởi tạo/huấn luyện (không phải width hữu hạn nhỏ như $n + 4$ ở đây); hai hướng nghiên cứu (approximation theory của width hữu hạn vs. infinite-width statistics) phát triển khá độc lập trong lý thuyết deep learning, dù cùng dùng từ "width".
- **math 25** (The Principles of Deep Learning Theory): dùng tỉ số $r = L/n$ (depth-to-width) làm biến trung tâm; bài 21 góp phần định tính cho "mẫu số" n của tỉ số đó từ góc độ xấp xỉ hàm hữu hạn-width, trong khi math 25 phân tích $n \to \infty$ với L cố định rồi mở rộng nhiễu loạn theo $1/n$.
- **pruning content/32_2024_unreasonable_ineffectiveness_deeper_layers.md** (đã kiểm tồn tại bằng `ls`): quan sát thực nghiệm rằng LLM hiện đại chịu được xoá khá nhiều layer sâu; liên hệ (suy luận của người ghi chú): nếu width của mỗi layer còn lại trong các LLM thực tế đã rất lớn hơn ngưỡng phase-transition n (số chiều input/hidden dimension) của bài 21, thì mạng còn nhiều "dư thừa" width để hấp thụ việc giảm depth, khớp với quan sát hiện tượng loại bỏ layer ít ảnh hưởng.
- **pruning content/20_2023_depgraph.md** (đã kiểm tồn tại bằng `ls`): framework cắt theo đồ thị phụ thuộc giữa các thành phần mạng (có thể cắt kênh/width một cách có cấu trúc); bài 21 cho cảnh báo lý thuyết (yếu, trên mạng ReLU đơn giản) rằng cắt width xuống dưới ngưỡng n (số chiều input cục bộ của một khối) có nguy cơ gây "sập" khả năng biểu diễn chứ không suy giảm từ từ - một điều nên lưu ý khi tự động hoá quyết định tỉ lệ cắt theo width.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU $\leq$ 8 GB, batch 1). Giống bài 20, đây là paper lý thuyết trên mạng ReLU fully-connected đơn giản, nên liên hệ chỉ mang tính **suy luận định tính**:

- **Cảnh báo "ngưỡng sập" khi cắt width**: Định lý 2/3 cho thấy việc giảm width xuống dưới một ngưỡng (liên quan tới "số chiều thông tin cần giữ qua lớp đó") có thể gây mất khả năng biểu diễn **đột ngột** (không suy giảm từ từ) chứ không như trực giác thường nghĩ "cắt ít thì mất ít". Khi quyết định tỉ lệ cắt width cho các khối GatedDeltaNet hoặc attention của LightOnOCR (ví dụ giảm d_{model} hoặc số kênh trạng thái ẩn), nên thận trọng với việc giảm xuống gần hoặc dưới "chiều thông tin hiệu dụng" cần truyền qua (ví dụ số chiều cần để mã hoá vị trí + nội dung token OCR) hơn là coi việc cắt width là tuyến tính an toàn.
- **Gợi ý thực nghiệm nén**: nếu phải nén cả depth và width, Định lý 1 gợi ý rằng việc "đền bù" width nhỏ bằng depth lớn hơn là khả thi về mặt biểu diễn (dù chỉ chứng minh cho ReLU fully-connected rất khác hybrid attention/SSM của LightOnOCR) - gợi ý một chiến lược nén (suy luận, chưa kiểm chứng trên kiến trúc thực): nếu cắt width mạnh ở một khối, có thể cân nhắc **không cắt tương ứng depth** ở khối đó để giữ khả năng biểu diễn, rồi bù hiệu năng suy luận (latency) bằng cắt depth ở khối khác.
- Mức độ liên quan: **trung bình - thấp** (không có công thức áp dụng trực tiếp cho attention/SSM, nhưng trực giác "ngưỡng sập theo width" và "đền bù depth-width" có giá trị khi thiết kế lưới quét kiến trúc

nén tự động, ví dụ dùng chung với các phương pháp trong pruning content/20 DepGraph hay pruning content/41 Flextron).

9. Thí nghiệm gợi ý

1. **Tái dựng phase transition của Định lý 2/3 (toy, CPU hoặc 1 GPU nhỏ)**: với $n = 3$, huấn luyện mạng ReLU fit một hàm mục tiêu cố định (ví dụ mạng ReLU ngẫu nhiên khác, width 20 depth 3) bằng mạng xấp xỉ có width quét qua $\{1, 2, 3, 4, 5, 6, 7\}$ (quanh ngưỡng $n = 3$), depth cố định lớn (ví dụ 10), đo MSE. Kỳ vọng: MSE giảm mạnh (”rơi vực”) khi width vượt qua $n = 3$ sang $n + 1 = 4$ trở lên, minh hoạ trực quan Định lý 2/3 (lưu ý: đây không phải chứng minh lại định lý, vì định lý nói về sai số **0 hoặc vô hạn trên toàn không gian**, còn thực nghiệm đo trên miền bị chặn hữu hạn điểm).
2. **Tái lập đúng Bảng 1 ở quy mô nhỏ**: dựng target network width $2k^2$ depth 3 cho $k = 3$ (width 18), approximator width $3k^{1.5} \approx 16$ depth $k + 2 = 5$, train bằng Adam hoặc AdaDelta lr nhỏ, so khớp MSE với số liệu paper (0.002248 worst, 0.000345 average cho $n = 1$). Nếu số liệu lệch nhiều, kiểm tra lại cách sinh tham số target (standard normal weight, uniform $[-1, 1)$ bias) và cách chia train/test.
3. **Depth bù width (đền bù cho khối hybrid)**: lấy một khối nhỏ kiểu GatedDeltaNet/attention đơn giản hoá (hoặc thay bằng MLP residual nhỏ làm proxy, vì môi trường toy không cần đúng kiến trúc), cắt width hidden dimension xuống 50%, rồi thử tăng số lớp lặp lại gấp đôi/gấp ba để bù; đo loss trên 1 batch nhỏ dữ liệu tổng hợp (ví dụ chuỗi ký tự ngẫu nhiên). Kỳ vọng: tăng depth bù được một phần nhưng không hoàn toàn (vì kiến trúc hybrid không phải ReLU fully-connected như trong lý thuyết) - thí nghiệm này dùng để **kiểm tra giới hạn áp dụng** của trực giác Mục 8, không phải tái lập paper.
4. **Đo ”số chiều thông tin hiệu dụng” bằng rank**: với một checkpoint LightOnOCR nhỏ (hoặc mô hình proxy cỡ nhỏ), đo hạng (rank, hoặc singular value decay) của ma trận trọng số từng lớp attention/SSM, so sánh với d_{model} hiện tại; nếu hạng hiệu dụng đã nhỏ hơn nhiều so với d_{model}, đó là tín hiệu (suy luận, giống tinh thần ”ngưỡng width” của Định lý 2/3 nhưng đo gián tiếp qua rank thực nghiệm) rằng có thể cắt width xuống gần ngưỡng đó mà ít rủi ro ”sập” biểu diễn.

10. Tóm tắt công thức / khái niệm

Khái niệm/công thức	Ý nghĩa
depth h, width d_m, input dim n	số lớp (không tính input), số node tối đa/lớp, số chiều đầu vào
Định lý 1	$\forall f$ khả tích Lebesgue, $\exists$ mạng ReLU width $\leq n + 4$ xấp xỉ L1 sai $< \varepsilon$ tùy ý
Định lý 2	width $\leq n$: trừ tập đo 0, sai số L1 trên $\mathbb{R}^n$ chỉ là 0 hoặc $+\infty$
Định lý 3	miền $[-1, 1]^n$, width $\leq n - 1$: sai số L1 $\geq \varepsilon^* > 0$ cố định với hàm không hằng theo hướng nào
Định lý 4	$\exists$ mạng rộng (width $2k^2$, depth 3) không xấp xỉ L2 được bởi mạng hẹp (width $\leq k^{1.5}$, depth $\leq k + 2$)
SRU (Single ReLU Unit)	khối ReLU width $n + 4$ depth $4n + 1$ dùng để ”chặt” + ”nhớ” + ”cộng” khi dựng chứng minh Định lý 1
Bảng 1 (tự tính kiểm khớp)	width xấp xỉ $\approx 3k^{1.5}$: $k = 3 \to 16$, $k = 4 \to 24$, $k = 5 \to 34$; depth xấp xỉ $= k + 2$
MSE Bảng 1, $n = 1, k = 3$	worst 0.002248, average 0.000345 (AdaDelta lr=1.0, 100 epoch, 20000 điểm)
Số chiều tự do mạng hẹp	$d^2(h - 2) + d(h - 1) + 2d + 1$ (dùng trong Lemma 5, lập luận đo-0)

11. Kết luận ngắn

Bài báo mở ra hướng ”width efficiency” đối ngẫu với ”depth efficiency” (bài 20): chứng minh mạng ReLU width $n + 4$ (depth tùy ý) là universal approximator theo L1, trong khi width $\leq n$ (toàn miền) hoặc $\leq n - 1$

(miền bị chặn) gần như không bao giờ xấp xỉ tốt - một **phase transition rõ rệt** quanh width = số chiều đầu vào. Về hiệu quả định lượng, chặn dưới cho width efficiency (Định lý 4) chỉ là **đa thức** ($k^{1.5}$ so k^2), yếu hơn hẳn chặn **mũ** của depth efficiency, và thực nghiệm nhỏ (Mục 4.1, với một điểm số liệu không khớp công thức ở $k = 4$ cần lưu ý) gợi ý depth tăng đa thức có thể đủ bù width hẹp. Cả chặn trên đa thức cho width và chặn dưới mũ cho width đều là bài toán mở. Kết luận thực hành của paper: depth và width đều quan trọng và cần tinh chỉnh cùng nhau, không nên chỉ tối ưu một chiều.

Nguồn gốc: https://arxiv.org/abs/1709.02540

Bài 22

Deep Neural Networks as Gaussian Processes (2017)

Tác giả: Jaehoon Lee, Yasaman Bahri (hai tác giả đóng góp ngang nhau), Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, Jascha Sohl-Dickstein (Google Brain, Google AI Residency Program)
Venue / arXiv: ”Published as a conference paper at ICLR 2018”; bản đọc arXiv:1711.00165v3 (3/3/2018)
Nguồn: arXiv:1711.00165
Chủ đề: Lý thuyết mạng nơ-ron vô hạn rộng (infinite-width), tương ứng với Gaussian Process, suy luận Bayes cho deep learning
Vị trí trong lộ trình: math 22/35 trong ”math content/”, cùng cặp lý thuyết NN vô hạn rộng với math 23 (Neural Tangent Kernel)

> Ghi chú này là bản tóm tắt, diễn giải lại theo cách hiểu của người viết sau khi đọc trực tiếp bài báo gốc, không phải bản dịch nguyên văn.

1. Bối cảnh

Neal (1994) đã chỉ ra rằng một mạng nơ-ron 1 lớp ẩn, khi số unit của lớp ẩn tiến ra vô hạn và các trọng số/bias được khởi tạo i.i.d. Gaussian, hội tụ (theo luật) về một Gaussian Process (GP). Kết quả này gần như bị ”ngủ quên” trong giai đoạn deep learning bùng nổ bằng mạng hữu hạn rộng huấn luyện bằng SGD. Bài báo này (Lee, Bahri và đồng nghiệp tại Google Brain) đặt lại câu hỏi: điều gì xảy ra nếu ta mở rộng kết quả của Neal cho mạng nhiều lớp ẩn (deep, không chỉ 1 lớp)?

Bối cảnh cụ thể: xét mạng fully-connected (FC) có L lớp ẩn, độ rộng (số unit) mỗi lớp ẩn là $N_1, \dots, N_L$, input $x \in \mathbb{R}^{d_{in}}$, output $z^L \in \mathbb{R}^{d_{out}}$. Pre-activation và post-activation của lớp l là z_i^l và $x_i^l = \phi(z_i^l)$ với ϕ là hàm phi tuyến. Trọng số W_{ij}^l i.i.d. zero-mean, variance σ_w^2/N_l; bias b_i^l i.i.d. zero-mean, variance σ_b^2. Câu hỏi: khi $N_1, \dots, N_{L-1} \to \infty$ (lấy giới hạn tuần tự, lớp này rồi lớp kia), các output function $z_i^L(\cdot)$ có còn là GP không, và nếu có, kernel (hàm hiệp phương sai) của GP đó là gì?

Động lực thực tế: nếu trả lời được câu hỏi trên, ta có thể làm suy luận Bayes ”đúng” (closed-form, không cần SGD) cho một họ mạng deep rất rộng, và so sánh trực tiếp hiệu năng của suy luận Bayes này với mạng hữu hạn huấn luyện thông thường.

2. Ý tưởng chính

Ý tưởng cốt lõi gồm 3 phần:

1. **Tương ứng NN vô hạn rộng vs. GP (NNGP):** bằng cách áp dụng Định lý giới hạn trung tâm (CLT) lặp lại qua từng lớp (lấy $N_1 \to \infty$, rồi $N_2 \to \infty$, ...), output của mạng deep FC hội tụ về một GP có kernel $K^L(x, x')$ được tính **đệ quy** từ lớp này sang lớp kia, chỉ phụ thuộc vào $\sigma_w^2, \sigma_b^2, \phi$ và kernel của lớp trước.
2. **Suy luận Bayes đóng (closed-form):** một khi đã có kernel K^L, bài toán ”huấn luyện” mạng vô hạn rộng này trở thành bài toán GP regression chuẩn: mean và covariance dự đoán tại điểm test có công thức đại số tuyến tính rõ ràng, không cần gradient descent.
3. **Thực nghiệm:** so sánh NNGP (suy luận Bayes đúng) với mạng FC hữu hạn huấn luyện bằng Adam trên MNIST/CIFAR-10, và liên hệ hiệu năng NNGP với lý thuyết signal propagation (ranh giới ”critical line” giữa các pha ordered/chaotic hoặc bounded/unbounded).

Điểm thú vị nhất: NNGP không học biểu diễn (representation learning) theo cách người ta thường gán công cho sức mạnh của deep learning, nó dùng một kernel cố định (fixed basis function). Vậy mà nó thường **không hề thua**, thậm chí vượt mạng SGD-trained ở nhiều kích thước tập huấn luyện.

3. Phương pháp chi tiết

3.1 Trường hợp 1 lớp ẩn (ôn lại, mục 2.2 của bài)

Với $L = 1$: $z_i^1(x) = b_i^1 + \sum_j W_{ij}^1 x_j^1(x)$. Vì các số hạng là i.i.d. và tổng trên $N_1 \to \infty$ số hạng, theo CLT $z_i^1 \sim GP(0, K^1)$ với

$$K^1(x, x') \equiv \mathbb{E}[z_i^1(x) z_i^1(x')] = \sigma_b^2 + \sigma_w^2 \, \mathbb{E}[x_i^1(x) x_i^1(x')] \equiv \sigma_b^2 + \sigma_w^2 C(x, x') \quad (2)$$

3.2 Trường hợp deep (mục 2.3), đệ quy qua các lớp

Lấy giới hạn tuần tự $N_1 \to \infty$, rồi $N_2 \to \infty$,... Giả sử z_j^{l-1} đã là GP (giống và độc lập theo j). Sau $l-1$ bước, $z_i^l(x) = b_i^l + \sum_j W_{ij}^l x_j^l(x)$ với $x_j^l(x) = \phi(z_j^{l-1}(x))$, và lại là tổng i.i.d. nên $z_i^l \sim GP(0, K^l)$ với

$$K^l(x, x') \equiv \mathbb{E}[z_i^l(x) z_i^l(x')] = \sigma_b^2 + \sigma_w^2 \, \mathbb{E}_{z^{l-1} \sim GP(0, K^{l-1})} \left[\phi(z_i^{l-1}(x)) \phi(z_i^{l-1}(x')) \right] \quad (4)$$

Kỳ vọng trong (4) chỉ phụ thuộc vào 3 đại lượng $K^{l-1}(x, x'), K^{l-1}(x, x), K^{l-1}(x', x')$, nên bài báo viết gọn thành một hàm đệ quy F_ϕ:

$$K^l(x, x') = \sigma_b^2 + \sigma_w^2 \, F_\phi\big(K^{l-1}(x, x'),\, K^{l-1}(x, x),\, K^{l-1}(x', x')\big) \quad (5)$$

Trường hợp gốc (base case), với $W_{ij}^0 \sim N(0, \sigma_w^2/d_{in})$, $b_j^0 \sim N(0, \sigma_b^2)$:

$$K^0(x, x') = \sigma_b^2 + \sigma_w^2 \, \frac{x \cdot x'}{d_{in}} \quad (6)$$

Bài báo nhấn mạnh: công thức đệ quy (5)-(6) **trùng** với công thức mean-field signal propagation của Poole et al. (2016) và Schoenholz et al. (2017) (math 26, math 27), và cũng xuất hiện trong văn liệu "compositional kernel" (Cho & Saul 2009; Daniely et al. 2016). Với ReLU, (5) có dạng đóng, đó là "arccosine kernel" (Cho & Saul 2009):

$$K^l(x, x') = \sigma_b^2 + \frac{\sigma_w^2}{2\pi} \sqrt{K^{l-1}(x, x) K^{l-1}(x', x')} \left[\sin\theta_{x,x'}^{l-1} + (\pi - \theta_{x,x'}^{l-1}) \cos\theta_{x,x'}^{l-1} \right], \quad \theta_{x,x'}^l = \cos^{-1}\left(\frac{K^l(x, x')}{\sqrt{K^l(x, x) K^l(x', x')}} \right)$$

Ví dụ tự tính (dùng đúng công thức (6) và (11)): lấy $d_{in} = 2$, $\sigma_w^2 = 1, \sigma_b^2 = 0$, hai input trực giao $x_1 = (1, 0)$, $x_2 = (0, 1)$, ReLU.

- Lớp 0 (công thức 6): $K^0(x_1, x_1) = \frac{1 \cdot 1 + 0 \cdot 0}{2} = 0.5$; tương tự $K^0(x_2, x_2) = 0.5$; $K^0(x_1, x_2) = \frac{1 \cdot 0 + 0 \cdot 1}{2} = 0$.
- Góc ở lớp 0: $\theta_{x_1, x_2}^0 = \cos^{-1}(0/\sqrt{0.5 \cdot 0.5}) = \cos^{-1}(0) = \pi/2$. Vậy $\sin\theta^0 = 1, \cos\theta^0 = 0$, ngoặc vuông $= 1 + (\pi - \pi/2) \cdot 0 = 1$.
- Lớp 1, cặp khác nhau (công thức 11): $K^1(x_1, x_2) = 0 + \frac{1}{2\pi}\sqrt{0.5 \cdot 0.5} \cdot 1 = \frac{0.5}{2\pi} = \frac{1}{4\pi} \approx 0.0796$.
- Lớp 1, cùng điểm: $\theta_{x_1, x_1}^0 = \cos^{-1}(1) = 0 \Rightarrow \sin 0 = 0, \cos 0 = 1$, ngoặc vuông $= 0 + \pi \cdot 1 = \pi$. $K^1(x_1, x_1) = 0 + \frac{1}{2\pi} \cdot 0.5 \cdot \pi = 0.25$ (tương tự $K^1(x_2, x_2) = 0.25$).
- Hệ số tương quan sau 1 lớp: $K^1(x_1, x_2)/\sqrt{K^1(x_1, x_1) K^1(x_2, x_2)} = 0.0796/0.25 \approx 0.318$.

Nhận xét từ phép tự tính này: hai input trực giao ở lớp 0 (tương quan 0) sau một bước ReLU-NNGP đã có tương quan dương (≈ 0.318), minh họa trực quan cho hiện tượng "trộn tương quan theo chiều sâu" mà mục 3.2/Phân tích bên dưới sẽ khai triển ở quy mô lớn (critical line).

3.3 Suy luận Bayes với GP prior (mục 2.4)

Với dữ liệu $D = \{(x_1, t_1), ..., (x_n, t_n)\}$ và nhiễu quan sát Gaussian variance σ_ϵ^2, phân phối posterior dự đoán tại điểm test x^* là Gaussian $z^*|D, x^* \sim N(\bar{\mu}, \bar{K})$ với

$$\bar{\mu} = K_{x^*,D}(K_{D,D} + \sigma_\epsilon^2 I_n)^{-1} t \quad (8)$$

$$\bar{K} = K_{x^*,x^*} - K_{x^*,D}(K_{D,D} + \sigma_\epsilon^2 I_n)^{-1} K_{x^*,D}^T \quad (9)$$

Đây là công thức GP regression chuẩn (Rasmussen & Williams 2006), không cần lan truyền ngược, chỉ cần dựng được K^L.

3.4 Thuật toán tính kernel hiệu quả (mục 2.5)

Với nonlinearity không có dạng đóng, tích phân Gauss trong (4) phải tính số. Cách ”ngây thơ” (tính riêng từng cặp điểm, từng lớp) có chi phí $O(n_g^2 L(n_{train}^2 + n_{train} n_{test}))$. Bài báo giảm chi phí bằng một bảng tra (lookup table) dựng trước trên lưới cố định của pre-activation u, variance s và hệ số tương quan c:

$$F_{ij} = \frac{\sum_{ab} \phi(u_a)\phi(u_b) \exp\big(-\frac{1}{2}(u_a, u_b)\,\Sigma^{-1}\,(u_a, u_b)^T\big)}{\sum_{ab} \exp\big(-\frac{1}{2}(u_a, u_b)\,\Sigma^{-1}\,(u_a, u_b)^T\big)}, \quad \Sigma = \begin{pmatrix} s_i & s_i c_j \\ s_i c_j & s_i \end{pmatrix} \quad (10)$$

sau đó tính $K^l(x, x')$ cho mọi cặp điểm bằng nội suy song tuyến tính (bilinear interpolation) vào bảng này, chi phí còn lại chỉ là $O(n_g^2 n_v n_c + L(n_{train}^2 + n_{train} n_{test}))$. Cài đặt mã nguồn mở: github.com/brain-research/nngp.

4. Thí nghiệm và kết quả chính

Thiết lập: MNIST và CIFAR-10 dạng ”permutation invariant” (làm phẳng ảnh, bỏ cấu trúc không gian), mạng baseline FC cùng độ rộng ở mọi lớp ẩn, loss MSE (nhãn lớp mã hóa one-hot dạng regression target: -0.1 cho lớp sai, 0.9 cho lớp đúng), optimizer Adam, siêu tham số (learning rate, weight decay, σ_w, σ_b, batch size) dò bằng Google Vizier, trung bình khoảng 250 trial cho mỗi cấu hình (kích thước tập train, depth, width, nonlinearity). Không dùng dropout. Chia dữ liệu: MNIST 50k/10k/10k (train/val/test), CIFAR-10 45k/5k/10k. Với GP, quét lưới 30×30 điểm cho $(\sigma_w^2 \in [0.1, 5.0],\ \sigma_b^2 \in [0, 2.0])$, tức 900 lần đánh giá, chọn cấu hình tốt nhất trên validation.

Bảng 1 (số liệu nguyên văn, test accuracy):

Tập train	NN (ReLU)	Acc	GP (ReLU)	Acc	NN (Tanh)	Acc	GP (Tanh)	Acc
MNIST: 1k	NN-2-5000-3.19-0.00	0.9252	GP-20-1.45-0.28	0.9279	NN-2-1000-0.60-0.00	0.9254	GP-20-1.96-0.62	0.9266
MNIST: 10k	NN-2-2000-0.42-0.16	0.9771	GP-7-0.61-0.07	0.9765	NN-2-2000-2.41-1.84	0.9745	GP-2-1.62-0.28	0.9773
MNIST: 50k	NN-2-2000-0.60-0.44	0.9864	GP-1-0.10-0.48	0.9875	NN-2-5000-0.28-0.34	0.9857	GP-1-1.28-0.00	0.9879
CIFAR: 1k	NN-5-500-1.29-0.28	0.3225	GP-7-1.28-0.00	0.3608	NN-1-200-1.45-0.12	0.3378	GP-50-2.97-0.97	0.3702

Tập train	NN (ReLU)	Acc	GP (ReLU)	Acc	NN (Tanh)	Acc	GP (Tanh)	Acc
CIFAR:10k	NN-5-2000-1.60-1.07	0.4545	GP-5-2.97-0.28	0.4780	NN-1-500-1.48-1.59	0.4429	GP-7-3.48-2.00	0.4766
CIFAR:45k	NN-3-5000-0.53-0.01	0.5313	GP-3-3.31-1.86	0.5566	NN-2-2000-1.05-2.08	0.5034	GP-3-3.48-1.52	0.5558

(Ký hiệu model: NN là "depth-width-σ_w^2-σ_b^2", GP là "depth-σ_w^2-σ_b^2". Bảng 2 ở Appendix là bản mở rộng của Bảng 1 với nhiều kích thước tập train hơn (100 đến 50k cho MNIST, 100 đến 45k cho CIFAR), theo đúng mẫu hình đã thấy ở Bảng 1.)

Hình 1: accuracy/MSE của NNGP tốt nhất so với NN tốt nhất theo từng width, vẽ theo kích thước tập train; NN càng rộng thì hiệu năng càng tiến gần NNGP.

Hình 2: generalization gap của 180 mạng FC 5 lớp ẩn trên CIFAR-10 (random hyperparameter), lọc lấy các mạng đạt 100% train accuracy (125 mạng Tanh, 55 mạng ReLU) theo bề rộng (width); mạng rộng nhất luôn generalize tốt nhất.

Hình 3 (uncertainty): vì NNGP là Bayes, mỗi điểm test có một ước lượng phương sai dự đoán (công thức 9); phương sai này tương quan cao với MSE thực tế. Số liệu tương quan nguyên văn: MNIST-50k, Tanh-corr: 0.9330, ReLU-corr: 0.9573; CIFAR-45k, Tanh-corr: 0.7428, ReLU-corr: 0.8223 (hyperparameter ví dụ: depth=3, $\sigma_w^2 = 2.0$, $\sigma_b^2 = 0.2$).

Thời gian tính toán (Appendix D): dựng lưới (grid generation, $n_g = n_v = 501, n_c = 500$) tốn 440-460s trên 6 CPU; dựng $K_{D,D}$ đầy đủ cho MNIST 50k tốn 90-140s trên 64 CPU; giải Cholesky cho 1000 điểm test tốn 180-220s; noise mặc định $\sigma_\epsilon^2 = 10^{-10}$, tăng gấp 10 khi Cholesky thất bại.

5. Phân tích / ablation

Ba quan sát phân tích quan trọng rút ra trực tiếp từ bài:

1. **NNGP thường thắng NN nhưng không phải luôn luôn.** Nhìn kỹ Bảng 1: GP thắng ở 11/12 cặp so sánh, nhưng ở MNIST:10k với ReLU, NN (0.9771) nhích hơn GP (0.9765) một chút. Đây là ngoại lệ thật trong số liệu của chính bài báo, không phải lỗi đọc, cho thấy ưu thế của NNGP là một xu hướng mạnh ở dữ liệu nhỏ/kernel method phù hợp, không phải định lý tuyệt đối cho mọi (kích thước tập, nonlinearity).
2. **Mức tương quan uncertainty-error phụ thuộc độ khó bài toán.** Tương quan cao hơn ở MNIST (0.93-0.96) so với CIFAR-10 (0.74-0.82). Điều này gợi ý: GP uncertainty là tín hiệu định lượng tốt khi bài toán "gần tuyến tính hoá được" bởi kernel cố định, còn với bài toán khó hơn (CIFAR), phần sai số không giải thích được bởi kernel cố định tăng lên.
3. **Hiệu năng theo cấu trúc "critical line" (mục 3.2).** Bài báo liên hệ trực tiếp đến lý thuyết signal propagation (Poole et al. 2016; Schoenholz et al. 2017): với Tanh có hai pha "ordered" ($K^l(x, x') \to q^*$ với mọi cặp, mọi input "quên" sự khác biệt) và "chaotic" ($K^l(x, x) \to q^*$ nhưng $K^l(x, x') \to q^* c^*$ với $c^* < 1$ khi $x \neq x'$); với ReLU có hai pha "bounded" (q^* hữu hạn, khác 0) và "unbounded" (q^* tiến vô cùng hoặc về 0). Trên đường biên giữa hai pha (critical line), tốc độ hội tụ về điểm cố định là phi mũ (chậm hơn hẳn). Heatmap accuracy của NNGP theo lưới (σ_w^2, σ_b^2) (Hình 4, và Hình 9 ở Appendix cho nhiều depth khác nhau) bám rất sát critical line này; xa critical line, độ chính xác rơi về mức đoán ngẫu nhiên, và hiện tượng này xảy ra nhanh hơn khi depth L tăng, vì lượng thông tin về dữ liệu chỉ còn nằm ở phần sai khác $K^L(x, x') - K^\infty(x, x')$, phần này suy giảm theo cấp số nhân với depth và cuối cùng bị nuốt bởi lỗi số (numerical precision). Bài báo còn chỉ ra (Hình 10, Appendix E) rằng hyperparameter

tốt nhất của NNGP tập trung quanh critical line một cách nhất quán qua nhiều kích thước tập train khác nhau, cho thấy đây không phải trùng hợp ngẫu nhiên của một thí nghiệm đơn lẻ.

6. Giới hạn

- Chỉ xét mạng fully-connected, dữ liệu được dùng ở dạng ”permutation invariant” (làm phẳng ảnh), bỏ hoàn toàn cấu trúc không gian của ảnh; bài không thử nghiệm kiến trúc convolutional.
- NNGP mô tả **phân phối tại khởi tạo và suy luận Bayes đúng**, không mô tả quá trình SGD thực tế huấn luyện một mạng hữu hạn. Việc SGD có ”xấp xỉ” suy luận Bayes hay không vẫn là câu hỏi mở mà bài chỉ nêu ra (trích dẫn Mandt et al. 2017, tức math 11 trong lộ trình) chứ không trả lời, đây chính là khe hở mà math 23 (NTK) lấp vào theo một hướng khác (mô tả động lực gradient descent bằng kernel, thay vì chỉ mô tả trạng thái tại khởi tạo).
- Suy luận GP đúng cần giải hệ phương trình tuyến tính cỡ $n_{train} \times n_{train}$ (Cholesky), chi phí $O(n^3)$ theo kích thước tập train; bài chỉ dừng ở quy mô MNIST/CIFAR (tối đa 50k/45k điểm) và nêu các phương pháp GP có thể mở rộng quy mô (Quinonero-Candela & Rasmussen 2005; Hensman et al. 2013) là hướng tương lai, chưa triển khai trong bài này.
- Chỉ dùng loss MSE (hồi quy one-hot) để có thể so sánh trực tiếp với công thức GP; chưa đánh giá với cross-entropy loss (bài nêu rõ đây là việc để lại cho công việc tương lai). Không dùng dropout.
- Đã kiểm tra văn bản của bản v3 (3/3/2018, ”Published as a conference paper at ICLR 2018”) được đọc cho ghi chú này: không thấy ghi chú erratum/correction nào trong nội dung đã đọc.

7. Liên hệ với các bài khác

- **math 23 (Jacot, Gabriel, Hongler, ”Neural Tangent Kernel”):** đây là bài nối tiếp trực tiếp và hay bị nhầm lẫn với bài hiện tại. NNGP mô tả kernel **tại khởi tạo** ($\Sigma^{(L)}$ trong ký hiệu của math 23, tương đương K^L ở đây) và dùng nó để làm suy luận Bayes tĩnh (công thức 8-9), không nói gì về việc gradient descent sẽ đi thế nào. NTK mô tả một kernel khác (ký hiệu $\Theta^{(L)}$), xuất hiện khi lấy đạo hàm của hàm mạng theo tham số, và chứng minh rằng **toàn bộ quá trình gradient descent** của mạng vô hạn rộng tương đương kernel gradient descent theo $\Theta_\infty^{(L)}$ cố định. Tóm lại: NNGP = kernel-tại-khởi-tạo cho suy luận Bayes; NTK = kernel-trong-suốt-huấn-luyện cho gradient descent. Hai kernel này khác nhau về bản chất toán học (một là hiệp phương sai của hàm, một là tích các đạo hàm riêng), tuy công thức đệ quy của $\Sigma^{(L)}$ trong NTK trùng với K^L ở đây.
- **math 24 (”On Lazy Training in Differentiable Programming”):** mở rộng ý tưởng của NTK, chỉ ra ”chế độ lazy” (lazy regime, gần tuyến tính hoá quanh khởi tạo) không chỉ xảy ra vì mạng vô hạn rộng mà còn vì cách tham số hoá/scale của mô hình; liên hệ trực tiếp đến NTK (math 23) hơn là NNGP, nhưng cùng một họ ý tưởng ”mạng rất rộng hành xử như một đối tượng tuyến tính hoá/kernel”.
- **math 26 (Poole et al., ”Exponential expressivity in deep neural networks through transient chaos”) và math 27 (Schoenholz et al., ”Deep Information Propagation”):** đây chính là nguồn gốc của công thức đệ quy (5)-(6) và khái niệm ”critical line”/pha ordered-chaotic mà mục 3.2 của bài hiện tại dùng để giải thích tại sao NNGP hoạt động tốt gần critical line. Về mặt công thức, $K^l(x, x')$ ở đây và q^{ab} trong lý thuyết signal propagation của hai bài đó là một.
- (Tham khảo thêm, không bắt buộc đọc): math 25 (”The Principles of Deep Learning Theory”) là một tài liệu tổng hợp muộn hơn, mở rộng NNGP sang các hiệu chỉnh độ rộng hữu hạn (finite-width corrections), nằm ngoài phạm vi của bài 1711.00165 nên không được khai triển ở đây.

8. Ứng dụng cho dự án (suy luận)

(Toàn bộ mục này là suy luận/inference của người viết ghi chú, áp dụng ý tưởng của bài vào bối cảnh dự án, không phải nội dung trong bài báo.)

Bối cảnh dự án: post-training/nén LightOnOCR-3-0.8B, một VLM OCR dùng kiến trúc lai GatedDeltaNet +

full attention, khoảng 3.600 token/trang (~1.500 token văn bản + 1.6-2.1K token thị giác), chạy trên GPU ≤ 8GB, batch size 1.

Hạn chế cần nói rõ trước: NNGP chỉ đúng khi độ rộng tiến ra **vô hạn**; LightOnOCR-3-0.8B có các lớp ẩn với độ rộng hữu hạn, cụ thể (thường ở tầm hàng nghìn chiều ẩn cho một mô hình cỡ 0.8B tham số), nên áp tương đương NNGP (coi mạng là một GP chính xác) lên mô hình này là **không hợp lệ về mặt toán học**, chỉ có thể dùng như một phép ẩn dụ định tính, không phải một công cụ định lượng.

Hai hướng suy luận định tính (không phải áp dụng trực tiếp công thức GP):

1. **Khởi tạo gần "critical line" cho các thành phần mới thêm.** Khi distillation/phục hồi sau pruning cần khởi tạo lại một số lớp (ví dụ adapter, lớp routing mới chèn vào, hoặc lớp thay thế sau khi cắt bớt attention head), bài học định tính từ mục 3.2 (hiệu năng NNGP sụp đổ xa critical line, tốt nhất gần critical line) gợi ý nên chọn scale khởi tạo (σ_w^2, σ_b^2 hiệu dụng) gần vùng mà tín hiệu không bùng nổ cũng không tắt theo chiều sâu, tương tự tinh thần của math 26/math 27, thay vì khởi tạo ngẫu nhiên "theo quán tính" (ví dụ copy std của lớp gốc mà không kiểm tra lại sau khi đã thay đổi kiến trúc/độ rộng lớp).
2. **Mượn ý tưởng "uncertainty tương quan với error" (Hình 3) để làm proxy rẻ, không phải GP thật.** Vì không thể dựng GP kernel chính xác cho mạng 0.8B, có thể mượn tinh thần: tìm một đại lượng rẻ, tính được bằng forward-pass (ví dụ entropy của phân phối token OCR đầu ra, hoặc độ lệch giữa vài lần forward có dropout/nhiễu nhỏ) và kiểm tra xem đại lượng đó có tương quan với lỗi OCR thực tế (qua một tập nhỏ có nhãn) không, giống cách bài báo kiểm tra tương quan giữa phương sai GP và MSE thực tế. Đây chỉ là việc mượn ý tưởng kiểm định tương quan, không mượn công thức (8)-(9).

Tóm lại, giá trị thực tế của bài này cho dự án không nằm ở việc áp trực tiếp công thức GP (vì model quá hẹp), mà ở hai bài học định tính: (a) khởi tạo nên nhắm gần vùng ổn định tín hiệu theo chiều sâu, và (b) nguyên lý "đo được độ không chắc chắn rồi kiểm tra tương quan với lỗi thật" là một thói quen kiểm định hữu ích có thể tái dùng dù không có GP.

9. Thí nghiệm gợi ý

1. **Tái lập mini-Table 1:** lấy một tập con nhỏ của MNIST (ví dụ 1.000-5.000 ảnh), dựng kernel NNGP bằng công thức arccosine ReLU (11) tính đệ quy qua vài lớp, giải GP regression bằng (8)-(9) (dùng numpy/Cholesky), so sánh test accuracy với một mạng FC nhỏ huấn luyện bằng Adam trên cùng tập con. Chạy hoàn toàn trên CPU hoặc GPU 8GB, không cần huấn luyện lớn.
2. **Tái lập Hình 3 ở quy mô nhỏ:** với cùng kernel ở thí nghiệm 1, tính phương sai dự đoán GP (công thức 9) cho một batch điểm test, đối chiếu với sai số bình phương thực tế, tính hệ số tương quan, kiểm tra xem có nằm trong khoảng tương tự báo cáo của bài (0.7-0.97) không.
3. **Mini phase-diagram:** huấn luyện vài chục mạng FC nhỏ (2-4 lớp ẩn, vài trăm unit) trên một tập con MNIST với lưới thô (σ_w^2, σ_b^2) cho khởi tạo, đo accuracy sau một số bước cố định, vẽ heatmap và xem có xuất hiện dải "tốt" giống critical line của Hình 4/9 không. Rất rẻ, chạy được trên một GPU 8GB với batch nhỏ.
4. **Chẩn đoán tín hiệu theo chiều sâu cho encoder của LightOnOCR (chỉ forward, không train):** với mô hình đã có sẵn, đo phương sai activation trung bình theo từng lớp (text tower và/hoặc vision tower) trên một batch ảnh thật, xem phương sai có ổn định theo chiều sâu hay bùng nổ/tắt dần, lấy cảm hứng trực tiếp từ liên hệ signal-propagation ở mục 3.2. Chỉ cần vài forward pass, chi phí rất thấp, batch 1 vẫn chạy được.
5. **Kiểm tra tính nửa xác định dương của kernel tự tính:** mở rộng phép tính tay ở mục 3 (hai điểm x_1, x_2 trực giao) ra 4-5 điểm ngẫu nhiên trong $\mathbb{R}^2$, tính ma trận K^0 rồi K^1 bằng công thức (6)/(11) bằng code ngắn, kiểm tra eigenvalue không âm, một bài tập kiểm chứng công thức rất nhẹ, không cần GPU.

10. Tóm tắt công thức / khái niệm

- **NNGP:** mạng FC vô hạn rộng với trọng số/bias Gaussian hội tụ về $GP(0, K^L)$ khi lấy giới hạn tuần tự theo từng lớp.
- Công thức (2): kernel lớp 1, $K^1 = \sigma_b^2 + \sigma_w^2 C(x, x')$.
- Công thức (4)-(5): đệ quy kernel lớp l qua kỳ vọng Gaussian của ϕ ở lớp $l-1$, viết gọn bằng hàm F_ϕ.
- Công thức (6): kernel gốc (lớp 0), tuyến tính theo $x \cdot x'/d_{in}$.
- Công thức (7)-(9): suy luận Bayes GP chuẩn, mean/covariance posterior dạng đóng.
- Công thức (10): lookup table tính F_ϕ số học, dùng nội suy song tuyến tính để giảm độ phức tạp.
- Công thức (11): arccosine kernel, dạng đóng của đệ quy (5) cho ReLU, có ví dụ tự tính ở mục 3.
- **Critical line / pha ordered-chaotic (Tanh), bounded-unbounded (ReLU):** ranh giới hyperparameter (σ_w^2, σ_b^2) nơi tốc độ hội tụ của K^l về điểm cố định là phi mũ (chậm), tương ứng vùng NNGP hoạt động tốt nhất.
- **Permutation invariant MNIST/CIFAR-10:** cách đóng khung ảnh thành vector phẳng để dùng được với mạng FC, bỏ cấu trúc không gian.

11. Kết luận ngắn

Bài báo chứng minh (và kiểm chứng thực nghiệm) rằng mạng FC deep vô hạn rộng tương ứng chính xác với một Gaussian Process, với kernel tính đệ quy qua từng lớp; suy luận Bayes dùng kernel này (NNGP) thường đạt hiệu năng ngang hoặc vượt mạng hữu hạn huấn luyện bằng SGD trên MNIST/CIFAR-10, và độ không chắc chắn của GP tương quan tốt với sai số thực tế. Hiệu năng NNGP còn được giải thích bằng lý thuyết signal propagation (critical line). Đây là nền tảng lý thuyết đầu tiên của một nhánh ”lý thuyết NN vô hạn rộng” mà math 23 (NTK) tiếp nối theo hướng mô tả động lực huấn luyện, và math 24, math 25 mở rộng thêm.

Nguồn gốc: https://arxiv.org/abs/1711.00165

Bài 23

Neural Tangent Kernel: Convergence and Generalization in Neural Networks (2018)

Tác giả: Arthur Jacot, Franck Gabriel, Clément Hongler (École Polytechnique Fédérale de Lausanne; Franck Gabriel đồng thời thuộc Imperial College London)
Venue / arXiv: ”32nd Conference on Neural Information Processing Systems (NIPS 2018), Montréal, Canada.”; bản đọc arXiv:1806.07572v4 (10/2/2020, bản chỉnh sửa sau hội nghị NIPS 2018)
Nguồn: arXiv:1806.07572
Chủ đề: Neural Tangent Kernel (NTK), động lực huấn luyện mạng nơ-ron vô hạn rộng bằng gradient descent, kernel gradient descent
Vị trí trong lộ trình: math 23/35 trong ”math content/”, cùng cặp lý thuyết NN vô hạn rộng với math 22 (Deep Neural Networks as Gaussian Processes)

> Ghi chú này là bản tóm tắt, diễn giải lại theo cách hiểu của người viết sau khi đọc trực tiếp bài báo gốc, không phải bản dịch nguyên văn.

1. Bối cảnh

Bài toán tối ưu của ANN không lồi (có nhiều saddle point có thể làm chậm hội tụ, theo Dauphin et al. và Choromanska et al., được bài trích dẫn), nhưng thực tế ANN vẫn huấn luyện tốt và generalize tốt dù over-parametrized, điều nghe có vẻ nghịch lý: mạng đủ lớn có thể khớp cả nhãn ngẫu nhiên nhưng vẫn đạt test accuracy cao khi huấn luyện trên dữ liệu thật (bài trích dẫn Zhang et al. 2017, ”Understanding deep learning requires rethinking generalization”). Bài báo cũng ghi nhận rằng kernel method có tính chất tương tự.

Nền tảng đã biết trước bài này (trích dẫn [16; 4; 7; 13; 6], trong đó [13] chính là Lee, Bahri, Novak, Schoenholz, Pennington, Sohl-Dickstein, ”Deep neural networks as gaussian processes”, ICLR 2018, tức math 22 trong lộ trình này): tại khởi tạo, ANN vô hạn rộng tương đương một Gaussian Process. Câu hỏi còn bỏ ngỏ mà bài này đặt ra: **trong khi huấn luyện** (không chỉ tại khởi tạo), hành vi của ANN vô hạn rộng có thể mô tả được bằng một đối tượng tương tự kernel không? Tác giả gọi đối tượng đó là Neural Tangent Kernel (NTK).

Thiết lập: ANN fully-connected, lớp đánh số từ 0 (input) đến L (output), mỗi lớp có $n_0, \dots, n_L$ neuron, nonlinearity $\sigma : \mathbb{R} \to \mathbb{R}$ Lipschitz, khả vi hai lần với đạo hàm bậc hai bị chặn (giả thiết làm đơn giản chứng minh, bài ghi rõ có thể không cần thiết nghiêm ngặt nhưng không chứng minh điều đó). Hàm mạng $F^{(L)} : \mathbb{R}^P \to \mathcal{F}$ biến tham số θ thành hàm f_θ, với $P = \sum_{l=0}^{L-1}(n_l + 1)n_{l+1}$ tham số gồm ma trận $W^{(l)} \in \mathbb{R}^{n_l \times n_{l+1}}$ và bias $b^{(l)} \in \mathbb{R}^{n_{l+1}}$, khởi tạo i.i.d. $N(0, 1)$.

2. Ý tưởng chính

Ba khối ý tưởng chính:

1. **Kernel gradient descent:** huấn luyện f_θ bằng gradient descent trên θ tương đương (trong hàm không gian $\mathcal{F}$) với kernel gradient descent theo một kernel nào đó, ngay cả khi hàm cost hợp thành $C \circ F^{(L)}$ không lồi theo θ, trong khi cost C theo $f \in \mathcal{F}$ có thể lồi. Kernel ở đây chính là NTK $\Theta^{(L)}(\theta) = \sum_p \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta)$, ngẫu nhiên và biến đổi theo $\theta(t)$ trong trường hợp chung (vì $F^{(L)}$ không tuyến tính theo θ).
2. **Giới hạn vô hạn rộng:** khi $n_1, \dots, n_{L-1} \to \infty$ (tuần tự), NTK hội tụ về một kernel giới hạn **xác định (deterministic)** $\Theta_\infty^{(L)}$ tại khởi tạo, và quan trọng hơn, **giữ nguyên không đổi suốt quá trình huấn luyện**. Đây là khác biệt căn bản so với math 22 (NNGP): NNGP chỉ nói về phân phối của f_θ **tại khởi tạo**; bài này nói về **toàn bộ động lực huấn luyện**.

3. **Hệ quả cho least-squares regression:** khi cố định kernel $\Theta_\infty^{(L)}$, động lực của $f_\theta(t)$ trong giới hạn vô hạn rộng trở thành một phương trình vi phân **tuyến tính**, giải được bằng khai triển theo các thành phần chính (kernel PCA) của $\Theta_\infty^{(L)}$; tốc độ hội tụ theo mỗi thành phần tỉ lệ với eigenvalue tương ứng, điều này cho một lý do lý thuyết cho early stopping.

3. Phương pháp chi tiết

3.1 Tham số hoá NTK (khác tham số hoá cổ điển, Remark 1)

Hàm mạng được định nghĩa theo chuẩn tham số hoá "NTK parametrization":

$$\tilde{\alpha}^{(0)}(x;\theta) = x, \qquad \tilde{\alpha}^{(l+1)}(x;\theta) = \frac{1}{\sqrt{n_l}} W^{(l)} \alpha^{(l)}(x;\theta) + \beta b^{(l)}, \qquad \alpha^{(l)}(x;\theta) = \sigma(\tilde{\alpha}^{(l)}(x;\theta))$$

và $f_\theta(x) := \tilde{\alpha}^{(L)}(x;\theta)$. Khác với tham số hoá cổ điển (không có hệ số $1/\sqrt{n_l}$ và β, dùng khởi tạo LeCun $W_{ij}^{(l)} \sim N(0, 1/n_l)$), tập hàm biểu diễn được là như nhau giữa hai cách tham số hoá, nhưng **đạo hàm** của $F^{(L)}$ theo $W^{(l)}$ và $b^{(l)}$ bị scale khác nhau ($1/\sqrt{n_l}$ và β tương ứng). Hệ số β dùng để cân bằng ảnh hưởng của bias so với trọng số khi n_l lớn (vì $1/\sqrt{n_l}$ làm giảm mạnh ảnh hưởng của trọng số khi n_l lớn). Trong thực nghiệm của bài, $\beta = 0.1$ với learning rate 1.0 (tương đương một mạng cổ điển rộng 100 với learning rate 0.01).

3.2 Kernel gradient descent (mục 3)

Một kernel đa chiều $K : \mathbb{R}^{n_0} \times \mathbb{R}^{n_0} \to \mathbb{R}^{n_L \times n_L}$ (đối xứng $K(x,x') = K(x',x)^T$) sinh một dạng song tuyến tính $\langle f, g \rangle_K := \mathbb{E}_{x,x' \sim p_{in}}[f(x)^T K(x,x') g(x')]$. K được gọi là xác định dương (positive definite) theo $\|\cdot\|_{p_{in}}$ nếu $\|f\|_{p_{in}} > 0 \Rightarrow \|f\|_K > 0$. Với $\mathcal{F}^*$ là đối ngẫu của $\mathcal{F}$ theo p_{in}, ánh xạ $\Phi_K : \mathcal{F}^* \to \mathcal{F}$ biến $\mu = \langle d, \cdot \rangle_{p_{in}}$ thành hàm f_μ với $f_{\mu,i}(x) = \langle d, K_{i,\cdot}(x,\cdot) \rangle_{p_{in}}$.

Với cost C chỉ phụ thuộc giá trị của f tại các điểm dữ liệu, đạo hàm hàm $\partial_{fin} C|_{f_0} \in \mathcal{F}^*$ có phần tử đối ngẫu $d|_{f_0}$. **Kernel gradient** $\nabla_K C|_{f_0} := \Phi_K(\partial_{fin} C|_{f_0})$, với dạng tường minh trên tập hữu hạn N điểm:

$$\nabla_K C|_{f_0}(x) = \frac{1}{N} \sum_{j=1}^{N} K(x, x_j)\, d|_{f_0}(x_j)$$

Hàm $f(t)$ theo "kernel gradient descent" với K thoả $\partial_t f(t) = -\nabla_K C|_{f(t)}$, và cost tiến hoá theo $\partial_t C|_{f(t)} = -\|d|_{f(t)}\|_K^2$: cost giảm chặt trừ khi $\|d|_{f(t)}\|_{p_{in}} = 0$. Nếu K xác định dương và C lồi, bị chặn dưới, $f(t)$ hội tụ về cực tiểu toàn cục.

3.3 Trực giác: random functions approximation (mục 3.1)

Lấy P hàm ngẫu nhiên độc lập $f^{(p)}$ với hiệp phương sai K ($\mathbb{E}[f_k^{(p)}(x) f_{k'}^{(p)}(x')] = K_{kk'}(x,x')$), tham số hoá tuyến tính $f_\theta^{lin} = \frac{1}{\sqrt{P}} \sum_p \theta_p f^{(p)}$. Gradient descent trên θ của cost $C \circ F^{lin}$ tương đương kernel gradient descent với "tangent kernel" ngẫu nhiên $\tilde{K} = \frac{1}{P} \sum_p f^{(p)} \otimes f^{(p)}$, và khi $P \to \infty$, $\tilde{K} \to K$ (luật số lớn). Đây là mô hình đồ chơi giải thích trực quan tại sao, với $F^{(L)}$ (phi tuyến theo θ) thay vì F^{lin} (tuyến tính theo θ), người ta kỳ vọng một hiện tượng tương tự xảy ra trong giới hạn vô hạn rộng.

3.4 Định nghĩa NTK và kết quả tại khởi tạo (mục 4, 4.1)

$$\Theta^{(L)}(\theta) = \sum_{p=1}^{P} \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta)$$

Mệnh đề 1 (Proposition 1): tại khởi tạo, với $n_1, \dots, n_{L-1} \to \infty$ (tuần tự), $f_{\theta,k}$ ($k = 1, \dots, n_L$) tiến về i.i.d. GP tâm 0, hiệp phương sai $\Sigma^{(L)}$ xác định đệ quy:

$$\Sigma^{(1)}(x, x') = \frac{1}{n_0} x^T x' + \beta^2, \qquad \Sigma^{(L+1)}(x, x') = \mathbb{E}_{f \sim N(0, \Sigma^{(L)})}[\sigma(f(x))\sigma(f(x'))] + \beta^2$$

Định lý 1 (Theorem 1): trong cùng giới hạn, NTK hội tụ theo xác suất về kernel giới hạn xác định: $\Theta^{(L)} \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$, với kernel vô hướng $\Theta_\infty^{(L)}$ xác định đệ quy:

$$\Theta_\infty^{(1)}(x, x') = \Sigma^{(1)}(x, x'), \qquad \Theta_\infty^{(L+1)}(x, x') = \Theta_\infty^{(L)}(x, x')\, \dot{\Sigma}^{(L+1)}(x, x') + \Sigma^{(L+1)}(x, x')$$

trong đó $\dot{\Sigma}^{(L+1)}(x, x') = \mathbb{E}_{f \sim N(0, \Sigma^{(L)})}[\dot{\sigma}(f(x))\dot{\sigma}(f(x'))]$, $\dot{\sigma}$ là đạo hàm của σ. $\Theta_\infty^{(L)}$ chỉ phụ thuộc σ, depth và variance khởi tạo (ở đây là 1).

Bài chú giải ý nghĩa (Remark 4): trong công thức đệ quy, số hạng $\Sigma^{(L+1)}$ đại diện cho phần học của **lớp cuối**, còn số hạng $\Theta_\infty^{(L)} \dot{\Sigma}^{(L+1)}$ đại diện cho phần học của **các lớp thấp hơn**. Dù biến động của từng activation riêng lẻ ở lớp ẩn co lại khi width lớn, hiệu ứng tập thể (collective) qua rất nhiều neuron vẫn đáng kể, nên các lớp thấp vẫn học được.

Ví dụ tự tính (trường hợp cơ sở $L = 1$, không có lớp ẩn, đúng theo chứng minh của Định lý 1 trong Appendix A.1): với $L = 1$, $f_\theta(x) = \frac{1}{\sqrt{n_0}} W^{(0)} x + \beta b^{(0)}$. Lấy $n_0 = 2$, $\beta = 0.1$, hai điểm $x = (1, 2)$, $x' = (3, 1)$.

- $\partial_{W_1} f(x) = x_1/\sqrt{n_0} = 1/\sqrt{2}$, $\partial_{W_2} f(x) = x_2/\sqrt{n_0} = 2/\sqrt{2}$, $\partial_b f(x) = \beta = 0.1$ (tương tự cho x': $3/\sqrt{2}, 1/\sqrt{2}, 0.1$).
- $\Theta(x, x') = \partial_{W_1} f(x) \partial_{W_1} f(x') + \partial_{W_2} f(x) \partial_{W_2} f(x') + \partial_b f(x) \partial_b f(x') = \frac{1 \cdot 3}{2} + \frac{2 \cdot 1}{2} + 0.1 \cdot 0.1 = 1.5 + 1.0 + 0.01 = 2.51$.
- Đối chiếu công thức tổng quát: $\Sigma^{(1)}(x, x') = \frac{x \cdot x'}{n_0} + \beta^2 = \frac{(1 \cdot 3 + 2 \cdot 1)}{2} + 0.01 = \frac{5}{2} + 0.01 = 2.51$. Khớp đúng với tính trực tiếp từ đạo hàm, xác nhận công thức cơ sở của Định lý 1 bằng tay cho trường hợp đơn giản nhất ($L = 1$, không hidden layer).

3.5 Kết quả trong khi huấn luyện (mục 4.2)

Tham số cập nhật theo hướng huấn luyện $d_t \in \mathcal{F}$ (tổng quát hơn gradient descent thường, bao gồm cả GAN): $\partial_t \theta_p(t) = \langle \partial_{\theta_p} F^{(L)}(\theta(t)), d_t \rangle_{p_{in}}$, với gradient descent thì $d_t = -d|_{f_{\theta(t)}}$.

Định lý 2 (Theorem 2): nếu σ Lipschitz, khả vi hai lần, đạo hàm hai bị chặn, và $\int_0^T \|d_t\|_{p_{in}} dt$ bị chặn ngẫu nhiên, thì khi $n_1, \dots, n_{L-1} \to \infty$ (tuần tự), đều trên $t \in [0, T]$: $\Theta^{(L)}(t) \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$ (NTK giữ nguyên không đổi suốt huấn luyện). Khi đó động lực của f_θ là:

$$\partial_t f_{\theta(t)} = \Phi_{\Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}}(\langle d_t, \cdot \rangle_{p_{in}})$$

Chứng minh (Appendix A.2) dựa trên quy nạp theo depth, chặn biến động của activation/weight theo tốc độ $O(1/\sqrt{n_l})$ bằng bất đẳng thức Grönwall (Lemma 1, Appendix A.3): mỗi activation riêng lẻ biến đổi rất ít khi width lớn, nhưng tổng hợp qua nhiều neuron vẫn tạo ra học tập có ý nghĩa ở các lớp thấp.

Mệnh đề 2 (Proposition 2, xác định dương): với σ Lipschitz, không phải đa thức, giới hạn $\Theta_\infty^{(L)}$ hạn chế trên mặt cầu đơn vị $S^{n_0 - 1}$ là xác định dương nếu $L \geq 2$. Chứng minh (Appendix A.4) dùng kỹ thuật khai triển Hermite và hàm "dual activation" (Daniely et al. 2016) cùng một định lý tổng quát hoá kết quả Schönberg (Gneiting 2013) về tính xác định dương của kernel dạng $f(x^T x')$ trên mặt cầu, dựa trên dấu các hệ số khai triển chuỗi lũy thừa của f.

3.6 Least-squares regression và early stopping (mục 5)

Với cost $C(f) = \frac{1}{2}\|f - f^*\|^2_{p_{in}}$, động lực $\partial_t f_t = \Phi_K(\langle f^* - f, \cdot\rangle_{p_{in}})$ có nghiệm $f_t = f^* + e^{-t\Pi}(f_0 - f^*)$ với $\Pi : f \mapsto \Phi_K(\langle f, \cdot\rangle_{p_{in}})$. Π có tối đa Nn_L eigenfunction dương $f^{(1)}, \ldots, f^{(Nn_L)}$ (chính là thành phần chính kernel PCA của dữ liệu theo K), eigenvalue λ_i. Phân rã $f^* - f_0 = \Delta^0_f + \sum_i \Delta^i_f$, nghiệm trở thành:

$$f_t = f^* + \Delta^0_f + \sum_{i=1}^{Nn_L} e^{-t\lambda_i}\Delta^i_f$$

Hội tụ **nhanh hơn** theo các thành phần có eigenvalue lớn hơn; early stopping do đó tập trung hội tụ vào các thành phần chính quan trọng nhất, tránh khớp các thành phần "nhiễu" (eigenvalue nhỏ, ví dụ ứng với tần số cao nếu K là RBF kernel). Khi $t \to \infty$ (và K xác định dương trên dữ liệu, tức ma trận Gram $\tilde{K}$ khả nghịch):

$$f_{\infty,k}(x) = \kappa^T_{x,k}\tilde{K}^{-1}y^* + \left(f_0(x) - \kappa^T_{x,k}\tilde{K}^{-1}y_0\right)$$

Số hạng đầu (mean) có ý nghĩa thống kê: đó là ước lượng MAP với Gaussian prior $f_k \sim N(0, \Theta^{(L)}_\infty)$ và điều kiện $f_k(x_i) = f^*_k(x_i)$, tương đương kernel ridge regression khi hệ số regularize $\lambda \to 0$. Số hạng sau là một Gaussian tâm 0 có variance tiến về 0 tại các điểm dữ liệu.

4. Thí nghiệm và kết quả chính

Mục 6 làm 3 thí nghiệm số, tất cả FC-ANN cùng width $n = n_1 = \cdots = n_{L-1}$ ở các lớp ẩn, ReLU, $n_L = 1$, $\beta = 0.1$.

6.1 Sự hội tụ của NTK: mạng depth $L = 4$, width $n = 500$ và $n = 10000$, input trên vòng tròn đơn vị $x = (\cos\gamma, \sin\gamma)$, $x_0 = (1, 0)$ cố định. Vẽ $\Theta^{(4)}(x_0, x)$ cho 10 lần khởi tạo độc lập mỗi width, tại $t = 0$ và sau 200 bước gradient descent (lr $= 1.0$), với mục tiêu hồi quy $f^*(x) = x_1x_2$ trên input $N(0, 1)$ ngẫu nhiên. Kết quả: mạng rộng hơn có NTK ít phương sai hơn, mượt hơn; kỳ vọng của NTK gần như giống nhau ở cả hai width; sau huấn luyện, NTK có xu hướng "phình to" (inflate), hiệu ứng này nhỏ hơn rõ rệt ở mạng rộng ($n = 10000$).

6.2 Kernel regression: so sánh phân phối Gaussian lý thuyết tại $t \to \infty$ với phân phối thực nghiệm của mạng hữu hạn tại $T = 1000$ (lr $= 1.0$), width $n = 50, 1000$, 10 lần khởi tạo, huấn luyện trên 4 điểm trên vòng tròn đơn vị. Đồng thời xấp xỉ $\Theta^{(4)}_\infty$ và $\Sigma^{(4)}$ bằng mạng rất rộng ($n = 10000$) để vẽ percentile thứ 10, 50, 90 của phân phối giới hạn. Kết quả: phân phối của cả hai width khớp khá tốt với phân phối giới hạn; ngay cả $n = 50$ cũng cho chỉ dấu tốt về phân phối tại $t \to \infty$.

6.3 Hội tụ theo thành phần chính, trên MNIST ($n_0 = 784$): tính 3 thành phần chính đầu của NTK (width $n = 10000$) trên batch $N = 512$ ảnh, bằng power iteration. Eigenvalue thu được: $\lambda_1 = 0.0457$, $\lambda_2 = 0.00108$, $\lambda_3 = 0.00078$. Vì kernel PCA không centering, thành phần đầu gần như là hàm hằng, giải thích khoảng cách lớn giữa eigenvalue 1 và 2. Đặt mục tiêu $f^* = f_{\theta(0)} + 0.5f^{(2)}$, phân rã $f_{\theta(t)} - f^*$ thành g_t (tỉ lệ với $f^{(2)}$) và h_t (trực giao). Trong giới hạn vô hạn rộng: $\|g_t\|_{p_{in}} = 0.5e^{-\lambda_2 t}$, $h_t = 0$. Thử với width $n = 100, 1000, 10000$: mạng rộng hơn lệch khỏi đường thẳng lý thuyết ít hơn (Hình 3b); quỹ đạo theo thành phần chính thứ 2 hội tụ về giới hạn lý thuyết (Hình 3c) khi width tăng.

Chú thích (footnote 3): nếu chọn $\beta = 1.0$ thay cho $\beta = 0.1$ (khuyến nghị), khoảng cách giữa eigenvalue 1 và 2 lớn hơn khoảng 10 lần, khiến việc huấn luyện khó hơn.

5. Phân tích / ablation

- **Quan sát "nghịch lý" về tốc độ hội tụ theo số bước:** mạng nhỏ hơn có vẻ hội tụ nhanh hơn theo số bước (Hình 3c), điều này giải thích được bởi hiệu ứng "inflation" của NTK quan sát ở thí nghiệm 6.1:

nhân NTK với một hệ số a tương đương nhân learning rate với a; nhưng vì NTK của mạng rộng ổn định hơn trong khi huấn luyện, về nguyên tắc có thể chọn learning rate lớn hơn cho mạng rộng. Tác giả cảnh báo: phải cẩn thận khi so sánh tốc độ hội tụ theo số bước (steps) so với theo thời gian liên tục t, vì cả hiệu ứng inflation và learning rate đều ảnh hưởng.

- **Ablation về β (footnote 3):** tăng β từ 0.1 lên 1.0 làm khoảng cách giữa eigenvalue thứ nhất và thứ hai của NTK lớn hơn khoảng 10 lần, và bài báo nói rõ điều này "khiến huấn luyện khó hơn" (đây là một ví dụ ablation thật có trong bài, không phải suy luận thêm), cho thấy tầm quan trọng của việc cân bằng ảnh hưởng bias/trọng số đúng cách trong tham số hoá NTK.
- **Độ rộng và độ ổn định của kernel:** xuyên suốt cả ba thí nghiệm, mẫu số chung là "mạng rộng hơn luôn hành xử gần giới hạn lý thuyết hơn" theo mọi tiêu chí được đo (phương sai NTK ở 6.1, khớp phân phối ở 6.2, lệch khỏi đường thẳng lý thuyết ở 6.3); đây chính là bằng chứng thực nghiệm trực tiếp cho hai định lý chính (Định lý 1, Định lý 2), không chỉ là minh họa định tính.

6. Giới hạn

- Các định lý chính (Mệnh đề 1, Định lý 1, Định lý 2) được chứng minh khi lấy giới hạn width **tuần tự** ($n_1 \to \infty$ rồi $n_2 \to \infty$, ...), không phải lấy đồng thời $\min(n_1, \ldots, n_{L-1}) \to \infty$; bài báo tự ghi rõ trong Appendix A: "kết quả của chúng tôi về nguyên tắc có thể được làm mạnh hơn cho thiết lập chung hơn" nhưng không chứng minh điều đó trong bài này.
- Giả thiết σ Lipschitz, khả vi hai lần, đạo hàm bậc hai bị chặn, được dùng cho toàn bộ máy chứng minh (đặc biệt Định lý 2 và Lemma 1 dùng bất đẳng thức Grönwall); ReLU **không khả vi hai lần theo nghĩa cổ điển** (đạo hàm bậc hai không tồn tại tại 0 theo nghĩa thông thường), nhưng thí nghiệm số (mục 6) vẫn dùng ReLU, tức nằm ngoài giả thiết chứng minh chặt của Định lý 2; bài không bàn thêm về khoảng cách này giữa lý thuyết và thực nghiệm.
- Mệnh đề 2 (xác định dương) chỉ chứng minh cho dữ liệu nằm trên **mặt cầu đơn vị** và σ không phải đa thức, với $L \geq 2$; không bao quát phân phối input tổng quát.
- Chứng minh hội tụ tuyến tính, nghiệm f_t dạng đóng, và lý giải early stopping (mục 5) chỉ được triển khai đầy đủ cho **least-squares regression**; với cost tổng quát (ví dụ cross-entropy cho phân loại), bài chỉ đưa ra khung kernel gradient descent ở mức mục 3 (không có bảo đảm hội tụ tuyến tính tường minh như mục 5).
- Thí nghiệm số có quy mô nhỏ: hai thí nghiệm đầu dùng input 2 chiều trên vòng tròn đơn vị (dữ liệu tổng hợp), thí nghiệm thứ ba dùng MNIST nhưng chỉ một batch 512 ảnh để tính kernel PCA; không có thí nghiệm ở quy mô dữ liệu/mạng lớn như thực tế hiện đại.
- Đã rà soát toàn văn bản v4 (10/2/2020) được đọc cho ghi chú này: không tìm thấy ghi chú erratum/correction nào trong nội dung, mặc dù đây là bản sửa đổi sau hội nghị NIPS 2018 gốc; không thể xác định cụ thể những gì đã được sửa giữa các phiên bản vì chỉ đọc được văn bản của phiên bản v4 duy nhất.

7. Liên hệ với các bài khác

- **math 22 (Lee et al., "Deep Neural Networks as Gaussian Processes"):** đây là liên kết **trực tiếp, được chính bài này trích dẫn** (reference [13], "Deep neural networks as gaussian processes, ICLR 2018"). Phân biệt rõ: NNGP (math 22) nói về $\Sigma^{(L)}$, kernel hiệp phương sai của hàm mạng **tại khởi tạo**, dùng cho suy luận Bayes tĩnh (công thức GP regression). Bài hiện tại (math 23) nói về $\Theta^{(L)}$, kernel sinh từ tích các đạo hàm riêng của hàm mạng theo tham số, mô tả **toàn bộ quỹ đạo huấn luyện bằng gradient descent**. Về công thức, $\Sigma^{(L)}$ ở hai bài gần như cùng một đệ quy (chỉ khác ký hiệu σ_w^2, σ_b^2 so với tham số hoá $1/\sqrt{n_l}, \beta$ của bài này), nhưng $\Theta_\infty^{(L)}$ là một đối tượng hoàn toàn khác, không xuất hiện trong math 22. Đây chính là điểm hay bị người học nhầm lẫn nhất giữa hai bài: NNGP không nói gì về động lực gradient descent, NTK mới là bài nói về điều đó.
- **math 24 ("On Lazy Training in Differentiable Programming"):** mở rộng và hình thức hoá thêm ý

tưởng ”chế độ lazy” (lazy regime) mà NTK ngầm giả định (mạng tuyến tính hoá quanh khởi tạo trong giới hạn rộng), chỉ ra hiện tượng lazy training không chỉ đến từ việc width lớn mà còn từ cách scale tham số hoá (gần giống đúng nhận xét ở mục 3.1 của bài hiện tại về vai trò của β và $1/\sqrt{n_l}$).

- **math 34 (Mei, Montanari, Nguyen, ”A Mean Field View of the Landscape of Two-Layers Neural Networks”):** cũng được trích dẫn trực tiếp trong bài này (reference [15]), ở đúng câu ”phân tích động lực huấn luyện trong giới hạn rộng cho mạng nông (shallow) đã có tiến triển gần đây” (một hướng nghiên cứu song song, dùng công cụ mean-field PDE thay vì kernel, cho mạng 2 lớp).
- **math 33 (”Gradient Descent Finds Global Minima of Deep Neural Networks”):** cùng chủ đề lộ trình (không phải trích dẫn trực tiếp trong bài 1806.07572, mà là liên kết theo mạch lý thuyết hội tụ toàn cục của deep network dựa trên các tính chất giống NTK/positive-definite Gram matrix), nên được xếp gần math 23 trong lộ trình nghiên cứu hội tụ của gradient descent trên mạng rộng.

8. Ứng dụng cho dự án (suy luận)

(Toàn bộ mục này là suy luận/inference của người viết ghi chú, áp dụng ý tưởng của bài vào bối cảnh dự án, không phải nội dung trong bài báo.)

Bối cảnh dự án: post-training/nén LightOnOCR-3-0.8B, VLM OCR kiến trúc lai GatedDeltaNet + full attention, khoảng 3.600 token/trang, GPU ≤ 8GB, batch 1.

Hạn chế cần nói rõ trước: định lý NTK hội tụ về kernel cố định chỉ đúng khi width tiến ra vô hạn **và kiến trúc giữ cố định suốt huấn luyện**; mô hình 0.8B có width hữu hạn, và quan trọng hơn, quá trình phục hồi sau pruning của dự án thường **thay đổi kiến trúc** (cắt head, giảm chiều ẩn, thay thế module) trước khi fine-tune, nên giả thiết nền tảng của Định lý 2 (kiến trúc cố định, chỉ θ thay đổi theo thời gian) không còn đúng nguyên vẹn ngay từ đầu quá trình phục hồi.

Hai hướng suy luận định tính có thể mượn:

1. **Chế độ ”lazy”/kernel như một thước đo định tính cho fine-tuning sau pruning.** Theo mục 5, nếu cập nhật tham số đủ nhỏ (ví dụ fine-tune với learning rate rất nhỏ, ít bước), hàm mạng gần như tuyến tính hoá quanh điểm khởi tạo hiện tại và tiến hoá theo các thành phần chính của một kernel xấp xỉ cố định, hội tụ nhanh theo các hướng ”lớn” và chậm theo hướng ”nhỏ” (ý tưởng early-stopping ở mục 3.6). Điều này gợi ý: khi phục hồi mô hình sau pruning bằng một vài epoch distillation ở learning rate nhỏ, có thể kỳ vọng việc học diễn ra chủ yếu theo một số ít ”hướng chính” quan trọng nhất của bài toán OCR, và early stopping hợp lý (dừng sớm) có cơ sở lý thuyết tương tự mục 3.6, hơn là một con số tùy ý. Ngược lại, nếu learning rate hoặc số bước phục hồi lớn, mô hình rời khỏi chế độ gần-kernel và học biểu diễn mới (feature learning) nhiều hơn, đây có thể là điều **mong muốn** (vì lazy regime vốn bị coi là một giới hạn, không phải một mục tiêu) khi cần phục hồi mạnh sau khi pruning làm mất nhiều thông tin.
2. **Kiểm tra gián tiếp mức độ ”lazy” bằng empirical NTK cục bộ.** Có thể tính một xấp xỉ rẻ của NTK (tích ngoài của gradient theo một tập nhỏ tham số, ví dụ chỉ của lớp adapter/LoRA) trước và sau một đoạn fine-tune ngắn, đo độ thay đổi tương đối (chuẩn Frobenius) để ước lượng mô hình đang ở gần hay xa chế độ lazy, từ đó quyết định learning rate/scheduler cho giai đoạn phục hồi tiếp theo một cách có cơ sở hơn là dò ngẫu nhiên.

Cả hai đều mang tính định tính/chẩn đoán, không áp dụng trực tiếp công thức $\Theta_\infty^{(L)}$ (vì không hợp lệ cho mạng hữu hạn, kiến trúc thay đổi), mà mượn khung khái niệm ”lazy vs. feature-learning” để ra quyết định thực nghiệm.

9. Thí nghiệm gợi ý

1. **Tái lập mini-6.1 (hội tụ/inflation của NTK):** với mạng toy depth 4 trên input vòng tròn đơn vị, so sánh hai width nhỏ (ví dụ 200 và 2000) thay vì 500/10000 của bài, tính NTK bằng autograd (Jacobian

theo tham số rồi nhân ngoài), quan sát hiện tượng ”phình to” sau vài trăm bước gradient descent. Rất rẻ, chạy trên CPU hoặc GPU 8GB trong vài giây đến vài phút.

2. **Tái lập mini-6.3 (kernel PCA trên MNIST):** lấy batch nhỏ (128-512 ảnh) MNIST, tính empirical NTK của một mạng width vài nghìn bằng autograd, trích 3 eigenvalue đầu bằng power iteration, so sánh thứ tự độ lớn với $\lambda_1 = 0.0457, \lambda_2 = 0.00108, \lambda_3 = 0.00078$ của bài (không kỳ vọng khớp tuyệt đối vì kiến trúc/seed khác, chỉ kỳ vọng khớp về thứ tự độ lớn và khoảng cách lớn giữa λ_1 và λ_2).
3. **Kiểm chứng tốc độ hội tụ theo thành phần chính:** đặt $f^* = f_{\theta(0)} + 0.5 f^{(2)}$ như mục 6.3, huấn luyện mạng nhỏ, đo $\|g_t\|$ theo thời gian, fit tốc độ suy giảm mũ, so sánh với λ_2 ước lượng ở thí nghiệm 2; làm với vài width để xem độ lệch khỏi $0.5e^{-\lambda_2 t}$ giảm khi width tăng, như Hình 3b/3c của bài.
4. **Ablation β:** lặp lại thí nghiệm 2 hoặc 3 với $\beta = 0.1$ và $\beta = 1.0$, đo khoảng cách $\lambda_1 - \lambda_2$ ở mỗi trường hợp, kiểm tra có thấy xu hướng tăng rõ rệt giống ghi chú của bài (footnote 3) không.
5. **Chẩn đoán ”lazy regime” cho một adapter nhỏ gắn vào LightOnOCR:** tính empirical tangent kernel (Jacobian outer product) trên một tập nhỏ token OCR, trước và sau một đoạn fine-tune ngắn ở learning rate nhỏ, đo thay đổi tương đối chuẩn Frobenius của kernel này, liên hệ trực tiếp tới mục 8; chỉ cần vài forward/backward pass trên một batch nhỏ, khả thi trên GPU 8GB, batch 1.

10. Tóm tắt công thức / khái niệm

- **NTK:** $\Theta^{(L)}(\theta) = \sum_p \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta)$, ngẫu nhiên tại khởi tạo, biến đổi theo $\theta(t)$ nói chung.
- **Kernel gradient descent:** $\partial_t f(t) = -\nabla_K C|_{f(t)}$, với $\nabla_K C|_{f_0}(x) = \frac{1}{N}\sum_j K(x, x_j) d|_{f_0}(x_j)$; cost giảm theo $\partial_t C|_{f(t)} = -\|d|_{f(t)}\|_K^2$.
- **Mệnh đề 1:** $\Sigma^{(1)}(x, x') = \frac{1}{n_0} x^T x' + \beta^2$; $\Sigma^{(L+1)} = \mathbb{E}_{f\sim N(0,\Sigma^{(L)})}[\sigma(f(x))\sigma(f(x'))] + \beta^2$ (GP tại khởi tạo).
- **Định lý 1:** $\Theta^{(L)} \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$; $\Theta_\infty^{(L+1)} = \Theta_\infty^{(L)} \dot\Sigma^{(L+1)} + \Sigma^{(L+1)}$, với $\dot\Sigma^{(L+1)} = \mathbb{E}[\dot\sigma(f(x))\dot\sigma(f(x'))]$ (kernel giới hạn tại khởi tạo).
- **Định lý 2:** $\Theta^{(L)}(t) \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$ đều trên $[0, T]$ (NTK không đổi suốt huấn luyện); động lực $\partial_t f_{\theta(t)} = \Phi_{\Theta_\infty^{(L)}\otimes \mathrm{Id}}(\langle d_t, \cdot\rangle_{p_{in}})$.
- **Mệnh đề 2:** $\Theta_\infty^{(L)}$ hạn chế trên mặt cầu đơn vị là xác định dương nếu σ không phải đa thức, $L \geq 2$.
- **Nghiệm least-squares:** $f_t = f^* + \Delta_f^0 + \sum_i e^{-t\lambda_i} \Delta_f^i$ (hội tụ nhanh theo eigenvalue lớn, cơ sở cho early stopping); f_∞ = MAP/kernel ridge regression khi $\lambda \to 0$.
- **Chế độ lazy:** vùng tham số hoá/width mà $\Theta^{(L)}(t) \approx \Theta_\infty^{(L)}$ gần như không đổi, mạng hành xử gần tuyến tính hoá quanh khởi tạo (liên hệ math 24).

11. Kết luận ngắn

Bài báo chứng minh rằng ANN vô hạn rộng, trong khi huấn luyện bằng gradient descent, tuân theo kernel gradient descent với một kernel xác định, cố định gọi là Neural Tangent Kernel; điều này biến bài toán huấn luyện không lồi theo tham số thành bài toán kernel method lồi theo hàm, cho phép phân tích hội tụ (qua tính xác định dương của NTK) và sinh ra một lý giải lý thuyết rõ ràng cho early stopping trong hồi quy least-squares. Đây là bước tiếp nối trực tiếp của math 22 (chuyển từ ”tại khởi tạo” sang ”trong suốt huấn luyện”), và là nền tảng cho các phân tích lazy training sâu hơn như math 24.

Nguồn gốc: https://arxiv.org/abs/1806.07572

Bài 24

On Lazy Training in Differentiable Programming (2018)

Tác giả: Lénaïc Chizat (CNRS, Université Paris-Sud), Edouard Oyallon (CentraleSupélec, INRIA), Francis Bach (INRIA, ENS, PSL Research University).
Venue / arXiv: arXiv:1812.07956 (v5, 07/01/2020, bản mở rộng từ bản đầu "A Note on Lazy Training in Supervised Differential Programming" online 12/2018); venue ghi trong paper: "33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada."
Nguồn: https://arxiv.org/abs/1812.07956
Chủ đề: định nghĩa và phân tích hiện tượng "lazy training" (mô hình phi tuyến hành xử gần như phiên bản tuyến tính hoá quanh điểm khởi tạo khi được scale lên), chỉ ra đây là hệ quả của cách chọn **scale** (không chỉ của over-parameterization), và thực nghiệm cho thấy CNN hiệu năng cao không nằm trong chế độ này.
Vị trí trong lộ trình: đọc sau math 22 (Deep NN as Gaussian Processes) và math 23 (Neural Tangent Kernel), vì bài này dùng lại khái niệm tangent kernel của NTK nhưng đặt trong khung tổng quát hơn (differentiable programming, không chỉ neural network) và đưa thêm bằng chứng thực nghiệm phản biện. Nên đọc trước math 25 (Principles of Deep Learning Theory), nơi "lazy"/infinite-width limit cũng xuất hiện như một trường hợp biên ($r = L/n \to 0$). Liên hệ math 19 (Intrinsic Dimension, cùng hỏi "mô hình cần di chuyển bao nhiêu trong không gian tham số để học").

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Khoảng 2018-2019, một loạt công trình lý thuyết (Du et al. 2019 x2, Li & Liang 2018, Allen-Zhu et al. 2019, Zou et al. 2019, và đặc biệt Jacot, Gabriel, Hongler 2018 tức "math 23" NTK) cho thấy mạng nơ-ron **over-parameterized** (rất nhiều tham số so với dữ liệu) được huấn luyện bằng gradient descent có thể hội tụ **tuyến tính (exponentially fast)** về loss huấn luyện bằng 0, trong khi **tham số hầu như không đổi** nhiều so với lúc khởi tạo. Đây là hiện tượng paper này gọi tên là "lazy training": mô hình phi tuyến hành xử giống **phiên bản tuyến tính hoá (linearization)** của nó quanh điểm khởi tạo.

Câu hỏi paper đặt ra: hiện tượng này có thực sự là lý do khiến mạng nơ-ron hoạt động tốt trong thực tế hay không, hay chỉ là một chế độ huấn luyện đặc biệt, dễ phân tích về lý thuyết nhưng không phải là nguyên nhân thành công thực sự?

Khái niệm cần nhớ: cho một mô hình $h : \mathbb{R}^p \to \mathcal{F}$ (ví dụ mạng nơ-ron, $\mathcal{F}$ là không gian Hilbert các hàm dự đoán), **tuyến tính hoá (linearization)** của h quanh điểm khởi tạo w_0 là

$$\bar{h}(w) = h(w_0) + Dh(w_0)(w - w_0),$$

tức khai triển Taylor bậc 1. Nếu h là mạng nơ-ron, $Dh(w_0)$ chính là **Jacobian** của mạng theo tham số tại điểm khởi tạo, và $\bar{h}$ tương đương một **mô hình kernel method tuyến tính** với kernel $K(x, x') = D_w f(w_0, x) D_w f(w_0, x')^\top$ (gọi là **tangent kernel**, cùng khái niệm Jacot et al. 2018 dùng cho NTK, nhưng ở đây công thức cho mạng hữu hạn chiều, không lấy giới hạn width $\to \infty$).

2. Ý tưởng chính

Luận điểm trung tâm: lazy training **không đặc trưng riêng cho mạng over-parameterized**, mà là hệ quả của một **lựa chọn scale**, thường là ngầm định (implicit). Paper chứng minh: với hầu như **bất kỳ** mô hình

tham số hoá trơn h, nếu đầu ra ban đầu gần 0 ($h(w_0) \approx 0$) và ta nhân đầu ra với một hệ số α đủ lớn, mô hình sẽ rơi vào chế độ lazy (gần tuyến tính hoá) khi $\alpha \to \infty$, bất kể số tham số p là bao nhiêu.

Để định lượng "mức độ lazy", paper định nghĩa **tỉ số nghịch đảo scale tương đối (inverse relative scale)** $\kappa_h(w_0)$; với hàm loss bình phương $R(y) = \frac{1}{2}\|y - y^\star\|^2$:

$$\kappa_h(w_0) := \|h(w_0) - y^\star\| \cdot \frac{\|D^2 h(w_0)\|}{\|Dh(w_0)\|^2} \tag{1}$$

$\kappa_h(w_0) \ll 1$ là điều kiện **đủ** (không phải cần) để lazy training xảy ra (Định lý 2.3 định lượng chính xác hơn). Trực giác: κ_h nhỏ khi (a) điểm khởi tạo đã gần đích ($\|h(w_0) - y^\star\|$ nhỏ, qua việc scale α lớn), hoặc (b) mô hình "gần tuyến tính" cục bộ ($\|D^2 h\|$ nhỏ so với $\|Dh\|^2$).

Hệ quả quan trọng paper rút ra bằng thực nghiệm: CNN hiệu năng cao trong thực tế (VGG, ResNet trên CIFAR-10) **không** nằm trong, và khi bị **ép vào** chế độ lazy (bằng cách tăng α), hiệu năng **giảm mạnh**, không vượt qua được một số phương pháp kernel/feature cố định đơn giản. Điều này cho thấy lazy training **không giải thích** được vì sao mạng nơ-ron thực tế hoạt động tốt; thành công thực sự nằm ở chế độ phi tuyến, nơi "neuron di chuyển" (feature learning), điều mà tại thời điểm paper viết (2018-2019) vẫn **chưa có lý thuyết đầy đủ**.

3. Phương pháp chi tiết

3.1 Thiết lập và tiêu chí lazy training

Cho mô hình $h : \mathbb{R}^p \to \mathcal{F}$, loss $R : \mathcal{F} \to \mathbb{R}_+$, mục tiêu $F(w) := R(h(w))$ và mục tiêu tuyến tính hoá $\bar{F}(w) := R(\bar{h}(w))$. **Lazy training** là tình huống (không hiển nhiên) mà quỹ đạo tối ưu của F và $\bar{F}$ (xuất phát cùng w_0) **vẫn gần nhau cho tới khi thuật toán dừng**, chứ không chỉ gần nhau ở vài bước đầu (điều này luôn đúng với mọi mô hình trơn).

Tiêu chí hình thức (Mục 1.2): xét một bước gradient descent $w_1 = w_0 - \eta \nabla F(w_0)$. Thay đổi tương đối của loss là $\Delta(F) \approx \eta \|\nabla F(w_0)\|^2 / F(w_0)$; thay đổi tương đối của Jacobian là $\Delta(Dh) \lesssim \eta \, \|\nabla F(w_0)\| \cdot \|D^2 h(w_0)\| / \|Dh(w_0)\|$. Lazy training xảy ra khi $\Delta(F) \gg \Delta(Dh)$ (loss giảm mạnh mà Jacobian gần như không đổi), dẫn tới điều kiện $\|\nabla F(w_0)\|/F(w_0) \gg \|D^2 h(w_0)\|/\|Dh(w_0)\|$, và với loss bình phương dẫn tới định nghĩa $\kappa_h(w_0)$ ở công thức (1).

3.2 Mô hình được scale lại (rescaled models)

Với hệ số scale $\alpha > 0$, xét $\kappa_{\alpha h}(w_0) = \frac{1}{\alpha}\|\alpha h(w_0) - y^\star\| \cdot \|D^2 h(w_0)\|/\|Dh(w_0)\|^2$. Nếu $\|\alpha h(w_0) - y^\star\|$ bị chặn (ví dụ $h(w_0) = 0$ nên không đổi theo α), thì $\kappa_{\alpha h}(w_0)$ giảm theo α^{-1}, tức α **lớn luôn dẫn tới lazy training**, không cần over-parameterization.

Mô hình thuần nhất bậc q (homogeneous): nếu $h(\lambda w) = \lambda^q h(w)$ với mọi $\lambda > 0$ (đúng với mạng q-layer gồm toàn phép tuyến tính không affine và phi tuyến thuần nhất, ví dụ ReLU), thì nhân khởi tạo với λ tương đương nhân scale α với λ^q: $\kappa_h(\lambda w_0) = \lambda^{-q}\|\lambda^q h(w_0) - y^\star\| \cdot \|D^2 h(w_0)\|/\|Dh(w_0)\|^2$. Vậy **tăng phương sai khởi tạo** cũng đẩy mạng vào chế độ lazy, miễn đầu ra ban đầu có norm bị chặn.

Mạng 2 lớp với m neuron: $h_m(w) = \alpha(m) \sum_{i=1}^m \phi(\theta_i)$, khởi tạo i.i.d. với $\mathbb{E}[\phi(\theta_i)] = 0$. Paper chứng minh (Phụ lục A.2, dựa trên luật số lớn):

$$\mathbb{E}[\kappa_{h_m}(w_0)] \lesssim m^{-1/2} + (m\,\alpha(m))^{-1} \qquad \text{(bound trong Mục 1.2)}$$

Vậy nếu $m\,\alpha(m) \to \infty$ khi $m \to \infty$ (ví dụ $\alpha(m) = 1/\sqrt{m}$), mạng **chắc chắn** rơi vào lazy khi m lớn, vì norm đầu ra ban đầu trở nên "không đáng kể" so với scale, do triệt tiêu thống kê (statistical cancellation) từ

giả thiết trung bình 0. Ngược lại, **scale tới hạn** $\alpha(m) = 1/m$ cho giới hạn **mean-field** (khi $m \to \infty$) không suy biến, mô tả bởi một phương trình đạo hàm riêng, và **không** rơi vào lazy training (đây là chế độ "feature learning" đối lập, liên hệ math 34 "mean field view of landscape").

3.3 Các định lý chặn khoảng cách quỹ đạo

Gọi $w_\alpha(t)$ là gradient flow của $F_\alpha(w) := \frac{1}{\alpha^2} R(\alpha h(w))$ (scale lại objective để α lớn tương ứng tốc độ "đúng" của động lực học), và $\bar{w}_\alpha(t)$ là gradient flow của $\bar{F}_\alpha$ tương tự với mô hình tuyến tính hoá.

Định lý 2.2 (tổng quát, không cần lồi): nếu $h(w_0) = 0$, với mọi khoảng thời gian hữu hạn $[0, T]$:

$$\sup_{t\in[0,T]} \|w_\alpha(t) - w_0\| = O(1/\alpha), \quad \sup_{t\in[0,T]} \|w_\alpha(t) - \bar{w}_\alpha(t)\| = O(1/\alpha^2), \quad \sup_{t\in[0,T]} \|\alpha h(w_\alpha(t)) - \alpha \bar{h}(\bar{w}_\alpha(t))\| = O(1/\alpha).$$

Định lý 2.3 (loss bình phương, định lượng): với K đóng vai trò "số bước lặp" ($T = K/\mathrm{Lip}(h)^2$):

$$\frac{\|\alpha h(w_\alpha(T)) - \alpha \bar{h}(\bar{w}_\alpha(T))\|}{\|\alpha h(w_0) - y^\star\|} \le \frac{K^2 \,\mathrm{Lip}(Dh)}{\alpha \,\mathrm{Lip}(h)^2} \|\alpha h(w_0) - y^\star\|$$

với điều kiện $\alpha \ge K\|\alpha h(w_0) - y^\star\|/(r\,\mathrm{Lip}(h))$. Vế trái là sai số tương đối giữa động lực phi tuyến và tuyến tính hoá; vế phải tỉ lệ nghịch với α, nên α **lớn làm sai số nhỏ**, nhưng hệ số K^2 cho thấy sai số **tăng theo bình phương số bước huấn luyện**.

Định lý 2.4 (over-parameterized, rank $Dh(w_0)$ **đầy/surjective):** với R là m-lồi mạnh, M-trơn (số điều kiện $\kappa = M/m$), nếu σ_{min} (giá trị kỳ dị nhỏ nhất của $Dh(w_0)^\top$) dương và α đủ lớn, $\alpha h(w_\alpha(t))$ hội tụ **tuyến tính** (hàm mũ) về $y^\star$ với tốc độ $\exp(-m\sigma_{min}^2 t/4)$ (sai số nhân với $\sqrt{\kappa}$), đồng thời $\sup_t \|w_\alpha(t) - \bar{w}_\alpha(t)\| = O(\log\alpha/\alpha^2)$.

Định lý 2.5 (under-parameterized): nếu F vô hạn chiều (ví dụ population risk) và rank $Dh(w)$ không đổi quanh w_0, tồn tại α_0 sao cho với $\alpha \ge \alpha_0$, gradient flow hội tụ với **tốc độ hình học không phụ thuộc** α về một **cực tiểu cục bộ** (không nhất thiết toàn cục) của F_α. Điểm quan trọng: kết quả này **không mâu thuẫn** với kết quả hội tụ toàn cục của Chizat & Bach (2018, mean-field, $p \to \infty$), vì Định lý 2.5 giả định p **hữu hạn**.

3.4 Kernel tương đương cho mạng 2 lớp

Với mạng 2 lớp $f_m(w, x) = \alpha(m) \sum_{j=1}^m b_j\, \sigma(a_j \cdot x)$, chọn $\alpha(m) = 1/\sqrt{m}$ (giới hạn không suy biến), tangent kernel $K_m = K_m^{(a)} + K_m^{(b)}$ (một phần cho mỗi lớp) hội tụ (khi $m \to \infty$) về kernel ngẫu nhiên-đặc trưng (random feature kernel). Với σ là ReLU và phân bố trọng số a_j đối xứng xoay (rotation invariant):

$$K^{(a)}(x, x') = \frac{(x \cdot x')\,\mathbb{E}(b^2)}{2\pi}(\pi - \varphi), \qquad K^{(b)}(x, x') = \frac{\|x\|\|x'\|\,\mathbb{E}(\|a\|^2)}{2\pi d}\big((\pi - \varphi)\cos\varphi + \sin\varphi\big) \tag{8}$$

với φ là góc giữa x, x'. $K^{(b)}$ (đạo hàm theo lớp output) là kernel truyền thống liên kết mạng nơ-ron với random features (Rahimi & Recht); $K^{(a)}$ (đạo hàm theo lớp ẩn) là phần mới hơn, đặc trưng cho tangent kernel đầy đủ.

4. Thí nghiệm và kết quả chính

Paper có 2 nhóm thí nghiệm số: (a) mạng 2 lớp trên dữ liệu tổng hợp (teacher-student), (b) CNN sâu (VGG-11, ResNet) trên CIFAR-10. Đây không phải paper lý thuyết thuần, có minh hoạ số thật.

Mạng 2 lớp, teacher-student (Hình 1, Hình 2). Student ReLU 2 lớp, teacher cùng kiến trúc với 3 neuron ẩn. Hình 1: với τ (độ lệch chuẩn khởi tạo) nhỏ ($\tau = 0, 1$), gradient descent "tìm ra" đúng 3 hướng neuron giáo viên (non-lazy); với $\tau = 2$ (lazy), neuron chỉ di chuyển rất ít, nội suy được train loss về gần 0 nhưng **generalization xấu dần khi τ tăng** (Hình 1c, không gian 100 chiều), đạt một plateau trùng khớp với hiệu năng của mô hình tuyến tính hoá. Hình 2a: quét số neuron m với 2 cách scale $\alpha(m) = 1/\sqrt{m}$ (dẫn tới lazy, test error tăng theo m) so với $\alpha(m) = 1/m$ (mean-field, test error giữ gần 0 khi m tăng).

CNN trên CIFAR-10 (Mục 3.2, Hình 3). VGG-11 chuẩn (centered để $h(w_0) = 0$), huấn luyện objective $R(\alpha h(w))/\alpha^2$ với $\alpha \in \{10^0, ..., 10^7\}$, 70 epoch, batch 128. Kết quả chính (Bảng trong Hình 3b, số liệu lấy đúng từ paper):

Mô hình	Train acc. (%)	Test acc. (%)
ResNet wide, linearized ($\alpha = 10^7$)	55,0	56,7
VGG-11 wide, linearized ($\alpha = 10^7$)	61,0	61,7
Prior features (scattering transform, không học)	-	82,3
Random features (không học)	-	84,2
VGG-11 wide, standard ($\alpha = 1$)	99,9	89,7
ResNet wide, standard ($\alpha = 1$)	99,4	91,0

Nhận xét trực tiếp từ bảng: **mô hình linearized (lazy, $\alpha = 10^7$) kém hơn cả các phương pháp feature cố định không học (random features, scattering/prior features)**, và kém xa mô hình standard cùng kiến trúc. "Stability of activations" (tỉ lệ neuron ReLU giữ đúng dấu kích hoạt từ đầu đến cuối huấn luyện) tiến gần 100% khi α tăng, xác nhận mô hình thực sự tuyến tính hoá khi α lớn (Hình 3a).

Phân tích phổ tangent kernel (Phụ lục C.3, Hình 7). Lý do linearized VGG train accuracy không về 0 dù over-parameterized: phổ giá trị riêng σ_i^2 của $Dh(w_0)Dh(w_0)^\top$ có **khoảng cách 1 bậc độ lớn giữa 0,2% giá trị riêng lớn nhất và phần còn lại**, sau đó giảm theo $O(1/i)$ (quan sát giống nhau trên ảnh CIFAR-10 thật và ảnh pixel ngẫu nhiên, cho thấy đây là tính chất nội tại của kiến trúc VGG tuyến tính hoá, không phụ thuộc dữ liệu), gây ra hiện tượng **điều kiện số kém (ill-conditioning)**.

5. Phân tích / ablation

Vai trò over- vs. under-parameterization. Định lý 2.4 (over-parameterized) cho hội tụ **toàn cục**; Định lý 2.5 (under-parameterized, ví dụ population risk vô hạn chiều) chỉ cho hội tụ về **cực tiểu cục bộ**, minh chứng thực nghiệm ở Hình 2b: với $d = 100$, $m = 50$ (dưới tham số hoá), SGD trên population loss hội tụ tốt khi τ nhỏ nhưng mắc ở cực tiểu cục bộ khi τ lớn (lazy); có cả vùng trung gian (hatched area trong Hình 2b) hội tụ rất chậm, chưa kết thúc khi dừng thí nghiệm.

Widening không cứu được lazy regime. Thử mở rộng CNN (x8 kênh cho VGG, x7 cho ResNet) để tăng over-parameterization trong chế độ lazy ($\alpha = 10^7$): hiệu năng linearized nhúc nhích lên (VGG: 61,7% như bảng trên) nhưng **vẫn kém đáng kể** so với random/prior features và kém xa non-lazy, cho thấy vấn đề không chỉ là thiếu tham số mà là **điều kiện số của tangent kernel cố định** (phổ riêng lệch mạnh, xem Hình 7).

Ghi chú về khởi tạo chuẩn (standard init). Paper trích Jacot et al. (2018, NTK): mạng khởi tạo chuẩn (phương sai $O(1/n_\ell)$ ở lớp ℓ) **tất yếu** tiến tới lazy khi mọi lớp rộng vô hạn, và cách khởi tạo này **gần như bắt buộc** để tránh gradient nổ/biến mất ở mạng rất sâu (trích He et al. 2015, Hanin & Rolnick 2018). Paper nhấn mạnh: **không phủ nhận** mạng rộng có xu hướng lazy, chỉ khẳng định các mạng **hoạt động tốt trong thực tế còn cách xa giới hạn tiệm cận đó**.

6. Giới hạn

Paper tự nêu một số giới hạn: (a) các chặn lý thuyết cho $\kappa_h(w_0)$ chỉ là điều kiện **đủ**, không phải cần, nên lazy training "có thể xảy ra ngay cả khi κ_h lớn" (ghi chú footnote 3); (b) chặn của Định lý 2.2 phụ thuộc **hàm mũ** vào horizon T (hằng số ẩn tăng theo $\exp(T)$), chỉ Định lý 2.3 (loss bình phương) loại bỏ được sự phụ thuộc này; (c) Định lý 2.5 (under-parameterized) chỉ dừng ở phát biểu **định tính** (tồn tại α_0), không cho công thức tường minh vì liên quan tới giá trị kỳ dị dương nhỏ nhất của $Dh(w_0)$, "khó kiểm soát" theo lời paper; (d) câu hỏi "liệu có thể thiết kế kiến trúc/pipeline khác để linearized CNN cạnh tranh được với non-lazy" được paper để **mở**, chỉ trích một kết quả khác (Arora et al. 2019, kiến trúc đặc biệt giải quyết ill-conditioning, đạt 77,4% không data augmentation) nhưng không tự thử nghiệm.

Một lưu ý khi đọc: đoạn Phụ lục C.3 trong bản PDF gốc dùng dấu gạch ngang dài kiểu typeset (nguyên văn có đoạn chêm ý kiểu "...which causes the ill conditionning, and then a decrease...", bản gốc dùng ký hiệu chêm câu khác) để chêm ý trong câu; đây là văn phong gốc của paper, không phải lỗi, nhưng ghi chú này diễn đạt lại bằng dấu phẩy/ngoặc theo quy tắc không dùng em dash.

7. Liên hệ với các bài khác

- **math 23 (Neural Tangent Kernel)**: bài 24 dùng lại đúng khái niệm tangent kernel $K(x, x') = D_w f(w_0, x) D_w f(w_0, x')^\top$ của bài 23, nhưng đặt câu hỏi khác: bài 23 hỏi "giới hạn width vô hạn thì NTK có tính chất gì", còn bài 24 hỏi "với mạng hữu hạn, khi nào và vì sao mô hình rơi vào chế độ giống NTK (lazy), và chế độ đó có tốt không". Bài 24 cũng chỉ ra rằng bản thân khái niệm "lazy" rộng hơn neural network, áp dụng cho mọi "differentiable programming".
- **math 22 (Deep NN as Gaussian Processes)**: paper trích dẫn [20,23] (Lee et al., Matthews et al.) về việc phân phối $h(w_0)$ hội tụ về Gaussian process khi mạng over-parameterized với khởi tạo chuẩn, dùng để giải thích nguồn gốc ngẫu nhiên của "offset" $h(w_0)$ trong kernel method tương đương (Mục A.1).
- **math 19 (Measuring the Intrinsic Dimension of Objective Landscapes)**: cả hai bài cùng quan tâm "mạng cần di chuyển/dùng bao nhiêu độ tự do để học", nhưng bài 19 đo tĩnh (subspace cố định, không phụ thuộc động lực học), còn bài 24 đo động (quỹ đạo gradient flow di chuyển bao xa khỏi w_0, cụ thể $\|w_\alpha(t) - w_0\| = O(1/\alpha)$ trong chế độ lazy).
- **math 25 (Principles of Deep Learning Theory)**: sách này mô tả chính xác chế độ "$r = L/n \to 0$" (infinite-width) là nơi mạng hành xử tuyến tính và **mất khả năng học biểu diễn** (representation learning), đúng là định nghĩa "lazy" của bài 24 nhìn từ góc độ width/depth thay vì scale α của output; sách cũng dùng khai triển $1/n$ để mô tả "độ lệch khỏi lazy" tương tự cách bài 24 dùng khai triển $1/\alpha$.
- **pruning 38 (Sheared LLaMA, đã kiểm tồn tại bằng `ls "pruning content"`)** và **distillation content (fine-tune nhẹ sau khi nén, đã kiểm tồn tại thư mục)**: về mặt thực hành, bài học "huấn luyện với learning rate/scale nhỏ gần giữ nguyên biểu diễn ban đầu" (gần giống chế độ lazy nhưng theo chiều ngược: scale nhỏ, không lớn) liên quan tới cách fine-tune nhẹ một mô hình đã prune mà không phá vỡ biểu diễn đã học, một chủ đề các bài trong "pruning content" và "distillation content" xử lý thực nghiệm.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1.6-2.1K token visual, GPU ≤ 8GB, batch 1).

- **Bài học quan trọng nhất (suy luận):** kết quả thực nghiệm của bài 24 (CNN lazy kém hơn CNN thường rất nhiều) là lời cảnh báo cho việc fine-tune mô hình đã prune/distill với **learning rate quá nhỏ hoặc quá nhiều regularization khiến tham số gần như không di chuyển**: nếu việc hậu huấn luyện vô tình rơi vào chế độ "giống lazy" (tham số gần như đứng yên, chỉ học được một kernel cố định quanh điểm sau-khi-prune), hiệu năng phục hồi có thể bị chặn trên bởi một "kernel method" yếu, giống cách CNN lazy bị chặn dưới hiệu năng của random features trong bài 24. Đây là suy luận tương tự hoá: paper

không thử nghiệm trên mô hình đã prune hay fine-tune LoRA, chỉ trên VGG/ResNet huấn luyện từ đầu với scale α khác nhau.

- **Liên hệ với LoRA/adapter ở chế độ "lazy" có kiểm soát:** ngược lại, có tình huống **mong muốn** gần-lazy: khi chỉ fine-tune một adapter nhỏ (LoRA) và muốn giữ gần nguyên biểu diễn đã học qua pretrain/distill (tránh "catastrophic forgetting" kỹ năng OCR), việc tham số thay đổi ít quanh checkpoint ban đầu (giống tinh thần "lazy" nhưng quanh điểm **đã huấn luyện**, không phải điểm khởi tạo ngẫu nhiên) có thể là **đặc tính tốt** cần giữ, trái với bối cảnh bài 24 (khởi tạo ngẫu nhiên). Cần phân biệt rõ hai bối cảnh để không áp dụng sai kết luận "lazy training là xấu" của bài 24 cho mọi tình huống fine-tune.
- **Giới hạn khi áp dụng:** paper hoàn toàn không đề cập Transformer/attention/SSM/VLM, chỉ MLP 2 lớp và CNN (VGG/ResNet) kiểu ảnh phân loại; việc ngoại suy sang kiến trúc hybrid Gated-DeltaNet+attention cho OCR là suy luận có rủi ro, cần kiểm chứng thực nghiệm riêng trước khi tin tưởng.

9. Thí nghiệm gợi ý

1. **Tái lập Hình 1 của paper ở quy mô nhỏ hơn:** mạng ReLU 2 lớp, $n = 15$ điểm dữ liệu tổng hợp, $m = 20$ neuron, teacher 3 neuron, quét $\tau \in \{0,1;\ 0,5;\ 1;\ 2;\ 5\}$. Đo: quỹ đạo $|a_j(t)|b_j(t)$ trong không gian 2D và test loss cuối cùng. Kỳ vọng: τ nhỏ tìm đúng hướng teacher (non-lazy), τ lớn chỉ dịch chuyển nhẹ quanh gốc (lazy), test loss tăng theo τ. Diễn giải: xác nhận hiện tượng cơ bản trước khi tin vào các kết luận CNN phức tạp hơn.
2. **Đo $\kappa_h(w_0)$ (công thức 1) trực tiếp bằng autograd cho một MLP nhỏ** (2-3 lớp, vài trăm tham số) ở vài mức phương sai khởi tạo τ^2, dùng Hessian-vector product để ước lượng $\|D^2h(w_0)\|$ (hoặc xấp xỉ bằng finite difference của Jacobian). Đo: κ_h có giảm theo $1/\tau^q$ (q là bậc thuần nhất) như công thức Mục 3.2 dự đoán không? Diễn giải: kiểm tra trực tiếp công thức lý thuyết bằng số, một bài tập autograd hữu ích.
3. **Huấn luyện VGG nhỏ (hoặc ResNet-8) trên CIFAR-10 với vài giá trị** $\alpha \in \{1, 10, 100, 1000\}$ trên 1 GPU 8GB (batch nhỏ, vài epoch), đo train/test accuracy và "stability of activations" (tỉ lệ neuron ReLU giữ dấu kích hoạt từ đầu đến cuối). Kỳ vọng: giống xu hướng Hình 3a của paper (accuracy giảm, stability tăng khi α tăng), dù quy mô nhỏ hơn nhiều so với paper (không cần tới $\alpha = 10^7$ hay widen x7-x8 để thấy xu hướng). Diễn giải: xác nhận hiện tượng ở quy mô vừa với 8GB.
4. **So sánh phổ giá trị riêng của tangent kernel** ($Dh(w_0)Dh(w_0)^\top$, ước lượng qua ma trận Jacobian trên một batch nhỏ, ví dụ 100 ảnh) cho 1 mạng nhỏ **trước và sau khi prune 50% kênh** (ví dụ cắt ngẫu nhiên 1 nửa filter của 1 layer). Đo: phổ riêng có xuất hiện khoảng cách lớn giữa vài giá trị riêng đầu và phần còn lại không, và có thay đổi sau prune không? Diễn giải: liên hệ trực tiếp ý tưởng "ill-conditioning của tangent kernel" với bối cảnh pruning của dự án.
5. **Fine-tune một adapter nhỏ (LoRA rank thấp) trên một mô hình ngôn ngữ nhỏ cho 1 tác vụ, với 2 learning rate khác nhau (rất nhỏ vs. bình thường)**, đo khoảng cách tham số $\|w(T) - w_0\|$ và hiệu năng cuối. Kỳ vọng: learning rate rất nhỏ giữ tham số gần w_0 (giống "lazy" quanh checkpoint) nhưng có thể đạt hiệu năng tốt nếu checkpoint ban đầu đã gần tối ưu cho tác vụ, trái với kết luận "lazy luôn xấu" của paper (vì bối cảnh khác: không phải khởi tạo ngẫu nhiên). Diễn giải: minh hoạ sự khác biệt giữa "lazy quanh init ngẫu nhiên" (xấu, theo paper) và "ít thay đổi quanh checkpoint đã huấn luyện tốt" (có thể tốt, tuỳ tác vụ).

10. Tóm tắt công thức / khái niệm

- **Linearization:** $\bar{h}(w) = h(w_0) + Dh(w_0)(w - w_0)$; tangent kernel $K(x, x') = D_w f(w_0, x) D_w f(w_0, x')^\top$ (7).
- **Inverse relative scale (điều kiện đủ cho lazy, loss bình phương):** $\kappa_h(w_0) = \|h(w_0) - y^\star\| \cdot \|D^2h(w_0)\|/\|Dh(w_0)\|^2$ (1); $\kappa_h \ll 1 \Rightarrow$ lazy.
- **Rescaled objective:** $F_\alpha(w) = \frac{1}{\alpha^2} R(\alpha h(w))$ (2); $\kappa_{\alpha h}(w_0)$ giảm $\propto \alpha^{-1}$ khi $\alpha \to \infty$ (nếu $\|\alpha h(w_0) -$

$y^\star\|$ bị chặn).

- **Mô hình thuần nhất bậc q:** nhân khởi tạo λ tương đương nhân scale α với λ^q; mạng q-layer ReLU/homogeneous rơi vào lazy khi phương sai khởi tạo tăng.
- **Chặn chính (Định lý 2.2, 2.3):** $\sup_t \|w_\alpha(t) - \bar{w}_\alpha(t)\| = O(1/\alpha^2)$ (horizon hữu hạn, tổng quát); với loss bình phương, sai số tương đối $\leq (K^2 \mathrm{Lip}(Dh)/\alpha \,\mathrm{Lip}(h)^2) \cdot \|\alpha h(w_0) - y^\star\|$.
- **Hội tụ toàn cục (over-parameterized, Định lý 2.4):** $\|\alpha h(w_\alpha(t)) - y^\star\| \leq \sqrt{\kappa}\,\|\alpha h(w_0) - y^\star\| \exp(-m\sigma_{min}^2 t/4)$.
- **Tự tính (ví dụ số nhỏ minh hoạ công thức 1):** giả sử một mô hình toy có $\|h(w_0) - y^\star\| = 2$, $\|D^2 h(w_0)\| = 0,5$, $\|Dh(w_0)\| = 10$. Khi đó $\kappa_h(w_0) = 2 \times 0,5/10^2 = 1/100 = 0,01 \ll 1$, nên theo tiêu chí (1), mô hình này (với các số giả định trên) đang ở chế độ lazy mạnh. Nếu scale h lên $\alpha = 5$ (giữ $y^\star$ cố định), theo công thức Mục 3.2 ta có $\kappa_{\alpha h}(w_0) = \kappa_h(w_0)/\alpha = 0,01/5 = 0,002$, càng nhỏ hơn, tức lazy hơn nữa. Đây là số tự chọn để minh hoạ cách dùng công thức, không phải số đo thực từ paper.

11. Kết luận ngắn

Bài này tách rõ ”lazy training” (mô hình phi tuyến hành xử như tuyến tính hoá quanh điểm khởi tạo) ra khỏi ”over-parameterization”: lazy chủ yếu do **lựa chọn scale** (ngầm định qua cách khởi tạo/chuẩn hoá), và có thể xảy ra ở mọi kích thước mô hình khi scale đủ lớn. Bằng cả lý thuyết (các chặn với horizon hữu hạn và vô hạn, cho cả trường hợp over- và under-parameterized) và thực nghiệm (CNN thật trên CIFAR-10), paper đưa ra một kết luận mang tính phản biện quan trọng cho cả nhánh lý thuyết NTK: **chế độ lazy, dù dễ phân tích, không giải thích được vì sao mạng nơ-ron hoạt động tốt trong thực tế**; chìa khoá thành công thực sự nằm ở chế độ phi tuyến (neuron di chuyển, feature learning), vẫn còn thiếu lý thuyết đầy đủ tại thời điểm paper được viết.

Nguồn gốc: https://arxiv.org/abs/1812.07956

Bài 25

The Principles of Deep Learning Theory (2021)

Tác giả: Daniel A. Roberts, Sho Yaida (nghiên cứu dựa trên hợp tác với Boris Hanin).
Venue / arXiv: arXiv:2106.10165 (v2, 24/08/2021); paper không nêu venue hội nghị/journal. Sau này nội dung được Cambridge University Press in thành sách, nhưng đó là nhà xuất bản sách, không phải ”venue” theo nghĩa hội nghị khoa học.
Nguồn: https://arxiv.org/abs/2106.10165
Chủ đề: Đây là một cuốn sách giáo trình dài khoảng 470 trang, xây dựng một ”lý thuyết hiệu dụng” (effective theory, mượn từ vật lý) cho mạng nơ-ron sâu tại thời điểm khởi tạo và trong lúc huấn luyện, với tham số trung tâm là tỉ lệ độ sâu trên độ rộng $r = L/n$. **Quan trọng:** ghi chú này CHỈ đọc kỹ các phần sau: Preface, Chương 0 (Initialization), Chương 3 (Effective Theory of Deep Linear Networks, toàn bộ §3.1 đến §3.4), Chương 5 §5.1-5.2 (Criticality Analysis of the Kernel, Criticality for Scale-Invariant Activations), Epilogue, Appendix A (Information in Deep Learning), Appendix B (Residual Learning). KHÔNG đọc Chương 1, 2, 4, 6, 7-9, 10, 11, ∞ (Pretraining, Neural Networks, phần còn lại của Effective Theory of Deep Nonlinear Networks, RG flow, Gradients/NTK chi tiết, Kernel Prediction, Representation Learning, End of Training) và phần References/Index. Đây là lựa chọn đọc ”core parts” cho một cuốn sách rất dài, theo đúng tinh thần brief cho phép với tài liệu khảo sát/sách dài.
Vị trí trong lộ trình: nên đọc sau math 26 (Poole et al., transient chaos) và math 27 (Schoenholz et al., Deep Information Propagation), vì PDLT kế thừa trực tiếp khái niệm criticality/edge-of-chaos của hai bài đó (chính PDLT ghi trong Chương 5 rằng ”initial part of this analysis was first carried out in a series of papers” dùng kỹ thuật khác nhưng tương đương ở giới hạn vô hạn rộng) và hệ thống hoá nó bằng một khung lý thuyết nhiễu loạn chặt chẽ hơn. Cũng nên đọc sau math 22 (Deep Neural Networks as Gaussian Processes, NNGP) và math 23 (Neural Tangent Kernel), vì Chương 0 và Epilogue của PDLT dùng lại đúng hai khái niệm NNGP và NTK làm hai đối tượng trung tâm của giới hạn vô hạn rộng. Nên đọc cùng nhóm với math 19 (Intrinsic Dimension) và math 24 (Lazy Training) vì cả ba đều xoay quanh câu hỏi ”độ phức tạp hiệu dụng của mô hình là gì, nếu không phải là số tham số”.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Tính đến khoảng 2021, lý thuyết về mạng nơ-ron sâu tại thời điểm khởi tạo (trước khi huấn luyện) có hai nhánh kết quả ”giải được” nhưng tách rời nhau. Nhánh thứ nhất là giới hạn vô hạn rộng (infinite width): khi số neuron mỗi lớp $n \to \infty$, output của mạng tại khởi tạo trở thành một Gaussian process (NNGP, xem math 22), và động lực huấn luyện bằng gradient descent trở thành tuyến tính, mô tả bởi Neural Tangent Kernel (NTK, xem math 23). Nhánh thứ hai là các kết quả về ”truyền tín hiệu” (signal propagation) và ”edge of chaos” của Poole et al. (math 26) và Schoenholz et al. (math 27): các tác giả này nhận ra rằng nếu tinh chỉnh phương sai trọng số và bias lúc khởi tạo tới một giá trị ”tới hạn” (critical), mạng rất sâu vẫn truyền được tín hiệu mà không bị bùng nổ hay triệt tiêu theo cấp số nhân theo độ sâu.

PDLT đặt câu hỏi: có thể xây một khung lý thuyết DUY NHẤT, chặt chẽ về mặt toán học, vừa chứa cả hai nhánh trên như các trường hợp riêng, vừa cho phép tính toán có hệ thống các hiệu ứng độ rộng hữu hạn (finite-width) để mạng vẫn ”học được representation” (biểu diễn) thay vì chỉ là một bộ làm mượt tuyến tính (linear smoother) không đổi theo dữ liệu? Cách tiếp cận của sách là mượn nguyên ”effective theory approach” từ vật lý: giống như nhiệt động lực học (thermodynamics) là một lý thuyết ”hiệu dụng” ở quy mô lớn, mô tả máy hơi nước mà không cần theo dõi từng phân tử khí riêng lẻ (đó là việc của cơ học thống kê, statistical

mechanics), các tác giả muốn một lý thuyết hiệu dụng mô tả hành vi mạng nơ-ron ở quy mô ”nhiều neuron, nhiều lớp” mà không cần theo dõi từng tham số riêng lẻ.

Thuật ngữ cần nắm trước: **lý thuyết hiệu dụng (effective theory)** là một lý thuyết xấp xỉ, chỉ đúng và chỉ cần đúng trong một phạm vi (scale) nhất định, bỏ qua chi tiết vi mô không ảnh hưởng tới hành vi vĩ mô ở phạm vi đó. **Tới hạn (criticality)**, mượn từ lý thuyết chuyển pha trong vật lý thống kê (ví dụ sắt từ chuyển từ có từ tính sang không có từ tính tại một nhiệt độ tới hạn), ở đây chỉ việc tinh chỉnh siêu tham số khởi tạo (initialization hyperparameters) tới một điểm đặc biệt mà hệ thống không ”đổ” về một trong hai hành vi tầm thường (bùng nổ hoặc triệt tiêu theo cấp số nhân) khi độ sâu tăng.

2. Ý tưởng chính

Toàn bộ sách xoay quanh việc khai triển Taylor hàm output đã huấn luyện $z(x_\delta;\theta^\star)$ quanh điểm khởi tạo θ:

$$z(x_\delta;\theta^\star) = z(x_\delta;\theta) + \sum_\mu (\theta^\star-\theta)_\mu \frac{dz_\delta}{d\theta_\mu} + \frac{1}{2}\sum_{\mu,\nu}(\theta^\star-\theta)_\mu(\theta^\star-\theta)_\nu \frac{d^2 z_\delta}{d\theta_\mu d\theta_\nu} + \dots$$

Khai triển này (Epilogue gọi lại là eq ε.2, tương ứng eq 0.5 ở Chương 0) gặp ba ”Vấn đề” được paper nêu rõ ngay từ Chương 0 (eq 0.6, 0.7, 0.8):

- **Vấn đề 1:** về nguyên tắc phải tính vô hạn số hạng đạo hàm $dz/d\theta, d^2z/d\theta^2, d^3z/d\theta^3, \dots$
- **Vấn đề 2:** cần biết phân phối đồng thời (joint distribution) của output z và tất cả các đạo hàm đó, ánh xạ từ phân phối khởi tạo $p(\theta)$ sang phân phối của $\left(z, dz/d\theta, d^2z/d\theta^2, \dots\right)$, nói chung không khả giải (intractable).
- **Vấn đề 3:** điểm hội tụ $\theta^\star$ sau huấn luyện phụ thuộc vào gần như mọi thứ: thuật toán học, learning rate, dữ liệu huấn luyện, v.v.

Sách đưa ra hai cách giải quyết bộ ba vấn đề này, cả hai đều dựa trên một quan sát gọi là **”nguyên lý thưa” (principle of sparsity)**: khi số neuron n mỗi lớp lớn, rất nhiều tương tác giữa các neuron tự triệt tiêu lẫn nhau (tương tự định luật số lớn), làm phân phối trở nên đơn giản hơn hẳn so với khi n nhỏ.

- **Cách 1 (giới hạn vô hạn rộng nghiêm ngặt, $n\to\infty$):** giải quyết triệt để cả 3 vấn đề, cho nghiệm closed-form (dạng Gaussian, đây chính là NNGP và NTK). Tuy nhiên cái giá phải trả: trong giới hạn này các biểu diễn ẩn (hidden representations) ở mọi lớp không thay đổi so với lúc khởi tạo trong suốt quá trình huấn luyện, nghĩa là mạng **không học được representation** theo nghĩa thông thường, chỉ là một bộ hồi quy tuyến tính trên các đặc trưng ngẫu nhiên cố định (random features).
- **Cách 2 (khai triển nhiễu loạn $1/n$, giữ n lớn nhưng hữu hạn):** giữ lại hiệu ứng độ rộng hữu hạn ở bậc dẫn đầu (leading order). Paper viết dưới dạng một khai triển cho phân phối của output đã huấn luyện (eq 0.14-0.15 ở Chương 0):

$$p(f^\star) = p^{\{0\}}(f^\star) + \frac{1}{n}p^{\{1\}}(f^\star) + O\left(\frac{1}{n^2}\right)$$

Số hạng $p^{\{0\}}$ chính là nghiệm Gaussian của Cách 1; số hạng $p^{\{1\}}/n$ là hiệu chỉnh độ rộng hữu hạn đầu tiên, và CHÍNH số hạng này mang lại khả năng học representation thật sự.

Điểm mấu chốt của toàn sách: khi thực hiện khai triển $1/n$ này cho một mạng có L lớp, tham số khai triển thực sự xuất hiện không phải là $1/n$ đơn thuần, mà là tỉ lệ

$$r \equiv \frac{L}{n}$$

gọi là **aspect ratio** (tỉ lệ độ sâu trên độ rộng, eq 0.19 ở Chương 0). Ba miền giá trị của r (thuật ngữ chính xác từ sách, giải thích thêm bằng tiếng Việt):

- $r \to 0$: mạng hành xử "giống vô hạn rộng" (infinite-width-like); dù L có lớn, nếu n lớn hơn nhiều thì $r \approx 0$ và mạng "không thực sự sâu" theo nghĩa hiệu dụng.
- $0 < r \ll 1$: mạng "hiệu dụng là sâu" (effectively deep); đây là vùng mà khai triển $1/n$ cắt ở bậc 1 chính xác, và mạng thực tế hay rơi vào vùng này.
- $r \gg 1$: mạng "quá sâu" (overly deep); các neuron tương tác mạnh (chaotic), không còn mô tả hiệu dụng nào có giá trị thực tế, và theo paper, mạng như vậy "vô dụng" (useless) trong thực hành.

3. Phương pháp chi tiết

Mạng tuyến tính sâu làm mô hình đồ chơi (Chương 3)

Sách gọi deep linear network là "giống dao động điều hòa đơn giản (simple harmonic oscillator) đối với deep learning": một mô hình đồ chơi tối giản nhưng vẫn lộ ra toàn bộ cấu trúc khái niệm (criticality, chaos, aspect ratio) mà các chương sau sẽ tổng quát hoá. Mạng được định nghĩa bởi (eq 3.1, bias đặt bằng 0 trong chương này):

$$z_{i;\alpha}^{(\ell+1)} = \sum_j W_{ij}^{(\ell+1)} z_{j;\alpha}^{(\ell)}$$

với thống kê trọng số (eq 3.4): $E[W_{ij}^{(\ell)}] = 0$ và $E[W_{i_1 j_1}^{(\ell)} W_{i_2 j_2}^{(\ell)}] = \frac{C_W}{n_{\ell-1}} \delta_{i_1 i_2} \delta_{j_1 j_2}$ (chú ý hệ số $1/n_{\ell-1}$: đây là cách khởi tạo "chuẩn hoá theo độ rộng" phổ biến, để phương sai output không phụ thuộc n).

Correlator hai điểm (two-point correlator) $G_{\alpha_1\alpha_2}^{(\ell)} \equiv \frac{1}{n_\ell} \sum_i E[z_{i;\alpha_1}^{(\ell)} z_{i;\alpha_2}^{(\ell)}]$ đo "độ giống nhau" trung bình giữa preactivation của hai mẫu đầu vào α_1, α_2 ở lớp ℓ. Dùng **Wick contraction** (một hệ quả của việc các biến W là Gaussian: kỳ vọng của tích 4 biến Gaussian phân rã thành tổng các tích từng cặp hiệp phương sai), paper suy ra công thức đệ quy đơn giản (eq 3.14): $G_{\alpha_1\alpha_2}^{(\ell+1)} = C_W G_{\alpha_1\alpha_2}^{(\ell)}$, giải chính xác (eq 3.15): $G^{(\ell)} = (C_W)^\ell G^{(0)}$.

Đây là chỗ khái niệm **criticality** xuất hiện lần đầu, dưới dạng đơn giản nhất: nếu $C_W > 1$, $G^{(\ell)}$ bùng nổ theo cấp số nhân khi ℓ tăng (điểm cố định tầm thường $G^\star = \infty$); nếu $C_W < 1$, $G^{(\ell)}$ triệt tiêu theo cấp số nhân ($G^\star = 0$, mọi thông tin phân biệt giữa các mẫu bị xoá sạch). Chỉ tại $C_W = 1$ (gọi là **critical initialization**), correlator được bảo toàn đúng nguyên giá trị ban đầu qua mọi lớp: đây là "điểm cố định không tầm thường" (nontrivial fixed point), tương tự điểm chuyển pha trong vật lý thống kê.

Correlator bốn điểm và dao động (fluctuations, §3.3): ở độ rộng hữu hạn, correlator bậc 4 có đệ quy (eq 3.24): $G_4^{(\ell+1)} = C_W^2 \left(1 + \frac{2}{n_\ell}\right) G_4^{(\ell)}$, giải ra (eq 3.25): $G_4^{(\ell)} = \left[\prod_{\ell'} \left(1 + \frac{2}{n_{\ell'}}\right)\right] \left(G_2^{(\ell)}\right)^2$. Ở giới hạn vô hạn rộng, số hạng $2/n_{\ell'}$ biến mất, cho $G_4 = (G_2)^2$: đây đúng là tính chất của phân phối Gaussian thuần tuý (không có tương quan "dư" giữa các neuron trong cùng lớp). Ở độ rộng hữu hạn, tại đúng điểm tới hạn $C_W = 1$, độ lệch khỏi tính Gaussian (non-Gaussianity) ở bậc dẫn đầu là (eq 3.28):

$$\frac{2(\ell-1)}{n} \left(G_2^{(\ell)}\right)^2 + O\left(\frac{1}{n^2}\right)$$

Paper nhấn mạnh đây "có lẽ là chủ đề lặp lại quan trọng nhất của toàn sách": độ lệch này tỉ lệ thuận với độ sâu, tỉ lệ nghịch với độ rộng, nghĩa là tỉ lệ với chính $r = L/n$. Về ý nghĩa: đây chính là tương tác thực sự giữa các neuron trong một lớp (intralayer interaction), và chính nó là thứ cho phép mạng học representation; mạng vô hạn rộng (không có số hạng này) không thể có tương tác neuron-neuron nên không học được representation theo nghĩa phi tầm thường.

Hỗn loạn (chaos, §3.4): tổng quát hoá cho correlator bậc $2m$ bất kỳ, đệ quy có dạng (eq 3.45): $G_{2m}^{(\ell+1)} =$

$c_{2m}(n_\ell) C_W^m G_{2m}^{(\ell)}$ với $c_{2m}(n)$ là một hệ số tổ hợp học (eq 3.46). Hai cách lấy giới hạn cho kết quả khác nhau hẳn, và đây là điểm rất quan trọng: nếu lấy $n \to \infty$ trước (giữ L cố định), $c_{2m} \to 1$, output hoàn toàn Gaussian, và mạng như vậy (theo cách diễn giải của sách) thực chất tương đương một mạng 1 lớp, "không thực sự sâu". Ngược lại nếu lấy $L \to \infty$ trước (giữ n cố định), ngay cả tại đúng điểm tới hạn $C_W = 1$, các correlator bậc cao vẫn bùng nổ theo cấp số nhân theo L: mạng hoàn toàn hỗn loạn (chaotic), không Gaussian, và theo lời paper là "không dùng được" trên thực tế. Hai giới hạn này không giao hoán (eq 3.51).

Paper giải quyết bằng cách lấy **giới hạn kép** (double-scaling limit): cho cả $n, L \to \infty$ nhưng giữ $r = L/n$ cố định (eq 3.52). Dùng khai triển $c_{2m}(n) = 1 + \frac{m(m-1)}{n} + O(1/n^2)$ (eq 3.53) và công thức giới hạn mũ $\lim_{L\to\infty}(1 + a/L)^L = e^a$ (eq 3.54), kết quả là (eq 3.55):

$$G_{2m}^{(L)} \to e^{m(m-1)r} \left(G_2^{(L)}\right)^m$$

Công thức này nội suy liên tục giữa giới hạn Gaussian ($r \to 0$) và giới hạn hỗn loạn ($r \to \infty$), với r là tham số duy nhất điều khiển vị trí trên "thanh trượt" đó. Tại criticality, correlator liên thông (connected correlator, phần "mới" sau khi trừ đi phần suy ra được từ các correlator bậc thấp hơn) bậc 4 là $(e^{2r} - 1)(G_2^{(0)})^2$, xấp xỉ $2r(G_2^{(0)})^2$ khi r nhỏ (eq 3.56); correlator liên thông bậc 6 xấp xỉ $12r^2(G_2^{(0)})^3$ (eq 3.57). Các correlator liên thông bậc càng cao bị triệt giảm bởi luỹ thừa càng cao của CÙNG MỘT tham số nhỏ r: đây là cấu trúc "phân bậc" (hierarchical) làm cho việc cắt khai triển nhiễu loạn ở một bậc cố định là hợp lý khi r nhỏ.

Tự tính (ví dụ minh họa bằng số): lấy một mạng tuyến tính cụ thể với $L = 20$ lớp, $n = 200$ neuron mỗi lớp, chuẩn hoá $G_2^{(0)} = 1$. Tính $r = L/n = 20/200 = 0.1$ (tự tính). Giá trị đúng của correlator liên thông bậc 4 tại criticality: $e^{2r} - 1 = e^{0.2} - 1$. Khai triển Taylor $e^{0.2} = 1 + 0.2 + \frac{0.2^2}{2} + \frac{0.2^3}{6} + \frac{0.2^4}{24} \approx 1 + 0.2 + 0.02 + 0.001333 + 0.0000667 \approx 1.2214$ (tự tính), nên $e^{0.2} - 1 \approx 0.2214$ (tự tính). Xấp xỉ bậc nhất mà paper dùng cho r nhỏ là $2r = 0.2$. Sai số tương đối giữa hai giá trị: $(0.2214 - 0.2)/0.2214 \approx 9.7\%$ (tự tính). Nhận xét riêng (suy luận của người ghi chú, không phải kết luận paper nêu rõ bằng số): ngay ở $r = 0.1$, một tỉ lệ sâu/rộng nghe có vẻ "an toàn", xấp xỉ tuyến tính bậc nhất theo r đã lệch gần 10% so với giá trị đúng, cho thấy biên an toàn thực tế của khai triển nhiễu loạn hẹp hơn cảm giác "chỉ cần $r < 1$".

Criticality cho activation phi tuyến tổng quát (Chương 5, §5.1-5.2)

Với activation phi tuyến $\sigma(z)$ bất kỳ, kernel (ký hiệu K, chính là giới hạn vô hạn rộng của G) có đệ quy (eq 5.1): $K_{\alpha\beta}^{(\ell+1)} = C_b + C_W \langle \sigma_\alpha \sigma_\beta \rangle_{K^{(\ell)}}$, trong đó $\langle \cdot \rangle_K$ là kỳ vọng Gaussian với ma trận hiệp phương sai K. Khác với mạng tuyến tính, với activation phi tuyến xuất hiện một phức tạp mới: với hai mẫu đầu vào khác nhau $\alpha \neq \beta$, thành phần ngoài đường chéo $K_{\alpha\beta}$ bị trộn với các thành phần trên đường chéo $K_{\alpha\alpha}, K_{\beta\beta}$ (paper minh hoạ bằng ví dụ activation bậc hai $\sigma(z) = z^2$, eq 5.3, cho thấy cần giải 3 phương trình đệ quy phi tuyến liên kết nhau).

Một đầu vào: định nghĩa hàm trợ giúp $g(K) \equiv \langle \sigma(z)\sigma(z) \rangle_K$ (eq 5.5), đệ quy một-đầu-vào là $K_{00}^{(\ell+1)} = C_b + C_W\, g\left(K_{00}^{(\ell)}\right)$ (eq 5.4). Tuyến tính hoá quanh điểm cố định $K_{00}^\star$ (nghiệm của eq 5.7) cho **susceptibility song song (parallel susceptibility)** $\chi_\parallel(K) \equiv C_W g'(K)$ (eq 5.10), và điều kiện tới hạn đầu tiên là $\chi_\parallel(K_{00}^\star) = 1$ (eq 5.11). Paper liệt kê ba kiểu criticality có thể xảy ra tuỳ activation: (i) một đường điểm cố định được bảo toàn chính xác (giống mạng tuyến tính), xảy ra với activation scale-invariant (§5.2 dưới đây); (ii) suy giảm theo luật lũy thừa (power law) về $K^\star = 0$, xảy ra với tanh, sin (§5.3.3, không đọc chi tiết); (iii) suy giảm luật lũy thừa về $K^\star \neq 0$, xảy ra với SWISH, GELU (§5.3.4, không đọc chi tiết).

Hai đầu vào: phân tích ma trận kernel 2×2 (ứng với 2 mẫu) trong cơ sở $\gamma^{[0]}, \gamma^{[1]}, \gamma^{[2]}$ (eq 5.15-5.16), với ý nghĩa vật lý: $K_{[0]}$ là độ lớn trung bình ("khối tâm", eq 5.17), $K_{[1]} = R/2$ là nửa hiệu độ lớn hai đầu vào (eq 5.18), $K_{[2]} = D/4$ liên quan bình phương khoảng cách giữa hai đầu vào sau khi lan truyền qua ℓ lớp (eq 5.19). Sau một phép tính khá dài (khai triển quanh giới hạn hai đầu vào trùng nhau, dùng phân tích trị riêng của kernel), paper thu được ba đệ quy "bootstrapped" (giải lần lượt, cái sau dùng nghiệm cái trước, eq 5.46-5.48):

$$K_{00}^{(\ell+1)} = C_b + C_W\, g\left(K_{00}^{(\ell)}\right), \qquad \delta K_{[1]}^{(\ell+1)} = \chi_{\parallel}\left(K_{00}^{(\ell)}\right)\delta K_{[1]}^{(\ell)}, \qquad \delta\delta K_{[2]}^{(\ell+1)} = \chi_{\perp}\left(K_{00}^{(\ell)}\right)\delta\delta K_{[2]}^{(\ell)} + h\left(K_{00}^{(\ell)}\right)\left(\delta K_{[1]}^{(\ell)}\right)^2$$

với **susceptibility vuông góc (perpendicular susceptibility)** mới $\chi_{\perp}(K) \equiv C_W\langle\sigma'(z)\sigma'(z)\rangle_K$ (eq 5.51). Điều kiện tới hạn tổng quát bây giờ cần CẢ HAI điều kiện (eq 5.55): $\chi_{\parallel}(K^\star) = 1$ **và** $\chi_{\perp}(K^\star) = 1$, nhiều hơn một điều kiện so với mạng tuyến tính (chỉ cần $C_W = 1$), vì activation phi tuyến làm $\chi_{\parallel} \neq \chi_{\perp}$ nói chung.

Activation scale-invariant (§5.2): lớp activation thoả $\sigma(\lambda z) = \lambda\sigma(z)$ với mọi $\lambda > 0$ (eq 5.58), có dạng tuyến tính từng đoạn $\sigma(z) = a_+ z$ ($z \geq 0$) hoặc $a_- z$ ($z < 0$) (eq 5.59). Lớp này bao gồm cả activation tuyến tính ($a_+ = a_- = 1$) và ReLU ($a_+ = 1, a_- = 0$) như hai trường hợp riêng. Với lớp này, $g(K) = A_2 K$ (eq 5.60) với $A_2 \equiv (a_+^2 + a_-^2)/2$ (eq 5.61), nghĩa là g tuyến tính đúng trong K, và hệ quả là $\chi_{\parallel} = \chi_{\perp} = A_2 C_W \equiv \chi$ **không phụ thuộc** K (eq 5.62), đơn giản hoá cực lớn. Điểm tới hạn: $C_b = 0, C_W = 1/A_2$ (eq 5.67), cho một ĐƯỜNG điểm cố định (bất kỳ $K_{00}^\star$ nào cũng được, tham số hoá bởi norm đầu vào). Với ReLU, $A_2 = 1/2$, nên $C_W = 2$: đây chính xác là **Kaiming initialization** $(C_b, C_W)^{\text{critical}} = (0, 2)$, ở đây được suy ra từ nguyên lý criticality, khác với lập luận gốc của Kaiming (bảo toàn phương sai forward pass theo kiểu heuristic hơn).

4. Thí nghiệm và kết quả chính

Điểm đặc biệt quan trọng cần nói rõ ngay: trong Preface, các tác giả nói thẳng rằng họ chủ động KHÔNG đưa các biểu đồ kiểm chứng thực nghiệm (experimental verification plots) vào sách, mà giữ các kiểm chứng đó trong ”sổ tay máy tính riêng” của mình. Nói cách khác, cuốn sách (ở các phần đã đọc) hoàn toàn là suy diễn giải tích (closed-form derivations), KHÔNG có benchmark trên dữ liệu thật, KHÔNG có bảng số liệu thực nghiệm kiểu accuracy/loss trên MNIST hay ImageNet. Vì vậy mục này liệt kê các ”kết quả chính” dưới dạng công thức/định lý suy ra được, không phải số liệu đo thực nghiệm:

1. Điều kiện criticality cho mạng tuyến tính: $C_W = 1$ duy nhất (một điều kiện, vì activation tuyến tính không gây mixing giữa các thành phần kernel).
2. Kaiming/He initialization $(C_b, C_W) = (0, 2)$ được tái suy ra từ nguyên lý criticality cho ReLU, không cần giả định heuristic ban đầu của Kaiming et al.
3. Công thức giới hạn kép thống nhất hai chế độ ”Gaussian” và ”hỗn loạn” của mạng tuyến tính: $G_{2m}^{(L)} \to e^{m(m-1)r}(G_2^{(L)})^m$, với $r = L/n$ là tham số liên tục duy nhất.
4. Hai điều kiện criticality tổng quát cho activation phi tuyến bất kỳ: $\chi_{\parallel} = \chi_{\perp} = 1$, thay vì chỉ một điều kiện như mạng tuyến tính.
5. (Epilogue) Công thức độ phức tạp mô hình: mô tả Gaussian vô hạn rộng cần $O(N_D^2)$ số để đặc tả đầy đủ (eq ε.14); mô tả nearly-Gaussian độ rộng hữu hạn (cắt ở bậc $1/n$) cần $O(N_D^4)$ số (eq ε.15); tổng quát, độ chính xác bậc $1/n^k$ cần $O(N_D^{2k})$ số (eq ε.25), với N_D là tổng số mẫu train+test.
6. (Appendix A) Công thức tỉ lệ sâu/rộng tối ưu theo tiêu chí tối đa hoá mutual information giữa hai nhóm neuron: $r^\star = \frac{4}{20+3n_L} \cdot \frac{1}{\nu}$ (eq A.73), với ν là hằng số phụ thuộc activation ($\nu = 2/3$ cho lớp $K^\star = 0$, hoặc $\nu = 3A_4/A_2^2 - 1$ cho lớp scale-invariant, trong đó $A_4 \equiv (a_+^4 + a_-^4)/2$; riêng ReLU cho $\nu = 5$, tuyến tính cho $\nu = 2$).
7. (Appendix B) Với residual network, điều kiện tới hạn mới $C_W(\gamma) = \frac{1}{A_2}(1-\gamma^2)$ (lớp scale-invariant, eq B.20) hoặc $C_W(\gamma) = \frac{1}{\sigma_1^2}(1-\gamma^2)$ (lớp $K^\star = 0$, eq B.21), với γ là hệ số residual (độ mạnh của nhánh identity/skip connection). Lựa chọn ”mặc định” $\gamma = 1$ (không có residual hyperparameter được tinh chỉnh) làm điều kiện tới hạn KHÔNG THỂ thoả mãn, trừ trường hợp suy biến $C_W = 0$.

5. Phân tích / ablation

Sách không chạy ablation thực nghiệm (xem mục 4), nhưng có hai dạng ”ablation giải tích” (so sánh các lựa chọn kiến trúc/activation bằng công thức, không phải bằng số liệu đo):

So sánh các "universality class" của activation (§5.2 vs §5.3, chỉ §5.2 đọc chi tiết): activation scale-invariant (tuyến tính, ReLU) cho một đường điểm cố định được bảo toàn đúng, trong khi các activation khác (tanh, sin, SWISH, GELU, không đọc chi tiết trong ghi chú này) cho suy giảm theo luật lũy thừa về 0 hoặc về một giá trị khác 0. Paper gọi các activation cùng hành vi tại criticality là cùng một "universality class" (mượn thuật ngữ nhóm tái chuẩn hoá trong vật lý: nhiều hệ vi mô khác nhau có thể chảy về cùng một lý thuyết hiệu dụng ở quy mô lớn). Hệ quả thực tiễn được paper rút ra: mạng dùng các activation trong cùng một class sẽ có hành vi gần giống nhau ở các lớp sâu, bất kể chi tiết cụ thể của activation.

Ablation theo hệ số residual γ (Appendix B, §B.3): $\gamma = 0$ cho lại mạng MLP thường (vanilla); $\gamma \to 1$ (nhánh identity chiếm ưu thế hoàn toàn) đẩy aspect ratio tối ưu $r^\star(\gamma)$ lên vô hạn (do $\nu(\gamma) \to 0$ đơn điệu khi $\gamma \to 1$, cả cho lớp scale-invariant eq B.29 và lớp $K^\star = 0$ eq B.35), nhưng đồng thời $C_W(\gamma) \to 0$: nhánh trọng số ngẫu nhiên gần như bị "tắt". Đây là một phát hiện khá bất ngờ: lựa chọn $\gamma = 1$ "theo mặc định" (không tinh chỉnh gì thêm), vốn là lựa chọn phổ biến khi mô tả residual connection ở dạng $z^{(\ell+1)} = R(z^{(\ell)}) + z^{(\ell)}$ với trọng số hệ số 1 trước $z^{(\ell)}$, về lý thuyết LÀM HỎNG hoàn toàn điều kiện tới hạn, trừ khi có thêm heuristic khác bù lại (paper nêu rõ layer normalization, dùng trong kiến trúc transformer, là một heuristic thực tế cho mục đích này). Ngược hướng, công thức mutual information bậc $1/n^3$ (eq A.71) có hai số hạng ngược dấu (một dương bậc r^2, một âm bậc r^3), nên tồn tại một điểm tối ưu nội tại: cả mạng quá nông (r quá nhỏ) và mạng quá sâu (r quá lớn) đều kém hơn theo tiêu chí này, không phải "sâu hơn luôn tốt hơn".

6. Giới hạn

- **Không có kiểm chứng thực nghiệm trong sách** (đã nêu ở mục 4): mọi kết quả là suy diễn giải tích, các tác giả tự nhận không đưa số liệu thực nghiệm vào văn bản chính.
- **Giới hạn đọc của ghi chú này:** chỉ đọc 6 trong khoảng 14 chương/phụ lục của sách (Preface, Chương 0, Chương 3, Chương 5 §5.1-5.2, Epilogue, Appendix A, Appendix B); KHÔNG đọc Chương 1 (Pretraining), Chương 2 (Neural Networks, mô tả kiến trúc tổng quát hơn MLP), Chương 4, Chương 6 (RG flow), Chương 7-9 (Gradients, Backpropagation, NTK chi tiết), Chương 10 (Kernel Prediction), Chương 11 (Representation Learning), Chương ∞ (End of Training), và References/Index. Do đó ghi chú KHÔNG thể xác nhận hoặc phủ nhận liệu sách có phân tích kiến trúc convolutional hay self-attention ở các chương chưa đọc hay không; không đưa ra khẳng định nào về nội dung những chương đó.
- **Giả định đơn giản hoá tường minh của chính paper:** ở Chương 5, các siêu tham số C_b, C_W được giả định không đổi theo lớp (layer-independent) để giảm số siêu tham số cần xét; đây là lựa chọn mô hình hoá rõ ràng của tác giả, không phải thiếu sót.
- **Caveat do chính tác giả nêu về $r^\star$:** công thức aspect ratio tối ưu ở Appendix A (eq A.72-A.74) dựa HOÀN TOÀN trên phân phối PRIOR (trước huấn luyện), dùng tiêu chí thông tin (mutual information) chứ không phải hiệu năng sau huấn luyện thật. Paper tự ghi chú (footnote 30 của Appendix A) rằng giá trị $r^\star$ tính ra "có thể trông nhỏ", và khuyên người đọc đừng quên bản chất tiêu chí này. Số liệu tự tính ở mục 10 dưới đây cho thấy cụ thể $r^\star$ nhỏ tới mức nào.
- **Không phát hiện mâu thuẫn nội bộ rõ ràng** giữa văn bản chính và công thức trong các phần đã đọc (khác với một số bài khác trong series này, ví dụ math 19 hoặc math 21, nơi phát hiện được sai khác số liệu giữa bảng và công thức). Tuy nhiên vì phần lớn cuốn sách không được kiểm tra trực tiếp, đây chỉ là nhận xét trong phạm vi đã đọc, không phải khẳng định toàn sách nhất quán.

7. Liên hệ với các bài khác

- **math 26 (Poole et al., transient chaos) và math 27 (Schoenholz et al., Deep Information Propagation):** là tiền thân trực tiếp của khái niệm criticality/edge-of-chaos mà Chương 3 và Chương 5 của PDLT hệ thống hoá lại bằng ngôn ngữ correlator/susceptibility chặt chẽ hơn; PDLT tự ghi nhận hai dòng công trình này trong Chương 5.
- **math 22 (Deep Neural Networks as Gaussian Processes, NNGP):** chính là nội dung "Cách 1" (giới hạn vô hạn rộng nghiêm ngặt) ở mục 2 của ghi chú này; PDLT dùng lại trực tiếp kết quả NNGP làm

điểm khởi đầu $p^{\{0\}}$ của khai triển $1/n$.

- **math 23 (Neural Tangent Kernel):** xuất hiện trực tiếp trong Epilogue (eq ε.6-ε.9): đạo hàm bậc nhất của output theo tham số, $dz/d\theta$, chính là random feature, và kernel tương ứng với các feature đó chính là NTK Θ.
- **math 24 (Lazy Training, do người viết ghi chú này tự đọc và viết):** liên hệ trực tiếp đến giới hạn vô hạn rộng/NTK ở mục 2: ”lazy training” mô tả đúng chế độ mà PDLT gọi là giới hạn vô hạn rộng nghiêm ngặt, nơi mạng gần như tuyến tính hoá quanh khởi tạo và không học representation; khái niệm $\kappa_h(w_0)$ trong math 24 và aspect ratio r trong PDLT đều đóng vai trò ”tham số điều khiển mức độ phi tuyến/representation learning”, dù định nghĩa khác nhau.
- **math 19 (Intrinsic Dimension) và math 20/21 (Benefits of Depth, Expressive Power of Width):** cùng nhóm câu hỏi ”độ phức tạp hiệu dụng của mô hình là gì nếu không phải số tham số”; Epilogue của PDLT (công thức $O(N_D^{2k})$) đưa ra một câu trả lời định lượng khác hẳn với ”intrinsic dimension” của math 19 (đếm theo codimension tập nghiệm) nhưng cùng tinh thần phản bác trực giác ”đếm tham số thô”.
- **pruning content, file `01_1989_1993_optimal_brain_damage_surgeon.md`** (đã `ls` xác nhận tồn tại): chính Appendix A của PDLT, ở footnote 32, gợi ý liên hệ giữa kết quả tripartite information (thông tin được lưu trữ dư thừa giữa các neuron, eq A.75) với ý tưởng Optimal Brain Damage/Surgeon.
- **pruning content, file `08_2018_lottery_ticket_hypothesis.md`** (đã `ls` xác nhận tồn tại): cùng được PDLT nhắc tên trong footnote 32 của Appendix A, như một hướng diễn giải khả dĩ cho hiện tượng thông tin dư thừa giữa neuron.
- **llm content, file `01_2017_attention_is_all_you_need.md`** (đã `ls` xác nhận tồn tại): Appendix B của PDLT, footnote 1, nêu rõ residual connection là thành phần của ”transformer-based language models”, và §B.4 (Residual Building Blocks) đưa ra công thức tổng quát áp dụng được về nguyên tắc cho bất kỳ residual block nào, bao gồm kiến trúc attention.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận của người ghi chú, không phải khẳng định của paper. Dự án liên quan: compression/post-training cho LightOnOCR-3-0.8B, một VLM OCR có kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3600 token/trang (khoảng 1500 token văn bản + 1600-2100 token hình ảnh), chạy trên GPU ≤ 8GB với batch size 1.

Mức độ liên quan: **trung bình, chủ yếu ở mức khái niệm, không có công thức áp dụng trực tiếp được.** Lý do: PDLT (ở các phần đã đọc) chỉ phân tích MLP thuần và residual-MLP tại thời điểm khởi tạo với trọng số Gaussian độc lập theo lớp; GatedDeltaNet (gating theo kiểu SSM/linear attention) và full attention không nằm trong phạm vi các công thức đã đọc, nên không thể áp trực tiếp bất kỳ công thức criticality/aspect ratio nào của PDLT vào các lớp đó mà không cần công việc lý thuyết mở rộng đáng kể.

Dù vậy, ba ý khái niệm có thể hữu ích để suy nghĩ về dự án:

1. **Tỉ lệ sâu/rộng $r = L/n$ như một ”biến số ẩn” khi cắt lớp (depth pruning):** khi cắt bớt k lớp của LightOnOCR (dù là lớp GatedDeltaNet hay full-attention), về mặt khái niệm ta đang giảm L mà giữ n (chiều ẩn) không đổi, tức giảm r. PDLT gợi ý rằng tồn tại một ”vùng hiệu dụng” $0 < r \ll 1$ mà bên ngoài đó (quá nông, hoặc ở hướng ngược lại quá sâu) hành vi mạng trở nên kém dự đoán được hơn. Đây là một lăng kính khái niệm để hiểu TẠI SAO một số lớp có thể cắt được gần như không mất gì (nếu việc cắt chỉ đẩy r về gần 0 từ một vùng đã ”dư dả độ sâu”), còn cắt quá tay lại gây suy giảm phi tuyến đột ngột, nhưng đây KHÔNG phải một công cụ đo được trực tiếp từ PDLT cho kiến trúc hybrid này.
2. **Tinh chỉnh hệ số residual γ sau khi cắt lớp:** Appendix B cho thấy việc thêm/giữ residual connection đòi hỏi tinh chỉnh lại $C_W(\gamma)$ để duy trì criticality, và $\gamma = 1$ ”mặc định” về lý thuyết là sai lệch nếu không có heuristic bù (ví dụ layer normalization). Suy luận cho dự án: sau khi cắt bớt lớp hoặc thay

đổi cấu trúc residual quanh các block GatedDeltaNet/attention còn lại, có thể cần xem lại hệ số khởi tạo/scale của nhánh residual (không chỉ fine-tune ngay) để tránh tín hiệu bùng nổ/triệt tiêu ngay tại bước đầu của giai đoạn hồi phục (recovery fine-tuning), trước khi đổ lỗi hoàn toàn cho ”mất năng lực do cắt”.

3. **”Độ phức tạp mô hình ≠ số tham số” (Epilogue):** khi đánh giá một phiên bản đã pruning/distill của LightOnOCR, quan điểm của PDLT nhắc ta rằng hành vi tổng quát hoá (generalization) của mô hình nhỏ hơn không nên chỉ đo bằng số tham số còn lại, mà (về nguyên tắc, dù PDLT chỉ chứng minh cho MLP) còn phụ thuộc cấu trúc các ”data-dependent coupling” hiệu dụng của mô hình. Đây là một khung suy nghĩ, không phải một chỉ số có thể tính ngay cho GatedDeltaNet.

Những gì PDLT KHÔNG giúp được trực tiếp cho dự án: không có công thức nào trong các phần đã đọc áp dụng được cho token hình ảnh/văn bản hỗn hợp, cho cơ chế gating của GatedDeltaNet, hay cho ràng buộc GPU ≤ 8GB/batch size 1; mọi liên hệ ở trên chỉ là phép loại suy khái niệm.

9. Thí nghiệm gợi ý

Cả 5 thí nghiệm dưới đây chỉ cần forward pass hoặc fine-tune rất ngắn, khả thi trên một GPU 8GB với batch size 1 (đúng ràng buộc của dự án).

1. **Đo ”độ bùng nổ/triệt tiêu” của preactivation theo lớp tại khởi tạo.** Lấy một vài block MLP/projection nhỏ trong LightOnOCR (hoặc dựng lại một MLP đồ chơi cùng độ rộng để so sánh), đo phương sai trung bình của preactivation ở mỗi lớp qua một forward pass batch size 1. Kỳ vọng: nếu khởi tạo ”gần tới hạn” (như Kaiming cho ReLU, tương ứng $C_W = 2$ theo PDLT), phương sai giữa các lớp tương đối ổn định; nếu lệch xa, phương sai tăng/giảm rõ rệt theo lớp. Diễn giải: cho biết mô hình gốc có đang ở gần vùng ”hiệu dụng” theo trực giác criticality hay không, trước khi can thiệp pruning.
2. **Tính tỉ lệ $r = L/n$ thực tế cho từng nhóm lớp (GatedDeltaNet vs full-attention) của LightOnOCR.** Chỉ cần đếm số lớp L và chiều ẩn n cho mỗi nhóm kiến trúc. Kỳ vọng: hai nhóm có thể có r khác nhau đáng kể. Diễn giải: không dùng được công thức $r^\star$ chính xác của PDLT (vì đó suy cho MLP thường), nhưng có thể dùng r như một chỉ số tương đối để so sánh ”mức độ sâu tương đối” giữa hai nhánh kiến trúc, tham khảo khi quyết định nên ưu tiên cắt lớp ở nhánh nào.
3. **Sweep hệ số residual γ ngay sau khi cắt lớp, đo ổn định tín hiệu TRƯỚC khi fine-tune.** Sau khi xoá k lớp liên tiếp và nối lại bằng một residual connection có hệ số $\gamma \in \{0, 0.5, 0.9, 0.99, 1.0\}$ nhân vào nhánh identity/skip, đo norm trung bình của activation tại lớp ngay sau điểm cắt, trên một batch nhỏ dữ liệu OCR thật. Kỳ vọng: theo Appendix B, γ gần 1 không tinh chỉnh thêm có thể gây lệch khỏi hành vi ”ổn định” quan sát được ở mô hình gốc. Diễn giải: nếu norm activation thay đổi đột ngột theo γ, đó là tín hiệu nên tinh chỉnh scale tại điểm nối trước khi fine-tune, thay vì fine-tune ngay và hy vọng mô hình tự bù.
4. **Đo dao động giữa các lần forward (instantiation-to-instantiation fluctuation) như một proxy thực nghiệm cho tính ”Gaussian/hỗn loạn”.** Chạy nhiều lần forward pass trên cùng một batch OCR với các mặt nạ dropout khác nhau (hoặc nhiều bản sao mô hình với seed khởi tạo khác nhau nếu có thể huấn luyện lại một phần nhỏ), đo độ lệch giữa các lần của một đại lượng tổng hợp như norm trung bình activation ở lớp cuối. Kỳ vọng: dao động lớn gợi ý mô hình/cấu hình đang ở vùng ”r lớn, nhiều tương tác neuron” theo trực giác PDLT. Diễn giải: dùng để so sánh TƯƠNG ĐỐI giữa bản gốc và bản đã pruning, xem việc cắt lớp có làm tăng độ ”nhạy cảm, khó đoán” của mô hình hay không, trước khi đổ nhiều công fine-tune.
5. **So sánh gradient/NTK-proxy trước và sau pruning trên một batch OCR nhỏ (liên hệ math 23/24).** Tính cosine similarity giữa gradient của vài mẫu OCR khác nhau (một proxy rẻ cho NTK) trên mô hình gốc và mô hình đã cắt lớp, với batch size 1 lần lượt cho từng mẫu. Kỳ vọng: nếu độ tương tự gradient giữa các mẫu thay đổi mạnh sau pruning, mô hình đã bị đẩy ra khỏi chế độ ”giống nhau về cấu trúc kernel” với bản gốc. Diễn giải: giúp đoán trước liệu giai đoạn recovery fine-tuning cần học lại representation sâu (tốn nhiều bước) hay chỉ cần hiệu chỉnh tuyến tính nhẹ (ít bước), tương tự tinh thần

”lazy vs feature learning” của math 24.

10. Tóm tắt công thức / khái niệm

- **Aspect ratio:** $r \equiv L/n$ (eq 0.19/ε.1); tham số nhiễu loạn duy nhất của toàn lý thuyết hiệu dụng.
- **Khai triển $1/n$:** $p(f^\star) = p^{\{0\}}(f^\star) + \frac{1}{n}p^{\{1\}}(f^\star) + O(1/n^2)$ (eq 0.14-0.15); $p^{\{0\}}$ là NNGP/NTK (vô hạn rộng), $p^{\{1\}}/n$ mang lại representation learning.
- **Criticality mạng tuyến tính:** $C_W = 1$ (eq 3.2 vùng lân cận, suy từ 3.14-3.15).
- **Non-Gaussianity bậc dẫn đầu tại criticality (mạng tuyến tính):** $\frac{2(\ell-1)}{n}(G_2^{(\ell)})^2$ (eq 3.28), tỉ lệ với r.
- **Giới hạn kép, hợp nhất Gaussian và hỗn loạn:** $G_{2m}^{(L)} \to e^{m(m-1)r}(G_2^{(L)})^m$ (eq 3.55).
- **Susceptibility song song/vuông góc (activation phi tuyến tổng quát):** $\chi_\parallel(K) = C_W g'(K)$ (eq 5.10), $\chi_\perp(K) = C_W\langle\sigma'\sigma'\rangle_K$ (eq 5.51); điều kiện tới hạn tổng quát $\chi_\parallel = \chi_\perp = 1$ (eq 5.55).
- **Criticality cho activation scale-invariant:** $(C_b, C_W)^{\text{critical}} = (0, 1/A_2)$ với $A_2 = (a_+^2 + a_-^2)/2$ (eq 5.67); ReLU cho $(0, 2)$ = Kaiming init.
- **Độ phức tạp mô hình theo bậc khai triển:** $O(N_D^{2k})$ số để đặc tả đầy đủ phân phối ở độ chính xác $1/n^k$ (eq ε.25); vô hạn rộng ($k = 0$, thực ra $k = 1$ theo cách đếm của paper cho Gaussian) là $O(N_D^2)$ (eq ε.14), $1/n$ (nearly-Gaussian) là $O(N_D^4)$ (eq ε.15).
- **Aspect ratio tối ưu theo mutual information (Appendix A):** $r^\star = \frac{4}{20+3n_L} \cdot \frac{1}{\nu}$ (eq A.73), với $\nu = 2/3$ (lớp $K^\star = 0$) hoặc $\nu = 3A_4/A_2^2 - 1$ (lớp scale-invariant, cho $\nu = 2$ với tuyến tính, $\nu = 5$ với ReLU).
- **Criticality cho residual network:** $C_W(\gamma) = \frac{1-\gamma^2}{A_2}$ (lớp scale-invariant, eq B.20); $r^\star(\gamma)$ tăng đơn điệu theo γ (eq B.36), giải thích vì sao residual connection cho phép huấn luyện mạng sâu hơn.

Tự tính (ví dụ số cho $r^\star$): áp dụng eq A.73 cho ReLU ($\nu = 5$) với một lớp rộng $n_L = 256$ (giả định, chỉ để minh hoạ công thức): $r^\star = \frac{4}{20+3\times256} \times \frac{1}{5} = \frac{4}{20+768} \times \frac{1}{5} = \frac{4}{788} \times \frac{1}{5} \approx 0.005076 \times 0.2 \approx 0.0010152$ (tự tính). Suy ra số lớp ”tối ưu” theo tiêu chí này: $L^\star = r^\star \times n_L \approx 0.0010152 \times 256 \approx 0.26$ lớp (tự tính), tức là CHƯA ĐẦY MỘT LỚP. Đây khớp với đúng caveat mà paper tự nêu ở footnote 30 Appendix A: giá trị $r^\star$ (và do đó $L^\star$) từ tiêu chí mutual-information-trên-prior là rất nhỏ, và không nên hiểu đây là ”số lớp nên dùng trong thực tế”, mà chỉ là một tính chất hình thức của tham số khai triển nhiễu loạn khi tối ưu hoá một tiêu chí thông tin trừu tượng trước khi huấn luyện.

11. Kết luận ngắn

PDLT xây một khung lý thuyết hiệu dụng thống nhất cho mạng nơ-ron sâu bằng cách nhận ra tỉ lệ độ sâu/rộng $r = L/n$ là tham số nhiễu loạn tự nhiên duy nhất, nối liền hai giới hạn trước đó tách rời (vô hạn rộng/NNGP/NTK và edge-of-chaos/signal propagation) thành một phổ liên tục. Trong các phần đã đọc (Chương 0, 3, 5 §5.1-5.2, Epilogue, Appendix A, B), đóng góp cụ thể gồm: tổng quát hoá điều kiện criticality cho activation phi tuyến bất kỳ (hai susceptibility $\chi_\parallel, \chi_\perp$), tái suy Kaiming init từ nguyên lý vật lý, định nghĩa lại ”độ phức tạp mô hình” theo số coupling phụ thuộc dữ liệu thay vì số tham số, và chỉ ra bằng công thức rằng residual connection dịch chuyển vùng ”hiệu dụng là sâu” lên các giá trị r lớn hơn. Hạn chế lớn nhất khi dùng ghi chú này: đây là một cuốn sách rất dài, ghi chú chỉ đọc có chọn lọc khoảng 6/14 phần, và sách tự nhận không có kiểm chứng thực nghiệm, nên mọi kết quả nêu trên cần được hiểu là định lý/công thức giải tích, không phải số liệu đo đạc.

Nguồn gốc: https://arxiv.org/abs/2106.10165

Bài 26

Exponential Expressivity in Deep Neural Networks through Transient Chaos (2016)

Tác giả: Ben Poole (Stanford University), Subhaneil Lahiri (Stanford University), Maithra Raghu (Google Brain and Cornell University), Jascha Sohl-Dickstein (Google Brain), Surya Ganguli (Stanford University).
Venue / arXiv: arXiv:1606.05340 (v2, 17/06/2016); paper không nêu venue hội nghị hay tạp chí nào trong toàn văn (đã kiểm bằng grep các từ khóa ”conference”, ”workshop”, ”published as”, ”proceedings”, ”NIPS”, ”NeurIPS”, ”ICLR”, ”ICML” trên toàn bộ file text trích từ PDF, không tìm thấy kết quả nào).
Nguồn: https://arxiv.org/abs/1606.05340
Chủ đề: lý thuyết mean-field cho sự lan truyền tín hiệu qua mạng nơ-ron ngẫu nhiên (chưa huấn luyện) nhiều lớp; ranh giới order-to-chaos (trật tự - hỗn loạn) theo phương sai trọng số/bias; dùng hình học Riemann (độ cong, độ dài Gauss) để định lượng độ phức tạp của hàm mà mạng tính được theo độ sâu.
Vị trí trong lộ trình: đọc cùng/trước math 27 (Deep Information Propagation), vì math 27 lấy đúng khung lý thuyết mean-field của bài này làm nền (”Background”) rồi mở rộng thêm depth-scale và lý thuyết backprop. Trong nhóm math, bài này còn liên hệ math 18 (Shattered Gradients), math 20 (Benefits of depth), math 21 (Expressive Power, View from Width), math 22 (Deep NN as Gaussian Processes); chi tiết ở mục 7.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (đọc toàn văn bản text trích từ PDF arXiv, bản v2 ngày 17/06/2016, kể cả phụ lục A-F). Các phép tính đánh dấu ”tự tính” là của người ghi chú, không phải số có sẵn trong paper. Mục ”Ứng dụng cho dự án” là suy luận (inference), không phải kết luận của paper.

1. Bối cảnh

Trước năm 2016, câu hỏi ”tại sao mạng sâu biểu diễn hàm phức tạp tốt hơn mạng rộng/nông” đã có một số lời giải, nhưng hầu hết đi theo hướng hạn chế loại phi tuyến để có thể dùng công cụ toán đặc thù: Montufar et al. (2014) đếm số vùng tuyến tính (linear regions) của mạng ReLU; Delalleau và Bengio (2011), Martens et al. (2013) dùng mạng sum-product; Eldan và Shamir (2015), Telgarsky (2015) dùng các họ hàm Boolean hoặc hàm dao động đặc biệt. Những phân tích này đúng, nhưng gắn chặt với một dạng phi tuyến cụ thể, khó tổng quát hoá.

Bài này chọn một hướng khác, không phụ thuộc dạng phi tuyến cụ thể: xét mạng nơ-ron **ngẫu nhiên, chưa huấn luyện** (random, untrained) như một hệ động lực học, dùng lý thuyết mean-field (vật lý thống kê của hệ hỗn loạn/mất trật tự) kết hợp hình học Riemann để đo độ phức tạp của ánh xạ mà mạng tính được, khi tín hiệu lan truyền qua nhiều lớp. Ý tưởng nền: nếu ngay cả một mạng ngẫu nhiên (không học gì) đã có thể tính hàm phức tạp theo cấp số nhân với độ sâu, thì độ sâu tự nó là nguồn sức mạnh biểu diễn, độc lập với việc huấn luyện.

Thiết lập: mạng truyền thẳng (feedforward) kết nối đầy đủ, có D lớp trọng số $W^1, \dots, W^D$ và $D+1$ lớp hoạt động $x^0, \dots, x^D$, lớp l có N_l nơ-ron. Động lực truyền thẳng: $x^l = \phi(h^l)$, $h^l = W^l x^{l-1} + b^l$, với ϕ là hàm phi tuyến áp dụng theo từng thành phần. Trọng số W^l_{ij} lấy i.i.d. từ Gauss 0 trung bình, phương sai σ_w^2/N_{l-1} (chia theo chiều rộng lớp trước để giữ tổng đầu vào mỗi nơ-ron ở mức $O(1)$ không phụ thuộc N_{l-1}); bias b^l_i lấy i.i.d. Gauss 0 trung bình, phương sai σ_b^2.

Với N_l lớn, phân phối thực nghiệm của h^l_i trên các nơ-ron i hội tụ về Gauss (do là tổng nhiều biến ngẫu nhiên độc lập), nên toàn bộ động lực lan truyền một đầu vào có thể tóm gọn trong một đại lượng vô hướng: độ dài bình phương chuẩn hoá $q^l = \frac{1}{N_l}\sum_i (h^l_i)^2$.

2. Ý tưởng chính

Ý tưởng cốt lõi gồm bốn phần, nối tiếp nhau:

(a) Order-to-chaos cho một cặp điểm. Với hai đầu vào $x^0_{;1}, x^0_{;2}$, mối quan hệ giữa chúng ở lớp l được đo bằng hệ số tương quan c^l_{12}. Khi $l \to \infty$, c^l_{12} hội tụ về điểm cố định $c^*(\sigma_w, \sigma_b)$. Độ ổn định của điểm cố định $c^* = 1$ (hai điểm trùng nhau) được quyết định bởi hệ số giãn (stretch factor) χ_1: nếu $\chi_1 < 1$, mọi cặp điểm hội tụ về nhau khi qua nhiều lớp (**pha trật tự**, ordered); nếu $\chi_1 > 1$, các điểm gần nhau tách xa nhau theo cấp số nhân (**pha hỗn loạn**, chaotic), xuất hiện điểm cố định mới $c^* < 1$. Đường $\chi_1(\sigma_w, \sigma_b) = 1$ là ranh giới pha. $\log \chi_1$ chính là tương tự của số mũ Lyapunov trong lý thuyết hệ động lực học, nhưng vì mạng truyền thẳng chỉ có độ sâu D hữu hạn (bị "cắt cụt"), hiện tượng này được gọi là **transient chaos** (hỗn loạn tạm thời/nhất thời), chứ không phải hỗn loạn đúng nghĩa của hệ lặp vô hạn.

(b) Hình học Riemann cho một đa tạp (manifold). Thay vì chỉ xét 2 điểm rời rạc, bài báo xét một đường cong 1 chiều (ví dụ hình tròn) trong không gian đầu vào, lan truyền qua mạng. Ở mỗi lớp, đường cong có độ dài Euclid L^E và độ cong ngoại tại (extrinsic curvature) $\kappa(\theta)$; kết hợp hai đại lượng này qua ánh xạ Gauss (Gauss map, gửi mỗi điểm tới vector tiếp tuyến đơn vị của nó trên mặt cầu S^{N-1}) cho ra độ dài Grassmannian L^G, một thước đo độ phức tạp "tổng hợp" của đường cong. Kết quả bất ngờ: trong pha hỗn loạn, L^E và L^G tăng theo cấp số nhân với độ sâu, nhưng độ cong κ **không** suy biến về 0 như trong một phép co giãn tuyến tính đơn giản (scale-up tuyến tính giữ L^G không đổi vì độ dài tăng bù cho độ cong giảm); ở đây độ cong hội tụ về một điểm cố định $\bar{\kappa}^*$ khác không. Nói cách khác, đường cong không chỉ "dài ra" mà còn "ngoằn ngoèo" liên tục ở mọi độ sâu, nên độ phức tạp hình học tăng theo cấp số nhân thật sự, không phải chỉ là giãn phẳng.

(c) Mạng nông không thể đạt độ phức tạp này. Định lý 1 của paper chứng minh: với một mạng 1 lớp ẩn rộng N_1, độ dài Euclid tối đa có thể đạt được chỉ tăng tuyến tính theo N_1 (nên độ dài chuẩn hoá theo nơ-ron chỉ tăng như $\sqrt{N_1}$), bất kể chọn trọng số/bias thế nào. Trong khi đó mạng sâu ở pha hỗn loạn đạt tăng trưởng mũ theo độ sâu. Đây là bằng chứng định lượng cho "độ sâu mạnh hơn độ rộng" theo đúng nghĩa hình học.

(d) Ranh giới quyết định cũng cong theo cấp số nhân. Nếu đặt một bộ phân loại tuyến tính ở lớp cuối, ranh giới quyết định (decision boundary) là một siêu phẳng ở lớp cuối, nhưng khi lan truyền ngược (backpropagate) ranh giới này về lớp đầu vào, nó trở thành một đa tạp cong; các độ cong chính (principal curvatures) của đa tạp này tăng theo cấp số nhân với độ sâu. Mạng sâu, ngay cả khi ngẫu nhiên, đã "gấp" không gian đầu vào đủ nhiều để một siêu phẳng đơn giản ở đầu ra tương ứng với một ranh giới rất phức tạp ở đầu vào.

Ngoài bốn ý trên, phụ lục F còn chỉ ra tính đối ngẫu (duality): thay vì cố định trọng số và thay đổi đầu vào, nếu cố định đầu vào và thay đổi (nhiễu) trọng số ở một lớp giữa, hàm mà mạng tính được cũng decorrelate (mất tương quan) theo cấp số nhân với độ sâu tính từ lớp bị nhiễu đến lớp cuối, y hệt cơ chế input-chaos. Hiện tượng này được gọi là **weight chaos**.

3. Phương pháp chi tiết

Length map (bản đồ độ dài). Với một đầu vào, q^l tuân theo công thức lặp:

$$q^l = \mathcal{V}(q^{l-1} \mid \sigma_w, \sigma_b) \equiv \sigma_w^2 \int \mathcal{D}z\, \phi\Big(\sqrt{q^{l-1}}\, z\Big)^2 + \sigma_b^2,$$

với $\mathcal{D}z$ là đo Gauss chuẩn. Với $\sigma_b = 0$, $\sigma_w < 1$: điểm cố định duy nhất là $q^* = 0$ (mạng co tín hiệu về gốc). Với $\sigma_w > 1, \sigma_b = 0$: $q^* = 0$ mất ổn định, xuất hiện điểm cố định không-không ổn định. Với $\sigma_b \neq 0$ bất kỳ: luôn có một điểm cố định ổn định khác không.

Correlation map (bản đồ tương quan). Với 2 đầu vào, hiệp phương sai q^l_{12} (và hệ số tương quan $c^l_{12} =$

$q^l_{12}/\sqrt{q^l_{11}q^l_{22}}$) tuân theo:

$$q^l_{12} = \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi(u_1)\phi(u_2) + \sigma_b^2, \quad u_1 = \sqrt{q^{l-1}_{11}}\, z_1,\ u_2 = \sqrt{q^{l-1}_{22}} \left(c^{l-1}_{12} z_1 + \sqrt{1-(c^{l-1}_{12})^2}\, z_2 \right).$$

Hệ số giãn tại điểm tới hạn $c = 1$:

$$\chi_1 \equiv \left. \frac{\partial c^l_{12}}{\partial c^{l-1}_{12}} \right|_{c=1} = \sigma_w^2 \int \mathcal{D}z \left[\phi'\left(\sqrt{q^*}\, z \right) \right]^2.$$

Nhận xét quan trọng về tốc độ hội tụ (không phải công thức, mà là quan sát thực nghiệm của paper, dùng ở mục 4): bản đồ tương quan nằm gần đường chéo (unity line) hơn bản đồ độ dài, nên c^l_{12} hội tụ về c^* chậm hơn nhiều so với q^l hội tụ về q^*.

Hình học Riemann của đường cong lan truyền. Với đường $h(\theta)$, vector tốc độ $v(\theta) = \partial_\theta h(\theta)$, gia tốc $a(\theta) = \partial_\theta v(\theta)$; đường tròn osculating (tiếp xúc) tại θ có bán kính $R(\theta)$, độ cong $\kappa(\theta) = 1/R(\theta)$. Metric Euclid $g^E = v \cdot v$ cho độ dài $L^E = \int \sqrt{g^E}\, d\theta$. Ánh xạ Gauss gửi θ tới vector tiếp tuyến đơn vị $\hat{v}(\theta) \in S^{N-1}$; metric Gauss $g^G = (\partial_\theta \hat{v}) \cdot (\partial_\theta \hat{v}) = \kappa^2 g^E$ cho độ dài $L^G = \int \sqrt{g^G}\, d\theta$.

Chuẩn hoá theo chiều rộng ($\bar{g}^E = g^E/N$, $\bar{\kappa} = \sqrt{N}\kappa$), paper suy ra công thức lặp:

$$\bar{g}^{E,l} = \chi_1\, \bar{g}^{E,l-1}, \qquad (\bar{\kappa}^l)^2 = \frac{\chi_2}{\chi_1^2} + \frac{1}{\chi_1}(\bar{\kappa}^{l-1})^2, \qquad \chi_2 = \sigma_w^2 \int \mathcal{D}z \left[\phi''\left(\sqrt{q^*}\, z \right) \right]^2.$$

Đọc công thức này: mỗi lớp nhân $\bar{g}^E$ với χ_1 (tăng mũ nếu $\chi_1 > 1$); độ cong $\bar{\kappa}^2$ của lớp trước bị suy giảm hệ số $1/\chi_1$ nhưng được "bù" thêm một lượng χ_2/χ_1^2 sinh ra từ độ cong của chính hàm phi tuyến ϕ (qua ϕ''). Hai hiệu ứng này cân bằng ở một điểm cố định $\bar{\kappa}^* \neq 0$, khác hẳn phép giãn tuyến tính đơn giản (ở đó κ giảm đúng bằng tỉ lệ nghịch với độ giãn, giữ L^G không đổi).

Định lý 1 (chặn trên cho mạng nông). Nếu ϕ đơn điệu không giảm với "dynamic range" $R = \max_h \phi(h) - \min_h \phi(h)$ hữu hạn, và đường cong đầu vào $x^0(\theta)$ có mọi phép chiếu 1 chiều của $\partial_\theta x^0(\theta)$ đổi dấu không quá s lần, thì với **mọi** cách chọn W^1, b^1: $L^E(x^1) \leq N_1(1+s)R$. Với đường tròn ($s = 1$) và $\phi = \tanh$ ($R = 2$), paper rút ra dạng đơn giản cho độ dài chuẩn hoá: $\bar{L}^E \leq 2\sqrt{N_1}$.

Độ cong của ranh giới quyết định. Với phân loại tuyến tính $y = \mathrm{sgn}(\beta \cdot x^D - \beta_0)$, ranh giới ở lớp đầu vào là nghiệm của $G(x^0) = \beta \cdot x^D(x^0) - \beta_0 = 0$; các độ cong chính $\kappa_1 \geq \cdots \geq \kappa_{N-1}$ là trị riêng của Hessian chuẩn hoá $H = \|\nabla G\|^{-1} P\, \partial^2 G/\partial x \partial x^T\, P$ chiếu lên mặt phẳng tiếp tuyến (P là toán tử chiếu). Paper tính số các κ_i này theo độ sâu bằng mô phỏng (không có công thức closed-form đơn giản như length/correlation map).

Weight chaos (phụ lục F). Xét họ mạng tham số hoá bởi $\Delta \in [-1, 1]$, chỉ trọng số lớp 2 thay đổi theo Δ: $W^2(\Delta) = \sqrt{1-|\Delta|}\, W + \sqrt{|\Delta|}\, dW$ (W, dW i.i.d. cùng phương sai σ_w^2). Với một đầu vào cố định (hình tròn ở bán kính q^*), tương quan hàm số $C^l(\Delta)$ giữa mạng tại $\Delta = 0$ và tại Δ tuân theo công thức lặp cùng dạng toán học với correlation map ở phần (b), nên trong pha hỗn loạn, $C^l(\Delta)$ cũng giảm theo cấp số nhân với độ sâu tính từ lớp 2: một nhiễu nhỏ ở trọng số lớp giữa gây ra thay đổi lớn, tăng mũ theo số lớp còn lại, trong không gian hàm mà mạng tính được.

4. Thí nghiệm và kết quả chính

Tất cả thí nghiệm dùng mạng sigmoidal ($\phi = \tanh$), chiều rộng 1000 nơ-ron mỗi lớp, so sánh dự đoán lý thuyết (đường liền) với mô phỏng mạng thật (điểm rời rạc).

- **Hình 1 (length map):** với $\sigma_b = 0{,}3$ cố định và 3 giá trị σ_w khác nhau, đường lý thuyết của bản đồ độ dài khớp tốt với mô phỏng; q^l hội tụ về điểm cố định q^* rất nhanh, **thường chỉ trong khoảng 4 lớp**,

không phụ thuộc điều kiện khởi đầu.

- **Hình 2 (correlation map):** cùng $\sigma_b = 0{,}3$ và các σ_w ở Hình 1, mô phỏng với $N_l = 1000$; tương quan c_{12}^l hội tụ về c^* **chậm hơn nhiều, khoảng 20 lớp**, đúng như lý giải ở mục 3 (bản đồ tương quan gần đường chéo hơn). Mặt phẳng (σ_w, σ_b) được chia bởi đường $\chi_1 = 1$ thành vùng trật tự ($\chi_1 < 1$, $c^* = 1$) và vùng hỗn loạn ($\chi_1 > 1$, $c^* < 1$).
- **Hình 3 (lan truyền hình tròn):** một hình tròn (tạo từ 2 vector cơ sở trực chuẩn u_0, u_1) được lan truyền qua 3 mạng ngẫu nhiên với σ_w khác nhau, $\sigma_b = 0{,}3$ cố định. Ở σ_w lớn (chế độ hỗn loạn), phổ trị riêng (singular values) của hình chiếu phẳng dần và đường cong "rối" hơn rõ rệt khi nhìn ở lớp 5 và lớp 10 (chiếu lên 3 thành phần chính đầu). Tự tương quan $c_{12}^l(\Delta\theta)$ đo theo lớp khớp định lượng với lý thuyết lặp từ công thức tương quan ở mục 3 (đường lý thuyết trùng số liệu mô phỏng, kể cả độ lệch chuẩn theo θ).
- **Hình 4 (độ cong/độ dài):** mạng 1000 nơ-ron, xác nhận định lượng rằng $\bar{g}^E$ tăng và $\bar{\kappa}$ hội tụ về điểm cố định khác không (không suy biến về 0), đúng như công thức (8)-(9) ở mục 3; lý thuyết và mô phỏng khớp nhau trên các panel C-E.
- **Hình 5 (sâu thắng rộng):** dùng chế độ hỗn loạn cụ thể $\sigma_b = 0{,}3, \sigma_w = 4{,}0$, mạng rộng 1000, $\phi = \tanh$. (A) hoạt động của 4 nơ-ron đầu ra theo θ ngày càng phức tạp khi mạng sâu hơn. (B) hồi quy tuyến tính lên một hàm mục tiêu ngẫu nhiên khớp tốt hơn rõ rệt với mạng sâu hơn. (C) phân rã lỗi theo tần số: mạng nông không bắt được thành phần tần số cao của hàm ngẫu nhiên dù mạng sâu làm được. (D) **tăng chiều rộng mạng 1 lớp ẩn lên tới 10.000 nơ-ron không làm giảm lỗi ở tần số cao** - xác nhận thực nghiệm trực tiếp cho Định lý 1 (độ rộng chỉ giúp theo $\sqrt{N_1}$, độ sâu giúp theo cấp số nhân).
- **Hình 4G (độ cong ranh giới quyết định):** các độ cong chính lớn nhất và nhỏ nhất (4 cặp $\kappa_{\pm 1}, \ldots, \kappa_{\pm 4}$) của ranh giới quyết định, tính số bằng mô phỏng, tăng theo cấp số nhân khi lan truyền ngược về lớp đầu vào.

5. Phân tích / ablation

Đây là paper lý thuyết trên mạng ngẫu nhiên (không huấn luyện trên dữ liệu thật), nên không có ablation kiểu "bỏ một thành phần kiến trúc rồi đo accuracy" như các paper thực nghiệm khác. Thay vào đó, các "ablation" ở đây là quét tham số một cách có hệ thống:

- **Quét σ_w, σ_b:** Hình 1C/D và Hình 2C/D vẽ q^*, số lớp cần để hội tụ 1%, c^*, và χ_1 như hàm của toàn bộ mặt phẳng (σ_w, σ_b), không chỉ 3 điểm minh hoạ ở Hình 1A/2A, cho thấy biên $\chi_1 = 1$ là một đường cong trơn trong mặt phẳng này, không phải một hiện tượng cô lập ở vài giá trị cụ thể.
- **Quét chiều rộng mạng nông:** Hình 5D là "ablation" rõ nhất về mặt khái niệm: tăng N_1 từ nhỏ lên tới 10.000, lỗi ở tần số cao gần như không giảm, trực tiếp kiểm chứng Định lý 1 (chặn trên $\sqrt{N_1}$, không phải tăng theo độ sâu).
- **Vai trò của bias:** so sánh định tính case $\sigma_b = 0$ (chỉ có điểm cố định $q^* = 0$ khi $\sigma_w < 1$, mạng co tín hiệu về gốc) với case $\sigma_b \neq 0$ (luôn có điểm cố định khác không), cho thấy bias đóng vai trò "chống suy biến" tín hiệu ngay cả khi trọng số nhỏ.
- **Paper không** chạy ablation trên loại phi tuyến ϕ khác tanh một cách hệ thống trong văn bản chính; phần lý thuyết (length/correlation map, χ_1) viết tổng quát cho ϕ đơn điệu bất kỳ, nhưng mọi hình vẽ minh hoạ số liệu đều dùng tanh.

6. Giới hạn

- Toàn bộ phân tích áp dụng cho mạng **ngẫu nhiên, chưa huấn luyện**; paper tự coi đây là một "null model" (mô hình nền) để so sánh với mạng đã huấn luyện, không trực tiếp nói điều gì xảy ra sau khi học.
- Chỉ xét mạng **kết nối đầy đủ** (fully-connected) với trọng số/bias i.i.d. Gauss; không mở rộng cho CNN, RNN hay kiến trúc có trọng số chia sẻ/cấu trúc đặc biệt trong bài này (paper tiếp theo, math 27, cũng tự nêu đúng hạn chế này).

- Lý thuyết dựa trên xấp xỉ mean-field ở giới hạn chiều rộng lớn ($N_l \to \infty$, giả thiết self-averaging); với N_l hữu hạn (ví dụ $N_l = 1000$ trong mô phỏng), chỉ là xấp xỉ tốt, không phải đẳng thức chính xác, dù khớp rất tốt trong các hình vẽ.
- Định lý 1 yêu cầu ϕ đơn điệu với dynamic range R **hữu hạn**; với các hàm không chặn như ReLU, $R = \infty$ nên chặn trên trở nên vô nghĩa theo đúng công thức đã nêu, dù paper không bàn trực tiếp vấn đề này trong văn bản chính (math 27 sau này mới nêu rõ hạn chế tương tự của khung mean-field với "unbounded activations" như ReLU).
- Paper tự nhận đây chỉ là "khảo sát sơ bộ" (preliminary exploration); việc đếm số lượng mẫu phân loại đạt được (classification patterns) được nhường hẳn cho bài đồng hành (companion paper) [16] = Raghu, Poole, Kleinberg, Ganguli, Sohl-Dickstein, "On the expressive power of deep neural networks" (arXiv:1606.05336). Cần lưu ý: đây **không** phải là math 21 trong bộ ghi chú này (math 21 là "The Expressive Power of Neural Networks: A View from the Width", arXiv:1709.02540, của Lu, Pu, Wang, Hu, Wang, một nhóm tác giả và nội dung hoàn toàn khác dù tên bài tương tự); ghi chú này không gộp hai bài lại.

7. Liên hệ với các bài khác

- **math 27 (Deep Information Propagation):** là bài kế tiếp trực tiếp, lấy đúng bộ công thức q^l, c_{12}^l, χ_1 của bài này làm "Background" (mục 2 của math 27), rồi mở rộng thêm hai depth-scale ξ_q, ξ_c (trong đó ξ_c phân kỳ đúng tại $\chi_1 = 1$), hiệu ứng dropout phá vỡ điểm tới hạn, và một lý thuyết mean-field song song cho gradient backprop nối trực tiếp $\chi_1 < 1/ > 1$ với vanishing/exploding gradient.
- **math 18 (Shattered Gradients Problem):** khảo sát hiện tượng có liên hệ gần: gradient của mạng ReLU sâu ngẫu nhiên trở nên giống nhiễu trắng (mất tương quan) theo độ sâu nếu không có kiến trúc residual phù hợp; đây là mặt "gradient" của chính hiện tượng chaos (mất tương quan tín hiệu) mà bài 26 nghiên cứu ở chiều truyền thẳng; math 27 xây cầu nối hình thức giữa hai hiện tượng này qua cùng hệ số χ_1.
- **math 21 (Expressive Power of Neural Networks, View from the Width):** cũng nghiên cứu đánh đổi độ sâu/độ rộng cho tính biểu diễn, nhưng bằng kỹ thuật khác (chặn dưới độ rộng cần thiết để mạng ReLU rộng xấp xỉ mạng hẹp sâu), bổ sung cho lập luận hình học/chaos của bài 26 về việc vì sao mạng nông không theo kịp mạng sâu.
- **math 22 (Deep Neural Networks as Gaussian Processes):** dùng chính cơ chế mean-field/Gauss ở giới hạn chiều rộng lớn (công thức lặp cho q^l ở đây về bản chất là công thức lặp cho kernel NNGP) nhưng cho mục đích khác: suy luận Bayes chính xác ở giới hạn vô hạn chiều rộng, thay vì đo độ hỗn loạn/biểu diễn.
- **math 20 (Benefits of depth in neural networks):** đưa lập luận sớm hơn, bằng công cụ lý thuyết xấp xỉ/lớp hàm, cho việc độ sâu có lợi; là một góc nhìn đối chứng hữu ích với lập luận hệ động lực học/hình học của bài 26 cho cùng câu hỏi.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, GPU ≤ 8GB, batch 1).

- **Ý tưởng chính (suy luận):** phương sai trọng số σ_w^2/N_{l-1} trong lý thuyết mean-field phụ thuộc trực tiếp vào chiều rộng lớp N_{l-1}. Khi pruning loại bỏ neuron/kênh, chiều rộng hiệu dụng N_{l-1} giảm; nếu không rescale lại phương sai trọng số còn lại cho đúng quy ước σ_w^2/N_{l-1} (với N_{l-1} **mới**, sau khi đã bớt), hệ số giãn χ_1 hiệu dụng của mạng con sau pruning có thể trôi ra khỏi đường tới hạn $\chi_1 \approx 1$ mà bài 26 mô tả, đẩy mạng con về phía quá trật tự (dễ làm tín hiệu/tính năng bị "co phẳng", khó phục hồi biểu diễn phức tạp khi fine-tune lại) hoặc quá hỗn loạn (khó tối ưu, nhạy với khởi tạo). Gợi ý thực hành: khi hậu huấn luyện LightOnOCR-3-0.8B sau pruning, nên kiểm tra/điều chỉnh lại thang phương sai khởi

tạo (hoặc dùng warm restart với learning rate nhỏ + theo dõi độ lớn kích hoạt theo lớp) để mạng con không bị lệch quá xa khỏi vùng gần tới hạn, trước khi đổ nhiều compute vào fine-tune phục hồi.

- **Ý tưởng phụ, liên hệ yếu hơn (suy luận, paper không kiểm chứng trực tiếp):** tính đối ngẫu weight chaos ở phụ lục F cho thấy nhiễu trọng số ở lớp giữa có thể gây thay đổi hàm tính được lớn theo cấp số nhân nếu mạng đang ở/gần pha hỗn loạn. Điều này gợi ý rằng xoá (zero-out) các kênh bị prune ở các lớp **giữa** của kiến trúc lai GatedDeltaNet + attention có thể rủi ro hơn (thay đổi hàm nhiều hơn trên mỗi tham số bị xoá) so với prune gần lớp đầu ra, nếu mạng đang hoạt động gần vùng hỗn loạn. Đây chỉ là một phép tương tự; bài 26 không nghiên cứu pruning, GatedDeltaNet, hay token hình ảnh/văn bản, nên liên hệ này cần kiểm chứng thực nghiệm riêng, không nên áp dụng trực tiếp làm quy tắc cứng.
- **Lưu ý trung thực:** bài 26 chỉ xét mạng FC tanh/sigmoidal ngẫu nhiên, chưa huấn luyện; không có bằng chứng nào trong paper cho kiến trúc gated/recurrent hoá hoặc đa phương thức (text + visual token) như LightOnOCR. Mọi liên hệ ở mục này là suy luận ngoại suy, không phải kết quả đã kiểm chứng.

9. Thí nghiệm gợi ý

1. **Đo tốc độ hội tụ của length map và correlation map** trên mạng FC tanh ngẫu nhiên nhỏ (rộng 500-1000, sâu tới 50 lớp) tại vài điểm (σ_w, σ_b) trải từ pha trật tự tới pha hỗn loạn, trên 1 GPU 8GB (hoặc chỉ CPU vì mạng nhỏ). Kỳ vọng: q^l hội tụ trong khoảng 4 lớp, c^l_{12} hội tụ trong khoảng 20 lớp, đúng như Hình 1-2 của paper; gần đường $\chi_1 = 1$, tốc độ hội tụ chậm hẳn lại (vì $|\log \chi_1| \to 0$).
2. **Tái lập rút gọn Hình 5:** so sánh một mạng 1 lớp ẩn với chiều rộng tăng dần (100 đến 5.000, vừa với 8GB) với một mạng sâu hẹp ở chế độ hỗn loạn (ví dụ $\sigma_w = 4{,}0, \sigma_b = 0{,}3$, rộng cố định 300-500, độ sâu tăng dần), cùng khớp một hàm mục tiêu ngẫu nhiên nhiều tần số. Đo lỗi theo tần số; kỳ vọng mạng rộng bão hoà còn mạng sâu tiếp tục cải thiện, giống Hình 5C/D.
3. **Đo điểm cố định độ cong** $\bar{\kappa}^*$ cho một hình tròn lan truyền qua mạng tanh nhỏ (rộng 500) ở vài giá trị σ_w trong pha hỗn loạn, so với giá trị lý thuyết giải từ công thức $(\bar{\kappa}^*)^2 = \chi_2/(\chi_1^2 - \chi_1)$ (rút ra từ công thức lặp ở mục 3 khi đặt $\bar{\kappa}^l = \bar{\kappa}^{l-1} = \bar{\kappa}^*$). Kiểm tra độ khớp lý thuyết - mô phỏng.
4. **Thí nghiệm prune-và-rescale đơn giản:** khởi tạo một mạng FC tanh nhỏ gần $\chi_1 \approx 1$, prune ngẫu nhiên 50% nơ-ron mỗi lớp, đo χ_1 của mạng con **trước** và **sau** khi rescale lại σ_w^2 theo chiều rộng mới N_{l-1}. Mục tiêu: kiểm tra trực tiếp giả thuyết ở mục 8, rằng pruning không rescale có thể làm χ_1 trôi khỏi vùng tới hạn.
5. **Đo weight chaos thực nghiệm:** với mạng tanh nhỏ ở pha hỗn loạn, cố định đầu vào (hình tròn ở bán kính q^*), nhiễu trọng số một lớp giữa theo tham số Δ như công thức ở mục 3, đo tương quan hàm $C^l(\Delta)$ ở lớp cuối theo Δ và theo khoảng cách từ lớp bị nhiễu tới lớp cuối; kỳ vọng giảm theo cấp số nhân, khớp với lập luận ở phụ lục F.

10. Tóm tắt công thức / khái niệm

- **Length map:** $q^l = \sigma_w^2 \int \mathcal{D}z\, \phi(\sqrt{q^{l-1}}z)^2 + \sigma_b^2$; điểm cố định q^*.
- **Correlation map và χ_1:** $q^l_{12} = \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi(u_1)\phi(u_2) + \sigma_b^2$; $\chi_1 = \sigma_w^2 \int \mathcal{D}z\, [\phi'(\sqrt{q^*}z)]^2$; $\chi_1 < 1$ là pha trật tự, $\chi_1 > 1$ là pha hỗn loạn, $\chi_1 = 1$ là đường tới hạn.
- **Transient chaos:** $\log \chi_1$ tương tự số mũ Lyapunov, nhưng "cắt cụt" ở độ sâu hữu hạn D (khác hỗn loạn đúng nghĩa của hệ lặp vô hạn).
- **Tiến hoá độ dài/độ cong:** $\bar{g}^{E,l} = \chi_1 \bar{g}^{E,l-1}$; $(\bar{\kappa}^l)^2 = \chi_2/\chi_1^2 + (\bar{\kappa}^{l-1})^2/\chi_1$, với $\chi_2 = \sigma_w^2 \int \mathcal{D}z[\phi''(\sqrt{q^*}z)]^2$; độ cong hội tụ về điểm cố định khác không trong pha hỗn loạn.
- **Định lý 1 (chặn mạng nông):** $L^E \le N_1(1+s)R$; với hình tròn + tanh: $\bar{L}^E \le 2\sqrt{N_1}$.
- **Tốc độ hội tụ quan sát được:** length map hội tụ trong khoảng 4 lớp; correlation map hội tụ trong khoảng 20 lớp (chậm hơn vì gần đường chéo hơn).
- **Tự tính (ví dụ 1, theo đúng công thức Định lý 1 mà paper đã rút gọn cho hình tròn + tanh):** với $N_1 = 1.000$: $\bar{L}^E \le 2\sqrt{1.000} \approx 2 \times 31{,}62 \approx 63{,}2$. Với $N_1 = 10.000$ (như thí nghiệm Hình 5D): $\bar{L}^E \le 2\sqrt{10.000} = 200$, tức tăng độ rộng gấp 10 lần chỉ cho phép chặn trên tăng khoảng $\sqrt{10} \approx 3{,}16$

lần, không phải 10 lần, giải thích tại sao Hình 5D cho thấy lỗi tần số cao gần như không đổi khi tăng N_1.

- **Tự tính (ví dụ 2, minh hoạ tăng trưởng mũ của $\bar{g}^E$):** giả sử $\chi_1 = 1{,}5$ (pha hỗn loạn) và $q^* = 1$; theo công thức $\bar{g}^{E,l} = \chi_1\, \bar{g}^{E,l-1}$, sau 19 lớp: $\bar{g}^{E,20} = 1{,}5^{19} \times 1$. Tính: $1{,}5^{10} \approx 57{,}67$; $1{,}5^{9} \approx 38{,}44$; nhân lại $1{,}5^{19} \approx 57{,}67 \times 38{,}44 \approx 2.218$. Vậy sau 20 lớp, độ dài bình phương trên mỗi nơ-ron đã tăng khoảng 2.218 lần chỉ từ $\chi_1 = 1{,}5$, tương ứng độ dài L^E tăng khoảng $\sqrt{2.218} \approx 47$ lần, minh hoạ cụ thể tốc độ tăng trưởng mũ mà paper mô tả bằng lời.

11. Kết luận ngắn

Bài này chỉ ra, bằng lý thuyết mean-field kết hợp hình học Riemann trên mạng ngẫu nhiên chưa huấn luyện, rằng ở pha hỗn loạn ($\chi_1 > 1$) độ dài và độ cong toàn cục của một đa tạp đầu vào tăng theo cấp số nhân với độ sâu, trong khi mạng nông chỉ có thể đạt độ phức tạp tăng theo căn bậc hai của chiều rộng (Định lý 1); ranh giới quyết định cũng cong theo cấp số nhân khi lan truyền ngược về đầu vào, và hiện tượng đối ngẫu ”weight chaos” cho thấy nhiễu trọng số nhỏ cũng có thể gây thay đổi hàm lớn ở pha hỗn loạn. Những kết quả này đặt nền cho math 27 mở rộng sang depth-scale và lý thuyết gradient backprop.

Nguồn gốc: https://arxiv.org/abs/1606.05340

Bài 27

Deep Information Propagation (2016)

Tác giả: Samuel S. Schoenholz (Google Brain), Justin Gilmer (Google Brain), Surya Ganguli (Stanford University), Jascha Sohl-Dickstein (Google Brain).
Venue / arXiv: arXiv:1611.01232 (v2, 04/04/2017); venue ghi trong paper: ”Published as a conference paper at ICLR 2017”.
Nguồn: https://arxiv.org/abs/1611.01232
Chủ đề: hai depth-scale (ξ_q, ξ_c) điều khiển lan truyền tín hiệu trong mạng ngẫu nhiên chưa huấn luyện; ảnh hưởng của dropout lên điểm tới hạn order-to-chaos; lý thuyết mean-field cho backprop gradient; thực nghiệm trainability trên MNIST/CIFAR-10 theo độ sâu và dropout.
Vị trí trong lộ trình: đọc ngay sau math 26 (Exponential expressivity in deep neural networks through transient chaos), vì bài này lấy nguyên khung lý thuyết mean-field của math 26 (length map, correlation map, χ_1) làm phần ”Background”, rồi mở rộng thành câu hỏi thực dụng hơn: với tham số nào thì mạng sâu **huấn luyện được**. Trong nhóm math, còn liên hệ math 18, math 20, math 22, math 23; chi tiết ở mục 7.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (đọc toàn văn bản text trích từ PDF arXiv, bản v2 ngày 04/04/2017, gồm cả phụ lục 7.1-7.10). Các phép tính đánh dấu ”tự tính” là của người ghi chú, không phải số có sẵn trong paper. Mục ”Ứng dụng cho dự án” là suy luận (inference), không phải kết luận của paper.

1. Bối cảnh

Poole et al. (2016, = math 26 trong bộ ghi chú này) và bài đồng hành Raghu et al. (2016) đã chứng minh mạng ngẫu nhiên chưa huấn luyện có thể biểu diễn hàm phức tạp theo cấp số nhân với độ sâu, nhờ lý thuyết mean-field mô tả ranh giới order-to-chaos. Nhưng biểu diễn được không có nghĩa là huấn luyện được. Paper này (Schoenholz, Gilmer, Ganguli, Sohl-Dickstein, ICLR 2017) đặt câu hỏi bổ sung, mang tính thực dụng hơn: với lựa chọn siêu tham số nào thì một mạng sâu thực sự **train được**?

Bối cảnh rộng hơn: việc chọn khởi tạo cho mạng sâu là bài toán nửa khoa học nửa ”nghệ thuật”, với nhiều kỹ thuật trực giác hoặc gián tiếp đã tồn tại trước bài này: khởi tạo ”edge of chaos” trực tiếp (Bertschinger et al., 2005; Glorot và Bengio, 2010), hoặc gián tiếp qua các kỹ thuật thúc đẩy lan truyền tín hiệu sâu như batch normalization (Ioffe và Szegedy, 2015), khởi tạo ma trận trực giao (Saxe et al., 2014), khởi tạo random-walk (Sussillo và Abbott, 2014), composition kernel (Daniely et al., 2016), hay kiến trúc residual (He et al., 2015). Paper tự nhận điểm mới của mình là hai điều: (1) khung lý thuyết dự đoán được độ sâu khả huấn luyện **ngay cả khi mạng ở xa điểm tới hạn** (ví dụ với dropout, nơi không hề có điểm tới hạn sắc nét); (2) đưa ra một giải thích **hình thức** (formal), không chỉ trực giác, cho hiện tượng ”chỉ huấn luyện được mạng rất sâu gần edge of chaos”.

2. Ý tưởng chính

(a) Hai depth-scale. Khai triển tiệm cận bản đồ độ dài và bản đồ tương quan của math 26 quanh điểm cố định cho ra hai thước đo độ sâu riêng biệt: ξ_q (độ sâu mà thông tin về **biên độ** một đầu vào đơn lẻ có thể lan truyền) và ξ_c (độ sâu mà thông tin về **tương quan** giữa hai đầu vào có thể lan truyền). Khác biệt mấu chốt: ξ_q luôn **hữu hạn và ngắn**, bất kể tham số nào, còn ξ_c **phân kỳ đúng tại đường tới hạn** $\chi_1 = 1$ của math 26. Đây là cách hình thức hoá trực giác ”edge of chaos cho phép mạng vô hạn sâu”: chỉ có thông tin tương quan (không phải thông tin biên độ) mới có thể lan truyền xa vô hạn, và chỉ khi mạng ở đúng điểm tới hạn.

(b) Dropout phá vỡ điểm tới hạn hoàn toàn. Với tỉ lệ dropout $\rho < 1$ bất kỳ, dù rất nhỏ, điểm $c^* = 1$ **không còn là điểm cố định** của bản đồ tương quan nữa (khác hẳn trường hợp không dropout, nơi $c^* = 1$ luôn là

điểm cố định, chỉ có ổn định hay không mới phụ thuộc χ_1). Hệ quả: ξ_c không bao giờ phân kỳ khi có dropout, nên độ sâu khả huấn luyện bị chặn trên một cách nghiêm ngặt, là hàm của ρ.

(c) Đối ngẫu forward-backward cho gradient. Paper xây một lý thuyết mean-field song song cho backprop: phương sai lỗi $\tilde{q}^l_{aa}$ lan truyền ngược qua các lớp tuân theo công thức lặp dùng **đúng** hệ số χ_1 của forward pass. Ba chế độ: pha trật tự ($\chi_1 < 1$) gradient **biến mất** (vanish) theo cấp số nhân; điểm tới hạn ($\chi_1 \to 1$) gradient **ổn định** ở mọi độ sâu; pha hỗn loạn ($\chi_1 > 1$) gradient **nổ** (explode) theo cấp số nhân. Đây được trình bày như lời giải thích hình thức đầu tiên (bằng mean-field) cho hiện tượng vanishing/exploding gradient đã được quan sát từ lâu (Bengio et al., 1993).

(d) Hiệp phương sai gradient cũng bị chặn bởi ξ_c. Không chỉ phương sai gradient của một mẫu, mà hiệp phương sai gradient **giữa hai mẫu đầu vào khác nhau** cũng suy giảm theo cùng depth-scale ξ_c chi phối tương quan tín hiệu forward. Điều này nối trực tiếp "thông tin tương quan có lan truyền được không" với "mạng có học được cấu trúc chung giữa các mẫu huấn luyện không".

(e) Giả thuyết trainability và kiểm chứng thực nghiệm. Từ (a)-(d), paper đưa ra giả thuyết: điều kiện cần để một mạng ngẫu nhiên huấn luyện được là thông tin phải lan truyền được cả hai chiều (forward và backward), nghĩa là độ sâu mạng L không được lớn hơn nhiều so với ξ_c. Giả thuyết này **không phụ thuộc dữ liệu**, chỉ phụ thuộc kiến trúc/siêu tham số, nên được gọi là chặn "phổ quát" (universal). Paper kiểm chứng bằng cách huấn luyện hàng loạt mạng FC sâu (độ sâu, σ_w^2, có/không dropout khác nhau) trên MNIST và CIFAR-10.

3. Phương pháp chi tiết

Nhắc lại khung mean-field (mục 2 của paper, trùng math 26). Mạng FC chưa huấn luyện, $z_i^l = \sum_j W_{ij}^l y_j^l + b_i^l$, $y_i^{l+1} = \phi(z_i^l)$, $W_{ij}^l \sim \mathcal{N}(0, \sigma_w^2/N_l)$, $b_i^l \sim \mathcal{N}(0, \sigma_b^2)$. Phương sai một đầu vào $q_{aa}^l = \sigma_w^2 \int \mathcal{D}z\, \phi^2(\sqrt{q_{aa}^{l-1}} z) + \sigma_b^2$; hiệp phương sai hai đầu vào $q_{ab}^l = \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi(u_1)\phi(u_2) + \sigma_b^2$; hệ số giãn $\chi_1 = \sigma_w^2 \int \mathcal{D}z\, [\phi'(\sqrt{q^*} z)]^2$; $\chi_1 = 1$ là ranh giới order-to-chaos.

Khai triển tiệm cận cho ξ_q. Đặt $q_{aa}^l = q^* + \epsilon_l$, khai triển bậc nhất quanh điểm cố định (xem phụ lục 7.1 của paper):

$$\epsilon_{l+1} = \epsilon_l \left[\chi_1 + \sigma_w^2 \int \mathcal{D}z\, \phi''(\sqrt{q^*} z) \phi(\sqrt{q^*} z) \right] + O(\epsilon_l^2),$$

cho nghiệm tiệm cận $\epsilon_l \sim e^{-l/\xi_q}$ với

$$\xi_q^{-1} = -\log \left[\chi_1 + \sigma_w^2 \int \mathcal{D}z\, \phi''(\sqrt{q^*} z) \phi(\sqrt{q^*} z) \right].$$

Với $\phi = \tanh$, số hạng thứ hai trong dấu ngoặc luôn **âm** (do tính đối xứng lẻ của tanh và ϕ''), nên dù $\chi_1 \to 1$, cả biểu thức trong ngoặc vẫn nhỏ hơn 1 một khoảng đáng kể; đây là lý do ξ_q luôn ngắn, không phân kỳ, kể cả gần điểm tới hạn.

Khai triển tiệm cận cho ξ_c. Đặt $c_{ab}^l = c^* + \epsilon_l$, khai triển tương tự (phụ lục 7.2):

$$\epsilon_{l+1} = \epsilon_l\, \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1^*)\phi'(u_2^*) + O(\epsilon_l^2), \qquad \xi_c^{-1} = -\log \left[\sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1^*)\phi'(u_2^*) \right].$$

Trong pha trật tự ($c^* = 1$), biểu thức trong ngoặc rút gọn đúng thành χ_1, nên $\xi_c^{-1} = -\log \chi_1$: khi $\chi_1 \to 1$ (tức tiến tới ranh giới order-to-chaos), $\xi_c \to \infty$. Đây là kết quả trung tâm của mục này: ξ_c **phân kỳ chính xác** tại điểm tới hạn.

Dropout. Với dropout tỉ lệ ρ, tiền kích hoạt trở thành $z_i^l = \frac{1}{\rho} \sum_j W_{ij}^l p_j^l y_j^l + b_i^l$, $p_j^l \sim \text{Bernoulli}(\rho)$ (chọn độc lập cho mỗi đầu vào, giống cách dropout dùng trong thực tế). Phương sai một đầu vào (phụ lục 7.3): $\bar{q}_{aa}^l = \frac{\sigma_w^2}{\rho} \int \mathcal{D}z\, \phi^2(\sqrt{\bar{q}_{aa}^{l-1}} z) + \sigma_b^2$, tức tương đương việc rescale $\sigma_w^2 \to \sigma_w^2/\rho$ cho trường hợp một đầu vào.

Hiệp phương sai hai đầu vào (phụ lục 7.4) có **cùng dạng công thức** như không dropout. Nhưng khi thay $\bar{c}^l_{ab} = 1$ vào công thức này (phụ lục 7.5), kết quả là:

$$\bar{c}^{l+1}_{ab} = 1 - \frac{1-\rho}{\rho \bar{q}^*} \sigma_w^2 \int \mathcal{D}z\, \phi^2(\sqrt{\bar{q}^*} z),$$

số hạng trừ đi **luôn dương** với mọi $\rho < 1$, nên $\bar{c}^{l+1}_{ab} < 1$ bất kể $\bar{c}^l_{ab} = 1$: nói cách khác, $c^* = 1$ **không còn là điểm cố định nào cả** của động lực này, với mọi $\rho < 1$, dù nhỏ tới đâu.

Mean-field cho gradient. Backprop: $\partial E/\partial W^l_{ij} = \delta^l_i\, \phi(z^{l-1}_j)$, $\delta^l_i = \phi'(z^l_i) \sum_j \delta^{l+1}_j W^{l+1}_{ji}$. Với giả thiết bổ sung (giống feedback alignment, Lillicrap et al. 2014) rằng trọng số dùng ở forward và trọng số ”nhìn thấy” ở backward là độc lập, paper suy ra (phụ lục 7.6-7.7) công thức lặp cho $\tilde{q}^l_{aa} = E[(\delta^l_i)^2]$:

$$\tilde{q}^l_{aa} = \tilde{q}^{l+1}_{aa} \frac{N_{l+1}}{N_l} \chi_1(q^l_{aa}).$$

Với mạng chiều rộng không đổi ($N_{l+1}/N_l = 1$), nghiệm là $\tilde{q}^l_{aa} = \tilde{q}^L_{aa}\, e^{-(L-l)/\xi_\nabla}$, $\xi_\nabla^{-1} = -\log \chi_1$ (đúng cả hai phía của điểm tới hạn, nên ξ_∇ có thể âm hoặc dương). Ba chế độ: $\chi_1 < 1 \Rightarrow \xi_\nabla > 0$ (gradient biến mất theo độ sâu $|\xi_\nabla|$); $\chi_1 \to 1 \Rightarrow \xi_\nabla \to \infty$ (gradient ổn định); $\chi_1 > 1 \Rightarrow \xi_\nabla < 0$ (gradient nổ theo độ sâu $|\xi_\nabla|$).

Hiệp phương sai gradient giữa hai mẫu. Tương tự (phụ lục 7.8-7.9): $\tilde{q}^l_{ab} = \tilde{q}^{l+1}_{ab} \frac{N_{l+1}}{N_{l+2}} \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1)\phi'(u_2)$, có nghiệm tiệm cận bậc không với **cùng** ξ_c như ở forward pass.

4. Thí nghiệm và kết quả chính

- **Hình 1 (phase diagram và hội tụ):** (a) biểu đồ pha (σ_w^2, σ_b^2) cho $\phi = \tanh$, phân chia order/chaos. (b)-(c) phần dư $|q^* - q^l_{aa}|$ và $|c^* - c^l_{ab}|$ theo độ sâu, trục log, với $\sigma_b^2 = 0{,}05$ cố định và σ_w^2 từ $0{,}01$ (đỏ) tới $1{,}7$ (tím): cả hai cho thấy **hành vi mũ rõ ràng** qua nhiều bậc độ lớn.
- **Hình 2 (đo depth-scale):** (a) bản đồ tương quan lặp với 3 giá trị $\sigma_w^2 = 0{,}5$ (trật tự, xa), $1{,}8$ (gần edge of chaos), $3{,}5$ (hỗn loạn sâu); đường tuyến tính hoá quanh điểm tới hạn có hệ số góc e^{-1/ξ_c}. (b) ξ_q theo σ_w^2 với σ_b^2 từ $0{,}01$ tới $0{,}3$: lý thuyết (nét đứt) khớp đo đạc (nét liền), ξ_q luôn ngắn và hữu hạn. (c) ξ_c theo σ_w^2, cùng dải σ_b^2: **phân kỳ rõ rệt tại điểm tới hạn**. Phương pháp đo: lặp công thức với $q^0_{aa} = q^0_{bb} = 0{,}8$, $c^0_{ab} = 0{,}6$, $\sigma_w^2 \in [0{,}1, 3{,}0]$, $\sigma_b^2 \in [0{,}01, 0{,}3]$, fit phần dư theo hàm mũ để suy ra depth-scale đo được.
- **Hình 3 (dropout phá điểm tới hạn):** (a) bản đồ tương quan với dropout $\rho = 1{,}0$, $0{,}95$, $0{,}9$ gần điểm tới hạn: $\bar{c}^l_{ab} = 1$ không còn là điểm cố định, điểm cố định mới giảm dần khi ρ giảm. (b) c^* theo σ_w^2 với ρ từ $0{,}8$ đến $1{,}0$: **không còn chuyển pha sắc nét** với mọi $\rho < 1$. (c) ξ_c theo σ_w^2 cùng dải ρ: **không còn phân kỳ** với $\rho < 1$, và ξ_c giảm khi ρ giảm ở cùng σ_w^2.
- **Hình 4 (gradient backprop):** mạng ngẫu nhiên sâu $L = 240$, chiều rộng $N_l = 300$, hàm lỗi cross-entropy trên MNIST. (a) chuẩn-2 gradient theo lớp $\|\nabla W^l_{ab} E\|_2^2$ với σ_w^2 từ $1{,}0$ (xanh) tới $4{,}0$ (đỏ): hành vi mũ rõ rệt qua nhiều bậc độ lớn, gradient biến mất ở pha trật tự, nổ ở pha hỗn loạn. (b) $|\xi_\nabla|$ đo được so với lý thuyết: khớp tốt, lệch chủ yếu gần điểm tới hạn do hiệu ứng kích thước hữu hạn.
- **Mục 5 (thực nghiệm trainability, kết quả thực dụng chính của paper):** mạng FC độ sâu $10 \le L \le 300$, $1 \le \sigma_w^2 \le 4$, $\sigma_b^2 = 0{,}05$; huấn luyện bằng SGD và RMSProp trên MNIST và CIFAR-10. Learning rate: SGD 10^{-3} khi $L \lesssim 200$, 10^{-4} khi L lớn hơn; RMSProp 10^{-5}; các giá trị này chọn bằng grid search từ 10^{-6} đến 10^{-2} theo bước nhân 10. Hình 5: (a) MNIST, SGD, 200 bước; (b) CIFAR-10, SGD, 2000 bước; (c) MNIST, SGD, 14000 bước; (d) MNIST, RMSProp, 300 bước. Ở mọi trường hợp, mạng cuối cùng overfit tới train accuracy 100%, test accuracy 98% trên MNIST và 55% trên CIFAR-10 (mục tiêu thí nghiệm là chứng minh ”huấn luyện được”, không phải tối ưu test accuracy). Overlay các đường $n\xi_c$ ($n = 1$ đến 6) lên panel (a): mạng có vẻ **huấn luyện được khi** $L \lesssim 6\xi_c$ (paper tự ghi ”sẽ thú vị nếu hiểu được tại sao là hệ số 6”); overlay riêng $6\xi_c$ lên các panel (b)-(d) cho thấy ranh giới này **nhất quán giữa dữ liệu, thời gian huấn luyện, và optimizer**, nên được gọi là chặn ”phổ quát”.
- **Hình 6 (trainability với dropout):** cùng khung thí nghiệm trên MNIST với $\rho = 0{,}99$, $0{,}98$, $0{,}94$. Ngay

cả dropout rất nhẹ ($\rho = 0{,}99$) cũng **giới hạn độ sâu khả huấn luyện tối đa xuống khoảng** $L = 100$; ξ_c (tính theo công thức dropout ở mục 3) vẫn dự đoán đúng vùng huấn luyện được ứng với từng mức dropout.

- **Phụ lục 7.10:** Hình 7 (độ chính xác huấn luyện MNIST sau 45/304/2048/13780 bước SGD, lr 10^{-3}) và Hình 8 (tương tự, sau 45/304/2048/13780 bước RMSProp, lr 10^{-5}) cho thấy quan hệ L so với ξ_c giữ nguyên dạng qua các mốc thời gian huấn luyện khác nhau.

5. Phân tích / ablation

Paper quét hệ thống nhiều trục cùng lúc, và chính sự **nhất quán xuyên suốt** các trục này là luận điểm thực nghiệm trung tâm:

- **Độ sâu L:** từ 10 đến 300 lớp.
- **Phương sai trọng số σ_w^2:** từ 1 đến 4 cho thí nghiệm trainability (và rộng hơn, 0,1 đến 3,0, khi đo depth-scale ở Hình 2).
- **Phương sai bias σ_b^2:** cố định $0{,}05$ cho thí nghiệm trainability; quét $0{,}01$ đến $0{,}3$ khi đo depth-scale.
- **Dữ liệu:** MNIST và CIFAR-10.
- **Optimizer:** SGD và RMSProp.
- **Thời gian huấn luyện:** từ 45 đến 14.000 bước.
- **Dropout:** ρ từ $0{,}8$ đến $1{,}0$ cho phân tích lý thuyết; $\rho = 0{,}99/0{,}98/0{,}94$ cho thực nghiệm trainability.

Kết quả ”ablation” đáng chú ý nhất: ranh giới $L \lesssim 6\xi_c$ **không đổi dạng** khi hoán đổi dữ liệu (MNIST/CIFAR-10), optimizer (SGD/RMSProp), hay thời gian huấn luyện (200 đến 14.000 bước), ủng hộ tuyên bố ”chặn phổ quát, không phụ thuộc dữ liệu”. Riêng trục dropout cho một ”ablation” định lượng rõ ràng: $\rho = 0{,}99 \to 0{,}98 \to 0{,}94$ làm độ sâu khả huấn luyện tối đa giảm dần, và ξ_c (công thức dropout) tiếp tục dự đoán đúng từng mức. Cuối cùng, việc so sánh trực tiếp Hình 2(b) và 2(c) (tức ξ_q so với ξ_c trên cùng dải tham số) là một phép ”ablation” khái niệm: cho thấy chính ξ_c, không phải ξ_q, là đại lượng quyết định trainability, vì ξ_q luôn ngắn nên không đổi nhiều theo việc mạng gần hay xa điểm tới hạn.

6. Giới hạn

Paper tự nêu rõ một số hạn chế ở phần Discussion:

- Khung lý thuyết **không áp dụng cho hàm hoạt hoá không chặn** như ReLU, vì có những vùng tham số nơi công thức độ dài (eq. 3, giống math 26) **không có điểm cố định**.
- Phân tích chỉ áp dụng trực tiếp cho **mạng kết nối đầy đủ, truyền thẳng**; paper tự nhận cần mở rộng cho kiến trúc có ma trận trọng số có cấu trúc, như CNN.
- Mục tiêu thí nghiệm mục 5 là chứng minh ”huấn luyện được” (trainable), **không phải** tối ưu test accuracy; test accuracy đạt được (98% MNIST, 55% CIFAR-10) chỉ là sản phẩm phụ, không nên dùng để so sánh với mô hình SOTA.
- Hệ số 6 trong ranh giới $L \lesssim 6\xi_c$ là quan sát thực nghiệm, **chưa có suy luận lý thuyết** cho giá trị cụ thể này; paper tự đặt câu hỏi mở này, không trả lời.
- Lý thuyết gradient cần thêm một giả thiết (trọng số forward và ”nhìn thấy” ở backward độc lập, giống feedback alignment) **ngoài** giả thiết mean-field gốc; paper không định lượng sai số của giả thiết này, chỉ kiểm chứng gián tiếp qua khớp số liệu ở Hình 4.
- Công thức gradient (eq. 15) giả định chiều rộng các lớp bằng nhau ($N_{l+1}/N_l = 1$); trường hợp chiều rộng không đều được paper **nhường lại** cho tài liệu khác (Glorot và Bengio, 2010), không tự suy ra trong bài này.

7. Liên hệ với các bài khác

- **math 26 (Exponential expressivity... through transient chaos):** nền tảng trực tiếp; mọi công thức ở mục ”Background” của bài 27 (length map, correlation map, χ_1) là lấy nguyên từ bài 26, chỉ đổi mục tiêu phân tích từ ”biểu diễn được gì” sang ”huấn luyện được không”.
- **math 18 (Shattered Gradients Problem):** cùng nói về gradient sâu trở nên ”giống nhiễu trắng” theo độ sâu nếu không có kiến trúc phù hợp; math 27 cho công cụ định lượng (ξ_∇, χ_1) để hình thức hoá đúng hiện tượng mà math 18 quan sát bằng thực nghiệm trên ReLU/residual network.
- **math 22 (Deep Neural Networks as Gaussian Processes):** paper tự liên hệ trực tiếp trong phần Discussion: hiệp phương sai q^l_{ab} chính là kernel của Gaussian Process cảm sinh (Neal, 2012) khi mạng vô hạn rộng; những trường hợp thông tin tương quan lan truyền được (tức ξ_c lớn) tương ứng chính xác với những trường hợp GP cảm sinh ”well-defined” (có kernel không suy biến).
- **math 23 (Neural Tangent Kernel):** cũng dùng công cụ kernel/mean-field ở giới hạn chiều rộng lớn để phân tích động lực gradient của mạng, nhưng theo hướng khác (kernel NTK không đổi trong suốt huấn luyện ở giới hạn vô hạn rộng), bổ sung góc nhìn cho lý thuyết backprop ”hữu hạn độ sâu, hữu hạn chiều rộng” của bài 27.
- **math 20 (Benefits of depth in neural networks):** cùng câu hỏi lớn ”độ sâu có ích gì”, nhưng bài 27 trả lời ở khía cạnh ”huấn luyện được hay không” thay vì ”biểu diễn được gì”, bổ khuyết cho nhau.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, GPU ≤ 8GB, batch 1).

- **Liên hệ trực tiếp nhất (suy luận):** quy tắc thực nghiệm $L \lesssim 6\xi_c$ gợi ý một cách kiểm tra cụ thể trước khi đổ compute vào fine-tune phục hồi sau pruning: nếu pruning làm thay đổi hiệu dụng σ_w, σ_b (do đổi chiều rộng N_l, như đã nêu ở math 26 mục 8) tới mức ξ_c hiệu dụng của mạng con trở nên nhỏ hơn nhiều so với số lớp thực tế của kiến trúc lai GatedDeltaNet + attention, mô hình con có thể rơi vào vùng khó huấn luyện theo đúng cơ chế mà bài 27 mô tả, bất kể có đủ dữ liệu/compute hay không. Nên coi đây là một bước kiểm tra chẩn đoán rẻ, trước khi chạy fine-tune phục hồi đầy đủ.
- **Cảnh báo cụ thể về dropout (suy luận):** nếu quy trình hậu huấn luyện/fine-tune phục hồi của LightOnOCR-3-0.8B dùng dropout để chống overfit, kết quả của bài 27 (ngay cả $\rho = 0{,}99$ cũng giới hạn độ sâu khả huấn luyện khoảng $L = 100$ trên mạng FC tanh) là một lời nhắc cụ thể: dropout có thể áp một trần độ sâu khả huấn luyện ẩn, độc lập với compute, lên các khối sâu của kiến trúc lai. Cần kiểm tra số lớp hiệu dụng của mô hình so với ngưỡng kiểu này trước khi quyết định giữ/bỏ dropout trong công thức phục hồi.
- **Chẩn đoán rẻ bằng gradient-norm theo lớp (suy luận, actionable):** vì hiệp phương sai gradient giữa các mẫu cũng bị chặn bởi cùng ξ_c, có thể đo thống kê chuẩn gradient theo lớp (giống Hình 4a) trên một batch nhỏ dữ liệu OCR thật, chạy trên mô hình con sau pruning, **không cần huấn luyện đầy đủ**, để ước lượng sớm ξ_∇/ξ_c hiệu dụng và phát hiện rủi ro vanishing/exploding gradient trước khi commit vào một lần fine-tune phục hồi tốn kém.
- **Lưu ý trung thực:** các hằng số cụ thể của bài 27 (hệ số 6, ngưỡng $L \approx 100$ với dropout nhẹ) được đo trên mạng FC tanh ngẫu nhiên, trên MNIST/CIFAR-10, **không** phải trên kiến trúc lai GatedDeltaNet + attention có residual connection, layer norm, và trọng số đã học (không còn i.i.d. Gauss ngẫu nhiên) như LightOnOCR. Logic định tính (depth-scale chặn độ sâu khả huấn luyện; dropout làm giảm depth-scale) nhiều khả năng vẫn có giá trị tham khảo, nhưng các hằng số cụ thể gần như chắc chắn **không** chuyển trực tiếp sang kiến trúc này; đây là suy luận ngoại suy, cần kiểm chứng riêng, không phải kết luận đã được paper hay dự án xác nhận.

9. Thí nghiệm gợi ý

1. **Tái lập Hình 2(b)(c) ở quy mô nhỏ:** lặp số bản đồ độ dài/tương quan cho $\phi = \tanh$ (hoàn toàn trên CPU, không cần GPU), đo ξ_q, ξ_c theo σ_w^2 ở vài giá trị σ_b^2, xác nhận ξ_c phân kỳ tại $\chi_1 = 1$ còn ξ_q luôn hữu hạn/ngắn.
2. **Lưới trainability thu nhỏ:** huấn luyện mạng FC tanh sâu (độ sâu 10-100, rộng 100-300, vừa 1 GPU 8GB) trên tập con MNIST, quét σ_w^2 quanh điểm tới hạn, kiểm tra xem ranh giới $L \lesssim 6\xi_c$ có xấp xỉ đúng ở quy mô nhỏ hơn nhiều so với Hình 5 của paper (độ sâu tới 300) hay không.
3. **Tái lập rút gọn Hình 4 (gradient theo lớp):** dựng mạng ngẫu nhiên sâu 100-240 lớp, rộng 200-300 (vừa 8GB dễ dàng), đo chuẩn-2 gradient theo lớp trên **một batch** MNIST (chỉ cần 1 lần forward + backward, không cần huấn luyện), so tốc độ tăng/giảm đo được với dự đoán lý thuyết $\xi_\nabla^{-1} = -\log \chi_1$.
4. **Trainability với dropout thu nhỏ:** lặp lại thí nghiệm 2 nhưng thêm dropout $\rho = 0{,}99/0{,}95/0{,}9$, kiểm tra độ sâu khả huấn luyện tối đa có giảm dần đúng như Hình 6 của paper hay không, ở quy mô mạng nhỏ hơn (độ sâu 20-150).
5. **Chẩn đoán gradient-norm trên mô hình dự án:** trên một checkpoint LightOnOCR-3-0.8B (trước và sau một lần pruning), đo thống kê chuẩn gradient theo lớp/khối trên một batch nhỏ dữ liệu OCR thật (tuân thủ batch size 1, GPU ≤ 8GB), để kiểm tra trực quan xem mạng con sau pruning có dấu hiệu vanishing/exploding gradient hay không, trước khi chạy fine-tune phục hồi đầy đủ.

10. Tóm tắt công thức / khái niệm

- **Depth-scale biên độ:** $\xi_q^{-1} = -\log\left[\chi_1 + \sigma_w^2 \int \mathcal{D}z\, \phi''(\sqrt{q^*}z)\phi(\sqrt{q^*}z)\right]$; luôn hữu hạn, ngắn.
- **Depth-scale tương quan:** $\xi_c^{-1} = -\log\left[\sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1^*)\phi'(u_2^*)\right]$; trong pha trật tự bằng $-\log \chi_1$; **phân kỳ tại** $\chi_1 = 1$.
- **Dropout phá điểm tới hạn:** $\tilde{c}_{ab}^{l+1} = 1 - \frac{1-\rho}{\rho \tilde{q}^*}\sigma_w^2 \int \mathcal{D}z\, \phi^2(\sqrt{\tilde{q}^*}z) < 1$ với mọi $\rho < 1$; không còn điểm cố định $c^* = 1$.
- **Depth-scale gradient:** $\xi_\nabla^{-1} = -\log \chi_1$ (cả hai phía điểm tới hạn); $\chi_1 < 1$ gradient biến mất, $\chi_1 > 1$ gradient nổ, $\chi_1 \to 1$ gradient ổn định.
- **Quy tắc trainability thực nghiệm:** mạng huấn luyện được khi độ sâu $L \lesssim 6\xi_c$; với dropout, ngưỡng này co lại nhanh (ví dụ $\rho = 0{,}99$ còn khoảng $L \approx 100$).
- **Tự tính (ví dụ 1, depth-scale gradient):** giả sử $\chi_1 = 0{,}95$ (pha trật tự, hơi dưới điểm tới hạn). $\xi_\nabla = -1/\ln(0{,}95)$. Tính $\ln(0{,}95) \approx -0{,}05129$, nên $\xi_\nabla \approx 1/0{,}05129 \approx 19{,}5$ lớp. Sau $L = 240$ lớp (đúng cấu hình Hình 4 của paper), gradient suy giảm theo hệ số $e^{-240/19{,}5} = e^{-12{,}3} \approx 4{,}5 \times 10^{-6}$ (tự tính), khớp với mô tả "biến mất qua nhiều bậc độ lớn" của paper ở pha trật tự.
- **Tự tính (ví dụ 2, quy tắc $6\xi_c$):** nếu đo được $\xi_c = 50$ lớp tại một cấu hình (σ_w, σ_b) cho trước (số giả định để minh hoạ cách dùng công thức, không phải số trong paper), quy tắc thực nghiệm của paper dự đoán độ sâu khả huấn luyện tối đa xấp xỉ $6 \times 50 = 300$ lớp (tự tính).

11. Kết luận ngắn

Bài này mở rộng khung mean-field của math 26 bằng cách định nghĩa hai depth-scale ξ_q (luôn ngắn) và ξ_c (phân kỳ đúng tại điểm tới hạn order-to-chaos), chỉ ra dropout phá vỡ điểm tới hạn này với mọi tỉ lệ $\rho < 1$, xây một lý thuyết mean-field đối ngẫu cho gradient backprop nối trực tiếp pha trật tự/hỗn loạn với vanishing/exploding gradient, và kiểm chứng bằng thực nghiệm rằng mạng sâu huấn luyện được chính xác khi độ sâu không vượt quá khoảng $6\xi_c$, một quy tắc có vẻ phổ quát qua dữ liệu, optimizer, và thời gian huấn luyện khác nhau.

Nguồn gốc: https://arxiv.org/abs/1611.01232

Bài 28

Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021)

Tác giả: Michael M. Bronstein (Imperial College London / Twitter), Joan Bruna (New York University), Taco Cohen (Qualcomm AI Research), Petar Veličković (DeepMind).
Venue / arXiv: arXiv:2104.13478 (v2, 02/05/2021). Paper không nêu venue hội nghị được bình duyệt: đây là một cuốn sách/monograph dài (phần nội dung chính dài khoảng 127 trang, cộng thêm danh mục tham khảo rất dài phía sau). Lời cảm ơn ở cuối bài có nhắc việc nội dung được trình bày dưới dạng bài keynote tại ICLR 2021, nhưng bản thân văn bản không phải một bài báo hội nghị qua phản biện.
Nguồn: https://arxiv.org/abs/2104.13478
Chủ đề: một khung lý thuyết (gọi là ”Geometric Deep Learning”, GDL) thống nhất phần lớn kiến trúc deep learning phổ biến (CNN, GNN, Deep Sets, Transformer, Mesh CNN, Spherical CNN, RNN/LSTM) dưới bốn nguyên lý hình học chung: đối xứng (nhóm bất biến/tương đương), ổn định biến dạng, tách biệt tỉ lệ, và một ”blueprint” gồm 4 khối xây dựng; áp dụng lên 5 miền hình học gọi là ”5 Gs”: Grids (lưới), Groups (nhóm/không gian đồng nhất), Graphs (đồ thị/tập hợp), Geodesics (đa tạp/metric nội tại), Gauges (trường quy chiếu cục bộ trên bundle). Phạm vi đọc của ghi chú này: đọc kỹ (deep read) chương 1, 2 (2.1-2.2), 3 (toàn bộ 3.1-3.5), 4 (toàn bộ 4.1-4.6), và 5.1-5.5; chương 5.6-5.8 (Mesh CNN, RNN, LSTM), chương 6 (ứng dụng) và chương 7 (lịch sử) được đọc ở mức lướt (skim) theo đúng phạm vi được giao cho bài này, dù trên thực tế người ghi chú đã đọc các phần này khá chi tiết (không chỉ dò từ khoá).
Vị trí trong lộ trình: nên đọc sau math 01 (matrix calculus) và math 02 (automatic differentiation) để quen ký hiệu đạo hàm/toán tử tuyến tính dùng xuyên suốt bài. Nên đọc cùng lúc hoặc trước llm 01 (Attention is All You Need), llm 29 (ViT), llm 21 (Mamba): bài này cung cấp một ”kính lý thuyết” (nhóm đối xứng, bất biến/tương đương) để nhìn lại ba kiến trúc đó như các trường hợp riêng của cùng một blueprint, chứ không phải ba phát minh độc lập.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu, công thức và số thứ tự phương trình lấy từ bản full text arXiv (tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú, không có trong paper. Mục ”Ứng dụng cho dự án” là suy luận. Đây là một cuốn sách rất dài, ghi chú chỉ chọn phần cốt lõi theo đúng phạm vi đọc đã nêu ở mục ”Chủ đề”.

1. Bối cảnh

Thập kỷ 2010, deep learning giải được nhiều bài toán tưởng như bất khả thi (thị giác máy tính, chơi cờ Go, dự đoán cấu trúc protein), nhưng các kiến trúc thành công (CNN, RNN, GNN, Transformer) ra đời khá độc lập, mỗi kiến trúc có động cơ và ký hiệu riêng. Paper này, viết bởi 4 tác giả từng trực tiếp đề xuất nhiều kiến trúc kể trên (Bronstein: Geodesic CNN/MoNet trên mesh; Bruna: spectral GNN đầu tiên trên graph; Cohen: group-equivariant CNN, gauge-equivariant CNN; Veličković: Graph Attention Network), đặt câu hỏi: liệu có một nguyên lý chung giải thích tại sao các kiến trúc này hoạt động?

Động lực toán học nằm ở ”curse of dimensionality” (lời nguyền chiều cao, chương 2 của paper). Nếu hàm mục tiêu f chỉ được giả định là 1-Lipschitz trên không gian d chiều, số mẫu cần để học f với sai số cố định tăng theo cấp lũy mũ với d: paper định nghĩa độ phân kỳ cực đại (maximum discrepancy) $\kappa(d) \simeq N^{-1/d}$ cho lớp hàm Lipschitz(1), nên để có $\kappa(d) \leq \epsilon$ cần $N = \Theta(\epsilon^{-d})$ mẫu. Thay giả định Lipschitz bằng lớp Sobolev H^s cũng không giúp nhiều: tốc độ hội tụ minimax là $\epsilon^{-d/s}$, chỉ cải thiện đáng kể khi độ mịn s tỉ lệ với d, một giả định phi thực tế. Mạng fully-connected với regularization thưa (sparsity) có thể phá vỡ lời nguyền này (Bach, 2017) nhưng phải giả định f chỉ phụ thuộc vào một số ít tổ hợp tuyến tính chiều thấp của đầu vào, điều cũng phi thực tế với ảnh, âm thanh, hoá học.

Lối thoát mà paper đề xuất là khai thác cấu trúc hình học sẵn có của miền dữ liệu Ω (lưới ảnh, đồ thị xã hội, mặt cầu, đa tạp 3D...) thông qua hai nguyên lý: đối xứng (symmetry) và tách biệt tỉ lệ (scale separation), lấy cảm hứng trực tiếp từ Erlangen Programme của Felix Klein (ngoài paper, kiến thức chung: 1872): ”hình học là nghiên cứu các bất biến dưới một nhóm biến đổi”. Tên ”Geometric Deep Learning” được một trong các tác giả (Bronstein) đặt ra từ 2015; cái tên rút gọn ”4G: Grids, Graphs, Groups, Gauges” do Max Welling đề xuất cho chương trình ELLIS; paper này thêm chữ G thứ 5, Geodesics, để nói về các bất biến metric nội tại trên đa tạp. Các tác giả tự nhận ”chữ G cuối (Gauges) hơi gượng ép” vì cấu trúc bên dưới thực ra là bundle/manifold chứ không hẳn là ”gauge”, nhưng vẫn giữ tên cho vần điệu.

2. Ý tưởng chính

Luận điểm trung tâm: phần lớn kiến trúc deep learning hiện có đều là một thực thể hoá của cùng một **blueprint** gồm 4 khối xây dựng (xem mục 3.4), khác nhau chỉ ở lựa chọn miền hình học Ω và nhóm đối xứng G tác động lên nó. Paper hệ thống hoá 5 miền hình học, gọi tắt ”5 Gs”:

G	Miền Ω	Nhóm đối xứng G	Kiến trúc tiêu biểu
Graphs / Sets	đồ thị, tập hợp không thứ tự	hoán vị Σ_n	GNN, Deep Sets, Transformer (trên đồ thị đầy đủ)
Grids	lưới Euclid (ảnh, chuỗi)	tịnh tiến	CNN
Groups	không gian đồng nhất (mặt cầu...)	$SO(3)$, nhóm Lie khác	Spherical CNN
Geodesics	đa tạp Riemann	đẳng cự (isometry)	Mesh/Intrinsic CNN (phần metric)
Gauges	bundle tiếp xúc / bundle đặc trưng	$SO(2)$ (đối khung quy chiếu)	gauge-equivariant Mesh CNN

Paper còn liệt kê thêm LSTM ứng với miền lưới 1D thời gian và nhóm ”time warping” (không phải nhóm tịnh tiến thường), minh hoạ rằng ngay cả tính bất biến ”lạ” (bất biến lớp, không phải bất biến zero-shot) cũng rơi vào cùng khung lý thuyết.

Hai ý chính cần nắm:

1. **Đối xứng tạo ra lớp hàm equivariant/invariant hẹp hơn, học hiệu quả hơn.** Một hàm f là G-bất biến (invariant) nếu $f(\rho(g)x) = f(x)$, và G-tương đương (equivariant) nếu $f(\rho(g)x) = \rho(g)f(x)$, với $\rho(g)$ là biểu diễn (representation) của nhóm G tác động lên không gian tín hiệu. Ví dụ convolution trong CNN là tịnh tiến-tương đương, không phải tịnh tiến-bất biến; global average pooling sau đó mới tạo ra bất biến.
2. **Bất biến đúng nhất khi kết hợp với tách biệt tỉ lệ (locality + đa tỉ lệ).** Hàm tuyến tính bất biến tuyệt đối chỉ có thể là phép lấy trung bình toàn cục (quá nghèo thông tin), nên cần xen lớp phi tuyến giữa các lớp tương đương cục bộ (local), rồi mới bất biến hoá ở lớp cuối. Đây chính là mô hình ”convolution + nonlinearity + pooling + global pooling” của CNN, tổng quát hoá thành blueprint 4 khối ở mục 3.4.

3. Phương pháp chi tiết

3.1 Nguyên lý 1: đối xứng, biểu diễn nhóm, bất biến và tương đương (chương 3.1-3.2)

Một nhóm G là tập hợp cùng phép hợp thành $\circ$ thoả 4 tiên đề: kết hợp, có phần tử đơn vị e, mọi phần tử có nghịch đảo, đóng kín. Nhóm không bắt buộc giao hoán; nhóm giao hoán ($gh = hg$ với mọi g, h) gọi là nhóm Abel. Một nhóm tác động (group action) lên miền Ω là ánh xạ $(g, u) \mapsto g.u$ thoả $g.(h.u) = (gh).u$. Từ đó suy ra tác động của G lên không gian tín hiệu $\mathcal{X}(\Omega)$ qua phương trình (3) của paper:

$$(g.x)(u) = x(g^{-1}u)$$

Biểu diễn nhóm (group representation) $\rho : G \to \mathbb{R}^{n\times n}$ gán cho mỗi g một ma trận khả nghịch $\rho(g)$ thoả $\rho(gh) = \rho(g)\rho(h)$. Hai định nghĩa trung tâm của toàn bài:

- f là G-bất biến nếu $f(\rho(g)x) = f(x)$ với mọi $g \in G$.
- f là G-tương đương nếu $f(\rho(g)x) = \rho(g)f(x)$ với mọi $g \in G$.

Ví dụ kinh điển: một lớp convolution trong CNN là tịnh tiến-tương đương (dịch ảnh đầu vào thì feature map đầu ra dịch theo cùng lượng), còn hàm phân loại ảnh toàn cục (dán nhãn "mèo"/"chó") cần tịnh tiến-bất biến. Multi-layer Perceptron xấp xỉ được mọi hàm trơn nhưng không có tính tương đương này, một phần lý do các mô hình pattern recognition đầu tiên (thập niên 1970) thất bại trên ảnh.

Chương 3.2 phân biệt **isomorphism** (ánh xạ song ánh bảo toàn cấu trúc giữa hai đối tượng khác nhau, ví dụ hai đồ thị đẳng cấu chỉ khác thứ tự đánh số nút) và **automorphism** (isomorphism từ một đối tượng về chính nó, tức là một đối xứng thật sự). Tuỳ mức cấu trúc cần bảo toàn mà ta có các lớp ánh xạ lồng nhau: song ánh (set) $\supset$ homeomorphism (bảo toàn liên tục, không gian topo) $\supset$ diffeomorphism (bảo toàn vi phân, đa tạp trơn) $\supset$ isometry (bảo toàn khoảng cách, không gian metric). Thêm cấu trúc thì nhóm đối xứng càng nhỏ: $Diff(\Omega) \supset E(2) \supset SE(2)$ (nhóm Euclid đầy đủ rồi tới nhóm chỉ giữ hướng), phản ánh đúng thứ bậc hình học Xạ ảnh $\supset$ Affine $\supset$ Euclid của Erlangen Programme.

3.2 Nguyên lý 2: ổn định biến dạng (chương 3.3)

Yêu cầu bất biến/tương đương chính xác với một nhóm cố định là quá cứng: biến dạng nhỏ (ví dụ hai vật thể trong video di chuyển theo hai hướng khác nhau) không tạo thành nhóm (hợp của nhiều biến dạng nhỏ có thể thành biến dạng lớn, phá vỡ tính đóng kín). Paper thay bằng khái niệm **ổn định hình học** (geometric stability), đo "khoảng cách" $c(\tau)$ từ một phép biến dạng $\tau \in Diff(\Omega)$ tới nhóm đối xứng G ($c(\tau) = 0$ khi $\tau \in G$), và yêu cầu (phương trình 4):

$$\|f(\rho(\tau)x) - f(x)\| \le C\, c(\tau)\, \|x\|, \quad \forall x \in \mathcal{X}(\Omega)$$

Một lựa chọn phổ biến cho ảnh liên tục là năng lượng Dirichlet $c^2(\tau) := \int_\Omega \|\nabla\tau(u)\|^2 du$, đo độ "đàn hồi" của τ so với trường dịch chuyển hằng. Khi miền Ω chính nó biến dạng (ví dụ đồ thị xã hội thay đổi theo thời gian, hoặc vật thể 3D không cứng), paper định nghĩa khoảng cách domain $d_{\mathcal{D}}(\Omega, \tilde{\Omega})$ (ví dụ graph edit distance, hoặc khoảng cách Gromov-Hausdorff cho đa tạp Riemann) và yêu cầu tương tự (phương trình 5):

$$\|f(x,\Omega) - f(\tilde{x},\tilde{\Omega})\| \le C\|x\|\, d_{\mathcal{D}}(\Omega,\tilde{\Omega})$$

Ý quan trọng: biểu diễn Fourier (toàn cục) ổn định với đúng nhóm đối xứng (ví dụ tịnh tiến) nhưng **không** ổn định với biến dạng nhỏ gần nhóm đó: với tịnh tiến gần đúng $\tau(u) = u - \tilde{\tau}(u)$, $\|\nabla\tilde{\tau}\|_\infty \le \epsilon$ nhỏ bất kỳ, hàm module-Fourier $f(x) = |\hat{x}|$ vẫn có $\|f(\rho(\tau)x) - f(x)\|/\|x\| = O(1)$, không giảm theo ϵ. Ngược lại biểu diễn wavelet (đa tỉ lệ, cục bộ cả không gian và tần số) ổn định hơn: $\|\rho(\tau)(W_\psi x) - W_\psi(\rho(\tau)x)\|/\|x\| = O(\epsilon)$ (Mallat, 2012). Đây là lý do toán học cho việc dùng filter cục bộ (như convolution kernel 3x3) thay vì biến đổi Fourier toàn cục.

3.3 Nguyên lý 3: tách biệt tỉ lệ (chương 3.4)

Ổn định biến dạng thu hẹp không gian hàm hợp lệ nhưng chưa đủ phá lời nguyền chiều cao. Paper đề xuất phân rã Ω thành một dãy miền thô dần $\Omega_1, \dots, \Omega_J$ (coarsening, ví dụ pooling trong CNN hoặc gộp cụm trong GNN), và nói một hàm f "ổn định cục bộ tại tỉ lệ j" nếu $f \approx f_j \circ P_j$ với $P_j : \mathcal{X}(\Omega) \to \mathcal{X}_j(\Omega_j)$ là một phép rút gọn phi tuyến. Nghĩa là thay vì học tương tác tầm xa trực tiếp, mô hình học tương tác cục bộ trước, rồi

truyền lên tỉ lệ thô hơn dần, đúng tinh thần renormalization group trong vật lý thống kê và Fast Multipole Method trong tính toán số.

3.4 Blueprint của Geometric Deep Learning (chương 3.5, "mục quan trọng nhất của paper")

Quan sát mở đầu: nếu f tuyến tính và G-bất biến thì f chỉ có thể là trung bình nhóm $Ax = \frac{1}{\mu(G)} \int_G (g.x)\, d\mu(g)$ (với ảnh và nhóm tịnh tiến, điều này suy biến thành "chỉ dùng màu RGB trung bình của ảnh", quá nghèo). Ngược lại họ hàm tuyến tính G-**tương đương** (không bất biến) phong phú hơn nhiều, và có thể ghép với phi tuyến σ bất kỳ mà vẫn giữ tương đương: nếu B là G-tương đương và σ áp dụng theo từng điểm thì $\sigma \circ B$ vẫn G-tương đương. Từ đó paper định nghĩa 4 khối xây dựng:

- **Lớp tuyến tính G-tương đương** $B : \mathcal{X}(\Omega, C) \to \mathcal{X}(\Omega', C')$, $B(g.x) = g.B(x)$.
- **Phi tuyến theo điểm** $\sigma : C \to C'$, áp dụng từng toạ độ: $(\sigma(x))(u) = \sigma(x(u))$.
- **Local pooling (coarsening)** $P : \mathcal{X}(\Omega, C) \to \mathcal{X}(\Omega', C)$ với Ω' là bản thu gọn của Ω.
- **Lớp G-bất biến (global pooling)** $A : \mathcal{X}(\Omega, C) \to Y$, $A(g.x) = A(x)$.

Ghép lại thành công thức chung cho toàn bộ "Geometric Deep Learning Blueprint":

$$f = A \circ \sigma_J \circ B_J \circ P_{J-1} \circ \cdots \circ P_1 \circ \sigma_1 \circ B_1$$

Mỗi lớp B_j có thể dùng nhóm G khác nhau. Bảng tương ứng kiến trúc, miền, nhóm mà paper đưa ra ngay sau blueprint này chính là nguồn của bảng 5G ở mục 2; CNN = (Grid, tịnh tiến), Spherical CNN = (mặt cầu, $SO(3)$), Intrinsic/Mesh CNN = (đa tạp, đẳng cự hoặc gauge $SO(2)$), GNN/Deep Sets/Transformer = (đồ thị/tập/đồ thị đầy đủ, hoán vị Σ_n), LSTM = (lưới 1D, time warping).

3.5 Năm miền hình học, "5 Gs" (chương 4.1-4.6)

Graphs và Sets (4.1). Tập hợp là đồ thị không cạnh. Với ma trận đặc trưng $X \in \mathbb{R}^{n \times d}$ (mỗi dòng một nút), phép hoán vị $g \in \Sigma_n$ ứng với ma trận hoán vị P (mỗi dòng/cột đúng một số 1). $f(X)$ bất biến hoán vị nếu $f(PX) = f(X)$; dạng đơn giản nhất (phương trình 6): $f(X) = \phi(\sum_u \psi(x_u))$. $F(X)$ tương đương hoán vị nếu $F(PX) = PF(X)$; phép tuyến tính đơn giản nhất là $F_\Theta(X) = X\Theta$ (phương trình 7). Khi có cạnh, ma trận kề A (phương trình 8) cũng bị hoán vị theo $PAP^\top$, và định nghĩa bất biến/tương đương tổng quát hoá thành $f(PX, PAP^\top) = f(X, A)$ (phương trình 9) và $F(PX, PAP^\top) = PF(X, A)$ (phương trình 10). Với lân cận 1-hop N_u (phương trình 11) và multiset đặc trưng lân cận X_{N_u} (phương trình 12), blueprint cho đồ thị là áp một hàm cục bộ $\phi(x_u, X_{N_u})$ bất biến hoán vị lên từng nút (phương trình 13): $F(X, A) = (\phi(x_1, X_{N_1}), \ldots, \phi(x_n, X_{N_n}))^\top$. Khi ϕ là đơn ánh (injective), một bước của ϕ tương đương một bước của phép kiểm thử đẳng cấu Weisfeiler-Lehman (WL test).

Grids (4.2). Lưới là đồ thị đặc biệt với thứ tự nút cố định (vòng $\mathbb{Z}_n$). Vì lân cận có thứ tự ("trái"/"phải"), ϕ không cần bất biến hoán vị nữa: $\phi(x_{u-1}, x_u, x_{u+1}) = \theta_{-1}x_{u-1} + \theta_0 x_u + \theta_1 x_{u+1}$ tạo ra ma trận circulant $C(\theta)$, và $C(\theta)x = x \star \theta$ chính là convolution rời rạc. Các ma trận circulant giao hoán với nhau và với toán tử dịch (shift) S: $SC(\theta)x = C(\theta)Sx$, tức convolution là tịnh tiến-tương đương; ngược lại, một toán tử tuyến tính tịnh tiến-tương đương bắt buộc phải là convolution. Paper suy ra biến đổi Fourier rời rạc (DFT) từ nguyên lý đối xứng thuần túy: các ma trận circulant giao hoán nên đồng chéo hoá được, và vì toán tử dịch S là circulant, các trị riêng của nó (căn bậc n của đơn vị) chính là cơ sở Fourier $\phi_k = \frac{1}{\sqrt{n}}(1, e^{2\pi i k/n}, \ldots)$. Đây là một trong những đoạn đẹp nhất của paper: Fourier transform không phải "cho sẵn" mà được suy ra từ yêu cầu đồng chéo hoá các toán tử tương đương-tịnh tiến.

Groups và Homogeneous spaces (4.3). Tổng quát hoá convolution: với nhóm G tác động lên Ω và biểu diễn ρ, group convolution (phương trình 14):

$$(x \star \theta)(g) = \langle x, \rho(g)\theta \rangle = \int_\Omega x(u)\, \theta(g^{-1}u)\, du$$

luôn G-tương đương, suy trực tiếp từ $\langle x, \rho(g)\theta\rangle = \langle \rho(g^{-1})x, \theta\rangle$. Với $\Omega = \mathbb{Z}_n$, $G = \mathbb{Z}_n$ (dịch vòng), công thức này quy về convolution rời rạc thông thường. Với mặt cầu $\Omega = S^2$, $G = SO(3)$ (xoay 3D), $x \star \theta$ là hàm trên $SO(3)$ (không còn trên Ω), nên lớp tiếp theo phải xử lý tín hiệu trên chính nhóm G (dùng "regular representation" của G tác động lên $\mathcal{X}(G)$). Ω với tính chất "mọi điểm nhìn giống nhau" (với mọi u, v tồn tại g sao cho $g.u = v$) gọi là không gian đồng nhất (homogeneous space).

Geodesics và Manifolds (4.4). Trên đa tạp Riemann, không còn nhóm tịnh tiến toàn cục; thay vào đó dùng cấu trúc metric nội tại: độ dài đường đi geodesic, tensor metric $g_u(X, Y) = \langle X, Y\rangle_u$ trên không gian tiếp xúc $T_u\Omega$. Toán tử Laplace-Beltrami $\Delta = \nabla^*\nabla$ là tự liên hợp (self-adjoint), có hệ trị riêng không âm $\lambda_1 \le \lambda_2 \le \dots$ và hệ hàm riêng trực giao φ_k ($\Delta\varphi_k = \lambda_k\varphi_k$) đóng vai trò cơ sở Fourier tổng quát hoá cho tín hiệu trên đa tạp: $x = \sum_k \langle x, \varphi_k\rangle\varphi_k$. Từ đó có hai cách định nghĩa convolution trên đa tạp: (i) spectral, áp hàm truyền $\hat{p}(\lambda)$ lên từng trị riêng, $\hat{p}(\Delta)x = \Phi\,\hat{p}(\Lambda)\,\Phi^\top x$ (công thức này xuất hiện lại y hệt ở mục 4.6 cho mesh); (ii) spatial, dùng ánh xạ mũ (exponential map) để "trải" lân cận u ra mặt phẳng tiếp xúc và tích chập cục bộ trên đó, cần chọn một khung quy chiếu (gauge) tại mỗi điểm, đồng thời đây chính là chỗ nảy sinh vấn đề mơ hồ gauge dẫn tới mục 4.5.

Gauges và Bundles (4.5). Một gauge là một khung cơ sở (frame) cho không gian tiếp xúc $T_u\Omega$ tại mỗi điểm u. Đổi gauge là một ánh xạ $g : \Omega \to G$ (với $G \subseteq GL(s)$, ví dụ $SO(2)$ hay $O(2)$ cho đa tạp 2D), gọi là "structure group" của bundle. Ví dụ trực quan paper dùng: ảnh RGB có thể coi là một bundle với fibre $\mathbb{R}^3$ (3 kênh màu) gắn tại mỗi pixel, và hoán vị thứ tự R-G-B tại mỗi pixel (nhóm Σ_3) là một phép đổi gauge; một phép convolution 1x1 $f : \mathbb{R}^3 \to \mathbb{R}^C$ là "gauge-tương đương" nếu $f \circ \rho_{in}(g) = \rho_{out}(g) \circ f$, nghĩa là biểu diễn toạ độ của f không đổi khi đổi gauge (giống hệt tinh thần permutation-tương đương trên đồ thị, chỉ khác là phép biến đổi tác động riêng lẻ lên từng điểm u chứ không tác động lên toàn miền Ω). Với trường vô hướng (scalar field, không có hướng), mọi phép tuyến tính đều tự động gauge-tương đương (trường hợp "tầm thường"). Với trường vector (ví dụ vận tốc), cần dùng connection để chuyển song song (parallel transport) vector từ điểm v sang điểm u trước khi áp filter: $(x \star \Theta)(u) = \int_\Omega \Theta(u, v)\,\rho(g_{v\to u})\,x(v)\,dv$ (phương trình 23), với $g_{v\to u} \in G$ là phần tử nhóm mô tả phép chuyển song song dọc geodesic nối v, u.

Geometric graphs và Meshes (4.6). Mesh tam giác $T = (V, E, F)$ là đồ thị có thêm cấu trúc mặt (face), xấp xỉ một đa tạp 2D liên tục. Laplacian rời rạc tổng quát (phương trình 24): $(\Delta X)_u = \sum_{v\in N_u} w_{uv}(x_u - x_v)$, viết dưới dạng ma trận $\Delta = D - W$ (D ma trận bậc). Với đồ thị thường, $W = A$ (ma trận kề); với mesh, trọng số cotangent (phương trình 25) $w_{uv} = \frac{\cot\angle uqv + \cot\angle upv}{2a_u}$ cho Laplacian hội tụ về toán tử Laplace-Beltrami liên tục khi mesh càng mịn, và bất biến đẳng cự (isometry): biến dạng không kéo giãn cạnh (giữ nguyên ℓ_{uv}) không làm thay đổi Laplacian. Vì Fourier phổ trên mesh/graph rất nhạy với nhiễu nhỏ trên cấu trúc (không ổn định khi so sánh hai mesh/graph khác nhau, xem mục 6), paper khuyến nghị dùng filter đa thức (polynomial, Defferrard et al. 2016, cục bộ, độ phức tạp $O(|E|)$) hoặc filter hữu tỉ (rational, ví dụ Cayley transform, không cục bộ nhưng decay mũ và ổn định hơn với biến dạng gần-đẳng-cự).

3.6 Năm kiến trúc tương ứng (chương 5.1-5.5, đọc kỹ)

- **CNN (5.1).** Mọi filter compact support viết được là tổ hợp tuyến tính của các "đỉnh đơn vị" θ_{vw} (phương trình 26), cho ra convolution 2D quen thuộc $F(x)_{uv} = \sum_{a,b} \alpha_{ab} x_{u+a,v+b}$ (phương trình 27), tổng quát hoá đa kênh ở phương trình 28. Một lớp CNN "chuẩn" là $h = P(\sigma(F(x)))$ (phương trình 29: tương đương-tuyến tính, phi tuyến, coarsening); ResNet đổi thành $h = P(x + \sigma(F(x)))$ (phương trình 30), tương đương một bước Euler của ODE $\dot{x} = \sigma(F(x))$, lý do huấn luyện ResNet sâu dễ hơn.
- **Group-equivariant CNN (5.2).** Tổng quát hoá CNN sang nhóm rời rạc lớn hơn tịnh tiến, ví dụ $G = \mathbb{Z}^3 \rtimes O_h$ (tịnh tiến + xoay 90 độ rời rạc) cho ảnh y khoa 3D, hoặc nhóm reverse-complement $\mathbb{Z}_2$ cho chuỗi DNA (hai mạch đọc ngược và bù nhau). Thực hiện bằng "transform rồi convolve": tạo các bản sao filter đã xoay $\theta_h = \rho(h)\theta$ cho mỗi $h \in H$ rồi convolution tịnh tiến như bình thường (phương trình 32), nên tính toán hiệu quả trên GPU như CNN thường.
- **GNN (5.3).** Ba "hương vị" (flavour) tổng quát hoá $\phi(x_u, X_{N_u})$: convolutional (trọng số cố định c_{uv}, phương trình 33), attentional (trọng số học qua attention $a(x_u, x_v)$, phương trình 34), message-passing

(message vector tuỳ ý $\psi(x_u, x_v)$, phương trình 35). Có quan hệ bao hàm chặt: convolutional $\subseteq$ attentional $\subseteq$ message-passing (mỗi hương vị sau biểu diễn được hương vị trước bằng cách chọn hàm phù hợp).

- **Deep Sets, Transformer, Latent graph inference (5.4).** Với tập hợp không đồ thị, ba lựa chọn cạnh: đồ thị trống $A = I$ (Deep Sets, $h_u = \psi(x_u)$), đồ thị đầy đủ $A = \mathbf{1}\mathbf{1}^\top$ với attention (phương trình 36) chính là self-attention, "trái tim" của Transformer, hay đồ thị suy luận (latent graph inference, A học được). Điểm mấu chốt: **Transformer = GNN hương vị attentional trên đồ thị đầy đủ** (complete graph), cộng thêm positional encoding để bù lại việc đồ thị đầy đủ "quên" thứ tự chuỗi; paper chỉ ra positional encoding kiểu sin/cos liên hệ trực tiếp tới DFT, tức là tới các vector riêng (eigenvector) của Laplacian của một "lưới vòng" (circular grid) đúng như mục 4.2.
- **Equivariant Message Passing Networks (5.5).** Khi đặc trưng nút gồm cả toạ độ không gian $x_u \in \mathbb{R}^3$ (ví dụ nguyên tử trong phân tử), cần tương đương với nhóm Euclid $E(3)$ (tịnh tiến, xoay, phản xạ) ngoài tương đương hoán vị. Mô hình của Satorras et al. (2021): $f'_u = \phi(f_u, \bigoplus_v \psi_f(f_u, f_v, \|x_u - x_v\|^2))$ và $x'_u = x_u + \sum_v (x_u - x_v)\psi_c(f_u, f_v, \|x_u - x_v\|^2)$, tương đương $E(3)$ nhờ chỉ phụ thuộc khoảng cách (bất biến $E(3)$). Hai hướng tổng quát hơn: biểu diễn khả quy (irreducible representations, dùng ma trận Wigner-D, ví dụ Tensor Field Networks, SE(3)-Transformer) và biểu diễn chính qui (regular representations, dùng với nhóm Lie qua exponential/logarithm map, ví dụ LieConv).

3.7 Phép tính "tự tính": kiểm chứng số tính tương đương hoán vị

Để kiểm tra cụ thể công thức $F(PX, PAP^\top) = PF(X, A)$ (phương trình 10, mục 4.1), lấy đồ thị đường đi 3 nút $1-2-3$ (cạnh $(1,2)$ và $(2,3)$), đặc trưng vô hướng $x = (2, 5, 9)$ cho các nút $1, 2, 3$. Hàm cục bộ chọn đơn giản $\phi(x_u, X_{N_u}) = \sum_{v \in N_u} x_v$ (tổng đặc trưng lân cận, dạng đặc biệt của phương trình 13 với $\psi = \text{id}$).

Ma trận kề: $A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$. Tính $F(x, A) = Ax = (x_2,\ x_1 + x_3,\ x_2) = (5,\ 11,\ 5)$.

Lấy phép hoán vị P đổi nhãn nút 1 và 2 (giữ nút 3): $P = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$. Tín hiệu hoán vị: $Px = (5, 2, 9)$. Đồ thị hoán vị tương ứng $A' = PAP^\top$: đổi nhãn 1 với 2 làm cạnh $(1,2)$ giữ nguyên (đối xứng) còn cạnh $(2,3)$ cũ thành cạnh $(1,3)$ mới, nên $A' = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$.

Tính $F(Px, A') = A'(Px)$: nút 1 mới có lân cận $\{2,3\}$ nên giá trị $= (Px)_2 + (Px)_3 = 2 + 9 = 11$; nút 2 mới có lân cận $\{1\}$ nên giá trị $= (Px)_1 = 5$; nút 3 mới có lân cận $\{1\}$ nên giá trị $= 5$. Vậy $F(Px, A') = (11, 5, 5)$.

So sánh với $PF(x, A)$: $F(x, A) = (5, 11, 5)$, áp P (đổi chỗ phần tử 1 và 2) cho $(11, 5, 5)$. Hai kết quả khớp nhau: $F(Px, A') = PF(x, A) = (11, 5, 5)$ (tự tính), xác nhận bằng số tính tương đương hoán vị của phép tổng lân cận. Ngược lại, nếu dùng hàm tổng toàn cục $f(x) = \sum_u x_u$ (không phụ thuộc A) thì $f(x) = f(Px) = 16$ với mọi hoán vị (tự tính): đây là ví dụ số cho tính **bất biến** (không phải tương đương) của khối global pooling A trong blueprint mục 3.4, minh hoạ sự khác biệt giữa lớp tương đương cục bộ B (giá trị theo từng nút, đổi theo hoán vị) và lớp bất biến toàn cục (giá trị vô hướng, không đổi).

4. Thí nghiệm và kết quả chính

Đây là paper lý thuyết/khảo sát (expository monograph), không có thí nghiệm mới do 4 tác giả tự chạy, không có bảng benchmark riêng. "Kết quả chính" của paper là các suy diễn toán học (trình bày ở mục 3): (i) blueprint 4 khối thống nhất CNN/GNN/Transformer/Mesh CNN/Spherical CNN/LSTM; (ii) chứng minh DFT là hệ hàm

riêng của toán tử dịch (suy từ nguyên lý đối xứng, không giả định trước); (iii) công thức group convolution tổng quát (phương trình 14); (iv) các điều kiện tương đương gauge trên mesh (phương trình 23, 37).

Chương 6 (đọc lướt) tổng hợp lại rất nhiều kết quả thực nghiệm **của các paper khác** để minh chứng blueprint có giá trị thực tiễn, không phải thí nghiệm của chính 4 tác giả: GNN tìm ra Halicin, một kháng sinh mới hiệu quả cả với vi khuẩn kháng thuốc (Stokes et al., 2020); AlphaFold của DeepMind dùng contact graph để dự đoán cấu trúc protein; mô hình GNN dự đoán ETA (thời gian đến) của Google Maps, cải thiện 40%+ ở một số thành phố như Sydney (Derrow-Pinion et al., 2021); CNN (AlexNet) giảm mạnh lỗi phân loại trên ImageNet (2012) mở đầu làn sóng deep learning hiện đại; AlphaGo/AlphaZero/MuZero dùng CNN xử lý bàn cờ 19×19; GPT-3 (Transformer, 175 tỷ tham số, Brown et al., 2020) minh chứng self-attention trên đồ thị đầy đủ của từ; IceCube (khối băng 1 km^3 ở Nam Cực) dùng geometric neural network xử lý hình học bất thường của detector để phát hiện neutrino vũ trụ. Paper dùng các ví dụ này như ”bằng chứng gián tiếp” rằng kiến trúc tuân theo đúng nhóm đối xứng của bài toán (chứ không phải kiến trúc tổng quát bất kỳ) mới đạt hiệu năng cao trong thực tế.

5. Phân tích / ablation

Vì không có thí nghiệm ablation thật, mục này tổng hợp các phân tích lý thuyết/so sánh trường hợp đặc biệt mà paper trình bày:

- **Thứ bậc biểu cảm của 3 hương vị GNN:** convolutional $\subseteq$ attentional $\subseteq$ message-passing (chứng minh trực tiếp bằng cách chọn $a(x_u, x_v) = c_{uv}$ hoặc $\psi(x_u, x_v) = c_{uv}\psi(x_v)$). Message-passing mạnh nhất nhưng tốn bộ nhớ nhất (phải giữ vector message trên từng cạnh); convolutional rẻ nhất, phù hợp với đồ thị ”homophilous” (nút liên kết có xu hướng cùng nhãn).
- **Giới hạn biểu cảm bởi WL test:** Xu et al. (2018) và Morris et al. (2019) chứng minh bất kỳ GNN nào thuộc 3 hương vị trên không thể mạnh hơn phép kiểm thử đẳng cấu Weisfeiler-Lehman (1-WL). Điều kiện để đạt đúng mức WL là hàm tổng hợp lân cận phải đơn ánh (injective); tổng (sum) là đơn ánh, còn max/mean thì không (ví dụ không phân biệt được multiset $\{\{a, b\}\}$ và $\{\{a, a, b, b\}\}$), dẫn tới thiết kế Graph Isomorphism Network (GIN) dùng sum-aggregation.
- **Nâng cấp biểu cảm vượt WL:** k-WL test / k-GNN (Morris et al., 2019) mạnh hơn nhưng phải giữ biểu diễn trên k-tuple nút, không scale quá $k = 3$ trong thực tế; Maron et al. (2018, 2019) cho thấy không gian hàm tương đương/bất biến trên k-tuple là tổ hợp tuyến tính của một số generator hữu hạn (không phụ thuộc n, chỉ phụ thuộc k).
- **Isotropic vs anisotropic filter trên mesh (5.6):** filter đẳng hướng $\theta(r)$ (như spectral/Laplacian) bất biến gauge hoàn toàn nhưng mất thông tin hướng (không bắt được đặc trưng dạng biên/cạnh); filter không đẳng hướng cần chọn gauge cố định (Monti et al., 2017, dùng hướng cong chính, không nội tại, mơ hồ tại điểm phẳng) hoặc dùng angular max-pooling (bất biến nhưng tốn tính toán quét mọi góc xoay θ_0) hoặc dùng gauge-equivariant filter (phương trình 37, tổng quát nhất, cần parallel transport $\rho(g_{v\to u})$ giữa các gauge).
- **Biểu diễn khả quy vs chính quy cho $E(3)$-equivariance (5.5):** irreducible representations (dùng Wigner-D, Clebsch-Gordan) chỉ áp dụng cho nhóm compact, cần giải constraint tương đương trực tiếp; regular representations tổng quát hơn (dùng được với nhóm Lie qua Lie algebra) nhưng tốn bộ nhớ lưu bản sao đặc trưng cho mọi phần tử nhóm.
- **Data augmentation vs kiến trúc tương đương:** augmentation (xoay/lật ảnh khi huấn luyện) phổ biến và hiệu quả thực nghiệm, nhưng được chứng minh là ”sub-optimal” về sample complexity so với kiến trúc có nhóm bất biến lớn hơn ngay từ kiến trúc (Mei et al., 2021), tức augmentation là giải pháp ”vá” chứ không tối ưu bằng việc mã hoá đối xứng trực tiếp vào mô hình.
- **SimpleRNN không tịnh tiến-tương đương, trừ khi đệm 0 và chọn $h^{(0)}$ là điểm bất động:** paper chứng minh rõ (phương trình 40-42) rằng dịch chuỗi đi 1 bước chỉ cho ra tính tương đương nếu $h^{(0)} = R(0, h^{(0)})$, một điều kiện không tự động thoả, đòi hỏi đệm 0 đủ dài và lặp R tới điểm bất động (dùng Banach Fixed Point Theorem).

6. Giới hạn

- Giới hạn biểu cảm cơ bản: mọi GNN 3-hương-vị không mạnh hơn 1-WL test, nên không phân biệt được một số cặp đồ thị không đẳng cấu đơn giản (ví dụ 6-cycle và hai tam giác rời).
- Biến đổi Fourier phổ trên đồ thị/mesh (graph Fourier transform) phụ thuộc cấu trúc cụ thể của từng đồ thị, không chuyển giao được giữa hai đồ thị khác nhau, và rất nhạy với nhiễu nhỏ trên cấu trúc (xem Hình 12 của paper, không đọc kỹ phần này): mô hình huấn luyện trên một tập mesh/graph rồi test trên tập khác (rất phổ biến trong computer graphics và nhiều bài toán ML trên graph) sẽ gặp vấn đề nếu dùng trực tiếp Fourier phổ, nên phải chuyển sang filter đa thức/hữu tỉ.
- Tính bất biến thời gian-co-dãn (time warping invariance) của gated RNN (LSTM) là bất biến theo **lớp mô hình** (class invariance), không phải zero-shot: huấn luyện trên một phép co dãn τ_1 rồi test trên $\tau_2 \neq \tau_1$ nói chung cần huấn luyện lại tham số (trừ trường hợp đặc biệt $\tau_2 = \alpha\tau_1$, chỉ cần scale lại gate).
- Lý thuyết depth-vs-width tối ưu cho CNN ”vẫn còn chưa có câu trả lời chặt” (lời paper tự nhận ở 5.1); lý thuyết giải thích vì sao batch normalization hiệu quả ”vẫn còn thiếu” (paper tự nhận ở 5.1).
- Graph isomorphism (bài toán quyết định hai đồ thị có đẳng cấu hay không, nền tảng của WL test) chưa biết có thuộc lớp P hay không; thuật toán tốt nhất đã biết chắc chắn là của Babai và Luks (1983); Babai (2016) đưa ra một đề xuất (chưa được review đầy đủ tại thời điểm paper viết) ngụ ý một giải pháp thời gian quasi-polynomial.
- Bản thân cách đặt tên ”5 Gs” có phần khiên cưỡng: tác giả tự nhận chữ G thứ 5 (Gauges) ”hơi gượng ép” vì cấu trúc bên dưới là bundle/manifold, không hẳn là một khái niệm ”gauge” độc lập.
- Higher-order k-GNN mạnh hơn WL nhưng không scale thực tế quá $k = 3$ do chi phí lưu trữ k-tuple.
- Phần đọc lướt của ghi chú (chương 6, 7): đây là các chương tổng hợp ứng dụng và lịch sử, không có thêm định lý/giới hạn kỹ thuật mới so với chương 3-5, nên việc đọc lướt không bỏ lỡ nội dung lý thuyết cốt lõi, nhưng ghi chú này không đảm bảo đã nắm hết mọi trích dẫn ứng dụng liệt kê trong hai chương đó.

7. Liên hệ với các bài khác

Đã chạy `ls "math content/"` ngay trước khi viết mục này: các file 29, 30, 32 đã tồn tại (29_2022_understanding_diffusion_models.md, 30_2017_opening_black_box_information.md, 32_2019_benign_overfitting_linear_regression.md), còn 31, 33, 34, 35 chưa có. Không có liên hệ nội dung trực tiếp đủ chặt với 3 bài đó (diffusion model, information bottleneck, benign overfitting đều không nằm trong phạm vi đã đọc của bài 28 này), nên không trích dẫn gượng ép. Ba liên hệ dưới đây đã được kiểm tra tồn tại file thật bằng `ls "llm content/"` và kiểm tra nội dung tương ứng ngay trong paper 28 (không suy diễn):

- **llm 01 (Attention is All You Need).** Mục 5.4 của paper này nói thẳng: ”we can pose Transformers exactly as attentional GNNs over a complete graph” (Transformer chính là GNN hương vị attentional trên đồ thị đầy đủ các từ), và mục 7 (lịch sử) nhắc lại: cơ chế attention đầy đủ (fully-connected GNN) ”kicked off” Transformer, ”de-throned both recurrent and convolutional models” trong dịch máy. Positional encoding của Transformer được paper liên hệ trực tiếp tới DFT, tức tới các vector riêng của Laplacian của lưới vòng (mục 4.2). Nên đọc bài 28 trước hoặc cùng llm 01 để thấy self-attention không phải phát minh độc lập mà là một thực thể hoá cụ thể của blueprint mục 3.4 với $G = \Sigma_n$, $\Omega =$ đồ thị đầy đủ.
- **llm 29 (ViT, Vision Transformer).** ViT chia ảnh thành các patch rồi xử lý như một tập token bằng Transformer, tức là chuyển miền từ Grid (4.2, nơi CNN khai thác đối xứng tịnh tiến bằng weight sharing dạng circulant) sang Graph/Set đầy đủ (4.1/5.4, nơi vị trí patch chỉ còn là positional encoding cộng thêm, không còn là đối xứng cấu trúc). Đọc bài 28 giúp thấy rõ ViT ”đánh đổi” một đối xứng mạnh (tịnh tiến, có cấu trúc tham số ít $\Theta(1)$/lớp như mục 4.2 chỉ ra) để lấy một mô hình tương tác toàn cục (attention đầy đủ, tốn $O(n^2)$).
- **llm 21 (Mamba, SSM).** Mục 5.7-5.8 của paper 28 (đọc kỹ) phân tích RNN/LSTM dưới góc nhìn hình học: RNN chỉ tịnh tiến-tương đương nếu đệm 0 đủ dài và trạng thái khởi tạo là điểm bất động (mục

3.7 ở trên), và lớp gated RNN (LSTM/GRU) đạt được tính bất biến lớp với phép co-dãn thời gian (time warping) nhờ cơ chế gate xấp xỉ đạo hàm $d\tau/dt$ (phương trình 49-53). Mamba là một state-space model (SSM) chọn lọc, cũng xử lý tín hiệu trên lưới 1D thời gian và cũng cần cơ chế gate/chọn lọc để kiểm soát tốc độ "quên" theo thời gian, tức nằm trong đúng họ kiến trúc "LSTM-như" mà mục 5.8 mô tả bằng ngôn ngữ hình học (dù paper 28 viết năm 2021, trước Mamba, nên không nhắc tên Mamba; đây là liên hệ do người ghi chú suy ra, không phải paper nói thẳng).

8. Ứng dụng cho dự án (suy luận)

Dự án đang nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm 1.500 token text và 1.6-2.1K token ảnh, chạy trên GPU ≤ 8GB, batch size 1). Đánh giá trung thực mức độ liên quan (đây hoàn toàn là suy luận, không phải nội dung paper nói):

- Phần **full attention** trong LightOnOCR đúng là một lớp attentional-GNN trên đồ thị đầy đủ của token (mục 5.4), nên tương đương hoán vị đối với thứ tự token nếu bỏ positional encoding, giống hệt phân tích Transformer ở mục 7. Hiểu điều này giúp giải thích tại sao thứ tự token ảnh (patch) chỉ ảnh hưởng qua positional encoding cộng thêm, không qua cấu trúc attention, nên khi nén (ví dụ cắt bớt token ảnh, pruning theo token) không phá vỡ "tính đối xứng cấu trúc" của attention, chỉ làm giảm số phần tử trong tổng.
- Phần **GatedDeltaNet** (SSM có cổng, họ hàng với kiến trúc gated RNN) có thể soi dưới góc nhìn mục 5.7-5.8: cơ chế gate kiểm soát tốc độ quên theo thời gian của SSM đóng vai trò tương tự gate LSTM trong việc đạt bất biến lớp với time-warping. Nếu dự án cần nén/distill phần GatedDeltaNet, phân tích "gate kiểm soát $1/\Gamma$ tỉ lệ với thời gian quên đặc trưng" (mục 5.8, phương trình 53) có thể gợi ý: khi prune/quantize các tham số gate, nên ưu tiên giữ độ chính xác số cho bias gate (quyết định range thời gian nhớ) hơn là trọng số input/hidden, vì paper chỉ ra riêng bias gate đã quyết định hành vi "chrono initialisation" của hệ thống.
- Tuy nhiên, cần nói thẳng: **phần lớn nội dung paper 28 (chọn miền Ω, nhóm G, thiết kế blueprint) là bài toán thiết kế kiến trúc từ đầu, còn dự án hiện tại dùng kiến trúc LightOnOCR đã cố định và chỉ còn bài toán nén/pruning/distillation/RL/interpretability** (đã có thư mục riêng trong repo). Paper này **không** đưa ra công cụ nén/pruning cụ thể nào, nên với bài toán nén cụ thể, mức độ liên quan trực tiếp của paper 28 là **thấp đến trung bình**: nó hữu ích như "ngôn ngữ" để hiểu và mô tả tại sao hai khối kiến trúc (attention, SSM có cổng) của LightOnOCR có hình dạng như vậy, nhưng không trực tiếp trả lời "nên prune layer nào, theo chuẩn gì".

9. Thí nghiệm gợi ý

1. **Kiểm chứng tương đương hoán vị bằng PyTorch (mở rộng mục 3.7).** Tạo một graph nhỏ (ví dụ 6 nút, ngẫu nhiên), một lớp GNN convolutional 1 tầng (phương trình 33 với $c_{uv} = 1/\deg(u)$), áp dụng lên X và lên $(PX, PAP^\top)$ với P hoán vị ngẫu nhiên, so sánh $F(PX, PAP^\top)$ với $PF(X, A)$ bằng `torch.allclose`. Kỳ vọng: khớp đến sai số số học (1e-6); nếu không khớp, lỗi thường do dùng sai chiều khi nhân ma trận hoán vị.
2. **So sánh sum vs mean vs max aggregation trên cặp multiset gây nhầm (ý tưởng mục 5, WL test).** Tạo hai multiset đặc trưng $\{\{1, 2\}\}$ và $\{\{1, 1, 2, 2\}\}$ (biểu diễn bằng tensor đệm 0), tính sum/mean/max của mỗi multiset. Kỳ vọng: sum phân biệt được hai multiset (tổng khác nhau: 3 vs 6), mean và max không phân biệt được (mean: 1.5 cả hai nếu chia đúng số phần tử thực; max: 2 cả hai), minh hoạ trực tiếp lý do GIN chọn sum.
3. **Group-equivariant CNN tối giản trên ảnh 8x8.** Dùng nhóm xoay rời rạc 90 độ C_4: tạo 4 bản xoay của một filter 3x3 (`torch.rot90`), convolve ảnh với cả 4 bản, ghép thành 4 "orientation channel". Kiểm tra: xoay ảnh đầu vào 90 độ thì 4 channel đầu ra phải hoán vị đúng theo chu kỳ (channel k thành

channel $k+1 \mod 4$), đo bằng sai số tuyệt đối trung bình giữa kết quả xoay-trước-rồi-convolve và convolve-trước-rồi-xoay-hoán vị channel.

4. **Tịnh tiến-tương đương của CNN 1D và tịnh tiến-bất biến của global average pooling.** Tạo tín hiệu 1D độ dài 64, một lớp conv1d (kernel 5), dịch tín hiệu đi k bước (có đệm 0 ở biên để tránh hiệu ứng biên giả). Đo sai số giữa feature map dịch-trước-rồi-conv và conv-trước-rồi-dịch (kỳ vọng xấp xỉ 0 ở vùng không chạm biên); sau đó thêm global average pooling, đo xem đầu ra cuối có đổi theo k không (kỳ vọng gần như không đổi, trừ hiệu ứng biên).
5. **Laplacian cotangent trên mesh tam giác nhỏ và độ nhạy khi mesh biến dạng.** Tạo mesh hình chữ nhật chia thành khoảng 20 tam giác (`numpy`/`trimesh` hoặc tự viết), tính ma trận Laplacian cotangent (phương trình 25), lấy vài trị riêng/vector riêng đầu. Làm biến dạng mesh nhẹ (dịch một vài nút một khoảng nhỏ, giữ metric gần như cũ) và biến dạng mạnh (tam giác gần suy biến), so sánh độ lệch trị riêng. Kỳ vọng: biến dạng nhẹ giữ trị riêng gần như không đổi (ổn định đẳng cự), biến dạng mạnh (tam giác gần suy biến) làm trị riêng/vector riêng thay đổi mạnh, minh hoạ trực tiếp cảnh báo ở mục 6 về độ nhạy của phương pháp phổ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn (mục paper)
Tác động nhóm lên tín hiệu	$(g.x)(u) = x(g^{-1}u)$	Eq. (3), 3.1
G-bất biến	$f(\rho(g)x) = f(x)$	3.1
G-tương đương	$f(\rho(g)x) = \rho(g)f(x)$	3.1
Ổn định biến dạng tín hiệu	$\|f(\rho(\tau)x) - f(x)\| \le Cc(\tau)\|x\|$	Eq. (4), 3.3
Ổn định biến dạng domain	$\|f(x,\Omega) - f(\tilde{x},\tilde{\Omega})\| \le C\|x\|d_{\mathcal{D}}(\Omega,\tilde{\Omega})$	Eq. (5), 3.3
Bất biến hoán vị (set)	$f(X) = \phi(\sum_u \psi(x_u))$	Eq. (6), 4.1
Tương đương hoán vị (đồ thị)	$F(PX, PAP^\top) = PF(X,A)$	Eq. (9)-(10), 4.1
Blueprint cho đồ thị	$F(X,A) = (\phi(x_u, X_{N_u}))_{u\in V}$	Eq. (13), 4.1
Group convolution	$(x \star \theta)(g) = \langle x, \rho(g)\theta\rangle$	Eq. (14), 4.3
Blueprint GDL tổng quát	$f = A \circ \sigma_J \circ B_J \circ P_{J-1} \circ \cdots \circ \sigma_1 \circ B_1$	3.5
Laplacian rời rạc / cotangent	$(\Delta X)_u = \sum_v w_{uv}(x_u - x_v)$, $w_{uv} = \frac{\cot\angle uqv + \cot\angle upv}{2a_u}$	Eq. (24)-(25), 4.6
Filter phổ tổng quát	$\hat{p}(\Delta)x = \Phi\hat{p}(\Lambda)\Phi^\top x$	4.4, 4.6
Gauge-tương đương	$f \circ \rho_{in}(g) = \rho_{out}(g) \circ f$	4.5
3 hương vị GNN	conv $\subseteq$ attentional $\subseteq$ message-passing	Eq. (33)-(35), 5.3
Self-attention = GNN trên đồ thị đầy đủ	$h_u = \phi(x_u, \bigoplus_v a(x_u,x_v)\psi(x_v))$	Eq. (36), 5.4
$E(3)$-equivariant message passing	phụ thuộc $\|x_u - x_v\|^2$ (bất biến khoảng cách)	5.5
Time-warping invariant RNN (gated)	$h^{(t+1)} = \Gamma R + (1-\Gamma)h^{(t)}$	Eq. (53), 5.8
Bảng domain/group/kiến trúc	CNN-Grid-tịnh tiến; GNN-Graph-Σ_n; LSTM-lưới 1D-time warping	3.5

11. Kết luận ngắn

Paper không đề xuất một mô hình mới mà một **ngôn ngữ chung**: mọi kiến trúc deep learning phổ biến (CNN, GNN, Deep Sets, Transformer, Mesh CNN, Spherical CNN, LSTM) đều có thể viết lại thành bốn khối (tuyến tính tương đương, phi tuyến, coarsening, bất biến toàn cục) tác động trên một trong 5 miền hình học ứng với một nhóm đối xứng cụ thể. Giá trị thực hành là một ”quy trình thiết kế”: xác định miền dữ liệu Ω và nhóm đối xứng G phù hợp với bài toán trước, rồi suy ra lớp equivariant/invariant tương ứng, thay vì chọn kiến trúc

theo thói quen. Hạn chế chính nằm ở biểu cảm (WL test), ở tính không ổn định của Fourier phổ khi so sánh domain khác nhau, và ở việc một số ”bất biến” (như time-warping của gated RNN) chỉ là bất biến lớp mô hình, không phải bất biến zero-shot tuyệt đối.

Nguồn gốc: https://arxiv.org/abs/2104.13478

Bài 29

Understanding Diffusion Models: A Unified Perspective (2022)

Tác giả: Calvin Luo (Google Research, Brain Team).
Venue / arXiv: arXiv:2208.11970 (v1, 25/08/2022); paper không nêu venue hội nghị/journal phản biện, đây là 1 bài tutorial/technical note độc lập đăng trực tiếp trên arXiv.
Nguồn: https://arxiv.org/abs/2208.11970
Chủ đề: suy ra lại toàn bộ họ mô hình diffusion (Variational Diffusion Model) từ một góc nhìn thống nhất: Markovian Hierarchical VAE, 3 cách tham số hoá mục tiêu tương đương (dự đoán x_0, dự đoán noise ϵ, dự đoán score), liên hệ với Score-based Generative Model, và 2 kỹ thuật guidance (classifier, classifier-free).
Vị trí trong lộ trình: paper thuần suy diễn công thức (tutorial), không phụ thuộc thứ tự với các bài còn lại trong batch 28-35. Nên đọc sau khi đã nắm ELBO/biến phân cơ bản (nếu có math về Variational Inference/VAE trong batch 01-27); liên hệ gần với math 10 (Hamiltonian Monte Carlo, cùng là kỹ thuật MCMC lấy mẫu như Langevin dynamics dùng trong phần Score-based Generative Models của bài này).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu/công thức lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận. Đã đọc toàn bộ văn bản (22 trang, ~1500 dòng text sau pdftotext): Introduction, Background (ELBO/VAE/HVAE), Variational Diffusion Models (gồm "Learning Diffusion Noise Parameters" và "Three Equivalent Interpretations"), Score-based Generative Models, Guidance (Classifier và Classifier-Free), Closing, và danh sách tham khảo. Paper là tutorial suy diễn công thức, KHÔNG có benchmark/thí nghiệm số của riêng tác giả.

1. Bối cảnh

Mục tiêu của generative model: học phân phối thật $p(x)$ từ các mẫu quan sát x, để sau đó có thể sinh mẫu mới hoặc (với một số công thức) tính likelihood. Paper liệt kê các hướng chính đang có: GAN (học sinh mẫu qua đối kháng), các mô hình "likelihood-based" (autoregressive, normalizing flow, VAE), energy-based model, và score-based model (học gradient của log-density thay vì density/energy trực tiếp). Paper tập trung vào **diffusion model**, vốn có cả cách nhìn likelihood-based (qua ELBO) và score-based, và cam kết trình bày "the math behind such models in excruciating detail" (chính câu của tác giả) để người đọc hiểu từ gốc.

Khái niệm cần nắm trước:

- **Biến latent** z: biến ẩn không quan sát được, giả định sinh ra dữ liệu quan sát x (ẩn dụ "hang động Plato" mà paper dùng: người trong hang chỉ thấy bóng 2D của vật thể 3D).
- **ELBO (Evidence Lower BOund)**: một chặn dưới có thể tính được của $\log p(x)$ (gọi là "evidence"), dùng làm proxy objective khi không tính trực tiếp $\log p(x)$ được.
- **Reparameterization trick**: viết 1 biến ngẫu nhiên Gauss như hàm xác định của 1 biến nhiễu chuẩn, để lan truyền gradient qua được bước lấy mẫu ngẫu nhiên.

2. Ý tưởng chính

Paper trình bày **Variational Diffusion Model (VDM)** như 1 trường hợp đặc biệt của **Markovian Hierarchical VAE (MHVAE)** với 3 ràng buộc: (i) số chiều latent bằng đúng số chiều dữ liệu, (ii) encoder tại mỗi bước thời gian **không học**, mà cố định sẵn là 1 Gaussian tuyến tính quanh output bước trước (quá trình "làm nhiễu" $x_0 \to x_1 \to \ldots \to x_T$), (iii) tham số Gaussian của encoder thay đổi theo thời gian sao cho x_T hội tụ về Gaussian chuẩn thuần.

Từ đó, suy ra ELBO của VDM theo 2 cách: cách đầu cho ra 1 "consistency term" tính trên 2 biến ngẫu nhiên

mỗi bước (phương sai Monte Carlo cao); cách thứ hai (dùng $q(x_t|x_{t-1}, x_0)$ qua Bayes) cho ra 1 "denoising matching term" chỉ tính trên 1 biến ngẫu nhiên mỗi bước (phương sai thấp hơn, dùng được trong thực hành). Tối ưu ELBO quy về: học 1 mạng nơ-ron dự đoán x_0 gốc từ x_t nhiễu bất kỳ ở bước t bất kỳ. Paper sau đó chỉ ra việc dự đoán x_0, dự đoán **noise** ϵ_0 đã thêm vào, và dự đoán **score** $\nabla \log p(x_t)$ là **3 cách tham số hoá tương đương** (khác nhau hằng số tỉ lệ) của cùng 1 bài toán. Cách nhìn score dẫn tới liên hệ trực tiếp với **Score-based Generative Model** (Langevin dynamics, multi-noise-level score matching, SDE liên tục). Cuối cùng, để sinh có điều kiện (text-to-image, super-resolution), paper trình bày 2 kỹ thuật **guidance**: Classifier Guidance (dùng classifier riêng trên ảnh nhiễu) và Classifier-Free Guidance (không cần classifier riêng, kết hợp tuyến tính score có điều kiện và không điều kiện).

3. Phương pháp chi tiết

3.1 ELBO và VAE (Background)

Với $p(x, z)$ đồng thời của dữ liệu x và latent z, ELBO định nghĩa (Eq. 3-4):

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{p(x,z)}{q_\phi(z|x)}\Big].$$

Paper suy ra 2 lần (1 lần bằng bất đẳng thức Jensen, 1 lần tường minh hơn bằng quy tắc dây chuyền) để cho thấy (Eq. 15):

$$\log p(x) = \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{p(x,z)}{q_\phi(z|x)}\Big] + D_{KL}\big(q_\phi(z|x) \,\|\, p(z|x)\big),$$

nghĩa là evidence = ELBO + 1 số hạng KL luôn $\geq 0 \Rightarrow$ ELBO luôn là chặn dưới, và **tối đa ELBO** tương đương **tối thiểu KL** giữa posterior xấp xỉ $q_\phi(z|x)$ và posterior thật $p(z|x)$ (vì evidence là hằng số không phụ thuộc ϕ).

Với VAE, phân tách ELBO (Eq. 19) thành 2 số hạng:

$$\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x) \,\|\, p(z)\big),$$

gọi là **reconstruction term** và **prior matching term**. Với $q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \sigma^2_\phi(x)I)$ và prior $p(z) = \mathcal{N}(0, I)$, số hạng KL tính được dạng đóng (closed form), số hạng reconstruction xấp xỉ Monte Carlo qua **reparameterization trick**: $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ (Eq. 20-22).

Hierarchical VAE (HVAE) tổng quát hoá: nhiều tầng latent $z_1, \ldots, z_T$. Paper chỉ xét **Markovian HVAE**: $p(x, z_{1:T}) = p(z_T)p_\theta(x|z_1)\prod_{t=2}^{T} p_\theta(z_{t-1}|z_t)$ (Eq. 23) - về bản chất là nhiều VAE xếp chồng lên nhau.

3.2 Variational Diffusion Model: 2 cách phân tách ELBO

VDM = MHVAE với 3 ràng buộc nêu ở mục 2. Ký hiệu x_0 = dữ liệu gốc, $x_1, \ldots, x_T$ = các bước nhiễu dần, $x_T \approx$ Gaussian chuẩn thuần. Encoder cố định (Eq. 31):

$$q(x_t|x_{t-1}) = \mathcal{N}\big(x_t; \sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t)I\big),$$

với α_t là hệ số (siêu tham số hoặc học được) kiểm soát tốc độ nhiễu, chọn sao cho phương sai được giữ ổn định ("variance-preserving"). Đặt $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$.

Cách phân tách thứ nhất (Eq. 45): dùng trực tiếp định nghĩa Markov, ra 3 số hạng: reconstruction, prior matching (xấp xỉ 0 nếu T đủ lớn), và 1 **consistency term** dạng

$\sum_{t=1}^{T-1} \mathbb{E}_{q(x_{t-1},x_{t+1}|x_0)}\big[D_{KL}(q(x_t|x_{t-1}) \,\|\, p_\theta(x_t|x_{t+1}))\big]$: ép phân phối tại x_t nhất quán giữa chiều forward (từ x_{t-1}) và chiều backward (từ x_{t+1}). Nhược điểm: mỗi số hạng lấy kỳ vọng trên **2** biến ngẫu nhiên, phương sai Monte Carlo cao khi T lớn.

Cách phân tách thứ hai (Eq. 58, dùng Bayes rule $q(x_{t-1}|x_t,x_0) = \frac{q(x_t|x_{t-1},x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)}$) cho 3 số hạng khác: reconstruction, prior matching, và 1 **denoising matching term**:

$$\sum_{t=2}^{T} \mathbb{E}_{q(x_t|x_0)}\Big[D_{KL}\big(q(x_{t-1}|x_t,x_0) \,\|\, p_\theta(x_{t-1}|x_t)\big)\Big],$$

mỗi số hạng chỉ lấy kỳ vọng trên **1** biến ngẫu nhiên $\Rightarrow$ phương sai thấp hơn, dùng được trong thực hành. $q(x_{t-1}|x_t,x_0)$ đóng vai trò "target" chuẩn (vì có thể tính closed-form khi biết x_0), còn $p_\theta(x_{t-1}|x_t)$ là bước denoise mô hình cần học xấp xỉ nó.

3.3 Dạng đóng của $q(x_{t-1}|x_t,x_0)$ và mục tiêu huấn luyện

Nhờ encoder là Gaussian tuyến tính, suy ra được (bằng truy hồi lặp reparameterization trick, Eq. 61-70):

$$q(x_t|x_0) = \mathcal{N}\big(x_t; \sqrt{\bar\alpha_t}\,x_0, (1-\bar\alpha_t)I\big).$$

Dùng Bayes rule với dạng Gaussian trên, paper suy ra (sau nhiều bước đại số "hoàn thành bình phương", Eq. 71-84) rằng $q(x_{t-1}|x_t,x_0)$ cũng là Gaussian với:

$$\mu_q(x_t,x_0) = \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})x_t + \sqrt{\bar\alpha_{t-1}}(1-\alpha_t)x_0}{1-\bar\alpha_t}, \qquad \sigma_q^2(t) = \frac{(1-\alpha_t)(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}.$$

Vì phương sai 2 phía cố định và giống nhau, tối thiểu D_{KL} giữa 2 Gaussian cùng phương sai quy về tối thiểu khoảng cách bình phương giữa 2 **mean** (Eq. 87-92): $\arg\min_\theta \frac{1}{2\sigma_q^2(t)}\|\mu_\theta - \mu_q\|^2$. Nếu tham số hoá $\mu_\theta(x_t,t)$ qua 1 mạng dự đoán $\hat x_\theta(x_t,t) \approx x_0$ theo cùng dạng công thức của μ_q, mục tiêu rút về (Eq. 99):

$$\arg\min_\theta \frac{1}{2\sigma_q^2(t)} \frac{\bar\alpha_{t-1}(1-\alpha_t)^2}{(1-\bar\alpha_t)^2} \big\|\hat x_\theta(x_t,t) - x_0\big\|_2^2,$$

tức **học 1 mạng dự đoán ảnh gốc x_0 từ phiên bản nhiễu x_t bất kỳ**, lấy kỳ vọng ngẫu nhiên theo $t \sim \text{Uniform}\{2,\dots,T\}$ (Eq. 100).

3.4 Học tham số nhiễu và SNR (Learning Diffusion Noise Parameters)

Định nghĩa **signal-to-noise ratio** $\text{SNR}(t) = \bar\alpha_t/(1-\bar\alpha_t)$ (từ dạng Gaussian của $q(x_t|x_0)$, Eq. 109). Mục tiêu Eq. 99 viết lại gọn bằng chênh lệch SNR giữa 2 bước liên tiếp (Eq. 110): $\propto (\text{SNR}(t-1) - \text{SNR}(t))\|\hat x_\theta - x_0\|_2^2$. Vì SNR phải giảm đơn điệu theo t, paper tham số hoá trực tiếp $\text{SNR}(t) = \exp(-\omega_\eta(t))$ với ω_η là mạng tăng đơn điệu theo t (Eq. 111-114), cho phép **học chung α_t cùng lúc với mô hình** thay vì cố định trước.

3.5 Ba cách tham số hoá tương đương

Từ $x_0 = \frac{x_t - \sqrt{1-\bar\alpha_t}\,\epsilon_0}{\sqrt{\bar\alpha_t}}$ (đảo công thức forward), thay vào μ_q cho ra mục tiêu dự đoán **noise** ϵ_0 (Eq. 115-130): $\arg\min_\theta \frac{(1-\alpha_t)^2}{2\sigma_q^2(t)(1-\bar\alpha_t)\alpha_t}\|\epsilon_0 - \hat\epsilon_\theta(x_t,t)\|_2^2$. Thực nghiệm (paper dẫn chiếu Ho et al. 2020, DDPM) cho thấy dự đoán noise **tốt hơn** dự đoán x_0 trực tiếp.

Dùng **công thức Tweedie** ($\mathbb{E}[\mu_z|z] = z + \Sigma_z \nabla_z \log p(z)$ cho Gaussian z), paper suy ra ước lượng tốt nhất của mean thật từ 1 mẫu quan sát gắn với **score** $\nabla \log p(x_t)$ (Eq. 131-133), từ đó suy ra tham số hoá thứ 3: học

mạng $s_\theta(x_t, t) \approx \nabla \log p(x_t)$ (Eq. 134-148). Paper chỉ rõ mối liên hệ định lượng giữa score và noise (Eq. 149-151): $\nabla \log p(x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_0$ - **2 đại lượng chỉ khác nhau 1 hằng số tỉ lệ phụ thuộc** t.

3.6 Score-based Generative Models và Guidance

Với energy-based model $p_\theta(x) = \frac{1}{Z_\theta}e^{-f_\theta(x)}$, lấy đạo hàm log loại bỏ hằng số chuẩn hoá khó tính: $\nabla_x \log p_\theta(x) = -\nabla_x f_\theta(x) \approx s_\theta(x)$ (Eq. 152-156) $\Rightarrow$ học trực tiếp score bằng cách tối thiểu **Fisher divergence** $\mathbb{E}_{p(x)}\|s_\theta(x) - \nabla \log p(x)\|^2$ (Eq. 157), sinh mẫu bằng **Langevin dynamics** (Eq. 158): $x_{i+1} \leftarrow x_i + c\nabla \log p(x_i) + \sqrt{2c}\,\epsilon$.

Paper nêu 3 vấn đề của score matching ”vanilla” (theo Song & Ermon): score không xác định ngoài manifold thấp chiều, kém chính xác ở vùng mật độ thấp, Langevin dynamics có thể không ”mix” đúng tỉ lệ trọng số giữa các mode (ví dụ hỗn hợp Gaussian 2 thành phần). Giải pháp: thêm **nhiều mức nhiễu Gaussian** $\{\sigma_t\}$, học 1 mạng $s_\theta(x, t)$ cho mọi mức nhiễu (Eq. 160-161) - đây chính là **cùng một mục tiêu với VDM** (paper nói ”almost exactly matches”). Khi số bước $T \to \infty$, quá trình nhiễu liên tục được mô tả bằng 1 **phương trình vi phân ngẫu nhiên (SDE)**, và sinh mẫu = giải SDE ngược.

Guidance: để sinh có điều kiện $p(x_{0:T}|y)$, thêm y vào mỗi bước denoise: $p_\theta(x_{0:T}|y) = p(x_T)\prod_t p_\theta(x_{t-1}|x_t, y)$ (Eq. 162). Vấn đề: mô hình có thể học ”bỏ qua” y. **Classifier Guidance** (Eq. 163-166): bằng Bayes, $\nabla \log p(x_t|y) = \nabla \log p(x_t) + \nabla \log p(y|x_t)$; nhân hệ số γ vào gradient classifier để kiểm soát mức tuân theo điều kiện: $\nabla \log p(x_t|y) = \nabla \log p(x_t) + \gamma\nabla \log p(y|x_t)$ - nhược điểm: cần huấn luyện riêng 1 classifier chịu được input nhiễu bất kỳ. **Classifier-Free Guidance** (Eq. 167-170) khử classifier bằng cách viết lại: $\nabla \log p(x_t|y) = \gamma\nabla \log p(x_t|y) + (1-\gamma)\nabla \log p(x_t)$, chỉ cần 1 mô hình (có và không điều kiện, học chung bằng cách ”dropout” điều kiện y thành giá trị cố định khi train).

4. Thí nghiệm và kết quả chính

Đây là paper **tutorial suy diễn công thức, không có thí nghiệm/benchmark số nào do tác giả thực hiện** (không có bảng số liệu, không huấn luyện mô hình nào trong bài). ”Kết quả chính” ở đây là các **công thức/đẳng thức toán học** đã suy ra ở mục 3: (a) 2 cách phân tách ELBO của VDM (số hạng phương sai cao vs thấp), (b) dạng closed-form của $q(x_{t-1}|x_t, x_0)$, (c) 3 mục tiêu huấn luyện tương đương (dự đoán x_0, ϵ, score) chỉ khác hệ số trọng số theo t, (d) liên hệ định lượng giữa VDM và Score-based model qua SNR và qua Eq. 149-151, (e) công thức guidance (classifier và classifier-free). Các hình minh hoạ (Hình 1-6) đều là sơ đồ khái niệm (VAE, HVAE, VDM, quá trình match 2 phân phối, quỹ đạo Langevin dynamics trên hỗn hợp Gaussian) chứ không phải kết quả benchmark trên dataset chuẩn.

5. Phân tích / ablation

Không có ablation thực nghiệm; ”phân tích” ở đây là các **so sánh định lượng giữa 3 cách tham số hoá** và giữa 2 cách phân tách ELBO mà paper tự thực hiện bằng đại số:

- **Phương sai ước lượng Monte Carlo**: cách phân tách ELBO thứ nhất (consistency term, Eq. 45) có phương sai cao hơn cách thứ hai (denoising matching term, Eq. 58) vì lấy kỳ vọng trên 2 biến ngẫu nhiên mỗi bước thay vì 1; đây là lý do thực hành chỉ dùng cách thứ hai.
- **Dự đoán x_0 vs dự đoán noise ϵ**: 2 mục tiêu **tương đương về đại số** (chỉ khác hệ số nhân phụ thuộc t trong Eq. 99 vs Eq. 130), nhưng paper dẫn chứng (trích Ho et al. 2020 và Saharia et al. 2022, không tự thực nghiệm) rằng dự đoán noise cho kết quả thực nghiệm tốt hơn.
- **Trường hợp đặc biệt $T = 1$**: cả 2 cách phân tách ELBO của VDM **suy biến đúng về ELBO của VAE thường (Eq. 19)** khi $T = 1$ - đây là 1 phép kiểm tra tính nhất quán toán học (sanity check) mà paper tự nêu rõ (ngay sau Eq. 58).
- **Giá trị γ trong guidance**: paper phân tích định tính 3 trường hợp $\gamma = 0$ (bỏ điều kiện), $\gamma = 1$ (điều kiện thường, không guidance), $\gamma > 1$ (tăng cường bám điều kiện, giảm diversity) - không có số liệu định lượng cụ thể minh hoạ ảnh hưởng của γ trong chính bài này.

6. Giới hạn

Paper tự liệt kê ở mục ”Closing”:

- Quá trình sinh mẫu bằng khử nhiễu dần từ nhiễu ngẫu nhiên **không giống cách con người tạo ra dữ liệu** (paper tự nhận đây khó có vẻ ”tự nhiên” về mặt nhận thức).
- VDM **không cho latent có thể diễn giải được**: khác VAE thường (latent có thể mã hoá cấu trúc ngữ nghĩa), ở VDM encoder mỗi bước là Gaussian tuyến tính cố định, không linh hoạt học được $\Rightarrow$ latent chỉ là ”ảnh gốc + nhiễu”, không mang cấu trúc riêng.
- Latent có **cùng số chiều với dữ liệu gốc** (ràng buộc (i) của VDM), nên không học được biểu diễn nén (compressed representation) như mục tiêu ban đầu của latent variable model.
- **Sinh mẫu tốn kém**: cần lặp qua rất nhiều bước T (để đảm bảo x_T đủ gần Gaussian chuẩn), không sinh được trong 1 bước như GAN.
- Paper không tự thực nghiệm kiểm chứng bất kỳ công thức nào trên dữ liệu thật (toàn bộ dẫn chứng thực nghiệm đều trích từ paper khác: DDPM, Imagen, DALL-E 2, score-based SDE) - đây không phải ”giới hạn” paper tự nhận nhưng là điều người đọc cần biết: đây là bài tổng hợp lý thuyết, không phải báo cáo thực nghiệm độc lập.

7. Liên hệ với các bài khác

Đã kiểm tra bằng `ls` các thư mục trước khi trỏ (math content và ml basics content hiện đang được nhiều bài viết song song trong phiên này; các bài lý tưởng nhất về chủ đề - VAE, GAN, DDPM theo titles.tsv ml 14/15/16 - CHƯA có file lúc kiểm tra, nên không trỏ tới để tránh link treo):

- **”math content/10_2017_hamiltonian_monte_carlo.md”** (đã xác nhận tồn tại): cùng là phương pháp MCMC lấy mẫu dựa trên gradient của log-density, cùng họ với Langevin dynamics mà bài 29 dùng trong phần Score-based Generative Models (Langevin dynamics có thể xem như HMC với 1 bước ”leapfrog”, không có thành phần động lượng).
- **”ml basics content/01_2016_rethinking_generalization.md”** (đã xác nhận tồn tại): không trực tiếp về generative model, nhưng cùng nằm trong mạch câu hỏi lớn ”vì sao mô hình sâu học và tổng quát hoá tốt” - liên hệ gián tiếp về phương pháp luận (dùng công thức toán chặt để giải thích hiện tượng thực nghiệm).
- **”interp content/05_2022_toy_models_of_superposition.md”** (đã xác nhận tồn tại): khác chủ đề (interpretability của representation) nhưng cùng phong cách phương pháp luận ”dùng mô hình toy/suy diễn công thức chặt để hiểu hiện tượng của mạng nơ-ron” như cách bài 29 suy diễn VDM từng bước.

(Ghi chú trung thực: các liên hệ lý tưởng nhất cho bài này, theo titles.tsv, là ml basics 14 ”Generative Adversarial Networks”, 15 ”Auto-Encoding Variational Bayes” (VAE, được bài 29 trích dẫn trực tiếp là tài liệu tham khảo [1]), và 16 ”Denoising Diffusion Probabilistic Models” (DDPM, tài liệu tham khảo [5], chính là công thức dự đoán-noise mà bài 29 suy ra lại) - các bài này đang được viết song song trong cùng phiên và chưa có file lúc note này được hoàn thành.)

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, batch 1, GPU $\leq$ 8GB; các hướng pruning/distillation/RL/interpretability đã có thư mục riêng).

(Suy luận) Diffusion model **không phải kiến trúc của LightOnOCR** (model này không sinh ảnh bằng khử nhiễu, mà encode ảnh trang tài liệu rồi decode ra text bằng kiến trúc tự hồi quy/attention) nên nội dung bài này **ít liên quan trực tiếp** tới việc train/nén/tune LightOnOCR. Tuy vậy có 2 liên hệ gián tiếp đáng ghi nhận: (1) kỹ thuật ELBO/biến phân và ”3 cách tham số hoá tương đương” là bài học phương pháp luận hữu ích nói chung khi thiết kế lại loss cho 1 mục tiêu huấn luyện (ví dụ khi đổi cách tham số hoá loss distillation hoặc loss OCR để giảm phương sai gradient, ý tưởng ”chọn tham số hoá có phương sai ước lượng thấp hơn” ở

mục 3.2-3.3 của note này áp dụng được tổng quát); (2) nếu dự án sau này có hướng dùng mô hình sinh (ví dụ augment dữ liệu training OCR bằng ảnh tài liệu tổng hợp qua diffusion) thì nền tảng công thức ở đây sẽ cần thiết - nhưng đây là hướng mở rộng xa, không nằm trong phạm vi nén/hậu huấn luyện hiện tại.

9. Thí nghiệm gợi ý

1. **Cài forward/reverse process trên dữ liệu 1D/2D toy**: lấy dữ liệu tổng hợp đơn giản (ví dụ hỗn hợp 2 Gaussian 2D), cài quá trình forward $q(x_t|x_{t-1})$ theo Eq. 31 với lịch α_t tuyến tính giảm, huấn luyện 1 MLP nhỏ dự đoán ϵ_0 theo loss Eq. 130. Đo: ảnh/điểm sinh ra sau khi denoise từ nhiễu Gauss có khớp lại 2 cluster ban đầu không. Chạy được trên CPU/GPU 8GB trong vài phút.
2. **So sánh dự đoán x_0 vs dự đoán ϵ**: huấn luyện 2 mô hình như thí nghiệm 1 nhưng 1 dự đoán x_0 (loss Eq. 99) và 1 dự đoán ϵ (loss Eq. 130), cùng kiến trúc/hyperparameter. Đo tốc độ giảm loss và chất lượng mẫu sinh (ví dụ khoảng cách tới phân phối thật bằng MMD hoặc histogram). Kỳ vọng: 2 cách tương đương về công thức nhưng có thể khác nhau về ổn định huấn luyện thực tế, minh hoạ nhận xét ở mục 5.
3. **Minh hoạ Langevin dynamics trên hỗn hợp Gaussian 2 mode**: cho biết trước score thật $\nabla \log p(x)$ của 1 hỗn hợp 2 Gaussian (tính giải tích), chạy Eq. 158 từ nhiều điểm khởi tạo. Đo tỉ lệ mẫu rơi vào mỗi mode so với trọng số c_1, c_2 thật. Kỳ vọng: nếu dùng đúng score 1 mức nhiễu (không multi-noise-level), tỉ lệ các mode có thể SAI lệch so với c_1, c_2 thật (đúng như giới hạn 3 mà paper nêu ở mục Score-based Generative Models).
4. **Thêm multi-noise-level score matching**: lặp lại thí nghiệm 3 nhưng học $s_\theta(x, t)$ cho nhiều mức nhiễu σ_t (Eq. 160-161) và sinh bằng annealed Langevin dynamics. Kỳ vọng: tỉ lệ mode khớp c_1, c_2 tốt hơn thí nghiệm 3, minh hoạ đúng lý do paper nêu cho việc dùng nhiều mức nhiễu.
5. **Thử Classifier-Free Guidance ở quy mô nhỏ**: với dữ liệu 2D có nhãn lớp (ví dụ 3 cluster, mỗi cluster 1 nhãn), huấn luyện 1 mô hình diffusion điều kiện (dropout ngẫu nhiên nhãn để học cả bản không điều kiện), sinh mẫu với $\gamma \in \{0, 1, 3\}$ theo Eq. 170. Đo: độ "thuần" của mẫu sinh theo nhãn (bao nhiêu % mẫu rơi đúng cluster điều kiện) và độ đa dạng (variance trong cluster). Kỳ vọng: γ tăng thì độ thuần tăng, đa dạng giảm, đúng như phân tích định tính ở mục 3.6/5.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ý nghĩa
ELBO	$\log p(x) \geq \mathbb{E}_{q_\phi(z\|x)}[\log \frac{p(x,z)}{q_\phi(z\|x)}]$ (Eq. 3-4)	Chặn dưới của log-likelihood, dùng làm proxy objective
Encoder VDM	$q(x_t\|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$ (Eq. 31)	1 bước làm nhiễu Gaussian tuyến tính
Forward closed-form	$q(x_t\|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$, $\bar{\alpha}_t = \prod_{i\leq t}\alpha_i$ (Eq. 70)	Nhiễu trực tiếp từ x_0 tới bước t bất kỳ, không cần lặp
Denoising matching loss	$\arg\min_\theta \mathbb{E}_{t,x_0,\epsilon}\|\epsilon_0 - \hat{\epsilon}_\theta(x_t, t)\|_2^2$ (dạng đơn giản hoá của Eq. 130)	Mục tiêu huấn luyện kiểu DDPM: dự đoán noise
SNR	$\text{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$ (Eq. 109)	Tỉ lệ tín hiệu/nhiễu, phải giảm đơn điệu theo t
Công thức Tweedie	$\mathbb{E}[\mu_z\|z] = z + \Sigma_z \nabla_z \log p(z)$	Liên hệ score với ước lượng mean thật
Score ↔ noise	$\nabla \log p(x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_0$ (Eq. 151)	2 tham số hoá chỉ khác hằng số tỉ lệ theo t
Classifier Guidance	$\nabla \log p(x_t\|y) = \nabla \log p(x_t) + \gamma \nabla \log p(y\|x_t)$ (Eq. 166)	Cần classifier riêng chịu nhiễu

Khái niệm	Công thức	Ý nghĩa
Classifier-Free Guidance	$\nabla \log p(x_t\|y) = \gamma\nabla \log p(x_t\|y) + (1-\gamma)\nabla \log p(x_t)$ (Eq. 170)	Không cần classifier, chỉ cần 1 mô hình

Ví dụ tính tay (tự tính): minh hoạ công thức truy hồi Eq. 61-70 với $x_0 = 10$ (số 1 chiều cho đơn giản), $\alpha_1 = 0.9$, $\alpha_2 = 0.8$. Lấy $\epsilon_1 = \epsilon_2 = 1$ (chọn cố định để minh hoạ, thực tế là ngẫu nhiên):

Bước 1: $x_1 = \sqrt{0.9} \times 10 + \sqrt{0.1} \times 1 \approx 0.9487 \times 10 + 0.3162 \times 1 = 9.487 + 0.316 = 9.803$.

Bước 2: $x_2 = \sqrt{0.8} \times x_1 + \sqrt{0.2} \times 1 \approx 0.8944 \times 9.803 + 0.4472 \times 1 = 8.770 + 0.447 = 9.217$.

Theo công thức trực tiếp (Eq. 70) với $\bar{\alpha}_2 = \alpha_1\alpha_2 = 0.9 \times 0.8 = 0.72$: $x_2 \approx \sqrt{0.72} \times 10 + \sqrt{0.28} \times \epsilon_{\text{combined}} = 8.485 + 0.529 \times \epsilon_{\text{combined}}$. Với $\epsilon_{\text{combined}} \approx 1.38$ (giá trị cần để 2 cách khớp nhau khi gộp 2 nhiễu độc lập theo Eq. 61-67, không bằng đơn giản $\epsilon_1 + \epsilon_2$ vì có hệ số tỉ lệ khác nhau), kết quả 2 cách tính xấp xỉ nhau (9.217 so với $8.485 + 0.529 \times 1.38 \approx 9.215$) - xác nhận công thức truy hồi Eq. 68-70 cho phép nhảy thẳng từ x_0 tới x_2 bằng 1 bước, không cần lặp qua từng t, đúng như paper suy ra.

11. Kết luận ngắn

Paper suy ra lại toàn bộ diffusion model (VDM) từ góc nhìn Markovian Hierarchical VAE, cho thấy việc tối ưu ELBO quy về 3 bài toán tương đương (dự đoán x_0, dự đoán noise, dự đoán score), nối liền VDM với Score-based Generative Model qua liên hệ định lượng giữa score và noise, và trình bày 2 kỹ thuật guidance (classifier, classifier-free) để sinh có điều kiện. Đây là tài liệu tham khảo toán học tốt để hiểu "tại sao" các công thức DDPM/score-based có dạng như vậy, nhưng không có đóng góp thực nghiệm mới và không tự kiểm chứng số liệu nào; người đọc cần tài liệu gốc (Ho et al. 2020, Song & Ermon 2019-2020) để xem kết quả thực nghiệm thật.

Nguồn gốc: https://arxiv.org/abs/2208.11970

Bài 30

Opening the Black Box of Deep Neural Networks via Information (2017)

Tác giả: Ravid Schwartz-Ziv, Naftali Tishby (Edmond and Lilly Safra Center for Brain Sciences, The Hebrew University of Jerusalem).
Venue / arXiv: arXiv:1703.00810 (v3, 29/04/2017); paper không nêu venue hội nghị/journal phản biện thông thường, chỉ tự ghi ”Editor: ICRI-CI” (báo cáo thuộc chuỗi ”Why & When Deep Learning works” của Intel Collaborative Research Institute for Computational Intelligence).
Nguồn: https://arxiv.org/abs/1703.00810
Chủ đề: quan sát thực nghiệm quá trình huấn luyện Deep Neural Network (DNN) qua lăng kính lý thuyết thông tin (Information Bottleneck, IB): phát hiện 2 pha của SGD (drift và diffusion) tương ứng 2 pha trong ”information plane” (ERM và nén biểu diễn).
Vị trí trong lộ trình: trong batch ”toán cho AI” (28-35) các bài không phụ thuộc thứ tự chặt. Bài này hợp lý đọc cùng mạch với math 07 (giới thiệu information geometry) và math 14 (Deep Variational Information Bottleneck, mở rộng IB sang biến phân cho mạng sâu) nếu đọc theo chủ đề lý thuyết thông tin; cũng liên hệ gần với math 31 (Deep learning: a statistical viewpoint) vì cả hai cùng tìm cách lý giải tại sao DNN overparameterized vẫn tổng quát hoá tốt, chỉ khác công cụ (information-theoretic ở đây, thống kê/NTK ở bài 31).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Đã đọc toàn bộ văn bản paper (905 dòng text sau pdftotext, gồm Abstract, mục 1-4, Acknowledgements, References); phần ”(see appendix)” được tác giả nhắc tới ở mục 3.4 không xuất hiện dưới dạng mục riêng trong bản PDF đã tải (có thể là tài liệu bổ sung ngoài bản chính), nên không trích dẫn được nội dung appendix đó.

1. Bối cảnh

Tới 2017, Deep Learning đã thành công rực rỡ trong thực nghiệm nhưng gần như không có lý thuyết giải thích tại sao: (a) tối ưu non-convex bằng SGD lại tìm được nghiệm tốt, (b) mạng fit gần như hoàn hảo dữ liệu train (không có regularization tường minh rõ ràng) nhưng vẫn generalize tốt trên test. DNN hay bị chê là ”hộp đen bí ẩn” (paper dẫn Alain and Bengio 2016).

Ý tưởng nền (từ Tishby and Zaslavsky 2015, công trình trước của chính nhóm tác giả): một mạng nhiều lớp tạo ra một **chuỗi Markov** các biểu diễn kế tiếp của input X: $Y \to X \to T_1 \to T_2 \to \ldots \to T_K \to \hat{Y}$. Mỗi lớp ẩn T_i được đặc trưng hoàn toàn (không quan tâm trọng số/neuron cụ thể) bởi 2 số: $I(X;T_i)$ (lượng thông tin giữ lại về input) và $I(T_i;Y)$ (lượng thông tin giữ lại về nhãn, tức khả năng dự đoán). Vẽ 2 số này cho mỗi lớp ta được 1 điểm trên ”**information plane**” (mặt phẳng thông tin), và cả mạng tạo thành 1 đường đi (path) qua các điểm đó.

Khung **Information Bottleneck (IB)** (Tishby, Pereira, Bialek 1999) định nghĩa ”biểu diễn tối ưu” T của X đối với Y: nén X nhiều nhất có thể (min $I(X;T)$) trong khi vẫn giữ được lượng thông tin $I(T;Y)$ cho trước về nhãn. Bài báo 2015 của Tishby & Zaslavsky mới đề xuất framework này cho DNN; bài báo 2017 này (Schwartz-Ziv & Tishby) là bài đầu tiên **thực nghiệm kiểm chứng** framework đó một cách có hệ thống: vẽ quỹ đạo thật của các lớp DNN trong information plane suốt quá trình train, và xem chúng có hành xử đúng như IB dự đoán không.

Khái niệm cần nắm trước (xem thêm math 00 nếu có, hoặc phần 10 cuối bài này):

- **Entropy** $H(X) = -\sum_x p(x) \log p(x)$: số bit trung bình cần để mã hoá X.

- **Mutual information (thông tin tương hỗ)** $I(X;Y)$: lượng thông tin X và Y "chia sẻ" với nhau; bằng 0 khi độc lập, lớn khi phụ thuộc mạnh.
- **Data Processing Inequality (DPI)**: nếu $X \to Y \to Z$ là chuỗi Markov (Z chỉ phụ thuộc X qua Y) thì $I(X;Y) \geq I(X;Z)$: xử lý dữ liệu qua nhiều bước chỉ có thể làm mất thông tin, không thể sinh thêm.

2. Ý tưởng chính

Tác giả huấn luyện các mạng fully-connected nhỏ (tối đa 7 lớp ẩn) nhiều lần với trọng số khởi tạo ngẫu nhiên khác nhau, tính chính xác $I(X;T_i)$ và $I(T_i;Y)$ cho từng lớp tại từng epoch (vì bài toán nhân tạo nên biết chính xác phân phối $P(X,Y)$, tính MI chính xác bằng tổng hữu hạn, không cần ước lượng từ mẫu), rồi vẽ quỹ đạo các lớp trong information plane qua thời gian huấn luyện.

5 kết quả chính (theo Abstract, mục 1.1):

1. **Phần lớn số epoch huấn luyện được dùng để nén biểu diễn (compression)**, không phải để fit nhãn (fitting) - fitting xảy ra nhanh, nén xảy ra rất chậm sau đó.
2. **SGD có 2 pha động học rõ rệt**: pha đầu ("**drift**", trôi) gradient có mean lớn hơn hẳn độ lệch chuẩn (SNR cao) ứng với pha train error giảm nhanh (ERM); pha sau ("**diffusion**", khuếch tán) gradient gần như nhiễu Gauss (SNR thấp), ứng với pha các lớp **nén thông tin về input** ($I(X;T_i)$ giảm) mà hầu như không còn cải thiện $I(T_i;Y)$ nhiều.
3. **Các lớp đã hội tụ nằm rất gần đường cong IB lý thuyết** (information curve), và thoả mãn các phương trình tự hợp (self-consistent) encoder-decoder của IB.
4. **Lợi ích chính của việc thêm lớp ẩn là tính toán (computational), không phải biểu diễn**: thêm lớp giúp giảm mạnh số epoch cần để hội tụ (vì pha diffusion với nhiều lớp mỏng tương đương nhiều bước nén nhỏ, nhanh hơn theo cấp số nhân so với 1 bước nén lớn).
5. Các lớp ẩn có xu hướng hội tụ gần các **điểm tới hạn (critical points)** trên đường cong IB (do hiện tượng "critical slowing down" của quá trình khuếch tán gần chuyển pha).

Thông điệp cốt lõi: **tính ngẫu nhiên của SGD không chỉ để thoát local minima** (giải thích truyền thống) mà còn đóng vai trò tạo ra biểu diễn nén hiệu quả thông qua một quá trình khuếch tán (diffusion) kiểu entropy-maximization, và đây có thể là lý do DNN generalize tốt dù không có regularization tường minh.

3. Phương pháp chi tiết

3.1 Công cụ lý thuyết thông tin (mục 2 của paper)

Mutual information giữa 2 biến ngẫu nhiên X, Y với phân phối đồng thời $p(x,y)$ (Eq. 1-2 của paper):

$$I(X;Y) = D_{KL}\big[p(x,y)\|p(x)p(y)\big] = \sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} = H(X) - H(X|Y).$$

Hai tính chất MI quan trọng cho DNN mà paper nhấn mạnh:

- **Bất biến với biến đổi khả nghịch (invertible)**: $I(X;Y) = I(\psi(X);\varphi(Y))$ với ψ, φ khả nghịch bất kỳ (Eq. 3). Nghĩa là nếu một lớp chỉ "xáo trộn" thông tin mà không mất gì (biến đổi 1-1), MI của nó với X và Y không đổi - dù các neuron riêng lẻ mã hoá đặc trưng gì đi nữa. Đây là lý do tác giả coi cả lớp T_i như **1 biến ngẫu nhiên duy nhất**, bỏ qua việc "đọc hiểu" từng neuron.
- **Data Processing Inequality (DPI)**: với chuỗi Markov $X \to Y \to Z$: $I(X;Y) \geq I(X;Z)$ (Eq. 4). Áp dụng cho chuỗi các lớp DNN $Y \to X \to T_1 \to \ldots \to T_K \to \hat{Y}$ cho ra 2 chuỗi bất đẳng thức đơn điệu (Eq. 5-6):

$$I(X;Y) \geq I(T_1;Y) \geq \ldots \geq I(T_K;Y) \geq I(\hat{Y};Y), \qquad H(X) \geq I(X;T_1) \geq \ldots \geq I(X;T_K) \geq I(X;\hat{Y}).$$

Information Bottleneck (IB): biểu diễn "tối ưu" $T(X)$ đối với Y được định nghĩa là **thống kê đủ tối thiểu** (minimal sufficient statistic): nén X nhiều nhất mà vẫn giữ nguyên $I(T;Y) = I(X;Y)$ (Eq. 7). Vì thống kê đủ tối thiểu chính xác chỉ tồn tại với họ phân phối rất đặc biệt (họ hàm mũ - exponential family), Tishby et al. (1999) nới lỏng bài toán: cho T là ánh xạ **ngẫu nhiên** $p(t|x)$, và không cần giữ 100% $I(X;Y)$ mà đánh đổi bằng 1 hệ số Lagrange β (Eq. 8):

$$\min_{p(t|x),p(y|t),p(t)} \Big\{ I(X;T) - \beta I(T;Y) \Big\}.$$

Nghiệm (ẩn) thoả 3 phương trình tự hợp (**self-consistent equations**, Eq. 9):

$$p(t|x) = \frac{p(t)}{Z(x;\beta)} \exp\big(-\beta D_{KL}[p(y|x)\|p(y|t)]\big), \qquad p(t) = \sum_x p(t|x)p(x), \qquad p(y|t) = \sum_x p(y|x)p(x|t).$$

Khi β chạy, tập các nghiệm $(I(X;T), I(T;Y))$ vẽ ra 1 đường cong lồi/đơn điệu gọi là **information curve** (đường cong IB), là biên trên của vùng khả đạt trong information plane: không biểu diễn nào vượt qua được đường này với cùng mức nén.

3.2 Vai trò của noise (mục 2.4) và lựa chọn bài toán thực nghiệm

Nếu $y = f(x)$ là hàm xác định (deterministic), $I(X;Y)$ **không nhạy cảm với độ phức tạp của** f (vì MI bất biến với hoán vị bất kỳ của patterns khi $|X|$ hữu hạn) - đây là vấn đề: không thể dùng MI để phân biệt hàm đơn giản (VC-dimension thấp) với hàm phức tạp chỉ bằng MI của input-output xác định. Cách khắc phục: biến quy tắc thành **stochastic** bằng cách thêm nhiễu nhỏ, ví dụ dùng sigmoid $p(y = 1|x) = \psi(w \cdot x)$ với $\psi(x) = 1/(1 + e^{-x})$ (ví dụ Perceptron) - độ "trải" của sigmoid mã hoá độ phức tạp biên quyết định.

Thiết lập thực nghiệm (mục 3.1): mạng fully-connected feed-forward, hàm kích hoạt **tanh** cho mọi neuron ẩn, lớp cuối dùng dạng sigmoidal; huấn luyện bằng SGD với **cross-entropy loss**, không có regularization tường minh nào khác. Kiến trúc chuẩn trong bài: tối đa **7 lớp ẩn, độ rộng 12-10-7-5-4-3-2 neuron** (Hình 4). Bài toán: luật quyết định nhị phân **bất biến với phép quay** $O(3)$ **của mặt cầu**, với **12 input nhị phân** biểu diễn 12 điểm phân bố đều trên mặt cầu 2D $\Rightarrow 4096$ pattern input khác nhau, chia thành **64 quỹ đạo (orbit) rời nhau** dưới nhóm quay (đây chính là 1 phân hoạch đủ tối thiểu cho luật đối xứng cầu). Hàm $f(x)$ đối xứng cầu tính qua phổ công suất điều hoà cầu (spherical harmonics), so với ngưỡng θ cho nhãn $0/1$, rồi làm "mềm" bằng sigmoid gain γ: $p(y = 1|x) = \psi(f(x) - \theta)$ (Eq. 10). Ngưỡng θ chọn sao cho $p(y = 1) \approx 0.5$; gain γ đủ lớn để $I(X;Y) \approx 0.99$ bit (gần như xác định nhưng không hoàn toàn).

Ước lượng MI (mục 3.2): vì bài toán nhân tạo có đúng 4096 pattern đồng xác suất và biết trước $P(X,Y)$, tác giả **không cần ước lượng từ mẫu** mà tính MI chính xác: rời rạc hoá output kích hoạt arctan của mỗi neuron thành **30 khoảng đều giữa** -1 **và** 1, từ đó tính trực tiếp $P(T_i, X)$ và $P(T_i, Y) = \sum_x P(x,Y)P(T_i|x)$ theo chuỗi Markov $Y \to X \to T_i$. Lặp lại với **50 lần khởi tạo trọng số ngẫu nhiên** khác nhau và lấy mẫu train khác nhau để lấy trung bình/sai số.

4. Thí nghiệm và kết quả chính

Đây là paper **có thí nghiệm số thật** (không phải chỉ lý thuyết), nhưng trên mạng nhỏ và bài toán tổng hợp (synthetic) để kiểm soát được $P(X,Y)$ chính xác, không phải benchmark chuẩn như ImageNet/GLUE.

(a) Hai pha SGD trong information plane (mục 3.3-3.5, Hình 2-4): với 85% dữ liệu train, 50 mạng ngẫu nhiên, các lớp lúc đầu giảm mạnh I_Y (information trên nhãn) ở lớp sâu (do khởi tạo ngẫu nhiên), sau đó: **pha ERM** (ngắn, vài trăm epoch) tăng nhanh I_Y mọi lớp, giữ đúng thứ tự DPI; **pha nén biểu diễn** (dài hơn nhiều) giảm I_X (information trên input) mà gần như không mất I_Y thêm. Khớp với động học gradient (Hình 4):

drift phase (mean gradient $\gg$ std, SNR cao) $\rightarrow$ **diffusion phase** (std $\gg$ mean, giống Wiener process, SNR thấp), chuyển pha ở khoảng ~ 350 **epoch** với kiến trúc chuẩn của bài. Với mẫu train nhỏ (5%), pha nén còn làm **giảm cả** I_Y (dấu hiệu overfit, có thể tránh bằng early stopping) nhưng với mẫu lớn (85%) I_Y chủ yếu tăng trong cả 2 pha.

(b) Giải thích bằng Focker-Planck: pha diffusion mô tả bằng phương trình Fokker-Planck; phân phối dừng của nó **tối đa hoá entropy** của trọng số dưới ràng buộc train error $\Rightarrow$ tối đa $H(X|T_i) \Rightarrow$ tối thiểu $I(X;T_i) = H(X) - H(X|T_i)$ (vì $H(X)$ không đổi). Đây chính là cơ chế "nén bằng khuếch tán" (compression by diffusion).

(c) Lợi ích tính toán của lớp ẩn (mục 3.6, Hình 5): huấn luyện 6 kiến trúc với 1-6 lớp ẩn (80% dữ liệu, 50 lần lặp). Quan sát: (1) thêm lớp ẩn **giảm mạnh số epoch** cần để đạt I_Y tốt - với 1 lớp ẩn mạng không đạt I_Y tốt dù sau 10^4 epoch, với 6 lớp đạt gần như đầy đủ information trong vòng 400 epoch; (2) pha nén của mỗi lớp **ngắn hơn** nếu lớp trước đã được nén sẵn (so sánh 4 vs 5 lớp: 5 lớp hội tụ nhanh gần gấp đôi); (3) nén nhanh hơn ở các lớp sâu/hẹp hơn (gần output), các lớp này "kéo" lớp dưới nén theo; (4) **thêm độ rộng (width) không giúp** - lớp ẩn dù rộng cũng dần nén trong pha diffusion, lớp đầu tiên (rộng 12, bằng chiều input) gần như không nén được vì gần như là ánh xạ 1-1.

(d) Lợi ích tính toán nhìn theo lý thuyết khuếch tán (mục 3.7): khuếch tán đơn giản làm entropy tăng theo $\Delta H \propto \log(Dt)$ (D = hằng số khuếch tán), suy ra số bước (epoch) cần để nén ΔI_X là $\sim \exp(\Delta I_X / D)$. Nếu K lớp mỗi lớp chỉ cần nén một phần nhỏ ΔI_X^k ($\sum_k \Delta I_X^k \approx \Delta I_X$ tổng), Eq. 11 của paper chỉ ra:

$$\exp\Big(\sum_k \Delta I_X^k\Big) \geq \sum_k \exp(\Delta I_X^k),$$

nghĩa là chia nén thành K bước nhỏ **giảm số epoch theo cấp số nhân** so với 1 bước lớn (dù số phép toán mỗi epoch chỉ tăng tuyến tính theo K) - đây là lý giải định lượng cho "lợi ích tính toán của việc sâu hơn".

(e) Hội tụ về biên IB (mục 3.8, Hình 6, Eq. 12): với mỗi lớp đã hội tụ, tác giả tìm β_i^* tối thiểu hoá $\mathbb{E}_x D_{KL}[p_i(t|x) \| p_\beta^{IB}(t|x)]$ (encoder thật so với encoder IB lý thuyết ứng với decoder thật của lớp đó). Kết quả: 5 lớp thực nghiệm nằm **rất gần đường cong IB lý thuyết**, với độ dốc đường cong tại điểm đó (β^{-1}) khớp với β_i^* ước lượng - tức thoả mãn gần đúng hệ phương trình tự hợp Eq. 9.

(f) Ảnh hưởng kích thước mẫu train (mục 3.9, Hình 7): huấn luyện mạng 6 lớp với mẫu train từ 3% đến 85% dữ liệu. Mẫu lớn hơn $\Rightarrow I_Y$ (generalization) được đẩy lên gần biên IB lý thuyết hơn; các lớp hội tụ nằm trên 1 đường cong trơn (ứng với "finite-sample IB bound", biên IB khi decoder là phân phối thực nghiệm chứ không phải $P(X,Y)$ thật).

(g) Kiểm tra tổng quát hoá (mục 4, Hình 8 + thí nghiệm MNIST): lặp lại với luật **không đối xứng** ("committee machine") - quan sát 2 pha và các kết luận tương tự (pha nén nhanh hơn nhưng bản chất giống). Quan trọng: tác giả cũng **kiểm tra thống kê gradient trên MNIST** (bài toán lớn, thực - không chỉ synthetic) và **tìm thấy chuyển pha sang diffusion** tương tự, nhưng **không** vẽ đầy đủ information-plane cho MNIST (vì ước lượng MI trên bài toán lớn/liên tục khó hơn, đòi hỏi estimator phức tạp hơn, tác giả nói "this can be done" nhưng để công trình tương lai).

5. Phân tích / ablation

Paper không có bảng ablation theo nghĩa thông thường (so sánh thành phần kiến trúc), nhưng có các kiểm tra độ nhạy đóng vai trò ablation:

- **Đổi luật quyết định (symmetric $\rightarrow$ committee machine không đối xứng, mục 4 câu hỏi 1)**: 2 pha drift/diffusion và các kết luận chính **không đổi** $\Rightarrow$ tác giả kết luận hiện tượng không phụ thuộc cấu trúc đối xứng đặc biệt của bài toán gốc.

- **Số lớp ẩn (1-6 lớp, mục 3.6)**: biến thiên có kiểm soát, cho thấy lợi ích đơn điệu của việc thêm lớp tới tốc độ hội tụ (không phải biểu diễn cuối cùng, vì cùng 1 đường cong IB là đích).
- **Kích thước mẫu train (3%-85%, mục 3.9)**: cho thấy overfitting liên hệ trực tiếp với pha nén (mẫu nhỏ $\Rightarrow$ nén làm giảm cả I_Y, dấu hiệu mất thông tin liên quan tới nhãn vì mẫu không đại diện).
- **Kiểm tra trên MNIST (dataset thật)**: chỉ xác nhận **tồn tại chuyển pha gradient drift $\rightarrow$ diffusion**, KHÔNG xác nhận đầy đủ các kết luận về information plane/biên IB trên dữ liệu thật - đây là giới hạn quan trọng paper tự thừa nhận một phần (xem mục 6).
- Tác giả cũng lưu ý (mục 3.5) **không có dấu hiệu trọng số tiến về 0 hoặc giảm norm** khi hội tụ, và tương quan giữa in-weights của các neuron trong cùng 1 điểm hội tụ **rất nhỏ** $\Rightarrow$ kết luận có **rất nhiều mạng khác nhau (hàm mũ theo số trọng số)** đạt hiệu năng tối ưu tương đương, nên diễn giải 1 neuron/trọng số cụ thể là ”vô nghĩa” (paper dùng đúng từ ”practically meaningless”).

6. Giới hạn

Theo paper tự nêu (mục 3.4, 4 và các câu hỏi tự đặt ra):

- Hiện tượng pha nén quan sát trên **mạng nhỏ, hàm kích hoạt tanh**, trên **bài toán tổng hợp có cấu trúc đối xứng** đơn giản; paper tự nói ”nên được kiểm chứng trên các bài toán lớn hơn” (mục 3.4: ”it should be verified on larger problems”).
- Trên **MNIST**, tác giả chỉ kiểm tra được **thống kê gradient** (xác nhận có chuyển pha drift/diffusion) nhưng **không** vẽ được đầy đủ information-plane / kiểm tra biên IB, vì ước lượng MI trên bài toán lớn/liên tục đòi hỏi estimator phức tạp hơn (paper nói ”this can be done” nhưng để sau).
- Paper tự đặt 4 câu hỏi mở chưa giải quyết triệt để trong chính bài: (1) tính phổ quát với luật/kiến trúc khác (một phần trả lời bằng committee machine), (2) liệu pha nén bằng nhiễu có xảy ra ở các mô hình học sâu khác (Bayesian network, random forest) hay không - **chỉ nêu giả thuyết, chưa kiểm chứng**, (3) có scale lên bài toán thực lớn không - **chỉ kiểm chứng phần gradient trên MNIST**, (4) hệ quả thuật toán thực tiễn (ví dụ dùng Monte-Carlo relaxation thay SGD trong pha diffusion) - **chỉ là hướng đang nghiên cứu, không có kết quả cụ thể trong bài này**.
- Paper nói ”we are convinced that our analysis and observations are general” (mục 2.5) nhưng đây là **niềm tin/giả thuyết của tác giả**, không phải kết quả đã chứng minh cho mạng lớn.

(Ngoài paper, kiến thức chung) Đây là 1 trong các paper gây tranh cãi nhiều nhất về lý thuyết Deep Learning: nghiên cứu sau đó (Saxe, Bansal, Dapello, Advani, Kolchinsky, Tracey, Cox, ”On the information bottleneck theory of deep learning”, ICLR 2018) phản bác một phần kết luận ở đây, cho rằng (a) hiện tượng ”pha nén” quan sát được **phụ thuộc mạnh vào hàm kích hoạt bão hoà (saturating) như tanh/sigmoid** và **không xuất hiện rõ với ReLU** (hàm kích hoạt phổ biến hơn trong Deep Learning thực tế), và (b) các phép đo ”mutual information” trên mạng với kích hoạt liên tục/xác định phải dựa vào kỹ thuật binning cụ thể, kết quả MI đo được có thể phụ thuộc vào cách binning hơn là phản ánh hiện tượng nén ”thật”. Người đọc nên xem bài 1703.00810 này là 1 giả thuyết thú vị và có thực nghiệm ủng hộ trên mạng nhỏ, không phải 1 định lý đã được cộng đồng đồng thuận hoàn toàn.

7. Liên hệ với các bài khác

Đã kiểm tra bằng `ls` các thư mục liên quan trước khi trỏ (math content hiện còn trống vì các bài 01-27 đang được viết song song trong phiên này; sẽ không trỏ tới các bài đó để tránh link treo):

- **”interp content/01_2022_mech_interp_essay.md”**: một hướng hoàn toàn khác để ”mở hộp đen” DNN, mechanistic interpretability (lần theo circuit/feature cụ thể) so với cách tiếp cận information-theoretic/thống kê tổng hợp (không nhìn vào trọng số cụ thể) của bài này - 2 triết lý bổ sung cho nhau: bài 30 nói ”từng neuron vô nghĩa”, mech interp thì cố gắng tìm ý nghĩa ở mức circuit/feature.
- **”interp content/05_2022_toy_models_of_superposition.md”**: cũng nghiên cứu cấu trúc biểu diễn ẩn của mạng nhỏ bằng mô hình toán học/thực nghiệm toy có kiểm soát, giống phương pháp luận của bài

30 (dùng bài toán tổng hợp nhỏ để hiểu hiện tượng tổng quát).

- **"ml basics content/01_2016_rethinking_generalization.md"** (Zhang et al., "Understanding deep learning requires rethinking generalization"): bài này được **chính paper 30 trích dẫn trực tiếp** (mục 3.5, "Zhang et al. (2016)") làm bằng chứng rằng các dạng regularization tường minh (weight decay, dropout, data augmentation) không giải thích đủ khả năng generalize của DNN - cùng câu hỏi lớn mà bài 30 cố trả lời bằng information bottleneck.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU $\leq$ 8GB, batch 1); các hướng pruning/distillation/RL/interpretability đã có thư mục riêng.

(Suy luận) Ý tưởng "information plane" ở đây **khó áp dụng trực tiếp** để quyết định layer nào nên prune trong LightOnOCR, vì 3 lý do: (1) paper chỉ kiểm chứng trên mạng fully-connected nhỏ với kích hoạt tanh/sigmoid, không phải Transformer/SSM lớn với attention và GatedDeltaNet; (2) ước lượng MI chính xác cần biết $P(X, Y)$ hoặc binning hợp lý, điều này rất khó cho không gian token hình ảnh/văn bản nhiều chiều của OCR; (3) như mục 6 đã nêu, hiện tượng nén có thể phụ thuộc kích hoạt bão hoà, còn LightOnOCR dùng kiến trúc hiện đại (SiLU/GELU, attention, gating) không bão hoà theo kiểu tanh.

Tuy vậy, có 1 liên hệ **gián tiếp hợp lý**: ý tưởng đo "bao nhiêu thông tin về input X còn được giữ ở mỗi lớp" gợi hướng heuristic để đánh giá **độ dư thừa (redundancy)** của 1 lớp/head trước khi quyết định prune: nếu có thể ước lượng (xấp xỉ, ví dụ qua entropy của activation hoặc proxy khác) rằng 1 lớp của LightOnOCR đã "nén" gần hết thông tin input (giữ lại ít $I(X;T)$ mà $I(T;Y)$ vẫn cao), lớp đó có thể là ứng viên tốt để giảm chiều/prune mà ít ảnh hưởng độ chính xác OCR - nhưng đây là loại suy, chưa có công cụ đo trực tiếp áp dụng được từ chính paper này cho mô hình quy mô ~0.8B tham số.

9. Thí nghiệm gợi ý

1. **Tái tạo thu nhỏ hiện tượng 2 pha trên MLP nhỏ (tanh)**: huấn luyện MLP 4-5 lớp (ví dụ 20-15-10-5-2 neuron), kích hoạt tanh, trên 1 bài toán phân loại nhị phân tổng hợp nhỏ (ví dụ XOR mở rộng hoặc tập dữ liệu dạng "circle"/"moons" của scikit-learn). Đo và vẽ $I(X;T_i)$, $I(T_i;Y)$ qua epoch bằng binning giống paper (rời rạc hoá activation thành ~30 bin). Kỳ vọng: thấy pha tăng I_Y nhanh rồi pha giảm I_X chậm hơn, tương tự Hình 3 của paper. Chạy được trên CPU/GPU 8GB trong vài phút.
2. **So sánh tanh vs ReLU**: lặp lại thí nghiệm 1 nhưng đổi activation sang ReLU. Đo xem pha "nén" (I_X giảm rõ) còn xuất hiện rõ như với tanh không. Kỳ vọng theo phản biện Saxe et al. (ngoài paper): hiện tượng nén sẽ **yếu hơn hoặc không rõ** với ReLU - nếu đúng, xác nhận giới hạn đã nêu ở mục 6.
3. **Đo SNR của gradient theo epoch**: trong cùng thí nghiệm 1, log lại mean và std của gradient mỗi lớp qua epoch (giống Hình 4 paper), vẽ trên thang log-log. Kỳ vọng thấy điểm chuyển giữa 2 pha (SNR cao $\rightarrow$ thấp) khớp thời điểm I_X bắt đầu giảm mạnh.
4. **Ảnh hưởng số lớp tới tốc độ hội tụ I_Y**: lặp lại thí nghiệm 1 với 1, 2, 4, 6 lớp ẩn (giữ tổng số neuron gần tương đương hoặc để width giảm dần như paper), đo số epoch cần để I_Y đạt một ngưỡng cố định (ví dụ 90% giá trị hội tụ). Kỳ vọng: mạng nhiều lớp hơn đạt ngưỡng này **nhanh hơn** (ít epoch hơn), minh hoạ lại mục 4(c)/(d) của note.
5. **Ảnh hưởng kích thước mẫu train tới overfitting qua information plane**: lặp thí nghiệm 1 với tỉ lệ dữ liệu train 5%, 45%, 85% (giữ tập test cố định để đo I_Y "thật"). Kỳ vọng: với mẫu nhỏ, I_Y **giảm** trong pha nén (dấu hiệu overfit), với mẫu lớn I_Y tiếp tục tăng hoặc giữ nguyên - khớp quan sát mục 3.4 của paper.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Mutual information	$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} = H(X) - H(X\|Y)$ (Eq. 1-2)	Lượng thông tin X, Y chia sẻ; 0 khi độc lập
Bất biến khả nghịch	$I(X;Y) = I(\psi(X); \varphi(Y))$ (Eq. 3)	Lớp biến đổi 1-1 không đổi MI với input/label
Data Processing Inequality	$X \to Y \to Z \Rightarrow I(X;Y) \geq I(X;Z)$ (Eq. 4)	Xử lý thêm chỉ mất thông tin, không sinh thêm
Information Bottleneck	$\min_{p(t\|x)} I(X;T) - \beta I(T;Y)$ (Eq. 8)	Đánh đổi nén X và giữ thông tin về Y
Self-consistent equations	$p(t\|x) \propto p(t) \exp(-\beta D_{KL}[p(y\|x)\|p(y\|t)])$ (Eq. 9)	Điều kiện biểu diễn tối ưu trên đường cong IB
Compression theo diffusion	số epoch $\sim \exp(\Delta I_X / D)$; với K lớp: $\exp(\sum_k \Delta I_X^k) \geq \sum_k \exp(\Delta I_X^k)$ (Eq. 11)	Lý giải lợi ích tính toán của việc thêm lớp sâu
β_i^* tối ưu mỗi lớp	$\beta_i^* = \arg\min_\beta \mathbb{E}_x D_{KL}[p_i(t\|x)\|p_\beta^{IB}(t\|x)]$ (Eq. 12)	Đo lớp thực nghiệm gần đường cong IB tới mức nào

Ví dụ tính tay (tự tính): minh hoạ công thức Eq. 1-2 với 1 phân phối rời rạc nhỏ, $X, Y \in \{0, 1\}$, phân phối đồng thời $p(0,0) = 0.4$, $p(0,1) = 0.1$, $p(1,0) = 0.1$, $p(1,1) = 0.4$ (kênh nhiễu, tương quan dương giữa X và Y). Biên: $p(X=0) = p(X=1) = 0.5$, $p(Y=0) = p(Y=1) = 0.5$, nên $p(x)p(y) = 0.25$ với mọi cặp. Tính từng số hạng (log cơ số 2):

$$I(X;Y) = 0.4 \log_2 \frac{0.4}{0.25} + 0.1 \log_2 \frac{0.1}{0.25} + 0.1 \log_2 \frac{0.1}{0.25} + 0.4 \log_2 \frac{0.4}{0.25}.$$

$\log_2(1.6) \approx 0.678$ và $\log_2(0.4) \approx -1.322$, nên (tự tính): $I(X;Y) \approx 2 \times (0.4 \times 0.678) + 2 \times (0.1 \times (-1.322)) = 2 \times 0.2711 - 2 \times 0.1322 = 0.5422 - 0.2644 \approx 0.278$ bit. So với trường hợp X, Y độc lập hoàn toàn ($I = 0$) và trường hợp xác định hoàn toàn ($I(X;Y) = H(Y) = 1$ bit vì $p(y=1) = 0.5$), giá trị ≈ 0.278 bit phản ánh đúng mức tương quan vừa phải mà ta đặt vào ví dụ - minh hoạ trực quan cho đại lượng mà paper dùng để đo $I(X;T_i)$ và $I(T_i;Y)$ ở mỗi lớp.

11. Kết luận ngắn

Bằng cách coi mỗi lớp DNN là 1 biến ngẫu nhiên và vẽ quỹ đạo $(I(X;T_i), I(T_i;Y))$ qua các epoch huấn luyện trên mạng fully-connected nhỏ (tanh) với bài toán tổng hợp có $P(X,Y)$ biết trước, paper phát hiện SGD có 2 pha rõ rệt (drift/ERM rồi diffusion/nén), các lớp hội tụ gần đường cong Information Bottleneck lý thuyết, và lợi ích chính của việc thêm lớp sâu là **giảm số epoch cần để nén** (lợi ích tính toán) chứ không hẳn là biểu diễn cuối cùng tốt hơn. Đây là 1 trong những cố gắng thực nghiệm sớm và có ảnh hưởng lớn để giải thích "tại sao Deep Learning hoạt động", nhưng kết luận giới hạn ở mạng nhỏ/kích hoạt bão hoà/bài toán tổng hợp, và bị phản biện đáng kể sau đó (Saxe et al. 2018, ngoài phạm vi paper này) về tính phổ quát với ReLU và độ nhạy của phép đo MI với cách binning.

Nguồn gốc: https://arxiv.org/abs/1703.00810

Bài 31

Deep Learning: A Statistical Viewpoint (2021)

Tác giả: Peter L. Bartlett, Andrea Montanari, Alexander Rakhlin
Venue / arXiv: arXiv:2103.09177 (v1, 16/03/2021); paper không nêu venue trong phần text trích được (được biết đến rộng rãi là Acta Numerica 2021, nhưng dòng chữ này không xuất hiện trong văn bản arXiv trích xuất được, nên ghi ”paper không nêu venue” theo đúng text trích được)
Nguồn: https://arxiv.org/abs/2103.09177
Chủ đề: Bài khảo sát (survey) tổng hợp các kết quả gần đây của lý thuyết học thống kê nhằm lý giải hai hiện tượng bí ẩn của deep learning: (a) gradient descent giải được bài toán tối ưu không lồi một cách hiệu quả khi mô hình quá tham số hoá (tractability via overparametrization), và (b) mô hình vẫn tổng quát hoá tốt dù khớp hoàn hảo dữ liệu huấn luyện có nhiễu (generalization via implicit regularization / benign overfitting).
Vị trí trong lộ trình: Đọc sau math 32 (Benign Overfitting in Linear Regression, một trong các kết quả chính được khảo sát lại ở đây), math 33 (Gradient Descent Finds Global Minima of DNNs) và math 34 (Mean Field View), vì bài này tổng hợp và đặt cả ba vào một khung lý thuyết thống nhất; là bài cuối cùng được viết trong batch math 28-35 nhưng đóng vai trò ”bản đồ tổng quan” nên có thể đọc trước hoặc sau các bài 32/33/34 tuỳ mục đích.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper mở đầu bằng việc nêu rõ **ba mục tiêu cạnh tranh** mà mọi phương pháp dự đoán thống kê phải cân bằng: (a) **tính biểu diễn (expressivity)**, để mô hình hoá được độ phong phú của dữ liệu thực; (b) **kiểm soát độ phức tạp thống kê (statistical complexity)**, để dùng tốt lượng dữ liệu huấn luyện hạn chế; (c) **hiệu quả tính toán (computational efficiency)**. Cách tiếp cận cổ điển là dùng một mô hình giàu, kết hợp với **regularization tường minh** (ví dụ ridge, early stopping) để kiểm soát độ phức tạp, và dùng **loss lồi** để tối ưu hiệu quả.

Deep learning phá vỡ cả hai nguyên tắc cổ điển này qua hai quan sát thực nghiệm gây ”bất ngờ”:

1. **Tractability via overparametrization**: khi mô hình đủ giàu tham số (quá tham số hoá), bài toán tối ưu **không lồi** lại trở nên dễ giải bằng các phương pháp gradient đơn giản, dù về lý thuyết cổ điển điều này ”có vẻ hoàn toàn vô lý” (paper dùng chính từ ”foolish”).
2. **Generalization via implicit regularization**: mô hình được huấn luyện **không có regularization tường minh nào**, đạt loss huấn luyện gần 0 (khớp hoàn toàn dữ liệu, kể cả dữ liệu có nhiễu), nhưng vẫn **tổng quát hoá tốt** trên dữ liệu mới. Đây là hiện tượng paper gọi là **”benign overfitting”** (tạm dịch: overfitting lành tính) - dùng đúng nghĩa đen của từ ”overfitting” (khớp tốt hơn mức cần thiết), không mang nghĩa tiêu cực như cách dùng thông thường trong giáo trình thống kê cổ điển.

Thuật ngữ cần làm rõ: **uniform convergence** (hội tụ đều) là tính chất mà sai số thực nghiệm (empirical risk) xấp xỉ tốt sai số kỳ vọng (true risk) đồng thời cho **mọi** hàm trong một lớp hàm $\mathcal{F}$, không chỉ cho một hàm cụ thể; đây là công cụ cốt lõi của lý thuyết học thống kê cổ điển (VC theory, Rademacher complexity). **Implicit regularization** (điều chuẩn ẩn/ngầm) là hiện tượng thuật toán tối ưu (ví dụ gradient descent) tự động chọn ra một nghiệm ”đặc biệt” trong số rất nhiều nghiệm khớp hoàn hảo dữ liệu, mà không cần một số hạng regularization tường minh nào trong hàm mục tiêu.

Paper phát biểu rõ **hai giả thuyết làm việc (working hypotheses)** xuyên suốt toàn bài (trích dẫn gần nguyên văn, dịch nghĩa): (1) độ khó của bài toán tối ưu phụ thuộc vào quan hệ giữa số chiều tham số và số lượng ràng buộc (kích thước mẫu); bài toán trở nên dễ (tractable) khi và chỉ khi mô hình đủ ”thiếu ràng buộc” (quá tham

số hoá); (2) các thuật toán huấn luyện tạo ra một độ lệch (bias) phá vỡ sự tương đương giữa mọi mô hình khớp hoàn hảo dữ liệu quan sát; mô hình deep learning có thể phân rã thành một **thành phần độ phức tạp thấp** (nơi uniform convergence cổ điển vẫn áp dụng) và một **thành phần độ phức tạp cao** (cho phép khớp hoàn hảo dữ liệu), và nếu mô hình đủ quá tham số hoá, thành phần sau không ảnh hưởng đáng kể đến độ chính xác dự đoán.

Paper cũng tự nói rõ một giới hạn phạm vi ngay từ Introduction: **không có kết quả nào trong bài chứng minh lợi ích của độ sâu (depth)** cho tối ưu hoá hay tổng quát hoá; độ sâu được paper cho là quan trọng chủ yếu cho **tính biểu diễn (expressivity)**, một khía cạnh nằm ngoài phạm vi khảo sát này.

2. Ý tưởng chính

Ý tưởng tổ chức xuyên suốt bài là **phân rã "đơn giản + gai nhọn" (simple-plus-spiky decomposition)**: mọi nghiệm nội suy (interpolating solution) tốt mà paper khảo sát đều có thể viết dưới dạng

$$\hat{f}(x) = \hat{f}_0(x) + \Delta(x),$$

với $\hat{f}_0$ là một thành phần **đơn giản** (ví dụ trơn/smooth, hoặc nằm trong một không gian con chiều thấp) chịu trách nhiệm cho khả năng dự đoán tốt, và Δ là một thành phần **gai nhọn (spiky)** chịu trách nhiệm cho việc khớp hoàn hảo dữ liệu (kể cả nhiễu) nhưng, trong điều kiện thuận lợi (quá tham số hoá đủ lớn), **không gây hại** cho độ chính xác dự đoán ($L(\hat{f}_0 + \Delta) \approx L(\hat{f}_0)$).

Lý thuyết hội tụ đều cổ điển **không thể** giải thích hiện tượng benign overfitting khi có nhiễu. Paper chứng minh điều này một cách định lượng (mục 2.9, đọc kỹ): với bất kỳ cận trên dựa trên độ phức tạp (ví dụ Rademacher complexity) của hàm nội suy có độ phức tạp tối thiểu, nếu dữ liệu có nhiễu (không có quan hệ tất định $y = f^*(x)$), thì cận trên này **nhất thiết phải** $\geq$ **một hằng số dương** (bằng đúng mức nhiễu $L(f^*)$), bất kể kích thước mẫu n lớn đến đâu. Lý do: cận hội tụ đều không phụ thuộc vào tính chất của phân phối X, nên vẫn đúng cả trong bối cảnh "fixed design" (thiết kế cố định $x_1, \ldots, x_n$), nơi excess risk của một hàm nội suy hoàn hảo **chắc chắn** $\geq$ mức nhiễu. Điều này nghĩa là: **cần một công cụ lý thuyết khác** (không chỉ dựa vào độ phức tạp của riêng hàm $\hat{f}$) để giải thích benign overfitting; công cụ đó phải khai thác thêm thông tin về phân phối dữ liệu/cơ chế sinh dữ liệu, không chỉ về bản thân hàm $\hat{f}$.

3. Phương pháp chi tiết

3.1. Implicit regularization qua gradient descent (Mục 3)

Ví dụ đơn giản nhất: hồi quy tuyến tính bình phương tối thiểu trong chế độ quá tham số hoá ($d > n$). Gradient descent khởi tạo từ $\theta_0 = 0$ trên loss $\hat{L}(\theta) = \frac{1}{n}\|X\theta - y\|_2^2$ hội tụ về **nghiệm có norm nhỏ nhất (minimum-norm interpolant)**:

$$\hat{\theta} = \operatorname*{argmin}_{\theta} \{\|\theta\|_2 : \langle \theta, x_i \rangle = y_i \; \forall i \leq n\} = X^{\dagger} y, \qquad \text{(14-15, paper)}$$

là giới hạn của ridge regression khi $\lambda \to 0^+$. Chứng minh dựa vào điều kiện KKT: $X\theta = y$ và $\theta + X^T\mu = 0$, cả hai được thoả bởi gradient descent khi nó khớp dữ liệu trong khi vẫn ở trong không gian sinh bởi các hàng của X.

Ví dụ thứ hai: **boosting**. AdaBoost tương ứng với coordinate descent trên loss hàm mũ (18); với dữ liệu phân tách tuyến tính (separable), Zhu & Yu (2005)/Telgarsky (2013) chứng minh hướng của θ_t hội tụ về nghiệm **max-margin chuẩn** ℓ_1. Soudry et al. (2018)/Ji & Telgarsky (2018) chứng minh **gradient** descent (không phải coordinate descent) trên loss logistic cho dữ liệu separable hội tụ về nghiệm **max-margin chuẩn** ℓ_2 (Định lý 3.1 của paper, trích đầy đủ): $\theta_t = \hat{\theta} \log t + \rho_t$ với $\|\rho_t\| = O(\log \log t)$, nên $\theta_t / \|\theta_t\|_2 \to \hat{\theta}/\|\hat{\theta}\|_2$.

Ví dụ thứ ba: **tham số hoá lại (reparametrization)**. Với $\theta(u)_i = u_i^2$ (bình phương theo toạ độ), gradient flow trên bài toán tham số hoá lại với khởi tạo $\alpha \to 0$ hội tụ về nghiệm **chuẩn ℓ_1 nhỏ nhất** trong không gian θ gốc (Gunasekar et al. 2017), khác hẳn với chuẩn ℓ_2 của tham số hoá gốc - cho thấy **cách tham số hoá mô hình cũng tạo ra implicit regularization**, không chỉ thuật toán tối ưu.

3.2. Benign overfitting: ví dụ Nadaraya-Watson (Mục 4.1)

Ví dụ sư phạm đơn giản nhất: ước lượng Nadaraya-Watson $\hat{f}(x) = \sum_i y_i\, \omega_i(x)$ với $\omega_i(x) = K((x - x_i)/h) / \sum_j K((x - x_j)/h)$. Với kernel **kỳ dị tại 0** (ví dụ $K(u) = \|u\|_2^{-a} \mathbb{1}[\|u\|_2 \le 1]$ với $0 < a < d/2$), ước lượng này **tự động nội suy dữ liệu** với mọi bandwidth h. Định lý 4.1 (trích từ Belkin, Rakhlin, Tsybakov 2019): với f^* thuộc lớp Hölder $H(\beta, L)$,

$$\mathbb{E}[\widehat{\text{bias}}^2] \lesssim h^{2\beta}, \qquad \mathbb{E}[\widehat{\text{var}}] \lesssim \sigma_\xi^2 (nh^d)^{-1}, \tag{31, paper}$$

chọn $h = n^{-1/(2\beta+d)}$ cho tốc độ minimax tối ưu. Điểm mấu chốt paper nhấn mạnh: **việc kernel có kỳ dị hay không (quyết định có nội suy hay không) hoàn toàn độc lập với mức độ "làm mượt" (bandwidth h, quyết định bias-variance trade-off)** - đây là minh chứng sư phạm đầu tiên cho thấy nội suy (fit hoàn hảo) không nhất thiết xung đột với một bias-variance trade-off tốt.

3.3. Benign overfitting trong hồi quy tuyến tính (Mục 4.2, liên hệ trực tiếp math 32)

Với $f^*(x) = \langle \theta^*, x \rangle$ và nghiệm minimum-norm $\hat{\theta}$ (Eq. 32-33), decomposition theo **effective rank** của ma trận hiệp phương sai $\Sigma = \mathbb{E}[xx^T]$: chọn chỉ số k, định nghĩa

$$r_k(\Sigma) = \frac{\sum_{i>k} \lambda_i}{\lambda_{k+1}}, \tag{Lemma 4.3/định nghĩa effective rank, paper}$$

nếu $r_k(\Sigma) \gtrsim n$, phần "tail" của ma trận dữ liệu $X_{>k}^T X_{>k}$ hành xử gần như đẳng hướng (near-isotropic), tạo ra một **self-induced regularization** với quy mô $\gamma = \sum_{i>k} \lambda_i$ (tương đương một ridge penalty tự nhiên). Đây đúng là kết quả chính của Bartlett-Long-Lugosi-Tsigler (2019/2020) đã được trình bày đầy đủ ở ghi chú **math 32**; paper này trình bày lại cùng kết quả ở dạng súc tích hơn, nhấn mạnh **sự khác biệt giữa self-induced regularization (phụ thuộc cơ chế sinh dữ liệu, tức tính chất của Σ) và implicit regularization (phụ thuộc thuật toán tối ưu, tức việc chọn nghiệm minimum-norm)** - một phân biệt quan trọng mà paper tự highlight bằng một footnote (chú thích số 4 trong văn bản).

3.4. Chế độ tuyến tính (linear regime) cho mạng nơ-ron (Mục 5.1)

Với mạng $f(x; \theta)$, gradient flow $\frac{d\theta_t}{dt} = \frac{1}{n} Df_n(\theta_t)^T (y - f_n(\theta_t))$ (Eq. 77) được so sánh với phiên bản **tuyến tính hoá (linearized)** quanh khởi tạo θ_0 (Eq. 78-79). **Định lý 5.1** (trích đầy đủ): nếu

$$\text{Lip}(Df_n)\, \|y - f_n(\theta_0)\|_2 < \frac{1}{4} \sigma_{\min}^2(Df_n(\theta_0)), \tag{84, paper}$$

thì (a) empirical risk giảm theo cấp số mũ với tốc độ $\lambda_0 = \sigma_{\min}^2/(2n)$; (b) tham số θ_t ở gần khởi tạo và được theo dõi sát bởi luồng tuyến tính hoá; (c) hàm $f(\theta_t)$ và $f_{\text{lin}}(\theta_t)$ gần nhau trên dữ liệu test. Áp dụng cho mạng 2 lớp $f(x; \theta) = \frac{\alpha}{\sqrt{m}} \sum_j b_j \sigma(\langle w_j, x \rangle)$ (Eq. 90) với hai cách khởi tạo khác nhau (một khởi tạo "chuẩn" $\theta_0^{(1)}$, một khởi tạo "đối xứng-trừ-ban-đầu" $\theta_0^{(2)}$ có $f(x; \theta_0^{(2)}) = 0$), **Định lý 5.4** cho thấy: với $\alpha = \Theta(1)$, cần $md \gg n^2$ (mạnh hơn hẳn điều kiện "chỉ cần quá tham số hoá" $md \gg n$); nhưng với $\alpha \to \infty$ và khởi tạo $\theta_0^{(2)}$, chỉ cần $md \gg n \log n$ (gần với mức quá tham số hoá tối thiểu) để đạt chế độ tuyến tính.

3.5. Mean-field vượt ra ngoài chế độ tuyến tính (Mục 5.2, liên hệ trực tiếp math 34)

Với scaling $\alpha = \alpha_0/\sqrt{m}$ (khác với chế độ tuyến tính), paper trình bày **Định lý 5.5** (trích dẫn công trình Mei-Montanari-Nguyen chính là **math 34**): giới hạn $m \to \infty$ của gradient flow là PDE

$$\partial_t \rho_t(w,b) = \alpha_0 \nabla(\rho_t(w,b)\nabla\Psi(w,b;\rho_t)), \qquad \text{(106, paper)}$$

và paper tự tính toán định lượng (Eq. 112-115) rằng trong chế độ mean-field, độ lệch trọng số **không triệt tiêu** khi $m \to \infty$ ($\frac{1}{\sqrt{m}}\|W_t - W_0\|_F$ bị chặn dưới bởi $c_1\sqrt{d/n}$), đối lập hẳn với chế độ tuyến tính (nơi độ lệch này $\to 0$). Đây là minh chứng định lượng cho thấy **cùng một kiến trúc mạng 2 lớp vô hạn rộng có thể có hai chế độ huấn luyện hoàn toàn khác nhau** (tuyến tính/"lazy" vs. mean-field/"feature learning"), chỉ phụ thuộc vào cách chọn hệ số scaling α theo m.

3.6. Ba mô hình tuyến tính hoá: NT-norm, random features, neural tangent (Mục 6.1-6.4)

Paper định nghĩa 3 "họ hàm" được xây từ việc tuyến tính hoá mạng 2 lớp theo các cách khác nhau:

- $\mathcal{F}_{RF}^m$ **(random features)**: chỉ huấn luyện lớp 2, lớp 1 cố định ngẫu nhiên: $f_{\text{lin}}(x;a) = \sum_i a_i \sigma(\langle w_i, x\rangle)$ (Eq. 124).
- $\mathcal{F}_{NT}^m$ **(neural tangent)**: chỉ huấn luyện lớp 1 (tuyến tính hoá), lớp 2 cố định: $f_{\text{lin}}(x;a) = \sum_i \langle a_i, x\rangle \sigma'(\langle w_i, x\rangle)$ (Eq. 125).

Cả hai tương ứng với **kernel ridge regression (KRR)** với các kernel ngẫu nhiên $K_{RF,m}$, $K_{NT,m}$ (Eq. 126-127), hội tụ về kernel kỳ vọng K_{RF}, K_{NT} khi $m \to \infty$, là các hàm của $\langle x_1, x_2\rangle$ (bất biến quay).

4. Thí nghiệm và kết quả chính

Paper này là một bài khảo sát lý thuyết, **không tiến hành thí nghiệm mới trên dữ liệu thực**; phần "kết quả" chính là các định lý được tổng hợp từ nhiều công trình, cùng **một hình mô phỏng số minh hoạ (Hình 1)** để trực quan hoá một hiện tượng lý thuyết.

Hình 1 (double descent, mục 6.3.2, đọc kỹ): mô phỏng random features model với $d = 100, n = 400, \tau^2 = 0.5$, hàm đích $f^* = \langle \beta_0, x\rangle$ với $\|\beta_0\|_2 = 1$, ridge regression với $\lambda = 10^{-3}(m/d)$, quét tỷ lệ quá tham số hoá m/n từ 0 đến 5, trung bình trên 20 lần thực hiện độc lập. Kết quả: train error giảm đơn điệu theo m/n; **test error có một đỉnh (peak) tại ngưỡng nội suy** $m/n = 1$ rồi giảm và hội tụ về một giá trị tiệm cận khi $m/n \gg 1$. Đây là hiện tượng **"double descent"** (lần đầu đặt tên bởi Belkin, Hsu, Ma, Mandal 2019), và paper chỉ ra các đường cong trong Hình 1 là **dự đoán tiệm cận chính xác (exact asymptotic)** từ lý thuyết ma trận ngẫu nhiên phi tuyến (nonlinear random matrix theory), không phải chỉ là xu hướng định tính.

Định lý 6.1 (polynomial scaling, đọc kỹ): với $d^\ell \ll m \ll d^{\ell+1}$ (phân tách lũy thừa rõ rệt giữa m, n, d), test error của random features model bị quyết định bởi $\min(m,n)$: nếu $m \ll n$ (lỗi xấp xỉ chiếm ưu thế), mô hình chỉ khớp được thành phần đa thức bậc $\le \ell$ của f^*; nếu $n \ll m$ (lỗi thống kê chiếm ưu thế), cũng chỉ học được $P_{\le \ell} f^*$. Cả hai được paper mô tả bằng một "parameter-counting heuristic": không gian đa thức bậc ℓ có chiều $\Theta(d^\ell)$, cần số tham số/mẫu tương ứng cỡ đó để học được.

Định lý 6.2 + Hệ quả 6.3 (universality và tối ưu của nội suy ở SNR cao, đọc kỹ): ở chế độ tỷ lệ proportional ($m/d \to \psi_w$, $n/d \to \psi_s$), risk của random features model **tiệm cận bằng** risk của một "noisy features model" đơn giản hơn (ma trận với các thành phần độc lập), một hiện tượng **universality**. Hệ quả: tồn tại một ngưỡng SNR^* sao cho nếu tỷ số tín hiệu/nhiễu (SNR) của bài toán $> \text{SNR}^*$, thì **regularization tối ưu là** $\lambda = 0^+$ (tức nội suy hoàn hảo là tối ưu, không chỉ tối ưu về tốc độ mà về chính giá trị risk).

Định lý 6.4 (neural tangent model, chỉ giới thiệu kết quả, không đọc chi tiết phần proof dài): tổng quát hoá một phần Định lý 6.1 cho NT model; paper kết luận NT model có tính chất tổng quát hoá **rất giống RF model** (nếu giữ số tham số không đổi), nhưng NT có **lợi thế tính toán**: với cùng số tham số p, độ phức tạp

dự đoán là $O(p)$ cho NT so với $O(pd)$ cho RF (vì NT không cần nhân ma trận W lúc dự đoán, chỉ cần dùng Jacobian đã tính).

5. Phân tích / ablation

Vì đây là bài khảo sát, "phân tích" ở đây tổng hợp các so sánh định lượng mà chính paper đưa ra giữa các mô hình/chế độ:

So sánh 2 khởi tạo trong chế độ tuyến tính (Định lý 5.4, mục 3.4 của ghi chú): sự khác biệt giữa yêu cầu $md \gg n^2$ (khởi tạo chuẩn, $\alpha = \Theta(1)$) và $md \gg n \log n$ (khởi tạo "triệt tiêu ban đầu", $\alpha \to \infty$) cho thấy **cách khởi tạo ảnh hưởng mạnh đến mức độ quá tham số hoá cần thiết** để đạt chế độ tuyến tính - một bài học thực hành quan trọng (dù chỉ chứng minh cho mạng 2 lớp đơn giản).

So sánh chế độ tuyến tính vs. mean-field (mục 3.5 của ghi chú): cùng kiến trúc, chỉ khác hệ số scaling α theo m, cho hai hành vi huấn luyện khác hẳn về chất (trọng số gần như không đổi vs. trọng số dịch chuyển đáng kể). Paper tự nhấn mạnh đây **không phải là một hạn chế kỹ thuật của phép phân tích**, mà là một **sự khác biệt thực sự về chế độ huấn luyện** (chứng minh bằng chặn dưới định lượng, Eq. 113-115, không chỉ chặn trên).

So sánh RF vs NT vs KRR (mục 6.4): paper liệt kê 4 câu hỏi dẫn dắt phần này (dịch nghĩa): (i) RF/NT có hành xử giống KRR không? (ii) m cần lớn đến đâu để khớp giới hạn kernel? (iii) sai khác gì khi m không đủ lớn? (iv) có mô hình nào ưu thế hơn? Trả lời tổng hợp của paper: về mặt **thống kê** (generalization), ba mô hình tương đương khi số tham số đủ lớn so với n; về mặt **tính toán**, NT có lợi thế rõ rệt với cùng số tham số.

Phân tích vai trò effective dimension/SNR trong Hệ quả 6.3: kết quả "nội suy tối ưu khi SNR cao" là một đóng góp ý nghĩa: nó **phản bác trực giác cổ điển** rằng luôn cần một chút regularization; ở chế độ "dữ liệu sạch, nhiễu thấp", mô hình nội suy hoàn hảo ($\lambda = 0^+$) mới là lựa chọn tối ưu đúng nghĩa (optimal risk, không chỉ optimal rate).

6. Giới hạn

Các giới hạn dưới đây paper **tự nêu rõ**, đặc biệt tập trung ở Mục 7 (Conclusions and future directions, đọc kỹ):

- **Chỉ giải quyết được các mô hình tuyến tính trong một không gian đặc trưng (feature space)** mà bản đồ đặc trưng (featurization) không phụ thuộc dữ liệu: RKHS, random features, neural tangent features. Paper tự nói: "the scope of this analysis might appear to be limited (neural networks are notoriously nonlinear in their parameters)".
- **Phân rã "đơn giản + gai nhọn" chỉ được hiểu đầy đủ cho mô hình tuyến tính**: trong mô hình tuyến tính, hai thành phần là **trực giao** và **không phụ thuộc dữ liệu** (chỉ phụ thuộc kernel/eigenbasis); với mô hình phi tuyến (mạng nơ-ron thật), paper tự nói "the two components need not be orthogonal and the associated subspaces are likely to be data-dependent" - tức cơ chế này **chưa được hiểu đầy đủ** ngoài chế độ tuyến tính.
- **Cơ chế implicit regularization chỉ được hiểu trong "a handful of cases"** (paper tự dùng đúng cụm từ này): chưa có lý thuyết tổng quát cho việc gradient descent chọn nghiệm nào trong không gian các nghiệm nội suy, đặc biệt khi mô hình phi tuyến.
- **Chưa có định nghĩa toán học rõ ràng cho "effective/intrinsic dimension"** trong bối cảnh deep learning thực tế (ảnh, văn bản): paper tự đặt câu hỏi mở "What is the proper mathematical definition of effective dimension?" ở mục Role of dimension.
- **Không biết lượng quá tham số hoá cần thiết trong thực tế**: paper tự nói "we are far from understanding precisely what amount of overparametrization is necessary, even in simple neural network models" - một giới hạn ngay cả cho các mô hình đơn giản nhất mà paper khảo sát.

- **Không chứng minh lợi ích của độ sâu**: đã nêu ở §1, paper chủ động không đưa ra bất kỳ kết quả nào về lợi ích tối ưu hoá/tổng quát hoá của độ sâu, dù công nhận độ sâu ”seems likely crucial” cho tính biểu diễn.
- **Neural tangent model (§6.4) kém được hiểu hơn random features**: paper tự nói nguyên nhân kỹ thuật là ma trận đặc trưng của NT **không có các cột độc lập** (Eq. 147), khó phân tích hơn bằng lý thuyết ma trận ngẫu nhiên hiện có.
- **Universality (Định lý 6.2) mới được chứng minh đầy đủ cho ridge regression với loss bình phương**; paper nói việc tổng quát hoá cho các loss khác (ví dụ max-margin, logistic) dựa trên ”universality hypothesis” ở một số công trình liên quan, chưa có chứng minh trực tiếp tổng quát trong chính bài này.

7. Liên hệ với các bài khác

- **math content/32 (Benign Overfitting in Linear Regression, 2019)**: mục 4.2 của bài này **trình bày lại chính xác cùng kết quả** (effective rank $r_k(\Sigma)$, self-induced regularization) mà math 32 đã phân tích đầy đủ, kèm chứng minh. Bài này cho khung nhìn tổng quan/tổng hợp, math 32 cho chi tiết chứng minh và điều kiện chính xác (Định lý 4 của BLLT); nên đọc hai bài theo cặp.
- **math content/33 (Gradient Descent Finds Global Minima of DNNs, 2018)**: mục 5.1 của bài này (chế độ tuyến tính, Định lý 5.1/5.4, dùng Jacobian/kernel $K_{m,0} = Df_n(\theta_0)Df_n(\theta_0)^T$) là **cùng dòng lý thuyết NTK** với math 33 (dùng ma trận Gram $K^{(H)}$); paper này thậm chí trích dẫn trực tiếp [DZPS19] (Du et al., ”Gradient descent provably optimizes...”) là nguồn gốc kỹ thuật cho Định lý 5.1.
- **math content/34 (A Mean Field View of the Landscape of Two-Layer Networks, 2018)**: mục 5.2 của bài này **trích dẫn trực tiếp và tóm tắt lại** PDE distributional dynamics của math 34 (Định lý 5.5 = Theorem 3 của math 34), đặt nó trong tương phản rõ với chế độ tuyến tính của mục 5.1; đây là liên hệ chặt nhất và rõ ràng nhất trong cả batch, vì bài này **thực sự tổng hợp cả hai bài kia** vào một bức tranh chung.
- **math content/22 (Deep Neural Networks as Gaussian Processes, 2017) và math content/24 (Lazy Training, 2018)**: cùng chủ đề giới hạn rộng vô hạn và chế độ ”lazy”/kernel mà mục 5.1, 6.4 của bài này thảo luận kỹ (neural tangent kernel, random features kernel như các trường hợp đặc biệt của kernel giới hạn rộng vô hạn).

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU <= 8 GB, batch 1; các hướng pruning, distillation, RL, interpretability đã có thư mục riêng).

Vì đây là bài khảo sát tổng hợp lý thuyết về over-parameterization/benign-overfitting/implicit-regularization cho các mô hình đơn giản (tuyến tính, 2-layer), nó **ít liên quan trực tiếp** đến kỹ thuật nén/hậu huấn luyện cụ thể cho LightOnOCR-3-0.8B. Tuy nhiên, khung khái niệm tổng quát của bài gợi ý một số câu hỏi suy luận hữu ích khi thiết kế thí nghiệm nén/fine-tune:

1. **Khung ”đơn giản + gai nhọn” cho việc đánh giá tác động của pruning**: nếu coi trọng số/representation của LightOnOCR trước khi pruning là $\hat{f} = \hat{f}_0 + \Delta$ (một phần ”đơn giản” chịu trách nhiệm cho năng lực OCR tổng quát, một phần ”gai nhọn” có thể là dư thừa/overfit vào các chi tiết huấn luyện cụ thể), việc pruning hiệu quả có thể hiểu như một chiến lược **nhắm trúng và loại bỏ phần** Δ **mà không đụng đến** $\hat{f}_0$. Đây là một ẩn dụ định hướng tư duy (paper không nói về pruning), không phải một công thức/kỹ thuật cụ thể có thể áp dụng trực tiếp.
2. **Bài học về implicit regularization khi fine-tune sau pruning**: mục 3 (Implicit regularization) nhấn mạnh rằng **lựa chọn thuật toán tối ưu và cách tham số hoá** (không chỉ hàm loss) quyết định nghiệm nào được chọn trong không gian các nghiệm khớp dữ liệu. Khi fine-tune lại LightOnOCR sau pruning (với ngân sách GPU hạn chế, thường dùng ít bước gradient/learning rate nhỏ), lựa chọn optimizer (SGD

vs Adam), scheme khởi tạo (tiếp tục từ trọng số đã pruning, có thể cần một chiến lược rescale giống phần khởi tạo $\theta_0^{(2)}$ ở mục 5.1) có thể ảnh hưởng đáng kể đến việc mô hình phục hồi năng lực theo hướng nào, dù paper không trực tiếp nghiên cứu bối cảnh fine-tune sau pruning.

3. **Cảnh báo về double descent khi scale ngân sách nén**: nếu quá trình nén/fine-tune tạo ra một "ngưỡng nội suy" (ví dụ số tham số còn lại sau pruning gần bằng số mẫu/batch hiệu dụng trong một giai đoạn fine-tune hẹp), Hình 1 của paper (§4) gợi ý rủi ro: hiệu năng kiểm định có thể có một **vùng tệ hơn** (test error peak) ngay tại ngưỡng đó, trước khi cải thiện trở lại nếu tiếp tục giảm (hoặc tăng) số tham số. Đây là một gợi ý để kiểm tra thực nghiệm (không phải dự đoán chắc chắn áp dụng cho VLM thật), đặc biệt khi quét một dải tỷ lệ pruning (sparsity ratio) liên tục.

9. Thí nghiệm gợi ý

1. **Tái tạo hiện tượng double descent (Hình 1) ở quy mô nhỏ**: cài đúng thiết lập random features ridge regression của paper nhưng với $d = 20, n = 80$ (thay vì $d = 100, n = 400$ để chạy trên CPU/GPU nhỏ), $f^* = \langle \beta_0, x \rangle$, quét m/n từ 0.2 đến 5, $\lambda = 10^{-3}(m/d)$. Đo: train/test error theo m/n. Kỳ vọng: thấy đỉnh test error gần $m/n = 1$, minh hoạ trực quan hiện tượng double descent.
2. **Minh hoạ implicit regularization với GD trên bài toán quá tham số hoá**: tạo bài toán hồi quy tuyến tính $d = 50 > n = 20$, chạy gradient descent từ $\theta_0 = 0$ đến khi hội tụ (loss ≈ 0), so sánh θ_{GD} với nghiệm minimum-norm tính trực tiếp bằng pseudo-inverse $X^\dagger y$. Đo: $\|\theta_{GD} - X^\dagger y\|_2$. Kỳ vọng: khoảng cách rất nhỏ (gần 0), xác nhận Eq. 14-15 bằng thực nghiệm đơn giản.
3. **So sánh hai khởi tạo trong chế độ tuyến tính (Định lý 5.4)**: cài mạng 2 lớp nhỏ ($d = 20, m \in \{200, 2000\}, n = 50$), huấn luyện với 2 cách khởi tạo như paper mô tả ($\theta_0^{(1)}$ chuẩn và $\theta_0^{(2)}$ "triệt tiêu ban đầu" bằng cách lấy nửa neuron đối xứng với nửa còn lại), đo độ lệch $\|f(\theta_t) - f_{\text{lin}}(\theta_t)\|$ trên tập test theo thời gian huấn luyện cho mỗi m. Đo: độ lệch này giảm theo m nhanh hơn với khởi tạo nào. Kỳ vọng: khởi tạo $\theta_0^{(2)}$ cần m nhỏ hơn để đạt độ lệch nhỏ, xác nhận trực giác của Định lý 5.4.
4. **Kiểm tra tối ưu của nội suy ở SNR cao/thấp (Hệ quả 6.3)**: với cùng random features model nhỏ như thí nghiệm 1, cố định $m/n = 3$ (quá tham số hoá), thử hai bài toán với SNR khác nhau (SNR cao: τ^2 nhỏ so với $\|P_1 f^*\|^2$; SNR thấp: τ^2 lớn), quét λ từ 10^{-4} đến 10, đo test error theo λ cho mỗi SNR. Đo: vị trí λ tối ưu. Kỳ vọng: ở SNR cao, λ tối ưu gần 0 (near-interpolation tối ưu); ở SNR thấp, λ tối ưu lớn hơn 0 rõ rệt, minh hoạ Hệ quả 6.3.
5. **Mô phỏng "simple+spiky" bằng pruning một mạng toy**: huấn luyện một mạng nhỏ nội suy hoàn hảo một bộ dữ liệu có nhiễu, sau đó thử "nén" (ví dụ chiếu trọng số lên không gian con k chiều tương ứng top-k eigenvector của ma trận đặc trưng, với k tăng dần) và đo test error theo k. Đo: test error có giảm rồi ổn định khi k đủ lớn hay không, so với train error. Kỳ vọng: test error giảm nhanh rồi bão hoà ở một k khá nhỏ so với số tham số đầy đủ, minh hoạ trực quan ý tưởng phân rã "đơn giản" (k nhỏ) + "gai nhọn" (phần còn lại) của §2/§8.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Nghiệm minimum-norm (implicit reg. của GD)	$\hat{\theta} = \text{argmin}\{\|\theta\|_2 : X\theta = y\} = X^\dagger y$	Eq. 14-15 (paper)
Max-margin ℓ_2 (gradient descent trên loss logistic)	$\hat{\theta} = \text{argmin}\{\|\theta\|_2 : y_i \langle \theta, x_i \rangle \geq 1\}$	Eq. 21, Theorem 3.1 (paper)
Phân rã đơn giản + gai nhọn	$\hat{f}(x) = \hat{f}_0(x) + \Delta(x)$	Eq. 157 (paper)
Nadaraya-Watson với kernel kỳ dị	$\hat{f}(x) = \sum_i y_i \omega_i(x)$, $K(u) = \|u\|_2^{-a} \mathbb{1}[\|u\|_2 \leq 1]$	Eq. 29-30 (paper)
Effective rank	$r_k(\Sigma) = \dfrac{\sum_{i>k} \lambda_i}{\lambda_{k+1}}$	mục 4.2 (paper), = math 32

Khái niệm	Công thức / định nghĩa	Nguồn
Điều kiện chế độ tuyến tính (NTK)	$\mathrm{Lip}(Df_n)\|y - f_n(\theta_0)\|_2 < \frac{1}{4}\sigma^2_{\min}(Df_n(\theta_0))$	Eq. 84, Theorem 5.1 (paper)
PDE mean-field (= Theorem 3 của math 34)	$\partial_t \rho_t = \alpha_0 \nabla(\rho_t \nabla \Psi(\cdot;\rho_t))$	Eq. 106, Theorem 5.5 (paper)
Kernel random features / neural tangent	$K_{RF,m} = \frac{1}{m}\sum_i \sigma(\langle w_i, x_1\rangle)\sigma(\langle w_i, x_2\rangle)$; $K_{NT,m} = \frac{1}{m}\sum_i \langle x_1, x_2\rangle \sigma'(\cdot)\sigma'(\cdot)$	Eq. 126-127 (paper)
Double descent, universality	$L_{RF}(\lambda) = L_{NF}(\lambda)\cdot(1 + o_n(1))$	Theorem 6.2 (paper)
Ví dụ tự tính (minh hoạ effective rank, mượn từ math 32)	$\Sigma = \mathrm{diag}(4,1,1,1)$: $r_0(\Sigma) = (1+1+1)/4 = 0.75$	tự tính

11. Kết luận ngắn

Bài khảo sát này đưa ra một khung khái niệm thống nhất để hiểu hai hiện tượng bí ẩn của deep learning: **tính dễ giải nhờ quá tham số hoá** và **tổng quát hoá nhờ implicit regularization**, bằng cách tổng hợp một loạt kết quả lý thuyết (uniform convergence cổ điển và lý do nó thất bại với benign overfitting, các ví dụ nội suy cụ thể như Nadaraya-Watson và hồi quy tuyến tính effective-rank, chế độ tuyến tính/NTK cho mạng 2 lớp, chế độ mean-field thay thế, và các mô hình tuyến tính hoá random-features/neural-tangent với hiện tượng double descent). Đóng góp quan trọng nhất về mặt tổ chức tri thức là ý tưởng **phân rã "đơn giản + gai nhọn"**, xuất hiện lặp lại trong mọi ví dụ cụ thể, và việc phân biệt rõ **self-induced regularization** (do cấu trúc dữ liệu) với **implicit regularization** (do thuật toán tối ưu). Paper tự thừa nhận phần lớn các kết quả chỉ áp dụng cho mô hình tuyến tính trong không gian đặc trưng cố định, và việc mở rộng sang chế độ phi tuyến thực sự (mean-field, mạng sâu nhiều lớp) vẫn là hướng nghiên cứu mở.

Nguồn gốc: https://arxiv.org/abs/2103.09177

Bài 32

Benign Overfitting in Linear Regression (2019)

Tác giả: Peter L. Bartlett, Philip M. Long, Gábor Lugosi, Alexander Tsigler (UC Berkeley, Google, Pompeu Fabra University).
Venue / arXiv: arXiv:1906.11300 (v3, 29/01/2020; ID gốc tháng 6/2019); paper không nêu venue hội nghị/journal phản biện trong bản text (phần đầu bài chỉ có affiliations, không có dòng ”Proceedings of...” hay ”Journal...”).
Nguồn: https://arxiv.org/abs/1906.11300
Chủ đề: đặc trưng hoá chính xác (điều kiện cần và đủ, qua 2 khái niệm ”effective rank” của ma trận hiệp phương sai) khi nào bộ ước lượng minimum-norm interpolator trong hồi quy tuyến tính overparameterized vẫn có excess risk gần tối ưu, dù khớp hoàn hảo dữ liệu train có nhiễu (”benign overfitting”).
Vị trí trong lộ trình: liên hệ gần nhất với math 31 (”Deep learning: a statistical viewpoint”, cùng batch 28-35, cùng tác giả Bartlett tham gia cả 2 bài, bài 31 khảo sát rộng hơn và trích dẫn kết quả của bài 32 này như 1 ví dụ cụ thể cho hiện tượng benign overfitting). Nên đọc bài 32 (cụ thể, có theorem chặt) trước hoặc song song với bài 31 (survey rộng hơn, đặt bài 32 trong bức tranh lớn hơn gồm cả NTK, random features).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu/công thức lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Đã đọc toàn bộ Introduction, Definitions and Notation (mục 2), Main Results (mục 3, gồm 3.1 Effective Ranks and Overparameterization), Deep neural networks (mục 4), Conclusions (mục 6), và References; mục 5 ”Proof” (dài nhất, khoảng 500 dòng text, chiếm hơn 1/3 bài) chỉ đọc LƯỚT để nắm ý tưởng chứng minh chính (phân tách bias/variance, biểu diễn bằng vector subgaussian độc lập, các bất đẳng thức concentration cho eigenvalue), KHÔNG theo dõi chi tiết từng bước đại số/hằng số trong các Lemma 7-20 và các Appendix (A, B, C, H, I, J, K được paper dẫn tới nhưng nằm trong ”supporting material” riêng, không có trong bản PDF chính đã tải).

1. Bối cảnh

Lý thuyết học thống kê cổ điển cho rằng có **đánh đổi (tradeoff)** giữa độ khớp với dữ liệu train và độ phức tạp của mô hình: khớp hoàn hảo (nội suy, interpolation) dữ liệu có nhiễu thường bị xem là dấu hiệu overfitting, dự đoán kém trên dữ liệu mới. Deep Learning phá vỡ trực giác này: mạng nơ-ron huấn luyện tới khi loss train ≈ 0 (kể cả khi có label noise) vẫn dự đoán tốt trên test (paper dẫn chứng thí nghiệm ảnh phân loại, dẫn chiếu tài liệu tham khảo [39] - không phải thí nghiệm của chính bài này).

Paper chọn bài toán **đơn giản nhất** có thể để nghiên cứu hiện tượng này: **hồi quy tuyến tính (linear regression)** với loss bình phương (quadratic loss), trong không gian tham số có số chiều đủ lớn để khớp hoàn hảo được đảm bảo (overparameterized). Vì có nhiều nghiệm nội suy (hệ thiếu xác định), paper chọn nghiệm tự nhiên nhất: $\hat{\theta}$ có **chuẩn nhỏ nhất (minimum norm)** trong số các nghiệm khớp hoàn hảo dữ liệu train.

Khái niệm cần nắm trước:

- **Ma trận hiệp phương sai (covariance operator)** $\Sigma = \mathbb{E}[xx^\top]$ của biến covariate x (có thể vô hạn chiều, trong không gian Hilbert khả ly H).
- **Excess risk** $R(\hat{\theta})$: chênh lệch kỳ vọng loss giữa ước lượng $\hat{\theta}$ và tham số tối ưu thật θ^* (tham số nhỏ nhất kỳ vọng loss bình phương).
- **Minimum-norm interpolator**: trong số các θ khớp hoàn hảo dữ liệu ($X\theta = y$), chọn θ có chuẩn Euclid/Hilbert nhỏ nhất, tương đương dùng pseudoinverse của ma trận thiết kế.

2. Ý tưởng chính

Kết quả chính (Theorem 4): excess risk của minimum-norm interpolator được chặn trên và dưới (gần khớp nhau, ”nearly matching”) bằng công thức phụ thuộc vào **2 khái niệm effective rank** $r_k(\Sigma)$ và $R_k(\Sigma)$ của ma trận hiệp phương sai. Kết luận cốt lõi: **overparameterization là điều kiện THIẾT YẾU (essential) cho benign overfitting trong thiết lập này**: số ”hướng” trong không gian tham số không quan trọng cho dự đoán (eigenvalue nhỏ của Σ) phải **vượt xa** kích thước mẫu n, để nhiễu label có thể ”ẩn” vào các hướng đó mà không ảnh hưởng dự đoán.

Paper cũng phát hiện vai trò thú vị của **số chiều hữu hạn nhưng lớn**: trong không gian **vô hạn chiều**, benign overfitting chỉ xảy ra với 1 dải hẹp tốc độ suy giảm eigenvalue (Theorem 6 phần 1: chính xác $\mu_k \propto k^{-1}\ln^{-\beta}(k+1)$ với $\beta > 1$); còn trong không gian **hữu hạn chiều lớn hơn** n, benign overfitting xảy ra với dải rộng hơn nhiều các kiểu suy giảm eigenvalue (Theorem 6 phần 2, kể cả eigenvalue gần như hằng số).

3. Phương pháp chi tiết

3.1 Định nghĩa và giả thiết (mục 2)

Bài toán hồi quy tuyến tính: $x \in H$ (Hilbert space, có thể vô hạn chiều), $y \in \mathbb{R}$, $\Sigma = \mathbb{E}[xx^\top]$, $\theta^* = \arg\min_\theta \mathbb{E}(y - x^\top\theta)^2$ (Định nghĩa 1). Giả thiết chính: x viết được $x = V\Lambda^{1/2}z$ với z có thành phần độc lập σ_x^2-subgaussian (tức x là ảnh tuyến tính của 1 vector có thành phần độc lập); nhiễu $y - x^\top\theta^*$ có phương sai chặn dưới σ^2 và σ_y^2-subgaussian; và (giả thiết 5) hầu như chắc chắn, phần dữ liệu X chiếu lên không gian vuông góc với bất kỳ eigenvector nào của Σ vẫn trải đủ chiều n (đảm bảo luôn có thể nội suy hoàn hảo).

Minimum norm estimator (Định nghĩa 2): $\hat\theta = \arg\min\{\|\theta\|^2 : X^\top X\theta = X^\top y\} = (X^\top X)^\dagger X^\top y = X^\top(XX^\top)^{-1}y$ (Eq. 1, dùng pseudoinverse).

Effective rank (Định nghĩa 3): với $\lambda_i = \mu_i(\Sigma)$ là eigenvalue giảm dần của Σ:

$$r_k(\Sigma) = \frac{\sum_{i>k}\lambda_i}{\lambda_{k+1}}, \qquad R_k(\Sigma) = \frac{\left(\sum_{i>k}\lambda_i\right)^2}{\sum_{i>k}\lambda_i^2}.$$

r_k đo ”số chiều hiệu dụng” của phần đuôi eigenvalue (từ $k+1$ trở đi) so với eigenvalue thứ $k+1$ (eigenvalue lớn nhất trong đuôi); R_k đo tương tự nhưng theo chuẩn bình phương (nhạy với việc các eigenvalue trong đuôi có đồng đều không). Paper chứng minh (Lemma 5, chỉ nêu phát biểu, chứng minh ở phụ lục không có trong bản PDF chính): $r_k(\Sigma) \geq 1$, $r_k^2(\Sigma) = r_k(\Sigma^2)R_k(\Sigma)$, và $r_k(\Sigma^2) \leq r_k(\Sigma) \leq R_k(\Sigma) \leq r_k^2(\Sigma)$.

3.2 Theorem 4: chặn excess risk chính

Đặt $k^* = \min\{k \geq 0 : r_k(\Sigma) \geq bn\}$ (b là hằng số phổ quát). Nếu $k^* \geq n/c_1$ (không đủ ”đuôi” eigenvalue lớn so với n), excess risk kỳ vọng $\geq \sigma^2/c$ (KHÔNG nhỏ, overfitting có hại). Ngược lại, với xác suất $\geq 1-\delta$:

$$R(\hat\theta) \leq c\|\theta^*\|^2\|\Sigma\|\max\left\{\sqrt{\tfrac{r_0(\Sigma)}{n}},\ \tfrac{r_0(\Sigma)}{n},\ \sqrt{\tfrac{\log(1/\delta)}{n}}\right\} + c\log(1/\delta)\sigma_y^2\left(\tfrac{k^*}{n} + \tfrac{n}{R_{k^*}(\Sigma)}\right),$$

và chặn dưới tương ứng $\mathbb{E}R(\hat\theta) \geq \frac{\sigma^2}{c}(\frac{k^*}{n} + \frac{n}{R_{k^*}(\Sigma)})$ (đây chính là Theorem 4 của paper). Diễn giải: số hạng đầu (liên quan $r_0(\Sigma)/n$) là ”**bias**” (độ lệch do nhìn θ^* qua mẫu hữu hạn), cần $r_0(\Sigma)$ **nhỏ** so với n (tổng eigenvalue không quá lớn); số hạng sau (liên quan $k^*/n + n/R_{k^*}(\Sigma)$) là ”**variance**” (ảnh hưởng của nhiễu label lên tham số ước lượng), cần $R_{k^*}(\Sigma)$ **lớn** so với n.

3.3 Overparameterization và ví dụ eigenvalue (mục 3.1)

Từ Theorem 4: để excess risk nhỏ, cần $r_0(\Sigma)$ nhỏ so với n (tổng eigenvalue nhỏ) **và** $r_{k^*}(\Sigma), R_{k^*}(\Sigma)$ lớn so với n. Kết hợp lại: số eigenvalue không-triệt-tiêu phải **lớn hơn nhiều** n, tổng của chúng phải **nhỏ** so

với n, và phải có **nhiều** eigenvalue nhỏ hơn hoặc bằng λ_{k^*} (đây chính là nội dung "overparameterization thiết yếu"). Paper mô tả cụ thể bằng khái niệm "symmetry" $s(\Sigma), S(\Sigma) \in [1/p, 1]$: với Σ hạng p hữu hạn, $r_0(\Sigma) = \text{rank}(\Sigma) \cdot s(\Sigma)$, $R_0(\Sigma) = \text{rank}(\Sigma) \cdot S(\Sigma)$, trong đó s, S gần 1 khi các eigenvalue gần đều nhau, gần $1/p$ khi 1 eigenvalue trội hẳn.

Theorem 6 (2 ví dụ cụ thể về điều kiện eigenvalue): (1) với Σ vô hạn chiều cố định, $\mu_k(\Sigma) = k^{-\alpha} \ln^{-\beta}(k+1)$: benign overfitting (định nghĩa: $r_0(\Sigma_n)/n \to 0$, $k_n^*/n \to 0$, $n/R_{k_n^*}(\Sigma_n) \to 0$ khi $n \to \infty$) xảy ra **khi và chỉ khi** $\alpha = 1$ và $\beta > 1$ - một dải **rất hẹp** tốc độ suy giảm. (2) với Σ_n hữu hạn chiều p_n, eigenvalue dạng hằng số cộng nhiễu isotropic nhỏ ($\mu_k = \gamma_k + n$ cho $k \le p_n$, $\gamma_k = \Theta(e^{-k/\tau})$): benign overfitting xảy ra khi và chỉ khi $p_n = \omega(n)$ (chiều tăng nhanh hơn n) và $n/p_n = ne^{-o(n)}$ (thành phần isotropic không quá nhỏ) - một dải **rộng hơn nhiều**.

3.4 Liên hệ với mạng nơ-ron sâu (mục 4)

Paper tự thảo luận: trong chế độ **Neural Tangent Kernel (NTK)** (mạng rất rộng, gần tuyến tính theo tham số), eigenvalue của ma trận hiệp phương sai NTK **có đuôi dài (heavy tail)**, và số chiều rất lớn nhưng hữu hạn - đúng kiểu điều kiện cần cho benign overfitting theo Theorem 4/6. NHƯNG paper tự nói rõ: **giả thiết của Theorem 4 (các thành phần z độc lập theo biến đổi tuyến tính) KHÔNG thoả mãn cho NTK thực tế**, nên kết quả của bài này **không áp dụng trực tiếp** cho trường hợp đó; paper gọi đây là "thiết lập khá phi thực tế" (somewhat unrealistic). Đây là 1 liên hệ quan trọng nhưng paper tự giới hạn, không khẳng định đã chứng minh cho mạng nơ-ron.

3.5 Ý tưởng chứng minh (mục 5, chỉ lướt)

Bước đầu: phân tách excess risk thành 2 số hạng (Lemma 7): $R(\hat{\theta}) \le 2\theta^{*\top} B\theta^* + c\sigma^2 \log(1/\delta)\text{tr}(C)$, với B, C là các ma trận xây từ X, Σ qua pseudoinverse. Số hạng $\theta^{*\top} B\theta^*$ là "bias" (méo do nhìn θ^* qua mẫu hữu hạn), số hạng $\sigma^2\text{tr}(C)$ là "variance" (ảnh hưởng nhiễu). Phần khó nhất là chặn $\text{tr}(C)$: biểu diễn qua tổng các outer-product của vector subgaussian độc lập đơn vị z_i (Lemma 8), rồi dùng các bất đẳng thức concentration (Lemma 9-12, dựa trên lý thuyết ma trận ngẫu nhiên) để chỉ ra eigenvalue nhỏ nhất/lớn nhất của các ma trận tổng A_k, A_{-i} tập trung quanh giá trị lý thuyết, cuối cùng suy ra $\text{tr}(C)$ nằm trong khoảng hằng số lần $k^*/n + n/R_{k^*}(\Sigma)$ - khớp đúng công thức Theorem 4.

4. Thí nghiệm và kết quả chính

Đây là paper **lý thuyết hoàn toàn (lý thuyết hồi quy tuyến tính), KHÔNG có thí nghiệm số/benchmark nào** do tác giả tự chạy; không có bảng số liệu, không có hình minh hoạ thực nghiệm trong bài. "Kết quả chính" là:

1. **Theorem 4**: chặn trên/dưới gần khớp cho excess risk của minimum-norm interpolator, theo $r_0(\Sigma)$ (bias) và $k^*/n + n/R_{k^*}(\Sigma)$ (variance) - đã trình bày ở mục 3.2.
2. **Theorem 6**: 2 ví dụ cụ thể (vô hạn chiều vs hữu hạn chiều lớn) minh hoạ điều kiện eigenvalue cần và đủ cho benign overfitting - mục 3.3.
3. **Nhận xét định lượng ở mục 4**: liên hệ (không chứng minh đầy đủ) giữa điều kiện của Theorem 4/6 và chế độ NTK của mạng nơ-ron rộng.

Không có "benchmark" theo nghĩa thông thường (không dataset, không mô hình huấn luyện thật); các "ví dụ" trong Theorem 6 là các cấu hình eigenvalue toán học cụ thể, không phải dữ liệu thực.

5. Phân tích / ablation

Không có ablation thực nghiệm; "phân tích điều kiện" chính là nội dung mục 3.3 của note này (2 phần Theorem 6 đóng vai trò như 2 "trường hợp đặc biệt" đối lập):

- **Vô hạn chiều, eigenvalue cố định** (Theorem 6 phần 1): dải tốc độ suy giảm cho phép benign overfitting **rất hẹp** (chính xác $\alpha = 1, \beta > 1$) - paper gọi đây là hiện tượng "bất thường" (unusual) trong trường hợp vô hạn chiều.
- **Hữu hạn chiều lớn + nhiễu isotropic nhỏ** (Theorem 6 phần 2): dải điều kiện cho phép benign overfitting **rộng hơn nhiều** (eigenvalue hằng số hoặc suy giảm chậm bất kỳ đều được, miễn số chiều tăng nhanh hơn n) - paper gọi đây là trường hợp "**tổng quát hơn**" (more generic). Với $p_n = \Omega(n)$ và $n/p_n = ne^{-o(n)}$, paper cho bound cụ thể (Theorem 6 phần 2 cuối): $R(\hat{\theta}) = O(\frac{n}{p_n} + \frac{\ln(n/(n/p_n))}{n} + \max\{\frac{1}{n}, \frac{n}{p_n}\})$.
- **Trường hợp cực đoan $p = n$, $\Sigma = I$ (ngay sau Lemma 7)**: paper tự chỉ ra bằng lý thuyết ma trận ngẫu nhiên rằng excess risk **tối thiểu là 1 hằng số** (không nhỏ dần theo n) - minh hoạ trực quan rằng KHÔNG overparameterized (không có đuôi eigenvalue lớn so với n) thì nội suy chắc chắn có hại.
- **Căng kéo giữa 2 điều kiện** (mục 3.3 cuối, paper tự nêu): cần eigenvalue suy giảm **đủ chậm** để $k/n + n/R_k$ nhỏ, nhưng cũng cần **đủ nhanh** (summable) để $r_0(\Sigma)/n$ nhỏ. Paper nêu rõ 2 cách giải quyết căng kéo này tương ứng chính là 2 phần của Theorem 6.

6. Giới hạn

Theo paper tự nêu (mục 6 Conclusions và rải rác trong bài):

- Giả thiết $\mathbb{E}[y|x]$ là hàm **tuyến tính** của x (well-specified); paper tự nói "quan trọng để hiểu liệu kết quả còn đúng trong trường hợp misspecified hay không" - **chưa giải quyết** trong bài này.
- Giả thiết covariate x là **ảnh tuyến tính của vector có thành phần độc lập** (Assumption 2); paper tự nói giả thiết này **loại trừ** một số trường hợp quan trọng, ví dụ reproducing kernel Hilbert space vô hạn chiều với kernel liên tục trên không gian hữu hạn chiều.
- Chỉ xét loss **bình phương** và ước lượng **minimum-norm interpolator** cụ thể; mở rộng sang loss khác hoặc estimator khác được paper liệt là hướng tương lai, **chưa làm** trong bài.
- Như đã nêu ở mục 3.4: kết quả **không áp dụng trực tiếp** cho mạng nơ-ron/NTK thực tế vì giả thiết về tính độc lập thành phần covariate không thoả mãn - đây là giới hạn quan trọng nhất paper tự thừa nhận khi nói về động lực ban đầu của bài (hiểu benign overfitting trong Deep Learning).
- Hướng mở rộng sang **lớp hàm tham số hoá phi tuyến (như mạng nơ-ron)** được paper gọi là "hướng tương lai thú vị nhất" (most interesting future direction) - nghĩa là bài này **chưa** trả lời câu hỏi gốc cho Deep Learning, chỉ trả lời cho trường hợp tuyến tính đơn giản nhất.

7. Liên hệ với các bài khác

Đã kiểm tra bằng `ls` trước khi trỏ:

- **"math content/31" (Deep learning: a statistical viewpoint)**: chưa thấy file lúc kiểm tra `ls "math content/"` (agent khác đang viết song song trong phiên này); đây là liên hệ gần nhất về nội dung (survey của Bartlett, Montanari, Rakhlin khảo sát rộng hơn và dùng chính kết quả bài 32 làm ví dụ cho benign overfitting), ghi nhận trung thực ở đây và sẽ được bài 31 trỏ ngược lại khi viết xong.
- **"llm content/03_2020_scaling_laws_kaplan.md"** (đã xác nhận tồn tại): liên hệ qua chủ đề overparameterization/quy mô hoá mô hình; bài 32 cho góc nhìn lý thuyết chặt (hồi quy tuyến tính) về lý do "mô hình nhiều tham số hơn dữ liệu" không nhất thiết overfit có hại, bổ sung cho góc nhìn thực nghiệm quy mô lớn của Kaplan et al.
- **"pruning content/11_2018_rethinking_value_of_pruning.md"** (đã xác nhận tồn tại): bài pruning này thảo luận các mạng overparameterized và cấu trúc dư thừa tham số; liên hệ gián tiếp với ý tưởng "nhiều hướng tham số không quan trọng" (eigenvalue nhỏ) của bài 32 - những hướng dư thừa đó, theo lý thuyết benign overfitting, chính là nơi "ẩn" nhiễu mà không hại dự đoán, cũng là ứng viên tự nhiên để cắt giảm khi pruning.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, batch 1, GPU $\leq$ 8GB; các hướng pruning/distillation/RL/interpretability đã có thư mục riêng).

(Suy luận) Liên hệ trực tiếp **khá xa**: lý thuyết ở đây chỉ chứng minh chặt cho **hồi quy tuyến tính**, không phải Transformer/SSM sâu như LightOnOCR, và paper tự nói rõ giả thiết không áp dụng cho NTK/mạng nơ-ron thực (mục 3.4/6). Tuy vậy, có 2 bài học định hướng (loại suy, không phải kết luận toán học) hữu ích khi nghĩ về nén mô hình: (1) ý tưởng "cần nhiều hướng tham số không quan trọng (eigenvalue nhỏ) để benign overfitting xảy ra" gợi ý rằng khi **nén** LightOnOCR (giảm chiều tham số hiệu dụng), nếu cắt đúng vào các "hướng ít quan trọng" đó thì model nén có thể vẫn generalize tốt trên dữ liệu fine-tune nhỏ (tương tự tinh thần minimum-norm interpolator), nhưng nếu cắt gần hết dư thừa tham số, model sẽ mất chế độ overparameterized và excess risk có thể tăng mạnh (đúng như trường hợp cực đoan $p = n$ ở mục 5); (2) bài học về "effective rank" như 1 công cụ đo độ dư thừa tham số có thể dùng (sau khi điều chỉnh phù hợp, không trực tiếp từ công thức paper) để chẩn đoán layer nào của LightOnOCR còn "dư thừa" (eigenvalue hiệp phương sai activation nhỏ, đồng đều) trước khi quyết định prune - đây là loại suy xa, cần kiểm chứng thực nghiệm riêng, không phải điều paper khẳng định.

9. Thí nghiệm gợi ý

1. **Tái tạo Theorem 4 trên dữ liệu tổng hợp nhỏ**: sinh $x \sim \mathcal{N}(0, \Sigma)$ với $\Sigma = \text{diag}(\lambda_1, \ldots, \lambda_p)$ (ví dụ $p = 200$, eigenvalue suy giảm theo $\lambda_i = i^{-1}$, giống Theorem 6 phần 1 với $\alpha = 1$), $y = x^\top \theta^* + \epsilon$, n nhỏ (ví dụ $n = 50$). Tính $\hat{\theta}$ bằng pseudoinverse (`numpy.linalg.pinv`), đo excess risk thực nghiệm trên tập test lớn, so sánh với chặn Theorem 4 (tính $r_0(\Sigma), k^*, R_{k^*}(\Sigma)$ từ Σ đã biết). Chạy được trên CPU trong vài giây.
2. **Quét tốc độ suy giảm eigenvalue** α: lặp thí nghiệm 1 với $\lambda_i = i^{-\alpha}$, $\alpha \in \{0.5, 1, 1.5, 2\}$ (giữ p, n cố định, p hữu hạn lớn hơn n nhiều). Đo excess risk theo α. Kỳ vọng: vùng α gần 1 (và p đủ lớn hữu hạn) cho excess risk nhỏ nhất, minh hoạ "dải rộng hơn trong không gian hữu hạn chiều" (Theorem 6 phần 2) so với yêu cầu chặt $\alpha = 1$ đúng của trường hợp vô hạn chiều.
3. **Trường hợp cực đoan** $p = n$: lặp thí nghiệm 1 với $p = n$ (không overparameterized), $\Sigma = I$. Đo excess risk thực nghiệm nhiều lần lặp (nhiều seed). Kỳ vọng: excess risk **không giảm** khi tăng n (giữ $p = n$), xác nhận đúng nhận xét ở mục 5 (ngay sau Lemma 7) rằng không overparameterized thì nội suy có hại.
4. **Đo effective rank của activation 1 mạng nơ-ron nhỏ**: huấn luyện 1 MLP nhỏ (2-3 lớp) trên MNIST/toy dataset, trích activation của 1 lớp ẩn trên tập validation, tính ma trận hiệp phương sai thực nghiệm của activation đó, tính r_0, R_0 theo công thức Định nghĩa 3. Đo xem r_0, R_0 lớn hay nhỏ so với n (số mẫu). Đây KHÔNG kiểm chứng Theorem 4 (vì giả thiết không thoả mãn cho mạng nơ-ron, đúng như mục 3.4), nhưng minh hoạ cách tính effective rank thực tế và kết nối (có kiểm soát, không suy diễn quá đà) với ý tưởng ở mục 8.
5. **So sánh minimum-norm vs ridge regression**: lặp thí nghiệm 1 nhưng thêm 1 ước lượng ridge regression (có regularization $\lambda > 0$) để so sánh excess risk với minimum-norm interpolator ($\lambda = 0$). Đo excess risk theo λ quét từ 0 tới giá trị lớn. Kỳ vọng: với Σ trong "vùng benign" (theo Theorem 6), minimum-norm ($\lambda = 0$) có excess risk gần với ridge tối ưu, minh hoạ trực quan hiện tượng "benign overfitting" (nội suy không cần regularization vẫn tốt).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ý nghĩa
Minimum-norm estimator	$\hat{\theta} = X^\top (XX^\top)^{-1} y$ (Eq. 1)	Nghiệm nội suy có chuẩn nhỏ nhất

Khái niệm	Công thức	Ý nghĩa
Effective rank r_k	$r_k(\Sigma) = \dfrac{\sum_{i>k}\lambda_i}{\lambda_{k+1}}$	Số chiều hiệu dụng của đuôi eigenvalue
Effective rank R_k	$R_k(\Sigma) = \dfrac{(\sum_{i>k}\lambda_i)^2}{\sum_{i>k}\lambda_i^2}$	Tương tự r_k nhưng theo chuẩn bình phương
Ngưỡng k^*	$k^* = \min\{k \geq 0 : r_k(\Sigma) \geq bn\}$	Điểm cắt giữa eigenvalue ”lớn” và ”nhỏ”
Theorem 4 (chặn trên)	$R(\hat{\theta}) \lesssim \|\theta^*\|^2\|\Sigma\|\sqrt{r_0(\Sigma)/n} + \sigma_y^2\left(\frac{k^*}{n} + \frac{n}{R_{k^*}(\Sigma)}\right)$	Bias (số hạng đầu) + variance (số hạng sau)
Benign (định nghĩa, mục 3.3)	$\lim_n r_0(\Sigma_n)/n = \lim_n k_n^*/n = \lim_n n/R_{k_n^*}(\Sigma_n) = 0$	Điều kiện dãy Σ_n cho excess risk $\to 0$
Theorem 6(1)	$\mu_k = k^{-\alpha}\ln^{-\beta}(k+1)$ benign $\iff \alpha = 1, \beta > 1$	Dải hẹp cho trường hợp vô hạn chiều

Ví dụ tính tay (tự tính): cho $\Sigma = \mathrm{diag}(4,1,1,1)$ (hạng $p = 4$, eigenvalue giảm dần $\lambda_1 = 4, \lambda_2 = \lambda_3 = \lambda_4 = 1$). Tính $r_0(\Sigma)$ và $R_0(\Sigma)$ theo Định nghĩa 3 với $k = 0$:

$$r_0(\Sigma) = \frac{\sum_{i>0}\lambda_i}{\lambda_1} = \frac{4+1+1+1}{4} = \frac{7}{4} = 1.75.$$

$$R_0(\Sigma) = \frac{\left(\sum_{i>0}\lambda_i\right)^2}{\sum_{i>0}\lambda_i^2} = \frac{7^2}{4^2+1^2+1^2+1^2} = \frac{49}{19} \approx 2.579.$$

So sánh với $\Sigma' = I_4$ (4 eigenvalue bằng nhau $= 1$): $r_0(\Sigma') = \frac{4}{1} = 4 = p$, $R_0(\Sigma') = \frac{16}{4} = 4 = p$ (khớp đúng nhận xét của paper: khi mọi eigenvalue bằng nhau, $r_0 = R_0 = \mathrm{rank}(\Sigma)$). Ví dụ Σ ban đầu có 1 eigenvalue trội ($\lambda_1 = 4$) làm r_0, R_0 **nhỏ hơn hẳn** $p = 4$, minh hoạ trực quan: hiệp phương sai ”mất cân bằng” (1 hướng chiếm phần lớn phương sai) có effective rank thấp hơn số chiều thật, tức ít ”chỗ ẩn nhiễu” hơn theo tinh thần Theorem 4.

11. Kết luận ngắn

Bằng cách phân tích chặt bài toán hồi quy tuyến tính overparameterized, paper đưa ra đặc trưng hoá (điều kiện cần và đủ, qua 2 khái niệm effective rank r_k, R_k của ma trận hiệp phương sai) cho khi nào minimum-norm interpolator vẫn có excess risk gần tối ưu dù khớp hoàn hảo dữ liệu nhiễu. Kết luận cốt lõi: overparameterization (nhiều hướng tham số ”không quan trọng”, eigenvalue nhỏ, vượt xa kích thước mẫu) là điều kiện thiết yếu, và không gian hữu hạn chiều lớn cho phép benign overfitting với dải điều kiện eigenvalue rộng hơn nhiều so với không gian vô hạn chiều. Đây là 1 trong các kết quả lý thuyết nền tảng và chặt chẽ nhất về benign overfitting, nhưng paper tự nhận giả thiết không áp dụng trực tiếp cho mạng nơ-ron/NTK thực tế, nên việc nối kết đầy đủ với hiện tượng gốc trong Deep Learning còn là câu hỏi mở.

Nguồn gốc: https://arxiv.org/abs/1906.11300

Bài 33

Gradient Descent Finds Global Minima of Deep Neural Networks (2018)

Tác giả: Simon S. Du, Jason D. Lee, Haochuan Li, Liwei Wang, Xiyu Zhai
Venue / arXiv: arXiv:1811.03804 (v4, 28/05/2019); ICML 2019 (Proceedings of the 36th International Conference on Machine Learning, PMLR 97, 2019, theo chính văn bản paper tự ghi ở trang đầu)
Nguồn: https://arxiv.org/abs/1811.03804
Chủ đề: Chứng minh lý thuyết rằng gradient descent khởi tạo ngẫu nhiên đạt loss huấn luyện bằng 0 với tốc độ tuyến tính trên mạng nơ-ron sâu quá tham số hoá (over-parameterized), cho ba kiến trúc: fully-connected sâu, ResNet, và convolutional ResNet.
Vị trí trong lộ trình: Đọc sau math 22 (Deep NN như Gaussian Process) và math 24 (Lazy training) vì cùng dòng lý thuyết NTK/Gram-matrix cho mạng quá tham số hoá; là bài thứ 7 trong batch math 28-35, liền trước math 34 (Mean Field View, một hướng lý thuyết khác cho cùng câu hỏi).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper mở đầu bằng việc nêu **hai hiện tượng bí ẩn (mysterious phenomena)** trong huấn luyện mạng nơ-ron sâu:

1. Các phương pháp bậc một khởi tạo ngẫu nhiên (random initialization) như gradient descent (GD) đạt loss huấn luyện bằng 0, **thậm chí khi nhãn là ngẫu nhiên** (dẫn chứng từ Zhang et al. 2016/2017, ”Understanding deep learning requires rethinking generalization”), trong khi bài toán tối ưu là **không lồi (non-convex)**.
2. ”Mạng sâu hơn thì khó huấn luyện hơn” - vấn đề mà kiến trúc **ResNet** (residual network, He et al. 2016) giải quyết thực nghiệm bằng kết nối tắt (skip connection), cho phép huấn luyện mạng có số lớp lớn hơn một bậc độ lớn, nhưng (tính đến thời điểm paper này công bố) **chưa có lý giải lý thuyết đầy đủ** cho lợi ích của residual connection với hàm kích hoạt phi tuyến.

Thuật ngữ cần làm rõ: **over-parameterization** (quá tham số hoá) nghĩa là mạng có nhiều tham số hơn số điểm dữ liệu huấn luyện cần fit; **Gram matrix** (ma trận Gram) trong ngữ cảnh này là ma trận $n \times n$ (n = số điểm dữ liệu) đo ”độ tương quan” giữa gradient của các điểm dữ liệu theo tham số mô hình; **linear convergence rate** (tốc độ hội tụ tuyến tính) nghĩa là loss giảm theo cấp số nhân (hình học) qua các bước lặp, dạng $L(k) \leq (1-c)^k L(0)$.

Paper đặt bài toán cụ thể: có n điểm dữ liệu (x_i, y_i), mạng có H lớp, độ rộng m mỗi lớp, hàm kích hoạt σ thoả **Condition 3.1** (Lipschitz và smooth, ví dụ hàm softplus $\sigma(z) = \log(1+e^z)$) và **Condition 3.2** (giải tích, không là đa thức). Mục tiêu: chứng minh với m đủ lớn, GD khởi tạo ngẫu nhiên hội tụ về loss $= 0$ với tốc độ tuyến tính, cho 3 kiến trúc: (a) mạng fully-connected sâu, (b) ResNet, (c) convolutional ResNet.

2. Ý tưởng chính

Ý tưởng cốt lõi (kế thừa từ Du et al. 2018b cho mạng 2 lớp, mở rộng cho mạng sâu) là theo dõi động lực học của **sai số dự đoán** $y - u(k)$, với $u_i(k) = f(\theta(k), x_i)$ là dự đoán ở bước k. Paper chứng minh động lực này tuân theo:

$$y - u(k+1) = (I - \eta G(k))(y - u(k)),$$

với $G(k)$ là ma trận Gram tại bước k (xây dựng từ gradient của u theo trọng số lớp cuối). Nếu m đủ lớn, $G(k)$ **gần như không đổi** trong suốt quá trình huấn luyện và gần một ma trận cố định $K^{(H)}$ (chỉ phụ thuộc dữ liệu đầu vào và kiến trúc, không phụ thuộc θ). Khi đó động lực xấp xỉ tuyến tính $y-u(k+1) \approx (I-\eta K^{(H)})(y-u(k))$, và theo lý thuyết power method chuẩn, nếu $K^{(H)}$ xác định dương ($\lambda_{\min}(K^{(H)}) > 0$), sai số giảm theo cấp số nhân với tốc độ quyết định bởi $\lambda_{\min}(K^{(H)})$.

Điểm mới của paper (so với Du et al. 2018b chỉ làm cho mạng 2 lớp) là chứng minh **tính ổn định của ma trận Gram xuyên suốt H lớp**, đòi hỏi phân tích cách "nhiễu" (perturbation) từ khởi tạo ngẫu nhiên và từ quá trình huấn luyện lan truyền qua nhiều lớp phi tuyến. Đây là phần khó kỹ thuật nhất, dùng công cụ từ **giải tích Malliavin** (Malliavin calculus, một nhánh giải tích ngẫu nhiên).

Kết quả chính, phát biểu trực giác: với mạng **fully-connected** sâu, độ rộng cần thiết m tăng **theo cấp số mũ với số lớp** H ($2^{O(H)}$); với **ResNet**, nhờ hệ số scaling residual $c_{\text{res}}/(\sqrt{H} \cdot \sqrt{m})$, độ rộng cần thiết chỉ tăng **đa thức theo** H ($\text{poly}(n, H)$). Đây chính là lời giải lý thuyết cho hiện tượng bí ẩn thứ hai: residual connection giúp ổn định sự lan truyền nhiễu qua các lớp, nên không cần độ rộng tăng theo cấp số mũ với độ sâu.

3. Phương pháp chi tiết

3.1. Ba kiến trúc được xét (mục 3.3 của paper)

Fully-connected: với $x^{(0)} = x \in \mathbb{R}^d$,

$$x^{(h)} = \sigma\left(\sqrt{\frac{c_\sigma}{m}}\, W^{(h)} x^{(h-1)}\right), \quad f(x,\theta) = a^\top x^{(H)}, \tag{2, paper}$$

với $c_\sigma = \left(\mathbb{E}_{x\sim N(0,1)}[\sigma(x)^2]\right)^{-1}$ là hệ số chuẩn hoá.

ResNet (paper tự định nghĩa đây là kiến trúc có skip-connection ở **mọi lớp**, khác ResNet chuẩn vốn skip mỗi 2 lớp, nhưng paper nói rõ phân tích tổng quát hoá được cho skip mỗi $k \geq 2$ lớp):

$$x^{(h)} = x^{(h-1)} + \frac{c_{\text{res}}}{\sqrt{H}\sqrt{m}}\, \sigma\left(W^{(h)} x^{(h-1)}\right), \quad 2 \leq h \leq H, \tag{3, paper}$$

với $0 < c_{\text{res}} < 1$ là hằng số nhỏ. **Hệ số scaling $1/\sqrt{H}$ chính là chìa khoá kỹ thuật** giúp độ rộng cần thiết không nổ theo cấp số mũ với H.

Convolutional ResNet: tương tự ResNet nhưng $W^{(h)}$ nhân với một phép trích patch $\phi_h(\cdot)$ (ví dụ 3×3 patch với $q = 3$), áp dụng cho ảnh có p pixel.

Tất cả tham số khởi tạo $W_{ij}^{(h)} \sim N(0,1)$ và $a \sim N(0,1)$ độc lập; **huấn luyện toàn bộ các lớp** (không đóng băng lớp nào) bằng gradient descent với step size cố định η.

3.2. Ma trận Gram $K^{(H)}$ (Định nghĩa 5.1, 6.1, 7.1)

Với mạng fully-connected, $K^{(H)}$ định nghĩa quy nạp: $K_{ij}^{(0)} = \langle x_i, x_j \rangle$, rồi với $h = 1, \ldots, H-1$:

$$A_{ij}^{(h)} = \begin{pmatrix} K_{ii}^{(h-1)} & K_{ij}^{(h-1)} \\ K_{ji}^{(h-1)} & K_{jj}^{(h-1)} \end{pmatrix}, \qquad K_{ij}^{(h)} = c_\sigma\, \mathbb{E}_{(u,v)^\top \sim N(0, A_{ij}^{(h)})}[\sigma(u)\sigma(v)], \tag{7, paper}$$

và lớp cuối dùng đạo hàm σ' thay σ. Đây chính là **ma trận hiệp phương sai của Gaussian Process tương đương** với mạng ở độ rộng vô hạn (kernel NNGP, Neural Network Gaussian Process), paper trích dẫn rõ liên

hệ này với các công trình Gaussian Process cho mạng vô hạn rộng (Lee et al. 2017, Matthews et al. 2018). Với ResNet, $K^{(H)}$ có thêm các số hạng $b_i^{(h)}$ (trung bình kích hoạt) do cấu trúc skip-connection.

Ví dụ tự tính nhỏ để minh hoạ bước đệ quy: giả sử $n = 2$ điểm dữ liệu chuẩn hoá $\|x_i\|_2 = 1$ với $\langle x_1, x_2\rangle = 0.5$ (góc giữa hai vector khoảng $60°$), $c_\sigma = 1$ và σ là hàm tuyến tính giả định $\sigma(z) = z$ chỉ để minh hoạ cơ chế (paper dùng σ phi tuyến thật, đây chỉ là ví dụ tính tay đơn giản hoá). Khi đó $K^{(0)} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, và với σ tuyến tính, $\mathbb{E}[\sigma(u)\sigma(v)] = \mathrm{Cov}(u, v) = K_{ij}^{(0)}$, nên $K^{(1)} = K^{(0)}$ không đổi qua lớp (tự tính, chỉ minh hoạ trực giác đệ quy, không phải kết quả của paper vì paper yêu cầu σ không phải đa thức).

3.3. Kỹ thuật chứng minh (mục 4, ”Technique Overview”)

Chứng minh gồm hai bước chính (paper trình bày tường minh ở mục 4):

Bước 1 (phân tích khởi tạo): chứng minh $G^{(H)}(0)$ (ma trận Gram tại $k = 0$) gần $K^{(H)}$. Với mạng fully-connected, nhiễu lan truyền theo công thức

$$\|G^{(H)}(0) - K^{(H)}\|_2 \lesssim 2^{O(H)}\epsilon_1, \qquad \text{(4, paper)}$$

tức **khuếch đại theo cấp số mũ với** H. Với ResNet, nhờ hệ số $1/\sqrt{H}$:

$$\|G^{(H)}(0) - K^{(H)}\|_2 \lesssim \left(1 + \frac{1}{H}\right)^H \epsilon_1 = O(\epsilon_1), \qquad \text{(5, paper)}$$

tức **bị chặn đều theo** H (vì $(1 + 1/H)^H \to e$ khi $H \to \infty$). Đây chính là phần giải thích định lượng cho lý do ResNet ”ổn định” hơn.

Bước 2 (phân tích nhiễu trong huấn luyện): chứng minh $G^{(H)}(k)$ gần $G^{(H)}(0)$ với mọi k, bằng cách chặn $\frac{1}{\sqrt{m}}\|W^{(h)}(k) - W^{(h)}(0)\|_F$ (độ lệch trung bình Frobenius-norm của trọng số so với khởi tạo) nhỏ với mọi k. Cơ chế trực giác tương tự Bước 1.

Proof Sketch (Appendix A, đọc lướt): chứng minh chính dùng quy nạp trên điều kiện $\|y - u(k)\|_2^2 \le (1 - \eta\lambda_0/2)^k\|y - u(0)\|_2^2$ (với $\lambda_0 = \lambda_{\min}(K^{(H)})$), tách $u(k+1) - u(k)$ bằng khai triển Taylor thành số hạng chính $I_1(k)$ (tỷ lệ η, làm giảm loss) và số hạng nhiễu $I_2(k)$ (tỷ lệ η^2, nhỏ). Đây là kỹ thuật chuẩn trong các chứng minh hội tụ tuyến tính kiểu NTK (Neural Tangent Kernel).

4. Thí nghiệm và kết quả chính

Paper này không có thí nghiệm thực nghiệm (benchmark) nào - đây là một bài chứng minh toán học thuần tuý, không có mục ”Experiments” hay bất kỳ bảng số liệu đo trên dữ liệu thực/tổng hợp. Kết quả chính là 3 định lý hội tụ:

Định lý 5.1 (Fully-connected): với $\|x_i\|_2 = 1$, $|y_i| = O(1)$, nếu

$$m = \Omega\left(2^{O(H)} \max\left\{\frac{n}{\lambda_{\min}(K^{(H)})^4\delta}, \frac{n^2\log(Hn/\delta)}{\lambda_{\min}(K^{(H)})^2}, n^4\right\}\right)$$

và step size $\eta = O\left(\dfrac{\lambda_{\min}(K^{(H)})}{n^2 2^{O(H)}}\right)$, thì với xác suất $\ge 1 - \delta$:

$$L(\theta(k)) \le \left(1 - \frac{\eta\lambda_{\min}(K^{(H)})}{2}\right)^k L(\theta(0)). \qquad \text{(Theorem 5.1, paper)}$$

Định lý 6.1 (ResNet): cùng dạng kết luận, nhưng độ rộng cần thiết

$$m = \Omega\left(\max\left\{\frac{n^2}{\lambda_{\min}(K^{(H)})^4}, \frac{n^4}{H^6\lambda_{\min}(K^{(H)})^2}, \frac{n^2\log(Hn/\delta)}{\delta\lambda_{\min}(K^{(H)})^2}\right\}\right), \quad \eta = O\left(\frac{\lambda_{\min}(K^{(H)})H^2}{n^2}\right).$$

(Theorem 6.1, paper)

Khác biệt quan trọng nhất giữa hai định lý: Định lý 5.1 có số hạng $2^{O(H)}$ (mũ theo số lớp), Định lý 6.1 **hoàn toàn đa thức** theo n và H (không có số hạng mũ). Đây là bằng chứng lý thuyết định lượng cho lợi ích của kiến trúc ResNet.

Định lý 7.1 (Convolutional ResNet): tương tự Định lý 6.1 nhưng thêm số hạng poly(p) (p = số patch/pixel) trong yêu cầu độ rộng và nghịch đảo của nó trong step size; về bản chất mở rộng kỹ thuật ResNet sang kiến trúc tích chập.

So sánh với công trình đồng thời (concurrent work) Allen-Zhu et al. (2018c): paper này tự so sánh (phần Related Work) rằng với ResNet, yêu cầu độ rộng của Allen-Zhu et al. là $m \gtrsim n^{30}H^{30}\log^2(1/\epsilon)$ và độ phức tạp lặp $T \gtrsim n^6H^2\log(1/\epsilon)$, trong khi của paper này là $m \gtrsim n^4H^2$ và $T \gtrsim n^2\log(1/\epsilon)$ - **không phụ thuộc** ϵ vì định lý của paper này đảm bảo hội tụ về đúng global minimum (loss $= 0$), không chỉ đạt sai số ϵ. Paper tự nhận hai công trình "không thể so sánh công bằng hoàn toàn" (not directly comparable) vì giả định dữ liệu khác nhau.

5. Phân tích / ablation

Vì đây là bài lý thuyết không có mã nguồn/thí nghiệm, "phân tích" ở đây là phân tích điều kiện và vai trò của từng thành phần trong định lý, theo chính paper trình bày:

Vai trò của từng số hạng trong yêu cầu độ rộng m (paper tự giải thích, mục ngay sau Theorem 5.1 và 6.1):

- Số hạng đầu dùng để chứng minh ma trận Gram **ổn định trong quá trình huấn luyện**.
- Số hạng thứ hai dùng để đảm bảo đầu ra mỗi lớp được **chuẩn hoá xấp xỉ đúng tại khởi tạo**.
- Số hạng thứ ba dùng để chặn **nhiễu của ma trận Gram tại khởi tạo**.

Điều kiện cần của hàm kích hoạt: Condition 3.2 (giải tích, không là đa thức) được paper giải thích là cần để đảm bảo $K^{(H)}$ **xác định dương chặt** (Proposition F.1, F.2, chỉ lướt qua); paper không cho ví dụ phản chứng cụ thể nếu vi phạm điều kiện này trong phần chính, chỉ nói "cần điều kiện này để đảm bảo positive-definiteness".

So sánh định lượng ResNet vs fully-connected (paper tự nêu, mục 6): "the primary reason that we do not have any exponential factor here is that the skip connection block makes the overall architecture more stable in both the initialization phase and the training phase" - đây là câu paper tự kết luận, không phải diễn giải của ghi chú.

Hướng cải thiện mở (paper tự nêu ở mục 8, Conclusion): paper liệt kê 4 hướng nghiên cứu tiếp theo mà bản thân nó chưa giải quyết:

1. Paper chỉ chứng minh loss huấn luyện (training loss), **không đề cập loss kiểm định (test loss / generalization)**; paper tự nói các công trình hiện có (gồm công trình của chính nhóm tác giả) chỉ cho thấy GD hoạt động trong cùng điều kiện với kernel method / random feature method.
2. Độ rộng m là đa thức nhưng **vẫn rất lớn** so với mạng thực tế (paper tự nhận: "mạng thực tế có số tham số là hằng số nhân với n, không phải độ rộng").
3. Phân tích hiện tại chỉ cho **gradient descent toàn batch**, chưa cho stochastic gradient descent (SGD), tuy paper tin rằng có thể mở rộng được.
4. Tốc độ hội tụ có thể cải thiện nếu tính đến đóng góp của ma trận Gram ở **mọi lớp** (hiện paper chỉ dùng $G^{(H)}$, ma trận Gram của lớp cuối, để đơn giản hoá, đánh đổi với tốc độ hội tụ kém hơn mức tối ưu khả

dĩ).

6. Giới hạn

Các giới hạn dưới đây đều được paper **tự nêu rõ** (không phải suy luận của ghi chú, trừ khi ghi chú thích):

- **Không chứng minh generalization**: như đã nói ở §5, định lý chỉ bảo đảm loss huấn luyện $\to 0$, không nói gì về hiệu năng trên dữ liệu chưa thấy (test set). Đây là giới hạn quan trọng nhất về phạm vi của kết quả.
- **Độ rộng m cần thiết rất lớn trong thực tế**: dù là đa thức (không mũ) với ResNet, số mũ cụ thể ($n^4, n^4/H^6, \ldots$) vẫn lớn hơn rất nhiều so với độ rộng dùng trong thực hành (paper tự nhận ở mục Conclusion điểm 2).
- **Chỉ xét gradient descent toàn batch (full-batch GD), không phải SGD**: thực tế hầu hết huấn luyện sâu dùng SGD/minibatch; paper chỉ suy đoán (không chứng minh) rằng kết quả mở rộng được.
- **Kiến trúc ResNet trong paper không hoàn toàn giống ResNet chuẩn**: paper tự chú thích (footnote 4 trong văn bản) rằng kiến trúc họ định nghĩa có skip-connection ở **mọi lớp**, khác ResNet gốc (He et al. 2016) có skip-connection mỗi 2 lớp; paper nói việc tổng quát hoá cho skip mỗi $k \geq 2$ lớp "không khó" nhưng **không chứng minh chi tiết** trường hợp đó trong bài.
- **Giả định dữ liệu "không suy biến" (non-degenerate)**: điều kiện $\lambda_{\min}(K^{(H)}) > 0$ cần dữ liệu đủ "khác biệt" (không có hai điểm trùng hướng); paper không thảo luận điều gì xảy ra khi dữ liệu gần suy biến (ví dụ ảnh gần giống nhau, một tình huống phổ biến với dữ liệu OCR có nhiều trang giống mẫu).
- **Không có thí nghiệm nào xác nhận lý thuyết trên mạng thực tế**: một giới hạn phương pháp luận paper không nêu trực tiếp nhưng là sự thật khách quan về phạm vi bài: không có một con số thực nghiệm nào (không mô phỏng, không dữ liệu thật) để kiểm tra liệu các hằng số/độ rộng dự đoán bởi lý thuyết có khớp với quan sát thực nghiệm hay không.
- **So sánh với Allen-Zhu et al. (2018c) không hoàn toàn công bằng**: paper tự nói rõ "a fair comparison is not possible due to the difference in setting and data assumptions", nên các con số so sánh độ rộng/độ phức tạp lặp ở §4 chỉ mang tính minh hoạ bậc độ lớn, không phải so sánh trực tiếp hiệu năng.

7. Liên hệ với các bài khác

- **math content/22 (Deep Neural Networks as Gaussian Processes, 2017)**: ma trận Gram $K^{(H)}$ định nghĩa đệ quy ở Định nghĩa 5.1/6.1/7.1 của bài này chính là **kernel NNGP (Neural Network Gaussian Process)** mà math 22 nghiên cứu ở độ rộng vô hạn; paper này tự trích dẫn trực tiếp dòng nghiên cứu Gaussian Process cho mạng vô hạn rộng (Lee et al. 2017, Matthews et al. 2018) trong phần Related Work.
- **math content/24 (Lazy Training, 2018)**: cùng hiện tượng cốt lõi "trọng số gần như không đổi so với khởi tạo trong suốt huấn luyện" (weights stay close to initialization) mà bài này dựa vào (kế thừa từ Li & Liang 2018, Du et al. 2018b) chính là hiện tượng "lazy training"/NTK mà math 24 phân tích tổng quát hơn; hai bài bổ sung cho nhau: math 24 giải thích *vì sao* hiện tượng này xảy ra ở mức tổng quát, bài này chứng minh *hệ quả cụ thể* (hội tụ global minimum) cho 3 kiến trúc cụ thể.
- **math content/32 (Benign Overfitting in Linear Regression, 2019)**: cùng dòng lý thuyết liên quan đến NTK (chính ghi chú math 32 đã có mục thảo luận NTK ở §4, với giới hạn tương tự về việc các giả định không khớp hoàn toàn với NTK thực). Cả hai bài đều thuộc nhóm lý thuyết "over-parameterization giúp tối ưu/generalization tốt hơn dự kiến trực giác cổ điển".
- **math content/34 (A Mean Field View of the Landscape of Two-Layer Neural Networks, 2018, cùng batch)**: một hướng lý thuyết khác (mean-field/optimal transport) cho cùng câu hỏi "vì sao GD tìm được nghiệm tốt cho mạng quá tham số hoá"; bài 33 này dùng kỹ thuật Gram-matrix/NTK-style, trong khi bài 34 dùng công cụ xác suất đo-đạc (measure-valued dynamics); paper 33 tự liệt kê các công trình optimal-transport (Mei et al. 2018, Chizat & Bach 2018a, Rotskoff & Vanden-Eijnden 2018) như

một hướng thay thế trong Related Work, nhưng tự nhận các công trình đó ”giới hạn ở mạng 2 lớp và có thể cần over-parameterization theo cấp số mũ”, một sự đối lập thú vị với chính bài 34.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU <= 8 GB, batch 1; các hướng pruning, distillation, RL, interpretability đã có thư mục riêng).

Bài này **ít liên quan trực tiếp** đến việc nén/hậu huấn luyện LightOnOCR-3-0.8B theo nghĩa thực hành cụ thể: các định lý chỉ áp dụng cho mạng **khởi tạo ngẫu nhiên huấn luyện từ đầu** bằng full-batch GD với độ rộng rất lớn (polynomial/exponential theo n, H), hoàn toàn khác bối cảnh thực tế của dự án là **fine-tune/pruning một mô hình đã huấn luyện sẵn** (LightOnOCR-3-0.8B) với ngân sách GPU rất hạn chế (<=8GB, batch 1). Những con số độ rộng trong các định lý (ví dụ $m = \Omega(n^4 H^2/\lambda_{\min}^4)$) hoàn toàn không thực tế để áp dụng định lượng cho một mô hình cụ thể.

Tuy vậy, có một liên hệ gián tiếp mang tính trực giác/suy luận đáng ghi nhận: nguyên lý cốt lõi của bài (”skip-connection với scaling $1/\sqrt{H}$ giúp ổn định sự lan truyền nhiễu/gradient qua nhiều lớp, tránh khuếch đại theo cấp số mũ”) là lý do nền tảng cho việc **các kiến trúc hybrid có attention/SSM xen residual** (như GatedDeltaNet + full attention trong LightOnOCR) thường dùng kết nối residual ở mọi block; nếu trong quá trình pruning/distillation cần **loại bỏ hoặc thay đổi một số residual connection** (ví dụ pruning một khối attention/SSM nguyên block), bài học định lượng này gợi ý rủi ro: việc phá vỡ cấu trúc skip-connection có thể làm mất tính ổn định huấn luyện (gradient có thể bùng nổ/biến mất) khi fine-tune lại mô hình đã pruning, đặc biệt nếu mô hình sau pruning trở nên ”sâu hơn tương đối” (depth-to-residual ratio tăng). Đây là một lưu ý kiến trúc mang tính suy luận gián tiếp, không phải khuyến nghị kỹ thuật cụ thể có thể trích dẫn trực tiếp từ paper.

9. Thí nghiệm gợi ý

1. **Minh hoạ ổn định Gram matrix theo độ sâu**: xây một mạng fully-connected toy (ví dụ $H = 2, 4, 8, 16$ lớp, độ rộng $m = 256$, dữ liệu $n = 20$ điểm ngẫu nhiên chuẩn hoá), tính ma trận Gram $G^{(H)}(0)$ bằng autograd (đạo hàm u_i theo $W^{(H)}$) ngay tại khởi tạo, so sánh $\|G^{(H)}(0) - G^{(H)}_{\text{ước lượng theo công thức (7)}}\|$ qua các H khác nhau. Đo: độ lệch này tăng theo H như thế nào. Kỳ vọng: độ lệch tăng rõ rệt (gần cấp số mũ) khi H tăng cho fully-connected thuần.
2. **So sánh fully-connected vs ResNet khi tăng độ sâu**: huấn luyện hai mạng toy (fully-connected và ResNet với scaling $c_{\text{res}}/\sqrt{H}$ như Eq. 3) trên cùng bài toán hồi quy tổng hợp nhỏ ($n = 50$, $d = 10$), với $H \in \{5, 10, 20, 40\}$, độ rộng $m = 128$ cố định, cùng step size. Đo: liệu loss có giảm về 0 ổn định hay phân kỳ/dao động. Kỳ vọng: ResNet vẫn hội tụ ổn định ở H lớn, fully-connected có thể phân kỳ hoặc hội tụ rất chậm khi H tăng, minh hoạ trực quan Định lý 5.1 vs 6.1 (chạy được trên CPU/GPU nhỏ, mạng toy).
3. **Kiểm tra độ lệch trọng số so với khởi tạo (lazy training signature)**: huấn luyện một MLP nhỏ quá tham số hoá ($m \gg n$) trên dữ liệu toy, theo dõi $\frac{1}{\sqrt{m}}\|W^{(h)}(k) - W^{(h)}(0)\|_F$ qua các bước k cho từng lớp h. Đo: đại lượng này có nhỏ và ổn định hay tăng dần. Kỳ vọng: nhỏ và gần như không đổi nếu m đủ lớn (xác nhận giả thuyết cốt lõi của chứng minh, Eq. 6 trong paper), giúp hiểu trực giác ”lazy training” mà không cần đọc chứng minh đầy đủ.
4. **Thử nghiệm điều kiện xác định dương của $K^{(H)}$ khi dữ liệu gần trùng nhau**: tạo dữ liệu tổng hợp với các điểm ngày càng ”gần trùng” (giảm dần khoảng cách cosine giữa các cặp điểm), tính $\lambda_{\min}(K^{(H)})$ bằng mô phỏng Monte Carlo (lấy mẫu nhiều (u, v) từ công thức đệ quy Eq. 7). Đo: $\lambda_{\min}(K^{(H)})$ giảm về 0 như thế nào khi dữ liệu suy biến dần. Kỳ vọng: giảm nhanh, minh hoạ tại sao điều kiện ”dữ liệu không suy biến” quan trọng, liên hệ tới rủi ro khi dữ liệu OCR có nhiều trang/token rất giống nhau.
5. **Ablation gỡ residual connection trên mạng nhỏ đã huấn luyện**: lấy một mạng ResNet toy nhỏ đã

huấn luyện tốt (ví dụ 10 lớp, bài toán phân loại tổng hợp), sau đó ”zero-out” hệ số c_{res} của một vài block (giữ nguyên trọng số còn lại) để giả lập việc loại bỏ một phần residual path, rồi fine-tune lại với vài bước GD, đo tốc độ giảm loss trước/sau khi loại residual. Đo: tốc độ hội tụ của GD khi fine-tune. Kỳ vọng: tốc độ giảm loss chậm hơn/kém ổn định hơn khi residual path bị loại, một kiểm chứng nhỏ cho suy luận ở §8 về rủi ro pruning residual connection.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Kiến trúc fully-connected	$x^{(h)} = \sigma\left(\sqrt{c_\sigma/m}\, W^{(h)} x^{(h-1)}\right)$, $f = a^\top x^{(H)}$	Eq. 2 (paper)
Kiến trúc ResNet	$x^{(h)} = x^{(h-1)} + \frac{c_{\text{res}}}{\sqrt{H}\sqrt{m}}\sigma(W^{(h)}x^{(h-1)})$	Eq. 3 (paper)
Động lực sai số dự đoán	$y - u(k+1) = (I - \eta G(k))(y - u(k))$	mục 4 (paper)
Ma trận Gram đệ quy (fully-connected)	$K_{ij}^{(h)} = c_\sigma\, \mathbb{E}_{(u,v)\sim N(0,A_{ij}^{(h)})}[\sigma(u)\sigma(v)]$	Eq. 7 (paper)
Lan truyền nhiễu, fully-connected	$\|G^{(H)}(0) - K^{(H)}\| \lesssim 2^{O(H)}\epsilon_1$	Eq. 4 (paper)
Lan truyền nhiễu, ResNet	$\|G^{(H)}(0) - K^{(H)}\| \lesssim (1+1/H)^H \epsilon_1 = O(\epsilon_1)$	Eq. 5 (paper)
Định lý hội tụ (dạng chung 3 định lý)	$L(\theta(k)) \le \left(1 - \frac{\eta\lambda_{\min}(K^{(H)})}{2}\right)^k L(\theta(0))$	Theorem 5.1/6.1/7.1 (paper)
Độ rộng cần, fully-connected	$m = \Omega\big(2^{O(H)}\,\text{poly}(n, 1/\lambda_{\min})\big)$	Theorem 5.1 (paper)
Độ rộng cần, ResNet	$m = \Omega\big(\text{poly}(n, H, 1/\lambda_{\min})\big)$ (không có số mũ H)	Theorem 6.1 (paper)
Ví dụ tự tính (bước đệ quy với σ tuyến tính giả định)	$K^{(0)} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \Rightarrow K^{(1)} = K^{(0)}$	tự tính, chỉ minh hoạ, không phải điều kiện thật của paper

11. Kết luận ngắn

Paper chứng minh định lượng rằng gradient descent khởi tạo ngẫu nhiên hội tụ tuyến tính về loss huấn luyện bằng 0 cho ba kiến trúc mạng sâu quá tham số hoá (fully-connected, ResNet, convolutional ResNet), với mấu chốt kỹ thuật là chứng minh tính ổn định của ma trận Gram $K^{(H)}$ xuyên suốt quá trình huấn luyện. Kết quả quan trọng nhất là **giải thích lý thuyết định lượng cho lợi ích của residual connection**: độ rộng mạng cần thiết giảm từ tăng theo cấp số mũ ($2^{O(H)}$, fully-connected) xuống đa thức (poly(H), ResNet) theo số lớp H, nhờ hệ số scaling $1/\sqrt{H}$ trong kiến trúc residual. Đây là một trong những bài nền tảng của dòng lý thuyết NTK/Gram-matrix cho mạng sâu, nhưng tự paper cũng thẳng thắn nêu các giới hạn quan trọng: chỉ chứng minh hội tụ trên tập huấn luyện (không phải generalization), yêu cầu độ rộng vẫn rất lớn so với thực hành, và chỉ áp dụng cho full-batch gradient descent.

Nguồn gốc: https://arxiv.org/abs/1811.03804

Bài 34

A Mean Field View of the Landscape of Two-Layer Neural Networks (2018)

Tác giả: Song Mei, Andrea Montanari, Phan-Minh Nguyen
Venue / arXiv: arXiv:1804.06561 (v2, 28/08/2018); paper không nêu venue trong phần text trích được (được biết đến rộng rãi là PNAS 2018, nhưng vì dòng chữ này không xuất hiện trong văn bản arXiv trích xuất được, ghi ”paper không nêu venue” theo đúng những gì trích được từ full text)
Nguồn: https://arxiv.org/abs/1804.06561
Chủ đề: Chứng minh rằng, trong giới hạn tỷ lệ (scaling limit) phù hợp, động lực SGD khi huấn luyện mạng nơ-ron 2 lớp được mô tả xấp xỉ bởi một phương trình đạo hàm riêng phi tuyến (PDE) gọi là ”distributional dynamics”, cho phép phân tích landscape tối ưu hoá độc lập với số lượng neuron.
Vị trí trong lộ trình: Đọc liền sau math 33 (Gradient Descent Finds Global Minima of DNNs) vì cùng trả lời câu hỏi ”vì sao SGD tìm được nghiệm tốt cho mạng quá tham số hoá” nhưng bằng công cụ khác (mean-field/PDE/Wasserstein gradient flow thay vì Gram-matrix/NTK); là bài áp cuối trong batch math 28-35, liền trước math 35 (Thompson Sampling).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper đặt câu hỏi cốt lõi giống nhiều bài lý thuyết deep learning cùng giai đoạn: huấn luyện mạng nơ-ron là một bài toán tối ưu hoá **không lồi (non-convex)** trên một hàm rủi ro (risk function) nhiều chiều, vậy tại sao stochastic gradient descent (SGD) vẫn thường tìm được nghiệm tốt? Hai câu hỏi con mà paper muốn trả lời: (a) SGD hội tụ về optimum toàn cục vì không có local minimum xấu, hay vì SGD ”tránh” được chúng? (b) Nếu SGD chỉ đạt local minimum, vì sao các local minimum đó vẫn tổng quát hoá (generalize) tốt?

Paper giới hạn phạm vi vào trường hợp đơn giản nhất có ý nghĩa: **mạng nơ-ron 2 lớp (two-layer neural network)**:

$$\hat{y}(x;\theta) = \frac{1}{N}\sum_{i=1}^{N} \sigma_*(x;\theta_i), \qquad \sigma_*(x;\theta_i) = a_i\,\sigma(\langle w_i, x\rangle + b_i), \tag{1-2, paper}$$

với N là số neuron ẩn, $\theta_i = (a_i, b_i, w_i)$ là tham số của neuron i. Mục tiêu là tối thiểu hoá **risk** (hàm rủi ro tổng thể, kỳ vọng loss trên phân phối dữ liệu thật) $R_N(\theta) = \mathbb{E}\{\ell(y, \hat{y}(x;\theta))\}$ với loss bình phương $\ell(y,\hat{y}) = (y-\hat{y})^2$.

Thuật ngữ cần làm rõ: **one-pass SGD** là giả định mỗi điểm dữ liệu chỉ được dùng đúng một lần (không lặp lại qua nhiều epoch), hợp lý khi dữ liệu rất lớn; **mean-field limit** (giới hạn trường trung bình) là kỹ thuật vật lý thống kê coi một tập hợp rất nhiều ”hạt” (ở đây là neuron) tương tác với nhau như một **phân phối liên tục** thay vì theo dõi từng hạt riêng lẻ; **Wasserstein metric** W_2 là một khoảng cách giữa hai phân phối xác suất, định nghĩa qua bài toán vận chuyển tối ưu (optimal transport), đo ”chi phí tối thiểu” để biến phân phối này thành phân phối khác.

2. Ý tưởng chính

Quan sát khởi đầu (mục 1.1 của paper): risk $R_N(\theta)$ chỉ phụ thuộc vào $\theta_1, \dots, \theta_N$ **qua phân phối thực nghiệm** của chúng, $\hat{\rho}^{(N)} = N^{-1}\sum_{i=1}^{N}\delta_{\theta_i}$. Viết lại:

$$R_N(\theta) = R_\# + \frac{2}{N}\sum_{i=1}^{N} V(\theta_i) + \frac{1}{N^2}\sum_{i,j=1}^{N} U(\theta_i, \theta_j), \qquad \text{(4, paper)}$$

với $V(\theta) = -\mathbb{E}[y\,\sigma_*(x;\theta)]$ và $U(\theta_1,\theta_2) = \mathbb{E}[\sigma_*(x;\theta_1)\sigma_*(x;\theta_2)]$ là một **kernel đối xứng, xác định dương bán phần (PSD)**. Điều này gợi ý định nghĩa risk cho một phân phối $\rho \in \mathcal{P}(\mathbb{R}^D)$ bất kỳ:

$$R(\rho) = R_\# + 2\int V(\theta)\rho(d\theta) + \int\int U(\theta_1,\theta_2)\,\rho(d\theta_1)\rho(d\theta_2). \qquad \text{(5, paper)}$$

Vì U là PSD, $R(\rho)$ **trở thành hàm lồi (convex)** theo ρ ở giới hạn $N \to \infty$. Nhưng điều này **không** suy ra rằng SGD sẽ luôn tìm được optimum toàn cục: paper dùng một phép tương tự vật lý (coi $\theta_1, \ldots, \theta_N$ như vị trí N "hạt" trong không gian D chiều) để chỉ ra rằng động lực của "khí hạt" này **không thể thay đổi mật độ một cách tuỳ ý**: khối lượng được bảo toàn cục bộ (local mass conservation) vì các hạt không thể di chuyển gián đoạn. Đây chính là lý do động lực PDE giới hạn không đơn giản là "gradient descent trực tiếp trên $R(\rho)$", mà là một **gradient flow trong không gian Wasserstein** $(\mathcal{P}(\mathbb{R}^D), W_2)$.

Kết quả chính (Định lý 3, phát biểu chi tiết ở §3): khi $N \to \infty, \varepsilon \to 0$ với tốc độ step size $s_k = \varepsilon\,\xi(k\varepsilon)$, phân phối thực nghiệm $\hat{\rho}^{(N)}_{t/\varepsilon}$ hội tụ yếu về ρ_t là nghiệm của PDE:

$$\partial_t \rho_t = 2\xi(t)\,\nabla_\theta \cdot \big(\rho_t\,\nabla_\theta \Psi(\theta;\rho_t)\big), \qquad \Psi(\theta;\rho) \equiv V(\theta) + \int U(\theta,\theta')\rho(d\theta'). \qquad \text{(7-8, paper)}$$

Paper gọi đây là **distributional dynamics (DD)**. Điểm mấu chốt: **PDE này không phụ thuộc vào** N (số neuron), chỉ phụ thuộc vào kích thước tham số D và phân phối dữ liệu. Do đó nếu DD hội tụ về gần optimum trong thời gian $t^*(D)$ không phụ thuộc N, thì với $N \gg D$ đủ lớn, SGD sẽ đạt risk gần tối ưu bằng một số bước lặp **chỉ phụ thuộc** D**, không phụ thuộc số neuron** N. Đây là lời giải thích định lượng cho hiện tượng over-parameterization không gây hại hiệu năng.

3. Phương pháp chi tiết

3.1. Noisy SGD và free energy (mục 1.1 cuối, §3.1)

Paper xét thêm biến thể **noisy SGD** (thêm nhiễu Gaussian và regularization ℓ_2):

$$\theta_i^{k+1} = (1 - 2\lambda s_k)\theta_i^k + 2s_k(y_k - \hat{y}_k)\nabla_{\theta_i}\sigma_*(x_k;\theta_i^k) + \sqrt{2s_k/\beta}\,g_i^k, \quad g_i^k \sim N(0, I_D). \qquad \text{(11, paper)}$$

Giới hạn tỷ lệ của phiên bản này thêm một số hạng khuếch tán (diffusion):

$$\partial_t \rho_t = 2\xi(t)\nabla_\theta \cdot (\rho_t \nabla_\theta \Psi_\lambda(\theta;\rho_t)) + 2\xi(t)\beta^{-1}\Delta_\theta \rho_t, \qquad \text{(12, paper)}$$

là gradient flow của **free energy** $F_{\beta,\lambda}(\rho) = \frac{1}{2}R(\rho) + \frac{\lambda}{2}\int \|\theta\|_2^2 \rho(d\theta) - \beta^{-1}\mathrm{Ent}(\rho)$, với $\mathrm{Ent}(\rho)$ là entropy của ρ. Số hạng entropy này (từ nhiễu Gaussian) chính là lý do **Định lý 4-5** của paper chứng minh được **hội tụ tổng quát (generic convergence)** về minimizer toàn cục của $F_{\beta,\lambda}$ (không cần điều kiện đặc biệt), trong khi với SGD không nhiễu (noiseless), paper **chỉ chứng minh được điều kiện ổn định cục bộ** (Định lý 6-7), không có kết quả hội tụ toàn cục tổng quát.

3.2. Bốn ví dụ minh hoạ cụ thể (mục 2)

Paper tự nói rõ các ví dụ này "không thực tế" (not realistic), chỉ để minh hoạ cơ chế lý thuyết bằng tính toán giải được:

(a) Centered isotropic Gaussians (§2.1): $x \sim N(0, (1 \pm \Delta)^2 I_d)$ tuỳ nhãn $y = \pm 1$; hàm kích hoạt $\sigma_*(x; \theta_i) = \sigma(\langle w_i, x \rangle)$ không có offset/trọng số đầu ra, σ là hàm tuyến tính-từng-đoạn ($t_1 = 0.5, t_2 = 1.5, s_1 = -2.5, s_2 = 7.5$). Vì phân phối dữ liệu P đối xứng cầu (spherically symmetric), ρ_t cũng đối xứng cầu nên **rút gọn về một PDE 1 chiều** theo $r = \|w\|_2$:

$$\partial_t \rho_t = 2\xi(t)\, \partial_r \big(\rho_t\, \partial_r \psi(r; \rho_t) \big). \qquad \text{(13, paper)}$$

Định lý 1 (phát biểu đầy đủ): với $\eta, \Delta, \delta > 0$ và khởi tạo ρ_0 thuộc một lớp "khởi tạo tốt" $\mathcal{P}_{\text{good}}$, tồn tại d_0, T, C_0 sao cho với $d \geq d_0$, số neuron $N \geq C_0 d$, step size $\varepsilon \in [1/N^{10}, 1/(C_0 d)]$: $R_N(\theta^k) \leq \inf_\theta R(\theta) + \eta$ với xác suất $\geq 1 - \delta$, cho mọi $k \in [T/\varepsilon, 10T/\varepsilon]$. Điểm quan trọng: **số bước SGD cần thiết $k = O(d)$, không phụ thuộc số neuron N**.

(b) Centered anisotropic Gaussians (§2.2): $\Sigma_\pm = I_d + (\tau_\pm^2 - 1) P_V$ với V là không gian con chiều s_0 cần được mạng "học ra" (biến relevant/irrelevant feature); **Định lý 2** mở rộng kết quả Định lý 1 cho trường hợp đối xứng giảm $O(s_0) \times O(d - s_0)$, với $s_0 = \gamma d$.

(c) A better activation function / ReLU (§2.3): dùng ReLU $\sigma(x) = \max(x, 0)$ với cả trọng số đầu ra a và offset b được học ($\theta = (w, a, b) \in \mathbb{R}^{d+2}$); thí nghiệm với $d = 320, s_0 = 60, N = 800$, cho thấy SGD hội tụ về risk rất nhỏ nhưng **có cấu trúc phi tầm thường với các vùng phẳng (long flat regions)** kéo dài, dự đoán khớp bởi PDE rút gọn 4 chiều (a, b, r_1, r_2).

(d) Predicting failure (§2.4): dùng hàm kích hoạt **không đơn điệu (non-monotone)** $\sigma(t) = -2.5$ với $t \leq 0$, nội suy tuyến tính qua điểm $(0.5, -4)$, rồi $\sigma(t) = 7.5$ với $t \geq 1.5$. Với hàm này (vi phạm giả định đơn điệu của Định lý 1), **SGD hội tụ về một trong hai giới hạn khác nhau tuỳ khởi tạo**: một có risk nhỏ, một có risk cao. Đây là ví dụ paper tự xây dựng để **chứng minh được trước (predict) sự thất bại** của SGD bằng chính lý thuyết PDE của mình, không phải quan sát thất bại rồi giải thích sau.

3.3. Điều kiện ổn định điểm bất động (§3.2)

Định lý 6: với $\rho_* = \delta_{\theta_*}$ (một điểm mass đơn) là điểm bất động của DD (nghĩa là $\nabla V(\theta_*) + \nabla_1 U(\theta_*, \theta_*) = 0$), nếu ma trận Hessian $H_0(\delta_{\theta_*}) = \nabla^2 V(\theta_*) + \int \nabla_{1,1}^2 U(\theta_*, \theta) \rho_*(d\theta)$ có $\lambda_{\min}(H_0) > 0$, thì ρ_* **ổn định (stable)**: nếu khởi tạo đủ gần, $\rho_t \to \rho_*$ với tốc độ mũ.

Định lý 7: ngược lại, nếu $\lambda_{\min}(H_0) < 0$ (điểm yên ngựa / saddle), thì ρ_* **không ổn định**: với khởi tạo có mật độ liên tục tuyệt đối (absolutely continuous), ρ_t không thể hội tụ về ρ_*.

Ví dụ tự tính nhỏ minh hoạ Định lý 6/7: giả sử bài toán 1 chiều đơn giản hoá với $\Psi(\theta; \rho_*) = \theta^2 - c$ gần một điểm $\theta_* = 0$ (minh hoạ ý tưởng, không phải thiết lập thật của paper). Hessian tại $\theta_* = 0$ là $H_0 = \Psi''(0) = 2 > 0$, nên theo logic Định lý 6, $\theta_* = 0$ là điểm bất động ổn định (tự tính, chỉ minh hoạ trực giác dấu của Hessian quyết định ổn định/bất ổn định, không phải một phép tính từ các công thức cụ thể của paper).

4. Thí nghiệm và kết quả chính

Paper có phần thí nghiệm số (Mục 11 trong Supplementary Information) thực sự là **mô phỏng SGD trên dữ liệu tổng hợp (synthetic)** để kiểm chứng (validate) lý thuyết PDE, không phải benchmark trên dữ liệu thực tế nào.

Thí nghiệm	Thiết lập cụ thể (paper)	Kết quả chính
Hình 1 (Isotropic Gaussians, DD)	$\Delta = 0.8$, $d = 40$, $N = 800$, $\varepsilon = 10^{-6}$, $\xi(t) = 1$, khởi tạo $w_i^0 \sim N(0, 0.8^2/d \cdot I_d)$, 1 lần chạy SGD, snapshot tại bước $10^3, 4 \times 10^6, 10^7$	Histogram của $\|w_i\|_2$ từ SGD khớp với nghiệm số của PDE rút gọn (Eq. 13)
Hình 2 (Isotropic Gaussians, Statics)	$d \in \{10, 20, 40, \infty\}$, quét $\Delta \in [0.1, 0.8]$, $N = 800$	Risk thực nghiệm (SGD, 1 lần chạy/điểm) khớp dự đoán lý thuyết từ việc tối thiểu hoá $R(\rho)$; ansatz "single-delta" sai khi $\Delta > \Delta_{hd} \approx 0.47$ (ở $d \to \infty$)
Hình 3 (Anisotropic Gaussians, ReLU)	$d = 320$, $s_0 = 60$, $N = 800$, $\xi(t) = t^{-1/4}$, $\varepsilon = 2 \times 10^{-4}$, khởi tạo $a_i = 1, b_i = 1, w_i^0 \sim N(0, 0.8^2/d \cdot I_d)$	SGD hội tụ về risk nhỏ nhưng có cấu trúc "vùng phẳng kéo dài"; PDE rút gọn 4 chiều khớp tốt
Hình 4 (Predicting failure)	$N = 800$, $d = 320$, $\Delta = 0.5$, hai khởi tạo $w_i^0 \sim N(0, \kappa^2/d \cdot I_d)$ với $\kappa = 0.1$ hoặc $\kappa = 0.4$	SGD hội tụ về 2 giới hạn khác nhau tuỳ κ (risk thấp vs risk cao), đúng như PDE 1 chiều dự đoán trước

Chi tiết kỹ thuật mô phỏng PDE (từ mục 11.1.1, đọc kỹ): vì giải PDE tổng quát (chiều D bất kỳ) tốn kém tính toán, paper chỉ mô phỏng ở **giới hạn** $d = \infty$ (công thức risk dạng tích phân đóng, Eq. 11.1 của SI) và dùng **ansatz nhiều-delta (multiple-deltas ansatz)**, xấp xỉ $\rho_t \approx \frac{1}{J}\sum_{i=1}^{J} \delta_{r_i(t)}$ với J điểm đại diện (ở ví dụ Hình 1, $J = 400$, lặp lại mô phỏng 100 lần với khởi tạo độc lập để lấy trung bình). Paper ghi rõ: trong một số trường hợp đặc biệt (ví dụ $\Delta = 0.2$) **chỉ cần** $J = 1$ (một điểm mass duy nhất) là đủ chính xác, vì nghiệm PDE thực sự hội tụ về một phân phối delta đơn.

5. Phân tích / ablation

Vai trò của hệ số nhiễu β (inverse temperature): paper phân tích rõ sự khác biệt giữa SGD không nhiễu và noisy SGD: với $\beta < \infty$ (có nhiễu), Định lý 4 đảm bảo **hội tụ tổng quát** về minimizer duy nhất của free energy $F_{\beta,\lambda}$ (vì entropy làm cho bài toán "lồi hoá" theo nghĩa thống kê); với $\beta = \infty$ (không nhiễu), paper **chỉ chứng minh được điều kiện ổn định cục bộ** (Định lý 6-7), không đảm bảo hội tụ toàn cục tổng quát - một sự đánh đổi giữa "đảm bảo lý thuyết mạnh hơn" và "gần với thực hành hơn" (SGD chuẩn không có nhiễu Gaussian tường minh).

Phân tích biên Δ_{ld}, Δ_{hd} (Lemma 1): với bài toán isotropic Gaussians, nghiệm tối ưu $\rho_{r_*}^{\text{unif}}$ (phân phối đều trên mặt cầu bán kính r_*) chỉ là **global minimizer trong một khoảng** $\Delta \in [\Delta_{ld}, \Delta_{hd}]$, với $\lim_{d\to\infty} \Delta_{ld} = 0$ và $\lim_{d\to\infty} \Delta_{hd} \approx 0.47$ (paper tính số bằng kiểm tra điều kiện trong Lemma 1, không có công thức closed-form cho Δ_{hd}). Ngoài khoảng này, ansatz "single-delta" (một điểm mass đơn) **không còn là lời giải tối ưu**, cho thấy landscape thực sự phức tạp hơn một điểm cực tiểu duy nhất khi Δ lớn.

So sánh cấu trúc hội tụ giữa các ví dụ: paper ghi nhận và giải thích (mục 11.1.1, "Comments") sự khác biệt về "độ sắc nét" trong quá trình hội tụ: ở $\Delta = 0.2$, ρ_t hội tụ nhanh về gần delta-function (chuyển pha sắc nét từ risk cao sang risk thấp); ở $\Delta = 0.8$, ρ_t **không tập trung (not concentrating)** dù hàm $R_d^{(1)}(r)$ vẫn "lành" (benign landscape 1 chiều) - cho thấy ngay cả khi landscape rút gọn trông đơn giản, động lực đầy đủ của phân phối vẫn có thể phức tạp hơn.

Ảnh hưởng của dữ liệu/kiến trúc lên thời gian hội tụ T: paper tự nêu (cuối mục 3.1, Định lý 5) rằng $T = T(\eta, V, U, \{K_i\}, D, \beta)$ phụ thuộc chiều D và phân phối dữ liệu, và "examining the proof of this theorem suggests $T = e^{O(D)}$ quite generally", nhưng paper cũng nói **có thể xây dựng ví dụ với $T = O(1)$ hoặc $T = e^{\Theta(D)}$** - tức cận trên tổng quát có thể bi quan hơn thực tế tuỳ cấu trúc cụ thể của bài toán.

6. Giới hạn

Các giới hạn dưới đây được paper **tự nêu rõ** (mục 4, Discussion and future directions), không phải suy luận của ghi chú trừ khi ghi rõ:

- **Chỉ áp dụng cho mạng 2 lớp**: toàn bộ lý thuyết (PDE, định lý hội tụ) chỉ xây dựng cho kiến trúc 2 lớp đơn giản nhất; paper không mở rộng cho mạng sâu nhiều lớp trong chính bài này.
- **Không chứng minh tính duy nhất của local minimum ở N hữu hạn**: paper tự nói rõ "we do not prove that the finite-N risk $R_N(\theta)$ has a unique local minimum, or that all local minima are close to each other... might be too strong for the case of neural networks" - đây là một giới hạn paper chủ động không theo đuổi, thay vào đó chỉ chứng minh PDE (giới hạn $N \to \infty$) hội tụ về near-global-optimum.
- **Với noiseless SGD, không có kết quả hội tụ toàn cục tổng quát**: chỉ có điều kiện ổn định cục bộ (Định lý 6-7) cho các điểm bất động dạng "một điểm mass + phần còn lại", không bao trùm mọi cấu hình điểm bất động có thể.
- **Sai số xấp xỉ tăng theo cấp số mũ với thời gian T** (paper tự nêu ngay dưới Định lý 3): "the error grows exponentially with the time horizon T, which limits its applicability to cases in which the DD converges rapidly to a good solution" - nghĩa là lý thuyết chỉ hữu dụng khi PDE hội tụ nhanh, không phải cho mọi bài toán.
- **Các ví dụ minh hoạ không thực tế**: paper tự nhận ở đầu mục 2: "these examples are not realistic. First, the data distribution P is extremely simple... Second, the activation function $\sigma_*(x;\theta)$ is not necessarily optimal" - các ví dụ (isotropic/anisotropic Gaussian, hàm kích hoạt tuyến tính-từng-đoạn tuỳ chỉnh) được chọn để **có thể tính toán closed-form**, không phản ánh dữ liệu/kiến trúc thực tế (ảnh, văn bản, v.v).
- **Chỉ xét loss bình phương (square loss) trong phần chứng minh chính**: paper nói việc mở rộng cho các loss khác "should be generalizable... cf. SI" (Appendix B) nhưng không chứng minh đầy đủ trong phần chính.
- **Mô phỏng PDE trong thí nghiệm chỉ thực hiện ở $d = \infty$ với ansatz nhiều-delta**: bản thân việc mô phỏng số PDE tổng quát (chiều D hữu hạn, không rút gọn đối xứng) được paper nói là "computationally intensive", nên mọi thí nghiệm số trong bài đều dựa vào các trường hợp đã **rút gọn được bằng đối xứng** (isotropic/anisotropic Gaussian); không có thí nghiệm nào cho trường hợp hoàn toàn tổng quát không có đối xứng đặc biệt.

7. Liên hệ với các bài khác

- **math content/33 (Gradient Descent Finds Global Minima of DNNs, 2018, cùng batch)**: hai bài trả lời cùng câu hỏi lớn ("vì sao GD/SGD hội tụ tốt cho mạng quá tham số hoá") bằng hai công cụ khác nhau: bài 33 dùng ma trận Gram/NTK-style (mạng sâu nhiều lớp, GD toàn batch), bài này dùng mean-field/PDE/Wasserstein gradient flow (chỉ mạng 2 lớp, SGD one-pass). Paper 34 tự liệt kê các công trình optimal-transport liên quan (Mei et al., Chizat & Bach, Rotskoff & Vanden-Eijnden) trong phần Discussion, và paper 33 cũng dẫn chính các công trình này ở Related Work, cho thấy hai hướng lý thuyết này được chính các tác giả đương thời coi là song song/bổ sung.
- **math content/22 (Deep Neural Networks as Gaussian Processes, 2017)**: cùng chủ đề "mạng ở giới hạn rộng vô hạn / số neuron vô hạn có cấu trúc toán học đơn giản hơn"; math 22 dùng giới hạn Gaussian Process (hàm số ngẫu nhiên), bài này dùng giới hạn trường trung bình (phân phối tham số); cả hai đều khai thác $N \to \infty$ để "làm phẳng" độ phức tạp của landscape, nhưng theo hai đối tượng toán học khác nhau (hàm số vs. phân phối tham số).
- **math content/20 (Benefits of Depth, 2016) và math content/21 (Expressive Power of Width, 2017)**: cùng dòng câu hỏi lý thuyết về vai trò của độ rộng/độ sâu trong mạng nơ-ron; bài này cho thấy cụ thể **độ rộng N không ảnh hưởng đến landscape tối ưu hoá** khi $N \gg D$ (chỉ ảnh hưởng tốc độ xấp xỉ $O(1/N)$ giữa R_N và R), một góc nhìn bổ sung cho câu hỏi "độ rộng cần bao nhiêu là đủ" mà hai bài kia tiếp cận từ phía biểu diễn hàm (expressivity) thay vì tối ưu hoá.

- **math content/24 (Lazy Training, 2018)**: đối lập thú vị cần nêu rõ: ”lazy training” (math 24) mô tả chế độ mà trọng số gần như không đổi so với khởi tạo (kernel regime, NTK); bài 34 này mô tả chế độ **ngược lại** (”feature learning” / mean-field regime), nơi phân phối tham số ρ_t **di chuyển đáng kể** theo thời gian (không ”lazy”) để tối thiểu hoá risk $R(\rho)$. Đây là hai chế độ scaling khác nhau của cùng kiến trúc mạng (phụ thuộc cách chọn hệ số chuẩn hoá theo N), một điểm đối chiếu quan trọng khi đọc cả ba bài 33/34/24 cùng nhau.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU <= 8 GB, batch 1; các hướng pruning, distillation, RL, interpretability đã có thư mục riêng).

Bài này **ít liên quan trực tiếp** đến việc nén/hậu huấn luyện LightOnOCR-3-0.8B: lý thuyết chỉ áp dụng cho mạng 2 lớp huấn luyện từ đầu bằng one-pass SGD trên dữ liệu tổng hợp đơn giản (Gaussian), hoàn toàn khác bối cảnh thực tế của dự án (mô hình VLM nhiều lớp, hybrid attention/SSM, đã huấn luyện sẵn, cần fine-tune/pruning với ngân sách hạn chế).

Có một liên hệ gián tiếp mang tính suy luận đáng ghi nhận: khái niệm **”feature learning” (phân phối tham số di chuyển đáng kể, đối lập với ”lazy training” như đã nêu ở §7)** mà bài này minh hoạ gợi ý một câu hỏi thực tế cho việc fine-tune sau pruning: nếu quá trình fine-tune khôi phục (recovery fine-tuning) sau khi pruning LightOnOCR chỉ làm trọng số ”dao động nhẹ quanh điểm pruning” (giống chế độ lazy/NTK, gần với bài 33/24) thì có thể cần ít bước fine-tune và learning rate nhỏ; ngược lại nếu cần trọng số ”di chuyển xa” để khôi phục năng lực OCR đã mất (giống chế độ feature-learning/mean-field của bài này) thì cần learning rate lớn hơn và nhiều bước hơn. Đây hoàn toàn là một khung suy luận định hướng thử nghiệm (không phải kết luận trực tiếp từ paper, paper không đề cập pruning hay fine-tune mô hình đã huấn luyện sẵn), không phải một khuyến nghị kỹ thuật cụ thể.

9. Thí nghiệm gợi ý

1. **Tái tạo thí nghiệm Hình 1 (isotropic Gaussians) ở quy mô nhỏ**: cài mạng 2 lớp với $\sigma_*(x;\theta_i) = \sigma(\langle w_i, x\rangle)$ dùng đúng hàm kích hoạt tuyến tính-từng-đoạn của paper ($t_1 = 0.5, t_2 = 1.5, s_1 = -2.5, s_2 = 7.5$), $d = 20$ (nhỏ hơn paper để chạy nhanh trên CPU/GPU nhỏ), $N = 200$, $\Delta = 0.8$, khởi tạo $w_i^0 \sim N(0, 0.8^2/d \cdot I_d)$. Đo: histogram của $\|w_i\|_2$ qua các bước huấn luyện. Kỳ vọng: phân phối ”nở rộng” dần, có thể thấy xu hướng tương tự Hình 1 của paper ở quy mô nhỏ hơn.
2. **Minh hoạ ”Predicting failure” (Hình 4)**: cài đúng hàm kích hoạt không đơn điệu của §2.4 ($\sigma(t) = -2.5$ với $t \le 0$, nội suy qua $(0.5, -4)$, rồi $\sigma(t) = 7.5$ với $t \ge 1.5$), $d = 40, N = 200, \Delta = 0.5$, thử 2 khởi tạo với $\kappa = 0.1$ và $\kappa = 0.4$ như paper. Đo: risk cuối cùng sau nhiều bước SGD cho mỗi khởi tạo. Kỳ vọng: hai khởi tạo hội tụ về hai mức risk khác biệt rõ rệt, minh hoạ trực quan hiện tượng SGD ”mắc kẹt” tuỳ khởi tạo khi vi phạm điều kiện đơn điệu của Định lý 1.
3. **Kiểm tra độc lập với số neuron** N: huấn luyện cùng bài toán isotropic Gaussians ($d = 20, \Delta = 0.4$ cố định) với nhiều giá trị $N \in \{50, 100, 400, 1600\}$, cùng số bước huấn luyện. Đo: risk cuối cùng theo N. Kỳ vọng: risk hội tụ về một giá trị gần như không đổi khi N đủ lớn (ví dụ từ $N = 400$ trở lên không cải thiện thêm đáng kể), minh hoạ trực giác ”SGD không phụ thuộc N khi $N \gg D$”.
4. **Quan sát chuyển pha theo Δ (tái tạo một phần Hình 2)**: quét $\Delta \in \{0.1, 0.2, \ldots, 0.8\}$ với $d = 20$ cố định, đo risk cuối cùng sau huấn luyện SGD hội tụ. Đo: đường cong risk theo Δ. Kỳ vọng: thấy xu hướng risk tăng dần theo Δ với khả năng có một điểm uốn/bất liên tục gần giá trị Δ_{hd} tương tự paper (dù quy mô d nhỏ hơn sẽ làm hiệu ứng kém rõ hơn).
5. **So sánh chế độ ”lazy” vs ”feature learning” bằng cách thay đổi scaling khởi tạo**: huấn luyện cùng mạng 2 lớp trên bài toán tổng hợp, nhưng thay đổi độ lớn khởi tạo $w_i^0 \sim N(0, \sigma_0^2/d \cdot I_d)$ với σ_0 nhỏ (ví dụ 0.01, gần chế độ lazy/NTK) và σ_0 lớn hơn (ví dụ 1.0, gần chế độ mean-field/feature-learning). Đo:

độ lệch trung bình $\frac{1}{N}\sum_i \|w_i(k) - w_i(0)\|_2$ theo thời gian huấn luyện cho mỗi σ_0. Kỳ vọng: độ lệch nhỏ và tương đối ổn định ở σ_0 nhỏ (lazy), độ lệch lớn dần theo thời gian ở σ_0 lớn (feature learning), minh hoạ trực quan sự khác biệt giữa hai chế độ nêu ở §7/§8.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Mạng 2 lớp	$\hat{y}(x;\theta) = \frac{1}{N}\sum_{i=1}^{N} a_i \sigma(\langle w_i, x\rangle + b_i)$	Eq. 1-2 (paper)
Risk dạng potential	$R_N(\theta) = R_\# + \frac{2}{N}\sum_i V(\theta_i) + \frac{1}{N^2}\sum_{i,j} U(\theta_i,\theta_j)$	Eq. 4 (paper)
Risk cho phân phối liên tục	$R(\rho) = R_\# + 2\int V\,d\rho + \int\int U\,d\rho\,d\rho$	Eq. 5 (paper)
Distributional dynamics (DD)	$\partial_t \rho_t = 2\xi(t)\nabla_\theta \cdot (\rho_t \nabla_\theta \Psi(\theta;\rho_t))$	Eq. 7 (paper)
Khoảng cách Wasserstein	$W_2(\rho_1,\rho_2) = \left(\inf_{\gamma\in C(\rho_1,\rho_2)} \int \|\theta_1-\theta_2\|_2^2\,\gamma(d\theta_1,d\theta_2)\right)^{1/2}$	Eq. 9 (paper)
Noisy SGD / diffusion PDE	$\partial_t \rho_t = 2\xi(t)\nabla_\theta\cdot(\rho_t\nabla_\theta\Psi_\lambda) + 2\xi(t)\beta^{-1}\Delta_\theta\rho_t$	Eq. 12 (paper)
Free energy	$F_{\beta,\lambda}(\rho) = \frac{1}{2}R(\rho) + \frac{\lambda}{2}\int\|\theta\|_2^2\,\rho(d\theta) - \beta^{-1}\mathrm{Ent}(\rho)$	mục 3.1 (paper)
Điều kiện ổn định điểm bất động	$\lambda_{\min}(H_0(\delta_{\theta_*})) > 0 \Rightarrow$ ổn định; $< 0 \Rightarrow$ bất ổn định	Theorem 6-7 (paper)
PDE rút gọn 1 chiều (isotropic Gaussian)	$\partial_t\rho_t = 2\xi(t)\partial_r(\rho_t\partial_r\psi(r;\rho_t))$	Eq. 13 (paper)
Ví dụ tự tính (dấu Hessian quyết định ổn định)	$\Psi(\theta)\approx\theta^2 - c \Rightarrow H_0 = 2 > 0 \Rightarrow$ ổn định tại $\theta_* = 0$	tự tính, chỉ minh hoạ trực giác

11. Kết luận ngắn

Paper xây dựng một khung lý thuyết mới (distributional dynamics, DD) để phân tích SGD trên mạng nơ-ron 2 lớp, chứng minh rằng ở giới hạn $N \to \infty$, động lực SGD được mô tả bởi một PDE dạng gradient flow trong không gian Wasserstein, hoàn toàn **không phụ thuộc số neuron** N. Khung lý thuyết này cho phép khai thác đối xứng của dữ liệu để giải PDE một cách tường minh trên một số ví dụ cụ thể (Gaussian isotropic/anisotropic), chứng minh SGD hội tụ gần optimum toàn cục trong một số trường hợp, và quan trọng hơn, **dự đoán trước được khi nào SGD sẽ thất bại** (ví dụ "Predicting failure" với hàm kích hoạt không đơn điệu) chỉ bằng phân tích PDE, không cần chạy thử nghiệm trước. Đóng góp lớn nhất của bài là cung cấp một cơ chế toán học khác (so với NTK/Gram-matrix của các bài cùng giai đoạn như math 33) để hiểu vì sao over-parameterization không gây hại hiệu năng tối ưu hoá, nhưng tự paper cũng thẳng thắn giới hạn phạm vi vào mạng 2 lớp, loss bình phương, và các ví dụ dữ liệu đơn giản hoá để có thể tính toán closed-form.

Nguồn gốc: https://arxiv.org/abs/1804.06561

Bài 35

A Tutorial on Thompson Sampling (2017)

Tác giả: Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, Zheng Wen
Venue / arXiv: arXiv:1707.02038 (v1, tháng 7/2017; bản tutorial hoàn chỉnh được in lại năm 2018), paper không nêu venue trong phần text trích được (dòng trích dẫn tự ghi ”Foundations and Trends...” bị mất tên tạp chí khi OCR, chỉ còn thấy ”Vol. 11, No. 1, pp 1-96, DOI: 10.1561/2200000070”; đây chính là Foundations and Trends in Machine Learning theo DOI prefix 10.1561, nhưng vì paper không hiện rõ chữ này trong văn bản trích xuất được, ghi nhận đây là suy luận từ DOI, không phải trích nguyên văn)
Nguồn: https://arxiv.org/abs/1707.02038
Chủ đề: Tutorial tổng hợp về thuật toán Thompson Sampling (TS) cho bài toán ra quyết định tuần tự dưới thông tin không đầy đủ (multi-armed bandit và mở rộng), từ công thức cơ bản, các phương pháp xấp xỉ hậu nghiệm, đến lý thuyết regret và giới hạn của TS.
Vị trí trong lộ trình: Đọc sau khi đã nắm trực giác về suy luận Bayes và MCMC (math 10 Hamiltonian Monte Carlo, math 11 SGD như suy luận Bayes xấp xỉ); đây là bài cuối trong batch math 28-35, đóng vai trò cầu nối giữa suy luận Bayes thống kê và các bài toán khám phá (exploration) trong RL (rl 13 RND exploration) và tối ưu hoá Bayes (optimization 32, 33).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Bài toán trung tâm của tutorial là **online decision problem**: một agent tương tác lặp lại với một hệ thống chưa biết đầy đủ, ở mỗi vòng t phải chọn một hành động x_t, nhận một phần thưởng (reward) r_t, và muốn tối đa hoá tổng phần thưởng tích lũy qua nhiều vòng. Vấn đề cốt lõi là **đánh đổi khám phá - khai thác (exploration-exploitation trade-off)**: agent phải vừa thử các hành động để học thêm về hệ thống (khám phá), vừa phải chọn hành động tốt nhất đã biết để thu lợi (khai thác).

Paper mở đầu bằng **Example 1.1 (Bernoulli Bandit)**: có K hành động (ví dụ K mẫu banner quảng cáo), mỗi hành động k có xác suất thành công (click) chưa biết $\theta_k \in [0, 1]$; mỗi lần chọn hành động k, agent quan sát một phần thưởng Bernoulli với tham số θ_k. Agent không biết trước $\theta = (\theta_1, \ldots, \theta_K)$ và phải học qua trải nghiệm.

Paper cũng đưa ra **Example 1.2 (Online Shortest Path)**: agent đi từ điểm xuất phát đến điểm đích trên một đồ thị có hướng mỗi ngày, quan sát thời gian đi qua từng cạnh đã chọn, với mục tiêu học đường đi ngắn nhất kỳ vọng. Đây là ví dụ cho bài toán có cấu trúc phức tạp hơn multi-armed bandit cơ bản (số hành động ”path” có thể là hàm mũ theo số cạnh).

Thompson Sampling (TS) được **Thompson đề xuất từ năm 1933** cho bài toán thử nghiệm y khoa (clinical trial), nhưng bị bỏ quên trong nhiều thập kỷ. TS chỉ được khám phá lại và phổ biến rộng rãi nhờ các công trình thực nghiệm của Chapelle và Li (2011) và Scott (2010), cho thấy TS hoạt động tốt trong các bài toán quảng cáo trực tuyến thực tế. Paper này (2017-2018) là nỗ lực hệ thống hoá: trình bày thuật toán, nền tảng lý thuyết, các cách xấp xỉ khi tính toán hậu nghiệm (posterior) đúng là bất khả thi, và một loạt case study ứng dụng. Nhóm tác giả công bố kèm một package Python (github.com/iosband/ts_tutorial) để tái tạo mọi hình và thí nghiệm trong bài.

Thuật ngữ quan trọng cần làm rõ ngay: **posterior distribution** (phân phối hậu nghiệm) là niềm tin (belief) của agent về tham số hệ thống θ sau khi đã quan sát một số dữ liệu, được tính bằng công thức Bayes từ **prior distribution** (phân phối tiên nghiệm, niềm tin trước khi quan sát) và **likelihood** (hàm hợp lý, xác suất quan

sát dữ liệu nếu biết θ). **Regret** (hối tiếc) là thước đo hiệu năng: chênh lệch giữa phần thưởng kỳ vọng của hành động tối ưu (nếu biết θ) và phần thưởng kỳ vọng thực tế đạt được.

2. Ý tưởng chính

Ý tưởng của TS cực kỳ đơn giản để diễn đạt nhưng mạnh về hiệu quả: ở mỗi vòng t, agent **lấy một mẫu (sample) duy nhất** $\hat{\theta}_t$ từ phân phối hậu nghiệm hiện tại của θ, rồi **hành động như thể** $\hat{\theta}_t$ **chính là giá trị thật**, chọn hành động x_t tối ưu đối với $\hat{\theta}_t$. Sau khi quan sát phần thưởng, agent cập nhật hậu nghiệm theo công thức Bayes và lặp lại.

Paper so sánh TS với chiến lược **greedy** (tham lam): greedy luôn chọn hành động có phần thưởng kỳ vọng cao nhất theo ước lượng hiện tại (ví dụ trung bình hậu nghiệm), không có cơ chế khám phá chủ động nào. Vấn đề của greedy minh hoạ qua một ví dụ 3 hành động: nếu hành động 3 có mật độ hậu nghiệm phẳng (độ chắc chắn thấp) nhưng trung bình thấp hơn hành động 1 (mật độ nhọn ở 0.6) một chút, greedy sẽ không bao giờ thử hành động 3 dù nó có thể thực chất tốt hơn, vì ước lượng điểm (point estimate) không phản ánh được độ không chắc chắn (uncertainty).

Một cách khác hay dùng là ϵ**-greedy**: với xác suất ϵ chọn ngẫu nhiên (dithering), với xác suất $1-\epsilon$ chọn greedy. Paper phê bình cách này là "mù": khi khám phá ngẫu nhiên, nó phân bổ đều cho mọi hành động kém, không ưu tiên các hành động "có khả năng là tối ưu nhưng chưa chắc chắn" hơn các hành động "chắc chắn là kém".

TS giải quyết vấn đề này một cách tự nhiên: hành động x được chọn với xác suất đúng bằng **xác suất hậu nghiệm rằng** x **là hành động tối ưu**. Nếu một hành động có độ không chắc chắn cao (hậu nghiệm phẳng) thì nó vẫn có xác suất được chọn tương đối cao dù giá trị trung bình không phải cao nhất, tạo ra khám phá tự nhiên theo đúng mức độ "đáng nghi ngờ" của mỗi hành động, mà không cần tinh chỉnh tham số ϵ nào.

3. Phương pháp chi tiết

3.1. Beta-Bernoulli Bandit (Example 3.1)

Đây là trường hợp cụ thể đơn giản nhất. Mỗi θ_k có tiên nghiệm beta(α_k, β_k) với mật độ

$$p(\theta_k) = \frac{\Gamma(\alpha_k+\beta_k)}{\Gamma(\alpha_k)\Gamma(\beta_k)}\,\theta_k^{\alpha_k-1}(1-\theta_k)^{\beta_k-1}.$$

Vì phân phối Bernoulli và Beta là **liên hợp (conjugate)**, sau khi chọn hành động $x_t = k$ và quan sát phần thưởng $r_t \in \{0,1\}$, việc cập nhật hậu nghiệm chỉ là cộng đơn giản:

$$(\alpha_k, \beta_k) \leftarrow (\alpha_k, \beta_k) + (r_t,\, 1-r_t) \quad \text{(chỉ hành động được chọn mới cập nhật).}$$

Thuật toán 3.2 (BernTS): ở mỗi vòng, lấy mẫu $\hat{\theta}_k \sim \text{beta}(\alpha_k, \beta_k)$ độc lập cho mọi k, chọn $x_t = \arg\max_k \hat{\theta}_k$, quan sát r_t, cập nhật $(\alpha_{x_t}, \beta_{x_t})$ như trên.

Ví dụ tự tính: giả sử 3 hành động có hậu nghiệm beta$(5,3)$, beta$(2,2)$, beta$(1,6)$ (trung bình tương ứng $5/8 = 0.625$, $2/4 = 0.5$, $1/7 \approx 0.143$). Paper minh hoạ bằng một ví dụ số với 3 hành động có xác suất được chọn bởi TS xấp xỉ $0.82/0/0.18$ khi hậu nghiệm đã khá tách biệt; đây là số của paper, không phải tự tính của ghi chú này. Để minh hoạ cơ chế liên hợp, người ghi chú tự tính thêm: nếu hành động 2 (hậu nghiệm beta$(2,2)$) được chọn và quan sát $r_t = 1$ (thành công), hậu nghiệm mới là beta$(3,2)$ với trung bình mới $3/5 = 0.6$ (tự tính), cho thấy việc cập nhật chỉ đơn giản là cộng 1 vào α khi thành công.

Paper mô tả một mô phỏng với $\theta = (0.9, 0.8, 0.7)$, 10.000 lần lặp mô phỏng độc lập, 1.000 vòng mỗi lần. Kết quả: greedy có thể "mắc kẹt" (converge sai vào hành động kém) ở một tỷ lệ các lần mô phỏng, trong khi TS

hội tụ ổn định về hành động tối ưu, thể hiện qua đồ thị per-period regret (Hình 3.2a/3.2b của paper).

3.2. Thompson Sampling tổng quát (Chương 4)

Paper tổng quát hoá: với mô hình bất kỳ $y_t = g(x_t, \theta, w_t)$ (trong đó w_t là nhiễu độc lập), hậu nghiệm cập nhật theo luật Bayes tổng quát:

$$p(\theta'|H_t) \propto p(\theta'|H_{t-1}) \cdot q_{\theta'}(y_t|x_t), \qquad \text{(4.2 trong paper)}$$

trong đó $q_{\theta'}(y_t|x_t)$ là hàm hợp lý (likelihood) của quan sát y_t dưới tham số θ' và hành động x_t. **Thuật toán 4.2 (Thompson)**: lấy mẫu $\hat{\theta}_t \sim p(\cdot|H_{t-1})$, chọn $x_t = \arg\max_x \mathbb{E}[r(y) \mid \hat{\theta}_t, x]$, quan sát, cập nhật theo công thức trên.

Paper minh hoạ với **Example 4.1 (Independent Travel Times)**: mỗi cạnh e có thời gian di chuyển log-Gaussian với tham số (μ_e, σ_e^2) chưa biết, dùng tiên nghiệm liên hợp Gaussian-chuẩn ngược (normal-inverse-gamma) để cập nhật dạng đóng (Eq 4.3 trong paper). Thí nghiệm với một đồ thị "binomial bridge" 20 tầng (184.756 đường đi khả dĩ) cho thấy TS hội tụ nhanh hơn hẳn greedy và ϵ-greedy.

3.3. Các phương pháp xấp xỉ hậu nghiệm (Chương 5)

Với nhiều mô hình thực tế (ví dụ logistic, neural network), hậu nghiệm đúng không có dạng đóng và không thể lấy mẫu chính xác. Paper trình bày 4 phương pháp xấp xỉ:

(a) Gibbs sampling: một thuật toán MCMC (Markov Chain Monte Carlo) tổng quát, lấy mẫu lần lượt từng thành phần của θ có điều kiện theo các thành phần còn lại, lặp nhiều "sweep" để mẫu hội tụ về phân phối hậu nghiệm đúng.

(b) Laplace approximation: xấp xỉ hậu nghiệm bằng một phân phối Gaussian tập trung ở **mode** (điểm cực đại) của log mật độ hậu nghiệm, dùng khai triển Taylor bậc 2:

$$C = -\nabla^2 \ln g(\phi)\Big|_{\phi=\text{mode}},$$

với C là ma trận hiệp phương sai nghịch đảo của xấp xỉ Gaussian. Paper nêu rõ hạn chế: phương pháp này **không bất biến (not invariant)** với phép đổi biến (nếu đổi tham số hoá, kết quả xấp xỉ khác đi), và trong thí nghiệm của paper (bài toán gợi ý đường đi với likelihood dạng logistic) nó cho hiệu năng **kém hơn** Langevin Monte Carlo.

(c) Langevin Monte Carlo (LMC): mô phỏng một phương trình vi phân ngẫu nhiên (SDE)

$$d\phi_t = \nabla \ln g(\phi_t)\, dt + \sqrt{2}\, dB_t,$$

có phân phối dừng chính là hậu nghiệm g mong muốn; rời rạc hoá Euler (Eq 5.2 trong paper) cho một chuỗi Markov xấp xỉ. Paper còn giới thiệu biến thể **stochastic-gradient Langevin MC** dùng minibatch (cỡ mini-batch 100 trong thí nghiệm của paper) để giảm chi phí tính gradient khi có nhiều dữ liệu, và một biến thể **preconditioning** dùng ma trận Hessian nghịch đảo âm tại mode để cải thiện tốc độ hội tụ hình học.

(d) Bootstrapping: tạo một "lịch sử giả định" bằng cách resample (lấy mẫu lại có hoàn lại) từ lịch sử quan sát thật, cộng thêm một mẫu tiên nghiệm giả θ_0, rồi giải một bài toán MAP (maximum a posteriori) ngẫu nhiên hoá trên dữ liệu giả này. Paper ghi rõ: phương pháp này **phi tham số (nonparametric)** nên linh hoạt, nhưng **thiếu cơ sở lý thuyết chặt chẽ** để chứng minh nó xấp xỉ đúng hậu nghiệm.

Kiểm tra đối chiếu (Sanity checks, mục 5.5): paper so sánh 3 phương pháp xấp xỉ trên với TS chính xác ở những bài toán có thể tính hậu nghiệm đúng (Bernoulli bandit cho Gibbs, shortest path với biến đổi biến cho

Laplace), cho thấy các xấp xỉ cho hiệu năng gần với TS chính xác (Hình 5.2 của paper).

Triển khai tăng dần (mục 5.6): paper nhấn mạnh yêu cầu thực tế là chi phí tính toán mỗi vòng phải **cố định**, không tăng theo số lượng dữ liệu đã quan sát (ví dụ không chạy lại MCMC từ đầu mỗi vòng), động lực cho các kỹ thuật cập nhật tăng dần (incremental) như ensemble sampling dựa trên bootstrap.

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial tổng hợp lý thuyết + mô phỏng minh hoạ, **không có benchmark trên dữ liệu thực tế công khai**; toàn bộ "thí nghiệm" trong paper là **mô phỏng Monte Carlo trên các mô hình tổng hợp (synthetic)** được paper tự thiết kế để minh hoạ từng luận điểm. Các kết quả thí nghiệm chính (đều đã nêu nguồn dữ liệu tổng hợp rõ ràng trong paper):

Thí nghiệm (mục trong paper)	Thiết lập	Kết quả chính
Bernoulli bandit 3 tay (3.3)	$\theta = (0.9, 0.8, 0.7)$, 10.000 mô phỏng, 1.000 vòng	TS hội tụ ổn định; greedy có thể mắc kẹt ở hành động kém
Shortest path (4.4)	Binomial bridge 20 tầng, 184.756 đường	TS vượt trội greedy và ϵ-greedy
News recommendation (7.1)	$K = 3$ bài báo, $d = 7$ đặc trưng, logit model, 2.000 instance	TS (qua Laplace/Langevin) vượt ϵ-greedy tối ưu ($\epsilon = 0.01$)
Product assortment (7.2)	$n = 6$ sản phẩm, $\sigma^2 = 0.04$	TS vượt greedy và ϵ-greedy (cả loại cố định và loại annealing)
Cascading bandit K=1000, J=100 (7.3)	Beta(1,40), 1000 instance, 20.000 vòng	CascadeTS vượt hẳn CascadeUCB (UCB1)
Cascading bandit K=50, J=10 (7.3)	cùng thiết lập, quy mô nhỏ hơn	CascadeUCB (đã tinh chỉnh c) vượt CascadeTS, trái với trường hợp trên
Active learning neural net (7.4)	mạng 2 lớp, M=100, D=50, K=100	Ensemble TS (ensemble sampling) vượt ϵ-greedy (cả annealing); chỉ cần khoảng 30 thành viên ensemble
Deep exploration trong MDP (7.5)	chuỗi trạng thái dài $2N+1$, thưởng ở hai đầu	Lấy mẫu một lần mỗi episode (PSRL) học đúng MDP trong 1 episode; lấy mẫu mỗi bước cần tối thiểu $2N$ episode
Nonstationary Bernoulli bandit (6.3)	$\gamma = 0.01$, 10.000 mô phỏng	Nonstationary TS duy trì hiệu năng khi hệ thống trôi dạt; TS thường bị suy giảm

Kết quả cascading bandit (hai dòng in đậm ở trên) là điểm đáng chú ý: **paper tự chỉ ra một trường hợp mà TS không phải lựa chọn tốt nhất** (CascadeUCB với c đã tinh chỉnh tốt hơn CascadeTS khi $K = 50, J = 10$), mặc dù trong trường hợp quy mô lớn hơn ($K = 1000, J = 100$) thì TS vượt trội. Paper giải thích khác biệt này bằng hình học của tập tin cậy (hyper-rectangular so với ellipsoidal) và hiệu ứng số chiều.

5. Phân tích / ablation

Vì đây là lý thuyết + mô phỏng minh hoạ, "ablation" ở đây hiểu theo nghĩa phân tích điều kiện khi nào TS hoạt động tốt và cơ sở lý thuyết.

5.1. Vì sao TS hoạt động tốt (mục 8.1)

Paper trình bày 3 hướng phân tích lý thuyết, đều dựa trên khái niệm **regret** tích lũy $\text{Regret}(T) = \sum_{t=1}^{T}(\mu(x^*, \theta) - \mu(x_t, \theta))$:

(a) Regret tiệm cận (asymptotic, mục 8.1.1): với bandit Bernoulli,

$$\lim_{T\to\infty} \frac{\mathbb{E}[\text{Regret}(T) \mid \theta]}{\log T} = \sum_{k\neq k^*} \frac{\theta_{k^*} - \theta_k}{d_{KL}(\theta_k \,\|\, \theta_{k^*})},$$

đúng với giới hạn dưới tối ưu của Lai và Robbins (1985), tức TS đạt **tối ưu tiệm cận** cho bài toán này (đã được chứng minh trong các công trình của Agrawal và Goyal, Kauffmann et al.).

(b) Regret không phụ thuộc instance (instance-independent, mục 8.1.2): với bandit Bernoulli tiên nghiệm đều, $\max_{\theta'} \mathbb{E}[\text{Regret}(T)|\theta = \theta'] = O(\sqrt{KT\log T})$ (Eq 8.3), gần tối ưu so với giới hạn dưới $\Omega(\sqrt{KT})$.

Với bài toán tổng quát hơn (mô hình tuyến tính, hàm tuyến tính tổng quát hoá, Gaussian process), paper trình bày mối liên hệ chặt giữa TS và các thuật toán **UCB (Upper Confidence Bound)**: với hàm U_t bất kỳ xác định từ lịch sử H_{t-1},

$$\mathbb{E}[U_t(x_t)] = \mathbb{E}[U_t(x^*)] \tag{8.4}$$

(vì x_t và x^* có cùng phân phối biên theo hậu nghiệm), từ đó mọi bound regret của UCB ”dịch” sang bound regret kỳ vọng của TS. Điểm khác biệt quan trọng: regret của UCB phụ thuộc vào lựa chọn cụ thể hàm U_t, còn regret của TS chỉ phụ thuộc vào **lựa chọn U_t tốt nhất có thể** (vì U_t chỉ là công cụ phân tích, không phải một phần của thuật toán TS).

Kết quả tổng hợp (Eq 8.6, của Russo và Van Roy 2014) biểu diễn regret theo hai đại lượng phức tạp của lớp hàm phần thưởng F: **log-covering number** (số lượng tập cần để phủ F ở độ chính xác T^{-2}, đo độ nhạy overfitting) và **eluder dimension** (đo mức độ giá trị của hành động chưa quan sát có thể suy ra từ hành động đã quan sát). Paper nhấn mạnh **eluder dimension là khái niệm mới do chính nhóm tác giả đưa ra**, vì các khái niệm độ phức tạp thống kê cổ điển (VC dimension, Kolmogorov) không đủ để chặn regret trong bài toán ra quyết định trực tuyến.

(c) Phân tích qua lý thuyết thông tin (information-theoretic, tiếp mục 8.1.2): định nghĩa **tỷ lệ thông tin (information ratio)**

$$\Gamma_t = \frac{(\mathbb{E}[\mu(x^*,\theta) - \mu(x_t,\theta)])^2}{I(x^*; (x_t, y_t) \mid H_{t-1})}, \tag{8.7}$$

đo ”chi phí regret bình phương trên mỗi bit thông tin thu được”. Từ đó suy ra bound tổng quát

$$\mathbb{E}[\text{Regret}(T)] \leq \sqrt{\Gamma\, H(x^*)\, T}, \tag{8.8}$$

với $H(x^*)$ là entropy của tiên nghiệm về hành động tối ưu. Với bài toán shortest path, paper chứng minh $\Gamma \leq d/2$ (d = số cạnh), nên bound phụ thuộc số cạnh thay vì số đường đi (có thể là hàm mũ số cạnh) - đây là lý do TS khai thác được cấu trúc bài toán hiệu quả hơn so với coi mỗi đường đi là một ”hành động” độc lập.

5.2. Vì sao cần ngẫu nhiên hoá (mục 8.1.3)

Paper chứng minh bằng **Example 8.1**: với bài toán 2 hành động, một hành động đã biết chắc (Bernoulli(1/2)) và một hành động phụ thuộc tham số nhị phân chưa biết, **mọi chiến lược dừng (stationary) tất định (deterministic)** sẽ có regret tuyến tính theo thời gian với một số tiên nghiệm p_0 nào đó. Do đó, để đạt regret dưới tuyến tính, chiến lược dừng phải **ngẫu nhiên hoá** (như TS), hoặc phải không dừng (như UCB, thay đổi theo thời gian một cách tất định).

6. Giới hạn

Paper dành toàn bộ mục 8.2 để tự phê bình TS, nêu rõ **4 nhóm bài toán mà TS không phải là lựa chọn tốt**:

(a) Bài toán không cần khám phá (8.2.1): nếu nhiễu ngữ cảnh (context) đủ đa dạng (ví dụ context Gaussian liên tục trong contextual bandit tuyến tính), greedy đã tự nhiên ”khám phá” đủ nhờ tính ngẫu nhiên của context, nên đầu tư thêm vào khám phá chủ động (như TS làm) chỉ lãng phí. Tuy nhiên paper cũng chỉ rõ: nếu context là biến nhị phân hoặc có ”offset” tham số, greedy vẫn có thể bị mắc kẹt vĩnh viễn (ví dụ bài toán news recommendation ở mục 7.1 chính là trường hợp này).

(b) Bài toán không cần khai thác (8.2.2): trong các bài toán ”ranking and selection” / ”best-arm identification” (chỉ cần tìm hành động tốt nhất, không cần tối đa hoá phần thưởng dọc đường), TS kém hiệu quả vì một khi đã tự tin vào một hành động, nó khai thác (chọn lại) hành động đó liên tục, làm chậm việc làm rõ các hành động còn lại. Paper dẫn một biến thể ”pure exploration TS” (Russo, 2016) giải quyết vấn đề này.

(c) Nhạy cảm thời gian (8.2.3): với **Example 8.3 (Many-Armed Deterministic Bandit)**, khi số hành động $K \to \infty$, TS không bao giờ lặp lại hành động nào (vì luôn có hành động ”có thể tốt hơn” chưa thử), khiến nó bị một chiến lược đơn giản ”duyệt tuần tự rồi dừng ở hành động đủ tốt” vượt mặt hoàn toàn trong khoảng thời gian ngắn. Đây là vấn đề về **time preference**: TS tối ưu cho khung thời gian dài vô hạn, không tối ưu nếu hiệu năng ngắn hạn quan trọng.

(d) Không đánh giá đúng giá trị thông tin của hành động ”không tối ưu nhưng hữu ích” (8.2.4): ba ví dụ minh họa (Example 8.4 ”A Revealing Action”, Example 8.5 ”Sparse Linear Model”, Example 8.6 ”Assortment Optimization”) đều cho thấy TS chỉ chọn trong số các hành động **có thể là tối ưu**, nên bỏ lỡ các hành động không tối ưu nhưng mang lại thông tin rất lớn (ví dụ hành động 0 trong Example 8.4 tiết lộ ngay giá trị θ dù phần thưởng kỳ vọng của nó luôn thấp hơn). Với Example 8.5, TS cần $O(d/2)$ vòng kỳ vọng còn một thuật toán bisection search chỉ cần $O(\log d)$ vòng, một khác biệt theo cấp độ mũ. Paper đề xuất **information-directed sampling (IDS)** (Russo và Van Roy 2018a) như một thuật toán thay thế tối thiểu hoá trực tiếp tỷ lệ thông tin (8.7), nhưng paper cũng tự nhận IDS **đòi hỏi tính toán phức tạp hơn nhiều** và có thể không thực tế cho các ứng dụng mà TS phù hợp.

Ngoài ra, bản thân kết quả thí nghiệm cascading bandit ở mục 4 trên đây (phần §4) đã tự cho thấy TS **không phải luôn vượt trội UCB đã tinh chỉnh tốt** ở quy mô nhỏ, một điểm tự-mâu-thuẫn-với-trực-giác chung mà paper nêu công khai thay vì giấu đi.

Một hạn chế về phương pháp luận cần ghi nhận rõ (không phải lỗi của paper, nhưng là giới hạn của phạm vi): **không có thí nghiệm trên dữ liệu thực** nào trong bài; mọi con số định lượng đều từ mô phỏng tổng hợp do nhóm tác giả tự thiết kế. Ngoài ra Laplace approximation được paper xác nhận minh bạch là **không bất biến dưới đổi biến** và cho hiệu năng kém hơn Langevin MC trong thí nghiệm cụ thể của paper, không phải một kết luận tổng quát cho mọi bài toán.

7. Liên hệ với các bài khác

- **math content/10 (Hamiltonian Monte Carlo, 2017)**: cùng thuộc nhóm phương pháp MCMC để lấy mẫu từ phân phối phức tạp; HMC là một kỹ thuật MCMC tinh vi hơn Gibbs/Langevin cơ bản mà tutorial này giới thiệu ở mục 5, có thể dùng để cải thiện bước lấy mẫu hậu nghiệm xấp xỉ trong TS cho mô hình phức tạp.
- **math content/11 (SGD như suy luận Bayes xấp xỉ, 2017)**: liên hệ trực tiếp với phần Langevin Monte Carlo và stochastic-gradient Langevin MC ở mục 5.3 của tutorial này; cả hai bài cùng khai thác ý tưởng rằng nhiễu trong SGD/minibatch có thể đóng vai trò như nguồn ngẫu nhiên cho lấy mẫu hậu nghiệm xấp xỉ.
- **optimization content/32 (Practical Bayesian Optimization, 2012) và optimization content/33 (Bayesian Optimization Tutorial, 2018)**: Bayesian Optimization và Thompson Sampling đều giải

quyết đánh đổi khám phá - khai thác trong không gian liên tục/rời rạc dưới mô hình Bayes; TS còn được dùng trực tiếp làm một chiến lược lựa chọn điểm thử trong Bayesian Optimization (xem ”Thompson sampling” là một trong các acquisition function phổ biến, dù chi tiết này không nằm trong phạm vi hai bài opt 32/33 theo như đã đọc ở các ghi chú khác).

- **rl content/13 (RND - Random Network Distillation, 2018)**: cả hai đều là chiến lược khám phá (exploration) trong RL, nhưng theo hai triết lý khác nhau: RND dùng phần thưởng nội tại (intrinsic reward) dựa trên lỗi dự đoán của mạng nơ-ron, còn TS/PSRL (mục 7.5 của tutorial này) dùng lấy mẫu hậu nghiệm Bayes để đạt ”deep exploration” có đảm bảo lý thuyết; so sánh hai hướng này là một câu hỏi mở về đánh đổi giữa tính thực dụng (RND dễ scale lên mạng lớn) và tính có đảm bảo lý thuyết (TS/PSRL).

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU <= 8 GB, batch 1; các hướng pruning, distillation, RL, interpretability đã có thư mục riêng).

Đây là bài **ít liên quan trực tiếp** đến việc huấn luyện hay nén LightOnOCR-3-0.8B theo nghĩa kỹ thuật cốt lõi (TS không phải là thuật toán tối ưu mô hình, không phải pruning hay distillation). Tuy nhiên có 2 liên hệ gián tiếp đáng ghi nhận (đều là suy luận):

1. **Lựa chọn siêu tham số bằng TS/Bayesian bandit**: nếu cần quét một tập hữu hạn các cấu hình nén (ví dụ: tỷ lệ sparsity khác nhau cho từng layer, hoặc tổ hợp (rank LoRA, learning rate) khác nhau cho fine-tuning sau pruning) trên một GPU 8GB với ngân sách thử nghiệm hạn chế, có thể coi mỗi cấu hình là một ”hành động” trong bandit, dùng TS (thay vì grid search đều) để tập trung ngân sách tính toán vào các cấu hình hứa hẹn nhất, nhanh hơn so với dò toàn bộ lưới. Đây là ứng dụng kiểu ”hyperparameter bandit”, phù hợp khi mỗi lần thử một cấu hình (chạy fine-tune + đánh giá) khá tốn kém trên GPU hạn chế.
2. **Active learning cho dữ liệu OCR** (liên hệ với mục 7.4 ”Active Learning with Neural Networks” của tutorial): nếu cần chọn lọc một tập con trang tài liệu để gán nhãn/đánh giá thêm (ví dụ để fine-tune lại sau pruning), nguyên lý ensemble sampling mà paper trình bày ở mục 7.4 (dùng nhiều mô hình/bootstrap để mô phỏng độ không chắc chắn, rồi chọn mẫu có độ không chắc chắn cao để lấy nhãn) có thể áp dụng để chọn trang ”khó” một cách có nguyên tắc hơn so với chọn ngẫu nhiên, giúp tiết kiệm ngân sách gán nhãn hạn chế.

Ngoài hai điểm trên, phần lớn nội dung tutorial (lý thuyết regret, cascading bandit, assortment optimization) không áp dụng trực tiếp cho bài toán nén mô hình VLM OCR của dự án.

9. Thí nghiệm gợi ý

1. **Tái tạo Bernoulli bandit cơ bản**: cài thuật toán 3.2 (BernTS) cho 3-5 hành động với θ tự chọn (ví dụ $(0.9, 0.8, 0.6)$), so sánh với greedy và ϵ-greedy ($\epsilon = 0.05$) qua 500-1000 vòng, lặp 1.000 lần mô phỏng (toy, chạy CPU, không cần GPU). Đo: regret tích lũy trung bình. Kỳ vọng: TS có regret thấp hơn ϵ-greedy, và không bị ”mắc kẹt” như greedy ở một tỷ lệ các lần lặp.
2. **So sánh Laplace vs Langevin Monte Carlo cho một bài toán logistic nhỏ**: tạo bài toán contextual bandit tuyến tính-logistic đồ chơi ($d = 5$, $K = 3$ hành động), cài cả Laplace approximation (mục 5.2) và Langevin MC đơn giản (mục 5.3, bước rời rạc Euler) để lấy mẫu hậu nghiệm xấp xỉ, so sánh regret. Đo: regret trung bình qua 200 vòng, 100 lần mô phỏng. Kỳ vọng: khác biệt nhỏ nếu hậu nghiệm gần Gaussian, khác biệt lớn nếu hậu nghiệm lệch (skewed).
3. **Minh hoạ deep exploration trong MDP chuỗi dài**: cài lại ví dụ Hình 7.6 (chuỗi trạng thái $\{-N, \ldots, N\}$, thưởng ở hai đầu) với $N = 10$, so sánh TS lấy mẫu mỗi episode (PSRL) với TS lấy mẫu mỗi bước. Đo: số episode cần để tìm được phần thưởng đầu tiên. Kỳ vọng: lấy mẫu mỗi episode học nhanh hơn theo cấp số mũ so với lấy mẫu mỗi bước, minh hoạ trực quan khái niệm ”deep exploration”.

4. **Hyperparameter bandit cho bài toán toy nén mô hình**: mô phỏng việc chọn một trong 5 ”cấu hình pruning” giả định (mỗi cấu hình có một ”độ chính xác kỳ vọng” ẩn, quan sát nhiễu Gaussian khi thử), áp dụng TS theo mô hình Gaussian liên hợp (tương tự Example 4.1) để tìm cấu hình tốt nhất trong ít lượt thử nhất. Đo: số lượt thử cần để tìm đúng cấu hình tốt nhất với xác suất 95%. Kỳ vọng: TS cần ít lượt thử hơn round-robin (chia đều ngân sách cho 5 cấu hình).
5. **Ensemble sampling cho active learning trên mạng nhỏ**: huấn luyện một mạng 2 lớp nhỏ (như mục 7.4, $M = 20, D = 10$) trên một hàm hồi quy tổng hợp, dùng ensemble 10-20 mô hình (bootstrap) để chọn điểm dữ liệu tiếp theo cần ”nhãn” (truy vấn), so sánh với chọn ngẫu nhiên. Đo: sai số dự đoán (MSE) trên tập test theo số điểm dữ liệu đã truy vấn. Kỳ vọng: ensemble sampling giảm MSE nhanh hơn chọn ngẫu nhiên khi ngân sách truy vấn nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Cập nhật liên hợp Beta-Bernoulli	$(\alpha_k, \beta_k) \leftarrow (\alpha_k, \beta_k) + (r_t, 1 - r_t)$	Eq. trong mục 3.1 (paper)
Cập nhật Bayes tổng quát	$p(\theta' \mid H_t) \propto p(\theta' \mid H_{t-1})\, q_{\theta'}(y_t \mid x_t)$	Eq. 4.2 (paper)
Langevin SDE	$d\phi_t = \nabla \ln g(\phi_t)\, dt + \sqrt{2}\, dB_t$	mục 5.3 (paper)
Laplace: ma trận hiệp phương sai nghịch đảo	$C = -\nabla^2 \ln g(\phi)$ tại mode	mục 5.2 (paper)
Regret tích lũy kỳ vọng	$\mathbb{E}[\text{Regret}(T)] = \mathbb{E}\big[\sum_{t=1}^{T}(\mu(x^*, \theta) - \mu(x_t, \theta))\big]$	mục 8.1.2 (paper)
Regret tiệm cận tối ưu (Bernoulli)	$\lim_{T\to\infty} \dfrac{\mathbb{E}[\text{Regret}(T) \mid \theta]}{\log T} = \sum_{k \neq k^*} \dfrac{\theta_{k^*} - \theta_k}{d_{KL}(\theta_k \| \theta_{k^*})}$	Eq. 8.1 (paper)
Regret bound tuyến tính qua UCB	$\mathbb{E}[\text{Regret}(T)] = O(d\sqrt{T} \log T)$	Eq. 8.5 (paper)
Tỷ lệ thông tin (information ratio)	$\Gamma_t = \dfrac{(\mathbb{E}[\mu(x^*, \theta) - \mu(x_t, \theta)])^2}{I(x^*; (x_t, y_t) \mid H_{t-1})}$	Eq. 8.7 (paper)
Bound regret qua thông tin	$\mathbb{E}[\text{Regret}(T)] \leq \sqrt{\Gamma\, H(x^*)\, T}$	Eq. 8.8 (paper)
Ví dụ tự tính: cập nhật Beta sau 1 thành công	$\text{beta}(2,2) \xrightarrow{r_t=1} \text{beta}(3,2)$, trung bình $0.5 \to 0.6$	tự tính

11. Kết luận ngắn

Thompson Sampling là một nguyên lý đơn giản (lấy mẫu từ hậu nghiệm rồi hành động như mẫu đó là sự thật) nhưng đạt hiệu quả khám phá gần tối ưu trên một phạm vi rất rộng các bài toán ra quyết định tuần tự, từ bandit cơ bản đến bài toán có cấu trúc phức tạp (shortest path, assortment, cascading, RL trong MDP). Sức mạnh của TS nằm ở khả năng tận dụng cấu trúc bài toán và tiên nghiệm một cách tự động, không cần thiết kế confidence bound riêng cho từng mô hình như UCB. Tuy nhiên, paper cũng trung thực chỉ ra 4 lớp bài toán mà TS không phải lựa chọn tốt nhất (không cần khám phá, không cần khai thác, nhạy cảm thời gian, cần đánh giá thông tin tinh vi hơn mức TS làm được), và đề xuất các hướng thay thế (pure-exploration TS, information-directed sampling, UCB, Gittins index) cho từng trường hợp. Về phương pháp luận, mọi kết quả trong tutorial đều dựa trên mô phỏng tổng hợp do nhóm tác giả tự thiết kế, không có benchmark trên dữ liệu thực tế công khai nào.

Nguồn gốc: https://arxiv.org/abs/1707.02038